

MSPM0 G-Series 80-MHz Microcontrollers

Technical Reference Manual



Literature Number: SLAU846A
JUNE 2023 – REVISED OCTOBER 2023

Table of Contents



Read This First	11
About This Manual	11
Notational Conventions	11
Glossary	11
Support Resources	11
Trademarks	11
1 Architecture	13
1.1 Architecture Overview	14
1.2 Bus Organization	14
1.3 Platform Memory Map	16
1.3.1 Code Region	16
1.3.2 SRAM Region	16
1.3.3 Peripheral Region	18
1.3.4 Subsystem Region	18
1.3.5 System PPB Region	18
1.4 Boot Configuration	18
1.4.1 Configuration Memory (NONMAIN)	18
1.4.2 Boot Configuration Routine (BCR)	20
1.4.3 Bootstrap Loader (BSL)	26
1.5 NONMAIN Registers	29
1.6 Factory Constants	57
1.6.1 FACTORYREGION Registers	58
2 PMCU	93
2.1 PMCU Overview	94
2.1.1 Power Domains	95
2.1.2 Operating Modes	95
2.2 Power Management (PMU)	99
2.2.1 Power Supply	100
2.2.2 Core Regulator	100
2.2.3 Supply Supervisors	100
2.2.4 Bandgap Reference	102
2.2.5 Temperature Sensor	102
2.2.6 VBOOST for Analog Muxes	103
2.2.7 Peripheral Power Enable Control	105
2.3 Clock Module (CKM)	106
2.3.1 Oscillators	106
2.3.2 Clocks	118
2.3.3 Clock Tree	127
2.3.4 Clock Monitors	129
2.3.5 Frequency Clock Counter (FCC)	132
2.4 System Controller (SYSCTL)	135
2.4.1 Resets and Device Initialization	135
2.4.2 Operating Mode Selection	143
2.4.3 Asynchronous Fast Clock Requests	144
2.4.4 SRAM Write Protection	146
2.4.5 Flash Wait States	146
2.4.6 Flash Bank Address Swap	147
2.4.7 Shutdown Mode Handling	147
2.4.8 Configuration Lockout	148

2.4.9 System Status.....	148
2.4.10 Error Handling.....	148
2.4.11 SYSCTL Events.....	150
2.5 Quick Start Reference.....	151
2.5.1 Default Device Configuration.....	152
2.5.2 Leveraging MFCLK.....	152
2.5.3 Optimizing Power Consumption in STOP Mode.....	152
2.5.4 Optimizing Power Consumption in STANDBY Mode.....	153
2.5.5 Increasing MCLK and ULPCLK Precision.....	153
2.5.6 Configuring MCLK for Maximum Speed.....	153
2.5.7 High Speed Clock (SYSPLL, HFCLK) Handling in Low-Power Modes.....	153
2.5.8 Optimizing for Lowest Wakeup Latency.....	154
2.5.9 Optimizing for Lowest Peak Current in RUN/SLEEP Mode.....	154
2.6 SYSCTL Registers.....	155
3 CPU.....	213
3.1 Overview.....	214
3.2 Arm Cortex-M0+ CPU.....	214
3.2.1 CPU Register File.....	215
3.2.2 Stack Behavior.....	218
3.2.3 Execution Modes and Privilege Levels.....	218
3.2.4 Address Space and Supported Data Sizes.....	218
3.3 Interrupts and Exceptions.....	219
3.3.1 Peripheral Interrupts (IRQs).....	220
3.3.2 Interrupt and Exception Table.....	224
3.3.3 Processor Lockup Scenario.....	226
3.4 CPU Peripherals.....	226
3.4.1 System Control Block (SCB).....	226
3.4.2 System Tick Timer (SysTick).....	227
3.4.3 Memory Protection Unit (MPU).....	228
3.5 Read-Only Memory (ROM).....	229
3.6 CPUSS Registers.....	230
3.7 WUC Registers.....	264
4 DMA.....	267
4.1 DMA Overview.....	268
4.2 DMA Operation.....	269
4.2.1 Addressing Modes.....	270
4.2.2 Channel Types.....	271
4.2.3 Transfer Modes.....	272
4.2.4 Extended Modes.....	274
4.2.5 Initiating DMA Transfers.....	275
4.2.6 Stopping DMA Transfers.....	275
4.2.7 Channel Priorities.....	275
4.2.8 Burst Block Mode.....	276
4.2.9 Using DMA with System Interrupts.....	276
4.2.10 DMA Controller Interrupts.....	276
4.2.11 DMA Trigger Event Status.....	276
4.2.12 DMA Operating Mode Support.....	277
4.2.13 DMA Address and Data Errors.....	277
4.2.14 Interrupt and Event Support.....	278
4.3 DMA Registers.....	279
5 MATHACL.....	327
5.1 Overview.....	327
5.2 Data Format.....	327
5.2.1 Unsigned 32-bit integers.....	327
5.2.2 Signed 32-bit integers.....	327
5.2.3 Unsigned 32-bit numbers.....	328
5.2.4 Signed 32-bit numbers.....	328
5.3 Basic Operation.....	328
5.4 Configuration Details with Examples.....	329
5.4.1 Sine and Cosine (SINCOS).....	329
5.4.2 Arc Tangent (ATAN2).....	330

5.4.3 Square Root (SQRT).....	332
5.4.4 Division (DIV).....	333
5.4.5 Multiplication.....	335
5.4.6 Multiply-Accumulate (MAC).....	339
5.4.7 Square Accumulate (SAC).....	341
5.5 MATHACL Registers.....	343
6 NVM (Flash).....	355
6.1 NVM Overview.....	356
6.1.1 Key Features.....	356
6.1.2 System Components.....	356
6.1.3 Terminology.....	356
6.2 Flash Memory Bank Organization.....	357
6.2.1 Banks.....	357
6.2.2 Flash Memory Regions.....	357
6.2.3 Addressing.....	357
6.2.4 Memory Organization Examples.....	358
6.3 Flash Controller.....	359
6.3.1 Overview of Flash Controller Commands.....	360
6.3.2 NOOP Command.....	360
6.3.3 PROGRAM Command.....	360
6.3.4 ERASE Command.....	365
6.3.5 READVERIFY Command.....	365
6.3.6 BLANKVERIFY Command.....	366
6.3.7 Command Diagnostics.....	367
6.3.8 Overriding the System Address With a Bank ID, Region ID, and Bank Address.....	367
6.3.9 FLASHCTL Events.....	368
6.4 Write Protection.....	368
6.4.1 Write Protection Resolution.....	368
6.4.2 Static Write Protection.....	369
6.4.3 Dynamic Write Protection.....	369
6.5 Read Interface.....	370
6.5.1 Bank Address Swapping.....	370
6.5.2 ECC Error Handling.....	370
6.6 FLASHCTL Registers.....	372
7 Events.....	437
7.1 Events Overview.....	438
7.1.1 Event Publisher.....	438
7.1.2 Event Subscriber.....	438
7.1.3 Event Fabric Routing.....	438
7.1.4 Event Routing Map.....	440
7.1.5 Event Propagation Latency.....	441
7.2 Events Operation.....	442
7.2.1 CPU Interrupt.....	442
7.2.2 DMA Trigger.....	442
7.2.3 Peripheral to Peripheral Event.....	443
7.2.4 Extended Module Description Register.....	444
7.2.5 Using Event Registers.....	444
8 IOMUX.....	447
8.1 IOMUX Overview.....	448
8.1.1 IO Types and Analog Sharing.....	448
8.2 IOMUX Operation.....	451
8.2.1 Peripheral Function (PF) Assignment.....	451
8.2.2 Logic High to Hi-Z Conversion.....	451
8.2.3 Logic Inversion.....	452
8.2.4 SHUTDOWN Mode Wakeup Logic.....	452
8.2.5 Pullup/Pulldown Resistors.....	453
8.2.6 Drive Strength Control.....	453
8.2.7 Hysteresis and Logic Level Control.....	453
8.3 IOMUX (PINCMx) Register Format.....	455
8.4 IOMUX Registers.....	457
9 GPIO.....	461

9.1 GPIO Overview.....	462
9.2 GPIO Operation.....	462
9.2.1 GPIO Ports.....	463
9.2.2 GPIO Read/Write Interface.....	463
9.2.3 GPIO Input Glitch Filtering and Synchronization.....	464
9.2.4 GPIO Fast Wake.....	465
9.2.5 GPIO DMA Interface.....	465
9.2.6 Event Publishers and Subscribers.....	465
9.3 GPIO Registers.....	467
10 ADC.....	575
10.1 ADC Overview.....	576
10.2 ADC Operation.....	577
10.2.1 ADC Core.....	578
10.2.2 Voltage Reference Options.....	578
10.2.3 Generic Resolution Modes.....	579
10.2.4 Hardware Averaging.....	579
10.2.5 ADC Clocking.....	580
10.2.6 Common ADC Use Cases.....	580
10.2.7 Power Down Behavior.....	581
10.2.8 Sampling Trigger Sources and Sampling Modes.....	582
10.2.9 Sampling Period.....	584
10.2.10 Conversion Modes.....	585
10.2.11 Data Format.....	586
10.2.12 Advanced Features.....	586
10.2.13 Status Register.....	590
10.2.14 ADC Events.....	591
10.3 ADC12 Registers.....	594
11 COMP.....	677
11.1 Comparator Overview.....	678
11.2 Comparator Operation.....	679
11.2.1 Comparator Configuration.....	679
11.2.2 Comparator Channels Selection.....	679
11.2.3 Comparator Output.....	680
11.2.4 Output Filter.....	680
11.2.5 Sampled Output Mode.....	681
11.2.6 Blanking Mode.....	681
11.2.7 Reference Voltage Generator.....	681
11.2.8 Window Comparator Mode.....	683
11.2.9 Comparator Hysteresis.....	684
11.2.10 Input SHORT Switch.....	684
11.2.11 Interrupt and Events Support.....	685
11.3 COMP Registers.....	688
12 OPA.....	717
12.1 OPA Overview.....	718
12.2 OPA Operation.....	719
12.2.1 Analog Core.....	719
12.2.2 Power Up Behavior.....	720
12.2.3 Inputs.....	720
12.2.4 Output.....	720
12.2.5 Clock Requirements.....	720
12.2.6 Chopping.....	721
12.2.7 OPA Amplifier Modes.....	721
12.2.8 OPA Configuration Selection.....	727
12.2.9 Burnout Current Source.....	728
12.3 OA Registers.....	729
13 GPAMP.....	739
13.1 GPAMP Overview.....	740
13.2 GPAMP Operation.....	740
13.2.1 Analog Core.....	740
13.2.2 Power Up Behavior.....	741
13.2.3 Inputs.....	741

13.2.4 Output.....	741
13.2.5 GPAMP Amplifier Modes.....	741
13.2.6 Chopping.....	743
13.3 GPAMP Registers.....	743
14 DAC.....	745
14.1 DAC Introduction.....	746
14.2 DAC Operation.....	747
14.2.1 DAC Core.....	747
14.2.2 DAC Output.....	747
14.2.3 DAC Voltage Reference.....	747
14.2.4 DAC Output Buffers.....	747
14.2.5 DAC Data Formats.....	748
14.2.6 Sample Time Generator.....	748
14.2.7 DAC FIFO Structure.....	749
14.2.8 DAC Operation With DMA Controller.....	749
14.2.9 DAC Operation With CPU.....	751
14.2.10 Data Register Format.....	751
14.2.11 DAC Output Amplifier Offset Calibration.....	752
14.2.12 Interrupt and Event Support.....	753
14.3 DAC12 Registers.....	756
15 VREF.....	787
15.1 VREF Overview.....	788
15.2 VREF Operation.....	788
15.2.1 Internal Reference Generation.....	788
15.2.2 External Reference Input.....	789
15.2.3 Analog Peripheral Interface.....	789
15.3 VREF Registers.....	791
16 UART.....	801
16.1 UART Overview.....	802
16.1.1 Purpose of the Peripheral.....	802
16.1.2 Features.....	802
16.1.3 Functional Block Diagram.....	803
16.2 UART Operation.....	804
16.2.1 Clock Control.....	804
16.2.2 Signal Descriptions.....	804
16.2.3 General Architecture and Protocol.....	804
16.2.4 Low Power Operation.....	819
16.2.5 Reset Considerations.....	819
16.2.6 Initialization.....	820
16.2.7 Interrupt and Events Support.....	820
16.2.8 Emulation Modes.....	822
16.3 UART Registers.....	823
17 SPI.....	881
17.1 SPI Overview.....	882
17.1.1 Purpose of the Peripheral.....	882
17.1.2 Features.....	882
17.1.3 Functional Block Diagram.....	883
17.1.4 External Connections and Signal Descriptions.....	883
17.2 SPI Operation.....	885
17.2.1 Clock Control.....	885
17.2.2 General Architecture.....	885
17.2.3 Protocol Descriptions.....	890
17.2.4 Reset Considerations.....	895
17.2.5 Initialization.....	895
17.2.6 Interrupt and Events Support.....	895
17.2.7 Emulation Modes.....	897
17.3 SPI Registers.....	898
18 I²C.....	971
18.1 I ² C Overview.....	972
18.1.1 Purpose of the Peripheral.....	972
18.1.2 Features.....	972

18.1.3 Functional Block Diagram.....	973
18.1.4 Environment and External Connections.....	973
18.2 I ² C Operation.....	974
18.2.1 Clock Control.....	974
18.2.2 Signal Descriptions.....	975
18.2.3 General Architecture.....	975
18.2.4 Protocol Descriptions.....	983
18.2.5 Reset Considerations.....	994
18.2.6 Initialization.....	995
18.2.7 Interrupt and Events Support.....	995
18.2.8 Emulation Modes.....	997
19 I2C Registers.....	999
20 CAN-FD.....	1065
20.1 MCAN Overview.....	1066
20.1.1 MCAN Features.....	1067
20.2 MCAN Environment.....	1067
20.3 CAN Network Basics.....	1068
20.4 MCAN Functional Description.....	1069
20.4.1 Clock Set up	1070
20.4.2 Module Clocking Requirements.....	1071
20.4.3 Interrupt Requests.....	1071
20.4.4 Operating Modes.....	1073
20.4.5 Software Initialization.....	1075
20.4.6 Transmitter Delay Compensation.....	1076
20.4.7 Restricted Operation Mode.....	1078
20.4.8 Bus Monitoring Mode.....	1079
20.4.9 Disabled Automatic Retransmission (DAR) Mode.....	1080
20.4.10 Clock Stop Mode.....	1080
20.4.11 Test Modes.....	1086
20.4.12 Timestamp Generation.....	1087
20.4.13 Timeout Counter.....	1088
20.4.14 Safety.....	1089
20.4.15 Tx Handling.....	1091
20.4.16 Rx Handling.....	1101
20.4.17 Rx FIFOs.....	1105
20.4.18 Dedicated Rx Buffers.....	1107
20.4.19 Message RAM.....	1107
20.5 MCAN Integration.....	1118
20.6 Interrupt and Event Support.....	1119
20.6.1 CPU Interrupt Event Publisher (CPU_INT).....	1119
20.7 MCAN Registers.....	1120
21 MCAN Registers.....	1121
22 CRC.....	1251
22.1 CRC Overview.....	1252
22.1.1 CRC16-CCITT.....	1252
22.1.2 CRC32-ISO3309.....	1252
22.2 CRC Operation.....	1252
22.2.1 CRC Generator Implementation.....	1253
22.2.2 Configuration.....	1253
22.3 CRC Registers.....	1255
23 AES.....	1267
23.1 AES Overview.....	1268
23.1.1 AES Performance.....	1268
23.2 AES Operation.....	1269
23.2.1 AES Register Access Rules.....	1270
23.2.2 Loading the Key.....	1271
23.2.3 Loading Data.....	1271
23.2.4 Reading Data.....	1271
23.2.5 Triggering an Encryption or Decryption.....	1271
23.2.6 Single Block Operations.....	1271
23.2.7 Block Cipher Mode Operations.....	1274

23.2.8 AES Events.....	1287
23.3 AES Registers.....	1290
24 TRNG.....	1331
24.1 TRNG Overview.....	1332
24.2 TRNG Operation.....	1332
24.2.1 TRNG Generation Data Path.....	1332
24.2.2 Clock Configuration and Output Rate.....	1333
24.2.3 Behavior in Low Power Modes.....	1333
24.2.4 Health Tests.....	1333
24.2.5 Configuration.....	1335
24.3 TRNG Registers.....	1339
25 Timers (TIMx).....	1357
25.1 TIMx Overview.....	1358
25.1.1 TIMG Overview.....	1358
25.1.2 TIMA Overview.....	1359
25.1.3 TIMx Instance Configuration.....	1361
25.2 TIMx Operation.....	1361
25.2.1 Timer Counter.....	1362
25.2.2 Counting Mode Control.....	1364
25.2.3 Capture/Compare Module.....	1370
25.2.4 Shadow Load and Shadow Compare.....	1384
25.2.5 Output Generator.....	1386
25.2.6 Fault Handler (TIMA only).....	1396
25.2.7 Synchronization With Cross Trigger.....	1402
25.2.8 Low Power Operation.....	1404
25.2.9 Interrupt and Event Support.....	1404
25.2.10 Debug Handler (TIMA only).....	1408
25.3 Timers (TIMx) Registers.....	1409
26 RTC.....	1503
26.1 Overview.....	1504
26.2 Basic Operation.....	1504
26.3 Configuration.....	1506
26.3.1 Clocking.....	1506
26.3.2 Reading and Writing to RTC Peripheral Registers.....	1506
26.3.3 Binary vs. BCD.....	1507
26.3.4 Leap Year Handling.....	1507
26.3.5 Calendar Alarm Configuration.....	1507
26.3.6 Interval Alarm Configuration.....	1508
26.3.7 Periodic Alarm Configuration.....	1509
26.3.8 Calibration.....	1509
26.3.9 RTC Events.....	1512
26.4 RTC Registers.....	1514
27 WWDT.....	1557
27.1 WWDT Overview.....	1558
27.1.1 Watchdog Mode.....	1558
27.1.2 Interval Timer Mode.....	1559
27.2 WWDT Operation.....	1559
27.2.1 Mode Selection.....	1559
27.2.2 Clock Configuration.....	1560
27.2.3 Low-Power Mode Behavior.....	1561
27.2.4 Debug Behavior.....	1561
27.2.5 WWDT Events.....	1562
27.3 WWDT Registers.....	1563
28 Debug.....	1581
28.1 Overview.....	1582
28.1.1 Debug Interconnect.....	1582
28.1.2 Physical Interface.....	1583
28.1.3 Debug Access Ports.....	1584
28.2 Debug Features.....	1584
28.2.1 Processor Debug.....	1584
28.2.2 Peripheral Debug.....	1586

28.2.3 EnergyTrace Technology.....	1586
28.3 Behavior in Low Power Modes.....	1586
28.4 Restricting Debug Access.....	1587
28.5 Mailbox (DSSM).....	1587
28.5.1 DSSM Events.....	1588
28.5.2 DEBUGSS Registers.....	1590
29 Revision History.....	1606



About This Manual

This manual describes the modules and peripherals of the MSPM0G family of devices. Each description presents the module or peripheral in a general sense. Not all features and functions of all modules or peripherals are present on all devices. In addition, modules or peripherals can differ in their exact implementation on different devices. Pin functions, internal signal connections, and operational parameters differ from device to device. See the device-specific data sheet for these details.

Notational Conventions

This document uses the following conventions.

- Hexadecimal numbers can be shown with the suffix h or the prefix 0x. For example, the following number is 40 hexadecimal (decimal 64): 40h or 0x40.
- Registers in this document are shown in figures and described in tables.
 - Each register figure shows a rectangle divided into fields that represent the fields of the register. Each field is labeled with its bit name, its beginning and ending bit numbers above, and its read/write properties with default reset value below. A legend explains the notation used for the properties.
 - Reserved bits in a register figure can have one of multiple meanings:
 - Not implemented on the device
 - Reserved for future device expansion
 - Reserved for TI testing
 - Reserved configurations of the device that are not supported
 - Writing nondefault values to the Reserved bits could cause unexpected behavior and should be avoided.

Glossary

[TI Glossary](#) This glossary lists and explains terms, acronyms, and definitions.

Support Resources

[TI E2E™ support forums](#) are an engineer's go-to source for fast, verified answers and design help — straight from the experts. Search existing answers or ask your own question to get the quick design help you need.

Linked content is provided "AS IS" by the respective contributors. They do not constitute TI specifications and do not necessarily reflect TI's views; see TI's [Terms of Use](#).

Trademarks

TI E2E™ and EnergyTrace™ are trademarks of Texas Instruments.
Arm® and Cortex® are registered trademarks of Arm Limited.
All trademarks are the property of their respective owners.

This page intentionally left blank.



The device architecture includes the bus organization, the platform memory map, and the boot configuration.

1.1 Architecture Overview	14
1.2 Bus Organization	14
1.3 Platform Memory Map	16
1.4 Boot Configuration	18
1.5 NONMAIN Registers	29
1.6 Factory Constants	57

1.1 Architecture Overview

MSPM0 G-series MCUs (MSPM0Gxx) combine 32-bit compute performance together with precision analog to enable a wide variety of sensing, interface, control, and housekeeping applications. The device architecture supports both high-performance and low-power applications through a flexible, easy-to-configure power management and clocking system.

MSPM0 G-series devices also offer enhanced robustness with ECC-protected flash memory, parity-protected SRAM, available dual window-watchdog timers, and support for 125°C ambient temperature and AEC-Q100 Grade 1 qualification.

This chapter introduces the device architecture, including an overview of the [power domains and bus organization](#), the [platform memory map](#), and the [device boot configuration](#).

1.2 Bus Organization

There are three main power domains on MSPM0Gxx devices:

- PD1 (power domain 1) which contains the CPU subsystem, memory interfaces, and high-speed peripherals
- PD0 (power domain 0) which contains the low-speed low-power peripherals
- The supply voltage (VDD) which powers IOs, analog modules, and limited logic directly from the supply

The PD1 domain supports higher clock speeds for performance, and is disabled in certain operating modes to minimize power consumption. The PD0 domain supports ultra-low-power performance and is always enabled in operating modes in which the core regulator is operating.

There are four main data buses on MSPM0Gxx devices:

- The AHB bus matrix, which interfaces the CPU to the device memory systems (ROM, SRAM, and flash memory) and the peripheral buses
- The PD1 (power domain 1) CPU-only peripheral bus, clocked from [MCLK](#)
- The PD1 (power domain 1) peripheral bus, clocked from [MCLK](#)
- The PD0 (power domain 0) peripheral bus, clocked from [ULPCLK](#)

The CPU and the DMA controller are the only two bus controllers in the device. Arbitration between the CPU and the DMA for shared peripherals happens between the CPU-only PD1 peripheral bus and the CPU/DMA PD1 peripheral bus. The DMA does not have access to peripherals on the CPU-only PD1 peripheral bus or the CPU bus matrix (the green components in the bus diagram). As such, the CPU can access peripherals on the CPU-only PD1 peripheral bus at the same time that the DMA is processing a transaction on the PD1 or PD0 bus.

Likewise, the CPU can access SRAM or flash memory through the AHB bus matrix at the same time that the DMA is processing a transaction, so long as the DMA is not accessing the same memory that the CPU is attempting to access. Arbitration between the CPU and the DMA for memory systems (SRAM or flash memory) happens at the memory interface itself. All arbitration between the CPU and DMA is done on a round-robin basis.

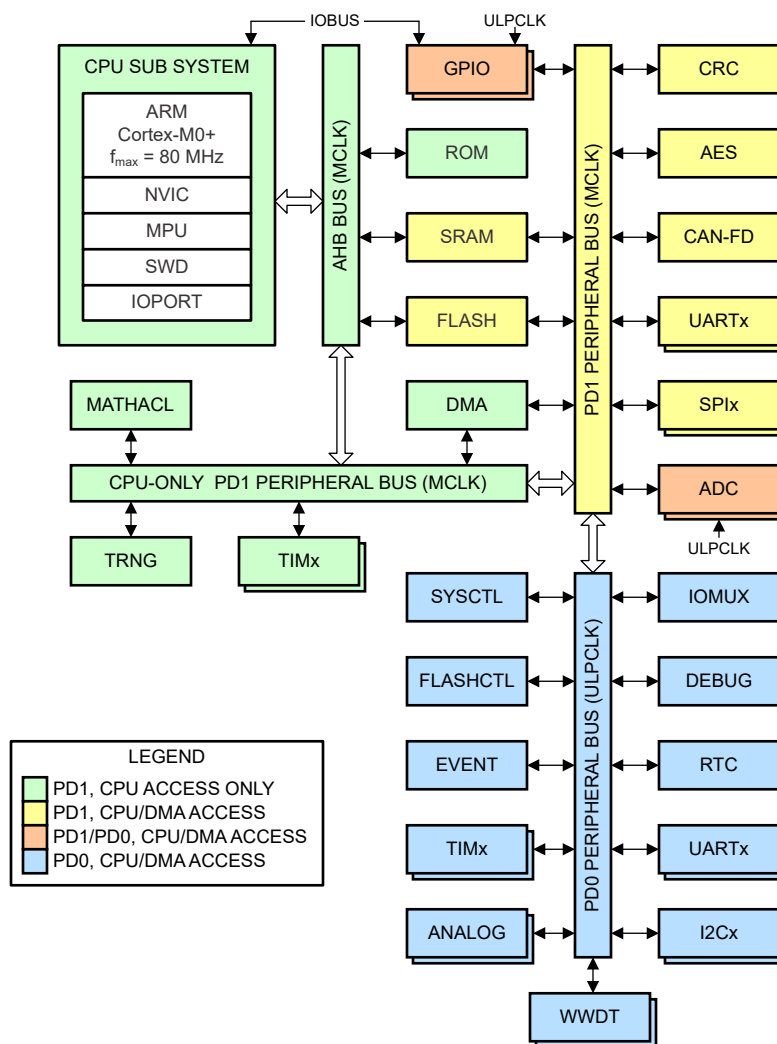


Figure 1-1. MSPM0Gxx Bus Organization

Note

This is a generic diagram of the typical peripherals on an MSPM0Gxx device and their respective bus locations. Not all devices have all peripheral options shown here. To determine the peripherals which are available on a given device, see the device-specific data sheet.

The GPIO and ADC peripherals (the orange components in the bus diagram) have special capabilities to enable both fast register access from the CPU and operation in low power operating modes.

- GPIO peripherals interface to the system through two mechanisms: the PD1 peripheral bus, and the Arm® Cortex®-M0+ single-cycle high-speed IO bus.
 - Accesses from the CPU to any GPIO registers are transacted through the single-cycle IO bus for best performance, enabling fast toggling of IO under software control.
 - The GPIO DOUT registers (data out) are also available on the PD1 peripheral bus, primarily so that the DMA can be used to load values to the GPIO DOUT registers.
 - While the bus interfaces to the GPIO peripherals are in the PD1 power domain (for best read/write performance), the GPIO logic itself is in the PD0 power domain so that it is available in all operating modes in which the core regulator is active.
- ADC peripherals interface to the system through the PD1 peripheral bus but contain functional logic in the PD0 power domain.

- ADC peripheral register accesses are processed through the PD1 peripheral bus (for best read/write performance)
- The ADC conversion logic is in the PD0 power domain to enable running timer-triggered ADC conversions without CPU interaction in certain low-power modes when PD1 is disabled.

1.3 Platform Memory Map

All MSPM0Gxx devices share a common platform memory map. Peripherals are assigned a fixed address space and have the same address space on all devices within the family. The memory map is compliant with the standard Arm Cortex-M memory regions.

Table 1-1. Top Level Memory Map

Memory Region	Start Address	End Address	Description
Code	0x0000.0000	0x1FFF.FFFF	Flash memory and ROM
SRAM	0x2000.0000	0x3FFF.FFFF	SRAM
Peripheral	0x4000.0000	0x5FFF.FFFF	Global peripheral memory-mapped registers and global non-executable data memory
Subsystem	0x6000.0000	0x7FFF.FFFF	Local CPU subsystem memory-mapped registers
System PPB	0xE000.0000	0xE00F.FFFF	Arm private peripheral bus

1.3.1 Code Region

The code region contains the flash memory used to store executable code and data. Accesses to the flash memory from the CPU through the code region are processed through the AHB bus matrix to the flash read interface directly. The flash memory is aliased to two address spaces in the code region: one returning ECC-corrected data, the other providing uncorrected (raw) data. See [Section 6.2.3.1](#) for the detailed flash memory map.

The code region also contains the read-only memory (ROM) used for the TI device boot code and the bootstrap loader. The ROM is only available during the initial device boot process.

1.3.2 SRAM Region

The SRAM region contains the system memory (SRAM). The SRAM supports zero wait state access at the maximum MCLK frequency (80 MHz). Accesses to the SRAM from the CPU are processed through the AHB bus matrix to the SRAM interface directly. The SRAM region supports devices with up to 1MB of SRAM. See the device-specific data sheet for the amount of SRAM present on a given device.

Certain devices optionally support parity or parity and ECC checking of the SRAM. Refer to the device-specific data sheet to determine if a device supports ECC or parity checked SRAM. For information on how parity and ECC errors are handled by the device, see [Section 2.4.10](#).

Parity Checking

In the case of parity checking (if available), 1 parity bit is provided per 8 data bits. Parity checking is capable of detecting a single bit error in the corresponding 8 bits of data (SED). Writing data to a parity checked address updates the corresponding parity bits based on the new data. Reading data from a parity checked address checks the read data against the corresponding parity bits. Upon a read, if the data does not match the corresponding parity bits, a parity error is generated. For information on how parity errors are handled by the device, see [Section 2.4.10](#).

ECC Checking

In the case of ECC checking (if available), 8 ECC bits are provided per 64 data bits. ECC is capable of correcting single bit errors (SEC) and detecting dual bit errors (DED) in the corresponding 64 data bits. Writing data to an ECC checked address updates the corresponding ECC code based on the new data. Reading data from an ECC checked address checks the read data against the corresponding ECC code. If a single bit error is found, it is corrected automatically and a correctable ECC error is generated. If a dual bit error is found, an uncorrectable ECC error is generated.

Aliased Subregions

The default subregion (0x2000.0000) is available on all MSPM0 devices, and when used, provides the highest level of integrity checking available on the device. The parity checked subregion (0x2010.0000) is available on devices which support ECC or parity checking, and accesses are always processed with parity checking. The unchecked subregion (0x2020.0000) is available on all devices, and when using this subregion, no integrity checks are performed. The parity/ECC code subregion (0x2030.0000) is available on devices with ECC or parity checking, and it returns the parity or ECC code which corresponds to the address being read.

Note

To improve robustness, there is no mechanism provided to disable parity checking of the parity-checked SRAM address space. To operate without parity checking, link the application against the unchecked SRAM address space.

Table 1-2. SRAM Region Memory Map

Subregion	Start	End	Description
Default	0x2000.0000	0x200F.FFFF	The highest available integrity check on the device is always applied to accesses in this subregion: - If the device supports ECC, this subregion is ECC checked. - If the device only supports parity (no ECC), this subregion is parity checked and accesses are equivalent to accesses to the parity checked subregion. - If the device does not support ECC or parity, no checks are applied to accesses in this subregion and the region is equivalent to the unchecked subregion.
Parity checked	0x2010.0000	0x201F.FFFF	If the device supports parity, accesses to this subregion are parity checked.
Unchecked	0x2020.0000	0x202F.FFFF	No ECC or parity checks are applied to accesses in this subregion.
Parity/ECC code	0x2030.0000	0x203F.FFFF	If the device supports parity or ECC, the parity or ECC codes may be directly accessed through this subregion: - If the device supports ECC, access to any address within a 64-bit boundary returns the 8 bits corresponding to the ECC code (if the application is linked against the default region) or the parity bits (one per byte) if the application is linked against the parity checked region. - If the device supports parity only, access to any address within a 32-bit boundary returns the 4 parity bits (one per byte) if the application is linked against the default region or parity checked region. - If the device does not support ECC or parity, access to this region always returns zeros.

Note

When ECC checking is used on devices which support ECC, writes to the SRAM require two cycles to complete. Read accesses only require a single cycle and do not incur any additional performance penalty.

On devices supporting parity or ECC, it is possible for application software to partition the usage of the physical SRAM into arbitrary zones which are intended to be ECC checked, parity checked, or unchecked. For example, if a device has 32KB of total SRAM memory, and it supports ECC and parity checking, it is possible to configure the application software to link against two subregions, one being ECC checked and the other being parity checked.

Note

It is not recommended to mix and match ECC checked, parity checked, and unchecked accesses to the same memory locations, as this can result in unintended parity/ECC errors. For example, if an SRAM memory location is written to through the parity checked subregion on a device which supports ECC, and then later that location is read through the default (ECC checked) subregion, an ECC error may be generated because the data was written with parity stored to the ECC/parity code memory, as opposed to the correct ECC code being stored.

Note

The SRAM contents may be random at power-up or when existing SHUTDOWN mode. A read of an SRAM location through an ECC/parity checked region which has not yet been written to may result in a ECC/parity error being detected, causing a hard fault in the processor. Ensure that all SRAM locations are initialized first before being read.

1.3.3 Peripheral Region

The peripheral region contains the memory-mapped peripherals on the [three peripheral buses](#). The flash memory is also aliased in the peripheral region.

Table 1-3. Peripheral Region Memory Map

Type	Start	End	Description
Peripherals	0x4000.0000	0x40FF.FFFF	Memory-mapped registers of peripherals on the peripheral buses
Aliased flash memory	0x4100.0000	0x41FF.FFFF	See Section 6.2.3.1

1.3.4 Subsystem Region

The subsystem region contains memory-mapped registers which are specific to the CPU subsystem and do not need to be accessed globally. See the CPU subsystem chapter for the memory-mapped registers in the subsystem region.

1.3.5 System PPB Region

The system PPB region contains memory-mapped registers on the Arm private peripheral bus. These registers are tightly coupled to the CPU and are the interface for peripherals such as the memory protection unit (MPU), SysTick timer, and CPU power management and reset functions.

1.4 Boot Configuration

After a [BOOTRST](#), the device executes the start-up boot routines to configure the device for operation before starting the main application. Boot routines are executed from read-only memory (ROM) before the main application is started. There are two boot routines: the boot configuration routine (BCR), and the bootstrap loader (BSL). The boot configuration routine sets up the device security policies, configures the device for operation, and optionally starts the BSL if it presents. The BSL, if started by the BCR, can be used to program or verify the device memory (flash and SRAM) through the use of a standard serial interface (UART or I2C).

After the start-up routines have successfully completed execution, the CPU is reset and the application is started by unconditionally fetching the stack pointer (SP) and reset vector from 0x0000.0000 and 0x0000.0004 of the flash memory. To enable secure boot, this single point of entry into the application code is enforced by the boot sequence. It is not possible to boot into a different memory location.

1.4.1 Configuration Memory (NONMAIN)

The NONMAIN is a dedicated region of flash memory which stores the configuration data used by the BCR and BSL to boot the device. The region is not used for any other purpose. The BCR and BSL both have configuration policies which can be left at their default values (as is typical during development and evaluation), or modified for specific purposes (as is typical during production programming) by altering the values programmed into the NONMAIN flash region.

The BCR and BSL configuration data structures are both contained within a single flash sector in the NONMAIN flash memory region. To change any parameter in the boot configuration, it is necessary to erase the entire NONMAIN sector and re-program both the BCR and BSL configuration structures with the desired settings.

The configuration data in the NONMAIN flash region is not affected by a mass erase command, but it is erased and re-programmed to factory defaults by a factory reset command sent to the BCR via the debug sub system mailbox (DSSM) over SWD.

The NONMAIN flash is also erased by a factory reset command sent to the BSL using the UART or I2C BSL interface. However, unlike the DSSM factory reset, the BSL factory reset does not program TI factory defaults to the NONMAIN memory following the erase. As such, it is the responsibility of the host which is connected to the MSPM0 target (via the BSL interface) to re-program the NONMAIN memory with a valid configuration before terminating the BSL session.

Note

If a factory reset command is executed through the BSL, and a valid NONMAIN configuration is not programmed back into the device before the BSL session is terminated, the device will assume a maximally restrictive state upon the next reset cycle, and it will not be possible to access the device via SWD or the BSL. Always ensure that a valid NONMAIN configuration is programmed back when using the BSL factory reset command.

The address ranges for the NONMAIN data structures are given in [Table 1-4](#). A detailed breakdown of the NONMAIN region is provided at the end of this section.

Table 1-4. NONMAIN Region Overview

NONMAIN Section	Start Address	End Address
BCR Configuration	41C0.0000h	41C0.005Bh
BCR Configuration CRC	41C0.005Ch	41C0.005Fh
BSL Configuration	41C0.0100h	41C0.0153h
BSL Configuration CRC	41C0.0154h	41C0.0157h

1.4.1.1 CRC-Backed Configuration Data

The BCR configuration data and BSL configuration data structures in the NONMAIN memory each include a CRC32 value corresponding to the CRC32 digest of the respective structure. During the device boot process, the BCR will compute the CRC digest of the data structures and compare it with the stored CRC values before the data contained within the structures is trusted for use.

BCR Configuration CRC Fail Handling

In the event that the BCR configuration data (which contains the SWD policies, BSL enable/disable policy, and flash memory protection and integrity check policies) fails its CRC check during boot, a catastrophic boot error results and the following limitations are imposed:

- The error cause will be logged in the CFG-AP as a boot diagnostic
- The BSL will not be invoked, even if it was configured to be enabled
- The user application is not started
- No application debug access is enabled
- A pending SWD factory reset command, if enabled or enabled with password, is honored
- A pending TI failure analysis flow entry, if enabled, is honored
- The boot process will re-attempt up to 3 times
 - If the 2nd or 3rd attempt pass, the device boots normally
 - If the 3rd attempt does not pass, no further boot attempts are made until the next BOR or POR

The benefit of this CRC check is that any bit flips in configuration data, such as the static write protection configuration (which is a pillar of secure boot), may be detected with high confidence during the boot process. The fail handling procedure explicitly prevents the BSL and user application from running, and the only supported options (SWD factory reset and TI FA) are protected by [16-bit pattern-match fields](#).

BSL Configuration CRC Fail Handling

If the BSL configuration data (which contains the BSL password and BSL policies) fails the CRC check during BSL invocation, a catastrophic boot error results and the following limitations are imposed:

- The error cause is logged in the CFG-AP as a boot diagnostic
- The BSL is not invoked, even if it was configured to be enabled
- The user application is not started
- No application debug access is enabled
- The boot process re-attempts up to 3 times
 - If the 2nd or 3rd attempt pass, the device boots normally
 - If the 3rd attempt does not pass, no further boot attempts are made until the next BOR or POR

The benefit of this CRC check is that any bit flips in the BSL configuration data may be detected with high confidence during the invoke process. The failure handling procedure prevents the BSL from starting with invalid data which could lead to a loss of security.

TI Factory Trim Data CRC Fail Handling

In addition to the user-specified configuration data, if the TI factory trim fails its CRC check during boot, a catastrophic boot error will also result with the following limitations:

- The error cause will be logged in the CFG-AP as a boot diagnostic
- The BSL will not be invoked, even if it was configured to be enabled
- The user application is not started
- No application debug access is enabled
- A pending TI failure analysis flow entry, if enabled, is honored
- The boot process will re-attempt up to 3 times
 - If the 2nd or 3rd attempt pass, the device boots normally
 - If the 3rd attempt does not pass, no further boot attempts are made until the next BOR or POR

1.4.1.2 16-bit Pattern Match for Critical Fields

Critical policies in the BCR configuration memory, such as the SWD security policies, are implemented as 16-bit pattern-match fields in the NONMAIN memory, with the following characteristics:

- An exact pattern match is required to enable lower security states
- Any value in the 16-bit field not matching the exact defined patterns results in a maximally secure state for the respective parameter

This behavior prevents single bit flips from causing the device to enter a lower security state than that which was originally specified.

1.4.2 Boot Configuration Routine (BCR)

The boot configuration routine is the first firmware to run on the device after a [BOOTRST](#). The BCR manages the following at boot time:

- Configuring the debug interface security policy
- Optionally executing a mass erase
- Optionally executing a factory reset
- Configuring the flash memory [static write protection](#) policy
- Optionally verifying the integrity of some or all of the application firmware (with a 32-bit CRC)
- Optionally starting the [bootstrap loader \(BSL\)](#)

1.4.2.1 Serial Wire Debug Related Policies

The serial wire debug related policies configure the functionality which is available through the device's physical debug interface (SWD). By default, MSPM0 devices come from TI in an unrestricted state. This state allows for easy production programming, evaluation, and development. However, this unrestricted state is not recommended for mass production, as it leaves a large attack surface present. To accommodate a variety of needs while keeping the configuration process simple, MSPM0 devices support three generic security levels: no restrictions ([Level 0](#)), custom restrictions ([Level 1](#)), and fully restricted ([Level 2](#)). [Table 1-5](#) shows the three generic security levels, from least restrictive to most restrictive.

There are 4 main uses of the SWD interface for which protection needs to be considered:

- Application debug access, which includes:
 - Full access to the processor, memory map, and peripherals through the AHB-AP
 - Access to the device EnergyTrace+ state information through the ET-AP
 - Access to the device power state controls for debug through the PWR-AP
- Mass erase access, which includes:
 - Ability to send a command through SWD to erase the MAIN memory region while leaving the NONMAIN device configuration memory intact
- Factory reset access, which includes:
 - Ability to send a command through SWD to erase the MAIN memory region and reset the NONMAIN device configuration memory to TI factory defaults (Level 0)
- TI failure analysis access, which includes:
 - Ability for TI to initiate a failure analysis return flow through SWD (note that the TI FA flow always forces a factory reset before FA access is given to TI; this ensures that TI does not have any mechanism to read proprietary customer information stored in the device flash memory when a failure analysis flow is initiated)

Table 1-5. Generic Security Levels

Level	Scenario	SW-DP Policy	App Debug Policy	Mass Erase Policy	Factory Reset Policy	TI FA Policy
0	No restrictions	EN	EN	EN	EN	EN
1	Custom restrictions	EN	EN, EN, EN with PW, DISDIS	EN, EN with PW, DIS	EN, EN with PW, DIS	EN, DIS
2	Fully restricted	DIS	Don't care (access not possible with SW-DP disabled) ⁽¹⁾			

(1) When the SW-DP policy is **SW-DP disabled**, the mass erase and factory reset policies are a don't care from the point of view of the SWD interface. However, if the bootstrap loader (BSL) is enabled, the mass erase and factory reset policies do impact what functionality is available through the BSL. See the BSL security section for details on securing the BSL.

1.4.2.1.1 SWD Security Level 0

SWD security level 0 is the least restrictive SWD security state. This is the default state of a new device from TI, and it is also the state of a device following a successful factory reset. There are no restrictions on application debug access, mass erase, factory reset, for failure analysis in this state.

When to Use This State

Level 0 is well suited for prototyping and development, as it allows programming of the device memory and debug of the processor and peripherals.

When to Not Use this State

Level 0 should not be used in mass production. An attacker would have full freedom to read the contents of the device memory, manipulate the execution of the device, and possibly change the flash memory contents (depending on the flash memory write protection scheme).

1.4.2.1.2 SWD Security Level 1

SWD security level 1 allows for a customized security configuration. The physical debug port (SW-DP) is left enabled, and each function (application debug, mass erase command, factory reset command, and TI failure analysis) may be individually enabled, disabled, or (in some cases) enabled through password authentication, providing considerable flexibility to tailor the device behavior to specific use-cases.

When to Use This State

Level 1 is well suited for restricted prototyping/development scenarios and for mass production scenarios where the desire is to retain certain SWD functions (such as factory reset and TI failure analysis) while disabling other functions (such as application debug). Common examples of Level 1 customized configurations are given in [Table 1-6](#).

Table 1-6. Examples of Level 1 Configurations

Level 1 Scenario	Configuration			
	App Debug	Mass Erase	Factory Reset	TI FA
This scenario restricts debug access with a user-specified password, but it leaves the factory reset and TI failure analysis available. This configuration allows field debug (with password), and it also allows the device to be brought back to the default "Level 0" state through factory reset.	EN with PW	DIS	EN	EN
This scenario does not allow debug. It does allow factory reset, but only with a user-specified password. This provides a way to open up a device in the field by clearing the MAIN memory contents and bringing the device back to a "Level 0" state if the password is known. Importantly, even if the factory reset password were compromised, it would not be possible for an attacker to read proprietary information in the MAIN flash memory.	DIS	DIS	EN with PW	EN
This scenario does not allow debug and it does not allow TI failure analysis. This prevents TI from performing a factory reset and further FA activities on the device, unless the user executes a factory reset with their user-specified password before returning the devices to TI for FA.	DIS	DIS	EN with PW	DIS

Note

Level 1 is the recommended configuration for most standard production use-cases. For applications which do not require secure boot, TI recommends using Level 1 in production with factory reset left enabled (with password) and TI failure analysis left enabled. In such a configuration, the device may be recovered to a less restrictive state after provisioning either by the user (with password) or by TI (through the failure analysis return flow). In use-cases requiring maximum secure boot assurance, a more restrictive Level 1 or Level 2 may be used for production, with the trade-off that devices may not be recoverable to a less restrictive state once provisioned.

When to Not Use this State

Level 1 should not be used during prototyping if complete access to the device is desired; in such a case, Level 0 should be used instead.

Level 1 should also not be used in a mass production scenario where a maximally restrictive state is desired and no SWD functions are to be enabled; in such a case, Level 2 should be used instead as it directly disables the complete SWD physical interface and minimizes the possibility of misconfiguration.

Note

If a device is configured with application debug and factory reset disabled, the only way for a user to restore debug access to the device is if the user application code provides a mechanism to change the NONMAIN configuration to a less restrictive state. If the [NONMAIN is locked through static write protection](#) then the state is not reversible and there is no way for a user to re-gain debug access.

1.4.2.1.3 SWD Security Level 2

SWD security level 2 configures the device in a maximally restrictive state. The physical debug port (SW-DP) is completely disabled, and all of the SWD-accessible functions (application debug, mass erase, factory reset, and TI failure analysis) are not accessible through SWD, regardless of their individual configuration.

When level 2 is selected (SW-DP disabled), the application debug configuration and TI failure analysis configuration fields are don't care fields which do not impact the device configuration.

If the BSL is disabled, then the mass erase and factory reset configuration fields are also don't care fields. However, if the BSL is enabled, then the mass erase and factory reset configuration fields are still used by the BSL to authorize mass erase or factory reset commands originating from the BSL interface.

When to Use This State

Use Level 2 only for mass production when no further access to any SWD functions is required and a maximally secure state is desired for the device.

When to Not Use this State

Do not use Level 2 in the following cases:

- Future application debug or reprogramming through SWD is required
- So that TI can perform failure analysis on the device
- To remove proprietary information from the flash memory by sending a mass erase or factory reset command through SWD

Note

After a device is configured for level 2 (SW-DP disabled), further access to the device through SWD **is not possible**. The only way to bring a device back to a level 0 or level 1 state with SWD access restored is if the BSL and factory reset are both enabled (allowing a BSL factory reset command to be sent), or a mechanism in the user application code is included which can change the NONMAIN configuration to a less restrictive state. In either scenario, if the [NONMAIN is locked through static write protection](#) then the level 2 state is not reversible and there is no way to re-gain SWD access.

1.4.2.2 SWD Mass Erase and Factory Reset Commands

The BCR provides mass erase and factory reset functionality through commands sent to the device over SWD from a debug probe using the [debug subsystem mailbox \(DSSM\)](#). These commands are not available in SWD security level 2, but they are optionally available in security level 0 and 1. When the device is not configured for SWD security level 2, the mass erase and factory reset commands can be individually configured to be enabled, enabled with a unique 128-bit password, or disabled. By default, both commands are enabled.

The SWD mass erase and factory reset DSSM commands superseded any static write protection policies. For example, if SWD factory reset is configured to be enabled or enabled with password, the BCR configuration data can be reset even if it is statically write protected.

SWD Mass Erase

A SWD mass erase is an erase of the MAIN flash regions only, which typically includes the user application. The BCR and BSL policies stored in the NONMAIN flash region are not affected by a mass erase. A mass erase is useful for erasing all application code and data while leaving the device configuration itself intact.

To set the mass erase command mode and password, configure the BOOTCFG3.MASSERASECMDACCESS field and the PWDMASSERASE password fields in the NONMAIN memory.

SWD Factory Reset

A SWD factory reset is an erase of the MAIN flash regions followed by a reset of the NONMAIN flash region to default values. Such an erase is useful for completely resetting the BCR and BSL device boot policies while also erasing the application code and data.

To set the factory reset command mode and password, configure the BOOTCFG3.FACTORYRESETCMDACCESS field and the PWDFACTORYRESET password fields in the NONMAIN memory.

1.4.2.3 Flash Memory Protection and Integrity Related Policies

The flash memory protection and integrity policies specify which sectors of flash memory are locked from modification, as well as which sectors are to be checked for integrity during the boot process before the user application is started.

1.4.2.3.1 Locking the Application (MAIN) Flash Memory

MSPM0 MCUs implement a static write protection scheme to lock out user defined sectors in the MAIN flash region from any program or erase operations at runtime. The desired static write protection scheme is configured as a part of the boot security policies in the NONMAIN flash region.

Purpose

Static write protection enables placement of a fixed, user-defined, application in the flash memory that has the following characteristics:

- Once programmed and locked, the application is not modifiable by the application code or ROM bootloader
- If placed at the beginning of the flash memory, the application is the first code that executes when the ROM boot configuration routine transfers execution to the user application

MSPM0 static write protection supports both characteristics, which must be satisfied to implement a secure boot image manager.

Capabilities

Any sector that is configured in the NONMAIN to be write-locked is functionally immutable when the boot configuration routine transfers execution to either the bootstrap loader or the user application code in MAIN flash. Any attempt to program or erase a statically protected sector by the application code or the bootstrap loader results in a hardware flash operation error, and the sector is not modified.

While static write protection prevents any modification by application code or the bootloader, a mass erase or factory reset command sent through the SWD interface is honored. If this behavior is not desired, the mass erase or factory reset SWD commands can be protected with unique passwords or disabled (see the [SWD policies](#)). To completely remove any means of modifying statically write protected MAIN flash sectors, the mass erase and factory reset commands (or the SW-DP) must be disabled, and the NONMAIN boot configuration memory must also be statically write protected to prevent application code from changing the underlying write protection scheme by modifying the contents NONMAIN region. This is discussed in the following section.

1.4.2.3.2 Locking the Configuration (NONMAIN) Flash Memory

MSPM0 MCUs implement a static write protection scheme to lock out the NONMAIN flash region from any program/erase operations at runtime. The write protection scheme is configured as a part of the boot security policies in the NONMAIN flash region.

Purpose

By default from TI, the NONMAIN configuration memory (which contains the user-specified boot security policies and bootstrap loader policies) is not write protected. This enables the NONMAIN to be erased by the user during provisioning and re-programmed with the user-specified policies which will be used in mass production.

In many cases, it is desirable for the configuration memory to be locked once it has been provisioned. Locking the configuration memory has the benefit of preventing any unauthorized modification of the security policies, bootstrap loader policies, and static write protection policies by either the bootstrap loader or the application code itself. In most applications, devices in mass production do not require modification of the configuration memory, even when the device firmware is updated.

Capabilities

When configured to be protected, the entire NONMAIN region will be write-locked and will be functionally immutable when the boot configuration routine transfers execution to either the bootstrap loader or the user application code in MAIN flash. Any attempt to program or erase the NONMAIN by the application code or the bootstrap loader will result in a hardware flash operation error, and the sector will not be modified.

While static write protection prevents any modification by application code or the boot loader, a factory reset command sent through the SWD interface would still be honored. If this behavior is not desired, the factory reset SWD command may be protected with a unique password or disabled altogether (see the [SWD policies](#)). To completely remove any means of modifying the NONMAIN configuration memory, the factory reset command and TI FA (or the SW-DP) must be disabled.

Note

When the NONMAIN is statically write protected, and the factory reset command and TI FA (or the SW-DP) are disabled, the NONMAIN is equivalent to immutable read-only memory, and it is no longer possible to change the device configuration by any means. Further, if any MAIN memory region sectors are configured with static protection, these sectors also can not be modified by any means and may be considered as immutable.

1.4.2.3.3 Static Write Protection NONMAIN Fields

Write protection may be enabled on a per-sector basis for the first 32 sectors of the MAIN flash memory. For the remaining sectors of flash memory, if present, write protection may be enabled per 8 sectors. To set a static write protection policy, configure the FLASHSWP0 and FLASHSWP1 fields in the NONMAIN memory.

Note

[Mass erase and factory reset commands](#) [Factory reset command](#) sent to the BCR via the debug sub system mailbox (DSSM) will override the specified static write protection policy. If this behavior is not desired, configure the mass erase and factory reset commands to be enabled with password or disabled. Note that mass erase and factory reset commands sent to the BSL will respect the specified static write protection policy (the BSL has the same permissions as application code).

1.4.2.4 Application CRC Verification

The BCR supports executing a complete CRC32 integrity check of the application code and data contained in the MAIN flash regions during the boot process before starting the user application. This is useful to ensure the integrity of some or all of the application code and data before executing it.

To enable the CRC32 integrity check at boot, the following information must be programmed into the BCR configuration in NONMAIN flash memory:

- 32-bit starting address of the CRC check (APPCRCSTART.ADDRESS field)
- The length of the application (specified in bytes) for which the CRC check is applied (APPCRCLEN.LENGTH field)
- The precalculated 32-bit CRC value to test against (APPCRC.DIGEST field)
- The enabled key value for the CRC check (BOOTCFG3.APPCRCMODE field)

In the event that an application CRC check fails at boot, the application in MAIN flash is not started. If the boot strap loader is enabled, it is entered. If the BSL is not enabled, then the boot fails.

1.4.2.5 Fast Boot

The execution time of the BCR can be reduced by enabling fast boot mode. Fast boot mode, when enabled, speeds up the boot process using the following methods:

- Limiting the BSL entry methods. When fast boot mode is enabled, only the SYSCTL register invoke method and the DSSM invoke method can be used to enter the bootloader. The other BSL invoke conditions are not tested (for example, pin based invoke).
- Bypassing the [application CRC check](#) (even if the application CRC check is enabled).

To enable fast boot mode, set the fast boot mode to enabled in the BOOTCFG2.FASTBOOTMODE field in the NONMAIN flash memory.

1.4.2.6 Bootstrap Loader (BSL) Enable/Disable Policy

The bootstrap loader (BSL) provides a means to program and verify the device memory through a standard serial interface (UART or I2C), as opposed to the serial wire debug interface. The BSL has its own configuration policy, but the BCR determines if the BSL is enabled to be invoked, or if it is to be disabled (non-invokable).

Since the BSL presents an additional attack surface, if it is not used in an application it may be disabled in the user-specified boot security policies. If the BSL is used in an application, then the BSL security settings (including the BSL access password) are managed in the [BSL configuration policy](#).

1.4.2.6.1 BSL Enable

The bootstrap loader ([BSL](#)) can be enabled or disabled by setting the bootloader mode to enabled or disabled in the BOOTCFG2.BSLMODE field in the NONMAIN flash memory. When the BSL is disabled, the bootloader cannot be entered through any invocation mechanism.

1.4.3 Bootstrap Loader (BSL)

The bootstrap loader (BSL) provides a method to program and/or verify the device memory through a standard UART or I2C serial interface. Key features of the BSL which are accessible through the serial interface include:

- Programming and erase of flash memory
- Ability to return a firmware version number through a pointer to the main flash
- Ability to specify a hardware invoke GPIO
- Ability to enable code/data read-out (disabled by default)
- Ability to return a 32-bit CRC of a code/data region (1KB minimum region size) to verify programming
- Access is always protected with a 256-bit password
- Configurable security alert handling for resisting brute force attacks
- Support for MAIN flash plug-ins to enable additional interfaces beyond UART and I2C

For a complete description of the BSL features, see the BSL user's guide.

The BSL can be completely disabled if desired by properly configuring the BSL mode in the BCR configuration in the NONMAIN flash. See the [BSL Enable](#) section for details on enabling or disabling the BSL.

1.4.3.1 GPIO Invoke

The bootloader supports hardware invoking after a BOOTRST through the use of a GPIO. The BSL configuration in the NONMAIN flash memory contains the pad, pin, and polarity definition for the GPIO invocation. Devices come configured from TI for a specific GPIO and polarity, but software can change this default by modifying the GPIO pin configuration in the BSL configuration in NONMAIN flash memory.

To specify the polarity of the BSL_invoke pin, configure the BSLCONFIG0.BSLIVK_LVL field in the NONMAIN memory.

To specify the device pin to be used for BSL_invoke, configure the following fields in the NONMAIN memory:

- Store the IOMUX PINCMx index into the BSLCONFIG0.BSLIVK_PAD_NUM field
- Store the GPIO port (A or B) into the BSLCONFIG0.BSLIVK_GPIOPORT field
- Store the GPIO pin (0-31) into the BSLCONFIG0.BSLIVK_GPIOPIN field

See the device specific data sheet to determine the default BSL invoke GPIO.

1.4.3.2 Bootstrap Loader (BSL) Security Policies

The BSL security policies are interpreted by the boot loader when it is invoked, and include the following parameters:

- BSL access password, described in [Section 1.4.3.2.1](#)
- BSL read-out policy, described in [Section 1.4.3.2.2](#)
- BSL security alert policy (tamper detection), described in [Section 1.4.3.2.3](#)

1.4.3.2.1 BSL Access Password

Access to the BSL is always protected by a 256-bit user-specified password. There is no option to disable the password. The password must be provided to the BSL after invocation for access to most BSL functions to be granted. When the password is not provided, the only BSL commands allowed are *Get Identity* and *Start Application*.

If a wrong password is provided to the BSL, the BSL halts for 2 seconds, after which an additional attempt can be made to send the correct password. After three failed password attempts, the security alert function is activated (see [Section 1.4.3.2.3](#)).

1.4.3.2.2 BSL Read-out Policy

The BSL optionally supports read-out of the device memory for debug and/or diagnostic purposes (after access to the BSL has been granted with a [correct password match](#)). By default, this capability is disabled for security to prevent extraction of sensitive code and/or data from the device. When the BSL read-out policy is disabled, the only information which may be provided to a host through the BSL interface is a CRC32 digest of a memory segment with a minimum segment length of 1 kilobyte. If direct read-out of the device memory is desired, it may be enabled in the BSL configuration.

1.4.3.2.3 BSL Security Alert Policy

The BSL provides an alert mechanism for taking action when tampering is suspected. Specifically, if an incorrect password is passed to the BSL 3 times during one BSL session, the security alert is activated and the BSL may respond in one of three different ways based on the specified security alert policy:

1. Issue a factory reset (erasing the MAIN flash and resetting the NONMAIN flash regions)
2. Disable the BSL (leaves the MAIN flash intact but re-configures the NONMAIN to block BSL access)
3. Ignore (do not modify the configuration and allow password attempts to continue)

Note

Options 1 and 2 require that the NONMAIN flash region not be statically write protected (see [Section 1.4.2.3.2](#)).

When option 1 is selected, any MAIN memory region which is configured to be statically write protected (see [Section 1.4.2.3.1](#)) will not be erased during the factory reset.

1.4.3.3 Application Version

The BSL supports returning an application version number through the BSL serial interface. This allows the BSL host to interrogate the firmware version without being able to read the firmware. The version field is 32 bits in length. To link the application version command to a version number programmed in the main flash, program the address of the version number in the BSLAPPVER.ADDRESS field in NONMAIN flash memory. The version data is only returned if the address specified in BSLAPPVER.ADDRESS corresponds to a valid flash memory address.

1.4.3.4 BSL Triggered Mass Erase and Factory Reset

It is possible to send a mass erase or factory reset command to the BSL. The commands work in a similar way as the [SWD mass erase and factory reset commands](#), with several key exceptions.

BSL Mass Erase

A mass erase command sent to the BSL will erase the MAIN flash memory. Any MAIN flash memory sectors which are configured to be statically write protected (via the FLASHSWP0 and FLASHSWP1 fields in the NONMAIN configuration memory) will not be erased. The NONMAIN device configuration memory is never erased by a mass erase.

BSL Factory Reset

A factory reset command sent to the BSL will first perform a BSL mass erase to erase the main flash memory (excluding any sectors which are statically write protected). Then, it will additionally erase the NONMAIN device configuration memory.

A BSL factory reset command is only accepted if the following are true:

1. The NONMAIN memory itself is not currently configured to be statically write protected (BOOTCFG4.NONMAINSWP field in the NONMAIN is set to not protected)
2. The factory reset command is not configured to be disabled (BOOTCFG3.FACTORYRESETCMDACCESS field in the NONMAIN is not set to disabled)

The BSL host must program valid configuration data back into the NONMAIN (via BSL commands) before terminating the BSL session, or the device may enter an unrecoverable state.

Note

If the NONMAIN is left unprogrammed after a BSL factory reset, the device will assume a maximally restrictive state on the next reset cycle, any application code in MAIN flash will not be started, and it will not be possible to access the device via any means. To prevent lockout, always ensure that valid configuration data is programmed into NONMAIN following a BSL factory reset.

1.5 NONMAIN Registers

Table 1-7 lists the memory-mapped registers for the NONMAIN registers. All register offset addresses not listed in Table 1-7 should be considered as reserved locations and the register contents should not be modified.

Table 1-7. NONMAIN Registers

Offset	Acronym	Register Name	Group	Section
41C0000 0h	BCRCONFIGID	Configuration ID of BCR Structure		Go
41C0000 4h	BOOTCFG0	Serial wire debug (SWD) lock policy.		Go
41C0000 8h	BOOTCFG1	BSL invoke pin policy.		Go
41C0000 Ch + formula	PWDDEBUGLOCK[y]	SWD command and password authentication request.		Go
41C0001 Ch	BOOTCFG2	Fast boot mode policy and BSL mode policy.		Go
41C0002 0h	BOOTCFG3	Mass erase and factory reset mode policies. These policies affect SWD initiated and BSL initiated mass erase and factory reset commands. If the SW-DP is disabled (SWDP_MODE is disabled), SWD initiated commands are not allowed as the SW-DP is fully disabled. If the BSL is disabled (BSLMODE is disabled), these settings are a a don't care for BSL initiated commands as the BSL is not enabled to be invoked.		Go
41C0002 4h + formula	PWDMASSErase[y]	SWD mass erase command password (must be provided via DSSM to authenticate a mass erase command).		Go
41C0003 4h + formula	PWDFACTORYRESET[y]	SWD factory reset command password (must be provided via DSSM to authenticate a factory reset command).		Go
41C0004 4h	FLASHSWP0	Static write protection policy for the first 32kB of flash memory. When protected, sectors will not be available for program or erase by either the bootloader or application code.		Go
41C0004 8h	FLASHSWP1	Static write protection policy for additional sectors of flash memory. When protected, sectors will not be available for program or erase by either the bootloader or application code.		Go
41C0004 Ch	BOOTCFG4			Go
41C0005 0h	APPCRCSTART	Start address of the application CRC check (must be an address in a MAIN flash region).		Go
41C0005 4h	APPCRCLENGTH	Length of the application area to include in the application CRC check (in bytes), starting from APPCRCSTART.		Go
41C0005 8h	APPCRC	Expected application CRC check digest (CRC-32) to test against during boot.		Go
41C0005 Ch	BOOTCRC	CRC digest (CRC-32) of the BCR (boot configuration) portion of the NONMAIN memory.		Go

Table 1-7. NONMAIN Registers (continued)

Offset	Acronym	Register Name	Group	Section
41C0010 0h	BSLCONFIGID	BSL configuration ID.		Go
41C0010 4h	BSLPINCFG0	BSL UART pin configuration.		Go
41C0010 8h	BSLPINCFG1	BSL I2C pin configuration.		Go
41C0010 Ch	BSLCONFIG0	BSL invoke pin configuration and memory read-out policy.		Go
41C0011 0h + formula	BSLPW[y]	256-bit BSL access password.		Go
41C0013 0h	BSLPLUGINCFG	Defines the presence and type of a BSL plug-in in MAIN flash memory.		Go
41C0013 4h + formula	BSLPLUGINHOOK[y]	Function pointers for plug-in init, receive, transmit, and de-init functions.		Go
41C0014 4h	PATCHHOOKID	Alternate BSL configuration.		Go
41C0014 8h	SBLADDRESS	Address of an alternate BSL.		Go
41C0014 Ch	BSLAPPVER	Address of the application version word.		Go
41C0015 0h	BSLCONFIG1	BSL security configuration.		Go
41C0015 4h	BSLCRC	CRC digest (CRC-32) of the BSL_CONFIG portion of the NONMAIN memory.		Go

Complex bit access types are encoded to fit into small table cells. [Table 1-8](#) shows the codes that are used for access types in this section.

Table 1-8. NONMAIN Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
W	W	Write
Reset or Default Value		
-n		Value after reset or the default value
Register Array Variables		
i,j,k,l,m,n		When these variables are used in a register name, an offset, or an address, they refer to the value of a register array where the register is part of a group of repeating registers. The register groups form a hierarchical structure and the array is represented with a formula.
y		When this variable is used in a register name, an offset, or an address it refers to the value of a register array.

1.5.1 BCRCONFIGID (Offset = 41C00000h) [Reset = 00000001h]

BCRCONFIGID is shown in [Figure 1-2](#) and described in [Table 1-9](#).

Return to the [Summary Table](#).

Configuration ID of BCR Structure

Figure 1-2. BCRCONFIGID

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CONFIG																															
R/W-00000001h																															

Table 1-9. BCRCONFIGID Field Descriptions

Bit	Field	Type	Reset	Description
31-0	CONFIG	R/W	00000001h	Configuration ID of the BOOTCFG

1.5.2 BOOTCFG0 (Offset = 41C00004h) [Reset = AABBAABBh]

BOOTCFG0 is shown in [Figure 1-3](#) and described in [Table 1-10](#).

Return to the [Summary Table](#).

Serial wire debug (SWD) lock policy.

Figure 1-3. BOOTCFG0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWDP_MODE																DEBUGACCESS															
R/W-AABBh																W-AABBh															

Table 1-10. BOOTCFG0 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	SWDP_MODE	R/W	AABBh	The serial wire debug port (SW-DP) access policy. This policy sets whether any communication is allowed with the device via the SWD pins (to any DAP). When disabled, no SWD communication is possible regardless of the configuration of the DEBUGACCESS field. 5566h = The SW-DP is fully disabled and no device access is possible via the SW-DP (0x5566 and all other values NOT 0xAABB). AABBh = The SW-DP is enabled and device access is set by the additional policies in NONMAIN.
15-0	DEBUGACCESS	W	AABBh	The debug access policy for accessing the AHB-AP, ET-AP, and PWR-AP debug access ports. Note that if SWDP_MODE is set to DISABLED, the value of this field is ignored and the debug port will remain fully locked. 5566h = Access to AHB-AP, ET-AP, and PWR-AP via SWD is disabled (0x5566 and all other values NOT 0xCCDD or 0xAABB). AABBh = Access to AHB-AP, ET-AP, and PWR-AP via SWD is enabled. CCDDh = Access to AHB-AP, ET-AP, and PWR-AP via SWD is only enabled when the correct password is provided via the DSSM before BCR execution.

1.5.3 BOOTCFG1 (Offset = 41C00008h) [Reset = AABBAABBh]

BOOTCFG1 is shown in [Figure 1-4](#) and described in [Table 1-11](#).

Return to the [Summary Table](#).

BSL invoke pin policy.

Figure 1-4. BOOTCFG1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BSL_PIN_INVOKE																TI_FA_MODE															
R/W-AABBh																R/W-AABBh															

Table 1-11. BOOTCFG1 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	BSL_PIN_INVOKE	R/W	AABBh	Boot strap loader (BSL) pin invoke method enable/disable policy. 5566h = The BSL_INVOKE pin is not checked during boot (0x5566 and all other values NOT 0xAABB). AABBh = The BSL_INVOKE pin is checked during boot.
15-0	TI_FA_MODE	R/W	AABBh	Sets the TI failure analysis enable/disable policy. If enabled, a re-test request through DSSM is allowed, else it is not allowed. Note that if SWDP_MODE is set to disabled, this field is ignored and failure analysis is not possible. 5566h = TI failure analysis is not allowed (0x5566 and all other values NOT 0xAABB). AABBh = TI failure analysis is allowed.

1.5.4 PWDDEBUGLOCK[y] (Offset = 41C0000Ch + formula) [Reset = FFFFFFFFh]

PWDDEBUGLOCK[y] is shown in [Figure 1-5](#) and described in [Table 1-12](#).

Return to the [Summary Table](#).

SWD command and password authentication request.

Offset = 41C0000Ch + (y * 4h); where y = 0h to 3h

Figure 1-5. PWDDEBUGLOCK[y]

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PW																															
R/W-FFFFFFFFh																															

Table 1-12. PWDDEBUGLOCK[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-0	PW	R/W	FFFFFFFFh	Password

1.5.5 BOOTCFG2 (Offset = 41C0001Ch) [Reset = AABBFfFh]

BOOTCFG2 is shown in [Figure 1-6](#) and described in [Table 1-13](#).

Return to the [Summary Table](#).

Fast boot mode policy and BSL mode policy.

Figure 1-6. BOOTCFG2

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BSLMODE																FASTBOOTMODE															
AAB Bh																FFF Fh															

Table 1-13. BOOTCFG2 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	BSLMODE		AAB Bh	BSLMODE configures the boot strap loader enable/disable policy. 5566h = The BSL is disabled (0x5566 and all other values NOT 0xAAB B). AAB Bh = The BSL is enabled.
15-0	FASTBOOTMODE		FFF Fh	FASTBOOTMODE configures the fast boot mode enable/disable policy. 5566h = Fast boot mode is disabled. All enabled BSL invoke conditions are evaluated (0x5566 and all other values NOT 0xAAB B). AAB Bh = Fast boot mode is enabled. Only the software BSL invoke condition is evaluated.

1.5.6 BOOTCFG3 (Offset = 41C00020h) [Reset = AABBAABBh]

BOOTCFG3 is shown in [Figure 1-7](#) and described in [Table 1-14](#).

Return to the [Summary Table](#).

Mass erase and factory reset mode policies. These policies affect SWD initiated and BSL initiated mass erase and factory reset commands. If the SW-DP is disabled (SWDP_MODE is disabled), SWD initiated commands are not allowed as the SW-DP is fully disabled. If the BSL is disabled (BSLMODE is disabled), these settings are a don't care for BSL initiated commands as the BSL is not enabled to be invoked.

Figure 1-7. BOOTCFG3

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
FACTORYRESETCMDACCESS															
R/W-AABBh															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MASSERASECMDACCESS															
R/W-AABBh															

Table 1-14. BOOTCFG3 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	FACTORYRESETCMDACCESS	R/W	AABBh	The factory reset command policy. 5566h = The factory reset command is not allowed (0x5566 and all other values NOT 0xAABB or 0xCCDD). AABBh = The factory reset command is allowed. CCDDh = The factory reset command is allowed only when the matching password is provided via the DSSM.
15-0	MASSERASECMDACCESS	R/W	AABBh	The mass erase command policy. 5566h = The mass erase command is not allowed (0x5566 and all other values NOT 0xAABB or 0xCCDD). AABBh = The mass erase command is allowed. CCDDh = The mass erase command is allowed only when the matching password is provided via the DSSM.

1.5.7 PWDMASSERASE[y] (Offset = 41C00024h + formula) [Reset = FFFFFFFFh]

PWDMASSERASE[y] is shown in [Figure 1-8](#) and described in [Table 1-15](#).

Return to the [Summary Table](#).

SWD mass erase command password (must be provided via DSSM to authenticate a mass erase command).

Offset = 41C00024h + (y * 4h); where y = 0h to 3h

Figure 1-8. PWDMASSERASE[y]

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PW																															
R/W-FFFFFFFh																															

Table 1-15. PWDMASSERASE[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-0	PW	R/W	FFFFFFFh	Password

1.5.8 PWDFACTORYRESET[y] (Offset = 41C00034h + formula) [Reset = FFFFFFFFh]

PWDFACTORYRESET[y] is shown in [Figure 1-9](#) and described in [Table 1-16](#).

Return to the [Summary Table](#).

SWD factory reset command password (must be provided via DSSM to authenticate a factory reset command).

Offset = 41C00034h + (y * 4h); where y = 0h to 3h

Figure 1-9. PWDFACTORYRESET[y]

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PW																															
R/W-FFFFFFFh																															

Table 1-16. PWDFACTORYRESET[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-0	PW	R/W	FFFFFFFh	Password

1.5.9 FLASHSWP0 (Offset = 41C00044h) [Reset = FFFFFFFFh]

FLASHSWP0 is shown in [Figure 1-10](#) and described in [Table 1-17](#).

Return to the [Summary Table](#).

Static write protection policy for the first 32kB of flash memory. When protected, sectors will not be available for program or erase by either the bootloader or application code.

Figure 1-10. FLASHSWP0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MAINLOW																															
R/W-FFFFFFFh																															

Table 1-17. FLASHSWP0 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	MAINLOW	R/W	FFFFFFFh	1 bit per sector (Setting a bit to 0 disables write, 1 enables write).

1.5.10 FLASHSWP1 (Offset = 41C00048h) [Reset = FFFFFFFFh]

FLASHSWP1 is shown in [Figure 1-11](#) and described in [Table 1-18](#).

Return to the [Summary Table](#).

Static write protection policy for additional sectors of flash memory. When protected, sectors will not be available for program or erase by either the bootloader or application code.

Figure 1-11. FLASHSWP1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MAINHIGH																															
R/W-FFFFFFFh																															

Table 1-18. FLASHSWP1 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	MAINHIGH	R/W	FFFFFFFh	1 bit per 8 sectors. Bits 3:0, not used as covered with FLASHSWP0. (Setting a bit to 0 disables write, 1 enables write)

1.5.11 BOOTCFG4 (Offset = 41C0004Ch) [Reset = FFFFFFFFh]

BOOTCFG4 is shown in [Figure 1-12](#) and described in [Table 1-19](#).

Return to the [Summary Table](#).

Figure 1-12. BOOTCFG4

31	30	29	28	27	26	25	24
APPCRCMODE							
R/W-FFFFh							
23	22	21	20	19	18	17	16
APPCRCMODE							
R/W-FFFFh							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED							NONMAINSWP
R/W-							R/W-FFFFh

Table 1-19. BOOTCFG4 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	APPCRCMODE	R/W	FFFFh	APPCRCMODE enables or disables the boot time CRC check of a segment of MAIN flash memory. 5566h = The boot time MAIN flash CRC check is disabled. The application code in MAIN flash is always started unless the reset vector or stack pointer are blank/unprogrammed (0x5566 and all other values NOT 0xAABB). AABBh = The boot time MAIN flash CRC check is enabled. If the boot time CRC check passes, the application code in MAIN flash is started unless the reset vector or stack pointer are blank (unprogrammed). In the event of a failing CRC check, the application code in MAIN flash will not be started and the boot process fails.
15-1	RESERVED	R/W	0h	
0	NONMAINSWP	R/W	FFFFh	Static write protection policy for entire NONMAIN device configuration memory. Setting bit to 0 disables program/erase of the NONMAIN by all means other than a SWD-initiated factory reset, 1 enables program/erase of the NONMAIN by normal means.

1.5.12 APPCRCSTART (Offset = 41C00050h) [Reset = FFFFFFFFh]

APPCRCSTART is shown in [Figure 1-13](#) and described in [Table 1-20](#).

Return to the [Summary Table](#).

Start address of the application CRC check (must be an address in a MAIN flash region).

Figure 1-13. APPCRCSTART

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADDRESS																															
R/W-FFFFFFFh																															

Table 1-20. APPCRCSTART Field Descriptions

Bit	Field	Type	Reset	Description
31-0	ADDRESS	R/W	FFFFFFFh	Application CRC check start address

1.5.13 APPCRLENGTH (Offset = 41C00054h) [Reset = FFFFFFFFh]

APPCRLENGTH is shown in [Figure 1-14](#) and described in [Table 1-21](#).

Return to the [Summary Table](#).

Length of the application area to include in the application CRC check (in bytes), starting from APPCRCSTART.

Figure 1-14. APPCRLENGTH

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LENGTH																															
R/W-FFFFFFFh																															

Table 1-21. APPCRLENGTH Field Descriptions

Bit	Field	Type	Reset	Description
31-0	LENGTH	R/W	FFFFFFFh	Application CRC check source data length

1.5.14 APPCRC (Offset = 41C00058h) [Reset = FFFFFFFFh]

APPCRC is shown in [Figure 1-15](#) and described in [Table 1-22](#).

Return to the [Summary Table](#).

Expected application CRC check digest (CRC-32) to test against during boot.

Figure 1-15. APPCRC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIGEST																															
R/W-FFFFFFFFh																															

Table 1-22. APPCRC Field Descriptions

Bit	Field	Type	Reset	Description
31-0	DIGEST	R/W	FFFFFFFFh	Application CRC check expected digest.

1.5.15 BOOTCRC (Offset = 41C0005Ch) [Reset = 1879DAC3h]

BOOTCRC is shown in [Figure 1-16](#) and described in [Table 1-23](#).

Return to the [Summary Table](#).

CRC digest (CRC-32) of the BCR (boot configuration) portion of the NONMAIN memory.

Figure 1-16. BOOTCRC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIGEST																															
R/W-1879DAC3h																															

Table 1-23. BOOTCRC Field Descriptions

Bit	Field	Type	Reset	Description
31-0	DIGEST	R/W	1879DAC3h	BCR boot configuration data CRC digest.

1.5.16 BSLCONFIGID (Offset = 41C00100h) [Reset = 00000001h]

BSLCONFIGID is shown in [Figure 1-17](#) and described in [Table 1-24](#).

Return to the [Summary Table](#).

BSL configuration ID.

Figure 1-17. BSLCONFIGID

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CONFIG																															
R/W-00000001h																															

Table 1-24. BSLCONFIGID Field Descriptions

Bit	Field	Type	Reset	Description
31-0	CONFIG	R/W	00000001h	Configuration ID of the BSL_CONFIG.

1.5.17 BSLPINCFG0 (Offset = 41C00104h) [Reset = 02180217h]

BSLPINCFG0 is shown in [Figure 1-18](#) and described in [Table 1-25](#).

Return to the [Summary Table](#).

BSL UART pin configuration.

Figure 1-18. BSLPINCFG0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
UARTTX_MUX_SEL								UARTTX_PAD_NUM							
R/W-02h								R/W-18h							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
UARTRX_MUX_SEL								UARTRX_PAD_NUM							
R/W-02h								R/W-17h							

Table 1-25. BSLPINCFG0 Field Descriptions

Bit	Field	Type	Reset	Description
31-24	UARTTX_MUX_SEL	R/W	02h	UART TX IOMUX PINCM mux selection.
23-16	UARTTX_PAD_NUM	R/W	18h	UART TX IOMUX PINCM register.
15-8	UARTRX_MUX_SEL	R/W	02h	UART RX IOMUX PINCM mux selection.
7-0	UARTRX_PAD_NUM	R/W	17h	UART RX IOMUX PINCM register.

1.5.18 BSLPINCFG1 (Offset = 41C00108h) [Reset = 03020301h]

BSLPINCFG1 is shown in [Figure 1-19](#) and described in [Table 1-26](#).

Return to the [Summary Table](#).

BSL I2C pin configuration.

Figure 1-19. BSLPINCFG1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
I2CSCL_MUX_SEL								I2CSCL_PAD_NUM							
R/W-03h								R/W-2h							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
I2CSDA_MUX_SEL								I2CSDA_PAD_NUM							
R/W-03h								R/W-1h							

Table 1-26. BSLPINCFG1 Field Descriptions

Bit	Field	Type	Reset	Description
31-24	I2CSCL_MUX_SEL	R/W	03h	I2C SCL IOMUX PINCM mux selection.
23-16	I2CSCL_PAD_NUM	R/W	2h	I2C SCL IOMUX PINCM register.
15-8	I2CSDA_MUX_SEL	R/W	03h	I2C SDA IOMUX PINCM mux selection.
7-0	I2CSDA_PAD_NUM	R/W	1h	I2C SDA IOMUX PINCM register.

1.5.19 BSLCONFIG0 (Offset = 41C0010Ch) [Reset = FFFF1293h]

BSLCONFIG0 is shown in [Figure 1-20](#) and described in [Table 1-27](#).

Return to the [Summary Table](#).

BSL invoke pin configuration and memory read-out policy.

Figure 1-20. BSLCONFIG0

31	30	29	28	27	26	25	24
READOUTEN							
R/W-FFFFh							
23	22	21	20	19	18	17	16
READOUTEN							
R/W-FFFFh							
15	14	13	12	11	10	9	8
RESERVED		BSLIVK_GPIOP ORT	BSLIVK_GPIOPIN				
R-0h		R/W-	R/W-12h				
7	6	5	4	3	2	1	0
BSLIVK_LVL	RESERVED	BSLIVK_PAD_NUM					
R/W-1h	R-0h	R/W-13h					

Table 1-27. BSLCONFIG0 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	READOUTEN	R/W	FFFFh	Sets the memory read-out policy for the BSL interface. 5566h = Memory read-out is not possible via the BSL interface (0x5566 and all other values NOT 0xAABB). AABBh = Memory contents may be read via the BSL interface.
15-14	RESERVED	R	0h	
13	BSLIVK_GPIOPORT	R/W	0h	The BSL_invoke GPIO port index corresponding to the pad used for BSL_invoke. 0h = The BSL_invoke pin is on GPIO port A. 1h = The BSL_invoke pin is on GPIO port B.
12-8	BSLIVK_GPIOPIN	R/W	12h	The BSL_invoke GPIO pin index corresponding to the pad used for BSL_invoke.
7	BSLIVK_LVL	R/W	1h	The BSL_invoke input logic level which shall invoke the BSL. 0h = LOW 1h = HIGH
6	RESERVED	R	0h	
5-0	BSLIVK_PAD_NUM	R/W	13h	The IOMUX PINCM register corresponding to the pad to be used for BSL_invoke.

1.5.20 BSLPW[y] (Offset = 41C00110h + formula) [Reset = FFFFFFFFh]

BSLPW[y] is shown in [Figure 1-21](#) and described in [Table 1-28](#).

Return to the [Summary Table](#).

256-bit BSL access password.

Offset = 41C00110h + (y * 4h); where y = 0h to 7h

Figure 1-21. BSLPW[y]

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PASSWORD																															
R/W-FFFFFFFFh																															

Table 1-28. BSLPW[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-0	PASSWORD	R/W	FFFFFFFFh	Password

1.5.21 BSLPLUGINCFG (Offset = 41C00130h) [Reset = FFFFFFFFh]

BSLPLUGINCFG is shown in [Figure 1-22](#) and described in [Table 1-29](#).

Return to the [Summary Table](#).

Defines the presence and type of a BSL plug-in in MAIN flash memory.

Figure 1-22. BSLPLUGINCFG

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SRAMEXISTS								FLASHEXISTS							
R/W-FFh								R/W-FFh							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PLUGINTYPE															
R/W-FFFFh															

Table 1-29. BSLPLUGINCFG Field Descriptions

Bit	Field	Type	Reset	Description
31-24	SRAMEXISTS	R/W	FFh	SRAM consumed by Flash plugin, from 0x00 to 0xFF.
23-16	FLASHEXISTS	R/W	FFh	The field tells if Flash Plugin exists are not. 0xBB - Flash Plugin exists; 0xFF (all other values) - Only ROM plugins will be used.
15-0	PLUGINTYPE	R/W	FFFFh	The type code for the BSL plug-in. 1000h = Plug-in is for UART. 2000h = Plug-in is for I2C. FFFFh = For all other values. Any other interfaces with valid hooks will be added to Plugin list.

1.5.22 BSLPLUGINHOOK[y] (Offset = 41C00134h + formula) [Reset = FFFFFFFFh]

BSLPLUGINHOOK[y] is shown in [Figure 1-23](#) and described in [Table 1-30](#).

Return to the [Summary Table](#).

Function pointers for plug-in init, receive, transmit, and de-init functions.

Offset = 41C00134h + (y * 4h); where y = 0h to 3h

Figure 1-23. BSLPLUGINHOOK[y]

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BSLPLUGIN																															
R/W-FFFFFFFFh																															

Table 1-30. BSLPLUGINHOOK[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-0	BSLPLUGIN	R/W	FFFFFFFFh	Address of the BSL plug-in hook. Byte [3-0] : Init; Byte [7-4] : Receive; Byte [11-8] : Send; Byte [15-12] : Deinit

1.5.23 PATCHHOOKID (Offset = 41C00144h) [Reset = FFFFFFFFh]

PATCHHOOKID is shown in [Figure 1-24](#) and described in [Table 1-31](#).

Return to the [Summary Table](#).

Alternate BSL configuration.

Figure 1-24. PATCHHOOKID

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																ID															
R/W-																R/W-FFFFFFFFh															

Table 1-31. PATCHHOOKID Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15-0	ID	R/W	FFFFFFFFh	ID field to invoke an alternate BSL. 5566h = Do not use an alternate BSL (0x5566 and all other values NOT 0xAABB). AABBh = Use the alternate BSL.

1.5.24 SBLADDRESS (Offset = 41C00148h) [Reset = FFFFFFFFh]

SBLADDRESS is shown in [Figure 1-25](#) and described in [Table 1-32](#).

Return to the [Summary Table](#).

Address of an alternate BSL.

Figure 1-25. SBLADDRESS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADDRESS																															
R/W-FFFFFFFh																															

Table 1-32. SBLADDRESS Field Descriptions

Bit	Field	Type	Reset	Description
31-0	ADDRESS	R/W	FFFFFFFh	Address of the alternate BSL, if present.

1.5.25 BSLAPPVER (Offset = 41C0014Ch) [Reset = FFFFFFFFh]

BSLAPPVER is shown in [Figure 1-26](#) and described in [Table 1-33](#).

Return to the [Summary Table](#).

Address of the application version word.

Figure 1-26. BSLAPPVER

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADDRESS																															
R/W-FFFFFFFh																															

Table 1-33. BSLAPPVER Field Descriptions

Bit	Field	Type	Reset	Description
31-0	ADDRESS	R/W	FFFFFFFh	Address of the application version word (must be a valid flash address to be returned).

1.5.26 BSLCONFIG1 (Offset = 41C00150h) [Reset = 0048FFFFh]

BSLCONFIG1 is shown in [Figure 1-27](#) and described in [Table 1-34](#).

Return to the [Summary Table](#).

BSL security configuration.

Figure 1-27. BSLCONFIG1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TARGETADDR																ALERTACTION															
R/W-0048h																R/W-FFFFh															

Table 1-34. BSLCONFIG1 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	TARGETADDR	R/W	0048h	I2C target address to be used for the BSL I2C communication.
15-0	ALERTACTION	R/W	FFFFh	Action to take upon a security alert condition. 5566h = Ignore the security alert condition (0x5566 and all other values NOT 0xAABB or 0xCCDD). AABBh = Trigger a factory reset. Note that if sectors in MAIN or NONMAIN flash are write protected they will not be affected by the BSL factory reset. CCDDh = Re-configure the NONMAIN region to disable the BSL. This is not supported if the NONMAIN region is configured to be write protected.

1.5.27 BSLCRC (Offset = 41C00154h) [Reset = 8C76DE95h]

BSLCRC is shown in [Figure 1-28](#) and described in [Table 1-35](#).

Return to the [Summary Table](#).

CRC digest (CRC-32) of the BSL_CONFIG portion of the NONMAIN memory.

Figure 1-28. BSLCRC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIGEST																															
R/W-8C76DE95h																															

Table 1-35. BSLCRC Field Descriptions

Bit	Field	Type	Reset	Description
31-0	DIGEST	R/W	8C76DE95h	BSL configuration data CRC digest

1.6 Factory Constants

All devices include a memory-mapped FACTORY region which provides read-only data describing the capabilities of a device as well as any factory-provided trim information for use by application software.

Key data provided in the FACTORY memory region includes:

- The device unique 96-bit identity
- The default [BSL](#) pins
- The total MAIN region flash memory size (in KB)
- The total DATA region flash memory size (in KB), if present
- The flash bank count
- The total SRAM memory size (in KB)
- The temperature sensor calibration value
- The SYSPLL startup parameters (if SYSPLL is present, else reserved)

1.6.1 FACTORYREGION Registers

Table 1-36 lists the memory-mapped registers for the FACTORYREGION registers. All register offset addresses not listed in Table 1-36 should be considered as reserved locations and the register contents should not be modified.

Table 1-36. FACTORYREGION Registers

Offset	Acronym	Register Name	Group	Section
41C4000 0h	TRACEID	Trace identifier		Go
41C4000 4h	DEVICEID	Device identifier		Go
41C4000 8h	USERID	Device variant identifier		Go
41C4000 Ch	BSLPIN_UART			Go
41C4001 0h	BSLPIN_I2C			Go
41C4001 4h	BSLPIN_INVOKE			Go
41C4001 8h	SRAMFLASH			Go
41C4001 Ch	PLLSTARTUP0_4_8MHZ			Go
41C4002 0h	PLLSTARTUP1_4_8MHZ	System PLL Parameter 1 MMR --- Data from Flash Table Lookup		Go
41C4002 4h	PLLSTARTUP0_8_16MHZ			Go
41C4002 8h	PLLSTARTUP1_8_16MHZ	System PLL Parameter 1 MMR --- Data from Flash Table Lookup		Go
41C4002 Ch	PLLSTARTUP0_16_32MHZ			Go
41C4003 0h	PLLSTARTUP1_16_32MHZ	System PLL Parameter 1 MMR --- Data from Flash Table Lookup		Go
41C4003 4h	PLLSTARTUP0_32_48MHZ			Go
41C4003 8h	PLLSTARTUP1_32_48MHZ	System PLL Parameter 1 MMR --- Data from Flash Table Lookup		Go
41C4003 Ch	TEMP_SENSE0	Temperature sensor room temperature calibration code. This is ADC conversion results of temperature sensor output voltage. Included in BOOTCRC calculation.		Go
41C4004 0h	RESERVED0			Go
41C4004 4h	RESERVED1			Go
41C4004 8h	RESERVED2			Go
41C4004 Ch	RESERVED3			Go
41C4005 0h	RESERVED4			Go
41C4005 4h	RESERVED5			Go
41C4005 8h	RESERVED6			Go

Table 1-36. FACTORYREGION Registers (continued)

Offset	Acronym	Register Name	Group	Section
41C4005	RESERVED7 Ch			Go
41C4006	RESERVED8 0h			Go
41C4006	RESERVED9 4h			Go
41C4006	RESERVED10 8h			Go
41C4006	RESERVED11 Ch			Go
41C4007	RESERVED12 0h			Go
41C4007	RESERVED13 4h			Go
41C4007	RESERVED14 8h			Go
41C4007	BOOTCRC Ch	BOOTCRC records the 32-bit CRC of all locations in OPEN including reserved locations.		Go

Complex bit access types are encoded to fit into small table cells. [Table 1-37](#) shows the codes that are used for access types in this section.

Table 1-37. FACTORYREGION Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
W	W	Write
Reset or Default Value		
-n		Value after reset or the default value

1.6.1.1 TRACEID (Offset = 41C40000h) [Reset = 00000000h]

TRACEID is shown in [Figure 1-29](#) and described in [Table 1-38](#).

Return to the [Summary Table](#).

Unique value per device shipped

Figure 1-29. TRACEID

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA																															
R-																															

Table 1-38. TRACEID Field Descriptions

Bit	Field	Type	Reset	Description
31-0	DATA	R	0h	

1.6.1.2 DEVICEID (Offset = 41C40004h) [Reset = 0BB8802Fh]

DEVICEID is shown in [Figure 1-30](#) and described in [Table 1-39](#).

Return to the [Summary Table](#).

Device identifier (die revision specific)

Figure 1-30. DEVICEID

31	30	29	28	27	26	25	24
VERSION				PARTNUM			
R-0h				R-BB88h			
23	22	21	20	19	18	17	16
PARTNUM							
R-BB88h							
15	14	13	12	11	10	9	8
PARTNUM				MANUFACTURER			
R-BB88h				R-17h			
7	6	5	4	3	2	1	0
MANUFACTURER							ALWAYS_1
R-17h							R-1h

Table 1-39. DEVICEID Field Descriptions

Bit	Field	Type	Reset	Description
31-28	VERSION	R	0h	Revision of the device
27-12	PARTNUM	R	BB88h	Part number of the device
11-1	MANUFACTURER	R	17h	TI's JEDEC bank and company code
0	ALWAYS_1	R	1h	This is always 1

1.6.1.3 USERID (Offset = 41C40008h) [Reset = 80000000h]

USERID is shown in [Figure 1-31](#) and described in [Table 1-40](#).

Return to the [Summary Table](#).

per Connectivity format, defines the variant feature set

Figure 1-31. USERID

31	30	29	28	27	26	25	24
START	MAJORREV			MINORREV			
R-1h	R-			R-			
23	22	21	20	19	18	17	16
VARIANT							
R-							
15	14	13	12	11	10	9	8
PART							
R-							
7	6	5	4	3	2	1	0
PART							
R-							

Table 1-40. USERID Field Descriptions

Bit	Field	Type	Reset	Description
31	START	R	1h	
30-28	MAJORREV	R	0h	Monotonic increasing value indicating a new revision of the SKU significant enough that users of the device may have to revise PCB or software design
27-24	MINORREV	R	0h	Monotonic increasing value indicating a new revision of the SKU that preserves compatibility with lesser minor rev values. New capability may be introduced such that lesser minor rev numbers may not be compatible with greater if the new capability is used.
23-16	VARIANT	R	0h	Bit pattern uniquely identifying the variant of a part
15-0	PART	R	0h	Bit pattern that uniquely identifying a part

1.6.1.4 BSLPIN_UART (Offset = 41C4000Ch) [Reset = 02180219h]

BSLPIN_UART is shown in [Figure 1-32](#) and described in [Table 1-41](#).

Return to the [Summary Table](#).

Figure 1-32. BSLPIN_UART

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
UART_TXT_PF								UART_TXD_PAD							
R-2h								R-18h							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
UART_RXD_PF								UART_RXD_PAD							
R-2h								R-19h							

Table 1-41. BSLPIN_UART Field Descriptions

Bit	Field	Type	Reset	Description
31-24	UART_TXT_PF	R	2h	
23-16	UART_TXD_PAD	R	18h	
15-8	UART_RXD_PF	R	2h	
7-0	UART_RXD_PAD	R	19h	

1.6.1.5 BSLPIN_I2C (Offset = 41C40010h) [Reset = 03020301h]

BSLPIN_I2C is shown in [Figure 1-33](#) and described in [Table 1-42](#).

Return to the [Summary Table](#).

Figure 1-33. BSLPIN_I2C

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
I2C_SCL_PF								I2C_SCL_PAD							
R-3h								R-2h							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
I2C_SDA_PF								I2C_SDA_PAD							
R-3h								R-1h							

Table 1-42. BSLPIN_I2C Field Descriptions

Bit	Field	Type	Reset	Description
31-24	I2C_SCL_PF	R	3h	
23-16	I2C_SCL_PAD	R	2h	
15-8	I2C_SDA_PF	R	3h	
7-0	I2C_SDA_PAD	R	1h	

1.6.1.6 BSLPIN_INVOKE (Offset = 41C40014h) [Reset = 000001ABh]

BSLPIN_INVOKE is shown in [Figure 1-34](#) and described in [Table 1-43](#).

Return to the [Summary Table](#).

Figure 1-34. BSLPIN_INVOKE

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							
R-							
15	14	13	12	11	10	9	8
GPIO_REG_SEL		GPIO_PIN_SEL					
R-		R-1h					
7	6	5	4	3	2	1	0
GPIO_LEVEL		BSL_PAD					
R-1h		R-2Bh					

Table 1-43. BSLPIN_INVOKE Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15-14	GPIO_REG_SEL	R	0h	
13-8	GPIO_PIN_SEL	R	1h	
7	GPIO_LEVEL	R	1h	
6-0	BSL_PAD	R	2Bh	

1.6.1.7 SRAMFLASH (Offset = 41C40018h) [Reset = 00200080h]

SRAMFLASH is shown in [Figure 1-35](#) and described in [Table 1-44](#).

Return to the [Summary Table](#).

Figure 1-35. SRAMFLASH

31	30	29	28	27	26	25	24
DATAFLASH_SZ						SRAM_SZ	
R-0h						R-20h	
23	22	21	20	19	18	17	16
SRAM_SZ							
R-20h							
15	14	13	12	11	10	9	8
RESERVED		MAINNUMBANKS			MAINFLASH_SZ		
R-		R-0h			R-80h		
7	6	5	4	3	2	1	0
MAINFLASH_SZ							
R-80h							

Table 1-44. SRAMFLASH Field Descriptions

Bit	Field	Type	Reset	Description
31-26	DATAFLASH_SZ	R	0h	The encoding of the field is that the value of the field is an integer to be interpreted as number of KB. For example, if the value of the field id 4, then it is 4KB, if the value is 32, then 32KB, and so on.
25-16	SRAM_SZ	R	20h	The encoding of the field is that the value of the field is an integer to be interpreted as number of KB. For example, if the value of the field id 4, then it is 4KB, if the value is 32, then 32KB, and so on.
15-14	RESERVED	R	0h	
13-12	MAINNUMBANKS	R	0h	
11-0	MAINFLASH_SZ	R	80h	The encoding of the field is that the value of the field is an integer to be interpreted as number of KB. For example, if the value of the field id 4, then it is 4KB, if the value is 32, then 32KB, and so on.

1.6.1.8 PLLSTARTUP0_4_8MHZ (Offset = 41C4001Ch) [Reset = 00004496h]

PLLSTARTUP0_4_8MHZ is shown in [Figure 1-36](#) and described in [Table 1-45](#).

Return to the [Summary Table](#).

Figure 1-36. PLLSTARTUP0_4_8MHZ

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
CAPBOVERRI DE	CAPBVAL					CPCURRENT	
R/W-	R/W-			R/W-4h			
15	14	13	12	11	10	9	8
CPCURRENT				STARTTIMELP			
R/W-4h				R/W-12h			
7	6	5	4	3	2	1	0
STARTTIMELP		STARTTIME					
R/W-12h				R/W-16h			

Table 1-45. PLLSTARTUP0_4_8MHZ Field Descriptions

Bit	Field	Type	Reset	Description
31-24	RESERVED	R	0h	
23	CAPBOVERRIDE	R/W	0h	Override Enable For Cap B 0h = 0 1h = 1
22-18	CAPBVAL	R/W	0h	Override Value for Cap B
17-12	CPCURRENT	R/W	4h	Charge Pump Current
11-6	STARTTIMELP	R/W	12h	Startup time from Low Power Exit to Locked Clock in resolution of 1usec
5-0	STARTTIME	R/W	16h	Startup time from Enable to Locked Clock in resolution of 1usec

1.6.1.9 PLLSTARTUP1_4_8MHZ (Offset = 41C40020h) [Reset = 0000805Ch]

PLLSTARTUP1_4_8MHZ is shown in [Figure 1-37](#) and described in [Table 1-46](#).

Return to the [Summary Table](#).

Figure 1-37. PLLSTARTUP1_4_8MHZ

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED									LPFRESC						
R-									R/W-1h						
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LPFRESC		LPFRESA									LPFCAPA				
R/W-1h					R/W-2h						R/W-1Ch				

Table 1-46. PLLSTARTUP1_4_8MHZ Field Descriptions

Bit	Field	Type	Reset	Description
31-23	RESERVED	R	0h	
22-15	LPFRESC	R/W	1h	Loop Filter Res C
14-5	LPFRESA	R/W	2h	Loop Filter Res A
4-0	LPFCAPA	R/W	1Ch	Loop Filter Cap A

1.6.1.10 PLLSTARTUP0_8_16MHZ (Offset = 41C40024h) [Reset = 000052CFh]

PLLSTARTUP0_8_16MHZ is shown in [Figure 1-38](#) and described in [Table 1-47](#).

Return to the [Summary Table](#).

Figure 1-38. PLLSTARTUP0_8_16MHZ

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
CAPBOVERRI DE	CAPBVAL					CPCURRENT	
R/W-	R/W-			R/W-5h			
15	14	13	12	11	10	9	8
CPCURRENT				STARTTIMELP			
R/W-5h				R/W-Bh			
7	6	5	4	3	2	1	0
STARTTIMELP		STARTTIME					
R/W-Bh		R/W-Fh					

Table 1-47. PLLSTARTUP0_8_16MHZ Field Descriptions

Bit	Field	Type	Reset	Description
31-24	RESERVED	R	0h	
23	CAPBOVERRIDE	R/W	0h	Override Enable For Cap B 0h = 0 1h = 1
22-18	CAPBVAL	R/W	0h	Override Value for Cap B
17-12	CPCURRENT	R/W	5h	Charge Pump Current
11-6	STARTTIMELP	R/W	Bh	Startup time from Low Power Exit to Locked Clock in resolution of 1usec
5-0	STARTTIME	R/W	Fh	Startup time from Enable to Locked Clock in resolution of 1usec

1.6.1.11 PLLSTARTUP1_8_16MHZ (Offset = 41C40028h) [Reset = 00008030h]

PLLSTARTUP1_8_16MHZ is shown in [Figure 1-39](#) and described in [Table 1-48](#).

Return to the [Summary Table](#).

Figure 1-39. PLLSTARTUP1_8_16MHZ

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED									LPFRESC						
R-									R/W-1h						
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LPFRE SC		LPFRESA									LPFCAPA				
R/W-1h		R/W-1h									R/W-10h				

Table 1-48. PLLSTARTUP1_8_16MHZ Field Descriptions

Bit	Field	Type	Reset	Description
31-23	RESERVED	R	0h	
22-15	LPFRESC	R/W	1h	Loop Filter Res C
14-5	LPFRESA	R/W	1h	Loop Filter Res A
4-0	LPFCAPA	R/W	10h	Loop Filter Cap A

1.6.1.12 PLLSTARTUP0_16_32MHZ (Offset = 41C4002Ch) [Reset = 0000520Ch]

PLLSTARTUP0_16_32MHZ is shown in [Figure 1-40](#) and described in [Table 1-49](#).

Return to the [Summary Table](#).

Figure 1-40. PLLSTARTUP0_16_32MHZ

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
CAPBOVERRI DE	CAPBVAL					CPCURRENT	
R/W-	R/W-				R/W-5h		
15	14	13	12	11	10	9	8
CPCURRENT				STARTTIMELP			
R/W-5h				R/W-8h			
7	6	5	4	3	2	1	0
STARTTIMELP		STARTTIME					
R/W-8h		R/W-Ch					

Table 1-49. PLLSTARTUP0_16_32MHZ Field Descriptions

Bit	Field	Type	Reset	Description
31-24	RESERVED	R	0h	
23	CAPBOVERRIDE	R/W	0h	Override Enable For Cap B 0h = 0 1h = 1
22-18	CAPBVAL	R/W	0h	Override Value for Cap B
17-12	CPCURRENT	R/W	5h	Charge Pump Current
11-6	STARTTIMELP	R/W	8h	Startup time from Low Power Exit to Locked Clock in resolution of 1usec
5-0	STARTTIME	R/W	Ch	Startup time from Enable to Locked Clock in resolution of 1usec

1.6.1.13 PLLSTARTUP1_16_32MHZ (Offset = 41C40030h) [Reset = 00008030h]

PLLSTARTUP1_16_32MHZ is shown in [Figure 1-41](#) and described in [Table 1-50](#).

Return to the [Summary Table](#).

Figure 1-41. PLLSTARTUP1_16_32MHZ

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED									LPFRESC						
R-									R/W-1h						
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LPFRESC		LPFRESA									LPFCAPA				
R/W-1h		R/W-1h									R/W-10h				

Table 1-50. PLLSTARTUP1_16_32MHZ Field Descriptions

Bit	Field	Type	Reset	Description
31-23	RESERVED	R	0h	
22-15	LPFRESC	R/W	1h	Loop Filter Res C
14-5	LPFRESA	R/W	1h	Loop Filter Res A
4-0	LPFCAPA	R/W	10h	Loop Filter Cap A

1.6.1.14 PLLSTARTUP0_32_48MHZ (Offset = 41C40034h) [Reset = 0000518Ah]

PLLSTARTUP0_32_48MHZ is shown in [Figure 1-42](#) and described in [Table 1-51](#).

Return to the [Summary Table](#).

Figure 1-42. PLLSTARTUP0_32_48MHZ

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
CAPBOVERRI DE	CAPBVAL					CPCURRENT	
R/W-	R/W-				R/W-5h		
15	14	13	12	11	10	9	8
CPCURRENT				STARTTIMELP			
R/W-5h				R/W-6h			
7	6	5	4	3	2	1	0
STARTTIMELP		STARTTIME					
R/W-6h		R/W-Ah					

Table 1-51. PLLSTARTUP0_32_48MHZ Field Descriptions

Bit	Field	Type	Reset	Description
31-24	RESERVED	R	0h	
23	CAPBOVERRIDE	R/W	0h	Override Enable For Cap B 0h = 0 1h = 1
22-18	CAPBVAL	R/W	0h	Override Value for Cap B
17-12	CPCURRENT	R/W	5h	Charge Pump Current
11-6	STARTTIMELP	R/W	6h	Startup time from Low Power Exit to Locked Clock in resolution of 1usec
5-0	STARTTIME	R/W	Ah	Startup time from Enable to Locked Clock in resolution of 1usec

1.6.1.15 PLLSTARTUP1_32_48MHZ (Offset = 41C40038h) [Reset = 00008030h]

PLLSTARTUP1_32_48MHZ is shown in [Figure 1-43](#) and described in [Table 1-52](#).

Return to the [Summary Table](#).

Figure 1-43. PLLSTARTUP1_32_48MHZ

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED									LPFRESC						
R-									R/W-1h						
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LPFRESC		LPFRESA									LPFCAPA				
R/W-1h		R/W-1h									R/W-10h				

Table 1-52. PLLSTARTUP1_32_48MHZ Field Descriptions

Bit	Field	Type	Reset	Description
31-23	RESERVED	R	0h	
22-15	LPFRESC	R/W	1h	Loop Filter Res C
14-5	LPFRESA	R/W	1h	Loop Filter Res A
4-0	LPFCAPA	R/W	10h	Loop Filter Cap A

1.6.1.16 TEMP_SENSE0 (Offset = 41C4003Ch) [Reset = 00000000h]

TEMP_SENSE0 is shown in [Figure 1-44](#) and described in [Table 1-53](#).

Return to the [Summary Table](#).

Temperature sensor room temperature calibration code. This is ADC conversion results of temperature sensor output voltage. Included in BOOTCRC calculation.

Figure 1-44. TEMP_SENSE0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA																															
R-																															

Table 1-53. TEMP_SENSE0 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	DATA	R	0h	

1.6.1.17 RESERVED0 (Offset = 41C40040h) [Reset = 00000000h]

RESERVED0 is shown in [Figure 1-45](#) and described in [Table 1-54](#).

Return to the [Summary Table](#).

Figure 1-45. RESERVED0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															
R-0h																															

Table 1-54. RESERVED0 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESERVED	R	0h	Reserved

1.6.1.18 RESERVED1 (Offset = 41C40044h) [Reset = 00000000h]

RESERVED1 is shown in [Figure 1-46](#) and described in [Table 1-55](#).

Return to the [Summary Table](#).

Figure 1-46. RESERVED1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															
R-0h																															

Table 1-55. RESERVED1 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESERVED	R	0h	Reserved

1.6.1.19 RESERVED2 (Offset = 41C40048h) [Reset = 00000000h]

RESERVED2 is shown in [Figure 1-47](#) and described in [Table 1-56](#).

Return to the [Summary Table](#).

Figure 1-47. RESERVED2

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															
R-0h																															

Table 1-56. RESERVED2 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESERVED	R	0h	Reserved

1.6.1.20 RESERVED3 (Offset = 41C4004Ch) [Reset = 00000000h]

RESERVED3 is shown in [Figure 1-48](#) and described in [Table 1-57](#).

Return to the [Summary Table](#).

Figure 1-48. RESERVED3

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															
R-0h																															

Table 1-57. RESERVED3 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESERVED	R	0h	Reserved

1.6.1.21 RESERVED4 (Offset = 41C40050h) [Reset = 00000000h]

RESERVED4 is shown in [Figure 1-49](#) and described in [Table 1-58](#).

Return to the [Summary Table](#).

Figure 1-49. RESERVED4

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															
R-0h																															

Table 1-58. RESERVED4 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESERVED	R	0h	Reserved

1.6.1.22 RESERVED5 (Offset = 41C40054h) [Reset = 00000000h]

RESERVED5 is shown in [Figure 1-50](#) and described in [Table 1-59](#).

Return to the [Summary Table](#).

Figure 1-50. RESERVED5

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															
R-0h																															

Table 1-59. RESERVED5 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESERVED	R	0h	Reserved

1.6.1.23 RESERVED6 (Offset = 41C40058h) [Reset = 00000000h]

RESERVED6 is shown in [Figure 1-51](#) and described in [Table 1-60](#).

Return to the [Summary Table](#).

Figure 1-51. RESERVED6

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															
R-0h																															

Table 1-60. RESERVED6 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESERVED	R	0h	Reserved

1.6.1.24 RESERVED7 (Offset = 41C4005Ch) [Reset = 00000000h]

RESERVED7 is shown in [Figure 1-52](#) and described in [Table 1-61](#).

Return to the [Summary Table](#).

Figure 1-52. RESERVED7

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															
R-0h																															

Table 1-61. RESERVED7 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESERVED	R	0h	Reserved

1.6.1.25 RESERVED8 (Offset = 41C40060h) [Reset = 00000000h]

RESERVED8 is shown in [Figure 1-53](#) and described in [Table 1-62](#).

Return to the [Summary Table](#).

Figure 1-53. RESERVED8

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															
R-0h																															

Table 1-62. RESERVED8 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESERVED	R	0h	Reserved

1.6.1.26 RESERVED9 (Offset = 41C40064h) [Reset = 00000000h]

RESERVED9 is shown in [Figure 1-54](#) and described in [Table 1-63](#).

Return to the [Summary Table](#).

Figure 1-54. RESERVED9

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															
R-0h																															

Table 1-63. RESERVED9 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESERVED	R	0h	Reserved

1.6.1.27 RESERVED10 (Offset = 41C40068h) [Reset = 00000000h]

RESERVED10 is shown in [Figure 1-55](#) and described in [Table 1-64](#).

Return to the [Summary Table](#).

Figure 1-55. RESERVED10

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															
R-0h																															

Table 1-64. RESERVED10 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESERVED	R	0h	Reserved

1.6.1.28 RESERVED11 (Offset = 41C4006Ch) [Reset = 00000000h]

RESERVED11 is shown in [Figure 1-56](#) and described in [Table 1-65](#).

Return to the [Summary Table](#).

Figure 1-56. RESERVED11

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															
R-0h																															

Table 1-65. RESERVED11 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESERVED	R	0h	Reserved

1.6.1.29 RESERVED12 (Offset = 41C40070h) [Reset = 00000000h]

RESERVED12 is shown in [Figure 1-57](#) and described in [Table 1-66](#).

Return to the [Summary Table](#).

Figure 1-57. RESERVED12

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															
R-0h																															

Table 1-66. RESERVED12 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESERVED	R	0h	Reserved

1.6.1.30 RESERVED13 (Offset = 41C40074h) [Reset = 00000000h]

RESERVED13 is shown in [Figure 1-58](#) and described in [Table 1-67](#).

Return to the [Summary Table](#).

Figure 1-58. RESERVED13

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															
R-0h																															

Table 1-67. RESERVED13 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESERVED	R	0h	Reserved

1.6.1.31 RESERVED14 (Offset = 41C40078h) [Reset = 00000000h]

RESERVED14 is shown in [Figure 1-59](#) and described in [Table 1-68](#).

Return to the [Summary Table](#).

Figure 1-59. RESERVED14

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															
R-0h																															

Table 1-68. RESERVED14 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESERVED	R	0h	Reserved

1.6.1.32 BOOTCRC (Offset = 41C4007Ch) [Reset = 00000000h]

BOOTCRC is shown in [Figure 1-60](#) and described in [Table 1-69](#).

Return to the [Summary Table](#).

BOOTCRC records the 32-bit CRC of all locations in OPEN including reserved locations.

Figure 1-60. BOOTCRC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA																															
R-																															

Table 1-69. BOOTCRC Field Descriptions

Bit	Field	Type	Reset	Description
31-0	DATA	R	0h	

This page intentionally left blank.



The power management and clock unit (PMCU) is a unified system module which provides all power management, clock configuration, and reset control functionality for the device. All power management unit (PMU) and clock module (CKM) policies for device operation are configured through memory-mapped registers in the system controller (SYSCTL).

2.1 PMCU Overview	94
2.2 Power Management (PMU)	99
2.3 Clock Module (CKM)	106
2.4 System Controller (SYSCTL)	135
2.5 Quick Start Reference	151
2.6 SYSCTL Registers	155

2.1 PMCU Overview

The power management and clock unit (PMCU) provides all power, clocking, reset, and system control services for the device. The PMCU contains three submodules to provide this functionality: the power management unit (**PMU**), the clock module (**CKM**), and the system controller (**SYSCTL**).

The **PMU** is an analog submodule that generates the internal regulated supplies for the device and supervises the condition of the external supply. The PMU also contains voltage and current reference circuits used by the on-chip regulators and analog peripherals.

The **CKM** is an analog submodule that provides clock sources (internal and external oscillators) and presents these clock sources to SYSCTL. SYSCTL distributes these clock sources to the CPU, buses, and peripherals on the device.

The **SYSCTL** is a digital submodule that provides the control logic for all functions in the PMCU. In addition, SYSCTL contains the memory-mapped registers used by software to configure power management and clocks, assess the status of the device, and control resets. SYSCTL also provides 4 bytes of general-purpose memory that is retained in SHUTDOWN mode and can be used to store status information in SHUTDOWN mode when SRAM and register contents are lost.

Figure 2-1

Note

shows the interfaces between the PMCU and the device supplies, clocks, and signals. Configuration of the PMCU by software is always done through memory-mapped registers in the SYSCTL submodule.

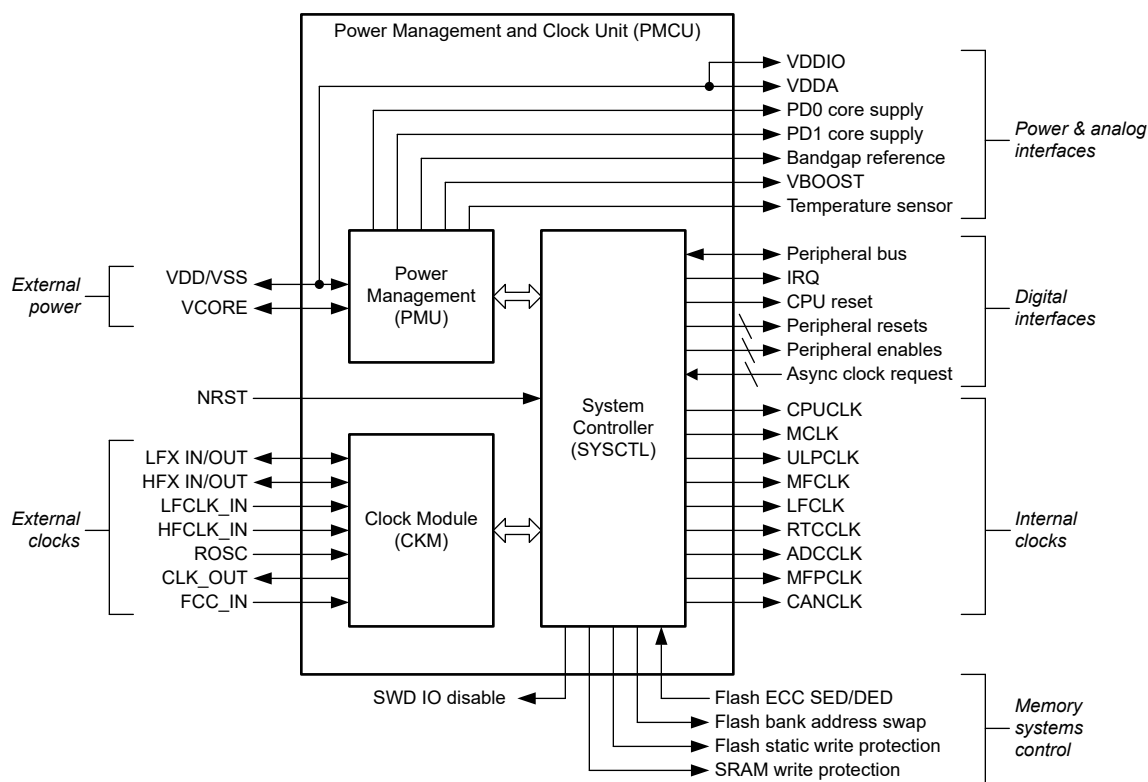


Figure 2-1. MSPM0Gxx PMCU Top Level

Note

Not all devices have all of the PMCU features shown in [Figure 2-1](#). For example, not all devices have LFXT (low-frequency crystal oscillator), HFXT (high-frequency crystal oscillator), and CANCLK. See the device-specific data sheet to understand the features present on a given device.

Using this Guide

The [PMU](#), [CKM](#), and [SYSCTL](#) sections of this chapter describe the functionality provided by each submodule in detail.

The [quick start](#) section describes overall system level operation of the PMCU and how to configure the PMCU for different application scenarios.

2.1.1 Power Domains

Two core power domains are provided on the device: PD1 and PD0. PD1 is always powered in RUN and SLEEP modes, but is disabled in all other modes. PD0 is always powered in RUN, SLEEP, STOP and STANDBY modes. PD1 and PD0 are both disabled in SHUTDOWN mode.

- The PD1 domain includes the CPU subsystem, the SRAM memory, PD1 peripherals, and the PD1 peripheral bus, which runs from [MCLK](#) (including the DMA) with a maximum frequency of 80 MHz. While PD1 is disabled in STOP and STANDBY mode, the CPU registers, SRAM, and peripheral MMR configuration registers are maintained in retention such that they are available to resume operation immediately when STOP or STANDBY modes are exited.
- The PD0 domain includes the PD0 peripherals and PD0 bus segment which runs from ULPCLK with a max frequency of 40 MHz in RUN and SLEEP mode, 4 MHz in STOP mode, and 32 kHz in STANDBY mode. The PD0 domain is powered in all modes except SHUTDOWN and can be thought of as an "always-on" domain.

The device-specific data sheet describes which peripherals on a device are in PD1 and which are in PD0.

The device also has a single external supply (VDD) domain that provides power to the IO and analog peripherals.

2.1.2 Operating Modes

Five operating modes (power modes) are provided to allow for optimization of the device power consumption based on application requirements. In order of decreasing power, the modes are: RUN, SLEEP, STOP, STANDBY, and SHUTDOWN. [Figure 2-2](#) shows the interaction between the modes.

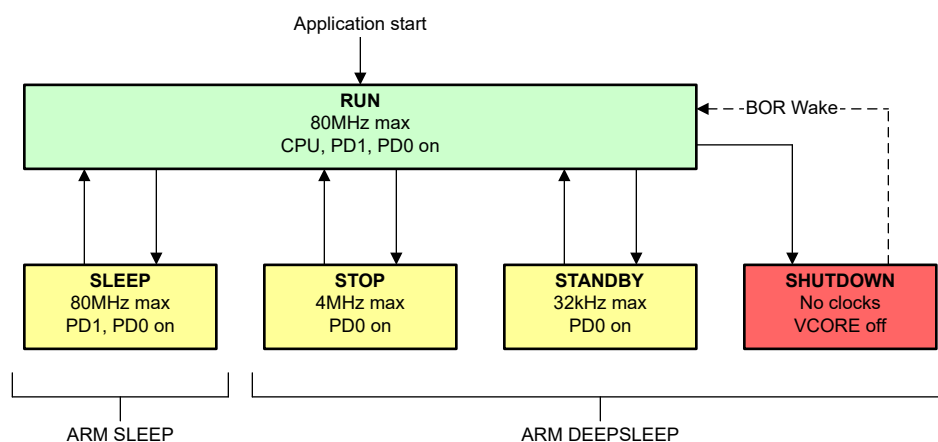


Figure 2-2. MSPM0Gxx Operating Modes

[Section 2.1.2.6](#) indicates what functions are available in each operating mode of the device. See the [operating mode selection](#) section for information on how to configure the device for a particular operating mode.

Operating Mode Concept

MSPM0 MCUs implement a policy-based power and clock management scheme. Policies can be configured through application software for how the clocking is to be managed in each operating mode to obtain the best balance of power and performance for a given application.

After the operating policy for each mode is configured, application software can enter and exit the various operating modes through simple register commands, and SYSCTL automatically manages all the necessary PMU states, oscillator and clock enable and disables, and the SYSOSC frequency according to the software-defined policies and software-selected mode.

A variety of hardware-triggered low-power mode suspension mechanisms also exist to enable on-demand access to a fast clock when requested by supported peripherals, as well as functions such as DMA and ADC triggering from low-power modes.

The policy-driven operating mode scheme together with the asynchronous low-power mode suspension mechanisms enable application software to select the operating mode and corresponding policy that provide the lowest possible power consumption for background activities, with transient foreground activities either bringing up the DMA, bringing up a fast clock, or bringing the device to RUN (in the case of an IRQ) for burst handling.

2.1.2.1 RUN Mode

In RUN mode, the CPU is active executing code and any peripheral can be enabled.

There are three RUN mode policy options: RUN0, RUN1, and RUN2.

- **RUN0:** The MCLK and the CPUCLK run from a fast clock source (SYSOSC, HFCLK, or SYSPLL).
- **RUN1:** The MCLK and the CPUCLK run from LFCLK (at 32 kHz) to reduce active power, but SYSOSC is left enabled to service analog modules such as an ADC, DAC, OPA, or COMP (in HS mode).
- **RUN2:** The MCLK and the CPUCLK run from LFCLK (at 32 kHz), and SYSOSC is completely disabled to save power. This is the lowest power state with the CPU running.

2.1.2.2 SLEEP Mode

In SLEEP mode, the CPU is disabled (clock gated) but otherwise the device configuration is the same as RUN. There are three SLEEP mode policy options: SLEEP0, SLEEP1, and SLEEP2. The SLEEPx policy is determined by the current RUNx policy when SLEEP mode is entered.

- **SLEEP0:** Identical to RUN0, with the CPU disabled.
- **SLEEP1:** Identical to RUN1, with the CPU disabled.
- **SLEEP2:** Identical to RUN2, with the CPU disabled.

2.1.2.3 STOP Mode

In STOP mode, the CPU, SRAM, and PD1 peripherals are disabled and in retention (if applicable). PD0 peripherals are available with a max ULPCLK frequency of 4 MHz. SYSOSC can run at higher frequencies to support ADC, OPA, or HS COMP operation, but ULPCLK will be automatically limited to the 4 MHz SYSOSC output by SYSCTL. High speed oscillators (SYSPLL, HFXT, HFCLK_IN) are automatically disabled.

DMA is available to be triggered. A DMA trigger wakes the PD1 power domain to make the SRAM and DMA available for processing the DMA transfer, and the DMA transfer is processed at the current MCLK and ULPCLK rate. After the transfer completes, the SRAM is returned to retention and PD1 is disabled automatically.

STOP mode is the lowest power mode that supports ADC, 12-bit DAC, , OPA, and high-speed COMP operation.

There are three policy options for STOP mode: STOP0, STOP1, and STOP2.

- **STOP0:** The SYSOSC is left running at the current frequency when entering STOP mode (either 32 MHz, 24 MHz, 16 MHz, or 4 MHz) . ULPCLK is always limited to 4 MHz automatically by hardware, but SYSOSC is not disturbed to support consistent operation of analog peripherals such as the ADC, OPA, or COMP.
 - **NOTE:** If STOP0 is entered from RUN1 (SYSOSC enabled but MCLK sourced from LFCLK), SYSOSC remains enabled as in RUN1, and ULPCLK remains at 32 kHz as in RUN1.

- NOTE: If STOP0 is entered from RUN2 (SYSOSC was disabled and MCLK was sourced from LFCLK), SYSOSC remains disabled as in RUN2, and ULPCLK remains at 32 kHz as in RUN2.
- **STOP1:** The SYSOSC is [gear shifted](#) from its current frequency to 4 MHz for the lowest power consumption in STOP mode with SYSOSC running. SYSOSC and ULPCLK both run at 4 MHz.
- **STOP2:** The SYSOSC is disabled and the ULPCLK is sourced from LFCLK at 32 kHz. This is the lowest power state in STOP mode.

2.1.2.4 STANDBY Mode

In STANDBY mode, the CPU, SRAM, and PD1 peripherals are disabled and in retention. PD0 peripherals, with the exception of the ADC, 12-bit DAC, and OPA, are available with a maximum ULPCLK frequency of 32 kHz. High-speed oscillators (SYSPLL, HFXT, HFCLK_IN) and SYSOSC are disabled.

DMA is available to be triggered. A DMA trigger wakes the PD1 power domain to make the SRAM and DMA available for processing the DMA transfer, and the DMA transfer is processed at the current MCLK and ULPCLK rate (32 kHz). After the transfer completes, the SRAM is returned to retention and PD1 is disabled automatically.

ADC, 12-bit DAC, OPA, and high-speed COMP operation is not supported in STANDBY mode.

There are 2 policy options for STANDBY mode: STANDBY0 and STANDBY1.

- **STANDBY0:** All PD0 peripherals receive the ULPCLK and LFCLK, and the RTC receives RTCCLK.
- **STANDBY1:** Only TIMG0 and TIMG1 receive ULPCLK or LFCLK. The RTC continues to receive RTCCLK. A TIMG0 and TIMG1 interrupt, RTC interrupt, or ADC trigger in STANDBY1 always triggers an [asynchronous fast clock request](#) to wake the system. Other PD0 peripherals (such as UART, I2C, GPIO, and COMP) can also wake the system upon an external event through an asynchronous fast clock request, but they are not actively clocked in STANDBY1.

2.1.2.5 SHUTDOWN Mode

In SHUTDOWN mode, no clocks are available. The core regulator is completely disabled and all SRAM and register contents are lost, with the exception of the 4 bytes of general-purpose memory in SYSCTL that can be used to store state information. The BOR and bandgap circuit are disabled.

The device can wake through a wake-up capable IO, a debug connection or NRST.

SHUTDOWN mode has the lowest current consumption of any operating mode. Exiting SHUTDOWN mode triggers a BOR.

2.1.2.6 Supported Functionality by Operating Mode

Supported functionality in each operating mode is given in [Table 2-1](#).

Functional key:

- **EN:** The function is enabled in the specified mode.
- **DIS:** The function is disabled (either clock or power gated) in the specified mode, but the function's configuration is retained.
- **OPT:** The function is optional in the specified mode, and remains enabled if configured to be enabled.
- **NS:** The function is not automatically disabled in the specified mode, but its use is not supported.
- **OFF:** The function is fully powered off in the specified mode, and no configuration information is retained.

Note

For a complete listing of the behavior for each peripheral instance on a given device, see the *Supported Functionality by Operating Mode* table in the detailed description section of the device-specific data sheet.

Table 2-1. MSPM0Gxx Supported Functionality by Operating Mode

Operating Mode		RUN			SLEEP			STOP			STANDBY		SHUTDOWN
		RUN0	RUN1	RUN2	SLEEP0	SLEEP1	SLEEP2	STOP0	STOP1	STOP2	STANDBY0	STANDBY1	
Oscillators	SYSOSC	EN	EN	DIS	EN	EN	DIS	OPT ⁽¹⁾	EN	DIS	DIS	DIS	OFF
	LFOSC or LFXT	EN (LFOSC or LFXT)											OFF
	HFXT	OPT	DIS	DIS	OPT	DIS	DIS	DIS	DIS	DIS	DIS	DIS	OFF
	SYSPLL	OPT	DIS	DIS	OPT	DIS	DIS	DIS	DIS	DIS	DIS	DIS	OFF
Clocks	CPUCLK	80M max	32k	32k	DIS								OFF
	MCLK to PD1	80M max	32k	32k	80M max	32k	32k	DIS					OFF
	ULPCLK to PD0	40M max	32k	32k	40M max	32k	32k	4M max ⁽¹⁾	4M	32k		DIS	OFF
	ULPCLK to TIMG0/1	40M max	32k	32k	40M max	32k	32k	4M max ⁽¹⁾	4M	32k		OFF	
	RTCCLK	32 kHz											OFF
	MFCLK	OPT	DIS		OPT	DIS		OPT		DIS		OFF	
	MFPCLK	OPT	DIS		OPT	DIS		OPT		DIS		OFF	
	LFCLK	32k										DIS	OFF
	LFCLK to TIMG0/1	32k											OFF
	LFCLK Monitor	OPT											OFF
	MCLK Monitor	OPT										DIS	OFF
PMU	POR monitor	EN											
	BOR monitor	EN											OFF
	Core regulator	FULL DRIVE						REDUCED DRIVE		LOW DRIVE		OFF	
Core Functions	CPU	EN			DIS								OFF
	DMA	OPT						DIS (triggers supported)					OFF
	Flash	EN						DIS					OFF
	SRAM	EN						DIS					OFF
Peripherals	PD1 Peripherals	OPT						DIS					OFF
	PD0 Peripherals	OPT										OPT ⁽²⁾	OFF
Analog	TRNG	OPT						OFF					
	ADC	OPT								NS (triggers supported)		OFF	
	12-bit DACs	OPT								NS		OFF	
	OPA	OPT	NS		OPT	NS		OPT		NS		OFF	
	GPAMP	OPT								NS		OFF	
	COMP / 8-bit DAC	OPT	OPT (ULP only)		OPT	OPT (ULP only)		OPT		OPT (ULP only)		OFF	
	VREF	OPT									OPT (sampled mode)		OFF
IOMUX and IO Wakeup		EN											DIS w/ WAKE
Wake Sources		N/A			ANY IRQ			PD0 IRQ					IOMUX, NRST

- (1) If STOP0 is entered from RUN1 (SYSOSC enabled but MCLK sourced from LFCLK), SYSOSC remains enabled as in RUN1, and ULPCLK remains at 32 kHz as in RUN1. If STOP0 is entered from RUN2 (SYSOSC was disabled and MCLK was sourced from LFCLK), SYSOSC remains disabled as in RUN2, and ULPCLK remains at 32 kHz as in RUN2.

- (2) When using the STANDBY1 policy for STANDBY, only , TIMG0, TIMG1 , and the RTC are clocked. Other PD0 peripherals can generate an [asynchronous fast clock request](#) on external activity but are not actively clocked.

2.1.2.7 Suspended Low Power Mode Operation

Some peripherals can be configured to temporarily suspend STOP or STANDBY mode operation to handle a temporary activity or process an event. There are two ways in which STOP or STANDBY mode can be suspended:

- An [asynchronous fast clock request](#)
- A DMA trigger

Suspended STOP or STANDBY for an Asynchronous Fast Clock Request

An asynchronous fast clock request temporarily suspends any active low-power mode and runs the [MCLK](#) and [ULPCLK](#) tree at 32 MHz , sourced from [SYSOSC](#). Asynchronous fast clock requests are also functional in RUN and SLEEP mode if MCLK is sourced from either [LFCLK](#) at 32 kHz or SYSOSC at a frequency lower than 32 MHz . While asynchronous fast clock requests suspend the low-power mode and change clock tree configuration to support 32 MHz operation, these requests do not enable the PD1 power domain if the device was in STOP or STANDBY mode. This functionality enables use cases such as:

- RTC and TIMG0 or TIMG1 wakeup from STANDBY1
- On-demand UART, I²C, or SPI communication
- Timer-triggered ADC sampling from STANDBY mode

Suspended STOP or STANDBY for a DMA Trigger

If a DMA trigger is asserted in STOP or STANDBY mode, the low-power mode is temporarily suspended and the PD1 power domain (including the SRAM and flash memory) is enabled to process the DMA request. Unlike the asynchronous fast clock request, DMA transfers do not change the clock tree configuration. A DMA request in STOP or STANDBY mode is processed at the current [ULPCLK](#) rate.

2.2 Power Management (PMU)

The power management unit (PMU) generates the regulated core supplies for the device and provides supervision of the external supply. It also contains a bandgap voltage reference used by the PMU and other analog peripherals.

Key PMU features include:

- Support for device operation across a wide supply range (1.62 V to 3.6 V)
- Low-dropout linear voltage regulator to generate the internal core logic supply, with multiple operating modes for reducing device current in low-power modes (managed automatically by SYSCCTL)
- Power-on reset (POR) supply monitor
- Brownout reset (BOR) supply monitor with four configurable threshold voltages
- Bandgap voltage reference supporting the BOR, core regulator, and analog peripherals
- Analog mux VBOOST unit for increasing analog mux performance
- Temperature sensor with connection to the ADC

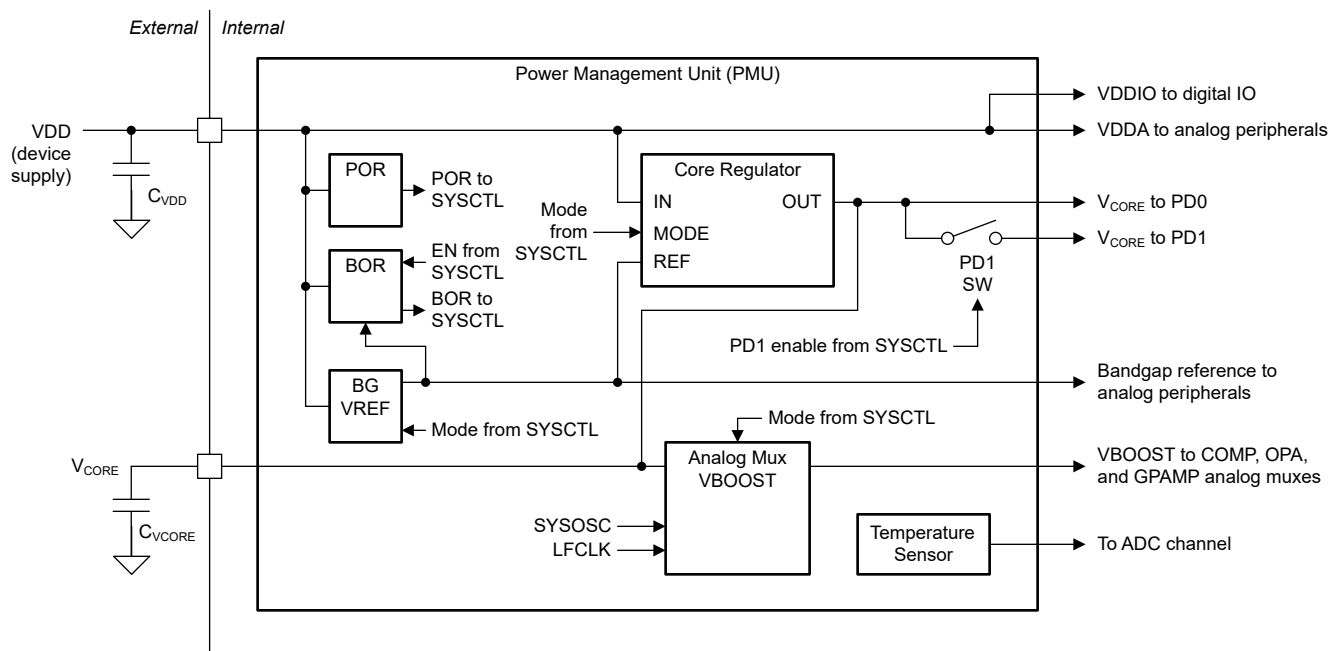


Figure 2-3. MSPM0Gxx PMU Block Diagram

2.2.1 Power Supply

Power is supplied to the device through the VDD and VSS connections. The device supports operation with a supply voltage of 1.62 V to 3.6 V and will start with a 1.62 V supply. A decoupling capacitor (C_{VDD}) must be placed across all VDD and VSS supply pairs. See the device-specific data sheet for the correct value and tolerance of C_{VDD} . Products with 64 pins or less typically have a single VDD/VSS power pair.

VDD is used directly to provide the IO supply (VDDIO) and the analog supply (VDDA). VDDIO and VDDA are internally connected to VDD so that additional power supply pins are not required.

2.2.2 Core Regulator

The PMU uses an on-chip, configurable, low-dropout linear voltage regulator to generate a 1.35V supply rail to power the device core. In general, the core regulator output (V_{CORE}) supplies power to the core logic, which includes the CPU, digital peripherals and the device memory. The core regulator requires an external capacitor (C_{VCORE}) which is connected between the device V_{CORE} pin and VSS (ground). See the device specific data sheet for the correct value and tolerance of C_{VCORE} .

The core regulator is active in all power modes except for SHUTDOWN. In all other power modes (RUN, SLEEP, STOP, and STANDBY) the drive strength of the regulator is configured automatically to support the max load current of each mode. This reduces the quiescent current of the regulator when using low-power modes, improving low power performance. SYSCTL automatically configures the regulator for best power consumption based on the power mode which is currently active.

2.2.3 Supply Supervisors

The PMU provides two supply supervisor circuits:

- A power-on reset (POR) circuit to indicate that the external supply has reached sufficient voltage to start the on-chip bandgap reference and BOR circuit
- A user-programmable brownout reset (BOR) circuit which ensures that the external supply is maintained at a sufficient voltage to support correct operation of the device

2.2.3.1 Power-on Reset (POR) Supervisor

The power-on reset (POR) supervisor monitors the external supply (VDD) and asserts or de-asserts a POR violation to SYSCTL. During cold power-up, the device is held in a POR state until VDD passes the POR+ threshold. When VDD has passed POR+, the POR state is released and the [bandgap reference](#) and [BOR monitor circuit](#) are started. If VDD drops below the POR- level, then a POR- violation is asserted and the device is again held in a POR reset state.

The POR supervisor does not indicate that VDD has reached a level high enough to support correct operation of the device. Rather, it is the first step in the boot process and is used to determine if the supply voltage is sufficient to power up the bandgap reference and BOR circuit, which are then used to determine if the supply has reached a level sufficient for the device to run correctly.

The POR supervisor is active in all power modes including SHUTDOWN and cannot be disabled.

2.2.3.2 Brownout Reset (BOR) Supervisor

The brownout reset (BOR) supervisor monitors the external supply (VDD) and asserts or de-asserts a BOR violation to SYSCTL. The primary responsibility of the BOR circuit is to make sure that the external supply is maintained high enough to enable correct operation of internal circuits, including the [core regulator](#). The BOR threshold reference is derived from the internal bandgap circuit. The threshold is programmable and is always higher than the [POR](#) threshold. During cold start, after VDD passes the POR+ threshold, the bandgap reference and BOR circuit are started. The device is then held in a BOR state until VDD passes the BOR0+ threshold. When VDD passes BOR0+, the BOR supervisor releases the device to continue the boot process, and the PMU is started.

There are four selectable BOR threshold levels (BOR0-BOR3). During startup, the BOR threshold is always BOR0 (the lowest value) to make sure that the device starts at the specified VDD minimum (1.62 V). After boot, software can optionally re-configure the BOR circuit to use a different (higher) threshold level (BOR1-BOR3).

When the BOR threshold is BOR0, a BOR0- violation always generates a BOR- violation signal to SYSCTL, generating a BOR level reset. When the BOR threshold is re-configured to BOR1, BOR2, or BOR3, the BOR circuit generates a SYSCTL interrupt rather than asserting the BOR- violation. This can be used to give the application an indication that the supply has dropped below a certain level without causing a reset.

To change the BOR level from the default (BOR0), first select the desired value in the LEVEL field of the BORTHRESHOLD register in SYSCTL. Then, activate the threshold set in the LEVEL field by setting the GO bit in the BORCLRCMD register. The change can be validated by testing the BORCURTHRESOLD field in the SYSSTATUS register, which returns a value corresponding to the currently active BOR threshold. The BOR threshold change takes approximately 15 μ s to complete, during which time the BOR circuit is blind to changes in the supply.

If the BOR is in interrupt mode (a threshold level of BOR1-BOR3), and the supply drops below the corresponding BORx- level, an interrupt is generated and the BOR circuit automatically switches the BOR threshold level to BOR0 to make sure that a BOR- violation is asserted if VDD drops below BOR0-. Application software can set the BOR level back to the level specified in the LEVEL field of the BORTHRESHOLD register by setting the GO bit again in the BORCLRCMD register.

The BOR supervisor is active in RUN, SLEEP, STOP, and STANDBY modes but is disabled automatically in SHUTDOWN mode.

2.2.3.3 POR and BOR Behavior During Supply Changes

When the supply voltage (VDD) drops below POR-, the entire device state is cleared. Small variations in VDD that do not pass below the BOR0- threshold do not cause a BOR- violation, and the device continues to run. Behavior for BORx thresholds other than BOR0 (for example, BOR1-BOR3) is the same as is shown for BOR0, except that the BOR circuit is configured to generate an interrupt rather than immediately triggering a BOR reset.

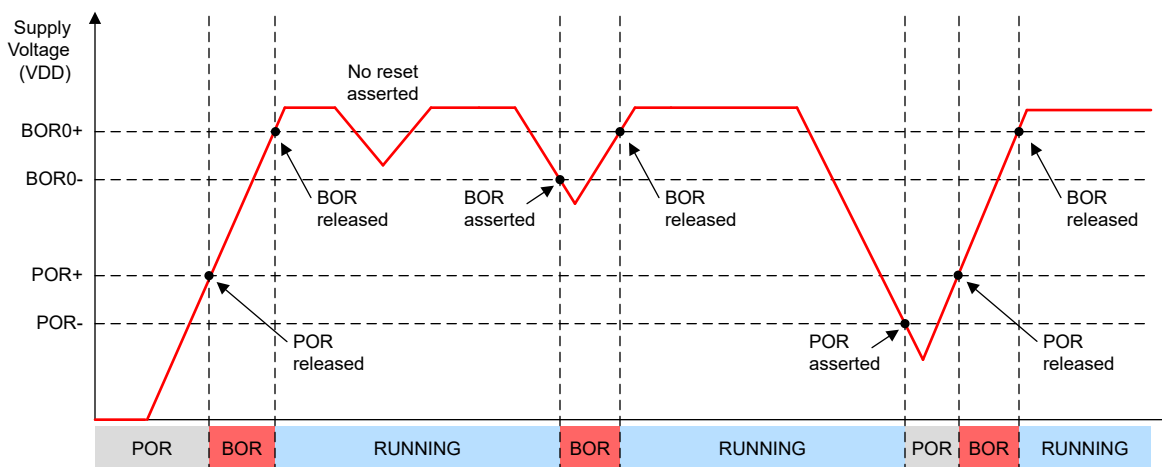


Figure 2-4. POR/BOR vs. Supply Voltage (VDD)

2.2.4 Bandgap Reference

The PMU provides a temperature and supply voltage stable bandgap voltage reference that is used by the device for internal functions, including:

- Deriving the brownout reset circuit thresholds
- Setting the output voltage for the core regulator
- Deriving the on-chip VREF levels for on-chip analog peripherals

The bandgap reference is enabled in RUN, SLEEP, STOP modes. This reference operates in a sampled mode in STANDBY to reduce power consumption and is disabled in SHUTDOWN mode. SYSCTL manages the bandgap state automatically; no user configuration is required.

2.2.5 Temperature Sensor

The PMU provides a basic temperature sensor which can be used to approximate the temperature of the device. The temperature sensor is connected internally to the ADC, and the ADC must be used to perform a temperature measurement. See the device-specific data sheet to determine the correct internal ADC channel to use when measuring the temperature sensor.

The temperature sensor outputs a voltage which has a linear relationship with temperature. The temperature coefficient (TS_C) is the slope of the temperature-voltage relationship (given in mV/C), and is given in the specifications section of the device-specific data sheet.

A unit-specific single-point trim value (TEMP_SENSE0.DATA) is provided in the [factory constants](#) memory of each device. This value indicates the temperature sensor output voltage at the factory trim temperature (TS_{TRIM}), in ADC result code format. The ADC result code in TEMP_SENSE0.DATA is based upon 12-bit sampling mode together with the 1.4-V internal voltage reference. The TS_{TRIM} temperature is also given in the specifications section of the device-specific data sheet.

The approximate temperature of the device can be computed through the use of the following parameters:

- TS_C , taken from the device data sheet
- TEMP_SENSE0.DATA, taken from the unit-specific factory constants memory
- TS_{TRIM} , taken from the device data sheet
- V_{SAMPLE} (voltage sample of the temperature sensor at time of interest, taken with the ADC)

The temperature is computed through the linear relationship given in [Equation 1](#), where V_{SAMPLE} is the current temperature sensor voltage, and V_{TRIM} is the factory calibrated temperature sensor voltage at the TS_{TRIM} temperature (derived from TEMP_SENSE0.DATA):

$$T_{SAMPLE} = (1 / TS_C) * (V_{SAMPLE} - V_{TRIM}) + TS_{TRIM} \quad (1)$$

The ADC_{CODE} raw result can be converted to a voltage equivalent (V_{SAMPLE}) as shown in the relationship in Equation 2, where RES is the ADC resolution in bits, and VREF is the ADC reference voltage.

$$V_{\text{SAMPLE}} = (V_{\text{REF}} / 2^{\text{RES}}) * (\text{ADC}_{\text{CODE}} - 0.5) \quad (2)$$

Example

To illustrate the process of converting an ADC sample of the temperature sensor into an approximate device temperature, an example is given below.

Example parameters:

- $TS_c = -2.04 \text{ mV/C}$
- TEMP_SENSE0.DATA = 1857 (ADC result code based on 12-bit mode and a 1.4-V reference)
- $TS_{\text{TRIM}} = 30\text{C}$
- $\text{ADC}_{\text{CODE}} = 1677$ (ADC result code based on 12-bit mode and a 1.4-V reference)

First, the current temperature sensor sample voltage is calculated using Equation 3:

$$V_{\text{SAMPLE}} = (1.4 \text{ V} / 4096) * (1677 - 0.5) = \mathbf{0.5730 \text{ V}} \quad (3)$$

Then, the single-point calibration voltage is calculated using the same means:

$$V_{\text{TRIM}} = (1.4 \text{ V} / 4096) * (1857 - 0.5) = \mathbf{0.6345 \text{ V}} \quad (4)$$

Then, the temperature is approximated using Equation 5:

$$T_{\text{SAMPLE}} = (1 / -0.002044) * (0.5730 \text{ V} - 0.6345) + 30^\circ\text{C} = \mathbf{60^\circ\text{C}} \quad (5)$$

Note

The temperature sensor is not available in STANDBY and SHUTDOWN operating modes.

2.2.6 VBOOST for Analog Muxes

The VBOOST circuit in the PMU generates an internal VBOOST supply that is used by the analog muxes in COMPs, GPAMPs, and OPAs, if present on a device. The VBOOST circuit enables consistent analog mux performance across the external supply voltage (VDD) range.

Enabling and Disabling VBOOST

SYSCTL automatically manages the enable request for the VBOOST circuit based on the following parameters:

- The COMP, OPA, and GPAMP peripheral PWREN settings
- The MODE setting of any COMP which is enabled (FAST vs. ULP mode)
- The ANACPUMPCFG control bits in the GENCLKCFG register in SYSCTL

VBOOST is disabled by default following a SYSRST. Application software is not required to enable the VBOOST circuit prior to using the COMP, OPA, or GPAMP. When a COMP, OPA, or the GPAMP is enabled by application software, SYSCTL also enables the VBOOST circuit to support the analog peripheral.

The VBOOST circuit has a startup time requirement (12 μs typical) to transition from a disabled state to an enabled state. If a COMP, OPA, or the GPAMP is enabled when VBOOST is disabled, then the corresponding analog peripheral ready status is not asserted until both the peripheral and the VBOOST circuit are ready. If the startup time of the COMP, OPA, or GPAMP is less than the VBOOST startup time, the peripheral startup time is extended to account for the VBOOST startup time.

Alternatively, application software can force the VBOOST circuit to be enabled at all times (with ANACPUMPCFG=0x2) or while in RUN or SLEEP mode (with ANACPUMPCFG=0x1) so that there is no

additional startup delay when enabling a COMP, OPA, or the GPAMP. [Table 2-2](#) gives the behavior of the ANACPUMPCFG control and corresponding application use cases.

Table 2-2. Forcing VBOOST Enable With ANACPUMPCFG

VBOOST Control (ANACPUMPCFG)		Behavior	
VAL	MODE	VBOOST Enable	Application Use Case
0x0	ONDEMAND	VBOOST is automatically enabled by SYSCTL only when a COMP, OPA, or the GPAMP is enabled.	This setting provides the lowest power consumption in all modes when fast startup of the COMP, OPA, or GPAMP is not critical.
0x1	ONACTIVE	VBOOST is forced to be enabled when the device is in RUN or SLEEP mode. VBOOST is also kept enabled in STOP or STANDBY mode if a COMP, OPA, or the GPAMP is enabled.	This setting provides low power consumption by disabling the VBOOST automatically in STOP and STANDBY modes when no peripherals requiring VBOOST are enabled. VBOOST is automatically re-enabled upon exit to RUN mode to provide fast startup of the COMP, OPA, or GPAMP in the event that application software enables a COMP, OPA, or GPAMP in RUN mode.
0x2	ONALWAYS	VBOOST is forced to be enabled in all operating modes except SHUTDOWN.	This setting makes sure that the COMP, OPA, and GPAMP never incur additional startup latency due to VBOOST startup in applications where fast COMP, OPA, or GPAMP startup is critical.

The VBOOST enable, VBOOST clock selection, and VBOOST clock error logic is shown in [Figure 2-5](#).

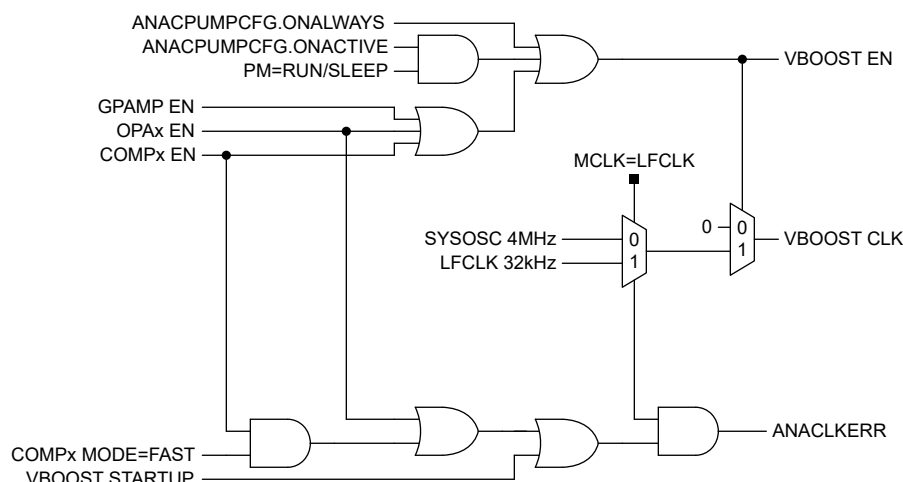


Figure 2-5. VBOOST Request Logic

VBOOST Clock

The VBOOST circuit requires a functional clock to operate. VBOOST is clocked by either the SYSOSC (4-MHz output) or the LFCLK (32 kHz) based on the currently active **MCLK** and **ULPCLK** tree source. The VBOOST clock is selected by automatically by SYSCTL according to the logic shown in [Figure 2-5](#) and described here:

- When the MCLK and ULPCLK are sourced from **LFCLK** (32 kHz), the VBOOST is also sourced from LFCLK at 32 kHz.
- When the MCLK and ULPCLK are sourced from any other source (for example, not sourced from LFCLK), VBOOST is sourced from the SYSOSC 4-MHz output. By definition, SYSOSC is always enabled when MCLK and ULPCLK are not sourced from LFCLK.

Certain VBOOST operating conditions require that the VBOOST clock be 4 MHz (sourced from SYSOSC) and not 32 kHz (sourced from LFCLK). Such conditions include:

- OPA operation or fast mode COMP operation

- VBOOST is starting up (transitioning from disabled to enabled)

The application software must make sure that MCLK and ULPCLK are not sourced from LFCLK when either of these conditions are present. SYSCTL does not change the current system clock configuration for a VBOOST request. If the MCLK and ULPCLK tree is sourced from LFCLK in one of these scenarios, SYSCTL asserts an ANACKERR status in the SYSSTATUS register in SYSCTL to indicate to application software that there is a mismatch between the VBOOST clock requirement and the current MCLK and ULPCLK configuration.

The complete clock requirements for VBOOST are given in [Table 2-3](#).

Table 2-3. VBOOST Clock Requirements

VBOOST Request		VBOOST Circuit Clock Requirement	MCLK and ULPCLK Source Requirement ⁽¹⁾	Supported Operating Modes or Policies
No active request (VBOOST is disabled)		N/A	Don't care	Don't care
VBOOST is starting up from a disabled state		4 MHz	Not LFCLK	RUN0, SLEEP0, STOP0, STOP1
OPA enabled		4 MHz	Not LFCLK	RUN0, SLEEP0, STOP0, STOP1
COMP enabled	Fast mode (FAST)	4 MHz	Not LFCLK	RUN0, SLEEP0, STOP0, STOP1
	Ultra-low-power mode (ULP) ⁽²⁾	4 MHz or 32 kHz	Don't care	RUN0, RUN1, RUN2, SLEEP0, SLEEP1, SLEEP2, STOP0, STOP1, STOP2, STANDBY0
GPAMP enabled ⁽²⁾		4 MHz or 32 kHz	Don't care	RUN0, RUN1, RUN2, SLEEP0, SLEEP1, SLEEP2, STOP0, STOP1, STOP2, STANDBY0
VBOOST running with no peripheral request active (due to ANACPUMPCFG!=0x0) ⁽²⁾		4 MHz or 32 kHz	Don't care	RUN0, RUN1, RUN2, SLEEP0, SLEEP1, SLEEP2, STOP0, STOP1, STOP2, STANDBY0

- (1) Application software must make sure that the clock system is configured as stated in the previous table to support proper VBOOST operation for each scenario. SYSCTL **does not change the system clock configuration when VBOOST is requested**. If a VBOOST request requires the VBOOST clock to be 4 MHz and MCLK is sourced from LFCLK, SYSCTL will assert an ANACKERR to indicate to the application that the current system clock configuration is not valid to support the current VBOOST request.
- (2) If no OPA is enabled and no COMP is enabled and configured in FAST mode, the VBOOST **does not require a 4-MHz clock** and VBOOST can run from LFCLK to reduce power consumption. However, if the VBOOST was previously disabled (no request present) and it is requested by a peripheral becoming enabled or by application software (ANACPUMPCFG!=0x0), **a 4-MHz VBOOST clock is required for VBOOST startup**. After VBOOST startup is complete, the VBOOST clock can be 32 kHz sourced from LFCLK if no OPA is enabled and no COMP is enabled in FAST mode. Changing the MCLK source to LFCLK during VBOOST startup can lead to incorrect operation of the VBOOST and inconsistent analog peripheral performance. When the VBOOST circuit has started and is ready, the ANACPUMPGOOD bit is set in the SYSSTATUS register in SYSCTL.

2.2.7 Peripheral Power Enable Control

All peripherals on a device, with the exception of infrastructure peripherals such as SYSCTL itself and the IOMUX, contain a power enable control register (PWREN) with a KEY and ENABLE field. Before any other peripheral registers are configured by software, the peripheral itself must be enabled by writing the ENABLE bit together with the appropriate KEY value to the peripheral's PWREN register.

When a peripheral ENABLE bit is cleared, the peripheral can be considered to be inactive and the remaining peripheral-specific registers are isolated from the peripheral bus and thus are not be accessible for read/write operations.

Note

After setting the ENABLE | KEY bits in the PWREN register to enable a peripheral, wait at least 4 [ULPCLK](#) clock cycles before accessing the rest of the peripheral's memory-mapped registers. The 4 cycles allow for the bus isolation signals at the peripheral's bus interface to update.

2.2.7.1 Automatic Peripheral Disable in Low Power Modes

Peripherals in power domain 1 (PD1) will be forced to a disabled state by SYSCTL upon entry into a STOP or STANDBY low-power mode. As such, these peripherals will not be available for use in STOP or STANDBY.

Most PD1 peripherals will retain their configuration settings after being automatically disabled, such that re-configuration is not required upon exit from STOP or STANDBY mode. See the peripheral-specific chapter in this guide for details on which peripheral registers are retained through STOP and STANDBY mode for PD1 peripherals.

If a PD1 peripheral was multiplexed to an IO pin (through the [IOMUX](#)) in an output configuration, the last valid output state (logic 0 or logic 1) from the peripheral to the IO is latched upon entry to STOP or STANDBY mode. This prevents external circuits from being disturbed by SYSCTL disabling a peripheral during low-power operation. Upon exit from STOP or STANDBY mode, the IO is again connected to the peripheral as the peripheral becomes re-enabled.

2.3 Clock Module (CKM)

The clock module contains the internal and external [oscillators](#), the [clock monitors](#), and the [clock selection and control logic](#). A [frequency clock counter](#) is also provided for checking and/or calibrating the frequency of high-speed clocks against either the LFXT/LFCLK_IN or a reference period/pulse provided on an IO pin.

2.3.1 Oscillators

Several internal and external oscillators are provided for generating low to high frequency clocks for use by the system. The CKM contains all the oscillators in the device and uses them to generate the system clocks.

Internal Oscillators

- [LFOSC](#): low frequency oscillator (32 kHz typical frequency)
- [SYSOSC](#): system oscillator (4 or 32 MHz factory-trimmed frequencies, 16 or 24 MHz user-trimmed frequencies)
- [SYSPLL](#): system PLL with programmable frequency

External Oscillators

- [LFXT](#): low frequency, low power crystal oscillator (32 kHz typical frequency)
- [HFXT](#): high frequency crystal oscillator (4-48MHz typical frequency)

In addition to the oscillators, low frequency (LFCLK_IN) and high frequency (HFCLK_IN) digital clock inputs are provided to support cases where the LFXT or HFXT is not used and a direct digital clock is supplied to the device.

2.3.1.1 Internal Low-Frequency Oscillator (LFOSC)

The low-frequency oscillator (LFOSC) is an on-chip low power oscillator which is factory trimmed to a frequency of 32.768 kHz. LFOSC provides a stable low frequency clock source in cases where the low frequency crystal oscillator (LFXT) is either not present or not used.

LFOSC can provide higher accuracy when used over a reduced temperature range. See the device-specific data sheet for details.

The LFOSC is active by default after a [BOOTRST](#), sourcing the [LFCLK](#). The [LFOSC startup monitor](#) sets the LFOSCGOOD bit in the CLKSTATUS register when LFOSC is ready.

If user software selects either LFXT or LFCLK_IN as the LFCLK source, the LFOSC is disabled until the next BOOTRST.

2.3.1.2 Internal System Oscillator (SYSOSC)

The system oscillator (SYSOSC) is an on-chip, accurate (4 or 32 MHz factory-trimmed frequencies, 16 or 24 MHz user-trimmed frequencies). The SYSOSC provides a flexible high-speed clock source to the system in

cases where the high frequency crystal oscillator (HFXT) is either not present or not used, or where fast wake-up from a low-power mode is required.

Key features of the SYSOSC include:

- High accuracy when using optional frequency correction loop (FCL) and reference resistor
 - The frequency correction loop may support correction via an external resistor (ROSC) or an internal resistor, depending on the device capabilities. Refer to the device-specific data sheet to determine if a device supports the FCL with an internal or external resistor, or both
- Fast start-up time from a disabled state
- Capable of switching from base frequency to low frequency, or low frequency to base frequency, in one clock cycle with no functional interruption (gear shift)
 - Phase-aligned transition to minimize disturbance to peripherals
 - Fast settling to specified accuracy
 - SYSCTL can initiate seamless gear shift frequency switch in STOP mode to reduce SYSOSC current
- A secondary output with a constant 4-MHz frequency for use by [MFCLK](#) and [MFPCLK](#)
 - When $f_{\text{SYSOSC}} = 32 \text{ MHz}$, the 4-MHz output is derived from SYSOSC divided digitally by 8. When $f_{\text{SYSOSC}} = 4 \text{ MHz}$, the 4-MHz output is derived from SYSOSC directly. User frequencies of 24 MHz and 16 MHz are supported with digital dividers of /6 and /4, respectively. SYSCTL manages the digital divider on this output to ensure a constant 4-MHz output regardless of the selected SYSOSC frequency.
 - Peripherals using this output through the MFCLK or MFPCLK see a continuous 4-MHz functional clock in RUN, SLEEP, and STOP modes, with reduced current in STOP mode when [gear shift](#) is enabled due to SYSOSC running natively at 4 MHz instead of at 32 MHz with a divide-by-8.

The SYSOSC is active at base frequency (32 MHz) by default after a brownout reset, sourcing MCLK.

2.3.1.2.1 SYSOSC Gear Shift

The SYSOSC can shift from base frequency (32 MHz) to low frequency (4 MHz), or low frequency back to base frequency, in a single SYSOSC clock cycle with the frequency change being phase aligned to minimize jitter. This feature is particularly well suited for low power, low cost microcontroller applications because it allows a single oscillator to be used to generate two frequencies:

- A high-speed 32-MHz clock (for running the CPU during compute), which is only enabled when needed
- A lower-speed 4-MHz clock (for running peripherals such as timers, UART interfaces, and I²C interfaces) which is continuously running

When a high-speed clock is needed (for example, in RUN and SLEEP modes), SYSOSC can run at its base frequency to provide high performance. In such a case, SYSOSC can be used directly to source MCLK so that MCLK can be used by the CPU, DMA, and peripherals clocked by the bus. For peripherals that run continuously but do not require the high clock frequency, the middle frequency system clock (MFCLK) is provided at 4 MHz by taking the 32-MHz SYSOSC and dividing down by 8.

When the fast 32-MHz clock is not needed (for example, if the CPU has completed processing data), then in STOP mode SYSOSC can be gear shifted down to 4 MHz in one cycle, and MFCLK switches to being clocked directly by SYSOSC- keeping a continuously running 4-MHz clock, but with a reduction in SYSOSC current and no need for a separate oscillator to provide the 4-MHz clock. When the CPU is again needed, SYSOSC can be switched back from 4 MHz to 32 MHz. This transition also occurs in one SYSOSC clock cycle. During the gear shift up, MFCLK switches back to SYSOSC/8 to keep the 4-MHz constant frequency. The timing transition between base frequency (32 MHz) and low frequency (4 MHz) during a gearshift down request is shown in [Figure 2-6](#).

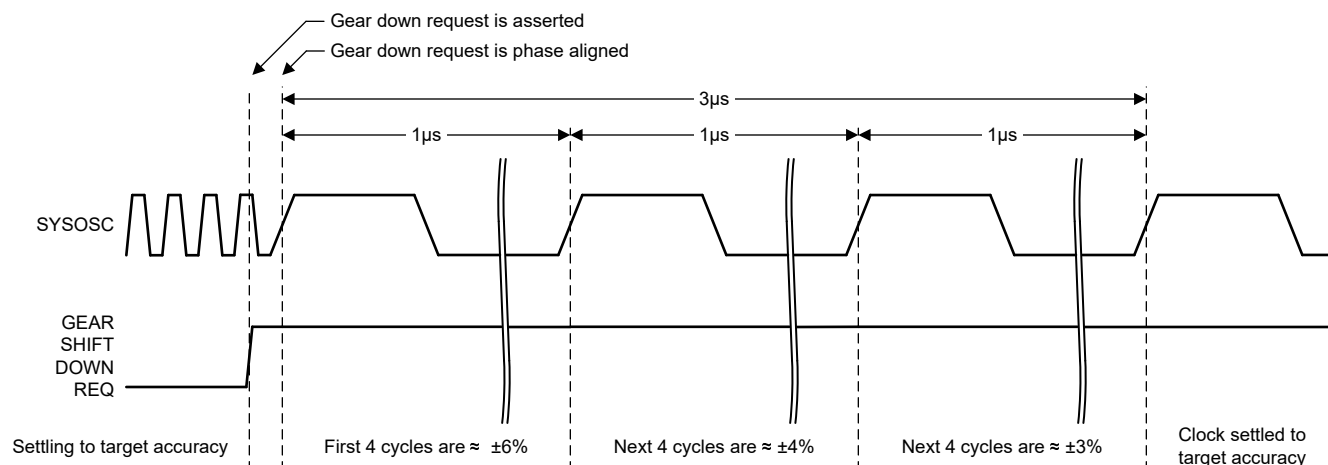


Figure 2-6. SYSOSC Gearshift Down Timing

The SYSOSC gearshift mode is tightly integrated with the power management scheme and is controlled by SYSCTL. Gearshift mode can be used when switching between RUN mode and STOP mode (gear down) and STOP mode and RUN mode (gear up). To use gearshift in STOP mode to reduce SYSOSC current, set the **USE4MHZSTOP** bit in the **SYSOSCCFG** register. Note that if SYSOSC is configured for 4-MHz operation in RUN and SLEEP mode as well as STOP mode (no gear shift), then the **USE4MHZSTOP** bit does not need to be set to keep the SYSOSC at 4 MHz when entering STOP mode.

2.3.1.2.2 SYSOSC Frequency and User Trims

SYSOSC comes factory trimmed for operation at either 32 MHz or 4 MHz. The default frequency of operation is the base frequency (32 MHz). It is possible to run SYSOSC continuously at 4 MHz for reduced current operation if 4 MHz provides suitable performance for the application. To change the SYSOSC frequency to 4 MHz, set the **FREQ** field in the **SYSOSCCFG** register to 4M.

Note

The **FREQ** field in the **SYSOSCCFG** register must only be changed when SYSOSC is the MCLK source. Do not change the SYSOSC **FREQ** configuration when MCLK is sourced from LFCLK or from HCLK.

SYSOSC also supports user-trimmed operation at either 24 MHz or 16 MHz. No other frequencies are allowed. To run SYSOSC at a user trimmed frequency, first ensure that SYSOSC is running at the base frequency (32 MHz). Then, configure the **SYSOSCTRIMUSER** register for the target frequency. Finally, select **USER** in the **FREQ** field of the **SYSOSCCFG** register. For trimming, see the [trim procedure](#).

Note

When running at a user frequency (16 MHz or 24 MHz), changing the SYSOSC frequency selection to low frequency (4 MHz) requires a transition through the base frequency (32 MHz) before SYSOSC switches to low frequency. The same is true when switching from low frequency to a user frequency. This change is managed entirely in hardware. Software can directly select the low frequency when running at a user frequency (and the reverse) but the hardware transitions through the base frequency when making the frequency change. If MFCLK (4 MHz) is enabled, it will be maintained at 4 MHz during the transition.

2.3.1.2.3 SYSOSC Frequency Correction Loop

The SYSOSC frequency accuracy can be improved through the use of the SYSOSC frequency correction loop (FCL) feature. The FCL circuit utilizes either an internal resistor or external resistor (populated between the ROSC pin and VSS), to stabilize the SYSOSC frequency by providing a precise reference current for the

SYSOSC. The overall frequency accuracy which is achievable depends on the operating temperature range together with the tolerance and temperature drift of the selected reference resistor.

Note

Not all devices support operation with an internal resistor and an external resistor for FCL. Some devices support external mode, some devices support internal mode, and some devices support both modes. See the device-specific data sheet to determine the features of a given device.

Note

The power consumption of SYSOSC is marginally higher with the FCL enabled due to the reference current that flows through ROSC. Settling time from startup to target accuracy can also be longer. See the device-specific data sheet for startup times.

2.3.1.2.3.1 SYSOSC FCL in External Resistor Mode (ROSC)

This section describes the procedure for selecting the external ROSC resistor (including computing the achievable SYSOSC accuracy) as well as the procedure for enabling the mode through the device SYSCTL registers.

SYSOSC Overall Frequency Accuracy with TI Recommended ROSC Resistor

The device-specific data sheet includes specifications for overall SYSOSC frequency accuracy when the TI-recommended $\pm 0.1\%$ tolerance, $\pm 25\text{-ppm}/^\circ\text{C}$, TCR resistor is used for ROSC. The TI-recommended resistor enables a variety of applications, including UART communication. If the overall accuracy values in the device-specific data sheet with the TI-recommended resistor specifications meets the application and cost requirements, then there is no need to compute the accuracy for an alternate resistor or alternate temperature ranges.

However, if a higher accuracy or reduced cost resistor is needed, the designer can use a more precise or less precise resistor for ROSC to balance cost with frequency accuracy for a given application scenario.

SYSOSC Overall Frequency Accuracy with Designer Selected Resistor

Assuming an ideal (zero error, zero temperature drift) reference resistor, the FCL mode improves the SYSOSC circuit accuracy to the level specified in the f_{SYSOSC} : "*SYSOSC frequency accuracy when frequency correction loop (FCL) is enabled and an ideal ROSC resistor is assumed*" section of the SYSOSC specifications table in the device-specific data sheet. The overall SYSOSC frequency accuracy in FCL mode which is achievable for a given external resistor and temperature range is determined by combining the following error sources to determine the total frequency error:

1. The ROSC resistor tolerance (at 25°C); this comes from the resistor specifications
2. The ROSC resistor temperature coefficient of resistance (TCR); this comes from the resistor specifications
3. The minimum and maximum expected operating temperatures for the system being designed
4. The SYSOSC frequency accuracy with an ideal resistor for the desired temperature range (taken from the MSPM0 device-specific data sheet)

The following table gives an example of how to compute the accuracy for different conditions, given the following typical application scenario:

- A target operating temperature range of -40 to 85°C .
- ROSC resistor with a $\pm 0.1\%$ tolerance and $\pm 25\text{ ppm}/^\circ\text{C}$ TCR selected

Table 2-4. Computing Overall SYSOSC FCL Frequency Accuracy based on External Resistor (ROSC) Specifications

Step	Description	MIN	TYP	MAX	UNIT
1	Define the operating temp range of the application.	-40	25	85	$^\circ\text{C}$
2	Select the target f_{SYSOSC} frequency (4 or 32 MHz)		32.00		MHz
3	Look up the corresponding f_{SYSOSC} ideal resistor accuracy from the device-specific data sheet based on the temperature range in Step 1.	-0.80		0.93	%

Table 2-4. Computing Overall SYSOSC FCL Frequency Accuracy based on External Resistor (ROSC) Specifications (continued)

Step	Description	MIN	TYP	MAX	UNIT
4	Compute the min and max f_{SYSOSC} with an ideal resistor based on the selected frequency in Step 2 and the accuracy range in Step 3.	31.744		32.298	MHz
5	Select an ROSC resistor with acceptable tolerance (the TI-recommended $\pm 0.1\%$ tolerance used in this example)	99.9	100	100.1	k Ω
6	Note the temperature coefficient of resistance (TCR) of the ROSC resistor.		± 25		ppm/ $^{\circ}\text{C}$
7	Compute the max temperature variation from 25 $^{\circ}\text{C}$ (nominal) based on the temperature range defined in Step 1.	$ 25 - (-40) = 65$			$^{\circ}\text{C}$
8	Compute the max ROSC drift over the max temperature variation (from Step 7) and the TCR (from Step 6).	-0.16		0.16	%
9	Compute the min and max ROSC resistance including tolerance (from Step 5) and temperature drift (from Step 8).	99.74		100.26	k Ω
10	Compute the overall ROSC resistor accuracy (in percent), using the overall ROSC resistance range from Step 9 vs. the nominal resistor value.	-0.26		0.26	%
11	Compute the variation in the min and max f_{SYSOSC} by degrading the min/max f_{SYSOSC} computed in Step 4 by the ROSC accuracy computed in Step 10.	31.66		32.38	MHz
12	If desired, convert the raw min/max f_{SYSOSC} computed in Step 11 into a %-based accuracy format relative to the target f_{SYSOSC} from Step 2.	-1.06		1.20	%

Enabling FCL with an External Resistor (ROSC)

To increase the SYSOSC accuracy with FCL, follow this procedure:

1. Verify that all digital functions on the ROSC pin are disabled in IOMUX (this is the default state after power up).
2. Verify that a ROSC reference resistor that meets the application accuracy requirements is placed between the ROSC pin and the device ground (VSS).
3. Enable FCL mode by setting the SETUSEFCL bit in the SYSOSCFCLCTL register.
 - a. If the device supports both internal resistor and external resistor FCL modes, also set the SETUSEEXRES bit in the SYSOSCFCLCTL register in addition to the SETUSEFCL bit.
4. When the FCL mode is enabled, software cannot disable the mode. A [BOOTRST](#) is required before a change to the FCL mode.

2.3.1.2.3.2 SYSOSC FCL in Internal Resistor Mode

This section describes the procedure for selecting the internal resistor mode through the device SYSCTL registers.

For devices which support the internal resistor FCL mode, the device-specific data sheet includes specifications for overall SYSOSC frequency accuracy across various temperature ranges. If the overall accuracy values in the device-specific data sheet meet the application and cost requirements, then there is no need to use the external resistor mode and the ROSC pin can be used for standard functions.

Enabling FCL with Internal Resistor

To increase the SYSOSC accuracy with FCL, follow this procedure:

1. Enable FCL mode by setting the SETUSEFCL bit in the SYSOSCFCLCTL register.
 - a. If the device supports both internal resistor and external resistor FCL modes, do not set the SETUSEEXRES bit in the SYSOSCFCLCTL register when setting the SETUSEFCL bit.
2. When the FCL mode is enabled, software cannot disable the mode. A [BOOTRST](#) is required before a change to the FCL mode.

2.3.1.2.4 SYSOSC User Trim Procedure

The SYSOSC can be trimmed to a user-trimmed value of 16 MHz or 24 MHz, if desired. Trimming is accomplished by manipulating the fields in the SYSOSCTRIMUSER to achieve a target frequency of 16 or 24 MHz. To trim the SYSOSC, the SYSOSC can be brought out externally to a pin using the [CLK_OUT](#) unit so that it can be measured. Follow the procedure below to derive the user trim values. Once trimmed values are obtained, they can be programmed in to the flash memory of the device to be recalled later. Each device must be trimmed individually for accuracy. SYSOSC can also be trimmed internally by using the frequency clock counter ([FCC](#)).

Trim procedure using CLK_OUT with frequency correction loop (FCL) disabled (No ROSC resistor)

1. Enable the CLK_OUT unit and select SYSOSC as the clock source, following the instructions in the [CLK_OUT](#) section
2. Set the SYSOSC frequency to BASE to start (ensure that the FREQ field in the SYSOSCCFG register is set to 0h for BASE), and leave the FCL mode disabled
3. Program an initial tuning for the target frequency in the SYSOSCTRIMUSER register:
 - a. Set the RCOARSE trim field in the SYSOSCTRIMUSER register to mid-range
 - b. Set the RFINE trim field in the SYSOSCTRIMUSER register to mid-range
4. Switch SYSOSC to the user-trimmed frequency by selecting USER in the FREQ field of the SYSOSCCFG register
5. Measure the SYSOSC frequency at the CLK_OUT pin
6. Switch SYSOSC back to BASE frequency by selecting BASE in the FREQ field of the SYSOSCCFG register
7. Adjust the SYSOSCTRIMUSER parameters to close on the target frequency:
 - a. Set the target frequency (16 or 24 MHz) in the FREQ field of the SYSOSCTRIMUSER register
 - b. Adjust trim parameters to achieve 16 or 24 MHz based on the selection made in step 7a
 - i. Increasing the CAP trim field in the SYSOSCTRIMUSER register will reduce the SYSOSC range. For 24 MHz, select CAP=0, and for 16 MHz select CAP=1
 - ii. Increasing the RCOARSE trim field in SYSOSCTRIMUSER will reduce the frequency in large steps of approximately 1 MHz
 - iii. Increasing the RFINE trim field in SYSOSCTRIMUSER will decrease the frequency in small steps of approximately 100 kHz
8. Repeat steps 4 through 7 until the desired accuracy has been achieved
9. Save the resulting SYSOSCTRIMUSER value for recall

Trim procedure using CLK_OUT with frequency correction loop (FCL) enabled (ROSC resistor present)

1. First, complete the FCL disabled trim procedure (defined above) to obtain starting values for the CAP, RCOARSE, and RFINE trim fields in the SYSOSCTRIMUSER register
2. Increase the RCOARSE value by 02h from the value obtained in Step 1
3. With CLK_OUT still enabled (SYSOSC as the source for CLK_OUT), ensure that the FREQ field in the SYSOSCCFG register is set to 0h for BASE, and enable the FCL mode by setting the SETUSEFCL bit in the SYSOSCFCLCTL register
4. Set the RDIV trim field in the SYSOSCUSERTRIM register to mid-range
5. Switch SYSOSC to the user-trimmed frequency by selecting USER in the FREQ field of the SYSOSCCFG register
6. Measure the SYSOSC frequency at the CLK_OUT pin
7. Switch SYSOSC back to BASE frequency by selecting BASE in the FREQ field of the SYSOSCCFG register
8. Adjust trim parameters to close on the target frequency:
 - a. Increasing the RDIV trim field in the SYSOSCTRIMUSER register will increase the frequency in small steps of approximately 50 kHz
 - b. Continue increasing or decreasing RDIV if the frequency is in saturation (not appearing to adjust)
9. Repeat steps 5-8 until the desired accuracy has been achieved
10. Save the resulting SYSOSCTRIMUSER value for recall

Note

SYSOSCTRIMUSER parameters only take effect when switching to the USER frequency. Once USER frequency is selected, changes to SYSOSCTRIMUSER have no effect. To refresh the USER values, first switch to BASE frequency, then update SYSOSCTRIMUSER, and then switch SYSOSC back to BASE frequency.

2.3.1.2.5 Disabling SYSOSC

SYSOSC can be disabled in STOP mode by setting the DISABLESTOP bit in the SYSOSCCFG register. Doing so forces the MCLK to use LFCLK in STOP mode (this is the STOP2 policy). This provides the lowest possible power consumption in STOP mode, as the system runs at 32 kHz and SYSOSC consumes no current. When exiting STOP mode to RUN mode, SYSCTL will automatically re-enable SYSOSC and switch MCLK back to SYSOSC.

SYSOSC can be disabled manually by setting the DISABLE bit in the SYSOSCCFG register. When SYSOSCCFG.DISABLE is set, the system will run from LFCLK in all power modes.

Note

The DISABLE and DISABLESTOP bits in SYSOSCCFG are mutually exclusive, and must not be set at the same time.

It is not possible to disable SYSOSC when using a different high frequency clock to source MCLK (such as HFCLK or the PLL). This is because SYSOSC is used by SYSCTL logic when MCLK is sourced from HFCLK or the PLL.

SYSOSC is always disabled automatically in STANDBY and SHUTDOWN modes.

2.3.1.3 System Phase-Locked Loop (SYSPLL)

The system phase locked loop (SYSPLL) takes an input reference clock SYSPLLREF and scales the input frequency to produce user-specified high frequency clocks (SYSPLLCLK0, SYSPLLCLK1, and SYSPLLCLK2X) for use by the device. Specifically, the SYSPLL clock outputs can be used as sources to MCLK and CANCLK.

Figure 2-7 shows the block diagram of the SYSPLL.

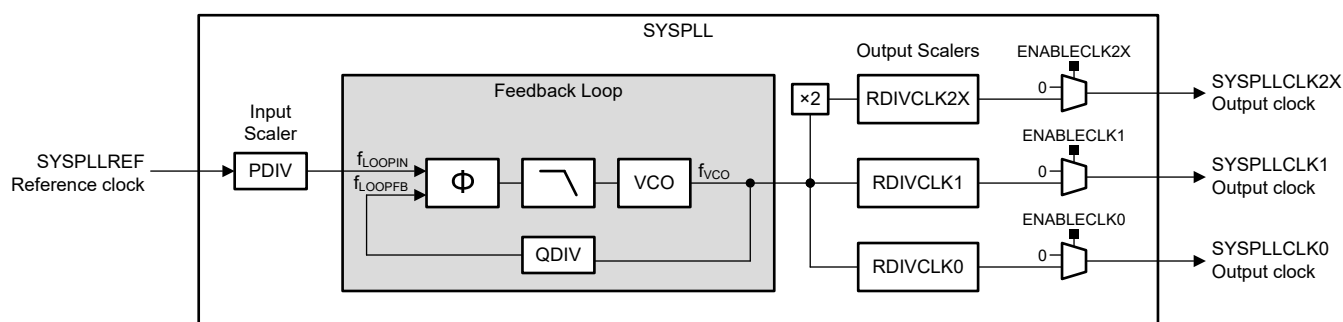


Figure 2-7. SYSPLL Diagram

Note

SYSOSC must not be disabled when the SYSPLL is enabled for use.

2.3.1.3.1 Configuring SYSPLL Output Frequencies

The SYSPLL accepts an input reference clock from 4-48 MHz. The available reference clocks include **SYSOSC** and **HFCLK**. The predivider PDIV scales the selected input reference clock ahead of the PLL feedback loop. PDIV can be selected as /1, /2, /4, or /8 by programming 0x0 to 0x3, respectively, into the PDIV field in the SYSPLLCFG1 register.

The effective divider can be calculated from the PDIV register setting as shown in Equation 6.

$$\text{SYSPLLREF}_{\text{DIV}} = 2^{\text{PDIV}} \quad (6)$$

The PLL feedback loop sets the voltage controlled oscillator (VCO) output equal to the divided input reference clock f_{LOOPIN} multiplied by the QDIV feedback divider. The QDIV divider is an integer divider with a valid range of /2 to /127. The desired QDIV divider is selected by programming 0x01 to 0x7E for /2 to /127, respectively, into the QDIV field of the SYSPLLCFG1 register. SYSPLLCFG1.QDIV=0x00 is an invalid configuration. The effective feedback divider can be calculated from the QDIV register setting as shown in [Equation 7](#).

$$\text{SYSPLLF}_{\text{DIV}} = \text{QDIV} + 1 \quad (7)$$

The output frequency of the VCO f_{VCO} is given in [Equation 8](#).

$$f_{\text{VCO}} = f_{\text{SYSPLLREF}} \times \text{SYSPLLF}_{\text{DIV}} / \text{SYSPLLREF}_{\text{DIV}} \quad (8)$$

The VCO output sources three separate SYSPLL outputs (SYSPLLCLK0, SYSPLLCLK1, and SYSPLLCLK2X). Each output has its own divider unit to enable generation of up to 3 different output frequencies for use by different modules in the device. The third output (SYSPLLCLK2X) also contains a frequency doubler before the divider unit to provide a wider range of output frequencies and lower power consumption.

For SYSPLLCLK2X, the output divider can be set from /1 to /16 in steps of 1. To set the SYSPLLCLK2X output divider, program 0x0-0xF for /1 up to /16, respectively, into the RDIVCLK2X field in the SYSPLLCFG0 register. [Equation 9](#) shows how to compute the effective SYSPLLCLK2X divider based on a given RDIVCLK2X register setting.

$$\text{SYSPLLCLK2X}_{\text{DIV}} = \text{RDIVCLK2X} + 1 \quad (9)$$

For SYSPLLCLK1 and SYSPLLCLK0, the output dividers can be set from /2 to /32 in steps of 2. To set the SYSPLLCLK0 or SYSPLLCLK1 output divider, program 0x0-0xF for /2 to /32, respectively, into the corresponding RDIVCLKx field in the SYSPLLCFG0 register. [Equation 10](#) shows how to compute the effective SYSPLLCLK0 divider based on a given RDIVCLK0 setting, and [Equation 11](#) shows how to compute the effective SYSPLLCLK1 divider based on a given RDIVCLK1 setting.

$$\text{SYSPLLCLK0}_{\text{DIV}} = 2 \times (\text{RDIVCLK0} + 1) \quad (10)$$

$$\text{SYSPLLCLK1}_{\text{DIV}} = 2 \times (\text{RDIVCLK1} + 1) \quad (11)$$

The SYSPLL output clock frequencies are thus set by the combination of f_{VCO} and the respective dividers:

$$f_{\text{SYSPLLCLK0}} = f_{\text{VCO}} / \text{SYSPLLCLK0}_{\text{DIV}} \quad (12)$$

$$f_{\text{SYSPLLCLK1}} = f_{\text{VCO}} / \text{SYSPLLCLK1}_{\text{DIV}} \quad (13)$$

$$f_{\text{SYSPLLCLK2X}} = 2 \times f_{\text{VCO}} / \text{SYSPLLCLK2X}_{\text{DIV}} \quad (14)$$

Enabling and Disabling the SYSPLL

After configuration, enable the SYSPLL by setting the SYSPLLEN bit in the HSCLKEN register. Before enabling the SYSPLL, make sure that the SYSPLL is in a disabled state by verifying that the SYSPLLOFF bit in the CLKSTATUS register is set. After the SYSPLL is enabled, application software must not disable it until the SYSPLLG00D or SYSPLLOFF bit is set in the CLKSTATUS register, indicating that the SYSPLL transitioned to a stable active state or a stable dead state. When the SYSPLL is enabled, the SYSPLL reference clock selection must not be changed.

Note

SYSOSC must be enabled and running at base frequency when the SYSPLL is enabled, even if HFCLK is used as the SYSPLL reference clock.

SYSPLL Usage Example

To illustrate the above relationships, take as an example the following requirements:

- The internal system oscillator (**SYSOSC**) is used as the SYSPLL reference (32 MHz)
- The SYSPLL must be configured with an output frequency of 80 MHz to source MCLK and 40 MHz to source CANCLK

To achieve this, the VCO can be configured for 80 MHz through the use of PDIV and QDIV. Then, SYSPLLCLK1 can feed CANCLK with an output divider of /2, and SYSPLLCLK2X can feed MCLK with an output divider of /2.

The steps below describe how to configure the CKM to use SYSPLL in this way:

1. Verify that the SYSPLL is disabled (SYSPLLOFF is set in CLKSTATUS)
2. Make sure that SYSOSC is running at base frequency (32 MHz); this is a requirement for SYSPLL operation even if HFCLK is used as the SYSPLL reference clock instead of SYSOSC
3. Set SYSOSC as the SYSPLL reference (make sure that the SYSPLLREF bit in the SYSPLLCFG0 register is cleared; this is the default state after reset)
4. Select a predivider PDIV to /2 (set SYSPLLCFG1.PDIV to 0x01), setting f_{LOOPIN} to 16 MHz (32 divided by 2)
5. Load the PLL parameters into SYSPLLPARAM0 and SYSPLLPARAM1 to support f_{LOOPIN} of 16 MHz
6. Set the feedback divider QDIV to 5 (set SYSPLLCFG1.QDIV to 4), giving $f_{\text{VCO}} = 80$ MHz (16 MHz multiplied by 5)
7. Set the SYSPLL output dividers for SYSPLLCLK1 and SYSPLLCLK2X to /2 (set SYSPLLCFG0.RDIVCLK1 to 0x0 and SYSPLLCFG0.RDIVCLK2X to 0x1) to get 40 MHz and 80 MHz at SYSPLLCLK1 and SYSPLLCLK2X, respectively
8. Enable SYSPLLCLK1 and SYSPLLCLK2X outputs by setting the ENABLECLK1 and ENABLECLK2X bits in the SYSPLLCFG0 register
9. With SYSOSC enabled and running at base frequency (32 MHz, this is the default state out of reset), enable the SYSPLL by setting SYSPLLEN in the HSCLKEN register
10. Wait for the SYSPLLG00D indication by testing SYSPLLG00D in the CLKSTATUS register
11. Select SYSPLLCLK2X as the PLL output to the HSCLK mux by setting MCLK2XVCO in the SYSPLLCFG0 register
12. Select the SYSPLL as the HSCLK source by ensuring that the HSCLKSEL bit is cleared in the HSCLKCFG register (this is the default state)
13. Select the high-speed clock (HSCLK) as the source for MCLK by setting the USEHSCLK bit in the MCLKCFG register. This will switch MCLK from SYSOSC to HSCLK. MCLK is now running from SYSPLLCLK2X at 80 MHz.
14. To configure the CANCLK, set the CANCLKSRC bit in the GENCLKCFG register.

The SYSPLL divider values used in this example are summarized below for reference.

Table 2-5. SYSPLL Divider Example Settings

Parameter	Register	Bit Field	Bit Field Value	Actual Divider
Input reference clock divider	SYSPLLCFG1	PDIV	0x1	/2
VCO feedback loop divider	SYSPLLCFG1	QDIV	0x4	/5
Output clock 1 divider	SYSPLLCFG0	RDIVCLK1	0x0	/2
Output clock 2X divider	SYSPLLCFG0	RDIVCLK2X	0x1	/2

Tuning Guidelines

In cases where there are multiple combinations of PDIV, QDIV, and RDIVCLKx that provide the desired output frequencies, consider these tuning guidelines to determine the best possible values for an application:

- Lower VCO frequencies (f_{VCO}) result in lower power consumption. Refer to the device data sheet for the allowable range of f_{VCO} .
- Higher feedback loop input frequencies (f_{LOOPIN}) have faster startup. For example, if an 80-MHz output frequency is desired with $f_{VCO} = 40\text{ MHz}$, $f_{VCO} = 40\text{ MHz}$ can be derived from a SYSPLLREF clock 32 MHz by setting PDIV to /8 and QDIV to /10. However, this gives slower startup because f_{LOOPIN} is <8 MHz. The same result can be achieved by setting PDIV to /4 and QDIV to /5, but because f_{LOOPIN} is 8 MHz in this case (32 MHz divided by 4), the [SYSPLL parameters](#) for the higher input range can be used which give faster startup.

2.3.1.3.2 Loading SYSPLL Lookup Parameters

Several tuning parameters must be configured in the SYSPLLPARAM0 and SYSPLLPARAM1 registers before using the SYSPLL. The values are determined by PLL feedback loop input clock frequency (f_{LOOPIN}).

SYSPLL supports four f_{LOOPIN} frequency ranges, given in [SYSPLL Parameter Lookup](#). Each frequency range has a 64-bit lookup value in the [FACTORY flash memory region](#) that must be copied from flash into the SYSPLLPARAM0 and SYSPLLPARAM1 registers in SYSCTL before enabling the SYSPLL.

Table 2-6. SYSPLL Parameter Lookup

f_{LOOPIN}	Lookup Address (PARAM0)	Lookup Address (PARAM1)
$32\text{ MHz} \leq \text{FREQ} \leq 48\text{ MHz}$	0x41C4.0034	0x41C4.0038
$16\text{ MHz} \leq \text{FREQ} < 32\text{ MHz}$	0x41C4.002C	0x41C4.0030
$8\text{ MHz} \leq \text{FREQ} < 16\text{ MHz}$	0x41C4.0024	0x41C4.0028
$4\text{ MHz} \leq \text{FREQ} < 8\text{ MHz}$	0x41C4.001C	0x41C4.0020

2.3.1.3.3 SYSPLL Startup Time

The PLL startup time depends on the PLL feedback loop input frequency f_{LOOPIN} and if the PLL is starting after previously running (for example, exiting a low-power mode) or starting for the first time after device boot. [Table 2-7](#) lists the PLL startup times.

Table 2-7. SYSPLL Startup Times

f_{LOOPIN}	SYSPLL Startup Time (μs)	SYSPLL Startup Time on Exit From Low-Power Mode (μs)
$32\text{ MHz} \leq \text{FREQ}$	5	3
$16\text{ MHz} \leq \text{FREQ} < 32\text{ MHz}$	15	12
$8\text{ MHz} \leq \text{FREQ} < 16\text{ MHz}$	25	20
$4\text{ MHz} \leq \text{FREQ} < 8\text{ MHz}$	40	35

2.3.1.4 Low Frequency Crystal Oscillator (LFXT)

The low-frequency crystal oscillator (LFXT) is an ultra-low-power crystal oscillator that supports a standard 32.768-kHz watch crystal.

To use the LFXT, place a watch crystal between the LFXIN and LFXOUT pins. Place loading capacitors on both pins to circuit ground (VSS). Size the crystal load capacitors according to the specifications of the crystal being used. A variety of crystal types are supported through a programmable drive mechanism that allows for balancing drive current with the required drive strength for a particular crystal.

The LFXT pins LFXIN and LFXOUT are shared with digital IO functions. To use the LFXT, first configure the IOMUX to support LFXT functionality on the LFXIN and LFXOUT pins. Configure IOMUX to disable any digital IO functionality on the LFXIN and LFXOUT pins. The LFXT defaults to the highest drive strength (LFCLKCFG.XT1DRIVE == 0x03 in SYSCTL), which is recommended for fast reliable start-up of the oscillator. If a crystal with ultra-low capacitance (<3 pF) is used, power consumption of the LFXT can be lowered further by setting the LOWCAP bit LFCLKCFG (LOWCAP is cleared by default for compatibility with most crystals).

After completing configuration, start the LFXT by setting the STARTLFXT bit in the LFXTCTL register in SYSCTL. When the oscillator has started successfully, the [LFXT startup monitor](#) sets the LFXTGOOD bit in

the CLKSTATUS register in SYSCTL. Crystal drive strength can then be reduced to conserve power, if desired. Once STARTLFXT is set by the application software, the internal LFOSC is disabled to conserve power while the LFXT starts in anticipation of LFCLK moving to the LFXT.

To switch the LFCLK tree to use LFXT as the 32-kHz clock source rather than LFOSC, set the SETUSELFXT bit in the LFXCTL register, and SYSCTL permanently switches the LFCLK source to LFXT.

After the LFXT is enabled, the internal LFOSC is permanently disabled. LFOSC cannot be re-enabled and LFCLK thus cannot be switched back to LFOSC other than by executing a BOOTRST.

Note

To source MCLK from LFCLK with LFCLK sourced from the LFXT, first configure LFCLK to use LFXT and then configure MCLK to use LFCLK. Do not switch MCLK to LFCLK while LFCLK runs from LFOSC, and then later switch LFCLK to LFXT.

Note

In the event that LFXT is selected for LFCLK at startup but the crystal fails to start properly, it is necessary to execute a BOOTRST to re-enable the internal LFOSC and run LFCLK from LFOSC or a 32kHz LFCLK will not be available for the system. In a scenario where LFXT fails to start, application software may leverage the backup memory to store a flag indicating that the crystal did not start properly, and then execute the BOOTRST. The backup memory is preserved through a BOOTRST and thus can be read after the BOOTRST cycle so that in the next startup sequence the application code can decide to keep LFCLK sourced from LFOSC versus the external crystal (in the event that there is a hardware issue with the crystal preventing it from starting).

2.3.1.5 LFCLK_IN (Digital Clock)

It is possible to bypass the LFXT circuit and bring in a 32.768kHz typical frequency digital clock into the device to use as the LFCLK source instead of LFOSC or LFXT. To configure LFCLK to use a digital clock input instead of LFXT or LFOSC, first configure the IOMUX to enable the LFCLK_IN function on the appropriate pin. When IOMUX is configured correctly and the external clock source is outputting a 32 kHz clock to LFCLK_IN, set the SETUSEEXLF bit in the EXLCTL register in SYSCTL.

LFCLK_IN is compatible with digital square wave CMOS clock inputs and should have a typical duty cycle of 50%.

It is possible to check for a valid clock signal on LFCLK_IN by enabling the LFCLK monitor before setting SETUSEEXLF in the EXLCTL register. By default, the LFCLK monitor will check LFCLK_IN if the LFXT was not started.

After LFCLK_IN is selected as the LFCLK source, it is not possible to change back to LFOSC or LFXT without going through a BOOTRST.

Note

If MCLK is to be sourced from LFCLK with LFCLK sourced from LFCLK_IN, first configure LFCLK to use LFCLK_IN and then configure MCLK to use LFCLK. Do not switch MCLK to LFCLK with LFCLK running from LFOSC, and then later switch LFCLK to LFCLK_IN.

Note

LFCLK_IN and LFXT are mutually exclusive and must not be enabled at the same time. Do not set the SETUSEEXLF bit in the EXLCTL register if the SETUSELFXT bit or the STARTLFXT bit is set in the LFXCTL register.

2.3.1.6 High Frequency Crystal Oscillator (HFXT)

The high frequency crystal oscillator (HFXT) can be used with standard crystals and resonators in the 4-48MHz range to generate a stable high-speed reference clock for the system. The HFXT can be used to clock the

primary device clock tree (MCLK) directly, or it can be used as a precision reference to the on-chip PLL where higher frequencies can be generated. In addition, the HFXT can be provided directly to the ADC as the sampling clock source or the CAN-FD peripheral as the functional clock source, asynchronous from the main system clock which can run from the PLL or SYSOSC.

To use the HFXT, a crystal or resonator must be populated between the HFXIN and HFXOUT pins. Loading capacitors must be placed on both pins to circuit ground (VSS). The crystal load capacitors must be sized according to the specifications of the crystal being used. The IOMUX must be configured to enable HFXT functionality on the HFXIN and HFXOUT pins. Configure IOMUX to disable any digital IO functionality on the HFXIN and HFXOUT pins. The HFXT frequency range must be set by configuring the HFXTRSEL bits in the HFCLKCLKCFG register in SYSCTL.

A programmable HFXT startup time is provided with 64µs resolution. Program an appropriate startup time based on the desired crystal or resonator specifications into the HFXTTIME field in the HFCLKCLKCFG register in SYSCTL before starting the HFXT.

Once configured properly, the HFXT is started by setting the HFXTEN bit in the HSCLKEN register in SYSCTL. When the oscillator has started successfully, the [HFCLK startup monitor](#) will assert the HFCLKGOOD bit in the CLKSTATUS register in SYSCTL.

Note

[SYSOSC](#) must be enabled at base frequency when the HFXT is enabled.

After setting HFXTEN to enable the HFXT, application software must verify that either an HFCLKGOOD indication or an HFCLKOFF (off/dead) indication in the CLKSTATUS register was asserted by hardware before attempting to disable the HFXT by clearing HFXTEN. When disabling the HFXT by clearing HFXTEN, the HFXT must not be re-enabled again until the HFCLKOFF bit in the CLKSTATUS register is set by hardware.

To use HFXT as the PLL reference after receiving an HFCLKGOOD status, set the SYSPLLREF bit in the SYSPLLCFG0 register in SYSCTL. If HFXT is selected as a reference for the SYSPLL and the SYSPLL is enabled, then the SYSPLL must be disabled and the SYSPLLOFF bit in the CLKSTATUS register must be set before the HFXT can be disabled.

To use the HFXT directly as the MCLK source after receiving an HFCLKGOOD status, first set the HSCLKSEL bit in the HSCLKCFG register to select HFCLK as the high-speed clock source (rather than the system PLL output). Then, set the USEHSCLK bit in the MCLKCFG register to select the high-speed clock source as the MCLK source. Once USEHSCLK is set, HSCLKCFG must not change and the HFXT must not be disabled until the MCLK source is switched back to SYSOSC by clearing USEHSCLK and verifying that the HSCLKMUX bit in CLKSTATUS is cleared by hardware.

2.3.1.7 HFCLK_IN (Digital clock)

It is possible to bypass the HFXT circuit and bring in a 4- to 48-MHz typical frequency digital clock into the device to use as the HFCLK source instead of HFXT. To configure HFCLK to use a digital clock input instead of HFXT, first configure the IOMUX to enable the HFCLK_IN function on the appropriate pin. When IOMUX is configured correctly and the clock source is outputting a clock to HFCLK_IN, set the USEEXTHFCLK bit in the HSCLKEN register in SYSCTL.

Note

[SYSOSC](#) must be enabled at base frequency when the HFCLK_IN is enabled.

To use HFCLK_IN as the PLL reference after selecting HFCLK_IN as the HFCLK source, set the SYSPLLREF bit in the SYSPLLCFG0 register in SYSCTL. If HFCLK_IN is selected as a reference for the SYSPLL (through HFCLK) and the SYSPLL is enabled, then the SYSPLL must be disabled and the SYSPLLOFF bit in the CLKSTATUS register must be set before the HFCLK_IN can be disabled by clearing the USEEXTHFCLK bit in HSCLKEN.

To source MCLK from HFCLK_IN after selecting HFCLK_IN as the HFCLK source, first set the HSCLKSEL bit in the HSCLKCFG register to select HFCLK as the high-speed clock source (rather than the system PLL output). Then, set the USEHSCLK bit in the MCLKCFG register to select the high-speed clock source as the MCLK source. Once USEHSCLK is set, HSCLKCFG must not change and the HFCLK_IN must not be disabled until the MCLK source is switched back to SYSOSC by clearing USEHSCLK and verifying that the HSCLKMUX bit in CLKSTATUS was cleared by hardware.

HFCLK_IN is compatible with digital square wave CMOS clock inputs and should have a typical duty cycle of 50%.

Note

HFCLK_IN and HFXT are mutually exclusive and must not be enabled at the same time. Do not set the USEEXTHFCLK bit if the HFXTEN bit is also set in HSCLKEN.

2.3.2 Clocks

The CKM takes oscillator outputs and generates a variety of functional clocks for use by the device.

Clocks

- System Clocks
 - **MCLK**: Main system clock for PD1 peripherals and PD1 bus
 - **CPUCLK**: CPU clock, derived from MCLK
 - **ULPCLK**: Main system clock for PD0 peripherals and PD0 bus, derived from MCLK
 - **MFCLK**: Fixed 4 MHz clock, synchronized to MCLK/ULPCLK
 - **MFPCLK**: Fixed 4 MHz clock
 - **LFCLK**: Fixed 32 kHz clock, synchronized to MCLK/ULPCLK
 - **HFCLK**: High frequency external clock
 - **HSCLK**: High speed configurable clock for use by MCLK, sourced from SYSPLL or HFCLK
- Peripheral Specific Clocks
 - **ADCCLK**: ADC sampling period clock
 - **CANCLK**: CAN-FD module functional clock
 - **RTCCLK**: Fixed 32 kHz clock direct to RTC
- External Clocks
 - **CLK_OUT**: External clock output with divider for pushing out a clock to external circuits

All clocks are disabled in SHUTDOWN mode.

In addition to the configurable clocks listed above, several direct clock connections are made to analog peripherals (see the [Section 2.3.2.13](#) section).

2.3.2.1 MCLK (Main Clock) Tree

The MCLK is the main system clock and the root point of synchronization for all synchronized clocks (MCLK, CPUCLK, ULPCLK, MFCLK, and LFCLK). It is typically the highest speed clock in the system and supports operation up to 80 MHz across the full temperature range of the device. The MCLK tree is the root source for the **CPUCLK** (in RUN mode), the PD1 high speed peripheral bus clock (in RUN and SLEEP modes), and the **ULPCLK** low power bus clock (in RUN, SLEEP, STOP, and STANDBY modes). In addition, the 4-MHz **MFCLK** and 32-kHz **LFCLK** outputs are synchronized to MCLK.

The MCLK output to PD1 peripherals is enabled in RUN and SLEEP modes, and disabled in all other power modes. While the MCLK output to PD1 is disabled in STOP and STANDBY modes, the MCLK tree is still running to source ULPCLK and to provide synchronization for MFCLK and LFCLK.

The MCLK source is selected with a glitch free clock mux and can be changed dynamically at runtime by user software. It can also be changed automatically by hardware when entering STOP and STANDBY modes or during an [asynchronous fast clock request](#).

The available sources for MCLK include:

- **HSCLK** (high-speed clock) at up to 80 MHz which can be sourced by:
 - **SYSPLLCLK0** or **SYSPLLCLK2X** (used to reach the max MCLK speed of 80 MHz)
 - **HFCLK** for applications where the main clock needs to be as accurate as possible
- **SYSOSC** at 4, 16, 24, or 32 MHz (the default clock source after reset)
- **SYSOSC** at 4 MHz with **MDIV** divider (for applications where the peripheral bus and CPU run between 250 kHz and 4 MHz)
- **LFCLK** at 32 kHz for applications where the entire system, including the CPU, runs at 32 kHz with low peak operating current

NOTE: Set **SYSPLLCLK0** or **SYSPLLCLK2X** as the source of **HSCLK** before setting **HSCLK** as the clock source of **MCLK**, otherwise the device can be in an unpredictable state.

Using MCLK in RUN and SLEEP Mode

After boot, MCLK is sourced from **SYSOSC** by default. The decision of which oscillator to use to source MCLK is important because MCLK sets both the CPUCLK frequency and the bus clock frequency for PD1 peripherals. As a result, the accuracy and the clock speed of the oscillator selected for MCLK must be appropriate not only for the operation of the CPU but also for the operation of the PD1 peripherals that use the bus clock as their functional clock.

The clock source and frequency selection decisions made for MCLK also affect ULPCLK in RUN and SLEEP modes. See the **ULPCLK** section for more information on how MCLK and ULPCLK are related in RUN and SLEEP mode.

Using MCLK in STOP and STANDBY Mode

In STOP and STANDBY modes, the MCLK output to PD1 peripherals is disabled, but the **ULPCLK**, which is the bus clock for PD0 peripherals, is still active in STOP and optionally active in STANDBY. See the **ULPCLK** section for more information on how the MCLK source and ULPCLK are related in STOP and STANDBY mode.

MCLK Source Selection

Application software can change the MCLK source from SYSOSC to the **SYSPLL** or to the **HFCLK** (which is either **HFXT** or **HFCLK_IN**) by configuring the MCLKCFG.USEHSCLK and HSCLKCFG.HSCLKSEL register bits appropriately in SYSCTL. It is also possible to select LFCLK as the MCLK source in all modes by setting MCLKCFG.USELFCLK, giving the low peak current consumption with the CPU and PD1 peripherals operational. [Table 2-8](#) gives the proper register bit configurations for selecting different clocks for MCLK in RUN and SLEEP modes.

Table 2-8. MSPM0Gxx MCLK Source Selection in RUN and SLEEP Mode

Desired Source	MCLKCFG.USEHSCLK	MCLKCFG.USELFCLK	HSCLKCFG.HSCLKSEL	SYSPLLCFG0.MCLK2XVCO
SYSOSC	0	0	Don't care	Don't care
SYSPLLCLK0	1	0	0	0
SYSPLLCLK2X	1	0	0	1
HFCLK	1	0	1	Don't care
LFCLK	0	1	Don't care	Don't care

To switch MCLK from SYSOSC to LFCLK in RUN mode:

1. If any high speed oscillators (SYSPLL, HFXT, HFCLK_IN) are enabled, disable them before proceeding
2. Verify that MCLK is sourced from SYSOSC (CLKSTATUS.CURMCLKSEL is cleared)
3. If SYSOSC is not running at base frequency, and SYSOSC is to be left enabled when switching MCLK to LFCLK, set SYSOSC to base frequency before proceeding
4. Set MCLKCFG.USELFCLK to switch MCLK to LFCLK and leave SYSOSC enabled, or set SYSOSCCFG.DISABLE to switch MCLK to LFCLK and disable SYSOSC

To switch MCLK from LFCLK to SYSOSC in RUN mode:

1. Verify that MCLK is sourced from LFCLK (CLKSTATUS.CURMCLKSEL is set)
2. Clear MCLKCFG.USELFCLK or SYSOSCCFG.DISABLE, whichever was set to switch MCLK to LFCLK

To switch MCLK from SYSOSC to HSCLK:

1. Verify that MCLK is sourced from SYSOSC (CLKSTATUS.HSCLKMUX is cleared).
2. Enable the desired high speed sources (SYSPLL, HFXT, HFCLK_IN) according to their respective requirements.
3. Select the desired HSCLK source through the HSCLKCFG.HSCLKSEL control
4. Verify that CLKSTATUS.HSCLKGOOD is set, indicating that the selected HSCLK source is valid.
5. Set MCLKCFG.USEHSCLK to switch MCLK to HSCLK.

To switch MCLK from HSCLK to SYSOSC:

1. Verify that MCLK is sourced from HSCLK (CLKSTATUS.HSCLKMUX is set).
2. Clear MCLKCFG.USEHSCLK to switch MCLK to SYSOSC.
3. Wait for CLKSTATUS.HSCLKMUX to clear, indicating that MCLK is now sourced from SYSOSC.
4. If desired, disable any high-speed clock sources (SYSPLL or HFXT)

As shown in [Table 2-8](#), the USELFCLK and USEHSCLK bits in MCLKCFG are mutually exclusive and must not be set at the same time. To switch MCLK from LFCLK to HSCLK, or HSCLK to LFCLK, MCLK is first switched to SYSOSC before switching to the final clock source. The procedures below describe how to switch MCLK from HSCLK to LFCLK or from LFCLK to HSCLK.

To switch MCLK from HSCLK to LFCLK:

1. Verify that MCLK is sourced from HSCLK (CLKSTATUS.HSCLKMUX is set).
2. Clear MCLKCFG.USEHSCLK to switch MCLK to SYSOSC
3. Wait for CLKSTATUS.HSCLKMUX to clear, indicating that MCLK is now sourced from SYSOSC
4. Disable any high speed sources (SYSPLL, HFXT, HFCLK_IN) which were enabled
5. Wait for CLKSTATUS.HSCLKSOFF to be asserted, indicating that high-speed clocks are off
6. Set MCLKCFG.USELFCLK to switch MCLK to LFCLK while leaving SYSOSC enabled, or set SYSOSCCFG.DISABLE to switch MCLK to LFCLK and disable SYSOSC

To switch MCLK from LFCLK to HSCLK:

1. Verify that MCLK is sourced from LFCLK (CLKSTATUS.CURMCLKSEL is set)
2. Clear SYSOSCCFG.DISABLE or MCLKCFG.USELFCLK, based on which was set previously
3. Wait for CLKSTATUS.CURMCLKSEL to clear, indicating that MCLK is now sourced from SYSOSC
4. Enable the desired high speed oscillators (SYSPLL, HFXT, HFCLK_IN) according to their respective requirements
5. Select the desired HSCLK source through the HSCLKCFG.HSCLKSEL control
6. Verify that CLKSTATUS.HSCLKGOOD is set, indicating that the selected HSCLK source is valid
7. Set MCLKCFG.USEHSCLK to switch MCLK to HSCLK

Note

When MCLK is actively sourced by HSCLK (the HSCLKMUX bit in the CLKSTATUS register is set), the HSCLK source selection must not be changed (the HSCLKSEL bit in the HSCLKCFG register and the MCLK2XVCO bit in the SYSPLLCFG0 register must not be changed). To change the HSCLK source, first switch MCLK to SYSOSC using the procedure given above, re-configure the HSCLK source, and then switch MCLK back to HSCLK.

MCLK Divider (MDIV)

An MCLK source divider (MDIV) is provided to enable MCLK operation in between the lowest SYSOSC frequency (4 MHz) and the LFCLK frequency (32 kHz). MDIV is for applications with a limited peak current but that still require a higher clock speed than 32 kHz. MDIV supports dividing the 4-MHz SYSOSC frequency

by up to 16, enabling the additional MCLK frequency options given in [Table 2-9](#). For example, a 500-kHz MCLK frequency can be obtained by setting SYSOSC to 4 MHz and setting MDIV to 7 (divide-by-8).

[Table 2-9](#) shows the MCLK frequency which is realized with /2, /4, /8, and /16 MDIV configurations, but MDIV can be set to any integer divider between /2 and /16 (MDIV register values of 0x1 through 0xF, respectively, with 0x0 disabling the MDIV).

Table 2-9. Typical MCLK Configurations for Operation Between 4 MHz and 32 kHz

MCLK Source	MDIV	MCLK Frequency
SYSOSC (4 MHz)	0 (Disabled)	4 MHz
	1 (/2)	2 MHz
	3 (/4)	1 MHz
	7 (/8)	500 kHz
	15 (/16)	250 kHz

To use MDIV to operate MCLK at an intermediate frequency below 4 MHz, follow the steps below:

1. Disable [asynchronous fast clock requests](#) (make sure that the BLOCKASYNCALL bit is set in the in SYSOSCCFG register)
2. Make sure that SYSOSC is selected as the MCLK source
3. Set the SYSOSC frequency to 4 MHz (set the FREQ field in the SYSOSCCFG register to 0x01)
4. Delay for 10 MCLK cycles
5. Set the desired divider value in the MDIV field in the MCLKCFG register (from 1 to 15, corresponding to /2 to /16, respectively)

Several rules apply when using MDIV to reduce the MCLK frequency:

- MDIV must not be used when high speed oscillators are used to source MCLK (SYSPLL, HFCLK); if SYSPLL or HFCLK are selected to source MCLK, MDIV must be disabled (MCLKCFG.MDIV=0x00)
- The SYSOSC frequency must be kept at 4 MHz when MDIV is enabled
- [Asynchronous requests to change the SYSOSC frequency](#) must remain blocked

To disable MDIV:

1. Disable MDIV (set the MDIV field in the MCLKCFG register to 0x0)
2. Wait for 16 MCLK cycles before changing the SYSOSC frequency from 4 MHz to another frequency

Note

MDIV does not apply to LFCLK, as MDIV is not in the LFCLK path when LFCLK is selected as the MCLK source. If LFCLK is selected for MCLK, MDIV must be disabled.

2.3.2.2 CPUCLK (Processor Clock)

The processor clock (CPU clock) is always derived directly from MCLK and is active in RUN mode at the MCLK frequency. In all other power modes, CPUCLK is disabled.

2.3.2.3 ULPCLK (Low-Power Clock)

The ULPCLK is the bus clock for peripherals in the PD0 power domain. It supports operation up to 40 MHz and is derived directly from the MCLK tree through a clock divider (UDIV) which is enabled only when MCLK is sourced from a high-speed clock (SYSPLL, HFXT, or HFCLK_IN). The ULPCLK frequency is dependent on the MCLK configuration and the selected power mode.

ULPCLK Behavior in RUN and SLEEP Modes

The PD0 power domain has a frequency limit of 40 MHz in **RUN** and **SLEEP** modes. As such, ULPCLK must be maintained to be ≤ 40 MHz at all times. When MCLK is configured to run from SYSOSC or LFCLK, SYSCTL disables UDIV automatically and $f_{ULPCLK} = f_{MCLK}$ as these clock sources are always ≤ 32 MHz.

However, when MCLK is configured to run from a high-speed clock (SYSPLL, HFXT, or HFCLK_IN), hardware cannot ensure that ULPCLK is ≤ 40 MHz because SYSCTL does not have knowledge of the MCLK frequency. In this case it is the responsibility of the application software to ensure that ULPCLK is ≤ 40 MHz in **RUN** and **SLEEP** modes by configuring UDIV appropriately. By default, the ULPCLK divider is set to divide-by-2 (MCLKCFG.UDIV==0x1) which provides safe operation up to the max MCLK frequency (80 MHz). If MCLK is configured to be ≤ 40 MHz, the ULPCLK speed is set equal to the MCLK speed by changing MCLKCFG.UDIV from 0x01 to 0x0 (divide-by-1). When UDIV==0x0, MCLK==ULPCLK and accesses to PD0 peripherals do not incur additional latency.

ULPCLK Behavior in STOP and STANDBY Modes

SYSCTL automatically disables UDIV in **STOP** and **STANDBY** modes.

- In **STOP** mode, the MCLK tree (and by extension, the ULPCLK) can run from SYSOSC with a 4 MHz rate (if SYSOSCCFG.DISABLESTOP=0x0) or from LFCLK at 32 kHz (if SYSOSCCFG.DISABLESTOP=0x1). When SYSOSC is used (SYSOSCCFG.DISABLESTOP=0x0), SYSCTL ensures that ULPCLK is always 4 MHz even if SYSOSC is running at a higher frequency (due to user configuration or due to an asynchronous request from a peripheral).
- In **STANDBY** mode, the MCLK tree (and by extension, the ULPCLK) either run from LFCLK (**STANDBY0**) or are disabled (**STANDBY1**) to conserve power. In **STANDBY1**, only the TIMG0 and TIMG1 timer peripherals receive ULPCLK.

Table 2-10. MSPM0Gxx ULPCLK by Operating Mode

Selected Power Mode	Configuration	Register Settings	ULPCLK Frequency
RUN or SLEEP (40 MHz maximum)	MCLK source is SYSOSC (RUN0, SLEEP0)	MCLKCFG.USEHSCLK=0x0, MCLKCFG.USELFCLK=0x0	ULPCLK is sourced from MCLK according to the MCLK configuration with $f_{ULPCLK} = f_{MCLK}$
	MCLK source is HSCLK (SYSPLL, HFXT, or HFCLK_IN) (RUN0, SLEEP0)	MCLKCFG.USEHSCLK=0x1, MCLKCFG.USELFCLK=0x0	ULPCLK is sourced from MCLK according to the MCLK configuration with $f_{ULPCLK} = f_{MCLK} / \text{UDIV}$
	MCLK source is LFCLK (RUN1/2, SLEEP1/2)	MCLKCFG.USELFCLK=0x1 or SYSOSCCFG.DISABLE=0x1	ULPCLK is sourced from LFCLK with $f_{ULPCLK} = f_{LFCLK} = 32$ kHz
STOP (4 MHz maximum)	STOP with SYSOSC enabled (STOP0/1)	SYSOSCCFG.DISABLESTOP = 0x0	ULPCLK is sourced from SYSOSC with $f_{ULPCLK} = 4$ MHz
	STOP with SYSOSC disabled (STOP2)	SYSOSCCFG.DISABLESTOP = 0x1	ULPCLK is sourced from LFCLK with $f_{ULPCLK} = f_{LFCLK} = 32$ kHz
STANDBY (32 kHz maximum)	STANDBY with ULPCLK and LFCLK enabled (STANDBY0)	MCLKCFG.STOPCLKSTBY= 0x0	ULPCLK is sourced from LFCLK with $f_{ULPCLK} = f_{LFCLK} = 32$ kHz
	STANDBY with ULPCLK and LFCLK disabled (STANDBY1)	MCLKCFG.STOPCLKSTBY= 0x1	ULPCLK is disabled to all peripherals except TIMG0 and TIMG1, which receive $f_{ULPCLK} = f_{LFCLK} = 32$ kHz
SHUTDOWN (Off)	-	-	ULPCLK is off

2.3.2.4 MFCLK (Middle Frequency Clock)

The MFCLK provides a continuous 4-MHz clock to a variety of peripherals on the device. The MFCLK 4-MHz rate is always derived from the **SYSOSC**. As the SYSOSC frequency is not fixed (it can be configured for 32, 24, 16, or 4 MHz), SYSCTL automatically applies a divider to SYSOSC to keep MFCLK at a constant 4-MHz rate regardless of the current SYSOSC frequency. MFCLK can be used by peripherals such as timers and serial interfaces that require a constant clock source in **RUN**, **SLEEP**, and **STOP** power modes.

After a SYSRST, MFCLK is initially disabled. MFCLK can be enabled in software by setting USEMFTICK in the MCLKCFG register in SYSCTL. MFCLK is active in RUN, SLEEP, and STOP power modes only, and SYSOSC must be enabled for MFCLK to operate.

All MFCLK edges are synchronized to the main system clocks (MCLK and ULPCLK), meaning that the registers of peripherals clocked by MFCLK can be read or written to at any time without any special handling.

Peripherals can select MFCLK as their functional clock source through their respective [CLKSEL](#) mux. Not all peripherals support running from MFCLK.

Using MFCLK in STOP Mode

When using MFCLK in STOP mode, SYSOSC can be configured to automatically switch to 4 MHz (low frequency) when entering STOP mode and automatically switch back to the previously selected frequency when exiting STOP mode to RUN mode (gear shift mode). As MFCLK is a 4 MHz clock source, running SYSOSC at 4 MHz in STOP mode reduces power consumption when in STOP mode. To configure SYSOSC for gear shift mode, see [Section 2.3.1.2.1](#).

Requirements for Using MFCLK

1. When using MFCLK, the MDIV (MCLK divider) must be disabled (set to /1). Disable MDIV by setting MDIV in the MCLKCFG register to 0x0. SYSCTL hardware does not allow MFCLK to run when MCLKCFG.MDIV != 0.
2. When using MFCLK, if MCLK is sourced from a source other than SYSOSC then the MCLK frequency must be ≥32 MHz for correct operation of MFCLK.
3. When using MFCLK, if MCLK is to be sourced from the high-speed clock (HSCLK), application software must enable MFCLK by setting the USEMFTICK bit before switching the MCLK source from SYSOSC to HSCLK. Do not change the state of USEMFTICK when MCLK is sourced from HSCLK.
4. When using MFCLK, do not enable MFCLK when MCLK is sourced from the low-frequency clock (LFCLK). Application software must enable MFCLK by setting the USEMFTICK bit before switching the MCLK source from SYSOSC to LFCLK. Software can switch MCLK to LFCLK after USEMFTICK is set. In this case, MFCLK halts when MCLK is sourced from LFCLK, and it resumes when MCLK is switched back to SYSOSC.
5. When MFCLK is enabled by setting USEMFTICK in the MCLKCFG register, it is considered by the hardware as a static policy. Do not clear USEMFTICK.

When MFCLK is configured to be enabled, it is only active when SYSOSC is active and MCLK is not sourced from LFCLK. When MCLK is sourced from LFCLK, MFCLK is stopped by hardware automatically. Note that if the device is in STANDBY, MCLK is always sourced from LFCLK and MFCLK is always disabled by hardware.

[Asynchronous fast clock requests](#), if configured, temporarily enable SYSOSC to handle specific peripheral events and activity. If MFCLK is configured to be enabled (USEMFTICK is set), then MFTICK runs when a peripheral asserts an asynchronous fast clock request.

2.3.2.5 MFPCLK (Middle Frequency Precision Clock)

MFPCLK provides a continuous 4-MHz clock to the external clock output ([CLK_OUT](#)) mux and 12-bit DAC modules in RUN, SLEEP, or STOP mode. Unlike [MFCLK](#), which also produces a continuous 4-MHz clock for use by most peripherals, MFPCLK is asynchronous to MCLK/ULPCLK and it can be sourced from either the SYSOSC or from HFCLK (HFXT or HFCLK_IN) to provide higher precision to the DACs, improving DAC performance.

The 12-bit DAC modules do not have a clock selection mux. The 12-bit DAC clock source is selected by configuring MFPCLK. To select HFCLK (HFXT or HFCLK_IN) to source the MFPCLK, set the MFPCLKSRC in the GENCLKCFG register in SYSCTL.

To enable the MFPCLK, set the MFPCLKEN bit in the GENCLKEN register in SYSCTL.

Note

HFCLK is disabled in STOP mode by SYSCTL automatically when entering STOP mode. If the 12-bit DACs are to be used in STOP mode, then SYSOSC must be the clock source for MFPCLK.

2.3.2.6 LFCLK (Low-Frequency Clock)

LFCLK provides a continuous 32-kHz clock to a variety of peripherals on the device. After a BOOTRST, LFCLK is initially sourced by the internal 32-kHz oscillator (LFOSC). After boot, LFCLK can be switched by software to either the low-frequency crystal oscillator (LFXT) or the low-frequency digital clock input (LFCLK_IN). See the respective oscillator section for instructions on switching the LFCLK source. When the LFCLK source is changed, the change is locked and LFOSC is disabled to save power. It is not possible to again select LFOSC as the LFCLK source without executing a BOOTRST.

Note

If LFCLK is to be selected as the MCLK source, and LFXT or LFCLK_IN are to be used as the LFCLK source, configure the LFXT or LFCLK_IN as the LFCLK source before selecting LFCLK as the source for MCLK.

LFCLK is active in RUN, SLEEP, STOP, and STANDBY power modes. It is possible to disable both ULPClk and LFCLK together to most peripherals in STANDBY mode to achieve the lowest possible STANDBY mode power consumption (STANDBY1). To do so, set the STOPCLKSTBY bit in the MCLKCFG register in SYSCTL before entering STANDBY. In this state, the RTC and TIMG0 and TIMG1 are the only clocked peripherals.

LFCLK is a synchronized clock. All LFCLK edges are synchronized to the main system clocks (MCLK and ULPClk), meaning that the registers of peripherals clocked by LFCLK can be read or written to at any time without any special handling. The RTC is the only exception to this rule, as the RTC receives RTCClk which is the asynchronous LFCLK.

Note

When MCLK/ULPClk are not sourced by LFCLK (for example, when they are sourced by SYSOSC) there is a 5 ULPClk cycle synchronization delay between the low frequency clock source's clock edge and the corresponding LFCLK edge as seen by peripherals running from LFCLK. When the MCLK/ULPClk frequency is constant, this delay is constant and it does not add jitter to LFCLK. If the MCLK/ULPClk frequency changes, the synchronization delay changes proportionally and this results in a small single-cycle LFCLK jitter at the MCLK/ULPClk frequency transition point. This jitter changes the duty cycle of one LFCLK period, but there is no accumulation of error (there is never a change in the number of LFCLK periods, ensuring an accurate LFCLK time base for peripherals). A common case would be MCLK/ULPClk sourced from SYSOSC at 32MHz, with SYSOSC occasionally switching to 4MHz to reduce power consumption. When SYSOSC (and, by definition, MCLK/ULPClk) switch from 32MHz to 4MHz, the synchronization delay increases from 0.16µs to 1.25µs (an increase of 1.1µs). Thus, during the SYSOSC frequency transition, one LFCLK period will appear 1.1µs (3.6%) longer than normal. When SYSOSC switches back from 4MHz to 32MHz, one LFCLK period will appear 1.1µs shorter than normal, and the phase shift is recovered and error is not accumulated in the long term.

Peripherals can select LFCLK as their functional clock source through their respective CLKSEL mux. Not all peripherals support running from LFCLK. It is possible to run the main clock (MCLK) from LFCLK, in which case the entire device runs at the LFCLK rate (32 kHz).

2.3.2.7 HFCLK (High-Frequency External Clock)

The high-frequency external clock (HFCLK) is the output of the high-frequency external clock selection mux, and can be selected as either the high-frequency oscillator (HFXT) output or the high-frequency digital clock input (HFCLK_IN).

The HFCLK can be used as the source for the following clocks:

- [HSCLK](#) source (to source MCLK)
- [SYSPLL](#) reference clock
- [CANCLK](#) source
- [ADCCLK](#) source
- [MFPCLK](#) source (with optional divider)

The HFCLK is only available in RUN and SLEEP modes. HFCLK is automatically disabled by SYSCTL in all other modes, along with the HFXT itself (if enabled).

2.3.2.8 HSCLK (High Speed Clock)

The high-speed clock (HSCLK) is the output of the high-speed clock selection mux, and can be selected to source MCLK by setting the USEHSCLK bit in the MCLKCFG register. The HSCLK is only a selection option for MCLK; it does not source any other functions. HSCLK can be configured to be sourced from the [SYSPLL](#) (SYSPLLCLK0 or SYSPLLCLK2X) or the HFCLK ([HFXT](#) or [HFCLK_IN](#)). By default, the HSCLK is sourced from the SYSPLL. To change the HSCLK source to HFCLK, set the HSCLKSEL bit in the HSCLKCFG register.

The HSCLK is only available in RUN and SLEEP modes. It is automatically disabled by SYSCTL in all other modes, along with the SYSPLL and HFXT (if enabled).

SYSCTL will not switch MCLK to HSCLK, even if requested by software, if the [HSCLK status](#) check indicates that selected HSCLK source was not started correctly.

2.3.2.9 ADCCLK (ADC Sample Period Clock)

ADCCLK is used by the ADC module to set the ADC sampling period. The ADCCLK for a given ADC is provided from the CKM to the ADC, but the ADCCLK clock selection is done within each ADC peripheral's configuration registers. See the ADC chapter for information on configuring the ADCCLK. ADCCLK can be selected as ULPCLK, SYSOSC, or HFCLK (HFXT or HFCLK_IN).

Note

On devices which contain more than one ADC, a unique ADCCLK is mapped to each ADC, making it possible for ADCs to have different sampling clocks.

2.3.2.10 CANCLK (CAN-FD Functional Clock)

The CANCLK is a functional clock provided directly to the CAN-FD module from either the HFCLK (HFXT or HFCLK_IN) or the SYSPLL (SYSPLLCLK1). This clock is provided as a functional clock to the CAN-FD module asynchronous from the main clock (MCLK) for the highest possible accuracy. The CANCLK source is selected in SYSCTL by configuring the CANCLKSRC bit in the GENCLKCFG register. Additional CAN-FD clock configuration is provided within the CAN-FD peripheral itself (review the CAN-FD chapter for more detail).

2.3.2.11 RTCCLK (RTC Clock)

RTCCLK is the sole clock source for the real-time clock peripheral. Because of the critical time-keeping nature of the RTC, the RTC is provided LFCLK directly before LFCLK is synchronized to the main clock (MCLK). This also enables the RTC to continue counting even when the system clock tree is completely gated in STANDBY mode to reduce power consumption. No special configuration is required for RTCCLK. Because the RTC logic is clocked asynchronously from the system main clock, special rules apply when reading certain RTC registers. See the RTC chapter to understand the rules for handling RTC registers.

2.3.2.12 External Clock Output (CLK_OUT)

A clock output unit is provided for sending digital clock signals from the device to external circuits or to the [frequency clock counter](#). This feature is useful for clocking external circuitry such as an external ADC that does not have a clock source. The clock output unit has a flexible set of sources to select and includes a programmable divider.

Available clock sources for CLK_OUT:

- SYSPLLCLK1
- HFCLK

- SYSOSC
- ULPCLK
- MFPCLK
- LFCLK

The selected clock source can be divided by 1 (no divide), 2, 4, 6, 8, 10, 12, 14, or 16 before being output to the pin or to the frequency clock counter.

To use the clock output unit:

1. Configure IOMUX to select the CLK_OUT function on the device pin with CLK_OUT.
2. Select the desired clock source in the EXCLKSRC field of the GENCLKCFG register.
3. Set the desired clock divider, if necessary, in the EXCLKDIVVAL field of the GENCLKCFG register, and enable the divider by setting the EXCLKDIVEN bit.
4. Enable the external clock output by setting the EXCLKEN bit in the GENCLKEN register.

Note

When the CLK_OUT source is selected as ULPCLK or MFPCLK, the clock divider must be enabled (EXCLKDIVEN must be set).

Note

When clearing the EXCLKEN bit to disable CLK_OUT, allow the clock source to run for 10 clock cycles to stabilize the EXCLKSRC mux.

Note

When disabling a clock source which is selected for CLK_OUT, it is recommended to disable the CLK_OUT function before disabling the clock source if it is important that CLK_OUT be logic low (0) when the clock source is disabled. If CLK_OUT is left enabled and the source for CLK_OUT is disabled, it is possible that CLK_OUT may stop in a logic high (1) state.

Note

When the CLK_OUT source is selected as SYSPLLCLK1, the SYSPLLCLK1 output must be ≤ 48 MHz. Further speed restrictions can exist depending on the IO capabilities of a specific device and pin; see the Digital IO specifications in the device-specific data sheet for details on supported IO speeds.

2.3.2.13 Direct Clock Connections for Infrastructure

Several direct clock connections are made in the device to support specific analog functionality:

- **SYSOSC** to ADCs
- **SYSOSC** to OPAs, if OPAs are present on a device
- **SYSOSC** and **LFCLK** to PMU analog mux **VBOOST** circuit

Direct Connections to ADCs

In addition to receiving ADCCLK to set the sampling window, the ADC modules also receive the direct output of SYSOSC. The SYSOSC direct output to the ADCs is used by the charge pump logic in the ADC modules. SYSOSC can be configured at any frequency to support this function. The ADC supports requesting SYSOSC automatically before a conversion, so there is not a requirement for application software to ensure that SYSOSC is running before triggering an ADC conversion.

Direct Connections to OPAs

The SYSOSC direct clock connection to the OPAs, if present, enable all OPA operating modes to be used even when the peripheral bus clock (ULPCLK) is running at a lower frequency than what is required by the OPA. Certain OPA modes require that SYSOSC be running at 32 MHz. See the OPA section for clocking details. Configuration of SYSOSC to a frequency which supports the selected OPA mode is the responsibility of the application software. If the OPA is enabled and configured for a mode which requires SYSOSC to be running at

a certain frequency, and software has not configured SYSOSC to run at this frequency, an OPAMPCLKERR will be asserted in the CLKSTATUS register in SYSCTL and the OPA can not function properly.

Direct Connections to the PMU Analog Mux VBOOST Circuit

Both the SYSOSC and the LFCLK are connected directly to the [analog mux VBOOST circuit](#) in the PMU, which is used to boost the performance of the analog muxes used by the COMPs, OPAs, and GPAMP. The VBOOST circuit is enabled automatically when a peripheral requires it for correct operation. It is the responsibility of the application software to ensure that the SYSOSC or LFCLK are enabled to support correct operation of the VBOOST circuit. If the VBOOST circuit requires SYSOSC or LFCLK, and the clock is not available, an ANACKERR is asserted in the CLKSTATUS register in SYSCTL. SYSCTL can also be configured to raise a SYSCTL interrupt when ANACKERR is asserted, to alert the application that a COMP, OPA, or the GPAMP can not be functioning properly as expected.

2.3.3 Clock Tree

[Figure 2-8](#) shows the top level clocking tree for MSPM0Gxx family devices. This diagram shows the mapping between oscillators (sources) and clocks (destinations), as well as the SYSCTL register bit fields for the selection muxes. Note that not all devices have all clock system features shown in [Figure 2-8](#).

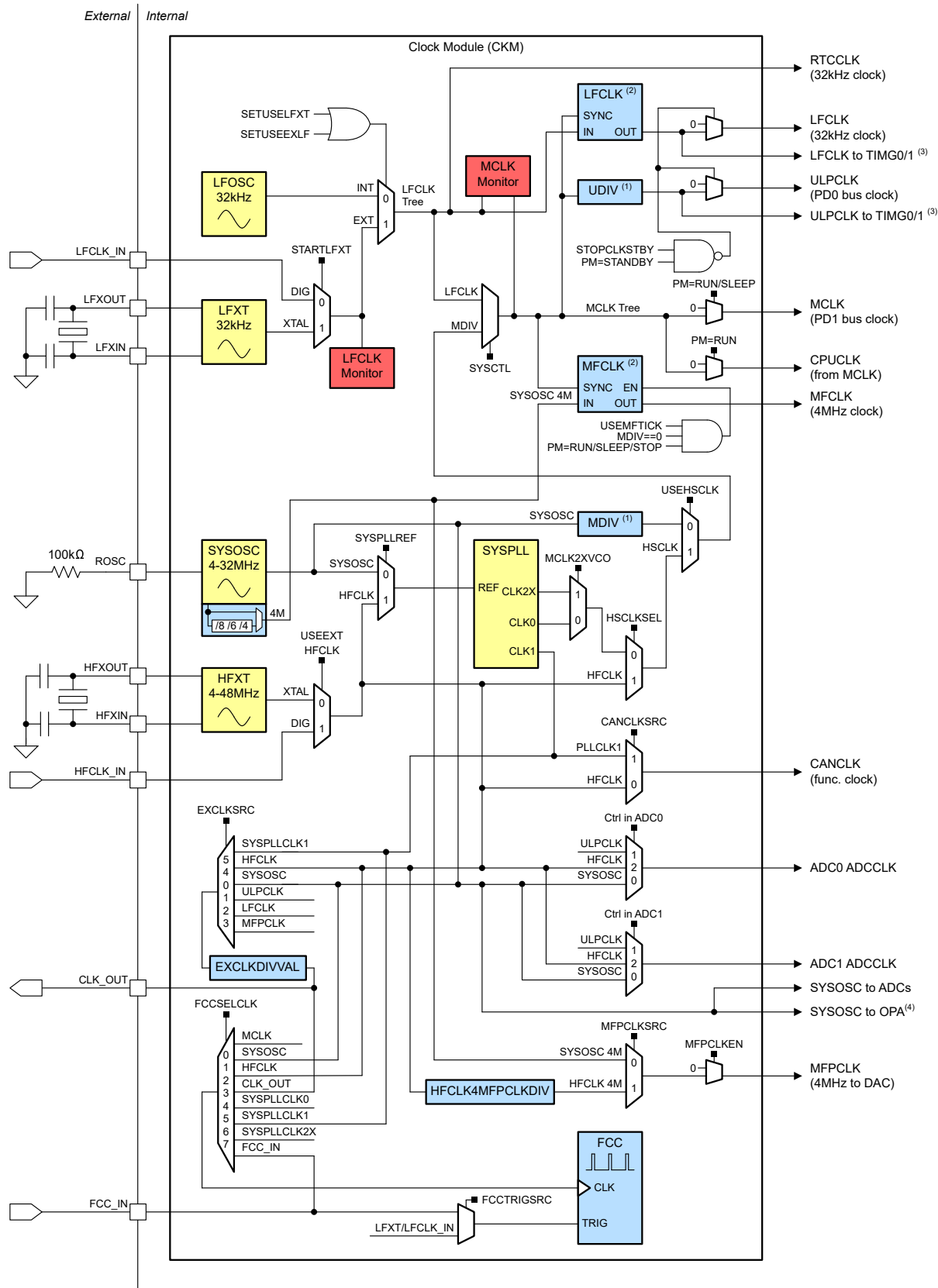


Figure 2-8. MSPM0Gxx Top Level Clocking Tree

1. MDIV and UDIV are clock dividers that support specific use-cases. Review the MCLK and ULPCLK sections, respectively, for specific design considerations regarding MDIV and UDIV.
2. LFCLK and MFCLK are fixed-frequency 32-kHz and 4-MHz clocks, respectively, that can be selected by certain peripherals for ensuring a constant clock rate even when MCLK or ULPCLK changes source or rate. LFCLK and MFCLK are always synchronized to each other and to MCLK and ULPCLK.
3. TIMG0 and TIMG1 (general purpose timers) receive an ungated LFCLK and ULPCLK, enabling them to continue operating even in STANDBY1 when STOPCLKSTBY is asserted to gate the LFCLK and ULPCLK to all other peripherals to save additional power in STANDBY mode.
4. SYSOSC is provided to the OPAs for use directly. The OPAs do not request SYSOSC automatically. It is the responsibility of application software to ensure that SYSOSC is enable and running at the necessary frequency to support correct OPA operation before enabling the OPA.

2.3.3.1 Peripheral Clock Source Selection

Most peripherals on the device contain an input clock selection mux which is used to select, and optionally divide down, the functional clock for the peripheral. Figure 2-9 shows the superset peripheral clock selection mux and optional clock divider. Note that not every peripheral has every clock source shown in Figure 2-9. For example, accelerators such as CRC, AES, and DMA run off of the bus clock. There is no option to select MFCLK or LFCLK for these peripherals. To determine the available clock sources for a peripheral, see the chapter for the specific peripheral and review the clock input selections.

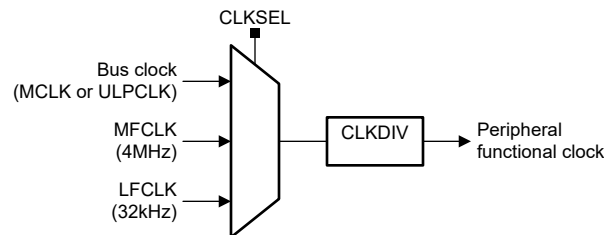


Figure 2-9. Peripheral Clock Selection Mux and Divider

Exceptions

There are also several peripherals that have a unique clock selection scheme and do not use the standard peripheral clock mux shown above. Typically this is due to a requirement for a peripheral to have a clock source that is asynchronous to the rest of the system. Cases where this occurs include:

- The real-time clock (RTC), where RTCCLK is used directly and there is not a clock selection option
- An analog-to-digital converter (ADC), where ADCCLK has a special selection mux to take asynchronous clock sources directly (the ADC sampling clock is selected with a special selection in the ADC control registers)
- The CAN-FD controller, where the CANCLK can be the a PLL output or the HFCLK directly (the CAN-FD functional clock source is selected in SYSCTL)
- The DAC, where MFPCLK is used directly and there is not a clock selection option

2.3.4 Clock Monitors

Several hardware clock monitors are provided to ensure that the CKM is functioning properly. Clock faults are processed through SYSCTL and result in either a brownout reset (in the event of a fatal fault) or a SYSCTL interrupt.

2.3.4.1 LFCLK Monitor

A low-power analog continuously operating clock monitor is provided to ensure that LFCLK is running when it is not sourced internally (for example, when LFCLK is sourced from [LFXT](#) or [LFCLK_IN](#) and not from LFOSC). The LFCLK monitor is only intended to check for clock stuck faults. It is not intended to be used to verify that the frequency of LFCLK is within a specific tolerance.

Enabling the LFCLK Monitor

The LFCLK monitor is disabled at startup after a BOOTRST, unless the cause of the BOOTRST was the NRST pin.

If the STARTLFXT bit in the LFXCTL register is set, the LFCLK monitor monitors the LFXT. If the STARTLFXT bit is left cleared, the LFCLK monitor monitors the LFCLK_IN.

Before enabling the LFCLK monitor, make sure that the source clock has started and is operating at 32 kHz. If STARTLFXT is set and the monitor is checking the LFXT, wait for the LFXGOOD indication before enabling the LFCLK monitor. If STARTLFXT is cleared and the monitor is checking the LFCLK_IN digital clock input, make sure that the external clock signal is active before enabling the LFCLK monitor. If a valid clock is not present when starting the monitor, the monitor will assert a fault.

The LFCLK continuous monitor can be enabled by setting the MONITOR bit in LFCLKCFG register.

Note

The LFCLK monitor requires 100 μ s to become active after being enabled. The LFCLK monitor can be enabled to monitor LFXT or LFCLK_IN before switching the LFCLK source from the internal LFOSC.

LFCLK Monitor Fault Handling

If an LFCLK stuck fault is detected, the system responds in one of two ways depending on the system clock configuration:

- If LFCLK is the source for MCLK (for example, if the entire system is running at 32 kHz derived from LFCLK), then a LFCLK fault is considered fatal and a BOOTRST is asserted. Following the BOOTRST, MCLK will be sourced from SYSOSC at BASE frequency, and LFCLK will be sourced from LFOSC.
- If LFCLK is not the source for MCLK at the time of the failure or after the failure, an LFCLK fault asserts a non-maskable interrupt (NMI) to the processor so that the application can immediately handle the fault and take any necessary action.

Falling Back to LFOSC After LFXT Failure

In the event that an LFXT or LFCLK_IN failure is detected and the failure is due to a system level / PCB level issue that is preventing the external clock source from being able to run reliably, it is possible to fall back to the internal 32kHz LFOSC rather than continuously trying to use the LFXT or LFCLK_IN. In this case, the following procedure is recommended:

1. Configure the device with LFXT or LFCLK_IN sourcing LFCLK and with the LFCLK monitor enabled, per the procedure given above in **Enabling the LFCLK Monitor**.
2. In the event of an LFCLK monitor fault detection:
 - a. If MCLK is sourced from LFCLK, the failure will generate a BOOTRST automatically.
 - b. If MCLK is not sourced from LFCLK, an NMI will be generated. In the NMI handling, application software can switch MCLK to LFCLK when the LFCLK fault is detected. This action will trigger the hardware to generate a BOOTRST automatically. Switching the MCLK source to LFCLK may be done by changing the operating mode from RUN0 to RUN1/RUN2 or STOP2/STANDBY.
3. Following the BOOTRST, MCLK will be sourced from SYSOSC at BASE frequency, and LFCLK will be sourced from LFOSC. Application software can check the cause of the reset was a fatal clock fault and decide what action to take.
 - a. Application software may attempt again to use LFXT/LFCLK_IN. If the LFCLK fails again, application software can return to step 2 above to again switch LFCLK back to LFOSC.
 - b. To track the number of LFCLK failures, application software can store diagnostic information in the [shutdown memory](#) of the device. The SHUTDNSTOREx memory locations are retained through a BOOTRST. As such, it is possible for the application software to track how many LFCLK failures have occurred, and if that number exceeded some threshold, application software can elect to remain with LFOSC as the LFCLK source rather than again attempting to use the LFXT/LFCLK_IN.

2.3.4.2 MCLK Monitor

A digital clock monitor can be used with MCLK. The MCLK monitor asserts an MCLK fault if there is no MCLK activity for a period of 1-12 LFCLK cycles. An MCLK fault is always considered fatal to the system and generates a BOOTRST.

The MCLK monitor can be enabled after the LFCLK is configured and running. To enable the MCLK monitor, set the MCLKDEADCHK bit in the MCLKCFG register in SYSCTL. When enabled, the MCLK monitor runs in all operating modes except for STANDBY1 and SHUTDOWN.

2.3.4.3 Startup Monitors

Clock startup monitors are provided for application software to check that the LFOSC, LFXT/LFCLK_IN, HFXT/HFCLK_IN, SYSPLL, and HCLK sources are alive before they are selected by software to be used to source a clock in the system. When a clock source has started successfully and is ready, a GOOD indication is given in the CLKSTATUS register in SYSCTL and an interrupt is generated. The startup monitors only provide a status indication when a related clock system configuration change is made. When an initial GOOD indication is given, the clock is not continuously monitored by the startup monitor. Continuous monitoring is provided for [LFCLK](#) and [MCLK](#).

2.3.4.3.1 LFOSC Startup Monitor

The LFOSC is started automatically after a BOOTRST. The LFOSC takes some time to start. A startup monitor is provided to indicate to the application software when LFOSC startup has completed, at which time the LFCLK is available for use by peripherals. When LFOSC startup has completed, the LFOSCGOOD bit in the CLKSTATUS register in SYSCTL and the LFOSCGOOD interrupt is asserted to alert the application. See the device-specific data sheet for the LFOSC startup time.

2.3.4.3.2 LFXT Startup Monitor

The [LFXT](#) takes time to start after being enabled. A startup monitor is provided to indicate to the application software that the LFXT has successfully started, at which point the LFXT can be selected as the LFCLK source. Once the LFXT startup has completed, the LFXT startup monitor will assert the LFXTGOOD bit in the CLKSTATUS register in SYSCTL and the LFXTGOOD interrupt will be asserted to alert the application. See the device-specific data sheet for the LFXT startup time.

2.3.4.3.3 HFCLK Startup Monitor

The [HFXT](#) takes time to start after being enabled. A startup monitor is provided to indicate to the application software if the HFXT has successfully started, at which point the [HFCLK](#) can be selected to source a variety of system functions. The HFCLK startup monitor also supports checking the [HFCLK_IN](#) digital clock input for a clock stuck fault.

To enable the HFCLK startup monitor, clear the HFCLKFLTCHK bit in the HFCLKCLKCFG register in SYSCTL (the default state is disabled).

When HFXT is started or the HFCLK_IN is selected as the HFCLK source, the HFCLKGOOD and HFCLKOFF bits in the CLKSTATUS register in SYSCTL are cleared.

In the case of HFXT being used, after the specified HFXT startup time has expired the HFXT status is tested. If the HFXT started successfully, the HFXT startup monitor asserts the HFCLKGOOD bit in the CLKSTATUS register and the HFCLKGOOD interrupt is also asserted. If the HFXT did not start within the specified startup time, the HFCLKOFF bit is set, indicating that the HFXT was dead at startup.

In the case of HFCLK_IN being used, after HFCLK_IN is selected a clock stuck check is performed. If the clock is alive, the HFCLKGOOD bit is set in the CLKSTATUS register and the HFCLKGOOD interrupt is also asserted. If the HFCLK_IN signal was stuck, the HFCLKOFF bit in CLKSTATUS register is set, indicating that the HFCLK_IN is dead.

If desired, checking of the HFCLK by the HFCLK startup monitor can be left disabled by keeping the HFCLKFLTCHK bit set in the HFCLKCLKCFG register in SYSCTL.

Note

The HFCLK must be in a stable state before attempting to enter STOP or STANDBY low power modes. Before entering STOP or STANDBY, make sure that either the HFCLKGOOD bit or the HFCLKOFF bit is set.

2.3.4.3.4 SYSPLL Startup Monitor

The **SYSPLL** takes time to start and settle after being enabled. A startup monitor is provided to indicate to the application software if the SYSPLL has successfully started, at which point the clock outputs from the SYSPLL can be selected to source a variety of system functions.

When the SYSPLL is started, the SYSPLLGOOD and SYSPLLOFF bits in the CLKSTATUS register in SYSTCTL are cleared. After the startup/settling time has expired, the SYSPLL status is tested. If the SYSPLL started successfully, the SYSPLL startup monitor will assert the SYSPLLGOOD bit in the CLKSTATUS register and the SYSPLLGOOD interrupt will also be asserted. If the SYSPLL did not start within the specified time, the SYSPLLOFF bit will be set, indicating that the SYSPLL was dead at startup.

Note

The SYSPLL must be in a stable state before attempting to enter STOP or STANDBY low power modes. Before entering STOP or STANDBY, make sure that either the SYSPLLGOOD bit or the SYSPLLOFF bit is set.

2.3.4.3.5 HSCLK Status

The **HSCLK** can be sourced from either the HFCLK or the SYSPLL. The CLKSTATUS register in SYSTCTL provides HSCLKGOOD and HSCLKDEAD indications, which indicate if the selected HSCLK source started successfully with a GOOD status or failed with a DEAD status, respectively. SYSTCTL will not switch MCLK over to HSCLK, even if requested, if HSCLKGOOD is not set.

In addition, the HSCLKSOFF bit is provided in CLKSTATUS to indicate if all HSCLK sources (SYSPLL, HFCLK) are either off (disabled) or started with a DEAD status.

2.3.5 Frequency Clock Counter (FCC)

The frequency clock counter (FCC) enables flexible in-system testing and calibration of a variety of oscillators and clocks on the device. The FCC counts the number of clock periods seen on the selected source clock within a known fixed trigger period (derived from a secondary reference source) to provide an estimation of the frequency of the source clock.

Application software can use the FCC to measure the frequency of the following source oscillators and clocks (selected through the FCCSELCLK field in the GENCLKCFG register):

- MCLK
- SYSOSC
- HFCLK
- CLK_OUT
- SYSPLL (any of the three SYSPLL outputs)
- The external FCC input (FCC_IN)

The reference clock used to set the trigger time over which pulses of the source clock are counted is configurable (through the FCCTRIGSRC field in the GENCLKCFG register), and can be driven by:

- The external FCC input (FCC_IN)
- LFCLK
- The output of the mux of LFOSC, LFXT, or LFCLK_IN (32 kHz)

The trigger time period can be set in one of two ways (through the FCCLVLTRIG field in the GENCLKCFG register):

- Level triggered (one rising edge to one falling edge of the reference clock input). Please note that LFCLK_IN cannot be used as a trigger clock source if using level triggering.
- Rising-edge to rising-edge triggered, for a defined number of clock periods of the reference clock (selectable from 1 to 32 through the FCCTRICNT field in the GENCLKCFG register)

When the trigger source is selected as the external FCC input in level-triggered mode, a user-specified counting period can be set by applying a logic high pulse on the FCC_IN pin of the desired trigger length.

When the trigger source is selected as the LFXT, using rising-edge to rising-edge triggering will cause the FCC to capture the number of source clock pulses which occurred within 1 to 32 32.768-kHz clock periods of the LFXT (30.5 μ s).

The FCC counter is 22 bits and supports counting from 0 up to $2^{22} - 1$ or 4 194 303.

While the external FCC input (FCC_IN function) can be used as either the FCC clock source or the FCC trigger input, it cannot be used for both functions during the same FCC capture. It must be configured as either the FCC clock source or the FCC trigger.

2.3.5.1 Using the FCC

Rising-Edge to Rising-Edge Triggered Mode with FCC_IN Trigger

The following steps describe how to use the FCC to count the number of source clock pulses within the trigger period set by the reference clock, with the FCC_IN pin being selected as the reference clock and the SYSOSC being selected as the source clock. This example would be useful for calibrating the SYSOSC frequency with respect to an accurate clock source provided to the FCC_IN pin externally.

1. Set the source clock to SYSOSC by configuring the FCCSELCLK field in the GENCLKCFG register.
2. Set the reference clock to FCC_IN by clearing the FCCTRIGSRC bit in the GENCLKCFG register.
3. Select rising-edge to rising-edge triggering by clearing the FCCLVLTRIG bit in the GENCLKCFG register.
4. Select the desired number of reference clock periods to count the source clock over in the FCCTRICNT field in the GENCLKCFG register.
5. Ensure that **SYSOSC** is enabled at the desired frequency, and that the external clock source connected to FCC_IN is running correctly before continuing.
6. Write the GO bit and KEY field to the FCCCMD register to start the FCC capture on the next trigger clock period.
7. Poll the FCCDONE status bit in the CLKSTATUS register. When the capture completes, FCCDONE will be set by hardware. FCCDONE is read-only and is automatically cleared by hardware when a new capture is started.
8. Extract the resulting count from the 22-bit DATA field in the FCC register.

Rising-Edge to Rising-Edge Triggered Mode with LFXT Trigger

The following steps describe how to use the FCC to count the number of source clock pulses within the reference clock period, with the LFXT being selected as the reference clock and the SYSOSC being selected as the source clock. This example would be useful for calibrating the SYSOSC frequency with respect to an accurate 32.768kHz watch crystal.

1. Set the source clock to SYSOSC by configuring the FCCSELCLK field in the GENCLKCFG register.
2. Set the reference clock to LFXT by setting the FCCTRIGSRC bit in the GENCLKCFG register.
3. Select rising-edge to rising-edge triggering by clearing the FCCLVLTRIG bit in the GENCLKCFG register.
4. Select the desired number of reference clock periods to count the source clock over in the FCCTRICNT field in the GENCLKCFG register.
5. Ensure that **SYSOSC** is enabled at the desired frequency, and that the **LFXT** is running correctly before continuing.
6. Write the GO bit and KEY field to the FCCCMD register to start the FCC capture on the next trigger clock period.

7. Poll the FCCDONE status bit in the CLKSTATUS register. When the capture completes, FCCDONE will be set by hardware. FCCDONE is read-only and is automatically cleared by hardware when a new capture is started.
8. Extract the resulting count from the 22-bit DATA field in the FCC register. If SYSOSC was running at 32 MHz and FCCTRIGCNT was set to '0' (one reference clock period), the result should be approximately 976 cycles counted within the single 32.768kHz period.
 - a. To calibrate SYSOSC for 24 MHz operation, the SYSOSC user trim must be adjusted until approximately 732 cycles are counted.
 - b. To calibrate SYSOSC for 16 MHz operation, the SYSOSC user trim must be adjusted until approximately 488 cycles are counted.

In general, increasing the FCCTRIGCNT value increases the accuracy of the measurement, at the expense of longer measurement time.

Level Triggered Mode with FCC_IN Trigger and HFCLK_IN Clock

The following steps describe how to use the FCC to count the number of source clock pulses within one external reference pulse window, with HFCLK_IN being selected as the source clock. This example would be useful for measuring the frequency of an external clock source with respect to a fixed pulse width driven by an external signal.

1. Set the source clock to HFCLK by configuring the FCCSELCLK field in the GENCLKCFG register.
2. Set the trigger clock to the FCC_IN pin function by clearing the FCCTRIGSRC bit in the GENCLKCFG register.
3. Set level triggering by setting the FCCLVLTRIG bit in the GENCLKCFG register.
4. Ensure that IOMUX is configured for FCC_IN, that HFCLK is configured for HFCLK_IN, and that an external clock is sourcing HFCLK_IN.
5. Write the GO bit and KEY field to the FCCCMD register to start the FCC capture when FCC_IN goes logic high. Note that if FCC_IN is already logic high when GO is asserted, counting starts immediately. When using level mode, FCC_IN should be low when GO is set, and the trigger pulse should be sent to FCC_IN after GO is set.
6. Poll the FCCDONE status bit in the CLKSTATUS register. When the capture completes, FCCDONE will be set by hardware. FCCDONE is read-only and is automatically cleared by hardware when a new capture is started.
7. Extract the resulting count from the 22-bit DATA field in the FCC register.

2.3.5.2 FCC Frequency Computation and Accuracy

The frequency of the source clock can be computed after capture if the trigger time is known. The frequency is computed by dividing the number of source clock cycles captured by the trigger time. For example, if the trigger source was a 32.768kHz clock, the trigger mode was rising-edge to rising-edge, and the period count was 1, then the trigger time is one 32.768-kHz clock period (30.5 μ s). If the captured source count were to come back as 122, the frequency of the source clock is computed as 122 divided by 30.5 μ s, giving a source clock frequency of approximately 3.99MHz.

$$f_{\text{source}} = \text{FCC.DATA} / ((\text{GENCLKCFG.FCCTRIGCNT} + 1) / f_{\text{ref}}) \quad (15)$$

The FCC accuracy is dependent on the trigger clock accuracy as well as the total number of clock cycles captured. The FCC intrinsic error is ≤ 2 source clock cycles per capture due to synchronization of the trigger to the source clock. Therefore, the impact of these two clock cycles is reduced as more cycles are counted (as the trigger time is increased and/or the source clock frequency is increased). Approximate intrinsic error of the FCC for various source clock frequencies captured against one 32.678kHz period (FCCTRIGCNT=0) and 32 clock periods (FCCTRIGCNT=31) are given in [Table 2-11](#).

Table 2-11. FCC Error

Use Case (Source Clock Frequency)	FCC Trigger Time	FCC Count Result	FCC Count Uncertainty	Approximate FCC Intrinsic Uncertainty Error
4-MHz source clock	30.5μs	122	2 cycles	1.6%
	976.6μs	3906		0.05%
16-MHz source clock	30.5μs	488		0.4%
	976.6μs	15625		0.01%
24-MHz source clock	30.5μs	732		0.27%
	976.6μs	23437		0.01%
32-MHz source clock	30.5μs	976		0.20%
	976.6μs	31250		0.01%
80-MHz source clock	30.5μs	2441		0.08%
	976.6μs	78125		0.003%

Note

When using the FCC_IN signal, it is recommended to have a fast slew rate of 10ns or less on the FCC_IN pin to minimize measurement uncertainty.

2.4 System Controller (SYSCTL)

The system controller (SYSCTL) contains all control logic for managing the configuration and state of the PMU and CKM analog circuitry. SYSCTL also provides reset management, control over NRST and SWD pin muxing, flash bank swap control, and flash ECC error handling.

All power, clock, and reset configuration is done through the SYSCTL memory-mapped register interface.

2.4.1 Resets and Device Initialization

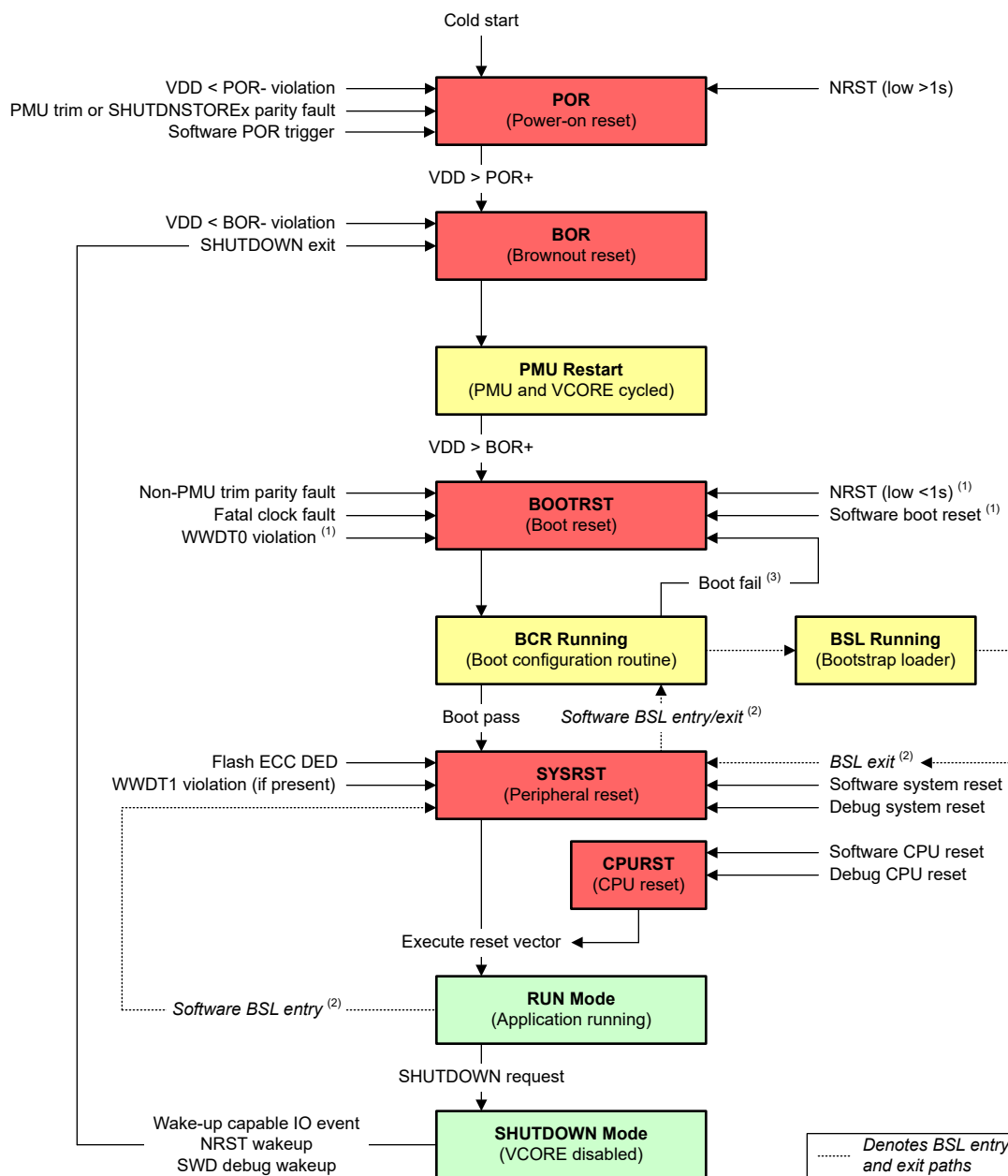
The SYSCTL manages device reset levels and device initialization.

2.4.1.1 Reset Levels

The device has five reset levels:

1. Power-on reset ([POR](#))
2. Brownout reset ([BOR](#))
3. Boot reset ([BOOTRST](#))
4. System reset ([SYSRST](#))
5. CPU reset ([CPURST](#))

The relationships between reset levels are given in [Figure 2-10](#).


Figure 2-10. MSPM0 Reset Levels

(1) An NRST (low <1 s), software boot reset, or WWD0 violation triggered BOOTRST runs the boot configuration routine but does not reset the RTC, LFXT, LFCLK, LFCLK_IN, and IOMUX configuration of any IO pins used by LFXT or LFCLK_IN. This allows the RTC to keep time through an external reset trigger.

(2) A software-triggered bootstrap loader (BSL) entry command first triggers a SYSRST, after which it runs the boot configuration routine (BCR) to authenticate the BSL entry before starting the BSL. After the BSL execution concludes, a SYSRST is generated and the BCR executes again. When the BCR completes, a final SYSRST is issued and the application is started. This entire process does not reset the RTC, LFXT/LFCLK/LFCLK_IN, the IOMUX configuration of any IO pins used by LFXT or LFCLK_IN, and SYSOSC FCL enable configurations, as only a SYSRST reset level is asserted throughout the process. This allows the RTC to keep time through an external reset trigger.

(3) If a boot fail occurs during execution of the boot configuration routine, a BOOTRST can be generated by SYCTL to attempt the boot process again from the BOOTRST level. See [Section 2.4.1.8](#).

Note

SLEEP, STOP, and STANDBY operating modes are not shown in this diagram. These modes originate from and return to RUN mode unless an exception occurs which causes a reset level to be asserted or a mode to be suspended.

2.4.1.1.1 Power-on Reset (POR) Reset Level

A power-on reset (POR) is a complete device reset.

The following conditions generate a POR:

- The device powers up (cold start)
- A [POR- supply monitor violation](#) (VDD drops below the POR supply monitor negative-going threshold)
- A parity fault on PMU trim data or the [shutdown memory](#)
- Software triggers a [POR through SYCTL](#) (*RESETLEVEL 0x03*)
- The NRST pin is held low for more than one second when in NRST mode

A POR always resets the [shutdown memory](#), reenables the NRST/SWD pin functions (if disabled), and triggers a BOR.

2.4.1.1.2 Brownout Reset (BOR) Reset Level

A brownout reset (BOR) resets the device power management unit (PMU). All regulated core logic powered from V_{CORE} is power cycled.

The following conditions generate a BOR:

- A [POR](#)
- A [BOR0- supply monitor violation](#) (VDD drops below the BOR0- supply monitor negative-going threshold)
- An exit from shutdown mode (through a wakeup-capable IO, NRST, or SWD)

The following are not reset by a BOR:

- The shutdown memory (SHUTDNSTOREx)
- The NRST state, if disabled by software
- The SWD state, if disabled by software
- The latched IO pin state, if the cause of the BOR is an exit from SHUTDOWN mode (see [SHUTDOWN mode handling](#))

A BOR always triggers a [BOOTRST](#) when VDD > BOR0+.

2.4.1.1.3 Boot Reset (BOOTRST) Reset Level

A boot reset (BOOTRST) triggers execution of the device [boot configuration routine](#) and resets the majority of the core logic, including the RTC, LFCLK, LFXT, and LFCLK_IN configuration and the IOMUX configuration of any IO pins used by LFXT or LFCLKIN (unless the BOOTRST was caused by NRST or a software triggered BOOTRST), and the [SYSOSC FCL mode](#) (if enabled). The system memory (SRAM) is also power cycled and SRAM contents are lost.

The following conditions generate a BOOTRST:

- A [BOR](#)
- A parity fault on non-PMU trim data
- A fatal clock failure (see [LFCLK Monitor](#) and [MCLK Monitor](#))
- A WWDTO violation
- Software triggers a [BOOTRST through SYCTL](#) (*RESETLEVEL 0x01*)
- The NRST pin is held low for longer than the minimum reset pulse time but less than one second when in NRST mode
- A BOOTRST followed by a boot failure (re-attempt of a failed boot sequence)

The following are not reset by a BOOTRST:

- The shutdown memory (SHUTDNSTOREx)
- The [NRST disable state](#), if disabled by software
- The [SWD disable state](#), if disabled by software
- The RTC, LFCLK, and LFXT and LFCLK_IN configuration, including the IOMUX settings for any IO pins used by LFXT and LFCLK_IN, **if the cause of the BOOTRST was NRST, a WWDT0 violation, or a software triggered BOOTRST**. As a result, if LFCLK is configured to run from the LFXT or LFCLK_IN, an NRST pin BOOTRST condition or a software triggered BOOTRST condition does not reset LFCLK to the default internal oscillator (LFOSC). LFCLK continues to run from LFXT or LFCLK_IN and cannot be reconfigured. This lets the RTC maintain a time base through an external reset. All other BOOTRST conditions trigger a reset of the RTC, LFCLK, the LFXT and LFCLK_IN state, and the IOMUX configuration of any IO pins used by LFXT or LFCLK_IN.

Following a BOOTRST, a [SYSRST](#) is always triggered if the boot configuration routine completes successfully. If the boot configuration routine fails to complete successfully, a BOOTRST is again generated and the boot process is attempted again from the BOOTRST point. The boot process attempts to complete successfully up to 3 times, after which the device state locks until a BOR or POR reset occurs (see [Section 2.4.1.8](#)).

2.4.1.1.4 System Reset (SYSRST) Reset Level

A system reset clears the state of the CPU and all the peripherals, with the exceptions listed below.

The following conditions generate a SYSRST:

- A [BOOTRST](#) followed by a boot pass
- A bootstrap loader (BSL) exit, which is always followed by execution of the boot configuration routine (BCR)
- A flash ECC uncorrectable (DED) error (if present)
- A WWDT1 violation (if present)
- A CPU lockup violation
- Software triggers a [SYSRST through SYSCTL \(RESETLEVEL 0x00\)](#)
- Software triggers a [SYSRST with BSL entry through SYSCTL \(RESETLEVEL 0x02\)](#)
- The debug subsystem triggers a system reset

The following are not reset by a SYSRST:

- The shutdown memory (SHUTDNSTOREx)
- The [NRST state](#), if disabled by software
- The [SWD state](#), if disabled by software
- The RTC, LFCLK, and LFXT/LFCLK_IN configuration, including the IOMUX settings for any IO pins configured to be used by LFXT/LFCLK_IN
- The SYSOSC frequency correction loop ([FCL](#)), if enabled by software

In most cases, the device is in RUN mode after a SYSRST, and the CPU executes the reset vector and begins execution of the application software. There are two exceptions to this:

- If the SYSRST was triggered with a BSL entry request, the BCR runs followed by the BSL.
- If the SYSRST was triggered due to a BSL exit request, the BCR runs, followed by an additional SYSRST, after which the application software starts.

2.4.1.1.5 CPU-only Reset (CPURST) Reset Level

A CPU-only reset clears the state of the CPU logic only. Peripheral states are not affected by a CPU reset. A CPU reset is only generated by software through the CPU AIRCR local register or by the debug subsystem.

2.4.1.2 Initial Conditions After POR

After a POR, when the boot process completes and the CPU starts the application, the initial device conditions are as follows:

- The NRST pin is configured in NRST mode

- Serial wire debug (SWD) IO are in SWD mode
- All other configurable I/O pins are high impedance (Hi-Z)
- Peripheral modules are reset as described in their respective chapters of this manual
- The device is in RUN mode
- MCLK is sourced from the internal SYSOSC at base frequency (32 MHz)
- LFCLK is sourced from the internal LFOSC (note that LFOSC requires time to start up before LFCLK can be used)
- MFCLK is disabled
- MFPCLK is disabled
- HFXT and SYSPLL are disabled
- Peripherals are disabled
- Any flash sectors configured to be write protected at boot are write protected

2.4.1.3 NRST Pin

After a cold start, the NRST pin is configured in NRST mode. The NRST pin must be high for the device to boot successfully. There is no internal pullup resistor on NRST. External circuitry (either a pullup resistor to VDD or a reset control circuit) must actively pull NRST high for the device to start. After the device is started, a low pulse on NRST <1 second in duration triggers a BOOTRST. If a low pulse on NRST is held for >1 second, a POR is triggered.

Some low pin count devices support reconfiguring the NRST pin to be a GPIO pin. See the pin configuration of the device-specific data sheet to see if GPIO functionality is shared with NRST. Application software can disable the NRST functionality of the NRST pin, allowing GPIO functionality to be enabled. To disable NRST, set the DISABLE bit in the EXRSTPIN register along with the KEY. Then configure IOMUX for the desired functionality.

After the NRST pin function is disabled, it can only be re-enabled by a POR.

Note

When the NRST pin is shared with the I2C open-drain pin, it is important for the user's system to ensure that the device is powered up and in I2C mode before any transactions occur on the I2C bus. If the device is inadvertently reset due to a low signal on the shared reset or I2C SDA line before this point, it may cause the device reset.

To prevent this, pullups on the NRST pin that is shared with the I2C open-drain IO should be selected to meet the I2C pullup requirements for minimum and maximum values.

2.4.1.4 SWD Pins

There are two serial wire debug (SWD) pins present on all devices:

- SWCLK (serial wire clock)
- SWDIO (serial wire data input/output)

After a cold start, the SWD pins are configured in SWD mode to allow a debug connection to be established. It is possible to re-configure the SWD pins as general purpose IO (GPIO) in software to enable use of these pins in an application when debug support is no longer required. To disable SWD functionality, set the DISABLE bit in the SWDCFG register in SYSCTL along with the KEY. Then configure IOMUX for the desired functionality.

Once the SWD pin functions are disabled, they can only be re-enabled by triggering a POR.

2.4.1.5 Generating Resets in Software

Software can generate a software POR, a software BOOTRST, a software SYSRST with bootstrap loader (BSL) entry, or a software SYSRST by issuing the appropriate command to SYSCTL. To issue a reset, first select the desired reset level in the RESETLEVEL register in SYSCTL. Then set the GO bit in the RESETCMD register along with the KEY value.

Table 2-12. Software Generated SYSCTL Reset Commands

LEVEL	Action
0x0	Software SYSRST
0x1	Software BOOTRST
0x2	Software SYSRST with BSL entry
0x3	Software POR

A CPU-only reset (CPURST) which does not reset the peripherals can also be triggered in software within the Cortex-M0+ CPU by setting the SYSRESETREQ bit in the AIRCR local CPU register. See the CPU Sub System chapter for more information.

Starting the BSL From Software

The software-triggered BSL entry (*RESETLEVEL 0x02*) is a special case of the SYSRST which provides a mechanism for the application software to start the ROM bootstrap loader (BSL). It is not possible to jump to the bootloader code directly during normal software execution in RUN mode. When application software commands a software-triggered BSL entry (*RESETLEVEL 0x02*), a SYSRST is generated first, followed by execution of the boot configuration routine (for authentication), after which the BSL is started (if the device security policy has the BSL configured to be enabled). Once the BSL has completed execution, a second SYSRST is issued and the BCR will execute. When the BCR completes, a final SYSTRST is asserted to return control of the system back to the application software. Any system configuration which is not reset by a SYSRST will be maintained through this entire process. As such, if the RTC can continue run without disruption during execution of a software-triggered BSL entry and exit.

2.4.1.6 Reset Cause

After a device reset occurs, the lowest level reset cause which occurred during reset processing is captured in hardware so that application software can interrogate the reason for the reset and take any appropriate action when starting the application. The lowest level reset cause is encoded into a 5-bit field in the reset cause register in SYSCTL. The contents of the reset cause register are always cleared upon a read, and return zero after being read if no reset has occurred after the read. The reset cause encodings are given in [Table 2-13](#).

Table 2-13. Reset Cause Encoding

Reset				Device Modules Reset										
Reset Level	Cause ID		Reset Cause	NRST/SWD Disables	SHUTDN STOREx	Core Regulator	Debug Subsystem	RTC, LFXT, LFCLK State	SRAM	BCR Execution	IOMUX	EVENT, DMA, FLASHCTL	Peripherals	CPU
	0x00	0	No reset since last read											
POR	0x01	1	VDD < POR- violation	R	R	R	R	R	R	R	R	R	R	R
			PMU trim parity fault											
			SHUTDNSTOREx parity fault											
	0x02	2	NRST pin reset (>1s)	R	R	R	R	R	R	R	R	R	R	
	0x03	3	Software-triggered POR	R	R	R	R	R	R	R	R	R	R	R
BOR	0x04	4	VDD < BOR- violation			R	R	R	R	R	R	R	R	R
	0x05	5	Wake from SHUTDOWN			R	R	R	R	R ⁽¹⁾	R	R	R	
	0x06	6	Reserved											
	0x07	7	Reserved											

Table 2-13. Reset Cause Encoding (continued)

Reset				Device Modules Reset											
Reset Level	Cause ID		Reset Cause	NRST/SWD Disables	SHUTDN STOREx	Core Regulator	Debug Subsystem	RTC, LFXT, LFCLK State	SRAM	BCR Execution	IOMUX	EVENT, DMA, FLASHCTL	Peripherals	CPU	
BOOTRST	0x08	8	Non-PMU trim parity fault					R	R	R	R	R	R	R	
	0x09	9	Fatal clock fault					R	R	R	R	R	R	R	
	0x0A	10	Reserved												
	0x0B	11	Reserved												
	0x0C	12	NRST pin reset (<1 s)						R	R	R ⁽²⁾	R	R	R	
	0x0D	13	Software-triggered BOOTRST						R	R	R ⁽²⁾	R	R	R	
	0x0E	14	WWDT0 violation						R	R	R ⁽²⁾	R	R	R	
	0x0F	15	Reserved												
SYSRST	0x10	16	BSL exit							R	R ⁽²⁾	R	R	R	
	0x11	17	BSL entry							R	R ⁽²⁾	R	R	R	
	0x12	18	Reserved												
	0x13	19	WWDT1 violation								R ⁽²⁾	R	R	R	
	0x14	20	Uncorrectable flash ECC error								R ⁽²⁾	R	R	R	
	0x15	21	CPULOCK violation								R ⁽²⁾	R	R	R	
	0x16	22	Reserved												
	0x17	23	Reserved												
	0x18	24	Reserved												
	0x19	25	Reserved												
	0x1A	26	Debug-triggered SYSRST									R ⁽²⁾	R	R	R
	0x1B	27	Software-triggered SYSRST									R ⁽²⁾	R	R	R
CPURST	0x1C	28	Debug-triggered CPURST											R	
	0x1D	29	Software-triggered CPURST											R	
	0x1E	30	Reserved												
	0x1F	31	Reserved												

- (1) In the case of a SHUTDOWN mode exit, the IOMUX registers are always reset but the IOs themselves retain their last state from the point of entry into SHUTDOWN until the user clears the RELEASE bit in the SHDNIOREL register in SYSCTL. This enables application software to be able to reconfigure IOMUX and any corresponding peripherals before releasing the IO after a SHUTDOWN exit. See [shutdown mode handling](#) and [IOMUX wake](#).
- (2) IOMUX is reset, but if LFXT or LFCLK_IN are enabled then the IOMUX settings for these pads does not reset. The RTC, LFXT, LFCLK_IN, and LFCLK continue to operate without interruption.

If two reset causes occur simultaneously, the lowest cause reset ID value is prioritized and reported. For example, if a WWDT0 violation (cause 0x12) occurs at the same time that a VDD < BOR- violation (cause 0x04) occurs, the reported reset cause is a BOR- violation (cause 0x04), as this is a lower level reset which clears additional aspects of the device state.

The reset cause encoding enables simple software handling during application startup. The reset cause value can be read by application software and tested to be within a certain value range to determine if the following occurred:

- **RESETCAUSE==0x00:** No reset since last read
- **RESETCAUSE<0x04:** The NRST/SWD disable state was reset and can need to be reconfigured

- **RESETCAUSE<0x04:** The SHUTDNSTOREx memory was reset and can need to be reconfigured
- **RESETCAUSE<0x08:** The regulated VCORE domain, including the SRAM, was power cycled
- **RESETCAUSE<0x0B:** The RTC and low frequency clock configuration were reset and can need to be reconfigured
- **RESETCAUSE<0x1C:** The peripherals were reset and can need to be reconfigured

The following example shows how the reset cause can be tested to take specific actions when starting an application after a reset:

```
// Read reset cause into SRAM variable
uint8_t cause = RESETCAUSE;

// Handle device re-configuration based on reset cause level
if (cause!=0)
{
    if (cause<0x04)
    {
        // NRST/SWD disable state was lost
        // SHUTDNSTOREx memory state was lost
        // PMU/VCORE domain state was lost
        // RTC/LFXT/LFCLK state was lost
    }
    if (cause<0x0B)
    {
        // RTC/LFXT/LFCLK state was lost
    }
    if (cause<0x1C)
    {
        // The peripherals were reset
    }
}
```

2.4.1.7 Peripheral Reset Control

Each peripheral on a device contains a reset control register (RSTCTL) and a status register (STAT).

The STAT register is a read-only register which contains a RESETSTKY bit, indicating if the peripheral was reset. This bit can be read by application software to determine if a peripheral was reset and needs to be re-configured. The RESETSTKY bit is cleared by writing the RESETSTKYCLR bit together with the KEY value to the RSTCTL register.

Application software can also force a reset of the peripheral by writing the RESETASSERT bit together with the KEY value to the RSTCTL register. This action will reset the peripheral to its default state, and will set the RESETSTKY bit in the STAT register.

2.4.1.8 Boot Fail Handling

If a boot fails during execution of the boot configuration routine ([BCR](#)), SYSCTL asserts a [BOOTRST](#) to attempt another boot. A boot fail can be caused by the following:

- Boot configuration data integrity error
- Device trim integrity error
- BCR timeout (BCR takes significantly longer than expected to complete for any other reason)

Up to three attempts to successfully boot the device are made by hardware. If the first, second, or third boot attempt is successful, the application starts normally. If the third attempt fails, then the boot process fails, no further boot attempts are made, and the application software is not started.

The purpose of the additional boot attempts is to allow the device to boot correctly if a transient (temporary) error was the cause of the boot fail. If three boot attempts are not successful, a steady-state error condition is likely present and the application is not started to prevent unexpected operation.

Note

If a device is locked due to three failed attempts to boot, and a BOR- violation occurs, a BOR and BOOTRST are still generated (by definition) and a single boot attempt is made. Under the same conditions, if power is completely removed from the device (triggering a POR- violation), then the device again attempts to boot up to three times.

2.4.2 Operating Mode Selection

The device operating mode is configured through the use of the following:

1. Policy bits in the SYSOSCCFG and MCLKCFG registers in SYSTL (to control the behavior of SYSOSC in RUN, SLEEP, and STOP modes)
2. Policy bits in the PMODECFG register in SYSTL (to set the deep sleep level of STOP, STANDBY or SHUTDOWN)
3. SLEEPDEEP policy bit in the SCR local CPU register (to select whether a WFI instruction triggers SLEEP mode or STOP/STANDBY/SHUTDOWN mode)
4. Use of the Arm WFI (wait for interrupt) CPU instruction (to enter the configured SLEEP/STOP/STANDBY/SHUTDOWN state)

Before entering an operating mode where the CPU is disabled, make sure that the appropriate peripheral that can wake the CPU from sleep has been configured to generate a CPU interrupt on the desired event.

For a detailed description of the behavior of each operating mode, see the [operating modes](#) section.

Policy Bit Configuration

[Table 2-14](#) defines how to configure the relevant policy bits for each operating mode. All values are indicated in binary format. A dash (-) indicates that the particular policy bit is a don't care for the specified operating mode.

Table 2-14. Operating Mode Policy Bit Configuration

Operating Mode Policy Control		RUN			SLEEP ⁽²⁾			STOP			STANDBY		SHUTDOWN
Register	Bit	RUN0	RUN1	RUN2	SLEEP0	SLEEP1	SLEEP2	STOP0	STOP1	STOP2 ⁽³⁾	STANDBY0	STANDBY1	
SYSOSCCFG	DISABLE ⁽¹⁾	0	0	1	0	0	1	-	-	(1)	-	-	-
	USE4MHZSTOP	-	-	-	-	-	-	0	1	0	-	-	-
	DISABLESTOP	-	-	-	-	-	-	0	0	1	-	-	-
MCLKCFG	USELFCCLK ⁽¹⁾	0	1	-	0	1	-	0	0	-	-	-	-
	STOPCLKSTBY	-	-	-	-	-	-	-	-	-	0	1	-
PMODECFG	DSLEEP	-	-	-	-	-	-	00	00	00	01	01	10
SCR	SLEEPDEEP	0	0	0	0	0	0	1	1	1	1	1	1

- (1) The SYSOSCCFG.DISABLE and MCLKCFG.USELFCCLK policy bits take effect immediately after being configured, as these bits affect the RUN mode behavior. Other policy bits only take effect when the CPU is put into deep sleep.
- (2) SLEEP mode behavior is always identical to RUN mode, except with the CPUCCLK disabled. As such, the SLEEP behavior is determined by the configuration of RUN mode.
- (3) The STOP2 policy for STOP mode can be configured by setting the DISABLESTOP bit or DISABLE bit in the SYSOSCCFG register before entering DEEPSLEEP. When DISABLESTOP is set and DISABLE is cleared, SYSOSC is only disabled when DEEPSLEEP is requested. SYSOSC continues to run in RUN and SLEEP modes. When DISABLE is set, DISABLESTOP becomes a don't care, and SYSOSC is disabled immediately and is kept disabled in STOP mode.

Entering SLEEP Mode

Entering **SLEEP** mode disables the CPU, but otherwise maintains the same configuration as **RUN**. To enter **SLEEP** mode:

1. Configure the Cortex-M0+ CPU for SLEEP by clearing the SLEEPDEEP bit in the Cortex-M0+ SCR local register
2. Enter sleep mode by executing a WFI (wait for interrupt) CPU instruction

Entering STOP or STANDBY Modes

To enter **STOP** or **STANDBY** mode:

1. Configure the PMODECFG register in SYSCCTL to 0b00 (STOP) or 0b01 (STANDBY)
2. Configure the Cortex-M0+ CPU for DEEP SLEEP by setting the SLEEPDEEP bit in the Cortex-M0+ SCR local register
3. Enter sleep mode by executing a WFI (wait for interrupt) CPU instruction

Entering SHUTDOWN Mode

To enter **SHUTDOWN** mode:

1. Configure the PMODECFG register in SYSCCTL to 0b10 (SHUTDOWN)
2. Configure the Cortex-M0+ CPU for DEEP SLEEP by setting the SLEEPDEEP bit in the Cortex-M0+ SCR local register
3. Enter sleep mode by executing a WFI (wait for interrupt) CPU instruction

2.4.3 Asynchronous Fast Clock Requests

Peripherals are configured to asynchronously assert a hardware request to the SYSCCTL for a fast clock source, even if the device is operating in STOP or STANDBY mode. This mechanism for applications where the MCLK/ULPCLK tree is normally sourced from either LFCLK (at 32 kHz) or SYSOSC (at 4 MHz), but a faster clock is temporarily needed to quickly handle a peripheral event (for example, a timer IRQ or GPIO IRQ) or peripheral activity (such as serial communication or an ADC conversion).

Asynchronous fast clock requests are also useful for scenarios where the device is running in STANDBY1 mode. In STANDBY1 (when STOPCLKSTBY is set), the ULPCLK and LFCLK are disabled to all peripherals except for TIMG0 and TIMG1, leaving TIMG0 and TIMG1 and the RTC as the only clocked peripherals. To wake up the device from this state where the bus clock (ULPCLK) is disabled, a TIMG0 and TIMG1 or RTC interrupt request forces an asynchronous fast clock request to wake the device to RUN mode. Other peripherals can also wake the device from this state if they support detecting an asynchronous event (for example GPIO, comparator, and serial interfaces).

Asynchronous fast clock requests temporarily provide peripherals with a 32 MHz bus clock ([MCLK/ULPCLK](#)), sourced from the [SYSOSC](#), for the duration of the request. [MFCLK](#), if enabled for use, is also enabled during the asynchronous request.

Asynchronous Fast Clock Behavior

When configured, SYSCCTL will respond to a peripheral fast clock request in the following way:

1. If the device is currently in a STOP or STANDBY mode, the low power state is temporarily suspended to support running the bus clock ([ULPCLK](#)) at the [SYSOSC](#) base frequency (32 MHz)
2. If SYSOSC is disabled, it is forced to be enabled; if SYSOSC is already running but at a different frequency than base frequency, it is forced to base frequency (32 MHz)
3. The [MCLK/ULPCLK](#) tree is forced to be sourced from [SYSOSC](#) at the 32 MHz rate; if the device is in RUN mode then the [CPUCLK](#) is also switches to the SYSOSC rate (the CPUCLK is always derived from MCLK)
4. If the [MFCLK](#) is configured to be used, it will be activated

After the configuration above is applied, it will be held for the duration of time that the asynchronous request remains asserted plus an additional 32 SYSOSC cycles (approximately 1µs). 32 SYSOSC cycles after the request is removed, the system will return to the configuration which existed before the fast clock request, provided the CPU did not change the configuration during the request.

Asynchronous fast clock requests are ignored and will have no effect on the device configuration if any of the following are true:

- MCLK is already sourced from SYSOSC at base frequency (32 MHz)
- MCLK is sourced from [HSCLK](#) (either [HFCLK](#) or [SYSPLL](#))
- Asynchronous fast clock requests are globally blocked by setting the BLOCKASYNCALL bit in the SYSOSCCFG register in SYCTL

Peripheral Support

The RTC, TIMG0, TIMG1, GPIO, comparator, SPI, I2C, UART, and ADC peripherals all provide support for generating an asynchronous fast clock request. The purpose, request source, and configuration requirements for these peripherals are given in [Table 2-15](#).

Table 2-15. Peripheral Support for Asynchronous Fast Clock Requests

Peripheral	Purpose	Request Source	Configuration
RTC	Fast CPU wake from RTC event	RTC IRQ to CPU	An RTC IRQ event always generates an asynchronous fast clock request when the device is in STANDBY1 mode; this is needed to wake the device as the main ULPClk is disabled to reduce power consumption. The RTC can also be configured to generate an asynchronous fast clock request in any operating mode by clearing the BLOCKASYNC bit in the RTC CLKCFG register; this provides the lowest latency RTC event response.
TIMG0 and TIMG1	Fast CPU wake from TIMG0/TIMG1 event	TIMG0 or TIMG1 IRQ to CPU	An IRQ event from TIMG0 or TIMG1 generates an asynchronous fast clock request when the device is in STANDBY1 mode and the corresponding IMASK interrupt is set in the TIMG registers. This is needed to wake the device as the ULPClk is disabled to reduce power consumption.
GPIO	Fast CPU wake from GPIO event	GPIO activity	The GPIO generates an asynchronous fast clock request through the GPIO configuration registers. This is for applications where GPIO wake from STANDBY mode is desired, as the fast clock request will cause the GPIO digital glitch filters to run at the 32 MHz 24 MHz rate. In addition to configuring the GPIO registers to request the fast clock, the BLOCKASYNCALL bit must be cleared in the SYSOSCCFG register to allow the request to propagate.
Comparator	Fast wake from a comparator event	Comparator event	A comparator event generates an asynchronous fast clock request to provide the lowest latency comparator event response. The BLOCKASYNC bit in the CLKCFG register of the respective comparator must be disabled.
SPI	Temporarily use fast clock for bit clock generation	SPI activity	SPI activity generates an asynchronous fast clock request when the BLOCKASYNC bit is cleared in the CLKCFG register of the respective SPI peripheral.
I2C	Temporarily use fast clock for bit clock generation	I2C activity	I2C activity generates an asynchronous fast clock request when the BLOCKASYNC bit is cleared in the CLKCFG register of the respective I2C peripheral.
UART	Temporarily use a fast clock for baud rate generation	UART activity	UART activity generates an asynchronous fast clock request when the BLOCKASYNC bit is cleared in the CLKCFG register of the respective UART peripheral.
ADC	Temporarily run the SYSOSC to support timer-triggered ADC operation from a low-power mode	ADC	If an ADC conversion is triggered when SYSOSC is disabled, an asynchronous fast clock request is generated to enable the SYSOSC (SYSOSC is required for correct ADC operation).

Fast CPU Event Handling

In addition to the peripheral event and activity fast clock request triggers, the SYCTL can be configured to generate an asynchronous fast clock request upon any IRQ request to the CPU. This provides the lowest latency interrupt handling when the system is running at the LFCLK rate (32 kHz), as the IRQ request will propagate

through the wake-up logic at the SYSOSC rate (32 MHz 24 MHz) vs. the LFCLK rate (32 kHz). When the FASTCPUEVENT bit is set in the SYSOSCCFG register in SYSCTL, any interrupt request to the CPU will also generate a fast clock request.

Asynchronous Fast Clock Request Logic

The logic for asserting a fast clock request is given in the following figure.

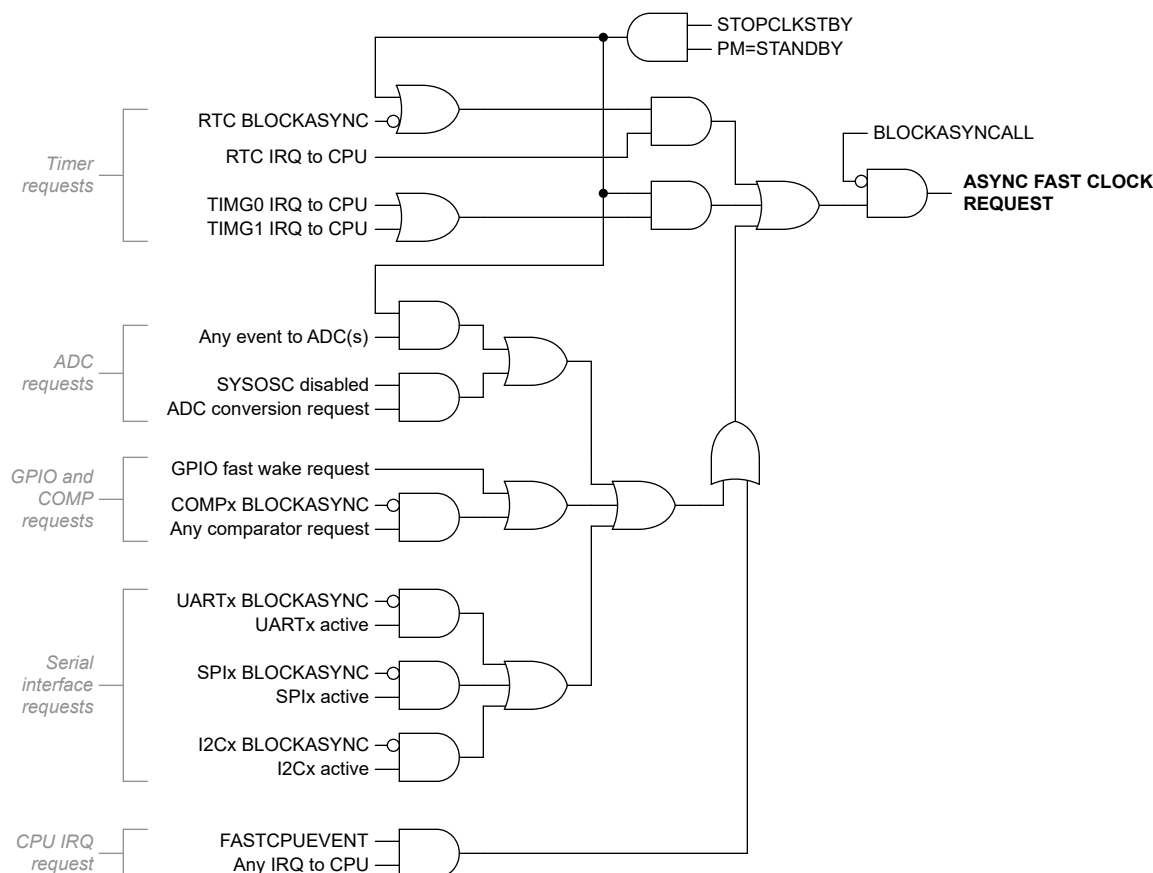


Figure 2-11. MSPM0Gxx Asynchronous Fast Clock Request Logic

2.4.4 SRAM Write Protection

Certain applications need to place read-only data into SRAM. This can occur if code is placed into SRAM (for zero wait state execution) or if critical lookup tables are placed in SRAM (for zero wait state reads). In these cases, especially when code is to be executed from the SRAM, it is desirable to prevent unintentional writes to SRAM addresses that can corrupt executable code in the event of a buffer overrun or a stack overflow. Likewise, it is desirable to prevent execution from non-write-protected SRAM addresses. To improve robustness of data in stored in SRAM, SYSCTL provides a write-exclusive-execute boundary mechanism.

To use this feature, first load the read-only data into the desired SRAM address, then configure the SRAM address range to be write protected. SRAM contents which are to be read-execute (no writes) should be placed into the upper portion of SRAM. SRAM contents which are to be read-write (no execute) should be placed into the lower portion of the SRAM. Then, the SRAMBOUNDARY register may be written with the desired boundary to partition the SRAM into two regions, with the lower region being RW and the upper region being RX.

2.4.5 Flash Wait States

Flash wait states are managed automatically by SYSCTL when MCLK is running from **SYSOSC** or **LFCLK**. If MCLK will not be switched to **SYSPLL**, **HFXT**, or **HFCLK_IN**, no wait state configuration is required.

If MCLK is to be configured to run from one of the high-speed clock sources such as [SYSPLL](#), [HFXT](#), or [HFCLK_IN](#), the flash wait state configuration in MCLKCFG.FLASHWAIT is applied. By default, MCLKCFG.FLASHWAIT is set to 0x2 (2 wait states) which supports operation at the maximum MCLK frequency of 80MHz. If MCLK is configured to run from a high-speed clock but the MCLK frequency allows operation with fewer than 2 wait states, then MCLKCFG.FLASHWAIT can be reduced.

Refer to the *Recommended Operation Conditions* section of the device specific data sheet to determine the max clock frequency supported with 0 or 1 wait state.

2.4.6 Flash Bank Address Swap

Devices with multiple flash banks provide a mechanism for swapping the address space of the upper banks with the address space of the lower banks to enable firmware updates where the firmware image itself does not need to have knowledge of the bank it resides in to be able to execute properly on the hardware.

To swap the addresses of the upper flash banks with the addresses of the lower flash banks, set the USEUPPER bit in the FLBANKSWAP register while also writing the KEY value.

Special considerations apply when swapping the flash bank address space. See the [non-volatile memory system](#) chapter of this guide for considerations when using flash bank address swapping.

2.4.7 Shutdown Mode Handling

When the device is configured to enter SHUTDOWN mode, the core regulator is powered down and the device register contents and SRAM contents are lost. An exit from SHUTDOWN mode generates a [BOR level reset](#). Two mechanisms are provided to preserve the device state when entering SHUTDOWN mode: IO latching and a small shutdown memory.

Shutdown IO State

The digital IO pin states (output low/high, pullup/pulldown, Hi-Z, drive configuration) are latched and retained upon entry to SHUTDOWN. After exiting SHUTDOWN mode, the IOs are held in the previous state until released by application software setting the RELEASE bit in the SHDNIOREL register along with the matching KEY value. When exiting SHUTDOWN mode, application software must first re-configure the IO to their proper state, then release the IO. To determine at startup if the cause of a reset was an exit from SHUTDOWN mode, application software must read the [RSTCAUSE register in SYSCTL](#).

Note

When exiting SHUTDOWN, **the serial wire debug (SWD) pins also remain locked until application software sets the RELEASE bit**. As a result, a debug connection cannot establish when waking up from SHUTDOWN mode until the IO are released by application software.

Note

When exiting SHUTDOWN, the bootstrap loader (BSL) invoke pin must be held at a logic low level to prevent unintended entry into the BSL during exit from SHUTDOWN. An entry to the BSL during SHUTDOWN exit prevents the application code from starting, the BSL interfaces are not be available, and a SWD connection is possible as the IO states remain latched through SHUTDOWN exit until application software releases the IOs.

Shutdown Memory

To enable saving of application state information before entering SHUTDOWN mode, 4 bytes of shutdown memory are provided in SYSCTL. These memory locations are retained in SHUTDOWN mode and are readable by the application after exiting SHUTDOWN. To save data to the SHUTDOWN memory, write to the SHUTDNSTORE0-SHUTDNSTORE3 registers in SYSCTL.

2.4.8 Configuration Lockout

Configuration registers in SYSCTL can be locked out from writes to add a layer of robustness against unintended changes to the PMCU at runtime. To lock out the configuration registers from writes, set the ACTIVE bit in the WRITELOCK register in SYSCTL.

All SYSCTL registers are protected by the WRITELOCK functionality except for those listed below:

- WRITELOCK
- PMODECFG
- FCC, FCCCMD
- FLBANKSWAP
- RSTCAUSE (read-to-clear), RESETLEVEL, RESETCMD
- BORTHRESHOLD, BORCLRCMD
- SHDNIOREL
- PMUOPAMP
- SHUTDNSTOREx

In addition to the overall SYSCTL configuration write lock feature, many SYSCTL registers also require a KEY value to be written in conjunction with the desired configuration data for the write to take effect.

2.4.9 System Status

The status of various aspects of the PMCU can be polled by software by reading the CLKSTATUS and SYSSTATUS registers in SYSCTL.

Checking Clock Status (CLKSTATUS)

The CLKSTATUS register in SYSCTL is a read-only register which indicates the current configuration and status of the clock module. Key status information provided in CLKSTATUS includes:

- The current SYSOSC frequency
- The current HSCLK selection
- The current LFCLK selection
- The current MCLK selection
- The HFCLK and SYSPLL status
- The LFXT status
- The LFOSC status
- Indications that the HSCLK, HFCLK, and SYSPLL are disabled
- Error indications if a peripheral requested a clock and the clock cannot be generated

This status information is useful to validate that a requested clock change has completed successfully, or to check the true SYSOSC frequency in applications where SYSOSC can have asynchronous activation or frequency requests issued by peripherals.

Checking System Status (SYSSTATUS)

The SYSSTATUS register in SYSCTL is a read-only register which indicates flash ECC errors (SED and DED) along with other peripheral-specific status information. ECC error bits in SYSSTATUS are sticky (they remain set when an ECC error occurs even if future reads do not have errors). These bits can be reset (cleared) by setting the ALLECC bit in the SYSSTATUSCLR register along with the KEY value.

2.4.10 Error Handling

MSPM0 devices include several diagnostic mechanisms to detect errors at runtime. [Table 2-16](#) lists error sources and their corresponding handling mechanism.

Note

Not all MSPM0 devices support all diagnostic features. For example, some devices do not have ECC/parity on memories and some devices do not have dual watchdog timers. Always refer to the device-specific data sheet to understand which diagnostic features are available for a given device. In the PMCU registers section, register maps are also provided for each MCU sub-family detailing the specific registers available for a given device.

Table 2-16. Error Sources and Handling Mechanisms

Error Source	Error	Handling Mechanism
Flash (if device has ECC)	Non-correctable ECC error (if device has ECC)	- For a CPU or DMA request, a FLASHDED non-maskable interrupt is generated to the processor or a SYSRST is generated depending on configuration of the FLASHECCRSTDIS bit - The FLASHDED sticky bit is set in the SYSSTATUS register in SYSCTL
	Correctable ECC error (if device has ECC)	- A FLASHSEC interrupt is also generated in SYSCTL - The FLASHSEC sticky bit is set in the SYSSTATUS register in SYSCTL
SRAM	Non-correctable ECC error (if device has ECC)	An SRAMDED non-maskable interrupt is generated to the processor
	Correctable ECC error (if device has ECC)	A SYSCTL SRAMSED interrupt is generated to the processor
	Parity error (if device has parity)	- Non-maskable interrupt is generated to the processor if the request was from the CPU - DMA data error interrupt is generated if the request was from the DMA
	Address error on CPU access	A hard fault is generated in the CPU
	Address error on DMA access	A DMA address error interrupt is generated in the DMA controller
	ECC error on CAN SRAM (if device has CAN-FD)	An interrupt is generated in the CAN-FD peripheral
SHUTDNSTOREx Memory	Parity error	A POR is generated
CKM	MCLK failure	A BOOTRST is generated
	LFCLK failure (if present)	- A BOOTRST is generated if LFCLK is sourcing MCLK - An LFCLKFAIL non-maskable interrupt is generated in the SYSCTL NMI registers.
CPUSS (if device has MPU)	Memory protection unit violation	A hard fault is generated in the CPU
WWDT	WWDT0 violation	A BOOTRST is generated or a non-maskable interrupt is generated in the SYSCTL NMI registers depending on configuration of the WWDTLPORSTDIS bit
	WWDT1 violation (if present)	A BOOTRST is generated or a non-maskable interrupt is generated in the SYSCTL NMI registers depending on configuration of the WWDTLP1RSTDIS bit

Table 2-16. Error Sources and Handling Mechanisms (continued)

Error Source	Error	Handling Mechanism
PMU	Trim parity error	• A POR is generated
	POR0- supply error	• A POR is generated
	BOR0- supply error	• A BOR is generated
	BOR1/2/3- supply error	• A BORLVL non-maskable interrupt is generated in the SYSCTL NMI registers
CPUSS	Memory protection unit violation (if present)	• A hard fault is generated in the CPU

Configurable NMI Triggers

Error sources can be configured to trigger either a non-maskable interrupt or a different handling mechanism. The SYSTEMCFG register in SYSCTL may be used to specify the desired error handling mechanism. For example, the WWDTR0 may be configured to generate either a BOOTRST or an NMI, with BOOTRST being the default case. Refer to the SYSTEMCFG register for the relevant device sub-family for the available error handling options.

2.4.11 SYSCTL Events

The SYSCTL module contains two [event publishers](#) and no [event subscribers](#). One event publisher manages SYSCTL interrupt requests (IRQs) to the CPU subsystem. The second publisher manages non-maskable interrupts to the CPU subsystem for critical diagnostics.

The SYSCTL events are summarized in [SYSCTL Events](#).

Table 2-17. SYSCTL Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU interrupt	Publisher	SYSCTL	CPU Subsystem	Static route	SYSCTL interrupt registers	Fixed interrupt route from SYSCTL to CPU
CPU non-maskable interrupt (NMI)	Publisher	SYSCTL	CPU Subsystem	Static route	NMI interrupt registers	Fixed interrupt route from SYSCTL to CPU

2.4.11.1 CPU Interrupt Event (CPU_INT)

The SYSCTL module provides several interrupt sources which can be configured to source a [CPU interrupt event](#). In order of decreasing interrupt priority, the CPU interrupt events from the SYSCTL are given in [Table 2-18](#).

Table 2-18. SYSCTL CPU Interrupt Event Sources

Index (IIDX)	Name	Description
0	NONE	No interrupt pending.
1	LFOSCGOOD	Indicates when LFOSC is ready during startup, as LFOSC takes ≈1ms to start.
2	ANACLKERR	Indicates that an analog function was enabled and expecting a SYSOSC to be operation at a certain frequency, but SYSOSC was either not available or not operating at the required frequency.
3	FLASHSEC	Indicates that a flash memory single-bit correctable error was detected
4	SRAMSEC	Indicates that a SRAM single-bit correctable error was detected

Table 2-18. SYSCTL CPU Interrupt Event Sources (continued)

Index (IIDX)	Name	Description
5	LFXTGOOD	Indicates when the low frequency external clock (either the LFXT oscillator or LFCLK_IN digital clock) are ready. This indication is useful when starting the clock system and waiting for LFXT or LFCLK_IN to be ready before switching the LFCLK source from LFOSC to an external source.
6	HFCLKGOOD	Indicates when the high frequency external clock (either the HFXT oscillator or HFCLK_IN digital clock) are ready. This indication is useful when starting the clock system and waiting for HFCLK to be ready before switching the MCLK source to HFCLK or before starting the SYSPLL with HFCLK as the SYSPLL reference.
7	SYSPLLGOOD	Indicates when the SYSPLL is ready and available for use. This indication is useful when starting the clock system and waiting for SYSPLL to be ready before switching the MCLK source to SYSPLL.
8	HSCLKGOOD	Indicates when the HSCLK (sourced by either HFCLK or a SYSPLL output) is ready. This indication is useful when waking up from STOP or STANDBY mode when HSCLK is configured as the MCLK source in RUN/SLEEP mode. When waking from STOP or STANDBY, MCLK will run from SYSOSC until the HSCLK is available, at which time SYSCTL automatically switches the MCLK source to the selected HSCLK source. This interrupt communicates that after wake-up from STOP/STANDBY, MCLK has switched over to the desired HSCLK source and it is now possible to use timing-sensitive peripherals sourced by MCLK.

The CPU interrupt event configuration is managed with the SYSCTL IIDX, IMASK, RIS, MIS, ISET, and ICLR event management registers. See [Section 7.2.5](#) for guidance on configuring these registers for CPU interrupts.

2.4.11.2 Non-maskable Interrupt Event (NMI)

The SYSCTL module provides x interrupt sources which can be configured to source a [CPU interrupt event](#). In order of decreasing interrupt priority, the CPU interrupt events from the SYSCTL are given in [Table 2-18](#).

Table 2-19. Non-maskable Interrupt Event Table

Index (IIDX)	Name	Description
0	NONE	No NMI pending.
1	BORLVL	Indicates that VDD has dropped below the specified VBOR- threshold.
2	WWDT0	A WWDT0 violation occurred.
3	WWDT1	A WWDT1 violation occurred.
4	LFCLKFAIL	Indicates that the LFXT or LFCLK_IN clock source is dead. This indication is useful for handling LFCLK errors when LFCLK is not sourcing MCLK but is sourcing a peripheral (for example, the RTC)
5	FLASHDED	Indicates that a flash memory double-bit uncorrectable error was detected.
6	SRAMDED	Indicates that an SRAM double-bit uncorrectable error was detected.

The CPU non-maskable interrupt event configuration is managed with the NMIIDX, NMIRIS, NMISET, and NMIICLR event management registers. See [Section 7.2.5](#) for guidance on configuring the interrupt management registers.

2.5 Quick Start Reference

The PMCU is designed to provide a simple, easy-to-use power management, clocking, and reset management functionality. This section describes the basic operating principles of the PMCU as well as tips and tricks for taking the default configuration out of reset and optimizing it for particular applications.

2.5.1 Default Device Configuration

The default operating configuration of the device provides basic functionality which can be suitable for many applications without modification.

MSPM0Gxx devices power up and release reset for execution of application code when the external supply (VDD and VSS) reaches 1.62 V. When the application code is released for execution, the device is in RUN mode with **MCLK**, which is sourced from the internal **SYSOSC** at 32 MHz. The **CPUCLK** and **ULPCLK** are also 32 MHz, derived from MCLK. **LFCLK** starts automatically, sourced from the internal **LFOSC**. In RUN mode with the default configuration, all peripherals are available to be enabled. Peripherals such as the DMA, CRC, and AES run directly from MCLK at the MCLK rate. Other peripherals, such as timers and serial interfaces, can run from the bus clock at 32 MHz or from the low-frequency 32-kHz clock (LFCLK) based on their [peripheral clock selection](#).

Power consumption can be reduced by entering SLEEP, STOP, STANDBY, or SHUTDOWN mode. By default, these modes behave in the following way:

- In SLEEP mode, the CPUCLK is disabled but all peripherals, including the DMA, continue to run as configured at either 32 MHz or 32 kHz. This power mode is designed for scenarios where PD1 peripherals are used and having the lowest possible wakeup latency is more important than having the lowest power consumption.
- In STOP mode, (STOP0 by default), the MCLK source to PD1 peripherals is disabled and PD1 peripherals are disabled and in retention (unavailable for use). The default STOP mode is designed for scenarios where power optimization is important but the ADC, OPA, DAC, or a clock faster than 32 kHz is needed.
 - SYSOSC by default will continue to run at 32 MHz 24 MHz, but the MCLK tree will run at 4 MHz (SYSOSC/8) and peripherals in PD0 which are still active will see the bus clock (ULPCLK) change from 32 MHz to 4 MHz.
 - PD0 peripherals configured to run from LFCLK continue to run at 32 kHz.
 - All OPA modes can be used
 - The ADC will always see SYSOSC at 32 MHz for sampling
- In STANDBY mode (STANDBY0 by default), the MCLK tree runs from LFCLK at 32 kHz and SYSOSC is disabled. PD0 peripherals running from the bus clock change to 32 kHz. PD0 peripherals running from LFCLK continue to run at 32 kHz with no change.

2.5.2 Leveraging MFCLK

When running with the default PMCU configuration, timers and serial interfaces can select either the bus clock (MCLK/ULPCLK) or the LFCLK as their clock source. LFCLK is always 32 kHz in RUN, SLEEP, STOP, and STANDBY, but MCLK/ULPCLK changes to 4 MHz in STOP and to 32 kHz in STANDBY, meaning that peripherals running from the bus clock see the source clock frequency change when transitioning power modes.

MFCLK, by contrast, works like LFCLK in that it provides a constant frequency clock source for peripherals across RUN, SLEEP, and STOP modes. MFCLK provides a constant 4 MHz as an alternative to LFCLK which runs at 32 kHz. The 4-MHz time base for MFCLK is always derived from SYSOSC. Peripherals, specifically PD0 peripherals that can be used in STOP mode, can select MFCLK as their clock source instead of ULPCLK. MFCLK is maintained at 4 MHz in RUN, SLEEP and STOP for peripherals like UART, I2C, and low-power timers that need a consistent clock but require a clock source greater than 32 kHz.

For information on using MFCLK, see the [MFCLK section](#).

2.5.3 Optimizing Power Consumption in STOP Mode

The STOP mode provides considerable flexibility for tailoring the device to an application's specific power and performance requirements. There are several options available for reducing power consumption in STOP mode:

- Enable SYSOSC gear shift mode in STOP to reduce SYSOSC current. In STOP mode, the MCLK tree (and ULPCLK) are always limited to 4 MHz maximum. By default, SYSOSC runs in STOP mode at 32 MHz (base frequency) with a divide-by-8 to meet the 4-MHz max frequency limit in STOP mode. Alternatively, SYSOSC

can be gear shifted to run natively at 4 MHz in STOP mode (STOP1), saving SYSOSC current. See [SYSOSC gear shift mode](#).

- If a 32-kHz clock is sufficient to run the needed peripherals, it is possible to run in STOP mode with MCLK sourced from LFCLK at 32 kHz. SYSOSC can be disabled to conserve power. To disable SYSOSC in STOP mode and run from LFCLK (STOP2), see [disabling SYSOSC](#) and [operating mode selection](#).

2.5.4 Optimizing Power Consumption in STANDBY Mode

In STANDBY mode, if only RTC, TIMG0, TIMG1, or asynchronous fast wake from GPIO, comparator (low-power mode), or a serial interface is desired, the lowest possible power consumption can be achieved by configuring the ULPCLK and LFCLK to be disabled when entering STANDBY, leaving only the RTC, TIMG0, and TIMG1 running (STANDBY1). See the [LFCLK section](#). In this state, RTC, TIMG0, TIMG1, or asynchronous activity/event will trigger an [asynchronous fast clock request](#) to wake the system.

2.5.5 Increasing MCLK and ULPCLK Precision

If an application requires high clocking accuracy for high-frequency peripherals, the best accuracy is achieved by using an external high-frequency crystal with the [HFXT](#) and sourcing the MCLK tree directly from HFCLK. By sourcing the MCLK directly from HFCLK, the bus clock to all peripherals in PD1 and PD0 will be the HFCLK.

Crystal frequencies up to 48 MHz are supported by the HFXT, but because PD0 peripherals using ULPCLK are limited to 40 MHz, 40 MHz is the highest crystal frequency supported to run the PD1 and PD0 peripherals at the same frequency. If a 48-MHz crystal is used, PD1 peripherals and the CPUCLK can run at 48 MHz directly from the crystal, but PD0 peripherals must run at MCLK/2 (24 MHz).

If precision clocking is needed for the CAN-FD controller or the ADC (for sampling window generation), and high CPU performance (high MCLK) is also required, it is also possible to source the CAN-FD controller (CANCLK) and the ADC sampling clock (ADCCLK) from HFCLK directly, asynchronous to MCLK, while running MCLK at maximum frequency using the PLL for best compute performance.

2.5.6 Configuring MCLK for Maximum Speed

The best CPU compute performance is obtained by using the SYSPLL to generate an 80MHz clock from either the SYSOSC reference or HFXT. To configure the SYSPLL to generate an 80MHz output sourcing MCLK, see the [SYSPLL configuration section](#).

Running MCLK at 80MHz also provides the best possible timer resolution for TIMA and TIMG peripherals in the PD1 domain (12.5ns).

2.5.7 High Speed Clock (SYSPLL, HFCLK) Handling in Low-Power Modes

The [SYSPLL](#) and [HFCLK](#) (HFXT, HFCLK_IN) high speed clock sources are not supported in the [STOP](#) and [STANDBY](#) operating modes. When a high-speed clock source ([SYSPLL](#), [HFCLK](#)) is enabled, entering either the STOP mode or STANDBY mode will cause SYSCTL to automatically disable the SYSPLL and/or HFCLK before entering STOP or STANDBY mode. Upon exit from STOP or STANDBY mode to RUN mode, SYSCTL will automatically re-enable the SYSPLL and/or HFCLK if they were previously enabled before entering the low-power mode.

Before entering STOP or STANDBY for the first time after enabling the SYSPLL or HFCLK, and after waking up from STOP or STANDBY mode, application software must wait to enter STOP or STANDBY mode until any previously enabled high speed clock sources have completed startup.

Application software must check the following before entering STOP or STANDBY mode:

- If the **SYSPLL was enabled**, then application software must wait for the SYSPLL to complete startup. When the SYSPLL startup is complete, the SYSPLLG00D bit will be set in the CLKSTATUS register in SYSCTL. If the SYSPLL fails to start, the SYSPLLOFF bit will be set instead. The SYSPLLOFF bit indicates that the SYSPLL was dead at startup or was not previously enabled. Ensure that either SYSPLLG00D or SYSPLLOFF is set before attempting to enter STOP or STANDBY.
- If the **HFCLK was enabled**, then application software must wait for the HFCLK to complete startup. When the HFCLK startup is complete, the HFCLKG00D bit will be set in the CLKSTATUS register in SYSCTL. If

the HFCLK fails to start, the HFCLKOFF bit will be set instead. The HFCLKOFF bit indicates that the HFCLK was dead at startup or was not previously enabled. Ensure that either HFCLKGOOD or HFCLKOFF is set before attempting to enter STOP or STANDBY.

In the event that the MCLK was configured to be sourced from HSCLK before entry to STOP or STANDBY, upon exit from STOP or STANDBY the MCLK will be sourced from SYSOSC initially and the CPU will be released to begin executing code at the SYSOSC frequency. SYSCTL will automatically restore the MCLK configuration to the previously selected high-speed clock when the high-speed clock has started and is ready for use. When MCLK switches back to the high-speed clock, SYSCTL will generate an HSCLK GOOD interrupt to alert the application that MCLK is again running from the high-speed clock.

2.5.8 Optimizing for Lowest Wakeup Latency

To ensure the lowest possible wakeup latency from STOP or STANDBY mode to RUN mode, set MCLK to SYSOSC with SYSOSC running at base frequency (32 MHz) before entering STOP or STANDBY. SYSOSC always starts at base frequency and latency is minimized if SYSOSC does not need to change to an alternate frequency.

2.5.9 Optimizing for Lowest Peak Current in RUN/SLEEP Mode

In applications which are peak current limited, there are two options for reducing active current in RUN and SLEEP modes:

- If 32 kHz provides sufficient performance, run MCLK from LFCLK. MCLK can be selected to run from LFCLK with SYSOSC disabled. If no fast handling of events is needed, SYSOSC asynchronous requests can be disabled to ensure that the device always runs from LFCLK. This provides the lowest possible current with the CPU still running (RUN2). See the [MCLK](#), [SYSOSC](#), and [Operating Mode Selection](#) sections.
- If 32 kHz does not provide sufficient performance, MCLK can be selected to run from SYSOSC with SYSOSC set to low frequency (4 MHz). With MCLK running from SYSOSC, the MDIV divider for MCLK can be applied to reduce current consumption. For example, MCLK can be configured to run as low as 250kHz by setting MDIV to /16 when sourced from SYSOSC running at 4 MHz. See the [MCLK](#) section.

2.6 SYSCTL Registers

Table 2-20 lists the memory-mapped registers for the SYSCTL registers. All register offset addresses not listed in Table 2-20 should be considered as reserved locations and the register contents should not be modified.

Table 2-20. SYSCTL Registers

Offset	Acronym	Register Name	Section
1020h	IIDX	SYSCTL interrupt index	Section 2.6.1
1028h	IMASK	SYSCTL interrupt mask	Section 2.6.2
1030h	RIS	SYSCTL raw interrupt status	Section 2.6.3
1038h	MIS	SYSCTL masked interrupt status	Section 2.6.4
1040h	ISSET	SYSCTL interrupt set	Section 2.6.5
1048h	ICLR	SYSCTL interrupt clear	Section 2.6.6
1050h	NMIIDX	NMI interrupt index	Section 2.6.7
1060h	NMIRIS	NMI raw interrupt status	Section 2.6.8
1070h	NMISET	NMI interrupt set	Section 2.6.9
1078h	NMIICLR	NMI interrupt clear	Section 2.6.10
1100h	SYSOSCCFG	SYSOSC configuration	Section 2.6.11
1104h	MCLKCFG	Main clock (MCLK) configuration	Section 2.6.12
1108h	HSCLKEN	High-speed clock (HSCLK) source enable/disable	Section 2.6.13
110Ch	HSCLKCFG	High-speed clock (HSCLK) source selection	Section 2.6.14
1110h	HFCLKCLKCFG	High-frequency clock (HFCLK) configuration	Section 2.6.15
1114h	LFCLKCFG	Low frequency crystal oscillator (LFXT) configuration	Section 2.6.16
1120h	SYSPLLCFG0	SYSPLL reference and output configuration	Section 2.6.17
1124h	SYSPLLCFG1	SYSPLL reference and feedback divider	Section 2.6.18
1128h	SYSPLLPARAM0	SYSPLL PARAM0 (load from FACTORY region)	Section 2.6.19
112Ch	SYSPLLPARAM1	SYSPLL PARAM1 (load from FACTORY region)	Section 2.6.20
1138h	GENCLKCFG	General clock configuration	Section 2.6.21
113Ch	GENCLKEN	General clock enable control	Section 2.6.22
1140h	PMODECFG	Power mode configuration	Section 2.6.23
1150h	FCC	Frequency clock counter (FCC) count	Section 2.6.24
1170h	SYSOSCUSER	SYSOSC user-specified trim	Section 2.6.25
1178h	SRAMBOUNDARY	SRAM Write Boundary	Section 2.6.26
1180h	SYSTEMCFG	System configuration	Section 2.6.27
1200h	WRITELOCK	SYSCTL register write lockout	Section 2.6.28
1204h	CLKSTATUS	Clock module (CKM) status	Section 2.6.29
1208h	SYSSTATUS	System status information	Section 2.6.30
120Ch	DEDERRADDR	Memory DED Address	Section 2.6.31
1220h	RSTCAUSE	Reset cause	Section 2.6.32
1300h	RESETLEVEL	Reset level for application-triggered reset command	Section 2.6.33
1304h	RESETCMD	Execute an application-triggered reset command	Section 2.6.34
1308h	BORTHRESHOLD	BOR threshold selection	Section 2.6.35
130Ch	BORCLRCMD	Set the BOR threshold	Section 2.6.36
1310h	SYSOSCFCLCTL	SYSOSC frequency correction loop (FCL) ROSC enable	Section 2.6.37
1314h	LFXTCTL	LFXT and LFCLK control	Section 2.6.38
1318h	EXLFCTL	LFCLK_IN and LFCLK control	Section 2.6.39
131Ch	SHDNIORL	SHUTDOWN IO release control	Section 2.6.40
1320h	EXRSTPIN	Disable the reset function of the NRST pin	Section 2.6.41

Table 2-20. SYSCTL Registers (continued)

Offset	Acronym	Register Name	Section
1324h	SYSSTATUSCLR	Clear sticky bits of SYSSTATUS	Section 2.6.42
1328h	SWDCFG	Disable the SWD function on the SWD pins	Section 2.6.43
132Ch	FCCCMD	Frequency clock counter start capture	Section 2.6.44
1380h	PMUOPAMP	GPAMP control	Section 2.6.45
1400h	SHUTDNSTORE0	Shutdown storage memory (byte 0)	Section 2.6.46
1404h	SHUTDNSTORE1	Shutdown storage memory (byte 1)	Section 2.6.47
1408h	SHUTDNSTORE2	Shutdown storage memory (byte 2)	Section 2.6.48
140Ch	SHUTDNSTORE3	Shutdown storage memory (byte 3)	Section 2.6.49

Complex bit access types are encoded to fit into small table cells. [Table 2-21](#) shows the codes that are used for access types in this section.

Table 2-21. SYSCTL Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
RC	R C	Read to Clear
Write Type		
W	W	Write
W1C	W 1C	Write 1 to clear
W1S	W 1S	Write 1 to set
Reset or Default Value		
-n		Value after reset or the default value

2.6.1 IIDX Register (Offset = 1020h) [Reset = X]

IIDX is shown in [Figure 2-12](#) and described in [Table 2-22](#).

Return to the [Summary Table](#).

SYSCTL interrupt index

Figure 2-12. IIDX Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																												STAT			
R-X																												R-0h			

Table 2-22. IIDX Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	X	
3-0	STAT	R	0h	The SYSCTL interrupt index (IIDX) register generates a value corresponding to the highest priority pending interrupt source. This value may be used as an address offset for fast, deterministic handling in the interrupt service routine. A read of the IIDX register will clear the corresponding interrupt status in the RIS and MIS registers. 0h = No interrupt pending 1h = LFOSCGOOD interrupt pending 2h = 2 3h = 3 4h = 4 5h = 5 6h = 6 7h = 7 8h = 8

2.6.2 IMASK Register (Offset = 1028h) [Reset = X]

IMASK is shown in [Figure 2-13](#) and described in [Table 2-23](#).

Return to the [Summary Table](#).

SYSCTL interrupt mask

Figure 2-13. IMASK Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED							
R/W-X							
15	14	13	12	11	10	9	8
RESERVED							
R/W-X							
7	6	5	4	3	2	1	0
HSCLKGOOD	SYSPLLGOOD	HFCLKGOOD	LFXTGOOD	SRAMSEC	FLASHSEC	ANACKERR	LFOSCGOOD
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 2-23. IMASK Register Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R/W	X	
7	HSCLKGOOD	R/W	0h	HSCLK GOOD 0h = 0 1h = 1
6	SYSPLLGOOD	R/W	0h	SYSPLL GOOD 0h = 0 1h = 1
5	HFCLKGOOD	R/W	0h	HFCLK GOOD 0h = 0 1h = 1
4	LFXTGOOD	R/W	0h	LFXT GOOD 0h = 0 1h = 1
3	SRAMSEC	R/W	0h	SRAM Single Error Correct 0h = 0 1h = 1
2	FLASHSEC	R/W	0h	Flash Single Error Correct 0h = 0 1h = 1
1	ANACKERR	R/W	0h	Analog Clocking Consistency Error 0h = 0 1h = 1
0	LFOSCGOOD	R/W	0h	Enable or disable the LFOSCGOOD interrupt. LFOSCGOOD indicates that the LFOSC has started successfully. 0h = Interrupt disabled 1h = Interrupt enabled

2.6.3 RIS Register (Offset = 1030h) [Reset = X]

RIS is shown in [Figure 2-14](#) and described in [Table 2-24](#).

Return to the [Summary Table](#).

SYSCTL raw interrupt status

Figure 2-14. RIS Register

31	30	29	28	27	26	25	24
RESERVED							
R-X							
23	22	21	20	19	18	17	16
RESERVED							
R-X							
15	14	13	12	11	10	9	8
RESERVED							
R-X							
7	6	5	4	3	2	1	0
HCLKGOOD	SYSPLLGOOD	HFCLKGOOD	LFXTGOOD	SRAMSEC	FLASHSEC	ANACKERR	LFOSCGOOD
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 2-24. RIS Register Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	X	
7	HCLKGOOD	R	0h	HCLK GOOD 0h = 0 1h = 1
6	SYSPLLGOOD	R	0h	SYSPLL GOOD 0h = 0 1h = 1
5	HFCLKGOOD	R	0h	HFCLK GOOD 0h = 0 1h = 1
4	LFXTGOOD	R	0h	LFXT GOOD 0h = 0 1h = 1
3	SRAMSEC	R	0h	SRAM Single Error Correct 0h = 0 1h = 1
2	FLASHSEC	R	0h	Flash Single Error Correct 0h = 0 1h = 1
1	ANACKERR	R	0h	Analog Clocking Consistency Error 0h = 0 1h = 1
0	LFOSCGOOD	R	0h	Raw status of the LFOSCGOOD interrupt. 0h = No interrupt pending 1h = Interrupt pending

2.6.4 MIS Register (Offset = 1038h) [Reset = X]

MIS is shown in [Figure 2-15](#) and described in [Table 2-25](#).

Return to the [Summary Table](#).

SYSTL masked interrupt status

Figure 2-15. MIS Register

31	30	29	28	27	26	25	24
RESERVED							
R-X							
23	22	21	20	19	18	17	16
RESERVED							
R-X							
15	14	13	12	11	10	9	8
RESERVED							
R-X							
7	6	5	4	3	2	1	0
HCLKGOOD	SYSPLLGOOD	HFCLKGOOD	LFXTGOOD	SRAMSEC	FLASHSEC	ANACKERR	LFOSCGOOD
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 2-25. MIS Register Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	X	
7	HCLKGOOD	R	0h	HCLK GOOD 0h = 0 1h = 1
6	SYSPLLGOOD	R	0h	SYSPLL GOOD 0h = 0 1h = 1
5	HFCLKGOOD	R	0h	HFCLK GOOD 0h = 0 1h = 1
4	LFXTGOOD	R	0h	LFXT GOOD 0h = 0 1h = 1
3	SRAMSEC	R	0h	SRAM Single Error Correct 0h = 0 1h = 1
2	FLASHSEC	R	0h	Flash Single Error Correct 0h = 0 1h = 1
1	ANACKERR	R	0h	Analog Clocking Consistency Error 0h = 0 1h = 1
0	LFOSCGOOD	R	0h	Masked status of the LFOSCGOOD interrupt. 0h = No interrupt pending 1h = Interrupt pending

2.6.5 ISET Register (Offset = 1040h) [Reset = X]

ISET is shown in [Figure 2-16](#) and described in [Table 2-26](#).

Return to the [Summary Table](#).

SYSCTL interrupt set

Figure 2-16. ISET Register

31	30	29	28	27	26	25	24
RESERVED							
W-X							
23	22	21	20	19	18	17	16
RESERVED							
W-X							
15	14	13	12	11	10	9	8
RESERVED							
W-X							
7	6	5	4	3	2	1	0
HCLKGOOD	SYSPLLGOOD	HFCLKGOOD	LFXTGOOD	SRAMSEC	FLASHSEC	ANACKERR	LFOSCGOOD
W1S-0h	W1S-0h	W1S-0h	W1S-0h	W1S-0h	W1S-0h	W1S-0h	W1S-0h

Table 2-26. ISET Register Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	W	X	
7	HCLKGOOD	W1S	0h	HCLK GOOD 0h = 0 1h = 1
6	SYSPLLGOOD	W1S	0h	SYSPLL GOOD 0h = 0 1h = 1
5	HFCLKGOOD	W1S	0h	HFCLK GOOD 0h = 0 1h = 1
4	LFXTGOOD	W1S	0h	LFXT GOOD 0h = 0 1h = 1
3	SRAMSEC	W1S	0h	SRAM Single Error Correct 0h = 0 1h = 1
2	FLASHSEC	W1S	0h	Flash Single Error Correct 0h = 0 1h = 1
1	ANACKERR	W1S	0h	Analog Clocking Consistency Error 0h = 0 1h = 1
0	LFOSCGOOD	W1S	0h	Set the LFOSCGOOD interrupt. 0h = Writing 0h has no effect 1h = Set interrupt

2.6.6 ICLR Register (Offset = 1048h) [Reset = X]

ICLR is shown in [Figure 2-17](#) and described in [Table 2-27](#).

Return to the [Summary Table](#).

SYSCTL interrupt clear

Figure 2-17. ICLR Register

31	30	29	28	27	26	25	24
RESERVED							
W-X							
23	22	21	20	19	18	17	16
RESERVED							
W-X							
15	14	13	12	11	10	9	8
RESERVED							
W-X							
7	6	5	4	3	2	1	0
HSCLKGOOD	SYSPLLGOOD	HFCLKGOOD	LFXTGOOD	SRAMSEC	FLASHSEC	ANACKERR	LFOSCGOOD
W1C-0h	W1C-0h	W1C-0h	W1C-0h	W1C-0h	W1C-0h	W1C-0h	W1C-0h

Table 2-27. ICLR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	W	X	
7	HSCLKGOOD	W1C	0h	HSCLK GOOD 0h = 0 1h = 1
6	SYSPLLGOOD	W1C	0h	SYSPLL GOOD 0h = 0 1h = 1
5	HFCLKGOOD	W1C	0h	HFCLK GOOD 0h = 0 1h = 1
4	LFXTGOOD	W1C	0h	LFXT GOOD 0h = 0 1h = 1
3	SRAMSEC	W1C	0h	SRAM Single Error Correct 0h = 0 1h = 1
2	FLASHSEC	W1C	0h	Flash Single Error Correct 0h = 0 1h = 1
1	ANACKERR	W1C	0h	Analog Clocking Consistency Error 0h = 0 1h = 1
0	LFOSCGOOD	W1C	0h	Clear the LFOSCGOOD interrupt. 0h = Writing 0h has no effect 1h = Clear interrupt

2.6.7 NMIIIDX Register (Offset = 1050h) [Reset = X]

NMIIIDX is shown in [Figure 2-18](#) and described in [Table 2-28](#).

Return to the [Summary Table](#).

NMI interrupt index

Figure 2-18. NMIIIDX Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																												STAT			
R-X																												R-0h			

Table 2-28. NMIIIDX Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	X	
3-0	STAT	R	0h	The NMI interrupt index (NMIIIDX) register generates a value corresponding to the highest priority pending NMI source. This value may be used as an address offset for fast, deterministic handling in the NMI service routine. A read of the NMIIIDX register will clear the corresponding interrupt status in the NMIRIS register. 0h = No NMI pending 1h = BOR Threshold NMI pending 2h = 2 3h = 3 4h = 4 5h = 5 6h = 6

2.6.8 NMIRIS Register (Offset = 1060h) [Reset = X]

NMIRIS is shown in [Figure 2-19](#) and described in [Table 2-29](#).

Return to the [Summary Table](#).

NMI raw interrupt status

Figure 2-19. NMIRIS Register

31	30	29	28	27	26	25	24
RESERVED							
R-X							
23	22	21	20	19	18	17	16
RESERVED							
R-X							
15	14	13	12	11	10	9	8
RESERVED							
R-X							
7	6	5	4	3	2	1	0
RESERVED		SRAMDED	FLASHDED	LFCLKFAIL	WWDT1	WWDT0	BORLVL
R-X		R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 2-29. NMIRIS Register Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R	X	
5	SRAMDED	R	0h	SRAM Double Error Detect 0h = 0 1h = 1
4	FLASHDED	R	0h	Flash Double Error Detect 0h = 0 1h = 1
3	LFCLKFAIL	R	0h	LFXT-EXLF Monitor Fail 0h = 0 1h = 1
2	WWDT1	R	0h	Watch Dog 0 Fault 0h = 0 1h = 1
1	WWDT0	R	0h	Watch Dog 0 Fault 0h = 0 1h = 1
0	BORLVL	R	0h	Raw status of the BORLVL NMI 0h = No interrupt pending 1h = Interrupt pending

2.6.9 NMIISET Register (Offset = 1070h) [Reset = X]

NMIISET is shown in [Figure 2-20](#) and described in [Table 2-30](#).

Return to the [Summary Table](#).

NMI interrupt set

Figure 2-20. NMIISET Register

31	30	29	28	27	26	25	24
RESERVED							
W-X							
23	22	21	20	19	18	17	16
RESERVED							
W-X							
15	14	13	12	11	10	9	8
RESERVED							
W-X							
7	6	5	4	3	2	1	0
RESERVED		SRAMDED	FLASHDED	LFCLKFAIL	WWDT1	WWDT0	BORLVL
W-X		W1S-0h	W1S-0h	W1S-0h	W1S-0h	W1S-0h	W1S-0h

Table 2-30. NMIISET Register Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	W	X	
5	SRAMDED	W1S	0h	SRAM Double Error Detect 0h = 0 1h = 1
4	FLASHDED	W1S	0h	Flash Double Error Detect 0h = 0 1h = 1
3	LFCLKFAIL	W1S	0h	LFXT-EXLF Monitor Fail 0h = 0 1h = 1
2	WWDT1	W1S	0h	Watch Dog 0 Fault 0h = 0 1h = 1
1	WWDT0	W1S	0h	Watch Dog 0 Fault 0h = 0 1h = 1
0	BORLVL	W1S	0h	Set the BORLVL NMI 0h = Writing 0h has no effect 1h = Set interrupt

2.6.10 NMIICLR Register (Offset = 1078h) [Reset = X]

NMIICLR is shown in [Figure 2-21](#) and described in [Table 2-31](#).

Return to the [Summary Table](#).

NMI interrupt clear

Figure 2-21. NMIICLR Register

31	30	29	28	27	26	25	24
RESERVED							
W-X							
23	22	21	20	19	18	17	16
RESERVED							
W-X							
15	14	13	12	11	10	9	8
RESERVED							
W-X							
7	6	5	4	3	2	1	0
RESERVED		SRAMED	FLASHDED	LFCLKFAIL	WWDT1	WWDT0	BORLVL
W-X		W1C-0h	W1C-0h	W1C-0h	W1C-0h	W1C-0h	W1C-0h

Table 2-31. NMIICLR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	W	X	
5	SRAMED	W1C	0h	SRAM Double Error Detect 0h = 0 1h = 1
4	FLASHDED	W1C	0h	Flash Double Error Detect 0h = 0 1h = 1
3	LFCLKFAIL	W1C	0h	LFXT-EXLF Monitor Fail 0h = 0 1h = 1
2	WWDT1	W1C	0h	Watch Dog 0 Fault 0h = 0 1h = 1
1	WWDT0	W1C	0h	Watch Dog 0 Fault 0h = 0 1h = 1
0	BORLVL	W1C	0h	Clear the BORLVL NMI 0h = Writing 0h has no effect 1h = Clear interrupt

2.6.11 SYSOSCCFG Register (Offset = 1100h) [Reset = X]

SYSOSCCFG is shown in [Figure 2-22](#) and described in [Table 2-32](#).

Return to the [Summary Table](#).

SYSOSC configuration

Figure 2-22. SYSOSCCFG Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED						FASTCPUEVENT	BLOCKASYNCALL
R/W-X						R/W-1h	R/W-0h
15	14	13	12	11	10	9	8
RESERVED					DISABLE	DISABLESTOP	USE4MHZSTOP
R/W-X					R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
RESERVED						FREQ	
R/W-X						R/W-0h	

Table 2-32. SYSOSCCFG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-18	RESERVED	R/W	X	
17	FASTCPUEVENT	R/W	1h	FASTCPUEVENT may be used to assert a fast clock request when an interrupt is asserted to the CPU, reducing interrupt latency. 0h = An interrupt to the CPU will not assert a fast clock request 1h = An interrupt to the CPU will assert a fast clock request
16	BLOCKASYNCALL	R/W	0h	BLOCKASYNCALL may be used to mask block all asynchronous fast clock requests, preventing hardware from dynamically changing the active clock configuration when operating in a given mode. 0h = Asynchronous fast clock requests are controlled by the requesting peripheral 1h = All asynchronous fast clock requests are blocked
15-11	RESERVED	R/W	X	
10	DISABLE	R/W	0h	DISABLE sets the SYSOSC enable/disable policy. SYSOSC may be powered off in RUN, SLEEP, and STOP modes to reduce power consumption. When SYSOSC is disabled, MCLK and ULPClk are sourced from LFCLK. 0h = Do not disable SYSOSC 1h = Disable SYSOSC immediately and source MCLK and ULPClk from LFCLK
9	DISABLESTOP	R/W	0h	DISABLESTOP sets the SYSOSC stop mode enable/disable policy. When operating in STOP mode, the SYSOSC may be automatically disabled. When set, ULPClk will run from LFCLK in STOP mode and SYSOSC will be disabled to reduce power consumption. 0h = Do not disable SYSOSC in STOP mode 1h = Disable SYSOSC in STOP mode and source ULPClk from LFCLK
8	USE4MHZSTOP	R/W	0h	USE4MHZSTOP sets the SYSOSC stop mode frequency policy. When entering STOP mode, the SYSOSC frequency may be automatically switched to 4MHz to reduce SYSOSC power consumption. 0h = Do not gear shift the SYSOSC to 4MHz in STOP mode 1h = Gear shift SYSOSC to 4MHz in STOP mode

Table 2-32. SYSOSCCFG Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
7-2	RESERVED	R/W	X	
1-0	FREQ	R/W	0h	Target operating frequency for the system oscillator (SYSOSC) 0h = Base frequency (32MHz) 1h = Low frequency (4MHz) 2h = User-trimmed frequency (16 or 24 MHz)

2.6.12 MCLKCFG Register (Offset = 1104h) [Reset = X]

MCLKCFG is shown in [Figure 2-23](#) and described in [Table 2-33](#).

Return to the [Summary Table](#).

Main clock (MCLK) configuration

Figure 2-23. MCLKCFG Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED	MCLKDEADCHK	STOPCLKSTBY	USELFCLK	RESERVED		USEHSCLK	
R/W-X	R/W-0h	R/W-0h	R/W-0h	R/W-X		R/W-0h	
15	14	13	12	11	10	9	8
RESERVED			USEMFTICK	FLASHWAIT			
R/W-X			R/W-0h	R/W-2h			
7	6	5	4	3	2	1	0
RESERVED		UDIV		MDIV			
R/W-X		R/W-1h		R/W-0h			

Table 2-33. MCLKCFG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-23	RESERVED	R/W	X	
22	MCLKDEADCHK	R/W	0h	MCLKDEADCHK enables or disables the continuous MCLK dead check monitor. LFCLK must be running before MCLKDEADCHK is enabled. 0h = The MCLK dead check monitor is disabled 1h = The MCLK dead check monitor is enabled
21	STOPCLKSTBY	R/W	0h	STOPCLKSTBY sets the STANDBY mode policy (STANDBY0 or STANDBY1). When set, ULPCCLK and LFCLK are disabled to all peripherals in STANDBY mode, with the exception of TIMG0 and TIMG1 which continue to run. Wake-up is only possible via an asynchronous fast clock request. 0h = ULPCCLK/LFCLK runs to all PD0 peripherals in STANDBY mode 1h = ULPCCLK/LFCLK is disabled to all peripherals in STANDBY mode except TIMG0 and TIMG1
20	USELFCLK	R/W	0h	USELFCLK sets the MCLK source policy. Set USELFCLK to use LFCLK as the MCLK source. Note that setting USELFCLK does not disable SYSOSC, and SYSOSC remains available for direct use by analog modules. 0h = MCLK will not use the low frequency clock (LFCLK) 1h = MCLK will use the low frequency clock (LFCLK)
19-17	RESERVED	R/W	X	
16	USEHSCLK	R/W	0h	USEHSCLK, together with USELFCLK, sets the MCLK source policy. Set USEHSCLK to use HSCLK (HFCLK or SYSPLL) as the MCLK source in RUN and SLEEP modes. 0h = MCLK will not use the high speed clock (HSCLK) 1h = MCLK will use the high speed clock (HSCLK) in RUN and SLEEP mode
15-13	RESERVED	R/W	X	

Table 2-33. MCLKCFG Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
12	USEMFTICK	R/W	0h	USEMFTICK specifies whether the 4MHz constant-rate clock (MFCLK) to peripherals is enabled or disabled. When enabled, MDIV must be disabled (set to 0h=/1). 0h = The 4MHz rate MFCLK to peripherals is enabled 1h = The 4MHz rate MFCLK to peripherals is disabled.
11-8	FLASHWAIT	R/W	2h	FLASHWAIT specifies the number of flash wait states when MCLK is sourced from HSCLK. FLASHWAIT has no effect when MCLK is sourced from SYSOSC or LFCLK. 0h = No flash wait states are applied 1h = One flash wait state is applied 2h = 2 flash wait states are applied
7-6	RESERVED	R/W	X	
5-4	UDIV	R/W	1h	UDIV specifies the ULPCLK divider when MCLK is sourced from HSCLK. UDIV has no effect when MCLK is sourced from SYSOSC or LFCLK. 0h = ULPCLK is not divided and is equal to MCLK 1h = ULPCLK is MCLK/2 (divided-by-2) 2h = ULPCLK is MCLK/3 (divided-by-3)
3-0	MDIV	R/W	0h	MDIV may be used to divide the MCLK frequency when MCLK is sourced from SYSOSC. MDIV=0h corresponds to /1 (no divider). MDIV=1h corresponds to /2 (divide-by-2). MDIV=Fh corresponds to /16 (divide-by-16). MDIV may be set between /1 and /16 on an integer basis.

2.6.13 HSCLKEN Register (Offset = 1108h) [Reset = X]

HSCLKEN is shown in [Figure 2-24](#) and described in [Table 2-34](#).

Return to the [Summary Table](#).

High-speed clock (HCLK) source enable/disable

Figure 2-24. HSCLKEN Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED							USEEXTHFCLK
R/W-X							R/W-0h
15	14	13	12	11	10	9	8
RESERVED							SYSPLLEN
R/W-X							R/W-0h
7	6	5	4	3	2	1	0
RESERVED							HFXTEN
R/W-X							R/W-0h

Table 2-34. HSCLKEN Register Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R/W	X	
16	USEEXTHFCLK	R/W	0h	USEEXTHFCLK selects the HFCLK_IN digital clock input to be the source for HFCLK. When disabled, HFXT is the HFCLK source and HFXTEN may be set. Do not set HFXTEN and USEEXTHFCLK simultaneously. 0h = Use HFXT as the HFCLK source 1h = Use the HFCLK_IN digital clock input as the HFCLK source
15-9	RESERVED	R/W	X	
8	SYSPLLEN	R/W	0h	SYSPLLEN enables or disables the system phase-lock loop (SYSPLL). 0h = Disable the SYSPLL 1h = Enable the SYSPLL
7-1	RESERVED	R/W	X	
0	HFXTEN	R/W	0h	HFXTEN enables or disables the high frequency crystal oscillator (HFXT). 0h = Disable the HFXT 1h = Enable the HFXT

2.6.14 HSCLKCFG Register (Offset = 110Ch) [Reset = X]

HSCLKCFG is shown in [Figure 2-25](#) and described in [Table 2-35](#).

Return to the [Summary Table](#).

High-speed clock (HSCLK) source selection

Figure 2-25. HSCLKCFG Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED							
R/W-X							
15	14	13	12	11	10	9	8
RESERVED							
R/W-X							
7	6	5	4	3	2	1	0
RESERVED							HSCLKSEL
R/W-X							R/W-0h

Table 2-35. HSCLKCFG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R/W	X	
0	HSCLKSEL	R/W	0h	HSCLKSEL selects the HSCLK source (SYSPLL or HFCLK). 0h = HSCLK is sourced from the SYSPLL 1h = HSCLK is sourced from the HFCLK

2.6.15 HFCLKCLKCFG Register (Offset = 1110h) [Reset = X]

HFCLKCLKCFG is shown in [Figure 2-26](#) and described in [Table 2-36](#).

Return to the [Summary Table](#).

High-frequency clock (HFCLK) configuration

Figure 2-26. HFCLKCLKCFG Register

31	30	29	28	27	26	25	24
RESERVED			HFCLKFLTCHK	RESERVED			
R/W-X			R/W-1h	R/W-X			
23	22	21	20	19	18	17	16
RESERVED							
R/W-X							
15	14	13	12	11	10	9	8
RESERVED		HFXTRSEL		RESERVED			
R/W-X		R/W-0h		R/W-X			
7	6	5	4	3	2	1	0
HFXTTIME							
R/W-0h							

Table 2-36. HFCLKCLKCFG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R/W	X	
28	HFCLKFLTCHK	R/W	1h	HFCLKFLTCHK enables or disables the HFCLK startup monitor. 0h = HFCLK startup is not checked 1h = HFCLK startup is checked
27-14	RESERVED	R/W	X	
13-12	HFXTRSEL	R/W	0h	HFXT Range Select 0h = 4MHz ≤ HFXT frequency ≤ 8MHz 1h = 8MHz < HFXT frequency ≤ 16MHz 2h = 16MHz < HFXT frequency ≤ 32MHz 3h = 32MHz < HFXT frequency ≤ 48MHz
11-8	RESERVED	R/W	X	
7-0	HFXTTIME	R/W	0h	HFXTTIME specifies the HFXT startup time in 64us resolution. If the HFCLK startup monitor is enabled (HFCLKFLTCHK), HFXT will be checked after this time expires. 0h = Minimum startup time (approximately zero) FFh = Maximum startup time (approximately 16.32ms)

2.6.16 LFCLKCFG Register (Offset = 1114h) [Reset = X]

LFCLKCFG is shown in [Figure 2-27](#) and described in [Table 2-37](#).

Return to the [Summary Table](#).

Low frequency crystal oscillator (LFXT) configuration

Figure 2-27. LFCLKCFG Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED							
R/W-X							
15	14	13	12	11	10	9	8
RESERVED							LOWCAP
R/W-X							R/W-0h
7	6	5	4	3	2	1	0
RESERVED			MONITOR	RESERVED		XT1DRIVE	
R/W-X			R/W-0h	R/W-X		R/W-3h	

Table 2-37. LFCLKCFG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-9	RESERVED	R/W	X	
8	LOWCAP	R/W	0h	LOWCAP controls the low-power LFXT mode. When the LFXT load capacitance is less than 3pf, LOWCAP may be set for reduced power consumption. 0h = LFXT low capacitance mode is disabled 1h = LFXT low capacitance mode is enabled
7-5	RESERVED	R/W	X	
4	MONITOR	R/W	0h	MONITOR enables or disables the LFCLK monitor, which continuously checks LFXT or LFCLK_IN for a clock stuck fault. 0h = Clock monitor is disabled 1h = Clock monitor is enabled
3-2	RESERVED	R/W	X	
1-0	XT1DRIVE	R/W	3h	XT1DRIVE selects the low frequency crystal oscillator (LFXT) drive strength. 0h = Lowest drive and current 1h = Lower drive and current 2h = Higher drive and current 3h = Highest drive and current

2.6.17 SYSPLLCFG0 Register (Offset = 1120h) [Reset = X]

SYSPLLCFG0 is shown in [Figure 2-28](#) and described in [Table 2-38](#).

Return to the [Summary Table](#).

SYSPLL reference and output configuration

Figure 2-28. SYSPLLCFG0 Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED				RDIVCLK2X			
R/W-X				R/W-0h			
15	14	13	12	11	10	9	8
RDIVCLK1				RDIVCLK0			
R/W-0h				R/W-0h			
7	6	5	4	3	2	1	0
RESERVED	ENABLECLK2X	ENABLECLK1	ENABLECLK0	RESERVED		MCLK2XVCO	SYSPLLREF
R/W-X	R/W-0h	R/W-0h	R/W-0h	R/W-X		R/W-0h	R/W-0h

Table 2-38. SYSPLLCFG0 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-20	RESERVED	R/W	X	
19-16	RDIVCLK2X	R/W	0h	RDIVCLK2X sets the final divider for the SYSPLLCLK2X output. 0h = SYSPLLCLK1 is divided by 1 1h = SYSPLLCLK1 is divided by 2 2h = SYSPLLCLK1 is divided by 3 3h = SYSPLLCLK1 is divided by 4 4h = SYSPLLCLK1 is divided by 5 5h = SYSPLLCLK1 is divided by 6 6h = SYSPLLCLK1 is divided by 7 7h = SYSPLLCLK1 is divided by 8 8h = SYSPLLCLK1 is divided by 9 9h = SYSPLLCLK1 is divided by 10 Ah = SYSPLLCLK1 is divided by 11 Bh = SYSPLLCLK1 is divided by 12 Ch = SYSPLLCLK1 is divided by 13 Dh = SYSPLLCLK1 is divided by 14 Eh = SYSPLLCLK1 is divided by 15 Fh = SYSPLLCLK1 is divided by 16
15-12	RDIVCLK1	R/W	0h	RDIVCLK1 sets the final divider for the SYSPLLCLK1 output. 0h = SYSPLLCLK1 is divided by 2 1h = SYSPLLCLK1 is divided by 4 2h = SYSPLLCLK1 is divided by 6 3h = SYSPLLCLK1 is divided by 8 4h = SYSPLLCLK1 is divided by 10 5h = SYSPLLCLK1 is divided by 12 6h = SYSPLLCLK1 is divided by 14 7h = SYSPLLCLK1 is divided by 16 8h = SYSPLLCLK1 is divided by 18 9h = SYSPLLCLK1 is divided by 20 Ah = SYSPLLCLK1 is divided by 22 Bh = SYSPLLCLK1 is divided by 24 Ch = SYSPLLCLK1 is divided by 26 Dh = SYSPLLCLK1 is divided by 28 Eh = SYSPLLCLK1 is divided by 30 Fh = SYSPLLCLK1 is divided by 32

Table 2-38. SYSPLLCFG0 Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11-8	RDIVCLK0	R/W	0h	RDIVCLK0 sets the final divider for the SYSPLLCLK0 output. 0h = SYSPLLCLK0 is divided by 2 1h = SYSPLLCLK0 is divided by 4 2h = SYSPLLCLK0 is divided by 6 3h = SYSPLLCLK0 is divided by 8 4h = SYSPLLCLK0 is divided by 10 5h = SYSPLLCLK0 is divided by 12 6h = SYSPLLCLK0 is divided by 14 7h = SYSPLLCLK0 is divided by 16 8h = SYSPLLCLK0 is divided by 18 9h = SYSPLLCLK0 is divided by 20 Ah = SYSPLLCLK0 is divided by 22 Bh = SYSPLLCLK0 is divided by 24 Ch = SYSPLLCLK0 is divided by 26 Dh = SYSPLLCLK0 is divided by 28 Eh = SYSPLLCLK0 is divided by 30 Fh = SYSPLLCLK0 is divided by 32
7	RESERVED	R/W	X	
6	ENABLECLK2X	R/W	0h	ENABLECLK2X enables or disables the SYSPLLCLK2X output. 0h = SYSPLLCLK2X is disabled 1h = SYSPLLCLK2X is enabled
5	ENABLECLK1	R/W	0h	ENABLECLK1 enables or disables the SYSPLLCLK1 output. 0h = SYSPLLCLK1 is disabled 1h = SYSPLLCLK1 is enabled
4	ENABLECLK0	R/W	0h	ENABLECLK0 enables or disables the SYSPLLCLK0 output. 0h = SYSPLLCLK0 is disabled 1h = SYSPLLCLK0 is enabled
3-2	RESERVED	R/W	X	
1	MCLK2XVCO	R/W	0h	MCLK2XVCO selects the SYSPLL output which is sent to the HSCLK mux for use by MCLK. 0h = The SYSPLLCLK0 output is sent to the HSCLK mux 1h = The SYSPLLCLK2X output is sent to the HSCLK mux
0	SYSPLLREF	R/W	0h	SYSPLLREF selects the system PLL (SYSPLL) reference clock source. 0h = SYSPLL reference is SYSOSC 1h = SYSPLL reference is HFCLK

2.6.18 SYSPLLCFG1 Register (Offset = 1124h) [Reset = X]

SYSPLLCFG1 is shown in [Figure 2-29](#) and described in [Table 2-39](#).

Return to the [Summary Table](#).

SYSPLL reference and feedback divider

Figure 2-29. SYSPLLCFG1 Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-X															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESE RVED	QDIV							RESERVED						PDIV	
R/W-X	R/W-0h							R/W-X						R/W-0h	

Table 2-39. SYSPLLCFG1 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	R/W	X	
14-8	QDIV	R/W	0h	QDIV selects the SYSPLL feedback path divider. 0h = Divide-by-one is not a valid QDIV option 1h = Feedback path is divided by 2 7Eh = Feedback path is divided by 127 (0x7E)
7-2	RESERVED	R/W	X	
1-0	PDIV	R/W	0h	PDIV selects the SYSPLL reference clock prescale divider. 0h = SYSPLLREF is not divided 1h = SYSPLLREF is divided by 2 2h = SYSPLLREF is divided by 4 3h = SYSPLLREF is divided by 8

2.6.19 SYSPLLPARAM0 Register (Offset = 1128h) [Reset = X]

SYSPLLPARAM0 is shown in [Figure 2-30](#) and described in [Table 2-40](#).

Return to the [Summary Table](#).

SYSPLL PARAM0 (load from FACTORY region)

Figure 2-30. SYSPLLPARAM0 Register

31	30	29	28	27	26	25	24
CAPBOVERRI DE	RESERVED				CAPBVAL		
R/W-0h	R/W-X				R/W-0h		
23	22	21	20	19	18	17	16
RESERVED		CPCURRENT					
R/W-X		R/W-0h					
15	14	13	12	11	10	9	8
RESERVED		STARTTIMELP					
R/W-X		R/W-0h					
7	6	5	4	3	2	1	0
RESERVED		STARTTIME					
R/W-X		R/W-0h					

Table 2-40. SYSPLLPARAM0 Register Field Descriptions

Bit	Field	Type	Reset	Description
31	CAPBOVERRIDE	R/W	0h	CAPBOVERRIDE controls the override for Cap B 0h = Cap B override disabled 1h = Cap B override enabled
30-29	RESERVED	R/W	X	
28-24	CAPBVAL	R/W	0h	Override value for Cap B
23-22	RESERVED	R/W	X	
21-16	CPCURRENT	R/W	0h	Charge pump current
15-14	RESERVED	R/W	X	
13-8	STARTTIMELP	R/W	0h	Startup time from low power mode exit to locked clock, in 1us resolution
7-6	RESERVED	R/W	X	
5-0	STARTTIME	R/W	0h	Startup time from enable to locked clock, in 1us resolution

2.6.20 SYSPLLPARAM1 Register (Offset = 112Ch) [Reset = X]

SYSPLLPARAM1 is shown in [Figure 2-31](#) and described in [Table 2-41](#).

Return to the [Summary Table](#).

SYSPLL PARAM1 (load from FACTORY region)

Figure 2-31. SYSPLLPARAM1 Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LPFRESC								RESERVED							
R/W-0h								R/W-X							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LPFRESA								RESERVED				LPFCAPA			
R/W-0h								R/W-X				R/W-0h			

Table 2-41. SYSPLLPARAM1 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	LPFRESC	R/W	0h	Loop filter Res C
23-18	RESERVED	R/W	X	
17-8	LPFRESA	R/W	0h	Loop filter Res A
7-5	RESERVED	R/W	X	
4-0	LPFCAPA	R/W	0h	Loop filter Cap A

2.6.21 GENCLKCFG Register (Offset = 1138h) [Reset = X]

GENCLKCFG is shown in [Figure 2-32](#) and described in [Table 2-42](#).

Return to the [Summary Table](#).

General clock configuration

Figure 2-32. GENCLKCFG Register

31	30	29	28	27	26	25	24
RESERVED				FCCTRICNT			
R/W-X				R/W-0h			
23	22	21	20	19	18	17	16
ANACPUMPCFG		FCCLVLTRIG	FCCTRIGSRC	FCCSELCLK			
R/W-0h		R/W-0h	R/W-0h	R/W-0h			
15	14	13	12	11	10	9	8
HFCLK4MFPCLKDIV				RESERVED		MFPCLKSRC	CANCLKSRC
R/W-0h				R/W-X		R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
EXCLKDIVEN	EXCLKDIVVAL			RESERVED	EXCLKSRC		
R/W-0h	R/W-0h			R/W-X	R/W-0h		

Table 2-42. GENCLKCFG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R/W	X	
28-24	FCCTRICNT	R/W	0h	FCCTRICNT specifies the number of trigger clock periods in the trigger window. FCCTRICNT=0h (one trigger clock period) up to 1Fh (32 trigger clock periods) may be specified.
23-22	ANACPUMPCFG	R/W	0h	ANACPUMPCFG selects the analog mux charge pump (VBOOST) enable method. 0h = VBOOST is enabled on request from a COMP, GPAMP, or OPA 1h = VBOOST is enabled when the device is in RUN or SLEEP mode, or when a COMP/GPAMP/OPA is enabled 2h = VBOOST is always enabled
21	FCCLVLTRIG	R/W	0h	FCCLVLTRIG selects the frequency clock counter (FCC) trigger mode. 0h = Rising edge to rising edge triggered 1h = Level triggered
20	FCCTRIGSRC	R/W	0h	FCCTRIGSRC selects the frequency clock counter (FCC) trigger source. 0h = FCC trigger is the external pin 1h = FCC trigger is the LFCLK
19-16	FCCSELCLK	R/W	0h	FCCSELCLK selects the frequency clock counter (FCC) clock source. 0h = FCC clock is MCLK 1h = FCC clock is SYSOSC 2h = FCC clock is HFCLK 3h = FCC clock is the CLK_OUT selection 4h = FCC clock is SYSPLLCLK0 5h = FCC clock is SYSPLLCLK1 6h = FCC clock is SYSPLLCLK2X 7h = FCC clock is the FCCIN external input

Table 2-42. GENCLKCFG Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
15-12	HFCLK4MFPCLKDIV	R/W	0h	HFCLK4MFPCLKDIV selects the divider applied to HFCLK when HFCLK is used as the MFPCLK source. Integer dividers from /1 to /16 may be selected. 0h = HFCLK is not divided before being used for MFPCLK 1h = HFCLK is divided by 2 before being used for MFPCLK 2h = HFCLK is divided by 3 before being used for MFPCLK 3h = HFCLK is divided by 4 before being used for MFPCLK 4h = HFCLK is divided by 5 before being used for MFPCLK 5h = HFCLK is divided by 6 before being used for MFPCLK 6h = HFCLK is divided by 7 before being used for MFPCLK 7h = HFCLK is divided by 8 before being used for MFPCLK 8h = HFCLK is divided by 9 before being used for MFPCLK 9h = HFCLK is divided by 10 before being used for MFPCLK Ah = HFCLK is divided by 11 before being used for MFPCLK Bh = HFCLK is divided by 12 before being used for MFPCLK Ch = HFCLK is divided by 13 before being used for MFPCLK Dh = HFCLK is divided by 14 before being used for MFPCLK Eh = HFCLK is divided by 15 before being used for MFPCLK Fh = HFCLK is divided by 16 before being used for MFPCLK
11-10	RESERVED	R/W	X	
9	MFPCLKSRC	R/W	0h	MFPCLKSRC selects the MFPCLK (middle frequency precision clock) source. 0h = MFPCLK is sourced from SYSOSC 1h = MFPCLK is sourced from HFCLK
8	CANCLKSRC	R/W	0h	CANCLKSRC selects the CANCLK source. 0h = CANCLK source is HFCLK 1h = CANCLK source is SYSPLLCLK1
7	EXCLKDIVEN	R/W	0h	EXCLKDIVEN enables or disables the divider function of the CLK_OUT external clock output block. 0h = Clock divider is disabled (pass through, EXCLKDIVVAL is not applied) 1h = Clock divider is enabled (EXCLKDIVVAL is applied)
6-4	EXCLKDIVVAL	R/W	0h	EXCLKDIVVAL selects the divider value for the divider in the CLK_OUT external clock output block. 0h = CLK_OUT source is divided by 2 1h = CLK_OUT source is divided by 4 2h = CLK_OUT source is divided by 6 3h = CLK_OUT source is divided by 8 4h = CLK_OUT source is divided by 10 5h = CLK_OUT source is divided by 12 6h = CLK_OUT source is divided by 14 7h = CLK_OUT source is divided by 16
3	RESERVED	R/W	X	
2-0	EXCLKSRC	R/W	0h	EXCLKSRC selects the source for the CLK_OUT external clock output block. ULPCLK and MFPCLK require the CLK_OUT divider (EXCLKDIVEN) to be enabled 0h = CLK_OUT is SYSOSC 1h = CLK_OUT is ULPCLK (EXCLKDIVEN must be enabled) 2h = CLK_OUT is LFCLK 3h = CLK_OUT is MFPCLK (EXCLKDIVEN must be enabled) 4h = CLK_OUT is HFCLK 5h = CLK_OUT is SYSPLLCLK1 (SYSPLLCLK1 must be ≤48MHz)

2.6.22 GENCLKEN Register (Offset = 113Ch) [Reset = X]

GENCLKEN is shown in [Figure 2-33](#) and described in [Table 2-43](#).

Return to the [Summary Table](#).

General clock enable control

Figure 2-33. GENCLKEN Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED							
R/W-X							
15	14	13	12	11	10	9	8
RESERVED							
R/W-X							
7	6	5	4	3	2	1	0
RESERVED			MFPCLKEN	RESERVED			EXCLKEN
R/W-X			R/W-0h	R/W-X			R/W-0h

Table 2-43. GENCLKEN Register Field Descriptions

Bit	Field	Type	Reset	Description
31-5	RESERVED	R/W	X	
4	MFPCLKEN	R/W	0h	MFPCLKEN enables the middle frequency precision clock (MFPCLK). 0h = MFPCLK is disabled 1h = MFPCLK is enabled
3-1	RESERVED	R/W	X	
0	EXCLKEN	R/W	0h	EXCLKEN enables the CLK_OUT external clock output block. 0h = CLK_OUT block is disabled 1h = CLK_OUT block is enabled

2.6.23 PMODECFG Register (Offset = 1140h) [Reset = X]

PMODECFG is shown in [Figure 2-34](#) and described in [Table 2-44](#).

Return to the [Summary Table](#).

Power mode configuration

Figure 2-34. PMODECFG Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-X															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED													DSLEEP		
R/W-X													R/W-0h		

Table 2-44. PMODECFG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R/W	X	
1-0	DSLEEP	R/W	0h	DSLEEP selects the operating mode to enter upon a DEEPSLEEP request from the CPU. 0h = STOP mode is entered 1h = STANDBY mode is entered 2h = SHUTDOWN mode is entered 3h = Reserved

2.6.24 FCC Register (Offset = 1150h) [Reset = X]

FCC is shown in [Figure 2-35](#) and described in [Table 2-45](#).

Return to the [Summary Table](#).

Frequency clock counter (FCC) count

Figure 2-35. FCC Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED											DATA																				
R-X											R-0h																				

Table 2-45. FCC Register Field Descriptions

Bit	Field	Type	Reset	Description
31-22	RESERVED	R	X	
21-0	DATA	R	0h	Frequency clock counter (FCC) count value.

2.6.25 SYSOSCTRIMUSER Register (Offset = 1170h) [Reset = X]

SYSOSCTRIMUSER is shown in [Figure 2-36](#) and described in [Table 2-46](#).

Return to the [Summary Table](#).

SYSOSC user-specified trim

Figure 2-36. SYSOSCTRIMUSER Register

31	30	29	28	27	26	25	24
RESERVED				RDIV			
R/W-X				R/W-0h			
23	22	21	20	19	18	17	16
RDIV				RESFINE			
R/W-0h				R/W-0h			
15	14	13	12	11	10	9	8
RESERVED				RESCOARSE			
R/W-X				R/W-0h			
7	6	5	4	3	2	1	0
RESERVED		CAP		RESERVED		FREQ	
R/W-X		R/W-0h		R/W-X		R/W-0h	

Table 2-46. SYSOSCTRIMUSER Register Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R/W	X	
28-20	RDIV	R/W	0h	RDIV specifies the frequency correction loop (FCL) resistor trim. This value changes with the target frequency.
19-16	RESFINE	R/W	0h	RESFINE specifies the resistor fine trim. This value changes with the target frequency.
15-14	RESERVED	R/W	X	
13-8	RESCOARSE	R/W	0h	RESCOARSE specifies the resistor coarse trim. This value changes with the target frequency.
7	RESERVED	R/W	X	
6-4	CAP	R/W	0h	CAP specifies the SYSOSC capacitor trim. This value changes with the target frequency.
3-2	RESERVED	R/W	X	
1-0	FREQ	R/W	0h	FREQ specifies the target user-trimmed frequency for SYSOSC. 0h = Reserved 1h = 16MHz user frequency 2h = 24MHz user frequency 3h = Reserved

2.6.26 SRAMBOUNDARY Register (Offset = 1178h) [Reset = X]

SRAMBOUNDARY is shown in [Figure 2-37](#) and described in [Table 2-47](#).

Return to the [Summary Table](#).

SRAM Write Boundary

Figure 2-37. SRAMBOUNDARY Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												ADDR												RESERVED							
R/W-X												R/W-0h												R/W-X							

Table 2-47. SRAMBOUNDARY Register Field Descriptions

Bit	Field	Type	Reset	Description
31-20	RESERVED	R/W	X	
19-5	ADDR	R/W	0h	SRAM boundary configuration. The value configured into this acts such that: SRAM accesses to addresses less than or equal value will be RW only. SRAM accesses to addresses greater than value will be RX only. Value of 0 is not valid (system will have no stack). If set to 0, the system acts as if the entire SRAM is RWX. Any non-zero value can be configured, including a value = SRAM size.
4-0	RESERVED	R/W	X	

2.6.27 SYSTEMCFG Register (Offset = 1180h) [Reset = X]

SYSTEMCFG is shown in [Figure 2-38](#) and described in [Table 2-48](#).

Return to the [Summary Table](#).

System configuration

Figure 2-38. SYSTEMCFG Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-X							
15	14	13	12	11	10	9	8
RESERVED							
R/W-X							
7	6	5	4	3	2	1	0
RESERVED					FLASHECCRS TDIS	WWDTLP1RST DIS	WWDTLP0RST DIS
R/W-X					R/W-1h	R/W-1h	R/W-0h

Table 2-48. SYSTEMCFG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	The key value of 1Bh (27) must be written to KEY together with contents to be updated. Reads as 0 1Bh = Issue write
23-3	RESERVED	R/W	X	
2	FLASHECCRSTDIS	R/W	1h	FLASHECCRSTDIS specifies whether a flash ECC double error detect (DED) will trigger a SYSRST or an NMI. 0h = Flash ECC DED will trigger a SYSRST 1h = Flash ECC DED will trigger a NMI
1	WWDTLP1RSTDIS	R/W	1h	WWDTLP1RSTDIS specifies whether a WWDT Error Event will trigger a SYSRST or an NMI. 0h = WWDTLP1 Error Event will trigger a SYSRST 1h = WWDTLP1 Error Event will trigger an NMI
0	WWDTLP0RSTDIS	R/W	0h	WWDTLP0RSTDIS specifies whether a WWDT Error Event will trigger a BOOTRST or an NMI. 0h = WWDTLP0 Error Event will trigger a BOOTRST 1h = WWDTLP0 Error Event will trigger an NMI

2.6.28 WRITELOCK Register (Offset = 1200h) [Reset = X]

WRITELOCK is shown in [Figure 2-39](#) and described in [Table 2-49](#).

Return to the [Summary Table](#).

SYSCTL register write lockout

Figure 2-39. WRITELOCK Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED							
R/W-X							
15	14	13	12	11	10	9	8
RESERVED							
R/W-X							
7	6	5	4	3	2	1	0
RESERVED							ACTIVE
R/W-X							R/W-0h

Table 2-49. WRITELOCK Register Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R/W	X	
0	ACTIVE	R/W	0h	ACTIVE controls whether critical SYSCTL registers are write protected or not. 0h = Allow writes to lockable registers 1h = Disallow writes to lockable registers

2.6.29 CLKSTATUS Register (Offset = 1204h) [Reset = X]

CLKSTATUS is shown in [Figure 2-40](#) and described in [Table 2-50](#).

Return to the [Summary Table](#).

Clock module (CKM) status

Figure 2-40. CLKSTATUS Register

31	30	29	28	27	26	25	24
ANACKERR	OPAMPCLKERR R	SYSPLLCLKUPD D	HFCLKCLKUPD D	RESERVED		FCCDONE	FCLMODE
R-0h	R-0h	R-0h	R-0h	R-X		R-0h	R-0h
23	22	21	20	19	18	17	16
LFCLKFAIL	RESERVED	HSCLKGOOD	HSCLKDEAD	RESERVED		CURMCLKSEL	CURHSCLKSEL
R-0h	R-X	R-0h	R-0h	R-X		R-0h	R-0h
15	14	13	12	11	10	9	8
RESERVED	SYSPLLOFF	HFCLKOFF	HSCLKSOFF	LFOSCGOOD	LFXTGOOD	SYSPLLG00D	HFCLKGOOD
R-X	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
LFCLKMUX		RESERVED	HSCLKMUX	RESERVED		SYSOSCFREQ	
R-0h		R-X	R-0h	R-X		R-0h	

Table 2-50. CLKSTATUS Register Field Descriptions

Bit	Field	Type	Reset	Description
31	ANACKERR	R	0h	ANACKERR is set when the device clock configuration does not support an enabled analog peripheral mode and the analog peripheral may not be functioning as expected. 0h = No analog clock errors detected 1h = Analog clock error detected
30	OPAMPCLKERR	R	0h	OPAMPCLKERR is set when the device clock configuration does not support an enabled OPA mode and the OPA may not be functioning as expected. 0h = No OPA clock generation errors detected 1h = OPA clock generation error detected
29	SYSPLLCLKUPD	R	0h	SYSPLLCLKUPD indicates when writes to SYSPLLCFG0/1 and SYSPLLPARAM0/1 are blocked. 0h = writes to SYSPLLCFG0/1 and SYSPLLPARAM0/1 are allowed 1h = writes to SYSPLLCFG0/1 and SYSPLLPARAM0/1 are blocked
28	HFCLKCLKUPD	R	0h	HFCLKCLKUPD indicates when writes to the HFCLKCLKCFG register are blocked. 0h = Writes to HFCLKCLKCFG are allowed 1h = Writes to HFCLKCLKCFG are blocked
27-26	RESERVED	R	X	
25	FCCDONE	R	0h	FCCDONE indicates when a frequency clock counter capture is complete. 0h = FCC capture is not done 1h = FCC capture is done
24	FCLMODE	R	0h	FCLMODE indicates if the SYSOSC frequency correction loop (FCL) is enabled. 0h = SYSOSC FCL is disabled 1h = SYSOSC FCL is enabled
23	LFCLKFAIL	R	0h	LFCLKFAIL indicates when the continuous LFCLK monitor detects a LFXT or LFCLK_IN clock stuck failure. 0h = No LFCLK fault detected 1h = LFCLK stuck fault detected

Table 2-50. CLKSTATUS Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
22	RESERVED	R	X	
21	HSCLKGOOD	R	0h	HSCLKGOOD is set by hardware if the selected clock source for HSCLK started successfully. 0h = The HSCLK source did not start correctly 1h = The HSCLK source started correctly
20	HSCLKDEAD	R	0h	HSCLKDEAD is set by hardware if the selected source for HSCLK was started but did not start successfully. 0h = The HSCLK source was not started or started correctly 1h = The HSCLK source did not start correctly
19-18	RESERVED	R	X	
17	CURMCLKSEL	R	0h	CURMCLKSEL indicates if MCLK is currently sourced from LFCLK. 0h = MCLK is not sourced from LFCLK 1h = MCLK is sourced from LFCLK
16	CURHSCLKSEL	R	0h	CURHSCLKSEL indicates the current clock source for HSCLK. 0h = HSCLK is currently sourced from the SYSPLL 1h = HSCLK is currently sourced from the HFCLK
15	RESERVED	R	X	
14	SYSPLLOFF	R	0h	SYSPLLOFF indicates if the SYSPLL is disabled or was dead at startup. When the SYSPLL is started, SYSPLLOFF is cleared by hardware. Following startup of the SYSPLL, if the SYSPLL startup monitor determines that the SYSPLL was not started correctly, SYSPLLOFF is set. 0h = SYSPLL started correctly and is enabled 1h = SYSPLL is disabled or was dead startup
13	HFCLKOFF	R	0h	HFCLKOFF indicates if the HFCLK is disabled or was dead at startup. When the HFCLK is started, HFCLKOFF is cleared by hardware. Following startup of the HFCLK, if the HFCLK startup monitor determines that the HFCLK was not started correctly, HFCLKOFF is set. 0h = HFCLK started correctly and is enabled 1h = HFCLK is disabled or was dead at startup
12	HSCLKSOFF	R	0h	HSCLKSOFF is set when the high speed clock sources (SYSPLL, HFCLK) are disabled or dead. It is the logical AND of HFCLKOFF and SYSPLLOFF. 0h = SYSPLL, HFCLK, or both were started correctly and remain enabled 1h = SYSPLL and HFCLK are both either off or dead
11	LFOSCGOOD	R	0h	LFOSCGOOD indicates when the LFOSC startup has completed and the LFOSC is ready for use. 0h = LFOSC is not ready 1h = LFOSC is ready
10	LFXTGOOD	R	0h	LFXTGOOD indicates if the LFXT started correctly. When the LFXT is started, LFXTGOOD is cleared by hardware. After the startup settling time has expired, the LFXT status is tested. If the LFXT started successfully the LFXTGOOD bit is set, else it is left cleared. 0h = LFXT did not start correctly 1h = LFXT started correctly
9	SYSPLLGOOD	R	0h	SYSPLLGOOD indicates if the SYSPLL started correctly. When the SYSPLL is started, SYSPLLGOOD is cleared by hardware. After the startup settling time has expired, the SYSPLL status is tested. If the SYSPLL started successfully the SYSPLLGOOD bit is set, else it is left cleared. 0h = SYSPLL did not start correctly 1h = SYSPLL started correctly

Table 2-50. CLKSTATUS Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
8	HFCLKGOOD	R	0h	HFCLKGOOD indicates that the HFCLK started correctly. When the HFXT is started or HFCLK_IN is selected as the HFCLK source, this bit will be set by hardware if a valid HFCLK is detected, and cleared if HFCLK is not operating within the expected range. 0h = HFCLK did not start correctly 1h = HFCLK started correctly
7-6	LFCLKMUX	R	0h	LFCLKMUX indicates if LFCLK is sourced from the internal LFOSC, the low frequency crystal (LFXT), or the LFCLK_IN digital clock input. 0h = LFCLK is sourced from the internal LFOSC 1h = LFCLK is sourced from the LFXT (crystal) 2h = LFCLK is sourced from LFCLK_IN (external digital clock input)
5	RESERVED	R	X	
4	HSCLKMUX	R	0h	HSCLKMUX indicates if MCLK is currently sourced from the high-speed clock (HSCLK). 0h = MCLK is not sourced from HSCLK 1h = MCLK is sourced from HSCLK
3-2	RESERVED	R	X	
1-0	SYSOSCFREQ	R	0h	SYSOSCFREQ indicates the current SYSOSC operating frequency. 0h = SYSOSC is at base frequency (32MHz) 1h = SYSOSC is at low frequency (4MHz) 2h = SYSOSC is at the user-trimmed frequency (16 or 24MHz) 3h = Reserved

2.6.30 SYSSTATUS Register (Offset = 1208h) [Reset = X]

SYSSTATUS is shown in [Figure 2-41](#) and described in [Table 2-51](#).

Return to the [Summary Table](#).

System status information

Figure 2-41. SYSSTATUS Register

31	30	29	28	27	26	25	24
REBOOTATTEMPTS		RESERVED					
R-0h		R-X					
23	22	21	20	19	18	17	16
RESERVED							
R-X							
15	14	13	12	11	10	9	8
RESERVED	SHDNIOLOCK	SWDCFGDIS	EXTRSTPINDIS	RESERVED		MCAN0READY	
R-X	R-0h	R-0h	R-0h	R-X		R-0h	
7	6	5	4	3	2	1	0
RESERVED	PMUIREFGOOD	ANACPUMPGOOD	BORLVL	BORCURTHRESHOLD		FLASHSEC	FLASHDED
R-X	R-0h	R-0h	R-0h	R-0h		R-0h	R-0h

Table 2-51. SYSSTATUS Register Field Descriptions

Bit	Field	Type	Reset	Description
31-30	REBOOTATTEMPTS	R	0h	REBOOTATTEMPTS indicates the number of boot attempts taken before the user application starts.
29-15	RESERVED	R	X	
14	SHDNIOLOCK	R	0h	SHDNIOLOCK indicates when IO is locked due to SHUTDOWN 0h = IO IS NOT Locked due to SHUTDOWN 1h = IO IS Locked due to SHUTDOWN
13	SWDCFGDIS	R	0h	SWDCFGDIS indicates when user has disabled the use of SWD Port 0h = SWD Port Enabled 1h = SWD Port Disabled
12	EXTRSTPINDIS	R	0h	EXTRSTPINDIS indicates when user has disabled the use of external reset pin 0h = External Reset Pin Enabled 1h = External Reset Pin Disabled
11-9	RESERVED	R	X	
8	MCAN0READY	R	0h	MCAN0READY indicates when the MCAN0 peripheral is ready. 0h = MCAN0 is not ready 1h = MCAN0 is ready
7	RESERVED	R	X	
6	PMUIREFGOOD	R	0h	PMUIREFGOOD is set by hardware when the PMU current reference is ready. 0h = IREF is not ready 1h = IREF is ready
5	ANACPUMPGOOD	R	0h	ANACPUMPGOOD is set by hardware when the VBOOST analog mux charge pump is ready. 0h = VBOOST is not ready 1h = VBOOST is ready

Table 2-51. SYSSTATUS Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
4	BORLVL	R	0h	BORLVL indicates if a BOR event occurred and the BOR threshold was switched to BOR0 by hardware. 0h = No BOR violation occurred 1h = A BOR violation occurred and the BOR threshold was switched to BOR0
3-2	BORCURTHRESHOLD	R	0h	BORCURTHRESHOLD indicates the active brown-out reset supply monitor configuration. 0h = Default minimum threshold; a BOR0- violation triggers a BOR 1h = A BOR1- violation generates a BORLVL interrupt 2h = A BOR2- violation generates a BORLVL interrupt 3h = A BOR3- violation generates a BORLVL interrupt
1	FLASHSEC	R	0h	FLASHSEC indicates if a flash ECC single bit error was detected and corrected (SEC). 0h = No flash ECC single bit error detected 1h = Flash ECC single bit error was detected and corrected
0	FLASHDED	R	0h	FLASHDED indicates if a flash ECC double bit error was detected (DED). 0h = No flash ECC double bit error detected 1h = Flash ECC double bit error detected

2.6.31 DEDERRADDR Register (Offset = 120Ch) [Reset = 00000000h]

DEDERRADDR is shown in [Figure 2-42](#) and described in [Table 2-52](#).

Return to the [Summary Table](#).

Memory DED Address

Figure 2-42. DEDERRADDR Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADDR																															
R-0h																															

Table 2-52. DEDERRADDR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-0	ADDR	R	0h	Address of MEMORY DED Error.

2.6.32 RSTCAUSE Register (Offset = 1220h) [Reset = X]

RSTCAUSE is shown in [Figure 2-43](#) and described in [Table 2-53](#).

Return to the [Summary Table](#).

Reset cause

Figure 2-43. RSTCAUSE Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																												ID			
R-X																												RC-0h			

Table 2-53. RSTCAUSE Register Field Descriptions

Bit	Field	Type	Reset	Description
31-5	RESERVED	R	X	
4-0	ID	RC	0h	ID is a read-to-clear field which indicates the lowest level reset cause since the last read. 0h = No reset since last read 1h = POR- violation, SHUTDOWNSTOREx or PMU trim parity fault 2h = NRST triggered POR (>1s hold) 3h = Software triggered POR 4h = BOR0- violation 5h = SHUTDOWN mode exit 8h = Non-PMU trim parity fault 9h = Fatal clock failure Ah = Software triggered BOOTRST Ch = NRST triggered BOOTRST (<1s hold) 10h = BSL exit 11h = BSL entry 12h = WWDT0 violation 13h = WWDT1 violation 14h = Flash uncorrectable ECC error 15h = CPULOCK violation 1Ah = Debug triggered SYSRST 1Bh = Software triggered SYSRST 1Ch = Debug triggered CPURST 1Dh = Software triggered CPURST

2.6.33 RESETLEVEL Register (Offset = 1300h) [Reset = X]

RESETLEVEL is shown in [Figure 2-44](#) and described in [Table 2-54](#).

Return to the [Summary Table](#).

Reset level for application-triggered reset command

Figure 2-44. RESETLEVEL Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-X															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												LEVEL			
R/W-X												R/W-0h			

Table 2-54. RESETLEVEL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R/W	X	
2-0	LEVEL	R/W	0h	LEVEL is used to specify the type of reset to be issued when RESETCMD is set to generate a software triggered reset. 0h = Issue a SYSRST (CPU plus peripherals only) 1h = Issue a BOOTRST (CPU, peripherals, and boot configuration routine) 2h = Issue a SYSRST and enter the boot strap loader (BSL) 3h = Issue a power-on reset (POR) 4h = Issue a SYSRST and exit the boot strap loader (BSL)

2.6.34 RESETCMD Register (Offset = 1304h) [Reset = X]

RESETCMD is shown in [Figure 2-45](#) and described in [Table 2-55](#).

Return to the [Summary Table](#).

Execute an application-triggered reset command

Figure 2-45. RESETCMD Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEY								RESERVED							
W-0h								W-X							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED															GO
W-X															W-0h

Table 2-55. RESETCMD Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	The key value of E4h (228) must be written to KEY together with GO to trigger the reset. E4h = Issue reset
23-1	RESERVED	W	X	
0	GO	W	0h	Execute the reset specified in RESETLEVEL.LEVEL. Must be written together with the KEY. 1h = Issue reset

2.6.35 BORTHRESHOLD Register (Offset = 1308h) [Reset = X]

BORTHRESHOLD is shown in [Figure 2-46](#) and described in [Table 2-56](#).

Return to the [Summary Table](#).

BOR threshold selection

Figure 2-46. BORTHRESHOLD Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-X															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED													LEVEL		
R/W-X													R/W-0h		

Table 2-56. BORTHRESHOLD Register Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R/W	X	
1-0	LEVEL	R/W	0h	LEVEL specifies the desired BOR threshold and BOR mode. 0h = Default minimum threshold; a BOR0- violation triggers a BOR 1h = A BOR1- violation generates a BORLVL interrupt 2h = A BOR2- violation generates a BORLVL interrupt 3h = A BOR3- violation generates a BORLVL interrupt

2.6.36 BORCLRCMD Register (Offset = 130Ch) [Reset = X]

BORCLRCMD is shown in [Figure 2-47](#) and described in [Table 2-57](#).

Return to the [Summary Table](#).

Set the BOR threshold

Figure 2-47. BORCLRCMD Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEY								RESERVED							
W-0h								W-X							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED															GO
W-X															W-0h

Table 2-57. BORCLRCMD Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	The key value of C7h (199) must be written to KEY together with GO to trigger the clear and BOR threshold change. C7h = Issue clear
23-1	RESERVED	W	X	
0	GO	W	0h	GO clears any prior BOR violation status indications and attempts to change the active BOR mode to that specified in the LEVEL field of the BORTHRESHOLD register. 1h = Issue clear

2.6.37 SYSOSCFCLCTL Register (Offset = 1310h) [Reset = X]

SYSOSCFCLCTL is shown in [Figure 2-48](#) and described in [Table 2-58](#).

Return to the [Summary Table](#).

SYSOSC frequency correction loop (FCL) ROSC enable

Figure 2-48. SYSOSCFCLCTL Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-X							
15	14	13	12	11	10	9	8
RESERVED							
W-X							
7	6	5	4	3	2	1	0
RESERVED						SETUSEEXRES	SETUSEFCL
W-X						W-0h	W-0h

Table 2-58. SYSOSCFCLCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	The key value of 2Ah (42) must be written to KEY together with SETUSEFCL to enable the FCL. 2Ah = Issue Command
23-2	RESERVED	W	X	
1	SETUSEEXRES	W	0h	Set SETUSEEXRES to specify that an external resistor will be used for the FCL. An appropriate resistor must be populated on the ROSC pin. This state is locked until the next BOOTRST. 1h = Enable the SYSOSC external Resistor
0	SETUSEFCL	W	0h	Set SETUSEFCL to enable the frequency correction loop in SYSOSC. Once enabled, this state is locked until the next BOOTRST. 1h = Enable the SYSOSC FCL

2.6.38 LFXTCTL Register (Offset = 1314h) [Reset = X]

LFXTCTL is shown in [Figure 2-49](#) and described in [Table 2-59](#).

Return to the [Summary Table](#).

LFXT and LFCLK control

Figure 2-49. LFXTCTL Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-X							
15	14	13	12	11	10	9	8
RESERVED							
W-X							
7	6	5	4	3	2	1	0
RESERVED						SETUSELFXT	STARTLFXT
W-X						W-0h	W-0h

Table 2-59. LFXTCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	The key value of 91h (145) must be written to KEY together with either STARTLFXT or SETUSELFXT to set the corresponding bit. 91h = Issue command
23-2	RESERVED	W	X	
1	SETUSELFXT	W	0h	Set SETUSELFXT to switch LFCLK to LFXT. Once set, SETUSELFXT remains set until the next BOOTRST. 0h = 0 1h = Use LFXT as the LFCLK source
0	STARTLFXT	W	0h	Set STARTLFXT to start the low frequency crystal oscillator (LFXT). Once set, STARTLFXT remains set until the next BOOTRST. 0h = LFXT not started 1h = Start LFXT

2.6.39 EXLFCTL Register (Offset = 1318h) [Reset = X]

EXLFCTL is shown in [Figure 2-50](#) and described in [Table 2-60](#).

Return to the [Summary Table](#).

LFCLK_IN and LFCLK control

Figure 2-50. EXLFCTL Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-X							
15	14	13	12	11	10	9	8
RESERVED							
W-X							
7	6	5	4	3	2	1	0
RESERVED							SETUSEEXLF
W-X							W-0h

Table 2-60. EXLFCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	The key value of 36h (54) must be written to KEY together with SETUSEEXLF to set SETUSEEXLF. 36h = Issue command
23-1	RESERVED	W	X	
0	SETUSEEXLF	W	0h	Set SETUSEEXLF to switch LFCLK to the LFCLK_IN digital clock input. Once set, SETUSEEXLF remains set until the next BOOTRST. 1h = Use LFCLK_IN as the LFCLK source

2.6.40 SHDNIORL Register (Offset = 131Ch) [Reset = X]

SHDNIORL is shown in [Figure 2-51](#) and described in [Table 2-61](#).

Return to the [Summary Table](#).

SHUTDOWN IO release control

Figure 2-51. SHDNIORL Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-X							
15	14	13	12	11	10	9	8
RESERVED							
W-X							
7	6	5	4	3	2	1	0
RESERVED							RELEASE
W-X							W-0h

Table 2-61. SHDNIORL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	The key value 91h must be written to KEY together with RELEASE to set RELEASE. 91h = Issue command
23-1	RESERVED	W	X	
0	RELEASE	W	0h	Set RELEASE to release the IO after a SHUTDOWN mode exit. 1h = Release IO

2.6.41 EXRSTPIN Register (Offset = 1320h) [Reset = X]

EXRSTPIN is shown in [Figure 2-52](#) and described in [Table 2-62](#).

Return to the [Summary Table](#).

Disable the reset function of the NRST pin

Figure 2-52. EXRSTPIN Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-X							
15	14	13	12	11	10	9	8
RESERVED							
W-X							
7	6	5	4	3	2	1	0
RESERVED							DISABLE
W-X							W-0h

Table 2-62. EXRSTPIN Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	The key value 1Eh must be written together with DISABLE to disable the reset function. 1Eh = Issue command
23-1	RESERVED	W	X	
0	DISABLE	W	0h	Set DISABLE to disable the reset function of the NRST pin. Once set, this configuration is locked until the next POR. 0h = Reset function of NRST pin is enabled 1h = Reset function of NRST pin is disabled

2.6.42 SYSSTATUSCLR Register (Offset = 1324h) [Reset = X]

SYSSTATUSCLR is shown in [Figure 2-53](#) and described in [Table 2-63](#).

Return to the [Summary Table](#).

Clear sticky bits of SYSSTATUS

Figure 2-53. SYSSTATUSCLR Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-X							
15	14	13	12	11	10	9	8
RESERVED							
W-X							
7	6	5	4	3	2	1	0
RESERVED							ALLECC
W-X							W-0h

Table 2-63. SYSSTATUSCLR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	The key value CEh (206) must be written to KEY together with ALLECC to clear the ECC state. CEh = Issue command
23-1	RESERVED	W	X	
0	ALLECC	W	0h	Set ALLECC to clear all ECC related SYSSTATUS indicators. 1h = Clear ECC error state

2.6.43 SWDCFG Register (Offset = 1328h) [Reset = X]

SWDCFG is shown in [Figure 2-54](#) and described in [Table 2-64](#).

Return to the [Summary Table](#).

Disable the SWD function on the SWD pins

Figure 2-54. SWDCFG Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-X							
15	14	13	12	11	10	9	8
RESERVED							
W-X							
7	6	5	4	3	2	1	0
RESERVED							DISABLE
W-X							W-0h

Table 2-64. SWDCFG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	The key value 62h (98) must be written to KEY together with DISBALE to disable the SWD functions. 62h = Issue command
23-1	RESERVED	W	X	
0	DISABLE	W	0h	Set DISABLE to disable the SWD function on SWD pins, allowing the SWD pins to be used as GPIO. 1h = Disable SWD function on SWD pins

2.6.44 FCCCMD Register (Offset = 132Ch) [Reset = X]

FCCCMD is shown in [Figure 2-55](#) and described in [Table 2-65](#).

Return to the [Summary Table](#).

Frequency clock counter start capture

Figure 2-55. FCCCMD Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEY								RESERVED							
W-0h								W-X							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED															GO
W-X															W-0h

Table 2-65. FCCCMD Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	The key value 0Eh (14) must be written with GO to start a capture. 0Eh = Issue command
23-1	RESERVED	W	X	
0	GO	W	0h	Set GO to start a capture with the frequency clock counter (FCC). 1h = 1

2.6.45 PMUOPAMP Register (Offset = 1380h) [Reset = X]

PMUOPAMP is shown in [Figure 2-56](#) and described in [Table 2-66](#).

Return to the [Summary Table](#).

GPAMP control

Figure 2-56. PMUOPAMP Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED							
R/W-X							
15	14	13	12	11	10	9	8
RESERVED				CHOPCLKMODE		CHOPCLKFREQ	
R/W-X				R/W-0h		R/W-0h	
7	6	5	4	3	2	1	0
RESERVED	OUTENABLE	RRI		NSEL		PCHENABLE	ENABLE
R/W-X	R/W-0h	R/W-0h		R/W-0h		R/W-0h	R/W-0h

Table 2-66. PMUOPAMP Register Field Descriptions

Bit	Field	Type	Reset	Description
31-12	RESERVED	R/W	X	
11-10	CHOPCLKMODE	R/W	0h	CHOPCLKMODE selects the GPAMP chopping mode. 0h = Chopping disabled 1h = Normal chopping 2h = ADC Assisted chopping 3h = Reserved
9-8	CHOPCLKFREQ	R/W	0h	CHOPCLKFREQ selects the GPAMP chopping clock frequency 0h = 16kHz 1h = 8kHz 2h = 4kHz 3h = 2kHz
7	RESERVED	R/W	X	
6	OUTENABLE	R/W	0h	Set OUTENABLE to connect the GPAMP output signal to the GPAMP_OUT pin 0h = GPAMP_OUT signal is not connected to the GPAMP_OUT pin 1h = GPAMP_OUT signal is connected to the GPAMP_OUT pin
5-4	RRI	R/W	0h	RRI selects the rail-to-rail input mode. 0h = PMOS input pairs 1h = NMOS input pairs 2h = Rail-to-rail mode 3h = Rail-to-rail mode
3-2	NSEL	R/W	0h	NSEL selects the GPAMP negative channel input. 0h = GPAMP_OUT pin connected to negative channel 1h = GPAMP_IN- pin connected to negative channel 2h = GPAMP_OUT signal connected to negative channel 3h = No channel selected
1	PCHENABLE	R/W	0h	Set PCHENABLE to enable the positive channel input. 0h = Positive channel disabled 1h = GPAMP_IN+ connected to positive channel
0	ENABLE	R/W	0h	Set ENABLE to turn on the GPAMP. 0h = GPAMP is disabled 1h = GPAMP is enabled

2.6.46 SHUTDNSTORE0 Register (Offset = 1400h) [Reset = X]

SHUTDNSTORE0 is shown in [Figure 2-57](#) and described in [Table 2-67](#).

Return to the [Summary Table](#).

Shutdown storage memory (byte 0)

Figure 2-57. SHUTDNSTORE0 Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED							
R/W-X							
15	14	13	12	11	10	9	8
RESERVED							
R/W-X							
7	6	5	4	3	2	1	0
DATA							
R/W-0h							

Table 2-67. SHUTDNSTORE0 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-10	RESERVED	R/W	X	
9	PARITYERR	R	0h	Parity error for SHUTDNSTORE0
8	PARITY	R/W	0h	Parity for SHUTDNSTORE0
7-0	DATA	R/W	0h	Shutdown storage byte 0

2.6.47 SHUTDNSTORE1 Register (Offset = 1404h) [Reset = X]

SHUTDNSTORE1 is shown in [Figure 2-58](#) and described in [Table 2-68](#).

Return to the [Summary Table](#).

Shutdown storage memory (byte 1)

Figure 2-58. SHUTDNSTORE1 Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED							
R/W-X							
15	14	13	12	11	10	9	8
RESERVED							
R/W-X							
7	6	5	4	3	2	1	0
DATA							
R/W-0h							

Table 2-68. SHUTDNSTORE1 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-10	RESERVED	R/W	X	
7-0	DATA	R/W	0h	Shutdown storage byte 1

2.6.48 SHUTDNSTORE2 Register (Offset = 1408h) [Reset = X]

SHUTDNSTORE2 is shown in [Figure 2-59](#) and described in [Table 2-69](#).

Return to the [Summary Table](#).

Shutdown storage memory (byte 2)

Figure 2-59. SHUTDNSTORE2 Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED							
R/W-X							
15	14	13	12	11	10	9	8
RESERVED							
R/W-X							
7	6	5	4	3	2	1	0
DATA							
R/W-0h							

Table 2-69. SHUTDNSTORE2 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-10	RESERVED	R/W	X	
7-0	DATA	R/W	0h	Shutdown storage byte 2

2.6.49 SHUTDNSTORE3 Register (Offset = 140Ch) [Reset = X]

SHUTDNSTORE3 is shown in [Figure 2-60](#) and described in [Table 2-70](#).

Return to the [Summary Table](#).

Shutdown storage memory (byte 3)

Figure 2-60. SHUTDNSTORE3 Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED							
R/W-X							
15	14	13	12	11	10	9	8
RESERVED							
R/W-X							
7	6	5	4	3	2	1	0
DATA							
R/W-0h							

Table 2-70. SHUTDNSTORE3 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-10	RESERVED	R/W	X	
7-0	DATA	R/W	0h	Shutdown storage byte 3



The CPU subsystem (MCPUSS) includes the Arm Cortex-M0+ processor, the interrupt logic, and the prefetch and cache logic.

3.1 Overview	214
3.2 Arm Cortex-M0+ CPU	214
3.3 Interrupts and Exceptions	219
3.4 CPU Peripherals	226
3.5 Read-Only Memory (ROM)	229
3.6 CPUSS Registers	230
3.7 WUC Registers	264

3.1 Overview

The MSP CPU subsystem (MCPUSS) contains the central processing unit (CPU) along with associated supporting logic and read-only memory (ROM). The functional blocks that comprise the MCPUSS include:

- The Arm Cortex-M0+ 32-bit CPU and internal peripherals
- The CPU bus splitter and router
- The interrupt management logic and **DEEPSLEEP** entry and exit logic
- The nonvolatile memory system prefetch and cache logic
- The CPU debug interface to the debug subsystem (**DEBUGSS**)
- The **processor trace memory**
- The **read-only memory** (ROM) used for the BCR and BSL

The top level architecture of the MCPUSS is shown in [Figure 3-1](#).

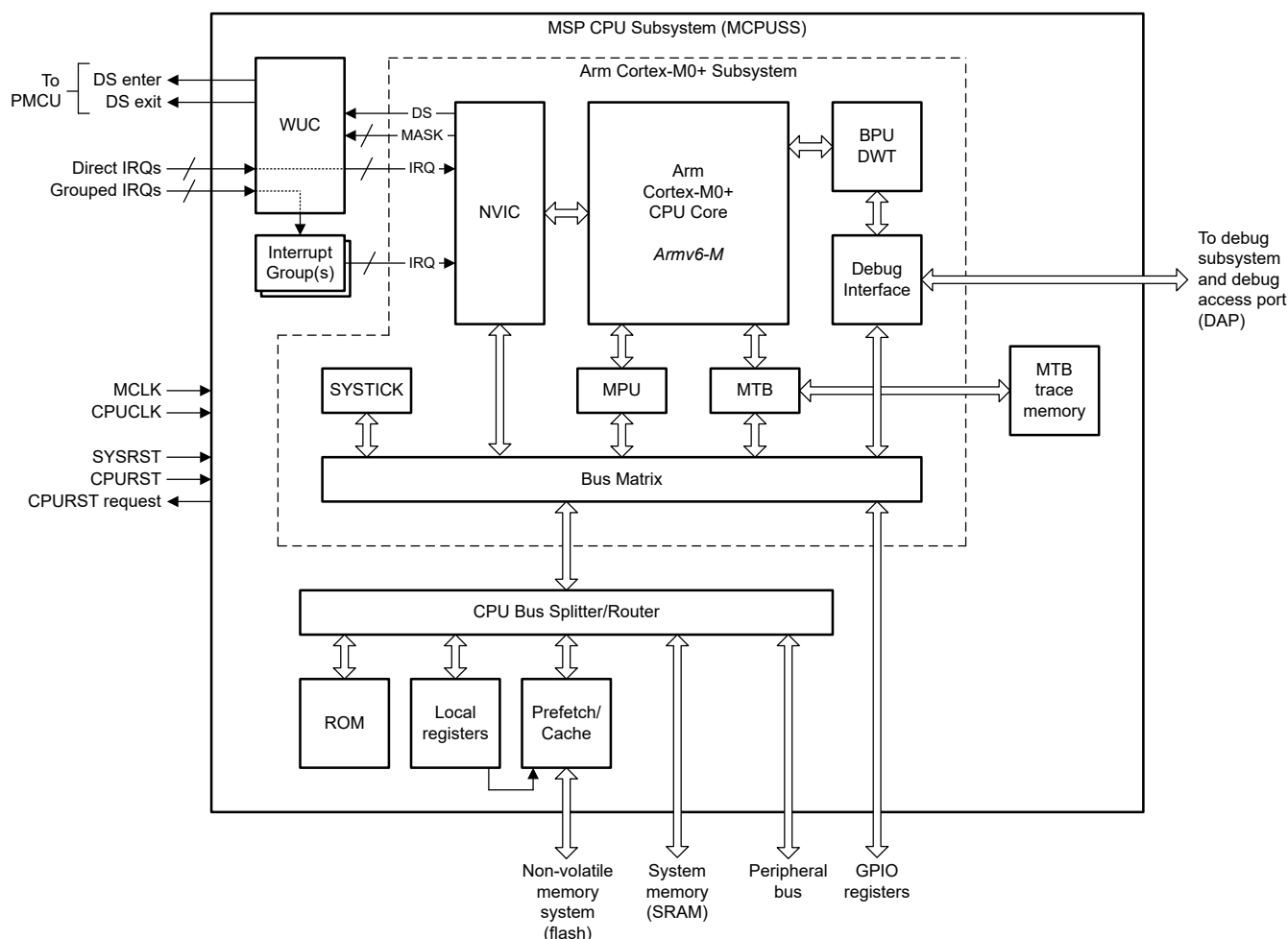


Figure 3-1. MSPM0Gxx MCPUSS Top Level Diagram

3.2 Arm Cortex-M0+ CPU

The MCPUSS contains an energy-efficient Arm Cortex-M0+ CPU implementing the Armv6-M instruction set architecture (ISA) with support for CPU clock speeds up to 80MHz. The Cortex-M0+ is a Von Neumann style 32-bit processor with a 2-stage ultra-low power pipeline and a single-cycle access port to the GPIO registers for efficient GPIO manipulation.

The Cortex-M0+ implementation on MSPM0Gxx devices has the following features:

- Up to 80 MHz execution frequency

- Little-endian (least significant byte at lowest byte address location)
- Support for 32-bit word instruction fetches
- Single cycle 32×32 multiply instruction
- Tightly integrated memory protection unit (MPU)
- User and privileged execution modes
- Integrated 24-bit system tick timer (SYSTICK)
- Four hardware breakpoints and two hardware watchpoints for debug
- Micro trace buffer
- Reset-all-registers support
- Vector table offset support

The Cortex-M0+ architecture enables excellent code density, deterministic interrupt handling, and upwards compatibility with other processor architectures in the Arm Cortex-M family.

A general overview of the Arm Cortex-M0+ is given in this section to provide a basic understanding of the features of the processor. For detailed information on developing with the Arm Cortex-M0+ processor, refer to the *Arm Cortex-M0+ Devices Generic User's Guide*.

3.2.1 CPU Register File

The Arm Cortex-M0+ processor instructions operate on registers in the CPU register file. The processor contains a register file consisting of 16 standard registers and 3 special registers as shown in [Figure 3-2](#).

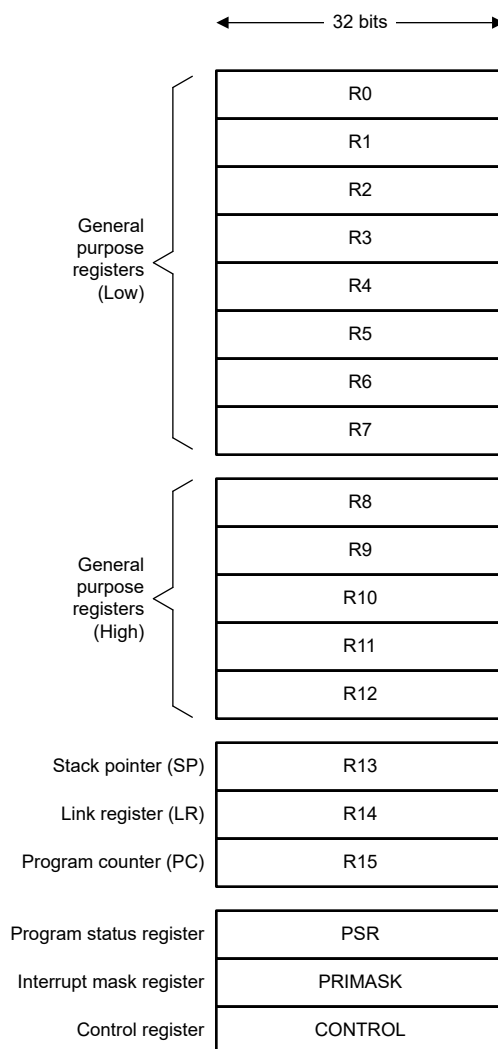


Figure 3-2. CPU Registers

General Purpose Registers (R0-R12)

The processor provides 13 general purpose registers, R0-R12, for operating on data. Registers R0 to R7 (low registers) are accessible by all instructions which specify a general purpose register. Registers R8 to R12 (high registers) are not accessible by 16-bit instructions but are accessible by any 32-bit instructions which specify a general purpose register.

Stack Pointer Register (R13)

The stack pointer is contained in R13, and can contain the main stack pointer (MSP) or the process stack pointer (PSP). When the processor is running in handler mode, the main stack pointer (MSP) is always used. When the processor is running in thread mode, the MSP or the process stack pointer (PSP) can be used, depending on the configuration of the SPSEL bit in the CONTROL register.

After a CPURST, the processor automatically and unconditionally fetches the default stack pointer from the first address of main flash (0x0000.0000) as the main stack pointer (MSP).

Link Register (R14)

R14 serves as the link register and contains the return value of function calls as well as exceptions. The link register must be set before being used as it is not reset to any known value. It is accessible in privileged and unprivileged mode.

Program Counter Register (R15)

The program counter register (R15) contains the address of the next instruction to be executed. The PC is accessible in privileged and unprivileged mode.

After a CPURST, the processor automatically and unconditionally fetches the default PC from the second word of main flash (0x0000.0004).

Special Registers

Special registers include the program status register (PSR), the interrupt mask register (PRIMASK), and the control register (CONTROL). Special registers are typically accessed by using the CPS, MRS, and MSR system instructions.

- **Program Status Register (PSR):** The PSR is a combination of the application status (APSR), interrupt status (IPSR), and execution status (EPSR) registers. Application software can access the PSR with MRS and MSR instructions, accessing either the complete PSR or a combination of one or more registers, with some restrictions. The PSR registers can be accessed with MRS and MSR instructions using the mnemonics given in [Table 3-1](#).
 - The application status register (APSR) contains the N, Z, C, and V flags which are used by the processor to evaluate conditional branch instructions. These bits are located in BIT31, BIT30, BIT29, and BIT28 of the PSR, respectively.
 - The interrupt status register (IPSR) reports the current exception number for a currently executing exception when in handler mode. In thread mode it reads as zero. The processor ignores writes to this register. The exception number field is presented in from BIT0 to BIT5 of the PSR.
 - The execution status register (EPSR) contains the T bit (BIT24), which defines whether the processor is in Thumb state. This bit cannot be read or written by software, but it is used by the processor.
- **Interrupt Mask Register (PRIMASK):** BIT0 of the PRIMASK register (PM) can be used to mask all interrupts to the processor which have configurable priority (see [Section 3.3](#)). This can be thought of as a global peripheral interrupt mask control. The processor ignores unprivileged writes to PRIMASK. Clearing PM to 0 enables interrupts. Setting PM to 1 disables interrupts. The CPS instruction can be used to change the PM bit value in the PRIMASK register.
- **Control Register (CONTROL):** The control register can be used to define whether code executing in thread mode is privileged or unprivileged by clearing or setting the nPRIV bit (BIT0), respectively. It can also be used to select the stack pointer used in R13 as either the main stack pointer (MSP) or process stack pointer (PSP) by clearing or setting the SPSEL bit (BIT1), respectively. A CPURST clears the CONTROL register to zero. The processor ignores unprivileged writes to the CONTROL register. The SPSEL stack pointer selection bit is updated by the processor automatically when entering and returning from exceptions. Note that software must implement an ISB barrier instruction after writing to CONTROL to ensure that any changes take effect before the next application instruction is executed by the processor.

Table 3-1. Program Status Register (PSR) Access Mnemonics

Mnemonic	Subregisters Included
APSR	APSR
IPSR	IPSR
EPSR	EPSR
IAPSR	IPSR and APSR
EAPSR	EPSR and APSR
XPSR	APSR, IPSR, EPSR

Table 3-1. Program Status Register (PSR) Access Mnemonics (continued)

Mnemonic	Subregisters Included
IEPSR	IPSR and EPSR

3.2.2 Stack Behavior

The Arm Cortex-M0+ processor implements a full descending stack protocol. The stack pointer register (SP) always indicates the location of the last stacked data. When new data is added to the call stack, the SP value is decremented and the data is written to the location indicated by the SP after being decremented.

The Arm Cortex-M0+ supports managing two independent stacks with two pointers: the main stack (MSP) and the process stack (PSP).

3.2.3 Execution Modes and Privilege Levels

The processor supports two primary modes of execution:

- **Thread mode** (for executing application software)
- **Handler mode** (for handling processor exceptions and peripheral interrupts)

By default, the processor is in thread mode out of reset. If an exception is issued to the processor, the processor will handle the exception in handler mode and return to thread mode after handler execution is complete. Code running in thread mode can be configured as being privileged or unprivileged, based on the configuration of the processor's internal CONTROL register. Code running in handler mode always executes as privileged.

In general, code which executes as privileged has complete control of the processor configuration, including control of the [MPU](#), [SysTick](#), [NVIC](#), and [SCB](#). Only privileged code can change the privilege level for code running in thread mode.

Code that is executing in thread mode in an unprivileged state cannot access the previously mentioned resources ([MPU](#), [SysTick](#), [NVIC](#), [SCB](#)) and can be subject to MPU restrictions if the MPU is configured and enabled.

3.2.4 Address Space and Supported Data Sizes

MSPM0 devices implement a flat memory map with a 32-bit byte-addressable address space. Byte addresses are unsigned numbers ranging from zero to $2^{32}-1$.

Address Space

The processor sees the address space as containing 2^{30} 32-bit words, with each word being word-aligned (4-byte aligned). Pointers are always 32 bits, and stack operations (for example, push, pop) increment the stack pointer by 4 addresses (4 bytes). Address calculations by the processor wrap around if they overflow or underflow the 32-bit memory space.

Instruction fetches by the processor are always 16-bit half-word aligned.

Data reads by the processor must be naturally aligned (for example, words must be word aligned, half words must be half-word aligned, etc.).

Supported Data Sizes

The processor supports 8-bit byte, 16-bit half-word, and 32-bit word data sizes. Signed and unsigned data is supported, and signed data is stored in CPU registers in 32-bit two's complement format. The Armv6-M instruction set does not provide native instructions supporting operations on 64-bit double-word data.

Load operations from memory to a CPU register can be signed or unsigned when the data size is less than 32 bits. When loading unsigned half-word or byte data to a CPU register, the value is zero-extended to 32 bits automatically. When loading signed half-word or byte data to a CPU register, the value is sign-extended to 32 bits automatically.

Stores from CPU registers to memory are sign agnostic.

All instruction and data accesses use little endian byte order.

3.3 Interrupts and Exceptions

Peripheral interrupt exceptions and system exceptions temporarily pause the processor's normal execution flow so that the processor can be used to handle an event.

The following can cause interruption of normal execution flow:

- A CPURST
- A nonmaskable interrupt (NMI, software trigger or hardware error signal from SYSCTL)
- A fault exception in the system (HardFault)
- Execution of a supervisor call instruction (SVCall)
- Setting of a pending supervisor service request (PendSV)
- A SysTick exception
- An enabled peripheral interrupt (IRQ)
- A breakpoint instruction (for debug)

Exception State

Each exception source to the processor will be in one of the below states at any given point in time:

- **Inactive** (not active, not pending)
- **Pending** (waiting to be serviced by the processor)
- **Active** (actively being serviced by the processor)
- **Active and pending** (actively being serviced by the processor when the same source generates another exception)

Exception Prioritization, Entry, and Exit

Exceptions are prioritized by the processor together with the nested vectored interrupt controller (NVIC). Each exception has either a fixed priority (reset, NMI, hard fault) or a configurable priority (SVCall, PendSV, SysTick, peripheral IRQs). Exceptions with configurable priority can be disabled by application software running in privileged mode. Exceptions with fixed priority cannot be disabled.

The processor exception model supports preemption, tail-chaining, and late-arrival features to boost exception handling performance:

- In the **preemption** case, if an exception of higher priority is pending when an exception of lower priority is executing, the higher priority exception will preempt the ongoing handler servicing the lower priority exception.
- In the **tail-chaining** case, if a valid-for-entry exception is pending at the time of completion of an exception handler, then the application context is not restored from the stack and control is given immediately to the pending exception.
- In the **late-arriving** case, if a higher priority exception occurs during entry to a lower priority exception, the higher priority exception will be serviced first after the processor state is saved to the stack. Once the higher priority exception handling is complete, the lower priority exception (which is still pending) is serviced based on the tail-chaining procedure.

An exception entry is issued if and when all of the following are true:

- An exception is in a **pending** state
- The priority of the pending exception is higher than the limit set by the exception mask register (PRIMASK)
- The processor is currently in either thread mode (not servicing an exception) or the newly pending exception is higher priority than the exception which is currently being serviced (resulting in a preemption)

Processor exceptions are vectored. When an exception occurs, the current processor state is pushed onto the stack which was active at the time of the event, and execution is vectored to the entry point address in the vector table corresponding to the exception which is to be processed.

If the exception is tail-chained to a previous handler which has completed, then there is no need to push any state to the stack and the interrupt service routine can be vectored to immediately. Likewise, if the exception is higher priority than a previous exception which started entry but did not complete entry, then there is no need to save the context again (late arrival).

Upon completion of an exception handler, if there is no exception pending which needs to be handled then the processor will pop the state from the stack and restore the processor to the previous state which it was in when the exception occurred.

3.3.1 Peripheral Interrupts (IRQs)

Peripheral interrupt functionality is managed by several components on the device:

- The nested vectored interrupt controller (NVIC)
- One or more interrupt groups (INT_GROUP)
- The wake-up controller (WUC)

MSPM0 devices include an Arm nested vectored interrupt controller (NVIC) with the Cortex-M0+ CPU for managing peripheral interrupts. The NVIC operation is tightly integrated with the processor and supports up to 32 native peripheral interrupt sources.

In addition to the NVIC, interrupt grouping modules can be present on a device to enable interfacing of more than 32 peripheral interrupts to the NVIC. High-priority interrupt sources that might require preemption capability are mapped directly to the NVIC and behave like normal NVIC interrupts. Lower priority interrupt sources which do not commonly require preemption capability are mapped to an interrupt grouping module, the output of which is then mapped to the NVIC as a native NVIC interrupt source. This routing arrangement is shown in Figure 3-3.

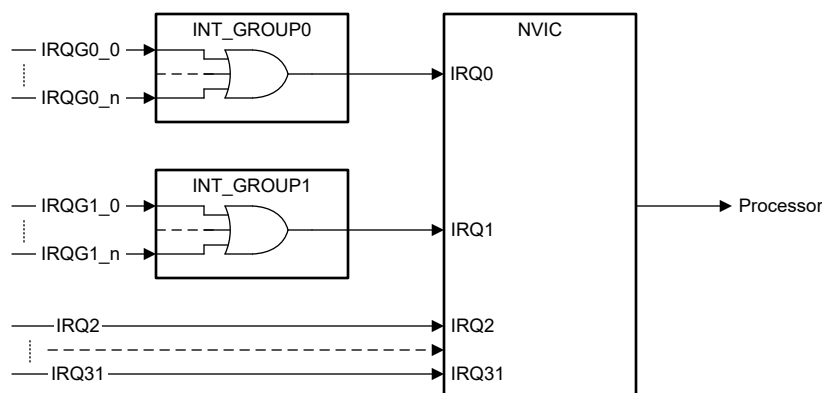


Figure 3-3. Peripheral Interrupt Hierarchy

The wake-up controller (WUC) determines if the PD1 power domain (containing the processor) needs to be powered up to service a peripheral interrupt if the PD1 domain is powered down in STOP or STANDBY mode.

3.3.1.1 Nested Vectored Interrupt Controller (NVIC)

The nested vectored interrupt controller (NVIC) is an industry-standard Arm component which interfaces peripheral interrupts (which are external to the processor) into the CPU. The NVIC supports connection of up to 32 native peripheral interrupt sources.

The NVIC is configured through memory-mapped registers in the system private peripheral bus (PPB) region. See Table 3-2 for the list of NVIC registers. The software development kit (SDK) provided with the devices supports the standard Arm Cortex Microcontroller Software Interface Standard (CMSIS) register access

definitions for the NVIC. Application software must use 32-bit aligned, word-size transactions when accessing any NVIC register.

In addition to interfacing peripheral interrupts to the processor, the NVIC also supports programmable priority for each interrupt.

Enabling and Disabling Interrupts

Peripheral interrupt enables can be read, set, and cleared through the interrupt set-enable (ISER) and interrupt clear-enable (ICER) registers in the NVIC. The 32 interrupts are mapped to the ISER and ICER registers with interrupt zero in the BIT0 position (LSB) and interrupt 31 in the BIT31 position (MSB) of each register. To enable an interrupt, set the corresponding enable bit in the ISER register. Writing a '0' to ISER has no effect. It is possible to read the ISER register to determine which interrupts are enabled. Upon a read, a '1' indicates that an interrupt is enabled; a '0' indicates disabled. To disable an interrupt, set the corresponding enable bit in the ICER register. Writing a '0' to ICER has no effect.

Note

In addition to enabling a peripheral interrupt at the NVIC, it is generally necessary to also configure the interrupt configuration of the corresponding peripheral as well. Most peripherals have multiple interrupt sources, which are merged together in the peripheral to source a single NVIC interrupt. Masking of individual peripheral interrupts is done within the peripheral's interrupt management registers.

In the event that an interrupt is disabled in the NVIC, if the interrupt is asserted by the corresponding peripheral then the NVIC interrupt will go to a pending state but the processor is not interrupted. If an interrupt is disabled when in an active state (when a handler is running) it will remain active until the exception handler returns or a reset occurs, but no further activations will happen.

Note

If a peripheral asserts an interrupt to the NVIC, but that peripheral's interrupt in the NVIC is disabled, the device may remain in a higher power mode than expected as the wake-up controller (WUC) is holding an event for the processor. To prevent this situation, ensure peripheral interrupts are masked at the peripheral directly, versus only masking interrupts at the NVIC.

Setting and Clearing Pending Interrupt Status

Pending interrupt status can be read, set, and cleared through the interrupt set-pending (ISPR) and interrupt clear-pending (ICPR) registers in the NVIC. The 32 interrupts are mapped to the ISPR and ICPR registers with interrupt zero in the BIT0 position (LSB) and interrupt 31 in the BIT31 position (MSB) of each register. To read if an interrupt is pending, read either the ISPR or the ICPR. Upon a read, a '1' indicates that an interrupt is pending; a '0' indicates not pending. To set an interrupt to a pending state through software, set the corresponding bit in the ISPR register. Writing a '0' to ISPR has no effect. To clear an interrupt pending state, set the corresponding bit in the ICPR register. Writing a '0' to ICPR has no effect. Note that if a peripheral interrupt condition is still present, the pending state will be set again by hardware even if it is cleared.

Setting Interrupt Priority

Interrupts on the NVIC have programmable priority. There are four priority levels possible. Priority is set by programming the eight IPRx registers in the NVIC. Each priority field is 8 bits in length, and the priority for 4 interrupts is configured per 32-bit register. The Arm Cortex-M0+ only implements the most significant 2 bits of each 8-bit priority field (giving the 4 priority levels). Lower priority values have higher priority. System exceptions (reset, NMI, hard fault) have fixed priorities of -3, -2, and -1, respectively. As such, these exceptions always have higher priority than peripheral interrupts. Peripheral interrupt priorities are programmable as 0, 64, 128, or 192, with 0 being highest priority and 192 being lowest priority.

If the processor is currently handling an exception, it can only be preempted by a higher priority exception. In the event that there are multiple exceptions in a pending state which all have the same priority level assigned, the exception with the lowest exception number is taken first.

Note

Application software must not change the priority of an interrupt while the corresponding interrupt is either active (being handled) or enabled. Doing so can result in unpredictable behavior.

Table 3-2. Arm Cortex-M0+ NVIC Registers

Address	Register	CMSIS	Description
0xE000.E100	NVIC_ISER	NVIC->ISER[0]	Interrupt set-enable register
0xE000.E180	NVIC_ICER	NVIC->ICER[0]	Interrupt clear-enable register
0xE000.E200	NVIC_ISPR	NVIC->ISPR[0]	Interrupt set-pending register
0xE000.E280	NVIC_ICPR	NVIC->ICPR[0]	Interrupt clear-pending register
0xE000.E400	NVIC_IPR0	NVIC->IP[0]	Interrupt priority register (0-3)
0xE000.E404	NVIC_IPR1	NVIC->IP[1]	Interrupt priority register (4-7)
0xE000.E408	NVIC_IPR2	NVIC->IP[2]	Interrupt priority register (8-11)
0xE000.E40C	NVIC_IPR3	NVIC->IP[3]	Interrupt priority register (12-15)
0xE000.E410	NVIC_IPR4	NVIC->IP[4]	Interrupt priority register (16-19)
0xE000.E414	NVIC_IPR5	NVIC->IP[5]	Interrupt priority register (20-23)
0xE000.E418	NVIC_IPR6	NVIC->IP[6]	Interrupt priority register (24-27)
0xE000.E41C	NVIC_IPR7	NVIC->IP[7]	Interrupt priority register (28-31)

3.3.1.2 Interrupt Groups

To support mapping of more than 32 peripheral interrupt sources to the NVIC, certain MSPM0 devices include interrupt grouping logic (INT_GROUP) in the MCPUSS to combine several interrupts together to source one native NVIC interrupt.

The INT_GROUP interrupt grouping uses the MSPM0 event management register structure with the key difference being that all peripheral interrupt sources to the interrupt group are always unmasked (always enabled) such that no additional enable configuration is needed beyond the peripheral interrupt configuration and the NVIC configuration. The IMASK register itself is read-only and hardwired to enable all sources to the interrupt group. Figure 3-4 shows the INT_GROUP structure.

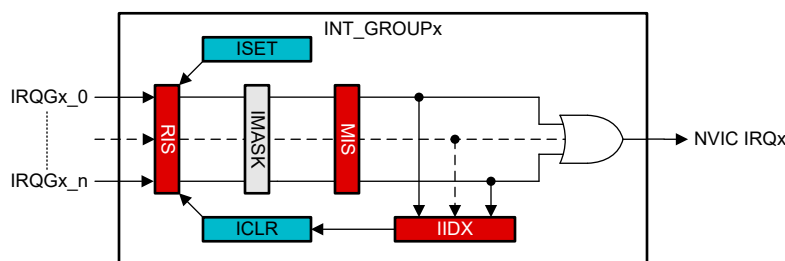


Figure 3-4. Interrupt Group (INT_GROUP)

Because no masking control is required, application software only needs to interface with the interrupt index (IIDX) register to efficiently handle a peripheral interrupt which is registered to the NVIC through an INT_GROUP.

Application software can read the IIDX register in the INT_GROUP to determine and clear the highest priority pending peripheral interrupt in the group. A read to IIDX will return an index corresponding to the highest priority peripheral which set an interrupt. The read action will also simultaneously clear the RIS and MIS bits

corresponding to the highest priority interrupt whose index was returned by the read. The value read from the IIDX register can then be used in a case statement, as shown below.

```
void GROUP_HANDLER(void)
{
    switch(IIDX)
    {
        case 0:          // no IRQ pending
            break;
        case 1:          // IRQ[0]
            do_peripheral_1_ISR();
            break;
        case 2:          // IRQ[1]
            do_peripheral_2_ISR();
            break;
        default:         // out of range
            illegal();
    }
}
```

Usage

Because peripheral interrupts that are grouped together into an INT_GROUP source a single NVIC interrupt, it is not possible to have one peripheral interrupt in a group preempt the execution of an active handler for the interrupt group. For example, take the scenario where the WWDT0 interrupt request line and the PMCU interrupt request line are connected to INT_GROUP0, and INT_GROUP0 sources NVIC peripheral interrupt 0. If the WWDT0 interrupt is asserted, INT_GROUP0 will assert an interrupt request to the NVIC. If no higher priority interrupt is active, the NVIC will vector the processor to the INT_GROUP0 handler. Application software can then read the INT_GROUP0 IIDX register to determine that it was the WWDT0 that triggered the INT_GROUP0 interrupt on the NVIC, and software can jump to the WWDT0 handler function.

If the PMCU asserts its interrupt line while the processor is still in handler mode servicing the WWDT0 function (which is really a part of the INT_GROUP0 handler), the PMCU interrupt can not preempt the WWDT0 handler. When the WWDT0 handler completes, The INT_GROUP0 handler will return. At that time, the processor will see that the INT_GROUP0 request was asserted again (this time, due to the PMCU), and it will tail-chain a second entry to the interrupt handler. This time, application software will read the IIDX and determine that the PMCU was the cause of the INT_GROUP0 interrupt being asserted to the NVIC.

If there are two or more peripheral interrupts pending to a single interrupt group, software can set the priority with which the interrupts are handled by first reading the RIS or MIS register to test which peripheral interrupts are asserted, followed by executing the software-determined priority. Alternatively, if the interrupt index (IIDX) register is used, the interrupt group hardware will return the highest priority index based on the index order.

3.3.1.3 Wake Up Controller (WUC)

The wake up controller (WUC) is responsible for monitoring for assertion of interrupts when the processor is powered down in the STOP or STANDBY operating mode. In these modes, the entire PD1 power domain is power gated, and as such, the processor and NVIC are not available to check for interrupts. The WUC retains a copy of which peripheral interrupt sources to the NVIC were enabled when the processor entered STOP or STANDBY mode. In the event that an enabled interrupt is issued, the WUC will handshake with the PMCU to bring the device out of STOP or STANDBY mode so that the CPU can service the interrupt. The WUC will capture the interrupt state and present it to the NVIC and processor when the processor is brought up, such that the processor will see the interrupt even if the raw interrupt status of the peripheral is removed before the processor finishes powering up to service the interrupt.

The WUC requires no configuration by application software upon entry to or exit from low-power modes, and operation is transparent to application software.

3.3.2 Interrupt and Exception Table

MSPM0 devices share a common interrupt and exception mapping across devices. Specific devices can not implement all interrupt sources, but in the event that a peripheral is common to two devices, it will have the same mapping to the NVIC on both devices.

See the device-specific data sheet for a complete list of which interrupts a particular device supports.

The Arm Cortex-M0+ interrupt vector table is 48 words long (192 bytes). The complete platform interrupt and exception table with vector table addresses is given in [Table 3-3](#).

Table 3-3. MSPM0 Platform Processor Interrupt and Exception Table

Exception Number	NVIC Number ⁽¹⁾	Priority Group	Exception or Interrupt	Vector Table Address	Vector Description
-	-	-	-	0x0000.0000	Stack pointer
1	-	-3	Reset	0x0000.0004	Reset vector
2	-	-2	NMI	0x0000.0008	NMI handler
3	-	-1	Hard fault	0x0000.000C	Hard fault handler
4	-	-	Reserved	0x0000.0010	-
5	-	-	Reserved	0x0000.0014	-
6	-	-	Reserved	0x0000.0018	-
7	-	-	Reserved	0x0000.001C	-
8	-	-	Reserved	0x0000.0020	-
9	-	-	Reserved	0x0000.0024	-
10	-	-	Reserved	0x0000.0028	-
11	-	Selectable	SVCALL	0x0000.002C	Supervisor call handler
12	-	-	Reserved	0x0000.0030	-
13	-	-	Reserved	0x0000.0034	-
14	-	Selectable	PendSV	0x0000.0038	Pended supervisor handler
15	-	Selectable	SysTick	0x0000.003C	SysTick handler
16	0	Selectable	INT_GROUP0	0x0000.0040	Combined peripheral group 0 handler (see INT_GROUP0 below)
17	1	Selectable	INT_GROUP1	0x0000.0044	Combined peripheral group 1 handler (see INT_GROUP1 below)
18	2	Selectable	TIMG8	0x0000.0048	Timer TIMG8 interrupt handler
19	3	Selectable	UART3	0x0000.004C	UART3 interrupt handler
20	4	Selectable	ADC0	0x0000.0050	ADC0 interrupt handler
21	5	Selectable	ADC1	0x0000.0054	ADC1 interrupt handler
22	6	Selectable	CANFD0	0x0000.0058	CAN-FD controller interrupt handler
23	7	Selectable	DAC0	0x0000.005C	DAC0 interrupt handler
24	8	Selectable	Reserved	0x0000.0060	
25	9	Selectable	SPI0	0x0000.0064	SPI0 interrupt handler
26	10	Selectable	SPI1	0x0000.0068	SPI1 interrupt handler
27	11	Selectable	Reserved	0x0000.006C	-
28	12	Selectable	Reserved	0x0000.0070	-
29	13	Selectable	UART1	0x0000.0074	UART1 interrupt handler
30	14	Selectable	UART2	0x0000.0078	UART2 interrupt handler
31	15	Selectable	UART0	0x0000.007C	UART0 interrupt handler
32	16	Selectable	TIMG0	0x0000.0080	Timer TIMG0 interrupt handler
33	17	Selectable	TIMG6	0x0000.0084	Timer TIMG6 interrupt handler
34	18	Selectable	TIMA0	0x0000.0088	Timer TIMA0 interrupt handler

Table 3-3. MSPM0 Platform Processor Interrupt and Exception Table (continued)

Exception Number	NVIC Number ⁽¹⁾	Priority Group	Exception or Interrupt	Vector Table Address	Vector Description
35	19	Selectable	TIMA1	0x0000.008C	Timer TIMA1 interrupt handler
36	20	Selectable	TIMG7	0x0000.0090	Timer TIMG7 interrupt handler
37	21	Selectable	TIMG12	0x0000.0094	Timer TIMG12 interrupt handler
38	22	Selectable	Reserved	0x0000.0098	-
39	23	Selectable	Reserved	0x0000.009C	-
40	24	Selectable	I2C0	0x0000.00A0	I2C0 interrupt handler
41	25	Selectable	I2C1	0x0000.00A4	I2C1 interrupt handler
42	26	Selectable	Reserved	0x0000.00A8	-
43	27	Selectable	Reserved	0x0000.00AC	-
44	28	Selectable	AES	0x0000.00B0	AES accelerator interrupt handler
45	29	Selectable	Reserved	0x0000.00B4	-
46	30	Selectable	RTC	0x0000.00B8	RTC interrupt handler
47	31	Selectable	DMA	0x0000.00BC	DMA interrupt handler

- (1) The NVIC number also indicates the relative interrupt priority if multiple NVIC interrupts have the same group priority. However, an interrupt will not preempt an active handler for another interrupt with the same group priority, even if the interrupt has a higher (numerically lower) NVIC position. For preemption to occur, the new interrupt must be configured to a higher priority group (numerically lower).

Non-Maskable Interrupt (NMI)

The CPU implements a non-maskable interrupt, which handles critical interrupts which must be serviced immediately by the processor. The NMI interrupt sources are managed by SYSCTL. See the corresponding NMI information in the SYSCTL section of the PMCU chapter.

INT_GROUP0 Peripheral Interrupt Group

The INT_GROUP0 peripheral interrupt group sources an interrupt to NVIC0 (exception 16) if any peripheral in the group has a pending interrupt. The peripheral interrupts mapped to INT_GROUP0 are given in [Table 3-4](#).

Table 3-4. INT_GROUP0 Interrupts

Priority	IIDX Index	Interrupt	Description
0	1	WWDT0	WWDT0 interrupt handler
1	2	WWDT1	WWDT1 interrupt handler
2	3	DEBUGSS	Debug subsystem interrupt handler
3	4	FLASHCTL	Flash controller interrupt handler
4	5	WUC FSUB0	Generic event subscriber 0 interrupt handler
5	6	WUC FSUB1	Generic event subscriber 1 interrupt handler
6	7	PMCU (SYSCTL)	PMCU (system controller) interrupt handler
7	8	Reserved	-

INT_GROUP1 Peripheral Interrupt Group

The INT_GROUP1 peripheral interrupt group sources an interrupt to NVIC1 (exception 17) if any peripheral in the group has a pending interrupt. The peripheral interrupts mapped to INT_GROUP1 are given in [Table 3-5](#).

Table 3-5. INT_GROUP1 Interrupts

Priority	IIDX Index	Interrupt	Description
0	1	GPIO0	GPIO0 interrupt handler
1	2	GPIO1	GPIO1 interrupt handler

Table 3-5. INT_GROUP1 Interrupts (continued)

Priority	IIDX Index	Interrupt	Description
2	3	COMP0	COMP0 interrupt handler
3	4	COMP1	COMP1 interrupt handler
4	5	COMP2	COMP2 interrupt handler
5	6	TRNG	TRNG interrupt handler
6	7	Reserved	-
7	8	Reserved	-

Note

Not all of the devices support Interrupt Group or NMI (Non-Maskable Interrupt). Refer to the device-specific data sheet to see which devices support them.

3.3.3 Processor Lockup Scenario

There are several exception conditions which can cause the processor to enter a lockup state. On MSPM0 devices, a processor lockup is considered a fatal fault which always triggers a [SYSRST](#) to clear the lockup condition and restart the system.

A lockup state is entered by the processor if an SVC (supervisor call) or fault condition occurs while the processor is handling an exception with priority of -1 or higher (numerically lower). Such a fault is considered by the processor to be unexpected under normal operating conditions.

The following examples are conditions that can trigger a lockup state in the processor:

- The processor cannot fetch the stack pointer or reset vector at reset
- The processor cannot fetch the NMI vector
- The processor cannot fetch the hard fault vector
- A memory fault occurs when the processor is already handling an exception with priority of -1 or -2 (hard fault or NMI)
- A supervisor call (SVC) occurs when the processor is already handling an exception with priority of -1 or -2 (hard fault or NMI)
- A usage fault or undefined instruction is fetched when the processor is already handling an exception with priority of -1 or -2 (hard fault or NMI)
- A BKPT instruction is executed when the processor is already handling an exception with priority of -1 or -2 (hard fault or NMI)

3.4 CPU Peripherals

The Arm Cortex-M0+ includes tightly coupled peripherals for system timing and memory protection.

3.4.1 System Control Block (SCB)

The system control block (SCB) provides system implementation information and system control functionality, as well as configuration, control, and reporting of processor exceptions.

The SCB is configured through memory-mapped registers in the system private peripheral bus ([PPB](#)) region. See [Table 3-6](#) for the list of SCB registers. The software development kit (SDK) provided with the devices supports the standard Arm Cortex Microcontroller Software Interface Standard (CMSIS) register access definitions for the SCB. Application software must use 32-bit aligned, word-size transactions when accessing any SCB register.

Table 3-6. Arm Cortex-M0+ System Control Block Registers

Address	Register	CMSIS	Description
0xE000.ED00	CPUID	SCB->CPUID	Read-only register indicating the CPU type and revision
0xE000.ED04	ICSR	SCB->ICSR	Provides specific interrupt controls and state
0xE000.ED08	VTOR	SCB->VTOR	Used to specify the vector table offset from 0x0000.0000

Table 3-6. Arm Cortex-M0+ System Control Block Registers (continued)

Address	Register	CMSIS	Description
0xE000.ED0C	AIRCR	SCB->AIRCR	Used to issue a CPU reset request (SYSRESETREQ)
0xE000.ED10	SCR	SCB->SCR	System control register, used to control low-power mode behavior
0xE000.ED14	CCR	SCB->CCR	Read-only register indicating behavior of the processor
0xE000.ED1C	SHPR2	SCB->SHP[2]	Used to configure the priority of the SVCALL system handler
0xE000.ED20	SHPR3	SCB->SHP[3]	Used to configure the priority of the SysTick and PendSV system handlers

For detailed information on the system control block register configuration, see the [SCB section of the Arm Cortex-M0+ devices generic user guide](#).

3.4.2 System Tick Timer (SysTick)

The system tick timer (SysTick) is a tightly-coupled timer clocked by MCLK, which can be used for system time keeping. The SysTick is available in RUN and SLEEP modes but is not available for use in STOP, STANDBY, or SHUTDOWN modes.

The SysTick is configured through memory-mapped registers in the system private peripheral bus (PPB) region. See [Table 3-7](#) for the list of SysTick configuration registers. The software development kit (SDK) provided with the devices supports the standard Arm Cortex Microcontroller Software Interface Standard (CMSIS) register access definitions for the SysTick.

The SysTick timer can be used in several different ways, including:

- As an RTOS timer which fires at a programmable rate (for example, 100Hz) and invokes a SysTick routine
- A high-speed alarm timer
- A simple counter used to measure time to completion and time used in an application
- A timeout counter to check that a routine has not timed out within a specified period

The SysTick timer is a simple 24-bit down counter, which counts down from its reload value to zero. Upon reaching zero, SysTick will re-load the value programmed into the reload value register (SYST_RVR) on the next clock cycle, and then again begin counting down to zero.

A SysTick event is generated when the SysTick counter reaches zero, and which point the COUNTFLAG status flag will be set. Reading the SYST_CSR register will clear the COUNTFLAG status bit. Writing to the current value register (SYST_CVR) clears the register and the COUNTFLAG status but does not generate an interrupt to the CPU. Reading SYST_CVR returns the counter value at time of access.

Table 3-7. SysTick Registers

Address	Register	CMSIS	Description
0xE000.E010	SYST_CSR	SysTick->CTRL	Control and status register, used to enable/disable SysTick and its related exception, and to check the COUNTFLAG status
0xE000.E014	SYST_RVR	SysTick->LOAD	Reload register used to program the counter reload value to set the SysTick interval in MCLK cycles
0xE000.E018	SYST_CVR	SysTick->VAL	Returns the current value of the SysTick counter; a write clears the counter to zero and clears COUNTFLAG in SYST_CSR
0xE000.E01C	SYST_CALIB	SysTick->CALIB	Not implemented

Application software must only use 32-bit word-aligned word accesses to the SysTick registers. To initialize the SysTick, follow the steps below:

1. Program the desired reload value (example: to generate a flag every 1000 MCLK cycles, program 999) to SYST_RVR
2. Clear the current value by writing to the SYST_CVR register
3. Program the SYST_CSR register to enable SysTick

Note

The SysTick counter does not decrement when the CPU is halted for debug.

3.4.3 Memory Protection Unit (MPU)

The memory protection unit (MPU) can be used to check all memory accesses made by the processor against a set of access permission policies which can be defined by the programmer. When used together with the privileged/unprivileged execution modes of the Arm Cortex-M0+, the MPU supports limiting access to certain memory locations to privileged code only. If unprivileged code accesses a non-restricted region, execution continues as if the MPU was not present. However, if unprivileged code issues an access to a restricted region, a hard fault is generated in the processor. It is also possible to restrict access to both privileged and unprivileged code (no access possible through the processor).

The MPU is configured through memory-mapped registers in the system private peripheral bus (PPB) region. See [Table 3-8](#) for the list of MPU configuration registers. The software development kit (SDK) provided with the devices supports the standard Arm Cortex Microcontroller Software Interface Standard (CMSIS) register access definitions for the MPU.

Because the MPU checks all memory accesses from the processor (including accesses to flash, SRAM, and peripherals), it is well suited for improving reliability and robustness in threaded applications involving an RTOS. The MPU can be used to restrict the memory access of individual threads, including establishing stack boundaries and limiting access to specific peripherals. Key state data used by the RTOS can be protected from modification by unprivileged threads.

Note

The MPU only monitors access to the memory map which originate from the processor. The MPU does not restrict the access of the DMA controller.

Note

Protecting flash memory regions with the MPU does not protect the flash memory from being read or modified by the [flash controller](#). This is because the flash controller can take control of the flash memory banks directly for read verify, program, and erase operations. Dedicated [write protection functions](#) are provided within the flash controller, and these mechanisms are for use when write protecting the flash memory. Software can configure the MPU to restrict access to the flash controller registers, thus preventing the CPU from configuring or initiating a flash command directly.

The MPU provides a mechanism to partition the device memory map into 8 regions (numbered 0-7) plus a default background region. Each region can be configured with access permissions and memory attributes, and regions can be configured to overlap if desired. In the case of overlapping regions, a memory access to a location existing in multiple regions is subjected to the attributes of the region with the highest number.

When the MPU is not enabled (ENABLE bit is cleared in the MPU_CTRL register), the device uses the default memory map and the CPU has access to the memory map as if the MPU was not present.

When the MPU is enabled for use, access to the vector table and the system control space are always permitted, but access to any other location depends on the following:

- The region configurations
- Whether the access was privileged or unprivileged
- The privileged software default memory map access control configuration (PRIVDEFENA)

Because the Arm Cortex-M0+ is a single-bus CPU architecture, there is no delineation between an instruction fetch and a data access by the MPU. Instructions and data are treated the same.

Note

If the MPU is enabled (ENABLE bit is set), then PRIVDEFENA must be set to give privileged software access to the memory map, or at least one memory region must be configured and enabled for the CPU to proceed without a hard fault. If PRIVDEFENA is cleared, no regions are defined and enabled, and the MPU is enabled, then the entire memory map is restricted, and any access generates a fault.

Table 3-8. MPU Registers

Address	Register	CMSIS	Description
0xE000.ED90	MPU_TYPE	MPU->TYPE	Type register indicating that the MPU is present.
0xE000.ED94	MPU_CTRL	MPU->CTRL	MPU control register for enabling and configuring the MPU.
0xE000.ED98	MPU_RNR	MPU->RNR	Region select register for using MPU_RBAR and MPU_RASR.
0xE000.ED9C	MPU_RBAR	MPU->RBAR	Region base address configuration register.
0xE000.EDA0	MPU_RASR	MPU->RASR	Region size and memory attributes register.

For detailed information on the MPU register configuration, see the [MPU section of the Arm Cortex-M0+ Devices Generic User Guide](#).

3.5 Read-Only Memory (ROM)

The MCPUSS contains a read-only memory which contains the executable code for the boot configuration routine (BCR) and bootstrap loader (BSL). The ROM is active after a BOOTRST, or after a SYSRST with BSL entry/exit. The ROM is disabled automatically when the application is started and is not accessible by application software.

3.6 CPUSS Registers

Table 3-9 lists the memory-mapped registers for the CPUSS registers. All register offset addresses not listed in Table 3-9 should be considered as reserved locations and the register contents should not be modified.

Table 3-9. CPUSS Registers

Offset	Acronym	Register Name	Group	Section
10E0h	EVT_MODE	Event Mode		Go
10FCh	DESC	Module Description		Go
1100h	IIDX	Interrupt index	CPU_INT_GR OUP0	Go
1108h	IMASK	Interrupt mask	CPU_INT_GR OUP0	Go
1110h	RIS	Raw interrupt status	CPU_INT_GR OUP0	Go
1118h	MIS	Masked interrupt status	CPU_INT_GR OUP0	Go
1120h	ISSET	Interrupt set	CPU_INT_GR OUP0	Go
1128h	ICLR	Interrupt clear	CPU_INT_GR OUP0	Go
1130h	IIDX	Interrupt index	CPU_INT_GR OUP1	Go
1138h	IMASK	Interrupt mask	CPU_INT_GR OUP1	Go
1140h	RIS	Raw interrupt status	CPU_INT_GR OUP1	Go
1148h	MIS	Masked interrupt status	CPU_INT_GR OUP1	Go
1150h	ISSET	Interrupt set	CPU_INT_GR OUP1	Go
1158h	ICLR	Interrupt clear	CPU_INT_GR OUP1	Go
1160h	IIDX	Interrupt index	CPU_INT_GR OUP2	Go
1168h	IMASK	Interrupt mask	CPU_INT_GR OUP2	Go
1170h	RIS	Raw interrupt status	CPU_INT_GR OUP2	Go
1178h	MIS	Masked interrupt status	CPU_INT_GR OUP2	Go
1180h	ISSET	Interrupt set	CPU_INT_GR OUP2	Go
1188h	ICLR	Interrupt clear	CPU_INT_GR OUP2	Go
1190h	IIDX	Interrupt index	CPU_INT_GR OUP3	Go
1198h	IMASK	Interrupt mask	CPU_INT_GR OUP3	Go
11A0h	RIS	Raw interrupt status	CPU_INT_GR OUP3	Go
11A8h	MIS	Masked interrupt status	CPU_INT_GR OUP3	Go
11B0h	ISSET	Interrupt set	CPU_INT_GR OUP3	Go

Table 3-9. CPUSS Registers (continued)

Offset	Acronym	Register Name	Group	Section
11B8h	ICLR	Interrupt clear	CPU_INT_GR OUP3	Go
11C0h	IIDX	Interrupt index	CPU_INT_GR OUP4	Go
11C8h	IMASK	Interrupt mask	CPU_INT_GR OUP4	Go
11D0h	RIS	Raw interrupt status	CPU_INT_GR OUP4	Go
11D8h	MIS	Masked interrupt status	CPU_INT_GR OUP4	Go
11E0h	ISSET	Interrupt set	CPU_INT_GR OUP4	Go
11E8h	ICLR	Interrupt clear	CPU_INT_GR OUP4	Go
1300h	CTL	Prefetch/Cache control		Go

Complex bit access types are encoded to fit into small table cells. [Table 3-10](#) shows the codes that are used for access types in this section.

Table 3-10. CPUSS Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
W	W	Write
Reset or Default Value		
-n		Value after reset or the default value

3.6.1 EVT_MODE (Offset = 10E0h) [Reset = 00000001h]

EVT_MODE is shown in [Figure 3-5](#) and described in [Table 3-11](#).

Return to the [Summary Table](#).

Event mode register. It is used to select whether each line is disabled, in software mode (software clears the RIS) or in hardware mode (hardware clears the RIS)

Figure 3-5. EVT_MODE

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							
R-							
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED						INT_CFG	
R-						R-1h	

Table 3-11. EVT_MODE Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1-0	INT_CFG	R	1h	Event line mode select 0h = The interrupt or event line is disabled. 1h = Event handled by software. Software must clear the associated RIS flag. 2h = Event handled by hardware. The hardware (another module) clears automatically the associated RIS flag.

3.6.2 DESC (Offset = 10FCh) [Reset = 27110000h]

DESC is shown in [Figure 3-6](#) and described in [Table 3-12](#).

Return to the [Summary Table](#).

This register identifies the peripheral and its exact version.

Figure 3-6. DESC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODULEID															
R-2711h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FEATUREVER				RESERVED				MAJREV				MINREV			
R-0h				R-				R-0h				R-0h			

Table 3-12. DESC Field Descriptions

Bit	Field	Type	Reset	Description
31-16	MODULEID	R	2711h	Module identification contains a unique peripheral identification number. The assignments are maintained in a central database for all of the platform modules to ensure uniqueness.
15-12	FEATUREVER	R	0h	Feature Set for the module *instance*
11-8	RESERVED	R	0h	
7-4	MAJREV	R	0h	Major rev of the IP
3-0	MINREV	R	0h	Minor rev of the IP

3.6.3 IIDX (Offset = 1100h) [Reset = 00000000h]

IIDX is shown in [Figure 3-7](#) and described in [Table 3-13](#).

Return to the [Summary Table](#).

Interrupt index register. This read-only register provides the interrupt index of the pending interrupt with the highest priority. It also indicates if no interrupt is pending. The priority order is fixed: lower index equals higher priority. Alternatively to the use of IIDX, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred.

On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flags in the RIS and MIS are cleared as well. After a read from the CPU (not from the debug interface), the register must be updated with the next highest priority interrupt or indicate that no interrupt is pending. Only interrupts which are selected via IMASK are indicated.

Figure 3-7. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 3-13. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 0h = No pending interrupt 1h = Interrupt 0 2h = Interrupt 1 3h = Interrupt 2 4h = Interrupt 3 5h = Interrupt 4 6h = Interrupt 5 7h = Interrupt 6 8h = Interrupt 7

3.6.4 IMASK (Offset = 1108h) [Reset = 00000FFh]

IMASK is shown in [Figure 3-8](#) and described in [Table 3-14](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.”

Figure 3-8. IMASK

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
R-0h																								R-FFh							

Table 3-14. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	INT	R	FFh	Masks the corresponding interrupt 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.5 RIS (Offset = 1110h) [Reset = 00000000h]

RIS is shown in [Figure 3-9](#) and described in [Table 3-15](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 3-9. RIS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
R-0h																								R-0h							

Table 3-15. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	INT	R	0h	Raw interrupt status for INT 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.6 MIS (Offset = 1118h) [Reset = 00000000h]

MIS is shown in [Figure 3-10](#) and described in [Table 3-16](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 3-10. MIS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
R-0h																								R-0h							

Table 3-16. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	INT	R	0h	Masked interrupt status for INT0 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.7 ISET (Offset = 1120h) [Reset = 00000000h]

ISET is shown in [Figure 3-11](#) and described in [Table 3-17](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 3-11. ISET

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
W-0h																								W-0h							

Table 3-17. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	W	0h	
7-0	INT	W	0h	Sets INT in RIS register 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.8 ICLR (Offset = 1128h) [Reset = 00000000h]

ICLR is shown in [Figure 3-12](#) and described in [Table 3-18](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 3-12. ICLR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
W-0h																								W-0h							

Table 3-18. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	W	0h	
7-0	INT	W	0h	Clears INT in RIS register 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.9 IIDX (Offset = 1130h) [Reset = 00000000h]

IIDX is shown in [Figure 3-13](#) and described in [Table 3-19](#).

Return to the [Summary Table](#).

Interrupt index register. This read-only register provides the interrupt index of the pending interrupt with the highest priority. It also indicates if no interrupt is pending. The priority order is fixed: lower index equals higher priority. Alternatively to the use of IIDX, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred.

On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flags in the RIS and MIS are cleared as well. After a read from the CPU (not from the debug interface), the register must be updated with the next highest priority interrupt or indicate that no interrupt is pending. Only interrupts which are selected via IMASK are indicated.

Figure 3-13. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 3-19. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 0h = No pending interrupt 1h = Interrupt 0 2h = Interrupt 1 3h = Interrupt 2 4h = Interrupt 3 5h = Interrupt 4 6h = Interrupt 5 7h = Interrupt 6 8h = Interrupt 7

3.6.10 IMASK (Offset = 1138h) [Reset = 00000FFh]

IMASK is shown in [Figure 3-14](#) and described in [Table 3-20](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.”

Figure 3-14. IMASK

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
R-0h																								R-FFh							

Table 3-20. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	INT	R	FFh	Masks the corresponding interrupt 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.11 RIS (Offset = 1140h) [Reset = 00000000h]

RIS is shown in [Figure 3-15](#) and described in [Table 3-21](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 3-15. RIS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
R-0h																								R-0h							

Table 3-21. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	INT	R	0h	Raw interrupt status for INT 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.12 MIS (Offset = 1148h) [Reset = 00000000h]

MIS is shown in [Figure 3-16](#) and described in [Table 3-22](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 3-16. MIS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
R-0h																								R-0h							

Table 3-22. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	INT	R	0h	Masked interrupt status for INTO 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.13 ISET (Offset = 1150h) [Reset = 00000000h]

ISET is shown in [Figure 3-17](#) and described in [Table 3-23](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 3-17. ISET

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
W-0h																								W-0h							

Table 3-23. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	W	0h	
7-0	INT	W	0h	Sets INT in RIS register 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.14 ICLR (Offset = 1158h) [Reset = 00000000h]

ICLR is shown in [Figure 3-18](#) and described in [Table 3-24](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 3-18. ICLR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
W-0h																								W-0h							

Table 3-24. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	W	0h	
7-0	INT	W	0h	Clears INT in RIS register 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.15 IIDX (Offset = 1160h) [Reset = 00000000h]

IIDX is shown in [Figure 3-19](#) and described in [Table 3-25](#).

Return to the [Summary Table](#).

Interrupt index register. This read-only register provides the interrupt index of the pending interrupt with the highest priority. It also indicates if no interrupt is pending. The priority order is fixed: lower index equals higher priority. Alternatively to the use of IIDX, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred.

On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flags in the RIS and MIS are cleared as well. After a read from the CPU (not from the debug interface), the register must be updated with the next highest priority interrupt or indicate that no interrupt is pending. Only interrupts which are selected via IMASK are indicated.

Figure 3-19. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 3-25. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 0h = No pending interrupt 1h = Interrupt 0 2h = Interrupt 1 3h = Interrupt 2 4h = Interrupt 3 5h = Interrupt 4 6h = Interrupt 5 7h = Interrupt 6 8h = Interrupt 7

3.6.16 IMASK (Offset = 1168h) [Reset = 00000FFh]

IMASK is shown in [Figure 3-20](#) and described in [Table 3-26](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.”

Figure 3-20. IMASK

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
R-0h																								R-FFh							

Table 3-26. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	INT	R	FFh	Masks the corresponding interrupt 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.17 RIS (Offset = 1170h) [Reset = 00000000h]

RIS is shown in [Figure 3-21](#) and described in [Table 3-27](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 3-21. RIS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
R-0h																								R-0h							

Table 3-27. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	INT	R	0h	Raw interrupt status for INT 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.18 MIS (Offset = 1178h) [Reset = 00000000h]

MIS is shown in [Figure 3-22](#) and described in [Table 3-28](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 3-22. MIS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
R-0h																								R-0h							

Table 3-28. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	INT	R	0h	Masked interrupt status for INT0 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.19 ISET (Offset = 1180h) [Reset = 00000000h]

ISET is shown in [Figure 3-23](#) and described in [Table 3-29](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 3-23. ISET

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
W-0h																								W-0h							

Table 3-29. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	W	0h	
7-0	INT	W	0h	Sets INT in RIS register 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.20 ICLR (Offset = 1188h) [Reset = 00000000h]

ICLR is shown in [Figure 3-24](#) and described in [Table 3-30](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 3-24. ICLR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
W-0h																								W-0h							

Table 3-30. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	W	0h	
7-0	INT	W	0h	Clears INT in RIS register 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.21 IIDX (Offset = 1190h) [Reset = 00000000h]

IIDX is shown in [Figure 3-25](#) and described in [Table 3-31](#).

Return to the [Summary Table](#).

Interrupt index register. This read-only register provides the interrupt index of the pending interrupt with the highest priority. It also indicates if no interrupt is pending. The priority order is fixed: lower index equals higher priority. Alternatively to the use of IIDX, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred.

On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flags in the RIS and MIS are cleared as well. After a read from the CPU (not from the debug interface), the register must be updated with the next highest priority interrupt or indicate that no interrupt is pending. Only interrupts which are selected via IMASK are indicated.

Figure 3-25. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 3-31. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 0h = No pending interrupt 1h = Interrupt 0 2h = Interrupt 1 3h = Interrupt 2 4h = Interrupt 3 5h = Interrupt 4 6h = Interrupt 5 7h = Interrupt 6 8h = Interrupt 7

3.6.22 IMASK (Offset = 1198h) [Reset = 00000FFh]

IMASK is shown in [Figure 3-26](#) and described in [Table 3-32](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.”

Figure 3-26. IMASK

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
R-0h																								R-FFh							

Table 3-32. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	INT	R	FFh	Masks the corresponding interrupt 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.23 RIS (Offset = 11A0h) [Reset = 00000000h]

RIS is shown in [Figure 3-27](#) and described in [Table 3-33](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 3-27. RIS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
R-0h																								R-0h							

Table 3-33. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	INT	R	0h	Raw interrupt status for INT 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.24 MIS (Offset = 11A8h) [Reset = 00000000h]

MIS is shown in [Figure 3-28](#) and described in [Table 3-34](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 3-28. MIS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
R-0h																								R-0h							

Table 3-34. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	INT	R	0h	Masked interrupt status for INTO 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.25 ISET (Offset = 11B0h) [Reset = 00000000h]

ISET is shown in [Figure 3-29](#) and described in [Table 3-35](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 3-29. ISET

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
W-0h																								W-0h							

Table 3-35. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	W	0h	
7-0	INT	W	0h	Sets INT in RIS register 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.26 ICLR (Offset = 11B8h) [Reset = 00000000h]

ICLR is shown in [Figure 3-30](#) and described in [Table 3-36](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 3-30. ICLR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
W-0h																								W-0h							

Table 3-36. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	W	0h	
7-0	INT	W	0h	Clears INT in RIS register 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.27 IIDX (Offset = 11C0h) [Reset = 00000000h]

IIDX is shown in [Figure 3-31](#) and described in [Table 3-37](#).

Return to the [Summary Table](#).

Interrupt index register. This read-only register provides the interrupt index of the pending interrupt with the highest priority. It also indicates if no interrupt is pending. The priority order is fixed: lower index equals higher priority. Alternatively to the use of IIDX, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred.

On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flags in the RIS and MIS are cleared as well. After a read from the CPU (not from the debug interface), the register must be updated with the next highest priority interrupt or indicate that no interrupt is pending. Only interrupts which are selected via IMASK are indicated.

Figure 3-31. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 3-37. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 0h = No pending interrupt 1h = Interrupt 0 2h = Interrupt 1 3h = Interrupt 2 4h = Interrupt 3 5h = Interrupt 4 6h = Interrupt 5 7h = Interrupt 6 8h = Interrupt 7

3.6.28 IMASK (Offset = 11C8h) [Reset = 00000FFh]

IMASK is shown in [Figure 3-32](#) and described in [Table 3-38](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.”

Figure 3-32. IMASK

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
R-0h																								R-FFh							

Table 3-38. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	INT	R	FFh	Masks the corresponding interrupt 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.29 RIS (Offset = 11D0h) [Reset = 00000000h]

RIS is shown in [Figure 3-33](#) and described in [Table 3-39](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 3-33. RIS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
R-0h																								R-0h							

Table 3-39. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	INT	R	0h	Raw interrupt status for INT 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.30 MIS (Offset = 11D8h) [Reset = 00000000h]

MIS is shown in [Figure 3-34](#) and described in [Table 3-40](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 3-34. MIS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
R-0h																								R-0h							

Table 3-40. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	INT	R	0h	Masked interrupt status for INTO 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.31 ISET (Offset = 11E0h) [Reset = 00000000h]

ISET is shown in [Figure 3-35](#) and described in [Table 3-41](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 3-35. ISET

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
W-0h																								W-0h							

Table 3-41. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	W	0h	
7-0	INT	W	0h	Sets INT in RIS register 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.32 ICLR (Offset = 11E8h) [Reset = 00000000h]

ICLR is shown in [Figure 3-36](#) and described in [Table 3-42](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 3-36. ICLR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								INT							
W-0h																								W-0h							

Table 3-42. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	W	0h	
7-0	INT	W	0h	Clears INT in RIS register 1h = Interrupt 0 2h = Interrupt 1 4h = Interrupt 2 8h = Interrupt 3 10h = Interrupt 4 20h = Interrupt 5 40h = Interrupt 6 80h = Interrupt 7

3.6.33 CTL (Offset = 1300h) [Reset = 00000007h]

CTL is shown in [Figure 3-37](#) and described in [Table 3-43](#).

Return to the [Summary Table](#).

Flash prefetch and cache control register.

Figure 3-37. CTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED					LITEN	ICACHE	PREFETCH
R/W-0h					R/W-1h	R/W-1h	R/W-1h

Table 3-43. CTL Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R/W	0h	
2	LITEN	R/W	1h	Literal caching and prefetch enable. This bit is a subset of ICACHE/PREFETCH bit i.e. literal caching or literal prefetching will only happen if ICACHE or PREFETCH bits have been set respectively When enabled, the cache and prefetcher structures inside CPUSS will cache and prefetch literals When disabled, the cache and prefetcher structures inside CPUSS will not cache and prefetch literals 0h = Literal caching disabled 1h = Literal caching enabled
1	ICACHE	R/W	1h	Used to enable/disable Instruction caching on flash access. 0h = Disable instruction caching. 1h = Enable instruction caching.
0	PREFETCH	R/W	1h	Used to enable/disable instruction prefetch to Flash. 0h = Disable instruction prefetch. 1h = Enable instruction prefetch.

3.7 WUC Registers

[Table 3-44](#) lists the memory-mapped registers for the WUC registers. All register offset addresses not listed in [Table 3-44](#) should be considered as reserved locations and the register contents should not be modified.

Table 3-44. WUC Registers

Offset	Acronym	Register Name	Section
400h	FSUB_0	Subscriber Port 0	FSUB_0 Register (Offset = 400h) [Reset = 00h]
404h	FSUB_1	Subscriber Port 1	FSUB_1 Register (Offset = 404h) [Reset = 00h]

3.7.1 FSUB_0 Register (Offset = 400h) [Reset = 00h]

FSUB_0 is shown in [Figure 3-38](#) and described in [Table 3-45](#).

Return to the [Summary Table](#).

Subscriber port

Figure 3-38. FSUB_0 Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-0h												R/W-0h			

Table 3-45. FSUB_0 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channel ID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum can be less than 15.

3.7.2 FSUB_1 Register (Offset = 404h) [Reset = 00h]

FSUB_1 is shown in [Figure 3-39](#) and described in [Table 3-46](#).

Return to the [Summary Table](#).

Subscriber port

Figure 3-39. FSUB_1 Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-0h												R/W-0h			

Table 3-46. FSUB_1 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channel ID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum can be less than 15.

This page intentionally left blank.



The direct memory access (DMA) controller module transfers data from one address to another, without CPU intervention. This chapter describes the operation of the DMA controller.

4.1 DMA Overview.....	268
4.2 DMA Operation.....	269
4.3 DMA Registers.....	279

4.1 DMA Overview

The DMA controller transfers data from a source address to a destination address without CPU intervention. For example, the DMA controller can be used to move data from ADC conversion memory to SRAM.

Devices can have up to sixteen DMA channels available. Therefore, depending on the number of DMA channels available, some features described in this chapter are not applicable to all devices. Please refer to the device-specific data sheet for the actual channel count of the DMA.

Using the DMA controller can increase the throughput of peripheral modules. It can also reduce system power consumption by allowing the CPU to remain in a low-power mode, without having to awaken to move data to or from a peripheral.

DMA controller features include:

- Up to sixteen independent transfer channels
- Configurable DMA [channel priorities](#)
- Byte (8-bit), short word (16-bit), word (32-bit) and long word (64-bit) or mixed byte and word transfer capability
- Transfer counter block size supports up to 64k transfers of any data type
- Configurable [DMA transfer trigger selection](#)
- Six flexible [addressing modes](#)
- Single or block [transfer modes](#)

The DMA controller block diagram is shown in [Figure 4-1](#).

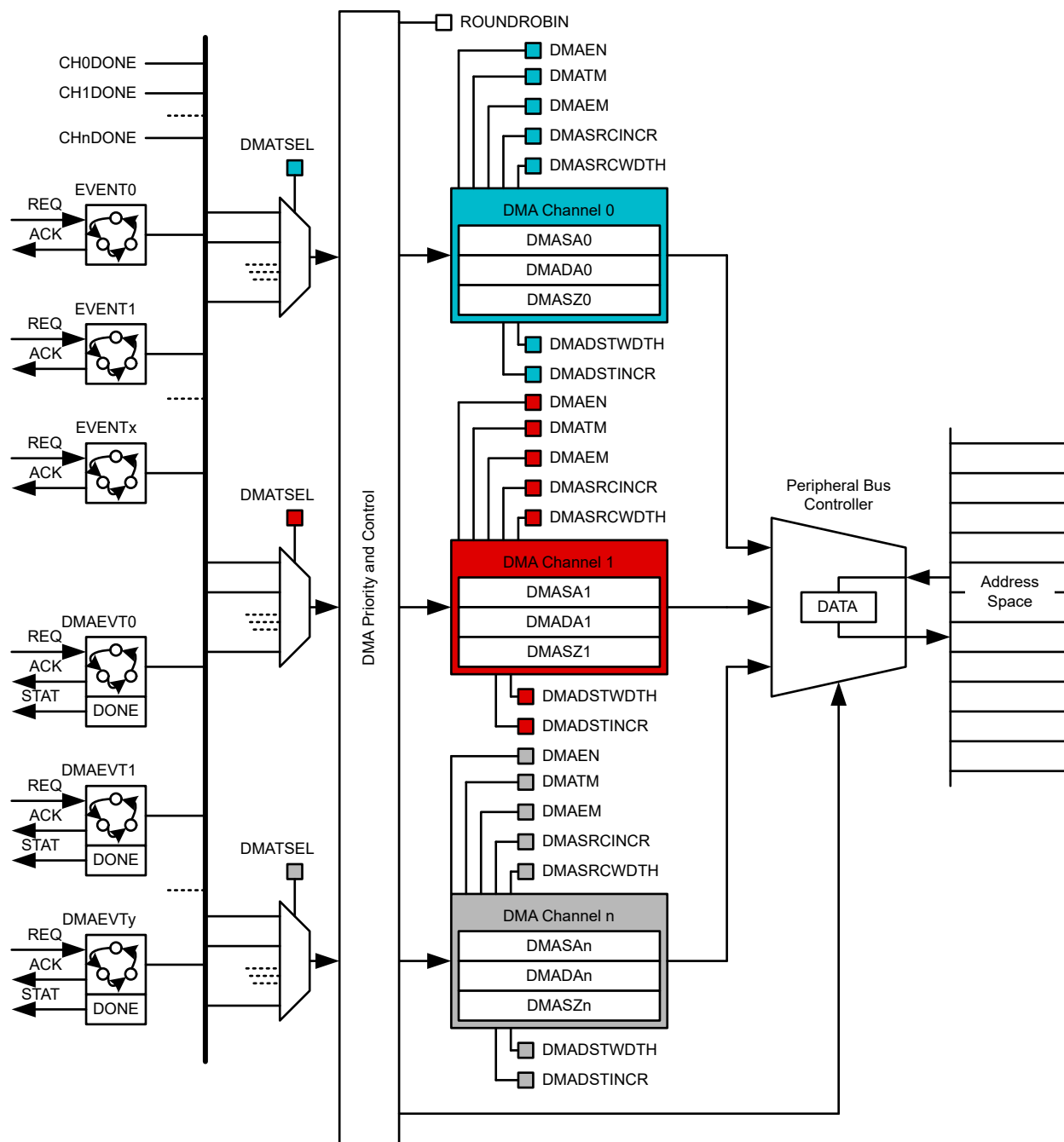


Figure 4-1. DMA Block Diagram

Note

DMA in some MSPM0C devices only support ADC, no supporting for other peripherals. Refer to the device-specific data sheet to see the details.

4.2 DMA Operation

The DMA controller is configured with user software. The setup and operation of the DMA is discussed in the following sections.

4.2.1 Addressing Modes

The DMA controller has six addressing modes. The addressing mode for each DMA channel is independently configurable. For example, channel 0 can transfer between two fixed addresses, while channel 1 transfers between two blocks of addresses. The addressing modes are shown in [Figure 4-2](#).

The addressing modes are:

1. Fixed address to fixed address
2. Fixed address to block of addresses
3. Block of addresses to fixed address
4. Block of addresses to block of addresses
5. Fill data to block of addresses
6. Data table to specific address

Addressing modes 1–4 shown above are simply configured with the DMASRCINCR and DMADSTINCR control bits. The DMASRCINCR bits select if the source address is incremented, decremented, or unchanged after each transfer. The DMADSTINCR bits select if the destination address is incremented, decremented, or unchanged after each transfer.

Addressing modes 5 and 6 shown above are also configured with the DMASRCINCR and DMADSTINCR control bits along with the help of additional parameters such as DMAEM for leveraging the [extended modes](#) of the DMA. Refer to [Section 4.2.4.1](#) and [Section 4.2.4.2](#) for more details on how to properly configure and use the DMA in Fill Mode and Table Mode.

Transfers can be byte to byte, short word to short word, word to word, long word to long word, or any combination of the four. When transferring (short or long) word to byte, only the lower byte of the source data transfers. When transferring (long) word to short word, only the lower short word of the source data transfers. When transferring byte to (short or long) word, the upper bytes of the destination word is cleared when the transfer occurs. When transferring short word to (long) word, the upper short word is cleared when the transfer occurs. When transferring word to long word, the upper word is cleared. There is no packing or unpacking support by combining several source byte transfers to one single destination (short) word or the reverse.

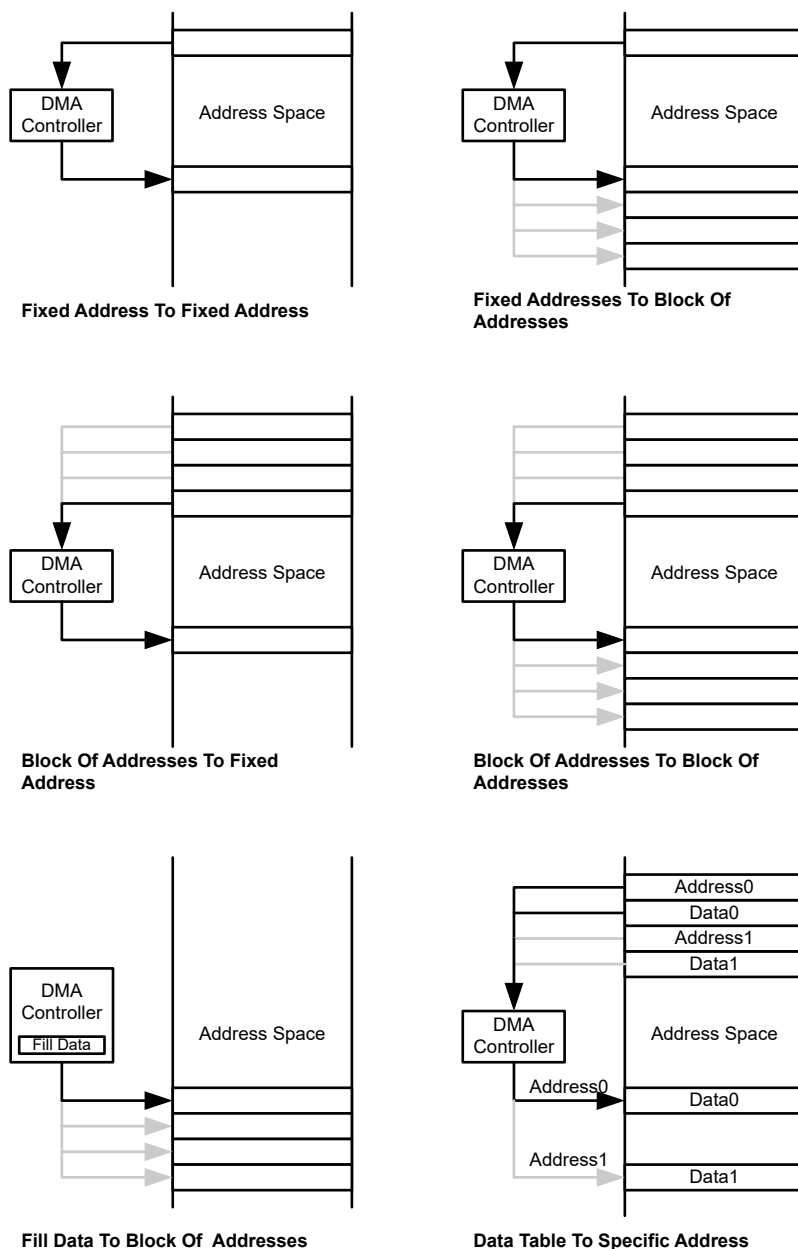


Figure 4-2. DMA Addressing Modes

4.2.2 Channel Types

There are two types of DMA channels: basic (BASIC) and full-featured (FULL) channels. BASIC channels support only single or block transfers, and FULL channels support repeated single and repeated block transfers with additional features such as early interrupt request generation and extended table and fill modes.

The highest priority DMA channels (starting with DMA0) are FULL channels, and the remaining priority channels are BASIC channels.

Note

See the device-specific data sheet to determine how many BASIC and FULL channels are available.

Table 4-1 shows the features supported in basic and full-feature DMA channels.

Table 4-1. Feature Comparison of Basic and Full-Feature DMA Channels

DMA Feature	Full-Feature Channel	Basic Channel
Repeated mode	✓	–
Early IRQ notification	✓	–
Block burst mode	✓	✓
Stride mode	✓	✓
Internal channel as trigger source	✓	✓
Extended Mode (Table and Fill Mode)	✓	–

4.2.3 Transfer Modes

The DMA controller has four transfer modes selected by the DMATM bits as listed in [Table 4-2](#). Each channel is individually configurable for its transfer mode. For example, channel 0 can be configured in repeated block transfer mode, while channel 1 is configured for block transfer mode, and channel 2 operates in single transfer mode. The transfer mode is configured independently from the addressing mode. Any addressing mode can be used with any transfer mode.

Four types of data can be transferred selectable by the DMADSTWDTH and DMASRCWDTH control bits. The source and destination locations can be either byte, short word, word, or long word data. It is also possible to transfer byte to byte, short word to short word, word to word, long word to long word, or any combination.

Additionally, all transfers modes support a stride mode where the DMA source and destination can be incremented to a higher value to support re-organization of data.

Table 4-2. DMA Transfer Modes

DMATM	Transfer Mode	Description	Channel Type
0h	Single transfer	Each transfer requires a trigger. DMAEN is automatically cleared when DMASZx transfers have been made.	Basic
1h	Block transfer	A complete block is transferred with one trigger. DMAEN is automatically cleared at the end of the block transfer.	Basic
2h	Repeated single transfer	Each transfer requires a trigger. DMAEN remains enabled.	Full-feature
3h	Repeated block transfer	A complete block is transferred with one trigger. DMAEN remains enabled.	Full-feature

4.2.3.1 Single Transfer

In single transfer mode (DMATM = 0h), each byte, half-word, word, or long-word transfer requires a separate trigger. Single transfer mode is available in basic and full-feature DMA channels.

The DMASZx register defines the number of transfers to be made. The DMADSTINCR and DMASRCINCR bits select if the destination address and the source address are incremented or decremented after each transfer. If DMASZx = 0, no transfers occur.

The DMASAx, DMADAx, and DMASZx registers are incremented or decremented after each transfer. The DMADSTWDTH will indicate whether the destination address will increment or decrement by 1, 2, 4, or 8 with each transfer cycle. The same is true for the DMASRCWDTH and the source address respectively. When the DMASZx register decrements to zero, the corresponding RIS flag is set.

The DMAEN bit is cleared automatically when DMASZx decrements to zero and must be set again for another transfer to occur.

4.2.3.2 Block Transfer

In block transfer mode (DMATM = 1h), a transfer of a complete block of data occurs after one trigger. Block transfer mode is available in basic DMA channels only.

The DMASZx register defines the size of the block, and the DMADSTINCR and DMASRCINCR bits select if the destination address and the source address are incremented or decremented after each transfer of the block. If DMASZx = 0, no transfers occur.

The DMASAx, DMADAx, and DMASZx registers are copied into temporary registers. The temporary values of DMASAx and DMADAx are incremented or decremented after each transfer in the block. The DMADSTWDTH will indicate whether the destination address will increment or decrement by 1, 2, 4 or 8 with each transfer cycle. The same is true for the DMASRCWDTH and the source address respectively. The DMASZx register is decremented after each transfer of the block and shows the number of blocks remained. When DMATM = 01, the DMAEN bit is cleared automatically when DMASZx decrements to zero and must be set again for another transfer to occur.

The DMAEN bit is cleared after the completion of the block transfer and must be set again before another block transfer can be triggered. After a block transfer has started, another trigger signal that occurs during the block transfer is ignored.

4.2.3.3 Repeated Single Transfer

In repeated single transfer mode (DMATM = 2h), the DMA controller remains enabled with DMAEN = 1, and a transfer occurs every time a trigger occurs. Repeated single transfer modes are available in full-featured DMA channels only.

The DMASAx, DMADAx, and DMASZx registers are copied into temporary registers. The temporary values of DMASAx and DMADAx are incremented or decremented after each transfer. The DMASZx register is decremented after each register. The DMADSTWDTH will indicate whether the destination address will increment or decrement by 1, 2, 4, or 8 with each transfer cycle. The same is true for the DMASRCWDTH and the source address respectively. When the DMASZx register decrements to zero, it is reloaded from its temporary register and the corresponding RIS flag is set.

Note

When using repeated single transfer mode, the DMA does not support pausing and continuing a transfer by disabling a channel (to pause) and then re-enabling the channel (to continue).

4.2.3.4 Repeated Block Transfer

In repeated block transfer mode (DMATM = 11), the DMAEN bit remains set after completion of the block transfer. The next trigger after the completion of a repeated block transfer starts another block transfer. Repeated block transfer modes are available in full-featured DMA channels only.

The DMASAx, DMADAx, and DMASZx registers are copied into temporary registers. The temporary values of DMASAx and DMADAx are incremented or decremented after each transfer in the block. The DMADSTWDTH will indicate whether the destination address will increment or decrement by 1, 2, 4 or 8 with each transfer cycle. The same is true for the DMASRCWDTH and the source address respectively. The DMASZx register is decremented after each transfer of the block and shows the number of transfers remaining in the block.

4.2.3.5 Stride Mode

All transfer modes support a "stride" mode where the DMA source and destination can be incremented to a higher value (rather than +1) after a transfer. This is helpful for re-organizing the order of data between the source and destination.

To support incremental strides, set the DMADSTINCR and/or DMASRCINCR to STRIDE_n, where n is the number of destination and/or source increments. The real increments are based in terms of the definitions DMADSTWDTH and/or DMASRCWDTH, respectively. For example, if external ADC data is transmitted to the MCU as a six-word SPI frame, DMADSTINCR can be set to STRIDE_6 during a block transfer so that the destination address is incremented by 6 and the data is organized to make processing easier.

4.2.4 Extended Modes

In FULL channels, the DMA controller has two extended modes selected by the DMAEM bits as listed in [Table 4-3](#). Each channel is individually configurable for the extended mode. For example, channel 0 can be configured in table mode while channel 1 is configured in fill mode.

Table 4-3. DMA Extended Modes

DMAEM	Extended Mode	Description
00 , 01	Normal mode	Operation is defined by DMATM
10	Fill mode	Used to fill predefined data patterns into memory
11	Table mode	Used to help configure a table of peripheral control registers

4.2.4.1 Fill Mode

In fill mode (DMAEM = 10b), the DMA controller takes a predefined FILL pattern and writes the pattern to a user defined segment of memory. The DMATM bits are ignored and the automatic transfer mode used is "block transfer".

The DMASAx register is used as the FILL pattern data. The DMASRCINCR bit field is used to indicate whether the FILL pattern data should be constant or incremented/decremented with every write cycle. This feature allows for filling a memory block with a sequential pattern (for example, 0, 1, 2, 3, ...). The DMASRCWDTH bit field indicates the magnitude of increment of the FILL mode data. Refer to [Table 4-4](#) for how to use DMASRCWDTH in fill mode.

Table 4-4. DMASRCWDTH in Fill Mode

DMASRCWDTH	FILL Mode Data Increment Value
0	±1
1	±2
2	±4
3	±8

The destination registers and bit fields DMADAx, DMADSTINCR, and DMADSTWDTH all behave as expected and influence where and how in memory the FILL pattern is written.

4.2.4.2 Table Mode

In table mode (DMAEM = 11b), the DMA controller executes 2 reads from the source and one write to a determined destination. This feature can be leveraged to interpret a table of addresses and data and uses the DMA to efficiently program that data to their associated addresses without CPU intervention. Table mode allows you to parse through a table of addresses and data to configure peripheral memory mapped registers in a single block transfer.

The DMASRCWDTH bit field should be set to "3" (64-bit mode) and the DMADSTWDTH bit field should be set to "2" (32-bit mode). The DMASRCINCR bit field can be set to 10b to decrement the source address or 11b to increment the source address after transfers. DMASZ is set to represent the number of entries in the table and DMATM should be set to "01" for block transfer mode. DMADAx and DMADSTINCR are ignored in table mode and can be treated as "don't care" values.

The DMASAx register needs to be programmed with the start address of the table, which needs to be aligned to 64-bit data (that is, DMASAx[2:0] = "000"). The address stored in the table needs to be on the lower word of a 64-bit data (ADDR[2:0] = "000" while the data needs to be on the upper word of a 64-bit data (ADDR[2:0] = "100"). [Table 4-5](#) is an example of a table in memory compatible with the DMA table mode.

Table 4-5. Example of an Incremental Table Compatible with DMA Table Mode

Table Address	Table Data
0x0000	Address 0
0x0004	Data 0

Table 4-5. Example of an Incremental Table Compatible with DMA Table Mode (continued)

Table Address	Table Data
0x0008	Address 1
0x000C	Data 1

4.2.5 Initiating DMA Transfers

Each DMA channel is independently configured for its trigger source with DMATSEL. The DMATSEL bits should be modified only when the DMACTLx.DMAEN bit is 0; otherwise, unpredictable DMA triggers can occur.

See the device-specific data sheet for the list of triggers available, along with their respective DMATSEL values.

When selecting the trigger, the trigger must not have already occurred, or the transfer does not take place.

DMA channels can be internally triggered upon the completion of activity on another channel to support cascading. Completion of activity occurs when a DMA channel's DMASZ counter reaches zero. This is beneficial for applications where data can be retrieved, transferred, and/or error-checked without an interrupt or event configuration.

Set the DMATINT bit to internally trigger the next DMA channel based on the DMATSEL trigger source. Once the DMATSEL trigger occurs, the next DMA channel begins to automatically execute.

For example, if UART data is received and transmitted to SRAM through DMA channel 0 and DMATSEL is set to UART RX, then DMA channel 1 can be internally triggered when the UART is finished receiving data. If DMA channel 1 is configured to transmit the data from SRAM to CRC, then the DMA transfer will trigger once the UART data is received. In this case, the DMA channels are cascaded from Channel 0 to Channel 1.

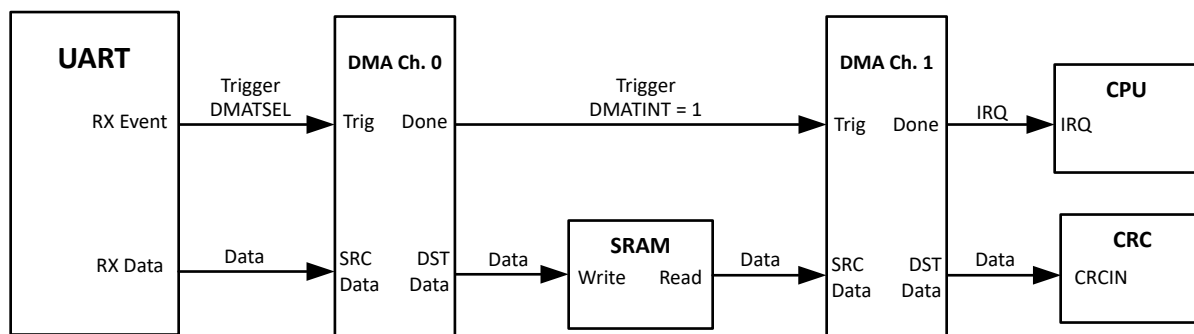


Figure 4-3. DMA Cascading Channels

4.2.6 Stopping DMA Transfers

A DMA block transfer in progress can be stopped by clearing the DMAEN bit. The DMA will stop after the completion of the ongoing transfer cycle and all the channel registers will stay in the current state. The block transfer can be continued as originally configured after setting the DMAEN bit again and resending the trigger. Please note that a trigger is necessary for halted transfer to resume.

Note

A single transfer in progress cannot be interrupted.

4.2.7 Channel Priorities

The default DMA channel priorities are DMA0 through DMA15. If two or three triggers happen simultaneously or are pending, the channel with the highest priority completes its transfer (single or block transfer) first, then the second priority channel, then the third priority channel. Transfers in progress are not halted if a higher-priority channel is triggered. The higher-priority channel waits until the transfer in progress completes before starting.

The DMA channel priorities are configurable with the ROUNDROBIN bit. When the ROUNDROBIN bit is set, the channel that completes a transfer becomes the lowest priority. The order of the priority of the channels always

stays the same, DMA0-DMA1-DMA2, for example, for three channels. When the ROUNDROBIN bit is cleared, the channel priority returns to the default priority.

Table 4-6. Round-Robin DMA Priority Example

Current DMA Priority	Transfer Occurs	New DMA Priority
DMA0 - DMA1 - DMA2	DMA1	DMA2 - DMA0 - DMA1
DMA2 - DMA0 - DMA1	DMA2	DMA0 - DMA1 - DMA2
DMA2 - DMA0 - DMA1	DMA0	DMA1 - DMA2 - DMA0

4.2.8 Burst Block Mode

The DMA module supports a burst block mode for suspending an active channel after a configurable number of transfers in order to service other pending channels. The burst block size is configurable by setting DMAPRIO.BURSTSZ to 8, 16, 32, or an infinite number of transfers. If a higher priority channel is pending after the burst block, the DMA will execute the higher priority channel and resume on the suspended channel once the higher priority channel is complete. If no other channel is pending, the priority logic assigns the control back to the block transfer for the next burst.

4.2.9 Using DMA with System Interrupts

System interrupt service routines are interrupted by DMA transfers. If an interrupt service routine or other routine must execute with no interruptions, the DMA controller should be disabled before executing the routine.

4.2.10 DMA Controller Interrupts

Each DMA channel has its own RIS flag. Each RIS flag is set in any mode when the corresponding DMASZx register counts to zero. If the corresponding MASK and RIS bits are set, an interrupt request is generated.

All RIS flags are prioritized, with DMA0 being the highest, and combined to source a single interrupt vector. The highest-priority enabled interrupt generates a number in the IIDX register. This number can be evaluated or added to the program counter (PC) to automatically enter the appropriate software routine.

Any access, read or write, of the IIDX register automatically resets the highest pending interrupt flag. If another interrupt flag is set, another interrupt is immediately generated after servicing the initial interrupt. For example, assume that DMA0 has the highest priority. If the DMA0-RIS and DMA2-RIS flags are set when the interrupt service routine accesses the IIDX register, DMA0-RIS is reset automatically. After the interrupt service routine is executed, the DMA2-RIS generates another interrupt.

4.2.11 DMA Trigger Event Status

The DMA controller supports dedicated DMA events. See [Section 7.2.2](#) for details on the DMA event trigger protocol. The idea is that the DMA can inform the event triggering peripheral about the status of the assigned DMA channel. This will allow the triggering peripheral to issue an interrupt itself after the completion of a repeated transfer, instead of the DMA issuing an interrupt event. The advantage is, that the DMA interrupt service routine does not need to keep track of the assigned function of the channel. As a result, the DMA triggering peripheral interrupt service routine will deal with the completion of the DMA transfer.

The status will reflect the value of the DMASZx register. If the last DMA transfer resulted in a size decrement to zero, the DMA will return the status of 1, indicating the end of the transfer. Otherwise the status will be 0.

Additionally, the DMA module can generate an early interrupt request to the CPU to indicate that a transfer will complete within a configurable number of transfers (1, 2, 4, 8, 32, 64, half-DMASZ).

An early IRQ event is enabled by setting DMAPREIRQ to the desired number of transfers. When the DMA has reached the number of transfers, the corresponding DMA channel's PREIRQ interrupt is set.

Early DMA interrupt generation is useful to:

- Reduce the interrupt latency in timing-critical applications where it would be beneficial to let the DMA preemptively generate the IRQ before the DMA transfer is complete
- Serve as a “progress notification” when scheduling other tasks for the CPU to complete

- Transfer weights of a neural network layer and notify the CPU to complete software configuration writes to the IP
- Implement a ping-pong buffer (by setting DMAPREIRQ to half)

Note

This feature is available on repeat-capable channels only.

4.2.12 DMA Operating Mode Support

The DMA supports triggered transfers in RUN mode, as well as in the SLEEP, STOP, and STANDBY low-power modes. Refer to the following sections for more details.

4.2.12.1 Transfer in RUN Mode

In RUN mode the system is fully operational. The CPU and all other peripherals and resources are available, therefore there is no restriction on the DMA functionality in RUN mode.

4.2.12.2 Transfer in SLEEP Mode

In SLEEP mode only the CPU is halted. All other peripherals and resources are available as when in RUN mode, therefore there is no restriction on the DMA functionality in SLEEP mode. All peripherals that can trigger a DMA transfer in RUN mode will also be able to trigger a DMA transfer in SLEEP mode.

4.2.12.3 Transfer in STOP Mode

In STOP mode the CPU is halted and the ULPCCLK is limited to 4 MHz operation. PD1 peripherals are disabled and in retention mode when applicable. Only PD0 peripherals are functional and therefore only PD0 peripherals are able to trigger a DMA transfer in STOP mode. The DMA itself is located in PD1 and is therefore in retention during STOP mode. The event manager will detect a DMA trigger event and request the PMU to enter a "suspended STOP" state. For more info on this state refer to [Section 2.1.2.7](#). While STOP mode is suspended, the DMA is fully functional and will work on the pending DMA trigger request. Once the DMA transfer is complete, the DMA will acknowledge the pending trigger event and the event subsystem removes the power mode request from the PMU. If the PMU has no other pending requests, the SoC will transition back into normal STOP mode.

4.2.12.4 Transfers in STANDBY Mode

In STANDBY mode the CPU is halted and the ULPCCLK is limited to 32 kHz operation. PD1 peripherals are disabled and in retention mode when applicable. Only PD0 peripherals are functional and therefore only PD0 peripherals are able to trigger a DMA transfer in STANDBY mode. The DMA itself is located in PD1 and therefore in retention during STANDBY mode. The event manager will detect a DMA trigger event and request the PMU to enter a "suspended STANDBY" state. For more info on this state refer to [Section 2.1.2.7](#). While STANDBY mode is suspended, the DMA is fully functional and will work on the pending DMA trigger request. Once the DMA transfer is complete, the DMA will acknowledge the pending trigger event and the event subsystem removes the power mode request from the PMU. If the PMU has no other pending requests, the SoC will transition back into normal STANDBY mode.

4.2.13 DMA Address and Data Errors

The DMA itself has the ability to flag address or data errors. Source or destination address errors can come from accessing a protected or non-existing memory range. If an address error occurs, the interrupt index IIDX[j].STAT flags a DMA address error (11h). Address error interrupts can be masked, set, and cleared using the ADDERR bit.

Note

The DMA itself does not perform range checking. If the DMA transfer occurs over a protected memory range, the destination data will report zeros (0h) for each byte of the DMA transaction that overlaps the protected or non-existing memory range.

Data errors can occur in SRAM or flash if it has an ECC or parity error. If a data error occurs, the interrupt index IIDX[j].STAT flags a DMA data error (12h). Data error interrupts can be masked, set, and cleared using the DATERR bit.

4.2.14 Interrupt and Event Support

The DMA module contains one [event publishers](#) and two [event subscribers](#).

- One event publisher (CPU_INT) manages DMA interrupt requests (IRQs) to the CPU subsystem through a [static event route](#).
- The second and third event (GEN_EVENT) are used to setup the generic event publishers and subscribers through [Generic route](#).

DMA events are summarized in [Table 4-7](#).

Table 4-7. DMA Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU interrupt	Publisher	DMA	CPU Subsystem	Static route	CPU_INT registers	Fixed interrupt route from TIMx to CPU
Generic publisher event	Publisher	DMA	Other peripherals	Generic route	GEN_EVENT and FPUB_1 registers	Configurable interrupt route from DMA to other peripherals
Generic subscriber event	Subscriber	Other peripherals	DMA	Generic route	FSUB_0	Configurable interrupt route from other peripherals to DMA
Generic subscriber event	Subscriber	Other peripherals	DMA	Generic route	FSUB_1	Configurable interrupt route from other peripherals to DMA

4.3 DMA Registers

Table 4-8 lists the memory-mapped registers for the DMA registers. All register offset addresses not listed in Table 4-8 should be considered as reserved locations and the register contents should not be modified.

Table 4-8. DMA Registers

Offset	Acronym	Register Name	Group	Section
400h	FSUB_0	Subscriber Port 0		Go
404h	FSUB_1	Subscriber Port 1		Go
444h	FPUB_1	Publisher Port 0		Go
1018h	PDBGCTL	Peripheral Debug Control		Go
1020h	IIDX	Interrupt index	CPU_INT	Go
1028h	IMASK	Interrupt mask	CPU_INT	Go
1030h	RIS	Raw interrupt status	CPU_INT	Go
1038h	MIS	Masked interrupt status	CPU_INT	Go
1040h	ISSET	Interrupt set	CPU_INT	Go
1048h	ICLR	Interrupt clear	CPU_INT	Go
1050h	IIDX	Interrupt index	GEN_EVENT	Go
1058h	IMASK	Interrupt mask	GEN_EVENT	Go
1060h	RIS	Raw interrupt status	GEN_EVENT	Go
1068h	MIS	Masked interrupt status	GEN_EVENT	Go
1070h	ISSET	Interrupt set	GEN_EVENT	Go
1078h	ICLR	Interrupt clear	GEN_EVENT	Go
10E0h	EVT_MODE	Event Mode		Go
10FCh	DESC	Module Description		Go
1100h	DMAprio	DMA Channel Priority Control		Go
1110h + formula	DMATCTL[j]	DMA Trigger Select		Go
1200h + formula	DMACTL[j]	DMA Channel Control		Go
1204h + formula	DMAA[j]	DMA Channel Source Address		Go
1208h + formula	DMAA[j]	DMA Channel Destination Address		Go
120Ch + formula	DMAZ[j]	DMA Channel Size		Go

Complex bit access types are encoded to fit into small table cells. Table 4-9 shows the codes that are used for access types in this section.

Table 4-9. DMA Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
W	W	Write
Reset or Default Value		
-n		Value after reset or the default value
Register Array Variables		

Table 4-9. DMA Access Type Codes (continued)

Access Type	Code	Description
i,j,k,l,m,n		When these variables are used in a register name, an offset, or an address, they refer to the value of a register array where the register is part of a group of repeating registers. The register groups form a hierarchical structure and the array is represented with a formula.
y		When this variable is used in a register name, an offset, or an address it refers to the value of a register array.

4.3.1 FSUB_0 (Offset = 400h) [Reset = 00000000h]

FSUB_0 is shown in [Figure 4-4](#) and described in [Table 4-10](#).

Return to the [Summary Table](#).

Subscriber port

Figure 4-4. FSUB_0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								CHANID							
R/W-0h																								R/W-0h							

Table 4-10. FSUB_0 Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R/W	0h	
7-0	CHANID	R/W	0h	0 = disconnected. 1-255 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected FFh = Consult your device data sheet as the actual allowed maximum may be less than 255.

4.3.2 FSUB_1 (Offset = 404h) [Reset = 00000000h]

FSUB_1 is shown in [Figure 4-5](#) and described in [Table 4-11](#).

Return to the [Summary Table](#).

Subscriber port

Figure 4-5. FSUB_1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								CHANID							
R/W-0h																								R/W-0h							

Table 4-11. FSUB_1 Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R/W	0h	
7-0	CHANID	R/W	0h	0 = disconnected. 1-255 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected FFh = Consult your device data sheet as the actual allowed maximum may be less than 255.

4.3.3 FPUB_1 (Offset = 444h) [Reset = 00000000h]

FPUB_1 is shown in [Figure 4-6](#) and described in [Table 4-12](#).

Return to the [Summary Table](#).

Publisher port

Figure 4-6. FPUB_1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								CHANID							
R/W-0h																								R/W-0h							

Table 4-12. FPUB_1 Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R/W	0h	
7-0	CHANID	R/W	0h	0 = disconnected. 1-255 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected FFh = Consult your device data sheet as the actual allowed maximum may be less than 255.

4.3.4 PDBGCTL (Offset = 1018h) [Reset = 00000003h]

PDBGCTL is shown in [Figure 4-7](#) and described in [Table 4-13](#).

Return to the [Summary Table](#).

This register can be used by the software developer to control the behavior of the peripheral relative to the 'Core Halted' input

Figure 4-7. PDBGCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED						SOFT	FREE
R/W-						R/W-1h	R/W-1h

Table 4-13. PDBGCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R/W	0h	
1	SOFT	R/W	1h	Soft halt boundary control. This function is only available, if FREE is set to 'STOP' 0h = The peripheral will halt immediately, even if the resultant state will result in corruption if the system is restarted 1h = The peripheral blocks the debug freeze until it has reached a boundary where it can resume without corruption
0	FREE	R/W	1h	Free run control 0h = The peripheral freezes functionality while the Core Halted input is asserted and resumes when it is deasserted. 1h = The peripheral ignores the state of the Core Halted input

4.3.5 IIDX (Offset = 1020h) [Reset = 00000000h]

IIDX is shown in [Figure 4-8](#) and described in [Table 4-14](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, . . . IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred.

On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in RIS [RIS] and MIS [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Note: The number of DMACH is device dependent. Please consult the data sheet of the specific device to map which channel number is implemented.

Figure 4-8. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 4-14. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 00h = No bit is set means there is no pending interrupt request 01h = DMA Channel 0 size counter reached zero (DMASZ=0). 02h = DMA Channel 1 size counter reached zero (DMASZ=0). 03h = DMA Channel 2 size counter reached zero (DMASZ=0). 04h = DMA Channel 3 size counter reached zero (DMASZ=0). 05h = DMA Channel 4 size counter reached zero (DMASZ=0). 06h = DMA Channel 5 size counter reached zero (DMASZ=0). 07h = DMA Channel 6 size counter reached zero (DMASZ=0). 08h = DMA Channel 7 size counter reached zero (DMASZ=0). 09h = DMA Channel 8 size counter reached zero (DMASZ=0). 0Ah = DMA Channel 9 size counter reached zero (DMASZ=0). 0Bh = DMA Channel 10 size counter reached zero (DMASZ=0). 0Ch = DMA Channel 11 size counter reached zero (DMASZ=0). 0Dh = DMA Channel 12 size counter reached zero (DMASZ=0). 0Eh = DMA Channel 13 size counter reached zero (DMASZ=0). 0Fh = DMA Channel 14 size counter reached zero (DMASZ=0). 10h = DMA Channel 15 size counter reached zero (DMASZ=0). 11h = PRE-IRQ event for DMA Channel 0. 12h = PRE-IRQ event for DMA Channel 1. 13h = PRE-IRQ event for DMA Channel 2. 14h = PRE-IRQ event for DMA Channel 3. 15h = PRE-IRQ event for DMA Channel 4. 16h = PRE-IRQ event for DMA Channel 5. 17h = PRE-IRQ event for DMA Channel 6. 18h = PRE-IRQ event for DMA Channel 7. 19h = DMA address error, SRC address not reachable. 1Ah = DMA data error, SRC data might be corrupted (PAR or ECC error).

4.3.6 IMASK (Offset = 1028h) [Reset = 00000000h]

IMASK is shown in [Figure 4-9](#) and described in [Table 4-15](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then the corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX [IIDX], as well as MIS [MIS].

Note: The number of DMACH is device dependent. Please consult the data sheet of the specific device to map which channel number is implemented.

Figure 4-9. IMASK

31	30	29	28	27	26	25	24
RESERVED						DATAERR	ADDRERR
R/W-0h						R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
PREIRQCH7	PREIRQCH6	PREIRQCH5	PREIRQCH4	PREIRQCH3	PREIRQCH2	PREIRQCH1	PREIRQCH0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
DMACH15	DMACH14	DMACH13	DMACH12	DMACH11	DMACH10	DMACH9	DMACH8
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
DMACH7	DMACH6	DMACH5	DMACH4	DMACH3	DMACH2	DMACH1	DMACH0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 4-15. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-26	RESERVED	R/W	0h	
25	DATAERR	R/W	0h	DMA data error, SRC data might be corrupted (PAR or ECC error). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
24	ADDRERR	R/W	0h	DMA address error, SRC address not reachable. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
23	PREIRQCH7	R/W	0h	Pre-IRQ for Channel 7. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
22	PREIRQCH6	R/W	0h	Pre-IRQ for Channel 6. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
21	PREIRQCH5	R/W	0h	Pre-IRQ for Channel 5. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
20	PREIRQCH4	R/W	0h	Pre-IRQ for Channel 4. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
19	PREIRQCH3	R/W	0h	Pre-IRQ for Channel 3. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
18	PREIRQCH2	R/W	0h	Pre-IRQ for Channel 2. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
17	PREIRQCH1	R/W	0h	Pre-IRQ for Channel 1. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit

Table 4-15. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
16	PREIRQCH0	R/W	0h	Pre-IRQ for Channel 0. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
15	DMACH15	R/W	0h	DMA Channel 15 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
14	DMACH14	R/W	0h	DMA Channel 14 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
13	DMACH13	R/W	0h	DMA Channel 13 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
12	DMACH12	R/W	0h	DMA Channel 12 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
11	DMACH11	R/W	0h	DMA Channel 11 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
10	DMACH10	R/W	0h	DMA Channel 10 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
9	DMACH9	R/W	0h	DMA Channel 9 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
8	DMACH8	R/W	0h	DMA Channel 8 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
7	DMACH7	R/W	0h	DMA Channel 7 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
6	DMACH6	R/W	0h	DMA Channel 6 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
5	DMACH5	R/W	0h	DMA Channel 5 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
4	DMACH4	R/W	0h	DMA Channel 4 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
3	DMACH3	R/W	0h	DMA Channel 3 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
2	DMACH2	R/W	0h	DMA Channel 2 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit

Table 4-15. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1	DMACH1	R/W	0h	DMA Channel 1 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
0	DMACH0	R/W	0h	DMA Channel 0 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit

4.3.7 RIS (Offset = 1030h) [Reset = 00000000h]

RIS is shown in [Figure 4-10](#) and described in [Table 4-16](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR [ICLR] register bit even if the corresponding IMASK [IMASK] bit is not enabled.

Note: The number of DMACH is device dependent. Please consult the data sheet of the specific device to map which channel number is implemented.

Figure 4-10. RIS

31	30	29	28	27	26	25	24
RESERVED						DATAERR	ADDRERR
R-0h						R-0h	R-0h
23	22	21	20	19	18	17	16
PREIRQCH7	PREIRQCH6	PREIRQCH5	PREIRQCH4	PREIRQCH3	PREIRQCH2	PREIRQCH1	PREIRQCH0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
DMACH15	DMACH14	DMACH13	DMACH12	DMACH11	DMACH10	DMACH9	DMACH8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
DMACH7	DMACH6	DMACH5	DMACH4	DMACH3	DMACH2	DMACH1	DMACH0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 4-16. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-26	RESERVED	R	0h	
25	DATAERR	R	0h	DMA data error, SRC data might be corrupted (PAR or ECC error). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
24	ADDRERR	R	0h	DMA address error, SRC address not reachable. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
23	PREIRQCH7	R	0h	Pre-IRQ for Channel 7. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
22	PREIRQCH6	R	0h	Pre-IRQ for Channel 6. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
21	PREIRQCH5	R	0h	Pre-IRQ for Channel 5. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
20	PREIRQCH4	R	0h	Pre-IRQ for Channel 4. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
19	PREIRQCH3	R	0h	Pre-IRQ for Channel 3. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
18	PREIRQCH2	R	0h	Pre-IRQ for Channel 2. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit

Table 4-16. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
17	PREIRQCH1	R	0h	Pre-IRQ for Channel 1. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
16	PREIRQCH0	R	0h	Pre-IRQ for Channel 0. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
15	DMACH15	R	0h	DMA Channel 15 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
14	DMACH14	R	0h	DMA Channel 14 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
13	DMACH13	R	0h	DMA Channel 13 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
12	DMACH12	R	0h	DMA Channel 12 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
11	DMACH11	R	0h	DMA Channel 11 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
10	DMACH10	R	0h	DMA Channel 10 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
9	DMACH9	R	0h	DMA Channel 9 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
8	DMACH8	R	0h	DMA Channel 8 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
7	DMACH7	R	0h	DMA Channel 7 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
6	DMACH6	R	0h	DMA Channel 6 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
5	DMACH5	R	0h	DMA Channel 5 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
4	DMACH4	R	0h	DMA Channel 4 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
3	DMACH3	R	0h	DMA Channel 3 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred

Table 4-16. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	DMACH2	R	0h	DMA Channel 2 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
1	DMACH1	R	0h	DMA Channel 1 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
0	DMACH0	R	0h	DMA Channel 0 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred

4.3.8 MIS (Offset = 1038h) [Reset = 00000000h]

MIS is shown in [Figure 4-11](#) and described in [Table 4-17](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK [IMASK] and RIS [RIS] registers.

Note: The number of DMACH is device dependent. Please consult the data sheet of the specific device to map which channel number is implemented.

Figure 4-11. MIS

31	30	29	28	27	26	25	24
RESERVED						DATAERR	ADDRERR
R-0h						R-0h	R-0h
23	22	21	20	19	18	17	16
PREIRQCH7	PREIRQCH6	PREIRQCH5	PREIRQCH4	PREIRQCH3	PREIRQCH2	PREIRQCH1	PREIRQCH0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
DMACH15	DMACH14	DMACH13	DMACH12	DMACH11	DMACH10	DMACH9	DMACH8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
DMACH7	DMACH6	DMACH5	DMACH4	DMACH3	DMACH2	DMACH1	DMACH0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 4-17. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-26	RESERVED	R	0h	
25	DATAERR	R	0h	DMA data error, SRC data might be corrupted (PAR or ECC error). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
24	ADDRERR	R	0h	DMA address error, SRC address not reachable. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
23	PREIRQCH7	R	0h	Pre-IRQ for Channel 7. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
22	PREIRQCH6	R	0h	Pre-IRQ for Channel 6. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
21	PREIRQCH5	R	0h	Pre-IRQ for Channel 5. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
20	PREIRQCH4	R	0h	Pre-IRQ for Channel 4. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
19	PREIRQCH3	R	0h	Pre-IRQ for Channel 3. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
18	PREIRQCH2	R	0h	Pre-IRQ for Channel 2. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
17	PREIRQCH1	R	0h	Pre-IRQ for Channel 1. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit

Table 4-17. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
16	PREIRQCH0	R	0h	Pre-IRQ for Channel 0. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
15	DMACH15	R	0h	DMA Channel 15 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
14	DMACH14	R	0h	DMA Channel 14 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
13	DMACH13	R	0h	DMA Channel 13 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
12	DMACH12	R	0h	DMA Channel 12 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
11	DMACH11	R	0h	DMA Channel 11 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
10	DMACH10	R	0h	DMA Channel 10 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
9	DMACH9	R	0h	DMA Channel 9 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
8	DMACH8	R	0h	DMA Channel 8 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
7	DMACH7	R	0h	DMA Channel 7 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
6	DMACH6	R	0h	DMA Channel 6 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
5	DMACH5	R	0h	DMA Channel 5 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
4	DMACH4	R	0h	DMA Channel 4 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
3	DMACH3	R	0h	DMA Channel 3 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
2	DMACH2	R	0h	DMA Channel 2 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred

Table 4-17. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1	DMACH1	R	0h	DMA Channel 1 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
0	DMACH0	R	0h	DMA Channel 0 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred

4.3.9 ISET (Offset = 1040h) [Reset = 00000000h]

ISET is shown in [Figure 4-12](#) and described in [Table 4-18](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS [RIS] bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS [MIS] bit is also set.

Note: The number of DMACH is device dependent. Please consult the data sheet of the specific device to map which channel number is implemented.

Figure 4-12. ISET

31	30	29	28	27	26	25	24
RESERVED						DATAERR	ADDRERR
W-0h						W-0h	W-0h
23	22	21	20	19	18	17	16
PREIRQCH7	PREIRQCH6	PREIRQCH5	PREIRQCH4	PREIRQCH3	PREIRQCH2	PREIRQCH1	PREIRQCH0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
DMACH15	DMACH14	DMACH13	DMACH12	DMACH11	DMACH10	DMACH9	DMACH8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
DMACH7	DMACH6	DMACH5	DMACH4	DMACH3	DMACH2	DMACH1	DMACH0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 4-18. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-26	RESERVED	W	0h	
25	DATAERR	W	0h	DMA data error, SRC data might be corrupted (PAR or ECC error). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
24	ADDRERR	W	0h	DMA address error, SRC address not reachable. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
23	PREIRQCH7	W	0h	Pre-IRQ for Channel 7. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
22	PREIRQCH6	W	0h	Pre-IRQ for Channel 6. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
21	PREIRQCH5	W	0h	Pre-IRQ for Channel 5. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
20	PREIRQCH4	W	0h	Pre-IRQ for Channel 4. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
19	PREIRQCH3	W	0h	Pre-IRQ for Channel 3. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
18	PREIRQCH2	W	0h	Pre-IRQ for Channel 2. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit

Table 4-18. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
17	PREIRQCH1	W	0h	Pre-IRQ for Channel 1. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
16	PREIRQCH0	W	0h	Pre-IRQ for Channel 0. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
15	DMACH15	W	0h	DMA Channel 0 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
14	DMACH14	W	0h	DMA Channel 14 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
13	DMACH13	W	0h	DMA Channel 13 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
12	DMACH12	W	0h	DMA Channel 12 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
11	DMACH11	W	0h	DMA Channel 11 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
10	DMACH10	W	0h	DMA Channel 0 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
9	DMACH9	W	0h	DMA Channel 9 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
8	DMACH8	W	0h	DMA Channel 8 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
7	DMACH7	W	0h	DMA Channel 7 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
6	DMACH6	W	0h	DMA Channel 6 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
5	DMACH5	W	0h	DMA Channel 5 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
4	DMACH4	W	0h	DMA Channel 4 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
3	DMACH3	W	0h	DMA Channel 3 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt

Table 4-18. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	DMACH2	W	0h	DMA Channel 2 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
1	DMACH1	W	0h	DMA Channel 1 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
0	DMACH0	W	0h	DMA Channel 0 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt

4.3.10 ICLR (Offset = 1048h) [Reset = 00000000h]

ICLR is shown in [Figure 4-13](#) and described in [Table 4-19](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Note: The number of DMACH is device dependent. Please consult the data sheet of the specific device to map which channel number is implemented.

Figure 4-13. ICLR

31	30	29	28	27	26	25	24
RESERVED						DATAERR	ADDRERR
W-0h						W-0h	W-0h
23	22	21	20	19	18	17	16
PREIRQCH7	PREIRQCH6	PREIRQCH5	PREIRQCH4	PREIRQCH3	PREIRQCH2	PREIRQCH1	PREIRQCH0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
DMACH15	DMACH14	DMACH13	DMACH12	DMACH11	DMACH10	DMACH9	DMACH8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
DMACH7	DMACH6	DMACH5	DMACH4	DMACH3	DMACH2	DMACH1	DMACH0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 4-19. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-26	RESERVED	W	0h	
25	DATAERR	W	0h	DMA data error, SRC data might be corrupted (PAR or ECC error). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
24	ADDRERR	W	0h	DMA address error, SRC address not reachable. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
23	PREIRQCH7	W	0h	Pre-IRQ for Channel 7. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
22	PREIRQCH6	W	0h	Pre-IRQ for Channel 6. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
21	PREIRQCH5	W	0h	Pre-IRQ for Channel 5. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
20	PREIRQCH4	W	0h	Pre-IRQ for Channel 4. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
19	PREIRQCH3	W	0h	Pre-IRQ for Channel 3. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
18	PREIRQCH2	W	0h	Pre-IRQ for Channel 2. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
17	PREIRQCH1	W	0h	Pre-IRQ for Channel 1. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit

Table 4-19. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
16	PREIRQCH0	W	0h	Pre-IRQ for Channel 0. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
15	DMACH15	W	0h	DMA Channel 15 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
14	DMACH14	W	0h	DMA Channel 14 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
13	DMACH13	W	0h	DMA Channel 13 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
12	DMACH12	W	0h	DMA Channel 12 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
11	DMACH11	W	0h	DMA Channel 11 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
10	DMACH10	W	0h	DMA Channel 10 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
9	DMACH9	W	0h	DMA Channel 9 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
8	DMACH8	W	0h	DMA Channel 8 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
7	DMACH7	W	0h	DMA Channel 7 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
6	DMACH6	W	0h	DMA Channel 6 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
5	DMACH5	W	0h	DMA Channel 5 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
4	DMACH4	W	0h	DMA Channel 4 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
3	DMACH3	W	0h	DMA Channel 3 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
2	DMACH2	W	0h	DMA Channel 2 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt

Table 4-19. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1	DMACH1	W	0h	DMA Channel 1 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
0	DMACH0	W	0h	DMA Channel 0 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt

4.3.11 IIDX (Offset = 1050h) [Reset = 00000000h]

IIDX is shown in [Figure 4-14](#) and described in [Table 4-20](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, . . . IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred.

On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in RIS [RIS] and MIS [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Note: The number of DMACH is device dependent. Please consult the data sheet of the specific device to map which channel number is implemented.

Figure 4-14. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 4-20. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 00h = No bit is set means there is no pending interrupt request 01h = DMA Channel 0 size counter reached zero (DMASZ=0). 02h = DMA Channel 1 size counter reached zero (DMASZ=0). 03h = DMA Channel 2 size counter reached zero (DMASZ=0). 04h = DMA Channel 3 size counter reached zero (DMASZ=0). 05h = DMA Channel 4 size counter reached zero (DMASZ=0). 06h = DMA Channel 5 size counter reached zero (DMASZ=0). 07h = DMA Channel 6 size counter reached zero (DMASZ=0). 08h = DMA Channel 7 size counter reached zero (DMASZ=0). 09h = DMA Channel 8 size counter reached zero (DMASZ=0). 0Ah = DMA Channel 9 size counter reached zero (DMASZ=0). 0Bh = DMA Channel 10 size counter reached zero (DMASZ=0). 0Ch = DMA Channel 11 size counter reached zero (DMASZ=0). 0Dh = DMA Channel 12 size counter reached zero (DMASZ=0). 0Eh = DMA Channel 13 size counter reached zero (DMASZ=0). 0Fh = DMA Channel 14 size counter reached zero (DMASZ=0). 10h = DMA Channel 15 size counter reached zero (DMASZ=0). 11h = PRE-IRQ event for DMA Channel 0. 12h = PRE-IRQ event for DMA Channel 1. 13h = PRE-IRQ event for DMA Channel 2. 14h = PRE-IRQ event for DMA Channel 3. 15h = PRE-IRQ event for DMA Channel 4. 16h = PRE-IRQ event for DMA Channel 5. 17h = PRE-IRQ event for DMA Channel 6. 18h = PRE-IRQ event for DMA Channel 7. 19h = DMA address error, SRC address not reachable. 1Ah = DMA data error, SRC data might be corrupted (PAR or ECC error).

4.3.12 IMASK (Offset = 1058h) [Reset = 00000000h]

IMASK is shown in [Figure 4-15](#) and described in [Table 4-21](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then the corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX [IIDX], as well as MIS [MIS].

Note: The number of DMACH is device dependent. Please consult the data sheet of the specific device to map which channel number is implemented.

Figure 4-15. IMASK

31	30	29	28	27	26	25	24
RESERVED						DATAERR	ADDRERR
R/W-0h						R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
PREIRQCH7	PREIRQCH6	PREIRQCH5	PREIRQCH4	PREIRQCH3	PREIRQCH2	PREIRQCH1	PREIRQCH0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
DMACH15	DMACH14	DMACH13	DMACH12	DMACH11	DMACH10	DMACH9	DMACH8
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
DMACH7	DMACH6	DMACH5	DMACH4	DMACH3	DMACH2	DMACH1	DMACH0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 4-21. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-26	RESERVED	R/W	0h	
25	DATAERR	R/W	0h	DMA data error, SRC data might be corrupted (PAR or ECC error). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
24	ADDRERR	R/W	0h	DMA address error, SRC address not reachable. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
23	PREIRQCH7	R/W	0h	Pre-IRQ for Channel 7. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
22	PREIRQCH6	R/W	0h	Pre-IRQ for Channel 6. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
21	PREIRQCH5	R/W	0h	Pre-IRQ for Channel 5. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
20	PREIRQCH4	R/W	0h	Pre-IRQ for Channel 4. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
19	PREIRQCH3	R/W	0h	Pre-IRQ for Channel 3. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
18	PREIRQCH2	R/W	0h	Pre-IRQ for Channel 2. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
17	PREIRQCH1	R/W	0h	Pre-IRQ for Channel 1. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit

Table 4-21. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
16	PREIRQCH0	R/W	0h	Pre-IRQ for Channel 0. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
15	DMACH15	R/W	0h	DMA Channel 15 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
14	DMACH14	R/W	0h	DMA Channel 14 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
13	DMACH13	R/W	0h	DMA Channel 13 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
12	DMACH12	R/W	0h	DMA Channel 12 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
11	DMACH11	R/W	0h	DMA Channel 11 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
10	DMACH10	R/W	0h	DMA Channel 10 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
9	DMACH9	R/W	0h	DMA Channel 9 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
8	DMACH8	R/W	0h	DMA Channel 8 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
7	DMACH7	R/W	0h	DMA Channel 7 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
6	DMACH6	R/W	0h	DMA Channel 6 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
5	DMACH5	R/W	0h	DMA Channel 5 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
4	DMACH4	R/W	0h	DMA Channel 4 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
3	DMACH3	R/W	0h	DMA Channel 3 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
2	DMACH2	R/W	0h	DMA Channel 2 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit

Table 4-21. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1	DMACH1	R/W	0h	DMA Channel 1 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
0	DMACH0	R/W	0h	DMA Channel 0 interrupt signal. Size counter reached zero (DMASZ=0). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit

4.3.13 RIS (Offset = 1060h) [Reset = 00000000h]

RIS is shown in [Figure 4-16](#) and described in [Table 4-22](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR [ICLR] register bit even if the corresponding IMASK [IMASK] bit is not enabled.

Note: The number of DMACH is device dependent. Please consult the data sheet of the specific device to map which channel number is implemented.

Figure 4-16. RIS

31	30	29	28	27	26	25	24
RESERVED						DATAERR	ADDRERR
R-0h						R-0h	R-0h
23	22	21	20	19	18	17	16
PREIRQCH7	PREIRQCH6	PREIRQCH5	PREIRQCH4	PREIRQCH3	PREIRQCH2	PREIRQCH1	PREIRQCH0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
DMACH15	DMACH14	DMACH13	DMACH12	DMACH11	DMACH10	DMACH9	DMACH8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
DMACH7	DMACH6	DMACH5	DMACH4	DMACH3	DMACH2	DMACH1	DMACH0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 4-22. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-26	RESERVED	R	0h	
25	DATAERR	R	0h	DMA data error, SRC data might be corrupted (PAR or ECC error). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
24	ADDRERR	R	0h	DMA address error, SRC address not reachable. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
23	PREIRQCH7	R	0h	Pre-IRQ for Channel 7. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
22	PREIRQCH6	R	0h	Pre-IRQ for Channel 6. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
21	PREIRQCH5	R	0h	Pre-IRQ for Channel 5. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
20	PREIRQCH4	R	0h	Pre-IRQ for Channel 4. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
19	PREIRQCH3	R	0h	Pre-IRQ for Channel 3. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
18	PREIRQCH2	R	0h	Pre-IRQ for Channel 2. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit

Table 4-22. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
17	PREIRQCH1	R	0h	Pre-IRQ for Channel 1. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
16	PREIRQCH0	R	0h	Pre-IRQ for Channel 0. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
15	DMACH15	R	0h	DMA Channel 15 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
14	DMACH14	R	0h	DMA Channel 14 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
13	DMACH13	R	0h	DMA Channel 13 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
12	DMACH12	R	0h	DMA Channel 12 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
11	DMACH11	R	0h	DMA Channel 11 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
10	DMACH10	R	0h	DMA Channel 10 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
9	DMACH9	R	0h	DMA Channel 9 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
8	DMACH8	R	0h	DMA Channel 8 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
7	DMACH7	R	0h	DMA Channel 7 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
6	DMACH6	R	0h	DMA Channel 6 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
5	DMACH5	R	0h	DMA Channel 5 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
4	DMACH4	R	0h	DMA Channel 4 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
3	DMACH3	R	0h	DMA Channel 3 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred

Table 4-22. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	DMACH2	R	0h	DMA Channel 2 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
1	DMACH1	R	0h	DMA Channel 1 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred
0	DMACH0	R	0h	DMA Channel 0 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur 1h = Interrupt occurred

4.3.14 MIS (Offset = 1068h) [Reset = 00000000h]

MIS is shown in [Figure 4-17](#) and described in [Table 4-23](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK [IMASK] and RIS [RIS] registers.

Note: The number of DMACH is device dependent. Please consult the data sheet of the specific device to map which channel number is implemented.

Figure 4-17. MIS

31	30	29	28	27	26	25	24
RESERVED						DATAERR	ADDRERR
R-0h						R-0h	R-0h
23	22	21	20	19	18	17	16
PREIRQCH7	PREIRQCH6	PREIRQCH5	PREIRQCH4	PREIRQCH3	PREIRQCH2	PREIRQCH1	PREIRQCH0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
DMACH15	DMACH14	DMACH13	DMACH12	DMACH11	DMACH10	DMACH9	DMACH8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
DMACH7	DMACH6	DMACH5	DMACH4	DMACH3	DMACH2	DMACH1	DMACH0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 4-23. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-26	RESERVED	R	0h	
25	DATAERR	R	0h	DMA data error, SRC data might be corrupted (PAR or ECC error). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
24	ADDRERR	R	0h	DMA address error, SRC address not reachable. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
23	PREIRQCH7	R	0h	Pre-IRQ for Channel 7. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
22	PREIRQCH6	R	0h	Pre-IRQ for Channel 6. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
21	PREIRQCH5	R	0h	Pre-IRQ for Channel 5. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
20	PREIRQCH4	R	0h	Pre-IRQ for Channel 4. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
19	PREIRQCH3	R	0h	Pre-IRQ for Channel 3. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
18	PREIRQCH2	R	0h	Pre-IRQ for Channel 2. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
17	PREIRQCH1	R	0h	Pre-IRQ for Channel 1. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit

Table 4-23. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
16	PREIRQCH0	R	0h	Pre-IRQ for Channel 0. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
15	DMACH15	R	0h	DMA Channel 15 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
14	DMACH14	R	0h	DMA Channel 14 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
13	DMACH13	R	0h	DMA Channel 13 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
12	DMACH12	R	0h	DMA Channel 12 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
11	DMACH11	R	0h	DMA Channel 11 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
10	DMACH10	R	0h	DMA Channel 10 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
9	DMACH9	R	0h	DMA Channel 9 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
8	DMACH8	R	0h	DMA Channel 8 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
7	DMACH7	R	0h	DMA Channel 7 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
6	DMACH6	R	0h	DMA Channel 6 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
5	DMACH5	R	0h	DMA Channel 5 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
4	DMACH4	R	0h	DMA Channel 4 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
3	DMACH3	R	0h	DMA Channel 3 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
2	DMACH2	R	0h	DMA Channel 2 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred

Table 4-23. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1	DMACH1	R	0h	DMA Channel 1 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred
0	DMACH0	R	0h	DMA Channel 0 interrupt signals that size counter reached zero (DMASZ=0). 0h = Interrupt did not occur or is masked out 1h = Interrupt occurred

4.3.15 ISET (Offset = 1070h) [Reset = 00000000h]

ISET is shown in [Figure 4-18](#) and described in [Table 4-24](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS [RIS] bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS [MIS] bit is also set.

Note: The number of DMACH is device dependent. Please consult the data sheet of the specific device to map which channel number is implemented.

Figure 4-18. ISET

31	30	29	28	27	26	25	24
RESERVED						DATAERR	ADDRERR
W-0h						W-0h	W-0h
23	22	21	20	19	18	17	16
PREIRQCH7	PREIRQCH6	PREIRQCH5	PREIRQCH4	PREIRQCH3	PREIRQCH2	PREIRQCH1	PREIRQCH0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
DMACH15	DMACH14	DMACH13	DMACH12	DMACH11	DMACH10	DMACH9	DMACH8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
DMACH7	DMACH6	DMACH5	DMACH4	DMACH3	DMACH2	DMACH1	DMACH0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 4-24. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-26	RESERVED	W	0h	
25	DATAERR	W	0h	DMA data error, SRC data might be corrupted (PAR or ECC error). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
24	ADDRERR	W	0h	DMA address error, SRC address not reachable. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
23	PREIRQCH7	W	0h	Pre-IRQ for Channel 7. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
22	PREIRQCH6	W	0h	Pre-IRQ for Channel 6. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
21	PREIRQCH5	W	0h	Pre-IRQ for Channel 5. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
20	PREIRQCH4	W	0h	Pre-IRQ for Channel 4. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
19	PREIRQCH3	W	0h	Pre-IRQ for Channel 3. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
18	PREIRQCH2	W	0h	Pre-IRQ for Channel 2. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit

Table 4-24. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
17	PREIRQCH1	W	0h	Pre-IRQ for Channel 1. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
16	PREIRQCH0	W	0h	Pre-IRQ for Channel 0. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
15	DMACH15	W	0h	DMA Channel 0 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
14	DMACH14	W	0h	DMA Channel 14 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
13	DMACH13	W	0h	DMA Channel 13 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
12	DMACH12	W	0h	DMA Channel 12 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
11	DMACH11	W	0h	DMA Channel 11 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
10	DMACH10	W	0h	DMA Channel 0 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
9	DMACH9	W	0h	DMA Channel 9 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
8	DMACH8	W	0h	DMA Channel 8 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
7	DMACH7	W	0h	DMA Channel 7 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
6	DMACH6	W	0h	DMA Channel 6 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
5	DMACH5	W	0h	DMA Channel 5 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
4	DMACH4	W	0h	DMA Channel 4 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
3	DMACH3	W	0h	DMA Channel 3 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt

Table 4-24. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	DMACH2	W	0h	DMA Channel 2 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
1	DMACH1	W	0h	DMA Channel 1 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt
0	DMACH0	W	0h	DMA Channel 0 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Set interrupt

4.3.16 ICLR (Offset = 1078h) [Reset = 00000000h]

ICLR is shown in [Figure 4-19](#) and described in [Table 4-25](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Note: The number of DMACH is device dependent. Please consult the data sheet of the specific device to map which channel number is implemented.

Figure 4-19. ICLR

31	30	29	28	27	26	25	24
RESERVED						DATAERR	ADDRERR
W-0h						W-0h	W-0h
23	22	21	20	19	18	17	16
PREIRQCH7	PREIRQCH6	PREIRQCH5	PREIRQCH4	PREIRQCH3	PREIRQCH2	PREIRQCH1	PREIRQCH0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
DMACH15	DMACH14	DMACH13	DMACH12	DMACH11	DMACH10	DMACH9	DMACH8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
DMACH7	DMACH6	DMACH5	DMACH4	DMACH3	DMACH2	DMACH1	DMACH0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 4-25. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-26	RESERVED	W	0h	
25	DATAERR	W	0h	DMA data error, SRC data might be corrupted (PAR or ECC error). 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
24	ADDRERR	W	0h	DMA address error, SRC address not reachable. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
23	PREIRQCH7	W	0h	Pre-IRQ for Channel 7. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
22	PREIRQCH6	W	0h	Pre-IRQ for Channel 6. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
21	PREIRQCH5	W	0h	Pre-IRQ for Channel 5. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
20	PREIRQCH4	W	0h	Pre-IRQ for Channel 4. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
19	PREIRQCH3	W	0h	Pre-IRQ for Channel 3. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
18	PREIRQCH2	W	0h	Pre-IRQ for Channel 2. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
17	PREIRQCH1	W	0h	Pre-IRQ for Channel 1. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit

Table 4-25. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
16	PREIRQCH0	W	0h	Pre-IRQ for Channel 0. Size counter reached Pre-IRQ threshold. 0h = Clear interrupt mask bit 1h = Set interrupt mask bit
15	DMACH15	W	0h	DMA Channel 15 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
14	DMACH14	W	0h	DMA Channel 14 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
13	DMACH13	W	0h	DMA Channel 13 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
12	DMACH12	W	0h	DMA Channel 12 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
11	DMACH11	W	0h	DMA Channel 11 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
10	DMACH10	W	0h	DMA Channel 10 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
9	DMACH9	W	0h	DMA Channel 9 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
8	DMACH8	W	0h	DMA Channel 8 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
7	DMACH7	W	0h	DMA Channel 7 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
6	DMACH6	W	0h	DMA Channel 6 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
5	DMACH5	W	0h	DMA Channel 5 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
4	DMACH4	W	0h	DMA Channel 4 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
3	DMACH3	W	0h	DMA Channel 3 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
2	DMACH2	W	0h	DMA Channel 2 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt

Table 4-25. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1	DMACH1	W	0h	DMA Channel 1 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt
0	DMACH0	W	0h	DMA Channel 0 interrupt signals that size counter reached zero (DMASZ=0). 0h = Writing 0 has no effect 1h = Clear interrupt

4.3.17 EVT_MODE (Offset = 10E0h) [Reset = 00000009h]

EVT_MODE is shown in [Figure 4-20](#) and described in [Table 4-26](#).

Return to the [Summary Table](#).

Event mode register. It is used to select whether each line is disabled, in software mode (software clears the RIS) or in hardware mode (hardware clears the RIS)

Figure 4-20. EVT_MODE

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED				EVT1_CFG		INT0_CFG	
R/W-				R-2h		R-1h	

Table 4-26. EVT_MODE Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-2	EVT1_CFG	R	2h	Event line mode select for event corresponding to generic event GEN_EVENT 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
1-0	INT0_CFG	R	1h	Event line mode select for event corresponding to interrupt event CPU_INT 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.

4.3.18 DESC (Offset = 10FCh) [Reset = 2511F000h]

DESC is shown in [Figure 4-21](#) and described in [Table 4-27](#).

Return to the [Summary Table](#).

This register identifies the peripheral and its exact version.

Figure 4-21. DESC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODULEID															
R-2511h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FEATUREVER				RESERVED				MAJREV				MINREV			
R-Fh				R-				R-0h				R-0h			

Table 4-27. DESC Field Descriptions

Bit	Field	Type	Reset	Description
31-16	MODULEID	R	2511h	Module identification contains a unique peripheral identification number. The assignments are maintained in a central database for all of the platform modules to ensure uniqueness. 0h = Smallest value FFFFh = Highest possible value
15-12	FEATUREVER	R	Fh	Feature Set for the DMA: number of DMA channel minus one (for example 0->1ch, 2->3ch, 15->16ch). 0h = Smallest value (1 channel) Fh = Highest value (16 channel)
11-8	RESERVED	R	0h	
7-4	MAJREV	R	0h	Major rev of the IP 0h = Smallest value Fh = Highest possible value
3-0	MINREV	R	0h	Minor rev of the IP 0h = Smallest value Fh = Highest possible value

4.3.19 DMAPRIO (Offset = 1100h) [Reset = 00000000h]

DMAPRIO is shown in [Figure 4-22](#) and described in [Table 4-28](#).

Return to the [Summary Table](#).

DMA Channel Priority Control

Figure 4-22. DMAPRIO

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED						BURSTSZ	
R/W-						R/W-0h	
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED							ROUNDROBIN
R/W-							R/W-0h

Table 4-28. DMAPRIO Field Descriptions

Bit	Field	Type	Reset	Description
31-18	RESERVED	R/W	0h	
17-16	BURSTSZ	R/W	0h	Define the burst size of a block transfer, before the priority is re-evaluated 0h = There is no burst size, the whole block transfer is completed on one transfer without interruption 1h = The burst size is 8, after 8 transfers the block transfer is interrupted and the priority is reevaluated 2h = The burst size is 16, after 16 transfers the block transfer is interrupted and the priority is reevaluated 3h = The burst size is 32, after 32 transfers the block transfer is interrupted and the priority is reevaluated
15-1	RESERVED	R/W	0h	
0	ROUNDROBIN	R/W	0h	Round robin. This bit enables the round-robin DMA channel priorities. 0h = Round robin priority disabled, DMA channel priority is fixed: DMA0-DMA1-DMA2-...-DMA16 1h = Round robin priority enabled, DMA channel priority changes with each transfer

4.3.20 DMATCTL[j] (Offset = 1110h + formula) [Reset = 00000000h]

DMATCTL[j] is shown in [Figure 4-23](#) and described in [Table 4-29](#).

Return to the [Summary Table](#).

DMA Trigger Control

Offset = 1110h + (j * 4h); where j = 0h to Fh

Figure 4-23. DMATCTL[j]

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
DMATINT	RESERVED	DMATSEL					
R/W-0h	R/W-	R/W-0h					

Table 4-29. DMATCTL[j] Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R/W	0h	
7	DMATINT	R/W	0h	DMA Trigger by Internal Channel 0h = DMATSEL will define external trigger select as transfer trigger. 1h = DMATSEL will define internal channel as transfer trigger select. 0-> Channel0-done, 1-> Channel1-done, ...
6	RESERVED	R/W	0h	
5-0	DMATSEL	R/W	0h	DMA Trigger Select Note: Reference the data sheet of the device to see the specific trigger mapping. 00h = Software trigger request 3Fh = Highest possible value

4.3.21 DMACTL[j] (Offset = 1200h + formula) [Reset = 00000000h]

DMACTL[j] is shown in [Figure 4-24](#) and described in [Table 4-30](#).

Return to the [Summary Table](#).

DMA Channel Control

Offset = 1200h + (j * 10h); where j = 0h to Fh

Figure 4-24. DMACTL[j]

31	30	29	28	27	26	25	24
RESERVED		DMATM		RESERVED		DMAEM	
R/W-		R/W-0h		R/W-		R/W-0h	
23	22	21	20	19	18	17	16
DMADSTINCR				DMASRCINCR			
R/W-0h				R/W-0h			
15	14	13	12	11	10	9	8
RESERVED		DMADSTWDTH		RESERVED		DMASRCWDTH	
R/W-		R/W-0h		R/W-		R/W-0h	
7	6	5	4	3	2	1	0
RESERVED	DMPREIRQ			RESERVED	DMAEN		DMAREQ
R/W-	R/W-0h			R/W-	R/W-0h		R/W-0h

Table 4-30. DMACTL[j] Field Descriptions

Bit	Field	Type	Reset	Description
31-30	RESERVED	R/W	0h	
29-28	DMATM	R/W	0h	DMA transfer mode register Note: The repeat-single (2h) and repeat-block (3h) transfer are only available in a FULL-channel configuration. Please consult the data sheet of the specific device to map which channel number has FULL or BASIC capability. In a BASIC channel configuration only the values for single (0h) and block (1h) transfer can be set. 0h = Single transfer. Each transfers requires a new trigger. When the DMASZ counts down to zero an event can be generated and the DMAEN is cleared. 1h = Block transfer. Each trigger transfers the complete block defined in DMASZ. After the transfer is complete an event can be generated and the DMAEN is cleared. 2h = Repeated single transfer. Each transfers requires a new trigger. When the DMASZ counts down to zero an event can be generated. After the last transfer the DMASA, DMADA, DAMSZ registers are restored to its initial value and the DMAEN stays enabled. 3h = Repeated block transfer. Each trigger transfers the complete block defined in DMASZ. After the last transfer the DMASA, DMADA, DAMSZ registers are restored to its initial value and the DMAEN stays enabled.
27-26	RESERVED	R/W	0h	
25-24	DMAEM	R/W	0h	DMA extended mode Note: The extended transfer modes are only available in a FULL-channel configuration. Please consult the data sheet of the specific device to map which channel number has FULL or BASIC capability. In a BASIC channel configuration this register is a read-only register and reads 0x0. 0h = Normal mode is related to transfers from SRC to DST 2h = Fill mode will copy the SA register content as data to DA 3h = Table mode will read an address and data value from SA and write the data to address

Table 4-30. DMACTL[j] Field Descriptions (continued)

Bit	Field	Type	Reset	Description
23-20	DMADSTINCR	R/W	0h	DMA destination increment. This bit selects automatic incrementing or decrementing of the destination address DMADA for each transfer. The amount of change to the DMADA is based on the definition in the DMADSTWDTH. For example an increment of 1 (+1) on a WORD transfer will increment the DMADA by 4. 0h = Address is unchanged (+0) 2h = Decrement by 1 (-1 * DMADSTWDTH) 3h = Incremented by 1 (+1 * DMADSTWDTH) 8h = Stride size 2 (+2 * DMADSTWDTH) 9h = Stride size 3 (+3 * DMADSTWDTH) Ah = Stride size 4 (+4 * DMADSTWDTH) Bh = Stride size 5 (+5 * DMADSTWDTH) Ch = Stride size 6 (+6 * DMADSTWDTH) Dh = Stride size 7 (+7 * DMADSTWDTH) Eh = Stride size 8 (+8 * DMADSTWDTH) Fh = Stride size 9 (+9 * DMADSTWDTH)
19-16	DMASRCINCR	R/W	0h	DMA source increment. This bit selects automatic incrementing or decrementing of the source address DMASA for each transfer. The amount of change to the DMASA is based on the definition in the DMASRCWDTH. For example an increment of 1 (+1) on a WORD transfer will increment the DMASA by 4. 0h = Address is unchanged (+0) 2h = Decrement by 1 (-1 * DMASRCWDTH) 3h = Incremented by 1 (+1 * DMASRCWDTH) 8h = Stride size 2 (+2 * DMASRCWDTH) 9h = Stride size 3 (+3 * DMASRCWDTH) Ah = Stride size 4 (+4 * DMASRCWDTH) Bh = Stride size 5 (+5 * DMASRCWDTH) Ch = Stride size 6 (+6 * DMASRCWDTH) Dh = Stride size 7 (+7 * DMASRCWDTH) Eh = Stride size 8 (+8 * DMASRCWDTH) Fh = Stride size 9 (+9 * DMASRCWDTH)
15-14	RESERVED	R/W	0h	
13-12	DMADSTWDTH	R/W	0h	DMA destination width. This bit selects the destination as a byte, half word, word or long word. 0h = Destination data width is BYTE (8-bit) 1h = Destination data width is HALF-WORD (16-bit) 2h = Destination data width is WORD (32-bit) 3h = Destination data width is LONG-WORD (64-bit)
11-10	RESERVED	R/W	0h	
9-8	DMASRCWDTH	R/W	0h	DMA source width. This bit selects the source data width as a byte, half word, word or long word. 0h = Source data width is BYTE (8-bit) 1h = Source data width is HALF-WORD (16-bit) 2h = Source data width is WORD (32-bit) 3h = Source data width is LONG-WORD (64-bit)
7	RESERVED	R/W	0h	

Table 4-30. DMACTL[j] Field Descriptions (continued)

Bit	Field	Type	Reset	Description
6-4	DMAPREIRQ	R/W	0h	Enable an early IRQ event. This can help software to react quicker to and DMA done event or allows some additional configuration before the channel is complete. Note: This register is only available in a FULL-channel configuration. Please consult the data sheet of the specific device to map which channel number has FULL or BASIC capability. In a BASIC configuration this register is a read only value and always reads as 0x0. 0h = Pre-IRQ event disabled. 1h = Issue Pre-IRQ event when DMASZ=1 2h = Issue Pre-IRQ event when DMASZ=2 3h = Issue Pre-IRQ event when DMASZ=4 4h = Issue Pre-IRQ event when DMASZ=8 5h = Issue Pre-IRQ event when DMASZ=32 6h = Issue Pre-IRQ event when DMASZ=64 7h = Issue Pre-IRQ event when DMASZ reached the half size point of the original transfer size
3-2	RESERVED	R/W	0h	
1	DMAEN	R/W	0h	DMA enable 0h = DMA channel disabled 1h = DMA channel enabled
0	DMAREQ	R/W	0h	DMA request. Software-controlled DMA start. DMAREQ is reset automatically. 0h = Default read value 1h = DMA transfer request (start DMA)

4.3.22 DMASA[j] (Offset = 1204h + formula) [Reset = 00000000h]

DMASA[j] is shown in [Figure 4-25](#) and described in [Table 4-31](#).

Return to the [Summary Table](#).

DMA Channel Source Address

Offset = 1204h + (j * 10h); where j = 0h to Fh

Figure 4-25. DMASA[j]

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADDR																															
R/W-0h																															

Table 4-31. DMASA[j] Field Descriptions

Bit	Field	Type	Reset	Description
31-0	ADDR	R/W	0h	DMA Channel Source Address 0h = Smallest value FFFFFFFFh = Highest possible value

4.3.23 DMADA[j] (Offset = 1208h + formula) [Reset = 00000000h]

DMADA[j] is shown in [Figure 4-26](#) and described in [Table 4-32](#).

Return to the [Summary Table](#).

DMA Channel Destination Address

Offset = 1208h + (j * 10h); where j = 0h to Fh

Figure 4-26. DMADA[j]

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADDR																															
R/W-0h																															

Table 4-32. DMADA[j] Field Descriptions

Bit	Field	Type	Reset	Description
31-0	ADDR	R/W	0h	DMA Channel Destination Address 0h = Smallest value FFFFFFFFh = Highest possible value

4.3.24 DMASZ[j] (Offset = 120Ch + formula) [Reset = 00000000h]

DMASZ[j] is shown in [Figure 4-27](#) and described in [Table 4-33](#).

Return to the [Summary Table](#).

DMA Channel Size

Offset = 120Ch + (j * 10h); where j = 0h to Fh

Figure 4-27. DMASZ[j]

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																SIZE															
R/W-																R/W-0h															

Table 4-33. DMASZ[j] Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15-0	SIZE	R/W	0h	DMA Channel Size in number of transfers 0h = Smallest value FFFFh = Highest possible value



The Math accelerator (MATHACL) is a collection of hardware-accelerated 32-bit math functions to improve system computational throughput. Software running on the CPU invokes MATHACL by writing to the MATHACL registers.

5.1 Overview

The MATHACL offloads mathematical calculations performed by the CPU to improve efficiency and CoreMark performance. The following hardware functions are available in the MATHACL:

- Sine/cosine (SINCOS)
- Arc tangent (ATAN2)
- Square root (SQRT)
- Divide (DIV)
- Multiply with 32-bit result (MPY32)
- Square with 32-bit result (SQUARE32)
- Multiply with 64-bit result (MPY64)
- Square with 64-bit results (SQUARE64)
- Multiply-accumulate (MAC)
- Square-accumulate (SAC)

5.2 Data Format

The operands that are used in MATHACL functions are 32-bit fixed point numbers. Operands can be expressed as unsigned 32-bit integers, signed 32-bit integers, unsigned 32-bit numbers, and signed 32-bit numbers.

[Table 5-1](#) summarizes the data formats that are used in the MATHACL.

Table 5-1. MATHACL data formats

Data format	Data type	Signed format	Data range
Unsigned 32-bit integers	Uint32_t	2's compliment	0 to $2^{32} - 1$
Signed 32-bit integers	int32_t	2's compliment	-2^{31} to $2^{31} - 1$
Unsigned 32-bit numbers	UQm.n	Signed magnitude	0 to $(2^{32})/2^n$
Signed 32-bit numbers	SQm.n	Signed magnitude	$(-2^{31})/2^n$ to $(2^{31})/2^n$

5.2.1 Unsigned 32-bit integers

An unsigned 32-bit integer is a 32-bit positive number ranging from 0 to $2^{32}-1$, and it is expressed as type `uint32_t`.

For instance, the number 123456 is equal to 0b 00000000 00000001 11100010 01000000, or 0x0001E240.

5.2.2 Signed 32-bit integers

A signed 32-bit integer is a 32-bit positive or negative number ranging from -2^{31} to $2^{31} - 1$, and it is expressed as type `int32_t`. To convert a signed 32-bit integer from decimal:

- Convert the integer to an unsigned 32-bit integer

- Invert the number
- Add 1 to the least significant bit (LSB)

For instance, the number -123456 is equal to 0b 11111111 11111110 00011101 11000000, or 0xFFFE1DC0.

5.2.3 Unsigned 32-bit numbers

Unsigned 32-bit numbers include an integer component and fractional component using Q number format. It is expressed as UQm.n, where:

- **m** is the integer component number of bits
- **n** is the fractional component number of bits

The m and n components vary depending on the operand type. For instance, an operand in UQ16.16 format has the signed number components as shown in [Table 5-2](#)

Table 5-2. Unsigned number components

Description	Value	Number of Bits	Bits
Integer component	I	m = 16	31:16
Fractional component	F	n = 16	15:0

To generate an unsigned number, such as 4.75 in UQ16.16:

1. Express the integer component (I = 4) as a number in m bits (0x0004)
2. Express the fractional component (F = 0.75) as a number in n bits (0xC000)
3. Express the resulting hexadecimal value as an unsigned value (0x0100C000)

5.2.4 Signed 32-bit numbers

Signed 32-bit numbers include a signed magnitude component, integer component and fractional component using Q number format. It is expressed as SQm.n, where:

- S is the Sign Bit
- **m** is the integer component number of bits
- **n** is the fractional component number of bits

The m and n components vary depending on the operand type. For instance, an operand in SQ15.16 format has the signed number components as shown in [Table 5-3](#).

Table 5-3. Signed number components

Description	Value	Number of Bits	Bits
Sign bit	+ or -	S = 1	31
Integer component	I	m = 15	30:16
Fractional component	F	n = 16	15:0

To generate a signed number, such as -1.5 in SQ15.16:

1. Express the integer component (I = 1) as a number in m bits (0x0001)
2. Express the fractional component (F = 0.5) as a number in n bits (0x8000)
3. Express the resulting hexadecimal value as an unsigned value (0x00018000)
4. If the number is signed, take the 2's compliment (0xFFFE7FFF)

5.3 Basic Operation

Power Enable

To power on the MATHACL, write the KEY to the PWREN register and set the ENABLE bit to power on the MATHACL. To reset the MATHACL, write the KEY to the RSTCTL register, set the RESETSTKYCLR bit, and set the RESETASSERT bit.

MATHACL Operation

All MATHACL operations follow a similar code sequence to call the functions, begin the compute operations, and read the results:

1. Write to the FUNC, QVAL, OPTYPE, SFACTOR, and NUMITER fields in the CTL register using a single write operation. See [Section 5.4](#) for which fields are required by the function being called.
2. Trigger the function to start computing based on the number of operands required in the operation.
 - a. In case of two operand functions, write to OP1 first, then write to OP2 to trigger the function to start computing.
 - b. In case of single operand functions, write to OP2 to trigger the function to start computing.
3. To read the computed results, read the RES1 and RES2 registers to check when the new data has been returned. Optionally, the STATUS.BUSY flag can be polled until it is 0 to indicate the computation is complete.

Note

The MATHACL control registers should not be changed while the current operation is ongoing, or else the operation will result in undefined behavior.

MATHACL Timing

Timing for each MATHACL calculation depends on the number of fractional bits (n), number of iterations (NUMITER) in SINCOS, ATAN2, and SQRT functions, or software overhead. See each function example in [Section 5.4](#) for the number of hardware cycles (if available).

5.4 Configuration Details with Examples

5.4.1 Sine and Cosine (SINCOS)

Sine and Cosine (SINCOS) functions are computed using the CORDIC algorithm. The algorithm relies on expressing the angle as a sum of angles in per unit format with a configurable number of iterations to incrementally calculate the result. The sine and cosine of the given angle is computed simultaneously for the number of iterations specified in CTL.NUMITER field and stored in the RES1 and RES2 registers.

[Table 5-4](#) shows the configuration registers for SINCOS.

Table 5-4. Sine/Cosine Configuration registers

Register (Bit Field)	Value	Description
OP1	User value	Angle in degrees per unit (SQ0.31 format). The angle in per unit is calculated by $n = \text{angle}/180$, where n is the angle in per unit between [-1,1) and angle is between [-180,180) degrees.
CTL.FUNC	1h	SINCOS
CTL.NUMITER	User value	Number of iterations
RES1	32-bit result	Cosine of the angle (SQ0.31 format)
RES2	32-bit result	Sine of the angle (SQ0.31 format)

A higher number of iterations (NUMITER) calculates more accurate results but will take longer to compute.

Status, Errors, and Overflow

There are no status, error, or overflow bits for SINCOS.

Configuration

To perform SINCOS:

1. In the CTL register:
 - a. Set FUNC to 1h.
 - b. Set NUMITER to the number of iterations for the calculation.
2. Set the angle per unit (SQ0.31 format) in OP2 to begin the computation.
3. Read the cosine of the angle in RES1 (SQ0.31 format).
4. Read the sine of the angle in RES2 (SQ0.31 format).

5.4.2 Arc Tangent (ATAN2)

The arctangent function (ATAN2) computes the arctangent of (y/x), where y and x are the normalized coordinates. The ATAN2 algorithm relies on expressing the resulting angle as a sum of constituent angles computed by incrementally accumulating the constituent angles until the y-coordinate value is 0. The number of iterations to calculate the arctangent is specified using the CTL.NUMITER field, and the final result for the angle in per unit is stored in RES1 as a signed number.

Before performing the ATAN2 function, the x- and y- coordinates should be normalized to a unit vector, which can be done with various types of normalization algorithms. One normalization algorithm is shown in [Figure 5-1](#).

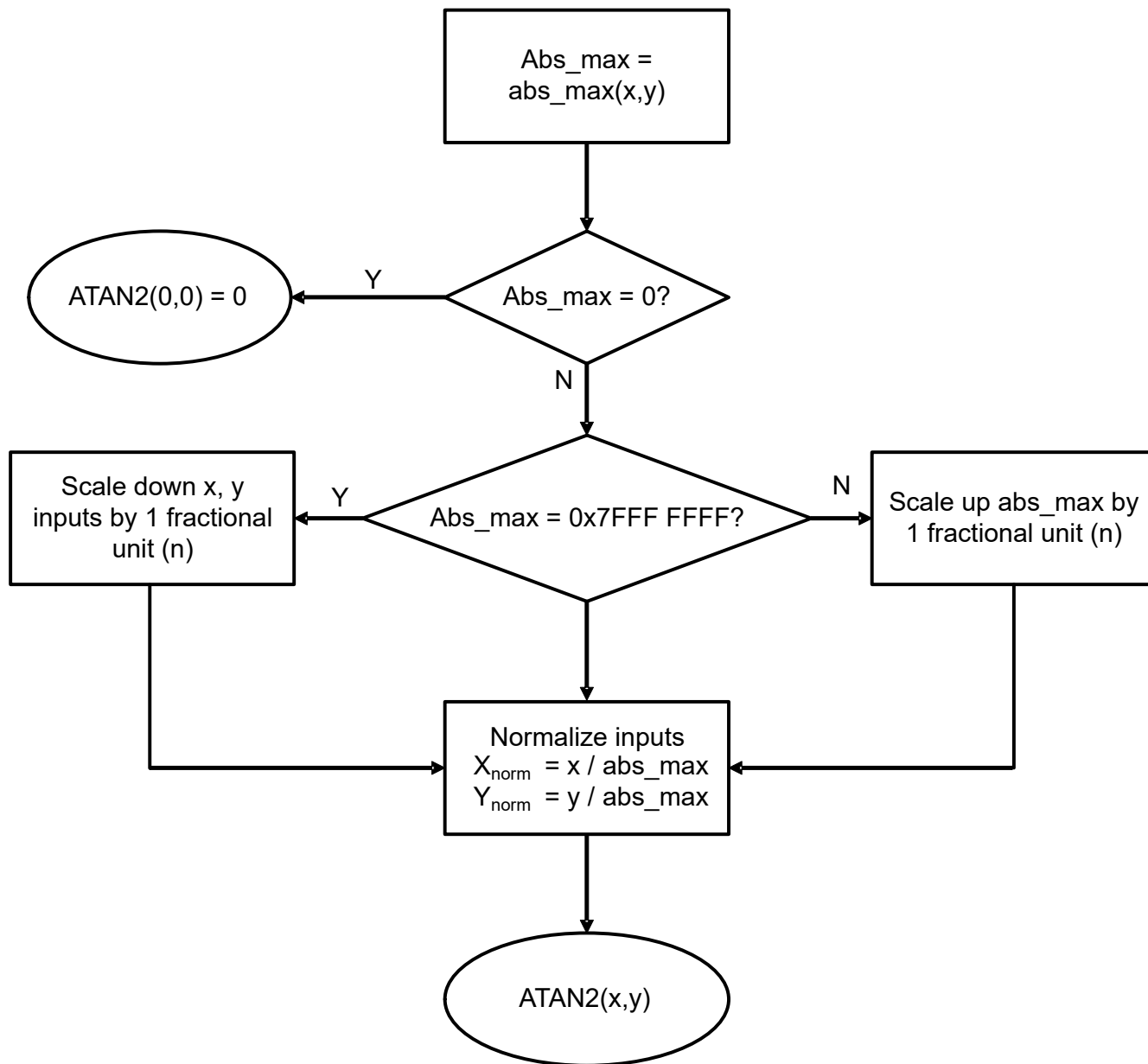


Figure 5-1. Normalization algorithm for ATAN2 inputs

Table 5-5 shows the configuration registers for the ATAN2 function.

Table 5-5. ATAN2 configuration registers

Register (Bit Field)	Value	Description
OP1	User value	X coordinate (SQ0.31 format)
OP2	User value	Y coordinate (SQ0.31 format)
CTL.FUNC	2h	ATAN2
CTL.NUMITER	User value	Number of iterations

Table 5-5. ATAN2 configuration registers (continued)

Register (Bit Field)	Value	Description
RES1	32-bit value	Angle in degrees per unit (SQ0.31 format) . The angle in per unit is calculated by $n = \text{angle}/180$, where n is the angle in per unit between [-1,1) and angle is between [-180,180) degrees.

A higher number of iterations (NUMITER) calculates more accurate results but will take longer to compute.

Status, Errors, and Overflow

There are no status, error, or overflow bits for ATAN2.

Configuration

To perform ATAN2:

- In the CTL register:
 - Set FUNC to 2h.
 - Set NUMITER to the number of iterations for the calculation.
- Set the normalized x-coordinate (SQ0.31 format) in OP1.
- Set the normalized y-coordinate (SQ0.31 format) in OP2 to begin the computation.
- Read the normalized arctangent result in RES1 (SQ0.31 format).

5.4.3 Square Root (SQRT)

The square root function (SQRT) computes the square root of a number (radicand). The square root is computed incrementally by the number of iterations in the CTL.NUMITER field. A higher number of iterations calculates more accurate results but takes longer to compute.

The radicand must be in UQm.n data type, and it is broken down into two components: an unscaled number (UN) and a scaled number (SN). The unscaled number is the floor of the number, and the scaled number is between 1.0 and 2.0 in UQ2.30 format. The relationship between the scaled number (SN) and unscaled number (UN) is the scale factor (SFACTOR), which is shown in [Equation 16](#).

$$\text{Scaled number (SN)} = \frac{\text{Unscaled number (UN)}}{2^{\text{SFACTOR}}} \quad (16)$$

To calculate the SN and SFACTOR from a radicand in UQm.n format, the algorithm below can be used.

```
//Compute scale factor and scaled number from radicand
UN = floor(radicand);           //unsigned number
SFACTOR = 0;                    //scale factor
count = 2;                      //scaled number must be between 1 and 2
do {
    SFACTOR++;                  //increase SFACTOR
    count <= 1;                 //multiply by 2
} while (count < UN)            //repeat until UN is less than count
SN = radicand / (count >> 1); //scale number = radicand / 2^SN
```

For example, if the 10.375 is the radicand, the unscaled number is 10, SFACTOR = 3 ($10.375 / 2^3 = 1.297$), and 1.297 is the scaled number between 1.0 and 2.0.

[Table 5-6](#) shows the configuration registers for the SQRT function.

Table 5-6. SQRT configuration registers

Register (Bit Field)	Value	Description
OP1	User value	Scaled number (SN) in UQ2.30 format. The scaled number is scaled from the unscaled number, and the scaled number has to be between 1.0 and 2.0.
CTL.FUNC	5h	SQRT
CTL.SFACTOR	User value	Scaling factor n. The scaled number (SN) = (Unscaled number)/2 ⁿ . This is a signed integer and takes a positive value or negative value in two's complement.
CTL.NUMITER	User value	Number of iterations
RES1	32-bit result	Square root of unscaled number in UQ16.16 format.

A higher number of iterations (NUMITER) calculates more accurate results but takes longer to compute.

Status, Errors, and Overflow

There are no status, error, or overflow bits for SQRT.

Configuration

To perform a SQRT:

- In the CTL register:
 - Set FUNC to 5h.
 - Set NUMITER to the number of iterations for the calculation.
 - Set SFACTOR to the scaling factor n.
- Set the scaled number (UQ2.30 format) in OP2 to begin the computation.
- Read the square root result in RES1 (UQ16.16 format).

5.4.4 Division (DIV)

The divide function (DIV) computes with a known dividend and divisor.

$$\text{Dividend} = \text{Divisor} * \text{Quotient} + \text{Remainder} \quad (17)$$

$$\text{mod}(\text{Remainder}) < \text{mod}(\text{Divisor}) \quad (18)$$

Table 5-7 shows the data types that the DIV function can perform.

Table 5-7. DIV Data Types Supported

Data Type (Dividend and Divisor)	Data Type (Quotient)	Data Type (Remainder)
Uint32_t	Uint32_t	Uint32_t
Int32_t	Int32_t	Int32_t
SQm.n	SQm.n	N/A
UQm.n	UQm.n	N/A

Table 5-8 shows the configuration registers for the DIV function.

Table 5-8. DIV Configuration Registers

Register (Bit Field)	Value	Description
OP1	User value	Dividend
OP2	User value	Divisor
CTL.FUNC	4h	DIV
CTL.QVAL	User value	Number of fractional bits (unsigned or signed number data type only)

Table 5-8. DIV Configuration Registers (continued)

Register (Bit Field)	Value	Description
CTL.OPTYPE	0 = unsigned operands 1 = signed operands	Operand sign
RES1	32-bit result	Quotient (same data type as dividend and divisor)
RES2	32-bit result	Remainder (same data type as dividend and divisor)

Note

OP1 and OP2 must be the same data type or use incorrect results will occur.

Signed integer (int32_t) calculations requires 4 cycles to compute results in RES1 and RES2 from an OP2 write. Examples of DIV for int32_t are in [Table 5-9](#).

Table 5-9. Division Examples for int32_t Data Type

Dividend	Divisor	Quotient	Remainder
15 (0x0000000F)	2 (0x00000002)	7 (0x00000007)	1 (0x00000001)
-15 (0xFFFFFFFF1)	2 (0x00000002)	-7 (0xFFFFFFFF9)	-1 (0xFFFFFFFF)
15 (0x0000000F)	-2 (0xFFFFFFFFE)	-7 (0xFFFFFFFF9)	1 (0x00000001)
-15 (0xFFFFFFFF1)	-2 (0xFFFFFFFFE)	7 (0x00000007)	-1 (0xFFFFFFFF)

Signed number (SQm.n) calculations requires n cycles to compute results in RES1 and RES2 from an OP2 write, where n is the number of fractional bits. Examples of DIV for SQ15.16 are in [Table 5-10](#).

Table 5-10. Division Examples for SQ15.16 Data Type

Dividend	Divisor	Quotient	Remainder
7.5 (0x00078000)	4.5 (0x00048000)	1.666657 (0x0001AAAA)	N/A
-7.5 (0xFFFF88000)	4.5 (0x00048000)	-1.666657 (0xFFFE5556)	N/A
7.5 (0x00078000)	-4.5 (0xFFFFB8000)	-1.666657 (0xFFFE5556)	N/A
-7.5 (0xFFFF88000)	-4.5 (0xFFFFB8000)	1.666657 (0x0001AAAA)	N/A

Status, Errors, and Overflow

If the divisor is 0, then the STATUS.ERR flag is set.

If the result of the operation is more than 32 bits, STATUS.OVF is set to indicate that an overflow has occurred. It will remain set until cleared by writing a 1 to CLR.CLR_OVF.

For signed operation, if result saturation is enabled (CTL.SATEN = 1) in the case of an overflow, the result will be saturated to the maximum positive result (0x7FFFFFFF) and the minimum negative result (0x80000000).

For unsigned operation, if result saturation is enabled (CTL.SATEN = 1) in the case of an overflow, the result will be saturated to the maximum positive result (0xFFFFFFFF).

Configuration

To perform a DIV:

- In the CTL register:
 - Set FUNC to 4h.
 - Set OPTYPE to 0 if the data type is unsigned, or 1 if the data type is signed.
 - Set QVAL to the number of fractional bits (n) for a signed (SQm.n) or unsigned number (UQm.n)
- Set the dividend in OP1.
- Set the divisor in OP2 to begin the computation.
- Read the quotient result in RES1 and remainder result in RES2.

5.4.5 Multiplication

Multiplication functions gives the result of multiplying two numbers. There are variations of the multiply function, including multiplying and squaring with 32-bit and 64-bit results.

5.4.5.1 Multiply32 (MPY32)

The Multiply32 (MPY32) function returns the results of multiplying two 32-bit numbers with the 32-bit result in RES1.

The MPY32 function can perform the data types shown in [Table 5-11](#).

Table 5-11. MPY32 Data Types Supported

Data type (Multiplicand and Multiplier)	Data type (Product)	Condition
Uint32_t	Uint32_t	Product is $< 2^{32} - 1$
Int32_t	Int32_t	Product is $> -2^{31}$ and $< 2^{31} - 1$
SQm.n	SQm.n	Product is not outside the range representable in the chosen data format
UQm.n	UQm.n	

[Table 5-12](#) shows the operands and descriptions for the MPY32 function.

Table 5-12. MPY32 Configuration Registers

Register (Bit Field)	Value	Description
OP1	User value	Multiplicand
OP2	User value	Multiplier
CTL.FUNC	6h	MPY32
CTL.QVAL	User value	Number of fractional bits (unsigned or signed number data type only)
CTL.OPTYPE	0h = unsigned operands 1h = signed operands	Operand sign
RES1	32-bit product	Product

Note

OP1 and OP2 must be the same data type or use incorrect results will occur.

Signed integer (int32_t) calculations requires 1 cycles to compute results in RES1 from an OP2 write. Examples of MPY32 for int32_t are in [Table 5-13](#).

Table 5-13. MPY32 Examples Using Signed Integers (int32_t)

Multiplicand	Multiplier	Product
15 (0x0000000F)	2 (0x00000002)	30 (0x0000001E)
-15 (0xFFFFFFF1)	2 (0x00000002)	-30 (0xFFFFFFE2)
15 (0x0000000F)	-2 (0xFFFFFFF2)	-30 (0xFFFFFFE2)
-15 (0xFFFFFFF1)	-2 (0xFFFFFFF2)	30 (0x0000001E)

Signed number (SQm.n) calculations requires n cycles to compute results in RES1 from an OP2 write, where n is the number of fractional bits. Examples of MPY32 for SQ15.16 are in [Table 5-14](#).

Table 5-14. MPY32 Examples Using SQ15.16 Signed Numbers

Multiplicand	Multiplier	Product
1.5 (0x00018000)	2.5 (0x00028000)	3.75 (0x0003C000)
-1.5 (0xFFFE8000)	2.5 (0x00028000)	-3.75 (0xFFFC0000)
1.5 (0x00018000)	-2.5 (0xFFFD8000)	-3.75 (0xFFFC0000)

Table 5-14. MPY32 Examples Using SQ15.16 Signed Numbers (continued)

Multiplicand	Multiplier	Product
-1.5 (0xFFFE8000)	-2.5 (0xFFFD8000)	3.75 (0x0003C000)

Status, Errors, and Overflow

If the result of the operation is more than 32 bits, STAT.OVF is set to indicate that an overflow has occurred. It will remain set until cleared by writing setting CLR_OVF = 1 in the CLR register.

For signed operation, if result saturation is enabled (CTL.SATEN = 1), in the case of an overflow, the STAT.OVF bit is set and the result saturates to the maximum positive result (0x7FFFFFFF).

For unsigned operation, if result saturation is enabled (CTL.SATEN = 1) in the case of an overflow, the result will be saturated to the maximum positive result (0xFFFFFFFF).

Configuration

To perform an MPY32:

- In the CTL register:
 - Set FUNC to 6h.
 - Set OPTYPE to 0 if the data type is unsigned, or 1 if the data type is signed.
 - Set QVAL to the number of fractional bits (n) for a signed (SQm.n) or unsigned number (UQm.n)
- Set the multiplier in OP1.
- Set the multiplicand in OP2 to begin the computation.
- Read the product result in RES1.

5.4.5.2 Square32 (SQUARE32)

The Square32 (SQUARE32) function is a variation of the MPY32 function and performs a square operation on operand OP2 with the 32-bit result in RES1.

The SQUARE32 function can perform the data types shown in [Table 5-15](#).

Table 5-15. SQUARE32 Data Types Supported

Data type (Base)	Data type (Result)	Condition
Uint32_t	Uint32_t	Result is $< 2^{32} - 1$
Int32_t	Int32_t	Result is $> -2^{31}$ and $< 2^{31} - 1$
SQm.n	SQm.n	Result is not outside the range representable in the chosen data format
UQm.n	UQm.n	

[Table 5-12](#) shows the operands and descriptions for the SQUARE32 function.

Table 5-16. SQUARE32 Configuration Registers

Register (Bit Field)	Value	Description
OP2	User value	Base
CTL.FUNC	7h	MPY32
CTL.QVAL	User value	Number of fractional bits (unsigned or signed number data type only)
CTL.OPTYPE	0h = unsigned operands 1h = signed operands	Operand sign
RES1	32-bit square result	Square result

Signed integer (int32_t) calculations requires 1 cycle to compute results in RES1 from an OP2 write. Examples of SQUARE32 for int32_t data type are in [Table 5-17](#).

Table 5-17. SQUARE32 Examples Using int32_t

Multiplicand/Multiplier	Square
4 (0x00000004)	16 (0x00000010)
-4 (0xFFFFFFFFB)	16 (0x00000010)

Signed number (SQm.n) calculations requires n cycles to compute results in RES1 from an OP2 write, where n is the number of fractional bits. Examples of division for SQ15.16 are in [Table 5-18](#).

Table 5-18. SQUARE32 Examples Using SQ15.16 Signed Numbers

Multiplicand/Multiplier	Square
1.5 (0x00018000)	2.25 (0x00024000)
-1.5 (0xFFFE8000)	2.25 (0x00024000)

Status, Errors, and Overflow

If the result of the operation is more than 32 bits, STAT.OVF is set to indicate that an overflow has occurred. It will remain set until cleared by writing setting CLR_OVF = 1 in the CLR register.

For signed operation, if result saturation is enabled (CTL.SATEN = 1), in the case of an overflow, the STAT.OVF bit is set and the result saturates to the maximum positive result (0x7FFFFFFF).

For unsigned operation, if result saturation is enabled (CTL.SATEN = 1) in the case of an overflow, the result will be saturated to the maximum positive result (0xFFFFFFFF).

Configuration

To perform a SQUARE32:

- In the CTL register:
 - Set FUNC to 6h.
 - Set OPTYPE to 0 if the data type is unsigned, or 1 if the data type is signed.
 - Set QVAL to the number of fractional bits (n) for a signed (SQm.n) or unsigned number (UQm.n)
- Set the base in OP2 to begin the computation.
- Read the product result in RES1.

5.4.5.3 Multiply64 (MPY64)

The Multiply64 function (MPY64) returns the results of multiplying two 32-bit numbers with the 64-bit result in RES1 and RES2.

The MPY64 function can perform the types of multiply shown in [Table 5-11](#).

Table 5-19. MPY64 Data Types Supported

Data type (Multiplicand and Multiplier)	Data type (Product)	Condition
Uint32_t	Uint64_t	Product is $< 2^{64} - 1$
Int32_t	Int64_t	Product is $> -2^{63}$ and $< 2^{63} - 1$

[Table 5-12](#) shows the operands and descriptions for the multiply function.

Table 5-20. MPY64 Configuration Registers

Register (Bit Field)	Value	Description
OP1	User value	Multiplicand
OP2	User value	Multiplier
CTL.FUNC	8h	MPY64
CTL.OPTYPE	0h = unsigned operands 1h = signed operands	Operand sign

Table 5-20. MPY64 Configuration Registers (continued)

Register (Bit Field)	Value	Description
RES1	Lower 32 bits of product	Product (64-bit)
RES2	Upper 32 bits of product	

Note

OP1 and OP2 must be the same data type or use incorrect results will occur.

Status, Errors, and Overflow

There are no status, error, or overflow bits for MPY64.

Configuration

To perform an MPY64:

- In the CTL register:
 - Set FUNC to 8h.
 - Set OPTYPE to 0 if the data type is unsigned, or 1 if the data type is signed.
- Set the multiplier in OP1.
- Set the multiplicand in OP2 to begin the computation.
- Read the product result in RES1 and RES2.

5.4.5.4 Square64 (SQUARE64)

The Square64 function (SQUARE64) is a variation of the MPY32 function and performs a square operation on operand OP2 with the 64-bit result in RES1 and RES2.

The SQUARE64 function can perform the types of multiply shown in [Table 5-11](#).

Table 5-21. SQUARE64 data types supported

Data type (Base)	Data type (Square Result)	Condition
Uint32_t	Uint64_t	Result is $< 2^{64} - 1$
Int32_t	Int64_t	Result is $> -2^{63}$ and $< 2^{63} - 1$

[Table 5-12](#) shows the operands and descriptions for the SQUARE64 function.

Table 5-22. SQUARE64 configuration registers

Register (Bit Field)	Value	Description
OP1	User value	Base
CTL.FUNC	9h	MPY64
CTL.OPTYPE	0h = unsigned operands 1h = signed operands	Operand sign
RES1	Lower 32 bits of product	Product (64-bit)
RES2	Upper 32 bits of product	

Status, Errors, and Overflow

There are no status, error, or overflow bits for SQUARE64.

Configuration

To perform an SQUARE64:

- In the CTL register:
 - Set FUNC to 8h.
 - Set OPTYPE to 0 if the data type is unsigned, or 1 if the data type is signed.

2. Set the base in OP2 to begin the computation.
3. Read the product result in RES1 and RES2.

5.4.6 Multiply-Accumulate (MAC)

Multiply-accumulate (MAC) functions accumulates the results of the product of OP1 and OP2 inside result registers RES1 and RES2 over multiple iterations until the MATHACL is reprogrammed for another function. The MATHACL supports only a 32-bit multiplier and multiplicand operand size, and a 64-bit accumulated result size.

Figure 5-2 shows the operation of the MATHACL in MAC mode.

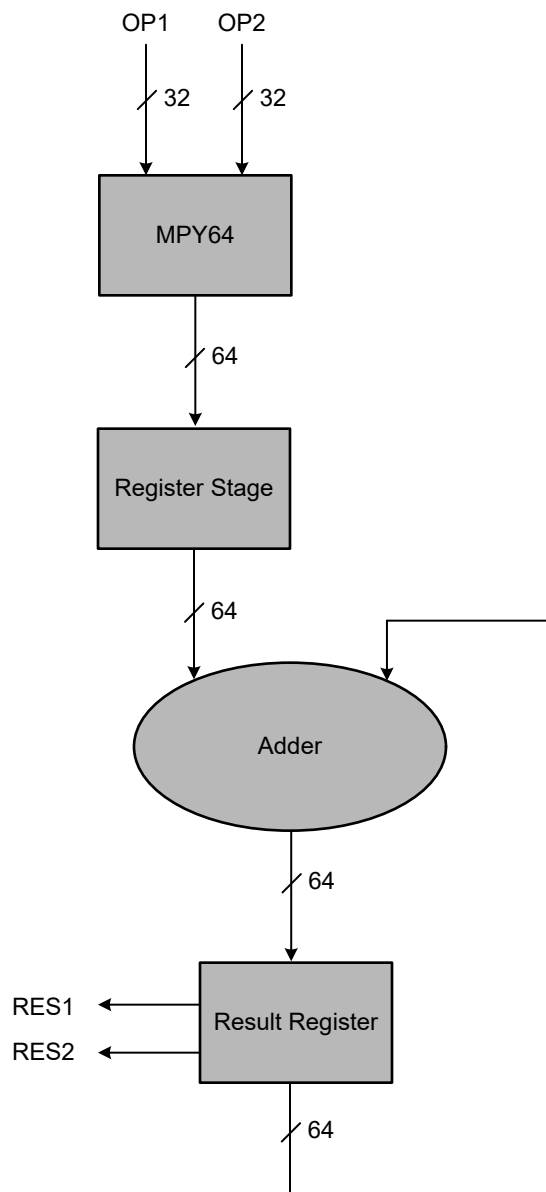


Figure 5-2. MAC Operation

Internal MAC operation performs the following steps:

- OP1 and OP2 are multiplied together, and the result is a 64-bit product
 - If using SQm.n or UQm.n data type, the product has the same number of fractional bits (n) as the operands
- The product result is stored into a Register Stage for correct timing

- The registered value is fed into a 64-bit Adder with the current Result Register value as the other input (Result Register += Register Stage)
- The result of the add operation is stored into RES1 and RES2

Note

Write 0 into RES1 and RES2 before starting a MAC operation to get the correct result.

The MAC function can perform the following data types of multiply-accumulate shown in [Table 5-23](#).

Table 5-23. MAC Data Types Supported

Data Type (Multiplicand and Multiplier)	Data Type (Product)	Condition
UInt32_t	UInt64_t	MAC result is $< 2^{64} - 1$
Int32_t	Int64_t	MAC result is $> -2^{63}$ and $< 2^{63} - 1$
SQm.n	SQ(32+m).n	MAC result is not outside the range representable in the chosen data format
UQm.n	UQ(32+m).n	

[Table 5-24](#) shows the configuration registers for the MAC function.

Table 5-24. MAC Configuration Registers

Register (Bit Field)	Value	Description
OP1	User value	Multiplicand
OP2	User value	Multiplier
CTL.FUNC	Ah	MAC
CTL.QVAL	User value	Number of fractional bits (unsigned or signed number data type only)
CTL.OPTYPE	0 = unsigned operands 1 = signed operands	Operand sign
RES1	Lower 32 bits of result	Multiply-accumulate result (64-bit)
RES2	Upper 32 bits of result	

Note

OP1 and OP2 must be the same data type or use incorrect results will occur.

The final result of the MAC operation can be read after 2 cycles after writing OP2.

Status, Errors, and Overflow

If the result of the operation is more than 64 bits, STAT.OVF is set to indicate that an overflow has occurred. It will remain set until cleared by writing a 1 to CLR.CLR_OVF.

For signed operation, if result saturation is enabled (CTL.SATEN = 1) in the case of an overflow, the result will be saturated to the maximum positive result (0x7FFF_FFFF_FFFF_FFFF) and the minimum negative result (0x8000_0000_0000_0000).

For unsigned operation, if result saturation is enabled (CTL.SATEN = 1) in the case of an overflow, the result will be saturated to the maximum positive result (0xFFFF_FFFF_FFFF_FFFF).

Configuration

To configure the MAC for operation:

1. In the CTL register:
 - a. Set FUNC to Ah.
 - b. Set OPTYPE to 0 if the data type is unsigned, or 1 if the data type is signed.
 - c. Set QVAL to the number of fractional bits (n) for a signed (SQm.n) or unsigned number (UQm.n)
2. Clear the result registers by setting RES1 and RES2 to 0.

3. Set the multiplier in OP1.
4. Set the multiplicand in OP2 to begin the computation.
5. Read the 64-bit product result in RES1 and RES2.
6. Repeat steps 3-5 to continue multiplying and accumulating data in RES1 and RES2.

5.4.7 Square Accumulate (SAC)

Square-accumulate (SAC) functions accumulates the results of the square of operand OP2 inside result registers RES1 and RES2 over multiple iterations until the MATHACL is reprogrammed for another function. It is a variation of the MAC function and operation details remain the same.

Note

Write 0 into RES1 and RES2 before starting a SAC operation to get the correct result.

The SAC function can perform the following data types of square-accumulate shown in [Table 5-25](#).

Table 5-25. SAC Data Types Supported

Data Type (Base)	Data Type (Result)	Condition
Uint32_t	Uint64_t	SAC result should be $< 2^{64} - 1$
Int32_t	Int64_t	SAC result should be $> -2^{63}$ and $< 2^{63} - 1$
SQm.n	SQ(32+m).n	SAC result should not be outside the range representable in the chosen data format
UQm.n	UQ(32+m).n	

[Table 5-26](#) shows the configuration registers for the SAC function.

Table 5-26. SAC configuration registers

Register (Bit Field)	Value	Description
OP1	User value	Base
CTL.FUNC	Bh	SAC
CTL.QVAL	User value	Number of fractional bits (unsigned or signed number data type only)
CTL.OPTYPE	0 = unsigned operands 1 = signed operands	Operand sign
RES1	Lower 32 bits of result	Square-accumulate result (64-bit)
RES2	Upper 32 bits of result	

The final result of the SAC operation can be read after 2 cycles after writing OP2.

Status, Errors, and Overflow

If the result of the operation is more than 64 bits, STAT.OVF is set to indicate that an overflow has occurred. It will remain set until cleared by writing a 1 to CLR.CLR_OVF.

For signed operation, if result saturation is enabled (CTL.SATEN = 1) in the case of an overflow, the result will be saturated to the maximum positive result (0x7FFF_FFFF_FFFF_FFFF) and the minimum negative result (0x8000_0000_0000_0000).

For unsigned operation, if result saturation is enabled (CTL.SATEN = 1) in the case of an overflow, the result will be saturated to the maximum positive result (0xFFFF_FFFF_FFFF_FFFF).

Configuration

To configure the SAC for operation:

1. In the CTL register:

- a. Set FUNC to Bh.
 - b. Set OPTYPE to 0 if the data type is unsigned, or 1 if the data type is signed.
 - c. Set QVAL to the number of fractional bits (n) for a signed (SQm.n) or unsigned number (UQm.n)
2. Clear the result registers by setting RES1 and RES2 to 0.
3. Set the base in OP1 to begin the computation.
4. Read the 64-bit square result in RES1 and RES2.
5. Repeat steps 3-4 to continue squaring and accumulating data in RES1 and RES2.

5.5 MATHACL Registers

Table 5-27 lists the memory-mapped registers for the MATHACL registers. All register offset addresses not listed in Table 5-27 should be considered as reserved locations and the register contents should not be modified.

Table 5-27. MATHACL Registers

Offset	Acronym	Register Name	Group	Section
800h	PWREN	Power Enable		Go
804h	RSTCTL	Reset Control		Go
814h	STAT	Status Register		Go
1100h	CTL	Control Register		Go
1118h	OP2	Operand 2 register.		Go
111Ch	OP1	Operand 1 Register		Go
1120h	RES1	Result 1 Register		Go
1124h	RES2	Result 2 Register		Go
1130h	STATUS	Status Register		Go
1140h	STATUSCLR	Status Flag Clear Register		Go

Complex bit access types are encoded to fit into small table cells. Table 5-28 shows the codes that are used for access types in this section.

Table 5-28. MATHACL Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
RH	R H	Read Set or cleared by hardware
Write Type		
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value

5.5.1 PWREN (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 5-3](#) and described in [Table 5-29](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 5-3. PWREN

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-0h							R/WK-0h

Table 5-29. PWREN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	ENABLE	R/WK	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

5.5.2 RSTCTL (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 5-4](#) and described in [Table 5-30](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 5-4. RSTCTL

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCL R	RESETASSERT
W-0h						WK-0h	WK-0h

Table 5-30. RSTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	W	0h	
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

5.5.3 STAT (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 5-5](#) and described in [Table 5-31](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 5-5. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED							
R-							

Table 5-31. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0h	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

5.5.4 CTL (Offset = 1100h) [Reset = 00000000h]

CTL is shown in [Figure 5-6](#) and described in [Table 5-32](#).

Return to the [Summary Table](#).

Control Register

Figure 5-6. CTL

31	30	29	28	27	26	25	24
RESERVED				NUMITER			
R/W-0h				R/W-0h			
23	22	21	20	19	18	17	16
RESERVED	SATEN	SFACTOR					
R/W-0h	R/W-0h	R/W-0h					
15	14	13	12	11	10	9	8
RESERVED				QVAL			
R/W-0h				R/W-0h			
7	6	5	4	3	2	1	0
RESERVED		OPTYPE	FUNC				
R/W-0h		R/W-0h	R/W-0h				

Table 5-32. CTL Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R/W	0h	
28-24	NUMITER	R/W	0h	Number of iterations, applicable if the function does the computations iteratively, for example sine/cosine/atan2/sqrt. Note: A value of 0 is interpreted as 31.
23	RESERVED	R/W	0h	
22	SATEN	R/W	0h	Saturation enable This bit is shared among DIV, SQUARE32, MPY32, MAC and SAC functions. When enabled, it will make the result to saturate to maximum value in case of an overflow event When disabled, the result will overflow to an unknown value. 0h = Saturation is disabled 1h = Saturation is enabled
21-16	SFACTOR	R/W	0h	Scaling factor. In case of SQRT function, the input operand needs to be in a range. If not it has to be scaled to $2^{+/-n}$. This field should be written with the value 'n'.
15-13	RESERVED	R/W	0h	

Table 5-32. CTL Field Descriptions (continued)

Bit	Field	Type	Reset	Description
12-8	QVAL	R/W	0h	Indicates the fractional bits in the operands, ranges from 0 to 31. Applicable to DIV function. 0h = Q0 operands 1h = Q1 operands 2h = Q2 operands 3h = Q3 operands 4h = Q4 operands 5h = Q5 operands 6h = Q6 operands 7h = Q7 operands 8h = Q8 operands 9h = Q9 operands Ah = Q10 operands Bh = Q11 operands Ch = Q12 operands Dh = Q13 operands Eh = Q14 operands Fh = Q15 operands 10h = Q16 operands 11h = Q17 operands 12h = Q18 operands 13h = Q19 operands 14h = Q20 operands 15h = Q21 operands 16h = Q22 operands 17h = Q23 operands 18h = Q24 operands 19h = Q25 operands 1Ah = Q26 operands 1Bh = Q27 operands 1Ch = Q28 operands 1Dh = Q29 operands 1Eh = Q30 operands 1Fh = Q31 operands
7-6	RESERVED	R/W	0h	
5	OPTYPE	R/W	0h	Operand type, could signed or unsigned. applicable to DIV function. 0h = Unsigned operands 1h = Signed operands.
4-0	FUNC	R/W	0h	ULP_ADCHP Enable Conversions. 0h = No operation 1h = Sine and Cosine operation 2h = Arc tangent with x and y values as operands. 4h = Divide, the operands are numerator, denominator, and the divide type. Result is the quotient and reminder. 5h = Do square root. Operand is the number whose square root needs to be computed. The number if outside the range needs to be scaled up down by 2 power 2n to bring it with in the range. 6h = 32-bit Multiply Result 7h = 32-bit square result 8h = 64-bit multiply result 9h = 64-bit multiply result Ah = Multiply and accumulate operation Bh = Square and accumulate operation

5.5.5 OP2 (Offset = 1118h) [Reset = 00000000h]

OP2 is shown in [Figure 5-7](#) and described in [Table 5-33](#).

Return to the [Summary Table](#).

Operand 2 Register

Figure 5-7. OP2

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA																															
R/W-0h																															

Table 5-33. OP2 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	DATA	R/W	0h	Operand 2 Register

5.5.6 OP1 (Offset = 111Ch) [Reset = 00000000h]

OP1 is shown in [Figure 5-8](#) and described in [Table 5-34](#).

Return to the [Summary Table](#).

Operand 1 register

Figure 5-8. OP1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA																															
R/W-0h																															

Table 5-34. OP1 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	DATA	R/W	0h	Operand 1 Register

5.5.7 RES1 (Offset = 1120h) [Reset = 00000000h]

RES1 is shown in [Figure 5-9](#) and described in [Table 5-35](#).

Return to the [Summary Table](#).

Result 1 register.

Figure 5-9. RES1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA																															
RH/W-0h																															

Table 5-35. RES1 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	DATA	RH/W	0h	Result 1 Register

5.5.8 RES2 (Offset = 1124h) [Reset = 00000000h]

RES2 is shown in [Figure 5-10](#) and described in [Table 5-36](#).

Return to the [Summary Table](#).

Result 2 Register

Figure 5-10. RES2

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA																															
RH/W-0h																															

Table 5-36. RES2 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	DATA	RH/W	0h	Result 2 Register

5.5.9 STATUS (Offset = 1130h) [Reset = 00000000h]

STATUS is shown in [Figure 5-11](#) and described in [Table 5-37](#).

Return to the [Summary Table](#).

Status Register

Figure 5-11. STATUS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							BUSY
R-0h							RH-0h
7	6	5	4	3	2	1	0
RESERVED				ERR		OVF	UF
R-0h				RH-0h		R-0h	R-0h

Table 5-37. STATUS Field Descriptions

Bit	Field	Type	Reset	Description
31-9	RESERVED	R	0h	
8	BUSY	RH	0h	MATHACL busy bit. 0h = Compute has completed. 1h = Compute ongoing
7-4	RESERVED	R	0h	
3-2	ERR	RH	0h	Incorrect inputs/outputs. 0h = No Error in computation. 1h = DIVBY0 error
1	OVF	R	0h	Overflow bit for MPY32, SQUARE32, DIV, MAC, and SAC functions This bit will be set on overflow and will retain its value until cleared by writing 1 into CLR.CLR_OVF 0h = Overflow error.
0	UF	R	0h	Underflow Flag 0h = No underflow error. 1h = Underflow error.

5.5.10 STATUSCLR (Offset = 1140h) [Reset = 00000000h]

STATUSCLR is shown in [Figure 5-12](#) and described in [Table 5-38](#).

Return to the [Summary Table](#).

Clear register for clearing flags in STATUS register

Figure 5-12. STATUSCLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED					CLR_ERR	CLR_OVF	CLR_UF
W-0h					W-0h	W-0h	W-0h

Table 5-38. STATUSCLR Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	W	0h	
2	CLR_ERR	W	0h	Write 1 to this bit to clear STATUS.ERR field 0h = Writing 0 has no effect 1h = Clear STATUS.ERR
1	CLR_OVF	W	0h	Write 1 to this bit to clear STATUS.OVF bit 0h = Writing 0 has no effect 1h = Clear STATUS.OVF
0	CLR_UF	W	0h	Write 1 to this bit to clear STATUS.UF bit 0h = Writing 0 has no effect 1h = Clear STATUS.UF



The non-volatile memory (NVM) system provides non-volatile flash memory for storing executable code and data.

6.1 NVM Overview.....	356
6.2 Flash Memory Bank Organization.....	357
6.3 Flash Controller.....	359
6.4 Write Protection.....	368
6.5 Read Interface.....	370
6.6 FLASHCTL Registers.....	372

6.1 NVM Overview

The nonvolatile memory system provides in-system programmable flash memory for storing executable code and data. This chapter describes the entire functionality provided by the nonvolatile memory system.

6.1.1 Key Features

Key features of the nonvolatile memory system include:

- In-circuit program and erase supported across the entire supply voltage range
- Internal programming voltage generation
- 64-bit flash word size (72-bit when optional ECC is present)
- Static write protection (latched at boot and held until BOR or POR)
- Dynamic write protection (configurable at runtime)
- Sector (1KB) and bank (up to 256KB) erase
- Automatic hardware preverification to extend flash bank longevity
- Automatic hardware post-verification of program/erase
- Optional ECC protection (SECDED)
- Optional bank address swap mode (for seamless dual-image firmware updates)
- Optional program register cache for time efficient programming (2, 4, or 8 flash words)

Note

To determine if a device has any of the optional features described above, review the nonvolatile memory system detailed description in the corresponding device data sheet.

6.1.2 System Components

The nonvolatile memory system consists of three components (listed below):

- One or more flash memory banks (for storing code and data)
- The flash controller (for managing all program/erase operations on the flash banks)
- The read interface (for interfacing the flash banks to the CPU and peripheral bus)

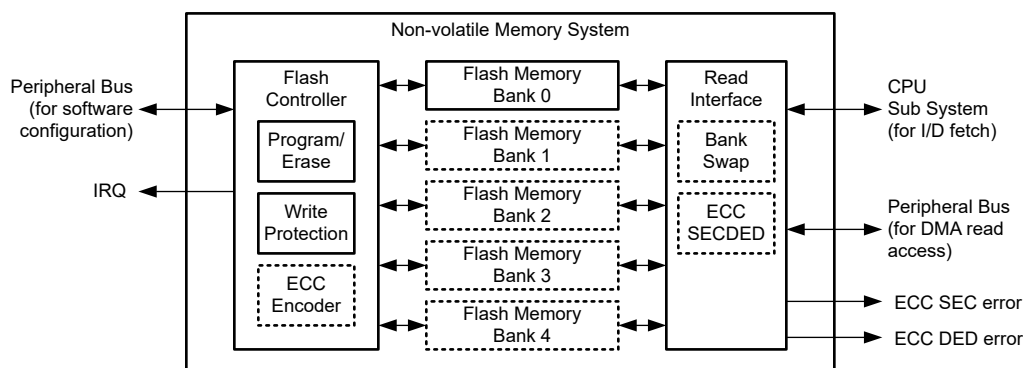


Figure 6-1. Non-volatile Memory System Components

6.1.3 Terminology

Key flash bank terms are defined in this section to be used as a reference for the rest of this chapter.

Table 6-1. NVM System Terminology

Term	Definition	Size
Flash word	Basic data size for program and read operations on the flash memory (also the read bus width to the system)	64 data bits (72 bits with ECC)
Word line	Group of flash words within a sector, with maximum program operation limit before sector erase	16 flash words (128 data bytes, optionally 16 ECC bytes)
Sector	Group of word lines that are erased together (minimum erase resolution of the flash memory)	8 word lines (1024 data bytes, optionally 128 ECC bytes)

Table 6-1. NVM System Terminology (continued)

Term	Definition	Size
Bank	Group of sectors that can be mass erased in one operation. Only one read, program, erase, or verify operation can run concurrently on a given bank.	Variable
Region	Logical assignment of a region of flash memory from a bank.	Variable

6.2 Flash Memory Bank Organization

The flash memory is used for storing application code and data, the device boot configuration, and parameters which are preprogrammed by TI from the factory. The flash memory is organized into one or more banks, and the memory in each bank is further mapped into one or more logical memory regions and assigned system address space for use by the application.

6.2.1 Banks

The nonvolatile memory system provides support for up to 5 flash memory banks (enumerated as BANK0 through BANK4). The number of flash banks present is device dependent. To determine the bank scheme of a particular device, review the detailed description section of the specific device data sheet. Most devices implement a single flash bank (BANK0).

On devices with a single flash bank, an ongoing program/erase operation will stall all read requests to the flash memory until the operation has completed and the flash controller has released control of the bank. On devices with more than one flash bank, a program/erase operation on a bank will also stall read requests issued to the bank which is executing the program/erase operation, but it will not stall read requests issued to any other bank. As such, the presence of multiple banks enables application cases such as:

- Dual-image firmware updates (an application can execute code out of one flash bank while a second image is programmed to a second symmetrical flash bank without stalling the application execution)
- EEPROM emulation (an application can execute code out of one flash bank while a second flash bank is used for writing data without stalling the application execution)

6.2.2 Flash Memory Regions

The memory within each bank is mapped to one or more logical regions based upon the functions that the memory in each bank supports. There are four regions: FACTORY, NONMAIN (Configuration NVM), MAIN (Flash Memory), and DATA.

Table 6-2. Flash Memory Regions

Flash Memory Region	Region Contents	Executable	Used by	Programmed by
FACTORY	Device ID and other parameters	No	Application	TI only (not modifiable)
NONMAIN (Configuration NVM)	Device boot configuration (BCR and BSL)	No	Boot ROM	TI, User
MAIN (Flash Memory)	Application code and data	Yes	Application	User
DATA	Data, or EEPROM emulation	No	Application	User

Devices with one bank implement the FACTORY, NONMAIN, and MAIN regions on BANK0 (the only bank present), and the data region is not available. Devices with multiple banks also implement FACTORY, NONMAIN, and MAIN regions on BANK0, but include additional banks (BANK1 through BANK4) that can implement MAIN or DATA regions.

For a detailed description of the contents of the read-only FACTORY region, see [Section 1.6](#).

6.2.3 Addressing

The flash memory regions are assigned to address space in the system memory map.

The NONMAIN, DATA, and FACTORY regions are assigned to the peripheral address space (0x4000.0000) as they do not contain any executable code. The CPU should not fetch executable instructions from this region.

The MAIN region is assigned to both the code address space (0x0000.0000) and the peripheral address space (0x4000.0000). Instruction and data fetches are recommended to always be done through the code address space as this gives the best performance. The CPU should not fetch executable instructions from this region.

ECC

On devices which have error correction code (ECC) support, the ECC codes for all memory regions are also assigned to address space and it is possible for software to read the ECC codes as data for diagnostic purposes. It is also possible to read the contents of any of the memory regions without ECC correction applied.

6.2.3.1 Flash Memory Map

The system address space assignments are given in [Table 6-3](#) and are consistent for all devices.

Table 6-3. Flash Region Memory Map

Region	Read Type	ECC Behavior	Base Address
NONMAIN	Data read	Corrected	0x41C0.0000
		Uncorrected	0x41C1.0000
		ECC code	0x41C2.0000
MAIN	Instruction fetch or data read	Corrected	0x0000.0000
		Uncorrected	0x0040.0000
	Data read	Corrected	0x4100.0000
		Uncorrected	0x4140.0000
		ECC code	0x4180.0000
DATA	Data read	Corrected	0x41D0.0000
		Uncorrected	0x41E0.0000
		ECC code	0x41F0.0000
FACTORY	Data read	Corrected	0x41C4.0000
		Uncorrected	0x41C5.0000
		ECC code	0x41C6.0000

NONMAIN, DATA, and FACTORY data reads are processed through the peripheral bus and peripheral address space only. MAIN regions can be accessed through either the CPU bus matrix or through the peripheral bus, depending on whether code address space or peripheral address space is used. The code address space is recommended for CPU accesses (instruction fetches or data reads), as these accesses do not cross the peripheral bus and thus do not compete with the DMA for control of the peripheral bus. See the [bus architecture section](#) for a detailed description of the bus interconnect.

On devices that have ECC, an access to an ECC code address returns the 8-bit ECC value for the entire 64-bit flash word that was accessed. On devices that do not have ECC, accesses to the corrected and uncorrected ECC address spaces with the same offset read the same, and ECC code addresses read as 0x0.

6.2.4 Memory Organization Examples

[Figure 6-2](#) is an example of a single bank configuration with a 64KB MAIN region. NONMAIN and FACTORY regions are also included in the single bank (BANK0) with the MAIN region. Most devices with a main region ≤128KB in size implement a single-bank configuration.

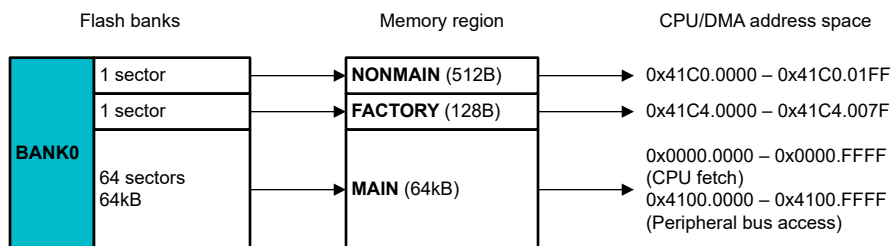


Figure 6-2. Memory Organization Example - Single Bank Configuration

Figure 6-3 is an example of a three bank configuration with a 512KB MAIN region split across BANK0 and BANK1, with a 16KB DATA region provided in BANK2. Like the single bank example, NONMAIN and FACTORY regions are included in BANK0. This example supports EEPROM emulation in the DATA region without stalling fetches to MAIN, and it also supports dual-image applications where BANK0 main can be written to without stalling fetches to BANK1 main (and the reverse). Most devices with a main region ≥ 256 KB in size implement a multibank configuration.

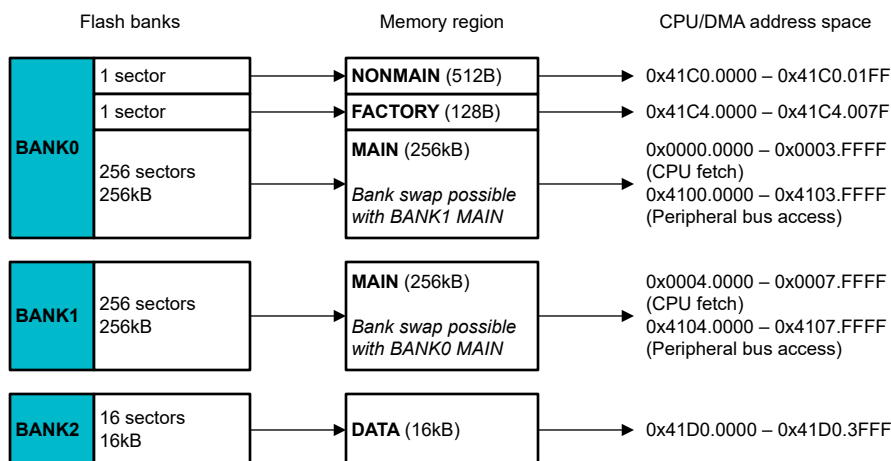


Figure 6-3. Memory Organization Example - Multiple Bank Configuration

6.3 Flash Controller

The flash controller manages all program, erase, and verification operations performed on the nonvolatile memory system. It contains memory-mapped registers in the peripheral region of the device memory map which must be configured by software to perform operations on the flash memory.

TI provides software abstraction for the flash controller as a part of the DriverLib layer of the software development kit (SDK). It is recommended to use the DriverLib abstraction layer when operating on the flash memory with software, but it is not mandatory to do so. To use the DriverLib software abstraction layer to perform operations on the flash memory, review the software development kit (SDK) documentation provided separately from this document. To directly operate on the flash memory with using low level register accesses to the flash controller, review the remainder of this section in detail.

Note

The FLASHCTL registers may not always be configured to default values after a reset. This may occur if the boot configuration routine (BCR) or boot strap loader (BSL) perform an operation on the flash memory during boot. When configuring the FLASHCTL registers for an operation, ensure that all registers which are relevant for the operation are correctly configured.

6.3.1 Overview of Flash Controller Commands

Operations on the flash memory are executed by configuring the CMDTYPE and CMDCTL registers for the desired command, along with any other registers which must be configured for a particular command, and writing 0x01 to the CMDEXEC register to initiate the command.

When 0x01 is set in CMDEXEC, the commanded operation begins executing. While an operation is executing, most configuration registers are blocked for writes until the operation completes. Some registers (for example, mask registers) can change state under hardware control while the operation runs to completion. The flash controller indicates completion of the commanded operation by setting the CMDDONE bit in the STATCMD register. The flash controller also sources an interrupt vector to the CPU subsystem to indicate a “DONE” status when an operation has completed.

The software sequence of setting the CMDEXEC bit and waiting for the CMDDONE response must be executed from either the device SRAM or from a different flash bank from the bank that is being operated on, as the flash controller will take control of the flash bank undergoing the operation. Reads to the flash bank that is being operated on while the flash controller is executing the command are not predictable.

The flash controller provides five basic commands for operating on the flash memory, specified in the COMMAND field of the CMDTYPE register. These commands are described in [Table 6-4](#).

Table 6-4. Flash Controller Commands

Command	Description
NOOP	No operation (default setting).
PROGRAM	Selects a program operation on the flash memory.
ERASE	Selects an erase operation on the flash memory.
READVERIFY	Selects a standalone read verify operation.
BLANKVERIFY	Selects a standalone blank verify operation.

6.3.2 NOOP Command

When not using the flash controller, it is best to set the COMMAND field to NOOP to prevent any unintentional operations on the flash memory in the event that the VAL bit in CMDEXEC is unintentionally set. Executing a NOOP command has no effect on the flash memory.

6.3.3 PROGRAM Command

The program command is used to write (program) the flash memory. Specifically, the purpose of a PROGRAM operation is to configure the flash bits in one or more flash words from the non-deterministic erased state to the deterministic programmed state. Once a byte is programmed using the PROGRAM command, the byte can not be re-programmed unless the sector is erased using the [ERASE](#) command.

All devices support single flash word programming of 64 data bits (plus 8 ECC bits on devices with ECC) at a time, with control to limit the scope of a program operation to specific bytes within a 64-bit flash word.

Some devices additionally have support for a multi-word programming mode where 2, 4, or 8 flash words can be written with a single commanded operation. Multi-word programming, when available, significantly speeds up programming when multiple words need to be programmed (for example, during production programming or firmware updates). See the device-specific data sheet to determine if multi-word programming is supported, and if so, how many flash word buffers are provided.

6.3.3.1 Program Bit Masking Behavior

The flash controller provides a program verification mechanism to extend the lifetime of the flash bank. During program operations, the CMDDATAx registers are used by the flash controller as a programming bit mask to indicate which specific bits in the flash word require program pulses. As a result, data which is loaded into the CMDDATAx registers before starting the program operation will be lost from the CMDDATAx registers during and/or upon completion of the program operation. If the same data is to be programmed again, the CMDDATAx registers must be re-loaded by software with the correct data values to be programmed.

6.3.3.2 Programming Less Than One Flash Word

In general, the simplest way to program the flash memory is one flash word at a time (64 bits plus 8 ECC bits if ECC is present). It is possible to program the flash memory with 32-, 16-, or 8-bit (byte) resolution, but special care must be taken when doing so to ensure the following:

1. On devices with ECC, ECC bits must be **handled properly** to prevent inadvertent ECC errors.
2. The number of program operations applied to a given word line must be monitored to ensure that the maximum word line program limit before erase is not violated.
3. Once a byte has been programmed, the sector containing the byte must be erased before attempting to re-program the same byte.

To program less than one flash word, the CMDBYTEN register must be configured to indicate which bytes in the flash word are to be programmed before starting the program operation. Each bit in CMDBYTEN corresponds to a byte in the flash word to be programmed, including the ECC bits.

Handling ECC

On devices with ECC, programming 64 bits of data at a time ensures that the 8 ECC bits which correspond to the 64-bit data word are also programmed both correctly and at the same time. Doing so prevents ECC errors from occurring if the memory locations are read by the CPU or DMA after programming.

If use of ECC is planned and partial programming is required, one approach is to mask (not program) the ECC bits until all 64 bits of a flash word are programmed, at which time the ECC bits can also be programmed. Programming of ECC bits is masked by clearing BIT8 (0x100) in the CMDBYTEN register. This prevents a situation where the entire sector must be erased each time a program operation is done to re-program ECC bits to match the new 64-bit data. However, in this case, a read to a partially programmed word where the ECC bits are not yet written would result in an ECC error. To avoid an ECC error, the software must either wait until the full 64-bit flash word and the 8 ECC bits are written, or read the data from the uncorrected address space.

Maximum Program Operations per Word Line Before Erase

The device data sheet specifies a maximum limit on the number of program operations per word line before erasure of the sector containing the word line is required. Exceeding this maximum can result in data corruption within the word line.

If 16-bit or greater program operations are performed, and no 16-bit location is programmed more than once before a sector is erased, **the maximum limit will never be reached and thus does not need to be considered.**

If 8-bit (byte) program operations are performed, the maximum program limit per word line must be considered and not exceeded. Program operations performed on ECC locations, if done independently from other program operations, count towards the number of writes before an erase is required.

6.3.3.3 Target Data Alignment (Devices with Single Flash Word Programming Only)

For devices which only support single word programming, only the CMDDATA0, CMDDATA1, and CMDECC0 registers are used to load data to be programmed to the flash memory. CMDDATA0 is always loaded with BIT31-BIT0 of the target data, and CMDDATA1 is always loaded with BIT63-BIT32 of the target data. ECC data, if specified directly and not computed automatically, is loaded into BIT7-BIT0 of CMDECC0. No other CMDDATAx or CMDECCx registers are used, and CMDDATAINDEX is not used. If fewer than 64 data bits are being programmed, see the special handling requirements section above for programming less than one flash word.

Single-word program operations must be flash word (64-bit) aligned. This means that the target system address specified in CMDADDR must be aligned to a 0b000 boundary (for example, the 3 LSBs in CMDADDR must be zero).

6.3.3.4 Target Data Alignment (Devices With Multiword Programming)

For devices that support 2-, 4-, or 8-word programming, there are two options for loading data to be programmed: direct mode or indexed mode. The programmer must select the mode that is most suitable to the application requirements.

Additional alignment rules apply when loading data into the CMDDATAx and CMDECCx registers on devices that support multiword programming, even if the multiword programming feature is not used and only single word programming is not used.

- 1-word program operations must have CMDADDR (the target system address) aligned to a 0b000 boundary (for example, the 3 LSBs in CMDADDR must be zero).
- 2-word program operations must have CMDADDR (the target system address) aligned to a 0b0000 boundary (for example, the 4 LSBs in CMDADDR must be zero).
- 4-word program operations must have CMDADDR (the target system address) aligned to a 0b0.0000 boundary (for example, the 5 LSBs in CMDADDR must be zero).
- 8-word program operations must have CMDADDR (the target system address) aligned to a 0b00.0000 boundary (for example, the 6 LSBs in CMDADDR must be zero).

Direct Data Load

To configure a program operation with direct data loading, the target data is loaded into the appropriate CMDDATAx registers based on the number of flash words supported by the device, the target address alignment, and the target data size. For example, if a 4-word program operation is to be initiated on a device supporting 4-word programming, the CMDDATA0-CMDDATA7 registers would be populated with the target data. If ECC is being specified directly (rather than automatically calculated by the flash controller) then the appropriate CMDECCx registers also needs to be populated with the ECC values for each data word being programmed.

Indexed Data Load

Rather than buffering data into all the CMDDATAx registers individually, it is possible to use only the CMDDATA0-CMDDATA1 registers in combination with an index register (CMDDATAINDEX) to indicate the flash word offset of the data being loaded. In this way, the index can be adjusted for each word loaded, and each target data word can be written to the same 64-bit space (CMDDATA0-CMDDATA1). When an index is applied, the loaded data will be mapped by the hardware into the appropriate CMDDATAx register. For example, if a 4-word program operation is to be initiated on a device supporting 4-word programming, the CMDDATA1:0 registers are loaded 4 times with the target data, with CMDDATAINDEX being incremented by one before each new word is loaded into CMDDATA1:0.

Alignment Rules

The alignment rules for each device configuration (2, 4, or 8 words) is given in the tables below with guidance on how target data must be placed with the CMDDATAx, CMDECCx, and CMDDATAINDEX registers for 1, 2, 4, or 8 word programming operations.

Table 6-5. Data Load Alignment for Devices Supporting Programming of 2 Flash Words

Direct Load Registers	Indexed Load Index	1 Word Aligned to 0b000	2 Words Aligned to 0b0000	4 Words Aligned to 0b0.0000	8 Words Aligned to 0b00.0000
CMDDATA1:0 / CMDECC0	CMDINDEX = 0	Target data word 0 if the target address ends in 0b0000	Target data word 0	Not supported	Not supported
CMDDATA3:2 / CMDECC1	CMDINDEX = 1	Target data word 0 if the target address ends in 0b1000	Target data word 1		

Table 6-6. Data Load Alignment for Devices Supporting Programming of 4 Flash Words

Direct Load Registers	Indexed Load Index	1 Word Aligned to 0b000	2 Words Aligned to 0b0000	4 Words Aligned to 0b0.0000	8 Words Aligned to 0b00.0000
CMDDATA1:0 / CMDECC0	CMDINDEX= 0	Target data word 0 if the target address ends in 0b0.0000	Target data word 0 if the target address ends in 0b0.0000	Target data word 0	Not supported
CMDDATA3:2 / CMDECC1	CMDINDEX= 1	Target data word 0 if the target address ends in 0b0.1000	Target data word 1 if the target address ends in 0b0.0000	Target data word 1	
CMDDATA5:4 / CMDECC2	CMDINDEX= 2	Target data word 0 if the target address ends in 0b1.0000	Target data word 0 if the target address ends in 0b1.0000	Target data word 2	
CMDDATA7:6 / CMDECC3	CMDINDEX= 3	Target data word 0 if the target address ends in 0b1.1000	Target data word 1 if the target address ends in 0b1.0000	Target data word 3	

Table 6-7. Data Load Alignment for Devices Supporting Programming of 8 Flash Words

Direct Load Registers	Indexed Load Index	1 Word Aligned to 0b000	2 Words Aligned to 0b0000	4 Words Aligned to 0b0.0000	8 Words Aligned to 0b00.0000
CMDDATA1:0 / CMDECC0	CMDINDEX = 0	Target data word 0 if the target address ends in 0b00.0000	Target data word 0 if the target address ends in 0b00.0000	Target data word 0 if the target address ends in 0b00.0000	Target data word 0
CMDDATA3:2 / CMDECC1	CMDINDEX = 1	Target data word 0 if the target address ends in 0b00.1000	Target data word 1 if the target address ends in 0b00.0000	Target data word 1 if the target address ends in 0b00.0000	Target data word 1
CMDDATA5:4 / CMDECC2	CMDINDEX = 2	Target data word 0 if the target address ends in 0b01.0000	Target data word 0 if the target address ends in 0b01.0000	Target data word 2 if the target address ends in 0b00.0000	Target data word 2
CMDDATA7:6 / CMDECC3	CMDINDEX = 3	Target data word 0 if the target address ends in 0b01.1000	Target data word 1 if the target address ends in 0b01.0000	Target data word 3 if the target address ends in 0b00.0000	Target data word 3
CMDDATA9:8 / CMDECC4	CMDINDEX = 4	Target data word 0 if the target address ends in 0b10.0000	Target data word 0 if the target address ends in 0b10.0000	Target data word 0 if the target address ends in 0b10.0000	Target data word 4
CMDDATA11:10 / CMDECC5	CMDINDEX = 5	Target data word 0 if the target address ends in 0b10.1000	Target data word 1 if the target address ends in 0b10.0000	Target data word 1 if the target address ends in 0b10.0000	Target data word 5
CMDDATA13:12 / CMDECC6	CMDINDEX = 6	Target data word 0 if the target address ends in 0b11.0000	Target data word 0 if the target address ends in 0b11.0000	Target data word 2 if the target address ends in 0b10.0000	Target data word 6
CMDDATA15:14 / CMDECC7	CMDINDEX = 7	Target data word 0 if the target address ends in 0b11.1000	Target data word 1 if the target address ends in 0b11.0000	Target data word 3 if the target address ends in 0b10.0000	Target data word 7

6.3.3.5 Executing a PROGRAM Operation

To program the flash memory:

- Select the command in the CMDTYPE register:
 - Set the COMMAND field in the CMDTYPE register to PROGRAM.
 - Set the SIZE field in the CMDTYPE register to the desired size (1, 2, 4, or 8 flash words). If a device does not support multi-word programming, select ONEWORD. If a device supports multi-word programming, and multi-word programming is desired, select the desired size which is less than or equal to the max size supported by the target device. The hardware will not check for invalid configuration of the SIZE field; software must ensure that the selection option is supported by the device. Note that SECTOR and BANK sizes are not valid sizes for PROGRAM operations. These sizes only apply to erase operations.
- Configure the program command in the CMDCTL register:

- a. On devices with ECC, the flash controller by default will generate the needed ECC bits from the data during the PROGRAM operation. Optionally, software can override the hardware ECC code generation and manually provide the ECC code to be programmed by setting the ECCGENOVR bit in CMDCTL register.
3. Select the target programming address in the CMDADDR and CMDBYTEN register:
 - a. Load the target system address into the CMDADDR register to indicate the base address from which programming will start. The target address must be a flash word address (64-bit aligned). The flash controller will translate the system address into the applicable flash region, bank ID, and bank address. If desired, after the operation completes the flash region, bank ID, and bank address can be read from the STATADDR register. In a multi-word program, STATADDR will indicate the bank ID and the final address which was programmed.
 - b. If sub-word programming (programming of less than the full 64 or 72 bit flash word) is desired, configure the CMDBYTEN register to set the bytes within the addressed flash word which are to be programmed. Each bit in CMDBYTEN corresponds to a byte in the addressed flash word to be programmed, including the ECC byte. For example, programming of the ECC code can be masked by clearing bit 8 in CMDBYTEN while programming the data bytes of the flash word. Note that there is a maximum number of program operations allowed per word line before a sector erase must be applied (see the device specific data sheet for the maximum).
4. Load the data to program into the CMDDATAx registers:
 - a. For a single flash word programming operation (64 or 72 bits depending on the presence of ECC), load the data into the CMDDATAx registers consistent with the alignment requirements (for devices which only support single-word programming, CMDDATA0 and CMDDATA1 are always used regardless of the target address).
 - b. For multi-word programming (if available and selected), load data into the CMDDATAx registers consistent with the alignment rules and the size of the multi-word program operation specified in step 1.
 - c. If ECCGENOVR in the CMDCTL register was set above (disabling hardware ECC code generation), then write the ECC data in the CMDDATAECC0 register (for single word programming) and optionally additional CMDDATAECCx registers as applicable for multi-word programming.
 - d. Note that the CMDDATA registers are used as bit masking registers by the flash controller during the program operation; after the operation completes, the values written to these registers will have been overwritten by the flash controller.
5. Ensure the [write protection](#) scheme is configured to allow writes to the target addresses (see the write protection section of this guide for additional information on configuring write protection).
6. Execute the program operation by writing 0x1 to the CMDEXEC register.
7. Monitor for completion of the program operation:
 - a. The STATCMD register can be polled to determine the status of the program operation. The CMDINPROGRESS bit will be set by hardware as soon as the command is initiated. The CMDDONE bit will be set when the operation terminates.
 - b. When CMDDONE is set, the CMDPASS bit will be reset or set at the same time to indicate whether the operation completed successfully or failed. If a program was attempted on a protected region, the FAILWEPROT bit is asserted. If the program operation cannot be completed successfully within the maximum program pulse count limit FAILVERIFY will be asserted. See the device-specific data sheet for maximum program times.
8. After completion of a program operation, the flash controller will configure several settings:
 - a. All dynamic write protection registers are set to a protected state (to protect against inadvertent programming)
 - b. All data registers are set to 1.
 - c. All program byte enables are cleared to 0.
9. Following programming of the flash memory, it is possible that there may be stale data in the processor's cache and prefetch logic. Before reading locations which were programmed, it is recommended to first flush the cache in the CPU subsystem.

6.3.4 ERASE Command

The erase command is used to erase individual sectors of flash memory (for MAIN, NONMAIN, or DATA regions) or a complete bank of flash memory (for MAIN regions only). From this erased state, bits can later be programmed to a '0' state or a '1' state as desired using the PROGRAM command. It is not possible to erase with a resolution lower than one sector (1KB) with sector alignment. For devices with multiple banks, a bank erase must be executed on all banks to erase the entire MAIN region on the device.

Note

After erasure, the memory contents of a sector are not deterministic until programmed. Erased bits do not always read as 1 after an erase. A memory location must be successfully programmed using the PROGRAM command before the memory location can be considered deterministic.

6.3.4.1 Erase Sector Masking Behavior

The flash controller provides an erase verification mechanism to extend the lifetime of the flash bank. The CMDWEPROTx registers are used as an erase mask and are manipulated by the flash controller during the execution of the erase operation. At the end of all erase operations, the CMDWEPROTx registers are set to a fully protected state to prevent against inadvertent programming and must be re-configured before attempting another program or erase operation.

6.3.4.2 Executing an ERASE Operation

To erase a sector or bank of the flash memory, follow these steps:

1. Select the command in the CMDTYPE register:
 - a. Set the COMMAND field in the CMDTYPE register to ERASE.
 - b. Set the SIZE field in the CMDTYPE register to SECTOR or BANK. Sizes other than SECTOR or BANK (for example, ONEWORD) are not supported by the ERASE command. It is the responsibility of software to check the configuration before issuing an ERASE command.
2. Select the target erase address in the CMDADDR register:
 - a. Store the target system address into the CMDADDR register to indicate the base address of the sector or bank to be erased. When performing a bank erase with write protection enabled (such that only unprotected sectors are erased), ensure that the address written to CMDADDR is in an unprotected sector. The flash controller will translate the system address into the applicable flash region, bank ID, and bank address. If desired, after the operation completes the flash region, bank ID, and bank address can be read from the STATADDR register.
3. Ensure the [write protection](#) scheme is configured to allow writes to the target sectors (see the write protection section of this guide for additional information on configuring write protection).
4. Execute the erase operation by writing 0x1 to the CMDEXEC register.
5. Monitor for completion of the erase operation:
 - a. The STATCMD register can be polled to determine the status of the erase operation. The CMDINPROGRESS bit will be set by hardware as soon as the command is initiated. The CMDDONE bit will be set when the operation terminates. When CMDDONE is set, the CMDPASS bit will be reset or set at the same time to indicate whether the operation completed successfully or failed.
 - b. If an erase was attempted on a protected region, the FAILWEPROT bit is asserted.
 - c. If the erase operation cannot be completed successfully within the maximum erase pulse count limit, FAILVERIFY will be asserted.
6. After completion of the erase operation, the flash controller will configure several settings to protect against inadvertent programming:
 - a. All dynamic write protection registers are set to a protected state.

6.3.5 READVERIFY Command

The read verify command can be used to read a flash location and compare the data to data which is preloaded into the CMDDATA registers of the flash controller. The command can be applied to a single flash word,

multiple flash words (if the device supports multi-word programming), an entire sector, or an entire bank. When performing a read verify on an entire sector or bank, the data in CMDDATAx will be re-used.

6.3.5.1 Executing a READVERIFY Operation

To execute a read verify command, follow these steps:

1. Select the command in the CMDTYPE register:
 - a. Set the COMMAND field in the CMDTYPE register to READVERIFY.
 - b. Set the SIZE field in the CMDTYPE register to the desired size.
2. Configure the read verify command in the CMDCTL register:
 - a. If the desire is to manually provide ECC bits along with the data, set the ECCGENOVR bit in the CMDCTL register. If ECCGENOVR is cleared, the flash controller will generate ECC bits for comparison based on the provided compare data.
3. Select the target address to verify on the CMDADDR register:
 - a. Load the target system address into the CMDADDR register to indicate the base address to be verified. The flash controller will translate the system address into the applicable flash region, bank ID, and bank address. If desired, after the operation completes the flash region, bank ID, and bank address can be read from the STATADDR register.
4. Load the data to verify into the CMDDATAx registers:
 - a. For single word verification, write the data to be verified to the CMDDATA0 and CMDDATA1 registers. For multi-word verification, if available on the target device, write the data to be verified to the appropriate CMDDATAx registers beyond CMDDATA0 and CMDDATA1.
5. Configure the byte enable settings in the CMDBYTEN register:
 - a. Any CMDBYTEN bit set to logic "0" will mask the associated data byte from being compared during the execution of the READVERIFY command. This can be used to verify data less than one flash word (less than 64 bits).
6. Execute the read verify operation by writing 0x1 to the CMDEXEC register.
7. Monitor for completion of the read verify operation:
 - a. The STATCMD register can be polled to determine the status of the erase operation. The CMDINPROGRESS bit will be set by hardware as soon as the command is initiated. The CMDDONE bit will be set when the operation terminates. When CMDDONE is set, the CMDPASS bit will be reset or set at the same time to indicate whether the read verification passed or failed.
 - b. The FAILVERIFY bit in STATCMD will be set if any data read from the flash did not match the expected data loaded in CMDDATAx.
8. After completion of the read verify operation, the flash controller will configure several settings:
 - a. All dynamic write protection registers are set to a protected state (to protect against inadvertent programming).
 - b. All data registers are set to '1's.
 - c. All program byte enables are cleared to '0's.

6.3.6 BLANKVERIFY Command

The blank verify (BLANKVERIFY) command can be used by application software to verify that a flash word is blank. A blank flash word is defined as a flash word which has been successfully erased with the [ERASE command](#) and not yet programmed away from that non-erased state with the [PROGRAM command](#).

After erase, a flash word is not in a deterministic state until it is programmed using the [PROGRAM command](#). This means that application software can not expect that erased bits will read back as '1's' after an erase. A memory location must be successfully programmed using the PROGRAM command before a read of that memory location can be considered deterministic and used by application software.

Because it is not possible to determine if a flash word is in an erased state by simply reading the location directly (as an erased location will return non-deterministic data when read), the BLANKVERIFY command can be used to test if a flash word is in a blank state, indicating it has not yet been programmed away from an erased state.

The BLANKVERIFY command can only be applied to a single flash word at a time.

6.3.6.1 Executing a BLANKVERIFY Operation

To execute a blank verify operation, follow these steps:

1. Select the command in the CMDTYPE register:
 - a. Set the COMMAND field in the CMDTYPE register to BLANKVERIFY.
 - b. Set the SIZE field in the CMDTYPE register to one word.
2. Select the target address to verify in the CMDADDR register:
 - a. Store the target system address into the CMDADDR register to indicate the base address to be verified. The flash controller will translate the system address into the applicable flash region, bank ID, and bank address. If desired, after the operation completes the flash region, bank ID, and bank address can be read from the STATADDR register. All 8 data bytes in the specified flash word, and the corresponding ECC byte, will be checked by BLANKVERIFY.
3. Execute the blank verify operation by writing 0x1 to the CMDEXEC register.
4. Monitor for completion of the blank verify operation:
 - a. The STATCMD register can be polled to determine the status of the erase operation. The CMDINPROGRESS bit will be set by hardware as soon as the command is initiated. The CMDDONE bit will be set when the operation terminates. When CMDDONE is set, the CMDPASS bit will be reset or set at the same time to indicate whether the blank verification passed or failed.
 - b. The FAILVERIFY bit in STATCMD will be set if the flash location is not erased.
5. After completion of the blank verify operation, the flash controller will configure several settings:
 - a. All dynamic write protection registers are set to a protected state (to protect against inadvertent programming)
 - b. All data registers are set to '1's.
 - c. All program byte enables are cleared to '0's.

6.3.7 Command Diagnostics

The flash controller updates several software-readable registers to communicate information about an initiated operation.

6.3.7.1 Command Status

The STATCMD register is a read-only register which provides diagnostic information about an operation which is been initiated or completed. The CMDINPROGRESS bit indicates that an operation is currently ongoing. The CMDDONE bit indicates that an operation has completed. These bits can be polled by software to determine the state of the flash controller during operations.

6.3.7.2 Address Translation

The STATADDR register is a read-only register which can be read to determine the current bank ID, region ID, and bank address which the flash controller is pointing to. These values can increment during execution of certain commands, in which case the value present after the completion of a command indicates the last address touched by the flash controller.

6.3.7.3 Pulse Counts

The STATPCNT register is a read-only register which can be read to determine the current pulse count applied during a program or erase operation.

6.3.8 Overriding the System Address With a Bank ID, Region ID, and Bank Address

Normally, flash controller commands are targeted to a specific flash location by loading a system memory map address into the CMDADDR register. This is the recommended way to specify the target address for a PROGRAM, ERASE, READVERIFY, or BLANKVERIFY command. In this mode of operation, the flash controller will automatically translate the system address into the corresponding bank ID, region ID, and bank address which are used to execute the command on the flash memory. Application software does not need to specify these items individually; only the system address is needed.

However, in some circumstances it can be desirable to directly specify the target flash bank, region, and address in the specified bank/region. For example, if the desire is to erase a complete bank when doing a mass erase operation on the device, application software actually does not need to have any knowledge of the system address of the bank to be erased- it only needs to specify the bank ID and region ID to the flash controller to erase the bank.

To use the flash controller to execute a command in address override mode, set the ADDRXLATEOVR bit in the CMDCTL register, and specify the bank ID, region, and bank address before executing the command. To return to system addressed mode, clear ADDRXLATEOVR. ADDRXLATEOVR is cleared by default (supporting system address operation).

Example Case - Bank Erase with ADDRXLATEOVR

To erase the MAIN region of BANK0 by specifying the bank ID and region instead of the system address, follow the steps in [Section 6.3.4.2](#), but replace step 2 with the alternate steps given below:

1. Set the ADDRXLATEOVR bit in CMDCTL to enable address translation override mode
2. Specify BANK0 by setting the BANKSEL field to 0x1 in the CMDCTL register
3. Specify the MAIN region by setting the REGIONSEL field to 0x1 in the CMDCTL register
4. Set the CMDADDR register to 0x0000.0000

6.3.9 FLASHCTL Events

The flash controller contains one [event publisher](#) and no [event subscribers](#). One event publisher (CPU_INT) manages FLASHCTL interrupt requests (IRQs) to the CPU subsystem through a [static event route](#).

[Table 6-8](#) summarizes the FLASHCTL events.

Table 6-8. FLASHCTL Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU Interrupt Event	Publisher	FLASHCTL	CPU Subsystem	Static route	CPU_INT registers	Fixed interrupt route from FLASHCTL to CPU

6.3.9.1 CPU Interrupt Event Publisher

The FLASHCTL module provides one interrupt source which can be configured to source a [CPU interrupt event](#). The FLASHCTL interrupt conditions are given in [Table 6-9](#).

Table 6-9. FLASHCTL CPU Interrupt Conditions

Index (IIDX)	Name	Description
0	DONE	Indicating that the FLASHCTL operation has completed

The CPU interrupt event configuration is managed with the CPU_INT event management registers. See [Section 7.2.5](#) for guidance on configuring these registers.

6.4 Write Protection

The flash controller provides two write protection mechanisms (one static, one dynamic) which are applied in parallel (logical OR) to protect user-specified sectors during any attempted program or erase operation. If a program or erase operation is issued to a write protected flash sector, the operation will terminate with a FAILWEPROT error reported in the STATCMD register.

6.4.1 Write Protection Resolution

The write protection resolution for both the static and dynamic write protection mechanisms is dependent on the flash bank and memory region which is being protected.

Table 6-10. Write Protection Resolution by Region

Flash Bank	Memory Region	Write Protection Resolution
0	NONMAIN	512B (entire region)
	MAIN	1KB (1 sector) for first 32KB (32 sectors) 8KB (8 sectors) for remaining sectors
1-4	MAIN	8KB (8 sectors)
	DATA	1KB (1 sector)

6.4.2 Static Write Protection

The static write protection scheme is configured and latched during device boot by the immutable ROM boot code before the main application in flash can execute. After a flash sector is configured to be write protected through the static write protection mechanism, software cannot remove the protection at runtime without a reboot. Thus, sectors protected by static write protection can be thought of as immutable after boot. This type of protection is useful for protecting a custom bootloader in a dual-image application, or for extending the secure root of trust from the ROM boot code into a portion of the main flash region to enable a secure boot manager with locked public keys.

The static write protection scheme is configured by programming the appropriate bits in the NONMAIN flash region, which is read by the boot code before the main application starts. The NONMAIN flash sector can be statically write protected, resulting in a system in which the static write protection scheme is fully permanent and cannot be modified. If the NONMAIN sector is configured to be statically protected, along with any other flash sectors, all statically protected flash sectors can be functionally viewed as read only memory which cannot be updated by any means.

Static write protection can be configured for all sectors of flash memory across all banks present on a device. Instructions for configuring the static write protection, along with the rest of the NONMAIN region, are given in [Section 1.4](#).

6.4.3 Dynamic Write Protection

The dynamic write protection scheme is intended to be configured at runtime by software. This scheme provides a simple way for application software to specify sectors to protect from modification by any program or erase operations that are issued with the flash controller. Unlike the static write protection mechanism, the dynamic mechanism is not lockable and thus does not provide any level of data security.

There are two primary uses for dynamic write protection. First, it provides an extra level of robustness against unintentional program or erase of specified sectors in applications that involve in-system programming for either firmware updates or EEPROM emulation. Second, it provides a way to simplify a situation where a bank erase is desired but a small number of sectors should not be erased when a bank erase command is issued. One example would be an application running on a single-bank device in which most of the MAIN region sectors are used to store the executable image, but a few sectors are used to store device-specific data that should not be erased during a firmware update. In that case, the sectors containing the device-specific data can be protected with dynamic write protection, and a bank erase command can be issued to erase all other sectors. This has the benefit that the majority of the MAIN region can be erased with a single command (a bank erase) rather than individual sector-by-sector commands, which would have a longer overall erase time and use more energy.

The dynamic write protection scheme is configured by setting up the CMDWEPROTx registers in the flash controller. The CMDWEPROTx registers cover one flash bank at a time. This means that these registers must be configured with knowledge of which flash bank an attempted program or erase operation will be applied to. Note that the CMDWEPROTx registers are reset to a protected state at the end of all program and erase operations. These registers must be re-configured by software before a new operation is initiated.

6.4.3.1 Configuring Protection for the MAIN Region

The CMDWEPROTA register is used to configure the dynamic write protection for the first 32 sectors (32KB) of the MAIN region in BANK0. Each bit corresponds to one sector of the MAIN region, starting from the beginning

of the MAIN region. CMDWEPROTA is only applicable to operations on the lower 32 sectors of the MAIN region of BANK0 (sectors 0-31). It is not used during program/erase operations applied to other sectors.

The CMDWEPROTB register is also used to configure the dynamic write protection for the MAIN region. There are two modes in which CMDWEPROTB is applied, depending on whether the program/erase operation is being applied to BANK0 or BANK1-4. In the case of a program/erase operation on BANK0, CMDWEPROTB protects sectors 32-255 in 8-sector increments (1 bit per 8KB), starting from BIT4. BIT0-BIT3 in CMDWEPROTB are ignored. The lower 32 sectors (sectors 0-31 of BANK0) are protected by CMDWEPROTA. In the case of a program/erase operation on BANK1-4, CMDWEPROTB protects sectors 0-255 in 8-sector increments (1 bit per 8 sectors), starting from BIT0 in CMDWEPROTB.

The CMDWEPROTC register configures the dynamic write protection for the MAIN region. For BANK0-4, CMDWEPROTC protects sectors 256-511 in 8-sector increments (1 bit per 8 sectors).

6.4.3.2 Configuring Protection for the NONMAIN Region

The CMDWEPROTNM register protects the NONMAIN region from program and erase. One protection bit is provided per sector. Devices only have one NONMAIN sector and it will be in BANK0.

6.5 Read Interface

The read interface provides the read path to the CPU subsystem (for instruction/data fetch), the read path to the peripheral bus (for use by the DMA controller or CPU), main bank address swapping, and detection and reporting of ECC SEC or DED errors.

6.5.1 Bank Address Swapping

Devices that contain more than one bank support swapping of the MAIN regions of the banks within the address space. This mechanism enables two versions of application firmware to be programmed into the device without the firmware needing to know which physical bank it exists in.

[Table 6-11](#) gives an example of the mapping before and after a bank swap is requested for a device with a 512KB main flash split across 2 banks (256KB each).

Table 6-11. Bank Address Swap Translation

Bank and Region	Address Space Before Swap	Address Space After Swap
BANK0 MAIN	0x0000.0000 – 0x0003.FFFF	0x0004.0000 – 0x0007.FFFF
BANK1 MAIN	0x0004.0000 – 0x0007.FFFF	0x0000.0000 – 0x0003.FFFF

After a device reset, the MAIN region of the lower bank is always mapped to the lowest address space. The application software is responsible for determining if a bank address swap is to be applied. The bank address swap control is contained within the SYSCTL module; see the SYSCTL chapter for register and bit definitions. During an address swap, the application software must meet the following constraints:

1. The software must disable interrupts before issuing the bank swap command.
2. The software must poll the bank swap status after issuing the bank swap command before proceeding with execution.
3. If the swap command and poll status routines are executing from flash, they must exist at the exact same location in both banks such that execution resumes from where it left off after the bank swap. This restriction does not apply if the bank swap and status polling is done from SRAM and not from flash memory.

6.5.2 ECC Error Handling

The read interface corrects single bit errors in a 64-bit flash word (SEC) and detects dual-bit errors in a 64-bit flash word (DED). ECC checks are ignored for an “all-1’s” scenario or an “all-0’s” scenario.

6.5.2.1 Single bit (correctable) errors

Single bit errors are corrected automatically before the requested data is returned to either the CPU subsystem or the peripheral bus (DMA). Errors occurring as a result of a debug access are not reported.

6.5.2.2 Dual bit (uncorrectable) errors

Dual bit errors are detectable but not correctable and are indicated to SYSCTL. SYSCTL can be configured to generate either a nonmaskable interrupt (NMI) or a reset, as uncorrectable errors can be fatal. Errors occurring as a result of a debug access are not reported.

6.6 FLASHCTL Registers

Table 6-12 lists the memory-mapped registers for the FLASHCTL registers. All register offset addresses not listed in Table 6-12 should be considered as reserved locations and the register contents should not be modified.

Table 6-12. FLASHCTL Registers

Offset	Acronym	Register Name	Group	Section
1020h	IIDX	Interrupt Index Register		Go
1028h	IMASK	Interrupt Mask Register		Go
1030h	RIS	Raw Interrupt Status Register		Go
1038h	MIS	Masked Interrupt Status Register		Go
1040h	ISSET	Interrupt Set Register		Go
1048h	ICLR	Interrupt Clear Register		Go
1100h	CMDEXEC	Command Execute Register		Go
1104h	CMDTYPE	Command Type Register		Go
1108h	CMDCTL	Command Control Register		Go
1120h	CMDADDR	Command Address Register		Go
1124h	CMDBYTEN	Command Program Byte Enable Register		Go
112Ch	CMDDATAINDEX	Command Data Index Register		Go
1130h	CMDDATA0	Command Data Register 0		Go
1134h	CMDDATA1	Command Data Register 1		Go
1138h	CMDDATA2	Command Data Register 2		Go
113Ch	CMDDATA3	Command Data Register Bits 127:96		Go
1140h	CMDDATA4	Command Data Register 4		Go
1144h	CMDDATA5	Command Data Register 5		Go
1148h	CMDDATA6	Command Data Register 6		Go
114Ch	CMDDATA7	Command Data Register 7		Go
1150h	CMDDATA8	Command Data Register 8		Go
1154h	CMDDATA9	Command Data Register 9		Go
1158h	CMDDATA10	Command Data Register 10		Go
115Ch	CMDDATA11	Command Data Register 11		Go
1160h	CMDDATA12	Command Data Register 12		Go
1164h	CMDDATA13	Command Data Register 13		Go
1168h	CMDDATA14	Command Data Register 14		Go
116Ch	CMDDATA15	Command Data Register 15		Go
1170h	CMDDATA16	Command Data Register 16		Go
1174h	CMDDATA17	Command Data Register 17		Go
1178h	CMDDATA18	Command Data Register 18		Go
117Ch	CMDDATA19	Command Data Register 19		Go
1180h	CMDDATA20	Command Data Register 20		Go
1184h	CMDDATA21	Command Data Register 21		Go
1188h	CMDDATA22	Command Data Register 22		Go
118Ch	CMDDATA23	Command Data Register 23		Go
1190h	CMDDATA24	Command Data Register 24		Go
1194h	CMDDATA25	Command Data Register 25		Go
1198h	CMDDATA26	Command Data Register 26		Go
119Ch	CMDDATA27	Command Data Register 27		Go
11A0h	CMDDATA28	Command Data Register 28		Go

Table 6-12. FLASHCTL Registers (continued)

Offset	Acronym	Register Name	Group	Section
11A4h	CMDDATA29	Command Data Register 29		Go
11A8h	CMDDATA30	Command Data Register 30		Go
11ACH	CMDDATA31	Command Data Register 31		Go
11B0h	CMDDATAECC0	Command Data Register ECC 0		Go
11B4h	CMDDATAECC1	Command Data Register ECC 1		Go
11B8h	CMDDATAECC2	Command Data Register ECC 2		Go
11BCh	CMDDATAECC3	Command Data Register ECC 3		Go
11C0h	CMDDATAECC4	Command Data Register ECC 4		Go
11C4h	CMDDATAECC5	Command Data Register ECC 5		Go
11C8h	CMDDATAECC6	Command Data Register ECC 6		Go
11CCh	CMDDATAECC7	Command Data Register ECC 7		Go
11D0h	CMDWEPROTA	Command Write Erase Protect A Register		Go
11D4h	CMDWEPROTB	Command Write Erase Protect B Register		Go
11D8h	CMDWEPROTC	Command Write Erase Protect C Register		Go
1210h	CMDWEPROTNM	Command Write Erase Protect Non-Main Register		Go
13B4h	CFGPCNT	Pulse Counter Configuration Register		Go
13D0h	STATCMD	Command Status Register		Go
13D4h	STATADDR	Address Status Register		Go
13D8h	STATPCNT	Pulse Count Status Register		Go

Complex bit access types are encoded to fit into small table cells. [Table 6-13](#) shows the codes that are used for access types in this section.

Table 6-13. FLASHCTL Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
W	W	Write
Reset or Default Value		
-n		Value after reset or the default value

6.6.1 IIDX (Offset = 1020h) [Reset = 00000000h]

IIDX is shown in [Figure 6-4](#) and described in [Table 6-14](#).

Return to the [Summary Table](#).

The interrupt index (IIDX) register provides the index of the highest priority pending and enabled interrupt.

Figure 6-4. IIDX

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							STAT
R-0h							R-0h

Table 6-14. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	Reserved
0	STAT	R	0h	Index corresponding to the highest priority pending interrupt source. This value may be used as an address offset for fast, deterministic handling in the interrupt service routine. A read of the IIDX register will clear the corresponding interrupt status in the RIS and MIS registers. 0h (R/W) = No Interrupt Pending 1h (R/W) = DONE Interrupt Pending

6.6.2 IMASK (Offset = 1028h) [Reset = 00000000h]

IMASK is shown in [Figure 6-5](#) and described in [Table 6-15](#).

Return to the [Summary Table](#).

The interrupt mask (IMASK) register holds the current interrupt mask settings.

Figure 6-5. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							DONE
R/W-0h							R/W-0h

Table 6-15. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R/W	0h	Reserved
0	DONE	R/W	0h	Enable or disable the DONE interrupt. 0h (R/W) = Interrupt is masked out 1h (R/W) = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set

6.6.3 RIS (Offset = 1030h) [Reset = 00000000h]

RIS is shown in [Figure 6-6](#) and described in [Table 6-16](#).

Return to the [Summary Table](#).

The raw interrupt status (RIS) register holds the current raw interrupt status.

Figure 6-6. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							DONE
R-0h							R-0h

Table 6-16. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	Reserved
0	DONE	R	0h	Raw status of the DONE interrupt. 0h (R/W) = Interrupt did not occur 1h (R/W) = Interrupt occurred

6.6.4 MIS (Offset = 1038h) [Reset = 00000000h]

MIS is shown in [Figure 6-7](#) and described in [Table 6-17](#).

Return to the [Summary Table](#).

The masked interrupt status (MIS) register holds the current masked interrupt status.

Figure 6-7. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							DONE
R-0h							R-0h

Table 6-17. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	Reserved
0	DONE	R	0h	Masked status of the DONE interrupt. 0h (R/W) = Masked interrupt did not occur 1h (R/W) = Masked interrupt occurred

6.6.5 ISET (Offset = 1040h) [Reset = 00000000h]

ISET is shown in [Figure 6-8](#) and described in [Table 6-18](#).

Return to the [Summary Table](#).

The interrupt set (ISET) register may be used to set an interrupt to pending from software.

Figure 6-8. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED							DONE
W-0h							W-0h

Table 6-18. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	W	0h	Reserved
0	DONE	W	0h	Set the DONE interrupt. 0h (R/W) = Writing a 0 has no effect 1h (R/W) = Set RIS bit

6.6.6 ICLR (Offset = 1048h) [Reset = 00000000h]

ICLR is shown in [Figure 6-9](#) and described in [Table 6-19](#).

Return to the [Summary Table](#).

The interrupt clear (ICLR) register may be used to clear a pending interrupt.

Figure 6-9. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED							DONE
W-0h							W-0h

Table 6-19. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	W	0h	Reserved
0	DONE	W	0h	Clear the DONE interrupt. 0h (R/W) = Writing a 0 has no effect 1h (R/W) = Clear RIS bit

6.6.7 CMDEXEC (Offset = 1100h) [Reset = 00000000h]

CMDEXEC is shown in [Figure 6-10](#) and described in [Table 6-20](#).

Return to the [Summary Table](#).

Command Execute Register

Initiates execution of the command specified in the CMDTYPE register. This register is blocked for writes after being written to 1 and prior to STATCMD.DONE being set by hardware. Hardware clears this register after the processing of the command has completed.

Figure 6-10. CMDEXEC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED															VAL
R/W-0h															R/W-0h

Table 6-20. CMDEXEC Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R/W	0h	Reserved
0	VAL	R/W	0h	Command Execute value Initiates execution of the command specified in the CMDTYPE register. 0h (R/W) = Command will not execute or is not executing in hardware 1h (R/W) = Command will execute or is executing in hardware

6.6.8 CMDTYPE (Offset = 1104h) [Reset = 00000000h]

CMDTYPE is shown in [Figure 6-11](#) and described in [Table 6-21](#).

Return to the [Summary Table](#).

Command Type Register

Specifies the type of command to be executed by hardware. This register is blocked for writes after CMDEXEC is written to a 1 and prior to STATCMD.DONE being set by the hardware to indicate that command execution has completed.

Figure 6-11. CMDTYPE

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED		SIZE		RESERVED		COMMAND	
R/W-0h		R/W-0h		R/W-0h		R/W-0h	

Table 6-21. CMDTYPE Field Descriptions

Bit	Field	Type	Reset	Description
31-7	RESERVED	R/W	0h	Reserved
6-4	SIZE	R/W	0h	Command size 0h (R/W) = Operate on 1 flash word 1h (R/W) = Operate on 2 flash words 2h (R/W) = Operate on 4 flash words 3h (R/W) = Operate on 8 flash words 4h (R/W) = Operate on a flash sector 5h (R/W) = Operate on an entire flash bank
3	RESERVED	R/W	0h	Reserved
2-0	COMMAND	R/W	0h	Command type 0h (R/W) = No Operation 1h (R/W) = Program 2h (R/W) = Erase 3h (R/W) = Read Verify - Perform a standalone read verify operation. 6h (R/W) = Blank Verify - Check whether a flash word is in the erased state. This command may only be used with CMDTYPE.SIZE = ONEWORD

6.6.9 CMDCTL (Offset = 1108h) [Reset = 00000000h]

CMDCTL is shown in [Figure 6-12](#) and described in [Table 6-22](#).

Return to the [Summary Table](#).

Command Control Register This register configures specific capabilities of the state machine for related to the execution of a command. This register is blocked for writes after CMDEXEC is written to a 1 and prior to STATCMD.DONE being set by the hardware to indicate that command execution has completed.

Figure 6-12. CMDCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED		RESERVED	SSERASEDIS	RESERVED		ECCGENOVR	ADDRXLATEOVR
R/W-0h		R/W-	R/W-0h	R/W-		R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
RESERVED		RESERVED	REGIONSEL				RESERVED
R/W-		R/W-0h	R/W-0h				R/W-
7	6	5	4	3	2	1	0
RESERVED			BANKSEL	RESERVED			
R/W-			R/W-0h	R/W-			

Table 6-22. CMDCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-22	RESERVED	R/W	0h	Reserved
21	RESERVED	R/W	0h	
20	SSERASEDIS	R/W	0h	Disable Stair-Step Erase. If set, the default VHV trim voltage setting will be used for all erase pulses. By default, this bit is reset, meaning that the VHV voltage will be stepped during successive erase pulses. The step count, step voltage, begin and end voltages are all hard-wired. 0h (R/W) = Enable 1h (R/W) = Disable
19-18	RESERVED	R/W	0h	
17	ECCGENOVR	R/W	0h	Override hardware generation of ECC data for program. Use data written to CMDDATAECC*. 0h (R/W) = Do not override 1h (R/W) = Override
16	ADDRXLATEOVR	R/W	0h	Override hardware address translation of address in CMDADDR from a system address to the corresponding bank address and bank ID. When set, CMDADDR will be used directly as the bank address, CMDCTL.REGIONSEL will be used directly as the region ID, and CMDCTL.BANKSEL will be used directly as the bank ID (if the device contains multiple banks). 0h (R/W) = Do not override 1h (R/W) = Override
15-14	RESERVED	R/W	0h	
13	RESERVED	R/W	0h	Reserved
12-9	REGIONSEL	R/W	0h	Bank Region A specific region ID can be written to this field to indicate to which region an operation should be applied if CMDCTL.ADDRXLATEOVR is set. 1h (R/W) = Main Region 2h (R/W) = Non-Main Region

Table 6-22. CMDCTL Field Descriptions (continued)

Bit	Field	Type	Reset	Description
8-5	RESERVED	R/W	0h	
4	BANKSEL	R/W	0h	Bank Select A specific Bank ID can be written to this field to indicate to which bank an operation should be applied if CMDCTL.ADDRXLATEOVR is set. 1h (R/W) = Bank 0 2h (R/W) = Bank 1 4h (R/W) = Bank 2 8h (R/W) = Bank 3 10h (R/W) = Bank 4
3-0	RESERVED	R/W	0h	

6.6.10 CMDADDR (Offset = 1120h) [Reset = 00000000h]

CMDADDR is shown in [Figure 6-13](#) and described in [Table 6-23](#).

Return to the [Summary Table](#).

Command Address Register:

This register forms the target address of a command. The use cases are as follows:

- 1) For single-word program, this address indicates the flash bank word to be programmed.
- 2) For multi-word program, this address indicates the first flash bank address for the program. The address will be incremented for further words.
- 3) For sector erase, this address indicates the sector to be erased.
- 4) For bank erase, the address indicates the bank to be erased.
- 5) For read verify, the address indications follow program/erase listed above.

Note the address written to this register will be submitted for translation to the flash address translation interface, and the translated address will be used to access the bank. However, if the CMDCTL.ADDRXLATEOVR bit is set, then the address written to this register will be used directly as the bank address.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

Figure 6-13. CMDADDR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-0h																															

Table 6-23. CMDADDR Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	0h	Address value 0h = Minimum value of [VAL] FFFFFFFFh = Maximum value of [VAL]

6.6.11 CMDBYTEN (Offset = 1124h) [Reset = 00000000h]

CMDBYTEN is shown in [Figure 6-14](#) and described in [Table 6-24](#).

Return to the [Summary Table](#).

Command Program Byte Enable Register:

This register forms a per-byte enable for programming data. For data bytes to be programmed, a 1 must be written to the corresponding bit in this register. Normally, all bits are written to 1, allowing program of full flash words. However, leaving some bits 0 allows programming of 8-bit, 16-bit, 32-bit or 64-bit portions of a flash word. In addition, the read verify command will ignore data bytes read from the flash in its comparison if the corresponding CMDBYTEN bit is 0.

For 64-bit flash word size devices, the CMDBYTEN register uses BIT7-0 to enable each data byte and BIT8 to enable the ECC code byte.

For 128-bit flash word size devices, the CMDBYTEN register uses BIT15-0 to enable each data byte and BIT17-16 to enable each ECC code byte.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is written to all 0 after the completion of all commands.

Figure 6-14. CMDBYTEN

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED														RESERVED										VAL							
R/W-0h														R/W-										R/W-0h							

Table 6-24. CMDBYTEN Field Descriptions

Bit	Field	Type	Reset	Description
31-18	RESERVED	R/W	0h	Reserved
17-8	RESERVED	R/W	0h	
7-0	VAL	R/W	0h	Command Byte Enable value. A 1-bit per flash word byte value is placed in this register. 0h = Minimum value of [VAL] 0003FFFFh = Maximum value of [VAL]

6.6.12 CMDDATAINDEX (Offset = 112Ch) [Reset = 00000000h]

CMDDATAINDEX is shown in [Figure 6-15](#) and described in [Table 6-25](#).

Return to the [Summary Table](#).

Command Program Data Index Register:

When multiple data registers are available for multi-word program, this register can be written with an index which points to one of the data registers. When a write to CMDDATA* is done, the data will be written to the physical data register indexed by the value in this register.

Up to 8 data registers can be present, so this register can be written with 0x0 to 0x7. If less than 8 data registers are present, successive MSB bits of this register are ignored when indexing the CMDDATA* registers.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

Figure 6-15. CMDDATAINDEX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															VAL
R/W-0h																															R/W-0h

Table 6-25. CMDDATAINDEX Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R/W	0h	Reserved
2-0	VAL	R/W	0h	Data register index 0h = Minimum value of [VAL] 7h = Maximum value of [VAL]

6.6.13 CMDDATA0 (Offset = 1130h) [Reset = FFFFFFFFh]

CMDDATA0 is shown in [Figure 6-16](#) and described in [Table 6-26](#).

Return to the [Summary Table](#).

Command Data Register 0

This register contains the data for a command.

This register represents bits 31:0 of flash word data register 0.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-16. CMDDATA0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFFh																															

Table 6-26. CMDDATA0 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFFh = Maximum value of [VAL]

6.6.14 CMDDATA1 (Offset = 1134h) [Reset = FFFFFFFFh]

CMDDATA1 is shown in [Figure 6-17](#) and described in [Table 6-27](#).

Return to the [Summary Table](#).

Command Data Register 1

This register contains the data for a command.

This register represents bits 63:32 of flash word data register 0.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-17. CMDDATA1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-27. CMDDATA1 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.15 CMDDATA2 (Offset = 1138h) [Reset = FFFFFFFFh]

CMDDATA2 is shown in [Figure 6-18](#) and described in [Table 6-28](#).

Return to the [Summary Table](#).

Command Data Register 2

This register contains the data for a command.

This register represents bits 31:0 of flash word data register 1.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-18. CMDDATA2

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-28. CMDDATA2 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.16 CMDDATA3 (Offset = 113Ch) [Reset = FFFFFFFFh]

CMDDATA3 is shown in [Figure 6-19](#) and described in [Table 6-29](#).

Return to the [Summary Table](#).

Command Data Register 3

This register contains the data for a command.

This register represents bits 63:32 of flash word data register 1.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-19. CMDDATA3

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFFh																															

Table 6-29. CMDDATA3 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFFh = Maximum value of [VAL]

6.6.17 CMDDATA4 (Offset = 1140h) [Reset = FFFFFFFFh]

CMDDATA4 is shown in [Figure 6-20](#) and described in [Table 6-30](#).

Return to the [Summary Table](#).

Command Data Register 4

This register contains the data for a command.

This register represents bits 31:0 of flash word data register 2.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-20. CMDDATA4

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-30. CMDDATA4 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. T 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.18 CMDDATA5 (Offset = 1144h) [Reset = FFFFFFFFh]

CMDDATA5 is shown in [Figure 6-21](#) and described in [Table 6-31](#).

Return to the [Summary Table](#).

Command Data Register 5

This register contains the data for a command.

This register represents bits 63:32 of flash word data register 2.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-21. CMDDATA5

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFFh																															

Table 6-31. CMDDATA5 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFFh = Maximum value of [VAL]

6.6.19 CMDDATA6 (Offset = 1148h) [Reset = FFFFFFFFh]

CMDDATA6 is shown in [Figure 6-22](#) and described in [Table 6-32](#).

Return to the [Summary Table](#).

Command Data Register 6

This register contains the data for a command.

This register represents bits 31:0 of flash word data register 3.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-22. CMDDATA6

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFFh																															

Table 6-32. CMDDATA6 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFFh = Maximum value of [VAL]

6.6.20 CMDDATA7 (Offset = 114Ch) [Reset = FFFFFFFFh]

CMDDATA7 is shown in [Figure 6-23](#) and described in [Table 6-33](#).

Return to the [Summary Table](#).

Command Data Register 7

This register contains the data for a command.

This register represents bits 63:32 of flash word data register 3.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-23. CMDDATA7

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFFh																															

Table 6-33. CMDDATA7 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFFh = Maximum value of [VAL]

6.6.21 CMDDATA8 (Offset = 1150h) [Reset = FFFFFFFFh]

CMDDATA8 is shown in [Figure 6-24](#) and described in [Table 6-34](#).

Return to the [Summary Table](#).

Command Data Register 8

This register contains the data for a command.

This register represents bits 31:0 of flash word data register 4.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-24. CMDDATA8

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-34. CMDDATA8 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.22 CMDDATA9 (Offset = 1154h) [Reset = FFFFFFFFh]

CMDDATA9 is shown in [Figure 6-25](#) and described in [Table 6-35](#).

Return to the [Summary Table](#).

Command Data Register 9

This register contains the data for a command.

This register represents bits 63:32 of flash word data register 4.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-25. CMDDATA9

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFFh																															

Table 6-35. CMDDATA9 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFFh = Maximum value of [VAL]

6.6.23 CMDDATA10 (Offset = 1158h) [Reset = FFFFFFFFh]

CMDDATA10 is shown in [Figure 6-26](#) and described in [Table 6-36](#).

Return to the [Summary Table](#).

Command Data Register 10

This register contains the data for a command.

This register represents bits 31:0 of flash word data register 5.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-26. CMDDATA10

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-36. CMDDATA10 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.24 CMDDATA11 (Offset = 115Ch) [Reset = FFFFFFFFh]

CMDDATA11 is shown in [Figure 6-27](#) and described in [Table 6-37](#).

Return to the [Summary Table](#).

Command Data Register 11

This register contains the data for a command.

This register represents bits 63:32 of flash word data register 5.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-27. CMDDATA11

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFFh																															

Table 6-37. CMDDATA11 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFFh = Maximum value of [VAL]

6.6.25 CMDDATA12 (Offset = 1160h) [Reset = FFFFFFFFh]

CMDDATA12 is shown in [Figure 6-28](#) and described in [Table 6-38](#).

Return to the [Summary Table](#).

Command Data Register 12

This register contains the data for a command.

This register represents bits 31:0 of flash word data register 6.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-28. CMDDATA12

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-38. CMDDATA12 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.26 CMDDATA13 (Offset = 1164h) [Reset = FFFFFFFFh]

CMDDATA13 is shown in [Figure 6-29](#) and described in [Table 6-39](#).

Return to the [Summary Table](#).

Command Data Register 13

This register contains the data for a command.

This register represents bits 63:32 of flash word data register 6.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-29. CMDDATA13

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFFh																															

Table 6-39. CMDDATA13 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFFh = Maximum value of [VAL]

6.6.27 CMDDATA14 (Offset = 1168h) [Reset = FFFFFFFFh]

CMDDATA14 is shown in [Figure 6-30](#) and described in [Table 6-40](#).

Return to the [Summary Table](#).

Command Data Register 14

This register contains the data for a command.

This register represents bits 31:0 of flash word data register 7.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-30. CMDDATA14

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-40. CMDDATA14 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.28 CMDDATA15 (Offset = 116Ch) [Reset = FFFFFFFFh]

CMDDATA15 is shown in [Figure 6-31](#) and described in [Table 6-41](#).

Return to the [Summary Table](#).

Command Data Register 15

This register contains the data for a command.

This register represents bits 63:32 of flash word data register 7.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all NoWrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-31. CMDDATA15

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFFh																															

Table 6-41. CMDDATA15 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFFh = Maximum value of [VAL]

6.6.29 CMDDATA16 (Offset = 1170h) [Reset = FFFFFFFFh]

CMDDATA16 is shown in [Figure 6-32](#) and described in [Table 6-42](#).

Return to the [Summary Table](#).

Command Data Register 16

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 31:0 of flash word data register 4.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-32. CMDDATA16

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-42. CMDDATA16 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.30 CMDDATA17 (Offset = 1174h) [Reset = FFFFFFFFh]

CMDDATA17 is shown in [Figure 6-33](#) and described in [Table 6-43](#).

Return to the [Summary Table](#).

Command Data Register 17

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 63:32 of flash word data register 4.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-33. CMDDATA17

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFFh																															

Table 6-43. CMDDATA17 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFFh = Maximum value of [VAL]

6.6.31 CMDDATA18 (Offset = 1178h) [Reset = FFFFFFFFh]

CMDDATA18 is shown in [Figure 6-34](#) and described in [Table 6-44](#).

Return to the [Summary Table](#).

Command Data Register 18

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 95:64 of flash word data register 4.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-34. CMDDATA18

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-44. CMDDATA18 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.32 CMDDATA19 (Offset = 117Ch) [Reset = FFFFFFFFh]

CMDDATA19 is shown in [Figure 6-35](#) and described in [Table 6-45](#).

Return to the [Summary Table](#).

Command Data Register 19

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 127:96 of flash word data register 4.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-35. CMDDATA19

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-45. CMDDATA19 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.33 CMDDATA20 (Offset = 1180h) [Reset = FFFFFFFFh]

CMDDATA20 is shown in [Figure 6-36](#) and described in [Table 6-46](#).

Return to the [Summary Table](#).

Command Data Register 20

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 31:0 of flash word data register 5.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-36. CMDDATA20

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-46. CMDDATA20 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.34 CMDDATA21 (Offset = 1184h) [Reset = FFFFFFFFh]

CMDDATA21 is shown in [Figure 6-37](#) and described in [Table 6-47](#).

Return to the [Summary Table](#).

Command Data Register 21

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 63:32 of flash word data register 5.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-37. CMDDATA21

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-47. CMDDATA21 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.35 CMDDATA22 (Offset = 1188h) [Reset = FFFFFFFFh]

CMDDATA22 is shown in [Figure 6-38](#) and described in [Table 6-48](#).

Return to the [Summary Table](#).

Command Data Register 22

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 95:64 of flash word data register 5.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-38. CMDDATA22

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-48. CMDDATA22 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.36 CMDDATA23 (Offset = 118Ch) [Reset = FFFFFFFFh]

CMDDATA23 is shown in [Figure 6-39](#) and described in [Table 6-49](#).

Return to the [Summary Table](#).

Command Data Register 23

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 127:96 of flash word data register 5.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-39. CMDDATA23

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-49. CMDDATA23 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.37 CMDDATA24 (Offset = 1190h) [Reset = FFFFFFFFh]

CMDDATA24 is shown in [Figure 6-40](#) and described in [Table 6-50](#).

Return to the [Summary Table](#).

Command Data Register 24

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 31:0 of flash word data register 6.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-40. CMDDATA24

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-50. CMDDATA24 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.38 CMDDATA25 (Offset = 1194h) [Reset = FFFFFFFFh]

CMDDATA25 is shown in [Figure 6-41](#) and described in [Table 6-51](#).

Return to the [Summary Table](#).

Command Data Register 25

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 63:32 of flash word data register 6.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-41. CMDDATA25

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-51. CMDDATA25 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.39 CMDDATA26 (Offset = 1198h) [Reset = FFFFFFFFh]

CMDDATA26 is shown in [Figure 6-42](#) and described in [Table 6-52](#).

Return to the [Summary Table](#).

Command Data Register 26

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 95:64 of flash word data register 6.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-42. CMDDATA26

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-52. CMDDATA26 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.40 CMDDATA27 (Offset = 119Ch) [Reset = FFFFFFFFh]

CMDDATA27 is shown in [Figure 6-43](#) and described in [Table 6-53](#).

Return to the [Summary Table](#).

Command Data Register 27

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 127:96 of flash word data register 6.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-43. CMDDATA27

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFFh																															

Table 6-53. CMDDATA27 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFFh = Maximum value of [VAL]

6.6.41 CMDDATA28 (Offset = 11A0h) [Reset = FFFFFFFFh]

CMDDATA28 is shown in [Figure 6-44](#) and described in [Table 6-54](#).

Return to the [Summary Table](#).

Command Data Register 28

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 31:0 of flash word data register 7.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-44. CMDDATA28

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-54. CMDDATA28 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.42 CMDDATA29 (Offset = 11A4h) [Reset = FFFFFFFFh]

CMDDATA29 is shown in [Figure 6-45](#) and described in [Table 6-55](#).

Return to the [Summary Table](#).

Command Data Register 29

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 63:32 of flash word data register 7.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-45. CMDDATA29

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFFh																															

Table 6-55. CMDDATA29 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFFh = Maximum value of [VAL]

6.6.43 CMDDATA30 (Offset = 11A8h) [Reset = FFFFFFFFh]

CMDDATA30 is shown in [Figure 6-46](#) and described in [Table 6-56](#).

Return to the [Summary Table](#).

Command Data Register 30

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 95:64 of flash word data register 7.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-46. CMDDATA30

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-56. CMDDATA30 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.44 CMDDATA31 (Offset = 11ACh) [Reset = FFFFFFFFh]

CMDDATA31 is shown in [Figure 6-47](#) and described in [Table 6-57](#).

Return to the [Summary Table](#).

Command Data Register 31

This register forms the data for a command.

For DATAWIDTH == 128: This register represents bits 127:96 of flash word data register 7.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by the flash wrapper hardware.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all flash wrapper commands.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

Figure 6-47. CMDDATA31

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFFh																															

Table 6-57. CMDDATA31 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFFh	A 32-bit data value is placed in this field. 0h = Minimum value of [VAL] FFFFFFFFh = Maximum value of [VAL]

6.6.45 CMDDATAECC0 (Offset = 11B0h) [Reset = 0000FFFFh]

CMDDATAECC0 is shown in [Figure 6-48](#) and described in [Table 6-58](#).

Return to the [Summary Table](#).

Command Data Register 0

This register forms the ECC portion of the data for a command. This ECC data in this register covers flash data register 0.

The hardware ECC generation can be overridden and ECC data developed elsewhere can be used. ECC data is placed in this register.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all commands.

Figure 6-48. CMDDATAECC0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																VAL1								VAL0							
R/W-0h																R/W-FFh								R/W-FFh							

Table 6-58. CMDDATAECC0 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	Reserved
15-8	VAL1	R/W	FFh	ECC data for bits 127:64 of the data is placed here. 0h = Minimum value FFh = Maximum value
7-0	VAL0	R/W	FFh	ECC data for bits 63:0 of the data is placed here. 0h = Minimum value FFh = Maximum value

6.6.46 CMDDATAECC1 (Offset = 11B4h) [Reset = 0000FFFFh]

CMDDATAECC1 is shown in [Figure 6-49](#) and described in [Table 6-59](#).

Return to the [Summary Table](#).

Command Data Register 1

This register forms the ECC portion of the data for a command. This ECC data in this register covers flash data register 1.

The hardware ECC generation can be overridden and ECC data developed elsewhere can be used. ECC data is placed in this register.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all commands.

Figure 6-49. CMDDATAECC1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																VAL1								VAL0							
R/W-0h																R/W-FFh								R/W-FFh							

Table 6-59. CMDDATAECC1 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	Reserved
15-8	VAL1	R/W	FFh	ECC data for bits 127:64 of the data is placed here. 0h = Minimum value FFh = Maximum value
7-0	VAL0	R/W	FFh	ECC data for bits 63:0 of the data is placed here. 0h = Minimum value FFh = Maximum value

6.6.47 CMDDATAECC2 (Offset = 11B8h) [Reset = 0000FFFFh]

CMDDATAECC2 is shown in [Figure 6-50](#) and described in [Table 6-60](#).

Return to the [Summary Table](#).

Command Data Register 2

This register forms the ECC portion of the data for a command. This ECC data in this register covers flash data register 2.

The hardware ECC generation can be overridden and ECC data developed elsewhere can be used. ECC data is placed in this register.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all commands.

Figure 6-50. CMDDATAECC2

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																VAL1								VAL0							
R/W-0h																R/W-FFh								R/W-FFh							

Table 6-60. CMDDATAECC2 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	Reserved
15-8	VAL1	R/W	FFh	ECC data for bits 127:64 of the data is placed here. 0h = Minimum value FFh = Maximum value
7-0	VAL0	R/W	FFh	ECC data for bits 63:0 of the data is placed here. 0h = Minimum value FFh = Maximum value

6.6.48 CMDDATAECC3 (Offset = 11BCh) [Reset = 0000FFFFh]

CMDDATAECC3 is shown in [Figure 6-51](#) and described in [Table 6-61](#).

Return to the [Summary Table](#).

Command Data Register 3

This register forms the ECC portion of the data for a command. This ECC data in this register covers flash data register 3.

The hardware ECC generation can be overridden and ECC data developed elsewhere can be used. ECC data is placed in this register.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all commands.

Figure 6-51. CMDDATAECC3

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																VAL1								VAL0							
R/W-0h																R/W-FFh								R/W-FFh							

Table 6-61. CMDDATAECC3 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	Reserved
15-8	VAL1	R/W	FFh	ECC data for bits 127:64 of the data is placed here. 0h = Minimum value FFh = Maximum value
7-0	VAL0	R/W	FFh	ECC data for bits 63:0 of the data is placed here. 0h = Minimum value FFh = Maximum value

6.6.49 CMDDATAECC4 (Offset = 11C0h) [Reset = 0000FFFFh]

CMDDATAECC4 is shown in [Figure 6-52](#) and described in [Table 6-62](#).

Return to the [Summary Table](#).

Command Data Register 4

This register forms the ECC portion of the data for a command. This ECC data in this register covers flash data register 4.

The hardware ECC generation can be overridden and ECC data developed elsewhere can be used. ECC data is placed in this register.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all commands.

Figure 6-52. CMDDATAECC4

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																VAL1								VAL0							
R/W-0h																R/W-FFh								R/W-FFh							

Table 6-62. CMDDATAECC4 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	Reserved
15-8	VAL1	R/W	FFh	ECC data for bits 127:64 of the data is placed here. 0h = Minimum value FFh = Maximum value
7-0	VAL0	R/W	FFh	ECC data for bits 63:0 of the data is placed here. 0h = Minimum value FFh = Maximum value

6.6.50 CMDDATAECC5 (Offset = 11C4h) [Reset = 0000FFFFh]

CMDDATAECC5 is shown in [Figure 6-53](#) and described in [Table 6-63](#).

Return to the [Summary Table](#).

Command Data Register 5

This register forms the ECC portion of the data for a command. This ECC data in this register covers flash data register 5.

The hardware ECC generation can be overridden and ECC data developed elsewhere can be used. ECC data is placed in this register.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all commands.

Figure 6-53. CMDDATAECC5

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																VAL1								VAL0							
R/W-0h																R/W-FFh								R/W-FFh							

Table 6-63. CMDDATAECC5 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	Reserved
15-8	VAL1	R/W	FFh	ECC data for bits 127:64 of the data is placed here. 0h = Minimum value FFh = Maximum value
7-0	VAL0	R/W	FFh	ECC data for bits 63:0 of the data is placed here. 0h = Minimum value FFh = Maximum value

6.6.51 CMDDATAECC6 (Offset = 11C8h) [Reset = 0000FFFFh]

CMDDATAECC6 is shown in [Figure 6-54](#) and described in [Table 6-64](#).

Return to the [Summary Table](#).

Command Data Register 6

This register forms the ECC portion of the data for a command. This ECC data in this register covers flash data register 6.

The hardware ECC generation can be overridden and ECC data developed elsewhere can be used. ECC data is placed in this register.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all commands.

Figure 6-54. CMDDATAECC6

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																VAL1								VAL0							
R/W-0h																R/W-FFh								R/W-FFh							

Table 6-64. CMDDATAECC6 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	Reserved
15-8	VAL1	R/W	FFh	ECC data for bits 127:64 of the data is placed here. 0h = Minimum value FFh = Maximum value
7-0	VAL0	R/W	FFh	ECC data for bits 63:0 of the data is placed here. 0h = Minimum value FFh = Maximum value

6.6.52 CMDDATAECC7 (Offset = 11CCh) [Reset = 0000FFFFh]

CMDDATAECC7 is shown in [Figure 6-55](#) and described in [Table 6-65](#).

Return to the [Summary Table](#).

Command Data Register 7

This register forms the ECC portion of the data for a command. This ECC data in this register covers flash data register 7.

The hardware ECC generation can be overridden and ECC data developed elsewhere can be used. ECC data is placed in this register.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

Use cases for the CMDDATA* registers are as follows:

- 1) Program - These registers contain the data to be programmed.
- 2) Erase - These registers are not used.
- 3) Read Verify - These registers contain data to be verified.

This register is used to aggregate masking for bits that do not require additional program pulses during program operations, and will be written to all 1 after the completion of all commands.

Figure 6-55. CMDDATAECC7

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																VAL1								VAL0							
R/W-0h																R/W-FFh								R/W-FFh							

Table 6-65. CMDDATAECC7 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	Reserved
15-8	VAL1	R/W	FFh	ECC data for bits 127:64 of the data is placed here. 0h = Minimum value FFh = Maximum value
7-0	VAL0	R/W	FFh	ECC data for bits 63:0 of the data is placed here. 0h = Minimum value FFh = Maximum value

6.6.53 CMDWEPROTA (Offset = 11D0h) [Reset = FFFFFFFFh]

CMDWEPROTA is shown in [Figure 6-56](#) and described in [Table 6-66](#).

Return to the [Summary Table](#).

Command WriteErase Protect A Register

This register allows the first 32 sectors of the main region to be protected from program or erase, with 1 bit protecting each sector. If the main region size is smaller than 32 sectors, then this register provides protection for the whole region.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

In addition, this register is used to aggregate masking for sectors that do not require additional erase pulses during bank erase operations, and will be written to all 1 after the completion of all commands.

Figure 6-56. CMDWEPROTA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-66. CMDWEPROTA Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	Each bit protects 1 sector. bit [0]: When 1, sector 0 of the flash memory will be protected from program and erase. bit [1]: When 1, sector 1 of the flash memory will be protected from program and erase. :: bit [31]: When 1, sector 31 of the flash memory will be protected from program and erase. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.54 CMDWEPROTB (Offset = 11D4h) [Reset = FFFFFFFFh]

CMDWEPROTB is shown in [Figure 6-57](#) and described in [Table 6-67](#).

Return to the [Summary Table](#).

Command WriteErase Protect B Register

This register allows main region sectors to be protected from program and erase. Each bit corresponds to a group of 8 sectors. There are 3 cases for how these protect bits are applied:

1. Single-bank system:

In the case where only a single flash bank is present, the first 32 sectors are protected via the CMDWEPROTA register. Thus, the protection given by the bits in CMDWEPROTB begin with sector 32.

2. Multi-bank system, Bank 0:

When multiple flash banks are present, the first 32 sectors of bank 0 are protected via the CMDWEPROTA register. Thus, only bits 4 and above of CMDWEPROTB would be applicable to bank 0. The protection of bit 4 and above would begin at sector 32. Bits 3:0 of WEPROTB are ignored for bank 0.

3. Multi-bank system, Banks 1-N:

For banks other than bank 0 in a multi-bank system, CMDWEPROTA has no effect, so the bits in CMDWEPROTB will protect these banks starting from sector 0.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

In addition, this register is used to aggregate masking for sectors that do not require additional erase pulses during bank erase operations, and will be written to all 1 after the completion of all commands.

Figure 6-57. CMDWEPROTB

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-67. CMDWEPROTB Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	Each bit protects a group of 8 sectors. When a bit is 1, the associated 8 sectors in the flash will be protected from program and erase. A maximum of 256 sectors can be protected with this register. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.55 CMDWEPROTC (Offset = 11D8h) [Reset = FFFFFFFFh]

CMDWEPROTC is shown in [Figure 6-58](#) and described in [Table 6-68](#).

Return to the [Summary Table](#).

Command WriteErase Protect C Register

This register allows main region sectors to be protected from program and erase. Each bit corresponds to a group of 8 sectors.

This register extends the protection bits from the CMDWEPROTB register to cover bank sizes larger than $32 \times 8 = 256$ sectors. This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

In addition, this register is used to aggregate masking for sectors that do not require additional erase pulses during bank erase operations, and will be written to all 1 after the completion of all commands.

Figure 6-58. CMDWEPROTC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFFh																															

Table 6-68. CMDWEPROTC Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFFh	Each bit protects a group of 8 sectors. When a bit is 1, the associated 8 sectors in the flash will be protected from program and erase. Note that the sectors protected with this register start at sector 256 in the flash, where the sectors protected by the CMDWEPROTB register end. 0h = Minimum value of [VAL] FFFFFFFFh = Maximum value of [VAL]

6.6.56 CMDWEPROTNM (Offset = 1210h) [Reset = FFFFFFFFh]

CMDWEPROTNM is shown in [Figure 6-59](#) and described in [Table 6-69](#).

Return to the [Summary Table](#).

Command WriteErase Protect Non-Main Register

This register allows non-main region sectors to be protected from program and erase. Each bit corresponds to 1 sector.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

In addition, this register is used to aggregate masking for sectors that do not require additional erase pulses during bank erase operations, and will be written to all 1 after the completion of all commands.

Figure 6-59. CMDWEPROTNM

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VAL																															
R/W-FFFFFFFh																															

Table 6-69. CMDWEPROTNM Field Descriptions

Bit	Field	Type	Reset	Description
31-0	VAL	R/W	FFFFFFFh	Each bit protects 1 sector. bit [0]: When 1, sector 0 of the non-main region will be protected from program and erase. bit [1]: When 1, sector 1 of the non-main region will be protected from program and erase. . . : bit [31]: When 1, sector 31 of the non-main will be protected from program and erase. 0h = Minimum value of [VAL] FFFFFFFh = Maximum value of [VAL]

6.6.57 CFGPCNT (Offset = 13B4h) [Reset = 00000000h]

CFGPCNT is shown in [Figure 6-60](#) and described in [Table 6-70](#).

Return to the [Summary Table](#).

Pulse Counter Configuration Register

This register allows further configuration of maximum pulse counts for program and erase operations.

This register is blocked for writes after a 1 is written to the CMDEXEC register and prior to STATCMD.DONE being set by hardware.

Figure 6-60. CFGPCNT

31	30	29	28	27	26	25	24
MAXERSPCNTVAL							
R/W-0h							
23	22	21	20	19	18	17	16
MAXERSPCNTVAL				RESERVED		MAXERSPCNT OVR	
R/W-0h				R/W-0h		R/W-0h	
15	14	13	12	11	10	9	8
RESERVED				MAXPCNTVAL			
R/W-0h				R/W-0h			
7	6	5	4	3	2	1	0
MAXPCNTVAL				RESERVED		MAXPCNTOVR	
R/W-0h				R/W-0h		R/W-0h	

Table 6-70. CFGPCNT Field Descriptions

Bit	Field	Type	Reset	Description
31-20	MAXERSPCNTVAL	R/W	0h	Override maximum pulse count for erase with this value. If MAXERSPCNTOVR = 0, then this field is ignored. If MAXERSPCNTOVR = 1, then this value will be used to override the max pulse count for erase. 0h = Minimum value FFFh = Maximum value
19-17	RESERVED	R/W	0h	Reserved
16	MAXERSPCNTOVR	R/W	0h	Override hard-wired maximum pulse count for erase. If set, then the value in MAXERSPCNTVAL will be used as the max pulse count for erase operations. By default, this bit is 0, and a hard-wired max pulse count is used. 0h = Use hard-wired (default) value for maximum pulse count 1h = Use value from MAXERSPCNTVAL field as maximum erase pulse count
15-12	RESERVED	R/W	0h	Reserved
11-4	MAXPCNTVAL	R/W	0h	Override maximum pulse counter with this value. If MAXPCNTOVR = 0, then this field is ignored. If MAXPCNTOVR = 1 and MAXERSPCNTOVR = 0, then this value will be used to override the max pulse count for both program and erase. Full max value will be {4'h0, MAXPCNTVAL}. If MAXPCNTOVR = 1 and MAXERSPCNTOVR = 1, then this value will be used to override the max pulse count for program only. Full max value will be {4'h0, MAXPCNTVAL}. 0h = Minimum value FFh = Maximum value
3-1	RESERVED	R/W	0h	Reserved

Table 6-70. CFGPCNT Field Descriptions (continued)

Bit	Field	Type	Reset	Description
0	MAXPCNTOVR	R/W	0h	Override hard-wired maximum pulse count. If MAXERSPCNTOVR is not set, then setting this value alone will override the max pulse count for both program and erase. If MAXERSPCNTOVR is set, then this bit will only control the max pulse count setting for program. By default, this bit is 0, and a hard-wired max pulse count is used. 0h = Use hard-wired (default) value for maximum pulse count 1h = Use value from MAXPCNTVAL field as maximum pulse count

6.6.58 STATCMD (Offset = 13D0h) [Reset = 00000000h]

STATCMD is shown in [Figure 6-61](#) and described in [Table 6-71](#).

Return to the [Summary Table](#).

Command Status Register This register contains status regarding completion and errors of command execution.

Figure 6-61. STATCMD

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED			FAILMISC	RESERVED			RESERVED
R-0h			R-0h	R-0h			R-
7	6	5	4	3	2	1	0
FAILMODE	FAILILLADDR	FAILVERIFY	FAILWEPROT	RESERVED	CMDINPROGR ESS	CMDPASS	CMDDONE
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 6-71. STATCMD Field Descriptions

Bit	Field	Type	Reset	Description
31-13	RESERVED	R	0h	Reserved
12	FAILMISC	R	0h	Command failed due to error other than write/erase protect violation or verify error. This is an extra bit in case a new failure mechanism is added which requires a status bit. 0h = No Fail 1h = Fail
11-9	RESERVED	R	0h	Reserved
8	RESERVED	R	0h	
7	FAILMODE	R	0h	Command failed because a bank has been set to a mode other than READ. Program and Erase commands cannot be initiated unless all banks are in READ mode. 0h = No Fail 1h = Fail
6	FAILILLADDR	R	0h	Command failed due to the use of an illegal address 0h = No Fail 1h = Fail
5	FAILVERIFY	R	0h	Command failed due to verify error 0h = No Fail 1h = Fail
4	FAILWEPROT	R	0h	Command failed due to Write/Erase Protect Sector Violation 0h = No Fail 1h = Fail
3	RESERVED	R	0h	Reserved
2	CMDINPROGRESS	R	0h	Command In Progress 0h = Complete 1h = In Progress
1	CMDPASS	R	0h	Command Pass - valid when CMD_DONE field is 1 0h = Fail 1h = Pass

Table 6-71. STATCMD Field Descriptions (continued)

Bit	Field	Type	Reset	Description
0	CMDDONE	R	0h	Command Done 0h = Not Done 1h = Done

6.6.59 STATADDR (Offset = 13D4h) [Reset = 00010000h]

STATADDR is shown in [Figure 6-62](#) and described in [Table 6-72](#).

Return to the [Summary Table](#).

Current Address Counter Value Read only register giving read access to the state machine current address. A bank id, region id and address are stored in this register and are incremented as necessary during execution of a command.

Figure 6-62. STATADDR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED						BANKID						REGIONID			
R-0h						R-0h						R-1h			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BANKADDR															
R-0h															

Table 6-72. STATADDR Field Descriptions

Bit	Field	Type	Reset	Description
31-26	RESERVED	R	0h	Reserved
25-21	BANKID	R	0h	Current Bank ID A bank indicator is stored in this register which represents the current bank on which the state machine is operating. There is 1 bit per bank. 1h (R/W) = Bank 0 2h (R/W) = Bank 1 4h (R/W) = Bank 2 8h (R/W) = Bank 3 10h (R/W) = Bank 4
20-16	REGIONID	R	1h	Current Region ID A region indicator is stored in this register which represents the current flash region on which the state machine is operating. 1h (R/W) = Main Region 2h (R/W) = Non-Main Region 4h (R/W) = Trim Region 8h (R/W) = Engr Region
15-0	BANKADDR	R	0h	Current Bank Address A bank offset address is stored in this register. 0h = Minimum value FFFFh = Maximum value

6.6.60 STATPCNT (Offset = 13D8h) [Reset = 00000000h]

STATPCNT is shown in [Figure 6-63](#) and described in [Table 6-73](#).

Return to the [Summary Table](#).

Current Pulse Count Register: Read only register giving read access to the state machine current pulse count value for program/erase operations.

Figure 6-63. STATPCNT

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0						
RESERVED																				PULSECNT																	
R-0h																				R-0h																	

Table 6-73. STATPCNT Field Descriptions

Bit	Field	Type	Reset	Description
31-12	RESERVED	R	0h	Reserved
11-0	PULSECNT	R	0h	Current Pulse Counter Value 0h = Minimum value FFFh = Maximum value



The event manager provides the peripheral-to-peripheral, peripheral-to-DMA, and peripheral-to-CPU (IRQ) event connections.

7.1 Events Overview.....	438
7.2 Events Operation.....	442

7.1 Events Overview

The event manager transfers digital events from one entity (for example, a peripheral) to another (for example, a second peripheral, the DMA, or the CPU). The event manager implements event transfer through a defined set of event publishers (generators) and subscribers (receivers) which are interconnected through an event fabric containing a combination of fixed (static) and programmable routes.

Events which are transferred by the event manager include:

- Peripheral event transferred to the CPU as an interrupt request (IRQ)
 - Example: RTC interrupt is sent to the CPU
- Peripheral event transferred to the DMA as a DMA trigger
 - Example: UART data receive trigger to DMA to request a DMA transfer
- Peripheral event transferred to another peripheral to directly trigger an action in hardware
 - Example: TIMx timer peripheral publishes a periodic event to the ADC subscriber port, and the ADC uses the event to trigger start-of-sampling

In addition to providing the event transfer logic, the event manager also interfaces with the power management and clock unit (PMCU) if an event requires the power and/or clock configuration of the device to change to handle the event properly. For example, if a peripheral asserts an event that targets the DMA, and the device is in a STOP or STANDBY operating mode (DMA is disabled), the event manager will handshake with the PMCU to [suspend the low power operating mode state temporarily](#) and enable the DMA such that the DMA transfer can be processed.

The event manager configuration is device dependent, as different devices support different peripherals. See the device-specific data sheet for information on the device-specific event implementation.

7.1.1 Event Publisher

An event publisher is the source of an event which is propagated on the event fabric. Peripherals contain event publishers for publishing CPU interrupts, DMA triggers and generic events to the event fabric through the publishing port FPUB_x. Publisher behavior is configured with standardized [event management registers](#).

7.1.2 Event Subscriber

Event subscribers are included within the processor, the DMA, and certain peripherals (see [Section 7.1.4](#)). Event subscribers subscribe the events through the subscribing port FSUB_x. Event subscribers enable modules to be able to subscribe to, and take a predefined action upon, events which are published to the event fabric by an [event publisher](#).

7.1.3 Event Fabric Routing

There are three different types of routes through the event fabric which are used to connect a publisher to a subscriber: [CPU interrupt events](#), [DMA trigger events](#), and [generic events](#).

7.1.3.1 CPU Interrupt Event Route (CPU_INT)

A CPU interrupt event route is a fixed, point-to-point connection between one event publisher (inside a peripheral module) and one event subscriber (the CPU subsystem) used to propagate CPU interrupts.

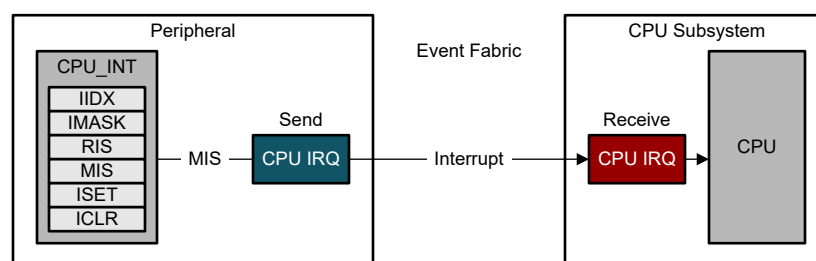


Figure 7-1. CPU Interrupt (Fixed Event Route)

For each peripheral which is capable of generating a CPU interrupt, a fixed route is provided from the peripheral's masked interrupt status (MIS) register to the [CPU subsystem's interrupt management logic](#).

If software does not clear the interrupt request in the peripheral's [event management registers](#), the request will remain pending to the CPU subsystem. See [Section 7.2.5.3](#) for guidance on setting and clearing interrupt status with the event management registers.

7.1.3.2 DMA Trigger Event Route (DMA_TRIGx)

A DMA route is a fixed route between a peripheral and the DMA controller, which optionally has additional side-band signals to pass a DMA done condition from the DMA controller back to the triggering peripheral to indicate when a DMA activity has run to completion. The DMA trigger route is shown in [Figure 7-2](#).

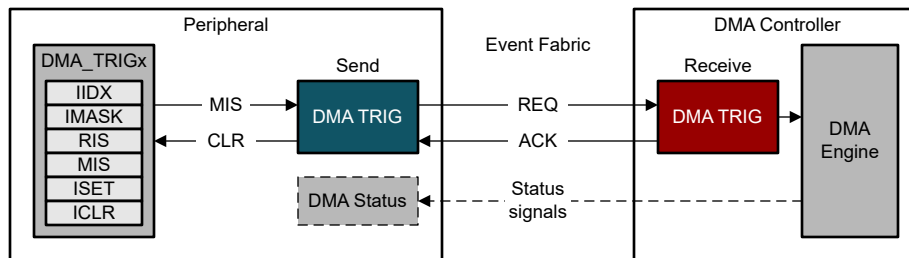


Figure 7-2. DMA Route

Most peripherals capable of generating a DMA trigger have an additional set of [event management registers](#) (in addition to the CPU_INT registers used for the CPU interrupt and any GEN_EVENTx generic route publishers). These registers can be used to select which peripheral condition to use for generating the DMA trigger.

When a trigger is received by the DMA, the DMA acknowledges the request and the peripheral clears the request. The DMA also acknowledges the cleared request, after which a new request can be asserted by the peripheral.

The DMA route can also contain status signals (for specific peripherals) to indicate to the triggering peripheral that a DMA transfer sequence has completed. For example, the DMA can be set up to transfer *N* number of bytes from an SRAM buffer into the UART TX data register based on the UART TX DMA trigger. Upon each trigger from the UART, the DMA will acknowledge that the transfer was successful. On the *N*th byte, the DMA will send a complete status signal to the UART, which the UART can use to propagate a transfer completion interrupt to the CPU.

Special Cases

Certain peripherals (for example, the 12-bit DAC) do not implement an event management register set for managing their DMA triggers. In these cases, the peripheral implements specific DMA configuration logic such that the management registers are not needed to interface with the DMA. [Figure 7-3](#) shows the model when the event management registers are not implemented. See the peripheral-specific section of this document for guidance on how to configure DMA channels in this case.

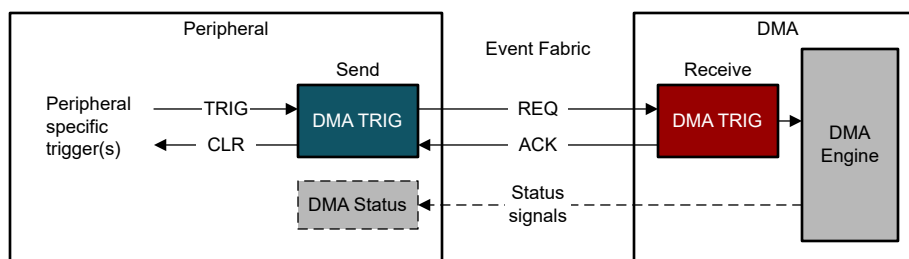


Figure 7-3. DMA Route without Event Management Registers

7.1.3.3 Generic Event Route (GEN_EVENTx)

A generic route is either a point-to-point (1:1) route or a point-to-two (1:2) splitter route in which the peripheral publishing the event uses one of several available generic route channels to publish its event to another entity (or entities, in the case of a splitter route), where an entity can be another peripheral, a generic DMA trigger event, or a generic CPU event, as shown in Figure 7-4.

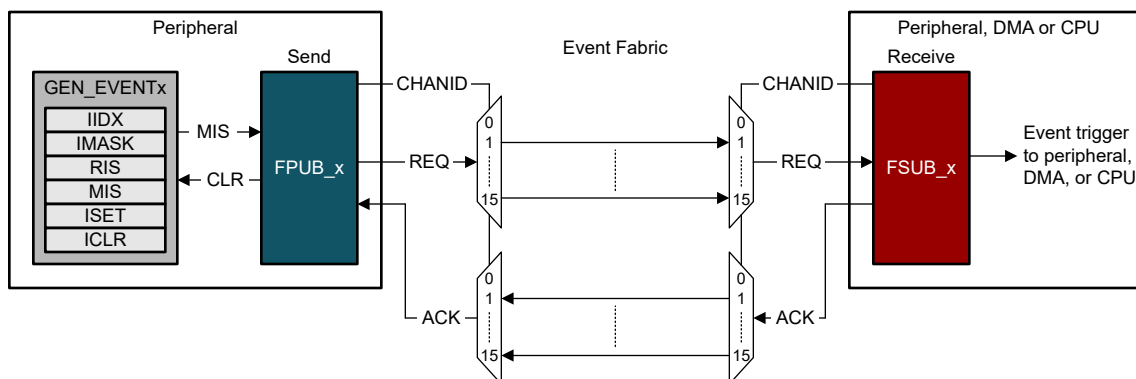


Figure 7-4. Generic Route

Peripherals capable of generating a generic event have an additional group (our groups) of GEN_EVENTx event management registers (in addition to the CPU_INT registers used for the CPU interrupt or DMA_TRIGx for DMA, if present). These registers can be used to select the peripheral condition to use for publishing a generic event. When configured, the event will broadcast out to the generic route channel selected by the FPUB_x register. A second peripheral, the DMA, or the CPU can subscribe to this event by configuring its generic subscriber port (FSUB_x) to listen on the same generic route channel to which the publishing peripheral is connected to.

Generic route channels can be configured with one subscriber (1:1 route) or two subscribers (1:2 splitter route), depending on which channel is selected. See the device data sheet for a complete listing of the available generic route channels and their type (1:1 or 1:2). Generic route channels can only be configured with one publishing peripheral at a time. Once a peripheral subscribes to a 1:1 generic route channel, no other peripheral will be able to select that channel to subscribe to, unless the originally connected peripheral is disconnected first. Generic route channels with splitter capability (1:2) enable exactly two peripherals to subscribe to the channel, after which additional attempts to add subscribers will be blocked by hardware until both of the two connected peripherals are disconnected from the splitter channel.

Each peripheral type has unique capabilities in terms of what can generate an event to publish, and what a subscribed event is capable of triggering within the peripheral. Review the chapter of this guide which corresponds to the peripheral of interest to understand what the publisher and subscriber ports on a given peripheral are capable of.

7.1.4 Event Routing Map

The event capabilities of each peripheral type are shown in Figure 7-5. Peripherals such as UART, SPI, and I2C generate CPU interrupt events which are routed to the CPU, and they also generate DMA trigger events which are routed to the DMA. Peripherals such as GPIO and ADC generate CPU interrupt events as well, but they also support generating and receiving events routed through a generic channel. For example, through the use of a generic event channel, it is possible to directly start an ADC conversion from a GPIO event by connecting a GPIO FPUB_x and ADC FSUB_0 to the same generic event channel.

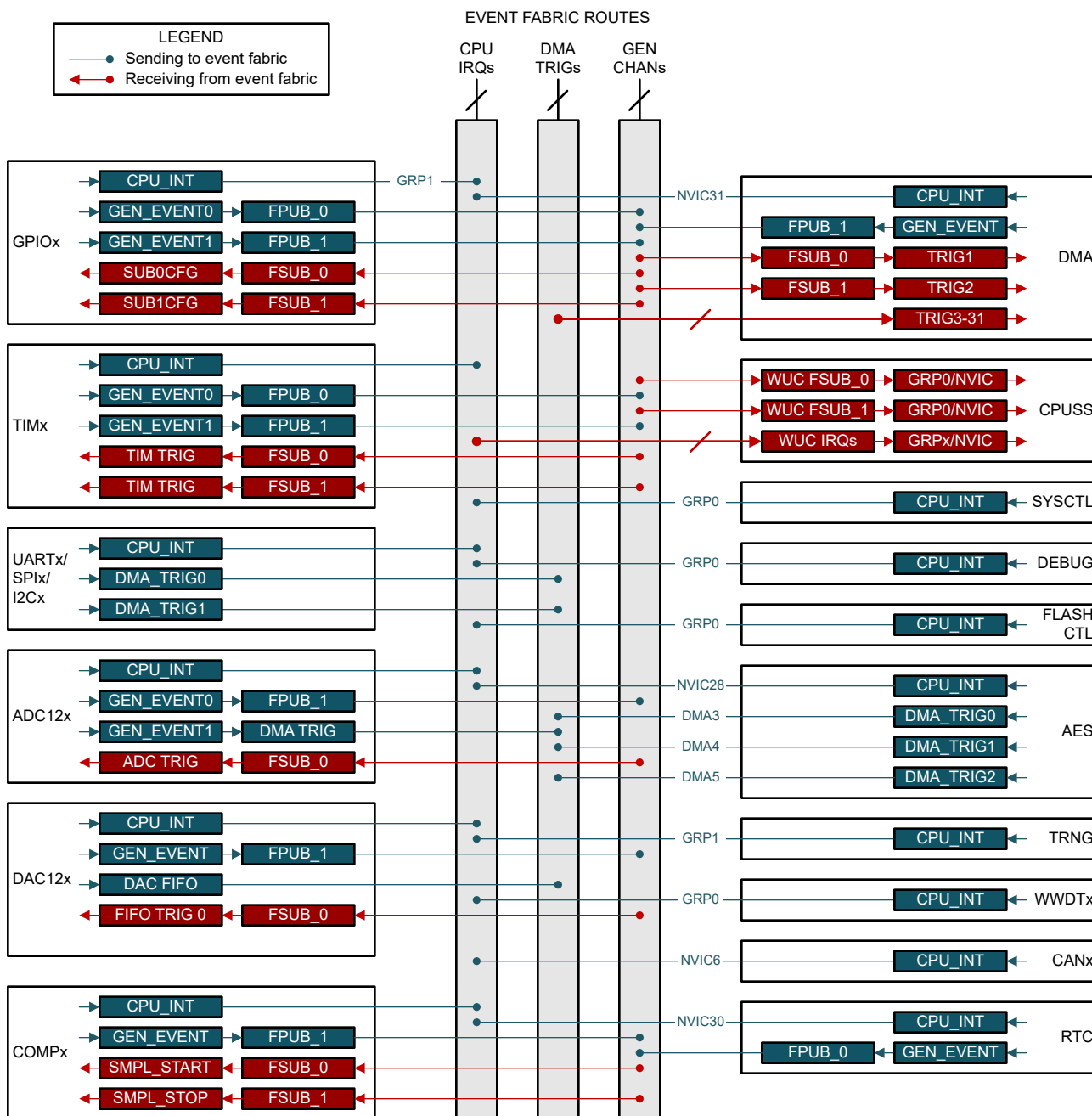


Figure 7-5. Event Map

7.1.5 Event Propagation Latency

Generic route channels implement a four-way hardware handshake between the publishing entity and the subscribing entity. This handshake requires four **ULPCLK** cycles to complete:

1. Request from publisher to subscriber
2. Acknowledge from subscriber to publisher
3. De-assert of request from publisher to subscriber
4. Acknowledge of de-assert from subscriber to publisher

If the publishing peripheral sends two requests and the first request has not cleared the handshake, the second request is dropped.

7.2 Events Operation

This section describes how to configure peripherals to use the event manager. Note that the event manager itself does not contain any configuration registers. All event configuration is done through the publishing and subscribing peripherals.

7.2.1 CPU Interrupt

Peripheral interrupt requests (IRQs) are propagated to the CPU subsystem through the event manager. Peripheral interrupt requests use fixed routes, but in addition the CPU subsystem may provide two generic event subscriber ports which can be used to trigger CPU interrupts through a generic route. See the device-specific data sheet for the complete list of interrupt assignments for a given device.

Standard CPU Interrupt Requests

No special configuration is required for fixed route interrupts. Interrupts can be managed through the peripheral's CPU_INT event management registers (IIDX, IMASK, RIS, MIS, ISET, and ICLR) and through the CPU subsystem interrupt configuration (see [Section 3.3](#)).

Generic Event Based CPU Interrupt Requests

The CPU subsystem contains two generic event subscriber ports (FSUB_x) which can be used to source a CPU interrupt from any of the device's generic event channels. This can be used to enable special cases where a particular function on a peripheral generates a dedicated interrupt to the CPU subsystem which is independent from, and in addition to, that peripheral's standard interrupt mechanism.

Consider the GPIO peripheral, which has a standard interrupt request as well as 2 publishers which can route a GPIO event to any of the generic event channels based on a defined state in the GPIO. For example, it can be desirable to have most GPIO events configured to source the standard interrupt, while a single specific GPIO event sources a second dedicated CPU interrupt through a generic route. This enables the application software to have two completely independent interrupt handlers for the GPIO.

To configure the event manager to trigger a CPU interrupt from a generic route, follow the steps below:

1. Configure the GEN_EVENT registers of the peripheral generating the event to select the desired peripheral state as an event generator.
2. Configure the FPUB_x register of the peripheral generating the event with the generic route channel ID which is to be used. This channel must not be in use by another peripheral.
3. Configure the FSUB_x register of the wake up controller (WUC), which captures generic route channel events to forward to the CPU subsystem.
4. Configure the [CPU subsystem interrupt management to enable the GENSUBx interrupt](#).

Note that when generating a CPU interrupt through a generic route, the generic event logic will automatically clear the pending interrupt request as a part of the four-way event handshake. Application software will not be able to read the cause of the interrupt from the peripheral registers, and it does not need to clear any interrupt status bits. Software can only read that the FSUB_x generic event generated an interrupt. This reduces the interrupt overhead.

7.2.2 DMA Trigger

DMA triggers are propagated to the DMA through the event manager. Most DMA triggers use fixed routes, but the DMA does provide two generic event subscriber ports which can be used to trigger DMA transfers through a generic route channel. See the device-specific data sheet for the complete list of DMA trigger assignments for a given device.

Standard DMA Triggers

To determine if a particular peripheral on a device provides a fixed DMA trigger (DMA_TRIGx) from the peripheral directly to the DMA, review the DMA triggers table in the detailed description section of the device-

specific data sheet. Certain peripherals can have more than one DMA trigger (for example, to enable a TX trigger and an RX trigger on a serial communication peripheral).

To select the specific peripheral event which triggers a static DMA route, configure the peripheral's DMA_TRIGx event management register set (IIDX, IMASK, RIS, MIS, ISET, and ICLR) which corresponds to the targeted DMA route. To determine which DMA_TRIGx register set corresponds with which DMA trigger, review the relevant chapter of this guide for the corresponding peripheral, or review [Section 7.1.4](#).

Certain peripherals (such as the 12-bit DAC) do not implement a DMA_TRIGx register set for managing DMA triggers. In these cases, the DMA trigger configuration is done through peripheral-specific configuration registers.

Generic Event Based DMA Triggers

The DMA contains two generic event subscriber ports (FSUB_x) which can be used to source a DMA trigger from any of the device's generic event channels. This can be used to enable special cases where a particular function on a peripheral generates a DMA trigger. For example, it can be desirable to trigger a DMA transfer from a timer.

To configure the event manager to trigger a DMA channel from a generic route, follow the steps below:

1. Configure the GEN_EVENTx registers of the peripheral generating the event to select the desired peripheral state as an event generator.
2. Configure the FPUB_x register of the peripheral generating the event with the generic event channel ID which is to be used. This channel must not be in use by another peripheral.
3. Configure the FSUB_x register of the DMA, which captures generic event channel events to be used as triggers in the DMA.
4. Configure the DMA according to the configuration instructions in the DMA chapter.

7.2.3 Peripheral to Peripheral Event

Peripheral to peripheral events enable a condition in one peripheral to trigger an action in a second (or third) peripheral, completely in hardware without any CPU interaction. The device provides a certain number of generic route channels which can be either published to or subscribed to by peripherals which include publisher and subscriber ports. Before establishing a configuration, follow these steps:

1. Review the device specific data sheet to determine the generic route channel count and channel type available on the target device. Select an appropriate channel type (point to point or splitter) based on the desired functionality, and determine the channel number to use for the connection (the channel must not already be used by other peripherals).
2. Review the publisher and subscriber capabilities of the peripherals which are to be connected. Some peripherals have more than one publisher and/or more than one subscriber port, and some peripherals have no publisher or subscriber ports. To understand the available ports for a peripheral, review the peripheral's reference chapter in this guide, or check the generic event channel connections in [Section 7.1.4](#).

Once the channel to be used is determined, and both the publisher and subscriber ports for the peripherals being connected are known, use the steps below to establish the event connection. In this example, a timer triggered ADC application will be configured, using TIMG0 to publish an event to generic channel 1, with ADC0 subscribing to generic channel 1 as a start-of-conversion trigger.

1. Configure the GEN_EVENTx event management registers of TIMG0 to set the event request based on the appropriate timer event (for example, a zero event).
2. Store 0x1 into the FPUB_0 register of TIMG0 to publish the TIMG0 event selected by the GEN_EVENTx registers to generic route channel 1. Channel 1 must not be in use by another peripheral.
3. Store 0x1 the FSUB_0 register of ADC0 so that ADC0 is listening for events published by the timer to channel 1.
4. Configure ADC0 to trigger from the subscriber port according to the configuration instructions in [Section 10.2.8](#).
5. Configure and enable TIMG0.

7.2.4 Extended Module Description Register

The DESC_EX register is a read-only register in the event manager which can be read by application software to determine how many point to point (single) generic route channels and splitter (dual) generic route channels are available on a given device.

7.2.5 Using Event Registers

The event management register group is a set of standard registers which are implemented by all peripherals capable of generating events (CPU interrupts, DMA triggers, or generic events). Each event generator in a peripheral contains its own event management register set. For example, if a peripheral supports generating a CPU interrupt and a DMA trigger, it will have an event management register set for the CPU interrupt (with the group name of CPU_INT) as well as a second event management register set for the DMA trigger (with the group name of DMA_TRIG).

The event management registers are used to:

- Configure which peripheral conditions are used to generate the event (masking)
- Communicate raw and masked peripheral event status
- Set or clear peripheral event status by software

In the "Registers" section for a given peripheral, the "Group" column displays the Group Name to indicate what functionality is mapped to each event management register group. See [Table 7-1](#) for which event management groups are mapped to specific functions in the group name for a peripheral's "Registers" section.

Table 7-1. Event Management Group Functionality and Mapping

Group Name (in Registers)	Functionality
CPU_INT	CPU interrupt (fixed route to the CPU subsystem)
DMA_TRIGx	DMA trigger (fixed route to the DMA controller)
GEN_EVENT	Generic event (programmable route for other module-to-module connections)

7.2.5.1 Event Registers

The event management register set contains 6 standard registers: RIS, IMASK, MIS, ISET, ICLR, and IIDX, given in [Table 7-2](#). The event registers are interconnected as shown in [Figure 7-6](#).

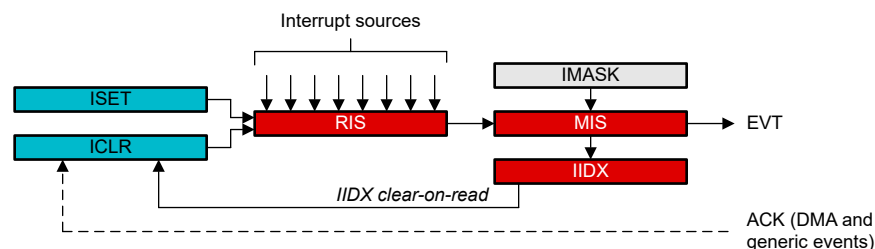


Figure 7-6. Event Management Register Relationship

The peripheral generating the event will contain one or more interrupt source signals which connect to the raw interrupt status (RIS) register. Software can poll RIS at any time to check the raw interrupt status. Software can also clear pending interrupts in the RIS register by writing to the corresponding bit position in the ICLR register. The RIS and IMASK registers are combined through a bit-wise AND function in the MIS register (masked interrupt status). To unmask an interrupt, set the corresponding bit in the IMASK register. Once unmasked, a pending interrupt will be indicated in both the RIS and MIS registers, and an event will be generated. The IIDX register will also be updated with the index of the highest priority pending interrupt.

In the case of a CPU interrupt (CPU_INT) with a [CPU interrupt event route](#), a read of the IIDX register will clear the highest priority pending interrupt in the RIS and MIS registers and return the index of the highest priority pending interrupt to application software.

In the case of a hardware event [DMA trigger route](#) (DMA_TRIGx) or [generic event route](#) (GEN_EVENTx), the hardware four-way handshake will send an ACK signal to the ICLR mechanism which will clear the pending interrupt in the RIS and MIS registers.

Table 7-2. Standardized Event Management Registers (Used for CPU_INT, DMA_TRIGx, GEN_EVENTx Configuration)

Register	Description	R/W	Functionality
RIS	Raw interrupt status	R	Indicates the current pending interrupt status, with one bit provided per interrupt condition. Writing to ICLR will clear the corresponding bit in the RIS register if the interrupt condition is no longer present.
IMASK	Interrupt mask	RW	Used by application software to configure which interrupt conditions propagate into an event, with one bit provided per interrupt condition.
MIS	Masked interrupt status	R	Indicates the current pending masked interrupt status to software and hardware, with one bit provided per interrupt condition. MIS is the bit-wise AND of the RIS and IMASK registers. Writing to ICLR will clear the corresponding bit in the RIS register if the interrupt condition is no longer present. If RIS is cleared, the corresponding bit in the MIS register is also automatically cleared.
ISSET	Software interrupt set control	W	Used by application software to force an interrupt condition for diagnostics. Writing to ISSET will set the corresponding bit in the RIS register. If the interrupt condition is enabled in IMASK, the corresponding bit in the MIS register is also set. Writing a '1' to a bit location in ISSET sets the respective interrupt status.
ICLR	Software interrupt clear control	W	Used by application software to clear a pending interrupt status in RIS. Writing a '1' to a bit location in ICLR clears the respective interrupt status. If an interrupt is enabled in IMASK, the corresponding bit location in MIS is also cleared automatically when RIS clears. If the interrupt condition is still present, clearing the status has no effect and the RIS will remain set.
IIDX	Pending interrupt index	R	Used by application software to read the highest priority pending interrupt while simultaneously clearing the highest priority interrupt status in RIS and MIS. A read of IIDX returns 0 if no unmasked interrupts are pending (MIS==0), else it returns an index value indicating the highest priority pending interrupt.

7.2.5.2 Configuring Events

To configure which peripheral interrupt source is to be used to trigger an event, set the bit which corresponds to the desired interrupt source in the IMASK register which corresponds to the desired event. Setting a bit in IMASK will cause the raw interrupt status in the RIS register to propagate to the MIS register. When an interrupt status bit in the MIS register is set (due to the interrupt being unmasked in the IMASK register and a raw interrupt being pending in the RIS register), an event is generated.

Multiple interrupt sources can be enabled for CPU interrupt events, as application software can determine the cause of the interrupt by reading the IIDX or MIS register.

For hardware events such as DMA triggers and generic event publishers, only one interrupt source should be unmasked in IMASK.

7.2.5.3 Responding to CPU Interrupts in Application Software

In the case of an event which generates a CPU interrupt, application software can determine which peripheral interrupt triggered the generation of the event by either reading the IIDX register or by reading the MIS and writing the ICLR registers.

CPU IRQ Interrupt Service Routine using CPU_INT IIDX Register

Application software can read the IIDX register in CPU_INT group to determine and clear the highest priority pending interrupt. A read to IIDX will return an index corresponding to the highest priority interrupt which was both set and unmasked. The read action will also simultaneously clear the RIS and MIS bits corresponding to the

highest priority interrupt whose index was returned by the read. The read value from the IIDX register can then be used in a case statement, as shown below.

```
void ISR_IIDX(void)
{
    switch(IIDX)
    {
        case 0:          // no IRQ pending
            break;
        case 1:          // IRQ[0]
            do_irq0();
            break;
        case 2:          // IRQ[1]
            do_irq1();
            break;
        default:         // out of range
            illegal();
    }
}
```

CPU IRQ Interrupt Service Routine using CPU_INT MIS and ICLR Registers

Alternatively, application software can read the MIS register to determine which bits are set, followed by using the ICLR register to clear the pending interrupt status bits.

```
void ISR(void)
{
    uint32_t pending = MIS;
    ICLR = pending;          // clear pending IRQ
    if (pending & 0x01)      // IRQ[0]
    {
        do_irq0();
    }
    if (pending & 0x02)      // IRQ[1]
    {
        do_irq1();
    }
}
```

7.2.5.4 Hardware Event Handling

In the case of an event which sources a DMA trigger (DMA_TRIG) or a generic event (GEN_EVENT), the IIDX register is not used. A four-way event handshake is performed between the peripheral generating the event and the hardware entity which is subscribed to the event (for example, the DMA or a secondary peripheral). The [four-way event handshake](#) will clear the corresponding interrupt status bits in the RIS and MIS registers automatically.



The IOMUX controls the configuration of all device pins with digital input-output (IO) functions, including: digital function selection, inversion control, drive strength (if applicable), the pullup or pulldown resistor (if applicable), and wake-up configuration (if applicable on certain IOs for wakeup from [SHUTDOWN](#) mode).

8.1 IOMUX Overview.....	448
8.2 IOMUX Operation.....	451
8.3 IOMUX (PINCMx) Register Format.....	455
8.4 IOMUX Registers.....	457

8.1 IOMUX Overview

The IOMUX manages the configuration of the digital IO. Key functions configured by IOMUX include:

- Selection of which peripheral is multiplexed to each digital IO pin (for example, a GPIO or UART peripheral)
- Digital input path configuration
 - Hysteresis control
 - Input path enable/disable
 - Input logic inversion control
- Digital output path configuration
 - Drive strength control
 - Output connection enable/disable
 - Output logic inversion (control shared with input logic inversion)
 - Logic-high to High-Z output conversion (for open-drain style interfaces)
 - Retention of "last state" when a peripheral connected to an IO is disabled
- Wakeup configuration (for wakeup from SHUTDOWN mode)
 - Read wake up source from the wake stat bit from the PINCM Register
 - Wake up compare level
 - Release SHDNIOREL
 - Wakeup enable/disable
- Pullup and pulldown resistor control

8.1.1 IO Types and Analog Sharing

The IOMUX manages the selection of which peripheral function is to be used on a digital IO. It also provides the controls for the output driver, input path, and the wakeup logic for wakeup from SHUTDOWN mode.

Digital IO Types

There are several digital IO types which can be included on a given device. Each digital IO type supports different features. [Table 8-1](#) lists the features which are included with each IO type. See the device-specific data sheet for which IO type is used on a given package pin.

Table 8-1. Digital IO Features by IO Type

IO Structure	Inversion Control	Drive Strength Control	Hysteresis Control	Pullup Resistor	Pulldown Resistor	Wakeup Logic
Standard-drive	Y			Y	Y	
Standard-drive with wake	Y			Y	Y	Y
High-drive	Y	Y		Y	Y	Y
High-speed	Y	Y		Y	Y	
5V tolerant open drain	Y		Y		Y	Y

Please note that the IOMUX will not support the inversion control and pseudo-open drain (output-high translated to high-impedance) setting on the SPI POCI pins. Also the IOMUX will not support inversion control on the SPI SCLK pins connected on an HSIO pin.

Digital IO Shared with Analog Functions

Certain pins on a device will be digital only and will not have any analog functions connected to the pin. Other pins can have one or more analog functions connected to the pin in addition to the digital IO functions. Analog functions are never selected within the IOMUX; they are always configured within of the respective analog peripheral. Analog peripherals have no knowledge of, or interaction with, the IOMUX.

In general, when analog functionality is used on a pin which also has digital functions, the IOMUX configuration for that pin should be left in its default (high-Z) state so as to not interfere with the proper operation of the analog

function. However, it is possible to have the IOMUX active on a pin when an analog peripheral is also interacting with the pin, provided that the application software ensures that there is not a conflict between the functions. For example, it is possible to have the pullup or pulldown resistor on an IO enabled at the same time that the ADC is running a conversion on the same IO. However, an invalid configuration would be enabling the output driver on an IO at the same time that an analog peripheral is driving the IO (for example, a DAC or OPA output). This would create an IO conflict.

Application software is responsible for ensuring that the IOMUX settings do not conflict with any analog peripheral functions which can be enabled on a shared pad.

IO Slice

The mixed-signal IO pin slice diagram for a full featured IO pin is shown in [Figure 8-1](#). Not all pins will have analog functions, wake-up logic, drive strength control, and pullup or pulldown resistors available. See the device-specific data sheet for detailed information on what features are supported for a specific pin.

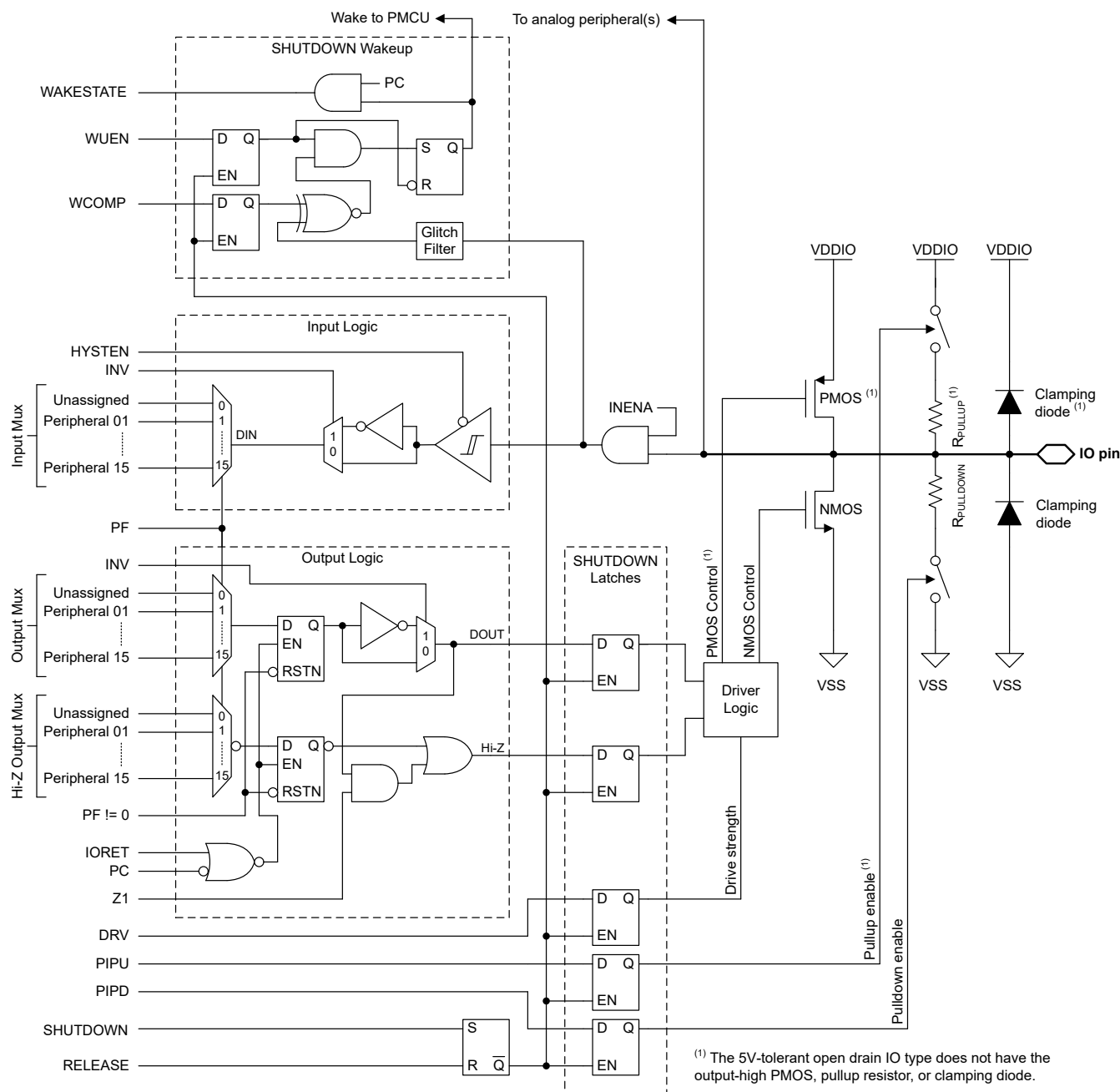


Figure 8-1. Superset IO Slice

Default IOMUX State

The initial state of the IOMUX pin slice for all digital IO after a BOOTRST is as follows:

- The digital IO is in a high-impedance state
- The peripheral function selection field (PF) is cleared (no peripheral function selected) and the peripheral connect (PC) and input enable states are cleared (disabled)
- The inversion logic is disabled
- The Hi-Z output high mode is disabled
- The pullup/pulldown resistors (if present) are disabled
- The input hysteresis control (if present) is disabled to save power
- The drive strength control (if present) is reset

- The wakeup logic (if present) is disabled

Note

The SWD pins are a specific exception to the above default state. The SWD debug pins are configured in SWD mode by default, and may be switched to an alternate setting after start-up. See [Section 2.4.1.4](#) in the SYSCCTL section of the PMCU chapter.

8.2 IOMUX Operation

Each digital IO on a device has a dedicated 32-bit PINCM register in the IOMUX peripheral register space which is used to configure the digital functions of the respective IO. See the device specific data sheet for determining the PINCM register index which corresponds to the IO to be configured.

8.2.1 Peripheral Function (PF) Assignment

When setting up the initial IOMUX configuration for an IO after a BOOTRST, application software can select which digital peripheral from the supported options is to be connect to an IO by writing the appropriate peripheral select value to the PF field while simultaneously setting the PC and INENA bits in the PINCMx register corresponding to the targeted pin. The IOMUX configurations for a given peripheral must be set before the peripheral connected to the IOs has been initialized for operation.

To change the peripheral function selection for a digital IO at runtime after a peripheral function has already been configured for that IO, the following procedure should be followed:

1. Disable the currently connected peripheral function
2. Clear the PC bit (output connect bit) and INENA (input connect bit) in the corresponding PINCMx register
3. Write 0x0 to the PF field in the PINCMx to clear the logic in the data path
4. Select the new peripheral function by writing the peripheral function ID to the PF register
5. Set the PC and INENA bits in the PINCMx register to connect the newly selected peripheral
6. Enable the newly selected peripheral for operation

At runtime, the INENA bit can be used to mask the input from the IO to the peripheral, if desired. When INENA is cleared, a connected peripheral function will see logic low (0) from the IO, regardless of the external state of the IO. If an IO supports wakeup from SHUTDOWN mode, the INENA bit also controls propagation of the IO state to the SHUTDOWN mode wakeup logic.

If a peripheral is assigned to an IO, but the peripheral is itself in a disabled state, the last valid output conditions (output logic level and Hi-Z state) are latched in the IOMUX output logic. When the peripheral is enabled, the IOMUX will release the latched state to allow the (now enabled) peripheral's output state to propagate to the IO. The PMCU indicates to the IOMUX when a peripheral is entering a disabled state via the IORET signal, which is combined with the PC signal via a logic OR to control the output state latches. This mechanism handles preservation of the last valid output state of peripherals in power domain 1 (PD1) when entering STOP or STANDBY mode, as PD1 peripherals are always temporarily disabled upon entry to STOP/STANDBY, and re-enabled upon exit from STOP/STANDBY modes.

When no peripheral function is selected (PF==0) the output latches are put into a reset state, causing the output NMOS and PMOS to be disabled (leaving the IO pin in a Hi-Z state with the exception of any enabled pullup/pulldown resistors). Note that the pullup/pulldown resistors are never controlled by either a connected peripheral or the peripheral muxing logic. They are only controlled by the IOMUX control bits ([see pullup/pulldown](#)).

8.2.2 Logic High to Hi-Z Conversion

The IOMUX supports translating an output high signal from a connected peripheral into a Hi-Z output state at the IO pin. This functionality is particularly useful for open-drain digital input/output applications. When this functionality is enabled, the IO pin state as a function of the peripheral output is as shown in [Table 8-2](#).

Table 8-2. Logic High to Hi-Z Truth Table

Connected Peripheral Output	IO Pin State (Z1 = 0x0)	IO Pin State (Z1 = 0x1)
Logic low (0)	Output low	Output low

Table 8-2. Logic High to Hi-Z Truth Table (continued)

Connected Peripheral Output	IO Pin State (Z1 = 0x0)	IO Pin State (Z1 = 0x1)
Logic high (1)	Output high	High impedance (Hi-Z)

To enable logic high to Hi-Z conversion on a digital IO, set the Z1 bit in the corresponding PINCMx register.

Note that for 5-V tolerant open-drain IO pins, the Z1 control has no effect as there is no high-side driver present. On these pins, a logic high output from the peripheral to the IO pin always results in a Hi-Z state.

8.2.3 Logic Inversion

The IOMUX supports logic inversion of the digital input/output path. Logic inversion is useful for scenarios where opposite polarity is required for UART functions or SPI chip select functions.

To enable logic inversion on a digital IO, set the INV bit in the corresponding PINCMx register. To disable logic inversion, clear the corresponding bit. Logic inversion is disabled by default.

When logic inversion is enabled for a 5V tolerant open drain IO, a connected peripheral which outputs a logic low state will cause the IO pin to go to a Hi-Z state. When the peripheral applies a logic high state, the IO pin will go to an output low state.

8.2.4 SHUTDOWN Mode Wakeup Logic

In SHUTDOWN mode, the entire regulated core supply of the device is disabled and the device wakes only from a wake-capable IO that is configured for wakeup, from NRST, or from a debug connection. The IO wake mechanism for exiting SHUTDOWN is managed by IOMUX and is level based. The 5-V tolerant open-drain IOs, high-drive IOs, and certain standard-drive IOs include the additional wakeup logic that can be used to wake the device from a [SHUTDOWN](#) operating mode upon a level match.

To configure a wake-capable IO for wakeup from SHUTDOWN mode:

1. Set the INENA bit to let the input state propagate from the IO to the wakeup logic.
2. Select the compare level to use for wake by setting or clearing the WCOMP bit in the PINCMx register corresponding to the targeted pin.
3. Enable wakeup by setting the WUEN bit in the PINCMx register corresponding to the targeted pin.

After the previous configuration, SHUTDOWN mode can be entered through the [appropriate command in SYSCTL](#). Pins on the device that contain digital IO controlled by IOMUX retain their current state when the device enters into SHUTDOWN. While the digital IO state is latched upon entry into SHUTDOWN mode, the IOMUX configuration registers (all PINCMx registers) lose their contents as the regulated core supply is shut down.

After SHUTDOWN is entered, a level match on any pin configured for wakeup triggers the exit sequence from SHUTDOWN. When the device exits SHUTDOWN, a BOR-level reset occurs but the state of the digital IO remains latched through the reset, keeping the IO state that was present upon entry into SHUTDOWN. This state is held until the [IO are released in SYSCTL](#). After the BOR, SYSCTL captures the [cause of the reset](#) as a SHUTDOWN exit so that software can identify this and take appropriate action to reconfigure the device.

If multiple pins were configured for wakeup from SHUTDOWN, application software can determine which wakeup-configured IO generated the wake by polling the WAKESTATE bit in any IOs that were enabled for wake before the SHUTDOWN exit.

Application software must apply the following process to restore the IO state upon exit from SHUTDOWN:

1. Check which IO triggered the wakeup from SHUTDOWN, if necessary, as follows:
 - a. Reconfigure the PINCMx register corresponding to the IO to be tested for wakeup status and set the peripheral connect (PC) bit (the PC bit gates the WAKESTATE indication).
 - b. Test the WAKESTATE bit in the PINCMx register corresponding to the IO to be tested to determine if that particular IO received a WAKE status based on the previously configured WCOMP and WUEN configuration.
2. Reconfigure any remaining IOMUX PINCM registers to the correct states.

3. Reconfigure the peripherals that are connected to pins through IOMUX, and enable them.
4. Release the [SHUTDOWN IO lock in SYSCTL](#).
5. Clear the WUEN bit in the PINCMx register to reset the WAKESTATE status.

Note

After waking from SHUTDOWN, if the WUEN bit not cleared and the shutdown release bit in SYSCTL is not set, then reentering SHUTDOWN results in an immediate wake event, because the WAKESTATE status was not cleared from the previous wake event.

8.2.5 Pullup/Pulldown Resistors

Programmable pullup/pulldown resistors are provided on most digital IO types, and are connected to VDD/VSS, respectively. The 5V tolerant open drain digital IO does not provide a pullup resistor due to the open drain configuration.

To enable the pullup or pulldown resistor on a digital IO, set the PIPU or PIPD bit, respectively, in the corresponding PINCMx register. To disable the pullup or pulldown resistor, clear the corresponding bit.

The pullup/pulldown resistors can be enabled at any time, and their configuration is independent from the [peripheral function configuration](#). It is possible to enable a pullup/pulldown resistor while changing the selected peripheral function.

8.2.6 Drive Strength Control

The high-drive and high-speed digital IO types have programmable drive strength (low drive and high drive). The default drive strength is low drive. Application software can request high drive by setting the DRV bit in the PINCMx register corresponding to the target digital IO. Drive strength control is not available for standard drive and open drain IO types.

The drive strength control is completely independent of the selected peripheral function (PF) and can be changed by application software at any time.

For detailed electrical specifications on the drive performance in each drive mode for a given IO, see the Digital IO parameters in the device-specific data sheet.

8.2.7 Hysteresis and Logic Level Control

The 5V-tolerant open drain digital IOs provide a hysteresis and logic level control to enable operation in input mode with standard CMOS logic (hysteresis enabled, CMOS logic levels) and TTL logic (hysteresis disabled, TTL logic levels).

The default mode for the 5V-tolerant open drain digital IO is TLL mode (HYSTEN bit in the PINCMx register is cleared). To use a 5V-tolerant open drain digital IO in CMOS mode with hysteresis enabled, set the HYSTEN bit in the PINCMx register which corresponds to the targeted IO.

The input logic level differences between TTL mode (left) and CMOS mode (right) are shown in [Figure 8-2](#).

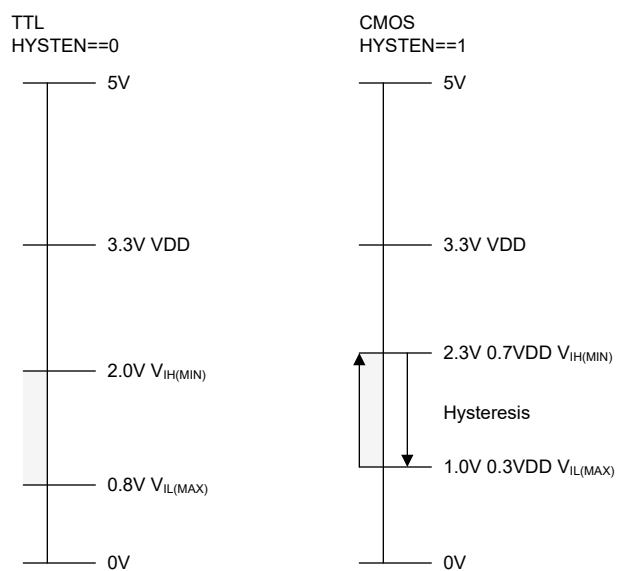


Figure 8-2. Input Logic Levels - 5V Tolerant Open Drain Digital IO

8.3 IOMUX (PINCMx) Register Format

8.3.1 PINCM (Offset = 4h) [Reset = X]

Pin Control Management Register

Figure 8-3. PINCM

31	30	29	28	27	26	25	24
RESERVED			WCOMP	WUEN	INV	HIZ1	RESERVED
R/W-0h			R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
RESERVED			DRV	HYSTEN	INENA	PIPU	PIPD
R/W-0h			R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
RESERVED		WAKESTAT	RESERVED				
R/W-0h		R-0h	R/W-0h				
7	6	5	4	3	2	1	0
PC	RESERVED	PF					
R/W-0h	R/W-0h	R/W-0h					

Table 8-3. PINCM Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R/W	0h	
28	WCOMP	R/W	0h	Wakeup Compare Value bit 0h = Wakeup on a match of 0 1h = Wakeup on a match of 1
27	WUEN	R/W	0h	Wakeup Enable bit 0h = wakeup is disabled. 1h = wakeup is enabled
26	INV	R/W	0h	Data inversion selection 0h = Data inversion is disabled. 1h = Data inversion is enabled
25	HIZ1	R/W	0h	High output value will tri-state the output when this bit is enabled 0h = open-drain is disabled. 1h = open-drain is enabled.
24-21	RESERVED	R/W	0h	
20	DRV	R/W	0h	Drive strength control selection, for HS IOCELL only 0h = Drive setting of 0 selected 1h = Drive setting of 1 selected
19	HYSTEN	R/W	0h	Hysteresis Enable Control Selection 0h = hysteresis is disabled. 1h = hysteresis is enabled
18	INENA	R/W	0h	Input Enable Control Selection 0h = Input enable is disabled. 1h = Input enable is enabled.
17	PIPU	R/W	0h	Pull Up control selection 0h = Pull up is disabled. 1h = Pull up is enabled
16	PIPD	R/W	0h	Pull Down control selection 0h = Pull down is disabled. 1h = Pull down is enabled
15-14	RESERVED	R/W	0h	

Table 8-3. PINCM Field Descriptions (continued)

Bit	Field	Type	Reset	Description
13	WAKESTAT	R	0h	This has the IOPAD WAKEUP signal as status bit. 0h = wakeup source is NOT from this IOCELL 1h = wakeup source is from this IOCELL
12-8	RESERVED	R/W	0h	
7	PC	R/W	0h	Peripheral is "Connected" 0h = The output of the peripheral (and its output enable) will not propagate to the IOCELL 1h = The output latch of the dataflow will be "transparent"
6	RESERVED	R/W	0h	
5-0	PF	R/W	0h	P channel Function selection bits 0h = Reserved as unconnected 3Fh = An encoding per function that can be connected to this pin.

8.4 IOMUX Registers

[Table 8-4](#) lists the memory-mapped registers for the IOMUX registers. All register offset addresses not listed in [Table 8-4](#) should be considered as reserved locations and the register contents should not be modified.

Table 8-4. IOMUX Registers

Offset	Acronym	Register Name	Group	Section
4h	PINCM	Pin Control Management Register in SECCFG region		Go

Complex bit access types are encoded to fit into small table cells. [Table 8-5](#) shows the codes that are used for access types in this section.

Table 8-5. IOMUX Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
W	W	Write
Reset or Default Value		
-n		Value after reset or the default value

8.4.1 PINCM (Offset = 4h) [Reset = X]

PINCM is shown in [Figure 8-4](#) and described in [Table 8-6](#).

Return to the [Summary Table](#).

Pin Control Management Register

Figure 8-4. PINCM

31	30	29	28	27	26	25	24
RESERVED			WCOMP	WUEN	INV	HIZ1	RESERVED
R/W-0h			R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
		RESERVED	DRV	HYSTEN	INENA	PIPU	PIPD
R/W-0h			R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
RESERVED		WAKESTAT	RESERVED				
R/W-0h		R-0h	R/W-0h				
7	6	5	4	3	2	1	0
PC	RESERVED	PF					
R/W-0h	R/W-0h	R/W-0h					

Table 8-6. PINCM Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R/W	0h	
28	WCOMP	R/W	0h	Wakeup Compare Value bit 0h = Wakeup on a match of 0 1h = Wakeup on a match of 1
27	WUEN	R/W	0h	Wakeup Enable bit 0h = wakeup is disabled. 1h = wakeup is enabled
26	INV	R/W	0h	Data inversion selection 0h = Data inversion is disabled. 1h = Data inversion is enabled
25	HIZ1	R/W	0h	High output value will tri-state the output when this bit is enabled 0h = open-drain is disabled. 1h = open-drain is enabled.
24	RESERVED	R/W	0h	
21	RESERVED	R/W	0h	
20	DRV	R/W	0h	Drive strength control selection, for HS IOCELL only 0h = Drive setting of 0 selected 1h = Drive setting of 1 selected
19	HYSTEN	R/W	0h	Hysteresis Enable Control Selection 0h = hysteresis is disabled. 1h = hysteresis is enabled
18	INENA	R/W	0h	Input Enable Control Selection 0h = Input enable is disabled. 1h = Input enable is enabled.
17	PIPU	R/W	0h	Pull Up control selection 0h = Pull up is disabled. 1h = Pull up is enabled
16	PIPD	R/W	0h	Pull Down control selection 0h = Pull down is disabled. 1h = Pull down is enabled
15-14	RESERVED	R/W	0h	

Table 8-6. PINCM Field Descriptions (continued)

Bit	Field	Type	Reset	Description
13	WAKESTAT	R	0h	This has the IOPAD WAKEUP signal as status bit. 0h = wakeup source is NOT from this IOCELL 1h = wakeup source is from this IOCELL
12-8	RESERVED	R/W	0h	
7	PC	R/W	0h	Peripheral is "Connected" 0h = The output of the peripheral (and its output enable) will not propagate to the IOCELL 1h = The output latch of the dataflow will be "transparent"
6	RESERVED	R/W	0h	
5-0	PF	R/W	0h	Peripheral Function selection bits 0h = Reserved as unconnected 3Fh = An encoding per function that can be connected to this pin.

This page intentionally left blank.



The GPIO peripheral provides the user with a means to write data out and read data in to and from the device pins. It also provides a way to detect wakeup events while the device is in a low power state. This chapter describes the operation of the GPIO peripheral.

9.1 GPIO Overview.....	462
9.2 GPIO Operation.....	462
9.3 GPIO Registers.....	467

9.1 GPIO Overview

The GPIO is used to read in digital data from the device pins and to send digital data out to the device pins.

The GPIO module features include:

- Zero wait state MMR access from CPU
- Set/clear/toggle multiple bits without the need of a read-modify-write construct in software
- Direct writes to individual GPIO output bits (DOUT) without the need of a read-modify-write construct in software
- Direct read comparisons of individual GPIO input bits (DIN) without the need to use masking in software
- DOUT serviceable by DMA to generate a predefined output sequence on specified pins
- "FastWake" feature enables low-power wakeup from STOP and STANDBY modes for any GPIO port
- User controlled input filtering (configurable per IO)
- Interconnection to the device event fabric through event publishers and event subscribers (GPIOA instance only)

Figure 9-1 shows the block diagram of the GPIO peripheral.

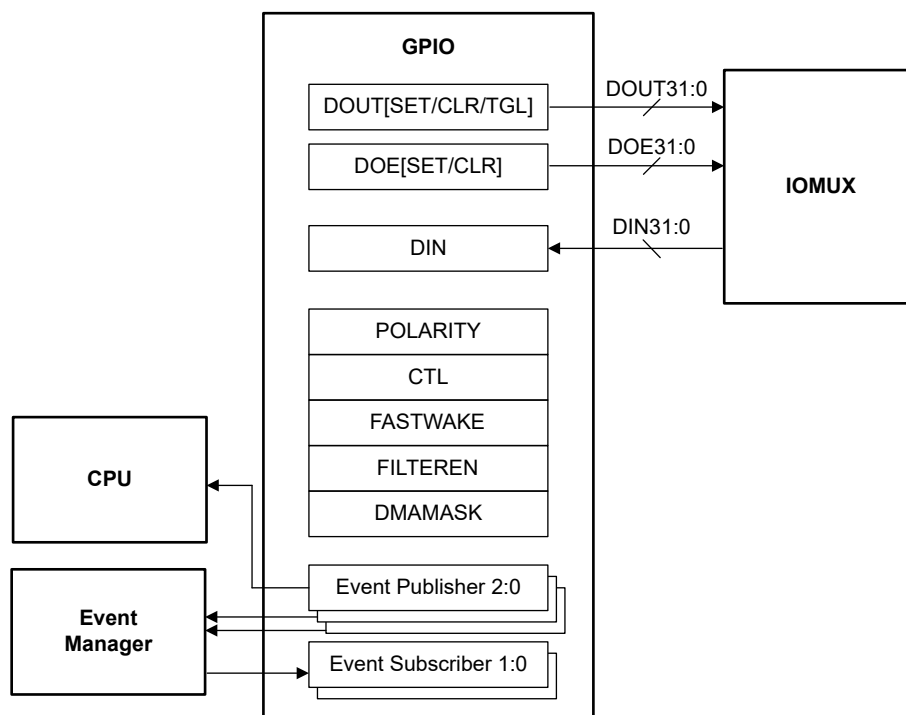


Figure 9-1. GPIO Block Diagram

Note

The GPIO module for the MSPM0 platform does not manage the complete digital IO functionality (for example, pullup, pulldown, or other functional muxing). For complete digital IO control details, refer to [Chapter 8](#). Similar to any other peripheral, the GPIO has inputs and outputs (with output enable) that allow the GPIO to interface with the IOMUX to make connections to the IO pins.

9.2 GPIO Operation

The GPIO peripheral is configured with user software. The setup and operation of the GPIO is discussed in the following sections.

9.2.1 GPIO Ports

An instance of the GPIO peripheral in the MSPM0 platform supports up to 32 data input/output (DIO) bits. For devices with greater than 32 GPIOs, multiple instances of the GPIO peripheral are used to address all of the device pins. The GPIO port and bit names are directly mapped to the signal names associated with each device pin in the *Pin Configuration and Functions* section of the device data sheet.

Table 9-1. GPIO Port and Device Pin Mapping

GPIO Port and Bit Name	Device Pin Signal Name
GPIO Port A Bit 0 (DIO0)	PA0
GPIO Port A Bit 1 (DIO1)	PA1
GPIO Port B Bit 0 (DIO0)	PB0
GPIO Port B Bit 1 (DIO1)	PB1
GPIO Port x Bit y (DIOy)	Pxy

9.2.2 GPIO Read/Write Interface

The GPIO peripheral has features and dedicated registers to allow for advanced bit manipulations without the need to execute a read-modify-write construct in software. These features are outlined below.

Read Interface

The DIN31_0 register is the data input register for a given GPIO port. Reading a bit in the DIN31_0 register corresponds to reading the signal voltage level on the associated device pin signal. The GPIO peripheral provides single-bit byte-read addresses for all bits of DIN. These registers are named DIN31_28, DIN27_24, ... , DIN3_0 and are essentially alias registers for all of the bits in the DIN31_0 data input register. Reading from the DIN31_28 register grants you byte level access to DIN31:28 which allows you to perform direct read comparisons without masking.

Write Interface

In many cases it is useful to write specific values for device pin signals based on the user's system. Register DOUT31_0 is the physical data output register for a given GPIO port. Setting a bit in the DOUT31_0 register sets the corresponding device pin signal when the output is enabled through the DOE31_0 register. Conversely, clearing a bit in the DOUT31_0 register clears the corresponding device pin signal.

The GPIO peripheral also provides single-bit byte-write addresses for all bits of DOUT. These registers are named DOUT31_28, DOUT27_24, ... , DOUT3_0 and are essentially alias registers for all of the bits in the DOUT31_0 data output register. Writing to the DOUT31_28 register grants you byte level access to DOUT31:28 which allows you to directly write to these individual bits without the need to use a read-modify-write construct.

Set/Clear/Toggle

Similar to the use-cases described above, it is also useful in an application to set, clear, or toggle a device pin or a collection of pins. The MSPM0 platform lets you execute these functions in one write cycle using the DOUTSET31_0, DOUTCLR31_0, and DOUTTGL31_0 registers. Setting a bit in these registers implements the corresponding function while writing a '0' has no effect. This feature allows you to directly set, clear, and toggle any bit in the DOUT31_0 register using a single command per GPIO port.

In the same way one can set and clear the data output bits, there are registers to do the same functionality with the data output enable register, DOE31_0. Writing a '1' to bits in the DOESET31_0 and DOECLR31_0 register implements the corresponding function while writing a '0' has no effect. This feature allows you to directly set or clear any bit in the DOE31_0 register using a single command per GPIO port.

9.2.3 GPIO Input Glitch Filtering and Synchronization

The GPIO module evaluates the state of input pins at the ULPCCLK (PD0 bus clock) rate, synchronizing the pin state to ULPCCLK through a 2-stage synchronizer before passing the GPIO state to the input glitch filter.

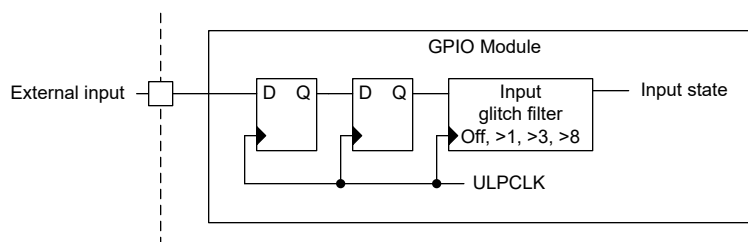


Figure 9-2. GPIO Input Synchronizer

A programmable input glitch filter is provided for suppressing noise on digital input pins. The glitch filter runs at ULPCCLK rate. Four levels of user-specified input filtering are possible:

- Sampled input without filtering (the minimum reliably detected pulse width is one ULPCCLK cycle due to synchronization of the pin state with ULPCCLK + Delay time from edge of asynchronous request to first 32MHz MCLK edge in case of fast wake enable for STANDBY0/1, STOP1/2 and SLEEP2 modes)
- Synchronized inputs which are not greater than 1 ULPCCLK periods are filtered out
- Synchronized inputs which are not greater than 3 ULPCCLK periods are filtered out
- Synchronized inputs which are not greater than 8 ULPCCLK periods are filtered out

This feature allows users to easily implement input filtering in hardware for cases where fast switching on the input pin is needed to be filtered out. The bit fields in the FILTEREN31_16 and FILTEREN15_0 registers allow users to configure the level of filtering needed for the corresponding GPIO bit.

Input pulses of the same pulse length can be passed in some cases while being filtered in other cases, due to 1 ULPCCLK cycle of uncertainty in the synchronization.

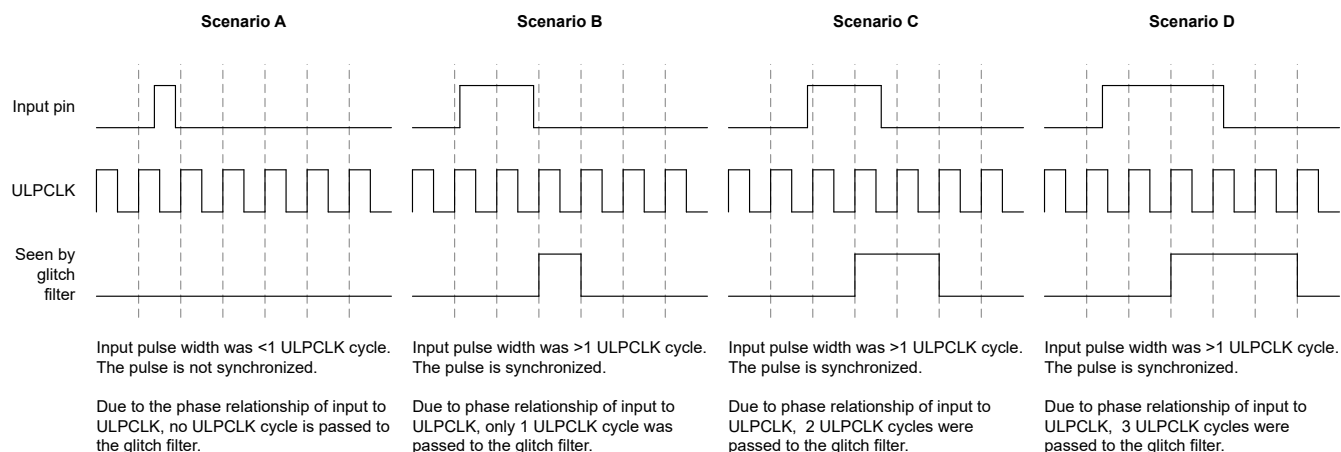


Figure 9-3. GPIO Input Synchronizer and Glitch Filtering Scenarios

- In Scenario A, the input pulse is less than one ULPCCLK cycle. Pulses less than one ULPCCLK cycle may not be captured. To ensure that GPIO inputs are always captured, the GPIO input pulse width must be greater than the ULPCCLK period.
- In Scenario B, the input pulse is nearly two ULPCCLK cycles in length, but because the rising edge occurs just after the ULPCCLK edge, the GPIO synchronizer only views the input pin as having been high for 1 ULPCCLK period. This scenario would not be filtered out by the glitch filter when the glitch filter is disabled, but a glitch filter value of >1 would result in this pulse being filtered. Conversely, the same input pulse width in Scenario C results in the input pin being considered high for two ULPCCLK periods, as the rising edge occurred just

before the ULPCLK edge. In this case, this scenario would not be filtered out when a glitch filter value of >1 is specified.

- In Scenario D, three ULPCLK cycles are passed to the glitch filter. In this case, the scenario would not be filtered out when a glitch filter value of >1 is specified, but it would be filtered out for a glitch filter value of >3 or >8.

Note

When the fast wake mode is enabled (SYSOSC is requested asynchronously upon input pin activity), the ULPCLK will switch from off (as would be the case in STANDBY1) or 32kHz (as would be the case in STANDBY0) to 32MHz, resulting in the input synchronization logic and glitch filter running at 32MHz after some latency. See the device specific data sheet for the asynchronous fast clock request wake time, and budget this time into any minimum pulse width calculations when using fast wake.

9.2.4 GPIO Fast Wake

The fast wake feature in the MSPM0 GPIO peripheral allows the GPIO module to stay in a low power state and detect interrupt events on the device pins without requiring a high-speed clock. This allows the device to support fast wakeup from low-power modes, such as STOP and STANDBY, on any GPIO pin.

Fast wake can be enabled on a bit-wise basis using the FASTWAKE register. Setting a bit in the FASTWAKE register enables the corresponding device pin signal to support fast wakeup functionality. The CTL register contains a bit field named FASTWAKEONLY which allows for global control of the fast wake feature. Setting the FASTWAKEONLY bit enables all of the bits in the corresponding GPIO port to support fast wakeup functionality.

Note

Do not enable fast wake in the GPIO while simultaneously blocking asynchronous fast clock requests in SYSCTL. When fast wake is enabled, the GPIO expects to handshake with SYSCTL for the fast clock. If SYSCTL ignores the request, the GPIO will not receive a clock until SYSCTL completes the asynchronous fast clock request handshake.

9.2.5 GPIO DMA Interface

The GPIO peripheral allows the DMA write-access to the DOUT31_0 register. This functionality allows users to generate predefined output sequences on specified device pins. Some applications require preloaded sequences of GPIO pin changes and the MSPM0 platform allows for the DMA to run that sequence so that the CPU can remain asleep and conserve energy.

The DMAMASK register is used to indicate which GPIO bits the DMA is allowed to modify. Setting a bit in the DMAMASK register enables the corresponding DOUT bit to be modified by the DMA.

Note

The CPU can write to any DOUT31_0 bit regardless of the DMAMASK value.

In cases where the DMA and the CPU both attempt to access and modify the DOUT31_0 register concurrently, it is the user's responsibility to manage the DMA and CPU bus transactions that are targeting the same bit to be modified.

- If a DMAMASK bit is set, the DMA will be prioritized to modify the corresponding DOUT bit.
- If a DMAMASK bit is cleared, the CPU will be prioritized to modify the corresponding DOUT bit.

9.2.6 Event Publishers and Subscribers

There are three independent event publishers available for GPIOx peripherals:

1. First Event Publisher (CPU_INT)
 - Used for generating CPU interrupt
 - Interrupt (RIS) flags are cleared upon software reading the IIDX register or writing to the respective ICLR register bits.

- An event to the CPU can be individually specified for each GPIO bit through the POLARITY register:
 - 0: Disabled
 - 1: Rise Event
 - 2: Fall Event
 - 3: Rise or Fall Event
- 2. Second Event Publisher (GEN_EVENT0), available on GPIOA only
 - Uses the same POLARITY register definition as CPU_INT
 - Applies to GPIO bits 15 down to 0 (DIO15:0)
- 3. Third Event Publisher (GEN_EVENT1), available on GPIOA only
 - Uses the same POLARITY register definition as CPU_INT
 - Applies to GPIO bits 31 down to 16 (DIO31:16)

There are **two event subscribers** (available for the GPIOA peripheral only):

1. First Event Subscriber (FSUB_0)
 - A specific pin can be directed to change state on an event
 - A subscriber event can only cause one single bit to have an action
 - Applies to GPIO bits 15 down to 0 (DIO15:0)
 - SUB0CFG register is used to enable the FSUB_0 event and define the output policy for a specific GPIO pin
2. Second Event Subscriber (FSUB_1)
 - A specific pin can be directed to change state on an event
 - A subscriber event can only cause one single bit to have an action
 - Applies to GPIO bits 31 down to 16 (DIO31:16)
 - SUB1CFG register is used to enable the FSUB_1 event and define the output policy for a specific GPIO pin

9.3 GPIO Registers

[Table 9-2](#) lists the memory-mapped registers for the GPIO registers. All register offset addresses not listed in [Table 9-2](#) should be considered as reserved locations and the register contents should not be modified.

Table 9-2. GPIO Registers

Offset	Acronym	Register Name	Group	Section
400h	FSUB_0	Subscriber Port 0		Go
404h	FSUB_1	Subscriber Port 1		Go
444h	FPUB_0	Publisher Port 0		Go
448h	FPUB_1	Publisher Port 1		Go
800h	PWREN	Power enable		Go
804h	RSTCTL	Reset Control		Go
814h	STAT	Status Register		Go
1010h	CLKOVR	Clock Override		Go
1018h	PDBGCTL	Peripheral Debug Control		Go
1020h	IIDX	Interrupt index	CPU_INT	Go
1028h	IMASK	Interrupt mask	CPU_INT	Go
1030h	RIS	Raw interrupt status	CPU_INT	Go
1038h	MIS	Masked interrupt status	CPU_INT	Go
1040h	ISSET	Interrupt set	CPU_INT	Go
1048h	ICLR	Interrupt clear	CPU_INT	Go
1050h	IIDX	Interrupt index	GEN_EVENT 0	Go
1058h	IMASK	Interrupt mask	GEN_EVENT 0	Go
1060h	RIS	Raw interrupt status	GEN_EVENT 0	Go
1068h	MIS	Masked interrupt status	GEN_EVENT 0	Go
1070h	ISSET	Interrupt set	GEN_EVENT 0	Go
1078h	ICLR	Interrupt clear	GEN_EVENT 0	Go
1080h	IIDX	Interrupt index	GEN_EVENT 1	Go
1088h	IMASK	Interrupt mask	GEN_EVENT 1	Go
1090h	RIS	Raw interrupt status	GEN_EVENT 1	Go
1098h	MIS	Masked interrupt status	GEN_EVENT 1	Go
10A0h	ISSET	Interrupt set	GEN_EVENT 1	Go
10A8h	ICLR	Interrupt clear	GEN_EVENT 1	Go
10E0h	EVT_MODE	Event Mode		Go
10FCh	DESC	Module Description		Go
1200h	DOUT3_0	Data output 3 to 0		Go
1204h	DOUT7_4	Data output 7 to 4		Go
1208h	DOUT11_8	Data output 11 to 8		Go
120Ch	DOUT15_12	Data output 15 to 12		Go

Table 9-2. GPIO Registers (continued)

Offset	Acronym	Register Name	Group	Section
1210h	DOUT19_16	Data output 19 to 16		Go
1214h	DOUT23_20	Data output 23 to 20		Go
1218h	DOUT27_24	Data output 27 to 24		Go
121Ch	DOUT31_28	Data output 31 to 28		Go
1280h	DOUT31_0	Data output 31 to 0		Go
1290h	DOUTSET31_0	Data output set 31 to 0		Go
12A0h	DOUTCLR31_0	Data output clear 31 to 0		Go
12B0h	DOUTTGL31_0	Data output toggle 31 to 0		Go
12C0h	DOE31_0	Data output enable 31 to 0		Go
12D0h	DOESET31_0	Data output enable set 31 to 0		Go
12E0h	DOECLR31_0	Data output enable clear 31 to 0		Go
1300h	DIN3_0	Data input 3 to 0		Go
1304h	DIN7_4	Data input 7 to 4		Go
1308h	DIN11_8	Data input 11 to 8		Go
130Ch	DIN15_12	Data input 15 to 12		Go
1310h	DIN19_16	Data input 19 to 16		Go
1314h	DIN23_20	Data input 23 to 20		Go
1318h	DIN27_24	Data input 27 to 24		Go
131Ch	DIN31_28	Data input 31 to 28		Go
1380h	DIN31_0	Data input 31 to 0		Go
1390h	POLARITY15_0	Polarity 15 to 0		Go
13A0h	POLARITY31_16	Polarity 31 to 16		Go
1400h	CTL	FAST WAKE GLOBAL EN		Go
1404h	FASTWAKE	FAST WAKE ENABLE		Go
1500h	SUB0CFG	Subscriber 0 configuration		Go
1508h	FILTEREN15_0	Filter Enable 15 to 0		Go
150Ch	FILTEREN31_16	Filter Enable 31 to 16		Go
1510h	DMAMASK	DMA Write MASK		Go
1520h	SUB1CFG	Subscriber 1 configuration		Go

Complex bit access types are encoded to fit into small table cells. [Table 9-3](#) shows the codes that are used for access types in this section.

Table 9-3. GPIO Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
K	K	Write protected by a key
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value

9.3.1 FSUB_0 (Offset = 400h) [Reset = 00000000h]

FSUB_0 is shown in [Figure 9-4](#) and described in [Table 9-4](#).

Return to the [Summary Table](#).

Subscriber port

Figure 9-4. FSUB_0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-0h												R/W-0h			

Table 9-4. FSUB_0 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 15.

9.3.2 FSUB_1 (Offset = 404h) [Reset = 00000000h]

FSUB_1 is shown in [Figure 9-5](#) and described in [Table 9-5](#).

Return to the [Summary Table](#).

Subscriber port

Figure 9-5. FSUB_1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-0h												R/W-0h			

Table 9-5. FSUB_1 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 15.

9.3.3 FPUB_0 (Offset = 444h) [Reset = 00000000h]

FPUB_0 is shown in [Figure 9-6](#) and described in [Table 9-6](#).

Return to the [Summary Table](#).

Publisher port

Figure 9-6. FPUB_0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-0h												R/W-0h			

Table 9-6. FPUB_0 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 15.

9.3.4 FPUB_1 (Offset = 448h) [Reset = 00000000h]

FPUB_1 is shown in [Figure 9-7](#) and described in [Table 9-7](#).

Return to the [Summary Table](#).

Publisher port

Figure 9-7. FPUB_1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-0h												R/W-0h			

Table 9-7. FPUB_1 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 15.

9.3.5 PWREN (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 9-8](#) and described in [Table 9-8](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 9-8. PWREN

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-							K-0h

Table 9-8. PWREN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	ENABLE	K	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

9.3.6 RSTCTL (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 9-9](#) and described in [Table 9-9](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 9-9. RSTCTL

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-							
15	14	13	12	11	10	9	8
RESERVED							
W-							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCL R	RESETASSERT
W-						WK-0h	WK-0h

Table 9-9. RSTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	W	0h	
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

9.3.7 STAT (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 9-10](#) and described in [Table 9-10](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 9-10. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED							
R-							

Table 9-10. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0h	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

9.3.8 CLKOVR (Offset = 1010h) [Reset = 00000000h]

CLKOVR is shown in [Figure 9-11](#) and described in [Table 9-11](#).

Return to the [Summary Table](#).

This register overrides the functional clock request by this peripheral to the system

Figure 9-11. CLKOVR

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED						RUN_STOP	OVERRIDE
R/W-0h						R/W-0h	R/W-0h

Table 9-11. CLKOVR Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R/W	0h	
1	RUN_STOP	R/W	0h	If OVERRIDE is enabled, this register is used to manually control the peripheral's clock request to the system 0h = Run/ungate functional clock 1h = Stop/gate functional clock
0	OVERRIDE	R/W	0h	Unlocks the functionality of RUN_STOP to override the automatic peripheral clock request 0h = Override disabled 1h = Override enabled

9.3.9 PDBGCTL (Offset = 1018h) [Reset = 00000001h]

PDBGCTL is shown in [Figure 9-12](#) and described in [Table 9-12](#).

Return to the [Summary Table](#).

This register can be used by the software developer to control the behavior of the peripheral relative to the 'Core Halted' input

Figure 9-12. PDBGCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							FREE
R/W-0h							R/W-1h

Table 9-12. PDBGCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R/W	0h	
0	FREE	R/W	1h	Free run control 0h = The peripheral freezes functionality while the Core Halted input is asserted and resumes when it is deasserted. 1h = The peripheral ignores the state of the Core Halted input

9.3.10 IIDX (Offset = 1020h) [Reset = 00000000h]

IIDX is shown in [Figure 9-13](#) and described in [Table 9-13](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 9-13. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 9-13. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 0h = No bit is set means there is no pending interrupt request 1h = DIO0 interrupt 2h = DIO1 interrupt 3h = DIO2 interrupt 4h = DIO3 interrupt 5h = DIO4 interrupt 6h = DIO5 interrupt 7h = DIO6 interrupt 8h = DIO7 interrupt 9h = DIO8 interrupt Ah = DIO9 interrupt Bh = DIO10 interrupt Ch = DIO11 interrupt Dh = DIO12 interrupt Eh = DIO13 interrupt Fh = DIO14 interrupt 10h = DIO15 interrupt 11h = DIO16 interrupt 12h = DIO17 interrupt 13h = DIO18 interrupt 14h = DIO19 interrupt 15h = DIO20 interrupt 16h = DIO21 interrupt 17h = DIO22 interrupt 18h = DIO23 interrupt 19h = DIO24 interrupt 1Ah = DIO25 interrupt 1Bh = DIO26 interrupt 1Ch = DIO27 interrupt 1Dh = DIO28 interrupt 1Eh = DIO29 interrupt 1Fh = DIO30 interrupt 20h = DIO31 interrupt

9.3.11 IMASK (Offset = 1028h) [Reset = 00000000h]

IMASK is shown in [Figure 9-14](#) and described in [Table 9-14](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 9-14. IMASK

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 9-14. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	R/W	0h	DIO31 event mask 0h = Event is masked 1h = Event is unmasked
30	DIO30	R/W	0h	DIO30 event mask 0h = Event is masked 1h = Event is unmasked
29	DIO29	R/W	0h	DIO29 event mask 0h = Event is masked 1h = Event is unmasked
28	DIO28	R/W	0h	DIO28 event mask 0h = Event is masked 1h = Event is unmasked
27	DIO27	R/W	0h	DIO27 event mask 0h = Event is masked 1h = Event is unmasked
26	DIO26	R/W	0h	DIO26 event mask 0h = Event is masked 1h = Event is unmasked
25	DIO25	R/W	0h	DIO25 event mask 0h = Event is masked 1h = Event is unmasked
24	DIO24	R/W	0h	DIO24 event mask 0h = Event is masked 1h = Event is unmasked
23	DIO23	R/W	0h	DIO23 event mask 0h = Event is masked 1h = Event is unmasked
22	DIO22	R/W	0h	DIO22 event mask 0h = Event is masked 1h = Event is unmasked

Table 9-14. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	DIO21	R/W	0h	DIO21 event mask 0h = Event is masked 1h = Event is unmasked
20	DIO20	R/W	0h	DIO20 event mask 0h = Event is masked 1h = Event is unmasked
19	DIO19	R/W	0h	DIO19 event mask 0h = Event is masked 1h = Event is unmasked
18	DIO18	R/W	0h	DIO18 event mask 0h = Event is masked 1h = Event is unmasked
17	DIO17	R/W	0h	DIO17 event mask 0h = Event is masked 1h = Event is unmasked
16	DIO16	R/W	0h	DIO16 event mask 0h = Event is masked 1h = Event is unmasked
15	DIO15	R/W	0h	DIO15 event mask 0h = Event is masked 1h = Event is unmasked
14	DIO14	R/W	0h	DIO14 event mask 0h = Event is masked 1h = Event is unmasked
13	DIO13	R/W	0h	DIO13 event mask 0h = Event is masked 1h = Event is unmasked
12	DIO12	R/W	0h	DIO12 event mask 0h = Event is masked 1h = Event is unmasked
11	DIO11	R/W	0h	DIO11 event mask 0h = Event is masked 1h = Event is unmasked
10	DIO10	R/W	0h	DIO10 event mask 0h = Event is masked 1h = Event is unmasked
9	DIO9	R/W	0h	DIO9 event mask 0h = Event is masked 1h = Event is unmasked
8	DIO8	R/W	0h	DIO8 event mask 0h = Event is masked 1h = Event is unmasked
7	DIO7	R/W	0h	DIO7 event mask 0h = Event is masked 1h = Event is unmasked
6	DIO6	R/W	0h	DIO6 event mask 0h = Event is masked 1h = Event is unmasked
5	DIO5	R/W	0h	DIO5 event mask 0h = Event is masked 1h = Event is unmasked
4	DIO4	R/W	0h	DIO4 event mask 0h = Event is masked 1h = Event is unmasked
3	DIO3	R/W	0h	DIO3 event mask 0h = Event is masked 1h = Event is unmasked

Table 9-14. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	DIO2	R/W	0h	DIO2 event mask 0h = Event is masked 1h = Event is unmasked
1	DIO1	R/W	0h	DIO1 event mask 0h = Event is masked 1h = Event is unmasked
0	DIO0	R/W	0h	DIO0 event mask 0h = Event is masked 1h = Event is unmasked

9.3.12 RIS (Offset = 1030h) [Reset = 00000000h]

RIS is shown in [Figure 9-15](#) and described in [Table 9-15](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 9-15. RIS

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 9-15. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	R	0h	DIO31 event 0h = DIO31 event did not occur 1h = DIO31 event occurred
30	DIO30	R	0h	DIO30 event 0h = DIO30 event did not occur 1h = DIO30 event occurred
29	DIO29	R	0h	DIO29 event 0h = DIO29 event did not occur 1h = DIO29 event occurred
28	DIO28	R	0h	DIO28 event 0h = DIO28 event did not occur 1h = DIO28 event occurred
27	DIO27	R	0h	DIO27 event 0h = DIO27 event did not occur 1h = DIO27 event occurred
26	DIO26	R	0h	DIO26 event 0h = DIO26 event did not occur 1h = DIO26 event occurred
25	DIO25	R	0h	DIO25 event 0h = DIO25 event did not occur 1h = DIO25 event occurred
24	DIO24	R	0h	DIO24 event 0h = DIO24 event did not occur 1h = DIO24 event occurred
23	DIO23	R	0h	DIO23 event 0h = DIO23 event did not occur 1h = DIO23 event occurred
22	DIO22	R	0h	DIO22 event 0h = DIO22 event did not occur 1h = DIO22 event occurred

Table 9-15. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	DIO21	R	0h	DIO21 event 0h = DIO21 event did not occur 1h = DIO21 event occurred
20	DIO20	R	0h	DIO20 event 0h = DIO20 event did not occur 1h = DIO20 event occurred
19	DIO19	R	0h	DIO19 event 0h = DIO19 event did not occur 1h = DIO19 event occurred
18	DIO18	R	0h	DIO18 event 0h = DIO18 event did not occur 1h = DIO18 event occurred
17	DIO17	R	0h	DIO17 event 0h = DIO17 event did not occur 1h = DIO17 event occurred
16	DIO16	R	0h	DIO16 event 0h = DIO16 event did not occur 1h = DIO16 event occurred
15	DIO15	R	0h	DIO15 event 0h = DIO15 event did not occur 1h = DIO15 event occurred
14	DIO14	R	0h	DIO14 event 0h = DIO14 event did not occur 1h = DIO14 event occurred
13	DIO13	R	0h	DIO13 event 0h = DIO13 event did not occur 1h = DIO13 event occurred
12	DIO12	R	0h	DIO12 event 0h = DIO12 event did not occur 1h = DIO12 event occurred
11	DIO11	R	0h	DIO11 event 0h = DIO11 event did not occur 1h = DIO11 event occurred
10	DIO10	R	0h	DIO10 event 0h = DIO10 event did not occur 1h = DIO10 event occurred
9	DIO9	R	0h	DIO9 event 0h = DIO9 event did not occur 1h = DIO9 event occurred
8	DIO8	R	0h	DIO8 event 0h = DIO8 event did not occur 1h = DIO8 event occurred
7	DIO7	R	0h	DIO7 event 0h = DIO7 event did not occur 1h = DIO7 event occurred
6	DIO6	R	0h	DIO6 event 0h = DIO6 event did not occur 1h = DIO6 event occurred
5	DIO5	R	0h	DIO5 event 0h = DIO5 event did not occur 1h = DIO5 event occurred
4	DIO4	R	0h	DIO4 event 0h = DIO4 event did not occur 1h = DIO4 event occurred
3	DIO3	R	0h	DIO3 event 0h = DIO3 event did not occur 1h = DIO3 event occurred

Table 9-15. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	DIO2	R	0h	DIO2 event 0h = DIO2 event did not occur 1h = DIO2 event occurred
1	DIO1	R	0h	DIO1 event 0h = DIO1 event did not occur 1h = DIO1 event occurred
0	DIO0	R	0h	DIO0 event 0h = DIO0 event did not occur 1h = DIO0 event occurred

9.3.13 MIS (Offset = 1038h) [Reset = 00000000h]

MIS is shown in [Figure 9-16](#) and described in [Table 9-16](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 9-16. MIS

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 9-16. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	R	0h	DIO31 event 0h = DIO31 event did not occur 1h = DIO31 event occurred
30	DIO30	R	0h	DIO30 event 0h = DIO30 event did not occur 1h = DIO30 event occurred
29	DIO29	R	0h	DIO29 event 0h = DIO29 event did not occur 1h = DIO29 event occurred
28	DIO28	R	0h	DIO28 event 0h = DIO28 event did not occur 1h = DIO28 event occurred
27	DIO27	R	0h	DIO27 event 0h = DIO27 event did not occur 1h = DIO27 event occurred
26	DIO26	R	0h	DIO26 event 0h = DIO26 event did not occur 1h = DIO26 event occurred
25	DIO25	R	0h	DIO25 event 0h = DIO25 event did not occur 1h = DIO25 event occurred
24	DIO24	R	0h	DIO24 event 0h = DIO24 event did not occur 1h = DIO24 event occurred
23	DIO23	R	0h	DIO23 event 0h = DIO23 event did not occur 1h = DIO23 event occurred
22	DIO22	R	0h	DIO22 event 0h = DIO22 event did not occur 1h = DIO22 event occurred

Table 9-16. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	DIO21	R	0h	DIO21 event 0h = DIO21 event did not occur 1h = DIO21 event occurred
20	DIO20	R	0h	DIO20 event 0h = DIO20 event did not occur 1h = DIO20 event occurred
19	DIO19	R	0h	DIO19 event 0h = DIO19 event did not occur 1h = DIO19 event occurred
18	DIO18	R	0h	DIO18 event 0h = DIO18 event did not occur 1h = DIO18 event occurred
17	DIO17	R	0h	DIO17 event 0h = DIO17 event did not occur 1h = DIO17 event occurred
16	DIO16	R	0h	DIO16 event 0h = DIO16 event did not occur 1h = DIO16 event occurred
15	DIO15	R	0h	DIO15 event 0h = DIO15 event did not occur 1h = DIO15 event occurred
14	DIO14	R	0h	DIO14 event 0h = DIO14 event did not occur 1h = DIO14 event occurred
13	DIO13	R	0h	DIO13 event 0h = DIO13 event did not occur 1h = DIO13 event occurred
12	DIO12	R	0h	DIO12 event 0h = DIO12 event did not occur 1h = DIO12 event occurred
11	DIO11	R	0h	DIO11 event 0h = DIO11 event did not occur 1h = DIO11 event occurred
10	DIO10	R	0h	DIO10 event 0h = DIO10 event did not occur 1h = DIO10 event occurred
9	DIO9	R	0h	DIO9 event 0h = DIO9 event did not occur 1h = DIO9 event occurred
8	DIO8	R	0h	DIO8 event 0h = DIO8 event did not occur 1h = DIO8 event occurred
7	DIO7	R	0h	DIO7 event 0h = DIO7 event did not occur 1h = DIO7 event occurred
6	DIO6	R	0h	DIO6 event 0h = DIO6 event did not occur 1h = DIO6 event occurred
5	DIO5	R	0h	DIO5 event 0h = DIO5 event did not occur 1h = DIO5 event occurred
4	DIO4	R	0h	DIO4 event 0h = DIO4 event did not occur 1h = DIO4 event occurred
3	DIO3	R	0h	DIO3 event 0h = DIO3 event did not occur 1h = DIO3 event occurred

Table 9-16. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	DIO2	R	0h	DIO2 event 0h = DIO2 event did not occur 1h = DIO2 event occurred
1	DIO1	R	0h	DIO1 event 0h = DIO1 event did not occur 1h = DIO1 event occurred
0	DIO0	R	0h	DIO0 event 0h = DIO0 event did not occur 1h = DIO0 event occurred

9.3.14 ISET (Offset = 1040h) [Reset = 00000000h]

ISET is shown in [Figure 9-17](#) and described in [Table 9-17](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 9-17. ISET

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 9-17. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	W	0h	DIO31 event 0h = No effect 1h = Sets DIO31 in RIS register
30	DIO30	W	0h	DIO30 event 0h = No effect 1h = Sets DIO30 in RIS register
29	DIO29	W	0h	DIO29 event 0h = No effect 1h = Sets DIO29 in RIS register
28	DIO28	W	0h	DIO28 event 0h = No effect 1h = Sets DIO28 in RIS register
27	DIO27	W	0h	DIO27 event 0h = No effect 1h = Sets DIO27 in RIS register
26	DIO26	W	0h	DIO26 event 0h = No effect 1h = Sets DIO26 in RIS register
25	DIO25	W	0h	DIO25 event 0h = No effect 1h = Sets DIO25 in RIS register
24	DIO24	W	0h	DIO24 event 0h = No effect 1h = Sets DIO24 in RIS register
23	DIO23	W	0h	DIO23 event 0h = No effect 1h = Sets DIO23 in RIS register
22	DIO22	W	0h	DIO22 event 0h = No effect 1h = Sets DIO22 in RIS register

Table 9-17. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	DIO21	W	0h	DIO21 event 0h = No effect 1h = Sets DIO21 in RIS register
20	DIO20	W	0h	DIO20 event 0h = No effect 1h = Sets DIO20 in RIS register
19	DIO19	W	0h	DIO19 event 0h = No effect 1h = Sets DIO19 in RIS register
18	DIO18	W	0h	DIO18 event 0h = No effect 1h = Sets DIO18 in RIS register
17	DIO17	W	0h	DIO17 event 0h = No effect 1h = Sets DIO17 in RIS register
16	DIO16	W	0h	DIO16 event 0h = No effect 1h = Sets DIO16 in RIS register
15	DIO15	W	0h	DIO15 event 0h = No effect 1h = Sets DIO15 in RIS register
14	DIO14	W	0h	DIO14 event 0h = No effect 1h = Sets DIO14 in RIS register
13	DIO13	W	0h	DIO13 event 0h = No effect 1h = Sets DIO13 in RIS register
12	DIO12	W	0h	DIO12 event 0h = No effect 1h = Sets DIO12 in RIS register
11	DIO11	W	0h	DIO11 event 0h = No effect 1h = Sets DIO11 in RIS register
10	DIO10	W	0h	DIO10 event 0h = No effect 1h = Sets DIO10 in RIS register
9	DIO9	W	0h	DIO9 event 0h = No effect 1h = Sets DIO9 in RIS register
8	DIO8	W	0h	DIO8 event 0h = No effect 1h = Sets DIO8 in RIS register
7	DIO7	W	0h	DIO7 event 0h = No effect 1h = Sets DIO7 in RIS register
6	DIO6	W	0h	DIO6 event 0h = No effect 1h = Sets DIO6 in RIS register
5	DIO5	W	0h	DIO5 event 0h = No effect 1h = Sets DIO5 in RIS register
4	DIO4	W	0h	DIO4 event 0h = No effect 1h = Sets DIO4 in RIS register
3	DIO3	W	0h	DIO3 event 0h = No effect 1h = Sets DIO3 in RIS register

Table 9-17. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	DIO2	W	0h	DIO2 event 0h = No effect 1h = Sets DIO2 in RIS register
1	DIO1	W	0h	DIO1 event 0h = No effect 1h = Sets DIO1 in RIS register
0	DIO0	W	0h	DIO0 event 0h = No effect 1h = Sets DIO0 in RIS register

9.3.15 ICLR (Offset = 1048h) [Reset = 00000000h]

ICLR is shown in [Figure 9-18](#) and described in [Table 9-18](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 9-18. ICLR

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 9-18. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	W	0h	DIO31 event 0h = No effect 1h = Clears DIO31 in RIS register
30	DIO30	W	0h	DIO30 event 0h = No effect 1h = Clears DIO30 in RIS register
29	DIO29	W	0h	DIO29 event 0h = No effect 1h = Clears DIO29 in RIS register
28	DIO28	W	0h	DIO28 event 0h = No effect 1h = Clears DIO28 in RIS register
27	DIO27	W	0h	DIO27 event 0h = No effect 1h = Clears DIO27 in RIS register
26	DIO26	W	0h	DIO26 event 0h = No effect 1h = Clears DIO26 in RIS register
25	DIO25	W	0h	DIO25 event 0h = No effect 1h = Clears DIO25 in RIS register
24	DIO24	W	0h	DIO24 event 0h = No effect 1h = Clears DIO24 in RIS register
23	DIO23	W	0h	DIO23 event 0h = No effect 1h = Clears DIO23 in RIS register
22	DIO22	W	0h	DIO22 event 0h = No effect 1h = Clears DIO22 in RIS register

Table 9-18. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	DIO21	W	0h	DIO21 event 0h = No effect 1h = Clears DIO21 in RIS register
20	DIO20	W	0h	DIO20 event 0h = No effect 1h = Clears DIO20 in RIS register
19	DIO19	W	0h	DIO19 event 0h = No effect 1h = Clears DIO19 in RIS register
18	DIO18	W	0h	DIO18 event 0h = No effect 1h = Clears DIO18 in RIS register
17	DIO17	W	0h	DIO17 event 0h = No effect 1h = Clears DIO17 in RIS register
16	DIO16	W	0h	DIO16 event 0h = No effect 1h = Clears DIO16 in RIS register
15	DIO15	W	0h	DIO15 event 0h = No effect 1h = Clears DIO15 in RIS register
14	DIO14	W	0h	DIO14 event 0h = No effect 1h = Clears DIO14 in RIS register
13	DIO13	W	0h	DIO13 event 0h = No effect 1h = Clears DIO13 in RIS register
12	DIO12	W	0h	DIO12 event 0h = No effect 1h = Clears DIO12 in RIS register
11	DIO11	W	0h	DIO11 event 0h = No effect 1h = Clears DIO11 in RIS register
10	DIO10	W	0h	DIO10 event 0h = No effect 1h = Clears DIO10 in RIS register
9	DIO9	W	0h	DIO9 event 0h = No effect 1h = Clears DIO9 in RIS register
8	DIO8	W	0h	DIO8 event 0h = No effect 1h = Clears DIO8 in RIS register
7	DIO7	W	0h	DIO7 event 0h = No effect 1h = Clears DIO7 in RIS register
6	DIO6	W	0h	DIO6 event 0h = No effect 1h = Clears DIO6 in RIS register
5	DIO5	W	0h	DIO5 event 0h = No effect 1h = Clears DIO5 in RIS register
4	DIO4	W	0h	DIO4 event 0h = No effect 1h = Clears DIO4 in RIS register
3	DIO3	W	0h	DIO3 event 0h = No effect 1h = Clears DIO3 in RIS register

Table 9-18. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	DIO2	W	0h	DIO2 event 0h = No effect 1h = Clears DIO2 in RIS register
1	DIO1	W	0h	DIO1 event 0h = No effect 1h = Clears DIO1 in RIS register
0	DIO0	W	0h	DIO0 event 0h = No effect 1h = Clears DIO0 in RIS register

9.3.16 IIDX (Offset = 1050h) [Reset = 00000000h]

IIDX is shown in [Figure 9-19](#) and described in [Table 9-19](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 9-19. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 9-19. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 0h = No bit is set means there is no pending interrupt request 1h = DIO0 interrupt 2h = DIO1 interrupt 3h = DIO2 interrupt 4h = DIO3 interrupt 5h = DIO4 interrupt 6h = DIO5 interrupt 7h = DIO6 interrupt 8h = DIO7 interrupt 9h = DIO8 interrupt Ah = DIO9 interrupt Bh = DIO10 interrupt Ch = DIO11 interrupt Dh = DIO12 interrupt Eh = DIO13 interrupt Fh = DIO14 interrupt 10h = DIO15 interrupt

9.3.17 IMASK (Offset = 1058h) [Reset = 00000000h]

IMASK is shown in [Figure 9-20](#) and described in [Table 9-20](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 9-20. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 9-20. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15	DIO15	R/W	0h	DIO15 event mask 0h = Event is masked 1h = Event is unmasked
14	DIO14	R/W	0h	DIO14 event mask 0h = Event is masked 1h = Event is unmasked
13	DIO13	R/W	0h	DIO13 event mask 0h = Event is masked 1h = Event is unmasked
12	DIO12	R/W	0h	DIO12 event mask 0h = Event is masked 1h = Event is unmasked
11	DIO11	R/W	0h	DIO11 event mask 0h = Event is masked 1h = Event is unmasked
10	DIO10	R/W	0h	DIO10 event mask 0h = Event is masked 1h = Event is unmasked
9	DIO9	R/W	0h	DIO9 event mask 0h = Event is masked 1h = Event is unmasked
8	DIO8	R/W	0h	DIO8 event mask 0h = Event is masked 1h = Event is unmasked
7	DIO7	R/W	0h	DIO7 event mask 0h = Event is masked 1h = Event is unmasked
6	DIO6	R/W	0h	DIO6 event mask 0h = Event is masked 1h = Event is unmasked

Table 9-20. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
5	DIO5	R/W	0h	DIO5 event mask 0h = Event is masked 1h = Event is unmasked
4	DIO4	R/W	0h	DIO4 event mask 0h = Event is masked 1h = Event is unmasked
3	DIO3	R/W	0h	DIO3 event mask 0h = Event is masked 1h = Event is unmasked
2	DIO2	R/W	0h	DIO2 event mask 0h = Event is masked 1h = Event is unmasked
1	DIO1	R/W	0h	DIO1 event mask 0h = Event is masked 1h = Event is unmasked
0	DIO0	R/W	0h	DIO0 event mask 0h = Event is masked 1h = Event is unmasked

9.3.18 RIS (Offset = 1060h) [Reset = 00000000h]

RIS is shown in [Figure 9-21](#) and described in [Table 9-21](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 9-21. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 9-21. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15	DIO15	R	0h	DIO15 event 0h = DIO15 event did not occur 1h = DIO15 event occurred
14	DIO14	R	0h	DIO14 event 0h = DIO14 event did not occur 1h = DIO14 event occurred
13	DIO13	R	0h	DIO13 event 0h = DIO13 event did not occur 1h = DIO13 event occurred
12	DIO12	R	0h	DIO12 event 0h = DIO12 event did not occur 1h = DIO12 event occurred
11	DIO11	R	0h	DIO11 event 0h = DIO11 event did not occur 1h = DIO11 event occurred
10	DIO10	R	0h	DIO10 event 0h = DIO10 event did not occur 1h = DIO10 event occurred
9	DIO9	R	0h	DIO9 event 0h = DIO9 event did not occur 1h = DIO9 event occurred
8	DIO8	R	0h	DIO8 event 0h = DIO8 event did not occur 1h = DIO8 event occurred
7	DIO7	R	0h	DIO7 event 0h = DIO7 event did not occur 1h = DIO7 event occurred

Table 9-21. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
6	DIO6	R	0h	DIO6 event 0h = DIO6 event did not occur 1h = DIO6 event occurred
5	DIO5	R	0h	DIO5 event 0h = DIO5 event did not occur 1h = DIO5 event occurred
4	DIO4	R	0h	DIO4 event 0h = DIO4 event did not occur 1h = DIO4 event occurred
3	DIO3	R	0h	DIO3 event 0h = DIO3 event did not occur 1h = DIO3 event occurred
2	DIO2	R	0h	DIO2 event 0h = DIO2 event did not occur 1h = DIO2 event occurred
1	DIO1	R	0h	DIO1 event 0h = DIO1 event did not occur 1h = DIO1 event occurred
0	DIO0	R	0h	DIO0 event 0h = DIO0 event did not occur 1h = DIO0 event occurred

9.3.19 MIS (Offset = 1068h) [Reset = 00000000h]

MIS is shown in [Figure 9-22](#) and described in [Table 9-22](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 9-22. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 9-22. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15	DIO15	R	0h	DIO15 event 0h = DIO15 event did not occur 1h = DIO15 event occurred
14	DIO14	R	0h	DIO14 event 0h = DIO14 event did not occur 1h = DIO14 event occurred
13	DIO13	R	0h	DIO13 event 0h = DIO13 event did not occur 1h = DIO13 event occurred
12	DIO12	R	0h	DIO12 event 0h = DIO12 event did not occur 1h = DIO12 event occurred
11	DIO11	R	0h	DIO11 event 0h = DIO11 event did not occur 1h = DIO11 event occurred
10	DIO10	R	0h	DIO10 event 0h = DIO10 event did not occur 1h = DIO10 event occurred
9	DIO9	R	0h	DIO9 event 0h = DIO9 event did not occur 1h = DIO9 event occurred
8	DIO8	R	0h	DIO8 event 0h = DIO8 event did not occur 1h = DIO8 event occurred
7	DIO7	R	0h	DIO7 event 0h = DIO7 event did not occur 1h = DIO7 event occurred
6	DIO6	R	0h	DIO6 event 0h = DIO6 event did not occur 1h = DIO6 event occurred

Table 9-22. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
5	DIO5	R	0h	DIO5 event 0h = DIO5 event did not occur 1h = DIO5 event occurred
4	DIO4	R	0h	DIO4 event 0h = DIO4 event did not occur 1h = DIO4 event occurred
3	DIO3	R	0h	DIO3 event 0h = DIO3 event did not occur 1h = DIO3 event occurred
2	DIO2	R	0h	DIO2 event 0h = DIO2 event did not occur 1h = DIO2 event occurred
1	DIO1	R	0h	DIO1 event 0h = DIO1 event did not occur 1h = DIO1 event occurred
0	DIO0	R	0h	DIO0 event 0h = DIO0 event did not occur 1h = DIO0 event occurred

9.3.20 ISET (Offset = 1070h) [Reset = 00000000h]

ISET is shown in [Figure 9-23](#) and described in [Table 9-23](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 9-23. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 9-23. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	W	0h	
15	DIO15	W	0h	DIO15 event 0h = No effect 1h = Sets DIO15 in RIS register
14	DIO14	W	0h	DIO14 event 0h = No effect 1h = Sets DIO14 in RIS register
13	DIO13	W	0h	DIO13 event 0h = No effect 1h = Sets DIO13 in RIS register
12	DIO12	W	0h	DIO12 event 0h = No effect 1h = Sets DIO12 in RIS register
11	DIO11	W	0h	DIO11 event 0h = No effect 1h = Sets DIO11 in RIS register
10	DIO10	W	0h	DIO10 event 0h = No effect 1h = Sets DIO10 in RIS register
9	DIO9	W	0h	DIO9 event 0h = No effect 1h = Sets DIO9 in RIS register
8	DIO8	W	0h	DIO8 event 0h = No effect 1h = Sets DIO8 in RIS register
7	DIO7	W	0h	DIO7 event 0h = No effect 1h = Sets DIO7 in RIS register

Table 9-23. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
6	DIO6	W	0h	DIO6 event 0h = No effect 1h = Sets DIO6 in RIS register
5	DIO5	W	0h	DIO5 event 0h = No effect 1h = Sets DIO5 in RIS register
4	DIO4	W	0h	DIO4 event 0h = No effect 1h = Sets DIO4 in RIS register
3	DIO3	W	0h	DIO3 event 0h = No effect 1h = Sets DIO3 in RIS register
2	DIO2	W	0h	DIO2 event 0h = No effect 1h = Sets DIO2 in RIS register
1	DIO1	W	0h	DIO1 event 0h = No effect 1h = Sets DIO1 in RIS register
0	DIO0	W	0h	DIO0 event 0h = No effect 1h = Sets DIO0 in RIS register

9.3.21 ICLR (Offset = 1078h) [Reset = 00000000h]

ICLR is shown in [Figure 9-24](#) and described in [Table 9-24](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 9-24. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 9-24. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	W	0h	
15	DIO15	W	0h	DIO15 event 0h = No effect 1h = Clears DIO15 in RIS register
14	DIO14	W	0h	DIO14 event 0h = No effect 1h = Clears DIO14 in RIS register
13	DIO13	W	0h	DIO13 event 0h = No effect 1h = Clears DIO13 in RIS register
12	DIO12	W	0h	DIO12 event 0h = No effect 1h = Clears DIO12 in RIS register
11	DIO11	W	0h	DIO11 event 0h = No effect 1h = Clears DIO11 in RIS register
10	DIO10	W	0h	DIO10 event 0h = No effect 1h = Clears DIO10 in RIS register
9	DIO9	W	0h	DIO9 event 0h = No effect 1h = Clears DIO9 in RIS register
8	DIO8	W	0h	DIO8 event 0h = No effect 1h = Clears DIO8 in RIS register
7	DIO7	W	0h	DIO7 event 0h = No effect 1h = Clears DIO7 in RIS register
6	DIO6	W	0h	DIO6 event 0h = No effect 1h = Clears DIO6 in RIS register

Table 9-24. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
5	DIO5	W	0h	DIO5 event 0h = No effect 1h = Clears DIO5 in RIS register
4	DIO4	W	0h	DIO4 event 0h = No effect 1h = Clears DIO4 in RIS register
3	DIO3	W	0h	DIO3 event 0h = No effect 1h = Clears DIO3 in RIS register
2	DIO2	W	0h	DIO2 event 0h = No effect 1h = Clears DIO2 in RIS register
1	DIO1	W	0h	DIO1 event 0h = No effect 1h = Clears DIO1 in RIS register
0	DIO0	W	0h	DIO0 event 0h = No effect 1h = Clears DIO0 in RIS register

9.3.22 IIDX (Offset = 1080h) [Reset = 00000000h]

IIDX is shown in [Figure 9-25](#) and described in [Table 9-25](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 9-25. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 9-25. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 0h = No bit is set means there is no pending interrupt request 1h = DIO0 interrupt 2h = DIO1 interrupt 3h = DIO2 interrupt 4h = DIO3 interrupt 5h = DIO4 interrupt 6h = DIO5 interrupt 7h = DIO6 interrupt 8h = DIO7 interrupt 9h = DIO8 interrupt Ah = DIO9 interrupt Bh = DIO10 interrupt Ch = DIO11 interrupt Dh = DIO12 interrupt Eh = DIO13 interrupt Fh = DIO14 interrupt 10h = DIO15 interrupt

9.3.23 IMASK (Offset = 1088h) [Reset = 00000000h]

IMASK is shown in [Figure 9-26](#) and described in [Table 9-26](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 9-26. IMASK

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							
R/W-0h							

Table 9-26. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	R/W	0h	DIO31 event mask 0h = Event is masked 1h = Event is unmasked
30	DIO30	R/W	0h	DIO30 event mask 0h = Event is masked 1h = Event is unmasked
29	DIO29	R/W	0h	DIO29 event mask 0h = Event is masked 1h = Event is unmasked
28	DIO28	R/W	0h	DIO28 event mask 0h = Event is masked 1h = Event is unmasked
27	DIO27	R/W	0h	DIO27 event mask 0h = Event is masked 1h = Event is unmasked
26	DIO26	R/W	0h	DIO26 event mask 0h = Event is masked 1h = Event is unmasked
25	DIO25	R/W	0h	DIO25 event mask 0h = Event is masked 1h = Event is unmasked
24	DIO24	R/W	0h	DIO24 event mask 0h = Event is masked 1h = Event is unmasked
23	DIO23	R/W	0h	DIO23 event mask 0h = Event is masked 1h = Event is unmasked
22	DIO22	R/W	0h	DIO22 event mask 0h = Event is masked 1h = Event is unmasked

Table 9-26. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	DIO21	R/W	0h	DIO21 event mask 0h = Event is masked 1h = Event is unmasked
20	DIO20	R/W	0h	DIO20 event mask 0h = Event is masked 1h = Event is unmasked
19	DIO19	R/W	0h	DIO19 event mask 0h = Event is masked 1h = Event is unmasked
18	DIO18	R/W	0h	DIO18 event mask 0h = Event is masked 1h = Event is unmasked
17	DIO17	R/W	0h	DIO17 event mask 0h = Event is masked 1h = Event is unmasked
16	DIO16	R/W	0h	DIO16 event mask 0h = Event is masked 1h = Event is unmasked
15-0	RESERVED	R/W	0h	

9.3.24 RIS (Offset = 1090h) [Reset = 00000000h]

RIS is shown in [Figure 9-27](#) and described in [Table 9-27](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 9-27. RIS

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							
R-0h							

Table 9-27. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	R	0h	DIO31 event 0h = DIO31 event did not occur 1h = DIO31 event occurred
30	DIO30	R	0h	DIO30 event 0h = DIO30 event did not occur 1h = DIO30 event occurred
29	DIO29	R	0h	DIO29 event 0h = DIO29 event did not occur 1h = DIO29 event occurred
28	DIO28	R	0h	DIO28 event 0h = DIO28 event did not occur 1h = DIO28 event occurred
27	DIO27	R	0h	DIO27 event 0h = DIO27 event did not occur 1h = DIO27 event occurred
26	DIO26	R	0h	DIO26 event 0h = DIO26 event did not occur 1h = DIO26 event occurred
25	DIO25	R	0h	DIO25 event 0h = DIO25 event did not occur 1h = DIO25 event occurred
24	DIO24	R	0h	DIO24 event 0h = DIO24 event did not occur 1h = DIO24 event occurred
23	DIO23	R	0h	DIO23 event 0h = DIO23 event did not occur 1h = DIO23 event occurred
22	DIO22	R	0h	DIO22 event 0h = DIO22 event did not occur 1h = DIO22 event occurred

Table 9-27. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	DIO21	R	0h	DIO21 event 0h = DIO21 event did not occur 1h = DIO21 event occurred
20	DIO20	R	0h	DIO20 event 0h = DIO20 event did not occur 1h = DIO20 event occurred
19	DIO19	R	0h	DIO19 event 0h = DIO19 event did not occur 1h = DIO19 event occurred
18	DIO18	R	0h	DIO18 event 0h = DIO18 event did not occur 1h = DIO18 event occurred
17	DIO17	R	0h	DIO17 event 0h = DIO17 event did not occur 1h = DIO17 event occurred
16	DIO16	R	0h	DIO16 event 0h = DIO16 event did not occur 1h = DIO16 event occurred
15-0	RESERVED	R	0h	

9.3.25 MIS (Offset = 1098h) [Reset = 00000000h]

MIS is shown in [Figure 9-28](#) and described in [Table 9-28](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 9-28. MIS

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							
R-0h							

Table 9-28. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	R	0h	DIO31 event 0h = DIO31 event did not occur 1h = DIO31 event occurred
30	DIO30	R	0h	DIO30 event 0h = DIO30 event did not occur 1h = DIO30 event occurred
29	DIO29	R	0h	DIO29 event 0h = DIO29 event did not occur 1h = DIO29 event occurred
28	DIO28	R	0h	DIO28 event 0h = DIO28 event did not occur 1h = DIO28 event occurred
27	DIO27	R	0h	DIO27 event 0h = DIO27 event did not occur 1h = DIO27 event occurred
26	DIO26	R	0h	DIO26 event 0h = DIO26 event did not occur 1h = DIO26 event occurred
25	DIO25	R	0h	DIO25 event 0h = DIO25 event did not occur 1h = DIO25 event occurred
24	DIO24	R	0h	DIO24 event 0h = DIO24 event did not occur 1h = DIO24 event occurred
23	DIO23	R	0h	DIO23 event 0h = DIO23 event did not occur 1h = DIO23 event occurred
22	DIO22	R	0h	DIO22 event 0h = DIO22 event did not occur 1h = DIO22 event occurred

Table 9-28. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	DIO21	R	0h	DIO21 event 0h = DIO21 event did not occur 1h = DIO21 event occurred
20	DIO20	R	0h	DIO20 event 0h = DIO20 event did not occur 1h = DIO20 event occurred
19	DIO19	R	0h	DIO19 event 0h = DIO19 event did not occur 1h = DIO19 event occurred
18	DIO18	R	0h	DIO18 event 0h = DIO18 event did not occur 1h = DIO18 event occurred
17	DIO17	R	0h	DIO17 event 0h = DIO17 event did not occur 1h = DIO17 event occurred
16	DIO16	R	0h	DIO16 event 0h = DIO16 event did not occur 1h = DIO16 event occurred
15-0	RESERVED	R	0h	

9.3.26 ISET (Offset = 10A0h) [Reset = 00000000h]

ISET is shown in [Figure 9-29](#) and described in [Table 9-29](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 9-29. ISET

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED							
W-0h							

Table 9-29. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	W	0h	DIO31 event 0h = No effect 1h = Sets DIO31 in RIS register
30	DIO30	W	0h	DIO30 event 0h = No effect 1h = Sets DIO30 in RIS register
29	DIO29	W	0h	DIO29 event 0h = No effect 1h = Sets DIO29 in RIS register
28	DIO28	W	0h	DIO28 event 0h = No effect 1h = Sets DIO28 in RIS register
27	DIO27	W	0h	DIO27 event 0h = No effect 1h = Sets DIO27 in RIS register
26	DIO26	W	0h	DIO26 event 0h = No effect 1h = Sets DIO26 in RIS register
25	DIO25	W	0h	DIO25 event 0h = No effect 1h = Sets DIO25 in RIS register
24	DIO24	W	0h	DIO24 event 0h = No effect 1h = Sets DIO24 in RIS register
23	DIO23	W	0h	DIO23 event 0h = No effect 1h = Sets DIO23 in RIS register
22	DIO22	W	0h	DIO22 event 0h = No effect 1h = Sets DIO22 in RIS register

Table 9-29. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	DIO21	W	0h	DIO21 event 0h = No effect 1h = Sets DIO21 in RIS register
20	DIO20	W	0h	DIO20 event 0h = No effect 1h = Sets DIO20 in RIS register
19	DIO19	W	0h	DIO19 event 0h = No effect 1h = Sets DIO19 in RIS register
18	DIO18	W	0h	DIO18 event 0h = No effect 1h = Sets DIO18 in RIS register
17	DIO17	W	0h	DIO17 event 0h = No effect 1h = Sets DIO17 in RIS register
16	DIO16	W	0h	DIO16 event 0h = No effect 1h = Sets DIO16 in RIS register
15-0	RESERVED	W	0h	

9.3.27 ICLR (Offset = 10A8h) [Reset = 00000000h]

ICLR is shown in [Figure 9-30](#) and described in [Table 9-30](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 9-30. ICLR

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED							
W-0h							

Table 9-30. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	W	0h	DIO31 event 0h = No effect 1h = Clears DIO31 in RIS register
30	DIO30	W	0h	DIO30 event 0h = No effect 1h = Clears DIO30 in RIS register
29	DIO29	W	0h	DIO29 event 0h = No effect 1h = Clears DIO29 in RIS register
28	DIO28	W	0h	DIO28 event 0h = No effect 1h = Clears DIO28 in RIS register
27	DIO27	W	0h	DIO27 event 0h = No effect 1h = Clears DIO27 in RIS register
26	DIO26	W	0h	DIO26 event 0h = No effect 1h = Clears DIO26 in RIS register
25	DIO25	W	0h	DIO25 event 0h = No effect 1h = Clears DIO25 in RIS register
24	DIO24	W	0h	DIO24 event 0h = No effect 1h = Clears DIO24 in RIS register
23	DIO23	W	0h	DIO23 event 0h = No effect 1h = Clears DIO23 in RIS register
22	DIO22	W	0h	DIO22 event 0h = No effect 1h = Clears DIO22 in RIS register

Table 9-30. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	DIO21	W	0h	DIO21 event 0h = No effect 1h = Clears DIO21 in RIS register
20	DIO20	W	0h	DIO20 event 0h = No effect 1h = Clears DIO20 in RIS register
19	DIO19	W	0h	DIO19 event 0h = No effect 1h = Clears DIO19 in RIS register
18	DIO18	W	0h	DIO18 event 0h = No effect 1h = Clears DIO18 in RIS register
17	DIO17	W	0h	DIO17 event 0h = No effect 1h = Clears DIO17 in RIS register
16	DIO16	W	0h	DIO16 event 0h = No effect 1h = Clears DIO16 in RIS register
15-0	RESERVED	W	0h	

9.3.28 EVT_MODE (Offset = 10E0h) [Reset = 00000029h]

EVT_MODE is shown in [Figure 9-31](#) and described in [Table 9-31](#).

Return to the [Summary Table](#).

Event mode register. It is used to select whether each line is disabled, in software mode (software clears the RIS) or in hardware mode (hardware clears the RIS)

Figure 9-31. EVT_MODE

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED		EVT2_CFG		EVT1_CFG		INT0_CFG	
R/W-		R-2h		R-2h		R-1h	

Table 9-31. EVT_MODE Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R/W	0h	
5-4	EVT2_CFG	R	2h	Event line mode select for event corresponding to none.GEN_EVENT1 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
3-2	EVT1_CFG	R	2h	Event line mode select for event corresponding to none.GEN_EVENT0 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
1-0	INT0_CFG	R	1h	Event line mode select for event corresponding to none.CPU_INT 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.

9.3.29 DESC (Offset = 10FCh) [Reset = 16110000h]

DESC is shown in [Figure 9-32](#) and described in [Table 9-32](#).

Return to the [Summary Table](#).

This register identifies the peripheral and its exact version.

Figure 9-32. DESC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODULEID															
R-1611h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FEATUREVER				RESERVED				MAJREV				MINREV			
R-				R-				R-				R-			

Table 9-32. DESC Field Descriptions

Bit	Field	Type	Reset	Description
31-16	MODULEID	R	1611h	Module identification contains a unique peripheral identification number. The assignments are maintained in a central database for all of the platform modules to ensure uniqueness. 0h = Smallest value FFFFh = Highest possible value
15-12	FEATUREVER	R	0h	Feature Set for the module *instance* 0h = Smallest value Fh = Highest possible value
11-8	RESERVED	R	0h	
7-4	MAJREV	R	0h	Major rev of the IP 0h = Smallest value Fh = Highest possible value
3-0	MINREV	R	0h	Minor rev of the IP 0h = Smallest value Fh = Highest possible value

9.3.30 DOUT3_0 (Offset = 1200h) [Reset = 00000000h]

DOUT3_0 is shown in [Figure 9-33](#) and described in [Table 9-33](#).

Return to the [Summary Table](#).

Data output for pins configured as DIO3 to DIO0. This is an alias register for byte access to bits 3 to 0 in DOUT31_0 register.

Figure 9-33. DOUT3_0

31	30	29	28	27	26	25	24
RESERVED							DIO3
W-0h							W-0h
23	22	21	20	19	18	17	16
RESERVED							DIO2
W-0h							W-0h
15	14	13	12	11	10	9	8
RESERVED							DIO1
W-0h							W-0h
7	6	5	4	3	2	1	0
RESERVED							DIO0
W-0h							W-0h

Table 9-33. DOUT3_0 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	W	0h	
24	DIO3	W	0h	This bit sets the value of the pin configured as DIO3 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
23-17	RESERVED	W	0h	
16	DIO2	W	0h	This bit sets the value of the pin configured as DIO2 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
15-9	RESERVED	W	0h	
8	DIO1	W	0h	This bit sets the value of the pin configured as DIO1 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
7-1	RESERVED	W	0h	
0	DIO0	W	0h	This bit sets the value of the pin configured as DIO0 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1

9.3.31 DOUT7_4 (Offset = 1204h) [Reset = 00000000h]

DOUT7_4 is shown in [Figure 9-34](#) and described in [Table 9-34](#).

Return to the [Summary Table](#).

Data output for pins configured as DIO7 to DIO4. This is an alias register for byte access to bits 7 to 4 in DOUT31_0 register.

Figure 9-34. DOUT7_4

31	30	29	28	27	26	25	24
RESERVED							DIO7
W-0h							W-0h
23	22	21	20	19	18	17	16
RESERVED							DIO6
W-0h							W-0h
15	14	13	12	11	10	9	8
RESERVED							DIO5
W-0h							W-0h
7	6	5	4	3	2	1	0
RESERVED							DIO4
W-0h							W-0h

Table 9-34. DOUT7_4 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	W	0h	
24	DIO7	W	0h	This bit sets the value of the pin configured as DIO7 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
23-17	RESERVED	W	0h	
16	DIO6	W	0h	This bit sets the value of the pin configured as DIO6 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
15-9	RESERVED	W	0h	
8	DIO5	W	0h	This bit sets the value of the pin configured as DIO5 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
7-1	RESERVED	W	0h	
0	DIO4	W	0h	This bit sets the value of the pin configured as DIO4 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1

9.3.32 DOUT11_8 (Offset = 1208h) [Reset = 00000000h]

DOUT11_8 is shown in [Figure 9-35](#) and described in [Table 9-35](#).

Return to the [Summary Table](#).

Data output for pins configured as DIO11 to DIO8. This is an alias register for byte access to bits 11 to 8 in DOUT31_0 register.

Figure 9-35. DOUT11_8

31	30	29	28	27	26	25	24
RESERVED							DIO11
W-0h							W-0h
23	22	21	20	19	18	17	16
RESERVED							DIO10
W-0h							W-0h
15	14	13	12	11	10	9	8
RESERVED							DIO9
W-0h							W-0h
7	6	5	4	3	2	1	0
RESERVED							DIO8
W-0h							W-0h

Table 9-35. DOUT11_8 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	W	0h	
24	DIO11	W	0h	This bit sets the value of the pin configured as DIO11 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
23-17	RESERVED	W	0h	
16	DIO10	W	0h	This bit sets the value of the pin configured as DIO10 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
15-9	RESERVED	W	0h	
8	DIO9	W	0h	This bit sets the value of the pin configured as DIO9 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
7-1	RESERVED	W	0h	
0	DIO8	W	0h	This bit sets the value of the pin configured as DIO8 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1

9.3.33 DOUT15_12 (Offset = 120Ch) [Reset = 00000000h]

DOUT15_12 is shown in [Figure 9-36](#) and described in [Table 9-36](#).

Return to the [Summary Table](#).

Data output for pins configured as DIO15 to DIO12. This is an alias register for byte access to bits 15 to 12 in DOUT31_0 register.

Figure 9-36. DOUT15_12

31	30	29	28	27	26	25	24
RESERVED							DIO15
W-0h							W-0h
23	22	21	20	19	18	17	16
RESERVED							DIO14
W-0h							W-0h
15	14	13	12	11	10	9	8
RESERVED							DIO13
W-0h							W-0h
7	6	5	4	3	2	1	0
RESERVED							DIO12
W-0h							W-0h

Table 9-36. DOUT15_12 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	W	0h	
24	DIO15	W	0h	This bit sets the value of the pin configured as DIO15 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
23-17	RESERVED	W	0h	
16	DIO14	W	0h	This bit sets the value of the pin configured as DIO14 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
15-9	RESERVED	W	0h	
8	DIO13	W	0h	This bit sets the value of the pin configured as DIO13 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
7-1	RESERVED	W	0h	
0	DIO12	W	0h	This bit sets the value of the pin configured as DIO12 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1

9.3.34 DOUT19_16 (Offset = 1210h) [Reset = 00000000h]

DOUT19_16 is shown in [Figure 9-37](#) and described in [Table 9-37](#).

Return to the [Summary Table](#).

Data output for pins configured as DIO19 to DIO16. This is an alias register for byte access to bits 19 to 16 in DOUT31_0 register.

Figure 9-37. DOUT19_16

31	30	29	28	27	26	25	24
RESERVED							DIO19
W-0h							W-0h
23	22	21	20	19	18	17	16
RESERVED							DIO18
W-0h							W-0h
15	14	13	12	11	10	9	8
RESERVED							DIO17
W-0h							W-0h
7	6	5	4	3	2	1	0
RESERVED							DIO16
W-0h							W-0h

Table 9-37. DOUT19_16 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	W	0h	
24	DIO19	W	0h	This bit sets the value of the pin configured as DIO19 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
23-17	RESERVED	W	0h	
16	DIO18	W	0h	This bit sets the value of the pin configured as DIO18 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
15-9	RESERVED	W	0h	
8	DIO17	W	0h	This bit sets the value of the pin configured as DIO17 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
7-1	RESERVED	W	0h	
0	DIO16	W	0h	This bit sets the value of the pin configured as DIO16 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1

9.3.35 DOUT23_20 (Offset = 1214h) [Reset = 00000000h]

DOUT23_20 is shown in [Figure 9-38](#) and described in [Table 9-38](#).

Return to the [Summary Table](#).

Data output for pins configured as DIO23 to DIO20. This is an alias register for byte access to bits 23 to 20 in DOUT31_0 register.

Figure 9-38. DOUT23_20

31	30	29	28	27	26	25	24
RESERVED							DIO23
W-0h							W-0h
23	22	21	20	19	18	17	16
RESERVED							DIO22
W-0h							W-0h
15	14	13	12	11	10	9	8
RESERVED							DIO21
W-0h							W-0h
7	6	5	4	3	2	1	0
RESERVED							DIO20
W-0h							W-0h

Table 9-38. DOUT23_20 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	W	0h	
24	DIO23	W	0h	This bit sets the value of the pin configured as DIO23 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
23-17	RESERVED	W	0h	
16	DIO22	W	0h	This bit sets the value of the pin configured as DIO22 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
15-9	RESERVED	W	0h	
8	DIO21	W	0h	This bit sets the value of the pin configured as DIO21 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
7-1	RESERVED	W	0h	
0	DIO20	W	0h	This bit sets the value of the pin configured as DIO20 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1

9.3.36 DOUT27_24 (Offset = 1218h) [Reset = 00000000h]

DOUT27_24 is shown in [Figure 9-39](#) and described in [Table 9-39](#).

Return to the [Summary Table](#).

Data output for pins configured as DIO27 to DIO24. This is an alias register for byte access to bits 27 to 24 in DOUT31_0 register.

Figure 9-39. DOUT27_24

31	30	29	28	27	26	25	24
RESERVED							DIO27
W-0h							W-0h
23	22	21	20	19	18	17	16
RESERVED							DIO26
W-0h							W-0h
15	14	13	12	11	10	9	8
RESERVED							DIO25
W-0h							W-0h
7	6	5	4	3	2	1	0
RESERVED							DIO24
W-0h							W-0h

Table 9-39. DOUT27_24 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	W	0h	
24	DIO27	W	0h	This bit sets the value of the pin configured as DIO27 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
23-17	RESERVED	W	0h	
16	DIO26	W	0h	This bit sets the value of the pin configured as DIO26 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
15-9	RESERVED	W	0h	
8	DIO25	W	0h	This bit sets the value of the pin configured as DIO25 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
7-1	RESERVED	W	0h	
0	DIO24	W	0h	This bit sets the value of the pin configured as DIO24 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1

9.3.37 DOUT31_28 (Offset = 121Ch) [Reset = 00000000h]

DOUT31_28 is shown in [Figure 9-40](#) and described in [Table 9-40](#).

Return to the [Summary Table](#).

Data output for pins configured as DIO31 to DIO28. This is an alias register for byte access to bits 31 to 28 in DOUT31_0 register.

Figure 9-40. DOUT31_28

31	30	29	28	27	26	25	24
RESERVED							DIO31
W-0h							W-0h
23	22	21	20	19	18	17	16
RESERVED							DIO30
W-0h							W-0h
15	14	13	12	11	10	9	8
RESERVED							DIO29
W-0h							W-0h
7	6	5	4	3	2	1	0
RESERVED							DIO28
W-0h							W-0h

Table 9-40. DOUT31_28 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	W	0h	
24	DIO31	W	0h	This bit sets the value of the pin configured as DIO31 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
23-17	RESERVED	W	0h	
16	DIO30	W	0h	This bit sets the value of the pin configured as DIO30 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
15-9	RESERVED	W	0h	
8	DIO29	W	0h	This bit sets the value of the pin configured as DIO29 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
7-1	RESERVED	W	0h	
0	DIO28	W	0h	This bit sets the value of the pin configured as DIO28 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1

9.3.38 DOUT31_0 (Offset = 1280h) [Reset = 00000000h]

DOUT31_0 is shown in [Figure 9-41](#) and described in [Table 9-41](#).

Return to the [Summary Table](#).

Data output for pins configured as DIO31 to DIO0.

Figure 9-41. DOUT31_0

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 9-41. DOUT31_0 Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	R/W	0h	This bit sets the value of the pin configured as DIO31 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
30	DIO30	R/W	0h	This bit sets the value of the pin configured as DIO30 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
29	DIO29	R/W	0h	This bit sets the value of the pin configured as DIO29 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
28	DIO28	R/W	0h	This bit sets the value of the pin configured as DIO28 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
27	DIO27	R/W	0h	This bit sets the value of the pin configured as DIO27 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
26	DIO26	R/W	0h	This bit sets the value of the pin configured as DIO26 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
25	DIO25	R/W	0h	This bit sets the value of the pin configured as DIO25 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
24	DIO24	R/W	0h	This bit sets the value of the pin configured as DIO24 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1

Table 9-41. DOUT31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
23	DIO23	R/W	0h	This bit sets the value of the pin configured as DIO23 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
22	DIO22	R/W	0h	This bit sets the value of the pin configured as DIO22 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
21	DIO21	R/W	0h	This bit sets the value of the pin configured as DIO21 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
20	DIO20	R/W	0h	This bit sets the value of the pin configured as DIO20 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
19	DIO19	R/W	0h	This bit sets the value of the pin configured as DIO19 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
18	DIO18	R/W	0h	This bit sets the value of the pin configured as DIO18 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
17	DIO17	R/W	0h	This bit sets the value of the pin configured as DIO17 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
16	DIO16	R/W	0h	This bit sets the value of the pin configured as DIO16 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
15	DIO15	R/W	0h	This bit sets the value of the pin configured as DIO15 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
14	DIO14	R/W	0h	This bit sets the value of the pin configured as DIO14 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
13	DIO13	R/W	0h	This bit sets the value of the pin configured as DIO13 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
12	DIO12	R/W	0h	This bit sets the value of the pin configured as DIO12 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
11	DIO11	R/W	0h	This bit sets the value of the pin configured as DIO11 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
10	DIO10	R/W	0h	This bit sets the value of the pin configured as DIO10 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1

Table 9-41. DOUT31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
9	DIO9	R/W	0h	This bit sets the value of the pin configured as DIO9 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
8	DIO8	R/W	0h	This bit sets the value of the pin configured as DIO8 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
7	DIO7	R/W	0h	This bit sets the value of the pin configured as DIO7 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
6	DIO6	R/W	0h	This bit sets the value of the pin configured as DIO6 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
5	DIO5	R/W	0h	This bit sets the value of the pin configured as DIO5 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
4	DIO4	R/W	0h	This bit sets the value of the pin configured as DIO4 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
3	DIO3	R/W	0h	This bit sets the value of the pin configured as DIO3 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
2	DIO2	R/W	0h	This bit sets the value of the pin configured as DIO2 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
1	DIO1	R/W	0h	This bit sets the value of the pin configured as DIO1 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1
0	DIO0	R/W	0h	This bit sets the value of the pin configured as DIO0 when the output is enabled through DOE31_0 register. 0h = Output is set to 0 1h = Output is set to 1

9.3.39 DOUTSET31_0 (Offset = 1290h) [Reset = 00000000h]

DOUTSET31_0 is shown in [Figure 9-42](#) and described in [Table 9-42](#).

Return to the [Summary Table](#).

Writing 1 to a bit position in this register sets the corresponding bit in the DOUT31_0 register.

Figure 9-42. DOUTSET31_0

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 9-42. DOUTSET31_0 Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	W	0h	Writing 1 to this bit sets the DIO31 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO31 in DOUT31_0
30	DIO30	W	0h	Writing 1 to this bit sets the DIO30 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO30 in DOUT31_0
29	DIO29	W	0h	Writing 1 to this bit sets the DIO29 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO29 in DOUT31_0
28	DIO28	W	0h	Writing 1 to this bit sets the DIO28 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO28 in DOUT31_0
27	DIO27	W	0h	Writing 1 to this bit sets the DIO27 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO27 in DOUT31_0
26	DIO26	W	0h	Writing 1 to this bit sets the DIO26 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO26 in DOUT31_0
25	DIO25	W	0h	Writing 1 to this bit sets the DIO25 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO25 in DOUT31_0
24	DIO24	W	0h	Writing 1 to this bit sets the DIO24 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO24 in DOUT31_0

Table 9-42. DOUTSET31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
23	DIO23	W	0h	Writing 1 to this bit sets the DIO23 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO23 in DOUT31_0
22	DIO22	W	0h	Writing 1 to this bit sets the DIO22 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO22 in DOUT31_0
21	DIO21	W	0h	Writing 1 to this bit sets the DIO21 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO21 in DOUT31_0
20	DIO20	W	0h	Writing 1 to this bit sets the DIO20 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO20 in DOUT31_0
19	DIO19	W	0h	Writing 1 to this bit sets the DIO19 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO19 in DOUT31_0
18	DIO18	W	0h	Writing 1 to this bit sets the DIO18 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO18 in DOUT31_0
17	DIO17	W	0h	Writing 1 to this bit sets the DIO17 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO17 in DOUT31_0
16	DIO16	W	0h	Writing 1 to this bit sets the DIO16 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO16 in DOUT31_0
15	DIO15	W	0h	Writing 1 to this bit sets the DIO15 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO15 in DOUT31_0
14	DIO14	W	0h	Writing 1 to this bit sets the DIO14 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO14 in DOUT31_0
13	DIO13	W	0h	Writing 1 to this bit sets the DIO13 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO13 in DOUT31_0
12	DIO12	W	0h	Writing 1 to this bit sets the DIO12 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO12 in DOUT31_0
11	DIO11	W	0h	Writing 1 to this bit sets the DIO11 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO11 in DOUT31_0
10	DIO10	W	0h	Writing 1 to this bit sets the DIO10 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO10 in DOUT31_0

Table 9-42. DOUTSET31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
9	DIO9	W	0h	Writing 1 to this bit sets the DIO9 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO9 in DOUT31_0
8	DIO8	W	0h	Writing 1 to this bit sets the DIO8 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO8 in DOUT31_0
7	DIO7	W	0h	Writing 1 to this bit sets the DIO7 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO7 in DOUT31_0
6	DIO6	W	0h	Writing 1 to this bit sets the DIO6 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO6 in DOUT31_0
5	DIO5	W	0h	Writing 1 to this bit sets the DIO5 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO5 in DOUT31_0
4	DIO4	W	0h	Writing 1 to this bit sets the DIO4 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO4 in DOUT31_0
3	DIO3	W	0h	Writing 1 to this bit sets the DIO3 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO3 in DOUT31_0
2	DIO2	W	0h	Writing 1 to this bit sets the DIO2 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO2 in DOUT31_0
1	DIO1	W	0h	Writing 1 to this bit sets the DIO1 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO1 in DOUT31_0
0	DIO0	W	0h	Writing 1 to this bit sets the DIO0 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO0 in DOUT31_0

9.3.40 DOUTCLR31_0 (Offset = 12A0h) [Reset = 00000000h]

DOUTCLR31_0 is shown in [Figure 9-43](#) and described in [Table 9-43](#).

Return to the [Summary Table](#).

Writing 1 to a bit position in this register clears the corresponding bit in the DOUT31_0 register.

Figure 9-43. DOUTCLR31_0

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 9-43. DOUTCLR31_0 Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	W	0h	Writing 1 to this bit clears the DIO31 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO31 in DOUT31_0
30	DIO30	W	0h	Writing 1 to this bit clears the DIO30 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO30 in DOUT31_0
29	DIO29	W	0h	Writing 1 to this bit clears the DIO29 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO29 in DOUT31_0
28	DIO28	W	0h	Writing 1 to this bit clears the DIO28 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO28 in DOUT31_0
27	DIO27	W	0h	Writing 1 to this bit clears the DIO27 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO27 in DOUT31_0
26	DIO26	W	0h	Writing 1 to this bit clears the DIO26 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO26 in DOUT31_0
25	DIO25	W	0h	Writing 1 to this bit clears the DIO25 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO25 in DOUT31_0
24	DIO24	W	0h	Writing 1 to this bit clears the DIO24 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO24 in DOUT31_0

Table 9-43. DOUTCLR31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
23	DIO23	W	0h	Writing 1 to this bit clears the DIO23 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO23 in DOUT31_0
22	DIO22	W	0h	Writing 1 to this bit clears the DIO22 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO22 in DOUT31_0
21	DIO21	W	0h	Writing 1 to this bit clears the DIO21 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO21 in DOUT31_0
20	DIO20	W	0h	Writing 1 to this bit clears the DIO20 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO20 in DOUT31_0
19	DIO19	W	0h	Writing 1 to this bit clears the DIO19 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO19 in DOUT31_0
18	DIO18	W	0h	Writing 1 to this bit clears the DIO18 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO18 in DOUT31_0
17	DIO17	W	0h	Writing 1 to this bit clears the DIO17 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO17 in DOUT31_0
16	DIO16	W	0h	Writing 1 to this bit clears the DIO16 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO16 in DOUT31_0
15	DIO15	W	0h	Writing 1 to this bit clears the DIO15 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO15 in DOUT31_0
14	DIO14	W	0h	Writing 1 to this bit clears the DIO14 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO14 in DOUT31_0
13	DIO13	W	0h	Writing 1 to this bit clears the DIO13 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO13 in DOUT31_0
12	DIO12	W	0h	Writing 1 to this bit clears the DIO12 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO12 in DOUT31_0
11	DIO11	W	0h	Writing 1 to this bit clears the DIO11 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO11 in DOUT31_0
10	DIO10	W	0h	Writing 1 to this bit clears the DIO10 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO10 in DOUT31_0

Table 9-43. DOUTCLR31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
9	DIO9	W	0h	Writing 1 to this bit clears the DIO9 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO9 in DOUT31_0
8	DIO8	W	0h	Writing 1 to this bit clears the DIO8 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO8 in DOUT31_0
7	DIO7	W	0h	Writing 1 to this bit clears the DIO7 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO7 in DOUT31_0
6	DIO6	W	0h	Writing 1 to this bit clears the DIO6 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO6 in DOUT31_0
5	DIO5	W	0h	Writing 1 to this bit clears the DIO5 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO5 in DOUT31_0
4	DIO4	W	0h	Writing 1 to this bit clears the DIO4 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO4 in DOUT31_0
3	DIO3	W	0h	Writing 1 to this bit clears the DIO3 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO3 in DOUT31_0
2	DIO2	W	0h	Writing 1 to this bit clears the DIO2 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO2 in DOUT31_0
1	DIO1	W	0h	Writing 1 to this bit clears the DIO1 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO1 in DOUT31_0
0	DIO0	W	0h	Writing 1 to this bit clears the DIO0 bit in the DOUT31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO0 in DOUT31_0

9.3.41 DOUTTGL31_0 (Offset = 12B0h) [Reset = 00000000h]

DOUTTGL31_0 is shown in [Figure 9-44](#) and described in [Table 9-44](#).

Return to the [Summary Table](#).

Writing 1 to a bit position in this register will invert the corresponding DIO output.

Figure 9-44. DOUTTGL31_0

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 9-44. DOUTTGL31_0 Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	W	0h	This bit is used to toggle DIO31 output. 0h = No effect 1h = Toggle output
30	DIO30	W	0h	This bit is used to toggle DIO30 output. 0h = No effect 1h = Toggle output
29	DIO29	W	0h	This bit is used to toggle DIO29 output. 0h = No effect 1h = Toggle output
28	DIO28	W	0h	This bit is used to toggle DIO28 output. 0h = No effect 1h = Toggle output
27	DIO27	W	0h	This bit is used to toggle DIO27 output. 0h = No effect 1h = Toggle output
26	DIO26	W	0h	This bit is used to toggle DIO26 output. 0h = No effect 1h = Toggle output
25	DIO25	W	0h	This bit is used to toggle DIO25 output. 0h = No effect 1h = Toggle output
24	DIO24	W	0h	This bit is used to toggle DIO24 output. 0h = No effect 1h = Toggle output
23	DIO23	W	0h	This bit is used to toggle DIO23 output. 0h = No effect 1h = Toggle output
22	DIO22	W	0h	This bit is used to toggle DIO22 output. 0h = No effect 1h = Toggle output

Table 9-44. DOUTTGL31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	DIO21	W	0h	This bit is used to toggle DIO21 output. 0h = No effect 1h = Toggle output
20	DIO20	W	0h	This bit is used to toggle DIO20 output. 0h = No effect 1h = Toggle output
19	DIO19	W	0h	This bit is used to toggle DIO19 output. 0h = No effect 1h = Toggle output
18	DIO18	W	0h	This bit is used to toggle DIO18 output. 0h = No effect 1h = Toggle output
17	DIO17	W	0h	This bit is used to toggle DIO17 output. 0h = No effect 1h = Toggle output
16	DIO16	W	0h	This bit is used to toggle DIO16 output. 0h = No effect 1h = Toggle output
15	DIO15	W	0h	This bit is used to toggle DIO15 output. 0h = No effect 1h = Toggle output
14	DIO14	W	0h	This bit is used to toggle DIO14 output. 0h = No effect 1h = Toggle output
13	DIO13	W	0h	This bit is used to toggle DIO13 output. 0h = No effect 1h = Toggle output
12	DIO12	W	0h	This bit is used to toggle DIO12 output. 0h = No effect 1h = Toggle output
11	DIO11	W	0h	This bit is used to toggle DIO11 output. 0h = No effect 1h = Toggle output
10	DIO10	W	0h	This bit is used to toggle DIO10 output. 0h = No effect 1h = Toggle output
9	DIO9	W	0h	This bit is used to toggle DIO9 output. 0h = No effect 1h = Toggle output
8	DIO8	W	0h	This bit is used to toggle DIO8 output. 0h = No effect 1h = Toggle output
7	DIO7	W	0h	This bit is used to toggle DIO7 output. 0h = No effect 1h = Toggle output
6	DIO6	W	0h	This bit is used to toggle DIO6 output. 0h = No effect 1h = Toggle output
5	DIO5	W	0h	This bit is used to toggle DIO5 output. 0h = No effect 1h = Toggle output
4	DIO4	W	0h	This bit is used to toggle DIO4 output. 0h = No effect 1h = Toggle output
3	DIO3	W	0h	This bit is used to toggle DIO3 output. 0h = No effect 1h = Toggle output

Table 9-44. DOUTTGL31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	DIO2	W	0h	This bit is used to toggle DIO2 output. 0h = No effect 1h = Toggle output
1	DIO1	W	0h	This bit is used to toggle DIO1 output. 0h = No effect 1h = Toggle output
0	DIO0	W	0h	This bit is used to toggle DIO0 output. 0h = No effect 1h = Toggle output

9.3.42 DOE31_0 (Offset = 12C0h) [Reset = 00000000h]

DOE31_0 is shown in [Figure 9-45](#) and described in [Table 9-45](#).

Return to the [Summary Table](#).

This register is used to enable the data outputs for DIO31 to DIO0.

Figure 9-45. DOE31_0

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 9-45. DOE31_0 Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	R/W	0h	Enables data output for DIO31. 0h = Output disabled 1h = Output enabled
30	DIO30	R/W	0h	Enables data output for DIO30. 0h = Output disabled 1h = Output enabled
29	DIO29	R/W	0h	Enables data output for DIO29. 0h = Output disabled 1h = Output enabled
28	DIO28	R/W	0h	Enables data output for DIO28. 0h = Output disabled 1h = Output enabled
27	DIO27	R/W	0h	Enables data output for DIO27. 0h = Output disabled 1h = Output enabled
26	DIO26	R/W	0h	Enables data output for DIO26. 0h = Output disabled 1h = Output enabled
25	DIO25	R/W	0h	Enables data output for DIO25. 0h = Output disabled 1h = Output enabled
24	DIO24	R/W	0h	Enables data output for DIO24. 0h = Output disabled 1h = Output enabled
23	DIO23	R/W	0h	Enables data output for DIO23. 0h = Output disabled 1h = Output enabled
22	DIO22	R/W	0h	Enables data output for DIO22. 0h = Output disabled 1h = Output enabled

Table 9-45. DOE31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	DIO21	R/W	0h	Enables data output for DIO21. 0h = Output disabled 1h = Output enabled
20	DIO20	R/W	0h	Enables data output for DIO20. 0h = Output disabled 1h = Output enabled
19	DIO19	R/W	0h	Enables data output for DIO19. 0h = Output disabled 1h = Output enabled
18	DIO18	R/W	0h	Enables data output for DIO18. 0h = Output disabled 1h = Output enabled
17	DIO17	R/W	0h	Enables data output for DIO17. 0h = Output disabled 1h = Output enabled
16	DIO16	R/W	0h	Enables data output for DIO16. 0h = Output disabled 1h = Output enabled
15	DIO15	R/W	0h	Enables data output for DIO15. 0h = Output disabled 1h = Output enabled
14	DIO14	R/W	0h	Enables data output for DIO14. 0h = Output disabled 1h = Output enabled
13	DIO13	R/W	0h	Enables data output for DIO13. 0h = Output disabled 1h = Output enabled
12	DIO12	R/W	0h	Enables data output for DIO12. 0h = Output disabled 1h = Output enabled
11	DIO11	R/W	0h	Enables data output for DIO11. 0h = Output disabled 1h = Output enabled
10	DIO10	R/W	0h	Enables data output for DIO10. 0h = Output disabled 1h = Output enabled
9	DIO9	R/W	0h	Enables data output for DIO9. 0h = Output disabled 1h = Output enabled
8	DIO8	R/W	0h	Enables data output for DIO8. 0h = Output disabled 1h = Output enabled
7	DIO7	R/W	0h	Enables data output for DIO7. 0h = Output disabled 1h = Output enabled
6	DIO6	R/W	0h	Enables data output for DIO6. 0h = Output disabled 1h = Output enabled
5	DIO5	R/W	0h	Enables data output for DIO5. 0h = Output disabled 1h = Output enabled
4	DIO4	R/W	0h	Enables data output for DIO4. 0h = Output disabled 1h = Output enabled
3	DIO3	R/W	0h	Enables data output for DIO3. 0h = Output disabled 1h = Output enabled

Table 9-45. DOE31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	DIO2	R/W	0h	Enables data output for DIO2. 0h = Output disabled 1h = Output enabled
1	DIO1	R/W	0h	Enables data output for DIO1. 0h = Output disabled 1h = Output enabled
0	DIO0	R/W	0h	Enables data output for DIO0. 0h = Output disabled 1h = Output enabled

9.3.43 DOESET31_0 (Offset = 12D0h) [Reset = 00000000h]

DOESET31_0 is shown in [Figure 9-46](#) and described in [Table 9-46](#).

Return to the [Summary Table](#).

Writing 1 to a bit position in this register sets the corresponding bit in the DOE31_0 register.

Figure 9-46. DOESET31_0

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 9-46. DOESET31_0 Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	W	0h	Writing 1 to this bit sets the DIO31 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO31 in DOE31_0
30	DIO30	W	0h	Writing 1 to this bit sets the DIO30 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO30 in DOE31_0
29	DIO29	W	0h	Writing 1 to this bit sets the DIO29 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO29 in DOE31_0
28	DIO28	W	0h	Writing 1 to this bit sets the DIO28 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO28 in DOE31_0
27	DIO27	W	0h	Writing 1 to this bit sets the DIO27 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO27 in DOE31_0
26	DIO26	W	0h	Writing 1 to this bit sets the DIO26 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO26 in DOE31_0
25	DIO25	W	0h	Writing 1 to this bit sets the DIO25 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO25 in DOE31_0
24	DIO24	W	0h	Writing 1 to this bit sets the DIO24 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO24 in DOE31_0

Table 9-46. DOESET31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
23	DIO23	W	0h	Writing 1 to this bit sets the DIO23 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO23 in DOE31_0
22	DIO22	W	0h	Writing 1 to this bit sets the DIO22 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO22 in DOE31_0
21	DIO21	W	0h	Writing 1 to this bit sets the DIO21 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO21 in DOE31_0
20	DIO20	W	0h	Writing 1 to this bit sets the DIO20 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO20 in DOE31_0
19	DIO19	W	0h	Writing 1 to this bit sets the DIO19 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO19 in DOE31_0
18	DIO18	W	0h	Writing 1 to this bit sets the DIO18 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO18 in DOE31_0
17	DIO17	W	0h	Writing 1 to this bit sets the DIO17 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO17 in DOE31_0
16	DIO16	W	0h	Writing 1 to this bit sets the DIO16 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO16 in DOE31_0
15	DIO15	W	0h	Writing 1 to this bit sets the DIO15 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO15 in DOE31_0
14	DIO14	W	0h	Writing 1 to this bit sets the DIO14 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO14 in DOE31_0
13	DIO13	W	0h	Writing 1 to this bit sets the DIO13 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO13 in DOE31_0
12	DIO12	W	0h	Writing 1 to this bit sets the DIO12 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO12 in DOE31_0
11	DIO11	W	0h	Writing 1 to this bit sets the DIO11 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO11 in DOE31_0
10	DIO10	W	0h	Writing 1 to this bit sets the DIO10 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO10 in DOE31_0

Table 9-46. DOESET31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
9	DIO9	W	0h	Writing 1 to this bit sets the DIO9 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO9 in DOE31_0
8	DIO8	W	0h	Writing 1 to this bit sets the DIO8 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO8 in DOE31_0
7	DIO7	W	0h	Writing 1 to this bit sets the DIO7 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO7 in DOE31_0
6	DIO6	W	0h	Writing 1 to this bit sets the DIO6 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO6 in DOE31_0
5	DIO5	W	0h	Writing 1 to this bit sets the DIO5 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO5 in DOE31_0
4	DIO4	W	0h	Writing 1 to this bit sets the DIO4 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO4 in DOE31_0
3	DIO3	W	0h	Writing 1 to this bit sets the DIO3 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO3 in DOE31_0
2	DIO2	W	0h	Writing 1 to this bit sets the DIO2 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO2 in DOE31_0
1	DIO1	W	0h	Writing 1 to this bit sets the DIO1 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO1 in DOE31_0
0	DIO0	W	0h	Writing 1 to this bit sets the DIO0 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Sets DIO0 in DOE31_0

9.3.44 DOECLR31_0 (Offset = 12E0h) [Reset = 00000000h]

DOECLR31_0 is shown in [Figure 9-47](#) and described in [Table 9-47](#).

Return to the [Summary Table](#).

Writing 1 to a bit position in this register clears the corresponding bit in the DOE31_0 register.

Figure 9-47. DOECLR31_0

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 9-47. DOECLR31_0 Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	W	0h	Writing 1 to this bit clears the DIO31 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO31 in DOE31_0
30	DIO30	W	0h	Writing 1 to this bit clears the DIO30 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO30 in DOE31_0
29	DIO29	W	0h	Writing 1 to this bit clears the DIO29 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO29 in DOE31_0
28	DIO28	W	0h	Writing 1 to this bit clears the DIO28 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO28 in DOE31_0
27	DIO27	W	0h	Writing 1 to this bit clears the DIO27 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO27 in DOE31_0
26	DIO26	W	0h	Writing 1 to this bit clears the DIO26 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO26 in DOE31_0
25	DIO25	W	0h	Writing 1 to this bit clears the DIO25 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO25 in DOE31_0
24	DIO24	W	0h	Writing 1 to this bit clears the DIO24 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO24 in DOE31_0

Table 9-47. DOECLR31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
23	DIO23	W	0h	Writing 1 to this bit clears the DIO23 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO23 in DOE31_0
22	DIO22	W	0h	Writing 1 to this bit clears the DIO22 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO22 in DOE31_0
21	DIO21	W	0h	Writing 1 to this bit clears the DIO21 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO21 in DOE31_0
20	DIO20	W	0h	Writing 1 to this bit clears the DIO20 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO20 in DOE31_0
19	DIO19	W	0h	Writing 1 to this bit clears the DIO19 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO19 in DOE31_0
18	DIO18	W	0h	Writing 1 to this bit clears the DIO18 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO18 in DOE31_0
17	DIO17	W	0h	Writing 1 to this bit clears the DIO17 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO17 in DOE31_0
16	DIO16	W	0h	Writing 1 to this bit clears the DIO16 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO16 in DOE31_0
15	DIO15	W	0h	Writing 1 to this bit clears the DIO15 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO15 in DOE31_0
14	DIO14	W	0h	Writing 1 to this bit clears the DIO14 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO14 in DOE31_0
13	DIO13	W	0h	Writing 1 to this bit clears the DIO13 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO13 in DOE31_0
12	DIO12	W	0h	Writing 1 to this bit clears the DIO12 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO12 in DOE31_0
11	DIO11	W	0h	Writing 1 to this bit clears the DIO11 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO11 in DOE31_0
10	DIO10	W	0h	Writing 1 to this bit clears the DIO10 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO10 in DOE31_0

Table 9-47. DOECLR31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
9	DIO9	W	0h	Writing 1 to this bit clears the DIO9 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO9 in DOE31_0
8	DIO8	W	0h	Writing 1 to this bit clears the DIO8 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO8 in DOE31_0
7	DIO7	W	0h	Writing 1 to this bit clears the DIO7 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO7 in DOE31_0
6	DIO6	W	0h	Writing 1 to this bit clears the DIO6 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO6 in DOE31_0
5	DIO5	W	0h	Writing 1 to this bit clears the DIO5 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO5 in DOE31_0
4	DIO4	W	0h	Writing 1 to this bit clears the DIO4 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO4 in DOE31_0
3	DIO3	W	0h	Writing 1 to this bit clears the DIO3 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO3 in DOE31_0
2	DIO2	W	0h	Writing 1 to this bit clears the DIO2 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO2 in DOE31_0
1	DIO1	W	0h	Writing 1 to this bit clears the DIO1 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO1 in DOE31_0
0	DIO0	W	0h	Writing 1 to this bit clears the DIO0 bit in the DOE31_0 register. Writing 0 has no effect. 0h = No effect 1h = Clears DIO0 in DOE31_0

9.3.45 DIN3_0 (Offset = 1300h) [Reset = 00000000h]

DIN3_0 is shown in [Figure 9-48](#) and described in [Table 9-48](#).

Return to the [Summary Table](#).

Data input from pins configured as DIO3 to DIO0. This is an alias register for byte access to bits 3 to 0 in DIN31_0 register.

Figure 9-48. DIN3_0

31	30	29	28	27	26	25	24
RESERVED							DIO3
R-0h							R-0h
23	22	21	20	19	18	17	16
RESERVED							DIO2
R-0h							R-0h
15	14	13	12	11	10	9	8
RESERVED							DIO1
R-0h							R-0h
7	6	5	4	3	2	1	0
RESERVED							DIO0
R-0h							R-0h

Table 9-48. DIN3_0 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	R	0h	
24	DIO3	R	0h	This bit reads the data input value of DIO3. 0h = Input value is 0 1h = Input value is 1
23-17	RESERVED	R	0h	
16	DIO2	R	0h	This bit reads the data input value of DIO2. 0h = Input value is 0 1h = Input value is 1
15-9	RESERVED	R	0h	
8	DIO1	R	0h	This bit reads the data input value of DIO1. 0h = Input value is 0 1h = Input value is 1
7-1	RESERVED	R	0h	
0	DIO0	R	0h	This bit reads the data input value of DIO0. 0h = Input value is 0 1h = Input value is 1

9.3.46 DIN7_4 (Offset = 1304h) [Reset = 00000000h]

DIN7_4 is shown in [Figure 9-49](#) and described in [Table 9-49](#).

Return to the [Summary Table](#).

Data input from pins configured as DIO7 to DIO4. This is an alias register for byte access to bits 7 to 4 in DIN31_0 register.

Figure 9-49. DIN7_4

31	30	29	28	27	26	25	24
RESERVED							DIO7
R-0h							R-0h
23	22	21	20	19	18	17	16
RESERVED							DIO6
R-0h							R-0h
15	14	13	12	11	10	9	8
RESERVED							DIO5
R-0h							R-0h
7	6	5	4	3	2	1	0
RESERVED							DIO4
R-0h							R-0h

Table 9-49. DIN7_4 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	R	0h	
24	DIO7	R	0h	This bit reads the data input value of DIO7. 0h = Input value is 0 1h = Input value is 1
23-17	RESERVED	R	0h	
16	DIO6	R	0h	This bit reads the data input value of DIO6. 0h = Input value is 0 1h = Input value is 1
15-9	RESERVED	R	0h	
8	DIO5	R	0h	This bit reads the data input value of DIO5. 0h = Input value is 0 1h = Input value is 1
7-1	RESERVED	R	0h	
0	DIO4	R	0h	This bit reads the data input value of DIO4. 0h = Input value is 0 1h = Input value is 1

9.3.47 DIN11_8 (Offset = 1308h) [Reset = 00000000h]

DIN11_8 is shown in [Figure 9-50](#) and described in [Table 9-50](#).

Return to the [Summary Table](#).

Data input from pins configured as DIO11 to DIO8. This is an alias register for byte access to bits 11 to 8 in DIN31_0 register.

Figure 9-50. DIN11_8

31	30	29	28	27	26	25	24
RESERVED							DIO11
R-0h							R-0h
23	22	21	20	19	18	17	16
RESERVED							DIO10
R-0h							R-0h
15	14	13	12	11	10	9	8
RESERVED							DIO9
R-0h							R-0h
7	6	5	4	3	2	1	0
RESERVED							DIO8
R-0h							R-0h

Table 9-50. DIN11_8 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	R	0h	
24	DIO11	R	0h	This bit reads the data input value of DIO11. 0h = Input value is 0 1h = Input value is 1
23-17	RESERVED	R	0h	
16	DIO10	R	0h	This bit reads the data input value of DIO10. 0h = Input value is 0 1h = Input value is 1
15-9	RESERVED	R	0h	
8	DIO9	R	0h	This bit reads the data input value of DIO9. 0h = Input value is 0 1h = Input value is 1
7-1	RESERVED	R	0h	
0	DIO8	R	0h	This bit reads the data input value of DIO8. 0h = Input value is 0 1h = Input value is 1

9.3.48 DIN15_12 (Offset = 130Ch) [Reset = 00000000h]

DIN15_12 is shown in [Figure 9-51](#) and described in [Table 9-51](#).

Return to the [Summary Table](#).

Data input from pins configured as DIO15 to DIO12. This is an alias register for byte access to bits 15 to 12 in DIN31_0 register.

Figure 9-51. DIN15_12

31	30	29	28	27	26	25	24
RESERVED							DIO15
R-0h							R-0h
23	22	21	20	19	18	17	16
RESERVED							DIO14
R-0h							R-0h
15	14	13	12	11	10	9	8
RESERVED							DIO13
R-0h							R-0h
7	6	5	4	3	2	1	0
RESERVED							DIO12
R-0h							R-0h

Table 9-51. DIN15_12 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	R	0h	
24	DIO15	R	0h	This bit reads the data input value of DIO15. 0h = Input value is 0 1h = Input value is 1
23-17	RESERVED	R	0h	
16	DIO14	R	0h	This bit reads the data input value of DIO14. 0h = Input value is 0 1h = Input value is 1
15-9	RESERVED	R	0h	
8	DIO13	R	0h	This bit reads the data input value of DIO13. 0h = Input value is 0 1h = Input value is 1
7-1	RESERVED	R	0h	
0	DIO12	R	0h	This bit reads the data input value of DIO12. 0h = Input value is 0 1h = Input value is 1

9.3.49 DIN19_16 (Offset = 1310h) [Reset = 00000000h]

DIN19_16 is shown in [Figure 9-52](#) and described in [Table 9-52](#).

Return to the [Summary Table](#).

Data input from pins configured as DIO19 to DIO16. This is an alias register for byte access to bits 19 to 16 in DIN31_0 register.

Figure 9-52. DIN19_16

31	30	29	28	27	26	25	24
RESERVED							DIO19
R-0h							R-0h
23	22	21	20	19	18	17	16
RESERVED							DIO18
R-0h							R-0h
15	14	13	12	11	10	9	8
RESERVED							DIO17
R-0h							R-0h
7	6	5	4	3	2	1	0
RESERVED							DIO16
R-0h							R-0h

Table 9-52. DIN19_16 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	R	0h	
24	DIO19	R	0h	This bit reads the data input value of DIO19. 0h = Input value is 0 1h = Input value is 1
23-17	RESERVED	R	0h	
16	DIO18	R	0h	This bit reads the data input value of DIO18. 0h = Input value is 0 1h = Input value is 1
15-9	RESERVED	R	0h	
8	DIO17	R	0h	This bit reads the data input value of DIO17. 0h = Input value is 0 1h = Input value is 1
7-1	RESERVED	R	0h	
0	DIO16	R	0h	This bit reads the data input value of DIO16. 0h = Input value is 0 1h = Input value is 1

9.3.50 DIN23_20 (Offset = 1314h) [Reset = 00000000h]

DIN23_20 is shown in [Figure 9-53](#) and described in [Table 9-53](#).

Return to the [Summary Table](#).

Data input from pins configured as DIO23 to DIO20. This is an alias register for byte access to bits 23 to 20 in DIN31_0 register.

Figure 9-53. DIN23_20

31	30	29	28	27	26	25	24
RESERVED							DIO23
R-0h							R-0h
23	22	21	20	19	18	17	16
RESERVED							DIO22
R-0h							R-0h
15	14	13	12	11	10	9	8
RESERVED							DIO21
R-0h							R-0h
7	6	5	4	3	2	1	0
RESERVED							DIO20
R-0h							R-0h

Table 9-53. DIN23_20 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	R	0h	
24	DIO23	R	0h	This bit reads the data input value of DIO23. 0h = Input value is 0 1h = Input value is 1
23-17	RESERVED	R	0h	
16	DIO22	R	0h	This bit reads the data input value of DIO22. 0h = Input value is 0 1h = Input value is 1
15-9	RESERVED	R	0h	
8	DIO21	R	0h	This bit reads the data input value of DIO21. 0h = Input value is 0 1h = Input value is 1
7-1	RESERVED	R	0h	
0	DIO20	R	0h	This bit reads the data input value of DIO20. 0h = Input value is 0 1h = Input value is 1

9.3.51 DIN27_24 (Offset = 1318h) [Reset = 00000000h]

DIN27_24 is shown in [Figure 9-54](#) and described in [Table 9-54](#).

Return to the [Summary Table](#).

Data input from pins configured as DIO27 to DIO24. This is an alias register for byte access to bits 27 to 24 in DIN31_0 register.

Figure 9-54. DIN27_24

31	30	29	28	27	26	25	24
RESERVED							DIO27
R-0h							R-0h
23	22	21	20	19	18	17	16
RESERVED							DIO26
R-0h							R-0h
15	14	13	12	11	10	9	8
RESERVED							DIO25
R-0h							R-0h
7	6	5	4	3	2	1	0
RESERVED							DIO24
R-0h							R-0h

Table 9-54. DIN27_24 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	R	0h	
24	DIO27	R	0h	This bit reads the data input value of DIO27. 0h = Input value is 0 1h = Input value is 1
23-17	RESERVED	R	0h	
16	DIO26	R	0h	This bit reads the data input value of DIO26. 0h = Input value is 0 1h = Input value is 1
15-9	RESERVED	R	0h	
8	DIO25	R	0h	This bit reads the data input value of DIO25. 0h = Input value is 0 1h = Input value is 1
7-1	RESERVED	R	0h	
0	DIO24	R	0h	This bit reads the data input value of DIO24. 0h = Input value is 0 1h = Input value is 1

9.3.52 DIN31_28 (Offset = 131Ch) [Reset = 00000000h]

DIN31_28 is shown in [Figure 9-55](#) and described in [Table 9-55](#).

Return to the [Summary Table](#).

Data input from pins configured as DIO31 to DIO28. This is an alias register for byte access to bits 31 to 28 in DIN31_0 register.

Figure 9-55. DIN31_28

31	30	29	28	27	26	25	24
RESERVED							DIO31
R-0h							R-0h
23	22	21	20	19	18	17	16
RESERVED							DIO30
R-0h							R-0h
15	14	13	12	11	10	9	8
RESERVED							DIO29
R-0h							R-0h
7	6	5	4	3	2	1	0
RESERVED							DIO28
R-0h							R-0h

Table 9-55. DIN31_28 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	R	0h	
24	DIO31	R	0h	This bit reads the data input value of DIO31. 0h = Input value is 0 1h = Input value is 1
23-17	RESERVED	R	0h	
16	DIO30	R	0h	This bit reads the data input value of DIO30. 0h = Input value is 0 1h = Input value is 1
15-9	RESERVED	R	0h	
8	DIO29	R	0h	This bit reads the data input value of DIO29. 0h = Input value is 0 1h = Input value is 1
7-1	RESERVED	R	0h	
0	DIO28	R	0h	This bit reads the data input value of DIO28. 0h = Input value is 0 1h = Input value is 1

9.3.53 DIN31_0 (Offset = 1380h) [Reset = 00000000h]

DIN31_0 is shown in [Figure 9-56](#) and described in [Table 9-56](#).

Return to the [Summary Table](#).

Data input value for pins configured as DIO31 to DIO0.

Figure 9-56. DIN31_0

31	30	29	28	27	26	25	24
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 9-56. DIN31_0 Field Descriptions

Bit	Field	Type	Reset	Description
31	DIO31	R	0h	This bit reads the data input value of DIO31. 0h = Input value is 0 1h = Input value is 1
30	DIO30	R	0h	This bit reads the data input value of DIO30. 0h = Input value is 0 1h = Input value is 1
29	DIO29	R	0h	This bit reads the data input value of DIO29. 0h = Input value is 0 1h = Input value is 1
28	DIO28	R	0h	This bit reads the data input value of DIO28. 0h = Input value is 0 1h = Input value is 1
27	DIO27	R	0h	This bit reads the data input value of DIO27. 0h = Input value is 0 1h = Input value is 1
26	DIO26	R	0h	This bit reads the data input value of DIO26. 0h = Input value is 0 1h = Input value is 1
25	DIO25	R	0h	This bit reads the data input value of DIO25. 0h = Input value is 0 1h = Input value is 1
24	DIO24	R	0h	This bit reads the data input value of DIO24. 0h = Input value is 0 1h = Input value is 1
23	DIO23	R	0h	This bit reads the data input value of DIO23. 0h = Input value is 0 1h = Input value is 1
22	DIO22	R	0h	This bit reads the data input value of DIO22. 0h = Input value is 0 1h = Input value is 1

Table 9-56. DIN31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	DIO21	R	0h	This bit reads the data input value of DIO21. 0h = Input value is 0 1h = Input value is 1
20	DIO20	R	0h	This bit reads the data input value of DIO20. 0h = Input value is 0 1h = Input value is 1
19	DIO19	R	0h	This bit reads the data input value of DIO19. 0h = Input value is 0 1h = Input value is 1
18	DIO18	R	0h	This bit reads the data input value of DIO18. 0h = Input value is 0 1h = Input value is 1
17	DIO17	R	0h	This bit reads the data input value of DIO17. 0h = Input value is 0 1h = Input value is 1
16	DIO16	R	0h	This bit reads the data input value of DIO16. 0h = Input value is 0 1h = Input value is 1
15	DIO15	R	0h	This bit reads the data input value of DIO15. 0h = Input value is 0 1h = Input value is 1
14	DIO14	R	0h	This bit reads the data input value of DIO14. 0h = Input value is 0 1h = Input value is 1
13	DIO13	R	0h	This bit reads the data input value of DIO13. 0h = Input value is 0 1h = Input value is 1
12	DIO12	R	0h	This bit reads the data input value of DIO12. 0h = Input value is 0 1h = Input value is 1
11	DIO11	R	0h	This bit reads the data input value of DIO11. 0h = Input value is 0 1h = Input value is 1
10	DIO10	R	0h	This bit reads the data input value of DIO10. 0h = Input value is 0 1h = Input value is 1
9	DIO9	R	0h	This bit reads the data input value of DIO9. 0h = Input value is 0 1h = Input value is 1
8	DIO8	R	0h	This bit reads the data input value of DIO8. 0h = Input value is 0 1h = Input value is 1
7	DIO7	R	0h	This bit reads the data input value of DIO7. 0h = Input value is 0 1h = Input value is 1
6	DIO6	R	0h	This bit reads the data input value of DIO6. 0h = Input value is 0 1h = Input value is 1
5	DIO5	R	0h	This bit reads the data input value of DIO5. 0h = Input value is 0 1h = Input value is 1
4	DIO4	R	0h	This bit reads the data input value of DIO4. 0h = Input value is 0 1h = Input value is 1
3	DIO3	R	0h	This bit reads the data input value of DIO3. 0h = Input value is 0 1h = Input value is 1

Table 9-56. DIN31_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	DIO2	R	0h	This bit reads the data input value of DIO2. 0h = Input value is 0 1h = Input value is 1
1	DIO1	R	0h	This bit reads the data input value of DIO1. 0h = Input value is 0 1h = Input value is 1
0	DIO0	R	0h	This bit reads the data input value of DIO0. 0h = Input value is 0 1h = Input value is 1

9.3.54 POLARITY15_0 (Offset = 1390h) [Reset = 00000000h]

POLARITY15_0 is shown in [Figure 9-57](#) and described in [Table 9-57](#).

Return to the [Summary Table](#).

This register is used to enable and configure the polarity for input edge detection on DIO15 to DIO0. The corresponding DIO bits in RIS register will be set when the input event matches the configured polarity.

Figure 9-57. POLARITY15_0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DIO15	DIO14	DIO13	DIO12	DIO11	DIO10	DIO9	DIO8								
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIO7	DIO6	DIO5	DIO4	DIO3	DIO2	DIO1	DIO0								
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h								

Table 9-57. POLARITY15_0 Field Descriptions

Bit	Field	Type	Reset	Description
31-30	DIO15	R/W	0h	Enables and configures edge detection polarity for DIO15. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
29-28	DIO14	R/W	0h	Enables and configures edge detection polarity for DIO14. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
27-26	DIO13	R/W	0h	Enables and configures edge detection polarity for DIO13. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
25-24	DIO12	R/W	0h	Enables and configures edge detection polarity for DIO12. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
23-22	DIO11	R/W	0h	Enables and configures edge detection polarity for DIO11. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
21-20	DIO10	R/W	0h	Enables and configures edge detection polarity for DIO10. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
19-18	DIO9	R/W	0h	Enables and configures edge detection polarity for DIO9. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
17-16	DIO8	R/W	0h	Enables and configures edge detection polarity for DIO8. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event

Table 9-57. POLARITY15_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
15-14	DIO7	R/W	0h	Enables and configures edge detection polarity for DIO7. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
13-12	DIO6	R/W	0h	Enables and configures edge detection polarity for DIO6. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
11-10	DIO5	R/W	0h	Enables and configures edge detection polarity for DIO5. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
9-8	DIO4	R/W	0h	Enables and configures edge detection polarity for DIO4. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
7-6	DIO3	R/W	0h	Enables and configures edge detection polarity for DIO3. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
5-4	DIO2	R/W	0h	Enables and configures edge detection polarity for DIO2. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
3-2	DIO1	R/W	0h	Enables and configures edge detection polarity for DIO1. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
1-0	DIO0	R/W	0h	Enables and configures edge detection polarity for DIO0. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event

9.3.55 POLARITY31_16 (Offset = 13A0h) [Reset = 00000000h]

POLARITY31_16 is shown in [Figure 9-58](#) and described in [Table 9-58](#).

Return to the [Summary Table](#).

This register is used to enable and configure the polarity for input edge detection on DIO31 to DIO16. The corresponding DIO bits in RIS register will be set when the input event matches the configured polarity.

Figure 9-58. POLARITY31_16

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DIO31	DIO30	DIO29	DIO28	DIO27	DIO26	DIO25	DIO24								
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIO23	DIO22	DIO21	DIO20	DIO19	DIO18	DIO17	DIO16								
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h								

Table 9-58. POLARITY31_16 Field Descriptions

Bit	Field	Type	Reset	Description
31-30	DIO31	R/W	0h	Enables and configures edge detection polarity for DIO31. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
29-28	DIO30	R/W	0h	Enables and configures edge detection polarity for DIO30. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
27-26	DIO29	R/W	0h	Enables and configures edge detection polarity for DIO29. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
25-24	DIO28	R/W	0h	Enables and configures edge detection polarity for DIO28. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
23-22	DIO27	R/W	0h	Enables and configures edge detection polarity for DIO27. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
21-20	DIO26	R/W	0h	Enables and configures edge detection polarity for DIO26. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
19-18	DIO25	R/W	0h	Enables and configures edge detection polarity for DIO25. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
17-16	DIO24	R/W	0h	Enables and configures edge detection polarity for DIO24. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event

Table 9-58. POLARITY31_16 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
15-14	DIO23	R/W	0h	Enables and configures edge detection polarity for DIO23. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
13-12	DIO22	R/W	0h	Enables and configures edge detection polarity for DIO22. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
11-10	DIO21	R/W	0h	Enables and configures edge detection polarity for DIO21. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
9-8	DIO20	R/W	0h	Enables and configures edge detection polarity for DIO20. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
7-6	DIO19	R/W	0h	Enables and configures edge detection polarity for DIO19. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
5-4	DIO18	R/W	0h	Enables and configures edge detection polarity for DIO18. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
3-2	DIO17	R/W	0h	Enables and configures edge detection polarity for DIO17. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event
1-0	DIO16	R/W	0h	Enables and configures edge detection polarity for DIO16. 0h = Edge detection disabled 1h = Detects rising edge of input event 2h = Detects falling edge of input event 3h = Detects both rising and falling edge of input event

9.3.56 CTL (Offset = 1400h) [Reset = 00000000h]

CTL is shown in [Figure 9-59](#) and described in [Table 9-59](#).

Return to the [Summary Table](#).

GPIO Control Register

Figure 9-59. CTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							FASTWAKEONLY
R/W-0h							R/W-0h

Table 9-59. CTL Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R/W	0h	
0	FASTWAKEONLY	R/W	0h	FASTWAKEONLY for the global control of fastwake 0h = The global control of fastwake is not enabled, per bit fast wake feature depends on FASTWAKE.DIN 1h = The global control of fastwake is enabled

9.3.57 FASTWAKE (Offset = 1404h) [Reset = 00000000h]

FASTWAKE is shown in [Figure 9-60](#) and described in [Table 9-60](#).

Return to the [Summary Table](#).

This is per bit fast wake enable for the bit slice, allows the GPIO module to stay in a low power state and not require high speed clocking of the input synchronizer or filter

Figure 9-60. FASTWAKE

31	30	29	28	27	26	25	24
DIN31	DIN30	DIN29	DIN28	DIN27	DIN26	DIN25	DIN24
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
DIN23	DIN22	DIN21	DIN20	DIN19	DIN18	DIN17	DIN16
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
DIN15	DIN14	DIN13	DIN12	DIN11	DIN10	DIN9	DIN8
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
DIN7	DIN6	DIN5	DIN4	DIN3	DIN2	DIN1	DIN0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 9-60. FASTWAKE Field Descriptions

Bit	Field	Type	Reset	Description
31	DIN31	R/W	0h	Enable fastwake feature for DIN31 0h = fastwake feature is disabled 1h = fastwake feature is enabled
30	DIN30	R/W	0h	Enable fastwake feature for DIN30 0h = fastwake feature is disabled 1h = fastwake feature is enabled
29	DIN29	R/W	0h	Enable fastwake feature for DIN29 0h = fastwake feature is disabled 1h = fastwake feature is enabled
28	DIN28	R/W	0h	Enable fastwake feature for DIN28 0h = fastwake feature is disabled 1h = fastwake feature is enabled
27	DIN27	R/W	0h	Enable fastwake feature for DIN27 0h = fastwake feature is disabled 1h = fastwake feature is enabled
26	DIN26	R/W	0h	Enable fastwake feature for DIN26 0h = fastwake feature is disabled 1h = fastwake feature is enabled
25	DIN25	R/W	0h	Enable fastwake feature for DIN25 0h = fastwake feature is disabled 1h = fastwake feature is enabled
24	DIN24	R/W	0h	Enable fastwake feature for DIN24 0h = fastwake feature is disabled 1h = fastwake feature is enabled
23	DIN23	R/W	0h	Enable fastwake feature for DIN23 0h = fastwake feature is disabled 1h = fastwake feature is enabled
22	DIN22	R/W	0h	Enable fastwake feature for DIN22 0h = fastwake feature is disabled 1h = fastwake feature is enabled

Table 9-60. FASTWAKE Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	DIN21	R/W	0h	Enable fastwake feature for DIN21 0h = fastwake feature is disabled 1h = fastwake feature is enabled
20	DIN20	R/W	0h	Enable fastwake feature for DIN20 0h = fastwake feature is disabled 1h = fastwake feature is enabled
19	DIN19	R/W	0h	Enable fastwake feature for DIN19 0h = fastwake feature is disabled 1h = fastwake feature is enabled
18	DIN18	R/W	0h	Enable fastwake feature for DIN18 0h = fastwake feature is disabled 1h = fastwake feature is enabled
17	DIN17	R/W	0h	Enable fastwake feature for DIN17 0h = fastwake feature is disabled 1h = fastwake feature is enabled
16	DIN16	R/W	0h	Enable fastwake feature for DIN16 0h = fastwake feature is disabled 1h = fastwake feature is enabled
15	DIN15	R/W	0h	Enable fastwake feature for DIN15 0h = fastwake feature is disabled 1h = fastwake feature is enabled
14	DIN14	R/W	0h	Enable fastwake feature for DIN14 0h = fastwake feature is disabled 1h = fastwake feature is enabled
13	DIN13	R/W	0h	Enable fastwake feature for DIN13 0h = fastwake feature is disabled 1h = fastwake feature is enabled
12	DIN12	R/W	0h	Enable fastwake feature for DIN12 0h = fastwake feature is disabled 1h = fastwake feature is enabled
11	DIN11	R/W	0h	Enable fastwake feature for DIN11 0h = fastwake feature is disabled 1h = fastwake feature is enabled
10	DIN10	R/W	0h	Enable fastwake feature for DIN10 0h = fastwake feature is disabled 1h = fastwake feature is enabled
9	DIN9	R/W	0h	Enable fastwake feature for DIN9 0h = fastwake feature is disabled 1h = fastwake feature is enabled
8	DIN8	R/W	0h	Enable fastwake feature for DIN8 0h = fastwake feature is disabled 1h = fastwake feature is enabled
7	DIN7	R/W	0h	Enable fastwake feature for DIN7 0h = fastwake feature is disabled 1h = fastwake feature is enabled
6	DIN6	R/W	0h	Enable fastwake feature for DIN6 0h = fastwake feature is disabled 1h = fastwake feature is enabled
5	DIN5	R/W	0h	Enable fastwake feature for DIN5 0h = fastwake feature is disabled 1h = fastwake feature is enabled
4	DIN4	R/W	0h	Enable fastwake feature for DIN4 0h = fastwake feature is disabled 1h = fastwake feature is enabled
3	DIN3	R/W	0h	Enable fastwake feature for DIN3 0h = fastwake feature is disabled 1h = fastwake feature is enabled

Table 9-60. FASTWAKE Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	DIN2	R/W	0h	Enable fastwake feature for DIN2 0h = fastwake feature is disabled 1h = fastwake feature is enabled
1	DIN1	R/W	0h	Enable fastwake feature for DIN1 0h = fastwake feature is disabled 1h = fastwake feature is enabled
0	DIN0	R/W	0h	Enable fastwake feature for DIN0 0h = fastwake feature is disabled 1h = fastwake feature is enabled

9.3.58 SUB0CFG (Offset = 1500h) [Reset = 00000000h]

SUB0CFG is shown in [Figure 9-61](#) and described in [Table 9-61](#).

Return to the [Summary Table](#).

This register is used to enable the subscriber 0 event and define the output policy on the selected DIO 0-15 pins.

Figure 9-61. SUB0CFG

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED				INDEX			
R/W-0h				R/W-0h			
15	14	13	12	11	10	9	8
RESERVED						OUTPOLICY	
R/W-0h						R/W-0h	
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-0h							R/W-0h

Table 9-61. SUB0CFG Field Descriptions

Bit	Field	Type	Reset	Description
31-20	RESERVED	R/W	0h	
19-16	INDEX	R/W	0h	Indicates the specific bit among lower 16 bits that is targeted by the subscriber action 0h = specific bit targeted by the subscriber action is bit0 Fh = specific bit targeted by the subscriber action is bit15
15-10	RESERVED	R/W	0h	
9-8	OUTPOLICY	R/W	0h	These bits configure the output policy for subscriber 0 event. 0h = Selected DIO pins are set 1h = Selected DIO pins are cleared 2h = Selected DIO pins are toggled
7-1	RESERVED	R/W	0h	
0	ENABLE	R/W	0h	This bit is used to enable subscriber 0 event. 0h = Subscriber 0 event is disabled 1h = Subscriber 0 event is enabled

9.3.59 FILTEREN15_0 (Offset = 1508h) [Reset = 00000000h]

FILTEREN15_0 is shown in [Figure 9-62](#) and described in [Table 9-62](#).

Return to the [Summary Table](#).

Programmable counter length of digital glitch filter for DIN0-DIN15

Figure 9-62. FILTEREN15_0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DIN15	DIN14	DIN13	DIN12	DIN11	DIN10	DIN9	DIN8								
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIN7	DIN6	DIN5	DIN4	DIN3	DIN2	DIN1	DIN0								
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h								

Table 9-62. FILTEREN15_0 Field Descriptions

Bit	Field	Type	Reset	Description
31-30	DIN15	R/W	0h	Programmable counter length of digital glitch filter for DIN15 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
29-28	DIN14	R/W	0h	Programmable counter length of digital glitch filter for DIN14 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
27-26	DIN13	R/W	0h	Programmable counter length of digital glitch filter for DIN13 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
25-24	DIN12	R/W	0h	Programmable counter length of digital glitch filter for DIN12 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
23-22	DIN11	R/W	0h	Programmable counter length of digital glitch filter for DIN11 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
21-20	DIN10	R/W	0h	Programmable counter length of digital glitch filter for DIN10 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
19-18	DIN9	R/W	0h	Programmable counter length of digital glitch filter for DIN9 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
17-16	DIN8	R/W	0h	Programmable counter length of digital glitch filter for DIN8 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample

Table 9-62. FILTEREN15_0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
15-14	DIN7	R/W	0h	Programmable counter length of digital glitch filter for DIN7 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
13-12	DIN6	R/W	0h	Programmable counter length of digital glitch filter for DIN6 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
11-10	DIN5	R/W	0h	Programmable counter length of digital glitch filter for DIN5 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
9-8	DIN4	R/W	0h	Programmable counter length of digital glitch filter for DIN4 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
7-6	DIN3	R/W	0h	Programmable counter length of digital glitch filter for DIN3 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
5-4	DIN2	R/W	0h	Programmable counter length of digital glitch filter for DIN2 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
3-2	DIN1	R/W	0h	Programmable counter length of digital glitch filter for DIN1 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
1-0	DIN0	R/W	0h	Programmable counter length of digital glitch filter for DIN0 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample

9.3.60 FILTEREN31_16 (Offset = 150Ch) [Reset = 00000000h]

FILTEREN31_16 is shown in [Figure 9-63](#) and described in [Table 9-63](#).

Return to the [Summary Table](#).

Programmable counter length of digital glitch filter for DIN16-DIN31

Figure 9-63. FILTEREN31_16

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DIN31	DIN30	DIN29	DIN28	DIN27	DIN26	DIN25	DIN24								
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIN23	DIN22	DIN21	DIN20	DIN19	DIN18	DIN17	DIN16								
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h								

Table 9-63. FILTEREN31_16 Field Descriptions

Bit	Field	Type	Reset	Description
31-30	DIN31	R/W	0h	Programmable counter length of digital glitch filter for DIN31 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPCLK minimum sample 2h = 3 ULPCLK minimum sample 3h = 8 ULPCLK minimum sample
29-28	DIN30	R/W	0h	Programmable counter length of digital glitch filter for DIN30 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPCLK minimum sample 2h = 3 ULPCLK minimum sample 3h = 8 ULPCLK minimum sample
27-26	DIN29	R/W	0h	Programmable counter length of digital glitch filter for DIN29 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPCLK minimum sample 2h = 3 ULPCLK minimum sample 3h = 8 ULPCLK minimum sample
25-24	DIN28	R/W	0h	Programmable counter length of digital glitch filter for DIN28 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPCLK minimum sample 2h = 3 ULPCLK minimum sample 3h = 8 ULPCLK minimum sample
23-22	DIN27	R/W	0h	Programmable counter length of digital glitch filter for DIN27 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPCLK minimum sample 2h = 3 ULPCLK minimum sample 3h = 8 ULPCLK minimum sample
21-20	DIN26	R/W	0h	Programmable counter length of digital glitch filter for DIN26 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPCLK minimum sample 2h = 3 ULPCLK minimum sample 3h = 8 ULPCLK minimum sample
19-18	DIN25	R/W	0h	Programmable counter length of digital glitch filter for DIN25 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPCLK minimum sample 2h = 3 ULPCLK minimum sample 3h = 8 ULPCLK minimum sample
17-16	DIN24	R/W	0h	Programmable counter length of digital glitch filter for DIN24 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPCLK minimum sample 2h = 3 ULPCLK minimum sample 3h = 8 ULPCLK minimum sample

Table 9-63. FILTEREN31_16 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
15-14	DIN23	R/W	0h	Programmable counter length of digital glitch filter for DIN23 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
13-12	DIN22	R/W	0h	Programmable counter length of digital glitch filter for DIN22 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
11-10	DIN21	R/W	0h	Programmable counter length of digital glitch filter for DIN21 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
9-8	DIN20	R/W	0h	Programmable counter length of digital glitch filter for DIN20 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
7-6	DIN19	R/W	0h	Programmable counter length of digital glitch filter for DIN19 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
5-4	DIN18	R/W	0h	Programmable counter length of digital glitch filter for DIN18 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
3-2	DIN17	R/W	0h	Programmable counter length of digital glitch filter for DIN17 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample
1-0	DIN16	R/W	0h	Programmable counter length of digital glitch filter for DIN16 0h = No additional filter beyond the CDC synchronization sample 1h = 1 ULPClk minimum sample 2h = 3 ULPClk minimum sample 3h = 8 ULPClk minimum sample

9.3.61 DMAMASK (Offset = 1510h) [Reset = 00000000h]

DMAMASK is shown in [Figure 9-64](#) and described in [Table 9-64](#).

Return to the [Summary Table](#).

DMA MASK which indicates which bit lanes the DMA is allowed to modify.

Figure 9-64. DMAMASK

31	30	29	28	27	26	25	24
DOUT31	DOUT30	DOUT29	DOUT28	DOUT27	DOUT26	DOUT25	DOUT24
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
DOUT23	DOUT22	DOUT21	DOUT20	DOUT19	DOUT18	DOUT17	DOUT16
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
DOUT15	DOUT14	DOUT13	DOUT12	DOUT11	DOUT10	DOUT9	DOUT8
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
DOUT7	DOUT6	DOUT5	DOUT4	DOUT3	DOUT2	DOUT1	DOUT0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 9-64. DMAMASK Field Descriptions

Bit	Field	Type	Reset	Description
31	DOUT31	R/W	0h	DMA is allowed to modify DOUT31 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
30	DOUT30	R/W	0h	DMA is allowed to modify DOUT30 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
29	DOUT29	R/W	0h	DMA is allowed to modify DOUT29 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
28	DOUT28	R/W	0h	DMA is allowed to modify DOUT28 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
27	DOUT27	R/W	0h	DMA is allowed to modify DOUT27 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
26	DOUT26	R/W	0h	DMA is allowed to modify DOUT26 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
25	DOUT25	R/W	0h	DMA is allowed to modify DOUT25 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
24	DOUT24	R/W	0h	DMA is allowed to modify DOUT24 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
23	DOUT23	R/W	0h	DMA is allowed to modify DOUT23 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
22	DOUT22	R/W	0h	DMA is allowed to modify DOUT22 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane

Table 9-64. DMAMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	DOUT21	R/W	0h	DMA is allowed to modify DOUT21 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
20	DOUT20	R/W	0h	DMA is allowed to modify DOUT20 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
19	DOUT19	R/W	0h	DMA is allowed to modify DOUT19 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
18	DOUT18	R/W	0h	DMA is allowed to modify DOUT18 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
17	DOUT17	R/W	0h	DMA is allowed to modify DOUT17 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
16	DOUT16	R/W	0h	DMA is allowed to modify DOUT16 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
15	DOUT15	R/W	0h	DMA is allowed to modify DOUT15 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
14	DOUT14	R/W	0h	DMA is allowed to modify DOUT14 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
13	DOUT13	R/W	0h	DMA is allowed to modify DOUT13 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
12	DOUT12	R/W	0h	DMA is allowed to modify DOUT12 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
11	DOUT11	R/W	0h	DMA is allowed to modify DOUT11 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
10	DOUT10	R/W	0h	DMA is allowed to modify DOUT10 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
9	DOUT9	R/W	0h	DMA is allowed to modify DOUT9 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
8	DOUT8	R/W	0h	DMA is allowed to modify DOUT8 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
7	DOUT7	R/W	0h	DMA is allowed to modify DOUT7 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
6	DOUT6	R/W	0h	DMA is allowed to modify DOUT6 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
5	DOUT5	R/W	0h	DMA is allowed to modify DOUT5 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
4	DOUT4	R/W	0h	DMA is allowed to modify DOUT4 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
3	DOUT3	R/W	0h	DMA is allowed to modify DOUT3 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane

Table 9-64. DMAMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	DOUT2	R/W	0h	DMA is allowed to modify DOUT2 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
1	DOUT1	R/W	0h	DMA is allowed to modify DOUT1 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane
0	DOUT0	R/W	0h	DMA is allowed to modify DOUT0 0h = DMA is not allowed to modify this bit lane 1h = DMA is allowed to modify this bit lane

9.3.62 SUB1CFG (Offset = 1520h) [Reset = 00000000h]

SUB1CFG is shown in [Figure 9-65](#) and described in [Table 9-65](#).

Return to the [Summary Table](#).

This register is used to enable the subscriber 1 event and define the output policy on the selected DIO 16-31 pins.

Figure 9-65. SUB1CFG

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED				INDEX			
R/W-0h				R/W-0h			
15	14	13	12	11	10	9	8
RESERVED						OUTPOLICY	
R/W-0h						R/W-0h	
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-0h							R/W-0h

Table 9-65. SUB1CFG Field Descriptions

Bit	Field	Type	Reset	Description
31-20	RESERVED	R/W	0h	
19-16	INDEX	R/W	0h	indicates the specific bit in the upper 16 bits that is targeted by the subscriber action 0h = specific bit targeted by the subscriber action is bit16 Fh = specific bit targeted by the subscriber action is bit31
15-10	RESERVED	R/W	0h	
9-8	OUTPOLICY	R/W	0h	These bits configure the output policy for subscriber 1 event. 0h = Selected DIO pins are set 1h = Selected DIO pins are cleared 2h = Selected DIO pins are toggled
7-1	RESERVED	R/W	0h	
0	ENABLE	R/W	0h	This bit is used to enable subscriber 1 event. 0h = Subscriber 1 event is disabled 1h = Subscriber 1 event is enabled



The ADC is a high-performance successive-approximation-register (SAR) analog-to-digital converter. This chapter describes the features and operation of the ADC peripheral.

10.1 ADC Overview	576
10.2 ADC Operation	577
10.3 ADC12 Registers	594

10.1 ADC Overview

The ADC supports measure analog signals and convert them to a digital representation with minimal CPU intervention.

The ADC supports fast 12-, 10-, and 8-bit analog-to-digital conversions. It implements a 12-bit SAR core, sample and conversion mode control, and up to 12 independent conversion-and-control buffers. The ADC allows up to 12 independent analog-to-digital converter (ADC) samples to be converted and stored without any CPU intervention.

ADC features include:

- 4-Msps conversion rate at a resolution of 12 bits
- Integrated hardware oversampling for averaging up to 128 samples
- Full-scale ADC operating voltage range
- Sample-and-hold with programmable sampling periods controlled by software or timers
- Two sampling trigger sources: software trigger and event trigger
- Software-selectable on-chip reference voltage of 1.4 V or 2.5 V
- Configurable ADC reference source: VDD, internal reference (VREF), or external reference (VREF+ and VREF-)
- Up to 16 individually configurable analog input channels
- Internal conversion channels for temperature sensing, supply monitoring, and analog signal chain (see the device-specific data sheet for availability and channel mapping)
- Configurable ADC clock source
- Single-channel, repeat-single-channel, sequence (autoscan), and repeat-sequence (repeated autoscan) conversion modes
- 12 conversion-result storage registers (MEMRES0:11)
- Window comparator for low-power monitoring of input signals from conversion-result registers
- DMA support with interrupt event generation on completion of transfer
- Operates in RUN, SLEEP, and STOP modes
- Can be triggered in any operating mode except for SHUTDOWN
- Automatic power, reference, and clock control for low-power operation
- Support for simultaneous and synchronous operation (sample and conversion) of two ADCs in parallel
- Semi-automatic calibration of CDAC trim values

Figure 10-1 shows the functional block diagram of the ADC peripheral.

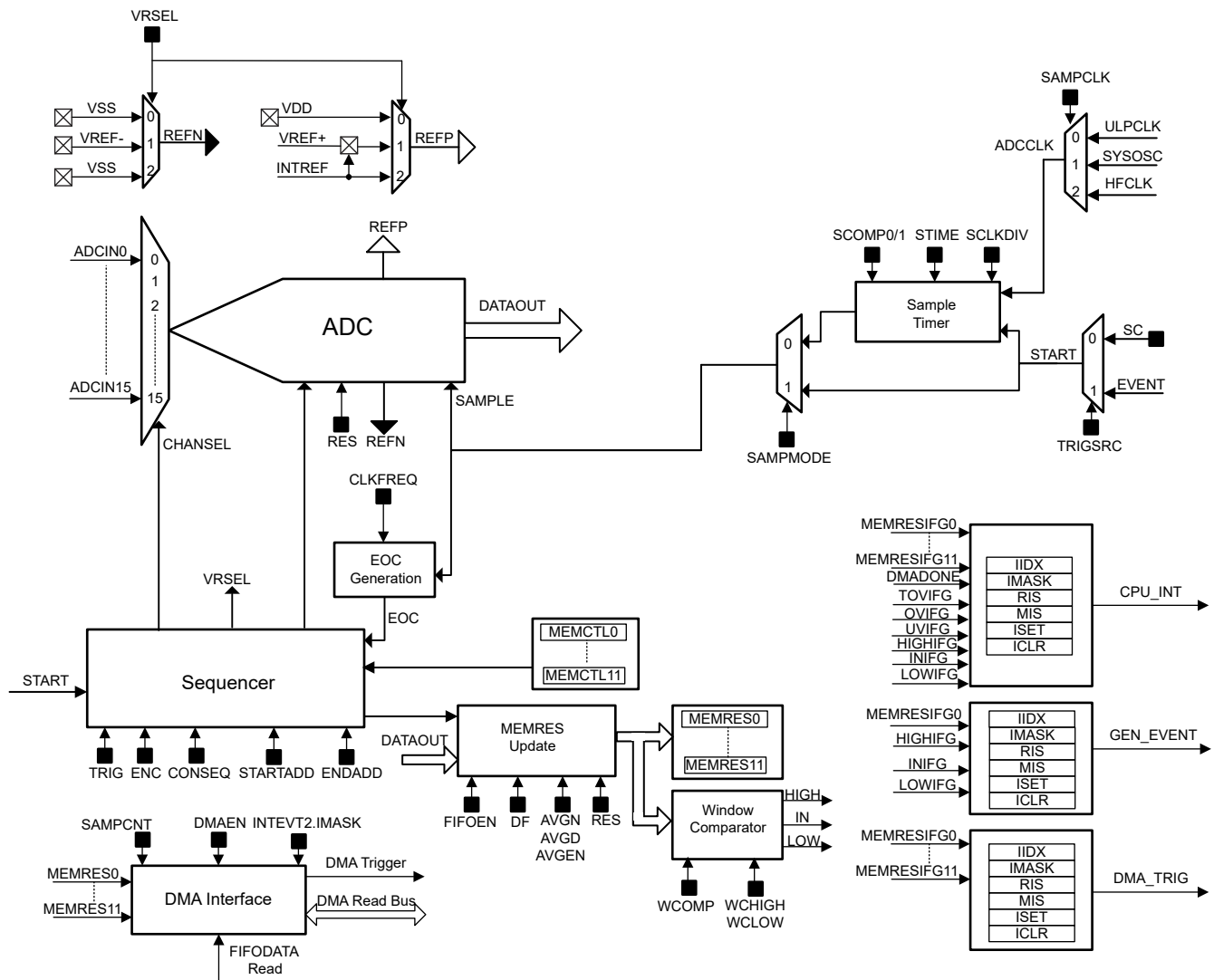


Figure 10-1. ADC Block Diagram

Note

The sample clock (SAMPCLK) is sourced from ULPCLK, SYSOSC or HFCLK, the conversion clock (CONVCLK) is sourced from an 80Mhz oscillator within the ADC IP.

10.2 ADC Operation

The ADC is configured with user software. The following sections describe the setup and operation of the ADC.

Note

The ADC result registers (MEMRES) are aliased between two address regions on the device:

- The primary region, through which all ADC registers can be accessed by the CPU or DMA at the ULPCLK rate
- The aliased region, through which the ADC MEMRES registers can be accessed by the CPU or DMA at the MCLK rate for fast read-out of ADC results

On devices which run MCLK and ULPCLK at the same frequency, there is not a performance benefit to accessing the ADC MEMRES registers through the aliased region. On device configurations where MCLK is greater than ULPCLK, the aliased region is recommended to be used. However, application software can use the aliased region on all devices for MEMRES access to keep the software implementation consistent.

10.2.1 ADC Core

The ADC core converts an analog input to its digital representation. The core uses two voltage levels (V_{R+} and V_{R-}) to define the upper and lower limits of the conversion. The digital output (N_{ADC}) is full scale when the input signal is equal to or higher than V_{R+} , and is zero when the input signal is equal to or lower than V_{R-} . The input channel and the positive reference voltage level (V_{R+}) are defined in the conversion-control memory.

Equation 19 below shows the conversion formula for the ADC result, N_{ADC} , for n-bit resolution mode.

$$N_{ADC} = \left(2^n - 1\right) \times \frac{(V_{in} + 0.5LSB) - V_{R-}}{V_{R+} - V_{R-}} \quad \text{Where, } LSB = \frac{V_{R+} - V_{R-}}{2^n} \quad (19)$$

Note

The supported V_{R-} for this ADC is 0V. All subsequent equations and sections will reflect this inherent property.

Given that V_{R-} is 0V in this ADC, the equation for N_{ADC} becomes:

$$N_{ADC} = \left(2^n - 1\right) \times \frac{(V_{in} + 0.5LSB)}{V_{R+}} \quad \text{Where, } LSB = \frac{V_{R+}}{2^n} \quad (20)$$

Equation 21 below describes the input voltage at which the ADC output saturates:

$$V_{in} = V_{R+} - 1.5LSB \quad (21)$$

Note

The ADC is NOT functional in STANDBY or SHUTDOWN mode.

10.2.2 Voltage Reference Options

The ADC voltage reference (V_{R+}) can be configured through the VRSEL bits in the MEMCTL register. Different reference sources can be selected for conversion on different channels. There are three options available for supplying a reference voltage to the ADC:

- External reference supplied to the ADC through the VREF+ and VREF- pins
- MCU supply voltage (VDD)
- Configurable internal reference voltage (INTREF) of 1.4 V and 2.5 V from

VREF module

When supplying an external reference to the ADC, connect the VREF+ pin to the reference source with the appropriate decoupling circuitry, and connect the VREF- pin to ground.

The internal reference (VREF) is a voltage reference module that is shared by multiple analog peripherals including the ADC peripheral. The internal reference is available on the VREF+ pin and it is recommended to connect a decoupling capacitor of specified value on the device pin. Refer to the device-specific data sheet for the recommended capacitor value. When supplying the internal reference as the reference to the ADC, there is no restriction on the ADC sampling rate and thus can be operated at full speed.

10.2.3 Generic Resolution Modes

The ADC supports operation in 12-bit (default), 10-bit, and 8-bit resolution modes. The resolution mode is configured using the RES bits in the CTL2 register.

- When 12-bit mode is selected, the conversion phase requires a total of 14 conversion clock cycles
- When 10-bit mode is selected, the conversion phase requires a total of 12 conversion clock cycles
- When 8-bit mode is selected, the conversion phase requires a total of 9 conversion clock cycles

The conversion clock (CONVCLK) is sourced from an 80MHz oscillator within the ADC IP. The conversion window is based on the resolution mode and the frequency of ADCCLK. For more details, refer to [Figure 10-2](#) and [Figure 10-3](#).

10.2.4 Hardware Averaging

This ADC implements digital sample averaging in hardware (HW averaging) to efficiently increase the effective resolution of the ADC without the need for SW or CPU intervention. The HW averaging functionality is configured using the AVGN and AVGD bits in the CTL1 register.

- AVGN defines the number of conversions to accumulate for the current MEMCTLx
- AVGD defines what the accumulated value gets divided by using bit shifting

Note

The MEMRES result register is a maximum of 16 bits long. If not shifted appropriately, the result will be truncated.

Table 10-1. Available Hardware Averaging Settings

Bit Field Value	AVGN Settings (number of samples accumulated)	AVGD Settings (number of bits to right shift)
0x0	0	0
0x1	2	1
0x2	4	2
0x3	8	3
0x4	16	4
0x5	32	5
0x6	64	6
0x7	128	7

The averaging configuration is global and it holds for any channel that enables the averaging feature. It is not possible to have different averaging configurations defined per channel. The averaging feature for each individual channel can be enabled though the AVGEN bit in the MEMCTL register. When the sample trigger is received for a channel with averaging enabled, the required number of conversions are performed automatically back-to-back and the final averaged value is stored in the MEMRES register or FIFODAT.

Note

The data format must be selected as unsigned binary while using the hardware averaging feature.

10.2.5 ADC Clocking

The ADC peripheral clock (ADCCLK) is provided by the [Section 2.4](#) and is used for the sampling clock (SAMPCLK). SYSOSC, HFCLK and ULPCLK are the available clock sources available for ADCCLK, which can support up to 48MHz. Refer to the device-specific data sheet for supported ADCCLK frequencies. Using the ULPCLK, which is the bus clock for all peripherals, is very useful for deterministic start of sampling and simultaneous sampling. Using the HFCLK as the clock source for ADCCLK is useful for when a very accurate, low-jitter, sampling period is needed. The ADC clock source can be selected by programming the SAMPCLK bits in the CLKCFG register. The conversion clock is sourced from a dedicated 80MHz local oscillator which enables high speed 12-bit conversions up to 4MSPS.

SYSOSC needs to be active for the ADC to operate properly. If SYSOSC is not running and the ADC is triggered, the ADC will automatically request SYSCTL to enable and set SYSOSC to base frequency during the conversion. If SYSOSC is already enabled, it will remain the same frequency. The only exception to this is in STOP1 operating mode where SYSOSC will go to base frequency when the ADC is triggered.

In order to provide a way to ensure predictable sample rate operation between power modes, the CCONRUN and CCONSTOP bits can be set to signal the ADC that it can expect that the SYSOSC will already be ON when the device is in RUN and STOP modes respectively. When these bits are set, the ADC will not wait for an ACK from SYSCTL to make sure SYSOSC is running before starting sampling. This feature gives users the flexibility to save power in applications where deterministic sample timing is not a requirement. Refer to [Section 10.2.6](#) for examples on how to properly use the CCONRUN and CCONSTOP control bits.

The user must configure the FRANGE bits in the CLKFREQ register to the appropriate setting based on the expected ADCCLK frequency. The ADC uses the CLKFREQ.FRANGE value and the resolution mode, configured by CTL2.RES bits, to determine how many ADCCLK clock cycles a conversion will take before signaling the End of Conversion (EOC). See [Table 10-2](#) below for more details on how to properly configure the CLKFREQ register and how that impacts the EOC signal generation.

Table 10-2. CLKFREQ Register Configuration

CLKFREQ.FRANGE Values	ADCCLK Frequency Range (MHz)	Conversion Duration (Clock cycles)		
		12-bit	10-bit	8-bit
0	>1 to 4	1	1	1
1	>4 to 8	2	2	1
2	>8 to 16	3	3	2
3	>16 to 20	4	4	3
4	>20 to 24	5	4	3
5	>24 to 32	6	6	4
6	>32 to 40	8	7	5
7	>40 to 48	9	8	6

10.2.6 Common ADC Use Cases

There are many ADC use-cases from an operating mode and clocking standpoint but the majority of them will fit into one of the items below:

- Triggers in RUN or SLEEP mode
 - If ADC is triggered to start a conversion (software or Event), and the device is in RUN0 or RUN1 or SLEEP0 or SLEEP1 mode (SYSOSC is already running at any frequency), then:
 - Sample clock can be ULPCLK, HFCLK, or SYSOSC in this mode
 - The conversion runs without changing SYSOSC frequency
 - 4-MHz, 16-MHz, 24-MHz, or 32-MHz SYSOSC frequencies are allowed
 - If ADC is triggered to start a conversion (software or Event), and the device is in RUN2 or SLEEP2 mode (SYSOSC is disabled, MCLK = LFCLK = 32 kHz), then:
 - Sample clock can be ULPCLK or SYSOSC in this mode

- SYSCTL interprets the ADC CLK REQ as an asynchronous fast clock request, enabling SYSOSC at 32 MHz and forcing MCLK or ULPCLK to 32 MHz until the ADC de-asserts the request
- CCONRUN must be cleared in this use case
- CCONSTOP must be cleared in this use case
- Triggers in STOP mode
 - Sample clock can be ULPCLK or SYSOSC in this mode
 - If ADC is triggered to start a conversion (Event), and the device is in STOP0 mode (SYSOSC runs at any frequency, ULPCLK = 4 MHz), then:
 - The conversion runs without changing SYSOSC frequency
 - 4-MHz, 16-MHz, 24-MHz, or 32-MHz SYSOSC frequencies are allowed
 - If ADC is triggered to start a conversion (Event), and the device is in STOP1 mode (SYSOSC was gear shifted to 4 MHz), then:
 - SYSCTL forces SYSOSC to BASE (consistent with RUN mode because USE4MHZSTOP was set) when receiving ADC CLK REQ, and SYSCTL releases SYSOSC back to 4 MHz after the ADC CLK REQ is removed
 - CCONRUN must be cleared
 - CCONSTOP must be cleared
 - If ADC is triggered to start a conversion (Event), and the device is in STOP2 mode (SYSOSC disabled), then:
 - The trigger event propagates through the event fabric at 32 kHz, the ADC receives the trigger and assert the ADC CLK REQ (CPCLK REQ) to SYSCTL, and SYSCTL receives the ADC CLK REQ as an asynchronous fast clock request, suspending STOP, enabling SYSOSC at 32 MHz, and forcing MCLK or ULPCLK to 32 MHz until the ADC de-asserts the ADC CLK REQ
 - CCONRUN must be cleared
 - CCONSTOP must be cleared
- Triggers in STANDBY mode
 - Sample clock can be ULPCLK or SYSOSC in this mode
 - If ADC is triggered to start a conversion (Event), and the device is in STANDBY0 mode (SYSOSC is disabled but ULPCLK is running), then:
 - The trigger event propagates through event fabric at 32 kHz, the ADC receives the trigger and asserts the ADC CLK REQ (CPCLK REQ) to SYSCTL, and SYSCTL interprets the ADC CLK REQ as an asynchronous fast clock request, suspending STANDBY, enabling SYSOSC at 32 MHz, and forcing MCLK/ULPCLK to 32 MHz until the ADC de-asserts the ADC CLK REQ
 - CCONRUN must be cleared
 - CCONSTOP must be cleared
 - If ADC is triggered to start a conversion (Event-TIMG0 or TIMG1), and the device is in STANDBY1 (ULPCLK is gated with STOPCLKSTBY set), then:
 - The TIMG0 or TIMG1 event triggers an asynchronous fast clock request to suspend STANDBY mode, start SYSOSC at 32 MHz, and force MCLK or ULPCLK to 32 MHz; there are then 32 SYSOSC cycles for the TIMG0 or TIMG1 event to proceed through the event fabric and for the ADC to capture the timer event and assert the ADC CLK REQ to hold the SYSOSC enabled to run the conversion
 - When the ADC de-asserts the ADC CLK REQ, ULPCLK runs for 32 additional cycles to allow any ADC event (DMA request or IRQ) to propagate, after which SYSCTL resumes STANDBY with STOPCLKSTBY (STANDBY1)
 - CCONRUN must be cleared
 - CCONSTOP must be cleared

10.2.7 Power Down Behavior

The ENABLE bit in the PWREN register enables or disables the ADC peripheral. The ADC should be disabled when it is not in use to save power. The PWRDN bit in the CTL0 register selects the ADC power down policy between AUTO and MANUAL. This takes effect when the ADC operates in RUN, SLEEP, and STOP MCU power modes.

PWRDN should be configured based on the max ADC sampling rate required and the operational needs in different MCU power modes. ADC hardware does not force the power down policy to AUTO during operation in STOP mode. It follows the user setting irrespective of the device power modes.

The reset value of PWRDN is 0, which has the default behavior of automatic power down of the ADC peripheral at the end of a conversion and when the next sample signal is not required to be asserted immediately. When the PWRDN bit is set to '1' it selects manual power down behavior. In this setting, the ADC is not powered down at the end of a conversion and remains enabled. This means that the ADC peripheral would only be powered down using the PWREN register.

Note

Refer to the device-specific data sheet for specifications on the ADC wakeup and enable time.

10.2.8 Sampling Trigger Sources and Sampling Modes

Sample Triggers

There are two sampling trigger sources available which can be selected through the TRIGSRC bit in the CTL1 register; one is a software trigger and the other is an event trigger.

When the software trigger is selected as the source, the application software can set the SC bit in the CTL1 register to initiate the sample phase. When the event trigger is selected as the source, a rising edge on the selected event from the event manager will initiate the sample phase. An event is always edge triggered.

Sampling Modes

There are two sampling modes available, AUTO and MANUAL, which can be selected through the SAMPMODE bit in the CTL1 register.

10.2.8.1 AUTO Sampling Mode

In AUTO mode, the sample signal is generated synchronous to the sampling clock (SAMPCLK) and can be programmed using an internal sampling timer to determine the duration of the sampling window. The sample timer is 10-bits wide and there are two sample time compare registers (SCOMPx) available to account for various source impedances to measure signals from. One of these two SCOMP registers can be selected using the STIME bit in the MEMCTL register.

There is a 2-3 cycle latency from when the sampling is triggered and when the sampling period starts. This latency can be bypassed by setting ULPCLK as the source for ADCCLK. This synchronization bypass feature is very useful for deterministic sampling and for simultaneous operation of two ADC peripherals.

[Figure 10-2](#) shows the ADC sample and conversion timing diagram when the ADC is configured in AUTO sampling mode.

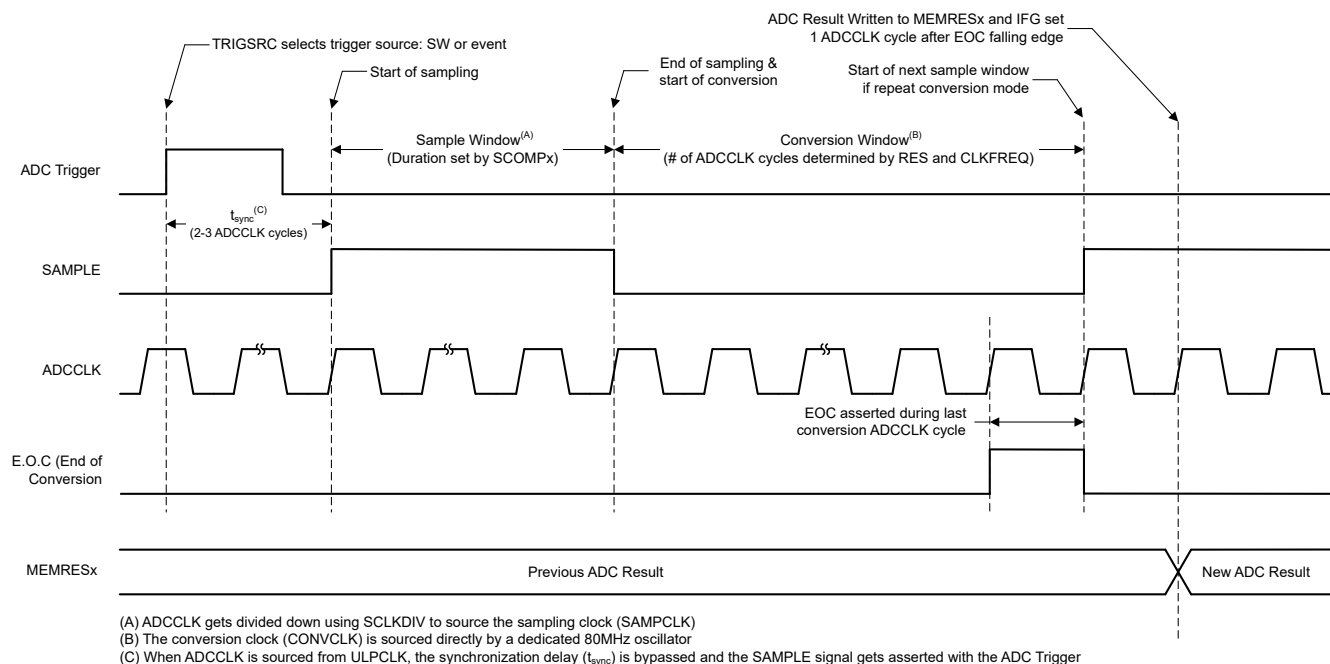


Figure 10-2. AUTO Sampling Mode ADC Sample and Conversion Timing Diagram

Note

When the reset value of PWRDN is set as '0' which has the default behavior of automatic power down, ADC wakeup/enable time needs to be considered in each sample window. Refer to the device-specific data sheet for specifications on the ADC wakeup/enable time. For example, if the maximum ADC wakeup/enable time is 5 μ S, it means the duration set by SCOMPx should be > (5 μ S + Duration for sample window).

10.2.8.2 MANUAL Sampling Mode

In MANUAL mode, the sample signal is generated when the SC bit is set which can be asynchronous to the sampling clock. The duration of the sampling window is controlled by software by holding the SC bit high. Because an event is always edge triggered, manual mode with event trigger is not supported for any of the conversion modes. Software trigger with manual sampling mode is supported only for single channel single conversion mode and it is not supported for any of the other three conversion modes.

There is a 2-3 cycle synchronization latency from when the sample window ends to when the conversion window begins.

Figure 10-3 shows the ADC sample and conversion timing diagram when the ADC is configured in MANUAL sampling mode:

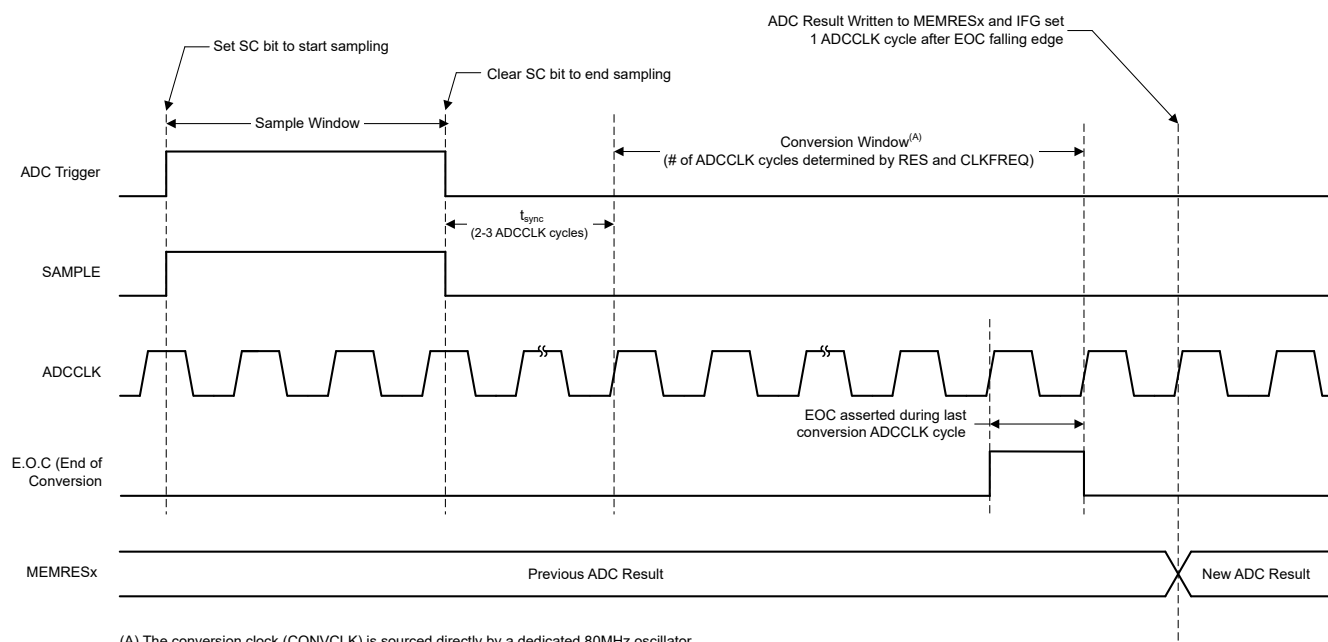


Figure 10-3. MANUAL Sampling Mode ADC Sample and Conversion Timing Diagram

Note

1. In MANUAL sampling mode the CCONRUN bit in the CLKCFG register must be set to 1.
2. When the reset value of PWRDN is set as '0' which has the default behavior of automatic power down, ADC wakeup/enable time needs to be considered before sample window. It means that after setting ENABLE bit in the PWREN register to enable ADC, then application software should set up a delay to wait the ADC wakeup/enable time before starting sampling. Refer to the device-specific data sheet for specifications on the ADC wakeup/enable time.
3. Once SC bit set by software, the SC bit is automatically cleared by hardware after the sample duration, which is the duration during which the analog signal is sampled. If the software attempts to set the SC bit before the sample duration + 2-3 cycles synchronization latency, TOVIFG flag will not be set.

10.2.9 Sampling Period

The sampling clock source is selected in the SYSCTL module using the SAMPCLK bits in the CLKCFG register. The desired sampling period for ADC operation can be generated using the internal clock divider and/or the sample timer, which applies to AUTO sampling mode. The internal clock divider is configured using the SCLKDIV bits in the CTL0 register and has divide options of 1, 2, 4, 8, 16, 24, 32, and 48.

The duration of the sampling period can be programmed to one of two user-defined values set by the SCOMP0 and SCOMP1 sample timer registers. The value in SCOMPx configures the sampling period by defining the number of sample time clocks to set the sample window to. The default SCOMPx sample timer value translates to 1 cycle wide sample pulse which allows the sampling period to be solely based on the sample clock and SCLKDIV. In general, there are three knobs that can be leveraged to control the sample period: SCOMPx, SCLKDIV, and the source of the sample clock.

When AUTO power down mode is selected using PWRDN=0, the module enable signal to the ADC peripheral is generated one sampling clock cycle after the sample signal is asserted. This should be considered by the user in the sample window calculation in addition to the ADC power time or settling time needs of other analog modules such as the Temperature Sensor, VREF, etc.

10.2.10 Conversion Modes

There are four conversion modes available in the ADC:

1. Single channel single conversion
 - The channel can be selected using MEMCTL
 - The selected channel is sampled and converted only once
 - Multiple conversions are performed when HW averaging is enabled
2. Repeat single channel conversion
 - The channel can be selected using MEMCTL
 - The selected channel is repeatedly sampled and converted until ENC is cleared by software
 - If the TRIG bit is set, a trigger is needed to move to the next conversion
 - Multiple conversions are performed when HW averaging is enabled
3. Sequence of channels conversion
 - Groups of channels can be formed using STARTADD, ENDADD, and MEMCTL registers
 - Each of the channels in the group is sampled and converted only once
 - Multiple conversions are performed on a channel during the sequence when HW averaging is enabled
 - The sequence will complete even if ENC is cleared in the middle of the sequence
4. Repeat sequence of channels conversion
 - Groups of channels can be formed using STARTADD, ENDADD, and MEMCTL registers
 - The group of channels are sampled and converted repeatedly until ENC is cleared by software
 - If the TRIG bit is set, a trigger is needed to move to the next conversion
 - When ENC is cleared the operation stops at the end of the last conversion
 - Multiple conversions are performed on a channel during a sequence when averaging is enabled

The following steps outline the recommended process for configuring the ADC for a desired conversion mode:

1. Use the CONSEQ bits in the CTL1 register to select the desired ADC conversion mode
2. Use the STARTADD bits in the CTL2 register to select which MEMCTLx is used for single conversion or as first MEMCTL for a sequence mode
3. If using a sequence mode, use the ENDADD bits in the CTL2 register to select which MEMCTLx is used for the last conversion of the sequence
4. Assign an ADC input channel to the appropriate MEMCTLx register using the CHANSEL bits
 - For sequence modes, you must assign an ADC input channel for each MEMCTLx that is part of the configured sequence
5. Select HW or SW trigger using the TRIGSRC bit in the CTL1 register
6. Select AUTO or MANUAL sampling mode using the SAMPMODE bit in the CTL1 register
 - If using AUTO mode, program the desired sample timer value in the SCOMPx register and use the STIME bits in the MEMCTLx register to select the appropriate sample timer source (SCOMP0 or SCOMP1)
7. If using repeat single channel or sequence conversion modes, program the TRIG bit in each MEMCTLx register to indicate if a trigger will be needed to step to the next MEMCTL in the sequence
8. Set the ENC bit in the CTL1 register to enable ADC conversions
9. The following table matrix depicts the next step of ADC configuration and usage based on the selected trigger and sampling modes:

Table 10-3. Trigger and Sample Mode ADC Usage Matrix

	Trigger Mode	
	SW Trigger	Event Trigger

Table 10-3. Trigger and Sample Mode ADC Usage Matrix (continued)

Sampling Mode	AUTO	- Set SC bit to start the sample phase (duration determined by sample timer) - Conversion starts once sample phase is over - In single channel single conversion, ENC is cleared when conversion is over - SC bit is automatically cleared once the trigger is captured - For repeat and sequence modes, if TRIG is set in MEMCTL, the SC bit needs to be set for the next conversion to proceed	- HW trigger starts the sample phase (duration determined by sample timer) - Conversion starts once sample phase is over - In single channel single conversion, ENC is cleared when conversion is over - For repeat and sequence modes, ADC waits for HW trigger or automatically starts the next conversion based on TRIG setting
	MANUAL	- Set SC bit to start the sample phase (SC bit is not automatically reset) - Clear the SC bit to end the sample phase and start the conversion - In single channel single conversion, ENC bit is cleared when conversion is over - Repeated/sequential conversion modes and HW averaging are NOT supported in this configuration	ADC operation is NOT supported in this configuration

10. The ADC results are stored in the MEMRES register of the associated MEMCTL (for example, the MEMCTL0 result is stored in MEMRES0).
- For repeat conversion modes, the result in MEMRES is updated after every associated MEMCTL conversion
11. For repeated conversion modes, clear the ENC bit to stop ADC operation

10.2.11 Data Format

The ADC supports two data formats – unsigned binary and 2's complement signed binary. Unsigned binary results are stored right-justified in the MEMRES register or FIFO. Signed binary results are stored left justified in the MEMRES register or FIFO.

Table 10-4. ADC Data Formats

Data Format	Resolution	Result Range (decimal)	Result Range (hex)
Unsigned	8-bit	0 to 255	0000h to 00FFh
	10-bit	0 to 1023	0000h to 03FFh
	12-bit	0 to 4095	0000h 0FFFh
Signed	8-bit	-128 to 127	8000h to 7F00h
	10-bit	-512 to 511	8000h to 7FC0h
	12-bit	-2048 to 2047	8000h to 7FF0h

10.2.12 Advanced Features

The following sections describe the additional features and benefits provided with the ADC peripheral and how to leverage them in an application.

10.2.12.1 Simultaneous Sampling

Some applications, such as current and voltage sensing, require measurement from multiple analog signals at the exact same time. In these circumstances, using multiple ADCs on a single MCU to perform simultaneous sampling is a requirement. Any device with multiple ADC peripherals in the MSPM0xx platform supports simultaneous sampling.

For this use-case, the ULPCLK should be used as the source for ADCCLK because it is also the clock for all of the peripherals in PD0. This means that it is already synchronized with the bus clock and therefore can ensure deterministic timing for the start of sampling. In addition to using the ULPCLK as the sample clock source for both ADC peripherals, the user is expected to select the same sample trigger and program the clock prescaler and/or SCOMP with the same values on both ADCs.

10.2.12.2 Window Comparator

There is one window comparator unit available in the ADC which can be used to check if the input signal is within predefined threshold values set by software. The ADC result that goes into MEMRES or FIFO is what gets checked against the threshold values of the window comparator.

Based on the comparison it can generate 3 interrupt conditions:

1. LOWIFG – Conversion result is below the Low threshold (WCLOW)
2. HIGHIFG – Conversion result is above the High threshold (WCHIGH)
3. INIFG – Conversion result is in between or equal to the Low and High thresholds

The window comparator low and high threshold values are global for all channels and the window comparison feature can be enabled for each channel as needed using the WINCOMP bit in the MEMCTL register.

When the ADC result data format (CTL2.DF) or resolution (CTL2.RES) configuration is changed, the window comparator threshold values are not reset by hardware and are retained as is. The software application is expected to reconfigure the threshold values as appropriate after changing the data format and/or resolution configuration.

10.2.12.3 DMA and FIFO Operation

The ADC has a dedicated interface for communicating to and from the DMA. This interface is useful to offload work from the CPU by using the DMA to store ADC results to memory automatically. [Figure 10-4](#) shows the signals that make up this interface:

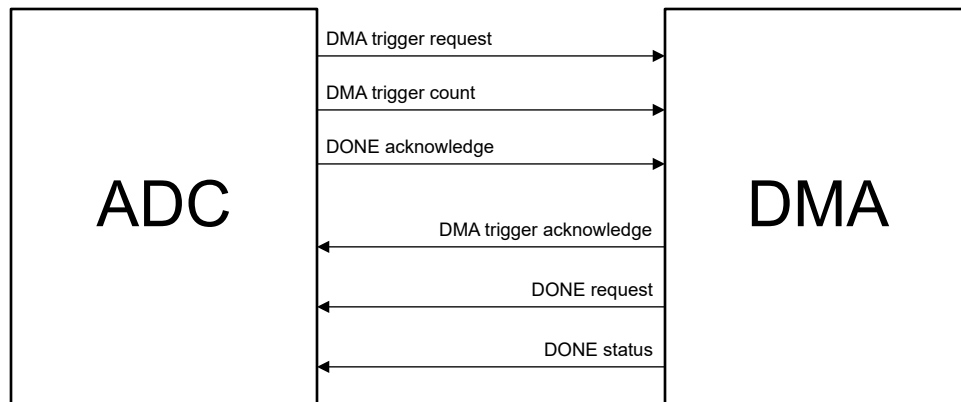


Figure 10-4. Internal ADC-DMA Interface

Note

The “DMA trigger count” signal indicates the number of samples that can be transferred by the DMA upon one trigger request. The “DONE status” signal is used by the ADC to generate the DMA DONE interrupt and it indicates if the DMA data transfer of programmed block size is completed.

The DMAEN bit in the CTL2 register is used to enable the DMA for ADC data transfer. The DMAEN bit is cleared by ADC hardware when the DMA “DONE status” signal is asserted. Software is expected to re-enable the DMA using DMAEN to arm the ADC to generate the next DMA trigger.

The ADC also incorporates an optional First-In-First-Out buffer to provide a way for ADC results to be stored for future use, such as transferring to memory by the DMA. Either the CPU or the DMA can be used to move data

from the ADC regardless of whether the FIFO is enabled or disabled. The memory result flags in the RIS register of the third event publisher serve as the FIFO threshold and can be unmasked to generate the DMA trigger.

The following sections explain the details of using the ADC+DMA/CPU in various conversion modes and with the FIFO enabled or disabled

ADC-DMA/CPU Operation in Non-FIFO Mode (FIFOEN=0)

ADC-DMA/CPU Operation in Non-FIFO Mode (FIFOEN=0)

- Single Conversion and Repeat Single Conversion
 - Configure STARTADD bits to select the desired MEMCTLx register
 - MEMCTLx **is correlated to** MEMRESx
 - MEMRESx **is correlated to** MEMRESIFGx
 - Configure MEMCTL CHANSEL bits to select the desired ADC channel
 - Conversion data is available in MEMRESx
 - MEMRESIFGx can be set to generate a CPU interrupt or the DMA trigger
 - SAMPCNT **must be programmed to 1** by SW for DMA operation
 - The conversion overflow flag OVIFG is set when the ADC updates MEMRESx before the previous sample is read by the CPU or DMA
 - The conversion underflow flag UVIFG is set when the CPU or DMA reads the MEMRESx register before the next conversion result is available
- Sequence Conversion and Repeat Sequence Conversion
 - Configure STARTADD bits to select the first MEMCTL in the sequence
 - Configure ENDADD bits to select the last MEMCTL in the sequence
 - MEMCTLx **is correlated to** MEMRESx
 - MEMRESx **is correlated to** MEMRESIFGx
 - Configure each MEMCTLx CHANSEL bits to select the desired ADC channels
 - Conversion data is available in MEMRESx
 - MEMRESIFGx can be set to generate a CPU interrupt or the DMA trigger
 - SAMPCNT must be programmed by SW to a **suitable value based on threshold setting** by SW for DMA operation
 - The conversion overflow flag OVIFG is set when the ADC updates MEMRESx before the previous sample is read by the CPU or DMA
 - The conversion underflow flag UVIFG is set when the CPU or DMA reads the MEMRESx register before the next conversion result is available

Note

For DMA based operation, the MEMCTL start address should be smaller than the end address for single sequence conversion as DMA source does not roll back. Repeat sequence conversion mode does not support DMA based data transfer because the DMA does not support circular addressing mode.

ADC-DMA/CPU Operation in FIFO Mode (FIFOEN=1)

- Single Conversion and Repeat Single Conversion
 - Configure STARTADD bits to select the desired MEMCTLx register
 - MEMCTLx **is NOT correlated to** MEMRESx
 - MEMRESx **is correlated to** MEMRESIFGx
 - Configure MEMCTL CHANSEL bits to select the desired ADC channel
 - Conversion data is loaded sequentially into MEMRES0,1,2,...N (organized as a FIFO)
 - The CPU or DMA must read ADC samples from the dedicated FIFODAT register and not from MEMRES registers directly
 - Data in the FIFO is always compacted with two samples and provided as 32-bit data upon a FIFODAT read by CPU or DMA

- MEMRESIFGx can be used as a threshold condition to generate a CPU interrupt or DMA trigger
 - For full use of the FIFO, the last MEMRESIFG can be used
- SAMPCNT must be programmed by SW to a **suitable value based on threshold setting** for DMA operation
- The conversion overflow flag OVIFG is set when the ADC updates MEMRESx before the previous sample is read by the CPU or DMA
- The conversion underflow flag UVIFG is set when the CPU or DMA reads the FIFODAT register before the conversion result is available in the MEMRESx registers.

Note

Single conversion mode with FIFO enabled is not recommended for CPU or DMA based operation. It will lead to underflow condition and unwanted 16-bit data will have to be discarded in software.

- Sequence Conversion and Repeat Sequence Conversion
 - Configure STARTADD bits to select the first MEMCTL in the sequence
 - Configure ENDADD bits to select the last MEMCTL in the sequence
 - MEMCTLx is **NOT correlated** to MEMRESx
 - MEMRESx is **correlated** to MEMRESIFGx
 - Configure each MEMCTLx CHANSEL bits to select the desired ADC channels
 - Conversion data is loaded sequentially into MEMRES0,1,2,...N (organized as a FIFO)
 - The CPU or DMA must read ADC samples from the dedicated FIFODAT register and not from MEMRES registers directly
 - Data in the FIFO is always compacted with two samples and provided as 32-bit data upon a FIFODAT read by CPU or DMA
 - MEMRESIFGx can be used as a threshold condition to generate a CPU interrupt or DMA trigger
 - For full use of the FIFO, the last MEMRESIFG can be used
 - SAMPCNT must be programmed by SW to a **suitable value based on threshold setting** for DMA operation
 - The conversion overflow flag OVIFG is set when the ADC updates MEMRESx before the previous sample is read by the CPU or DMA
 - The conversion underflow flag UVIFG is set when the CPU or DMA reads the FIFODAT register before the conversion result is available in the MEMRESx registers

Note

- The data in FIFODAT register won't be cleared automatically after CPU or DMA reads. New conversion data overwrites the previous data in FIFODAT register.
- To ensure synchronized reading of bytes from the 32-bit FIFO, which stores 16-bit samples, specific DMA triggers can be used. In particular, selecting MEMRES1, MEMRES3, MEMRES5, MEMRES7, MEMRES9 and MEMRES11 will synchronize the reading of bytes from the FIFO with the corresponding MEMRESx bytes.
- If the ADC is disabled during either the repeat sequence mode or normal repeat mode, it's worth noting that an additional conversion may occur before the ADC completely stops.

Table 10-5. ADC-DMA/CPU Operation Summary Matrix

Conversion Mode	FIFO Disabled (FIFOEN=0) Samples not compacted Read from MEMRESx registers directly		FIFO Enabled (FIFOEN=1) Samples always compacted Read from FIFODAT register only	
	CPU Read/Write	DMA Read/Write	CPU Read/Write	DMA Read/Write
Single	Supported	Supported SAMPCNT=1 Sample in 16 bits	Not recommended Underflow flag will set Unwanted 16-bits should be ignored	Not recommended Underflow flag will set Unwanted 16-bits should be ignored

Table 10-5. ADC-DMA/CPU Operation Summary Matrix (continued)

Conversion Mode	FIFO Disabled (FIFOEN=0) Samples not compacted Read from MEMRESx registers directly		FIFO Enabled (FIFOEN=1) Samples always compacted Read from FIFODAT register only	
	CPU Read/Write	DMA Read/Write	CPU Read/Write	DMA Read/Write
Repeat Single	Supported	Supported SAMP CNT=1 Sample in 16 bits	Supported MEMRESIFG=CPU interrupt FIFODAT read in 32-bits	Supported MEMRESIFG=DMA trigger SAMP CNT=Samples in 32-bits
Sequence	Supported	Supported SAMP CNT=Sample in 16 bits STARTADD<ENDADD	Supported MEMRESIFG=CPU interrupt FIFODAT read in 32-bits	Supported MEMRESIFG=DMA trigger SAMP CNT=Samples in 32-bits
Repeat Sequence	Supported	Not Supported	Supported MEMRESIFG=CPU interrupt FIFODAT read in 32-bits	Supported MEMRESIFG=DMA trigger SAMP CNT=Samples in 32-bits

10.2.12.4 Analog Peripheral Interconnection

The MSPM0 platform of MCUs provides a rich set of high-performance analog peripherals which can interact with each other to perform various analog signal chain functions. The items below describe how the ADC interacts with the other on-board analog peripherals:

ADC with Internal Reference Module (VREF)

The ADC has a dedicated enable request and ready interface with the internal voltage reference module. VREF enable is asserted upon sample trigger while internal reference buffer is selected for ADC operation. The ready response from VREF is captured in the ADC status register (REFBUFRDY).

The ready response from VREF **does not** gate the sample phase in the ADC. The ADC sample window is started upon the sample trigger and the settling time of the internal reference buffer needs to be considered in the sample period as appropriate. Software can enable the VREF module using the software enable bit so that it is already settled by the time the ADC starts sampling the input channel. In this case, the sampling time can be smaller and does not have to account for the enable time of VREF.

ADC with OPA

The ADC provides a burnout current source (BCS) enable signal to an OPA peripheral when it is selected as the ADC input channel and the BCSEN bit is set in the MEMCTL register. This signal allows the ADC peripheral to work in unison with the OPA peripheral to monitor sensor health and detect faults. The BCS enable signal remains high for all conversions during hardware averaging operation. The ADC receives a chop request signal and provides a chop state control signal to the OPA peripheral when ADC-assisted OPA chopping is activated and the hardware averaging feature is enabled. The chop state control signal toggles at the end of each conversion during hardware averaging operation. The number of samples averaged should be programmed to an even value when ADC-assisted OPA chopping is activated.

ADC with Temperature Sensor

The temperature sensor module enable signal is generated by the ADC when the temp sense channel is selected. The settling time of the temperature sensor should be accounted for in the sample period as there is no ready response from the temperature sensor.

10.2.13 Status Register

The ADC status register, STATUS, contains two bits – REFBUFRDY and BUSY.

- REFBUFRDY is set when the ADC receives the ready signal from the internal reference buffer (VREF/REFBUF) after asserting the enable request
- BUSY equaling '1' indicates that the ADC is busy performing a sample or convert operation

- For **single channel single conversion**, it signals that a trigger has been received and sample or conversion is ongoing. BUSY will be cleared when the conversion completes
- For **repeat single conversion**, it signals that repeat single operation has begun and has not ended. BUSY will be cleared when ENC is written '0' and the last conversion completes
- For **sequence of channels conversion**, it signals that the sequence of channels conversion has started. BUSY will be cleared at the end of the sequence
- For **repeat sequence of channels conversion**, it signals the repeat sequence is ongoing. BUSY will be cleared when ENC is written '0' and the last conversion completes

10.2.14 ADC Events

The ADC peripheral contains three [event publishers](#) and one [event subscriber](#).

One event publisher (CPU_INT) manages ADC interrupt requests (IRQs) to the CPU subsystem through a [static event route](#). The second event publisher (GEN_EVENT) can be used to publish ADC events to a subscriber through a [generic event route channel](#). The third event publisher (DMA_TRIG) can be used as an ADC-to- DMA trigger to send ADC events directly to the DMA through a [DMA event route](#).

The event subscriber (FSUB_0) can be used to subscribe to events which are published to the event fabric through a [generic event route channel](#).

The ADC events are summarized in [Table 10-6](#).

Table 10-6. ADC Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU interrupt event	Publisher	ADC	CPU Subsystem	Static route	CPU_INT registers	Fixed interrupt route from ADC to CPU
Generic publisher event	Publisher	ADC	Generic event channel	Generic route (FPUB_0)	GEN_EVENT registers, FPUB_0 register	Trigger generic event channel from ADC
DMA trigger event	Publisher	ADC	DMA	DMA route	DMA_TRIG registers	Fixed trigger route from ADC to DMA
Generic subscriber event	Subscriber	Other peripherals	ADC	Generic route (FSUB_0)	FSUB_0	ADC subscription to generic event channel

10.2.14.1 CPU Interrupt Event Publisher (CPU_INT)

The ADC peripheral provides many interrupt sources which can be configured to source a [CPU interrupt event](#). In order of decreasing interrupt priority, the CPU interrupt events from the ADC are given in [Table 10-7](#).

Table 10-7. ADC CPU Interrupt Event Conditions (CPU_INT)

Index (IIDX)	Name	Description
0x0	NO_INTR	No bit set (IIDX.STAT = 0) means there is no pending interrupt request
0x1	OVIFG	Conversion overflow interrupt flag is set when the ADC updates MEMRESx before the previous sample is read by the CPU or DMA
0x2	TOVIFG	Sequence conversion time overflow interrupt flag is set when the ADC receives a new sampling trigger while the previous sample+conversion is still in progress
0x3	HIGHIFG	High threshold compare interrupt flag is set when the MEMRESx result register is higher than the WCHIGH threshold of the window comparator
0x4	LOWIFG	Low threshold compare interrupt flag is set when the MEMRESx result register is lower than the WCLOW threshold of the window comparator
0x5	INIFG	In-range comparator interrupt flag is set when the MEMRESx result register is within the range of WCLOW and WCHIGH of the window comparator

Table 10-7. ADC CPU Interrupt Event Conditions (CPU_INT) (continued)

Index (IIDX)	Name	Description
0x6	DMADONE	DMA done interrupt flag is set when the DMA data transfer of programmed block size is completed
0x7	UVIFG	Conversion underflow interrupt flag, the UVIFG flag is set when the CPU or DMA reads the MEMRESx register before the next conversion result is available
0x9 up to 0x20	MEMRESIFG[0 up to 24] ⁽¹⁾	Memory register interrupt flag is set when MEMRESx is loaded with a new conversion result

(1) Refer to the device-specific data sheet to see how many conversion-result storage registers (MEMRES) your device supports.

The CPU interrupt event configuration is managed with the CPU_INT event management registers. Interrupt (RIS) flags are cleared upon software reading the IIDX register or writing to the respective ICLR register bits. See [Section 7.2.5](#) for guidance on configuring the Event registers for CPU interrupts.

10.2.14.2 Generic Event Publisher (GEN_EVENT)

The ADC peripheral provides 4 interrupt sources, one of which can be configured to publish an event to a generic event route channel. [Table 10-8](#) lists these interrupt sources.

Table 10-8. ADC Generic Event Publisher Conditions (GEN_EVENT)

Index	Name	Description
0x0	NO_INTR	No bit set means there is no pending interrupt request
0x3	HIGHIFG	High threshold compare interrupt flag is set when the MEMRESx result register is higher than the WCHIGH threshold of the window comparator
0x4	LOWIFG	Low threshold compare interrupt flag is set when the MEMRESx result register is lower than the WCLOW threshold of the window comparator
0x5	INIFG	In-range comparator interrupt flag is set when the MEMRESx result register is within the range of WCLOW and WCHIGH of the window comparator
0x9	MEMRESIFG0	Memory register interrupt flag is set when MEMRES0 is loaded with a new conversion result

The generic event publisher configuration is managed with the GEN_EVENT event management registers. Interrupt (RIS) flags are cleared based on acknowledgment (ACK) signal from the subscriber module received over the event fabric. See [Section 7.2.5](#) for guidance on configuring the Event registers for generic event publishers.

The generic event channel which GEN_EVENT is to publish to must be selected by writing the target generic channel ID to the **FPUB_0** register in the ADC. See [Section 7.1.3.3](#) for guidance on configuring generic event routes.

If this publisher is not used in an application, the FPUB_0 register can be left in a disconnected state (set equal to zero) and no events should be unmasked through the MIS register in the ADC GEN_EVENT register set.

10.2.14.3 DMA Trigger Event Publisher (DMA_TRIG)

The ADC module provides many interrupt sources which can be configured to source the DMA trigger. In order of decreasing interrupt priority, the DMA trigger events from the ADC are given in [Table 10-9](#). When the DMA channel is needed by the ADC, the DMA trigger should be unmasked in the IMASK register of DMA_TRIG and the DMA should be configured as needed to support the ADC operation.

Table 10-9. ADC DMA Trigger Event Conditions (DMA_TRIG)

Index (IIDX)	Name	Description
0x0	NO_INTR	No bit set (IIDX.STAT = 0) means there is no pending interrupt request
0x9 up to 0x20	MEMRESIFG[0 up to 24] ⁽¹⁾	Memory register interrupt flag is set when MEMRESx is loaded with a new conversion result

(1) Check the device-specific data sheet to see how many conversion-result storage registers (MEMRES) your device supports.

The DMA trigger event configuration is managed with the DMA_TRIG event management registers. The interrupt (RIS) flags are cleared based on ACK from DMA. See [Section 7.2.5](#) for guidance on configuring the Event registers for DMA triggers.

10.2.14.4 Generic Event Subscriber (FSUB_0)

The ADC peripheral supports receiving events routed through a generic channel from other peripherals. Refer to [Section 7.1.3.3](#) and [Section 7.2.3](#) for how generic event route works.

Once the channel to be used is determined, and both the publisher and subscriber ports for the peripherals being connected are known, use the steps below to establish the event connection. In this example, a GPIO triggered ADC application will be configured, using GPIO Port A to publish an event to generic channel 1, with ADC0 subscribing to generic channel 1 as a start-of-conversion trigger.

1. Configure the GEN_EVENT registers of GPIO Port A to set the event request based on the appropriate event (for example, a DIN rise event).
2. Store 0x1 into the FPUB_0 register of GPIO Port A to publish the GPIO event selected by the GEN_EVENT registers to generic route channel 1. Channel 1 must not be in use by another peripheral.
3. Store 0x1 the FSUB_0 register of ADC0 so that ADC0 is listening for events published by the timer to channel 1.
4. Configure ADC0 to trigger from the subscriber port according to the configuration instructions in [Section 10.2.8](#)
5. Configure and enable the appropriate GPIO pin to monitor input voltage events

10.3 ADC12 Registers

[Table 10-10](#) lists the memory-mapped registers for the ADC12 registers. All register offset addresses not listed in [Table 10-10](#) should be considered as reserved locations and the register contents should not be modified.

Table 10-10. ADC12 Registers

Offset	Acronym	Register Name	Group	Section
400h	FSUB_0	Subscriber Configuration Register.		Go
444h	FPUB_1	Publisher Configuration Register.		Go
800h	PWREN	Power enable		Go
804h	RSTCTL	Reset Control		Go
808h	CLKCFG	ADC clock configuration Register		Go
814h	STAT	Status Register		Go
1020h	IIDX	Interrupt index	CPU_INT	Go
1028h	IMASK	Interrupt mask	CPU_INT	Go
1030h	RIS	Raw interrupt status	CPU_INT	Go
1038h	MIS	Masked interrupt status	CPU_INT	Go
1040h	ISSET	Interrupt set	CPU_INT	Go
1048h	ICLR	Interrupt clear	CPU_INT	Go
1050h	IIDX	Interrupt index	GEN_EVENT	Go
1058h	IMASK	Interrupt mask	GEN_EVENT	Go
1060h	RIS	Raw interrupt status	GEN_EVENT	Go
1068h	MIS	Masked interrupt status	GEN_EVENT	Go
1070h	ISSET	Interrupt set	GEN_EVENT	Go
1078h	ICLR	Interrupt clear	GEN_EVENT	Go
1080h	IIDX	Interrupt index	DMA_TRIG	Go
1088h	IMASK	Interrupt mask extension	DMA_TRIG	Go
1090h	RIS	Raw interrupt status extension	DMA_TRIG	Go
1098h	MIS	Masked interrupt status extension	DMA_TRIG	Go
10A0h	ISSET	Interrupt set extension	DMA_TRIG	Go
10A8h	ICLR	Interrupt clear extension	DMA_TRIG	Go
10E0h	EVT_MODE	Event Mode		Go
10FCh	DESC	Module Description		Go
1100h	CTL0	Control Register 0		Go
1104h	CTL1	Control Register 1		Go
1108h	CTL2	Control Register 2		Go
110Ch	CTL3	Control Register 3		Go
1110h	CLKFREQ	Sample Clock Frequency Range Register		Go
1114h	SCOMP0	Sample Time Compare 0 Register		Go
1118h	SCOMP1	Sample Time Compare 1 Register		Go
111Ch	REFCFG	Reference Buffer Configuration Register		Go
1148h	WCLOW	Window Comparator Low Threshold Register		Go
1150h	WCHIGH	Window Comparator High Threshold Register		Go
1160h	FIFODATA	FIFO Data Register		Go
1170h	ASCRES	ASC Result Register		Go

Table 10-10. ADC12 Registers (continued)

Offset	Acronym	Register Name	Group	Section
1180h + formula	MEMCTL[y]	Conversion Memory Control Register		Go
1280h + formula	MEMRES[y]	Memory Result Register		Go
1340h	STATUS	Status Register		Go

Complex bit access types are encoded to fit into small table cells. [Table 10-11](#) shows the codes that are used for access types in this section.

Table 10-11. ADC12 Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
RH	R H	Read Set or cleared by hardware
Write Type		
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value
Register Array Variables		
i,j,k,l,m,n		When these variables are used in a register name, an offset, or an address, they refer to the value of a register array where the register is part of a group of repeating registers. The register groups form a hierarchical structure and the array is represented with a formula.
y		When this variable is used in a register name, an offset, or an address it refers to the value of a register array.

10.3.1 FSUB_0 (Offset = 400h) [Reset = 00000000h]

FSUB_0 is shown in [Figure 10-5](#) and described in [Table 10-12](#).

Return to the [Summary Table](#).

Subscriber port

Figure 10-5. FSUB_0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-0h												R/W-0h			

Table 10-12. FSUB_0 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 255.

10.3.2 FPUB_1 (Offset = 444h) [Reset = 00000000h]

FPUB_1 is shown in [Figure 10-6](#) and described in [Table 10-13](#).

Return to the [Summary Table](#).

Publisher port

Figure 10-6. FPUB_1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-0h												R/W-0h			

Table 10-13. FPUB_1 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 255.

10.3.3 PWREN (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 10-7](#) and described in [Table 10-14](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 10-7. PWREN

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-0h							R/WK-0h

Table 10-14. PWREN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	ENABLE	R/WK	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

10.3.4 RSTCTL (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 10-8](#) and described in [Table 10-15](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 10-8. RSTCTL

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCL R	RESETASSERT
W-0h						WK-0h	WK-0h

Table 10-15. RSTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	W	0h	
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

10.3.5 CLKCFG (Offset = 808h) [Reset = 00000000h]

CLKCFG is shown in [Figure 10-9](#) and described in [Table 10-16](#).

Return to the [Summary Table](#).

ADC clock configuration

Figure 10-9. CLKCFG

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED		CCONSTOP	CCONRUN	RESERVED		SAMPCLK	
R/W-0h		R/W-0h	R/W-0h	R/W-0h		R/W-0h	

Table 10-16. CLKCFG Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key A9h = KEY to allow write access to this register
23-6	RESERVED	R/W	0h	
5	CCONSTOP	R/W	0h	CCONSTOP: Forces SYSOSC to run at base frequency when device is in STOP mode which can be used as ADC sample or conversion clock source. 0h = ADC conversion clock source is not kept continuously on during STOP mode. 1h = ADC conversion clock source kept continuously on during STOP mode.
4	CCONRUN	R/W	0h	CCONRUN: Forces SYSOSC to run at base frequency when device is in RUN mode which can be used as ADC sample or conversion clock source. 0h = ADC conversion clock source is not kept continuously on during RUN mode. 1h = ADC conversion clock source kept continuously on during RUN mode.
3-2	RESERVED	R/W	0h	
1-0	SAMPCLK	R/W	0h	ADC sample clock source selection. 0h = ULPCLK is the source of ADC sample clock. 1h = SYSOSC is the source of ADC sample clock. 2h = HFCLK clock is the source of ADC sample clock. Note : HFCLK may not be available on all the devices.

10.3.6 STAT (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 10-10](#) and described in [Table 10-17](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 10-10. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED							
R-							

Table 10-17. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0h	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

10.3.7 IIDX (Offset = 1020h) [Reset = 00000000h]

IIDX is shown in [Figure 10-11](#) and described in [Table 10-18](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. 0x0 means no event pending. Interrupt 1 is the highest priority, 2 next highest, 4, 8, ... 2^{31} is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in the RIS and MIS are cleared as well. After a read from the CPU (not from the debug interface), the register must be updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 10-11. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																						STAT									
R-0h																						R-0h									

Table 10-18. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-10	RESERVED	R	0h	
9-0	STAT	R	0h	Interrupt index status 00h = No bit is set means there is no pending interrupt request 01h = MEMRESx overflow interrupt 02h = Sequence Conversion time overflow interrupt 03h = High threshold compare interrupt 04h = Low threshold compare interrupt 05h = Primary Sequence In range comparator interrupt 06h = DMA done interrupt, generated on DMA transfer completion, 07h = MEMRESx underflow interrupt 09h = MEMRES0 data loaded interrupt Ah = MEMRES1 data loaded interrupt Bh = MEMRES2 data loaded interrupt Ch = MEMRES3 data loaded interrupt Dh = MEMRES4 data loaded interrupt Eh = MEMRES5 data loaded interrupt Fh = MEMRES6 data loaded interrupt 10h = MEMRES7 data loaded interrupt 11h = MEMRES8 data loaded interrupt 12h = MEMRES9 data loaded interrupt 13h = MEMRES10 data loaded interrupt 14h = MEMRES11 data loaded interrupt 15h = MEMRES12 data loaded interrupt 16h = MEMRES13 data loaded interrupt 17h = MEMRES14 data loaded interrupt 18h = MEMRES15 data loaded interrupt 19h = MEMRES16 data loaded interrupt 1Ah = MEMRES17 data loaded interrupt 1Bh = MEMRES18 data loaded interrupt 1Ch = MEMRES19 data loaded interrupt 1Dh = MEMRES20 data loaded interrupt 1Eh = MEMRES21 data loaded interrupt 1Fh = MEMRES22 data loaded interrupt 20h = MEMRES23 data loaded interrupt

10.3.8 IMASK (Offset = 1028h) [Reset = 00000000h]

IMASK is shown in [Figure 10-12](#) and described in [Table 10-19](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 10-12. IMASK

31	30	29	28	27	26	25	24
MEMRESIFG23	MEMRESIFG22	MEMRESIFG21	MEMRESIFG20	MEMRESIFG19	MEMRESIFG18	MEMRESIFG17	MEMRESIFG16
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
MEMRESIFG15	MEMRESIFG14	MEMRESIFG13	MEMRESIFG12	MEMRESIFG11	MEMRESIFG10	MEMRESIFG9	MEMRESIFG8
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
MEMRESIFG7	MEMRESIFG6	MEMRESIFG5	MEMRESIFG4	MEMRESIFG3	MEMRESIFG2	MEMRESIFG1	MEMRESIFG0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
ASCDONE	UVIFG	DMADONE	INIFG	LOWIFG	HIGHIFG	TOVIFG	OVIFG
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 10-19. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31	MEMRESIFG23	R/W	0h	Raw interrupt status for MEMRES23. This bit is set to 1 when MEMRES23 is loaded with a new conversion result. Reading MEMRES23 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
30	MEMRESIFG22	R/W	0h	Raw interrupt status for MEMRES22. This bit is set to 1 when MEMRES22 is loaded with a new conversion result. Reading MEMRES22 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
29	MEMRESIFG21	R/W	0h	Raw interrupt status for MEMRES21. This bit is set to 1 when MEMRES21 is loaded with a new conversion result. Reading MEMRES21 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
28	MEMRESIFG20	R/W	0h	Raw interrupt status for MEMRES20. This bit is set to 1 when MEMRES20 is loaded with a new conversion result. Reading MEMRES20 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-19. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
27	MEMRESIFG19	R/W	0h	Raw interrupt status for MEMRES19. This bit is set to 1 when MEMRES19 is loaded with a new conversion result. Reading MEMRES19 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
26	MEMRESIFG18	R/W	0h	Raw interrupt status for MEMRES18. This bit is set to 1 when MEMRES18 is loaded with a new conversion result. Reading MEMRES18 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
25	MEMRESIFG17	R/W	0h	Raw interrupt status for MEMRES17. This bit is set to 1 when MEMRES17 is loaded with a new conversion result. Reading MEMRES17 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
24	MEMRESIFG16	R/W	0h	Raw interrupt status for MEMRES16. This bit is set to 1 when MEMRES16 is loaded with a new conversion result. Reading MEMRES16 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
23	MEMRESIFG15	R/W	0h	Raw interrupt status for MEMRES15. This bit is set to 1 when MEMRES15 is loaded with a new conversion result. Reading MEMRES15 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
22	MEMRESIFG14	R/W	0h	Raw interrupt status for MEMRES14. This bit is set to 1 when MEMRES14 is loaded with a new conversion result. Reading MEMRES14 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
21	MEMRESIFG13	R/W	0h	Raw interrupt status for MEMRES13. This bit is set to 1 when MEMRES13 is loaded with a new conversion result. Reading MEMRES13 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
20	MEMRESIFG12	R/W	0h	Raw interrupt status for MEMRES12. This bit is set to 1 when MEMRES12 is loaded with a new conversion result. Reading MEMRES12 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-19. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	MEMRESIFG11	R/W	0h	Raw interrupt status for MEMRES11. This bit is set to 1 when MEMRES11 is loaded with a new conversion result. Reading MEMRES11 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
18	MEMRESIFG10	R/W	0h	Raw interrupt status for MEMRES10. This bit is set to 1 when MEMRES10 is loaded with a new conversion result. Reading MEMRES10 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
17	MEMRESIFG9	R/W	0h	Raw interrupt status for MEMRES9. This bit is set to 1 when MEMRES9 is loaded with a new conversion result. Reading MEMRES9 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
16	MEMRESIFG8	R/W	0h	Raw interrupt status for MEMRES8. This bit is set to 1 when MEMRES8 is loaded with a new conversion result. Reading MEMRES8 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
15	MEMRESIFG7	R/W	0h	Raw interrupt status for MEMRES7. This bit is set to 1 when MEMRES7 is loaded with a new conversion result. Reading MEMRES7 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
14	MEMRESIFG6	R/W	0h	Raw interrupt status for MEMRES6. This bit is set to 1 when MEMRES6 is loaded with a new conversion result. Reading MEMRES6 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
13	MEMRESIFG5	R/W	0h	Raw interrupt status for MEMRES5. This bit is set to 1 when MEMRES5 is loaded with a new conversion result. Reading MEMRES5 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
12	MEMRESIFG4	R/W	0h	Raw interrupt status for MEMRES4. This bit is set to 1 when MEMRES4 is loaded with a new conversion result. Reading MEMRES4 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-19. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	MEMRESIFG3	R/W	0h	Raw interrupt status for MEMRES3. This bit is set to 1 when MEMRES3 is loaded with a new conversion result. Reading MEMRES3 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
10	MEMRESIFG2	R/W	0h	Raw interrupt status for MEMRES2. This bit is set to 1 when MEMRES2 is loaded with a new conversion result. Reading MEMRES2 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
9	MEMRESIFG1	R/W	0h	Raw interrupt status for MEMRES1. This bit is set to 1 when MEMRES1 is loaded with a new conversion result. Reading MEMRES1 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
8	MEMRESIFG0	R/W	0h	Raw interrupt status for MEMRES0. This bit is set to 1 when MEMRES0 is loaded with a new conversion result. Reading MEMRES0 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
7	ASCDONE	R/W	0h	Mask for ASC done raw interrupt flag 0h = Interrupt is not pending. 1h = Interrupt is pending.
6	UVIFG	R/W	0h	Raw interrupt flag for MEMRESx underflow. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
5	DMADONE	R/W	0h	Raw interrupt flag for DMADONE. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
4	INIFG	R/W	0h	Mask INIFG in MIS_EX register. 0h = Interrupt is not pending. 1h = Interrupt is pending.
3	LOWIFG	R/W	0h	Raw interrupt flag for the MEMRESx result register being below than the WCLOWx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
2	HIGHIFG	R/W	0h	Raw interrupt flag for the MEMRESx result register being higher than the WCHIGHx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
1	TOVIFG	R/W	0h	Raw interrupt flag for sequence conversion timeout overflow. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.

Table 10-19. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
0	OVIFG	R/W	0h	Raw interrupt flag for MEMRESx overflow. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.

10.3.9 RIS (Offset = 1030h) [Reset = 00000000h]

RIS is shown in [Figure 10-13](#) and described in [Table 10-20](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 10-13. RIS

31	30	29	28	27	26	25	24
MEMRESIFG23	MEMRESIFG22	MEMRESIFG21	MEMRESIFG20	MEMRESIFG19	MEMRESIFG18	MEMRESIFG17	MEMRESIFG16
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
MEMRESIFG15	MEMRESIFG14	MEMRESIFG13	MEMRESIFG12	MEMRESIFG11	MEMRESIFG10	MEMRESIFG9	MEMRESIFG8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
MEMRESIFG7	MEMRESIFG6	MEMRESIFG5	MEMRESIFG4	MEMRESIFG3	MEMRESIFG2	MEMRESIFG1	MEMRESIFG0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
ASCDONE	UVIFG	DMADONE	INIFG	LOWIFG	HIGHIFG	TOVIFG	OVIFG
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 10-20. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31	MEMRESIFG23	R	0h	Raw interrupt status for MEMRES23. This bit is set to 1 when MEMRES23 is loaded with a new conversion result. Reading MEMRES23 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
30	MEMRESIFG22	R	0h	Raw interrupt status for MEMRES22. This bit is set to 1 when MEMRES22 is loaded with a new conversion result. Reading MEMRES22 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
29	MEMRESIFG21	R	0h	Raw interrupt status for MEMRES21. This bit is set to 1 when MEMRES21 is loaded with a new conversion result. Reading MEMRES21 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
28	MEMRESIFG20	R	0h	Raw interrupt status for MEMRES20. This bit is set to 1 when MEMRES20 is loaded with a new conversion result. Reading MEMRES20 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-20. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
27	MEMRESIFG19	R	0h	Raw interrupt status for MEMRES19. This bit is set to 1 when MEMRES19 is loaded with a new conversion result. Reading MEMRES19 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
26	MEMRESIFG18	R	0h	Raw interrupt status for MEMRES18. This bit is set to 1 when MEMRES18 is loaded with a new conversion result. Reading MEMRES18 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
25	MEMRESIFG17	R	0h	Raw interrupt status for MEMRES17. This bit is set to 1 when MEMRES17 is loaded with a new conversion result. Reading MEMRES17 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
24	MEMRESIFG16	R	0h	Raw interrupt status for MEMRES16. This bit is set to 1 when MEMRES16 is loaded with a new conversion result. Reading MEMRES16 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
23	MEMRESIFG15	R	0h	Raw interrupt status for MEMRES15. This bit is set to 1 when MEMRES15 is loaded with a new conversion result. Reading MEMRES15 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
22	MEMRESIFG14	R	0h	Raw interrupt status for MEMRES14. This bit is set to 1 when MEMRES14 is loaded with a new conversion result. Reading MEMRES14 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
21	MEMRESIFG13	R	0h	Raw interrupt status for MEMRES13. This bit is set to 1 when MEMRES13 is loaded with a new conversion result. Reading MEMRES13 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
20	MEMRESIFG12	R	0h	Raw interrupt status for MEMRES12. This bit is set to 1 when MEMRES12 is loaded with a new conversion result. Reading MEMRES12 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-20. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	MEMRESIFG11	R	0h	Raw interrupt status for MEMRES11. This bit is set to 1 when MEMRES11 is loaded with a new conversion result. Reading MEMRES11 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
18	MEMRESIFG10	R	0h	Raw interrupt status for MEMRES10. This bit is set to 1 when MEMRES10 is loaded with a new conversion result. Reading MEMRES10 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
17	MEMRESIFG9	R	0h	Raw interrupt status for MEMRES9. This bit is set to 1 when MEMRES9 is loaded with a new conversion result. Reading MEMRES9 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
16	MEMRESIFG8	R	0h	Raw interrupt status for MEMRES8. This bit is set to 1 when MEMRES8 is loaded with a new conversion result. Reading MEMRES8 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
15	MEMRESIFG7	R	0h	Raw interrupt status for MEMRES7. This bit is set to 1 when MEMRES7 is loaded with a new conversion result. Reading MEMRES7 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
14	MEMRESIFG6	R	0h	Raw interrupt status for MEMRES6. This bit is set to 1 when MEMRES6 is loaded with a new conversion result. Reading MEMRES6 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
13	MEMRESIFG5	R	0h	Raw interrupt status for MEMRES5. This bit is set to 1 when MEMRES5 is loaded with a new conversion result. Reading MEMRES5 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
12	MEMRESIFG4	R	0h	Raw interrupt status for MEMRES4. This bit is set to 1 when MEMRES4 is loaded with a new conversion result. Reading MEMRES4 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-20. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	MEMRESIFG3	R	0h	Raw interrupt status for MEMRES3. This bit is set to 1 when MEMRES3 is loaded with a new conversion result. Reading MEMRES3 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
10	MEMRESIFG2	R	0h	Raw interrupt status for MEMRES2. This bit is set to 1 when MEMRES2 is loaded with a new conversion result. Reading MEMRES2 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
9	MEMRESIFG1	R	0h	Raw interrupt status for MEMRES1. This bit is set to 1 when MEMRES1 is loaded with a new conversion result. Reading MEMRES1 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
8	MEMRESIFG0	R	0h	Raw interrupt status for MEMRES0. This bit is set to 1 when MEMRES0 is loaded with a new conversion result. Reading MEMRES0 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
7	ASCDONE	R	0h	Raw interrupt flag for ASC done 0h = Interrupt is not pending. 1h = Interrupt is pending.
6	UVIFG	R	0h	Raw interrupt flag for MEMRESx underflow. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
5	DMADONE	R	0h	Raw interrupt flag for DMADONE. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
4	INIFG	R	0h	Mask INIFG in MIS_EX register. 0h = Interrupt is not pending. 1h = Interrupt is pending.
3	LOWIFG	R	0h	Raw interrupt flag for the MEMRESx result register being below than the WCLOWx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
2	HIGHIFG	R	0h	Raw interrupt flag for the MEMRESx result register being higher than the WCHIGHx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
1	TOVIFG	R	0h	Raw interrupt flag for sequence conversion trigger overflow. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.

Table 10-20. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
0	OVIFG	R	0h	Raw interrupt flag for MEMRESx overflow. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.

10.3.10 MIS (Offset = 1038h) [Reset = 00000000h]

MIS is shown in [Figure 10-14](#) and described in [Table 10-21](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 10-14. MIS

31	30	29	28	27	26	25	24
MEMRESIFG23	MEMRESIFG22	MEMRESIFG21	MEMRESIFG20	MEMRESIFG19	MEMRESIFG18	MEMRESIFG17	MEMRESIFG16
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
MEMRESIFG15	MEMRESIFG14	MEMRESIFG13	MEMRESIFG12	MEMRESIFG11	MEMRESIFG10	MEMRESIFG9	MEMRESIFG8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
MEMRESIFG7	MEMRESIFG6	MEMRESIFG5	MEMRESIFG4	MEMRESIFG3	MEMRESIFG2	MEMRESIFG1	MEMRESIFG0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
ASCDONE	UVIFG	DMADONE	INIFG	LOWIFG	HIGHIFG	TOVIFG	OVIFG
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 10-21. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31	MEMRESIFG23	R	0h	Raw interrupt status for MEMRES23. This bit is set to 1 when MEMRES23 is loaded with a new conversion result. Reading MEMRES23 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
30	MEMRESIFG22	R	0h	Raw interrupt status for MEMRES22. This bit is set to 1 when MEMRES22 is loaded with a new conversion result. Reading MEMRES22 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
29	MEMRESIFG21	R	0h	Raw interrupt status for MEMRES21. This bit is set to 1 when MEMRES21 is loaded with a new conversion result. Reading MEMRES21 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
28	MEMRESIFG20	R	0h	Raw interrupt status for MEMRES20. This bit is set to 1 when MEMRES20 is loaded with a new conversion result. Reading MEMRES20 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-21. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
27	MEMRESIFG19	R	0h	Raw interrupt status for MEMRES19. This bit is set to 1 when MEMRES19 is loaded with a new conversion result. Reading MEMRES19 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
26	MEMRESIFG18	R	0h	Raw interrupt status for MEMRES18. This bit is set to 1 when MEMRES18 is loaded with a new conversion result. Reading MEMRES18 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
25	MEMRESIFG17	R	0h	Raw interrupt status for MEMRES17. This bit is set to 1 when MEMRES17 is loaded with a new conversion result. Reading MEMRES17 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
24	MEMRESIFG16	R	0h	Raw interrupt status for MEMRES16. This bit is set to 1 when MEMRES16 is loaded with a new conversion result. Reading MEMRES16 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
23	MEMRESIFG15	R	0h	Raw interrupt status for MEMRES15. This bit is set to 1 when MEMRES15 is loaded with a new conversion result. Reading MEMRES15 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
22	MEMRESIFG14	R	0h	Raw interrupt status for MEMRES14. This bit is set to 1 when MEMRES14 is loaded with a new conversion result. Reading MEMRES14 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
21	MEMRESIFG13	R	0h	Raw interrupt status for MEMRES13. This bit is set to 1 when MEMRES13 is loaded with a new conversion result. Reading MEMRES13 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
20	MEMRESIFG12	R	0h	Raw interrupt status for MEMRES12. This bit is set to 1 when MEMRES12 is loaded with a new conversion result. Reading MEMRES12 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-21. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	MEMRESIFG11	R	0h	Raw interrupt status for MEMRES11. This bit is set to 1 when MEMRES11 is loaded with a new conversion result. Reading MEMRES11 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
18	MEMRESIFG10	R	0h	Raw interrupt status for MEMRES10. This bit is set to 1 when MEMRES10 is loaded with a new conversion result. Reading MEMRES10 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
17	MEMRESIFG9	R	0h	Raw interrupt status for MEMRES9. This bit is set to 1 when MEMRES9 is loaded with a new conversion result. Reading MEMRES9 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
16	MEMRESIFG8	R	0h	Raw interrupt status for MEMRES8. This bit is set to 1 when MEMRES8 is loaded with a new conversion result. Reading MEMRES8 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
15	MEMRESIFG7	R	0h	Raw interrupt status for MEMRES7. This bit is set to 1 when MEMRES7 is loaded with a new conversion result. Reading MEMRES7 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
14	MEMRESIFG6	R	0h	Raw interrupt status for MEMRES6. This bit is set to 1 when MEMRES6 is loaded with a new conversion result. Reading MEMRES6 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
13	MEMRESIFG5	R	0h	Raw interrupt status for MEMRES5. This bit is set to 1 when MEMRES5 is loaded with a new conversion result. Reading MEMRES5 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
12	MEMRESIFG4	R	0h	Raw interrupt status for MEMRES4. This bit is set to 1 when MEMRES4 is loaded with a new conversion result. Reading MEMRES4 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-21. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	MEMRESIFG3	R	0h	Raw interrupt status for MEMRES3. This bit is set to 1 when MEMRES3 is loaded with a new conversion result. Reading MEMRES3 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
10	MEMRESIFG2	R	0h	Raw interrupt status for MEMRES2. This bit is set to 1 when MEMRES2 is loaded with a new conversion result. Reading MEMRES2 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
9	MEMRESIFG1	R	0h	Raw interrupt status for MEMRES1. This bit is set to 1 when MEMRES1 is loaded with a new conversion result. Reading MEMRES1 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
8	MEMRESIFG0	R	0h	Raw interrupt status for MEMRES0. This bit is set to 1 when MEMRES0 is loaded with a new conversion result. Reading MEMRES0 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
7	ASCDONE	R	0h	Masked interrupt status for ASC done 0h = Interrupt is not pending. 1h = Interrupt is pending.
6	UVIFG	R	0h	Raw interrupt flag for MEMRESx underflow. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
5	DMADONE	R	0h	Raw interrupt flag for DMADONE. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
4	INIFG	R	0h	Mask INIFG in MIS_EX register. 0h = Interrupt is not pending. 1h = Interrupt is pending.
3	LOWIFG	R	0h	Raw interrupt flag for the MEMRESx result register being below than the WCLOWx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
2	HIGHIFG	R	0h	Raw interrupt flag for the MEMRESx result register being higher than the WCHIGHx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
1	TOVIFG	R	0h	Raw interrupt flag for sequence conversion timeout overflow. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.

Table 10-21. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
0	OVIFG	R	0h	Raw interrupt flag for MEMRESx overflow. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.

10.3.11 ISET (Offset = 1040h) [Reset = 00000000h]

ISET is shown in [Figure 10-15](#) and described in [Table 10-22](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 10-15. ISET

31	30	29	28	27	26	25	24
MEMRESIFG23	MEMRESIFG22	MEMRESIFG21	MEMRESIFG20	MEMRESIFG19	MEMRESIFG18	MEMRESIFG17	MEMRESIFG16
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
MEMRESIFG15	MEMRESIFG14	MEMRESIFG13	MEMRESIFG12	MEMRESIFG11	MEMRESIFG10	MEMRESIFG9	MEMRESIFG8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
MEMRESIFG7	MEMRESIFG6	MEMRESIFG5	MEMRESIFG4	MEMRESIFG3	MEMRESIFG2	MEMRESIFG1	MEMRESIFG0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
ASCDONE	UVIFG	DMADONE	INIFG	LOWIFG	HIGHIFG	TOVIFG	OVIFG
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 10-22. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31	MEMRESIFG23	W	0h	Raw interrupt status for MEMRES23. This bit is set to 1 when MEMRES23 is loaded with a new conversion result. Reading MEMRES23 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
30	MEMRESIFG22	W	0h	Raw interrupt status for MEMRES22. This bit is set to 1 when MEMRES22 is loaded with a new conversion result. Reading MEMRES22 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
29	MEMRESIFG21	W	0h	Raw interrupt status for MEMRES21. This bit is set to 1 when MEMRES21 is loaded with a new conversion result. Reading MEMRES21 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
28	MEMRESIFG20	W	0h	Raw interrupt status for MEMRES20. This bit is set to 1 when MEMRES20 is loaded with a new conversion result. Reading MEMRES20 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-22. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
27	MEMRESIFG19	W	0h	Raw interrupt status for MEMRES19. This bit is set to 1 when MEMRES19 is loaded with a new conversion result. Reading MEMRES19 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
26	MEMRESIFG18	W	0h	Raw interrupt status for MEMRES18. This bit is set to 1 when MEMRES18 is loaded with a new conversion result. Reading MEMRES18 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
25	MEMRESIFG17	W	0h	Raw interrupt status for MEMRES17. This bit is set to 1 when MEMRES17 is loaded with a new conversion result. Reading MEMRES17 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
24	MEMRESIFG16	W	0h	Raw interrupt status for MEMRES16. This bit is set to 1 when MEMRES16 is loaded with a new conversion result. Reading MEMRES16 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
23	MEMRESIFG15	W	0h	Raw interrupt status for MEMRES15. This bit is set to 1 when MEMRES15 is loaded with a new conversion result. Reading MEMRES15 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
22	MEMRESIFG14	W	0h	Raw interrupt status for MEMRES14. This bit is set to 1 when MEMRES14 is loaded with a new conversion result. Reading MEMRES14 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
21	MEMRESIFG13	W	0h	Raw interrupt status for MEMRES13. This bit is set to 1 when MEMRES13 is loaded with a new conversion result. Reading MEMRES13 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
20	MEMRESIFG12	W	0h	Raw interrupt status for MEMRES12. This bit is set to 1 when MEMRES12 is loaded with a new conversion result. Reading MEMRES12 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-22. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	MEMRESIFG11	W	0h	Raw interrupt status for MEMRES11. This bit is set to 1 when MEMRES11 is loaded with a new conversion result. Reading MEMRES11 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
18	MEMRESIFG10	W	0h	Raw interrupt status for MEMRES10. This bit is set to 1 when MEMRES10 is loaded with a new conversion result. Reading MEMRES10 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
17	MEMRESIFG9	W	0h	Raw interrupt status for MEMRES9. This bit is set to 1 when MEMRES9 is loaded with a new conversion result. Reading MEMRES9 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
16	MEMRESIFG8	W	0h	Raw interrupt status for MEMRES8. This bit is set to 1 when MEMRES8 is loaded with a new conversion result. Reading MEMRES8 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
15	MEMRESIFG7	W	0h	Raw interrupt status for MEMRES7. This bit is set to 1 when MEMRES7 is loaded with a new conversion result. Reading MEMRES7 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
14	MEMRESIFG6	W	0h	Raw interrupt status for MEMRES6. This bit is set to 1 when MEMRES6 is loaded with a new conversion result. Reading MEMRES6 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
13	MEMRESIFG5	W	0h	Raw interrupt status for MEMRES5. This bit is set to 1 when MEMRES5 is loaded with a new conversion result. Reading MEMRES5 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
12	MEMRESIFG4	W	0h	Raw interrupt status for MEMRES4. This bit is set to 1 when MEMRES4 is loaded with a new conversion result. Reading MEMRES4 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-22. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	MEMRESIFG3	W	0h	Raw interrupt status for MEMRES3. This bit is set to 1 when MEMRES3 is loaded with a new conversion result. Reading MEMRES3 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
10	MEMRESIFG2	W	0h	Raw interrupt status for MEMRES2. This bit is set to 1 when MEMRES2 is loaded with a new conversion result. Reading MEMRES2 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
9	MEMRESIFG1	W	0h	Raw interrupt status for MEMRES1. This bit is set to 1 when MEMRES1 is loaded with a new conversion result. Reading MEMRES1 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
8	MEMRESIFG0	W	0h	Raw interrupt status for MEMRES0. This bit is set to 1 when MEMRES0 is loaded with a new conversion result. Reading MEMRES0 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
7	ASCDONE	W	0h	Set ASC done flag in RIS 0h = Interrupt is not pending. 1h = Interrupt is pending.
6	UVIFG	W	0h	Raw interrupt flag for MEMRESx underflow. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
5	DMADONE	W	0h	Raw interrupt flag for DMADONE. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
4	INIFG	W	0h	Mask INIFG in MIS_EX register. 0h = Interrupt is not pending. 1h = Interrupt is pending.
3	LOWIFG	W	0h	Raw interrupt flag for the MEMRESx result register being below than the WCLOWx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
2	HIGHIFG	W	0h	Raw interrupt flag for the MEMRESx result register being higher than the WCHIGHx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
1	TOVIFG	W	0h	Raw interrupt flag for sequence conversion timeout overflow. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.

Table 10-22. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
0	OVIFG	W	0h	Raw interrupt flag for MEMRESx overflow. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.

10.3.12 ICLR (Offset = 1048h) [Reset = 00000000h]

ICLR is shown in [Figure 10-16](#) and described in [Table 10-23](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 10-16. ICLR

31	30	29	28	27	26	25	24
MEMRESIFG23	MEMRESIFG22	MEMRESIFG21	MEMRESIFG20	MEMRESIFG19	MEMRESIFG18	MEMRESIFG17	MEMRESIFG16
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
MEMRESIFG15	MEMRESIFG14	MEMRESIFG13	MEMRESIFG12	MEMRESIFG11	MEMRESIFG10	MEMRESIFG9	MEMRESIFG8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
MEMRESIFG7	MEMRESIFG6	MEMRESIFG5	MEMRESIFG4	MEMRESIFG3	MEMRESIFG2	MEMRESIFG1	MEMRESIFG0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
ASCDONE	UVIFG	DMADONE	INIFG	LOWIFG	HIGHIFG	TOVIFG	OVIFG
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 10-23. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31	MEMRESIFG23	W	0h	Raw interrupt status for MEMRES23. This bit is set to 1 when MEMRES23 is loaded with a new conversion result. Reading MEMRES23 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
30	MEMRESIFG22	W	0h	Raw interrupt status for MEMRES22. This bit is set to 1 when MEMRES22 is loaded with a new conversion result. Reading MEMRES22 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
29	MEMRESIFG21	W	0h	Raw interrupt status for MEMRES21. This bit is set to 1 when MEMRES21 is loaded with a new conversion result. Reading MEMRES21 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
28	MEMRESIFG20	W	0h	Raw interrupt status for MEMRES20. This bit is set to 1 when MEMRES20 is loaded with a new conversion result. Reading MEMRES20 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-23. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
27	MEMRESIFG19	W	0h	Raw interrupt status for MEMRES19. This bit is set to 1 when MEMRES19 is loaded with a new conversion result. Reading MEMRES19 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
26	MEMRESIFG18	W	0h	Raw interrupt status for MEMRES18. This bit is set to 1 when MEMRES18 is loaded with a new conversion result. Reading MEMRES18 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
25	MEMRESIFG17	W	0h	Raw interrupt status for MEMRES17. This bit is set to 1 when MEMRES17 is loaded with a new conversion result. Reading MEMRES17 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
24	MEMRESIFG16	W	0h	Raw interrupt status for MEMRES16. This bit is set to 1 when MEMRES16 is loaded with a new conversion result. Reading MEMRES16 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
23	MEMRESIFG15	W	0h	Raw interrupt status for MEMRES15. This bit is set to 1 when MEMRES15 is loaded with a new conversion result. Reading MEMRES15 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
22	MEMRESIFG14	W	0h	Raw interrupt status for MEMRES14. This bit is set to 1 when MEMRES14 is loaded with a new conversion result. Reading MEMRES14 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
21	MEMRESIFG13	W	0h	Raw interrupt status for MEMRES13. This bit is set to 1 when MEMRES13 is loaded with a new conversion result. Reading MEMRES13 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
20	MEMRESIFG12	W	0h	Raw interrupt status for MEMRES12. This bit is set to 1 when MEMRES12 is loaded with a new conversion result. Reading MEMRES12 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-23. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	MEMRESIFG11	W	0h	Raw interrupt status for MEMRES11. This bit is set to 1 when MEMRES11 is loaded with a new conversion result. Reading MEMRES11 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
18	MEMRESIFG10	W	0h	Raw interrupt status for MEMRES10. This bit is set to 1 when MEMRES10 is loaded with a new conversion result. Reading MEMRES10 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
17	MEMRESIFG9	W	0h	Raw interrupt status for MEMRES9. This bit is set to 1 when MEMRES9 is loaded with a new conversion result. Reading MEMRES9 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
16	MEMRESIFG8	W	0h	Raw interrupt status for MEMRES8. This bit is set to 1 when MEMRES8 is loaded with a new conversion result. Reading MEMRES8 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
15	MEMRESIFG7	W	0h	Raw interrupt status for MEMRES7. This bit is set to 1 when MEMRES7 is loaded with a new conversion result. Reading MEMRES7 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
14	MEMRESIFG6	W	0h	Raw interrupt status for MEMRES6. This bit is set to 1 when MEMRES6 is loaded with a new conversion result. Reading MEMRES6 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
13	MEMRESIFG5	W	0h	Raw interrupt status for MEMRES5. This bit is set to 1 when MEMRES5 is loaded with a new conversion result. Reading MEMRES5 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
12	MEMRESIFG4	W	0h	Raw interrupt status for MEMRES4. This bit is set to 1 when MEMRES4 is loaded with a new conversion result. Reading MEMRES4 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-23. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	MEMRESIFG3	W	0h	Raw interrupt status for MEMRES3. This bit is set to 1 when MEMRES3 is loaded with a new conversion result. Reading MEMRES3 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
10	MEMRESIFG2	W	0h	Raw interrupt status for MEMRES2. This bit is set to 1 when MEMRES2 is loaded with a new conversion result. Reading MEMRES2 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
9	MEMRESIFG1	W	0h	Raw interrupt status for MEMRES1. This bit is set to 1 when MEMRES1 is loaded with a new conversion result. Reading MEMRES1 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
8	MEMRESIFG0	W	0h	Raw interrupt status for MEMRES0. This bit is set to 1 when MEMRES0 is loaded with a new conversion result. Reading MEMRES0 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
7	ASCDONE	W	0h	Clear ASC done flag in RIS 0h = Interrupt is not pending. 1h = Interrupt is pending.
6	UVIFG	W	0h	Raw interrupt flag for MEMRESx underflow. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
5	DMADONE	W	0h	Raw interrupt flag for DMADONE. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
4	INIFG	W	0h	Mask INIFG in MIS_EX register. 0h = Interrupt is not pending. 1h = Interrupt is pending.
3	LOWIFG	W	0h	Raw interrupt flag for the MEMRESx result register being below than the WCLOWx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
2	HIGHIFG	W	0h	Raw interrupt flag for the MEMRESx result register being higher than the WCHIGHx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
1	TOVIFG	W	0h	Raw interrupt flag for sequence conversion timeout overflow. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.

Table 10-23. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
0	OVIFG	W	0h	Raw interrupt flag for MEMRESx overflow. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.

10.3.13 IIDX (Offset = 1050h) [Reset = 00000000h]

IIDX is shown in [Figure 10-17](#) and described in [Table 10-24](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. 0x0 means no event pending. Interrupt 1 is the highest priority, 2 next highest, 4, 8, ... 2^{31} is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in the RIS and MIS are cleared as well. After a read from the CPU (not from the debug interface), the register must be updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 10-17. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																						STAT									
R-0h																						R-0h									

Table 10-24. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-10	RESERVED	R	0h	
9-0	STAT	R	0h	Interrupt index status 00h = No bit is set means there is no pending interrupt request 03h = High threshold compare interrupt 04h = Low threshold compare interrupt 05h = Primary Sequence In range comparator interrupt 9h = MEMRES0 data loaded interrupt

10.3.14 IMASK (Offset = 1058h) [Reset = 00000000h]

IMASK is shown in [Figure 10-18](#) and described in [Table 10-25](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 10-18. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							MEMRESIFG0
R/W-0h							R/W-0h
7	6	5	4	3	2	1	0
RESERVED			INIFG	LOWIFG	HIGHIFG	RESERVED	
R/W-0h			R/W-0h	R/W-0h	R/W-0h	R/W-0h	

Table 10-25. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-9	RESERVED	R/W	0h	
8	MEMRESIFG0	R/W	0h	Raw interrupt status for MEMRES0. This bit is set to 1 when MEMRES0 is loaded with a new conversion result. Reading MEMRES0 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
7-5	RESERVED	R/W	0h	
4	INIFG	R/W	0h	Mask INIFG in MIS_EX register. 0h = Interrupt is not pending. 1h = Interrupt is pending.
3	LOWIFG	R/W	0h	Raw interrupt flag for the MEMRESx result register being below than the WCLOWx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
2	HIGHIFG	R/W	0h	Raw interrupt flag for the MEMRESx result register being higher than the WCHIGHx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
1-0	RESERVED	R/W	0h	

10.3.15 RIS (Offset = 1060h) [Reset = 00000000h]

RIS is shown in [Figure 10-19](#) and described in [Table 10-26](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 10-19. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							MEMRESIFG0
R-0h							R-0h
7	6	5	4	3	2	1	0
RESERVED			INIFG	LOWIFG	HIGHIFG	RESERVED	
R-0h			R-0h	R-0h	R-0h	R-0h	

Table 10-26. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-9	RESERVED	R	0h	
8	MEMRESIFG0	R	0h	Raw interrupt status for MEMRES0. This bit is set to 1 when MEMRES0 is loaded with a new conversion result. Reading MEMRES0 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
7-5	RESERVED	R	0h	
4	INIFG	R	0h	Mask INIFG in MIS_EX register. 0h = Interrupt is not pending. 1h = Interrupt is pending.
3	LOWIFG	R	0h	Raw interrupt flag for the MEMRESx result register being below than the WCLOWx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
2	HIGHIFG	R	0h	Raw interrupt flag for the MEMRESx result register being higher than the WCHIGHx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
1-0	RESERVED	R	0h	

10.3.16 MIS (Offset = 1068h) [Reset = 00000000h]

MIS is shown in [Figure 10-20](#) and described in [Table 10-27](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 10-20. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							MEMRESIFG0
R-0h							R-0h
7	6	5	4	3	2	1	0
RESERVED			INIFG	LOWIFG	HIGHIFG	RESERVED	
R-0h			R-0h	R-0h	R-0h	R-0h	

Table 10-27. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-9	RESERVED	R	0h	
8	MEMRESIFG0	R	0h	Raw interrupt status for MEMRES0. This bit is set to 1 when MEMRES0 is loaded with a new conversion result. Reading MEMRES0 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
7-5	RESERVED	R	0h	
4	INIFG	R	0h	Mask INIFG in MIS_EX register. 0h = Interrupt is not pending. 1h = Interrupt is pending.
3	LOWIFG	R	0h	Raw interrupt flag for the MEMRESx result register being below than the WCLOWx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
2	HIGHIFG	R	0h	Raw interrupt flag for the MEMRESx result register being higher than the WCHIGHx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
1-0	RESERVED	R	0h	

10.3.17 ISET (Offset = 1070h) [Reset = 00000000h]

ISET is shown in [Figure 10-21](#) and described in [Table 10-28](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 10-21. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							MEMRESIFG0
W-0h							W-0h
7	6	5	4	3	2	1	0
RESERVED			INIFG	LOWIFG	HIGHIFG	RESERVED	
W-0h			W-0h	W-0h	W-0h	W-0h	

Table 10-28. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-9	RESERVED	W	0h	
8	MEMRESIFG0	W	0h	Raw interrupt status for MEMRES0. This bit is set to 1 when MEMRES0 is loaded with a new conversion result. Reading MEMRES0 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
7-5	RESERVED	W	0h	
4	INIFG	W	0h	Mask INIFG in MIS_EX register. 0h = Interrupt is not pending. 1h = Interrupt is pending.
3	LOWIFG	W	0h	Raw interrupt flag for the MEMRESx result register being below than the WCLOWx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
2	HIGHIFG	W	0h	Raw interrupt flag for the MEMRESx result register being higher than the WCHIGHx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
1-0	RESERVED	W	0h	

10.3.18 ICLR (Offset = 1078h) [Reset = 00000000h]

ICLR is shown in [Figure 10-22](#) and described in [Table 10-29](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 10-22. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							MEMRESIFG0
W-0h							W-0h
7	6	5	4	3	2	1	0
RESERVED			INIFG	LOWIFG	HIGHIFG	RESERVED	
W-0h			W-0h	W-0h	W-0h	W-0h	

Table 10-29. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-9	RESERVED	W	0h	
8	MEMRESIFG0	W	0h	Raw interrupt status for MEMRES0. This bit is set to 1 when MEMRES0 is loaded with a new conversion result. Reading MEMRES0 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
7-5	RESERVED	W	0h	
4	INIFG	W	0h	Mask INIFG in MIS_EX register. 0h = Interrupt is not pending. 1h = Interrupt is pending.
3	LOWIFG	W	0h	Raw interrupt flag for the MEMRESx result register being below than the WCLOWx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
2	HIGHIFG	W	0h	Raw interrupt flag for the MEMRESx result register being higher than the WCHIGHx threshold of the window comparator. This bit is reset to 0 by IIDX read or when corresponding bit in ICLR_EX is set to 1. 0h = Interrupt is not pending. 1h = Interrupt is pending.
1-0	RESERVED	W	0h	

10.3.19 IIDX (Offset = 1080h) [Reset = 00000000h]

IIDX is shown in [Figure 10-23](#) and described in [Table 10-30](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. 0x0 means no event pending. Interrupt 1 is the highest priority, 2 next highest, 4, 8, ... 2^{31} is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in the RIS and MIS are cleared as well. After a read from the CPU (not from the debug interface), the register must be updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 10-23. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																						STAT									
R-0h																						R-0h									

Table 10-30. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-10	RESERVED	R	0h	
9-0	STAT	R	0h	Interrupt index status 00h = No bit is set means there is no pending interrupt request 9h = MEMRES0 data loaded interrupt Ah = MEMRES1 data loaded interrupt Bh = MEMRES2 data loaded interrupt Ch = MEMRES3 data loaded interrupt Dh = MEMRES4 data loaded interrupt Eh = MEMRES5 data loaded interrupt Fh = MEMRES6 data loaded interrupt 10h = MEMRES7 data loaded interrupt 11h = MEMRES8 data loaded interrupt 12h = MEMRES9 data loaded interrupt 13h = MEMRES10 data loaded interrupt 14h = MEMRES11 data loaded interrupt 15h = MEMRES12 data loaded interrupt 16h = MEMRES13 data loaded interrupt 17h = MEMRES14 data loaded interrupt 18h = MEMRES15 data loaded interrupt 19h = MEMRES16 data loaded interrupt 1Ah = MEMRES17 data loaded interrupt 1Bh = MEMRES18 data loaded interrupt 1Ch = MEMRES19 data loaded interrupt 1Dh = MEMRES20 data loaded interrupt 1Eh = MEMRES21 data loaded interrupt 1Fh = MEMRES22 data loaded interrupt 20h = MEMRES23 data loaded interrupt

10.3.20 IMASK (Offset = 1088h) [Reset = 00000000h]

IMASK is shown in [Figure 10-24](#) and described in [Table 10-31](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 10-24. IMASK

31	30	29	28	27	26	25	24
MEMRESIFG23	MEMRESIFG22	MEMRESIFG21	MEMRESIFG20	MEMRESIFG19	MEMRESIFG18	MEMRESIFG17	MEMRESIFG16
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
MEMRESIFG15	MEMRESIFG14	MEMRESIFG13	MEMRESIFG12	MEMRESIFG11	MEMRESIFG10	MEMRESIFG9	MEMRESIFG8
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
MEMRESIFG7	MEMRESIFG6	MEMRESIFG5	MEMRESIFG4	MEMRESIFG3	MEMRESIFG2	MEMRESIFG1	MEMRESIFG0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
RESERVED							
R/W-0h							

Table 10-31. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31	MEMRESIFG23	R/W	0h	Raw interrupt status for MEMRES23. This bit is set to 1 when MEMRES23 is loaded with a new conversion result. Reading MEMRES23 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
30	MEMRESIFG22	R/W	0h	Raw interrupt status for MEMRES22. This bit is set to 1 when MEMRES22 is loaded with a new conversion result. Reading MEMRES22 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
29	MEMRESIFG21	R/W	0h	Raw interrupt status for MEMRES21. This bit is set to 1 when MEMRES21 is loaded with a new conversion result. Reading MEMRES21 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
28	MEMRESIFG20	R/W	0h	Raw interrupt status for MEMRES20. This bit is set to 1 when MEMRES20 is loaded with a new conversion result. Reading MEMRES20 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-31. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
27	MEMRESIFG19	R/W	0h	Raw interrupt status for MEMRES19. This bit is set to 1 when MEMRES19 is loaded with a new conversion result. Reading MEMRES19 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
26	MEMRESIFG18	R/W	0h	Raw interrupt status for MEMRES18. This bit is set to 1 when MEMRES18 is loaded with a new conversion result. Reading MEMRES18 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
25	MEMRESIFG17	R/W	0h	Raw interrupt status for MEMRES17. This bit is set to 1 when MEMRES17 is loaded with a new conversion result. Reading MEMRES17 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
24	MEMRESIFG16	R/W	0h	Raw interrupt status for MEMRES16. This bit is set to 1 when MEMRES16 is loaded with a new conversion result. Reading MEMRES16 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
23	MEMRESIFG15	R/W	0h	Raw interrupt status for MEMRES15. This bit is set to 1 when MEMRES15 is loaded with a new conversion result. Reading MEMRES15 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
22	MEMRESIFG14	R/W	0h	Raw interrupt status for MEMRES14. This bit is set to 1 when MEMRES14 is loaded with a new conversion result. Reading MEMRES14 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
21	MEMRESIFG13	R/W	0h	Raw interrupt status for MEMRES13. This bit is set to 1 when MEMRES13 is loaded with a new conversion result. Reading MEMRES13 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
20	MEMRESIFG12	R/W	0h	Raw interrupt status for MEMRES12. This bit is set to 1 when MEMRES12 is loaded with a new conversion result. Reading MEMRES12 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-31. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	MEMRESIFG11	R/W	0h	Raw interrupt status for MEMRES11. This bit is set to 1 when MEMRES11 is loaded with a new conversion result. Reading MEMRES11 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
18	MEMRESIFG10	R/W	0h	Raw interrupt status for MEMRES10. This bit is set to 1 when MEMRES10 is loaded with a new conversion result. Reading MEMRES10 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
17	MEMRESIFG9	R/W	0h	Raw interrupt status for MEMRES9. This bit is set to 1 when MEMRES9 is loaded with a new conversion result. Reading MEMRES9 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
16	MEMRESIFG8	R/W	0h	Raw interrupt status for MEMRES8. This bit is set to 1 when MEMRES8 is loaded with a new conversion result. Reading MEMRES8 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
15	MEMRESIFG7	R/W	0h	Raw interrupt status for MEMRES7. This bit is set to 1 when MEMRES7 is loaded with a new conversion result. Reading MEMRES7 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
14	MEMRESIFG6	R/W	0h	Raw interrupt status for MEMRES6. This bit is set to 1 when MEMRES6 is loaded with a new conversion result. Reading MEMRES6 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
13	MEMRESIFG5	R/W	0h	Raw interrupt status for MEMRES5. This bit is set to 1 when MEMRES5 is loaded with a new conversion result. Reading MEMRES5 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
12	MEMRESIFG4	R/W	0h	Raw interrupt status for MEMRES4. This bit is set to 1 when MEMRES4 is loaded with a new conversion result. Reading MEMRES4 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-31. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	MEMRESIFG3	R/W	0h	Raw interrupt status for MEMRES3. This bit is set to 1 when MEMRES3 is loaded with a new conversion result. Reading MEMRES3 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
10	MEMRESIFG2	R/W	0h	Raw interrupt status for MEMRES2. This bit is set to 1 when MEMRES2 is loaded with a new conversion result. Reading MEMRES2 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
9	MEMRESIFG1	R/W	0h	Raw interrupt status for MEMRES1. This bit is set to 1 when MEMRES1 is loaded with a new conversion result. Reading MEMRES1 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
8	MEMRESIFG0	R/W	0h	Raw interrupt status for MEMRES0. This bit is set to 1 when MEMRES0 is loaded with a new conversion result. Reading MEMRES0 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
7-0	RESERVED	R/W	0h	

10.3.21 RIS (Offset = 1090h) [Reset = 00000000h]

RIS is shown in [Figure 10-25](#) and described in [Table 10-32](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 10-25. RIS

31	30	29	28	27	26	25	24
MEMRESIFG23	MEMRESIFG22	MEMRESIFG21	MEMRESIFG20	MEMRESIFG19	MEMRESIFG18	MEMRESIFG17	MEMRESIFG16
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
MEMRESIFG15	MEMRESIFG14	MEMRESIFG13	MEMRESIFG12	MEMRESIFG11	MEMRESIFG10	MEMRESIFG9	MEMRESIFG8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
MEMRESIFG7	MEMRESIFG6	MEMRESIFG5	MEMRESIFG4	MEMRESIFG3	MEMRESIFG2	MEMRESIFG1	MEMRESIFG0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
RESERVED							
R-0h							

Table 10-32. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31	MEMRESIFG23	R	0h	Raw interrupt status for MEMRES23. This bit is set to 1 when MEMRES23 is loaded with a new conversion result. Reading MEMRES23 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
30	MEMRESIFG22	R	0h	Raw interrupt status for MEMRES22. This bit is set to 1 when MEMRES22 is loaded with a new conversion result. Reading MEMRES22 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
29	MEMRESIFG21	R	0h	Raw interrupt status for MEMRES21. This bit is set to 1 when MEMRES21 is loaded with a new conversion result. Reading MEMRES21 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
28	MEMRESIFG20	R	0h	Raw interrupt status for MEMRES20. This bit is set to 1 when MEMRES20 is loaded with a new conversion result. Reading MEMRES20 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-32. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
27	MEMRESIFG19	R	0h	Raw interrupt status for MEMRES19. This bit is set to 1 when MEMRES19 is loaded with a new conversion result. Reading MEMRES19 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
26	MEMRESIFG18	R	0h	Raw interrupt status for MEMRES18. This bit is set to 1 when MEMRES18 is loaded with a new conversion result. Reading MEMRES18 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
25	MEMRESIFG17	R	0h	Raw interrupt status for MEMRES17. This bit is set to 1 when MEMRES17 is loaded with a new conversion result. Reading MEMRES17 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
24	MEMRESIFG16	R	0h	Raw interrupt status for MEMRES16. This bit is set to 1 when MEMRES16 is loaded with a new conversion result. Reading MEMRES16 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
23	MEMRESIFG15	R	0h	Raw interrupt status for MEMRES15. This bit is set to 1 when MEMRES15 is loaded with a new conversion result. Reading MEMRES15 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
22	MEMRESIFG14	R	0h	Raw interrupt status for MEMRES14. This bit is set to 1 when MEMRES14 is loaded with a new conversion result. Reading MEMRES14 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
21	MEMRESIFG13	R	0h	Raw interrupt status for MEMRES13. This bit is set to 1 when MEMRES13 is loaded with a new conversion result. Reading MEMRES13 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
20	MEMRESIFG12	R	0h	Raw interrupt status for MEMRES12. This bit is set to 1 when MEMRES12 is loaded with a new conversion result. Reading MEMRES12 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-32. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	MEMRESIFG11	R	0h	Raw interrupt status for MEMRES11. This bit is set to 1 when MEMRES11 is loaded with a new conversion result. Reading MEMRES11 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
18	MEMRESIFG10	R	0h	Raw interrupt status for MEMRES10. This bit is set to 1 when MEMRES10 is loaded with a new conversion result. Reading MEMRES10 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
17	MEMRESIFG9	R	0h	Raw interrupt status for MEMRES9. This bit is set to 1 when MEMRES9 is loaded with a new conversion result. Reading MEMRES9 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
16	MEMRESIFG8	R	0h	Raw interrupt status for MEMRES8. This bit is set to 1 when MEMRES8 is loaded with a new conversion result. Reading MEMRES8 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
15	MEMRESIFG7	R	0h	Raw interrupt status for MEMRES7. This bit is set to 1 when MEMRES7 is loaded with a new conversion result. Reading MEMRES7 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
14	MEMRESIFG6	R	0h	Raw interrupt status for MEMRES6. This bit is set to 1 when MEMRES6 is loaded with a new conversion result. Reading MEMRES6 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
13	MEMRESIFG5	R	0h	Raw interrupt status for MEMRES5. This bit is set to 1 when MEMRES5 is loaded with a new conversion result. Reading MEMRES5 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
12	MEMRESIFG4	R	0h	Raw interrupt status for MEMRES4. This bit is set to 1 when MEMRES4 is loaded with a new conversion result. Reading MEMRES4 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-32. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	MEMRESIFG3	R	0h	Raw interrupt status for MEMRES3. This bit is set to 1 when MEMRES3 is loaded with a new conversion result. Reading MEMRES3 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
10	MEMRESIFG2	R	0h	Raw interrupt status for MEMRES2. This bit is set to 1 when MEMRES2 is loaded with a new conversion result. Reading MEMRES2 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
9	MEMRESIFG1	R	0h	Raw interrupt status for MEMRES1. This bit is set to 1 when MEMRES1 is loaded with a new conversion result. Reading MEMRES1 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
8	MEMRESIFG0	R	0h	Raw interrupt status for MEMRES0. This bit is set to 1 when MEMRES0 is loaded with a new conversion result. Reading MEMRES0 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
7-0	RESERVED	R	0h	

10.3.22 MIS (Offset = 1098h) [Reset = 00000000h]

MIS is shown in [Figure 10-26](#) and described in [Table 10-33](#).

Return to the [Summary Table](#).

Extension of Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 10-26. MIS

31	30	29	28	27	26	25	24
MEMRESIFG23	MEMRESIFG22	MEMRESIFG21	MEMRESIFG20	MEMRESIFG19	MEMRESIFG18	MEMRESIFG17	MEMRESIFG16
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
MEMRESIFG15	MEMRESIFG14	MEMRESIFG13	MEMRESIFG12	MEMRESIFG11	MEMRESIFG10	MEMRESIFG9	MEMRESIFG8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
MEMRESIFG7	MEMRESIFG6	MEMRESIFG5	MEMRESIFG4	MEMRESIFG3	MEMRESIFG2	MEMRESIFG1	MEMRESIFG0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
RESERVED							
R-0h							

Table 10-33. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31	MEMRESIFG23	R	0h	Raw interrupt status for MEMRES23. This bit is set to 1 when MEMRES23 is loaded with a new conversion result. Reading MEMRES23 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
30	MEMRESIFG22	R	0h	Raw interrupt status for MEMRES22. This bit is set to 1 when MEMRES22 is loaded with a new conversion result. Reading MEMRES22 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
29	MEMRESIFG21	R	0h	Raw interrupt status for MEMRES21. This bit is set to 1 when MEMRES21 is loaded with a new conversion result. Reading MEMRES21 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
28	MEMRESIFG20	R	0h	Raw interrupt status for MEMRES20. This bit is set to 1 when MEMRES20 is loaded with a new conversion result. Reading MEMRES20 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-33. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
27	MEMRESIFG19	R	0h	Raw interrupt status for MEMRES19. This bit is set to 1 when MEMRES19 is loaded with a new conversion result. Reading MEMRES19 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
26	MEMRESIFG18	R	0h	Raw interrupt status for MEMRES18. This bit is set to 1 when MEMRES18 is loaded with a new conversion result. Reading MEMRES18 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
25	MEMRESIFG17	R	0h	Raw interrupt status for MEMRES17. This bit is set to 1 when MEMRES17 is loaded with a new conversion result. Reading MEMRES17 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
24	MEMRESIFG16	R	0h	Raw interrupt status for MEMRES16. This bit is set to 1 when MEMRES16 is loaded with a new conversion result. Reading MEMRES16 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
23	MEMRESIFG15	R	0h	Raw interrupt status for MEMRES15. This bit is set to 1 when MEMRES15 is loaded with a new conversion result. Reading MEMRES15 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
22	MEMRESIFG14	R	0h	Raw interrupt status for MEMRES14. This bit is set to 1 when MEMRES14 is loaded with a new conversion result. Reading MEMRES14 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
21	MEMRESIFG13	R	0h	Raw interrupt status for MEMRES13. This bit is set to 1 when MEMRES13 is loaded with a new conversion result. Reading MEMRES13 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
20	MEMRESIFG12	R	0h	Raw interrupt status for MEMRES12. This bit is set to 1 when MEMRES12 is loaded with a new conversion result. Reading MEMRES12 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-33. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	MEMRESIFG11	R	0h	Raw interrupt status for MEMRES11. This bit is set to 1 when MEMRES11 is loaded with a new conversion result. Reading MEMRES11 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
18	MEMRESIFG10	R	0h	Raw interrupt status for MEMRES10. This bit is set to 1 when MEMRES10 is loaded with a new conversion result. Reading MEMRES10 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
17	MEMRESIFG9	R	0h	Raw interrupt status for MEMRES9. This bit is set to 1 when MEMRES9 is loaded with a new conversion result. Reading MEMRES9 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
16	MEMRESIFG8	R	0h	Raw interrupt status for MEMRES8. This bit is set to 1 when MEMRES8 is loaded with a new conversion result. Reading MEMRES8 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
15	MEMRESIFG7	R	0h	Raw interrupt status for MEMRES7. This bit is set to 1 when MEMRES7 is loaded with a new conversion result. Reading MEMRES7 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
14	MEMRESIFG6	R	0h	Raw interrupt status for MEMRES6. This bit is set to 1 when MEMRES6 is loaded with a new conversion result. Reading MEMRES6 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
13	MEMRESIFG5	R	0h	Raw interrupt status for MEMRES5. This bit is set to 1 when MEMRES5 is loaded with a new conversion result. Reading MEMRES5 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
12	MEMRESIFG4	R	0h	Raw interrupt status for MEMRES4. This bit is set to 1 when MEMRES4 is loaded with a new conversion result. Reading MEMRES4 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-33. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	MEMRESIFG3	R	0h	Raw interrupt status for MEMRES3. This bit is set to 1 when MEMRES3 is loaded with a new conversion result. Reading MEMRES3 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
10	MEMRESIFG2	R	0h	Raw interrupt status for MEMRES2. This bit is set to 1 when MEMRES2 is loaded with a new conversion result. Reading MEMRES2 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
9	MEMRESIFG1	R	0h	Raw interrupt status for MEMRES1. This bit is set to 1 when MEMRES1 is loaded with a new conversion result. Reading MEMRES1 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
8	MEMRESIFG0	R	0h	Raw interrupt status for MEMRES0. This bit is set to 1 when MEMRES0 is loaded with a new conversion result. Reading MEMRES0 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
7-0	RESERVED	R	0h	

10.3.23 ISET (Offset = 10A0h) [Reset = 00000000h]

ISET is shown in [Figure 10-27](#) and described in [Table 10-34](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 10-27. ISET

31	30	29	28	27	26	25	24
MEMRESIFG23	MEMRESIFG22	MEMRESIFG21	MEMRESIFG20	MEMRESIFG19	MEMRESIFG18	MEMRESIFG17	MEMRESIFG16
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
MEMRESIFG15	MEMRESIFG14	MEMRESIFG13	MEMRESIFG12	MEMRESIFG11	MEMRESIFG10	MEMRESIFG9	MEMRESIFG8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
MEMRESIFG7	MEMRESIFG6	MEMRESIFG5	MEMRESIFG4	MEMRESIFG3	MEMRESIFG2	MEMRESIFG1	MEMRESIFG0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
RESERVED							
W-0h							

Table 10-34. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31	MEMRESIFG23	W	0h	Raw interrupt status for MEMRES23. This bit is set to 1 when MEMRES23 is loaded with a new conversion result. Reading MEMRES23 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
30	MEMRESIFG22	W	0h	Raw interrupt status for MEMRES22. This bit is set to 1 when MEMRES22 is loaded with a new conversion result. Reading MEMRES22 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
29	MEMRESIFG21	W	0h	Raw interrupt status for MEMRES21. This bit is set to 1 when MEMRES21 is loaded with a new conversion result. Reading MEMRES21 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
28	MEMRESIFG20	W	0h	Raw interrupt status for MEMRES20. This bit is set to 1 when MEMRES20 is loaded with a new conversion result. Reading MEMRES20 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-34. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
27	MEMRESIFG19	W	0h	Raw interrupt status for MEMRES19. This bit is set to 1 when MEMRES19 is loaded with a new conversion result. Reading MEMRES19 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
26	MEMRESIFG18	W	0h	Raw interrupt status for MEMRES18. This bit is set to 1 when MEMRES18 is loaded with a new conversion result. Reading MEMRES18 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
25	MEMRESIFG17	W	0h	Raw interrupt status for MEMRES17. This bit is set to 1 when MEMRES17 is loaded with a new conversion result. Reading MEMRES17 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
24	MEMRESIFG16	W	0h	Raw interrupt status for MEMRES16. This bit is set to 1 when MEMRES16 is loaded with a new conversion result. Reading MEMRES16 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
23	MEMRESIFG15	W	0h	Raw interrupt status for MEMRES15. This bit is set to 1 when MEMRES15 is loaded with a new conversion result. Reading MEMRES15 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
22	MEMRESIFG14	W	0h	Raw interrupt status for MEMRES14. This bit is set to 1 when MEMRES14 is loaded with a new conversion result. Reading MEMRES14 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
21	MEMRESIFG13	W	0h	Raw interrupt status for MEMRES13. This bit is set to 1 when MEMRES13 is loaded with a new conversion result. Reading MEMRES13 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
20	MEMRESIFG12	W	0h	Raw interrupt status for MEMRES12. This bit is set to 1 when MEMRES12 is loaded with a new conversion result. Reading MEMRES12 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-34. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	MEMRESIFG11	W	0h	Raw interrupt status for MEMRES11. This bit is set to 1 when MEMRES11 is loaded with a new conversion result. Reading MEMRES11 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
18	MEMRESIFG10	W	0h	Raw interrupt status for MEMRES10. This bit is set to 1 when MEMRES10 is loaded with a new conversion result. Reading MEMRES10 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
17	MEMRESIFG9	W	0h	Raw interrupt status for MEMRES9. This bit is set to 1 when MEMRES9 is loaded with a new conversion result. Reading MEMRES9 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
16	MEMRESIFG8	W	0h	Raw interrupt status for MEMRES8. This bit is set to 1 when MEMRES8 is loaded with a new conversion result. Reading MEMRES8 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
15	MEMRESIFG7	W	0h	Raw interrupt status for MEMRES7. This bit is set to 1 when MEMRES7 is loaded with a new conversion result. Reading MEMRES7 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
14	MEMRESIFG6	W	0h	Raw interrupt status for MEMRES6. This bit is set to 1 when MEMRES6 is loaded with a new conversion result. Reading MEMRES6 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
13	MEMRESIFG5	W	0h	Raw interrupt status for MEMRES5. This bit is set to 1 when MEMRES5 is loaded with a new conversion result. Reading MEMRES5 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
12	MEMRESIFG4	W	0h	Raw interrupt status for MEMRES4. This bit is set to 1 when MEMRES4 is loaded with a new conversion result. Reading MEMRES4 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-34. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	MEMRESIFG3	W	0h	Raw interrupt status for MEMRES3. This bit is set to 1 when MEMRES3 is loaded with a new conversion result. Reading MEMRES3 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
10	MEMRESIFG2	W	0h	Raw interrupt status for MEMRES2. This bit is set to 1 when MEMRES2 is loaded with a new conversion result. Reading MEMRES2 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
9	MEMRESIFG1	W	0h	Raw interrupt status for MEMRES1. This bit is set to 1 when MEMRES1 is loaded with a new conversion result. Reading MEMRES1 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
8	MEMRESIFG0	W	0h	Raw interrupt status for MEMRES0. This bit is set to 1 when MEMRES0 is loaded with a new conversion result. Reading MEMRES0 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
7-0	RESERVED	W	0h	

10.3.24 ICLR (Offset = 10A8h) [Reset = 00000000h]

ICLR is shown in [Figure 10-28](#) and described in [Table 10-35](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 10-28. ICLR

31	30	29	28	27	26	25	24
MEMRESIFG23	MEMRESIFG22	MEMRESIFG21	MEMRESIFG20	MEMRESIFG19	MEMRESIFG18	MEMRESIFG17	MEMRESIFG16
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
MEMRESIFG15	MEMRESIFG14	MEMRESIFG13	MEMRESIFG12	MEMRESIFG11	MEMRESIFG10	MEMRESIFG9	MEMRESIFG8
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
MEMRESIFG7	MEMRESIFG6	MEMRESIFG5	MEMRESIFG4	MEMRESIFG3	MEMRESIFG2	MEMRESIFG1	MEMRESIFG0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
RESERVED							
W-0h							

Table 10-35. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31	MEMRESIFG23	W	0h	Raw interrupt status for MEMRES23. This bit is set to 1 when MEMRES23 is loaded with a new conversion result. Reading MEMRES23 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
30	MEMRESIFG22	W	0h	Raw interrupt status for MEMRES22. This bit is set to 1 when MEMRES22 is loaded with a new conversion result. Reading MEMRES22 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
29	MEMRESIFG21	W	0h	Raw interrupt status for MEMRES21. This bit is set to 1 when MEMRES21 is loaded with a new conversion result. Reading MEMRES21 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
28	MEMRESIFG20	W	0h	Raw interrupt status for MEMRES20. This bit is set to 1 when MEMRES20 is loaded with a new conversion result. Reading MEMRES20 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-35. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
27	MEMRESIFG19	W	0h	Raw interrupt status for MEMRES19. This bit is set to 1 when MEMRES19 is loaded with a new conversion result. Reading MEMRES19 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
26	MEMRESIFG18	W	0h	Raw interrupt status for MEMRES18. This bit is set to 1 when MEMRES18 is loaded with a new conversion result. Reading MEMRES18 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
25	MEMRESIFG17	W	0h	Raw interrupt status for MEMRES17. This bit is set to 1 when MEMRES17 is loaded with a new conversion result. Reading MEMRES17 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
24	MEMRESIFG16	W	0h	Raw interrupt status for MEMRES16. This bit is set to 1 when MEMRES16 is loaded with a new conversion result. Reading MEMRES16 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
23	MEMRESIFG15	W	0h	Raw interrupt status for MEMRES15. This bit is set to 1 when MEMRES15 is loaded with a new conversion result. Reading MEMRES15 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
22	MEMRESIFG14	W	0h	Raw interrupt status for MEMRES14. This bit is set to 1 when MEMRES14 is loaded with a new conversion result. Reading MEMRES14 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
21	MEMRESIFG13	W	0h	Raw interrupt status for MEMRES13. This bit is set to 1 when MEMRES13 is loaded with a new conversion result. Reading MEMRES13 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
20	MEMRESIFG12	W	0h	Raw interrupt status for MEMRES12. This bit is set to 1 when MEMRES12 is loaded with a new conversion result. Reading MEMRES12 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-35. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	MEMRESIFG11	W	0h	Raw interrupt status for MEMRES11. This bit is set to 1 when MEMRES11 is loaded with a new conversion result. Reading MEMRES11 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
18	MEMRESIFG10	W	0h	Raw interrupt status for MEMRES10. This bit is set to 1 when MEMRES10 is loaded with a new conversion result. Reading MEMRES10 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
17	MEMRESIFG9	W	0h	Raw interrupt status for MEMRES9. This bit is set to 1 when MEMRES9 is loaded with a new conversion result. Reading MEMRES9 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
16	MEMRESIFG8	W	0h	Raw interrupt status for MEMRES8. This bit is set to 1 when MEMRES8 is loaded with a new conversion result. Reading MEMRES8 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
15	MEMRESIFG7	W	0h	Raw interrupt status for MEMRES7. This bit is set to 1 when MEMRES7 is loaded with a new conversion result. Reading MEMRES7 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
14	MEMRESIFG6	W	0h	Raw interrupt status for MEMRES6. This bit is set to 1 when MEMRES6 is loaded with a new conversion result. Reading MEMRES6 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
13	MEMRESIFG5	W	0h	Raw interrupt status for MEMRES5. This bit is set to 1 when MEMRES5 is loaded with a new conversion result. Reading MEMRES5 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
12	MEMRESIFG4	W	0h	Raw interrupt status for MEMRES4. This bit is set to 1 when MEMRES4 is loaded with a new conversion result. Reading MEMRES4 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.

Table 10-35. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	MEMRESIFG3	W	0h	Raw interrupt status for MEMRES3. This bit is set to 1 when MEMRES3 is loaded with a new conversion result. Reading MEMRES3 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
10	MEMRESIFG2	W	0h	Raw interrupt status for MEMRES2. This bit is set to 1 when MEMRES2 is loaded with a new conversion result. Reading MEMRES2 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
9	MEMRESIFG1	W	0h	Raw interrupt status for MEMRES1. This bit is set to 1 when MEMRES1 is loaded with a new conversion result. Reading MEMRES1 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
8	MEMRESIFG0	W	0h	Raw interrupt status for MEMRES0. This bit is set to 1 when MEMRES0 is loaded with a new conversion result. Reading MEMRES0 register will clear this bit, or when the corresponding bit in ICLR is set to 1 0h = No new data ready. 1h = A new data is ready to be read.
7-0	RESERVED	W	0h	

10.3.25 EVT_MODE (Offset = 10E0h) [Reset = 00000009h]

EVT_MODE is shown in [Figure 10-29](#) and described in [Table 10-36](#).

Return to the [Summary Table](#).

Event mode register. It is used to select whether each line is disabled, in software mode (software clears the RIS) or in hardware mode (hardware clears the RIS)

Figure 10-29. EVT_MODE

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							
R-							
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED				EVT1_CFG		INT0_CFG	
R-				R-2h		R-1h	

Table 10-36. EVT_MODE Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3-2	EVT1_CFG	R	2h	Event line mode select for event corresponding to GEN_EVENT 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
1-0	INT0_CFG	R	1h	Event line mode select for event corresponding to CPU_INT 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.

10.3.26 DESC (Offset = 10FCh) [Reset = 26110010h]

DESC is shown in [Figure 10-30](#) and described in [Table 10-37](#).

Return to the [Summary Table](#).

This register identifies the peripheral and its exact version.

Figure 10-30. DESC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODULEID															
R-2611h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FEATUREVER				INSTNUM				MAJREV				MINREV			
R-0h				R-0h				R-1h				R-0h			

Table 10-37. DESC Field Descriptions

Bit	Field	Type	Reset	Description
31-16	MODULEID	R	2611h	Module identification contains a unique peripheral identification number. The assignments are maintained in a central database for all of the platform modules to ensure uniqueness. 0h = Smallest value FFFFh = Highest possible value
15-12	FEATUREVER	R	0h	Feature Set for the module *instance* 0h = Smallest value Fh = Highest possible value
11-8	INSTNUM	R	0h	Instance Number within the device. This will be a parameter to the RTL for modules that can have multiple instances
7-4	MAJREV	R	1h	Major rev of the IP 0h = Smallest value Fh = Highest possible value
3-0	MINREV	R	0h	Minor rev of the IP 0h = Smallest value Fh = Highest possible value

10.3.27 CTL0 (Offset = 1100h) [Reset = 00000000h]

CTL0 is shown in [Figure 10-31](#) and described in [Table 10-38](#).

Return to the [Summary Table](#).

Control Register 0

Figure 10-31. CTL0

31	30	29	28	27	26	25	24
RESERVED					SCLKDIV		
R/W-0h					R/W-0h		
23	22	21	20	19	18	17	16
RESERVED							PWRDN
R/W-0h							R/W-0h
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							ENC
R/W-0h							RH/W-0h

Table 10-38. CTL0 Field Descriptions

Bit	Field	Type	Reset	Description
31-27	RESERVED	R/W	0h	
26-24	SCLKDIV	R/W	0h	Sample clock divider 0h = Do not divide clock source 1h = Divide clock source by 2 2h = Divide clock source by 4 3h = Divide clock source by 8 4h = Divide clock source by 16 5h = Divide clock source by 24 6h = Divide clock source by 32 7h = Divide clock source by 48
23-17	RESERVED	R/W	0h	
16	PWRDN	R/W	0h	Power down policy 0h = ADC is powered down on completion of a conversion if there is no pending trigger 1h = ADC remains powered on as long as it is enabled through software.
15-1	RESERVED	R/W	0h	
0	ENC	RH/W	0h	Enable conversion 0h = Conversion disabled. ENC change from ON to OFF will abort single or repeat sequence on a MEMCTLx boundary. The current conversion will finish and result stored in corresponding MEMRESx. 1h = Conversion enabled. ADC sequencer waits for valid trigger (software or hardware).

10.3.28 CTL1 (Offset = 1104h) [Reset = 00000000h]

CTL1 is shown in [Figure 10-32](#) and described in [Table 10-39](#).

Return to the [Summary Table](#).

Control Register 1

Figure 10-32. CTL1

31	30	29	28	27	26	25	24
RESERVED	AVGD			RESERVED	AVGN		
R/W-0h			R/W-0h			R/W-0h	
23	22	21	20	19	18	17	16
RESERVED			SAMPMODE	RESERVED			CONSEQ
R/W-0h			R/W-0h	R/W-0h			R/W-0h
15	14	13	12	11	10	9	8
RESERVED							SC
R/W-0h							RH/W-0h
7	6	5	4	3	2	1	0
RESERVED							TRIGSRC
R/W-0h							R/W-0h

Table 10-39. CTL1 Field Descriptions

Bit	Field	Type	Reset	Description
31	RESERVED	R/W	0h	
30-28	AVGD	R/W	0h	Hardware averager denominator. The number to divide the accumulated value by (this is a shift). Note result register is maximum of 16-bits long so if not shifted appropriately result will be truncated. 0h (R/W) = No shift 1h (R/W) = 1 bit shift 2h (R/W) = 2 bit shift 3h (R/W) = 3 bit shift 4h (R/W) = 4 bit shift 5h (R/W) = 5 bit shift 6h (R/W) = 6 bit shift 7h (R/W) = 7 bit shift
27	RESERVED	R/W	0h	
26-24	AVGN	R/W	0h	Hardware averager numerator. Selects number of conversions to accumulate for current MEMCTLx and then it is divided by AVGD. Result will be stored in MEMRESx. 0h (R/W) = Disables averager 1h (R/W) = Averages 2 conversions before storing in MEMRESx register 2h (R/W) = Averages 4 conversions before storing in MEMRESx register 3h (R/W) = Averages 8 conversions before storing in MEMRESx register 4h (R/W) = Averages 16 conversions before storing in MEMRESx register 5h (R/W) = Averages 32 conversions before storing in MEMRESx register 6h (R/W) = Averages 64 conversions before storing in MEMRESx register 7h (R/W) = Averages 128 conversions before storing in MEMRESx register
23-21	RESERVED	R/W	0h	

Table 10-39. CTL1 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
20	SAMPMODE	R/W	0h	Sample mode. This bit selects the source of the sampling signal. MANUAL option is not valid when TRIGSRC is selected as hardware event trigger. 0h = Sample timer high phase is used as sample signal 1h = Software trigger is used as sample signal
19-18	RESERVED	R/W	0h	
17-16	CONSEQ	R/W	0h	Conversion sequence mode 0h = ADC channel in MEMCTLx pointed by STARTADD will be converted once 1h = ADC channel sequence pointed by STARTADD and ENDADD will be converted once 2h = ADC channel in MEMCTLx pointed by STARTADD will be converted repeatedly 3h = ADC channel sequence pointed by STARTADD and ENDADD will be converted repeatedly
15-9	RESERVED	R/W	0h	
8	SC	RH/W	0h	Start of conversion 0h = When SAMPMODE is set to MANUAL, clearing this bit will end the sample phase and the conversion phase will start. When SAMPMODE is set to AUTO, writing 0 has no effect. 1h = When SAMPMODE is set to MANUAL, setting this bit will start the sample phase. Sample phase will last as long as this bit is set. When SAMPMODE is set to AUTO, setting this bit will trigger the timer based sample time.
7-1	RESERVED	R/W	0h	
0	TRIGSRC	R/W	0h	Sample trigger source 0h = Software trigger 1h = Hardware event trigger

10.3.29 CTL2 (Offset = 1108h) [Reset = 00000000h]

CTL2 is shown in [Figure 10-33](#) and described in [Table 10-40](#).

Return to the [Summary Table](#).

Control Register 2

Figure 10-33. CTL2

31	30	29	28	27	26	25	24
RESERVED				ENDADD			
R/W-0h				R/W-0h			
23	22	21	20	19	18	17	16
RESERVED				STARTADD			
R/W-0h				R/W-0h			
15	14	13	12	11	10	9	8
SAMPCNT					FIFOEN	RESERVED	DMAEN
R/W-0h					R/W-0h	R/W-0h	RH/W-0h
7	6	5	4	3	2	1	0
RESERVED					RES		DF
R/W-0h					R/W-0h		R/W-0h

Table 10-40. CTL2 Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R/W	0h	
28-24	ENDADD	R/W	0h	Sequence end address. These bits select which MEMCTLx is the last one for the sequence mode. The value of ENDADD is 0x00 to 0x17, corresponding to MEMRES0 to MEMRES23. 00h = MEMCTL0 is selected as end address of sequence. 01h = MEMCTL1 is selected as end address of sequence. 02h = MEMCTL2 is selected as end address of sequence. 03h = MEMCTL3 is selected as end address of sequence. 04h = MEMCTL4 is selected as end address of sequence. 05h = MEMCTL5 is selected as end address of sequence. 06h = MEMCTL6 is selected as end address of sequence. 07h = MEMCTL7 is selected as end address of sequence. 08h = MEMCTL8 is selected as end address of sequence. 09h = MEMCTL9 is selected as end address of sequence. 0Ah = MEMCTL10 is selected as end address of sequence. 0Bh = MEMCTL11 is selected as end address of sequence. 0Ch = MEMCTL12 is selected as end address of sequence. 0Dh = MEMCTL13 is selected as end address of sequence. 0Eh = MEMCTL14 is selected as end address of sequence. 0Fh = MEMCTL15 is selected as end address of sequence. 10h = MEMCTL16 is selected as end address of sequence. 11h = MEMCTL17 is selected as end address of sequence. 12h = MEMCTL18 is selected as end address of sequence. 13h = MEMCTL19 is selected as end address of sequence. 14h = MEMCTL20 is selected as end address of sequence. 15h = MEMCTL21 is selected as end address of sequence. 16h = MEMCTL22 is selected as end address of sequence. 17h = MEMCTL23 is selected as end address of sequence.
23-21	RESERVED	R/W	0h	

Table 10-40. CTL2 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
20-16	STARTADD	R/W	0h	Sequencer start address. These bits select which MEMCTLx is used for single conversion or as first MEMCTL for sequence mode. The value of STARTADD is 0x00 to 0x17, corresponding to MEMRES0 to MEMRES23. 00h = MEMCTL0 is selected as start address of a sequence or for a single conversion. 01h = MEMCTL1 is selected as start address of a sequence or for a single conversion. 02h = MEMCTL2 is selected as start address of a sequence or for a single conversion. 03h = MEMCTL3 is selected as start address of a sequence or for a single conversion. 04h = MEMCTL4 is selected as start address of a sequence or for a single conversion. 05h = MEMCTL5 is selected as start address of a sequence or for a single conversion. 06h = MEMCTL6 is selected as start address of a sequence or for a single conversion. 07h = MEMCTL7 is selected as start address of a sequence or for a single conversion. 08h = MEMCTL8 is selected as start address of a sequence or for a single conversion. 09h = MEMCTL9 is selected as start address of a sequence or for a single conversion. 0Ah = MEMCTL10 is selected as start address of a sequence or for a single conversion. 0Bh = MEMCTL11 is selected as start address of a sequence or for a single conversion. 0Ch = MEMCTL12 is selected as start address of a sequence or for a single conversion. 0Dh = MEMCTL13 is selected as start address of a sequence or for a single conversion. 0Eh = MEMCTL14 is selected as start address of a sequence or for a single conversion. 0Fh = MEMCTL15 is selected as start address of a sequence or for a single conversion. 10h = MEMCTL16 is selected as start address of a sequence or for a single conversion. 11h = MEMCTL17 is selected as start address of a sequence or for a single conversion. 12h = MEMCTL18 is selected as start address of a sequence or for a single conversion. 13h = MEMCTL19 is selected as start address of a sequence or for a single conversion. 14h = MEMCTL20 is selected as start address of a sequence or for a single conversion. 15h = MEMCTL21 is selected as start address of a sequence or for a single conversion. 16h = MEMCTL22 is selected as start address of a sequence or for a single conversion. 17h = MEMCTL23 is selected as start address of a sequence or for a single conversion.
15-11	SAMPCNT	R/W	0h	Number of ADC converted samples to be transferred on a DMA trigger 0h = Minimum value 18h = Maximum value
10	FIFOEN	R/W	0h	Enable FIFO based operation 0h = Disable 1h = Enable
9	RESERVED	R/W	0h	

Table 10-40. CTL2 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
8	DMAEN	RH/W	0h	Enable DMA trigger for data transfer. Note: DMAEN bit is cleared by hardware based on DMA done signal at the end of data transfer. Software has to re-enable DMAEN bit for ADC to generate DMA triggers. 0h (R/W) = DMA trigger not enabled 1h (R/W) = DMA trigger enabled
7-3	RESERVED	R/W	0h	
2-1	RES	R/W	0h	Resolution. These bits define the resolution of ADC conversion result. Note : A value of 3 defaults to 12-bits resolution. 0h = 12-bits resolution 1h = 10-bits resolution 2h = 8-bits resolution
0	DF	R/W	0h	Data read-back format. Data is always stored in binary unsigned format. 0h = Digital result reads as Binary Unsigned. 1h = Digital result reads Signed Binary. (2s complement), left aligned.

10.3.30 CTL3 (Offset = 110Ch) [Reset = 00000000h]

CTL3 is shown in [Figure 10-34](#) and described in [Table 10-41](#).

Return to the [Summary Table](#).

Control Register 3. This register is used to configure ADC for ad-hoc single conversion.

Figure 10-34. CTL3

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED		ASCVRSEL		RESERVED		ASCSTIME	
R/W-0h		R/W-0h		R/W-0h		R/W-0h	
7	6	5	4	3	2	1	0
RESERVED			ASCCHSEL				
R/W-0h			R/W-0h				

Table 10-41. CTL3 Field Descriptions

Bit	Field	Type	Reset	Description
31-14	RESERVED	R/W	0h	
13-12	ASCVRSEL	R/W	0h	Selects voltage reference for ASC operation. VEREFM must be connected to on-board ground when external reference option is selected. Note: Writing value 0x3 defaults to INTREF. 0h = VDDA reference. 1h = EXTREF pin reference. 2h = Internal reference.
11-9	RESERVED	R/W	0h	
8	ASCSTIME	R/W	0h	ASC sample time compare value select. This is used to select between SCOMP0 and SCOMP1 registers for ASC operation. 0h = Select SCOMP0 1h = Select SCOMP1
7-5	RESERVED	R/W	0h	

Table 10-41. CTL3 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
4-0	ASCCHSEL	R/W	0h	ASC channel select 00h = Selects channel 0 01h = Selects channel 1 02h = Selects channel 2 03h = Selects channel 3 04h = Selects channel 4 05h = Selects channel 5 06h = Selects channel 6 07h = Selects channel 7 08h = Selects channel 8 09h = Selects channel 9 0Ah = Selects channel 10 0Bh = Selects channel 11 0Ch = Selects channel 12 0Dh = Selects channel 13 0Eh = Selects channel 14 0Fh = Selects channel 15 10h = Selects channel 16 11h = Selects channel 17 12h = Selects channel 18 13h = Selects channel 19 14h = Selects channel 20 15h = Selects channel 21 16h = Selects channel 22 17h = Selects channel 23 18h = Selects channel 24 19h = Selects channel 25 1Ah = Selects channel 26 1Bh = Selects channel 27 1Ch = Selects channel 28 1Dh = Selects channel 29 1Eh = Selects channel 30 1Fh = Selects channel 31

10.3.31 CLKFREQ (Offset = 1110h) [Reset = 00000000h]

CLKFREQ is shown in [Figure 10-35](#) and described in [Table 10-42](#).

Return to the [Summary Table](#).

Sampling clock frequency range register.

Figure 10-35. CLKFREQ

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												FRANGE			
R/W-0h												R/W-0h			

Table 10-42. CLKFREQ Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R/W	0h	
2-0	FRANGE	R/W	0h	Frequency Range. 0h = 1 to 4 MHz 1h = >4 to 8 MHz 2h = >8 to 16 MHz 3h = >16 to 20 MHz 4h = >20 to 24 MHz 5h = >24 to 32 MHz 6h = >32 to 40 MHz 7h = >40 to 48 MHz

10.3.32 SCOMP0 (Offset = 1114h) [Reset = 00000000h]

SCOMP0 is shown in [Figure 10-36](#) and described in [Table 10-43](#).

Return to the [Summary Table](#).

Sample time compare 0 register. Specifies the sample time, in number of ADC sample clock cycles. CTL0.ENC must be 0 to write to this register.

Figure 10-36. SCOMP0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0						
RESERVED																						VAL															
R/W-0h																						R/W-0h															

Table 10-43. SCOMP0 Field Descriptions

Bit	Field	Type	Reset	Description
31-10	RESERVED	R/W	0h	
9-0	VAL	R/W	0h	Specifies the number of sample clocks. When VAL = 0 or 1, number of sample clocks = Sample clock divide value. When VAL > 1, number of sample clocks = VAL x Sample clock divide value. Note: Sample clock divide value is not the value written to SCLKDIV but the actual divide value (SCLKDIV = 2 implies divide value is 4). Example: VAL = 4, SCLKDIV = 3 implies 32 sample clock cycles.

10.3.33 SCOMP1 (Offset = 1118h) [Reset = 00000000h]

SCOMP1 is shown in [Figure 10-37](#) and described in [Table 10-44](#).

Return to the [Summary Table](#).

Sample time compare 1 register. Specifies the sample time, in number of ADC sample clock cycles. CTL0.ENC must be 0 to write to this register.

Figure 10-37. SCOMP1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0						
RESERVED																						VAL															
R/W-0h																						R/W-0h															

Table 10-44. SCOMP1 Field Descriptions

Bit	Field	Type	Reset	Description
31-10	RESERVED	R/W	0h	
9-0	VAL	R/W	0h	Specifies the number of sample clocks. When VAL = 0 or 1, number of sample clocks = Sample clock divide value. When VAL > 1, number of sample clocks = VAL x Sample clock divide value. Note: Sample clock divide value is not the value written to SCLKDIV but the actual divide value (SCLKDIV = 2 implies divide value is 4). Example: VAL = 4, SCLKDIV = 3 implies 32 sample clock cycles.

10.3.34 REFCFG (Offset = 111Ch) [Reset = 00000000h]

REFCFG is shown in [Figure 10-38](#) and described in [Table 10-45](#).

Return to the [Summary Table](#).

Reference buffer configuration register

Figure 10-38. REFCFG

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED			IBPROG		RESERVED	REFVSEL	REFEN
R/W-0h			R/W-0h		R/W-0h	R/W-0h	R/W-0h

Table 10-45. REFCFG Field Descriptions

Bit	Field	Type	Reset	Description
31-5	RESERVED	R/W	0h	
4-3	IBPROG	R/W	0h	Configures reference buffer bias current output value 0h = 1uA 1h = 0.5uA 2h = 2uA 3h = 0.67uA
2	RESERVED	R/W	0h	
1	REFVSEL	R/W	0h	Configures reference buffer output voltage 0h = Reference buffer generates 2.5 V output 1h = Reference buffer generates 1.4 V output
0	REFEN	R/W	0h	Reference buffer enable 0h = Disable 1h = Enable

10.3.35 WCLOW (Offset = 1148h) [Reset = 00000000h]

WCLOW is shown in [Figure 10-39](#) and described in [Table 10-46](#).

Return to the [Summary Table](#).

Window Comparator Low Threshold Register.

The data format that is used to write and read WCLOW depends on the value of DF bit in CTL2 register.

CTL0.ENC must be 0 to write to this register.

Note: Change in ADC data format or resolution does not reset WCLOW.

Figure 10-39. WCLOW

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																DATA															
R/W-0h																R/W-0h															

Table 10-46. WCLOW Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15-0	DATA	R/W	0h	If DF = 0, unsigned binary format has to be used. The value based on the resolution has to be right aligned with the MSB on the left. For 10-bits and 8-bits resolution, unused bits have to be 0s. If DF = 1, 2s-complement format has to be used. The value based on the resolution has to be left aligned with the LSB on the right. For 10-bits and 8-bits resolution, unused bits have to be 0s.

10.3.36 WCHIGH (Offset = 1150h) [Reset = 00000000h]

WCHIGH is shown in [Figure 10-40](#) and described in [Table 10-47](#).

Return to the [Summary Table](#).

Window Comparator High Threshold Register.

The data format that is used to write and read WCHIGH depends on the value of DF bit in CTL2 register.

CTL0.ENC must be 0 to write to this register.

Note: Change in ADC data format or resolution does not reset WCHIGH.

Figure 10-40. WCHIGH

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																DATA															
R/W-0h																R/W-0h															

Table 10-47. WCHIGH Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15-0	DATA	R/W	0h	If DF = 0, unsigned binary format has to be used. The threshold value has to be right aligned, with the MSB on the left. For 10-bits and 8-bits resolution, unused bit have to be 0s. If DF = 1, 2s-complement format has to be used. The value based on the resolution has to be left aligned with the LSB on the right. For 10-bits and 8-bits resolution, unused bit have to be 0s.

10.3.37 FIFODATA (Offset = 1160h) [Reset = 00000000h]

FIFODATA is shown in [Figure 10-41](#) and described in [Table 10-48](#).

Return to the [Summary Table](#).

FIFO data register. This is a virtual register used to do read from FIFO.

Figure 10-41. FIFODATA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA																															
R-																															

Table 10-48. FIFODATA Field Descriptions

Bit	Field	Type	Reset	Description
31-0	DATA	R	0h	Read from this data field returns the ADC sample from FIFO.

10.3.38 ASCRES (Offset = 1170h) [Reset = 00000000h]

ASCRES is shown in [Figure 10-42](#) and described in [Table 10-49](#).

Return to the [Summary Table](#).

ASC result register

Figure 10-42. ASCRES

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																DATA															
R-0h																R-0h															

Table 10-49. ASCRES Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15-0	DATA	R	0h	Result of ADC ad-hoc single conversion. If DF = 0, unsigned binary: The conversion result is right aligned. In 10 and 8 bit modes, the unused MSB bits are forced to 0. If DF = 1, 2s-complement format: The conversion result is left aligned. In 10 and 8 bit modes, the unused LSB bits are forced to 0. The data is stored in the right-justified format and is converted to the left-justified 2s-complement format during read back.

10.3.39 MEMCTL[y] (Offset = 1180h + formula) [Reset = 00000000h]

MEMCTL[y] is shown in [Figure 10-43](#) and described in [Table 10-50](#).

Return to the [Summary Table](#).

Conversion Memory Control Register.

CTL0.ENC must be 0 to write to this register.

Offset = 1180h + (y * 4h); where y = 0h to 17h

Figure 10-43. MEMCTL[y]

31	30	29	28	27	26	25	24
RESERVED			WINCOMP	RESERVED			TRIG
R/W-0h			R/W-0h	R/W-0h			R/W-0h
23	22	21	20	19	18	17	16
RESERVED			BCSEN	RESERVED			AVGEN
R/W-0h			R/W-0h	R/W-0h			R/W-0h
15	14	13	12	11	10	9	8
RESERVED			STIME	RESERVED			VRSEL
R/W-0h			R/W-0h	R/W-0h			R/W-0h
7	6	5	4	3	2	1	0
RESERVED			CHANSEL				
R/W-0h			R/W-0h				

Table 10-50. MEMCTL[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R/W	0h	
28	WINCOMP	R/W	0h	Enable window comparator. 0h = Disable 1h = Enable
27-25	RESERVED	R/W	0h	
24	TRIG	R/W	0h	Trigger policy. Indicates if a trigger will be needed to step to the next MEMCTL in the sequence or to perform next conversion in the case of repeat single channel conversions. 0h = Next conversion is automatic 1h = Next conversion requires a trigger
23-21	RESERVED	R/W	0h	
20	BCSEN	R/W	0h	Enable burn out current source. 0h = Disable 1h = Enable
19-17	RESERVED	R/W	0h	
16	AVGEN	R/W	0h	Enable hardware averaging. 0h (R/W) = Averaging disabled. 1h = Averaging enabled.
15-13	RESERVED	R/W	0h	
12	STIME	R/W	0h	Selects the source of sample timer period between SCOMP0 and SCOMP1. 0h = Select SCOMP0 1h = Select SCOMP1
11-10	RESERVED	R/W	0h	

Table 10-50. MEMCTL[y] Field Descriptions (continued)

Bit	Field	Type	Reset	Description
9-8	VRSEL	R/W	0h	Voltage reference selection. VEREFM must be connected to on-board ground when external reference option is selected. Note: Writing value 0x3 defaults to INTREF. 0h = VDDA reference 1h = External reference from pin 2h = Internal reference
7-5	RESERVED	R/W	0h	
4-0	CHANSEL	R/W	0h	Input channel select. 00h = Selects channel 0 01h = Selects channel 1 02h = Selects channel 2 03h = Selects channel 3 04h = Selects channel 4 05h = Selects channel 5 06h = Selects channel 6 07h = Selects channel 7 08h = Selects channel 8 09h = Selects channel 9 0Ah = Selects channel 10 0Bh = Selects channel 11 0Ch = Selects channel 12 0Dh = Selects channel 13 0Eh = Selects channel 14 0Fh = Selects channel 15 10h = Selects channel 16 11h = Selects channel 17 12h = Selects channel 18 13h = Selects channel 19 14h = Selects channel 20 15h = Selects channel 21 16h = Selects channel 22 17h = Selects channel 23 18h = Selects channel 24 19h = Selects channel 25 1Ah = Selects channel 26 1Bh = Selects channel 27 1Ch = Selects channel 28 1Dh = Selects channel 29 1Eh = Selects channel 30 1Fh = Selects channel 31

10.3.40 MEMRES[y] (Offset = 1280h + formula) [Reset = 00000000h]

MEMRES[y] is shown in [Figure 10-44](#) and described in [Table 10-51](#).

Return to the [Summary Table](#).

Memory Result Register

Offset = 1280h + (y * 4h); where y = 0h to 17h

Figure 10-44. MEMRES[y]

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																DATA															
R-																R-															

Table 10-51. MEMRES[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15-0	DATA	R	0h	MEMRES result register. If DF = 0, unsigned binary: The conversion results are right aligned. In 10 and 8 bit modes, the unused MSB bits are forced to 0. If DF = 1, 2s-complement format: The conversion results are left aligned. In 10 and 8 bit modes, the unused LSB bits are forced to 0. The data is stored in the right-justified format and is converted to the left-justified 2s-complement format during read back.

10.3.41 STATUS (Offset = 1340h) [Reset = 00000000h]

STATUS is shown in [Figure 10-45](#) and described in [Table 10-52](#).

Return to the [Summary Table](#).

Status Register

Figure 10-45. STATUS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED					ASCACT	REFBUFDRDY	BUSY
R-0h					R-0h	R-0h	R-0h

Table 10-52. STATUS Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R	0h	
2	ASCACT	R	0h	ASC active 0h = Idle or done 1h = ASC active
1	REFBUFDRDY	R	0h	Indicates reference buffer is powered up and ready. 0h = Not ready 1h = Ready
0	BUSY	R	0h	Busy. This bit indicates that an active ADC sample or conversion operation is in progress. 0h = No ADC sampling or conversion in progress. 1h = ADC sampling or conversion is in progress.



The comparator module (COMP) is an analog voltage comparator with general comparator functionality. This chapter describes the operation of the comparator module.

11.1 Comparator Overview.....	678
11.2 Comparator Operation.....	679
11.3 COMP Registers.....	688

11.1 Comparator Overview

The comparator module can be useful for supply voltage supervision and monitoring of external analog signals.

Features of comparator include:

- Fast and ultra-low-power modes of operation
- Inverting and non-inverting terminal input multiplexer
- Inverting and non-inverting terminal short and exchange capabilities
- Software-selectable analog filter for the comparator output
- Programmable hysteresis
- Reference voltage from internal voltage reference, VDD or from external reference pin
- Configurable reference voltage generator from integrated 8-bit DAC with two input codes
- Output connected to event system
- Window comparator mode
- Interrupt driven measurement system for low-power operation

Figure 11-1 shows the comparator block diagram.

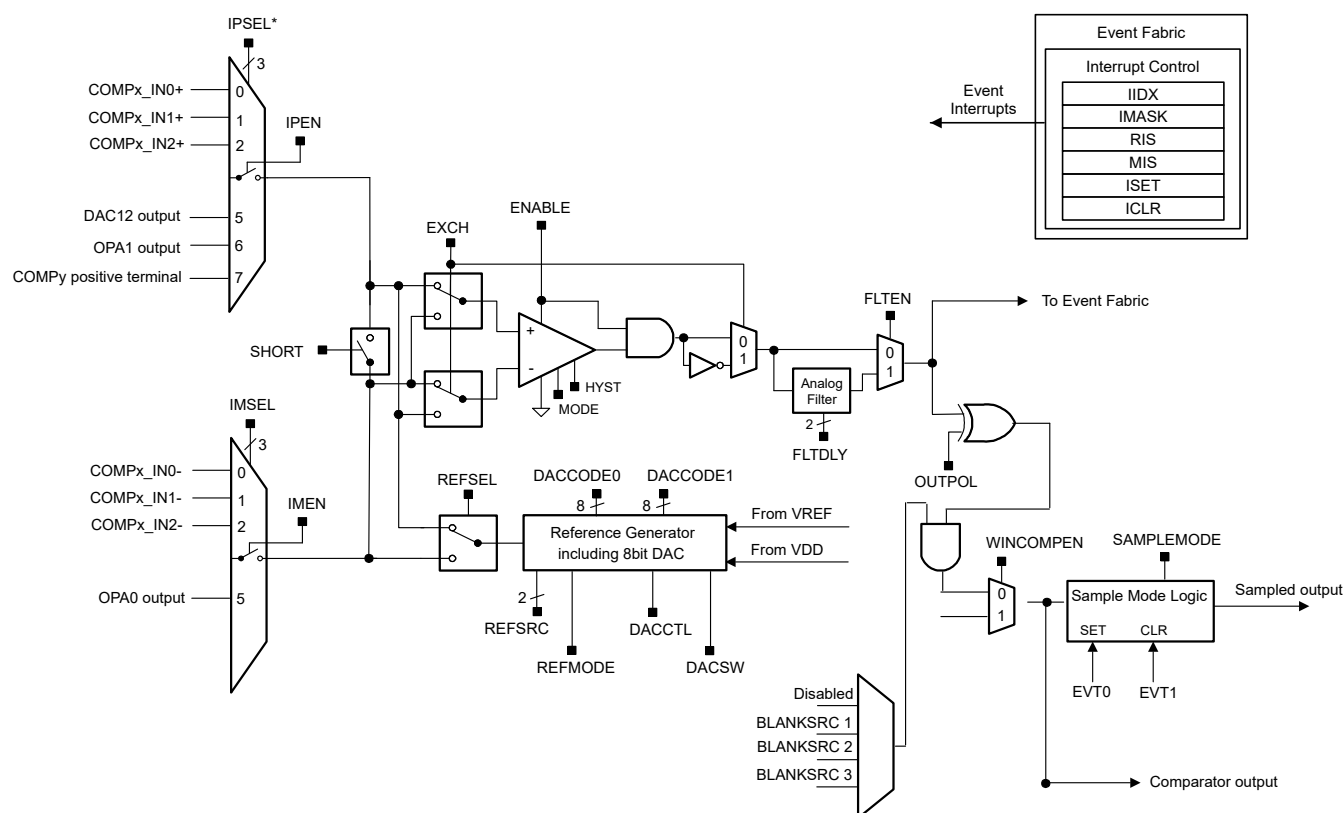


Figure 11-1. Comparator Block Diagram

11.2 Comparator Operation

The comparator module is configured by user software. The setup and operation of comparator is discussed in the following sections.

11.2.1 Comparator Configuration

The comparator compares the analog voltages at the positive (+) and negative (–) input terminals. If the + terminal is more positive than the – terminal, the comparator output is high. The polarity of the comparator output can be configured by using the OUTPOL bit in COMPx.CTL1 register. The comparator can be switched on or off using ENABLE bit in COMPx.CTL1 register. The comparator should be switched off when not in use to reduce current consumption. When the comparator is off, OUT is low when OUTPOL bit is set to 0, and OUT is high when OUTPOL bit is set to 1.

The comparator can be configured in fast or ultra-low-power mode using MODE bit in COMPx.CTL1 register. The default value of MODE bit is 0 which configures the comparator in fast mode. When MODE bit is set to 1 the comparator is configured in ultra-low-power mode. In the fast mode the comparator consumes higher current but the response time is faster. In the ultra-low power mode the comparator consumes very low current but response time is slower. To optimize current consumption for the application, the power mode that meets the comparator speed requirements should be selected (please see the device-specific data sheet for the comparator propagation delay).

The clock control for comparator is managed by System Controller (SYSCTL), SYSCTL knows if comparator module is enabled and it also knows if it is in ultra-low-power mode or fast mode. User needs to ensure the proper bus clock is selected for different comparator operation mode:

- For ultra-low-power mode comparator, bus clock can be LFCLK or any of the high speed clocks.
- For fast mode comparator, bus clock cannot be LFCLK, system will generate a clock error interrupt in SYSCTL if you enable a comparator in fast mode and the bus clock is LFCLK.

11.2.2 Comparator Channels Selection

The comparator channels on positive terminal is selected through IPSEL bits and enabled through IPEN bit in the register COMPx.CTL0. Similarly the comparator channels on negative terminal is selected through IMSEL bits and enabled through IMEN bit. The IPSEL and IMSEL bits can be used to select the comparator channel inputs from device pins or from internal analog modules. The switches in the comparator input channels multiplexer on positive and negative terminals are implemented using break-before-make arrangement to minimize the cross talk when the channel selection changes.

The IPSEL and IMSEL bits allow:

- Connection of an external signal to the positive and negative terminals of the comparator
- Connection of an internal analog signal (like DAC or OPA) to the positive and negative terminals of the comparator
- Connection of one comparator positive terminal signal to another comparator positive terminal in window compare mode

The EXCH bit in COMPx.CTL1 register controls the input multiplexer, exchange the input signals of the comparator positive and negative terminals. Additionally, when the comparator terminals are exchanged, the output signal from the comparator is inverted too. This allows the user to determine or compensate for the comparator input offset voltage.

Note

Comparator Input Connection

Please see the device data sheet COMP section for the input channel selection configuration.

When the comparator is on, the input terminals should be connected to a signal, power, or ground. Otherwise, floating levels can cause unexpected interrupts and increased current consumption.

Comparator Configuration

The configuration of the comparator must not change while it is in operation except for changes in IPSEL, IMSEL and EXCH settings in the register COMPx.CTL0. To change the configuration of the comparator it must be first disabled, reconfigured as needed and then re-enabled.

11.2.3 Comparator Output

The comparator output is readable from the OUT bit in COMPx.STAT register. The output is also brought out on device pin. For successful connection of comparator output with the device pin, it is necessary for the software to configure the corresponding device pin in IOMUX module. The comparator output also goes to the Event Interface to generate CPU interrupt event and generic publisher event for internal connects with other modules.

11.2.4 Output Filter

The output of the comparator can be used with or without internal filtering. Output filtering can only be enabled while Comparator is in fast mode (COMPx.CTL1.MODE = 0). When FLTEN bit in COMPx.CTL1 register is set, the output is filtered with an on-chip analog filter. The delay of the filter can be adjusted in four different steps using the FLTDLY bits in COMPx.CTL1 register (see [Table 11-1](#)).

Table 11-1. Typical Output Filter Delay Settings

FLTDLY bit in COMPx.CTL1	Typical filter delay
0	70ns
1	500ns
2	1200ns
3	2700ns

Filter delay settings are an addition to propagation delay outlined in device specific data sheet.

The comparator output oscillates if the voltage difference across the input terminals is small (see [Figure 11-2](#)). Internal and external parasitic effects and cross coupling on and between signal lines, power supply lines, and other parts of the system result in this behavior. The comparator output oscillation reduces the accuracy and resolution of the comparison result. Selecting the output filter can reduce errors associated with comparator oscillation.

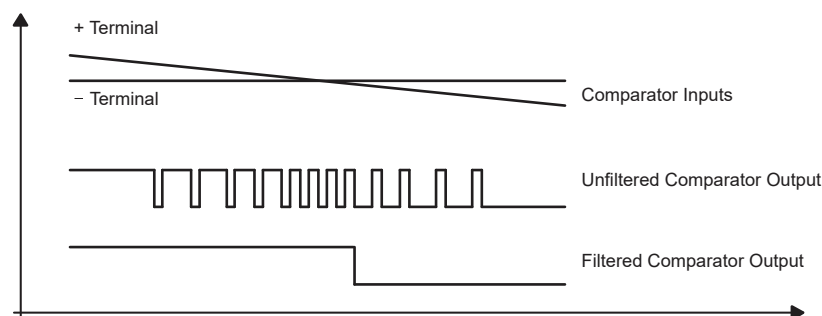


Figure 11-2. Analog Filter Response at the Output of the Comparator

Note

The output filter only works for the comparator when it operates in fast mode.

11.2.5 Sampled Output Mode

Sampled mode is a method of operation for the comparator, which compares input signals at discrete time intervals or samples, producing output signals that change only at the sampling points. The SAMPMODE bit in the register CTL2 enables or disables the sampled mode.

To define the sampling window, two events, EVT0 and EVT1, are generated from the timer. The sampling window can be aligned to less noisy phases with the use of these events. EVT0 sets the sampling window, while EVT1 clears it.

In sampled mode, the output of the comparator is captured only when the sampling window is high, and the output is not captured at other times. The captured output is used for interrupt and event generation.

By using sampled mode, the comparator can reduce the effects of noise in the input signals, resulting in more accurate and reliable comparisons. The use of EVT0 and EVT1 to define the sampling window allows for greater flexibility in aligning the window to the least noisy phases of the input signals, further improving the accuracy of the comparisons.

Please note following when internal DAC8 is used in sampled mode and user wants to enter Standby0 mode; for comparator output to work as expected, it is to be ensured that COMP is ready by polling the OUTRDYIFG flag before SoC enters Standby0 mode.

User can follow the below sequence for this scenario.

1. In run/active mode, do all the necessary COMP configurations (DAC8 in sampled mode, COMP in ULP mode, etc.)
2. Set power policy to standby0 (DL_SYSCCTL_setPowerPolicySTANDBY0)
3. Enable COMP (CTL1.EN bit to be written)
4. Wait for COMP OUTRDYIFG to be set (wait for bit number 3 in RIS register to be 1)
5. Enter Standby0 low power mode (using __WFI() function)

11.2.6 Blanking Mode

Due to the noise or other interference, the input signal may fluctuate around the threshold voltage, causing the output to switch rapidly and erratically. This can lead to false triggering. Comparator blanking mode introduces a brief delay between the time the input signal crosses the threshold voltage and the time the output changes state.

During this delay period, the comparator ignores any further changes in the input signal and maintains its current output state, this is mainly used in some uses cases like PWM waves indicated by a timer in motor drive, power control and so on.

The control bits BLANKSRC in the register CTL2 can be used to configure disable or which source should be working on blanking mode. Refer to the data sheet of the device to see the specific blanking sources supported.

Note

The timer is used as the source of comparator blanking mode, there will typically be a one TIMCLK cycle delay between the timer output and the comparator's output.

11.2.7 Reference Voltage Generator

Figure 11-3 shows the block diagram of the comparator reference voltage generator. The comparator reference voltage generator consists of a 8-bit DAC along with some configuration options. The REFSRC bits in COMPx.CTL2 register are used to select the reference source for the comparator.

- When REFSRC = 0, the reference voltage generator is disabled and the reference voltage generator cannot be used for comparator operation.

- When REFSRC = 1, the analog supply VDDA is selected as the reference input for the DAC and the DAC output is used as reference voltage for the comparator.
- When REFSRC = 2, the VREF from internal reference module output is selected as the reference input for the DAC and the DAC output is used as reference voltage for the comparator.
- When REFSRC = 3, the VREF from internal reference module output is used directly as reference voltage for the comparator and the DAC is switched off.

The reference voltage generator output can be applied to either the positive terminal or negative terminal of the comparator using the REFSEL bit COMPx.CTL2 register. If external signals are applied to both comparator input terminals, turn off the internal reference voltage generator to reduce current consumption.

When the reference voltage generator output is applied on a comparator terminal using REFSEL bit COMPx.CTL2 register and the comparator channel (from device pins or from internal analog modules) is also selected on the same terminal using IPSEL/IPEN or IMSEL/IMEN bits in COMPx.CTL0 register then the comparator channel selection takes precedence.

DAC8 output is also connected to the internal analog OPA module through OPAX (see [Figure 12-1](#)).

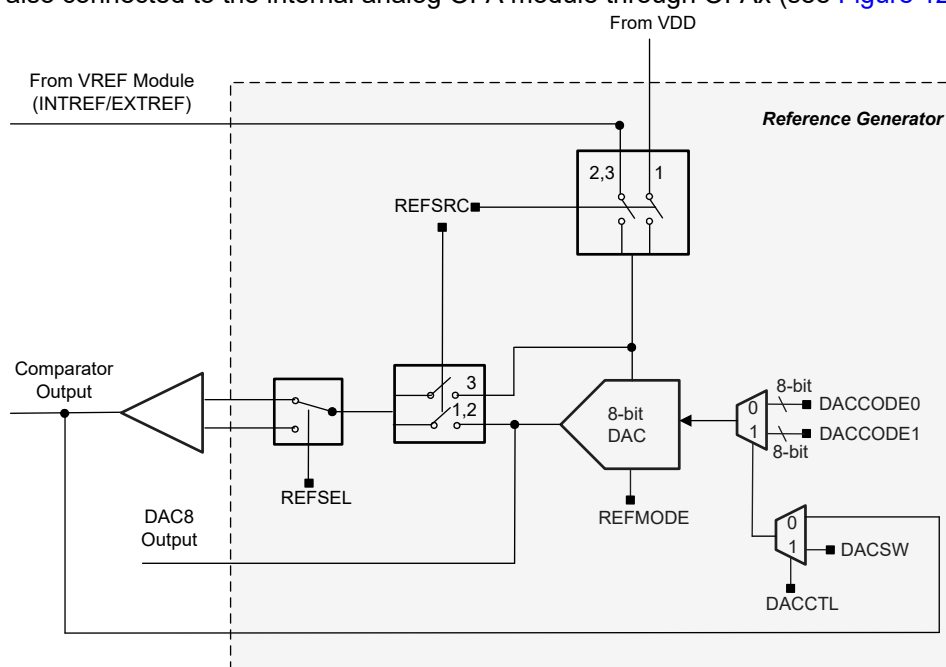


Figure 11-3. Reference Voltage Generator Block Diagram

Integrated 8-bit DAC

The 8-bit DAC input code can be provided through DACCODE0 or DACCODE1 bits in COMPx.CTL3 register. The DACCTL bit in COMPx.CTL2 register determines if the comparator output or a software control bit DACSW in COMPx.CTL2 register selects DACCODE0 or DACCODE1 bits as input to DAC.

- When DACCTL is 0 the comparator output value selects DACCODE0 or DACCODE1. If the comparator output value is 0, DACCODE0 will be the input to DAC. And DACCODE1 will be the input to DAC if the comparator output value is 1.
- When DACCTL is 1 the DACSW bit value that is programmed by software selects DACCODE0 or DACCODE1. If DACSW is 0, DACCODE0 will be the input to DAC. And DACCODE1 will be the input to DAC if DACSW is 1. With this provision it is possible to generate desired hysteresis levels for the comparator without using external components.
- The REFMODE bit in COMPx.CTL2 register determines if the comparator requests for internal VREF operation in fast or ultra-low-power and also the mode of operation of the 8-bit DAC. When REFMODE bit is 0, the internal VREF is requested for operation in fast mode and the 8-bit DAC in comparator is configured in

fast mode as well. When REFMODE bit is 1, the internal VREF is requested for operation in ultra-low-power and the 8-bit DAC in comparator in comparator is configured in ultra-low-power. Operation in fast mode offers higher accuracy but consumes higher current while the ultra-low-power operation consumes lower current but with relaxed reference voltage accuracy. Refer to comparator electrical specifications in device-specific data sheet for details.

11.2.8 Window Comparator Mode

The purpose of window comparator mode is to monitor the input signal within a specified threshold range defined by lower and upper thresholds.

As [Figure 11-4](#) shows, COMP0 and COMP1 can combine to create a window comparator. The input signal is connected to the positive terminal of comparators connected together and the upper and lower threshold voltages are connected to the negative terminal of the comparators.

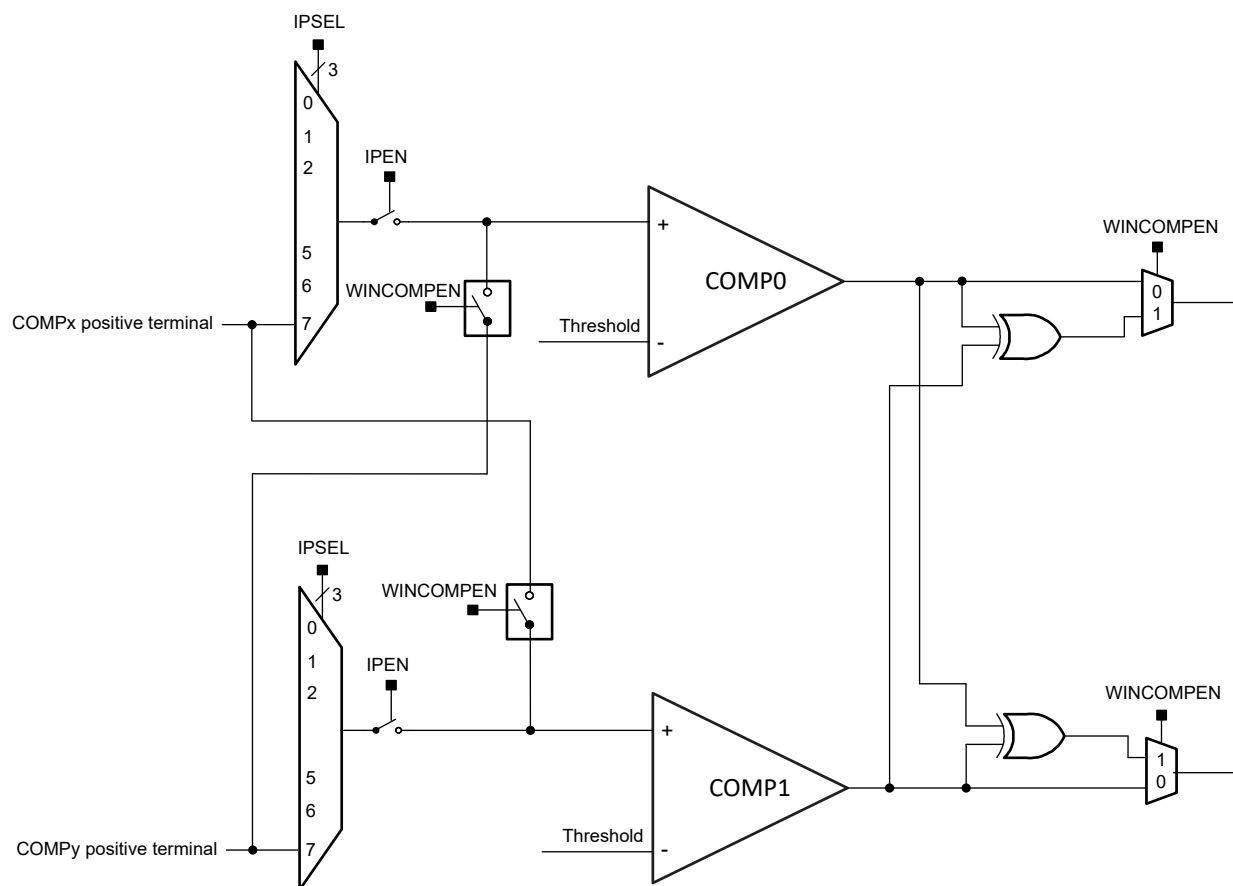


Figure 11-4. Window Comparator Mode

Window comparator mode can be enabled by set the WINCOMPEN bit in COMPx.CTL1 register. [Figure 11-5](#) below shows a example of a window mode configuration:

- Configure IPSEL bits in COMP0.CTL0 register to the corresponding input signal pin.
- Set WINCOMPEN bit in COMP0.CTL1 register to 1.
- Clear the WINCOMPEN bit in COMP1.CTL1 register.
- Configure IPSEL bits in COMP1.CTL0 register to 0x07, select the COMP0 positive terminal as the input.

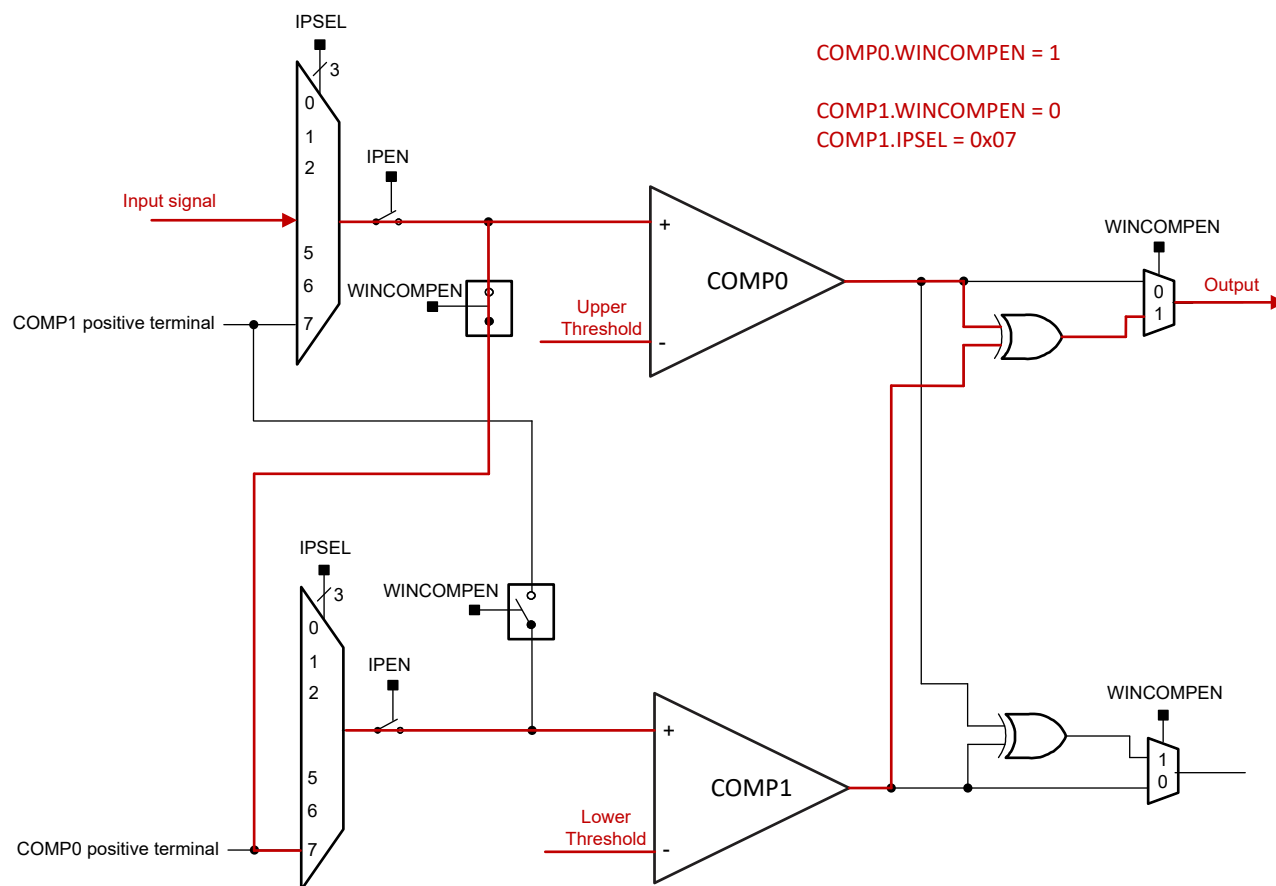


Figure 11-5. Window Comparator Mode Configuration

11.2.9 Comparator Hysteresis

The comparator supports programmable hysteresis voltages for fast and ultra-low-power modes to avoid spurious output transitions in case of noisy input signals. When HYST bits in COMPx.CTL1 register are 0, the comparator generates no hysteresis. 10 mV, 20 mV, and 30 mV typical hysteresis voltages are generated by the comparator for HYST bits values 1, 2, and 3 respectively.

The hysteresis can also be implemented by using the internal 8-bit DAC. The input of the reference generator 8-bit DAC can be provided by two values DACCODE0 and DACCODE1 in COMPx.CTL3 register. User can configure the DACCTL bit in COMPx.CTL2 register to 1 and the comparator output value will select the DAC input between DACCODE0 and DACCODE1 values. With this provision it is possible to generate desired hysteresis levels for the comparator without using external components. For more information about this configuration please refer to [Integrated 8-bit DAC](#) section.

11.2.10 Input SHORT Switch

The SHORT bit shorts the comparator inputs. This can be used to build a simple sample-and-hold for the comparator (see [Figure 11-6](#))

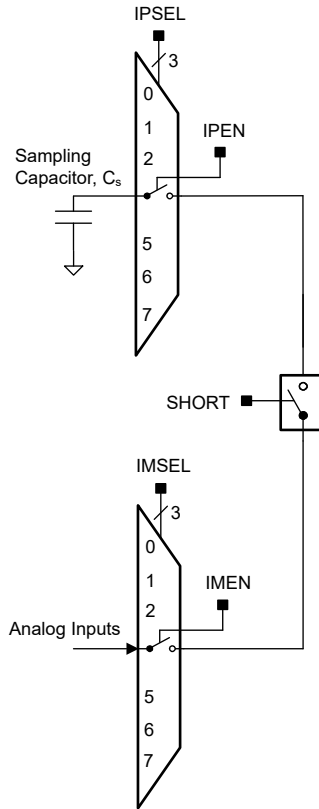


Figure 11-6. Comparator Short Switch

The required sampling time is proportional to the size of the sampling capacitor (CS), the resistance of the input switches in series with the short switch (RI) and the resistance of the external source (RS). The sampling capacitor CS should be greater than 100 pF. The time constant, Tau, to charge the sampling capacitor CS can be calculated with [Equation 22](#).

$$T_{au} = (R_I + R_S) \times C_S \quad (22)$$

Depending on the required accuracy, use 3 to 10 Tau as the sampling time. With 3 Tau the sampling capacitor is charged to approximately 95% of the input signals voltage level, with 5 Tau it is charged to more than 99%, and with 10 Tau the sampled voltage is sufficient for 12-bit accuracy.

11.2.11 Interrupt and Events Support

The comparator module contains two [event publishers](#) and two [event subscribers](#). One event publisher (CPU_INT) manages comparator interrupt requests (IRQs) to the CPU subsystem through a [static event route](#). The second event publisher (GEN_EVENT) is used to setup the generic event publisher through [Generic route](#).

The two event subscribers are used to define the start and stop points in [sampled output mode](#).

The comparator events are summarized in [Table 11-2](#).

Table 11-2. Comparator Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU interrupt	Publisher	Comparator	CPU Subsystem	Static route	CPU_INT registers	Fixed interrupt route from comparator to CPU

Table 11-2. Comparator Events (continued)

Event	Type	Source	Destination	Route	Configuration	Functionality
Generic publisher event	Publisher	Comparator	Other peripherals	Generic route	GEN_EVENT FPUB_1 registers	Configurable route from comparator to other peripherals
Generic subscriber event	Subscriber	Other peripherals	Comparator	Generic route	GEN_EVENT FSUB_0 registers	Configurable route from other peripherals to the comparator start of sample window
Generic subscriber event	Subscriber	Other peripherals	Comparator	Generic route	GEN_EVENT FSUB_1 registers	Configurable route from other peripherals to the comparator stop sampling window

11.2.11.1 CPU Interrupt Event Publisher (CPU_INT)

The comparator module provides 3 interrupt sources which can be configured to source a [CPU interrupt event](#). In order of decreasing interrupt priority, the CPU interrupt events from the comparator are:

Table 11-3. Comparator CPU Interrupt Event Conditions (CPU_INT)

IIDX STAT	Name	Description
0x01	COMPIFG	The interrupt flags COMPIFG and COMPINVIFG are set either on the rising or falling edge of the comparator output, selected by the IES bit. When IES bit is 0, rising edge of the comparator output sets the COMPIFG and falling edge sets the COMPINVIFG. When IES bit is 1, falling edge of the comparator output sets the COMPIFG and rising edge sets the COMPINVIFG.
0x02	COMPINVIFG	The interrupt flags COMPIFG and COMPINVIFG are set either on the rising or falling edge of the comparator output, selected by the IES bit. When IES bit is 0, rising edge of the comparator output sets the COMPIFG and falling edge sets the COMPINVIFG. When IES bit is 1, falling edge of the comparator output sets the COMPIFG and rising edge sets the COMPINVIFG.
0x03	OUTRDYIFG	Comparator output ready interrupt. This bit is set when the comparator output is valid.

The CPU interrupt event configuration is managed with the CPU_INT event management registers. See [Section 7.2.5](#) for guidance on configuring the Event registers for CPU interrupts.

11.2.11.2 Generic Event Publisher (GEN_EVENT)

A generic route is a route in which the comparator peripheral publishing the event is configured to use one of several available generic route channels to publish its event to another entity (or entities, in the case of a splitter route), where an entity can be another peripheral, a generic DMA trigger event, or a generic CPU event.

The GEN_EVENT register is used to select a peripheral condition ([Table 11-4](#)) to use for publishing an event. FPUB_1 is the publisher port register and it is used to configure which generic route channel to use to broadcast the event. A second peripheral, the DMA, or the CPU can subscribe to this event by configuring its subscriber port to listen on the same generic route channel which the publishing peripheral is connected to.

For example, through the use of a generic event channel, it is possible to directly trigger a timer capture from a comparator event, by connecting a comparator FPUB_1 and TIM FSUB_x to the same generic event channel. Refer to [Section 7.1.3.3](#) and [Section 7.2.3](#) for how generic event route works.

Table 11-4. Comparator Generic Event Conditions (GEN_EVENT)

IIDX STAT	Name	Description
0x01	COMPIFG	The interrupt flags COMPIFG and COMPINVIFG are set either on the rising or falling edge of the comparator output, selected by the IES bit. When IES bit is 0, rising edge of the comparator output sets the COMPIFG and falling edge sets the COMPINVIFG. When IES bit is 1, falling edge of the comparator output sets the COMPIFG and rising edge sets the COMPINVIFG.
0x02	COMPINVIFG	The interrupt flags COMPIFG and COMPINVIFG are set either on the rising or falling edge of the comparator output, selected by the IES bit. When IES bit is 0, rising edge of the comparator output sets the COMPIFG and falling edge sets the COMPINVIFG. When IES bit is 1, falling edge of the comparator output sets the COMPIFG and rising edge sets the COMPINVIFG.
0x03	OUTRDYIFG	Comparator output ready interrupt. This bit is set when the comparator output is valid.

See [Section 7.2.5](#) for guidance on configuring the Event registers.

11.2.11.3 Generic Event Subscribers

The two event subscribers are used to establish the sampling start and stop times when the comparator is used in the [sampled output mode](#).

- FSUB_0 selects the generic event route at the device level on which the start of sampling window event is sent.
- FSUB_1 selects the generic event route at the device level on which the stop of sampling window event is sent.

As the sampling window is driven from generic event routes, any peripheral module with generic publisher capabilities can set the comparator sampling window, including timers and GPIO.

11.3 COMP Registers

Table 11-5 lists the memory-mapped registers for the COMP registers. All register offset addresses not listed in Table 11-5 should be considered as reserved locations and the register contents should not be modified.

Table 11-5. COMP Registers

Offset	Acronym	Register Name	Group	Section
400h	FSUB_0	Subscriber Port 0		Go
404h	FSUB_1	Subscriber Port 1		Go
444h	FPUB_1	Publisher port 1		Go
800h	PWREN	Power enable		Go
804h	RSTCTL	Reset Control		Go
808h	CLKCFG	Peripheral Clock Configuration Register		Go
814h	STAT	Status Register		Go
1020h	IIDX	Interrupt index	CPU_INT	Go
1028h	IMASK	Interrupt mask	CPU_INT	Go
1030h	RIS	Raw interrupt status	CPU_INT	Go
1038h	MIS	Masked interrupt status	CPU_INT	Go
1040h	ISSET	Interrupt set	CPU_INT	Go
1048h	ICLR	Interrupt clear	CPU_INT	Go
1050h	IIDX	Interrupt index	GEN_EVENT	Go
1058h	IMASK	Interrupt mask	GEN_EVENT	Go
1060h	RIS	Raw interrupt status	GEN_EVENT	Go
1068h	MIS	Masked interrupt status	GEN_EVENT	Go
1070h	ISSET	Interrupt set	GEN_EVENT	Go
1078h	ICLR	Interrupt clear	GEN_EVENT	Go
10E0h	EVT_MODE	Event Mode		Go
10FCh	DESC	Module Description		Go
1100h	CTL0	Control 0		Go
1104h	CTL1	Control 1		Go
1108h	CTL2	Control 2		Go
110Ch	CTL3	Control 3		Go
1120h	STAT	Status		Go

Complex bit access types are encoded to fit into small table cells. Table 11-6 shows the codes that are used for access types in this section.

Table 11-6. COMP Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
K	K	Write protected by a key
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value

11.3.1 FSUB_0 (Offset = 400h) [Reset = 00000000h]

FSUB_0 is shown in [Figure 11-7](#) and described in [Table 11-7](#).

Return to the [Summary Table](#).

Subscriber port

Figure 11-7. FSUB_0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-												R/W-0h			

Table 11-7. FSUB_0 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 15.

11.3.2 FSUB_1 (Offset = 404h) [Reset = 00000000h]

FSUB_1 is shown in [Figure 11-8](#) and described in [Table 11-8](#).

Return to the [Summary Table](#).

Subscriber port

Figure 11-8. FSUB_1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-												R/W-0h			

Table 11-8. FSUB_1 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 15.

11.3.3 FPUB_1 (Offset = 444h) [Reset = 00000000h]

FPUB_1 is shown in [Figure 11-9](#) and described in [Table 11-9](#).

Return to the [Summary Table](#).

Publisher port

Figure 11-9. FPUB_1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-0h												R/W-0h			

Table 11-9. FPUB_1 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 15.

11.3.4 PWREN (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 11-10](#) and described in [Table 11-10](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 11-10. PWREN

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-							K-0h

Table 11-10. PWREN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	ENABLE	K	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

11.3.5 RSTCTL (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 11-11](#) and described in [Table 11-11](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 11-11. RSTCTL

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-							
15	14	13	12	11	10	9	8
RESERVED							
W-							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCL R	RESETASSERT
W-						WK-0h	WK-0h

Table 11-11. RSTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	W	0h	
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

11.3.6 CLKCFG (Offset = 808h) [Reset = 00000000h]

CLKCFG is shown in [Figure 11-12](#) and described in [Table 11-12](#).

Return to the [Summary Table](#).

Peripheral Clock Configuration Register

Figure 11-12. CLKCFG

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							BLOCKASYNC
R/W-0h							R/W-0h
7	6	5	4	3	2	1	0
RESERVED							
R/W-0h							

Table 11-12. CLKCFG Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to Allow State Change -- 0xA9 A9h = Key value to be used in writing to this register for the write to take effect.
23-9	RESERVED	R/W	0h	
8	BLOCKASYNC	R/W	0h	Async Clock Request is blocked from starting SYSOSC or forcing bus clock to 32MHz 0h = disable COMP to request SYSOSC 1h = enable COMP to request SYSOSC
7-0	RESERVED	R/W	0h	

11.3.7 STAT (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 11-13](#) and described in [Table 11-13](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 11-13. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED							
R-							

Table 11-13. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0h	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

11.3.8 IIDX (Offset = 1020h) [Reset = 00000000h]

IIDX is shown in [Figure 11-14](#) and described in [Table 11-14](#).

Return to the [Summary Table](#).

Interrupt index register. This read-only register provides the interrupt index of the pending interrupt with the highest priority. It also indicates if no interrupt is pending. The priority order is fixed: lower index equals higher priority. Alternatively to the use of IIDX, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred.

On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flags in the RIS and MIS are cleared as well. After a read from the CPU (not from the debug interface), the register must be updated with the next highest priority interrupt or indicate that no interrupt is pending. Only interrupts which are selected via IMASK are indicated.

Figure 11-14. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED														STAT	
R-0h														R-0h	

Table 11-14. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1-0	STAT	R	0h	Interrupt index status 0h = No pending interrupt 2h = Comparator output interrupt 3h = Comparator output inverted interrupt 4h = Comparator output ready interrupt

11.3.9 IMASK (Offset = 1028h) [Reset = 00000000h]

IMASK is shown in [Figure 11-15](#) and described in [Table 11-15](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.”

Figure 11-15. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED				OUTRDYIFG	COMPINVIFG	COMPIFG	RESERVED
R/W-0h				R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 11-15. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3	OUTRDYIFG	R/W	0h	Masks OUTRDYIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
2	COMPINVIFG	R/W	0h	Masks COMPINVIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
1	COMPIFG	R/W	0h	Masks COMPIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
0	RESERVED	R/W	0h	

11.3.10 RIS (Offset = 1030h) [Reset = 00000000h]

RIS is shown in [Figure 11-16](#) and described in [Table 11-16](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 11-16. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				OUTRDYIFG	COMPINVIFG	COMPIFG	RESERVED
R-0h				R-0h	R-0h	R-0h	R-0h

Table 11-16. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	OUTRDYIFG	R	0h	Raw interrupt status for comparator output ready interrupt flag. This bit is set when the comparator output is valid. 0h = No interrupt pending 1h = Interrupt pending
2	COMPINVIFG	R	0h	Raw interrupt status for comparator output inverted interrupt flag. The IES bit defines the transition of the comparator output setting this bit. 0h = No interrupt pending 1h = Interrupt pending
1	COMPIFG	R	0h	Raw interrupt status for comparator output interrupt flag. The IES bit defines the transition of the comparator output setting this bit. 0h = No interrupt pending 1h = Interrupt pending
0	RESERVED	R	0h	

11.3.11 MIS (Offset = 1038h) [Reset = 00000000h]

MIS is shown in [Figure 11-17](#) and described in [Table 11-17](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 11-17. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				OUTRDYIFG	COMPINVIFG	COMPIFG	RESERVED
R-0h				R-0h	R-0h	R-0h	R-0h

Table 11-17. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	OUTRDYIFG	R	0h	Masked interrupt status for OUTRDYIFG 0h = OUTRDYIFG does not request an interrupt service routine 1h = OUTRDYIFG requests an interrupt service routine
2	COMPINVIFG	R	0h	Masked interrupt status for COMPINVIFG 0h = COMPINVIFG does not request an interrupt service routine 1h = COMPINVIFG requests an interrupt service routine
1	COMPIFG	R	0h	Masked interrupt status for COMPIFG 0h = COMPIFG does not request an interrupt service routine 1h = COMPIFG requests an interrupt service routine
0	RESERVED	R	0h	

11.3.12 ISET (Offset = 1040h) [Reset = 00000000h]

ISET is shown in [Figure 11-18](#) and described in [Table 11-18](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 11-18. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED				OUTRDYIFG	COMPINVIFG	COMPIFG	RESERVED
W-0h				W-0h	W-0h	W-0h	W-0h

Table 11-18. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	W	0h	
3	OUTRDYIFG	W	0h	Sets OUTRDYIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to OUTRDYIFG is set
2	COMPINVIFG	W	0h	Sets COMPINVIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to COMPINVIFG is set
1	COMPIFG	W	0h	Sets COMPIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to COMPIFG is set
0	RESERVED	W	0h	

11.3.13 ICLR (Offset = 1048h) [Reset = 00000000h]

ICLR is shown in [Figure 11-19](#) and described in [Table 11-19](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 11-19. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED				OUTRDYIFG	COMPINVIFG	COMPIFG	RESERVED
W-0h				W-0h	W-0h	W-0h	W-0h

Table 11-19. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	W	0h	
3	OUTRDYIFG	W	0h	Clears OUTRDYIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to OUTRDYIFG is cleared
2	COMPINVIFG	W	0h	Clears COMPINVIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to COMPINVIFG is cleared
1	COMPIFG	W	0h	Clears COMPIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to COMPIFG is cleared
0	RESERVED	W	0h	

11.3.14 IIDX (Offset = 1050h) [Reset = 00000000h]

IIDX is shown in [Figure 11-20](#) and described in [Table 11-20](#).

Return to the [Summary Table](#).

Interrupt index register. This read-only register provides the interrupt index of the pending interrupt with the highest priority. It also indicates if no interrupt is pending. The priority order is fixed: lower index equals higher priority. Alternatively to the use of IIDX, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred.

On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flags in the RIS and MIS are cleared as well. After a read from the CPU (not from the debug interface), the register must be updated with the next highest priority interrupt or indicate that no interrupt is pending. Only interrupts which are selected via IMASK are indicated.

Figure 11-20. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED														STAT	
R-0h														R-0h	

Table 11-20. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1-0	STAT	R	0h	Interrupt index status 0h = No pending interrupt 2h = Comparator output interrupt 3h = Comparator output inverted interrupt 4h = Comparator output ready interrupt

11.3.15 IMASK (Offset = 1058h) [Reset = 00000000h]

IMASK is shown in [Figure 11-21](#) and described in [Table 11-21](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.”

Figure 11-21. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED				OUTRDYIFG	COMPINVIFG	COMPIFG	RESERVED
R/W-0h				R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 11-21. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3	OUTRDYIFG	R/W	0h	Masks OUTRDYIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
2	COMPINVIFG	R/W	0h	Masks COMPINVIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
1	COMPIFG	R/W	0h	Masks COMPIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
0	RESERVED	R/W	0h	

11.3.16 RIS (Offset = 1060h) [Reset = 00000000h]

RIS is shown in [Figure 11-22](#) and described in [Table 11-22](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 11-22. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				OUTRDYIFG	COMPINVIFG	COMPIFG	RESERVED
R-0h				R-0h	R-0h	R-0h	R-0h

Table 11-22. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	OUTRDYIFG	R	0h	Raw interrupt status for comparator output ready interrupt flag. This bit is set when the comparator output is valid. 0h = No interrupt pending 1h = Interrupt pending
2	COMPINVIFG	R	0h	Raw interrupt status for comparator output inverted interrupt flag. The IES bit defines the transition of the comparator output setting this bit. 0h = No interrupt pending 1h = Interrupt pending
1	COMPIFG	R	0h	Raw interrupt status for comparator output interrupt flag. The IES bit defines the transition of the comparator output setting this bit. 0h = No interrupt pending 1h = Interrupt pending
0	RESERVED	R	0h	

11.3.17 MIS (Offset = 1068h) [Reset = 00000000h]

MIS is shown in [Figure 11-23](#) and described in [Table 11-23](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 11-23. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				OUTRDYIFG	COMPINVIFG	COMPIFG	RESERVED
R-0h				R-0h	R-0h	R-0h	R-0h

Table 11-23. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	OUTRDYIFG	R	0h	Masked interrupt status for OUTRDYIFG 0h = OUTRDYIFG does not request an interrupt service routine 1h = OUTRDYIFG requests an interrupt service routine
2	COMPINVIFG	R	0h	Masked interrupt status for COMPINVIFG 0h = COMPINVIFG does not request an interrupt service routine 1h = COMPINVIFG requests an interrupt service routine
1	COMPIFG	R	0h	Masked interrupt status for COMPIFG 0h = COMPIFG does not request an interrupt service routine 1h = COMPIFG requests an interrupt service routine
0	RESERVED	R	0h	

11.3.18 ISET (Offset = 1070h) [Reset = 00000000h]

ISET is shown in [Figure 11-24](#) and described in [Table 11-24](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 11-24. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED				OUTRDYIFG	COMPINVIFG	COMPIFG	RESERVED
W-0h				W-0h	W-0h	W-0h	W-0h

Table 11-24. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	W	0h	
3	OUTRDYIFG	W	0h	Sets OUTRDYIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to OUTRDYIFG is set
2	COMPINVIFG	W	0h	Sets COMPINVIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to COMPINVIFG is set
1	COMPIFG	W	0h	Sets COMPIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to COMPIFG is set
0	RESERVED	W	0h	

11.3.19 ICLR (Offset = 1078h) [Reset = 00000000h]

ICLR is shown in [Figure 11-25](#) and described in [Table 11-25](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 11-25. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED				OUTRDYIFG	COMPINVIFG	COMPIFG	RESERVED
W-0h				W-0h	W-0h	W-0h	W-0h

Table 11-25. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	W	0h	
3	OUTRDYIFG	W	0h	Clears OUTRDYIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to OUTRDYIFG is cleared
2	COMPINVIFG	W	0h	Clears COMPINVIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to COMPINVIFG is cleared
1	COMPIFG	W	0h	Clears COMPIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to COMPIFG is cleared
0	RESERVED	W	0h	

11.3.20 EVT_MODE (Offset = 10E0h) [Reset = 00000009h]

EVT_MODE is shown in [Figure 11-26](#) and described in [Table 11-26](#).

Return to the [Summary Table](#).

Event mode register. It is used to select whether each line is disabled, in software mode (software clears the RIS) or in hardware mode (hardware clears the RIS)

Figure 11-26. EVT_MODE

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED				EVT1_CFG		INT0_CFG	
R/W-0h				R-2h		R-1h	

Table 11-26. EVT_MODE Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-2	EVT1_CFG	R	2h	Event line mode select for event corresponding to GEN_EVENT 0h = The interrupt or event line is disabled. 1h = Event handled by software. Software must clear the associated RIS flag. 2h = Event handled by hardware. The hardware (another module) clears automatically the associated RIS flag.
1-0	INT0_CFG	R	1h	Event line mode select for event corresponding to CPU_INT 0h = The interrupt or event line is disabled. 1h = Event handled by software. Software must clear the associated RIS flag. 2h = Event handled by hardware. The hardware (another module) clears automatically the associated RIS flag.

11.3.21 DESC (Offset = 10FCh) [Reset = 06110000h]

DESC is shown in [Figure 11-27](#) and described in [Table 11-27](#).

Return to the [Summary Table](#).

This register identifies the peripheral and its exact version.

Figure 11-27. DESC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODULEID															
R-611h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FEATUREVER				RESERVED				MAJREV				MINREV			
R-0h				R-				R-0h				R-0h			

Table 11-27. DESC Field Descriptions

Bit	Field	Type	Reset	Description
31-16	MODULEID	R	611h	Module identification contains a unique peripheral identification number. The assignments are maintained in a central database for all of the platform modules to ensure uniqueness.
15-12	FEATUREVER	R	0h	Feature Set for the module *instance*
11-8	RESERVED	R	0h	
7-4	MAJREV	R	0h	Major rev of the IP
3-0	MINREV	R	0h	Minor rev of the IP

11.3.22 CTL0 (Offset = 1100h) [Reset = 00000000h]

CTL0 is shown in [Figure 11-28](#) and described in [Table 11-28](#).

Return to the [Summary Table](#).

Control 0 register.

Figure 11-28. CTL0

31		30		29		28		27		26		25		24	
IMEN		RESERVED													
R/W-0h								R/W-0h							
23		22		21		20		19		18		17		16	
RESERVED										IMSEL					
R/W-0h															
15		14		13		12		11		10		9		8	
IPEN		RESERVED													
R/W-0h								R/W-0h							
7		6		5		4		3		2		1		0	
RESERVED										IPSEL					
R/W-0h															

Table 11-28. CTL0 Field Descriptions

Bit	Field	Type	Reset	Description
31	IMEN	R/W	0h	Channel input enable for the negative terminal of the comparator. 0h = Selected analog input channel for negative terminal is disabled 1h = Selected analog input channel for negative terminal is enabled
30-19	RESERVED	R/W	0h	
18-16	IMSEL	R/W	0h	Channel input selected for the negative terminal of the comparator if IMEN is set to 1. 0h = Channel 0 selected 1h = Channel 1 selected 2h = Channel 2 selected 3h = Channel 3 selected 4h = Channel 4 selected 5h = Channel 5 selected 6h = Channel 6 selected 7h = Channel 7 selected
15	IPEN	R/W	0h	Channel input enable for the positive terminal of the comparator. 0h = Selected analog input channel for positive terminal is disabled 1h = Selected analog input channel for positive terminal is enabled
14-3	RESERVED	R/W	0h	
2-0	IPSEL	R/W	0h	Channel input selected for the positive terminal of the comparator if IPEN is set to 1. 0h = Channel 0 selected 1h = Channel 1 selected 2h = Channel 2 selected 3h = Channel 3 selected 4h = Channel 4 selected 5h = Channel 5 selected 6h = Channel 6 selected 7h = Channel 7 selected

11.3.23 CTL1 (Offset = 1104h) [Reset = 00000000h]

CTL1 is shown in [Figure 11-29](#) and described in [Table 11-29](#).

Return to the [Summary Table](#).

Control 1 register.

Figure 11-29. CTL1

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED			WINCOMPEN	RESERVED	FLTDLY		FLTEN
R/W-0h			R/W-0h	R/W-0h	R/W-0h		R/W-0h
7	6	5	4	3	2	1	0
OUTPOL	HYST		IES	SHORT	EXCH	MODE	ENABLE
R/W-0h	R/W-0h		R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 11-29. CTL1 Field Descriptions

Bit	Field	Type	Reset	Description
31-13	RESERVED	R/W	0h	
12	WINCOMPEN	R/W	0h	This bit enables window comparator operation of comparator. 0h = window comparator is disable 1h = window comparator is enable
11	RESERVED	R/W	0h	
10-9	FLTDLY	R/W	0h	These bits select the comparator output filter delay. See the device-specific data sheet for specific values on comparator propagation delay for different filter delay settings. 0h = Typical filter delay of 70 ns 1h = Typical filter delay of 500 ns 2h = Typical filter delay of 1200 ns 3h = Typical filter delay of 2700 ns
8	FLTEN	R/W	0h	This bit enables the analog filter at comparator output. 0h = Comparator output filter is disabled 1h = Comparator output filter is enabled
7	OUTPOL	R/W	0h	This bit selects the comparator output polarity. 0h = Comparator output is non-inverted 1h = Comparator output is inverted
6-5	HYST	R/W	0h	These bits select the hysteresis setting of the comparator. 0h = No hysteresis 1h = Low hysteresis, typical 10mV 2h = Medium hysteresis, typical 20mV 3h = High hysteresis, typical 30mV
4	IES	R/W	0h	This bit selected the interrupt edge for COMPIFG and COMPINVIFG. 0h = Rising edge sets COMPIFG and falling edge sets COMPINVIFG 1h = Falling edge sets COMPIFG and rising edge sets COMPINVIFG
3	SHORT	R/W	0h	This bit shorts the positive and negative input terminals of the comparator. 0h = Comparator positive and negative input terminals are not shorted 1h = Comparator positive and negative input terminals are shorted

Table 11-29. CTL1 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	EXCH	R/W	0h	This bit exchanges the comparator inputs and inverts the comparator output. 0h = Comparator inputs not exchanged and output not inverted 1h = Comparator inputs exchanged and output inverted
1	MODE	R/W	0h	This bit selects the comparator operating mode. 0h = Comparator is in fast mode 1h = Comparator is in ultra-low power mode
0	ENABLE	R/W	0h	This bit turns on the comparator. When the comparator is turned off it consumes no power. 0h = Comparator is off 1h = Comparator is on

11.3.24 CTL2 (Offset = 1108h) [Reset = 00000000h]

CTL2 is shown in [Figure 11-30](#) and described in [Table 11-30](#).

Return to the [Summary Table](#).

Control 2 register.

Figure 11-30. CTL2

31	30	29	28	27	26	25	24
RESERVED							SAMPMODE
R/W-0h							R/W-0h
23	22	21	20	19	18	17	16
RESERVED						DACSW	DACCTL
R/W-0h						R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
RESERVED					BLANKSRC		
R/W-0h					R/W-0h		
7	6	5	4	3	2	1	0
REFSEL	RESERVED	REFSRC			RESERVED		REFMODE
R/W-0h	R/W-0h	R/W-0h			R/W-0h		R/W-0h

Table 11-30. CTL2 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	R/W	0h	
24	SAMPMODE	R/W	0h	Enable sampled mode of comparator. 0h = Sampled mode disabled 1h = Sampled mode enabled
23-18	RESERVED	R/W	0h	
17	DACSW	R/W	0h	This bit selects between DACCODE0 and DACCODE1 to 8-bit DAC when DACCTL bit is 1. 0h = DACCODE0 selected for 8-bit DAC 1h = DACCODE1 selected for 8-bit DAC
16	DACCTL	R/W	0h	This bit determines if the comparator output or DACSW bit controls the selection between DACCODE0 and DACCODE1. 0h = Comparator output controls selection between DACCODE0 and DACCODE1 1h = DACSW bit controls selection between DACCODE0 and DACCODE1
15-11	RESERVED	R/W	0h	
10-8	BLANKSRC	R/W	0h	These bits select the blanking source for the comparator. 0h = Blanking source disabled 1h = Select Blanking Source 1 2h = Select Blanking Source 2 3h = Select Blanking Source 3 4h = Select Blanking Source 4 5h = Select Blanking Source 5 6h = Select Blanking Source 6

Table 11-30. CTL2 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
7	REFSEL	R/W	0h	This bit selects if the selected reference voltage is applied to positive or negative terminal of the comparator. 0h = If EXCH bit is 0, the selected reference is applied to positive terminal. If EXCH bit is 1, the selected reference is applied to negative terminal. 1h = If EXCH bit is 0, the selected reference is applied to negative terminal. If EXCH bit is 1, the selected reference is applied to positive terminal.
6	RESERVED	R/W	0h	
5-3	REFSRC	R/W	0h	These bits select the reference source for the comparator. 0h = Reference voltage generator is disabled (local reference buffer as well as DAC). 1h = VDDA selected as the reference source to DAC and DAC output applied as reference to comparator. 2h = VREF selected as reference to DAC and DAC output applied as reference to comparator. 3h = In devices where internal VREF is buffered and connected to external VREF pin, VREF applied as reference to comparator. DAC is switched off. 5h = VDDA is used as comparator reference. 6h = Internal reference selected as the reference source to DAC and DAC output applied as reference to comparator. 7h = Internal VREF is used as the source of comparator. Not all devices will have this option.
2-1	RESERVED	R/W	0h	
0	REFMODE	R/W	0h	This bit requests ULP_REF bandgap operation in fast mode(static) or low power mode (sampled). The local reference buffer and 8-bit DAC inside comparator module are also configured accordingly. Fast mode operation offers higher accuracy but consumes higher current. Low power operation consumes lower current but with relaxed reference voltage accuracy. Comparator requests for reference voltage from ULP_REF only when REFLVL > 0. 0h = ULP_REF bandgap, local reference buffer and 8-bit DAC inside comparator operate in static mode. 1h = ULP_REF bandgap, local reference buffer and 8-bit DAC inside comparator operate in sampled mode.

11.3.25 CTL3 (Offset = 110Ch) [Reset = 00000000h]

CTL3 is shown in [Figure 11-31](#) and described in [Table 11-31](#).

Return to the [Summary Table](#).

Control 3 register.

Figure 11-31. CTL3

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED								DACCODE1								RESERVED								DACCODE0							
R/W-0h								R/W-0h								R/W-0h								R/W-0h							

Table 11-31. CTL3 Field Descriptions

Bit	Field	Type	Reset	Description
31-24	RESERVED	R/W	0h	
23-16	DACCODE1	R/W	0h	This is the second 8-bit DAC code. When the DAC code is 0x0 the DAC output will be selected reference voltage x 1/256 V. When the DAC code is 0xFF the DAC output will be selected reference voltage x 255/256 V. 0h = Minimum DAC code value FFh = Minimum DAC code value
15-8	RESERVED	R/W	0h	
7-0	DACCODE0	R/W	0h	This is the first 8-bit DAC code. When the DAC code is 0x0 the DAC output will be selected reference voltage x 1/256 V. When the DAC code is 0xFF the DAC output will be selected reference voltage x 255/256 V. 0h = Minimum DAC code value FFh = Minimum DAC code value

11.3.26 STAT (Offset = 1120h) [Reset = 00000000h]

STAT is shown in [Figure 11-32](#) and described in [Table 11-32](#).

Return to the [Summary Table](#).

Status register.

Figure 11-32. STAT

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED															OUT
R-0h															R-0h

Table 11-32. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	OUT	R	0h	This bit reflects the value of the comparator output. Writing to this bit has no effect on the comparator output. 0h = Comparator output is low 1h = Comparator output is high



The OPA is a zero-drift chopper stabilized operational amplifier with a programmable gain stage. This chapter describes the features and operation of the OPA peripheral.

12.1 OPA Overview	718
12.2 OPA Operation	719
12.3 OA Registers	729

12.1 OPA Overview

The purpose of the OPA peripheral integrated into the MSPM0xx MCU platform is to provide a way to buffer or amplify analog signals in a system without the need of discrete op-amps.

The OPA is based off of a high-performance factory trimmed amplifier core accompanied with a programmable gain stage feedback loop, configurable input muxes, and a burnout current source used to monitor sensor health.

OPA features include:

- Software selectable zero-drift chopper stabilization
- Factory trimming to remove offset error
- Rail-to-rail input & output
- Operation supported in full-scale supply voltage range
- Dual power modes for added flexibility
- Programmable gain amplifier (PGA) up to 32x
- Configurable amplifier modes include General Purpose Mode, Buffer Mode, Inverting/Non-inverting PGA Mode, Difference Amplifier Mode and Cascade Amplifier Mode
- Rich selection of external/internal op-amp terminal input options
- Dual-differential input pairs to connect multiple sensors to a single OPA
- Burnout current source (BCS) integrated to monitor sensor health
- Operates in RUN, SLEEP, and STOP modes

Note

Most devices integrate more than one OPA peripheral. If more than one OPA is present on a device, they all operate identically. Throughout this chapter, nomenclature appears such as OPAX and OPA[x-1] to describe different instances of the OPA peripheral.

Figure 12-1 shows the functional block diagram of the OPA peripheral.

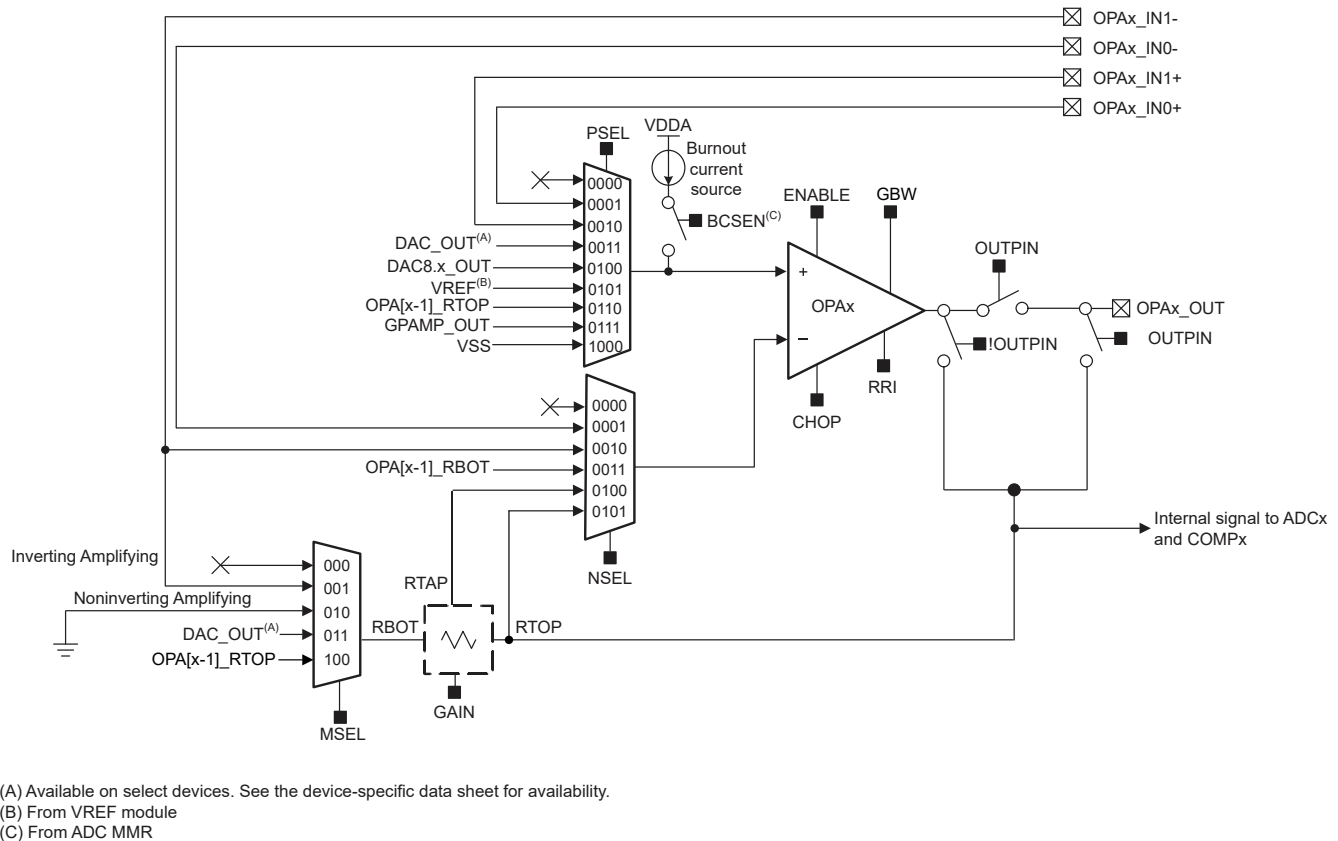


Figure 12-1. OPA Block Diagram

12.2 OPA Operation

The OPA is configured with application software. The following sections describe the setup and operation of the OPA peripheral.

12.2.1 Analog Core

The OPA integrates a high-performance chopper stabilized low-power rail-to-rail input/output operational amplifier. It can be reconfigured in real-time to meet the performance requirements of a specific application. The following table depicts all of the op-amp analog core features and parameters that are configurable:

Table 12-1. OPA Configurable Parameters

Parameter	Configuration Options
Rail-to-Rail Input (RRI)	Enabled / Disabled
Gain Bandwidth Product (GBW)	6MHz (STD Mode) / 1MHz (LP Mode)
Chopping (CHOP)	Disabled/Standard Chop/ADC-assisted Chop

In some basic signal conditioning applications, rail-to-rail input is not required and thus can be disabled to reduce overall power consumption. Rail-to-rail input can be disabled by clearing the RRI bit field in the PMUOPAMP register. The same concept applies to the gain bandwidth product for general purpose applications which require <1MHz GBW. The OPA can be configured to the low power operating mode (LP mode) by clearing the GBW bit in the CFGBASE register.

In some analog sensing applications, high accuracy and low offset drift is a requirement. For these use-cases, chopping can be enabled to significantly improve offset voltage and offset drift performance. Please refer to [Section 12.2.6](#) for more details on how to properly configure and leverage the chopping feature of the OPA.

12.2.2 Power Up Behavior

The ENABLE bit in the PWREN register connects the OPA peripheral to the bus and the clock system. The ENABLE bit in the CTL register activates the OPA analog and digital core. There is a ready bit (RDY) in the STAT register which is set when the OPA is ready for use. Please see the device-specific data sheet for the enable time of the OPA peripheral.

Note

PWREN.ENABLE must be set before writing to CTL.ENABLE or any other OPA memory mapped registers (MMRs).

12.2.3 Inputs

The OPA uses input muxes P-MUX and N-MUX to provide a selection of input channels for the non-inverting and inverting terminals of the amplifier. These inputs are configurable using the PSEL and NSEL control bits.

A generalized depiction of the input channel mapping is included in [Table 12-2](#) below. Please refer to the device-specific data sheet for a more detailed depiction of the available OPAX inputs.

Table 12-2. General OPAX Input Channels

PSEL	OPAx Non-inverting Channels	NSEL	OPAx Inverting Input Channels
0x0	Open	0x0	Open
0x1	OPAx_IN0+	0x1	OPAx_IN0-
0x2	OPAx_IN1+	0x2	OPAx_IN1-
0x3	DAC_OUT	0x3	OPA[x-1]_RBOT
0x4	DAC8.x_OUT	0x4	RTAP
0x5	VREF	0x5	RTOP
0x6	OPA[x-1]_RTOP		
0x7	GPAMP Output		
0x8	VSS		

Note

OPAx and OPA[x-1] refer to instances where there are 2 OPA peripherals in a device. In these cases, the peripherals are paired and the input/outputs are shared as shown in [Table 12-2](#). DAC8.x refers to the paired COMP reference DAC which can be routed to the non-inverting input of OPAX.

12.2.4 Output

The OPA output is connected to a network of complementary switches to allow the amplifier output to be a purely internal signal or allow the signal to be accessed through a device pin. If OUTPIN = 0, the output is internally connected to the programmable gain stage and capable of being routed to other analog peripherals. If OUTPIN = 1, the output goes to the device pin while still maintaining connection to the programmable gain stage and other analog peripherals. Having access to the amplifier output at the device pin allows for external filtering circuitry.

12.2.5 Clock Requirements

The OPA requires SYSOSC to be active for proper operation to support features such as chopping and rail-to-rail input. There are no provisions in place for the OPA to automatically request the SYSOSC from SYSCTL so the user must ensure that SYSOSC is enabled to use the OPA. The minimum requirements for the input clock depend on the OPA configuration as shown in the table below.

Table 12-3.

RRI	Supported SYSOSC Frequencies
0	4/16/24/32 MHz
1	32 MHz

Note

Application software needs to ensure that SYSOSC is programmed to an appropriate frequency for rail-to-rail input.

12.2.6 Chopping

The OPA peripheral implements chopper stabilization to reduce offset, drift, and 1/f noise. CFG.CHOP can be set to 0x1 for standard chopping mode which requires an external filter. Note that the OPA automatically scales the chopping frequency based on CFG.GAIN which is used to select the OPA PGA gain but is also needed with external filter when the standard chopping is used as shown in [Table 12-4](#).

Table 12-4. Standard Chopping Frequency for OPA

GAIN	Inverting / Non-inverting Gain	Chopping Frequency (Hz)	Resistor(kΩ)	Capacitor(nF)
0x0	1x	125k	1	10
0x1	-1x / 2x	62.5k	1	30
0x2	-3x / 4x	31.25k	1	90
0x3	-7x / 8x	15.6k	1	270
0x4	-15x / 16x	7.8k	1	810
0x5	-31x / 32x	3.9k	1	2400

12.2.7 OPA Amplifier Modes

The OPA uses an input mux, M-MUX, connected to the programmable gain stage resistor ladder to allow the user to program the OPA for various analog signal chain configurations. These configurations include unity gain buffer, inverting/non-inverting amplification, PGA, cascade amplification, and more.

A generalized depiction of the input channel mapping for M-MUX is included in [Table 12-5](#) below. Please refer to the device-specific data sheet for a more detailed depiction of the available M-MUX inputs.

Table 12-5. General OPA M-MUX Channel Mapping

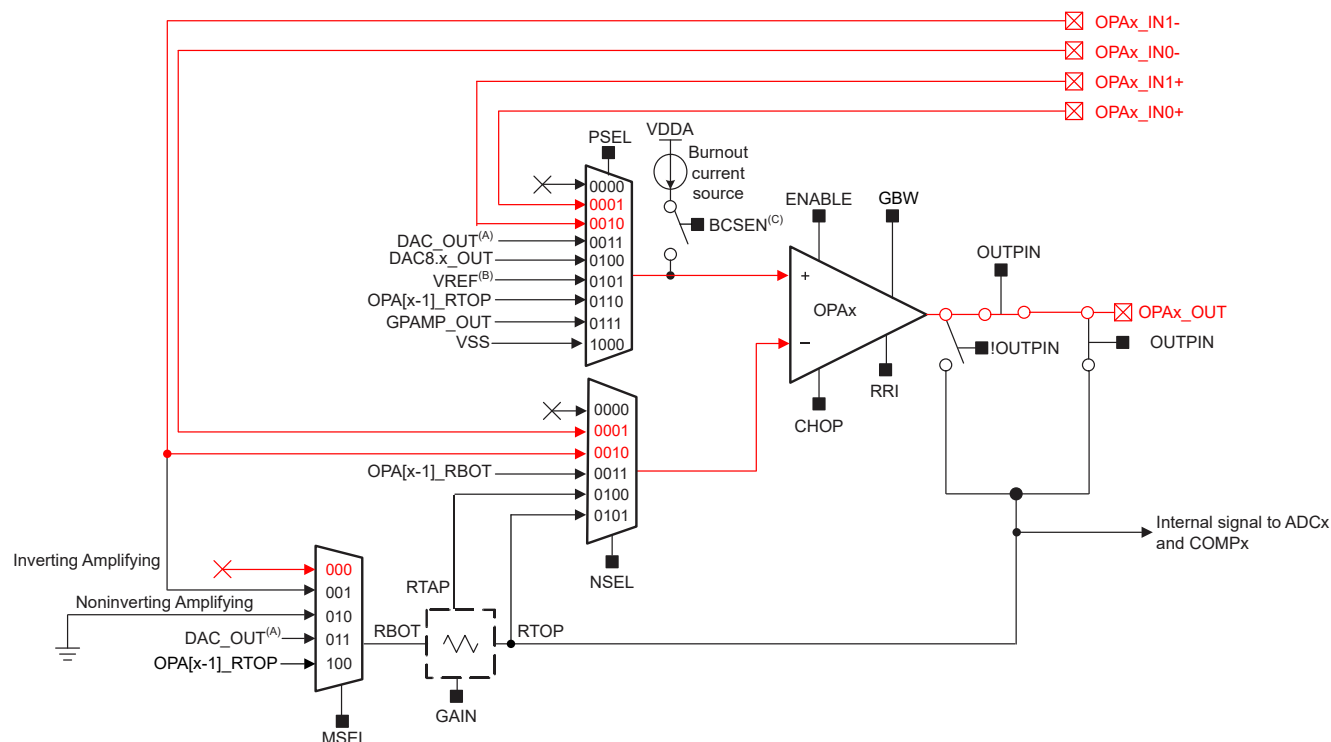
MSEL	OPAx M-MUX Input Channels
0x0	Open
0x1	OPAx_IN1-
0x2	GND
0x3	DAC_OUT
0x4	OPA[x-1]_RTOP

The following sections provide examples of how P-MUX, N-MUX, and M-MUX can be configured for various amplifier modes.

12.2.7.1 General-Purpose Mode

The OPA internal muxes support GP mode which allows all input and output terminals to be accessed from device pins. Using this mode with external components enables the amplifier to realize any analog signal chain circuitry that a discrete op-amp can. Dual differential inputs IN0+/- and IN1+/- allow for two different input networks to be connected to the same OPA peripheral.

[Figure 12-2](#) shows the block diagram of the OPA in GP mode.



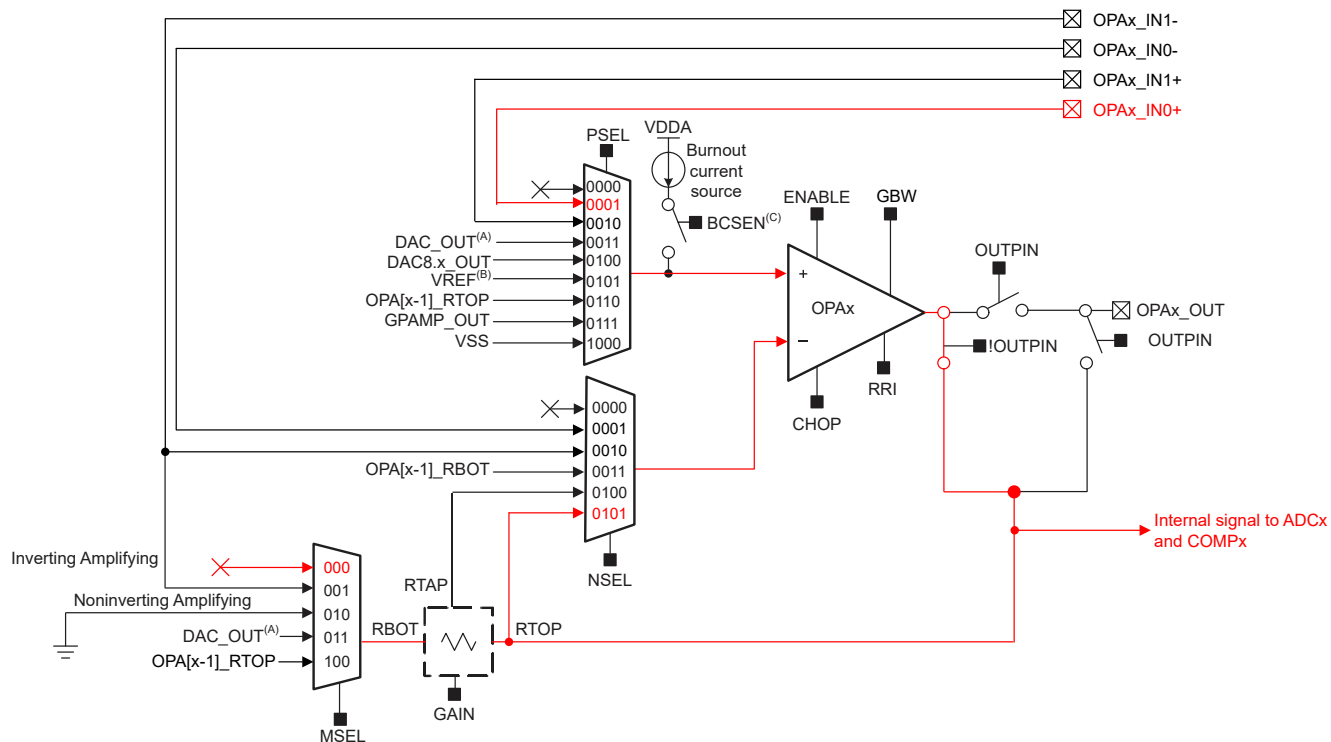
(A) Available on select devices. See the device-specific data sheet for availability.
(B) From VREF module
(C) From ADC MMR

Figure 12-2. OPA in General Purpose (GP) Mode

12.2.7.2 Buffer Mode

The internal feedback loop can be programmed to a unity gain configuration to support OPA buffer mode. The non-inverting input can be sourced from a variety of signals such as the external OPA_x_IN_x or the on-board DAC and VREF peripherals. The OPA can output to an external pin by setting OUTPIN = 0x1 or be internally routed to on-board analog peripherals such as the ADC, COMP, and/or paired OPA by setting OUTPIN = 0x0.

Figure 12-3 shows the block diagram of the OPA in buffer mode with external OPA_x_IN0+ as the noninverting input and the output internally routed to the ADC.



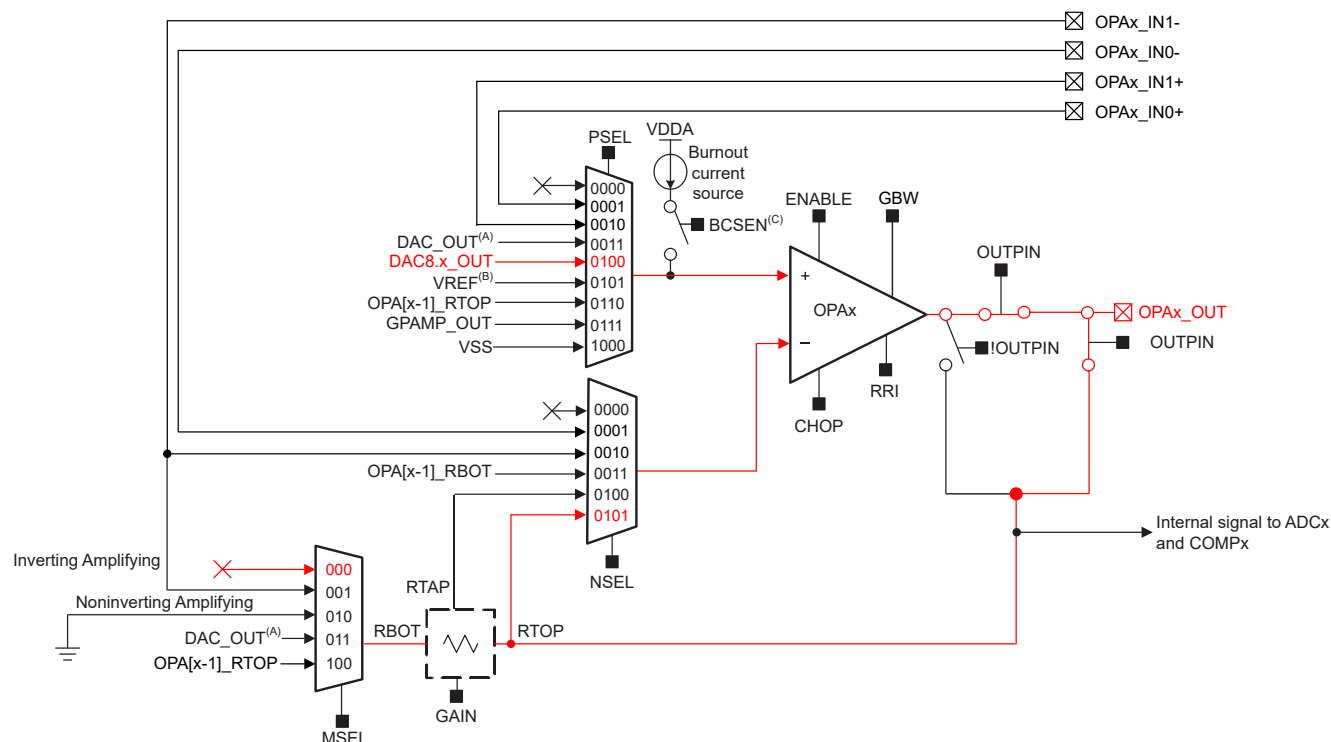
(A) Available on select devices. See the device-specific data sheet for availability.

(B) From VREF module

(C) From ADC MMR

Figure 12-3. OPA in ADC Buffer Configuration

Figure 12-4 shows the block diagram of the OPA in buffer mode with DAC8.x_OUT as the noninverting input and the output routed out to a device pin. This mode enables the user to use the OPA peripheral as an output buffer for the comparator onboard reference DAC.



(A) Available on select devices. See the device-specific data sheet for availability.

(B) From VREF module

(C) From ADC MMR

Figure 12-4. OPA in DAC Output Buffer Configuration

12.2.7.3 OPA PGA Mode

The OPA integrates a programmable gain stage in the feedback loop to configure the OPA as a PGA (programmable gain amplifier). The gain stage is a feedback resistance ladder and it supports up to 32x amplification. Table 12-6 lists the OPA PGA mode gain settings.

Table 12-6. PGA Mode and Gain Settings

PGA Mode Configuration	GAIN	Gain Value from PGA (Internal Gain)
Inverting PGA Mode	0x0	Not Valid
	0x1	-1x
	0x2	-3x
	0x3	-7x
	0x4	-15x
	0x5	-31x
Non-inverting PGA Mode	0x0	Not valid
	0x1	2x
	0x2	4x
	0x3	8x
	0x4	16x
	0x5	32x

The programmable gain stage works with P-MUX, N-MUX, and M-MUX to support various internal amplifier configurations which are detailed in the following sections.

12.2.7.3.1 Inverting PGA Mode

In this mode, the inverting input is sourced from the external OPAX_IN1- crossing the resistor ladder of the PGA. The MSEL bits select OPAX_IN1- as the source (MSEL = 0x1) and the NSEL bits must select the RTAP as the inverting input for the amplifier (NSEL = 0x4). Because the OPA is a single supply amplifier, the non-inverting input needs to be biased up to ensure that the output stays within the linear range. The non-inverting input can be sourced from the external OPAX_INx+ input or from a DAC input.

Figure 12-5 shows an example of the OPA inverting PGA mode with the 8-bit reference DAC as bias cancellation.

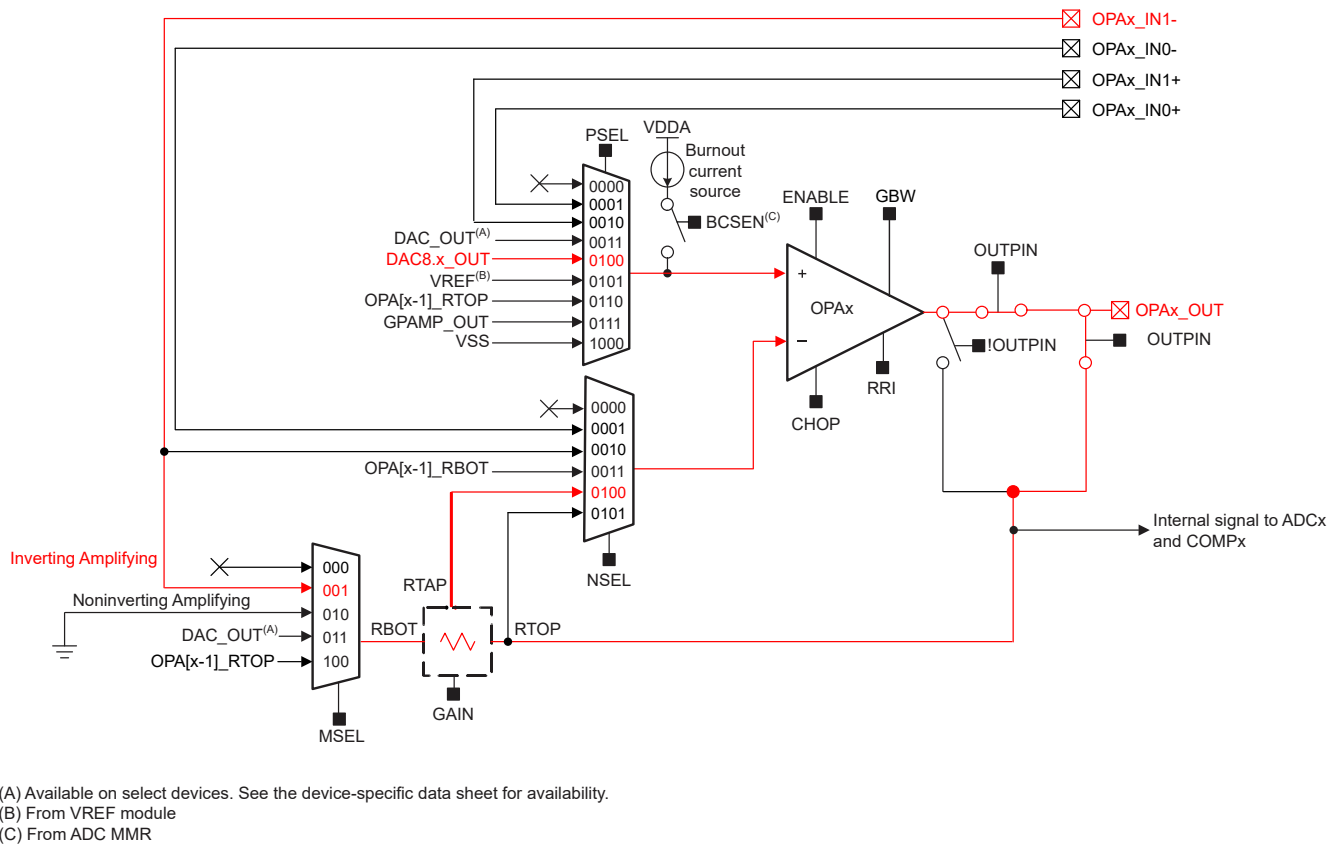
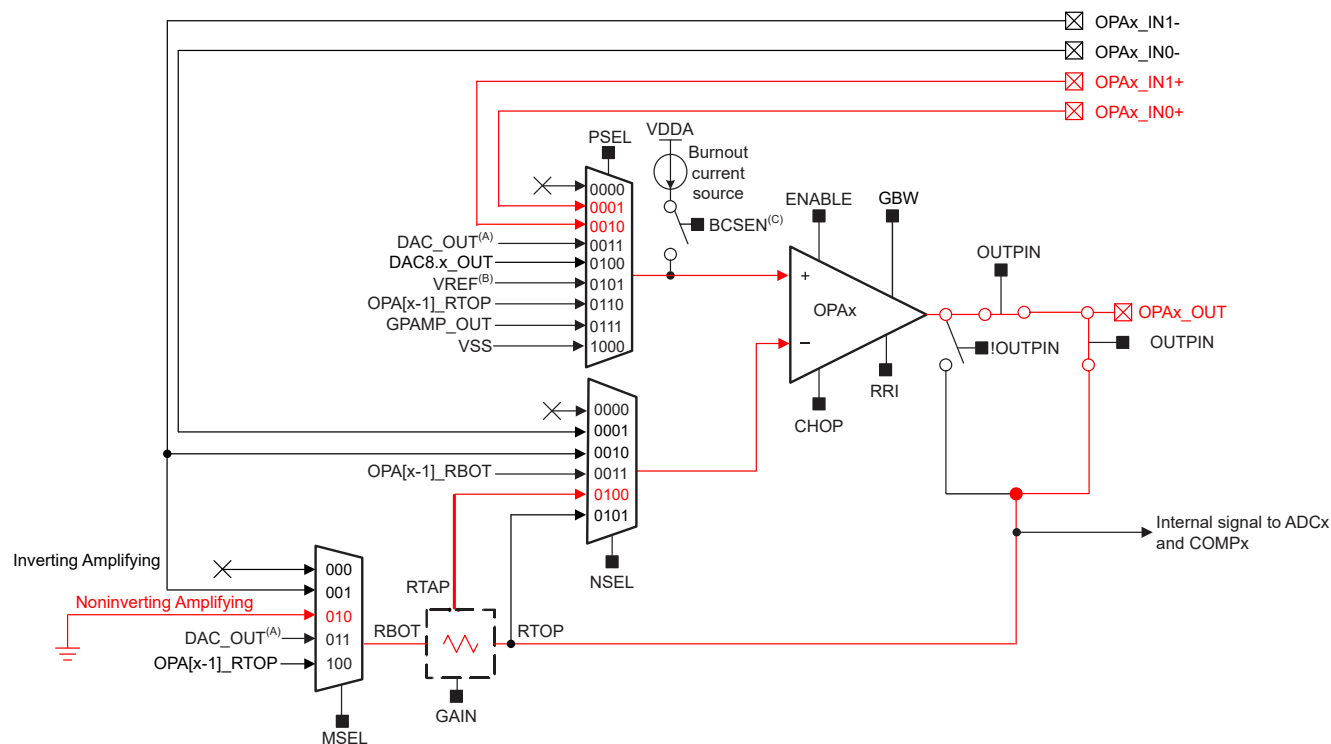


Figure 12-5. OPA in Inverting PGA Configuration

12.2.7.3.2 Non-inverting PGA Mode

In this mode, the non-inverting input is sourced from the external OPAX_INx+ input (PSEL = 0x1 or 0x2). The NSEL bits must select RTAP as the inverting input for the amplifier (NSEL = 0x4) and from the MSEL bits a bias is chosen from either ground or the 12-bit buffered DAC (DAC_OUT).

Figure 12-6 shows an example of the OPA in non-inverting PGA mode.



(A) Available on select devices. See the device-specific data sheet for availability.
(B) From VREF module
(C) From ADC MMR

Figure 12-6. OPA in Non-inverting PGA Configuration

12.2.7.4 Difference Amplifier Mode

When two or more OPAs are available on a device, two can be combined to form a difference amplifier. The output equation for the difference amplifier is given by the V_{diff} equation in Figure 12-7. The internally connected configuration for two OPAs with the output being sampled by the ADC is shown in Figure 12-8.

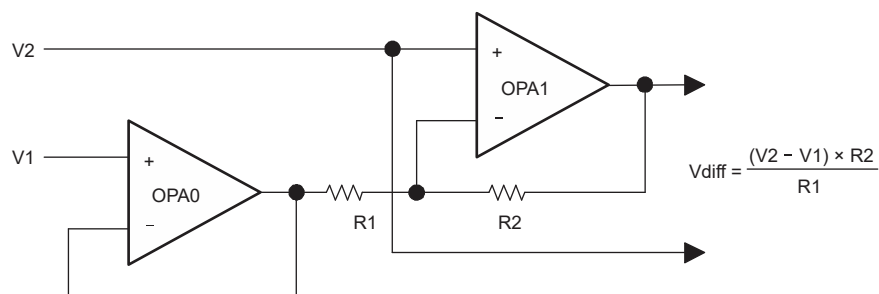


Figure 12-7. Two OPA Difference Amplifier Block Diagram and Equation

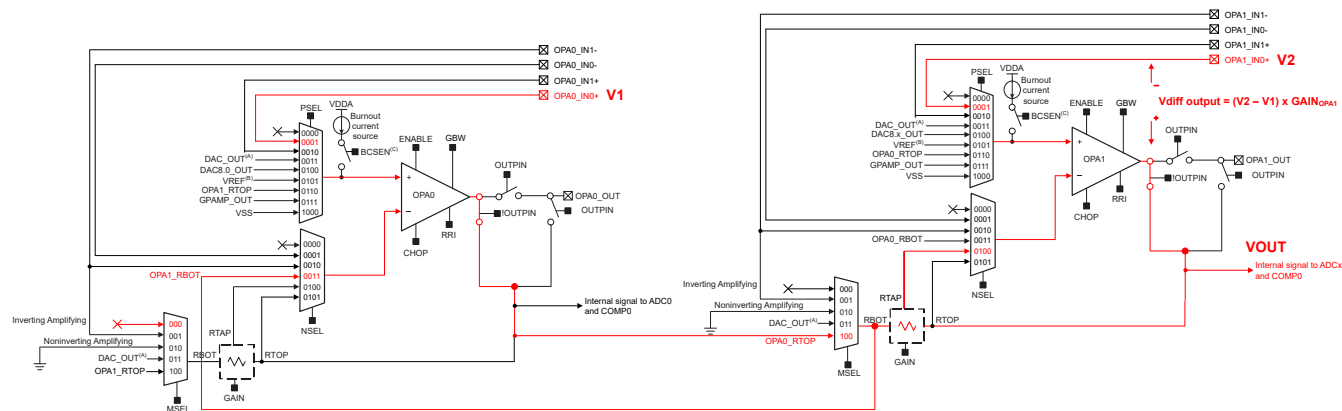


Figure 12-8. Two OPAs in Difference Amplifier Configuration

12.2.7.5 Cascade Amplifier Mode

When two or more OPAs are available on a device, two can be combined to form a multistage or cascaded amplifier. Using the programmable input muxes, all combinations of inverting and non-inverting multistage amplifiers can be implemented. The output equation for the non-inverting to non-inverting cascaded amplifier is given by the V_{out} equation in Figure 12-9. The internally connected configuration for two OPAs is shown in Figure 12-10.

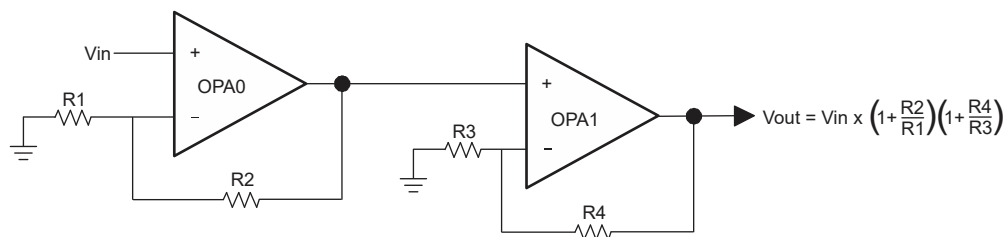


Figure 12-9. Two OPA Non-inverting to Non-inverting Cascade Amplifier Block Diagram and Equation

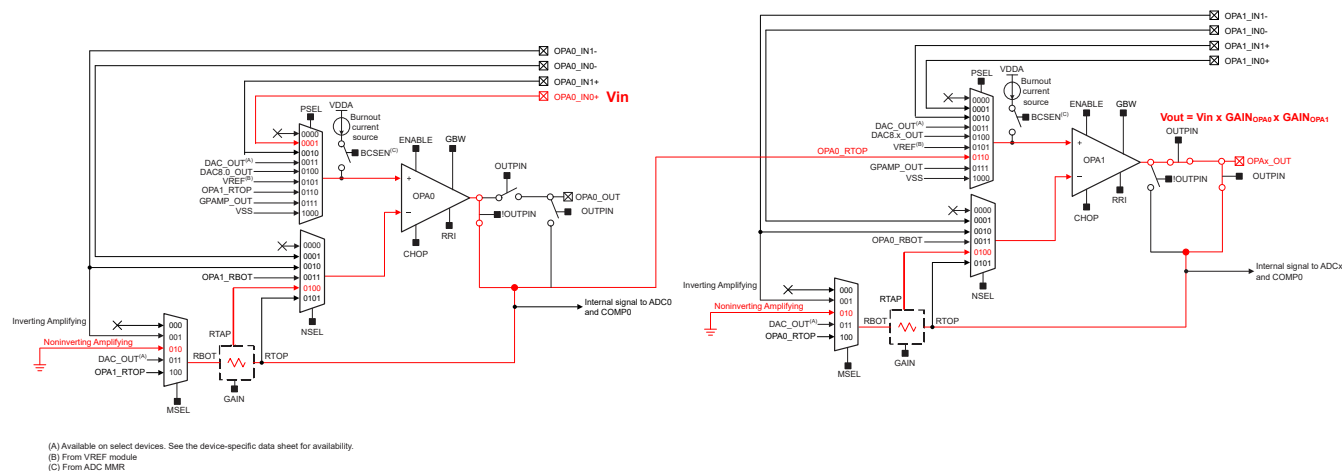


Figure 12-10. Two OPAs in Non-inverting to Non-inverting Cascade Amplifier Configuration

12.2.8 OPA Configuration Selection

The OPA peripheral allows customers to dynamically change the OPA gain without disabling OPA functionality by using the register CFG.GAIN control bits. When using the control bits CFG.GAIN to select between different

gain values, the user must ensure that the additional CFG control bits remain unchanged, and that the CFG.GAIN control bits do not get cleared to 0x0.

To ensure proper operation of the OPA, the configuration settings except the CFG.GAIN bits governed by the CFG register can only be changed when the OPA is disabled. This means that in order to change OPA inputs or mode within an application, a user must perform the following:

- Clear CTL.ENABLE bit
- Update CFG.MSEL or CFG.NSEL or CFG.PSEL bits
- Set CTL.ENABLE bit

Note

Applications must take into account OPA enable time when performing this procedure. Please see OPA Specification within the device-specific data sheet for details.

Note

This procedure must also be followed if changing the CFG.GAIN bits to or from a value of 0x0.

12.2.9 Burnout Current Source

The OPA has a burnout current source (BCS) that can be used when requested by the ADC. The ADC will provide a burnout current source enable signal to the OPA peripheral when it is selected as the ADC input channel and the BCSEN bit is set in the ADC MEMCTL register. When enabled, the burnout current source will inject current into the positive input of the OPA. The ADC should then be used to measure the voltage across the sensor or resistive Wheatstone bridge with the input signal going through the OPA configured in buffer mode (or whichever OPA configuration is normally used). An ADC reading with the burnout current source enabled can then detect if the sensor is open (that is, a broken wire resulting in full range signal) or shorted (0 V). The user can periodically enable the burnout current source to check if the sensor is operating properly. It should be turned off during measurements.

12.3 OA Registers

Table 12-7 lists the memory-mapped registers for the OA registers. All register offset addresses not listed in Table 12-7 should be considered as reserved locations and the register contents should not be modified.

Table 12-7. OA Registers

Offset	Acronym	Register Name	Group	Section
800h	PWREN	Power enable		Go
804h	RSTCTL	Reset Control		Go
814h	STAT	Status Register		Go
1010h	CLKOVR	Clock Override		Go
101Ch	PWRCTL	Power Control		Go
1100h	CTL	Control Register		Go
1108h	CFG	Configuration Register		Go
1118h	STAT	Status Register		Go

Complex bit access types are encoded to fit into small table cells. Table 12-8 shows the codes that are used for access types in this section.

Table 12-8. OA Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value

12.3.1 PWREN (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 12-11](#) and described in [Table 12-9](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 12-11. PWREN

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-0h							R/WK-0h

Table 12-9. PWREN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	ENABLE	R/WK	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

12.3.2 RSTCTL (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 12-12](#) and described in [Table 12-10](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 12-12. RSTCTL

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCL R	RESETASSERT
W-0h						WK-0h	WK-0h

Table 12-10. RSTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	W	0h	
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

12.3.3 STAT (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 12-13](#) and described in [Table 12-11](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 12-13. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED							
R-							

Table 12-11. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0h	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

12.3.4 CLKOVR (Offset = 1010h) [Reset = 00000000h]

CLKOVR is shown in [Figure 12-14](#) and described in [Table 12-12](#).

Return to the [Summary Table](#).

This register overrides the functional clock request by this peripheral to the system

Figure 12-14. CLKOVR

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED						RUN_STOP	OVERRIDE
R/W-0h						R/W-0h	R/W-0h

Table 12-12. CLKOVR Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R/W	0h	
1	RUN_STOP	R/W	0h	If OVERRIDE is enabled, this register is used to manually control the peripheral's clock request to the system 0h = Run/ungate functional clock 1h = Stop/gate functional clock
0	OVERRIDE	R/W	0h	Unlocks the functionality of RUN_STOP to override the automatic peripheral clock request 0h = Override disabled 1h = Override enabled

12.3.5 PWRCTL (Offset = 101Ch) [Reset = 00000001h]

PWRCTL is shown in [Figure 12-15](#) and described in [Table 12-13](#).

Return to the [Summary Table](#).

This register controls if a peripheral is disabled if it is in idle state.

Figure 12-15. PWRCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED							AUTO_OFF
R/W-							R/W-1h

Table 12-13. PWRCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R/W	0h	
0	AUTO_OFF	R/W	1h	When set the peripheral will remove its local IP “request for enable” so that it can be disabled if no other entities in the system are requesting it to be enabled. 0h = Disable automatic power off function 1h = Enable automatic power off function

12.3.6 CTL (Offset = 1100h) [Reset = X]

CTL is shown in [Figure 12-16](#) and described in [Table 12-14](#).

Return to the [Summary Table](#).

Control Register

Figure 12-16. CTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-0h							R/W-0h

Table 12-14. CTL Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R/W	0h	
0	ENABLE	R/W	0h	OAxn Enable. 0h (R/W) = OAxn OFF 1h (R/W) = OAxn ON

12.3.7 CFG (Offset = 1108h) [Reset = 00000000h]

CFG is shown in [Figure 12-17](#) and described in [Table 12-15](#).

Return to the [Summary Table](#).

Configuration Register

Figure 12-17. CFG

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
GAIN			MSEL			NSEL	
R/W-0h			R/W-0h			R/W-0h	
7	6	5	4	3	2	1	0
NSEL	PSEL			OUTPIN		CHOP	
R/W-0h	R/W-0h			R/W-0h		R/W-0h	

Table 12-15. CFG Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15-13	GAIN	R/W	0h	Gain setting. Refer to TRM for enumeration information. 0h = Minimum gain value 7h = Maximum gain value.
12-10	MSEL	R/W	0h	MSEL Mux selection. Please refer to the device specific data sheet for exact channels available. 0h (R/W) = no connect 1h (R/W) = external pin OAn-1 2h (R/W) = VSS 3h (R/W) = DAC12 Output 4h (R/W) = OA[n-1]Rtop
9-7	NSEL	R/W	0h	Negative OA input selection. Please refer to the device specific data sheet for exact channels available. 0h (R/W) = no connect 1h (R/W) = external pin OAn-0 2h (R/W) = external pin OAn-1 3h (R/W) = OA[n+1]Rbot 4h (R/W) = OA[n]Rtap 5h (R/W) = OA[n]Rtop 6h (R/W) = Spare input
6-3	PSEL	R/W	0h	Positive OA input selection. Please refer to the device specific data sheet for exact channels available. 0h (R/W) = No connect 1h (R/W) = external pin OA+0 2h (R/W) = external pin OAn+1 3h (R/W) = DAC12OUT 4h (R/W) = DAC8OUT 5h (R/W) = VREF Channel 6h (R/W) = OA[n-1]Rtop 7h (R/W) = GPAMP_OUT_INT Input 8h = Internal Ground Connection

Table 12-15. CFG Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	OUTPIN	R/W	0h	Enable output pin 0h (R/W) = Output pin disabled 1h (R/W) = Output pin enabled
1-0	CHOP	R/W	0h	Chopping enable. 0h (R/W) = Chopping disable. 1h (R/W) = Standard chopping enable. 2h (R/W) = Chop with post average on. Requires output to be connect to ADC in average mode.

12.3.8 STAT (Offset = 1118h) [Reset = X]

STAT is shown in [Figure 12-18](#) and described in [Table 12-16](#).

Return to the [Summary Table](#).

Status Register

Figure 12-18. STAT

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED															RDY
R-0h															R-0h

Table 12-16. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	RDY	R	0h	OA ready status. 0h (R) = OAxn is not ready. 1h (R) = OAxn is ready.



The GPAMP is a chopper stabilized general purpose operational amplifier with rail-to-rail input and output. This chapter describes the features and operation of the GPAMP peripheral.

13.1 GPAMP Overview.....	740
13.2 GPAMP Operation.....	740
13.3 GPAMP Registers.....	743

13.1 GPAMP Overview

The purpose of the GPAMP peripheral integrated into the MSPM0xx MCU platform is to provide a way to buffer or amplify analog signals in a system without the need of discrete op-amps.

The GPAMP is based off of a factory trimmed amplifier core accompanied with a configurable inverting input mux which allows for various analog signal chain configurations.

GPAMP features include:

- Software selectable chopper stabilization
- Rail-to-rail input & output
- Operation supported in full-scale supply voltage range
- Programmable internal unity gain feedback loop
- Operates in RUN, SLEEP, and STOP modes

Figure 13-1 show the functional block diagram of the GPAMP peripheral.

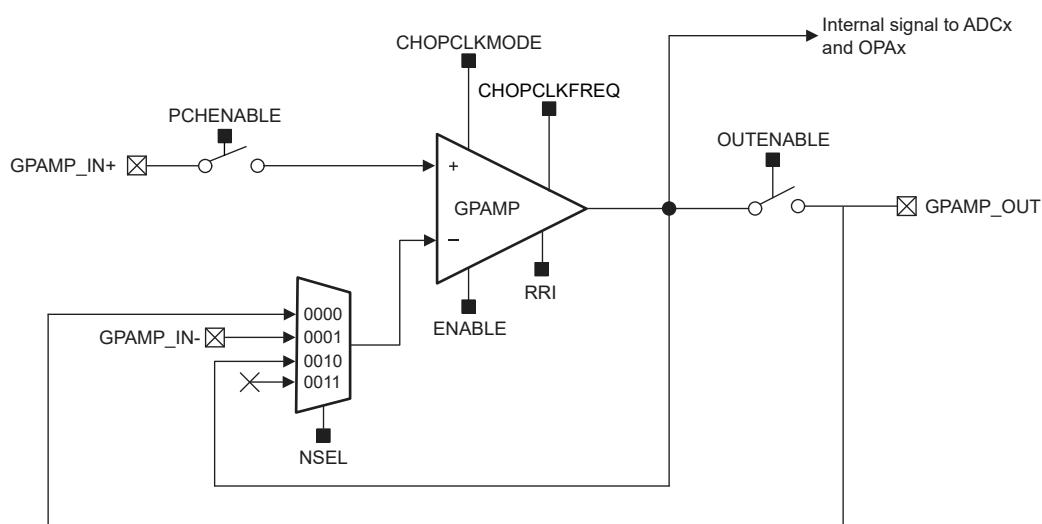


Figure 13-1. GPAMP Block Diagram

13.2 GPAMP Operation

The GPAMP peripheral is configured with user software using the PMUOPAMP register which is located in the PMCU region of memory mapped registers. For bit field definitions, you can refer to the end of this chapter or the register section of [PMCU](#). The setup and operation of GPAMP is discussed in the following sections.

13.2.1 Analog Core

The GPAMP integrates a chopper stabilized low-power rail-to-rail input/output operational amplifier. It can be reconfigured in real-time to meet the performance requirements of various applications. The following table depicts all of the amplifier analog core features and parameters that are configurable:

Table 13-1. GPAMP Configurable Parameters

Parameter	Configuration Options
Rail-to-Rail Input (RRI)	Mode 0 / Mode 1 / Mode 2
Chopping (CHOPCLKMODE)	Standard Chop / Disabled

In some basic signal conditioning applications, rail-to-rail input is not required and thus can be disabled to reduce overall power consumption. Rail-to-rail input can be disabled by programming the RRI bit field to 0x0 (Mode 0). Refer to the GPAMP specifications in the device specific data sheet for supported common-mode input ranges in the various RRI modes.

In some analog sensing applications, high accuracy and low offset drift is a requirement. For these use-cases, chopping can be enabled to significantly improve offset voltage and offset drift performance. Please refer to [Section 13.2.6](#) for more details on how to properly configure and leverage the chopping feature of the GPAMP.

13.2.2 Power Up Behavior

The ENABLE bit in the PMUOPAMP register activates the GPAMP analog and digital core. The GPAMP requires the VBOOST circuit in the PMU to be stabilized before use. Refer to [Section 2.2.6](#) for more details on what the VBOOST circuit is and why it is needed for proper GPAMP operation. Please see the device-specific data sheet for the enable time of the GPAMP peripheral.

13.2.3 Inputs

The GPAMP uses an input mux, N-MUX, to provide a selection of input channels for the inverting terminal of the amplifier. These inputs are configurable using the NSEL control bits. The non-inverting terminal of the amplifier is enabled using the PCHENABLE control bit and allows the input to be driven from the GPAMP_IN+ signal on a device pin.

A depiction of the input channel mapping is included in [Table 13-2](#) below.

Table 13-2. GPAMP Input Channels

PCHENABLE	GPAMP Non-inverting Input	NSEL	GPAMP Inverting Input Channels
0x0	Open	0x0	GPAMP output pad (GPAMP_OUT)
0x1	GPAMP_IN+	0x1	GPAMP_IN-
		0x2	GPAMP internal amplifier output
		0x3	Open

13.2.4 Output

The GPAMP output is connected to a switch to allow the amplifier output to be a purely internal signal or allow the signal to be accessed through a device pin. If OUTENABLE = 1, the amplifier output goes to the device pin while still maintaining connection to the inverting input mux and other analog peripherals. If OUTENABLE = 0, the amplifier output is disconnected from the device pin but is still internally connected to the inverting input mux and other analog peripherals on-board. Having access to the amplifier output at the device pin allows for external filtering circuitry. Refer to the [GPAMP Block Diagram](#) for a detailed schematic of the GPAMP output circuitry.

13.2.5 GPAMP Amplifier Modes

The GPAMP uses N-MUX and an internal unity gain feedback loop to allow the user to program the GPAMP for various analog signal chain configurations. These configurations include general purpose mode, ADC buffer, and unity gain mode.

The following sections provide examples of how GPAMP can be configured for these various amplifier modes.

13.2.5.1 General-Purpose Mode

The GPAMP supports general-purpose (GP) mode which allows all input and output terminals to be accessed from device pins. To realize this GP mode NSEL, PCHENABLE, and OUTENABLE must all be set to 0x1. Using this mode requires external components and an appropriate feedback loop to build the circuit desired.

[Figure 13-2](#) shows the block diagram of the GPAMP in GP mode.

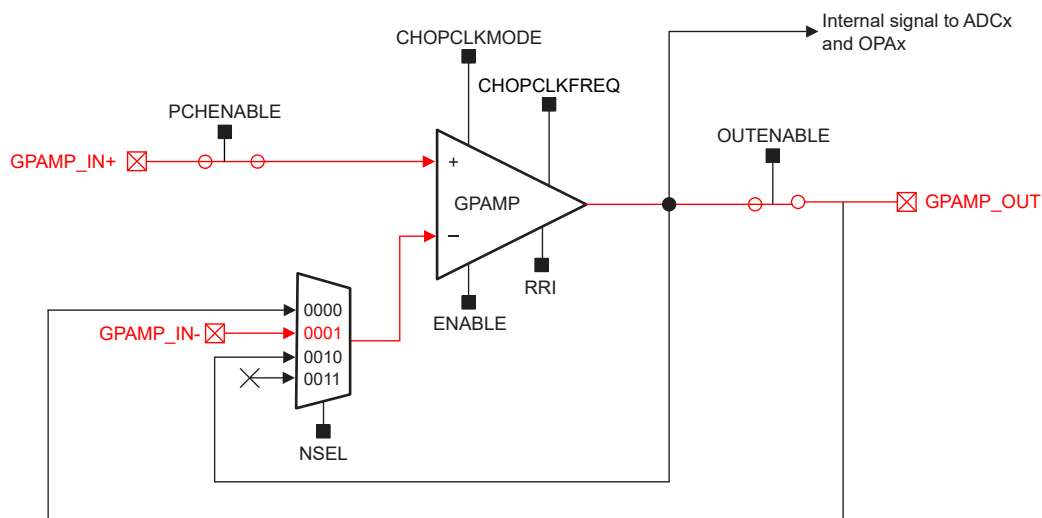


Figure 13-2. GPAMP in General Purpose (GP) Mode

13.2.5.2 ADC Buffer Mode

The GPAMP can be used to internally buffer an analog signal into an on-board ADC. To realize this ADC buffer mode, NSEL must be set to 0x2, PCHENABLE must be set to 0x1, and OUTENABLE must be set to 0x0.

Figure 13-3 shows the block diagram of the GPAMP in ADC buffer mode with GPAMP_IN+ as the noninverting input and the output internally routed to the ADC.

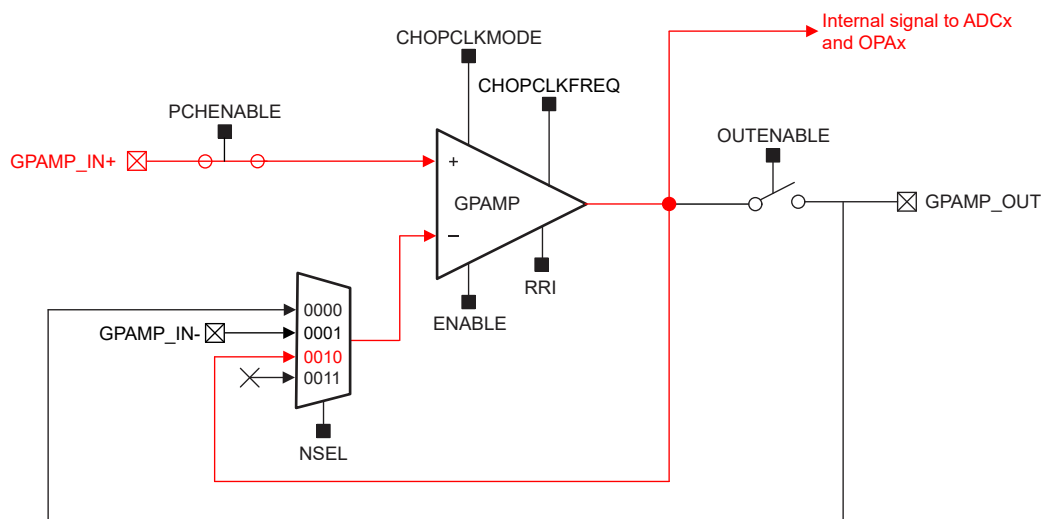


Figure 13-3. GPAMP in ADC Buffer Configuration

13.2.5.3 Unity Gain Mode

The internal feedback loop of the GPAMP can be used to implement a unity gain amplifier which can buffer an analog signal to other devices in a system. To realize this unity gain mode, NSEL must be set to 0x0, PCHENABLE must be set to 0x1, and OUTENABLE must be set to 0x1.

Figure 13-4 shows the block diagram of the GPAMP in unity gain mode.

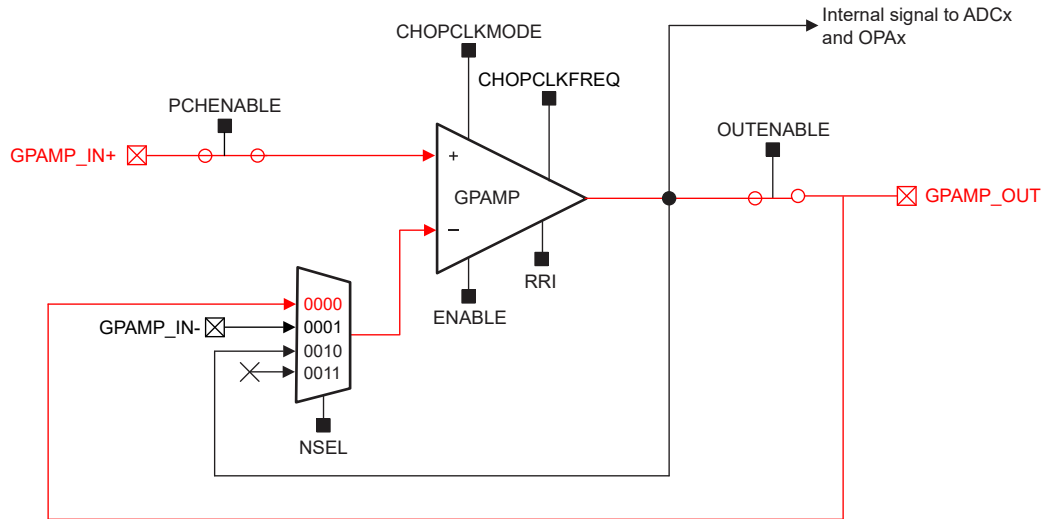


Figure 13-4. GPAMP in Unity Gain Configuration

13.2.6 Chopping

The GPAMP peripheral implements chopper stabilization to reduce offset, drift, and 1/f noise. CHOPCLKMODE can be set to 0x1 for standard chopping mode which requires an external filter. See device-specific data sheet for recommended values. The chopping frequency, which is set by CHOPCLKFREQ, needs to scale down as the gain increases. Refer to [Table 13-3](#) for info on what chopping frequency is required for a given gain.

Table 13-3. Standard Chopping Frequency for GPAMP

CHOPCLKFREQ	Supported External Gain Configuration	Chopping Frequency (Hz)
0x0	-1x / 2x	16k
0x1	-3x / 4x	8k
0x2	-7x / 8x	4k
0x3	-15x / 16x	2k

13.3 GPAMP Registers

The GPAMP control register (PMUOPAMP) is in the SYSCTL registers. See [PMUOPAMP Register](#) for details.

This page intentionally left blank.



The DAC module is a 12-bit voltage-output digital-to-analog converter (DAC). This chapter describes the operation of the DAC module.

14.1 DAC Introduction.....	746
14.2 DAC Operation.....	747
14.3 DAC12 Registers.....	756

14.1 DAC Introduction

The DAC module is a 12-bit voltage-output DAC. The DAC can be configured in 8-bit or 12-bit resolution setting and can be used in conjunction with the DMA controller.

Features of the DAC include:

- 8-bit or 12-bit voltage-output resolution
- Straight binary or twos-complement data format
- Internal sample time generator for generation of predefined sampling rates
- Two hardware triggers from event fabric for D/A conversion
- 4x12-bits internal FIFO for data flow rate control
- Operation with DMA controller without CPU Intervention
- Selectable voltage reference options
- Output internally connected to analog modules like OPA, ADC and COMP
- Self-calibration option for offset error correction

Figure 14-1 shows the block diagram of DAC module.

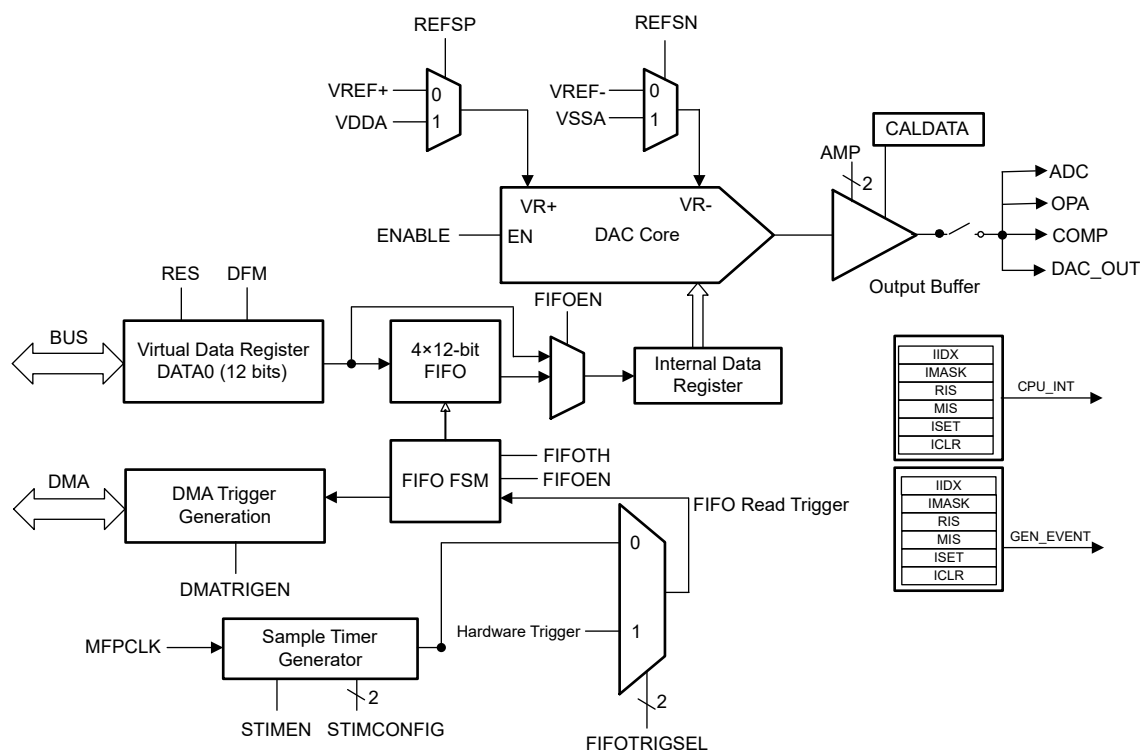


Figure 14-1. DAC Block Diagram

14.2 DAC Operation

The DAC module is configured with user software. The setup and operation of the DAC is discussed in the following sections.

14.2.1 DAC Core

The DAC can be configured to operate in 8-bit or 12-bit resolution setting using the CTL0.RES bit. The CTL0.DFM bit can be used to select the data format to be straight-binary or twos-complement. When using straight-binary data format, the formula for the output voltage is given in [Table 14-1](#).

Table 14-1. DAC Full-Scale Range

Resolution	CTL0.RES	Output Voltage Formula
12-bit	1	$V_{out} = V_{ref} \times (DATA_VALUE / 4096)$
8-bit	0	$V_{out} = V_{ref} \times (DATA_VALUE / 256)$

- In 8-bit resolution setting, the maximum useable value for DATA_VALUE in the DATA0 register is 255 (0xFF).
- In 12-bit resolution setting, the maximum useable value for DATA_VALUE in the DATA0 register is 4095(0xFFF).

The DAC can be enabled by setting the CTL0.ENABLE bit . When the DAC is enabled, the DAC core and output buffer start to settle and the module ready interrupt condition (MODRDYIFG) is generated once to indicate that the DAC is ready for use.

Note

The DAC must be enabled after the configuration of all control registers as required in the application. If a configuration change is required when the DAC is running, the DAC must be disabled first and re-enabled after the new configuration is programmed into the control registers. Any change in control registers while the DAC is running can cause unpredictable results.

14.2.2 DAC Output

DAC output can be connected to internal analog modules OPA, ADC & COMP or DAC_OUT pin by setting CTL1.OPS bit. Reset value of bits CTL1.OPS is 0. There are the following scenarios:

- If CTL1.OPS bit is set, DAC output is connected to OPA, ADC, COMP and DAC_OUT.
- If CTL1.OPS bit is not set, there is no DAC output. The pin DAC_OUT can be connected to ADC, OPA or COMP.

Note

The CTL1.OPS bit must be set as 0 when pin DAC_OUT is configured as the input for ADC, OPA or COMP.

14.2.3 DAC Voltage Reference

The CTL1.REFSP and CTL1.REFSN bits select the voltage reference for the DAC from the following options.

- Analog supply (VDDA) (see [MSPM0Gxx PMU Block Diagram](#)).
- An external reference voltage that can be provided on VREF+ and VREF- pins of the device.
- The internal reference voltage from the output of VREF module.

14.2.4 DAC Output Buffers

The voltage output buffers of the DAC can be configured to be enabled or disabled by setting the bit CTL1.AMPEN. When disabled, the output amplifier can be set either to ground or high impedance using the bit CTL1.AMPHIZ.

14.2.5 DAC Data Formats

The DAC supports both straight-binary and twos-complement data formats. When using straight-binary data format, the full-scale output value is 0xFFF in 12-bit resolution setting and 0xFF in 8-bit resolution settings (see Figure 14-2).

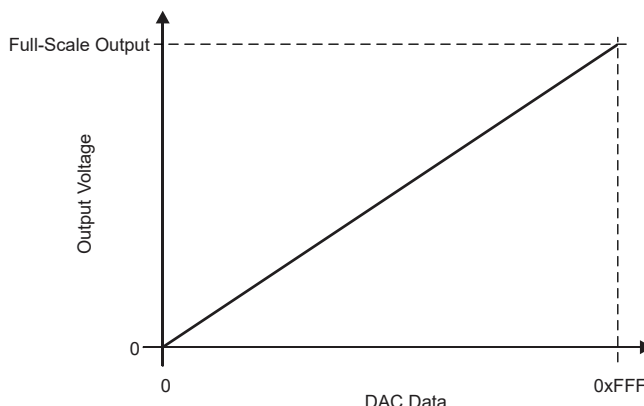


Figure 14-2. Output Voltage vs DAC Data, 12-Bit Straight-Binary Mode

When using twos-complement data format, the range is shifted so that a DAC data value of 0x800 in 12-bit resolution setting and 0x80 in 8-bit resolution settings results in a zero output voltage, 0x0 is the mid-scale output voltage, and 0x7FF in 12-bit resolution setting and 0x7F in 8-bit resolution settings is the full-scale voltage output (see Figure 14-3).

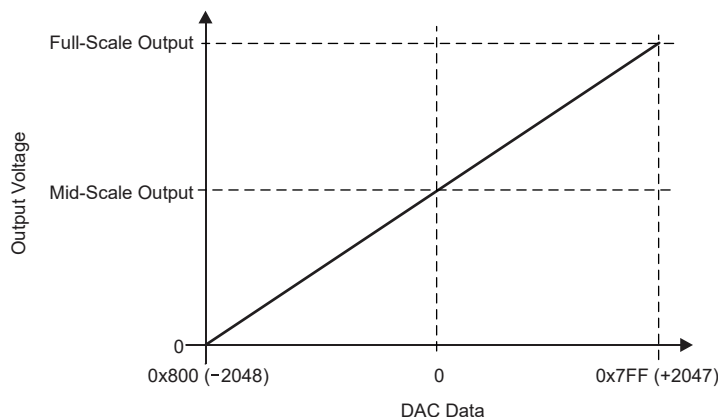


Figure 14-3. Output Voltage vs DAC Data, 12-Bit Twos-Complement Mode

14.2.6 Sample Time Generator

There is a sample time generator available in the DAC module. This can be used to generate DAC conversion trigger for certain predefined sampling rates. The sample time generator is disabled by default and it can be enabled by setting the CTL3.STIMEN bit. When the sample time generator is enabled, it automatically requests for the MFPCLK (4 MHz) and generates trigger according to the programmed trigger rate. The predefined sampling rates supported are 500sps, 1Ksps, 2Ksps, 4Ksps, 8Ksps, 16Ksps, 100Ksps, 200Ksps, 500Ksps, and 1Msps. These trigger rates are programmed by setting the bit CTL3.STIMCONFIG. The predefined sampling rates are given in Table 14-2

Table 14-2. Predefined Sampling Rates

CTL3.STIMCONFIG	Sampling Rate
0	500sps
1	1Ksps

Table 14-2. Predefined Sampling Rates (continued)

CTL3.STIMCONFIG	Sampling Rate
2	2Ksps
3	4Ksps
4	8Ksps
5	16Ksps
6	100Ksps
7	200Ksps
8	500Ksps
9	1Msps

Note

When the sample time generator is enabled, the bus clock must be equal to the MFPCLK or samples can be missed in the FIFO.

14.2.7 DAC FIFO Structure

The DAC module has an 4x12-bits FIFO. By default the FIFO operation is disabled and it is enabled by setting the CTL2.FIFOEN bit. When the CPU or DMA controller writes into the memory mapped data register DATA0, the data is written into FIFO at the location pointed by the write pointer. The data is read from FIFO and loaded to internal DAC data register when the FIFO read trigger is asserted. The data written into the FIFO can be in binary or twos complement format.

14.2.7.1 Loading Data From FIFO to Internal DAC Data Register

The CTL2.FIFOTRIGSEL register is used to select the trigger source for loading data from FIFO into internal DAC data register. There are three triggers sources:

- The sample time generator output can be selected as the trigger source.
- Hardware Trigger, listening for events published through GEN_EVENT configured in the register FSUB_0.

For the hardware trigger source, the DAC module can subscribe to the event through the event fabric. Setting CTR2.FIFOTRIGSEL as 1 is to subscribe to the event on FSUB_0.

When the sample time generator is used, the MFPCLK is used for trigger generation and for transferring the data from FIFO into internal DAC data register. In this scenario, the DAC sampling rate is predictable and has no connection with ULPCLK.

When the sample time generator is not used, the ULPCLK is used for transferring the data from FIFO into internal DAC data register upon the external event trigger. For a predictable DAC sampling rate, the application software must manage ULPCLK frequency to be deterministic. The sampling rate is affected if there is any change in ULPCLK frequency during DAC operation in this scenario.

Whenever the selected trigger source is asserted, the data in FIFO pointed by the read pointer is read and loaded into the internal DAC data register for conversion.

14.2.8 DAC Operation With DMA Controller

The DMA controller can move data into the DAC from the SRAM. Program the CTL2.DMATRIGEN and CTL2.FIFOEN bits to enable DAC operation with the DMA controller. When the DMA trigger mechanism is enabled in the DAC along with the FIFO, the FIFO hardware state machine evaluates the available empty locations in the FIFO and generates a DMA trigger. The FIFO threshold level can be selected using the CTL2.FIFOTH bits. The available FIFO level settings are empty, 1/4, 1/2, and 3/4.

During operation, empty locations in FIFO are continuously evaluated by the FIFO hardware state machine and are compared with the selected FIFO threshold level. The DAC generates a trigger to the DMA when the number of empty locations matches the programmed FIFO threshold.

The DAC-to-DMA controller interface is shown in [Figure 14-4](#) and explained in the following sections.

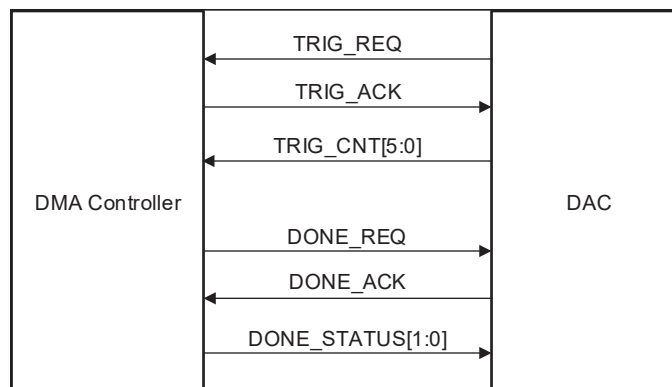


Figure 14-4. DAC Interface With DMA Controller

14.2.8.1 DMA Trigger Interface

There is a defined protocol between DAC and DMA:

1. The DAC generates a trigger TRIG_REQ to the DMA together with the trigger count TRIG_CNT. TRIG_CNT is a 6-bit signal that indicates the number of empty locations available in the DAC FIFO for the DMA to fill. The value of TRIG_CNT is in accordance with the FIFO element size.
2. When TRIG_REQ is asserted, the DMA provides a trigger acknowledge signal TRIG_ACK back to the DAC.

The trigger event and TRIG_ACK are routed through the event fabric and follow the standard 4-way request, acknowledge handshake protocol.

14.2.8.2 DMA Status Interface

When the DAC trigger is captured by DMA and trigger acknowledgment has been provided, both the DAC and the DMA are in triggered state. In this state, the DAC must not generate any new DMA trigger. When DMA starts to service the DAC, the DMA can perform data transfers to the actual value indicated by the trigger count or to a value less than the trigger count (it can be even no transfer). This is not deterministic and depends on factors like DMA bandwidth and channel priorities.

When the DMA performs a certain number of transfers, it asserts a DMA done signal DONE_REQ to the DAC along with the DMA status side band signal DONE_STATUS. The DAC module provides a DMA done acknowledgment DONE_ACK as a response. The DMA done event and DONE_ACK are routed through the event fabric and follow the standard 4-way request, acknowledge handshake protocol.

14.2.8.3 DMA Trigger Generation Scheme

DMA status signal DONE_STATUS value 0 indicates that DMA has more data elements to transfer to DAC. When DAC observes DONE_STATUS to be 0, it performs fresh assessment of the number of empty locations in the FIFO and if that matches with the programmed FIFO threshold level it makes the next TRIG_REQ along with TRIG_CNT. Then the whole sequence of operations mentioned above repeats.

When the DMA has finished transferring the complete data then it asserts DMA done signal DONE_REQ along with DMA status signal DONE_STATUS with a non-zero value. When DAC captures DMA status signal with a non-zero value it sets the DMADONE interrupt flag. This interrupt flag can be used to generate an interrupt to the CPU that owns the DAC for the appropriate action to be taken. CTL2.DMATRIGEN bit should be cleared by software to stop further DMA triggers. This interrupt flag can be used to generate an interrupt to the CPU that owns the DAC for the appropriate action to be taken.

The following section explain the details of using the DAC and DMA with the FIFO enabled

DAC-DMA Operation in FIFO Mode (CTL2.FIFOEN=1) and Sample Time Generator enabled

- Configure the clock, the MFPCLK should be enabled as Sample Time Generator will be used
- Configure DMA source and destination address. Destination address is FIFO data register DATA0
- Choose the transfer mode and DMA transfer size between byte (8-bit), short word (16-bit), word (32-bit) or long word (64-bit)
- Set DAC resolution (8-bit or 12-bit)
- Configure FIFO Trigger Threshold via the CTL2.FIFOTH bits which supports $\frac{1}{4}$, $\frac{1}{2}$ and $\frac{3}{4}$
- Configure and enable DAC and DMA interrupt
- Use CTL2.FIFOTRIGSEL to configure Sample Time Generator as the trigger source for loading data to FIFO data register. Ensure that the clock configurations of MCLK and ULPCLK are properly set to avoid FIFO underrun issues and distortion in the DAC output
 - Make sure that ULPCLK is equal to MFPCLK to maintain synchronization between the clock domains
 - Adjust the FIFO threshold setting to $\frac{1}{2}$ or $\frac{3}{4}$ when the source of MCLK is HFCLK or SYSPLL and its frequency is equal to or greater than 32MHz
- Enable DMA Trigger

14.2.9 DAC Operation With CPU

The CPU can directly load data into DAC for conversion which has the following scenarios:

- **Fixed DC voltage output and FIFO disabled:** CPU writes the data into the data register DATA0. This data will pass on to the internal DAC data register for conversion.
- **Fixed DC voltage output and FIFO enabled:** CPU writes the data into the data register DATA0 which is read from FIFO and loaded to internal DAC data register when the FIFO read trigger is asserted.
- **Fixed AC voltage output and FIFO enabled:** CPU writes the data into the data register DATA0 which is read from FIFO and loaded to internal DAC data register when the FIFO read trigger is asserted.

Note

1. If a fixed AC voltage output is desired, the FIFO must be enabled.
2. CPU must write into the data register DATA0 with byte-0 for 8-bit DAC resolution and lower half-word (bytes 0 and 1) for 12-bit DAC resolution.
3. DMA trigger generation mechanism must be kept disabled when CPU is used to load data into DAC.

For the FIFO read triggers, it is possible to use the sample time generator or the external event available for the event fabric.

14.2.9.1 Interrupt conditions for DAC operation with CPU

When the FIFO is enabled and CPU is used to load data into DAC, there are certain FIFO specific interrupt conditions generated in the DAC. These are FIFO empty, FIFO full, FIFO 1/4 empty, FIFO 1/2 empty and FIFO 3/4 empty. The reset value of all these interrupt flags is 0. When the DAC module is enabled along with the FIFO, the internal logic evaluates the FIFO status and updates the FIFO specific interrupt flags accordingly. These interrupt conditions can be enabled appropriately by software so that CPU is notified about the empty locations available in FIFO for loading data into DAC.

14.2.10 Data Register Format

The DAC supports programmable resolution setting and data format . The figures given in this section take DATA0 register as an example and describe the format in which CPU or DMA controller is expected to write data into DATA0 register for one data sample.

The CPU always writes only into DATA0 register. If FIFO is disabled then CPU can write into only byte-0 for 8-bit DAC resolution and lower half-word (bytes 0 and 1) for 12-bit DAC resolution. The upper half-word of DATA0 register is never written.

When the FIFO is enabled, the CPU can write up to 4 data samples into DATA0 register for 8-bit resolution or 12-bit resolution.

Figure 14-5 shows the data format in DATA0 register for 8-bit binary or two's complement data.

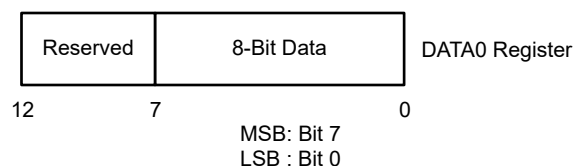


Figure 14-5. 8-Bit Data

14.2.11 DAC Output Amplifier Offset Calibration

The offset voltage of the DAC output amplifier can be positive or negative. When the offset is negative, the output amplifier attempts to drive the voltage negative but cannot do so. The output voltage remains at zero until the DAC digital input produces a sufficient positive output voltage to overcome the negative offset voltage, resulting in the transfer function in Figure 14-6.

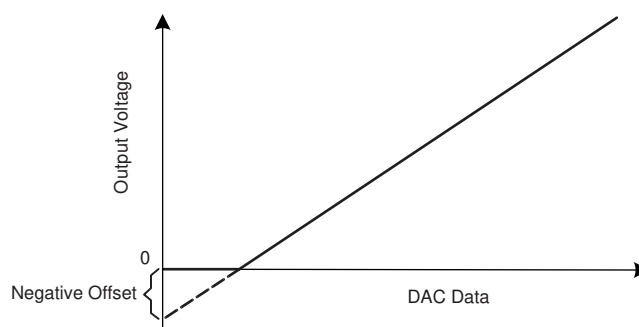


Figure 14-6. Negative Offset

When the output amplifier has a positive offset, a digital input of zero does not result in a zero output voltage. The DAC output voltage reaches the maximum output level before the DAC data reaches the maximum code (see Figure 14-7).

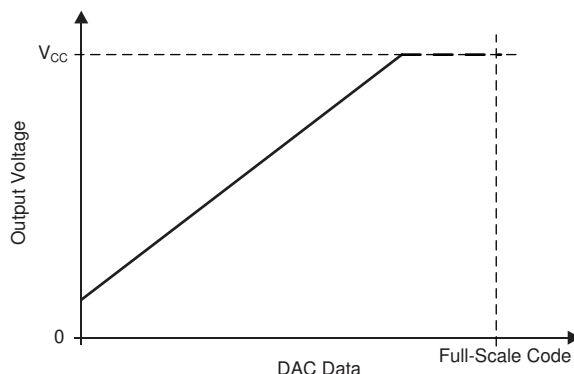


Figure 14-7. Positive Offset

The DAC can self-calibrate the offset voltage of the output amplifier. The application can perform self-calibration of the DAC output buffer to achieve the offset error given in the electrical specification. The CALCTL.CALON bit is used to initiate the offset error calibration. The DAC output is high impedance while the calibration is active. When the calibration is complete, the CALCTL.CALON bit is automatically reset. Software must configure the CTL1.AMP bits before calibration.

For best calibration results, activity on the device pins and in the CPU must be minimized during calibration. Calibration data is loaded into CALDATA; when DAC self-calibration is initiated using the CALCTL.CALON bit, the CALDATA register is continuously updated during the course of calibration. Read the CALDATA register only after the CALCTL.CALON bit is cleared, otherwise incorrect values can be read. The DAC calibration data format is twos complement. Only the lowest byte is used and the upper bytes have no effect on the offset calibration.

Note

Recalibrate the DAC output buffer after changing the reference voltage level in the VREF module when DAC is in use.

14.2.12 Interrupt and Event Support

The DAC module contains three [event publishers](#) and two [event subscribers](#). One event publisher (CPU_INT) manages DAC interrupt requests (IRQs) to the CPU subsystem via a [static event route](#). The second event publisher (GEN_EVENT) can be used to setup the generic event publisher via [Generic route](#). A direct DMA trigger can be used as a DAC-to-DMA trigger to send DAC events to the DMA through DMA event route.

The event subscriber (FSUB_0) can be used to subscribe to events which are published to the event fabric through a generic event route channel.

The DAC events are summarized in [Table 14-1](#).

Table 14-508. DAC Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU interrupt	Publisher	DAC	CPU Subsystem	Static route	CPU_INT registers	Fixed interrupt route from DAC to CPU
Generic publisher event	Publisher	DAC	Other peripherals	Generic route (FPUB_0)	GEN_EVENT and FPUB_0 registers	Trigger generic event channel from DAC
DMA Trigger event	Publisher	DAC	DMA	DMA route	DMA_TRIG and FPUB_1 registers	Fixed trigger route from DAC to DMA

Table 14-508. DAC Events (continued)

Event	Type	Source	Destination	Route	Configuration	Functionality
Generic subscriber event	Subscriber	Other peripherals	DAC	Generic route (FSUB_0)	FSUB_0	DAC subscription to generic event channel

14.2.12.1 CPU Interrupt Event Publisher (CPU_INT)

The DAC module provides different interrupt sources which can be configured to source a [CPU interrupt event](#). In order of decreasing interrupt priority, the CPU interrupt events from the DAC are:

Table 14-509. DAC Interrupt Event Conditions (CPU_INT)

Index (IIDX)	Name	Description
0x0	NO_INTR	No bit set (IIDX.STAT = 0) means there is no pending interrupt request
0x2	MODRDYIFG	The DAC output is settled after it is turned on using ENABLE in the CTL0 register
0x9	FIFOFULLIFG	FIFO full interrupt flag is set when the FIFO is full
0xA	FIFO1B4IFG	FIFO one fourth empty interrupt flag is set when one fourth of FIFO locations are empty.
0xB	FIFO1B2IFG	FIFO half empty interrupt flag is set when half of the FIFO locations are empty.
0xC	FIFO3B4IFG	FIFO three fourth empty interrupt flag is set when all the locations in FIFO are empty.
0xD	FIFOEMPTYIFG	FIFO empty interrupt flag is set when all data in the FIFO have been shifted out.
0xE	FIFOURUNIFG	FIFO underrun flag is set when the FIFO read trigger is asserted while the FIFO is empty.
0xF	DMADONEIFG	DMA done interrupt flag is set when the DMA data transfer of programmed block size is completed

The CPU interrupt event configuration is managed with the CPU_INT event management registers. Interrupt (RIS) flags are cleared upon software reading the IIDX register or writing to the respective ICLR register bits.

See [Section 7.2.5](#) section for guidance on configuring the event registers for CPU interrupts.

14.2.12.2 Generic Event Publisher (GEN_EVENT)

The DAC module provides the following interrupt sources, one of which can be configured to publish an event to a generic event route channel.

Table 14-510. DAC Generic Event Publisher Conditions (GEN_EVENT)

Index (IIDX)	Name	Description
0x0	NO_INTR	No bit set (IIDX.STAT = 0) means there is no pending interrupt request
0x2	MODRDYIFG	Module ready interrupt once DAC is enabled and DAC output is settled
0x9	FIFOFULLIFG	FIFO full interrupt flag is set when the FIFO is full
0xA	FIFO1B4IFG	FIFO one fourth empty interrupt flag is set when one fourth of FIFO locations are empty.
0xB	FIFO1B2IFG	FIFO half empty interrupt flag is set when half of the FIFO locations are empty.
0xC	FIFO3B4IFG	FIFO three fourth empty interrupt flag is set when all the locations in FIFO are empty.
0xD	FIFOEMPTYIFG	FIFO empty interrupt flag is set when all data in the FIFO have been shifted out.
0xE	FIFOURUNIFG	FIFO underrun flag is set when the FIFO read trigger is asserted while the FIFO is empty.
0xF	DMADONEIFG	DMA done interrupt flag is set when the DMA data transfer of programmed block size is completed

The generic event publisher configuration is managed with the GEN_EVENT event management registers. Interrupt (RIS) flags are cleared based on acknowledgment (ACK) signal from the subscriber module received over the event fabric. See [\[Using Event Registers\]](#) for guidance on configuring the event registers for generic event publishers. The generic event channel which GEN_EVENT is to publish to must be selected by writing the target generic channel ID to the **FPUB_0** register in the DAC. See [\[Generic Event Route\]](#) for guidance on configuring generic event routes. If this publisher is not used in an application, the FPUB_0 register can be left in

a disconnected state (set equal to zero) and no events should be unmasked through the MIS register in the ADC GEN_EVENT register set.

14.2.12.3 DMA Trigger Event Publisher

The DAC module provides different interrupt sources which can be configured to source the DMA trigger. In order of decreasing interrupt priority, the DMA trigger events from the DAC are given as the following table. When the DMA channel is needed by the DAC, the DMA trigger should be unmasked in the IMASK register.

Note

The DMA trigger from the DAC is a direct trigger in hardware and not a part of the DMA_TRIG group of configurable event management registers.

To configure the DMA controller to trigger from the DAC, please see [Section 14.2.8](#).

Table 14-6. DAC DMA Trigger Event Publisher Conditions

Index (IIDX)	Name	Description
0x0	NO_INTR	No bit set (IIDX.STAT = 0) means there is no pending interrupt request
0x2	MODRDYIFG	Module ready interrupt once DAC is enabled and DAC output is settled
0x9	FIFOFULLIFG	FIFO full interrupt flag is set when the FIFO is full
0xA	FIFO1B4IFG	FIFO one fourth empty interrupt flag is set when one fourth of FIFO locations are empty.
0xB	FIFO1B2IFG	FIFO half empty interrupt flag is set when half of the FIFO locations are empty.
0xC	FIFO3B4IFG	FIFO three fourth empty interrupt flag is set when all the locations in FIFO are empty.
0xD	FIFOEMPTYIFG	FIFO empty interrupt flag is set when all data in the FIFO have been shifted out.
0xE	FIFOURUNIFG	FIFO underrun flag is set when the FIFO read trigger is asserted while the FIFO is empty.
0xF	DMADONEIFG	DMA done interrupt flag is set when the DMA data transfer of programmed block size is completed

14.2.12.4 Generic Event Subscriber (FSUB_0)

The DAC module can trigger FIFO transfers by receiving events routed through a generic channel from other peripherals. When the event is received, the data in the FIFO is loaded into internal DAC data register. Refer to [Generic Event Route](#) and [Peripheral to Peripheral Event](#) for details on how the generic event route works. When the channel to be used is determined, and both the publisher and subscriber ports for the peripherals to connect are known, use the steps below to establish the event connection.

In this example, a GPIO-triggered DAC FIFO transfer is configured, using GPIO Port A to publish an event to generic channel 1, with DAC subscribing to generic channel 1 as a start-of-conversion trigger.

1. Configure the GEN_EVENT registers of GPIO Port A to set the event request based on the appropriate event (for example, a DIN rise event).
2. Store 0x1 into the FPUB_0 register of GPIO Port A to publish the GPIO event selected by the GEN_EVENT registers to generic route channel 1. Channel 1 must not be in use by another peripheral.
3. Store 0x1 the FSUB_0 register of DAC so that DAC is listening for events published by the timer to channel 1.
4. Configure DAC to trigger from the subscriber port according to the configuration instructions in [Loading Data From FIFO to Internal DAC Data Register](#).
5. Configure and enable the appropriate GPIO pin to monitor input voltage events.

14.3 DAC12 Registers

Table 14-7 lists the memory-mapped registers for the DAC12 registers. All register offset addresses not listed in Table 14-7 should be considered as reserved locations and the register contents should not be modified.

Table 14-7. DAC12 Registers

Offset	Acronym	Register Name	Group	Section
400h	FSUB_0	Subscriber Port 0		Go
444h	FPUB_1	Publisher port 1		Go
800h	PWREN	Power enable		Go
804h	RSTCTL	Reset Control		Go
814h	STAT	Status Register		Go
1020h	IIDX	Interrupt index	CPU_INT	Go
1028h	IMASK	Interrupt mask	CPU_INT	Go
1030h	RIS	Raw interrupt status	CPU_INT	Go
1038h	MIS	Masked interrupt status	CPU_INT	Go
1040h	ISSET	Interrupt set	CPU_INT	Go
1048h	ICLR	Interrupt clear	CPU_INT	Go
1050h	IIDX	Interrupt index	GEN_EVENT	Go
1058h	IMASK	Interrupt mask	GEN_EVENT	Go
1060h	RIS	Raw interrupt status	GEN_EVENT	Go
1068h	MIS	Masked interrupt status	GEN_EVENT	Go
1070h	ISSET	Interrupt set	GEN_EVENT	Go
1078h	ICLR	Interrupt clear	GEN_EVENT	Go
10E0h	EVT_MODE	Event Mode		Go
10FCh	DESC	Module Description		Go
1100h	CTL0	Control 0		Go
1110h	CTL1	Control 1		Go
1120h	CTL2	Control 2		Go
1130h	CTL3	Control 3		Go
1140h	CALCTL	Calibration control		Go
1160h	CALDATA	Calibration data		Go
1200h	DATA0	Data 0		Go

Complex bit access types are encoded to fit into small table cells. Table 14-8 shows the codes that are used for access types in this section.

Table 14-8. DAC12 Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
RH	R H	Read Set or cleared by hardware
Write Type		
K	K	Write protected by a key
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		

Table 14-8. DAC12 Access Type Codes (continued)

Access Type	Code	Description
$-n$		Value after reset or the default value

14.3.1 FSUB_0 (Offset = 400h) [Reset = 00000000h]

FSUB_0 is shown in [Figure 14-8](#) and described in [Table 14-9](#).

Return to the [Summary Table](#).

Subscriber port 0

Figure 14-8. FSUB_0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-												R/W-0h			

Table 14-9. FSUB_0 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. others = connected to channel_ID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 15.

14.3.2 FPUB_1 (Offset = 444h) [Reset = 00000000h]

FPUB_1 is shown in [Figure 14-9](#) and described in [Table 14-10](#).

Return to the [Summary Table](#).

Publisher port 1

Figure 14-9. FPUB_1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-												R/W-0h			

Table 14-10. FPUB_1 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. others = connected to channel_ID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 15.

14.3.3 PWREN (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 14-10](#) and described in [Table 14-11](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 14-10. PWREN

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-							K-0h

Table 14-11. PWREN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	ENABLE	K	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

14.3.4 RSTCTL (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 14-11](#) and described in [Table 14-12](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 14-11. RSTCTL

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-							
15	14	13	12	11	10	9	8
RESERVED							
W-							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCL R	RESETASSERT
W-						WK-0h	WK-0h

Table 14-12. RSTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	W	0h	
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

14.3.5 STAT (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 14-12](#) and described in [Table 14-13](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 14-12. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED							
R-							

Table 14-13. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0h	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

14.3.6 IIDX (Offset = 1020h) [Reset = 00000000h]

IIDX is shown in [Figure 14-13](#) and described in [Table 14-14](#).

Return to the [Summary Table](#).

Interrupt index register. This read-only register provides the interrupt index of the pending interrupt with the highest priority. It also indicates if no interrupt is pending. The priority order is fixed: lower index equals higher priority. Alternatively, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred.

On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flags in the RIS and MIS are cleared as well. After a read from the CPU (not from the debug interface), the register must be updated with the next highest priority interrupt or indicate that no interrupt is pending. Only interrupts which are selected via IMASK are indicated.

Figure 14-13. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																															STAT
R-0h																															R-0h

Table 14-14. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3-0	STAT	R	0h	Interrupt index status 0h = No pending interrupt 2h = Module ready interrupt 9h = FIFO full interrupt Ah = FIFO one fourth empty interrupt Bh = FIFO half empty interrupt Ch = FIFO three fourth empty interrupt Dh = FIFO empty interrupt Eh = FIFO underrun interrupt Fh = DMA done interrupt

14.3.7 IMASK (Offset = 1028h) [Reset = 00000000h]

IMASK is shown in [Figure 14-14](#) and described in [Table 14-15](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS

Figure 14-14. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED	DMADONEIFG	FIFOURUNIFG	FIFOEMPTYIFG	FIFO3B4IFG	FIFO1B2IFG	FIFO1B4IFG	FIFOFULLIFG
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
RESERVED						MODRDYIFG	RESERVED
R/W-0h						R/W-0h	R/W-0h

Table 14-15. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	R/W	0h	
14	DMADONEIFG	R/W	0h	Masks DMADONEIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
13	FIFOURUNIFG	R/W	0h	Masks FIFOURUNIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
12	FIFOEMPTYIFG	R/W	0h	Masks FIFOEMPTYIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
11	FIFO3B4IFG	R/W	0h	Masks FIFO3B4IFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
10	FIFO1B2IFG	R/W	0h	Masks FIFO1B2IFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
9	FIFO1B4IFG	R/W	0h	Masks FIFO1B4IFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
8	FIFOFULLIFG	R/W	0h	Masks FIFOFULLIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
7-2	RESERVED	R/W	0h	

Table 14-15. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1	MODRDYIFG	R/W	0h	Masks MODRDYIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
0	RESERVED	R/W	0h	

14.3.8 RIS (Offset = 1030h) [Reset = 00001E00h]

RIS is shown in [Figure 14-15](#) and described in [Table 14-16](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 14-15. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED	DMADONEIFG	FIFOURUNIFG	FIFOEMPTYIFG	FIFO3B4IFG	FIFO1B2IFG	FIFO1B4IFG	FIFOFULLIFG
R-0h	R-0h	R-0h	R-1h	R-1h	R-1h	R-1h	R-0h
7	6	5	4	3	2	1	0
RESERVED						MODRDYIFG	RESERVED
R-0h						R-0h	R-0h

Table 14-16. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	R	0h	
14	DMADONEIFG	R	0h	Raw interrupt status for DMADONEIFG 0h = DMA done condition did not occur 1h = DMA done condition occurred
13	FIFOURUNIFG	R	0h	Raw interrupt status for FIFOURUNIFG 0h = FIFO underrun condition did not occur 1h = FIFO underrun condition occurred
12	FIFOEMPTYIFG	R	1h	Raw interrupt status for FIFOEMPTYIFG 0h = FIFO empty condition did not occur 1h = FIFO empty condition occurred
11	FIFO3B4IFG	R	1h	Raw interrupt status for FIFO3B4IFG 0h = FIFO three fourth empty condition did not occur 1h = FIFO three fourth empty condition occurred
10	FIFO1B2IFG	R	1h	Raw interrupt status for FIFO1B2IFG 0h = FIFO half empty condition did not occur 1h = FIFO half empty condition occurred
9	FIFO1B4IFG	R	1h	Raw interrupt status for FIFO1B4IFG 0h = FIFO one fourth empty condition did not occur 1h = FIFO one fourth empty condition occurred
8	FIFOFULLIFG	R	0h	Raw interrupt status for FIFOFULLIFG 0h = FIFO full condition did not occur 1h = FIFO full condition occurred
7-2	RESERVED	R	0h	
1	MODRDYIFG	R	0h	Raw interrupt status for MODRDYIFG 0h = DAC module ready event did not occur 1h = DAC module ready event occurred
0	RESERVED	R	0h	

14.3.9 MIS (Offset = 1038h) [Reset = 00000000h]

MIS is shown in [Figure 14-16](#) and described in [Table 14-17](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 14-16. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED	DMADONEIFG	FIFOURUNIFG	FIFOEMPTYIFG	FIFO3B4IFG	FIFO1B2IFG	FIFO1B4IFG	FIFOFULLIFG
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
RESERVED						MODRDYIFG	RESERVED
R-0h						R-0h	R-0h

Table 14-17. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	R	0h	
14	DMADONEIFG	R	0h	Masked interrupt status for DMADONEIFG 0h = DMADONEIFG does not request an interrupt service routine 1h = DMADONEIFG requests an interrupt service routine
13	FIFOURUNIFG	R	0h	Masked interrupt status for FIFOURUNIFG 0h = FIFOURUNIFG does not request an interrupt service routine 1h = FIFOURUNIFG requests an interrupt service routine
12	FIFOEMPTYIFG	R	0h	Masked interrupt status for FIFOEMPTYIFG 0h = FIFOEMPTYIFG does not request an interrupt service routine 1h = FIFOEMPTYIFG requests an interrupt service routine
11	FIFO3B4IFG	R	0h	Masked interrupt status for FIFO3B4IFG 0h = FIFO3B4IFG does not request an interrupt service routine 1h = FIFO3B4IFG requests an interrupt service routine
10	FIFO1B2IFG	R	0h	Masked interrupt status for FIFO1B2IFG 0h = FIFO1B2IFG does not request an interrupt service routine 1h = FIFO1B2IFG requests an interrupt service routine
9	FIFO1B4IFG	R	0h	Masked interrupt status for FIFO1B4IFG 0h = FIFO1B4IFG does not request an interrupt service routine 1h = FIFO1B4IFG requests an interrupt service routine
8	FIFOFULLIFG	R	0h	Masked interrupt status for FIFOFULLIFG 0h = FIFOFULLIFG does not request an interrupt service routine 1h = FIFOFULLIFG requests an interrupt service routine
7-2	RESERVED	R	0h	
1	MODRDYIFG	R	0h	Masked interrupt status for MODRDYIFG 0h = MODRDYIFG does not request an interrupt service routine 1h = MODRDYIFG requests an interrupt service routine
0	RESERVED	R	0h	

14.3.10 ISET (Offset = 1040h) [Reset = 00000000h]

ISET is shown in [Figure 14-17](#) and described in [Table 14-18](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 14-17. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED	DMADONEIFG	FIFOURUNIFG	FIFOEMPTYIFG	FIFO3B4IFG	FIFO1B2IFG	FIFO1B4IFG	FIFOFULLIFG
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
RESERVED						MODRDYIFG	RESERVED
W-0h						W-0h	W-0h

Table 14-18. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	W	0h	
14	DMADONEIFG	W	0h	Sets DMADONEIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to DMADONEIFG is set
13	FIFOURUNIFG	W	0h	Sets FIFOURUNIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFOURUNIFG is set
12	FIFOEMPTYIFG	W	0h	Sets FIFOEMPTYIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFOEMPTYIFG is set
11	FIFO3B4IFG	W	0h	Sets FIFO3B4IFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFO3B4IFG is set
10	FIFO1B2IFG	W	0h	Sets FIFO1B2IFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFO1B2IFG is set
9	FIFO1B4IFG	W	0h	Sets FIFO1B4IFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFO1B4IFG is set
8	FIFOFULLIFG	W	0h	Sets FIFOFULLIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFOFULLIFG is set
7-2	RESERVED	W	0h	
1	MODRDYIFG	W	0h	Sets MODRDYIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to MODRDYIFG is set
0	RESERVED	W	0h	

14.3.11 ICLR (Offset = 1048h) [Reset = 00000000h]

ICLR is shown in [Figure 14-18](#) and described in [Table 14-19](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 14-18. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED	DMADONEIFG	FIFOURUNIFG	FIFOEMPTYIFG	FIFO3B4IFG	FIFO1B2IFG	FIFO1B4IFG	FIFOFULLIFG
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
RESERVED						MODRDYIFG	RESERVED
W-0h						W-0h	W-0h

Table 14-19. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	W	0h	
14	DMADONEIFG	W	0h	Clears DMADONEIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to DMADONEIFG is cleared
13	FIFOURUNIFG	W	0h	Clears FIFOURUNIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFOURUNIFG is cleared
12	FIFOEMPTYIFG	W	0h	Clears FIFOEMPTYIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFOEMPTYIFG is cleared
11	FIFO3B4IFG	W	0h	Clears FIFO3B4IFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFO3B4IFG is cleared
10	FIFO1B2IFG	W	0h	Clears FIFO1B2IFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFO1B2IFG is cleared
9	FIFO1B4IFG	W	0h	Clears FIFO1B4IFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFO1B4IFG is cleared
8	FIFOFULLIFG	W	0h	Clears FIFOFULLIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFOFULLIFG is cleared
7-2	RESERVED	W	0h	
1	MODRDYIFG	W	0h	Clears MODRDYIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to MODRDYIFG is cleared
0	RESERVED	W	0h	

14.3.12 IIDX (Offset = 1050h) [Reset = 00000000h]

IIDX is shown in [Figure 14-19](#) and described in [Table 14-20](#).

Return to the [Summary Table](#).

Interrupt index register. This read-only register provides the interrupt index of the pending interrupt with the highest priority. It also indicates if no interrupt is pending. The priority order is fixed: lower index equals higher priority. Alternatively, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred.

On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flags in the RIS and MIS are cleared as well. After a read from the CPU (not from the debug interface), the register must be updated with the next highest priority interrupt or indicate that no interrupt is pending. Only interrupts which are selected via IMASK are indicated.

Figure 14-19. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																												STAT			
R-0h																												R-0h			

Table 14-20. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3-0	STAT	R	0h	Interrupt index status 0h = No pending interrupt 2h = Module ready interrupt 9h = FIFO full interrupt Ah = FIFO one fourth empty interrupt Bh = FIFO half empty interrupt Ch = FIFO three fourth empty interrupt Dh = FIFO empty interrupt Eh = FIFO underrun interrupt Fh = DMA done interrupt

14.3.13 IMASK (Offset = 1058h) [Reset = 00000000h]

IMASK is shown in [Figure 14-20](#) and described in [Table 14-21](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS

Figure 14-20. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED	DMADONEIFG	FIFOURUNIFG	FIFOEMPTYIFG	FIFO3B4IFG	FIFO1B2IFG	FIFO1B4IFG	FIFOFULLIFG
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
RESERVED						MODRDYIFG	RESERVED
R/W-0h						R/W-0h	R/W-0h

Table 14-21. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	R/W	0h	
14	DMADONEIFG	R/W	0h	Masks DMADONEIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
13	FIFOURUNIFG	R/W	0h	Masks FIFOURUNIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
12	FIFOEMPTYIFG	R/W	0h	Masks FIFOEMPTYIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
11	FIFO3B4IFG	R/W	0h	Masks FIFO3B4IFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
10	FIFO1B2IFG	R/W	0h	Masks FIFO1B2IFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
9	FIFO1B4IFG	R/W	0h	Masks FIFO1B4IFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
8	FIFOFULLIFG	R/W	0h	Masks FIFOFULLIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
7-2	RESERVED	R/W	0h	

Table 14-21. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1	MODRDYIFG	R/W	0h	Masks MODRDYIFG 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
0	RESERVED	R/W	0h	

14.3.14 RIS (Offset = 1060h) [Reset = 00001E00h]

RIS is shown in [Figure 14-21](#) and described in [Table 14-22](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 14-21. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED	DMADONEIFG	FIFOURUNIFG	FIFOEMPTYIFG	FIFO3B4IFG	FIFO1B2IFG	FIFO1B4IFG	FIFOFULLIFG
R-0h	R-0h	R-0h	R-1h	R-1h	R-1h	R-1h	R-0h
7	6	5	4	3	2	1	0
RESERVED						MODRDYIFG	RESERVED
R-0h						R-0h	R-0h

Table 14-22. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	R	0h	
14	DMADONEIFG	R	0h	Raw interrupt status for DMADONEIFG 0h = DMA done condition did not occur 1h = DMA done condition occurred
13	FIFOURUNIFG	R	0h	Raw interrupt status for FIFOURUNIFG 0h = FIFO underrun condition did not occur 1h = FIFO underrun condition occurred
12	FIFOEMPTYIFG	R	1h	Raw interrupt status for FIFOEMPTYIFG 0h = FIFO empty condition did not occur 1h = FIFO empty condition occurred
11	FIFO3B4IFG	R	1h	Raw interrupt status for FIFO3B4IFG 0h = FIFO three fourth empty condition did not occur 1h = FIFO three fourth empty condition occurred
10	FIFO1B2IFG	R	1h	Raw interrupt status for FIFO1B2IFG 0h = FIFO half empty condition did not occur 1h = FIFO half empty condition occurred
9	FIFO1B4IFG	R	1h	Raw interrupt status for FIFO1B4IFG 0h = FIFO one fourth empty condition did not occur 1h = FIFO one fourth empty condition occurred
8	FIFOFULLIFG	R	0h	Raw interrupt status for FIFOFULLIFG 0h = FIFO full condition did not occur 1h = FIFO full condition occurred
7-2	RESERVED	R	0h	
1	MODRDYIFG	R	0h	Raw interrupt status for MODRDYIFG 0h = DAC module ready event did not occur 1h = DAC module ready event occurred
0	RESERVED	R	0h	

14.3.15 MIS (Offset = 1068h) [Reset = 00000000h]

MIS is shown in [Figure 14-22](#) and described in [Table 14-23](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 14-22. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED	DMADONEIFG	FIFOURUNIFG	FIFOEMPTYIFG	FIFO3B4IFG	FIFO1B2IFG	FIFO1B4IFG	FIFOFULLIFG
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
RESERVED						MODRDYIFG	RESERVED
R-0h						R-0h	R-0h

Table 14-23. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	R	0h	
14	DMADONEIFG	R	0h	Masked interrupt status for DMADONEIFG 0h = DMADONEIFG does not request an interrupt service routine 1h = DMADONEIFG requests an interrupt service routine
13	FIFOURUNIFG	R	0h	Masked interrupt status for FIFOURUNIFG 0h = FIFOURUNIFG does not request an interrupt service routine 1h = FIFOURUNIFG requests an interrupt service routine
12	FIFOEMPTYIFG	R	0h	Masked interrupt status for FIFOEMPTYIFG 0h = FIFOEMPTYIFG does not request an interrupt service routine 1h = FIFOEMPTYIFG requests an interrupt service routine
11	FIFO3B4IFG	R	0h	Masked interrupt status for FIFO3B4IFG 0h = FIFO3B4IFG does not request an interrupt service routine 1h = FIFO3B4IFG requests an interrupt service routine
10	FIFO1B2IFG	R	0h	Masked interrupt status for FIFO1B2IFG 0h = FIFO1B2IFG does not request an interrupt service routine 1h = FIFO1B2IFG requests an interrupt service routine
9	FIFO1B4IFG	R	0h	Masked interrupt status for FIFO1B4IFG 0h = FIFO1B4IFG does not request an interrupt service routine 1h = FIFO1B4IFG requests an interrupt service routine
8	FIFOFULLIFG	R	0h	Masked interrupt status for FIFOFULLIFG 0h = FIFOFULLIFG does not request an interrupt service routine 1h = FIFOFULLIFG requests an interrupt service routine
7-2	RESERVED	R	0h	
1	MODRDYIFG	R	0h	Masked interrupt status for MODRDYIFG 0h = MODRDYIFG does not request an interrupt service routine 1h = MODRDYIFG requests an interrupt service routine
0	RESERVED	R	0h	

14.3.16 ISET (Offset = 1070h) [Reset = 00000000h]

ISET is shown in [Figure 14-23](#) and described in [Table 14-24](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 14-23. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED	DMADONEIFG	FIFOURUNIFG	FIFOEMPTYIFG	FIFO3B4IFG	FIFO1B2IFG	FIFO1B4IFG	FIFOFULLIFG
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
RESERVED						MODRDYIFG	RESERVED
W-0h						W-0h	W-0h

Table 14-24. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	W	0h	
14	DMADONEIFG	W	0h	Sets DMADONEIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to DMADONEIFG is set
13	FIFOURUNIFG	W	0h	Sets FIFOURUNIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFOURUNIFG is set
12	FIFOEMPTYIFG	W	0h	Sets FIFOEMPTYIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFOEMPTYIFG is set
11	FIFO3B4IFG	W	0h	Sets FIFO3B4IFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFO3B4IFG is set
10	FIFO1B2IFG	W	0h	Sets FIFO1B2IFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFO1B2IFG is set
9	FIFO1B4IFG	W	0h	Sets FIFO1B4IFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFO1B4IFG is set
8	FIFOFULLIFG	W	0h	Sets FIFOFULLIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFOFULLIFG is set
7-2	RESERVED	W	0h	
1	MODRDYIFG	W	0h	Sets MODRDYIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to MODRDYIFG is set
0	RESERVED	W	0h	

14.3.17 ICLR (Offset = 1078h) [Reset = 00000000h]

ICLR is shown in [Figure 14-24](#) and described in [Table 14-25](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 14-24. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED	DMADONEIFG	FIFOURUNIFG	FIFOEMPTYIFG	FIFO3B4IFG	FIFO1B2IFG	FIFO1B4IFG	FIFOFULLIFG
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
RESERVED						MODRDYIFG	RESERVED
W-0h						W-0h	W-0h

Table 14-25. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	W	0h	
14	DMADONEIFG	W	0h	Clears DMADONEIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to DMADONEIFG is cleared
13	FIFOURUNIFG	W	0h	Clears FIFOURUNIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFOURUNIFG is cleared
12	FIFOEMPTYIFG	W	0h	Clears FIFOEMPTYIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFOEMPTYIFG is cleared
11	FIFO3B4IFG	W	0h	Clears FIFO3B4IFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFO3B4IFG is cleared
10	FIFO1B2IFG	W	0h	Clears FIFO1B2IFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFO1B2IFG is cleared
9	FIFO1B4IFG	W	0h	Clears FIFO1B4IFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFO1B4IFG is cleared
8	FIFOFULLIFG	W	0h	Clears FIFOFULLIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to FIFOFULLIFG is cleared
7-2	RESERVED	W	0h	
1	MODRDYIFG	W	0h	Clears MODRDYIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to MODRDYIFG is cleared
0	RESERVED	W	0h	

14.3.18 EVT_MODE (Offset = 10E0h) [Reset = 00000009h]

EVT_MODE is shown in [Figure 14-25](#) and described in [Table 14-26](#).

Return to the [Summary Table](#).

Event mode register. It is used to select whether each line is disabled, in software mode (software clears the RIS) or in hardware mode (hardware clears the RIS)

Figure 14-25. EVT_MODE

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED				EVT1_CFG		INT0_CFG	
R/W-0h				R-2h		R-1h	

Table 14-26. EVT_MODE Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-2	EVT1_CFG	R	2h	Event line mode select for event corresponding to none.GEN_EVENT 0h = The interrupt or event line is disabled. 1h = Event handled by software. Software must clear the associated RIS flag. 2h = Event handled by hardware. The hardware (another module) clears automatically the associated RIS flag.
1-0	INT0_CFG	R	1h	Event line mode select for event corresponding to none.CPU_INT 0h = The interrupt or event line is disabled. 1h = Event handled by software. Software must clear the associated RIS flag. 2h = Event handled by hardware. The hardware (another module) clears automatically the associated RIS flag.

14.3.19 DESC (Offset = 10FCh) [Reset = 03110000h]

DESC is shown in [Figure 14-26](#) and described in [Table 14-27](#).

Return to the [Summary Table](#).

This register identifies the peripheral and its exact version.

Figure 14-26. DESC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODULEID															
R-311h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FEATUREVER				RESERVED				MAJREV				MINREV			
R-0h				R-				R-0h				R-0h			

Table 14-27. DESC Field Descriptions

Bit	Field	Type	Reset	Description
31-16	MODULEID	R	311h	Module identification contains a unique peripheral identification number. The assignments are maintained in a central database for all of the platform modules to ensure uniqueness.
15-12	FEATUREVER	R	0h	Feature Set for the module *instance*
11-8	RESERVED	R	0h	
7-4	MAJREV	R	0h	Major rev of the IP
3-0	MINREV	R	0h	Minor rev of the IP

14.3.20 CTL0 (Offset = 1100h) [Reset = 00000000h]

CTL0 is shown in [Figure 14-27](#) and described in [Table 14-28](#).

Return to the [Summary Table](#).

Control 0 register.

Figure 14-27. CTL0

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							DFM
R/W-0h							R/W-0h
15	14	13	12	11	10	9	8
RESERVED							RES
R/W-0h							R/W-0h
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-0h							R/W-0h

Table 14-28. CTL0 Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R/W	0h	
16	DFM	R/W	0h	This bit defines the DAC input data format. 0h = Straight binary 1h = Twos complement
15-9	RESERVED	R/W	0h	
8	RES	R/W	0h	These bits define the DAC output voltage resolution. 0h = 8-bits resolution 1h = 12-bit resolution
7-1	RESERVED	R/W	0h	
0	ENABLE	R/W	0h	This bit enables the DAC module. 0h = DAC is disabled 1h = DAC is enabled

14.3.21 CTL1 (Offset = 1110h) [Reset = 00000000h]

CTL1 is shown in [Figure 14-28](#) and described in [Table 14-29](#).

Return to the [Summary Table](#).

Control 1 register.

Figure 14-28. CTL1

31	30	29	28	27	26	25	24
RESERVED							OPS
R/W-0h							R/W-0h
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED						REFSN	REFSP
R/W-0h						R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
RESERVED						AMPHIZ	AMPEN
R/W-0h						R/W-0h	R/W-0h

Table 14-29. CTL1 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	R/W	0h	
24	OPS	R/W	0h	These bits select the DAC output on device pin. 0h = No connect. Both DAC output switches are open. 1h = OUT0 output is selected
23-10	RESERVED	R/W	0h	
9	REFSN	R/W	0h	This bit selects the DAC voltage reference source + input. 0h = VEREFN pin as VR- 1h = Analog supply (VSSA) as VR-
8	REFSP	R/W	0h	This bit selects the DAC voltage reference source + input. 0h = Analog supply (VDDA) as VR+ 1h = VEREFP pin as VR+
7-2	RESERVED	R/W	0h	
1	AMPHIZ	R/W	0h	AMPHIZ - amplifier output value 0 : amplifier output is high impedance 1 : amplifier output is pulled down to ground 0h = HiZ when disable 1h = dacout pulldown when disable
0	AMPEN	R/W	0h	AMP_EN - output amplifier enabled or disabled 0 : disabled 1 : enabled 0h = disable 1h = enable

14.3.22 CTL2 (Offset = 1120h) [Reset = 00000000h]

CTL2 is shown in [Figure 14-29](#) and described in [Table 14-30](#).

Return to the [Summary Table](#).

Control 2 register.

Figure 14-29. CTL2

31	30	29	28	27	26	25	24
RESERVED							DMATRIGEN
R/W-0h							RH/W-0h
23	22	21	20	19	18	17	16
RESERVED							FIFOTRIGSEL
R/W-0h							R/W-0h
15	14	13	12	11	10	9	8
RESERVED							FIFOTH
R/W-0h							R/W-0h
7	6	5	4	3	2	1	0
RESERVED							FIFOEN
R/W-0h							R/W-0h

Table 14-30. CTL2 Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	R/W	0h	
24	DMATRIGEN	RH/W	0h	This bit enables the DMA trigger generation mechanism. When this bit is set along with FIFOEN, the DMA trigger is generated based on the empty FIFO locations qualified by FIFOTH settings. This bit should be cleared by software to stop further DMA triggers. 0h = DMA trigger generation mechanism is disabled 1h = DMA trigger generation mechanism is enabled
23-18	RESERVED	R/W	0h	
17-16	FIFOTRIGSEL	R/W	0h	These bits select the source for FIFO read trigger. When the selected FIFO read trigger is asserted, the data from FIFO (as indicated by read pointer) is moved into internal DAC data register. 0h = Sample time generator output 1h = Hardware trigger-0 from event fabric 2h = Reserved - unimplemented 3h = Reserved - unimplemented
15-10	RESERVED	R/W	0h	
9-8	FIFOTH	R/W	0h	These bits determine the FIFO threshold. In case of DMA based operation, DAC generates new DMA trigger when the number of empty locations in FIFO match the selected FIFO threshold level. In case of CPU based operation, the FIFO threshold bits are don't care and FIFO level is directly indicated through the respective bits in the RIS register. 0h = One fourth of the FIFO locations are empty 1h = Half of the FIFO locations are empty 2h = Three fourth of the FIFO locations are empty 3h = Reserved value. Defaults to same effect as FIFOTH = 0 (One fourth of the FIFO locations are empty).
7-1	RESERVED	R/W	0h	
0	FIFOEN	R/W	0h	This bit enables the FIFO and the FIFO hardware control state machine. 0h = FIFO is disabled 1h = FIFO is enabled

14.3.23 CTL3 (Offset = 1130h) [Reset = 00000000h]

CTL3 is shown in [Figure 14-30](#) and described in [Table 14-31](#).

Return to the [Summary Table](#).

Control 3 register.

Figure 14-30. CTL3

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED				STIMCONFIG			
R/W-0h				R/W-0h			
7	6	5	4	3	2	1	0
RESERVED							STIMEN
R/W-0h							R/W-0h

Table 14-31. CTL3 Field Descriptions

Bit	Field	Type	Reset	Description
31-12	RESERVED	R/W	0h	
11-8	STIMCONFIG	R/W	0h	These bits are used to configure the trigger rate from the sample time generator. The STIMCONFIG values 10 to 15 are reserved and default to same effect as value 0 (500SPS). 0h = Trigger rate is 500 sps (clock divide value is 4000) 1h = Trigger rate is 1 ksps (clock divide value is 2000) 2h = Trigger rate is 2 ksps (clock divide value is 1000) 3h = Trigger rate is 4 ksps (clock divide value is 500) 4h = Trigger rate is 8 ksps (clock divide value is 250) 5h = Trigger rate is 16 ksps (clock divide value is 125) 6h = Trigger rate is 100 ksps (clock divide value is 20) 7h = Trigger rate is 200 ksps (clock divide value is 10) 8h = Trigger rate is 500 ksps (clock divide value is 4) 9h = Trigger rate is 1 Msps (clock divide value is 2)
7-1	RESERVED	R/W	0h	
0	STIMEN	R/W	0h	This bit enables the sample time generator. 0h = Sample time generator is disabled 1h = Sample time generator is enabled

14.3.24 CALCTL (Offset = 1140h) [Reset = 00000000h]

CALCTL is shown in [Figure 14-31](#) and described in [Table 14-32](#).

Return to the [Summary Table](#).

Calibration control register.

Figure 14-31. CALCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED						CALSEL	CALON
R/W-0h						RH/W-0h	RH/W-0h

Table 14-32. CALCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R/W	0h	
1	CALSEL	RH/W	0h	This bit is used to select between factory trim or self calibration trim. 0h (R/W) = Factory Trim : Factory Trim Calibration Values are used when calibration is enabled 1h (R/W) = Self Calibration Trim : Self Calibration Trim Values are used when calibration is enabled
0	CALON	RH/W	0h	This bit when set initiates the DAC offset error calibration sequence and is automatically reset when the offset error calibration completes. 0h = Offset error calibration is not active 1h = Initiate offset error calibration or offset error calibration is already in progress

14.3.25 CALDATA (Offset = 1160h) [Reset = 00000000h]

CALDATA is shown in [Figure 14-32](#) and described in [Table 14-33](#).

Return to the [Summary Table](#).

This is the offset error calibration data register.

Figure 14-32. CALDATA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								DATA							
R-0h																								RH-0h							

Table 14-33. CALDATA Field Descriptions

Bit	Field	Type	Reset	Description
31-7	RESERVED	R	0h	
6-0	DATA	RH	0h	DAC offset error calibration data. The DAC offset error calibration data is represented in twos complement format providing a range of –64 to +63. This is read-only bit, reflecting the calibration data. Writing to this register will have no effect, it will not change the calibration value.

14.3.26 DATA0 (Offset = 1200h) [Reset = 00000000h]

DATA0 is shown in [Figure 14-33](#) and described in [Table 14-34](#).

Return to the [Summary Table](#).

Data 0 register. This register can be written with one 8-bit or one 12-bit digital input data.

Figure 14-33. DATA0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												DATA_VALUE																			
R/W-0h												R/W-0h																			

Table 14-34. DATA0 Field Descriptions

Bit	Field	Type	Reset	Description
31-12	RESERVED	R/W	0h	
11-0	DATA_VALUE	R/W	0h	This is the data written for digital to analog conversion.

This page intentionally left blank.



The VREF module contains a configurable voltage reference buffer which allows users to supply a stable internal reference to on-board analog peripherals. It also supports bringing in an external reference for applications where higher accuracy is required. This chapter describes the features and operation of the VREF module.

15.1 VREF Overview	788
15.2 VREF Operation	788
15.3 VREF Registers	791

15.1 VREF Overview

The VREF module for the MSPM0Gxx family is a shared voltage reference module which can be leveraged by a variety of on-board analog peripherals. VREF allows users to choose between using an internally generated reference voltage or using an externally provided reference voltage from outside the MCU.

The VREF module features include:

- 1.4V and 2.5V user-selectable internal references
- Support for receiving external reference on VREF+/- device pins
- Internal reference output available on VREF+ device pin
- Sample and hold mode supports VREF operation down to STANDBY operating mode
- Internal reference supports full speed ADC operation

Figure 15-1 shows the block diagram of the VREF module.

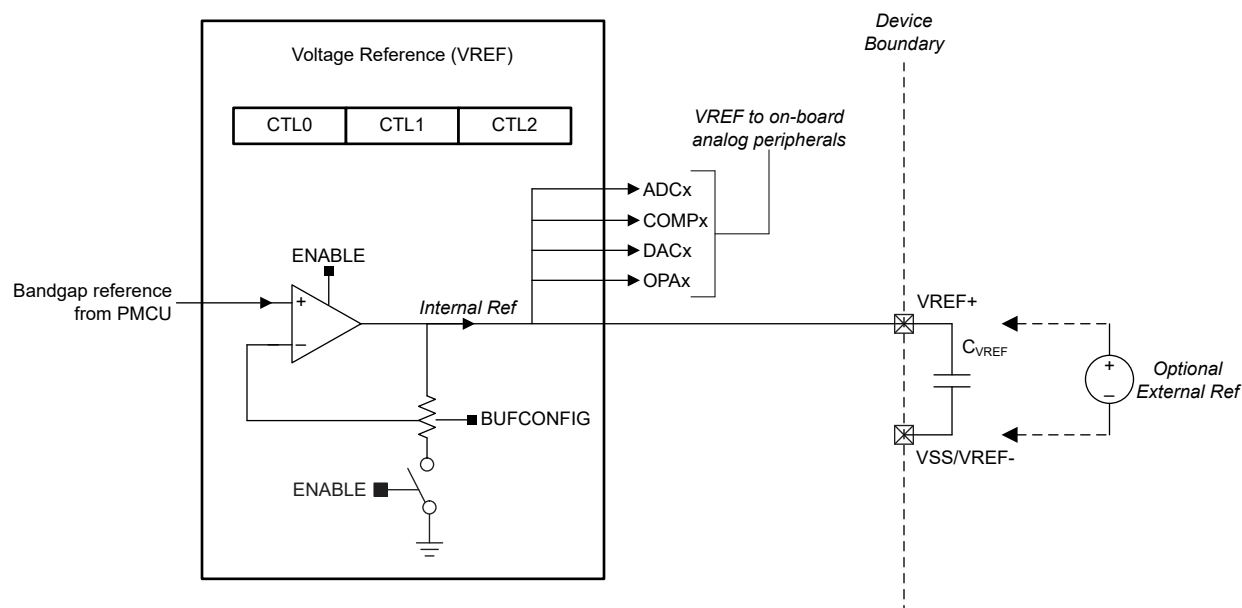


Figure 15-1. VREF Block Diagram

15.2 VREF Operation

The VREF module is configured with user software. The setup and operation of VREF is discussed in the following sections.

15.2.1 Internal Reference Generation

To use VREF to generate an internal voltage reference, the user must first enable the power to the module using the ENABLE control bit in the PWREN register and then enable the reference buffer using the ENABLE control bit in the CTL0 register. The VREF module generates voltage references based on the factory trimmed bandgap coming from the PMU. The bandgap reference is buffered through a non-inverting amplifier to generate one of two internal reference voltages (1.4V or 2.5V). Only one voltage can be selected at a time using the BUFCONFIG control bit in CTL0.

Note

The internal reference generated by VREF requires an external decoupling capacitor (C_{VREF}) for proper operation. Please refer to the device specific data sheet for more information on the value and required placement of C_{VREF} .

After it is enabled and settled, the internal reference can be used as an accurate and stable voltage reference for on-board analog peripherals. Refer to the analog peripheral chapters for more info on how can leverage this reference voltage.

The VREF provides a READY indication bit in the CTL1 register. The first time the VREF is enabled, the READY bit will remain cleared until the VREF is started and settled, after which the READY bit will be set by hardware. If the VREF is disabled, the READY bit will be cleared back to zero by hardware. If the VREF is re-enabled later, the READY bit will not be set, and application software must manage the VREF startup time.

Note

VREF configuration change from 2.5V to 1.4V mode has a very slow slew rate. If this change is needed; then follow below procedure:

- Disable VREF using the ENABLE bit in the CTL0 register
 - Configure pin PA23(VREF+) as GPIO output and drive this pin to logic low for 100us. A 1uF external capacitor is assumed on the VREF+ pin
 - Re-enable VREF in 1.4V mode by setting the BUGCONFIG bit in the CTL0 register to 1. After VREF is enabled and settled, the internal reference of 1.4V can be used
-

15.2.2 External Reference Input

For circumstances where accuracy is a key requirement, an external reference can be brought in to the MCU from the VREF+ and VREF- device pins. Only one type of reference (internal or external) can be selected at a time. When supplying the MCU with an external reference, it is recommended to connect a decoupling capacitor on the reference pins with a value based on the voltage source.

Note

Be sure the VREF reference buffer is disabled by clearing the ENABLE control bit in the CTL0 register before applying the external reference to the device pins.

15.2.3 Analog Peripheral Interface

VREF to ADC

The ADC peripheral can leverage the internal configurable reference or an external reference using the VRSEL control bits in the ADC MEMCTL register.

When using an internal reference as the ADC voltage reference (V_{R+}) the user must ensure that the VREF reference buffer has settled before triggering an ADC conversion. Refer to the device-specific data sheet for VREF settling times and refer to the COMP chapter for more details on all of the reference options available for the ADC peripheral.

VREF to COMP

The Comparator peripheral can leverage the internal configurable reference or an external reference using the REFSRC control bits in the CTL2 register.

Refer to the COMP chapter for more details on all of the reference options available for the Comparator peripheral.

VREF to DAC

The DAC peripheral can leverage the internal configurable reference or an external reference using the REFSP control bits in the DAC CTL1 register.

Refer to PLACEHOLDER FOR DAC REFERENCE CHAPTER for more details on all of the reference options available for the DAC peripheral.

VREF to OPA

The OPA peripheral can leverage the internal configurable reference or an external reference for setting a voltage bias on the non-inverting input terminal of the amplifier using the PSEL control bits in the CFG register. In cases where other devices in a system need to use the same reference as the MCU, the OPA can be used in unity gain buffer configuration to drive the VREF internal reference voltage to the OPA_OUT pin which can be used as a reference for other devices on the PCB.

Refer to the OPA chapter for more details on all of the channels available for the non-inverting amplifier input terminal of the amplifier.

15.3 VREF Registers

[Table 15-1](#) lists the memory-mapped registers for the VREF registers. All register offset addresses not listed in [Table 15-1](#) should be considered as reserved locations and the register contents should not be modified.

Table 15-1. VREF Registers

Offset	Acronym	Register Name	Group	Section
800h	PWREN	Power enable		Go
804h	RSTCTL	Reset Control		Go
814h	STAT	Status Register		Go
1000h	CLKDIV	Clock Divider		Go
1008h	CLKSEL	Clock Selection		Go
1100h	CTL0	Control 0		Go
1104h	CTL1	Control 1		Go
1108h	CTL2	Control 2		Go

Complex bit access types are encoded to fit into small table cells. [Table 15-2](#) shows the codes that are used for access types in this section.

Table 15-2. VREF Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
K	K	Write protected by a key
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value

15.3.1 PWREN (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 15-2](#) and described in [Table 15-3](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 15-2. PWREN

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-							K-0h

Table 15-3. PWREN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	ENABLE	K	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

15.3.2 RSTCTL (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 15-3](#) and described in [Table 15-4](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 15-3. RSTCTL

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-							
15	14	13	12	11	10	9	8
RESERVED							
W-							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCL R	RESETASSERT
W-						WK-0h	WK-0h

Table 15-4. RSTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	W	0h	
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

15.3.3 STAT (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 15-4](#) and described in [Table 15-5](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 15-4. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED							
R-							

Table 15-5. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0h	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

15.3.4 CLKDIV (Offset = 1000h) [Reset = 00000000h]

CLKDIV is shown in [Figure 15-5](#) and described in [Table 15-6](#).

Return to the [Summary Table](#).

Clock Divider

Figure 15-5. CLKDIV

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												RATIO			
R/W-0h												R/W-0h			

Table 15-6. CLKDIV Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R/W	0h	
2-0	RATIO	R/W	0h	Selects divide ratio of module clock to be used in sample and hold logic

15.3.5 CLKSEL (Offset = 1008h) [Reset = 00000000h]

CLKSEL is shown in [Figure 15-6](#) and described in [Table 15-7](#).

Return to the [Summary Table](#).

Clock Selection

Figure 15-6. CLKSEL

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED				BUSCLK_SEL	MFCLK_SEL	LFCLK_SEL	RESERVED
R/W-0h				R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 15-7. CLKSEL Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3	BUSCLK_SEL	R/W	0h	Selects BUSCLK as clock source if enabled
2	MFCLK_SEL	R/W	0h	Selects MFCLK as clock source if enabled
1	LFCLK_SEL	R/W	0h	Selects LFCLK as clock source if enabled
0	RESERVED	R/W	0h	

15.3.6 CTL0 (Offset = 1100h) [Reset = 00000000h]

CTL0 is shown in [Figure 15-7](#) and described in [Table 15-8](#).

Return to the [Summary Table](#).

Control 0 register.

Figure 15-7. CTL0

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							SHMODE
R/W-0h							R/W-0h
7	6	5	4	3	2	1	0
BUFCONFIG	RESERVED					COMP_VREF_ENABLE	ENABLE
R/W-0h	R/W-0h					R/W-0h	R/W-0h

Table 15-8. CTL0 Field Descriptions

Bit	Field	Type	Reset	Description
31-9	RESERVED	R/W	0h	
8	SHMODE	R/W	0h	This bit enable sample and hold mode 0h = Sample and hold mode is disable 1h = Sample and hold mode is enable
7	BUFCONFIG	R/W	0h	These bits configure output buffer. 0h = output 2p5v : Configure Output Buffer to 2.5v 1h = output 1p4v : Configure Output Buffer to 1.4v
6-2	RESERVED	R/W	0h	
1	COMP_VREF_ENABLE	R/W	0h	Comparator Vref Enable 0h = COMP VREF is disabled 1h = COMP VREF is enabled
0	ENABLE	R/W	0h	This bit enables the VREF module. 0h = VREF is disabled 1h = VREF is enabled

15.3.7 CTL1 (Offset = 1104h) [Reset = 00000000h]

CTL1 is shown in [Figure 15-8](#) and described in [Table 15-9](#).

Return to the [Summary Table](#).

Control 1 register.

Figure 15-8. CTL1

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED						READY	
R/W-0h						R-0h	

Table 15-9. CTL1 Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R/W	0h	
0	READY	R	0h	These bits defines status of VREF 0h = VREF output is not ready 1h = VREF output is ready

15.3.8 CTL2 (Offset = 1108h) [Reset = 00000000h]

CTL2 is shown in [Figure 15-9](#) and described in [Table 15-10](#).

Return to the [Summary Table](#).

Control 2 register.

Figure 15-9. CTL2

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
HCYCLE																SHCYCLE															
R/W-0h																R/W-0h															

Table 15-10. CTL2 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	HCYCLE	R/W	0h	Hold cycle count Total cycles of module clock for hold phase when VREF is working in sample and hold mode in STANDBY to save power. Please refer VREF section of data sheet for recommended values of sample and hold times. 0h = smallest hold cycle FFFFh = largest hold cycle
15-0	SHCYCLE	R/W	0h	Sample and Hold cycle count Total cycles of module clock for sample and hold phase when VREF is working in sample and hold mode in STANDBY to save power. This field should be greater than HCYCLE field. The difference between this field and HCYCLE gives the number of cycles of sample phase. Please refer VREF section of data sheet for recommended values of sample and hold times. 0h = smallest sample and hold cycle count FFFFh = largest sample and hold cycle

This page intentionally left blank.



The universal asynchronous receiver-transmitter (UART) module provides an interface for serial communication protocols.

16.1 UART Overview.....	802
16.2 UART Operation.....	804
16.3 UART Registers.....	823

16.1 UART Overview

16.1.1 Purpose of the Peripheral

This interface can be used transfer data between a MSPM0 device and another device with an asynchronous serial communication protocol like LIN (local interconnection network), ISO7816 (Smart card protocol), IrDA (infrared data association), hardware flow control (CTS/RTS) and multiprocessor communications are supported.

16.1.2 Features

The UART controller includes the following features:

- Fully programmable serial interface
 - 5, 6, 7 or 8 data bits
 - Even, odd, stick, or no-parity bit generation and detection
 - 1 or 2 stop bit generation
 - LSB-first or MSB-first data transmit and receive
 - Line-break detection
 - Glitch filter on the input signals
 - Programmable baud-rate generation with oversampling by 16, 8 or 3
- Separated transmit and receive 4 depth FIFOs reduce CPU interrupt service loading
- Supports DMA data transfer
- Standard FIFO-level and End-of-Transmission interrupts
- Active in all low-power mode including stop and standby mode
- Support for waking up SYSOSC via an asynchronous fast clock request upon start bit detection when operating in low power modes (supports up to 19200B rates when using SYSOSC in FCL mode (1% accuracy))
- Support loopback mode operation
- Support hardware flow control
- Support 9-bit multi-drop configuration
- Protocols supported:
 - Local Interconnect Network (LIN) support
 - DALI
 - IrDA
 - ISO7816 Smart card
 - RS485
 - Manchester coding
 - Idle-Line Multiprocessor

Note

This is a general overview, UART extend and UART main differences can be seen in [Table 16-1](#) below. Please refer to device data sheet for the specific UART configuration on UART extend and UART main.

Table 16-1. UART Extend and Main Features⁽¹⁾

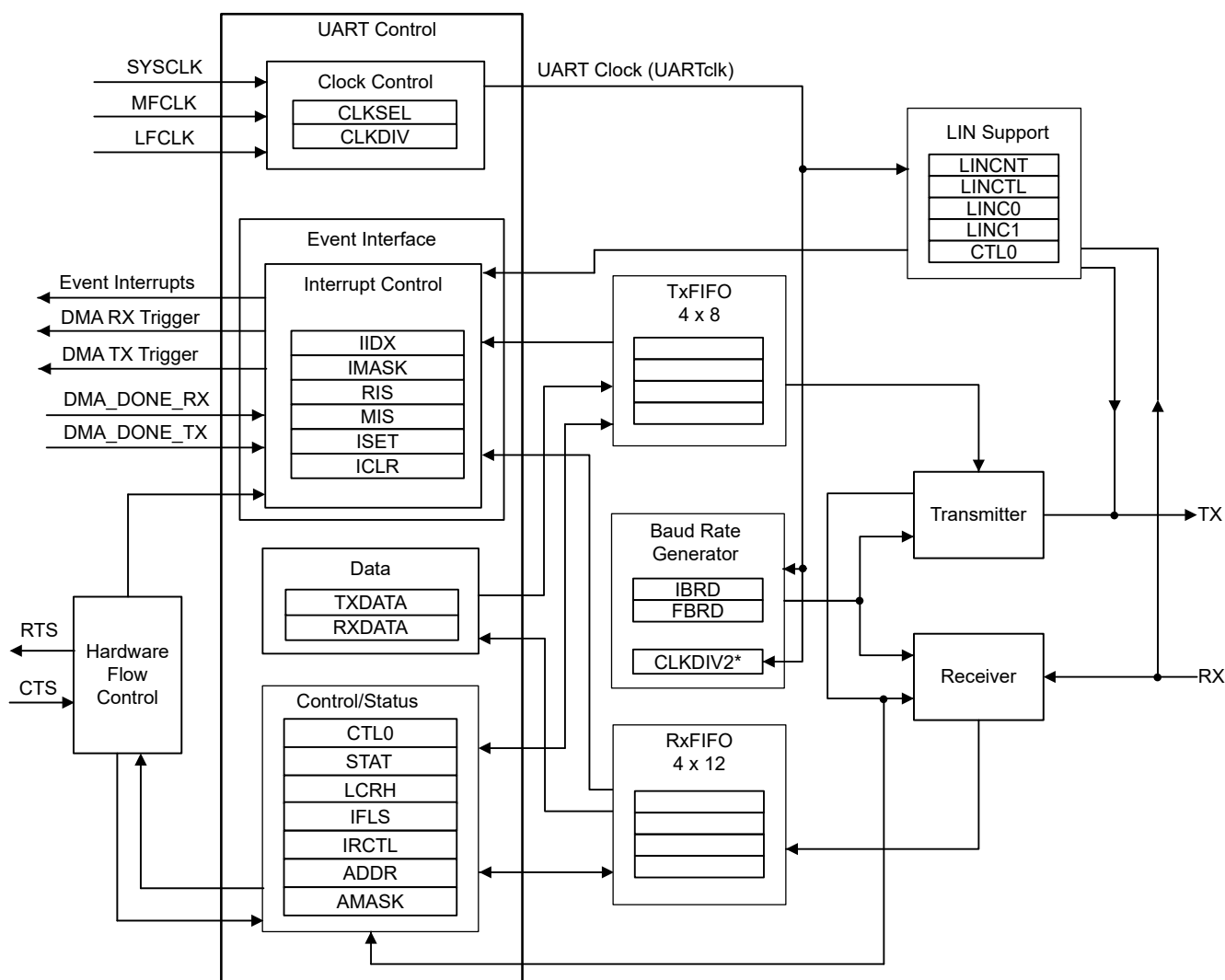
Features	UART Extend	UART Main
Hardware flow control	Yes	Yes
Oversampling options	3, 8, 16	3, 8, 16
Separate transmit and receive FIFOs	Yes	Yes
Active in all low-power modes	Yes ⁽²⁾	Yes ⁽²⁾
Wake-up with start bit	Yes ⁽³⁾	Yes ⁽³⁾
Digital Glitch filter	Yes	No
Analog Glitch filter	Yes	Yes
9-bit multi-drop configuration	Yes	Yes
Idle-Line Multiprocessor	Yes	Yes

Table 16-1. UART Extend and Main Features⁽¹⁾ (continued)

Features	UART Extend	UART Main
RS-485	Yes	Yes
Support LIN mode	Yes	-
Support DALI	Yes	-
Support IrDA	Yes	-
Support ISO7816 Smart card	Yes	-
Support Manchester code	Yes	-

- (1) Refer to the device-specific data sheet for the device-specific configuration of UART extend and UART main modules and their power domains.
- (2) UART can be active in all low-power modes including stop and standby unless the UART instance is in power domain 1 (PD1).
- (3) Only for UART instance in power domain 0 (PD0).

16.1.3 Functional Block Diagram



*CLKDIV2 is for IrDA mode only

Figure 16-1. UART Functional Block Diagram

16.2 UART Operation

This section describes the operation of the UART peripheral.

16.2.1 Clock Control

The UART internal functional clock is selected and divided from the functional clock of the IP.

- Use UARTx.CLKSEL register to select the source of the UART functional clock.
 - BUSSCLK: the current bus clock is selected as the source for UART. The current bus clock depends on power domain. If the UART instance is in power domain 1 (PD1) refer to MCLK, if the UART instance is in power domain 0 (PD0) refer to ULPCLK.
 - MFCLK: MFCLK is selected as the source for UART, refer to MFCLK.
 - LFCLK: LFCLK is selected as the source for UART, refer to LFCLK.
- Use UARTx.CLKDIV register to select the divide ratio of the UART function clock, options are from divide by 1 to 8. For UART Extend, there is a CLKDIV2 register to further divide the UART function clock to support the IrDA mode. When using IrDA mode, CLKDIV2.RATIO must be set to 1h for proper IrDA clocking.

The selected source clock is always available and the frequency depends on the power mode, for more information, see the [Clock Module \(CKM\)](#) section. After enabling the UART module by setting the ENABLE bit, the module will be ready to start receiving and transmitting data.

16.2.2 Signal Descriptions

UART communications require two pins: Receive Data (RX) and Transmit Data (TX):

- RX (Receive Data): RX is the serial data input. Glitch filter and oversampling techniques are used on the receive signal to ensure accurate incoming data.
- TX (Transmit Data): TX is the serial data output. When the transmitter is enabled and no data needs to be transmitted, the TX pin will be held high. In some bidirectional protocols like ISO7816 Smart card, this pin is also used to receive data.

In hardware flow control mode, the following pins are also used:

- CTS (Clear To Send): when driven high by external signal, this signal blocks the data transmission at the end of the current transfer.
- RTS (Request To Send): when low, this signal indicates that the UART is ready to receive data.

16.2.3 General Architecture and Protocol

UART transmits and receives characters at a bit rate that is asynchronous to another device. Timing for each character is based on the selected baud-rate. The transmit and receive functions use the same baud-rate frequency.

In general, control registers should only be programmed when the UART is disabled (ENABLE bit in the UARTx.CTL0 register is cleared). If the UART is operating and gets disabled during transmit or receive operation, the current transaction gets completed before the UART stops. The baud-rate divisor registers (UARTx.IBRD and UARTx.FBRD) can be modified without disabling the UART.

16.2.3.1 Transmit Receive Logic

The transmit logic performs parallel-to-serial conversion on the data read from the transmit FIFO. The control logic outputs the serial bit stream beginning with a start bit and followed by the data bits (LSB first), parity bit, and the stop bits according to the programmed configuration.

The receive logic performs serial-to-parallel conversion on the received bit stream after a valid start pulse has been detected. Overrun, parity, frame error checking, and line-break detection are also performed, and their status accompanies the data that is written to the receive FIFO, this is the reason for 12-bit receive FIFO.

UARTx.CTL0.ENABLE bit is used to enable and disable the UART module, UARTx.CTL0.TXE and RXE bits are used to enable the transmit and receive mode, UARTx.LCRH.WLEN bit is used to configure the number of data

bits transmitted or received in a frame, UARTx.LCRH.PEN is used to enable parity and UARTx.LCRH.STP2 is used to send two stop bits.

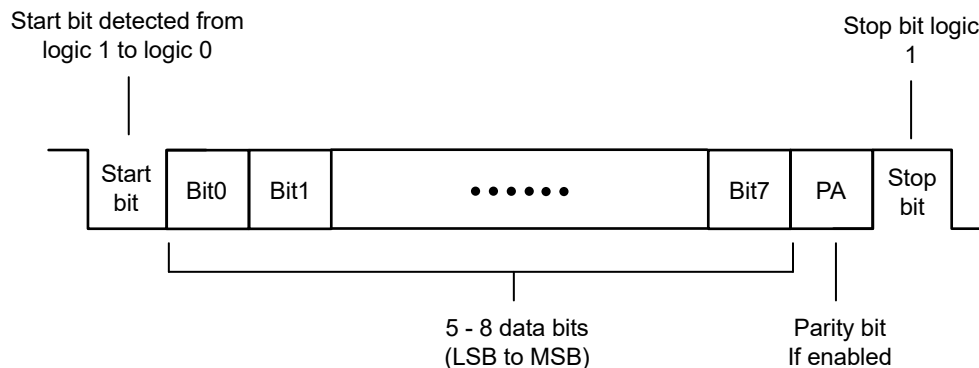


Figure 16-2. UART Character Frame

16.2.3.2 Bit Sampling

By default, UARTx.CTL0.HSE is set to 0 and 16 oversampling is selected and receiving bits are expected to have the length of 16 UART clock UARTclk cycles and is sampled on the 8th UARTclk cycle.

Setting the UARTx.CTL0.HSE bit to 1 selects the 8 oversampling where the receiving bits are expected to have the length of 8 UART clock UARTclk cycles and is sampled on the 4th UARTclk cycle.

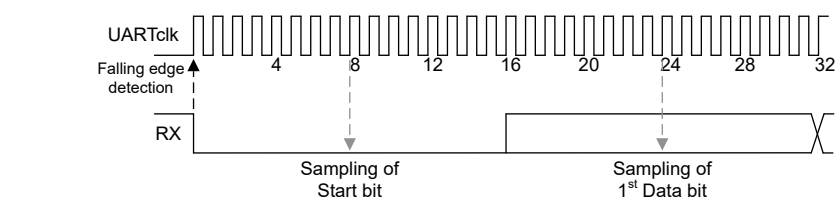
Setting the UARTx.CTL0.HSE bit to 2 selects the 3 oversampling, the receiving bits are expected to have the length of 3 UART clock UARTclk cycles and are sampled on the 2nd UARTclk cycle.

The aforementioned scenarios assume an IBRD =1 and FBRD =0.

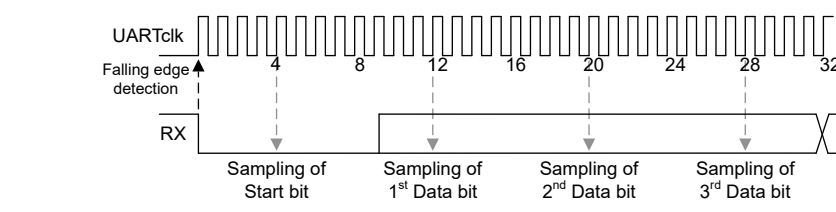
Depending on the application:

- Select oversampling by 3 or 8 to achieve higher speed with UARTclk/8 or UARTclk/3. In this case the receiver tolerance to clock deviation is reduced.
- Select oversampling by 16 to increase the tolerance of the receiver to clock deviations. The maximum speed is limited to UARTclk/16.

16x oversampling mode (HSE = 0)



8x oversampling mode (HSE = 1)



3x oversampling mode (HSE = 2)

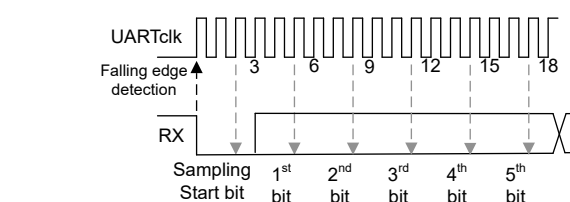


Figure 16-3. UART Oversampling mode

16.2.3.3 Majority Voting Feature

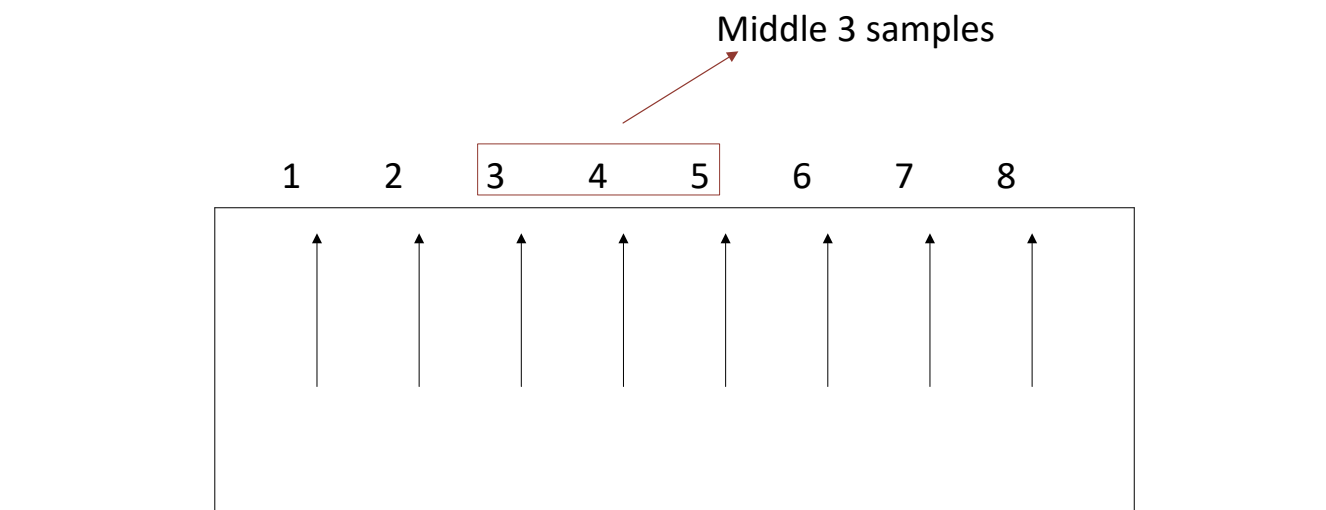


Figure 16-4. Majority voting for 8 oversampling

The majority voting feature of the UART provides noise immunity by sampling each bit 3 times in the center of the bit period. For example, in the case when the UARTx.CTL0.HSE is set to 0 with 16 oversampling; the 7th, 8th and 9th bit are sampled and the majority value is considered as final value to be sampled. When the UARTx.CTL0.HSE is set to 1 with 8 oversampling; then the 3rd, 4th and 5th bits are sampled and majority value is considered as final value to be sampled. The oversampling is only applicable for 16 and 8 oversampling.

When the 3 samples used for majority vote are not equal; the RIS.NE (noise error bit) is set. The received data is transferred inspite of the noise error. The NE bit will get appended to the received data before storing it in the RXDATA register at bit position 12. Please note that the majority voting feature is implemented only for data bits. One can select majority voting or single sample using the MAJVOTE control bit in the CTL0 register.

Note

Even though UART instances have noise filters, this feature provides an extra layer of noise immunity for longer glitches

16.2.3.4 Baud Rate Generation

The baud-rate divisor is a 22-bit number consisting of a 16-bit integer and a 6-bit fractional part. The number formed by these two values is used by the baud-rate generator to determine the bit sample period. Having a fractional baud-rate divisor allows the UART to generate all of the standard baud-rates very accurately

The 16-bit integer is loaded through the UART Integer Baud-Rate Divisor UARTx.IBRD register and the 6-bit fractional part is loaded with the UART Fractional Baud-Rate Divisor UARTx.FBRD register.

The baud-rate divisor can be calculated by using the following formula:

$$\text{BRD} = \text{UART Clock} / (\text{Oversampling} \times \text{Baud rate}) \quad (23)$$

UART clock is the clock output of the UART clock control logic, configured by CLKSEL and CLKDIV. Oversampling is selected by the HSE bit in the UARTx.CTL0 register and it can be 16, 8 or 3.

- UARTx.IBRD = INT(BRD), integer part of the BRD
- UARTx.FBRD = BRD % 64 , fractional part

The integer part of BRD is loaded into UARTx.IBRD register. The 6-bit fractional number must be loaded into the UARTx.FBRD register.

Note

When IBRD = 0, FBRD is ignored and no data gets transferred by the UART. Similarly, when IBRD = 65535 (that is 0xFFFF), then FBRD must not be greater than zero. If this is exceeded it results in an aborted transmission or reception.

The following example shows a simple method to calculate IBRD.DIVINT and FBRD.DIVFRAC for a baud rate of 19200 bit/s:

UART Clock = 40 MHz
Oversampling = 16
Baudrate = 19200 bit/s

$$\text{BRD} = \frac{\text{UARTclk}}{\text{OVS} \times \text{Baudrate}} = \frac{40\text{MHz}}{16 \times 19200 \text{ bit/s}} = 130.2083333$$

↑
UARTx.IBRD.DIVINT= 130 (=82h)

↓
UARTx.FBRD.DIVFRAC
= INT((.2083333 x 64) + 0.5)
= INT(13.833333)
= 13d (= Dh)

Note: The adder '+0.5' ensures rounding to the closest integer value to keep the rounding error as small as possible

Figure 16-5. Baud Rate Configuration

When updating the baud-rate divisor (UARTx.IBRD or UARTx.IFRD), the UART.LCRH register must also be written, so any changes to the baud-rate divisor must be followed by a write to the LCRH register for the changes to take effect. The contents of the UART.IBRD and UART.FBRD registers are not updated until transmission or reception of the current character is complete.

16.2.3.5 Data Transmission

Data received or transmitted is stored in two FIFOs, though the receive FIFO has an extra four bits per character for status information.

Transmit data:

For transmission, data is written into the transmit FIFO. If the UART is enabled, it causes a data frame to start transmitting with the parameters indicated in the UARTx.LCRH register. Data continues to be transmitted until there is no data left in the transmit FIFO. The BUSY bit in the UARTx.STAT register is asserted as soon as data is written to the transmit FIFO (that is, if the FIFO is non-empty) and remains asserted while data is being transmitted. The BUSY bit is negated only when the transmit FIFO is empty, and the last character has been transmitted to the shift register, including the stop bits. The UART can indicate that it is busy even though the UART can no longer be enabled. BUSY also is set during the generation of a BREAK signal.

Receive data:

When the receiver is idle (the RX signal is continuously 1), and the data input goes low (a start bit has been received), the receive counter begins running and data is sampled on the different cycle based on the oversampling setting of the HSE bit in UARTx.CTL0 register. The start bit is valid and recognized if the RX signal is still low after certain number for cycles based on the oversampling setting. After a valid start bit is detected, successive data bits are sampled according to the programmed length of the data characters. The parity bit is then checked if parity mode is enabled. Data length and parity are defined in the UART.LCRH register. Oversampling is explained in [Section 16.2.3.2](#).

Lastly, a valid stop bit is confirmed if the RXD signal is high, otherwise a framing error has occurred. When a full word is received, the data is stored in the receive FIFO along with any error bits associated with that word.

16.2.3.6 Error and Status

For received data, the data byte and the 4-bit status (break, frame, parity, and overrun) is pushed onto the 12-bit wide receive FIFO. The error and status can be retrieved by reading UARTx.RXDATA register as showing in [Table 16-2](#).

Table 16-2. UART Error and Conditions

Error Condition ⁽¹⁾	Bit Field	Description
Framing error	FRMERR	A framing error occurs when a low stop bit is detected. When two stop bits are used, both stop bits are checked for framing error. When a framing error is detected, the FRMERR bit is set.
Parity error	PARERR	A parity error is a mismatch between the number of 1s in a character and the value of the parity bit. When an address bit is included in the character, it is included in the parity calculation. When a parity error is detected, the PARERR bit is set.
Receive overrun	OVRERR	An overrun error occurs when a character is loaded into RXDATA/FIFO before the prior character has been read. When an overrun occurs, the OVRERR bit is set.
Break condition	BRKERR	A break is detected when all received data, parity, and stop bits are 0. When a break condition is detected, the BRKERR bit is set. A break condition can also set the interrupt flag RXINT if the break interrupt enable IMASK.BRKERR bit is set.

(1) Framing error and break condition are not set when LIN mode is enabled, this pattern is used to signal a Sync frame for LIN. Break detection is not available for IRDA, Manchester and Dali mode.

UART module flag status can also be checked by reading UARTx.STAT register as showing in [Table 16-3](#).

Table 16-3. UART Flag Status

Bit Field	Description
BUSY	This bit is set as soon as the transmit FIFO becomes non-empty (regardless of whether UART is enabled). In IDLE_Line mode the busy signal also stays set during the idle time generation.
RXFE	This bit is set when receive FIFO is empty. If the FIFO is disabled (FEN is 0), the receive holding register is full. If the FIFO is enabled (FEN is 1), the receive FIFO is full.
RXFF	This bit is set when receive FIFO is full. If the FIFO is disabled (FEN is 0), the receive holding register is empty. If the FIFO is enabled (FEN is 1), the receive FIFO is empty.
TXFE	This bit is set when transmit FIFO is empty. If the FIFO is disabled (FEN is 0), the transmit holding register is full. If the FIFO is enabled (FEN is 1), the transmit FIFO is full.
TXFF	This bit is set when transmit FIFO is full. If the FIFO is disabled (FEN is 0), the transmit holding register is empty. If the FIFO is enabled (FEN is 1), the transmit FIFO is empty.
CTS	This bit is set when CTS signal is asserted (low) and cleared when CTS signal is not asserted (high).
IDLE	IDLE mode has been detected in idle line multiprocessor mode. The IDLE bit is used as an address tag for each block of characters. In idle line multiprocessor format, this bit is set when a received character is an address.

16.2.3.7 Local Interconnect Network (LIN) Support

This section is only relevant for UART Extend, which supports LIN mode. Refer to the device data sheet for the device-specific configuration of UART extend and UART main.

For supporting local interconnect network (LIN) protocol, the following hardware enhancements are implemented in the UART module:

- 16 bit up-counter (LINCNT) clocked by the UART clock.
- Interrupt capability on counter overflow (CPU_INT.IMASK.LINOVF).
- 16 bit capture register (LINC0) with two configurable modes
 - Capture of LINCNT value on RXD falling edge. Interrupt capability on capture.
 - Compare of LINCNT with interrupt capability on match.
- 16 bit capture register (LINC1) can be configured:
 - Capture LINCNT value on RXD rising edge. Interrupt capability on capture.

LIN transmission

Sending the break signal can be done by setting the BRK bit in UARTx.LCRH register. This bit needs to be set before the data is written into the FIFO or transmit data register TXDATA.

To generate LIN responder signals such as wake signals, the TX pin can be configured by TXD_OUT and TXD_CTL_EN bits in register UARTx.CTL0 to be software controlled. By setting TXD_CTL_EN bit to 1, the TX output pin can be controlled by the TXD_OUT bit if the UART transmit is disabled (CTL0.TXE is cleared).

LIN reception

LIN commander issues a break field and sync field at the start of every frame. Hardware must be added such that the LIN responder software driver can reasonably detect BREAK-SYNC and measure the necessary timing parameters to adjust the baud rate or determine an error.

For LIN reception, break field detection and compare mode are needed. To configure these features:

1. Initialize LIN counter to 0 (UARTx.LINCNT = 0)
2. Enable counter compare match mode (UARTx.LINCTL.LINC0_MATCH = 1)
3. Load UARTx.LINC0 (counter capture 0 register) with counter value corresponding to $9.5 \times T_{\text{bit}}$ (refer to LIN Specification for details on this timing).
4. Enable LINC0 match interrupt (CPU_INT.IMASK.LINC0 = 1)
5. Setup LIN count control (UARTx.LINCTL):
 - Enable count while low signal on RXD (LINCTL.CNTRXLOW = 1)
 - Enable LIN counter clearing on RXD falling edge (LINCTL.ZERONE = 1)
 - Enable LIN counter (LINCTL.CTRENA = 1)

Optional:

- User can enable the rising edge on UART TX signal interrupt (CPU_INT.IMASK.RXPE = 1), when the RXPE interrupt fires, the software can read the LINCTR directly to see the BREAK field timing.
- User can enable the LIN counter overflow interrupt (CPU_INT.IMASK.LINOVF = 1) to detect the BREAK field is too long and overflows 16bit counter. The timeout can be calculated as $t_{\text{Timeout}} = 2^{16} / \text{UART clock}$.

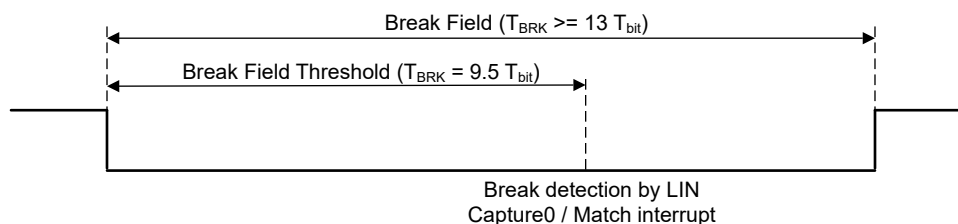


Figure 16-6. LIN Break Field Detection

Sync field validation is required to ensure accuracy of this LIN header part and to calculate the commander baud rate. The synch field consists of the data 0x55 inside a byte field (see Figure 16-7). After software detects a valid BREAK, it can then set the counter to measure the SYNC field. Both capture registers in LIN counter are used so that software sees fewer interrupts. Figure 16-7 shows the SYNC byte format. The LINCTR should be set to 0 on the start bit falling edge and count continuously. The LINC0 capture or RX falling edge interrupts fire at the falling edges of the RX line. During the interrupt processing, software can measure the individual HIGH-LOW times of the bits themselves using the values in the LINC0 and LINC1 registers to make sure all of the timings are valid.

The following flow describes a possible LIN sync field validation procedure:

1. Initialize LIN counter to 0 (UARTx.LINCNT = 0) after detecting a valid break field.
2. Enable interrupt on RX falling edge (CPU_INT.IMASK.RXNE = 1)
3. Setup LIN count control (LINCTL):
 - Enable LIN counter capture on raising RX edge (LINCTL.LINC1CAP = 1)
 - Enable LIN counter capture on falling RX edge (LINCTL.LINC0CAP = 1)
 - Enable LIN counter clearing on RX falling edge (LINCTL.ZERONE = 1)
 - Enable LIN counter (LINCTL.CTRENA = 1)

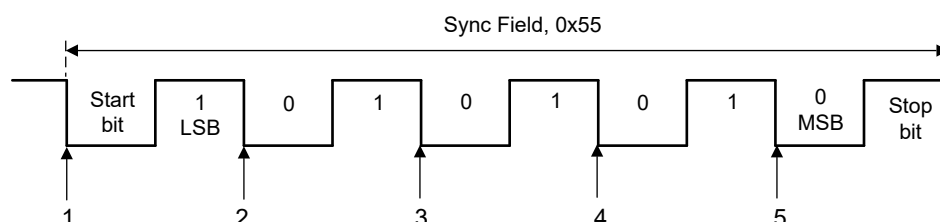


Figure 16-7. LIN SYNC Field Detection

Actions at each falling edge of the RX line for the sync field as showing below:

1. LIN counter is set to 0 and start counting on the falling RX edge. (LINCTL.ZERONE = 1)
2. RX falling edge interrupt trigger (RXNE):
 - Read capture register LINC0 (falling edge) and LINC1 (rising edge) values
 - Verify bit times
3. RX falling edge interrupt trigger (RXNE):
 - Read capture register LINC0 (falling edge) and LINC1 (rising edge) values
 - Verify bit times
4. RX falling edge interrupt trigger (RXNE):
 - Read capture register LINC0 (falling edge) and LINC1 (rising edge) values
 - Verify bit times
5. RX falling edge interrupt trigger (RXNE):
 - Read capture register LINC0 (falling edge) and LINC1 (rising edge) values

- Verify bit times
- Calculate the proper baud rate to set. Software must set the baud rate before the start bit of the PID field after sync field.

On each interrupt occurrence, the capture registers must be read and the bit times need to be validated by the application software. In case of a bit time verification error, the sync field analysis process must be aborted and the application software must switch back to break detection.

In case of errors like a breaking commander communication during sync field detection, a timeout interrupt can be generated by enabling the LIN counter overflow (IMASK.LINOVF = 1). When the counter overflows, the interrupt handler can abort the sync field analysis and switch back to break detection. The time the counter overflow interrupt occurs can be calculated as $t_{\text{Timeout}} = 2^{16} / \text{UART clock}$.

Note

The sync field is automatically stored in the RX FIFO and can be misread as the PID. Therefore, flush the RX FIFO after the sync field is received and before the PID is received.

16.2.3.7.1 LIN Responder Transmission Delay

The interrupt RXINT for starting the transmission on the responder line always occurs at the mid-point of the commander STOP bit. Depending on the BUSCLK and baud-rate; there might not be enough response space between the STOP bit of commander and START bit of the responder; and the data transmission may start before end of STOP bit.

To overcome this, a delay of half the STOP bit period can be added as response space time to make sure that there is sufficient delay time between STOP and START bit, before start of responder data transmission.

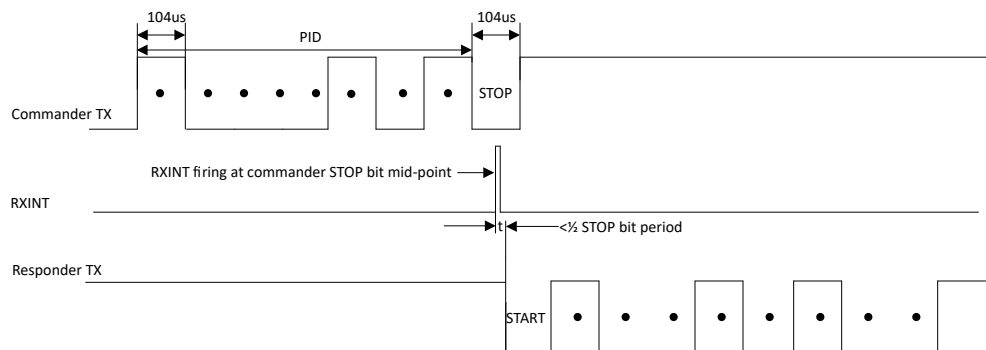


Figure 16-8. LIN Responder Transmission Delay

16.2.3.8 Flow Control

Flow control can be accomplished by hardware and the following sections describe the implementation method.

In UART mode (CTL0.MODE set to 0), hardware flow control between two devices is accomplished by connecting the RTS output to the CTS input on the receiving device, and connecting the RTS output on the receiving device to the CTS input. The RTS output signal is low active, the CTS input expects a low signal on a send request as shown in Figure 16-9.

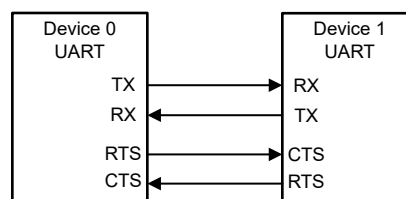


Figure 16-9. Flow Control

The CTS input controls the transmitter, the Device 0 and Device 1 transmitter can only transmit data when their CTS input is asserted low. When RTS flow control is enabled, the RTS output signal indicates the state of the receive FIFO. For example, the CTS of the Device 1 remains asserted low until the preprogrammed RX FIFO level of Device 0 is reached, indicating that the receive FIFO of Device 0 has no space to store additional characters.

The CTSEN and RTSEN bits in the UART.CTL0 register specify the flow control mode as shown in [Table 16-4](#).

Table 16-4. Flow Control Enable

CTSEN	RTSEN	Description
1	1	RTS and CTS flow control enabled
1	0	Only CTS flow control enabled
0	1	Only RTS flow control enabled
0	0	Both RTS and CTS flow control disabled

When RTSEN is set to 1, the value of the CTL0.RTS bit is ignored and the RTS output signal is generated by the hardware trigger levels as described below. When RTSEN bit is cleared, the RTS signal output is controlled by the CTL0.RTS bit for SW control.

RTS flow control:

The RTS flow control logic is linked to the programmable receive FIFO watermark levels, it can be configured using UARTx.IFLS register. When RTS flow control is enabled, the RTS is asserted (low) until the receive FIFO is filled up to the watermark level. When the receive FIFO watermark level is reached, the RTS signal is de-asserted (high), indicating that there is no more room to receive any more data. The transmission of data is expected to cease after the current character has been transmitted. The RTS signal is reasserted (low) when data has been read out of the receive FIFO so that it is filled to less than the watermark level. If RTS flow control is disabled and the UART is still enabled, then data is received until the receive FIFO is full, or no more data is transmitted to it.

As the RTS signal is de-asserted when the FIFO watermark level is reached by putting the last received character into the FIFO. This means on a back to back transmit another character transfer could already been started by the sender. Therefore, in such cases the watermark level should be set to one level lower to ensure all data can be received and put into the FIFO.

CTS flow control:

If CTS flow control is enabled, then the transmitter checks the CTS signal before transmitting the next byte. If the CTS signal is asserted (low), it transmits the byte otherwise transmission does not occur. The data continues to be transmitted while CTS is asserted (low), and the transmit FIFO is not empty. If the transmit FIFO is empty and the CTS signal is asserted (low) no data is transmitted. If the CTS signal is de-asserted (high) and CTS flow control is enabled, then the current character transmission is completed before stopping. If CTS flow control is disabled and the UART is enabled, then the data continues to be transmitted until the transmit FIFO is empty.

16.2.3.9 Idle-Line Multiprocessor

When IDLELINE is set in the CTL0.MODE register bits, the idle-line multiprocessor format is selected. Blocks of data are separated by an idle time on the transmit or receive lines ([Figure 16-10](#)). An idle receive line is detected when ten or more continuous ones (marks) are received after the one or two stop bits of a character. The baud-rate generator is switched off after reception of an idle line until the next start edge is detected. When an idle line is detected, the IDLE bit in UARTx.STAT is set. In Idle-Line mode the UART receiver operates in no parity mode and the UART word length (UARTx.LCRH.WLEN) must be set to 8bit.

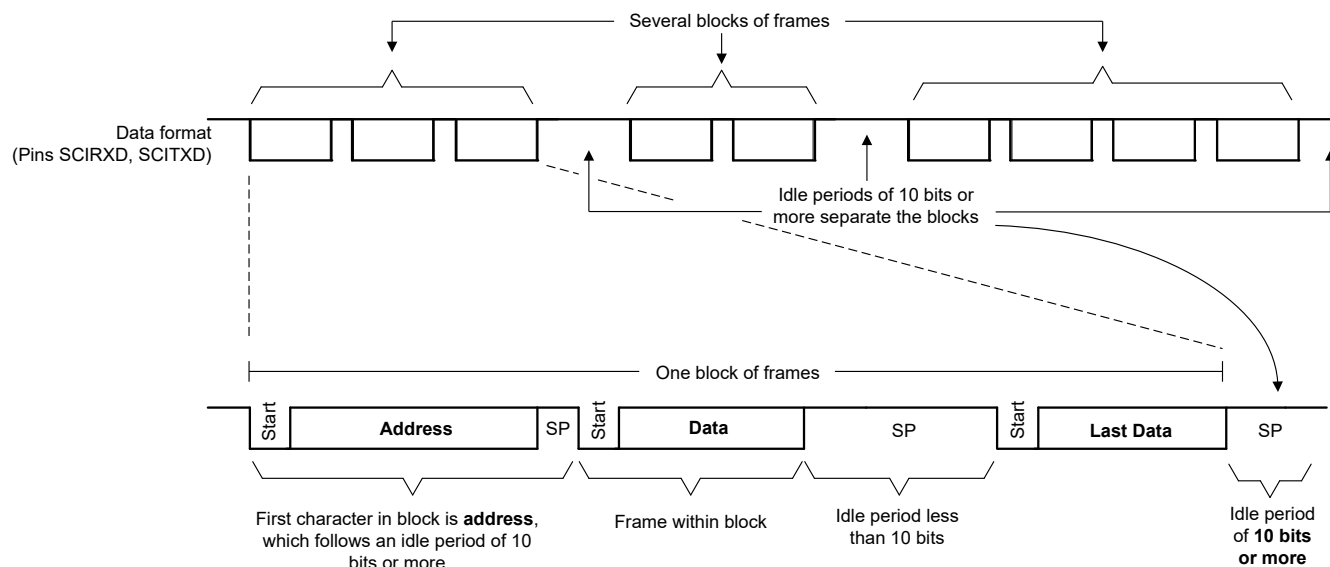


Figure 16-10. Idle-Line Multiprocessor

The first character received after an idle period is an address character. The IDLE bit in UARTx.STAT register is used as an address tag for each block of characters. In idle-line multiprocessor format, this bit is set when a received character is an address.

If an address character is received it is compared against the ADDR register with the AMASK applied. If the received character matches, the address character and all following received characters are transferred into the RXDATA buffer and interrupts/flags are generated until the next address without a match is received. The IDLE bit in UARTx.STAT register is automatically cleared when the address does match; otherwise the IDLE bit is set until the address is matched.

When the SENDIDLE bit in UARTx.LCRH register is set the UART inserts an idle period of 11 bit times on the bus, an ongoing transfer is finished first. The next transfer will be delayed till this idle period has finished. Then the next transfer can start with an address character.

The following procedure sends out an idle frame to indicate an address character followed by associated data:

1. Set SENDIDLE then write the address character to TXDATA. TXDATA must be ready for new data (TXINT interrupt must be 1). This generates an idle period of exactly 11 bits followed by the address character. SENDIDLE is reset automatically when the address character has been transferred (all bits are sent out of shift register).
2. Write desired data characters to TXDATA. TXDATA must be ready for new data (TXINT interrupt must be 1). The data written to TXDATA is transferred to the shift register and transmitted as soon as the shift register is ready for new data.

The idle-line time must not be exceeded between address and data transmission or between data transmissions. Otherwise, the transmitted data is misinterpreted as an address.

BUSY bit in IDLELINE mode:

- TX: BUSY is set during the generation of an IDLELINE signal and while the address and data bytes are sent
- RX: The BUSY signal set during the receive of data and till first 10 idle bits are received.

16.2.3.10 9-Bit UART Mode

9-bit mode is enabled by setting MODE bit to ADDR9BIT in the UARTx.CTL0 register. This feature is useful in a multi-drop configuration of the UART where a single controller connected to multiple peripherals can communicate with a particular peripheral through its address or set of addresses along with a qualifier for an address byte. In 9-bit UART mode, the parity enable/mode bits are ignored and the UART word length (UARTx.LCRH.WLEN) must be set to 8 bit.

Receive Transaction:

In 9-bit mode, a peripheral checks for the address qualifier at the location of the parity bit. If set, the received byte is compared with the preprogrammed address in UARTx.ADDR register:

- If the address matches, a 9-bit mode address match interrupt (ADDR_MATCH) is generated, if enabled and further data get received.
- If the address does not match, the address byte and the subsequent data bytes get dropped. .

The address can be predefined in the UART.ADDR register to match with the received byte. The matching can be extended to a set of addresses using the address mask in the UART.AMASK register. By default, the UART.AMASK is 0xFF, meaning that only the specified address must match

Transmit Transaction:

All the send transactions in 9-bit UART mode are interpreted as:

- Address bytes, if the 9th bit is set
- Data bytes, if the 9th bit is cleared

In 9-bit mode, the 9th bit can be controlled by software. The EPS bit setting of the LCRH register reflects the 9th bit for transmit transactions. To indicate an address byte, the software must set the EPS bit before the byte transmission. For data byte transmissions, the EPS bit must be cleared before the byte transmission. For a complete transmit transaction, the address byte must be transmitted as a single byte transaction with EPS bit set, followed by a data byte burst with EPS bit cleared.

9th bit handling:

Table 16-5. 9th Bit Handling

PEN	SPS	EPS	9 th Bit (Transmitted or Verified)
0	X	X	Not transmitted or verified
1	1	0	0 (= Data)
1	1	1	1 (= Address)

16.2.3.11 RS485 Support

RS485 is a standard used in serial communications systems. This standard can be used effectively over long distances and in electrically noisy environments. Multiple receivers can be connected to such a network. These characteristics make RS-485 useful in industrial control systems and similar applications.

With the RS485 direction signal an external RS485 PHY can be controlled. The RTS I/O is used in this mode for the direction signal. The signal is set automatically to high once a data transmit is started. It will stay set between bytes if they are sent back to back. If a data receive is ongoing a new transmit should be delayed till this data has been received and the direction signal has been set to transmit.

Data exchange sequence as show in [Figure 16-11](#):

- Wait till an ongoing receive has been finished.
- Activate direction signal to transmit on RTS Pin
- Send data (one or more Bytes)
- Wait till an ongoing receive has been finished.
- Deactivate direction signal to transmit on RTS Pin

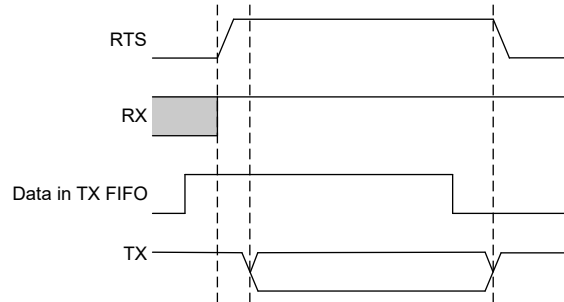


Figure 16-11. RS485 Data Exchange

Two bit fields to the LCRH register define the setup and hold time of the external driver direction control:

- EXTDIR_SETUP bits defines the number of UART clock ticks the signal to control the external driver for the RS485 will be set before the START bit is send. The generated setup time will be between EXTDIR_SETUP value and EXTDIR_SETUP + one baud rate cycle
- EXTDIR_HOLD bits defines the number of UART clock ticks the signal to control the external driver for the RS485 will be reset after the beginning of the stop bit. (If 2 STOP bits are enabled the beginning of the 2nd STOP bit.)

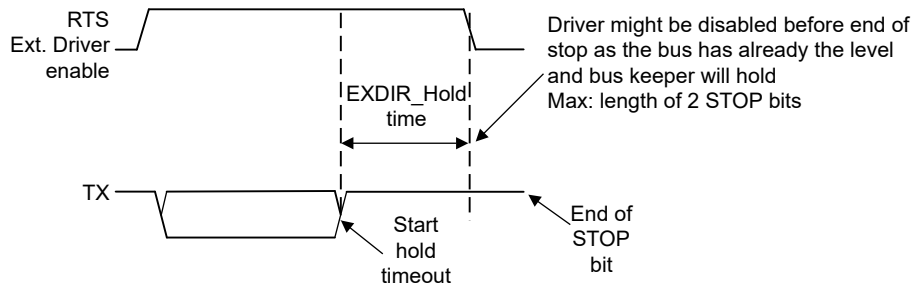


Figure 16-12. RS485 External Control

16.2.3.12 DALI Protocol

DALI stands for Digital Addressable Lighting Interface. It is an International Standard (IEC 62386) lighting control system, providing a single interface for all electronic control gear (light sources) and electronic control devices (lighting controllers).

The UART module supports the low level DALI Protocol sending and receiving the bit streams for forward and backward frames. The timing between any forward and backward frame sequence needs to be handled and checked by software.

Transmitting a forward or backward frame:

When transmitting a forward frame user needs to ensure to write second byte to buffer before first byte has been shifted out. The hardware will then send the two bytes without inserting the stop bits in-between. Otherwise the stop bits will be sent and the data will be handled like a backward frame.

Receiving data:

The UART module in DALI mode will check the 9th bit after the start bit to detect a Forward frame or backward frame. If this bit does have a change of the phase (= no stop bit) a forward frame is detected and:

- Address compare is done in software or hardware depending on MASK
- Address and data are always transferred into the RX Buffer

Otherwise a backward frame is detected and the data is transferred into the RX Buffer without setting the ADDR_MATCH interrupt.

The AMASK register can be used as group assignment used during multicast operation with the MSB to indicate if the device is part of a DALI group. To enable the UART in DALI mode to respond on all ADDRESS the AMASK register needs to be cleared.

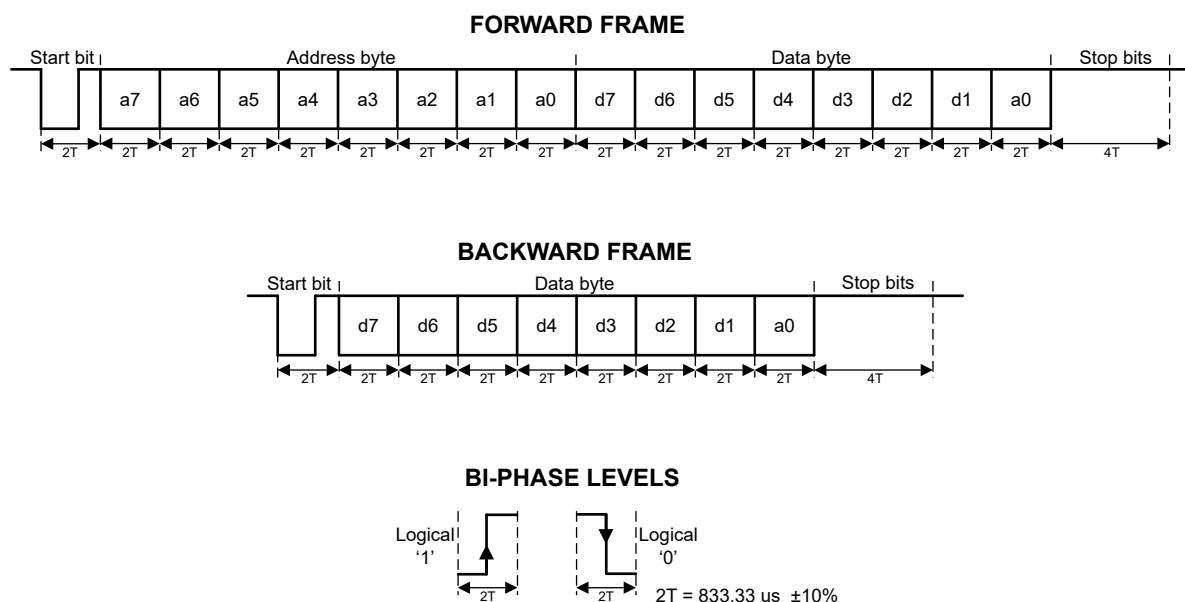


Figure 16-13. DALI Protocol

The limits for the times T shall be: $334 \mu s < T < 500 \mu s$ according to standard IEC 62386-102. DALI mode requires Manchester encoding to be enabled. When using DALI mode (CTL0.MODE set to DALI), the CTL0 and LCRH register must be set to:

- 8-bit word length (WLEN bit)
- No parity / 2 Stop Bit
- Manchester encoding enabled
- Baud-rate configuration set to match $2T = 833.33 \mu s$
- DALI mode requires the FIFO to be enabled

16.2.3.13 Manchester Encoding and Decoding

UART provides option to receive and transmit Manchester encoded data. The function is enabled by the MENC bit in CTL0 register to generate the IEEE 802.3 compliant waveform. With the invert function in GPIO control module the output signals can be inverted to generate the G. E. Thomas compliant waveform.

The output signal is generated by XORing the data with UART clock signal. The UART clock needs to be double the speed of the baud-rate. So for the data transmit there is an edge at the beginning and the middle of each data bit. For the receive signal the edge in the middle of the bit is detected to decode the RX data.

16.2.3.14 IrDA Encoding and Decoding

When IREN bit in UARTx.IRCTL register is set, the IrDA encoder and decoder are enabled and provide hardware bit shaping for IrDA communication. IrDA en/decoding should only be used with UART mode (UARTx.CTL0.MODE is 0)

IrDA Encoding

The encoder sends a pulse for every zero bit in the transmit bit stream coming from the UART (see [Figure 16-14](#)). The pulse duration is defined by IRTXPL bits specifying the number of one-half clock periods of the clock selected by IRTXCLK bit.

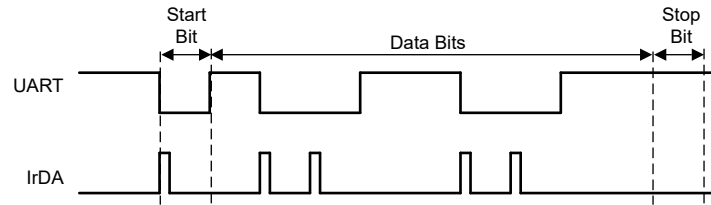


Figure 16-14. IrDA Protocol

(more information will be added in next revision)

IrDA Decoding

The decoder detects high pulses when IRRXPL = 0. Otherwise, it detects low pulses.

A programmable digital filter stage can be enabled by setting UARTx.GFCTL.DGFSEL > 0. When IRCTL.IREN is set, also the digital glitch filter should be set so that only pulses longer than the programmed filter length are passed and shorter pulses are discarded. (See the Glitch Suppression chapter on how to set the filter.)

16.2.3.15 ISO7816 Smart Card Support

The UART offers basic support to allow communication with an ISO7816 smart card. When configuring the MODE bits to smart card (0x4) of the UARTx.CTL0 register, the TXD signal is used as a bit clock, and the RXD signal is used as the half-duplex communication line connected to the smart card. Further smart card signals are not supported by the UART.

The clock rate of the UART clock in ISO7816 mode must be in the range of 1 MHz to 5 MHz when using ISO7816 mode, the UARTx.LCRH register must be set to:

- 8-bit word length (WLEN bits configured to 0x3)
- Even parity (PEN set and EPS set)
- No stick parity (SPS cleared)

In ISO7816 mode, the UART automatically uses 2 stop bits; therefore the STP2 bit of the LCRH register is ignored.

If a parity error is detected during a transmission, RXD is pulled low during the second stop bit. In this case, the UART aborts the transmission, it flushes the transmit FIFO and discards any data it contains. Additionally it raises a parity error interrupt, allowing the software to detect the problem and initiate retransmission of the affected data, as the UART does not support automatic retransmission in this case. The UART does not support automatic retransmission on parity errors. If a parity error is detected on transmission, all further transmit operations are aborted and software must handle retransmission of the affected byte or message.

In Smartcard Mode, the receiver in case of parity error will drive the line low and a parity interrupt flag is asserted. The transmitter responds based on the value of this bit.

16.2.3.16 Address Detection

The UARTx.ADDR register is used to set the specific address that should be matched with receiving address byte. This register is used in conjunction with UARTx.AMASK register to form a match for address-byte received. Only bits where the AMASK is set to '1' are considered. So for full address the AMASK register is set to 0xFF. This feature is used in DALI, UART 9-Bit or Idle-Line mode.

Table 16-6. Address Detection

Condition	DALI Mode	Idle Line Mode	9-Bit Mode
Address match	Address and Data is moved to RXDATA	Address and Data is moved to RXDATA	Address and Data is moved to RXDATA
Address mismatch	Address and Data will be dropped	Address and Data will be dropped	Address and Data will be dropped

16.2.3.17 FIFO Operation

The UART has two FIFOs with a depth of 4 entries, one for transmit and one for receive. The FIFOs are accessed through the UART Data (TXDATA/RXDATA) registers. Read operations of the RXDATA register return a 12-bit value consisting of 8 data bits and 4 error flags. Write operations to TXDATA place 8-bit data in the transmit FIFO.

Out of reset, both FIFOs are disabled and act as 1-byte-deep holding registers. The FIFOs are enabled by setting the FEN bit in UARTx.CTL0. FIFO status can be monitored through the UARTx.STAT register and the interrupt events.

Hardware monitors empty, full and overrun conditions

The UARTx.STAT register contains empty and full flags (TXFE, TXFF, RXFE, and RXFF bits), and the CPU_INT.RIS register shows overrun status of the receive FIFO through the OVRERR bit. There is no indicator for a transmit FIFO overrun. A write is just lost, in case it overruns the transmit FIFO. If the FIFOs are disabled, the empty and full flags are set according to the status of the 1-byte-deep holding registers. When receiving more data than the FIFO can capture the oldest data will be overwritten with the received data.

The trigger point at which the FIFOs generate interrupts is controlled through the UARTx.IFLS register. Both FIFOs can be individually configured to trigger interrupts at different levels. Available configurations for transmit FIFO include 3/4, 1/2, 1/4 and empty, for receive FIFO 1/4, 1/2, 3/4 and full. For example, if the 3/4 option is selected for the receive FIFO, the UART generates a receive interrupt after 3 data bytes are received. Out of reset, both FIFOs are configured to trigger an interrupt at the 1/2 mark. The FIFO integrity is indeterminate under the following conditions:

- After a UART Send Break has been initiated (LCRH.BRK = 1)
- If the software disables the UART in the middle of a transmission with data in the FIFO, and then re-enables it.

16.2.3.18 Loopback Operation

The UART can be placed into an internal loopback mode for diagnostic or debug work by setting the LBE bit in the UART.CTL0 register. In loopback mode, the UART operates with the following behavior:

- Data transmitted on the TX output is received on the RX input
- Data transmitted on the TX output is not propagated to the TX IO pin
- Data received on the RX IO pin is ignored

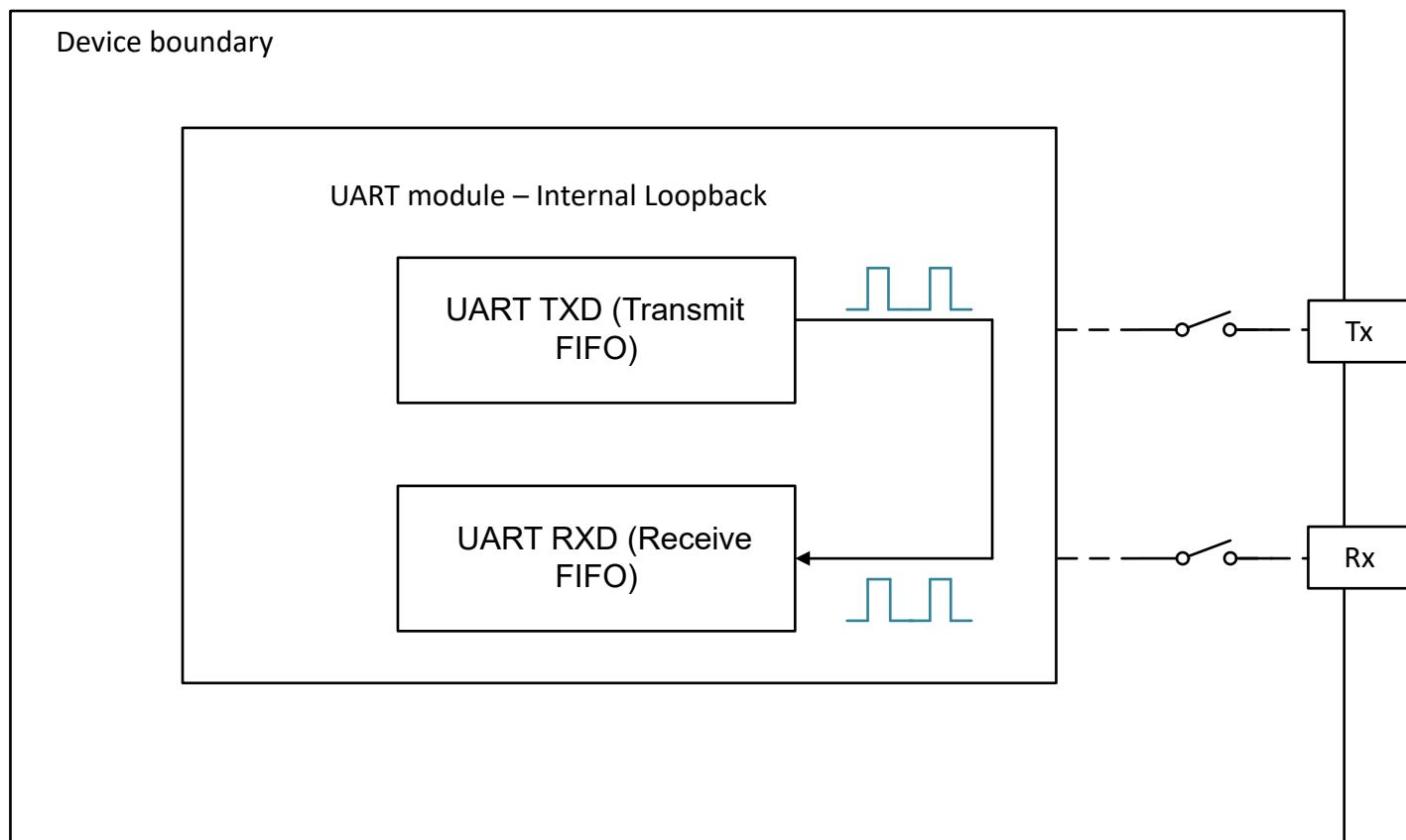


Figure 16-15. UART Loopback mode

16.2.3.19 Glitch Suppression

Digital filter

Digital filter is based on the UART functional clock. The DGFSEL bits in the UARTx.GFCTL register can be programmed to provide glitch suppression on the RX line and assure proper signal values. The glitch suppression value is in terms of functional clocks. All signals are delayed internally when glitch suppression is nonzero. For example, if DGFSEL is set to 0x5, 5 clocks should be added onto the calculation for the expected transaction time. The DGFSEL need to be configured for the glitch suppression pulse width to be shorter than 1/3 of a normal data pulse, to avoid a normal pulse is filtered unexpectedly.

Analog filter

The analog glitch suppression on the RX line is based on the analog glitch filter and it can be selected with the AGFSEL bits in the UARTx.GFCTL register. See data sheet for the select-able glitch filter values. The analog glitch filter is enabled with the AGFEN, if not set the input signals will be passed through to the UART module without filtering.

16.2.4 Low Power Operation

(More information will be added in next revision)

16.2.5 Reset Considerations

Software Reset Considerations

A software reset can be executed with setting the RESETASSERT together with the KEY in the RSTCTL register. An ongoing transfer will be terminated immediately and can leave the software in an undefined state. Therefore, before requesting a reset an ongoing transfer should be terminated.

Hardware Reset Considerations

A hardware reset also initializes the IO configuration. This sets the IOs to a high impedance state and the data lines can float. If this is critical for the application or connected devices on the UART interface external pull up or down resistors might be required.

16.2.6 Initialization

Before the UART is setup or configuration changes, the ENABLE bit should be cleared to avoid unpredictable behavior during the updates or for the first data receive or transmitted afterward.

To enable and initialize the UART, use the following steps:

1. Configure RX and TX pin functions by using the IOMUX registers.
2. Reset the peripheral using UARTx.RSTCTL register
3. Enable the power to UART peripheral using the UARTx.PWREN register
4. Select the UART function clock source and divide options using UART.CLKSEL and UART.CLKDIV registers.
5. Disable the UART by clearing the UART.CTL0.ENABLE bit.
6. Use the baud-rate equation in [Section 16.2.3.4](#) to calculate the UARTx.IBRD and UARTx.FBRD registers.
7. Write the integer portion of the BRD to the UART.IBRD register.
8. Write the fractional portion of the BRD to the UART.FBRD register.
9. Write the desired oversampling and FIFO configuration to the UART.CTL0 register
10. Write the desired serial parameters to the UART.LCRH register.
11. Enable the UART by setting the UART.CTL0.ENABLE bit.

16.2.7 Interrupt and Events Support

The UART module contains three [event publishers](#) and no [event subscribers](#). One event publisher (CPU_INT) manages UART interrupt requests (IRQs) to the CPU subsystem through a [static event route](#). The second and third event publishers (DMA_TRIG_RX, DMA_TRIG_TX) are used to setup the trigger signaling for the DMA through [DMA event route](#).

The UART events are summarized in [Table 16-7](#).

Table 16-7. UART Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU interrupt	Publisher	UART	CPU Subsystem	Static route	CPU_INT registers	Fixed interrupt route from UART to CPU
DMA trigger	Publisher	UART	DMA	DMA event route	DMA_TRIG_RX registers	Fixed interrupt route from UART to DMA
DMA trigger	Publisher	UART	DMA	DMA event route	DMA_TRIG_TX registers	Fixed interrupt route from UART to DMA

16.2.7.1 CPU Interrupt Event Publisher (CPU_INT)

The UART module provides 18 interrupt sources which can be configured to source a [CPU interrupt event](#). In order of decreasing interrupt priority, the CPU interrupt events from the UART are:

Table 16-8. UART CPU Interrupt Event Conditions (CPU_INT)

IIDX STAT	Name	Description
0x01	RTOUT	UART receive timeout interrupt, This interrupt is asserted when the receive FIFO is not empty, and no further data is received specified time in the UARTx.IFLS.RXTSEL bits. More information provided below.
0x02	FRMERR	UART framing error interrupt, see Section 16.2.3.6 for more information.
0x03	PARERR	UART parity error interrupt, see Section 16.2.3.6 for more information.

Table 16-8. UART CPU Interrupt Event Conditions (CPU_INT) (continued)

IIDX STAT	Name	Description
0x04	BRKERR	UART break error interrupt, see Section 16.2.3.6 for more information.
0x05	OVRERR	UART receive overrun error interrupt, see Section 16.2.3.6 for more information.
0x06	RXNE	Falling edge on RX interrupt, this interrupt triggers when there is a falling edge on RX line.
0x07	RXPE	Rising edge on RX interrupt, this interrupt triggers when there is a rising edge on RX line.
0x08	LINC0	LIN capture 0 match interrupt, this interrupt triggers when the defined capture 0 value is reached in LIN counter.
0x09	LINC1	LIN capture 1 match interrupt, this interrupt triggers when the defined capture 1 value is reached in LIN counter.
0x0A	LINOVF	LIN counter overflow interrupt, this interrupt triggers when the 16bit LIN counter overflows.
0x0B	RXINT	UART receive interrupt. More information provided below.
0x0C	TXINT	UART transmit interrupt. More information provided below.
0x0D	EOT	UART end of transmission interrupt, it indicates that the last bit of all transmitted data and status has left the serializer and without any further data in the TX FIFO.
0x0E	ADDR_MATCH	Address match interrupt, used in protocols with address to indicate address match happened.
0x0F	CTS	UART clear to send interrupt, indicate the CTS signal status.
0x10	DMA_DONE_RX	This interrupt is set if the RX DMA channel sends the DONE signal.
0x11	DMA_DONE_TX	This interrupt is set if the TX DMA channel sends the DONE signal.

The CPU interrupt event configuration is managed with the CPU_INT event management registers. See [Section 7.2.5](#) for guidance on configuring the Event registers for CPU interrupts.

The receive timeout interrupt is asserted when the receive FIFO is not empty, and no further data is received specified time in the IFLS.RXTSEL bits. The receive timeout interrupt is cleared either when the FIFO becomes empty through reading all the data (or by reading the holding register), by reading the interrupt index from IIDX or when a 1 is written to the corresponding bit in the ICLR register.

The receive interrupt (RXINT, 0x0B) changes state when one of the following events occurs:

- If the FIFOs are enabled and the receive FIFO reaches the programmed trigger level, the RXINT bit is set. The receive interrupt is cleared by reading data from the receive FIFO until it becomes less than the trigger level, by reading the interrupt index from IIDX or by writing a 1 to the RXINT bit in ICLR.
- If the FIFOs are disabled (have a depth of one location) and data is received thereby filling the location, the RXINT bit is set. The receive interrupt is cleared by performing a single read of the receive FIFO, by reading the interrupt index from IIDX or by writing a 1 to the RXINT bit in ICLR.

The transmit interrupt (TXINT, 0x0C) changes state when one of the following events occurs:

- If the FIFOs are enabled and the transmit FIFO progresses through the programmed trigger level, the TXINT bit is set. The transmit interrupt is based on a transition through level, therefore the FIFO must be written past the programmed trigger level otherwise no further transmit interrupts will be generated. The transmit interrupt is cleared by writing data to the transmit FIFO until it becomes greater than the trigger level, by reading the interrupt index from IIDX or by writing a 1 to the TXINT bit in ICLR.
- If the FIFOs are disabled (have a depth of one location) and there is no data present in the transmitters single location, the TXINT bit is set. It is cleared by performing a single write to the transmit FIFO, by reading the interrupt index from IIDX or by writing a 1 to the TXINT bit in ICLR.

16.2.7.2 DMA Trigger Publisher (DMA_TRIG_RX, DMA_TRIG_TX)

DMA_TRIG_RX and DMA_TRIG_TX registers are used to setup the trigger signaling for the DMA. This can be setup in a flexible way to trigger the DMA for receive or transmit events with the trigger conditions in [Table 16-9](#) and [Table 16-10](#).

DMA_TRIG_RX is used for triggering the DMA to do a receive data transfer and DMA_TRIG_TX is used for triggering the DMA to do a transmit data transfer.

Table 16-9. UART DMA Trigger Condition (DMA_TRIG_RX)

IIDX STAT	Name	Description
0x01	RTOUT	UART receive timeout interrupt. This interrupt is asserted when the receive FIFO is not empty, and no further data is received specified time in the UARTx.IFLS.RXTOSEL bits. More information provided below.
0x0B	RXINT	UART receive interrupt. More information provided below.

The receive timeout interrupt is asserted when the receive FIFO is not empty, and no further data is received specified time in the IFLS.RXTOSEL bits. The receive timeout interrupt is cleared either when the FIFO becomes empty through reading all the data (or by reading the holding register), by reading the interrupt index from IIDX or when a 1 is written to the corresponding bit in the ICLR register.

The receive interrupt (RXINT, 0x0B) changes state when one of the following events occurs:

- If the FIFOs are enabled and the receive FIFO reaches the programmed trigger level, the RXINT bit is set. The receive interrupt is cleared by reading data from the receive FIFO until it becomes less than the trigger level, by reading the interrupt index from IIDX or by writing a 1 to the RXINT bit in ICLR.
- If the FIFOs are disabled (have a depth of one location) and data is received thereby filling the location, the RXINT bit is set. The receive interrupt is cleared by performing a single read of the receive FIFO, by reading the interrupt index from IIDX or by writing a 1 to the RXINT bit in ICLR.

Table 16-10. UART DMA Trigger Condition (DMA_TRIG_TX)

IIDX STAT	Name	Description
0x0C	TXINT	UART transmit interrupt. More information provided below.

The transmit interrupt (TXINT, 0x0C) changes state when one of the following events occurs:

- If the FIFOs are enabled and the transmit FIFO progresses through the programmed trigger level, the TXINT bit is set. The transmit interrupt is based on a transition through level, therefore the FIFO must be written past the programmed trigger level otherwise no further transmit interrupts will be generated. The transmit interrupt is cleared by writing data to the transmit FIFO until it becomes greater than the trigger level, by reading the interrupt index from IIDX or by writing a 1 to the TXINT bit in ICLR.
- If the FIFOs are disabled (have a depth of one location) and there is no data present in the transmitters single location, the TXINT bit is set. It is cleared by performing a single write to the transmit FIFO, by reading the interrupt index from IIDX or by writing a 1 to the TXINT bit in ICLR.

The DMA trigger event configuration is managed with the DMA_TRIG_RX and DMA_TRIG_TX event management registers. See [Section 7.2.5](#) for guidance on configuring the Event registers and [Section 7.1.3.2](#) for on how DMA trigger event works.

16.2.8 Emulation Modes

The module behavior while the device is in debug mode is controlled by the FREE and SOFT bits in PDBGCTL register.

When the device is in debug mode and set into halt mode below behavior can be configured.

Table 16-11. Debug Mode Peripheral Behavior

PDBGCTL.FREE	PDBGCTL.SOFT	Function
1	x	Modules continues operation
0	0	Module stops immediately
0	1	Module stops after the next transfer has been finished

16.3 UART Registers

Table 16-12 lists the memory-mapped registers for the UART registers. All register offset addresses not listed in Table 16-12 should be considered as reserved locations and the register contents should not be modified.

Table 16-12. UART Registers

Offset	Acronym	Register Name	Group	Section
800h	PWREN	Power enable		Go
804h	RSTCTL	Reset Control		Go
808h	CLKCFG	Peripheral Clock Configuration Register		Go
814h	STAT	Status Register		Go
1000h	CLKDIV	Clock Divider		Go
1008h	CLKSEL	Clock Select for Ultra Low Power peripherals		Go
1018h	PDBGCTL	Peripheral Debug Control		Go
1020h	IIDX	Interrupt index	CPU_INT	Go
1028h	IMASK	Interrupt mask	CPU_INT	Go
1030h	RIS	Raw interrupt status	CPU_INT	Go
1038h	MIS	Masked interrupt status	CPU_INT	Go
1040h	ISSET	Interrupt set	CPU_INT	Go
1048h	ICLR	Interrupt clear	CPU_INT	Go
1050h	IIDX	Interrupt index	DMA_TRIG_RX	Go
1058h	IMASK	Interrupt mask	DMA_TRIG_RX	Go
1060h	RIS	Raw interrupt status	DMA_TRIG_RX	Go
1068h	MIS	Masked interrupt status	DMA_TRIG_RX	Go
1070h	ISSET	Interrupt set	DMA_TRIG_RX	Go
1078h	ICLR	Interrupt clear	DMA_TRIG_RX	Go
1080h	IIDX	Interrupt index	DMA_TRIG_TX	Go
1088h	IMASK	Interrupt mask	DMA_TRIG_TX	Go
1090h	RIS	Raw interrupt status	DMA_TRIG_TX	Go
1098h	MIS	Masked interrupt status	DMA_TRIG_TX	Go
10A0h	ISSET	Interrupt set	DMA_TRIG_TX	Go
10A8h	ICLR	Interrupt clear	DMA_TRIG_TX	Go
10E0h	EVT_MODE	Event Mode		Go
10E4h	INTCTL	Interrupt control register		Go
1100h	CTL0	UART Control Register 0		Go
1104h	LCRH	UART Line Control Register		Go
1108h	STAT	UART Status Register		Go
110Ch	IFLS	UART Interrupt FIFO Level Select Register		Go

Table 16-12. UART Registers (continued)

Offset	Acronym	Register Name	Group	Section
1110h	IBRD	UART Integer Baud-Rate Divisor Register		Go
1114h	FBRD	UART Fractional Baud-Rate Divisor Register		Go
1118h	GFCTL	Glitch Filter Control		Go
1120h	TXDATA	UART Transmit Data Register		Go
1124h	RXDATA	UART Receive Data Register		Go
1130h	LINCNT	UART LIN Mode Counter Register		Go
1134h	LINCTL	UART LIN Mode Control Register		Go
1138h	LINC0	UART LIN Mode Capture 0 Register		Go
113Ch	LINC1	UART LIN Mode Capture 1 Register		Go
1140h	IRCTL	eUSCI_Ax IrDA Control Word Register		Go
1148h	AMASK	Self Address Mask Register		Go
114Ch	ADDR	Self Address Register		Go
1160h	CLKDIV2	Clock Divider		Go

Complex bit access types are encoded to fit into small table cells. [Table 16-13](#) shows the codes that are used for access types in this section.

Table 16-13. UART Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value

16.3.1 PWREN (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 16-16](#) and described in [Table 16-14](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 16-16. PWREN

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-0h							R/WK-0h

Table 16-14. PWREN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	ENABLE	R/WK	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

16.3.2 RSTCTL (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 16-17](#) and described in [Table 16-15](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 16-17. RSTCTL

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCLR	RESETASSERT
W-0h						WK-0h	WK-0h

Table 16-15. RSTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	W	0h	
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

16.3.3 CLKCFG (Offset = 808h) [Reset = 00000000h]

CLKCFG is shown in [Figure 16-18](#) and described in [Table 16-16](#).

Return to the [Summary Table](#).

Peripheral Clock Configuration Register

Figure 16-18. CLKCFG

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							BLOCKASYNC
R/W-0h							R/W-0h
7	6	5	4	3	2	1	0
RESERVED							
R/W-0h							

Table 16-16. CLKCFG Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to Allow State Change -- 0xA9 A9h = 0xA9
23-9	RESERVED	R/W	0h	
8	BLOCKASYNC	R/W	0h	Async Clock Request is blocked from starting SYSOSC or forcing bus clock to 32MHz 0h = 0 1h = 1
7-0	RESERVED	R/W	0h	

16.3.4 STAT (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 16-19](#) and described in [Table 16-17](#).

Return to the [Summary Table](#).

Reset status register

Figure 16-19. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED							
R-							

Table 16-17. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0h	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

16.3.5 CLKDIV (Offset = 1000h) [Reset = 00000000h]

CLKDIV is shown in [Figure 16-20](#) and described in [Table 16-18](#).

Return to the [Summary Table](#).

This register is used to specify module-specific divide ratio of the functional clock

Figure 16-20. CLKDIV

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												RATIO			
R/W-0h												R/W-0h			

Table 16-18. CLKDIV Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R/W	0h	
2-0	RATIO	R/W	0h	Selects divide ratio of module clock 0h = Do not divide clock source 1h = Divide clock source by 2 2h = Divide clock source by 3 3h = Divide clock source by 4 4h = Divide clock source by 5 5h = Divide clock source by 6 6h = Divide clock source by 7 7h = Divide clock source by 8

16.3.6 CLKSEL (Offset = 1008h) [Reset = 00000000h]

CLKSEL is shown in [Figure 16-21](#) and described in [Table 16-19](#).

Return to the [Summary Table](#).

Clock source selection for peripherals

Figure 16-21. CLKSEL

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED				BUSCLK_SEL	MFCLK_SEL	LFCLK_SEL	RESERVED
R/W-0h				R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 16-19. CLKSEL Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3	BUSCLK_SEL	R/W	0h	Selects BUS CLK as clock source if enabled 0h = Does not select this clock as a source 1h = Select this clock as a source
2	MFCLK_SEL	R/W	0h	Selects MFCLK as clock source if enabled 0h = Does not select this clock as a source 1h = Select this clock as a source
1	LFCLK_SEL	R/W	0h	Selects LFCLK as clock source if enabled 0h = Does not select this clock as a source 1h = Select this clock as a source
0	RESERVED	R/W	0h	

16.3.7 PDBGCTL (Offset = 1018h) [Reset = 00000003h]

PDBGCTL is shown in [Figure 16-22](#) and described in [Table 16-20](#).

Return to the [Summary Table](#).

This register can be used by the software developer to control the behavior of the peripheral relative to the 'Core Halted' input

Figure 16-22. PDBGCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED						SOFT	FREE
R/W-						R/W-1h	R/W-1h

Table 16-20. PDBGCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R/W	0h	
1	SOFT	R/W	1h	Soft halt boundary control. This function is only available, if FREE is set to 'STOP' 0h = The peripheral will halt immediately, even if the resultant state will result in corruption if the system is restarted 1h = The peripheral blocks the debug freeze until it has reached a boundary where it can resume without corruption
0	FREE	R/W	1h	Free run control 0h = The peripheral freezes functionality while the Core Halted input is asserted and resumes when it is deasserted. 1h = The peripheral ignores the state of the Core Halted input

16.3.8 IIDX (Offset = 1020h) [Reset = 00000000h]

IIDX is shown in [Figure 16-23](#) and described in [Table 16-21](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 16-23. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 16-21. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	UART Module Interrupt Vector Value. This register provides the highest priority interrupt index. A read clears the corresponding interrupt flag in RIS and MIS registers. 15h-1Fh = Reserved 00h = No interrupt pending 01h = UART receive time-out interrupt; Interrupt Flag: RT; Interrupt Priority: Highest 02h = UART framing error interrupt; Interrupt Flag: FE 03h = UART parity error interrupt; Interrupt Flag: PE 04h = UART break error interrupt; Interrupt Flag: BE 05h = UART receive overrun error interrupt; Interrupt Flag: OE 06h = Negative edge on UARTxRXD interrupt; Interrupt Flag: RXNE 07h = Positive edge on UARTxRXD interrupt; Interrupt Flag: RXPE 08h = LIN capture 0 / match interrupt; Interrupt Flag: LINC0 09h = LIN capture 1 interrupt; Interrupt Flag: LINC1 0Ah = LIN hardware counter overflow interrupt; Interrupt Flag: LINOVF 0Bh = UART receive interrupt; Interrupt Flag: RX 0Ch = UART transmit interrupt; Interrupt Flag: TX 0Dh = UART end of transmission interrupt (transmit serializer empty); Interrupt Flag: EOT 0Eh = 9-bit mode address match interrupt; Interrupt Flag: MODE_9B Fh = UART Clear to Send Modem interrupt; Interrupt Flag: CTS 10h = DMA DONE on RX 11h = DMA DONE on TX 12h = Noise Error Event

16.3.9 IMASK (Offset = 1028h) [Reset = 00000000h]

IMASK is shown in [Figure 16-24](#) and described in [Table 16-22](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 16-24. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED						NERR	DMA_DONE_TX
R/W-0h						R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
DMA_DONE_RX	CTS	ADDR_MATCH	EOT	TXINT	RXINT	LINOVF	LINC1
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
LINC0	RXPE	RXNE	OVRRERR	BRKERR	PARERR	FRMERR	RTOUT
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 16-22. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-18	RESERVED	R/W	0h	
17	NERR	R/W	0h	Noise Error on triple voting. Asserted when the 3 samples of majority voting are not equal 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
16	DMA_DONE_TX	R/W	0h	Enable DMA Done on TX Event Channel Interrupt 0h = Interrupt disabled 1h = Set Interrupt Mask
15	DMA_DONE_RX	R/W	0h	Enable DMA Done on RX Event Channel Interrupt 0h = Interrupt disabled 1h = Set Interrupt Mask
14	CTS	R/W	0h	Enable UART Clear to Send Modem Interrupt. 0h = Interrupt disabled 1h = Set Interrupt Mask
13	ADDR_MATCH	R/W	0h	Enable Address Match Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
12	EOT	R/W	0h	Enable UART End of Transmission Interrupt Indicates that the last bit of all transmitted data and flags has left the serializer and without any further Data in the TX FIFO or Buffer. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
11	TXINT	R/W	0h	Enable UART Transmit Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
10	RXINT	R/W	0h	Enable UART Receive Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

Table 16-22. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
9	LINOVF	R/W	0h	Enable LIN Hardware Counter Overflow Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
8	LINC1	R/W	0h	Enable LIN Capture 1 Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
7	LINC0	R/W	0h	Enable LIN Capture 0 / Match Interrupt . 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
6	RXPE	R/W	0h	Enable Positive Edge on UARTxRXD Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
5	RXNE	R/W	0h	Enable Negative Edge on UARTxRXD Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
4	OVRERR	R/W	0h	Enable UART Receive Overrun Error Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
3	BRKERR	R/W	0h	Enable UART Break Error Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	PARERR	R/W	0h	Enable UART Parity Error Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	FRMERR	R/W	0h	Enable UART Framing Error Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	RTOUT	R/W	0h	Enable UARTOUT Receive Time-Out Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

16.3.10 RIS (Offset = 1030h) [Reset = 00000000h]

RIS is shown in [Figure 16-25](#) and described in [Table 16-23](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 16-25. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED						NERR	DMA_DONE_TX
R-						R-0h	R-0h
15	14	13	12	11	10	9	8
DMA_DONE_RX	CTS	ADDR_MATCH	EOT	TXINT	RXINT	LINOVF	LINC1
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
LINC0	RXPE	RXNE	OVRRERR	BRKERR	PARERR	FRMERR	RTOUT
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 16-23. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-18	RESERVED	R	0h	
17	NERR	R	0h	Noise Error on triple voting. Asserted when the 3 samples of majority voting are not equal 0h = Interrupt did not occur 1h = Interrupt occurred
16	DMA_DONE_TX	R	0h	DMA Done on TX Event Channel Interrupt 0h = Interrupt disabled 1h = Interrupt occurred
15	DMA_DONE_RX	R	0h	DMA Done on RX Event Channel Interrupt 0h = Interrupt disabled 1h = Interrupt occurred
14	CTS	R	0h	UART Clear to Send Modem Interrupt. 0h = Interrupt disabled 1h = Interrupt occurred
13	ADDR_MATCH	R	0h	Address Match Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
12	EOT	R	0h	UART End of Transmission Interrupt Indicates that the last bit of all transmitted data and flags has left the serializer and without any further Data in the TX FIFO or Buffer. 0h = Interrupt did not occur 1h = Interrupt occurred
11	TXINT	R	0h	UART Transmit Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
10	RXINT	R	0h	UART Receive Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred

Table 16-23. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
9	LINOVF	R	0h	LIN Hardware Counter Overflow Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
8	LINC1	R	0h	LIN Capture 1 Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
7	LINC0	R	0h	LIN Capture 0 / Match Interrupt . 0h = Interrupt did not occur 1h = Interrupt occurred
6	RXPE	R	0h	Positive Edge on UARTxRXD Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
5	RXNE	R	0h	Negative Edge on UARTxRXD Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
4	OVRERR	R	0h	UART Receive Overrun Error Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
3	BRKERR	R	0h	UART Break Error Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
2	PARERR	R	0h	UART Parity Error Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
1	FRMERR	R	0h	UART Framing Error Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
0	RTOUT	R	0h	UARTOUT Receive Time-Out Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred

16.3.11 MIS (Offset = 1038h) [Reset = 00000000h]

MIS is shown in [Figure 16-26](#) and described in [Table 16-24](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 16-26. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED						NERR	DMA_DONE_TX
R-0h						R-0h	R-0h
15	14	13	12	11	10	9	8
DMA_DONE_RX	CTS	ADDR_MATCH	EOT	TXINT	RXINT	LINOVF	LINC1
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
LINC0	RXPE	RXNE	OVRRERR	BRKERR	PARERR	FRMERR	RTOUT
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 16-24. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-18	RESERVED	R	0h	
17	NERR	R	0h	Noise Error on triple voting. Asserted when the 3 samples of majority voting are not equal 0h = Interrupt did not occur 1h = Interrupt occurred
16	DMA_DONE_TX	R	0h	Masked DMA Done on TX Event Channel Interrupt 0h = Interrupt did not occur 1h = Interrupt occurred
15	DMA_DONE_RX	R	0h	Masked DMA Done on RX Event Channel Interrupt 0h = Interrupt did not occur 1h = Interrupt occurred
14	CTS	R	0h	Masked UART Clear to Send Modem Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
13	ADDR_MATCH	R	0h	Masked Address Match Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
12	EOT	R	0h	UART End of Transmission Interrupt Indicates that the last bit of all transmitted data and flags has left the serializer and without any further Data in the TX FIFO or Buffer. 0h = Interrupt did not occur 1h = Interrupt occurred
11	TXINT	R	0h	Masked UART Transmit Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
10	RXINT	R	0h	Masked UART Receive Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred

Table 16-24. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
9	LINOVF	R	0h	Masked LIN Hardware Counter Overflow Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
8	LINC1	R	0h	Masked LIN Capture 1 Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
7	LINC0	R	0h	Masked LIN Capture 0 / Match Interrupt . 0h = Interrupt did not occur 1h = Interrupt occurred
6	RXPE	R	0h	Masked Positive Edge on UARTxRXD Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
5	RXNE	R	0h	Masked Negative Edge on UARTxRXD Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
4	OVRERR	R	0h	Masked UART Receive Overrun Error Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
3	BRKERR	R	0h	Masked UART Break Error Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
2	PARERR	R	0h	Masked UART Parity Error Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
1	FRMERR	R	0h	Masked UART Framing Error Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
0	RTOUT	R	0h	Masked UARTOUT Receive Time-Out Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred

16.3.12 ISET (Offset = 1040h) [Reset = 00000000h]

ISET is shown in [Figure 16-27](#) and described in [Table 16-25](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 16-27. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED						NERR	DMA_DONE_TX
W-0h						W-0h	W-0h
15	14	13	12	11	10	9	8
DMA_DONE_RX	CTS	ADDR_MATCH	EOT	TXINT	RXINT	LINOVF	LINC1
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
LINC0	RXPE	RXNE	OVRRERR	BRKERR	PARERR	FRMERR	RTOUT
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 16-25. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-18	RESERVED	W	0h	
17	NERR	W	0h	Noise Error on triple voting. Asserted when the 3 samples of majority voting are not equal 0h = Writing this has no effect 1h = Set the interrupt
16	DMA_DONE_TX	W	0h	Set DMA Done on TX Event Channel Interrupt 0h = Interrupt disabled 1h = Set Interrupt
15	DMA_DONE_RX	W	0h	Set DMA Done on RX Event Channel Interrupt 0h = Interrupt disabled 1h = Set Interrupt
14	CTS	W	0h	Set UART Clear to Send Modem Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
13	ADDR_MATCH	W	0h	Set Address Match Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
12	EOT	W	0h	Set UART End of Transmission Interrupt Indicates that the last bit of all transmitted data and flags has left the serializer and without any further Data in the TX FIFO or Buffer. 0h = Writing 0 has no effect 1h = Set Interrupt
11	TXINT	W	0h	Set UART Transmit Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
10	RXINT	W	0h	Set UART Receive Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt

Table 16-25. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
9	LINOVF	W	0h	Set LIN Hardware Counter Overflow Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
8	LINC1	W	0h	Set LIN Capture 1 Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
7	LINC0	W	0h	Set LIN Capture 0 / Match Interrupt . 0h = Writing 0 has no effect 1h = Set Interrupt
6	RXPE	W	0h	Set Positive Edge on UARTxRXD Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
5	RXNE	W	0h	Set Negative Edge on UARTxRXD Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
4	OVRERR	W	0h	Set UART Receive Overrun Error Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
3	BRKERR	W	0h	Set UART Break Error Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
2	PARERR	W	0h	Set UART Parity Error Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
1	FRMERR	W	0h	Set UART Framing Error Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
0	RTOUT	W	0h	Set UARTOUT Receive Time-Out Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt

16.3.13 ICLR (Offset = 1048h) [Reset = 00000000h]

ICLR is shown in [Figure 16-28](#) and described in [Table 16-26](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 16-28. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED						NERR	DMA_DONE_TX
W-0h						W-0h	W-0h
15	14	13	12	11	10	9	8
DMA_DONE_RX	CTS	ADDR_MATCH	EOT	TXINT	RXINT	LINOVF	LINC1
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
LINC0	RXPE	RXNE	OVRRERR	BRKERR	PARERR	FRMERR	RTOUT
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 16-26. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-18	RESERVED	W	0h	
17	NERR	W	0h	Noise Error on triple voting. Asserted when the 3 samples of majority voting are not equal 0h = Writing 0 has no effect 1h = Clear Interrupt
16	DMA_DONE_TX	W	0h	Clear DMA Done on TX Event Channel Interrupt 0h = Interrupt disabled 1h = Clear Interrupt
15	DMA_DONE_RX	W	0h	Clear DMA Done on RX Event Channel Interrupt 0h = Interrupt disabled 1h = Clear Interrupt
14	CTS	W	0h	Clear UART Clear to Send Modem Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
13	ADDR_MATCH	W	0h	Clear Address Match Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
12	EOT	W	0h	Clear UART End of Transmission Interrupt Indicates that the last bit of all transmitted data and flags has left the serializer and without any further Data in the TX FIFO or Buffer. 0h = Writing 0 has no effect 1h = Clear Interrupt
11	TXINT	W	0h	Clear UART Transmit Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
10	RXINT	W	0h	Clear UART Receive Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt

Table 16-26. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
9	LINOVF	W	0h	Clear LIN Hardware Counter Overflow Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
8	LINC1	W	0h	Clear LIN Capture 1 Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
7	LINC0	W	0h	Clear LIN Capture 0 / Match Interrupt . 0h = Writing 0 has no effect 1h = Clear Interrupt
6	RXPE	W	0h	Clear Positive Edge on UARTxRXD Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
5	RXNE	W	0h	Clear Negative Edge on UARTxRXD Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
4	OVRERR	W	0h	Clear UART Receive Overrun Error Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
3	BRKERR	W	0h	Clear UART Break Error Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
2	PARERR	W	0h	Clear UART Parity Error Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
1	FRMERR	W	0h	Clear UART Framing Error Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
0	RTOUT	W	0h	Clear UARTOUT Receive Time-Out Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt

16.3.14 IIDX (Offset = 1050h) [Reset = 00000000h]

IIDX is shown in [Figure 16-29](#) and described in [Table 16-27](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 16-29. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 16-27. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	UART Module Interrupt Vector Value. This register provides the highest priority interrupt index. A read clears the corresponding interrupt flag in UARTRIS and UARTMISC. 15h-1Fh = Reserved 00h = No interrupt pending 01h = UART receive time-out interrupt; Interrupt Flag: RT; Interrupt Priority: Highest 0Bh = UART receive interrupt; Interrupt Flag: RX

16.3.15 IMASK (Offset = 1058h) [Reset = 00000000h]

IMASK is shown in [Figure 16-30](#) and described in [Table 16-28](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 16-30. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED					RXINT	RESERVED	
R/W-0h					R/W-0h	R/W-0h	
7	6	5	4	3	2	1	0
RESERVED							RTOUT
R/W-0h							R/W-0h

Table 16-28. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-11	RESERVED	R/W	0h	
10	RXINT	R/W	0h	Enable UART Receive Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
9-1	RESERVED	R/W	0h	
0	RTOUT	R/W	0h	Enable UARTOUT Receive Time-Out Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

16.3.16 RIS (Offset = 1060h) [Reset = 00000000h]

RIS is shown in [Figure 16-31](#) and described in [Table 16-29](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 16-31. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							
R-							
15	14	13	12	11	10	9	8
RESERVED					RXINT	RESERVED	
R-					R-0h	R-	
7	6	5	4	3	2	1	0
RESERVED							RTOUT
R-							R-0h

Table 16-29. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-11	RESERVED	R	0h	
10	RXINT	R	0h	UART Receive Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
9-1	RESERVED	R	0h	
0	RTOUT	R	0h	UARTOUT Receive Time-Out Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred

16.3.17 MIS (Offset = 1068h) [Reset = 00000000h]

MIS is shown in [Figure 16-32](#) and described in [Table 16-30](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 16-32. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED					RXINT	RESERVED	
R-0h					R-0h	R-0h	
7	6	5	4	3	2	1	0
RESERVED							RTOUT
R-0h							R-0h

Table 16-30. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-11	RESERVED	R	0h	
10	RXINT	R	0h	Masked UART Receive Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
9-1	RESERVED	R	0h	
0	RTOUT	R	0h	Masked UARTOUT Receive Time-Out Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred

16.3.18 ISET (Offset = 1070h) [Reset = 00000000h]

ISET is shown in [Figure 16-33](#) and described in [Table 16-31](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 16-33. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED					RXINT	RESERVED	
W-0h					W-0h	W-0h	
7	6	5	4	3	2	1	0
RESERVED							RTOUT
W-0h							W-0h

Table 16-31. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-11	RESERVED	W	0h	
10	RXINT	W	0h	Set UART Receive Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
9-1	RESERVED	W	0h	
0	RTOUT	W	0h	Set UARTOUT Receive Time-Out Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt

16.3.19 ICLR (Offset = 1078h) [Reset = 00000000h]

ICLR is shown in [Figure 16-34](#) and described in [Table 16-32](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 16-34. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED					RXINT	RESERVED	
W-0h					W-0h	W-0h	
7	6	5	4	3	2	1	0
RESERVED							RTOUT
W-0h							W-0h

Table 16-32. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-11	RESERVED	W	0h	
10	RXINT	W	0h	Clear UART Receive Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
9-1	RESERVED	W	0h	
0	RTOUT	W	0h	Clear UARTOUT Receive Time-Out Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt

16.3.20 IIDX (Offset = 1080h) [Reset = 00000000h]

IIDX is shown in [Figure 16-35](#) and described in [Table 16-33](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 16-35. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 16-33. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	UART Module Interrupt Vector Value. This register provides the highest priority interrupt index. A read clears the corresponding interrupt flag in UARTRIS and UARTMISC. 15h-1Fh = Reserved 00h = No interrupt pending 0Ch = UART transmit interrupt; Interrupt Flag: TX

16.3.21 IMASK (Offset = 1088h) [Reset = 00000000h]

IMASK is shown in [Figure 16-36](#) and described in [Table 16-34](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 16-36. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED				TXINT	RESERVED		
R/W-0h				R/W-0h	R/W-0h		
7	6	5	4	3	2	1	0
RESERVED							
R/W-0h							

Table 16-34. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-12	RESERVED	R/W	0h	
11	TXINT	R/W	0h	Enable UART Transmit Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
10-0	RESERVED	R/W	0h	

16.3.22 RIS (Offset = 1090h) [Reset = 00000000h]

RIS is shown in [Figure 16-37](#) and described in [Table 16-35](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 16-37. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							
R-							
15	14	13	12	11	10	9	8
RESERVED				TXINT	RESERVED		
R-				R-0h	R-		
7	6	5	4	3	2	1	0
RESERVED							
R-							

Table 16-35. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-12	RESERVED	R	0h	
11	TXINT	R	0h	UART Transmit Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
10-0	RESERVED	R	0h	

16.3.23 MIS (Offset = 1098h) [Reset = 00000000h]

MIS is shown in [Figure 16-38](#) and described in [Table 16-36](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 16-38. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED				TXINT	RESERVED		
R-0h				R-0h	R-0h		
7	6	5	4	3	2	1	0
RESERVED							
R-0h							

Table 16-36. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-12	RESERVED	R	0h	
11	TXINT	R	0h	Masked UART Transmit Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
10-0	RESERVED	R	0h	

16.3.24 ISET (Offset = 10A0h) [Reset = 00000000h]

ISET is shown in [Figure 16-39](#) and described in [Table 16-37](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 16-39. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED				TXINT	RESERVED		
W-0h				W-0h	W-0h		
7	6	5	4	3	2	1	0
RESERVED							
W-0h							

Table 16-37. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-12	RESERVED	W	0h	
11	TXINT	W	0h	Set UART Transmit Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
10-0	RESERVED	W	0h	

16.3.25 ICLR (Offset = 10A8h) [Reset = 00000000h]

ICLR is shown in [Figure 16-40](#) and described in [Table 16-38](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 16-40. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED				TXINT	RESERVED		
W-0h				W-0h	W-0h		
7	6	5	4	3	2	1	0
RESERVED							
W-0h							

Table 16-38. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-12	RESERVED	W	0h	
11	TXINT	W	0h	Clear UART Transmit Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
10-0	RESERVED	W	0h	

16.3.26 EVT_MODE (Offset = 10E0h) [Reset = 00000029h]

EVT_MODE is shown in [Figure 16-41](#) and described in [Table 16-39](#).

Return to the [Summary Table](#).

Event mode register. It is used to select whether each line is disabled, in software mode (software clears the RIS) or in hardware mode (hardware clears the RIS)

Figure 16-41. EVT_MODE

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED		INT2_CFG		INT1_CFG		INT0_CFG	
R/W-		R-2h		R-2h		R-1h	

Table 16-39. EVT_MODE Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R/W	0h	
5-4	INT2_CFG	R	2h	Event line mode select for event corresponding to none.DMA_TRIG_TX 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
3-2	INT1_CFG	R	2h	Event line mode select for event corresponding to none.DMA_TRIG_RX 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
1-0	INT0_CFG	R	1h	Event line mode select for event corresponding to none.CPU_INT 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.

16.3.27 INTCTL (Offset = 10E4h) [Reset = 00000000h]

INTCTL is shown in [Figure 16-42](#) and described in [Table 16-40](#).

Return to the [Summary Table](#).

Interrupt control register

Figure 16-42. INTCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED							INTEVAL
R/W-							W-0h

Table 16-40. INTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R/W	0h	
0	INTEVAL	W	0h	Writing a 1 to this field re-evaluates the interrupt sources. 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS.

16.3.28 CTL0 (Offset = 1100h) [Reset = 00000038h]

CTL0 is shown in [Figure 16-43](#) and described in [Table 16-41](#).

Return to the [Summary Table](#).

UART Control Register

The CTL0 register is the control register. All the bits are cleared on reset except for the Transmit Enable (TXE) and Receive Enable (RXE) bits, which are set. To enable the UART module, the UARTEN bit must be set. If software requires a configuration change in the module, the UARTEN bit must be cleared before the configuration changes are written. If the UART is disabled during a transmit or receive operation, the current transaction is completed prior to the UART stopping. NOTE: The CTL0 register should not be changed while the UART is enabled or else the results are unpredictable. The following sequence is recommended for making changes to the CTL0 register.

1. Disable the UART.
2. Wait for the end of transmission or reception of the current character.
3. Flush the transmit FIFO by clearing bit FEN in the UART control register CTL0.
4. Reprogram the control register.
5. Enable the UART.

Figure 16-43. CTL0

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED				MSBFIRST	MAJVOTE	FEN	HSE
R/W-0h				R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
HSE	CTSEN	RTSEN	RTS	RESERVED	MODE		
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h		
7	6	5	4	3	2	1	0
MENC	TXD_OUT	TXD_OUT_EN	TXE	RXE	LBE	RESERVED	ENABLE
R/W-0h	R/W-0h	R/W-1h	R/W-1h	R/W-1h	R/W-0h	R/W-0h	R/W-0h

Table 16-41. CTL0 Field Descriptions

Bit	Field	Type	Reset	Description
31-20	RESERVED	R/W	0h	
19	MSBFIRST	R/W	0h	Most Significant Bit First This bit has effect both on the way protocol byte is transmitted and received. Notes: User needs to match the protocol to the correct value of this bit to send MSb or LSb first. The hardware engine will send the byte entirely based on this bit. 0h = Least significant bit is sent first in the protocol packet 1h = Most significant bit is sent first in the protocol packet
18	MAJVOTE	R/W	0h	Majority Vote Enable When Majority Voting is enabled, the three center bits are used to determine received sample value. In case of error (i.e. all 3 bits are not the same), noise error is detected and bits RIS.NERR and register RXDATA.NERR are set. Oversampling of 16 : bits 7, 8, 9 are used Oversampling of 8 : bits 3, 4, 5 are used Disabled : Single sample value (center value) used 0h = Majority voting is disabled 1h = Majority voting is enabled

Table 16-41. CTL0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
17	FEN	R/W	0h	UART Enable FIFOs 0h = The FIFOs are disabled (Character mode). The FIFOs become 1-byte-deep holding registers. 1h = The transmit and receive FIFO buffers are enabled (FIFO mode).
16-15	HSE	R/W	0h	High-Speed Bit Oversampling Enable NOTE: The bit oversampling influences the UART baud-rate configuration. The state of this bit has no effect on clock generation in ISO7816 smart card mode (the SMART bit is set). 0h = 16x oversampling. 1h = 8x oversampling. 2h = 3x oversampling. IrDA, Manchester and DALI not supported when 3x oversampling is enabled.
14	CTSEN	R/W	0h	Enable Clear To Send 0h = CTS hardware flow control is disabled. 1h = CTS hardware flow control is enabled. Data is only transmitted when the UARTxCTS signal is asserted.
13	RTSEN	R/W	0h	Enable hardware controlled Request to Send 0h = RTS hardware flow control is disabled. 1h = RTS hardware flow control is enabled. Data is only requested (by asserting UARTxRTS) when the receive FIFO has available entries.
12	RTS	R/W	0h	Request to Send If RTSEN is set the RTS output signals is controlled by the hardware logic using the FIFO fill level or TXDATA buffer. If RTSEN is cleared the RTS output is controlled by the RTS bit. The bit is the complement of the UART request to send, RTS modem status output. 0h = Signal not RTS 1h = Signal RTS
11	RESERVED	R/W	0h	
10-8	MODE	R/W	0h	Set the communication mode and protocol used. (Not defined settings uses the default setting: 0) 0h = Normal operation 1h = RS485 mode: UART needs to be IDLE with receiving data for the in EXTDIR_HOLD set time. EXTDIR_SETUP defines the time the RTS line is set to high before sending. When the buffer is empty the RTS line is set low again. A transmit will be delayed as long the UART is receiving data. 2h = The UART operates in IDLE Line Mode 3h = The UART operates in 9 Bit Address mode 4h = ISO7816 Smart Card Support The application must ensure that it sets 8-bit word length (WLEN set to 3h) and even parity (PEN set to 1, EPS set to 1, SPS set to 0) in UARTECRH when using ISO7816 mode. The value of the STP2 bit in UARTECRH is ignored and the number of stop bits is forced to 2. 5h = DALI Mode:
7	MENC	R/W	0h	Manchester Encode enable 0h = Disable Manchester Encoding 1h = Enable Manchester Encoding
6	TXD_OUT	R/W	0h	TXD Pin Control Controls the TXD pin when TXD_OUT_EN = 1 and TXE = 0. 0h = TXD pin is low 1h = TXD pin is high
5	TXD_OUT_EN	R/W	1h	TXD Pin Control Enable. When the transmit section of the UART is disabled (TXE = 0), the TXD pin can be controlled by the TXD_OUT bit. 0h = TXD pin can not be controlled by TXD_OUT 1h = TXD pin can be controlled by TXD_OUT

Table 16-41. CTL0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
4	TXE	R/W	1h	UART Transmit Enable If the UART is disabled in the middle of a transmission, it completes the current character before stopping. NOTE: To enable transmission, the UARTEN bit must be set. 0h = The transmit section of the UART is disabled. The UARTxTXD pin of the UART can be controlled by the TXD_CTL bit when enabled. 1h = The transmit section of the UART is enabled.
3	RXE	R/W	1h	UART Receive Enable If the UART is disabled in the middle of a receive, it completes the current character before stopping. NOTE: To enable reception, the UARTEN bit must be set. 0h = The receive section of the UART is disabled. 1h = The receive section of the UART is enabled.
2	LBE	R/W	0h	UART Loop Back Enable 0h = Normal operation. 1h = The UARTxTX path is fed through the UARTxRX path internally.
1	RESERVED	R/W	0h	
0	ENABLE	R/W	0h	UART Module Enable. If the UART is disabled in the middle of transmission or reception, it completes the current character before stopping. If the ENABLE bit is not set, all registers can still be accessed and updated. It is recommended to setup and change the UART operation mode with having the ENABLE bit cleared to avoid unpredictable behavior during the setup or update. If disabled the UART module will not send or receive any data and the logic is held in reset state. 0h = Disable Module 1h = Enable module

16.3.29 LCRH (Offset = 1104h) [Reset = 00000000h]

LCRH is shown in [Figure 16-44](#) and described in [Table 16-42](#).

Return to the [Summary Table](#).

UART Line Control Register The LCRH register is the line control register. Serial parameters such as data length, parity, and stop bit selection are implemented in this register. When updating the baud-rate divisor (UARTIBRD or UARTIFRD), the LCRH register must also be written. The write strobe for the baud-rate divisor registers is tied to the LCRH register.

Figure 16-44. LCRH

31	30	29	28	27	26	25	24
RESERVED						EXTDIR_HOLD	
R/W-0h						R/W-0h	
23	22	21	20	19	18	17	16
EXTDIR_HOLD				EXTDIR_SETUP			
R/W-0h				R/W-0h			
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
SENDIDLE	SPS	WLEN		STP2	EPS	PEN	BRK
R/W-0h	R/W-0h	R/W-0h		R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 16-42. LCRH Field Descriptions

Bit	Field	Type	Reset	Description
31-26	RESERVED	R/W	0h	
25-21	EXTDIR_HOLD	R/W	0h	Defines the number of UARTclk ticks the signal to control the external driver for the RS485 will be reset after the beginning of the stop bit. (If 2 STOP bits are enabled the beginning of the 2nd STOP bit.) 0h = Smallest value 1Fh = Highest possible value
20-16	EXTDIR_SETUP	R/W	0h	Defines the number of UARTclk ticks the signal to control the external driver for the RS485 will be set before the START bit is send 0h = Smallest value 1Fh = Highest possible value
15-8	RESERVED	R/W	0h	
7	SENDIDLE	R/W	0h	UART send IDLE pattern. When this bit is set an SENDIDLE period of 11 bit times will be sent on the TX line. The bit is cleared by hardware afterward. 0h = Disable Send Idle Pattern 1h = Enable Send Idle Pattern
6	SPS	R/W	0h	UART Stick Parity Select The Stick Parity Select (SPS) bit is used to set either a permanent '1' or a permanent '0' as parity when transmitting or receiving data. Its purpose is to typically indicate the first byte of a package or to mark an address byte, for example in a multi-drop RS-485 network. When bits PEN, EPS, and SPS of UARTLCRH are set, the parity bit is transmitted and checked as a 0. When bits PEN and SPS are set and EPS is cleared, the parity bit is transmitted and checked as a 1. 0h = Disable Stick Parity 1h = Enable Stick Parity

Table 16-42. LCRH Field Descriptions (continued)

Bit	Field	Type	Reset	Description
5-4	WLEN	R/W	0h	UART Word Length The bits indicate the number of data bits transmitted or received in a frame as follows: 0h = 5 bits (default) 1h = 6 bits 2h = 7 bits 3h = 8 bits
3	STP2	R/W	0h	UART Two Stop Bits Select When in 7816 smart card mode (the SMART bit is set in the UARTCTL register), the number of stop bits is forced to 2. 0h = One stop bit is transmitted at the end of a frame. 1h = Two stop bits are transmitted at the end of a frame. The receive logic checks for two stop bits being received and provide Frame Error if either is invalid.
2	EPS	R/W	0h	UART Even Parity Select This bit has no effect when parity is disabled by the PEN bit. For 9-Bit UART Mode transmissions, this bit controls the address byte and data byte indication (9th bit). 0 = The transferred byte is a data byte 1 = The transferred byte is an address byte 0h = Odd parity is performed, which checks for an odd number of 1s. 1h = Even parity generation and checking is performed during transmission and reception, which checks for an even number of 1s in data and parity bits.
1	PEN	R/W	0h	UART Parity Enable 0h = Parity is disabled and no parity bit is added to the data frame. 1h = Parity checking and generation is enabled.
0	BRK	R/W	0h	UART Send Break (for LIN Protocol) 0h = Normal use. 1h = A low level is continually output on the UARTxTXD signal, after completing transmission of the current character. For the proper execution of the break command, software must set this bit for at least two frames (character periods).

16.3.30 STAT (Offset = 1108h) [Reset = 00000144h]

STAT is shown in [Figure 16-45](#) and described in [Table 16-43](#).

Return to the [Summary Table](#).

UART Status Register

Figure 16-45. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							
R-							
15	14	13	12	11	10	9	8
RESERVED						IDLE	CTS
R-						R-0h	R-1h
7	6	5	4	3	2	1	0
TXFF	TXFE	RESERVED		RXFF	RXFE	RESERVED	BUSY
R-0h	R-1h	R-		R-0h	R-1h	R-	R-0h

Table 16-43. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-10	RESERVED	R	0h	
9	IDLE	R	0h	IDLE mode has been detected in Idleline-Multiprocessor-Mode. The IDLE bit is used as an address tag for each block of characters. In idle-line multiprocessor format, this bit is set when a received character is an address. 0h = IDLE has not been detected before last received character. (In idle-line multiprocessor mode). 1h = IDLE has been detected before last received character. (In idle-line multiprocessor mode).
8	CTS	R	1h	Clear To Send 0h = The CTS signal is not asserted (high). 1h = The CTS signal is asserted (low).
7	TXFF	R	0h	UART Transmit FIFO Full The meaning of this bit depends on the state of the FEN bit in the CTL0 register. 0h = The transmitter is not full. 1h = If the FIFO is disabled (FEN is 0), the transmit holding register is full. If the FIFO is enabled (FEN is 1), the transmit FIFO is full.
6	TXFE	R	1h	UART Transmit FIFO Empty The meaning of this bit depends on the state of the FEN bit in the CTL0 register. 0h = The transmitter has data to transmit. 1h = If the FIFO is disabled (FEN is 0), the transmit holding register is empty. If the FIFO is enabled (FEN is 1), the transmit FIFO is empty.
5-4	RESERVED	R	0h	
3	RXFF	R	0h	UART Receive FIFO Full The meaning of this bit depends on the state of the FEN bit in the CTL0 register. 0h = The receiver can receive data. 1h = If the FIFO is disabled (FEN is 0), the receive holding register is full. If the FIFO is enabled (FEN is 1), the receive FIFO is full.

Table 16-43. STAT Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	RXFE	R	1h	UART Receive FIFO Empty The meaning of this bit depends on the state of the FEN bit in the CTL0 register. 0h = The receiver is not empty. 1h = If the FIFO is disabled (FEN is 0), the receive holding register is empty. If the FIFO is enabled (FEN is 1), the receive FIFO is empty.
1	RESERVED	R	0h	
0	BUSY	R	0h	UART Busy This bit is set as soon as the transmit FIFO or TXDATA register becomes non-empty (regardless of whether UART is enabled) or if a receive data is currently ongoing (after the start edge have been detected until a complete byte, including all stop bits, has been received by the shift register). In IDLE_Line mode the Busy signal also stays set during the idle time generation. 0h = The UART is not busy. 1h = The UART is busy transmitting data. This bit remains set until the complete byte, including all stop bits, has been sent/received from/into the shift register.

16.3.31 IFLS (Offset = 110Ch) [Reset = 00000022h]

IFLS is shown in [Figure 16-46](#) and described in [Table 16-44](#).

Return to the [Summary Table](#).

The IFLS register is the interrupt FIFO level select register. You can use this register to define the levels at which the TX, RX and timeout interrupt flags are triggered. The interrupts are generated based on a transition through a level rather than being based on the level. That is, the interrupts are generated when the fill level progresses through the trigger level. For example, if the receive trigger level is set to the half-way mark, the interrupt is triggered when the receive FIFO is filled with two or more characters. Out of reset, the TXIFLSEL and RXIFLSEL bits are configured so that the FIFOs trigger an interrupt at the half-way mark.

Figure 16-46. IFLS

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED				RXTOSEL			
R/W-0h				R/W-0h			
7	6	5	4	3	2	1	0
RESERVED		RXIFLSEL		RESERVED		TXIFLSEL	
R/W-0h		R/W-2h		R/W-0h		R/W-2h	

Table 16-44. IFLS Field Descriptions

Bit	Field	Type	Reset	Description
31-12	RESERVED	R/W	0h	
11-8	RXTOSEL	R/W	0h	UART Receive Interrupt Timeout Select. When receiving no start edge for an additional character within the set bit times a RX interrupt is set even if the FIFO level is not reached. A value of 0 disables this function. 0h = Smallest value Fh = Highest possible value
7	RESERVED	R/W	0h	
6-4	RXIFLSEL	R/W	2h	UART Receive Interrupt FIFO Level Select The trigger points for the receive interrupt are as follows: Note: In ULP domain the trigger levels are used for: 0: LVL_1_4 4: LVL_FULL For undefined settings the default configuration is used. 0h = RX FIFO >= 1/4 full Note: For ULP Domain 1h = RX FIFO >= 1/4 full 2h = RX FIFO >= 1/2 full (default) 3h = RX FIFO >= 3/4 full 4h = RX FIFO is full Note: For ULP Domain 5h = RX FIFO is full 7h = RX FIFO >= 1 entry available Note: esp. required for DMA Trigger
3	RESERVED	R/W	0h	

Table 16-44. IFLS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2-0	TXIFLSEL	R/W	2h	UART Transmit Interrupt FIFO Level Select The trigger points for the transmit interrupt are as follows: Note: for undefined settings the default configuration is used. 1h = TX FIFO <= 3/4 empty 2h = TX FIFO <= 1/2 empty (default) 3h = TX FIFO <= 1/4 empty 5h = TX FIFO is empty 7h = TX FIFO >= 1 entry free Note: esp. required for DMA Trigger

16.3.32 IBRD (Offset = 1110h) [Reset = 00000000h]

IBRD is shown in [Figure 16-47](#) and described in [Table 16-45](#).

Return to the [Summary Table](#).

When changing the IBRD register, the new value does not take effect until transmission/reception of the current character is complete. Any changes to the baud-rate divisor must be followed by a write to the UARTLCRH register. See Baud-Rate Generation chapter for configuration details.

Figure 16-47. IBRD

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																DIVINT															
R/W-0h																R/W-0h															

Table 16-45. IBRD Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15-0	DIVINT	R/W	0h	Integer Baud-Rate Divisor 0h = Smallest value FFFFh = Highest possible value

16.3.33 FBRD (Offset = 1114h) [Reset = 00000000h]

FBRD is shown in [Figure 16-48](#) and described in [Table 16-46](#).

Return to the [Summary Table](#).

UART Fractional Baud-Rate Divisor Register The FBRD register is the fractional part of the baud-rate divisor value. All the bits are cleared on reset. When changing the FBRD register, the new value does not take effect until transmission/reception of the current character is complete. Any changes to the baud-rate divisor must be followed by a write to the UARTLCRH register. See Baud-Rate Generation chapter for configuration details.

Figure 16-48. FBRD

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED										DIVFRAC					
R/W-0h										R/W-0h					

Table 16-46. FBRD Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R/W	0h	
5-0	DIVFRAC	R/W	0h	Fractional Baud-Rate Divisor 0h = Smallest value 3Fh = Highest possible value

16.3.34 GFCTL (Offset = 1118h) [Reset = X]

GFCTL is shown in [Figure 16-49](#) and described in [Table 16-47](#).

Return to the [Summary Table](#).

This register control the glitch filter on the RX input.

Figure 16-49. GFCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED				CHAIN	AGFSEL		AGFEN
R/W-0h				R/W-0h	R/W-0h		R/W-0h
7	6	5	4	3	2	1	0
RESERVED		DGFSEL					
R/W-0h		R/W-0h					

Table 16-47. GFCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-12	RESERVED	R/W	0h	
11	CHAIN	R/W	0h	Analog and digital noise filters chaining enable. 0 DISABLE: When 0, chaining is disabled and only digital filter output is available to IP logic for sampling 1 ENABLE: When 1, analog and digital glitch filters are chained and the output of the combination is made available to IP logic for sampling 0h = Disabled 1h = Enabled
10-9	AGFSEL	R/W	0h	Analog Glitch Suppression Pulse Width This field controls the pulse width select for the analog glitch suppression on the RX line. See device data sheet for exact values. 0h = Pulses shorter then 5ns length are filtered. 1h = Pulses shorter then 10ns length are filtered. 2h = Pulses shorter then 25ns length are filtered. 3h = Pulses shorter then 50ns length are filtered.
8	AGFEN	R/W	0h	Analog Glitch Suppression Enable 0h = Analog Glitch Filter disable 1h = Analog Glitch Filter enable
7-6	RESERVED	R/W	0h	
5-0	DGFSEL	R/W	0h	Glitch Suppression Pulse Width This field controls the pulse width select for glitch suppression on the RX line. The value programmed in this field gives the number of cycles of functional clock up to which the glitch has to be suppressed on the RX line. In IRDA mode: The minimum pulse length for receive is given by: $t(MIN) = (DGFSEL) / f(IRTCLK)$ 0h = Bypass GF 3Fh = Highest Possible Value

16.3.35 TXDATA (Offset = 1120h) [Reset = 00000000h]

TXDATA is shown in [Figure 16-50](#) and described in [Table 16-48](#).

Return to the [Summary Table](#).

UART Transmit Data Register. This register is the transmit data register (the interface to the FIFOs). For transmitted data, if the FIFO is enabled, data written to this location is pushed onto the transmit FIFO. If the FIFO is disabled, data is stored in the transmitter holding register (the bottom word of the transmit FIFO). A write to this register initiates a transmission from the UART.

Figure 16-50. TXDATA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																DATA															
R/W-0h																R/W-0h															

Table 16-48. TXDATA Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R/W	0h	
7-0	DATA	R/W	0h	Data Transmitted or Received Data that is to be transmitted via the UART is written to this field. When read, this field contains the data that was received by the UART. 0h = Smallest value FFh = Highest possible value

16.3.36 RXDATA (Offset = 1124h) [Reset = 00000000h]

RXDATA is shown in [Figure 16-51](#) and described in [Table 16-49](#).

Return to the [Summary Table](#).

UART Receive Data Register. This register is the data receive register (the interface to the FIFOs). For received data, if the FIFO is enabled, the data byte and the 4-bit status (break, frame, parity, and overrun) is pushed onto the 12-bit wide receive FIFO. If the FIFO is disabled, the data byte and status are stored in the receiving holding register (the bottom word of the receive FIFO). The received data can be retrieved by reading this register.

Figure 16-51. RXDATA

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED			NERR	OVRERR	BRKERR	PARERR	FRMERR
R-0h			R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
DATA							
R-0h							

Table 16-49. RXDATA Field Descriptions

Bit	Field	Type	Reset	Description
31-13	RESERVED	R	0h	
12	NERR	R	0h	Noise Error. Writing to this bit has no effect. The flag is cleared by writing 1 to the NERR bit in the UART EVENT ICLR register. 0h = No noise error occurred 1h = Noise error occurred during majority voting
11	OVRERR	R	0h	UART Receive Overrun Error Writing to this bit has no effect. The flag is cleared by writing 1 to the OVRERR bit in the UART EVENT ICLR register. In case of a receive FIFO overflow, the FIFO contents remain valid because no further data is written when the FIFO is full. Only the contents of the shift register are overwritten. The CPU must read the data in order to empty the FIFO. 0h = No data has been lost due to a receive overrun. 1h = New data was received but could not be stored, because the previous data was not read (resulting in data loss).
10	BRKERR	R	0h	UART Break Error Writing to this bit has no effect. The flag is cleared by writing 1 to the BRKERR bit in the UART EVENT ICLR register. This error is associated with the character at the top of the FIFO. When a break occurs, only one 0 character is loaded into the FIFO. The next character is only enabled after the receive data input goes to a 1 (marking state) and the next valid start bit is received. 0h = No break condition has occurred 1h = A break condition has been detected, indicating that the receive data input was held low for longer than a full-word transmission time (defined as start, data, parity, and stop bits).
9	PARERR	R	0h	UART Parity Error Writing to this bit has no effect. The flag is cleared by writing 1 to the PARERR bit in the UART EVENT ICLR register. 0h = No parity error has occurred 1h = The parity of the received data character does not match the parity defined by bits 2 and 7 of the UARTLCRH register.

Table 16-49. RXDATA Field Descriptions (continued)

Bit	Field	Type	Reset	Description
8	FRMERR	R	0h	UART Framing Error Writing to this bit has no effect. The flag is cleared by writing 1 to the FRMERR bit in the UART EVENT ICLR register. This error is associated with the character at the top of the FIFO. 0h = No framing error has occurred 1h = The received character does not have a valid stop bit sequence, which is one or two stop bits depending on the UARTLCRH.STP2 setting (a valid stop bit is 1).
7-0	DATA	R	0h	Received Data. When read, this field contains the data that was received by the UART. 0h = Smallest value FFh = Highest possible value

16.3.37 LINCNT (Offset = 1130h) [Reset = 00000000h]

LINCNT is shown in [Figure 16-52](#) and described in [Table 16-50](#).

Return to the [Summary Table](#).

UART LIN Mode Counter Register

Figure 16-52. LINCNT

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																VALUE															
R/W-0h																R/W-0h															

Table 16-50. LINCNT Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15-0	VALUE	R/W	0h	16 bit up counter clocked by the functional clock of the UART. 0h = Smallest value FFFFh = Highest possible value

16.3.38 LINCTL (Offset = 1134h) [Reset = 00000000h]

LINCTL is shown in [Figure 16-53](#) and described in [Table 16-51](#).

Return to the [Summary Table](#).

UART LIN Mode Control Register

Figure 16-53. LINCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED	LINC0_MATCH	LINC1CAP	LINC0CAP	RESERVED	CNTRXLOW	ZERONE	CTRENA
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 16-51. LINCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-7	RESERVED	R/W	0h	
6	LINC0_MATCH	R/W	0h	Counter Compare Match Mode When this bit is set to 1 a counter compare match with LINC0 register triggers an LINC0 interrupt when enabled. 0h = Counter compare match mode disabled (capture mode enabled) 1h = Counter compare match enabled (capture mode disabled)
5	LINC1CAP	R/W	0h	Capture Counter on positive RXD Edge. When enabled the counter value is captured to LINC1 register on each positive RXD edge. A LINC1 interrupt is triggered when enabled. 0h = Capture counter on positive UARTxRXD edge disabled 1h = Capture counter on positive UARTxRXD edge enabled
4	LINC0CAP	R/W	0h	Capture Counter on negative RXD Edge. When enabled the counter value is captured to LINC0 register on each negative RXD edge. A LINC0 interrupt is triggered when enabled. 0h = Capture counter on negative UARTxRXD edge disabled 1h = Capture counter on negative UARTxRXD edge enabled
3	RESERVED	R/W	0h	
2	CNTRXLOW	R/W	0h	Count while low Signal on RXD When counter is enabled and the signal on RXD is low, the counter increments. 0h = Count while low Signal on UARTxRXD disabled 1h = Count while low Signal on UARTxRXD enabled
1	ZERONE	R/W	0h	Zero on negative Edge of RXD. When enabled the counter is set to 0 and starts counting on a negative edge of RXD 0h = Zero on negative edge disabled 1h = Zero on negative edge enabled
0	CTRENA	R/W	0h	LIN Counter Enable. LIN counter will only count when enabled. 0h = Counter disabled 1h = Counter enabled

16.3.39 LINC0 (Offset = 1138h) [Reset = 00000000h]

LINC0 is shown in [Figure 16-54](#) and described in [Table 16-52](#).

Return to the [Summary Table](#).

UART LIN Mode Capture 0 Register

Figure 16-54. LINC0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																DATA															
R/W-0h																R/W-0h															

Table 16-52. LINC0 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15-0	DATA	R/W	0h	16 Bit Capture / Compare Register Captures current LINCTR value on RXD falling edge and can generate a LINC0 interrupt when capture is enabled (LINC0CAP = 1). If compare mode is enabled (LINC0_MATCH = 1), a counter match can generate a LINC0 interrupt. 0h = Smallest value FFFFh = Highest possible value

16.3.40 LINC1 (Offset = 113Ch) [Reset = 00000000h]

LINC1 is shown in [Figure 16-55](#) and described in [Table 16-53](#).

Return to the [Summary Table](#).

UART LIN Mode Capture 1 Register

Figure 16-55. LINC1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																DATA															
R/W-0h																R/W-0h															

Table 16-53. LINC1 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15-0	DATA	R/W	0h	16 Bit Capture / Compare Register Captures current LINCTR value on RXD rising edge and can generate a LINC1 interrupt when capture is enabled (LINC1CAP = 1) 0h = Smallest value FFFFh = Highest possible value

16.3.41 IRCTL (Offset = 1140h) [Reset = X]

IRCTL is shown in [Figure 16-56](#) and described in [Table 16-54](#).

Return to the [Summary Table](#).

IrDA Control Register

Figure 16-56. IRCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED						IRRXPPL	RESERVED
R/W-0h						R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
IRTXPL						IRTXCLK	IREN
R/W-0h						R/W-0h	R/W-0h

Table 16-54. IRCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-10	RESERVED	R/W	0h	
9	IRRXPPL	R/W	0h	IrDA receive input UCAXRXD polarity 0h = HIGH : IrDA transceiver delivers a high pulse when a light pulse is seen 1h = LOW : IrDA transceiver delivers a low pulse when a light pulse is seen
8	RESERVED	R/W	0h	
7-2	IRTXPL	R/W	0h	Transmit pulse length. Pulse length t(PULSE) = (IRTXPLx + 1) / [2 * f(IRTXCLK)] (IRTXCLK = functional clock of the UART) 0h = Smallest value 3Fh = Highest possible value
1	IRTXCLK	R/W	0h	IrDA transmit pulse clock select 0h (R/W) = IrDA encode data is based on the functional clock. 1h (R/W) = IrDA encode data is based on the Baud Rate clock< when select 8x oversampling, the IRTXPL cycle should less 8; when select 16x oversampling, the IRTXPL cycle should less 16.
0	IREN	R/W	0h	IrDA encoder/decoder enable 0h (R/W) = IrDA encoder/decoder disabled 1h (R/W) = IrDA encoder/decoder enabled

16.3.42 AMASK (Offset = 1148h) [Reset = 000000FFh]

AMASK is shown in [Figure 16-57](#) and described in [Table 16-55](#).

Return to the [Summary Table](#).

Self Address Mask Register The AMASK register is used to enable the address mask for 9-bit or Idle-Line mode. The address bits are masked to create a set of addresses to be matched with the received address byte. Used in DALI, UART 9-Bit or Idle-Line mode.

Figure 16-57. AMASK

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								VALUE							
R/W-0h																								R/W-FFh							

Table 16-55. AMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R/W	0h	
7-0	VALUE	R/W	FFh	Self Address Mask for 9-Bit Mode This field contains the address mask that creates a set of addresses that should be matched. A 0 bit in the MSK bitfield configures, that the corresponding bit in the ADDR bitfield of the UARTxADDR register is don't care. A 1 bit in the MSK bitfield configures, that the corresponding bit in the ADDR bitfield of the UARTxADDR register must match. 0h = Smallest value FFh = Highest possible value

16.3.43 ADDR (Offset = 114Ch) [Reset = 00000000h]

ADDR is shown in [Figure 16-58](#) and described in [Table 16-56](#).

Return to the [Summary Table](#).

Self Address Register The ADDR register is used to write the specific address that should be matched with the receiving byte when the Address Mask (AMASK) is set to FFh. This register is used in conjunction with AMASK to form a match for address-byte received.

Used in DALI, UART 9-Bit or Idle-Line mode.

Figure 16-58. ADDR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																VALUE															
R/W-0h																R/W-0h															

Table 16-56. ADDR Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R/W	0h	
7-0	VALUE	R/W	0h	Self Address for 9-Bit Mode This field contains the address that should be matched when UARTxAMASK is FFh. 0h = Smallest value FFh = Highest possible value

16.3.44 CLKDIV2 (Offset = 1160h) [Reset = 00000000h]

CLKDIV2 is shown in [Figure 16-59](#) and described in [Table 16-57](#).

Return to the [Summary Table](#).

This register is used to specify module-specific divide ratio of the functional clock.
(Only in UART extended)

Figure 16-59. CLKDIV2

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												RATIO			
R/W-0h												R/W-0h			

Table 16-57. CLKDIV2 Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R/W	0h	
2-0	RATIO	R/W	0h	Selects divide ratio of module clock 0h = Do not divide clock source 1h = Divide clock source by 2 2h = Divide clock source by 3 3h = Divide clock source by 4 4h = Divide clock source by 5 5h = Divide clock source by 6 6h = Divide clock source by 7 7h = Divide clock source by 8

This page intentionally left blank.



The serial peripheral interface (SPI) module provides a standardized serial interface to transfer data between MSPM0 devices and other external devices with SPI interface.

17.1 SPI Overview.....	882
17.2 SPI Operation.....	885
17.3 SPI Registers.....	898

17.1 SPI Overview

The SPI module provides a standardized serial interface to transfer data between MSPM0 devices and other external devices (such as a Sensors, Memory, ADCs, or DACs) using SPI protocols

17.1.1 Purpose of the Peripheral

The SPI module acts as a controller or peripheral interface for synchronous serial communication with peripheral devices and other controllers. The transmit and receive paths are buffered with internal, independent FIFO memories allowing up to 4 entries with 16-bit width. A DMA interface is also provided to allow the data exchange with the transmit and receive FIFOs.

17.1.2 Features

The SPI modules have the following features:

- Configurable as a controller or a peripheral
- Programmable clock bit rate and prescaler
- Separate transmit (TX) and receive (RX) first-in first-out buffers (FIFOs)
- Programmable data frame size from 4-bits to 16-bits (Controller Mode)
- Programmable data frame size from 7-bits to 16-bits (Peripheral Mode)
- Supports PACKEN feature that allows the packing of two 16 bit FIFO entries into a 32-bit value to improve CPU performance. However, not all devices support packing. Please check device specific data sheet.
- Interrupts for transmit and receive FIFOs, overrun and timeout interrupts, and DMA done
- Programmable SPI mode support Motorola SPI, MICROWIRE, or Texas Instruments format
- Single bit parity will be supported in both transmit and receive paths using the CTL1.PTEN and CTL1.PREN bits
- Direct memory access controller interface (DMA):
 - Separate channels for transmit and receive
 - Transfer complete interrupt

17.1.3 Functional Block Diagram

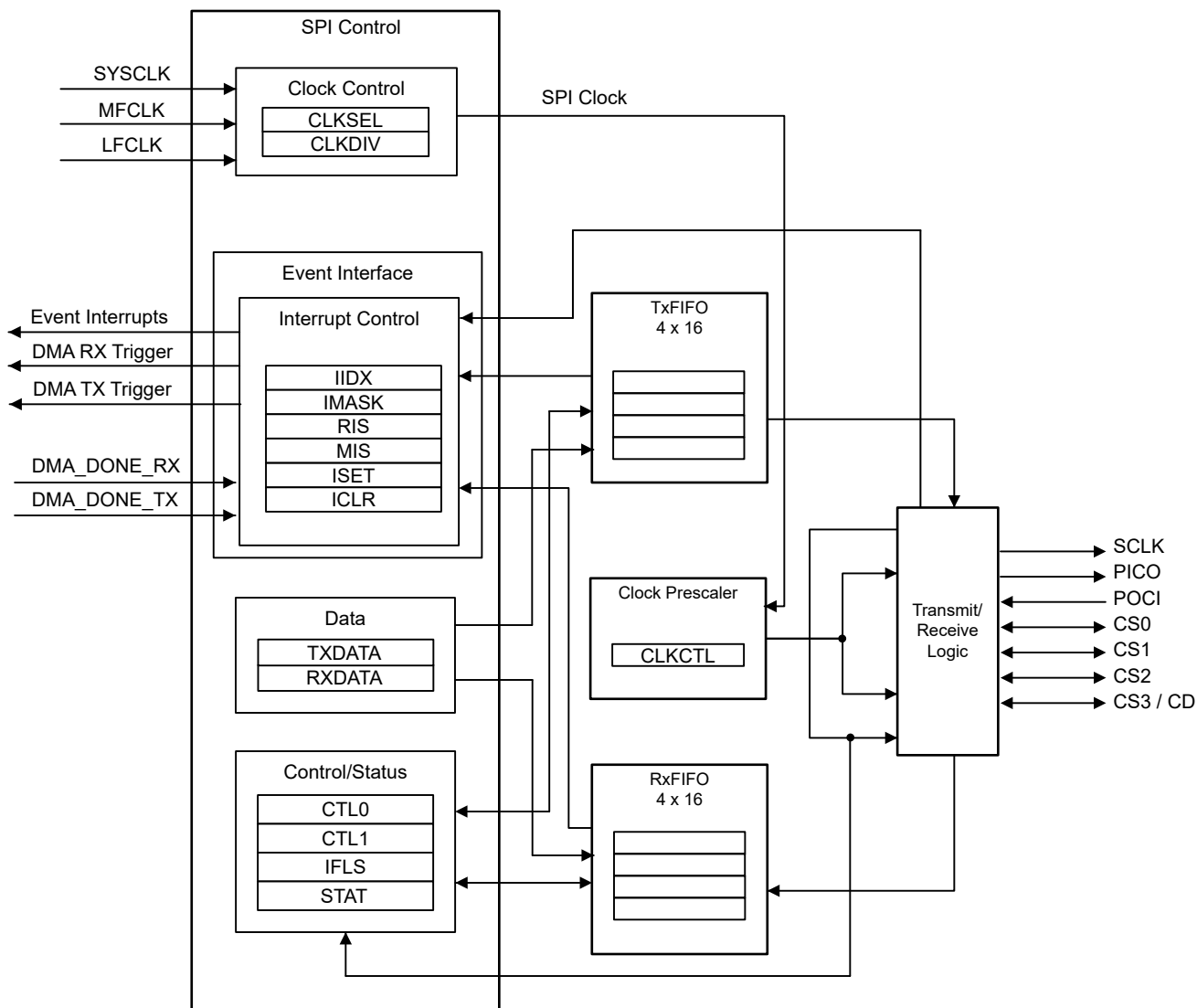


Figure 17-1. SPI Functional Block Diagram

17.1.4 External Connections and Signal Descriptions

Figure 17-2 and Table 17-1 show an overview of the pin functions for different operation modes of the SPI module.

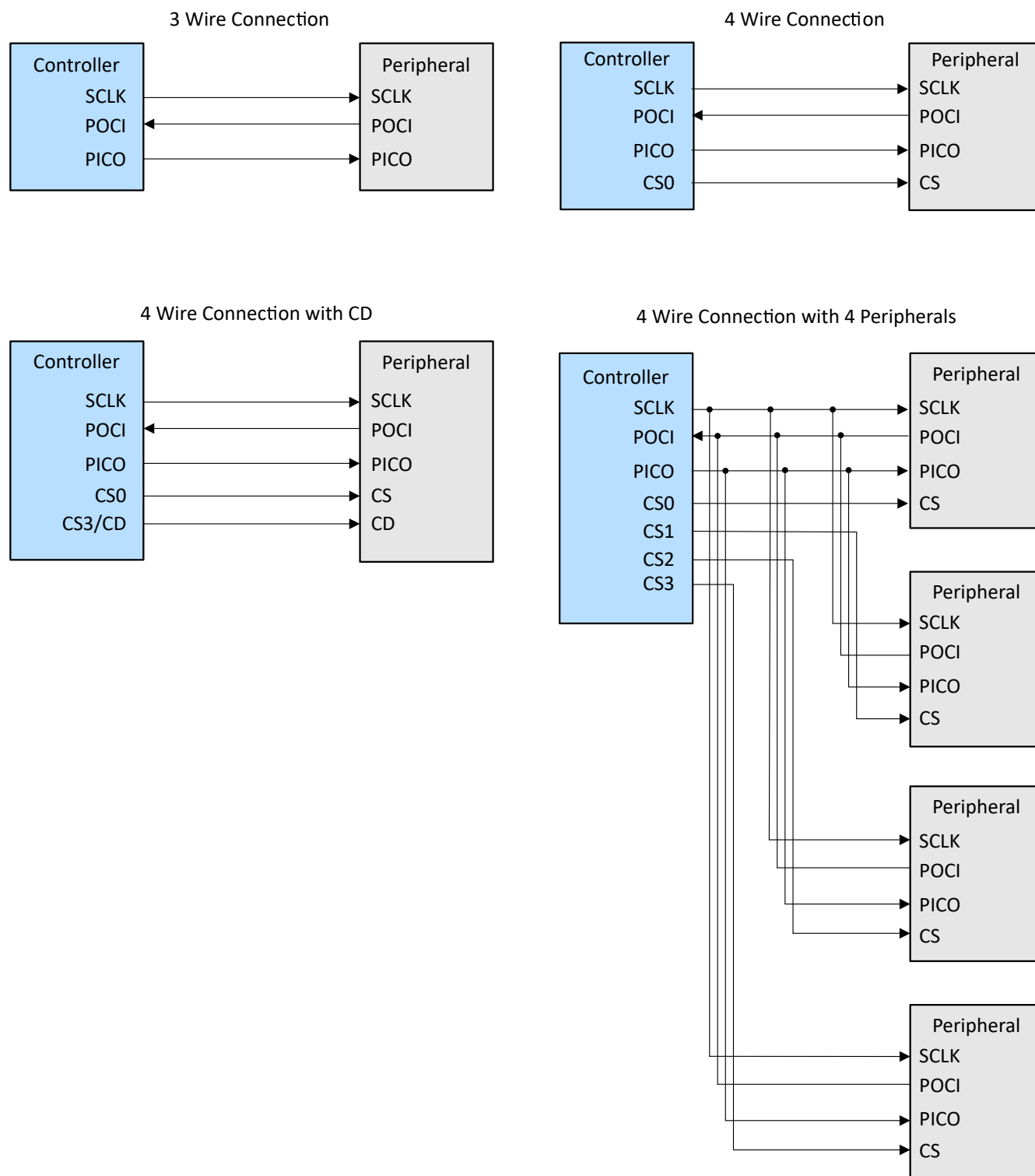


Figure 17-2. External Connections for Different SPI Configurations

Table 17-1. Pin Function Overview

Standard SPI	Feature
SCLK	SPI clock Controller mode: SCLK is an output Peripheral mode: SCLK is an input
PICO	Controller out, peripheral in Controller mode: PICO is the data output line Peripheral mode: PICO is the data input line
POCI	Controller in, peripheral out Controller mode: POCI is the data input line Peripheral mode: POCI is the data output line
CS0	Chip select signal 0, used in 4-pin mode
CS1	Chip select signal 1, used in 4-pin mode
CS2	Chip select signal 2, used in 4-pin mode
CS3/CD	Chip select signal 3 or Command, used in 4-pin mode

17.2 SPI Operation

17.2.1 Clock Control

The SPI internal functional clock is selected and divided from the clock sourced to this module.

- Use SPIx.CLKSEL register to select the source of the SPI functional clock.
 - BUSSCLK: the current bus clock is selected as the source for SPI. The current bus clock depends on power domain. If the SPI instance is in power domain 1 (PD1) please refer to [MCLK](#), if the SPI instance is in power domain 0 (PD0) refer to [ULPCLK](#).
 - MFCLK: MFCLK is selected as the source for SPI, refer to [MFCLK](#).
 - LFCLK: LFCLK is selected as the source for SPI, refer to [LFCLK](#).
- Use SPIx.CLKDIV register to select the divide ratio of the SPI function clock. Options are from divide by 1 to 8.

The SPI module must be enabled before being configured for use by using the ENABLE bit in SPIx.PWREN register (see [peripheral power enable](#)). When the SPI will be setup or the configuration should be changed the ENABLE bit should be cleared to avoid unpredictable behavior during the updates or for the first data receive or transmitted afterward.

The maximum SPI frequency supported with controller and peripheral mode depends on the device clock option and IO option. Please refer to specific device data sheet spec for more information.

17.2.2 General Architecture

17.2.2.1 Chip Select and Command Handling

17.2.2.1.1 Chip Select Control

One can configure the SPI to be controller mode by setting the CTL1.MS bit to 1, and in peripheral mode by clearing the CTL1.MS bit.

The CTL0.CSSEL bit selects which connected peripheral is addressed by the up to 4 CS signals. The bits are controlled by the SPI module in controller or target/peripheral mode. The selected signal is controlled during the transfers.

The chip select signal needs to be provided by the controller in four-wire mode and the chip select polarity can be inverted by configuring the PINCM.CSx.INV register.

In peripheral mode, the clock is provided by the controller and used by the peripheral to capture the data. The peripheral has the option to operate in 3-wire or 4-wire mode. 4-wire mode only accepts data transfers if the CS is activated.

When the CTL0.CSCLR bit is set, the transmit/receive shift register counter is cleared automatically when the CS goes to the inactive state. When using the Motorola 4-wire or National Microwire mode, follow these constraints:

- The CS disable period must be longer than 2 functional clock cycles before the CS pin is re-asserted
- The CS lead time (CS active to the first bit clock edge) must be at least 2 SPI functional clock cycles

Following these constraints helps the peripheral to synchronize again on the controller in case of a disturbance or glitch on the clock line or during initialization. This bit is relevant only in the peripheral mode.

- CTL0.CSCLR = 0: The transmit/receive bit counter state is retained when the CS signal disables the peripheral.
- CTL0.CSCLR = 1: The transmit/receive bit counter is cleared when the CS signal disables the peripheral.

Note

The CSCLR function requires the CS disable pulse to be longer than 2 SPI function clock cycles to properly detect and clear the bit counter in the SPI. The CS lead time (CS active to the first bit clock edge) also needs to be at least 2 SPI function clock cycles.

17.2.2.1.2 Command Data Control

When using the Motorola frame format, the CDMODE bit can be set to use the CS3/CD line as signal to distinguish between Command and Data information. This is often used for LCD or data storage devices.

- CD level low: command function
- CD level high: data function

The CTL1.CDMODE can be written with a value of 1-14 to specify the number of bytes and the CD line will go low for the given numbers of bytes which are sent by the SPI, starting with the next value to be transmitted. After the number of bytes are transmitted the CD will go high automatically. If a value of 0xF is set the C/D stays low permanently, a value of 0 set the CD line to high immediately after the current character has been transmitted.

This option is only available in controller mode. CTL1.CDENABLE can only be updated when the SPI module is disabled, CTL1.CDMODE can be updated between the different data packages. The counter will be reset with CDENABLE or SPI ENABLE set to disabled. Before setting a new value in CTL1.CDMODE the status of the FIFO should be checked to be empty and the SPI should be in Idle mode.

When writing a new value into CTL1.CDMODE, the internal counter will be reset and the new value will be used for counting. If the counter did count down to 0 and another command package should be sent the CDMODE needs to be set first again, otherwise the next data is sent as data with the CD pin signaling data mode.

17.2.2.2 Data Format

The control bit CTL1.MSB defines the direction of the data input and output with most-significant-bit (MSB) or least-significant-bit (LSB) first. If the parity is enabled the parity bits is always received as last bit.

With the control register bits CTL0.DSS the bit length per transfer will be defined between 4-16 bits for Controller mode and 7-16 bits for Peripheral mode.

A transfer will be triggered with writing to the TX buffer register. The data write needs to have at least the number of bits of the transfer. For example, if only a byte is written to the TX buffer but the length of the transfer is > 8 the missing bits will be filled with 0s. On the receive path the received data will be moved to the RXFIFO or RX buffer after the number of bit defined in CTL0.DSS register have been received.

The RX and TX buffer shall be accessed with at least the bits covering one transfer.

- 4-8 bits : byte access (Peripheral mode: 7-8 Bits)
- 9-16 bits : 16 bit access

Clock polarity (CTL0.SPO) is used to control the clock polarity when data is not being transferred and it is only used in the [Motorola SPI frame](#) mode.

- 0h = peripheral produces a steady state LOW value on the CLKOUT pin when data is not being transferred.
- 1h = peripheral produces a steady state HIGH value on the CLKOUT pin when data is not being transferred.

Clock phase (CTL0.SPH) bit selects the clock edge that captures data and enables it to change state. It has the most impact on the first bit transmitted by either permitting or not permitting a clock transition before the first data capture edge. Please refer to [Motorola SPI frame](#) mode section to check the diagrams.

- 0h = Data is captured on the first clock edge transition.
- 1h = Data is captured on the second clock edge transition.

The SPI can be configured to work in Peripheral mode with CTL1.MS bit = 0. In Peripheral mode the clock is provided by the controller and available for the peripheral on the CLK pins which needs to be configured for input. The Clock Select and divider control bits are not used. The CS input signal is used to select/enable the data receive path of the peripheral in 4 wire mode.

The SPI can be configured to work as Controller with CTL1.MS bit = 1. In Controller mode the clock needs to be generated by selecting the available clock sources with the clock select bits. It also needs to control the CS signal depending on the selected protocol.

When setting the CTL1.PEN bit the last bit will be used as parity to evaluate the integrity of the previous bits. The CTL1.PES bit selects the parity mode as even or odd. When detecting a fault, the interrupt flag RIS.PER is set to mark the data as invalid. Parity checking is a feature to improve the robustness of the communication.

17.2.2.3 Delayed data sampling

In circumstances when the input data arrives at the POCI pin with some delay due to runtime conditions and the following input data sampling stage, the previous data would be sampled at the sampling clock edge. To compensate for such condition, a delayed sampling can be set with the CLKCTL.DSAMPLE bits. The delayed sampling is only available in controller mode. The delay can be adjusted in steps of SPI input clock steps with setting the control register bits CLKCTL.DSAMPLE. The maximum allowed delay should not exceed the length of one data frame.

17.2.2.4 Clock Generation

The SPI includes a programmable bit rate clock divider and prescaler to generate the serial output clock (SCLK).

Bit rates supported are up to, the input clocks divided by 2. The input clock selection depends on the specific device, refer to the device data sheet and [Clock Control](#) section.

The SPI functionality can work with any of these selected inputs : SYSCLK, MFCLK and LFCLK

"SPI Clock" is the output after clock division performed according to ratio selected by the CLKDIV register. SPI clock = Selected input clock / (1 + CLKDIV)

SPI Sampling Clock (SCLK) is the output after dividing the SPI Clock by the Prescaler value. $SCLK = SPI\ Clock / ((1 + SCR) * 2)$

If the factor of two (*2) is set by CLKDIV the input clock must be at least 2 times faster than SPI clock.

17.2.2.5 FIFO Operation

Transmit FIFO

The TX FIFO is a 16-bit-wide, 4-location-deep, first-in first-out memory buffer. The CPU writes data to the FIFO by writing the SPI Data Register TXDATA.DATA, and data is stored in the FIFO until it is read out by the transmission logic.

When configured as a controller or a peripheral, parallel data is written into the TX FIFO before serial conversion and transmission to the attached peripheral or controller, respectively, through the PICO or POCI pin.

In peripheral mode, the SPI transmits data each time the controller initiates a transaction. If the TX FIFO is empty and the controller initiates a transfer, the peripheral transmits the most-recent value written to the transmit

FIFO. User or software is responsible to make valid data available to the FIFO as needed. The SPI can be configured to generate an interrupt or a DMA request when the FIFO is empty. The transmit FIFO has a TXFIFO_UNF interrupt to indicate a FIFO underflow condition.

Receive FIFO

The RX FIFO is a 16-bit-wide, 4-location-deep, first-in-first-out memory buffer. Received data from the serial interface is stored in the buffer until read out by the CPU or DMA, which accesses the read FIFO by reading the SPIx.RXDATA register.

When configured as a controller or peripheral, serial data is received through the POCI or PICO pin. As the access pointer for the FIFO will be updated with each access, the data needs to be accessed by single transfers.

With the FIFO fill level trigger signals located in the STAT register (TFE, TNF, RFE, RNF) the FIFO buffer allows an application to continuously stream serial data in one buffer while the application moves or process the data from the other buffer. If the FIFO is full and new data is written into the FIFO without reading data the RXFIFO overflow event is set. The receive FIFO has a RXFULL interrupt to indicate a FIFO full condition.

17.2.2.6 Loopback mode

The SPI module can be placed into an internal loopback mode for diagnostic or debug work by setting the LBM bit in the CTL1 register. In loopback mode, the data from the TX FIFO can be serially transmitted into the RX FIFO. The data from the RX FIFO can be read to check whether correct transmission has occurred or not. The external toggling of the IOs has no effect when the module is set in the internal loopback mode.

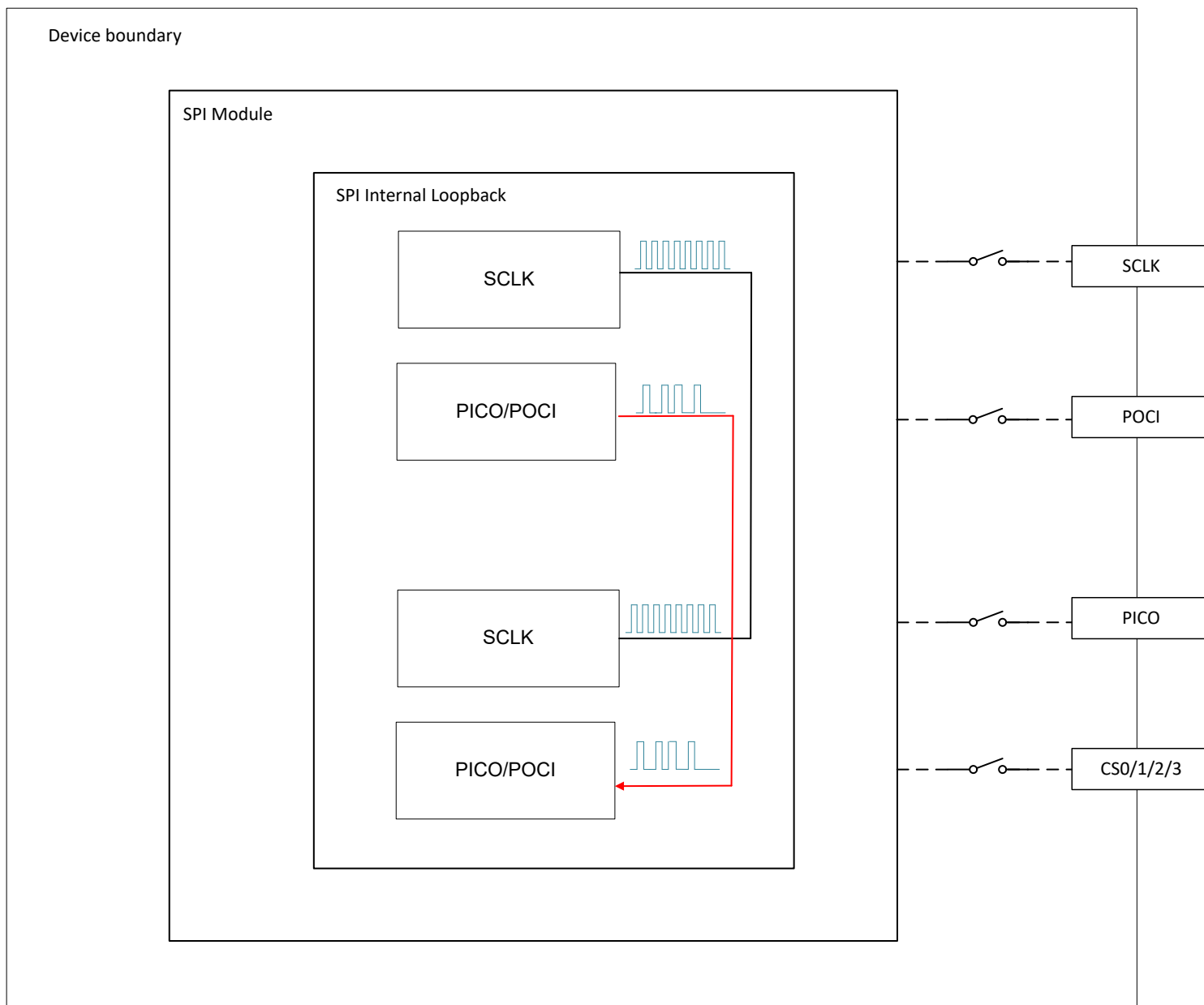


Figure 17-3. Internal Loop Back Mode

17.2.2.7 DMA Operation

The SPI provides an interface to the DMA controller with separate channels for transmit and receive. The DMA operation of the SPI is enabled through the SPI Event and DMA register. When DMA operation is enabled, the SPI asserts a DMA request on the receive or transmit channel when the associated FIFO can transfer data.

For the receive channel a transfer request is asserted whenever the amount of data in the receive FIFO is at or above the FIFO trigger level configured using the RXIFLSEL bit in IFLS register or the receive timeout has triggered, in this case the amount of data received so far will be transmitted.

For the transmit channel a transfer request is asserted whenever the transmit FIFO contains fewer characters than the FIFO trigger level configured using the TXIFLSEL bit in IFLS register. The DMA transfer requests are handled automatically by the DMA controller depending on how the DMA channel is configured.

The DMA transfers can be configured and aligned between the data width of the SPI transfers and the bus accesses width of 8/16 bits to make an efficient usage of the bus. The trigger and transfers are independent for receive and transmit.

See more information about the interrupt and event in [Section 17.2.6.2](#) section.

17.2.2.8 Repeat Transfer mode

With the CTL1.REPEATTX bits the last character transmit will be repeated as defined by the register bits. A value of 0 in CTL1.REPEATTX bits will disable this mode and this function is only available in Controller mode. The transfer will be started with writing a data into the TX Buffer. Then the data will be repeatedly sent with the given value. The behavior is identical as if the data would be written into the TX Buffer that many times as defined by the value here. It can be used to clean a transfer or to pull a certain amount of data from a peripheral.

When REPEATTX is used it needs to be aligned with the data in the FIFO. So the below shown sequence should be used:

- Wait and check till FIFO is empty
- Setup REPEATTX
- Write to TXDATA / FIFO
- Wait till requested data has been received

17.2.2.9 Low Power Mode

The SPI module is located in power domain 1 (PD1) and as such is only active in RUN and SLEEP modes. If the SPI module is enabled by application software, an entry into STOP or STANDBY low-power mode forces the SPI module to be temporarily disabled while the device is in STOP or STANDBY mode.

17.2.3 Protocol Descriptions

The protocol format mode can be selected by using CTL0.FRF register. The supported options include Motorola 3 wire, Motorola 4 wire, Texas Instruments Synchronous and MICROWIRE.

17.2.3.1 Motorola SPI Frame Format

The Motorola SPI interface is a 4-wire interface where the CS signal behaves as a peripheral select. In the 3-wire mode the CS signals is not required and the module behaves as if always selected. The main feature of the Motorola SPI format is that the inactive state and phase of the SCLK signal can be programmed through the SPO and SPH bits in the SPIx.CTL0 control register.

SPO Clock Polarity Bit

If the CTL0.SPO clock polarity control bit is clear, the bit produces a steady-state low value on the SCLK pin when data is not being transferred. If the CTL0.SPO bit is set, the bit places a steady-state high value on the SCLK pin when data is not being transferred.

SPH Phase-Control Bit

The CTL0.SPH phase-control bit selects the clock edge that captures data, and allows it to change state. The state of this bit has the most impact on the first bit transmitted, by either allowing or not allowing a clock transition before the first data capture edge. If the CTL0.SPH phase-control bit is clear, data is captured on the first clock edge transition. If the SPH bit is set, data is captured on the second clock edge transition.

Motorola SPI Frame Format with SPO = 0 and SPH = 0

[Figure 17-4](#) shows signal sequences for Motorola SPI format with SPO = 0 and SPH = 0.

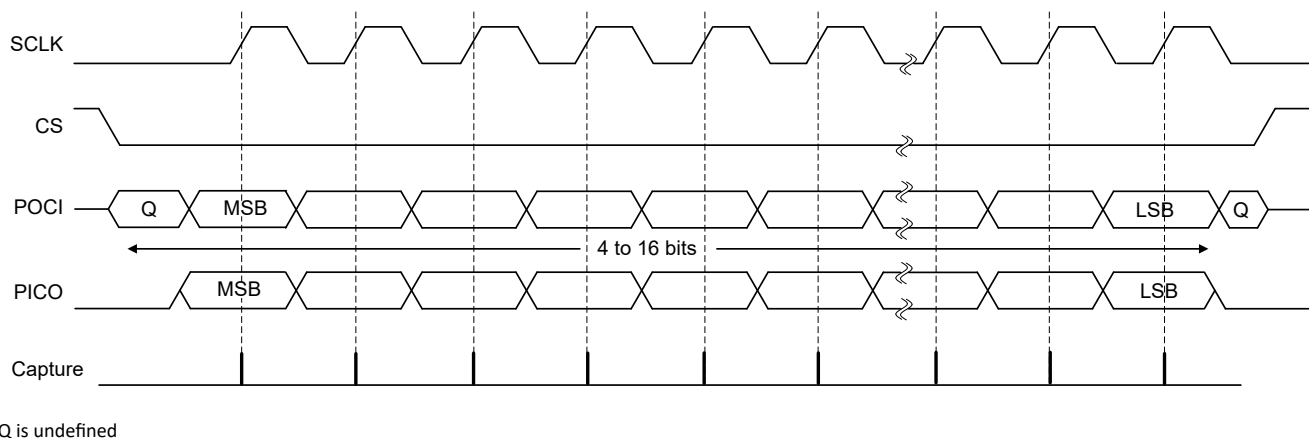


Figure 17-4. Motorola SPI Format With SPO = 0 and SPH = 0

In this configuration, the following occurs during idle periods:

- SCLK is forced low
- CS is forced high
- The transmit data line PICO is forced low
- When the SPI is configured as a controller, it enables the SCLK pin
- When the SPI is configured as a peripheral, it disables the SCLK pin

If the SPI is enabled and valid data is in the TX FIFO, the CS controller signal is driven low at the start of transmission which causes enabling of peripheral data onto the POCI input line of the controller. The controller PICO output pin is enabled.

One-half SCLK period later, valid controller data is transferred to the PICO pin. Once both the controller and peripheral data are set, the SCLK controller clock pin goes high after an additional one-half SCLK period. The data is now captured on the rising edges and propagated on the falling edges of the SCLK signal.

For a single-word transmission after all bits of the data word are transferred, the CS line is returned to its IDLE high state one SCLK period after the last bit is captured. For continuous back-to-back transmissions, the CS signal must pulse high between each data word transfer because the peripheral-select pin freezes the data in its serial peripheral register and does not allow altering of the data if the SPH bit is clear. The controller device must raise the CS pin of the peripheral device between each data transfer to enable the serial peripheral data write. When the continuous transfer completes, the CS pin is returned to its IDLE state one SCLK period after the last bit is captured.

Motorola SPI Frame Format with SPO = 0 and SPH = 1

Figure 17-5 shows the signal sequence for Motorola SPI format with SPO = 0 and SPH = 1.

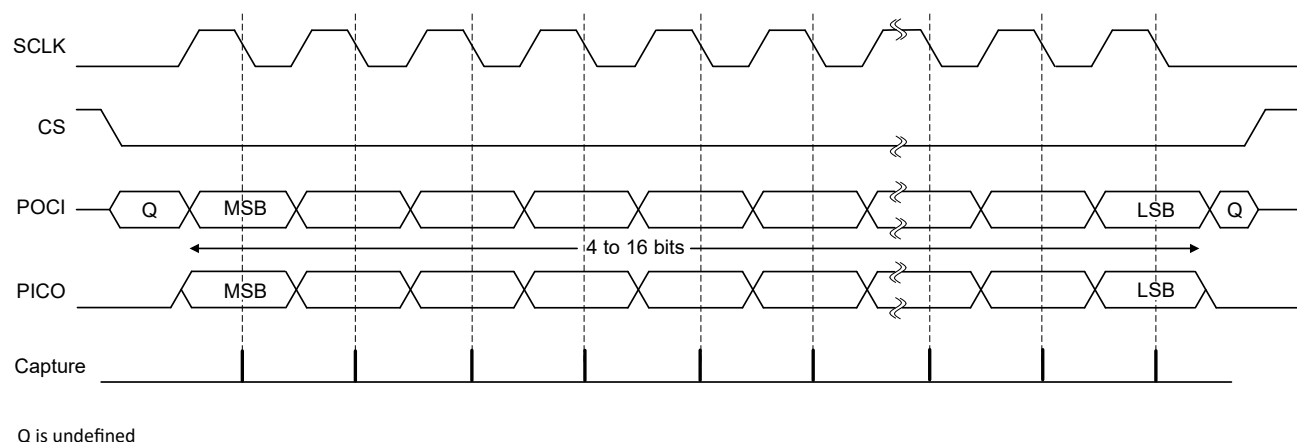


Figure 17-5. Motorola SPI Frame Format With SPO = 0 and SPH = 1

In this configuration, the following occurs during idle periods:

- SCLK is forced low
- CS is forced high
- The transmit data line PICO is forced low
- When the SPI is configured as a controller, it enables the SCLK pin
- When the SPI is configured as a peripheral, it disables the SCLK pin

If the SPI is enabled and valid data is in the TX FIFO, the CS controller signal goes low at the start of transmission. The controller PICO output is enabled. After an additional one-half SCLK period, both controller and peripheral valid data are enabled onto their respective transmission lines. At the same time, SCLK is enabled with a rising-edge transition. Data is then captured on the falling edges and propagated on the rising edges of the SCLK signal.

For a single-word transfer, after all bits are transferred, the CS line is returned to its IDLE high state one SCLK period after the last bit is captured. For continuous back-to-back transfers, the CS pin is held low between successive data words and terminates like a single-word transfer.

Motorola SPI Frame Format with SPO = 1 and SPH = 0

Figure 17-6 shows signal sequences for Motorola SPI format with SPO = 1 and SPH = 0.

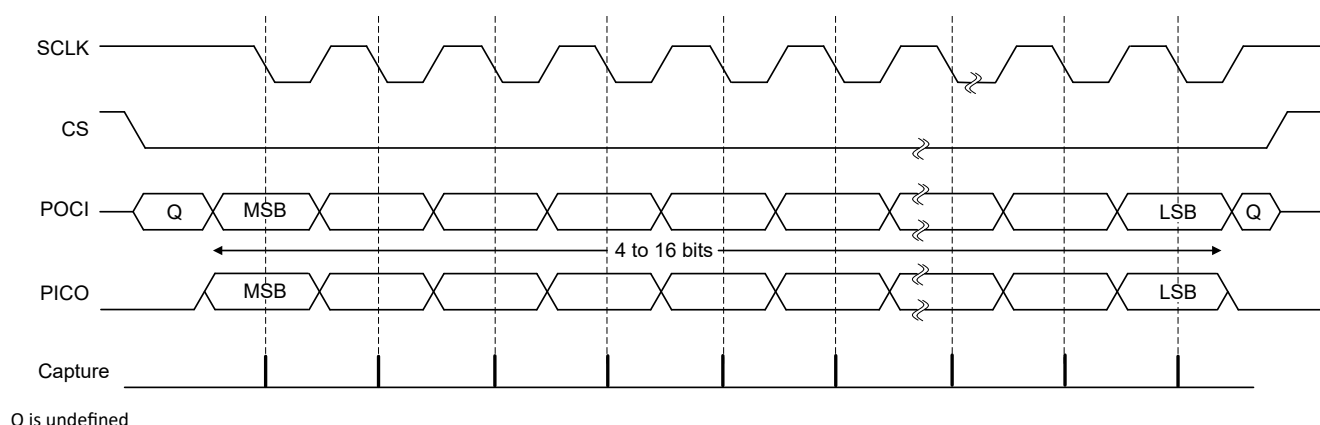


Figure 17-6. Motorola SPI Frame Format With SPO = 1 and SPH = 0

In this configuration, the following occurs during idle periods:

- SCLK is forced high
- CS is forced high

- The transmit data line PICO is arbitrarily forced low
- When the SPI is configured as a controller, it enables the SCLK pin
- When the SPI is configured as a peripheral, it disables the SCLK pin

If the SPI is enabled and valid data is in the TX FIFO, the SPI CS controller signal goes low at the start of transmission and transfers peripheral data onto the POCI line of the controller immediately. The controller PICO output pin is enabled.

One-half SCLK period later, valid controller data is transferred to the PICO line. When both the controller and peripheral data have been set, the SCLK controller clock pin becomes low after one additional half SCLK period. Data is captured on the falling edges and propagated on the rising edges of the SCLK signal.

For a single-word transmission after all bits of the data word are transferred, the CS line is returned to its IDLE high state one SCLK period after the last bit is captured. For continuous back-to-back transmissions, the CS signal must pulse high between each data word transfer as the peripheral-select pin freezes the data in its serial peripheral register and keeps it from being altered if the SPH bit is clear. The controller device must raise the CS pin of the peripheral device between each data transfer to enable the serial peripheral data write. When the continuous transfer completes, the CS pin returns to its IDLE state one SCLK period after the last bit is captured.

Motorola SPI Frame Format with SPO = 1 and SPH = 1

Figure 17-7 shows the signal sequence for Motorola SPI format with SPO = 1 and SPH = 1.

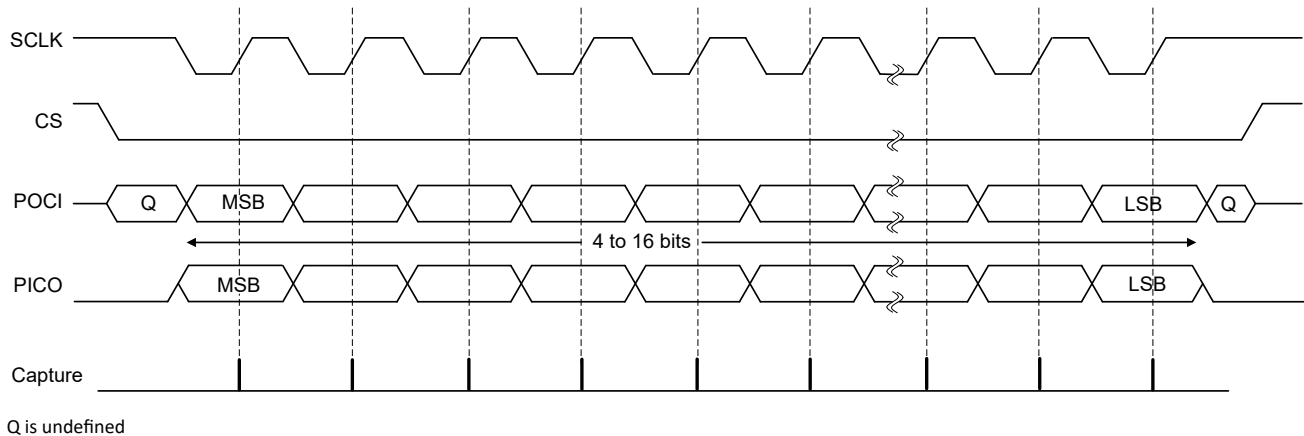


Figure 17-7. Motorola SPI Frame Format With SPO = 1 and SPH = 1

In this configuration, the following occurs during idle periods:

- SCLK is forced high
- CS is forced high
- The transmit data line PICO is arbitrarily forced low
- When the SPI is configured as a controller, it enables the SCLK pin
- When the SPI is configured as a peripheral, it disables the SCLK pin

If the SPI is enabled and valid data is in the TX FIFO, the start of transmission is signified by the CS controller signal going low. The controller PICO output pin is enabled. After an additional one-half SCLK period, both controller and peripheral data are enabled onto their respective transmission lines. At the same time, SCLK is enabled with a falling-edge transition. Data is then captured on the rising edges and propagated on the falling edges of the SCLK signal.

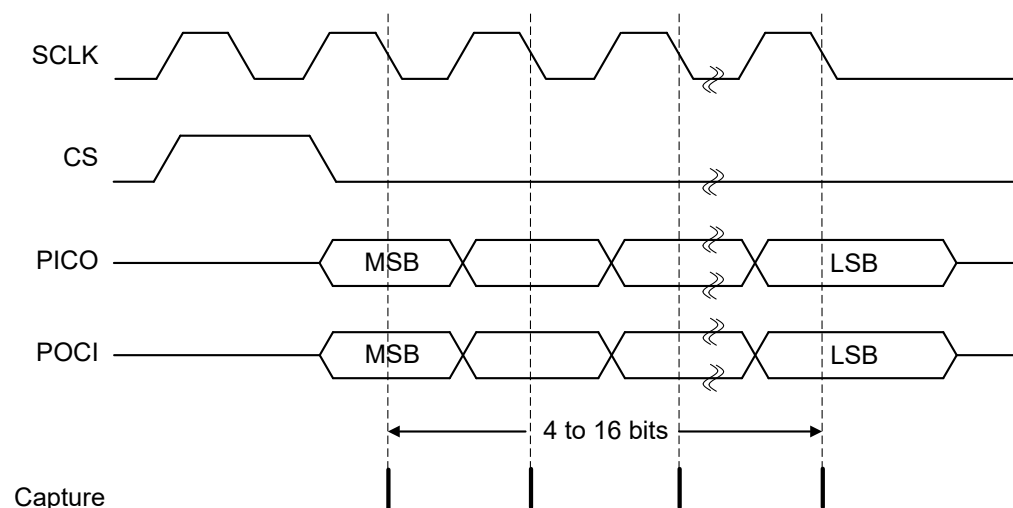
For a single word transmission, after all bits are transferred, the CS line returns to its IDLE high state one SCLK period after the last bit is captured. For continuous back-to-back transmissions, the CS pin remains in its active low state until the final bit of the last word is captured and then returns to its IDLE state. For continuous back-to-back transfers, the CS pin is held low between successive data words and terminates like a single-word transfer.

The serial clock (SCLK) is held inactive while the SPI is idle and SCLK transitions at the programmed frequency only during active transmission or reception of data. The IDLE state of SCLK provides a receive timeout indication that occurs when the RX FIFO still contains data after a timeout period.

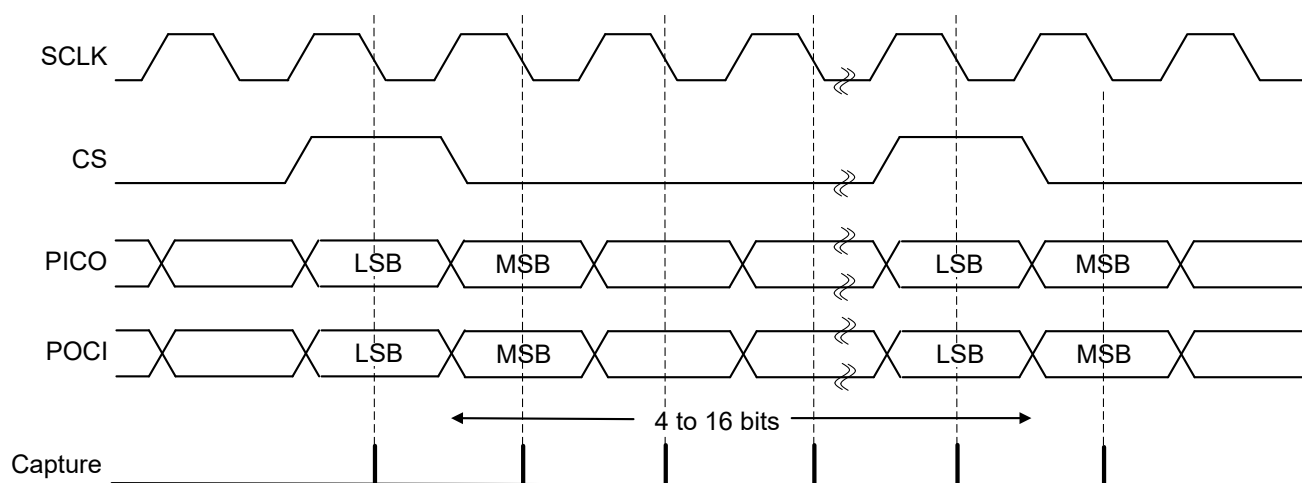
17.2.3.2 Texas Instruments Synchronous Serial Frame Format

The SPI peripheral is compatible with Texas Instruments Synchronous Serial frame format.

Figure 17-8 shows the TI synchronous serial frame format for a single and continuous transmitted frame.



Single Transmission Signal Sequences



Continuous Transmission Signal Sequences

Figure 17-8. TI Synchronous Serial Frame Format

SCLK and CS are forced low and the transmit data line PICO is put in tristate whenever the SPI is idle. When the bottom entry of the TX FIFO contains data, CS is pulsed high for one SCLK period. The transmitted value is also transferred from the TX FIFO to the serial shift register of the transmit logic. On the next rising edge of SCLK, the MSB of the 4- to 16-bit data frame is shifted out on the PICO pin. Likewise, the MSB of the received data is shifted onto the POCI pin by the off-chip serial peripheral device. Both the SPI and the off-chip serial peripheral device then clock each data bit into their serial shifter on each falling edge of SCLK. The received

data is transferred from the serial shifter to the RX FIFO on the first rising edge of SCLK after the least significant bit (LSB) is latched.

The serial clock (SCLK) is held inactive while the SPI is idle and SCLK transitions at the programmed frequency only during active transmission or reception of data. The IDLE state of SCLK provides a receive time-out indication that occurs when the RX FIFO still contains data after a time-out period.

17.2.4 Reset Considerations

Software Reset Considerations

A Software reset can be executed with setting the RESETASSERT together with the KEY in the RSTCTL register. An ongoing transfer will be terminated immediately and can leave the software in an undefined state. Therefore, before requesting a Reset an ongoing Transfer should be terminated.

Hardware Reset Considerations

A hardware reset also initializes the IO configuration. This sets the IOs to a high impedance state and the data lines can float. If this is critical for the application or connected devices on the SPI interface external pull up or down resistors might be required.

17.2.5 Initialization

To enable and initialize the SPI, the following steps are necessary:

1. Configure the IOMUX with the appropriate GPIO pins for which the SPI signals are multiplexed to

Note

Pull-ups can be used to avoid unnecessary toggles on the SPI pins, which can take the peripheral to a wrong state. In addition, if the SCLK signal is programmed to steady state High through the SPO bit in the CTL0 register, then software must also configure the GPIO port pin corresponding to the SCLK signal as a pull-up.

For each of the frame formats, the SPI is configured using the following steps:

1. Ensure that the ENABLE bit in the CTL1 register is clear before making any configuration changes.
2. Select and configure the clock prescale divisor by writing the CLKSEL and CLKDIV register.
3. Select whether the SPI is a controller or peripheral:
 - For controller operations, set the MS bit in the CTL1 register.
 - For peripheral mode, clear the MS bit in the CTL1 register.
4. Configure the clock divisor by writing the CLKCTL register.
5. Please note that a SPI Software reset (See section 15.3.4) is required when switching SPI protocol format.
6. Configure the CTL0 and CTL1 register with based on the desired protocol, data width and other special configurations.
7. Optionally configure DMA
8. Enable the SPI by setting the ENABLE bit in the CTL1 register.

17.2.6 Interrupt and Events Support

The SPI module contains three [event publishers](#) and no [event subscribers](#). One event publisher (CPU_INT) manages SPI interrupt requests (IRQs) to the CPU subsystem through a [static event route](#). The second and third event publishers (DMA_TRIG_RX, DMA_TRIG_TX) are used to setup the trigger signaling for the DMA through [DMA event route](#).

The SPI events are summarized in [Table 17-2](#).

Table 17-2. SPI Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU interrupt	Publisher	SPI	CPU Subsystem	Static route	CPU_INT registers	Fixed interrupt route from SPI to CPU

Table 17-2. SPI Events (continued)

Event	Type	Source	Destination	Route	Configuration	Functionality
DMA trigger	Publisher	SPI	DMA	DMA event route	DMA_TRIG_RX registers	Fixed interrupt route from SPI RX to DMA
DMA trigger	Publisher	SPI	DMA	DMA event route	DMA_TRIG_TX registers	Fixed interrupt route from SPI TX to DMA

17.2.6.1 CPU Interrupt Event Publisher (CPU_INT)

The SPI module provides 9 interrupt sources that can source a [CPU interrupt event](#). [Table 17-3](#) lists the CPU interrupt events from the SPI in order of decreasing priority.

Table 17-3. SPI CPU_INT Trigger Condition

IIDX STAT	Name	Description
0x01	RXFIFO_OVF	RXFIFO overflow event. This interrupt is set if an RX FIFO overflow has been detected.
0x02	PER	Parity error event. This bit is set if a Parity error has been detected.
0x03	RTOUT	Peripheral receive timeout event. When in peripheral mode and not receiving data for the CTL1.RXTIMEOUT selected number of functional clock cycles.
0x04	RX	Receive FIFO event. This interrupt is set if the selected receive FIFO level has been reached.
0x05	TX	Transmit FIFO event. This interrupt is set if the selected transmit FIFO level has been reached.
0x06	TXEMPTY	Transmit FIFO empty interrupt. This is set if all data in the transmit FIFO have been shifted out.
0x07	IDLE	SPI Idle. SPI has done finished transfers and changed into IDLE mode. This bit is set when STAT.BUSY bit goes low.
0x08	DMA_DONE1_RX	This interrupt is set if the RX DMA channel sends the DONE signal.
0x09	DMA_DONE1_TX	This interrupt is set if the TX DMA channel sends the DONE signal.

The CPU interrupt event configuration is managed with the CPU_INT event management registers. See [Section 7.2.5](#) for guidance on configuring the Event registers.

17.2.6.2 DMA Trigger Publisher (DMA_TRIG_RX, DMA_TRIG_TX)

DMA_TRIG_RX and DMA_TRIG_TX registers are used to setup the trigger signaling for the DMA. This can be setup in a flexible way to trigger the DMA for receive or transmit events with the trigger conditions in [Table 17-4](#) and [Table 17-5](#).

DMA_TRIG_RX is used for triggering the DMA to do a receive data transfer and DMA_TRIG_TX is used for triggering the DMA to do a transmit data transfer.

Table 17-4. SPI DMA_TRIG_RX DMA Trigger Condition

IIDX STAT	Name	Description
0x03	RTOUT	Peripheral receive timeout event. When in peripheral mode and not receiving data for the CTL1.RXTIMEOUT selected number of functional clock cycles.
0x04	RX	Receive FIFO event. This interrupt is set if the selected receive FIFO level has been reached.

Table 17-5. SPI DMA_TRIG_TX DMA Trigger Condition

IIDX STAT	Name	Description
0x05	TX	Transmit FIFO event. This interrupt is set if the selected transmit FIFO level has been reached.

The DMA trigger event configuration is managed with the DMA_TRIG_RX and DMA_TRIG_TX event management registers. See [Section 7.2.5](#) for guidance on configuring the Event registers and [Section 7.1.3.2](#) for on how DMA trigger event works.

17.2.7 Emulation Modes

The module behavior while the device is in debug mode is controlled by the FREE and SOFT bits in PDBGCTL register.

When the device is in debug mode and set into halt mode below behavior can be configured.

Table 17-6. Debug Mode Peripheral Behavior

PDBGCTL.FREE	PDBGCTL.SOFT	Function
1	x	Modules continues operation
0	0	Module stops immediately
0	1	Module stops after the next transfer has been finished

17.3 SPI Registers

Table 17-7 lists the memory-mapped registers for the SPI registers. All register offset addresses not listed in Table 17-7 should be considered as reserved locations and the register contents should not be modified.

Table 17-7. SPI Registers

Offset	Acronym	Register Name	Group	Section
4h	SCLK	SCLK		Go
8h	MOSI	MOSI		Go
Ch	MISO	MISO		Go
18h	CS0	SPI Chip Select 0		Go
1Ch	CS1_MISO1	SPI Chip Select 1		Go
20h	CS2_MISO2	SPI Chip Select 2		Go
24h	CS3_CD_MISO3	SPI Chip Select 3		Go
204h	SCLK	FUPDATE version of SCLK		Go
208h	MOSI	FUPDATE version of MOSI		Go
20Ch	MISO	FUPDATE version of MISO		Go
218h	CS0	FUPDATE version of CS0		Go
21Ch	CS1_MISO1	FUPDATE version of CS1		Go
220h	CS2_MISO2	FUPDATE version of CS2		Go
224h	CS3_CD_MISO3	FUPDATE version of CS3		Go
480h	CPU_CONNECT_0	CPU Connect		Go
504h	DMA_MAP_RX	DMA Map		Go
505h	DMA_TRIG_RX	DMA Trigger		Go
506h	DMA_ENTRY_RX	DMA Entry		Go
508h	DMA_MAP_TX	DMA Map		Go
509h	DMA_TRIG_TX	DMA Trigger		Go
50Ah	DMA_ENTRY_TX	DMA Entry		Go
800h	PWREN	Power enable		Go
804h	RSTCTL	Reset Control		Go
808h	CLKCFG	Peripheral Clock Configuration Register		Go
814h	STAT	Status Register		Go
1000h	CLKDIV	Clock Divider		Go
1004h	CLKSEL	Clock Select for Ultra Low Power peripherals		Go
1018h	PDBGCTL	Peripheral Debug Control		Go
1020h	IIDX	Interrupt Index Register	CPU_INT	Go
1028h	IMASK	Interrupt mask	CPU_INT	Go
1030h	RIS	Raw interrupt status	CPU_INT	Go
1038h	MIS	Masked interrupt status	CPU_INT	Go
1040h	ISSET	Interrupt set	CPU_INT	Go
1048h	ICLR	Interrupt clear	CPU_INT	Go
1050h	IIDX	Interrupt Index Register	DMA_TRIG_RX	Go
1058h	IMASK	Interrupt mask	DMA_TRIG_RX	Go
1060h	RIS	Raw interrupt status	DMA_TRIG_RX	Go
1068h	MIS	Masked interrupt status	DMA_TRIG_RX	Go

Table 17-7. SPI Registers (continued)

Offset	Acronym	Register Name	Group	Section
1070h	ISSET	Interrupt set	DMA_TRIG_RX	Go
1078h	ICLR	Interrupt clear	DMA_TRIG_RX	Go
1080h	IIDX	Interrupt Index Register	DMA_TRIG_TX	Go
1088h	IMASK	Interrupt mask	DMA_TRIG_TX	Go
1090h	RIS	Raw interrupt status	DMA_TRIG_TX	Go
1098h	MIS	Masked interrupt status	DMA_TRIG_TX	Go
10A0h	ISSET	Interrupt set	DMA_TRIG_TX	Go
10A8h	ICLR	Interrupt clear	DMA_TRIG_TX	Go
10E0h	EVT_MODE	Event Mode		Go
10E4h	INTCTL	Interrupt control register		Go
1100h	CTL0	SPI control register 0		Go
1104h	CTL1	SPI control register 1		Go
1108h	CLKCTL	Clock prescaler and divider register.		Go
110Ch	IFLS	Interrupt FIFO Level Select Register		Go
1110h	STAT	Status Register		Go
1130h	RXDATA	RXDATA Register		Go
1140h	TXDATA	TXDATA Register		Go
1E00h	TEST0	Test 0 Register		Go

Complex bit access types are encoded to fit into small table cells. [Table 17-8](#) shows the codes that are used for access types in this section.

Table 17-8. SPI Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
R-0	R-0	Read Returns 0s
Write Type		
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value

17.3.1 SCLK (Offset = 4h) [Reset = 00000000h]

SCLK is shown in [Figure 17-9](#) and described in [Table 17-9](#).

Return to the [Summary Table](#).

SCLK Signal Controller : Clock Output Peripheral: Clock Input

Figure 17-9. SCLK

31	30	29	28	27	26	25	24
RESERVED	GFLT	SLEW	WCOMP	WUEN	INV	HIGHZ1	HIGHZ0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
RESERVED		DRV		HYSTEN	INENA	PIPU	PIPD
R/W-0h		R/W-0h		R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
GSTATE							
R/W-0h							
7	6	5	4	3	2	1	0
PSTATE							
R/W-0h							

Table 17-9. SCLK Field Descriptions

Bit	Field	Type	Reset	Description
31	RESERVED	R/W	0h	
30	GFLT	R/W	0h	Glitch Filter Enable 0h = No internal glitch filter 1h = Use internal glitch filter
29	SLEW	R/W	0h	Reserved Slew Rate Control 0h = No Slew Rate Control 1h = Use Slew Rate Control
28	WCOMP	R/W	0h	Wake up compare value 0h = Match 0 will wake 1h = Match 1 will wake
27	WUEN	R/W	0h	Wake up enable 0h = Wake up not enabled 1h = Wake up enabled
26	INV	R/W	0h	Invert digital input/output relative to peripheral/GPIO 0h = Input and output are non-inverted 1h = Input and output are inverted
25	HIGHZ1	R/W	0h	High-Z instead of high output 0h = Pin can be driven high 1h = Pin is tri-stated instead of driven high
24	HIGHZ0	R/W	0h	High-Z instead of low output 0h = Pin can be driven low 1h = Pin is tri-stated instead of driven low
23	RESERVED	R/W	0h	
22-20	DRV	R/W	0h	Drive strength options 0h = Lowest drive strength 1h = Drive strength 2/8 2h = Drive strength 3/8 3h = Drive strength 4/8 4h = Drive strength 5/8 5h = Drive strength 6/8 6h = Drive strength 7/8 7h = Highest drive strength

Table 17-9. SCLK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	HYSTEN	R/W	0h	Hysteresis enable 0h = No hysteresis 1h = Hysteresis on
18	INENA	R/W	0h	Input enable 0h = Inputs 0 to connected core 1h = Inputs IO pad value to connected core
17	PIPU	R/W	0h	Pull up enable 0h = No pull up 1h = Pull up
16	PIPD	R/W	0h	Pull down enable 0h = No pull down 1h = Pull down
15-14	GSTATE	R/W	0h	GPIO Channel State 0h = G-Channel is in Unassigned State 1h = G-Channel is in Handover State 2h = G-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = G-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
13-8	RESERVED	R/W	0h	
7-6	PSTATE	R/W	0h	Peripheral-Analog Channel State 0h = P-Channel is in Unassigned State 1h = P-Channel is in Handover State 2h = P-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = P-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
5-0	RESERVED	R/W	0h	

17.3.2 MOSI (Offset = 8h) [Reset = 00000000h]

MOSI is shown in [Figure 17-10](#) and described in [Table 17-10](#).

Return to the [Summary Table](#).

MOSI Signal Controller : Data Output Peripheral: Data Input

Figure 17-10. MOSI

31	30	29	28	27	26	25	24
RESERVED	GFLT	SLEW	WCOMP	WUEN	INV	HIGHZ1	HIGHZ0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
RESERVED	DRV			HYSTEN	INENA	PIPU	PIPD
R/W-0h	R/W-0h			R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
GSTATE		RESERVED					
R/W-0h		R/W-0h					
7	6	5	4	3	2	1	0
PSTATE		RESERVED					
R/W-0h		R/W-0h					

Table 17-10. MOSI Field Descriptions

Bit	Field	Type	Reset	Description
31	RESERVED	R/W	0h	
30	GFLT	R/W	0h	Glitch Filter Enable 0h = No internal glitch filter 1h = Use internal glitch filter
29	SLEW	R/W	0h	Reserved Slew Rate Control 0h = No Slew Rate Control 1h = Use Slew Rate Control
28	WCOMP	R/W	0h	Wake up compare value 0h = Match 0 will wake 1h = Match 1 will wake
27	WUEN	R/W	0h	Wake up enable 0h = Wake up not enabled 1h = Wake up enabled
26	INV	R/W	0h	Invert digital input/output relative to peripheral/GPIO 0h = Input and output are non-inverted 1h = Input and output are inverted
25	HIGHZ1	R/W	0h	High-Z instead of high output 0h = Pin can be driven high 1h = Pin is tri-stated instead of driven high
24	HIGHZ0	R/W	0h	High-Z instead of low output 0h = Pin can be driven low 1h = Pin is tri-stated instead of driven low
23	RESERVED	R/W	0h	
22-20	DRV	R/W	0h	Drive strength options 0h = Lowest drive strength 1h = Drive strength 2/8 2h = Drive strength 3/8 3h = Drive strength 4/8 4h = Drive strength 5/8 5h = Drive strength 6/8 6h = Drive strength 7/8 7h = Highest drive strength

Table 17-10. MOSI Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	HYSTEN	R/W	0h	Hysteresis enable 0h = No hysteresis 1h = Hysteresis on
18	INENA	R/W	0h	Input enable 0h = Inputs 0 to connected core 1h = Inputs IO pad value to connected core
17	PIPU	R/W	0h	Pull up enable 0h = No pull up 1h = Pull up
16	PIPD	R/W	0h	Pull down enable 0h = No pull down 1h = Pull down
15-14	GSTATE	R/W	0h	GPIO Channel State 0h = G-Channel is in Unassigned State 1h = G-Channel is in Handover State 2h = G-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = G-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
13-8	RESERVED	R/W	0h	
7-6	PSTATE	R/W	0h	Peripheral-Analog Channel State 0h = P-Channel is in Unassigned State 1h = P-Channel is in Handover State 2h = P-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = P-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
5-0	RESERVED	R/W	0h	

17.3.3 MISO (Offset = Ch) [Reset = 00000000h]

MISO is shown in [Figure 17-11](#) and described in [Table 17-11](#).

Return to the [Summary Table](#).

MISO Signal Controller : Data Input Peripheral: Data Output

Figure 17-11. MISO

31	30	29	28	27	26	25	24
RESERVED	GFLT	SLEW	WCOMP	WUEN	INV	HIGHZ1	HIGHZ0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
RESERVED	DRV			HYSTEN	INENA	PIPU	PIPD
R/W-0h	R/W-0h			R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
GSTATE		RESERVED					
R/W-0h		R/W-0h					
7	6	5	4	3	2	1	0
PSTATE		RESERVED					
R/W-0h		R/W-0h					

Table 17-11. MISO Field Descriptions

Bit	Field	Type	Reset	Description
31	RESERVED	R/W	0h	
30	GFLT	R/W	0h	Glitch Filter Enable 0h = No internal glitch filter 1h = Use internal glitch filter
29	SLEW	R/W	0h	Reserved Slew Rate Control 0h = No Slew Rate Control 1h = Use Slew Rate Control
28	WCOMP	R/W	0h	Wake up compare value 0h = Match 0 will wake 1h = Match 1 will wake
27	WUEN	R/W	0h	Wake up enable 0h = Wake up not enabled 1h = Wake up enabled
26	INV	R/W	0h	Invert digital input/output relative to peripheral/GPIO 0h = Input and output are non-inverted 1h = Input and output are inverted
25	HIGHZ1	R/W	0h	High-Z instead of high output 0h = Pin can be driven high 1h = Pin is tri-stated instead of driven high
24	HIGHZ0	R/W	0h	High-Z instead of low output 0h = Pin can be driven low 1h = Pin is tri-stated instead of driven low
23	RESERVED	R/W	0h	
22-20	DRV	R/W	0h	Drive strength options 0h = Lowest drive strength 1h = Drive strength 2/8 2h = Drive strength 3/8 3h = Drive strength 4/8 4h = Drive strength 5/8 5h = Drive strength 6/8 6h = Drive strength 7/8 7h = Highest drive strength

Table 17-11. MISO Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	HYSTEN	R/W	0h	Hysteresis enable 0h = No hysteresis 1h = Hysteresis on
18	INENA	R/W	0h	Input enable 0h = Inputs 0 to connected core 1h = Inputs IO pad value to connected core
17	PIPU	R/W	0h	Pull up enable 0h = No pull up 1h = Pull up
16	PIPD	R/W	0h	Pull down enable 0h = No pull down 1h = Pull down
15-14	GSTATE	R/W	0h	GPIO Channel State 0h = G-Channel is in Unassigned State 1h = G-Channel is in Handover State 2h = G-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = G-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
13-8	RESERVED	R/W	0h	
7-6	PSTATE	R/W	0h	Peripheral-Analog Channel State 0h = P-Channel is in Unassigned State 1h = P-Channel is in Handover State 2h = P-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = P-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
5-0	RESERVED	R/W	0h	

17.3.4 CS0 (Offset = 18h) [Reset = 00000000h]

CS0 is shown in [Figure 17-12](#) and described in [Table 17-12](#).

Return to the [Summary Table](#).

SPI Chip Select 0: Controller : Output Peripheral: Input

Figure 17-12. CS0

31	30	29	28	27	26	25	24
RESERVED	GFLT	SLEW	WCOMP	WUEN	INV	HIGHZ1	HIGHZ0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
RESERVED	DRV			HYSTEN	INENA	PIPU	PIPD
R/W-0h	R/W-0h			R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
GSTATE		RESERVED					
R/W-0h		R/W-0h					
7	6	5	4	3	2	1	0
PSTATE		RESERVED					
R/W-0h		R/W-0h					

Table 17-12. CS0 Field Descriptions

Bit	Field	Type	Reset	Description
31	RESERVED	R/W	0h	
30	GFLT	R/W	0h	Glitch Filter Enable 0h = No internal glitch filter 1h = Use internal glitch filter
29	SLEW	R/W	0h	Reserved Slew Rate Control 0h = No Slew Rate Control 1h = Use Slew Rate Control
28	WCOMP	R/W	0h	Wake up compare value 0h = Match 0 will wake 1h = Match 1 will wake
27	WUEN	R/W	0h	Wake up enable 0h = Wake up not enabled 1h = Wake up enabled
26	INV	R/W	0h	Invert digital input/output relative to peripheral/GPIO 0h = Input and output are non-inverted 1h = Input and output are inverted
25	HIGHZ1	R/W	0h	High-Z instead of high output 0h = Pin can be driven high 1h = Pin is tri-stated instead of driven high
24	HIGHZ0	R/W	0h	High-Z instead of low output 0h = Pin can be driven low 1h = Pin is tri-stated instead of driven low
23	RESERVED	R/W	0h	
22-20	DRV	R/W	0h	Drive strength options 0h = Lowest drive strength 1h = Drive strength 2/8 2h = Drive strength 3/8 3h = Drive strength 4/8 4h = Drive strength 5/8 5h = Drive strength 6/8 6h = Drive strength 7/8 7h = Highest drive strength

Table 17-12. CS0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	HYSTEN	R/W	0h	Hysteresis enable 0h = No hysteresis 1h = Hysteresis on
18	INENA	R/W	0h	Input enable 0h = Inputs 0 to connected core 1h = Inputs IO pad value to connected core
17	PIPU	R/W	0h	Pull up enable 0h = No pull up 1h = Pull up
16	PIPD	R/W	0h	Pull down enable 0h = No pull down 1h = Pull down
15-14	GSTATE	R/W	0h	GPIO Channel State 0h = G-Channel is in Unassigned State 1h = G-Channel is in Handover State 2h = G-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = G-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
13-8	RESERVED	R/W	0h	
7-6	PSTATE	R/W	0h	Peripheral-Analog Channel State 0h = P-Channel is in Unassigned State 1h = P-Channel is in Handover State 2h = P-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = P-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
5-0	RESERVED	R/W	0h	

17.3.5 CS1_MISO1 (Offset = 1Ch) [Reset = 00000000h]

CS1_MISO1 is shown in [Figure 17-13](#) and described in [Table 17-13](#).

Return to the [Summary Table](#).

SPI Chip Select 1 / MISO1 Controller : Output / Input Peripheral: - / Output

Figure 17-13. CS1_MISO1

31	30	29	28	27	26	25	24
RESERVED	GFLT	SLEW	WCOMP	WUEN	INV	HIGHZ1	HIGHZ0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
RESERVED	DRV			HYSTEN	INENA	PIPU	PIPD
R/W-0h	R/W-0h			R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
GSTATE		RESERVED					
R/W-0h		R/W-0h					
7	6	5	4	3	2	1	0
PSTATE		RESERVED					
R/W-0h		R/W-0h					

Table 17-13. CS1_MISO1 Field Descriptions

Bit	Field	Type	Reset	Description
31	RESERVED	R/W	0h	
30	GFLT	R/W	0h	Glitch Filter Enable 0h = No internal glitch filter 1h = Use internal glitch filter
29	SLEW	R/W	0h	Reserved Slew Rate Control 0h = No Slew Rate Control 1h = Use Slew Rate Control
28	WCOMP	R/W	0h	Wake up compare value 0h = Match 0 will wake 1h = Match 1 will wake
27	WUEN	R/W	0h	Wake up enable 0h = Wake up not enabled 1h = Wake up enabled
26	INV	R/W	0h	Invert digital input/output relative to peripheral/GPIO 0h = Input and output are non-inverted 1h = Input and output are inverted
25	HIGHZ1	R/W	0h	High-Z instead of high output 0h = Pin can be driven high 1h = Pin is tri-stated instead of driven high
24	HIGHZ0	R/W	0h	High-Z instead of low output 0h = Pin can be driven low 1h = Pin is tri-stated instead of driven low
23	RESERVED	R/W	0h	
22-20	DRV	R/W	0h	Drive strength options 0h = Lowest drive strength 1h = Drive strength 2/8 2h = Drive strength 3/8 3h = Drive strength 4/8 4h = Drive strength 5/8 5h = Drive strength 6/8 6h = Drive strength 7/8 7h = Highest drive strength

Table 17-13. CS1_MISO1 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	HYSTEN	R/W	0h	Hysteresis enable 0h = No hysteresis 1h = Hysteresis on
18	INENA	R/W	0h	Input enable 0h = Inputs 0 to connected core 1h = Inputs IO pad value to connected core
17	PIPU	R/W	0h	Pull up enable 0h = No pull up 1h = Pull up
16	PIPD	R/W	0h	Pull down enable 0h = No pull down 1h = Pull down
15-14	GSTATE	R/W	0h	GPIO Channel State 0h = G-Channel is in Unassigned State 1h = G-Channel is in Handover State 2h = G-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = G-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
13-8	RESERVED	R/W	0h	
7-6	PSTATE	R/W	0h	Peripheral-Analog Channel State 0h = P-Channel is in Unassigned State 1h = P-Channel is in Handover State 2h = P-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = P-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
5-0	RESERVED	R/W	0h	

17.3.6 CS2_MISO2 (Offset = 20h) [Reset = 00000000h]

CS2_MISO2 is shown in [Figure 17-14](#) and described in [Table 17-14](#).

Return to the [Summary Table](#).

SPI Chip Select 2 / MISO2 Controller : Output / Input Peripheral: - / Output

Figure 17-14. CS2_MISO2

31	30	29	28	27	26	25	24
RESERVED	GFLT	SLEW	WCOMP	WUEN	INV	HIGHZ1	HIGHZ0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
RESERVED	DRV			HYSTEN	INENA	PIPU	PIPD
R/W-0h	R/W-0h			R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
GSTATE		RESERVED					
R/W-0h		R/W-0h					
7	6	5	4	3	2	1	0
PSTATE		RESERVED					
R/W-0h		R/W-0h					

Table 17-14. CS2_MISO2 Field Descriptions

Bit	Field	Type	Reset	Description
31	RESERVED	R/W	0h	
30	GFLT	R/W	0h	Glitch Filter Enable 0h = No internal glitch filter 1h = Use internal glitch filter
29	SLEW	R/W	0h	Reserved Slew Rate Control 0h = No Slew Rate Control 1h = Use Slew Rate Control
28	WCOMP	R/W	0h	Wake up compare value 0h = Match 0 will wake 1h = Match 1 will wake
27	WUEN	R/W	0h	Wake up enable 0h = Wake up not enabled 1h = Wake up enabled
26	INV	R/W	0h	Invert digital input/output relative to peripheral/GPIO 0h = Input and output are non-inverted 1h = Input and output are inverted
25	HIGHZ1	R/W	0h	High-Z instead of high output 0h = Pin can be driven high 1h = Pin is tri-stated instead of driven high
24	HIGHZ0	R/W	0h	High-Z instead of low output 0h = Pin can be driven low 1h = Pin is tri-stated instead of driven low
23	RESERVED	R/W	0h	
22-20	DRV	R/W	0h	Drive strength options 0h = Lowest drive strength 1h = Drive strength 2/8 2h = Drive strength 3/8 3h = Drive strength 4/8 4h = Drive strength 5/8 5h = Drive strength 6/8 6h = Drive strength 7/8 7h = Highest drive strength

Table 17-14. CS2_MISO2 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	HYSTEN	R/W	0h	Hysteresis enable 0h = No hysteresis 1h = Hysteresis on
18	INENA	R/W	0h	Input enable 0h = Inputs 0 to connected core 1h = Inputs IO pad value to connected core
17	PIPU	R/W	0h	Pull up enable 0h = No pull up 1h = Pull up
16	PIPD	R/W	0h	Pull down enable 0h = No pull down 1h = Pull down
15-14	GSTATE	R/W	0h	GPIO Channel State 0h = G-Channel is in Unassigned State 1h = G-Channel is in Handover State 2h = G-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = G-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
13-8	RESERVED	R/W	0h	
7-6	PSTATE	R/W	0h	Peripheral-Analog Channel State 0h = P-Channel is in Unassigned State 1h = P-Channel is in Handover State 2h = P-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = P-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
5-0	RESERVED	R/W	0h	

17.3.7 CS3_CD_MISO3 (Offset = 24h) [Reset = 00000000h]

CS3_CD_MISO3 is shown in [Figure 17-15](#) and described in [Table 17-15](#).

Return to the [Summary Table](#).

SPI Chip Select 3 / Command Data / MISO3 Controller : Output / Output / Input Peripheral: - / - / Output

Figure 17-15. CS3_CD_MISO3

31	30	29	28	27	26	25	24
RESERVED	GFLT	SLEW	WCOMP	WUEN	INV	HIGHZ1	HIGHZ0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
RESERVED	DRV			HYSTEN	INENA	PIPU	PIPD
R/W-0h	R/W-0h			R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
GSTATE		RESERVED					
R/W-0h		R/W-0h					
7	6	5	4	3	2	1	0
PSTATE		RESERVED					
R/W-0h		R/W-0h					

Table 17-15. CS3_CD_MISO3 Field Descriptions

Bit	Field	Type	Reset	Description
31	RESERVED	R/W	0h	
30	GFLT	R/W	0h	Glitch Filter Enable 0h = No internal glitch filter 1h = Use internal glitch filter
29	SLEW	R/W	0h	Reserved Slew Rate Control 0h = No Slew Rate Control 1h = Use Slew Rate Control
28	WCOMP	R/W	0h	Wake up compare value 0h = Match 0 will wake 1h = Match 1 will wake
27	WUEN	R/W	0h	Wake up enable 0h = Wake up not enabled 1h = Wake up enabled
26	INV	R/W	0h	Invert digital input/output relative to peripheral/GPIO 0h = Input and output are non-inverted 1h = Input and output are inverted
25	HIGHZ1	R/W	0h	High-Z instead of high output 0h = Pin can be driven high 1h = Pin is tri-stated instead of driven high
24	HIGHZ0	R/W	0h	High-Z instead of low output 0h = Pin can be driven low 1h = Pin is tri-stated instead of driven low
23	RESERVED	R/W	0h	
22-20	DRV	R/W	0h	Drive strength options 0h = Lowest drive strength 1h = Drive strength 2/8 2h = Drive strength 3/8 3h = Drive strength 4/8 4h = Drive strength 5/8 5h = Drive strength 6/8 6h = Drive strength 7/8 7h = Highest drive strength

Table 17-15. CS3_CD_MISO3 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	HYSTEN	R/W	0h	Hysteresis enable 0h = No hysteresis 1h = Hysteresis on
18	INENA	R/W	0h	Input enable 0h = Inputs 0 to connected core 1h = Inputs IO pad value to connected core
17	PIPU	R/W	0h	Pull up enable 0h = No pull up 1h = Pull up
16	PIPD	R/W	0h	Pull down enable 0h = No pull down 1h = Pull down
15-14	GSTATE	R/W	0h	GPIO Channel State 0h = G-Channel is in Unassigned State 1h = G-Channel is in Handover State 2h = G-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = G-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
13-8	RESERVED	R/W	0h	
7-6	PSTATE	R/W	0h	Peripheral-Analog Channel State 0h = P-Channel is in Unassigned State 1h = P-Channel is in Handover State 2h = P-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = P-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
5-0	RESERVED	R/W	0h	

17.3.8 SCLK (Offset = 204h) [Reset = 00000000h]

SCLK is shown in [Figure 17-16](#) and described in [Table 17-16](#).

Return to the [Summary Table](#).

FUPDATE version of SCLK

Figure 17-16. SCLK

31	30	29	28	27	26	25	24
RESERVED				IOADDR			
W-0h				W-0h			
23	22	21	20	19	18	17	16
IOADDR							
W-0h							
15	14	13	12	11	10	9	8
IOADDR							
W-0h							
7	6	5	4	3	2	1	0
IOADDR						LOCK	GSEL
W-0h						W-0h	W-0h

Table 17-16. SCLK Field Descriptions

Bit	Field	Type	Reset	Description
31-28	RESERVED	W	0h	
27-2	IOADDR	W	0h	IO Address. This is the address that corresponds to the SOC address[27:2] of the module IP instance specific IO signal in the "Full Write" subregion of the pinmux subregion.
1	LOCK	W	0h	Sets lock bit 0h = Writing this value has no effect 1h = Set channel lock bit
0	GSEL	W	0h	GPIO channel Select 0: Select the P-Channel for the F update 1: Select the G-Channel for the F update 0h = Select the P-Channel for the F update 1h = Select the G-Channel for the F update

17.3.9 MOSI (Offset = 208h) [Reset = 00000000h]

MOSI is shown in [Figure 17-17](#) and described in [Table 17-17](#).

Return to the [Summary Table](#).

FUPDATE version of MOSI

Figure 17-17. MOSI

31	30	29	28	27	26	25	24
RESERVED				IOADDR			
W-0h				W-0h			
23	22	21	20	19	18	17	16
IOADDR							
W-0h							
15	14	13	12	11	10	9	8
IOADDR							
W-0h							
7	6	5	4	3	2	1	0
IOADDR						LOCK	GSEL
W-0h						W-0h	W-0h

Table 17-17. MOSI Field Descriptions

Bit	Field	Type	Reset	Description
31-28	RESERVED	W	0h	
27-2	IOADDR	W	0h	IO Address. This is the address that corresponds to the SOC address[27:2] of the module IP instance specific IO signal in the "Full Write" subregion of the pinmux subregion.
1	LOCK	W	0h	Sets lock bit 0h = Writing this value has no effect 1h = Set channel lock bit
0	GSEL	W	0h	GPIO channel Select 0: Select the P-Channel for the F update 1: Select the G-Channel for the F update 0h = Select the P-Channel for the F update 1h = Select the G-Channel for the F update

17.3.10 MISO (Offset = 20Ch) [Reset = 00000000h]

MISO is shown in [Figure 17-18](#) and described in [Table 17-18](#).

Return to the [Summary Table](#).

FUPDATE version of MISO

Figure 17-18. MISO

31	30	29	28	27	26	25	24
RESERVED				IOADDR			
W-0h				W-0h			
23	22	21	20	19	18	17	16
IOADDR							
W-0h							
15	14	13	12	11	10	9	8
IOADDR							
W-0h							
7	6	5	4	3	2	1	0
IOADDR						LOCK	GSEL
W-0h						W-0h	W-0h

Table 17-18. MISO Field Descriptions

Bit	Field	Type	Reset	Description
31-28	RESERVED	W	0h	
27-2	IOADDR	W	0h	IO Address. This is the address that corresponds to the SOC address[27:2] of the module IP instance specific IO signal in the "Full Write" subregion of the pinmux subregion.
1	LOCK	W	0h	Sets lock bit 0h = Writing this value has no effect 1h = Set channel lock bit
0	GSEL	W	0h	GPIO channel Select 0: Select the P-Channel for the F update 1: Select the G-Channel for the F update 0h = Select the P-Channel for the F update 1h = Select the G-Channel for the F update

17.3.11 CS0 (Offset = 218h) [Reset = 00000000h]

CS0 is shown in [Figure 17-19](#) and described in [Table 17-19](#).

Return to the [Summary Table](#).

FUPDATE version of CS0

Figure 17-19. CS0

31	30	29	28	27	26	25	24
RESERVED				IOADDR			
W-0h				W-0h			
23	22	21	20	19	18	17	16
IOADDR							
W-0h							
15	14	13	12	11	10	9	8
IOADDR							
W-0h							
7	6	5	4	3	2	1	0
IOADDR						LOCK	GSEL
W-0h						W-0h	W-0h

Table 17-19. CS0 Field Descriptions

Bit	Field	Type	Reset	Description
31-28	RESERVED	W	0h	
27-2	IOADDR	W	0h	IO Address. This is the address that corresponds to the SOC address[27:2] of the module IP instance specific IO signal in the "Full Write" subregion of the pinmux subregion.
1	LOCK	W	0h	Sets lock bit 0h = Writing this value has no effect 1h = Set channel lock bit
0	GSEL	W	0h	GPIO channel Select 0: Select the P-Channel for the F update 1: Select the G-Channel for the F update 0h = Select the P-Channel for the F update 1h = Select the G-Channel for the F update

17.3.12 CS1_MISO1 (Offset = 21Ch) [Reset = 00000000h]

CS1_MISO1 is shown in [Figure 17-20](#) and described in [Table 17-20](#).

Return to the [Summary Table](#).

FUPDATE version of CS1_MISO1

Figure 17-20. CS1_MISO1

31	30	29	28	27	26	25	24
RESERVED				IOADDR			
W-0h				W-0h			
23	22	21	20	19	18	17	16
IOADDR							
W-0h							
15	14	13	12	11	10	9	8
IOADDR							
W-0h							
7	6	5	4	3	2	1	0
IOADDR						LOCK	GSEL
W-0h						W-0h	W-0h

Table 17-20. CS1_MISO1 Field Descriptions

Bit	Field	Type	Reset	Description
31-28	RESERVED	W	0h	
27-2	IOADDR	W	0h	IO Address. This is the address that corresponds to the SOC address[27:2] of the module IP instance specific IO signal in the "Full Write" subregion of the pinmux subregion.
1	LOCK	W	0h	Sets lock bit 0h = Writing this value has no effect 1h = Set channel lock bit
0	GSEL	W	0h	GPIO channel Select 0: Select the P-Channel for the F update 1: Select the G-Channel for the F update 0h = Select the P-Channel for the F update 1h = Select the G-Channel for the F update

17.3.13 CS2_MISO2 (Offset = 220h) [Reset = 00000000h]

CS2_MISO2 is shown in [Figure 17-21](#) and described in [Table 17-21](#).

Return to the [Summary Table](#).

FUPDATE version of CS2_MISO2

Figure 17-21. CS2_MISO2

31	30	29	28	27	26	25	24
RESERVED				IOADDR			
W-0h				W-0h			
23	22	21	20	19	18	17	16
IOADDR							
W-0h							
15	14	13	12	11	10	9	8
IOADDR							
W-0h							
7	6	5	4	3	2	1	0
IOADDR						LOCK	GSEL
W-0h						W-0h	W-0h

Table 17-21. CS2_MISO2 Field Descriptions

Bit	Field	Type	Reset	Description
31-28	RESERVED	W	0h	
27-2	IOADDR	W	0h	IO Address. This is the address that corresponds to the SOC address[27:2] of the module IP instance specific IO signal in the "Full Write" subregion of the pinmux subregion.
1	LOCK	W	0h	Sets lock bit 0h = Writing this value has no effect 1h = Set channel lock bit
0	GSEL	W	0h	GPIO channel Select 0: Select the P-Channel for the F update 1: Select the G-Channel for the F update 0h = Select the P-Channel for the F update 1h = Select the G-Channel for the F update

17.3.14 CS3_CD_MISO3 (Offset = 224h) [Reset = 00000000h]

CS3_CD_MISO3 is shown in [Figure 17-22](#) and described in [Table 17-22](#).

Return to the [Summary Table](#).

FUPDATE version of CS3_CD_MISO3

Figure 17-22. CS3_CD_MISO3

31	30	29	28	27	26	25	24
RESERVED				IOADDR			
W-0h				W-0h			
23	22	21	20	19	18	17	16
IOADDR							
W-0h							
15	14	13	12	11	10	9	8
IOADDR							
W-0h							
7	6	5	4	3	2	1	0
IOADDR						LOCK	GSEL
W-0h						W-0h	W-0h

Table 17-22. CS3_CD_MISO3 Field Descriptions

Bit	Field	Type	Reset	Description
31-28	RESERVED	W	0h	
27-2	IOADDR	W	0h	IO Address. This is the address that corresponds to the SOC address[27:2] of the module IP instance specific IO signal in the "Full Write" subregion of the pinmux subregion.
1	LOCK	W	0h	Sets lock bit 0h = Writing this value has no effect 1h = Set channel lock bit
0	GSEL	W	0h	GPIO channel Select 0: Select the P-Channel for the F update 1: Select the G-Channel for the F update 0h = Select the P-Channel for the F update 1h = Select the G-Channel for the F update

17.3.15 CPU_CONNECT_0 (Offset = 480h) [Reset = 00000000h]

CPU_CONNECT_0 is shown in [Figure 17-23](#) and described in [Table 17-23](#).

Return to the [Summary Table](#).

Directly connect peripheral publisher port to application processor

Figure 17-23. CPU_CONNECT_0

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED						CPUSS0_CON N	RESERVED
R/W-0h						R/W-0h	R/W-0h

Table 17-23. CPU_CONNECT_0 Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R/W	0h	
1	CPUSS0_CONN	R/W	0h	CPUSS0 connect bit. 0h = The CPU is not connected. 1h = The CPU is connected.
0	RESERVED	R/W	0h	

17.3.16 DMA_MAP_RX (Offset = 504h) [Reset = 00h]

DMA_MAP_RX is shown in [Figure 17-24](#) and described in [Table 17-24](#).

Return to the [Summary Table](#).

Trigger port ID in the DMA for this peripheral trigger

Figure 17-24. DMA_MAP_RX

7	6	5	4	3	2	1	0
RESERVED	TRIG_ID						
R-0h	R-0h						

Table 17-24. DMA_MAP_RX Field Descriptions

Bit	Field	Type	Reset	Description
7	RESERVED	R	0h	
6-0	TRIG_ID	R	0h	The trigger port ID in the DMA for this peripheral trigger 0h = No trigger selected 1h = Trigger 1 selected 7Fh = Trigger 127 selected

17.3.17 DMA_TRIG_RX (Offset = 505h) [Reset = 00h]

DMA_TRIG_RX is shown in [Figure 17-25](#) and described in [Table 17-25](#).

Return to the [Summary Table](#).

Trigger control and status register for this peripheral trigger

Figure 17-25. DMA_TRIG_RX

7	6	5	4	3	2	1	0
RESERVED	THRHL D			TRIGLOST	STATECLR	STATE	
R/W-0h	R-0h			R-0h	R-0/W-0h	R-0h	

Table 17-25. DMA_TRIG_RX Field Descriptions

Bit	Field	Type	Reset	Description
7	RESERVED	R/W	0h	
6-4	THRHL D	R	0h	The threshold for the DMA to accept the trigger request. 0h = Lowest threshold possible 6h = Highest threshold possible
3	TRIGLOST	R	0h	Sticky flag that is set whenever a trigger request is received while the trigger port is in TRIGGER_PEND or TRIGGERED. Cleared by writing to STATECLR. 0h = Trigger not lost 1h = Trigger was lost
2	STATECLR	R-0/W	0h	Clear trigger state. Writing 1 to this register clears any pending DMA trigger on this port and transitions the port to Untriggered state. 0h = Writing 0 has no effect 1h = Clear DMA trigger
1-0	STATE	R	0h	Returns the current state of the DMA tx trigger port 0h = Channel was not triggered 1h = Channel trigger is pending 2h = Channel was triggered

17.3.18 DMA_ENTRY_RX (Offset = 506h) [Reset = 0FFFh]

DMA_ENTRY_RX is shown in [Figure 17-26](#) and described in [Table 17-26](#).

Return to the [Summary Table](#).

Descriptor connect to peripheral DMA trigger

Figure 17-26. DMA_ENTRY_RX

15	14	13	12	11	10	9	8
RESERVED				ENTRY_ID			
R/W-				R/W-FFFh			
7	6	5	4	3	2	1	0
ENTRY_ID							
R/W-FFFh							

Table 17-26. DMA_ENTRY_RX Field Descriptions

Bit	Field	Type	Reset	Description
15-12	RESERVED	R/W	0h	
11-0	ENTRY_ID	R/W	FFFh	The ID of the DMA descriptor that this trigger is routed to. This allows to ensure that another DMA channel could not listen or influence the DMA channel responsible for handling the data of this peripheral. 0h = DCLB index i=0-15. This can only be used with dedicated DCLBs. Fh = DCLB index i=0-15. This can only be used with dedicated DCLBs. 10h = DMA entry in RACE memory at index i=16-4094. This can only be used if limitless DMA is enabled in the system. FFEh = DMA entry in RACE memory at index i=16-4094. This can only be used if limitless DMA is enabled in the system. FFFh = Trigger not enabled

17.3.19 DMA_MAP_TX (Offset = 508h) [Reset = 00h]

DMA_MAP_TX is shown in [Figure 17-27](#) and described in [Table 17-27](#).

Return to the [Summary Table](#).

Trigger port ID in the DMA for this peripheral trigger

Figure 17-27. DMA_MAP_TX

7	6	5	4	3	2	1	0
RESERVED	TRIG_ID						
R-0h	R-0h						

Table 17-27. DMA_MAP_TX Field Descriptions

Bit	Field	Type	Reset	Description
7	RESERVED	R	0h	
6-0	TRIG_ID	R	0h	The trigger port ID in the DMA for this peripheral trigger 0h = No trigger selected 1h = Trigger 1 selected 7Fh = Trigger 127 selected

17.3.20 DMA_TRIG_TX (Offset = 509h) [Reset = 00h]

DMA_TRIG_TX is shown in [Figure 17-28](#) and described in [Table 17-28](#).

Return to the [Summary Table](#).

Trigger control and status register for this peripheral trigger

Figure 17-28. DMA_TRIG_TX

7	6	5	4	3	2	1	0
RESERVED	THRHL D			TRIGLOST	STATECLR	STATE	
R/W-0h	R-0h			R-0h	R-0/W-0h	R-0h	

Table 17-28. DMA_TRIG_TX Field Descriptions

Bit	Field	Type	Reset	Description
7	RESERVED	R/W	0h	
6-4	THRHL D	R	0h	The threshold for the DMA to accept the trigger request. 0h = Lowest threshold possible 6h = Highest threshold possible
3	TRIGLOST	R	0h	Sticky flag that is set whenever a trigger request is received while the trigger port is in TRIGGER_PEND or TRIGGERED. Cleared by writing to STATECLR. 0h = Trigger not lost 1h = Trigger was lost
2	STATECLR	R-0/W	0h	Clear trigger state. Writing 1 to this register clears any pending DMA trigger on this port and transitions the port to Untriggered state. 0h = Writing 0 has no effect 1h = Clear DMA trigger
1-0	STATE	R	0h	Returns the current state of the DMA tx trigger port 0h = Channel was not triggered 1h = Channel trigger is pending 2h = Channel was triggered

17.3.21 DMA_ENTRY_TX (Offset = 50Ah) [Reset = 0FFFh]

DMA_ENTRY_TX is shown in [Figure 17-29](#) and described in [Table 17-29](#).

Return to the [Summary Table](#).

Descriptor connect to peripheral DMA trigger

Figure 17-29. DMA_ENTRY_TX

15	14	13	12	11	10	9	8
RESERVED				ENTRY_ID			
R/W-				R/W-FFFh			
7	6	5	4	3	2	1	0
ENTRY_ID							
R/W-FFFh							

Table 17-29. DMA_ENTRY_TX Field Descriptions

Bit	Field	Type	Reset	Description
15-12	RESERVED	R/W	0h	
11-0	ENTRY_ID	R/W	FFFh	The ID of the DMA descriptor that this trigger is routed to. This allows to ensure that another DMA channel could not listen or influence the DMA channel responsible for handling the data of this peripheral. 0h = DCLB index i=0-15. This can only be used with dedicated DCLBs. Fh = DCLB index i=0-15. This can only be used with dedicated DCLBs. 10h = DMA entry in RACE memory at index i=16-4094. This can only be used if limitless DMA is enabled in the system. FFEh = DMA entry in RACE memory at index i=16-4094. This can only be used if limitless DMA is enabled in the system. FFFh = Trigger not enabled

17.3.22 PWREN (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 17-30](#) and described in [Table 17-30](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 17-30. PWREN

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-0h							R/WK-0h

Table 17-30. PWREN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	ENABLE	R/WK	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

17.3.23 RSTCTL (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 17-31](#) and described in [Table 17-31](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 17-31. RSTCTL

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCL R	RESETASSERT
W-0h						WK-0h	WK-0h

Table 17-31. RSTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	W	0h	
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

17.3.24 CLKCFG (Offset = 808h) [Reset = 00000000h]

CLKCFG is shown in [Figure 17-32](#) and described in [Table 17-32](#).

Return to the [Summary Table](#).

Peripheral Clock Configuration Register

Figure 17-32. CLKCFG

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							BLOCKASYNC
R/W-0h							R/W-0h
7	6	5	4	3	2	1	0
RESERVED							
R/W-0h							

Table 17-32. CLKCFG Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to Allow State Change -- 0xA9 A9h = key value to allow change field of GPRCM
23-9	RESERVED	R/W	0h	
8	BLOCKASYNC	R/W	0h	Async Clock Request is blocked from starting SYSOSC or forcing bus clock to 32MHz 0h = Not block async clock request 1h = Block async clock request
7-0	RESERVED	R/W	0h	

17.3.25 STAT (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 17-33](#) and described in [Table 17-33](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 17-33. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED							
R-							

Table 17-33. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0h	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

17.3.26 CLKDIV (Offset = 1000h) [Reset = 00000000h]

CLKDIV is shown in [Figure 17-34](#) and described in [Table 17-34](#).

Return to the [Summary Table](#).

This register is used to specify module-specific divide ratio of the functional clock

Figure 17-34. CLKDIV

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												RATIO			
R/W-0h												R/W-0h			

Table 17-34. CLKDIV Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R/W	0h	
2-0	RATIO	R/W	0h	Selects divide ratio of module clock 0h = Do not divide clock source 1h = Divide clock source by 2 2h = Divide clock source by 3 3h = Divide clock source by 4 4h = Divide clock source by 5 5h = Divide clock source by 6 6h = Divide clock source by 7 7h = Divide clock source by 8

17.3.27 CLKSEL (Offset = 1004h) [Reset = 00000000h]

CLKSEL is shown in [Figure 17-35](#) and described in [Table 17-35](#).

Return to the [Summary Table](#).

Clock source selection for peripherals

Figure 17-35. CLKSEL

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED				SYSCLK_SEL	MFCLK_SEL	LFCLK_SEL	RESERVED
R/W-0h				R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 17-35. CLKSEL Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3	SYSCLK_SEL	R/W	0h	Selects SYSCLK as clock source if enabled 0h = Does not select this clock as a source 1h = Select this clock as a source
2	MFCLK_SEL	R/W	0h	Selects MFCLK as clock source if enabled 0h = Does not select this clock as a source 1h = Select this clock as a source
1	LFCLK_SEL	R/W	0h	Selects LFCLK as clock source if enabled 0h = Does not select this clock as a source 1h = Select this clock as a source
0	RESERVED	R/W	0h	

17.3.28 PDBGCTL (Offset = 1018h) [Reset = 00000003h]

PDBGCTL is shown in [Figure 17-36](#) and described in [Table 17-36](#).

Return to the [Summary Table](#).

This register can be used by the software developer to control the behavior of the peripheral relative to the 'Core Halted' input

Figure 17-36. PDBGCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED						SOFT	FREE
R/W-						R/W-1h	R/W-1h

Table 17-36. PDBGCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R/W	0h	
1	SOFT	R/W	1h	Soft halt boundary control. This function is only available, if FREE is set to 'STOP' 0h = The peripheral will halt immediately, even if the resultant state will result in corruption if the system is restarted 1h = The peripheral blocks the debug freeze until it has reached a boundary where it can resume without corruption
0	FREE	R/W	1h	Free run control 0h = The peripheral freezes functionality while the Core Halted input is asserted and resumes when it is deasserted. 1h = The peripheral ignores the state of the Core Halted input

17.3.29 IIDX (Offset = 1020h) [Reset = 00000000h]

IIDX is shown in [Figure 17-37](#) and described in [Table 17-37](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 17-37. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 17-37. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 00h = No interrupt pending 1h = RX FIFO Overflow Event/interrupt pending 2h = Transmit Parity Event/interrupt pending 3h = SPI receive time-out interrupt 4h = Receive Event/interrupt pending 5h = Transmit Event/interrupt pending 6h = Transmit Buffer Empty Event/interrupt pending 7h = End of Transmit Event/interrupt pending 8h = DMA Done for Receive Event/interrupt pending 9h = DMA Done for Transmit Event/interrupt pending Ah = TX FIFO underflow interrupt Bh = RX FIFO Full Interrupt

17.3.30 IMASK (Offset = 1028h) [Reset = 00000000h]

IMASK is shown in [Figure 17-38](#) and described in [Table 17-38](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 17-38. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED					RXFULL	TXFIFO_UNF	DMA_DONE_TX
R/W-0h					R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
DMA_DONE_RX	IDLE	TXEMPTY	TX	RX	RTOUT	PER	RXFIFO_OVF
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 17-38. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-11	RESERVED	R/W	0h	
10	RXFULL	R/W	0h	RX FIFO Full Interrupt Mask 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
9	TXFIFO_UNF	R/W	0h	TX FIFO underflow interrupt mask 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
8	DMA_DONE_TX	R/W	0h	DMA Done 1 event for TX event mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
7	DMA_DONE_RX	R/W	0h	DMA Done 1 event for RX event mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
6	IDLE	R/W	0h	SPI Idle event mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
5	TXEMPTY	R/W	0h	Transmit FIFO Empty event mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
4	TX	R/W	0h	Transmit FIFO event mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
3	RX	R/W	0h	Receive FIFO event. This interrupt is set if the selected Receive FIFO level has been reached 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	RTOUT	R/W	0h	Enable SPI Receive Time-Out event mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

Table 17-38. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1	PER	R/W	0h	Parity error event mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	RXFIFO_OVF	R/W	0h	RXFIFO overflow event mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

17.3.31 RIS (Offset = 1030h) [Reset = 00000000h]

RIS is shown in [Figure 17-39](#) and described in [Table 17-39](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 17-39. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED					RXFULL	TXFIFO_UNF	DMA_DONE_TX
R-0h					R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
DMA_DONE_RX	IDLE	TXEMPTY	TX	RX	RTOUT	PER	RXFIFO_OVF
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 17-39. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-11	RESERVED	R	0h	
10	RXFULL	R	0h	RX FIFO Full Interrupt 0h = Interrupt did not occur 1h = Interrupt occurred
9	TXFIFO_UNF	R	0h	TX FIFO Underflow Interrupt 0h = Interrupt did not occur 1h = Interrupt occurred
8	DMA_DONE_TX	R	0h	DMA Done 1 event for TX. This interrupt is set if the TX DMA channel sends the DONE signal. This allows the handling of the DMA event inside the mapped peripheral. 0h = Interrupt did not occur 1h = Interrupt occurred
7	DMA_DONE_RX	R	0h	DMA Done 1 event for RX. This interrupt is set if the RX DMA channel sends the DONE signal. This allows the handling of the DMA event inside the mapped peripheral. 0h = Interrupt did not occur 1h = Interrupt occurred
6	IDLE	R	0h	SPI has done finished transfers and changed into IDLE mode. This bit is set when BUSY goes low. 0h = Interrupt did not occur 1h = Interrupt occurred
5	TXEMPTY	R	0h	Transmit FIFO Empty interrupt mask. This interrupt is set if all data in the Transmit FIFO have been move to the shift register. 0h = Interrupt did not occur 1h = Interrupt occurred
4	TX	R	0h	Transmit FIFO event. This interrupt is set if the selected Transmit FIFO level has been reached. 0h = Interrupt did not occur 1h = Interrupt occurred

Table 17-39. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
3	RX	R	0h	Receive FIFO event. This interrupt is set if the selected Receive FIFO level has been reached 0h = Interrupt did not occur 1h = Interrupt occurred
2	RTOUT	R	0h	SPI Receive Time-Out event. 0h = Interrupt did not occur 1h = Interrupt occurred
1	PER	R	0h	Parity error event: this bit is set if a Parity error has been detected 0h = Interrupt did not occur 1h = Interrupt occurred
0	RXFIFO_OVF	R	0h	RXFIFO overflow event. This interrupt is set if an RX FIFO overflow has been detected. 0h = Interrupt did not occur 1h = Interrupt occurred

17.3.32 MIS (Offset = 1038h) [Reset = 00000000h]

MIS is shown in [Figure 17-40](#) and described in [Table 17-40](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 17-40. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED					RXFULL	TXFIFO_UNF	DMA_DONE_TX
R-0h				R-0h		R-0h	R-0h
7	6	5	4	3	2	1	0
DMA_DONE_RX	IDLE	TXEMPTY	TX	RX	RTOUT	PER	RXFIFO_OVF
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 17-40. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-11	RESERVED	R	0h	
10	RXFULL	R	0h	RX FIFO Full Interrupt 0h = Interrupt did not occur 1h = Interrupt occurred
9	TXFIFO_UNF	R	0h	TX FIFO underflow interrupt 0h = Interrupt did not occur 1h = Interrupt occurred
8	DMA_DONE_TX	R	0h	Masked DMA Done 1 event for TX. 0h = Interrupt did not occur 1h = Interrupt occurred
7	DMA_DONE_RX	R	0h	Masked DMA Done 1 event for RX. 0h = Interrupt did not occur 1h = Interrupt occurred
6	IDLE	R	0h	Masked SPI IDLE mode event. 0h = Interrupt did not occur 1h = Interrupt occurred
5	TXEMPTY	R	0h	Masked Transmit FIFO Empty event. 0h = Interrupt did not occur 1h = Interrupt occurred
4	TX	R	0h	Masked Transmit FIFO event. This interrupt is set if the selected Transmit FIFO level has been reached. 0h = Interrupt did not occur 1h = Interrupt occurred
3	RX	R	0h	Masked receive FIFO event. This interrupt is set if the selected Receive FIFO level has been reached 0h = Interrupt did not occur 1h = Interrupt occurred
2	RTOUT	R	0h	Masked SPI Receive Time-Out Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

Table 17-40. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1	PER	R	0h	Masked Parity error event: this bit is set if a Parity error has been detected 0h = Interrupt did not occur 1h = Interrupt occurred
0	RXFIFO_OVF	R	0h	Masked RXFIFO overflow event. This interrupt is set if an RX FIFO overflow has been detected. 0h = Interrupt did not occur 1h = Interrupt occurred

17.3.33 ISET (Offset = 1040h) [Reset = 00000000h]

ISET is shown in [Figure 17-41](#) and described in [Table 17-41](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 17-41. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED					RXFULL	TXFIFO_UNF	DMA_DONE_TX
W-0h					W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
DMA_DONE_RX	IDLE	TXEMPTY	TX	RX	RTOUT	PER	RXFIFO_OVF
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 17-41. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-11	RESERVED	W	0h	
10	RXFULL	W	0h	Set RX FIFO Full Event 0h = Writing has no effect 1h = Set Interrupt
9	TXFIFO_UNF	W	0h	Set TX FIFO Underflow Event 0h = Writing has no effect 1h = Set interrupt
8	DMA_DONE_TX	W	0h	Set DMA Done 1 event for TX. 0h = Writing 0 has no effect 1h = Set Interrupt
7	DMA_DONE_RX	W	0h	Set DMA Done 1 event for RX. 0h = Writing 0 has no effect 1h = Set Interrupt
6	IDLE	W	0h	Set SPI IDLE mode event. 0h = Writing 0 has no effect 1h = Set Interrupt
5	TXEMPTY	W	0h	Set Transmit FIFO Empty event. 0h = Writing 0 has no effect 1h = Set Interrupt
4	TX	W	0h	Set Transmit FIFO event. 0h = Writing 0 has no effect 1h = Set Interrupt
3	RX	W	0h	Set Receive FIFO event. 0h = Writing 0 has no effect 1h = Set Interrupt
2	RTOUT	W	0h	Set SPI Receive Time-Out Event. 0h = Writing 0 has no effect 1h = Set Interrupt Mask

Table 17-41. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1	PER	W	0h	Set Parity error event. 0h = Writing 0 has no effect 1h = Set Interrupt
0	RXFIFO_OVF	W	0h	Set RXFIFO overflow event. 0h = Writing 0 has no effect 1h = Set Interrupt

17.3.34 ICLR (Offset = 1048h) [Reset = 00000000h]

ICLR is shown in [Figure 17-42](#) and described in [Table 17-42](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 17-42. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED					RXFULL	TXFIFO_UNF	DMA_DONE_TX
W-0h					W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
DMA_DONE_RX	IDLE	TXEMPTY	TX	RX	RTOUT	PER	RXFIFO_OVF
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 17-42. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-11	RESERVED	W	0h	
10	RXFULL	W	0h	Clear RX FIFO underflow event 0h = Writing has no effect 1h = Clear interrupt
9	TXFIFO_UNF	W	0h	Clear TXFIFO underflow event 0h = Writing has no effect 1h = Clear interrupt
8	DMA_DONE_TX	W	0h	Clear DMA Done 1 event for TX. 0h = Writing 0 has no effect 1h = Clear Interrupt
7	DMA_DONE_RX	W	0h	Clear DMA Done 1 event for RX. 0h = Writing 0 has no effect 1h = Clear Interrupt
6	IDLE	W	0h	Clear SPI IDLE mode event. 0h = Writing 0 has no effect 1h = Clear Interrupt
5	TXEMPTY	W	0h	Clear Transmit FIFO Empty event. 0h = Writing 0 has no effect 1h = Clear Interrupt
4	TX	W	0h	Clear Transmit FIFO event. 0h = Writing 0 has no effect 1h = Clear Interrupt
3	RX	W	0h	Clear Receive FIFO event. 0h = Writing 0 has no effect 1h = Clear Interrupt
2	RTOUT	W	0h	Clear SPI Receive Time-Out Event. 0h = Writing 0 has no effect 1h = Set Interrupt Mask

Table 17-42. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1	PER	W	0h	Clear Parity error event. 0h = Writing 0 has no effect 1h = Clear Interrupt
0	RXFIFO_OVF	W	0h	Clear RXFIFO overflow event. 0h = Writing 0 has no effect 1h = Clear Interrupt

17.3.35 IIDX (Offset = 1050h) [Reset = 00000000h]

IIDX is shown in [Figure 17-43](#) and described in [Table 17-43](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 17-43. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 17-43. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 00h = No interrupt pending 3h = SPI receive time-out interrupt 4h = Receive Event/interrupt pending

17.3.36 IMASK (Offset = 1058h) [Reset = 00000000h]

IMASK is shown in [Figure 17-44](#) and described in [Table 17-44](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 17-44. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED				RX	RTOUT	RESERVED	
R/W-0h				R/W-0h	R/W-0h	R/W-0h	

Table 17-44. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3	RX	R/W	0h	Receive FIFO event mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	RTOUT	R/W	0h	SPI Receive Time-Out event mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1-0	RESERVED	R/W	0h	

17.3.37 RIS (Offset = 1060h) [Reset = 00000000h]

RIS is shown in [Figure 17-45](#) and described in [Table 17-45](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 17-45. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							
R-							
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED				RX	RTOUT	RESERVED	
R-				R-0h	R-0h	R-	

Table 17-45. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	RX	R	0h	Receive FIFO event. This interrupt is set if the selected Receive FIFO level has been reached 0h = Interrupt did not occur 1h = Interrupt occurred
2	RTOUT	R	0h	SPI Receive Time-Out Event. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1-0	RESERVED	R	0h	

17.3.38 MIS (Offset = 1068h) [Reset = 00000000h]

MIS is shown in [Figure 17-46](#) and described in [Table 17-46](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 17-46. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				RX	RTOUT	RESERVED	
R-0h				R-0h	R-0h	R-0h	

Table 17-46. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	RX	R	0h	Receive FIFO event mask. 0h = Interrupt did not occur 1h = Interrupt occurred
2	RTOUT	R	0h	SPI Receive Time-Out event mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1-0	RESERVED	R	0h	

17.3.39 ISET (Offset = 1070h) [Reset = 00000000h]

ISET is shown in [Figure 17-47](#) and described in [Table 17-47](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 17-47. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED				RX	RTOUT	RESERVED	
W-0h				W-0h	W-0h	W-0h	

Table 17-47. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	W	0h	
3	RX	W	0h	Set Receive FIFO event. 0h = Writing 0 has no effect 1h = Set Interrupt
2	RTOUT	W	0h	Set SPI Receive Time-Out event. 0h = Writing 0 has no effect 1h = Set Interrupt Mask
1-0	RESERVED	W	0h	

17.3.40 ICLR (Offset = 1078h) [Reset = 00000000h]

ICLR is shown in [Figure 17-48](#) and described in [Table 17-48](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 17-48. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED				RX	RTOUT	RESERVED	
W-0h				W-0h	W-0h	W-0h	

Table 17-48. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	W	0h	
3	RX	W	0h	Clear Receive FIFO event. 0h = Writing 0 has no effect 1h = Clear Interrupt
2	RTOUT	W	0h	Clear SPI Receive Time-Out event. 0h = Writing 0 has no effect 1h = Set Interrupt Mask
1-0	RESERVED	W	0h	

17.3.41 IIDX (Offset = 1080h) [Reset = 00000000h]

IIDX is shown in [Figure 17-49](#) and described in [Table 17-49](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 17-49. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 17-49. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 00h = No interrupt pending 5h = Transmit Event/interrupt pending

17.3.42 IMASK (Offset = 1088h) [Reset = 00000000h]

IMASK is shown in [Figure 17-50](#) and described in [Table 17-50](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 17-50. IMASK

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED											TX	RESERVED			
R/W-0h											R/W-0h		R/W-0h		

Table 17-50. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-5	RESERVED	R/W	0h	
4	TX	R/W	0h	Transmit FIFO event mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
3-0	RESERVED	R/W	0h	

17.3.43 RIS (Offset = 1090h) [Reset = 00000000h]

RIS is shown in [Figure 17-51](#) and described in [Table 17-51](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 17-51. RIS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED											TX	RESERVED			
R-											R-0h		R-		

Table 17-51. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-5	RESERVED	R	0h	
4	TX	R	0h	Transmit FIFO event: A read returns the current mask for transmit FIFO interrupt. On a write of 1, the mask for transmit FIFO interrupt is set which means the interrupt state will be reflected in MIS.TXMIS. A write of 0 clears the mask which means MIS.TXMIS will not reflect the interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
3-0	RESERVED	R	0h	

17.3.44 MIS (Offset = 1098h) [Reset = 00000000h]

MIS is shown in [Figure 17-52](#) and described in [Table 17-52](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 17-52. MIS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED											TX	RESERVED			
R-0h											R-0h	R-0h			

Table 17-52. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-5	RESERVED	R	0h	
4	TX	R	0h	Masked Transmit FIFO event 0h = Interrupt did not occur 1h = Interrupt occurred
3-0	RESERVED	R	0h	

17.3.45 ISET (Offset = 10A0h) [Reset = 00000000h]

ISET is shown in [Figure 17-53](#) and described in [Table 17-53](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 17-53. ISET

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED											TX	RESERVED			
W-0h											W-0h	W-0h			

Table 17-53. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-5	RESERVED	W	0h	
4	TX	W	0h	Set Transmit FIFO event. 0h = Writing 0 has no effect 1h = Set Interrupt
3-0	RESERVED	W	0h	

17.3.46 ICLR (Offset = 10A8h) [Reset = 00000000h]

ICLR is shown in [Figure 17-54](#) and described in [Table 17-54](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 17-54. ICLR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED											TX	RESERVED			
W-0h											W-0h	W-0h			

Table 17-54. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-5	RESERVED	W	0h	
4	TX	W	0h	Clear Transmit FIFO event. 0h = Writing 0 has no effect 1h = Clear Interrupt
3-0	RESERVED	W	0h	

17.3.47 EVT_MODE (Offset = 10E0h) [Reset = 00000029h]

EVT_MODE is shown in [Figure 17-55](#) and described in [Table 17-55](#).

Return to the [Summary Table](#).

Event mode register. It is used to select whether each line is disabled, in software mode (software clears the RIS) or in hardware mode (hardware clears the RIS)

Figure 17-55. EVT_MODE

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED		INT2_CFG		INT1_CFG		INT0_CFG	
R/W-		R-2h		R-2h		R-1h	

Table 17-55. EVT_MODE Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R/W	0h	
5-4	INT2_CFG	R	2h	Event line mode select for event corresponding to none.INT_EVENT2 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
3-2	INT1_CFG	R	2h	Event line mode select for event corresponding to none.INT_EVENT1 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
1-0	INT0_CFG	R	1h	Event line mode select for event corresponding to none.INT_EVENT0 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.

17.3.48 INTCTL (Offset = 10E4h) [Reset = 00000000h]

INTCTL is shown in [Figure 17-56](#) and described in [Table 17-56](#).

Return to the [Summary Table](#).

Interrupt control register

Figure 17-56. INTCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED							INTEVAL
R/W-							W-0h

Table 17-56. INTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R/W	0h	
0	INTEVAL	W	0h	Writing a 1 to this field re-evaluates the interrupt sources. 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS.

17.3.49 CTL0 (Offset = 1100h) [Reset = 00000000h]

CTL0 is shown in [Figure 17-57](#) and described in [Table 17-57](#).

Return to the [Summary Table](#).

SPI control register 0

Figure 17-57. CTL0

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED	CSCLR	CSSEL		RESERVED		SPH	SPO
R/W-0h	R/W-0h	R/W-0h		R/W-0h		R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
PACKEN	FRF		DSS				
R/W-0h	R/W-0h		R/W-0h				

Table 17-57. CTL0 Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	R/W	0h	
14	CSCLR	R/W	0h	Clear shift register counter on CS inactive This bit is relevant only in the peripheral, CTL1.CP=0. 0h = Disable automatic clear of shift register when CS goes to disable. 1h = Enable automatic clear of shift register when CS goes to disable.
13-12	CSSEL	R/W	0h	Select the CS line to control on data transfer This bit is applicable for both controller/target mode 0h (R/W) = CS line select: 0 1h (R/W) = CS line select: 1 2h (R/W) = CS line select: 2 3h (R/W) = CS line select: 3
11-10	RESERVED	R/W	0h	
9	SPH	R/W	0h	CLKOUT phase (Motorola SPI frame format only) This bit selects the clock edge that captures data and enables it to change state. It has the most impact on the first bit transmitted by either permitting or not permitting a clock transition before the first data capture edge. 0h = Data is captured on the first clock edge transition. 1h = Data is captured on the second clock edge transition.
8	SPO	R/W	0h	CLKOUT polarity (Motorola SPI frame format only) 0h = SPI produces a steady state LOW value on the CLKOUT 1h = SPI produces a steady state HIGH value on the CLKOUT
7	PACKEN	R/W	0h	Packing Enable. When 1, packing feature is enabled inside the IP When 0, packing feature is disabled inside the IP 0h = Packing feature disabled 1h = Packing feature enabled

Table 17-57. CTL0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
6-5	FRF	R/W	0h	Frame format Select 0h = Motorola SPI frame format (3 wire mode) 1h = Motorola SPI frame format (4 wire mode) 2h = TI synchronous serial frame format 3h = National Microwire frame format
4-0	DSS	R/W	0h	Data Size Select. Values 0 - 2 are reserved and shall not be used. 3h = 4_BIT : 4-bit data SPI allows only values up to 16 Bit 3h (R/W) = Data Size Select bits: 4 4h (R/W) = Data Size Select bits: 5 5h (R/W) = Data Size Select bits: 6 6h (R/W) = Data Size Select bits: 7 7h (R/W) = Data Size Select bits: 8 8h (R/W) = Data Size Select bits: 9 9h (R/W) = Data Size Select bits: 10 Ah (R/W) = Data Size Select bits: 11 Bh (R/W) = Data Size Select bits: 12 Ch (R/W) = Data Size Select bits: 13 Dh (R/W) = Data Size Select bits: 14 Eh (R/W) = Data Size Select bits: 15 Fh (R/W) = Data Size Select bits: 16

17.3.50 CTL1 (Offset = 1104h) [Reset = 00000004h]

CTL1 is shown in [Figure 17-58](#) and described in [Table 17-58](#).

Return to the [Summary Table](#).

SPI control register 1

Figure 17-58. CTL1

31	30	29	28	27	26	25	24
RESERVED				RXTIMEOUT			
R/W-0h				R/W-0h			
23	22	21	20	19	18	17	16
REPEATTX							
R/W-0h							
15	14	13	12	11	10	9	8
CDMODE				CDENABLE	RESERVED		PTEN
R/W-0h				R/W-0h	R/W-0h		R/W-0h
7	6	5	4	3	2	1	0
RESERVED	PES	PREN	MSB	POD	CP	LBM	ENABLE
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-1h	R/W-0h	R/W-0h

Table 17-58. CTL1 Field Descriptions

Bit	Field	Type	Reset	Description
31-30	RESERVED	R/W	0h	
29-24	RXTIMEOUT	R/W	0h	Receive Timeout (only for Peripheral mode) Defines the number of Clock Cycles before after which the Receive Timeout flag RTOUT is set. The time is calculated using the control register for the clock selection and divider in the Controller mode configuration. A value of 0 disables this function. 0h = Smallest value 3Fh = Highest possible value
23-16	REPEATTX	R/W	0h	Counter to repeat last transfer 0: repeat last transfer is disabled. x: repeat the last transfer with the given number. The transfer will be started with writing a data into the TX Buffer. Sending the data will be repeated with the given value, so the data will be transferred X+1 times in total. The behavior is identical as if the data would be written into the TX Buffer that many times as defined by the value here. It can be used to clean a transfer or to pull a certain amount of data by a peripheral. 0h = Smallest value FFh = Highest possible value
15-12	CDMODE	R/W	0h	Command/Data Mode Value When CTL1.CDENABLE is 1, CS3 line is used as C/D signal to distinguish between Command (C/D low) and Data (C/D high) information. When a value is written into the CTL1.CDMODE bits, the C/D (CS3) line will go low for the given numbers of byte which are sent by the SPI, starting with the next value to be transmitted after which, C/D line will go high automatically 0: Manual mode with C/D signal as High 1-14: C/D is low while this number of bytes are being sent after which, this field sets to 0 and C/D goes high. Reading this field at any time returns the remaining number of command bytes. 15: Manual mode with C/D signal as Low. 0h = Manual mode: Data 0h = Smallest value Fh = Manual mode: Command

Table 17-58. CTL1 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	CDENABLE	R/W	0h	Command/Data Mode enable 0h = CS3 is used for Chip Select 1h = CS3 is used as CD signal
10-9	RESERVED	R/W	0h	
8	PTEN	R/W	0h	Parity transmit enable If enabled, parity transmission will be done for both controller and peripheral modes. 0h = Parity transmission is disabled 1h = Parity transmission is enabled
7	RESERVED	R/W	0h	
6	PES	R/W	0h	Even Parity Select 0h = Odd Parity mode 1h = Even Parity mode
5	PREN	R/W	0h	Parity receive enable If enabled, parity reception check will be done for both controller and peripheral modes In case of a parity miss-match the parity error flag RIS.PER will be set. 0h = Disable Parity receive function 1h = Enable Parity receive function
4	MSB	R/W	0h	MSB first select. Controls the direction of the receive and transmit shift register. 0h = LSB first 1h = MSB first
3	POD	R/W	0h	Peripheral-mode: Data output disabled This bit is relevant only in Peripheral mode. In multiple-peripheral system topologies, SPI controller can broadcast a message to all peripherals, while only one peripheral drives the line. POD can be used by the SPI peripheral to disable driving data on the line. 0h = SPI can drive the MISO output in peripheral mode. 1h = SPI cannot drive the MISO output in peripheral mode.
2	CP	R/W	1h	Controller or peripheral mode select. This bit can be modified only when SPI is disabled, CTL1.ENABLE=0. 0h = Select Peripheral mode 1h = Select Controller Mode
1	LBM	R/W	0h	Loop back mode 0h = Disable loopback mode 1h = Enable loopback mode
0	ENABLE	R/W	0h	SPI enable 0h = Disable module function 1h = Enable module function

17.3.51 CLKCTL (Offset = 1108h) [Reset = 00000000h]

CLKCTL is shown in [Figure 17-59](#) and described in [Table 17-59](#).

Return to the [Summary Table](#).

Clock prescaler and divider register. This register contains the settings for the Clock prescaler and divider settings.

Figure 17-59. CLKCTL

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DSAMPLE				RESERVED											
R/W-0h								R/W-0h							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED						SCR									
R/W-0h								R/W-0h							

Table 17-59. CLKCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-28	DSAMPLE	R/W	0h	Delayed sampling value. In controller mode the data on the input pin will be delayed sampled by the defined clock cycles of internal functional clock hence relaxing the setup time of input data. This setting is useful in systems where the board delays and external peripheral delays are more than the input setup time of the controller. Please refer to the data sheet for values of controller input setup time and assess what DSAMPLE value meets the requirement of the system. Note: High values of DSAMPLE can cause HOLD time violations and must be factored in the calculations. 0h = Smallest value Fh = Highest possible value
27-10	RESERVED	R/W	0h	
9-0	SCR	R/W	0h	Serial clock divider: This is used to generate the transmit and receive bit rate of the SPI. The SPI bit rate is (SPI functional clock frequency)/((SCR+1)×2). SCR is a value from 0-1023. 0h = Smallest value 3FFh = Highest possible value

17.3.52 IFLS (Offset = 110Ch) [Reset = 00000012h]

IFLS is shown in [Figure 17-60](#) and described in [Table 17-60](#).

Return to the [Summary Table](#).

The IFLS register is the interrupt FIFO level select register. You can use this register to define the levels at which the TX, RX and timeout interrupt flags are triggered. The interrupts are generated based on a transition through a level rather than being based on the level. That is, the interrupts are generated when the fill level progresses through the trigger level. For example, if the receive trigger level is set to the half-way mark, the interrupt is triggered when the receive FIFO is filled with two or more characters. Out of reset, the TXIFLSEL and RXIFLSEL bits are configured so that the FIFOs trigger an interrupt at the half-way mark.

Figure 17-60. IFLS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED										RXIFLSEL			TXIFLSEL		
R/W-0h										R/W-2h			R/W-2h		

Table 17-60. IFLS Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R/W	0h	
5-3	RXIFLSEL	R/W	2h	SPI Receive Interrupt FIFO Level Select The trigger points for the receive interrupt are as follows: 0h = Reserved 1h = RX FIFO >= 1/4 full 2h = RX FIFO >= 1/2 full (default) 3h = RX FIFO >= 3/4 full 4h = Reserved 5h = RX FIFO is full 6h = Reserved 7h = Trigger when RX FIFO contains >= 1 frame
2-0	TXIFLSEL	R/W	2h	SPI Transmit Interrupt FIFO Level Select The trigger points for the transmit interrupt are as follows: 0h = Reserved 1h = TX FIFO <= 3/4 empty 2h = TX FIFO <= 1/2 empty (default) 3h = TX FIFO <= 1/4 empty 4h = Reserved 5h = TX FIFO is empty 6h = Reserved 7h = Trigger when TX FIFO has >= 1 frame free.

17.3.53 STAT (Offset = 1110h) [Reset = 0000000Fh]

STAT is shown in [Figure 17-61](#) and described in [Table 17-61](#).

Return to the [Summary Table](#).

Status Register

Figure 17-61. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							
R-							
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED			BUSY	RNF	RFE	TNF	TFE
R-			R-0h	R-1h	R-1h	R-1h	R-1h

Table 17-61. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-5	RESERVED	R	0h	
4	BUSY	R	0h	Busy 0h = SPI is in idle mode. 1h = SPI is currently transmitting and/or receiving data, or transmit FIFO is not empty.
3	RNF	R	1h	Receive FIFO not full 0h = Receive FIFO is full. 1h = Receive FIFO is not full.
2	RFE	R	1h	Receive FIFO empty. 0h = Receive FIFO is not empty. 1h = Receive FIFO is empty.
1	TNF	R	1h	Transmit FIFO not full 0h = Transmit FIFO is full. 1h = Transmit FIFO is not full.
0	TFE	R	1h	Transmit FIFO empty. 0h = Transmit FIFO is not empty. 1h = Transmit FIFO is empty.

17.3.54 RXDATA (Offset = 1130h) [Reset = 00000000h]

RXDATA is shown in [Figure 17-62](#) and described in [Table 17-62](#).

Return to the [Summary Table](#).

RXDATA Register

Reading this register returns value(s) of FIFO. If the FIFO is empty the last read value is returned.

Writing has not effect and is ignored.

When PACKEN=1,two entries of the FIFO are returned as a 32-bit value. When PACKEN=0, 1 entry of FIFO is returned as 16-bit value.

Figure 17-62. RXDATA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA																															
R-0h																															

Table 17-62. RXDATA Field Descriptions

Bit	Field	Type	Reset	Description
31-0	DATA	R	0h	Received Data When PACKEN=1,two entries of the FIFO are returned as a 32-bit value. When PACKEN=0, 1 entry of FIFO is returned as 16-bit value. As data values are removed by the receive logic from the incoming data frame, they are placed into the entry in the receive FIFO, pointed to by the current FIFO write pointer. Received data less than 16 bits is automatically right justified in the receive buffer. 0h = Smallest value FFFFFFFFh = Highest possible value

17.3.55 TXDATA (Offset = 1140h) [Reset = 00000000h]

TXDATA is shown in [Figure 17-63](#) and described in [Table 17-63](#).

Return to the [Summary Table](#).

TXDATA Register

Writing puts the data into the TX FIFO. Reading this register returns the last written value.

When PACKEN=0, only the lower 16-bits of data written into the register is transferred to one 16-bits wide TX FIFO entry

When PACKEN=1, upper and lower 16-bits of 32-bit write data are transferred to two 16-bits wide TX FIFO entry

Figure 17-63. TXDATA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA																															
R/W-0h																															

Table 17-63. TXDATA Field Descriptions

Bit	Field	Type	Reset	Description
31-0	DATA	R/W	0h	Transmit Data When read, last written value will be returned. If the last write to this field was a 32-bit write (with PACKEN=1), 32-bits will be returned and if the last write was a 16-bit write (PACKEN=0), those 16-bits will be returned. When written, one or two FIFO entries will be written depending on PACKEN value. Data values are removed from the transmit FIFO one value at a time by the transmit logic. It is loaded into the transmit serial shifter, then serially shifted out onto the TXD output pin at the programmed bit rate. When a data size of less than 16 bits is selected, the user must right-justify data written to the transmit FIFO. The transmit logic ignores the unused bits. 0h = Smallest value FFFFFFFFh = Highest possible value

17.3.56 TEST0 (Offset = 1E00h) [Reset = 00000000h]

TEST0 is shown in [Figure 17-64](#) and described in [Table 17-64](#).

Return to the [Summary Table](#).

Test 0 register.

Figure 17-64. TEST0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												DTB_MUX_SEL			
R/W-0h												R/W-0h			

Table 17-64. TEST0 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	DTB_MUX_SEL	R/W	0h	This bit field is used to select DTB mux digital output signals. 0h = Disables DTB MUX 1h = Selects test group 1 2h = Selects test group 2 3h = Selects test group 3 4h = Selects test group 4 5h = Selects test group 5 6h = Selects test group 6 7h = Selects test group 7 8h = Selects test group 8 9h = Selects test group 9

This page intentionally left blank.



The I²C module provides a standardized serial interface to transfer data between MSP devices and other external I²C devices (such as a sensors, memory, or DACs).

18.1 I²C Overview	972
18.2 I²C Operation	974

18.1 I²C Overview

The I²C module provides a standardized serial interface to transfer data between MSP devices and other external I²C devices (such as a sensors, memory, or DACs).

18.1.1 Purpose of the Peripheral

The I²C peripheral provides bidirectional data transfer through a two-wire serial bus consisting of a data (SDA) and clock (SCL) line. The I²C bus is widely used to interface with devices such as battery management ICs, sensors, other MCUs and so on. This I²C peripheral provides the ability to both transmit to and receive from other I²C devices on the bus.

18.1.2 Features

The controller includes I²C modules with the following features:

- Devices on the I²C bus can be designated as either a controller or a target with 7-bit addressing.
- Support four I²C modes
 - Controller transmit
 - Controller receive
 - Target transmit
 - Target receive
- Supported transmission speeds:
 - Standard-mode (Sm) with a bit rate up to 100 kbps
 - Fast-mode (Fm) with a bit rate up to 400 kbps
 - Fast-mode Plus (Fm+) with a bit rate up to 1 Mbps
- Independent 8-byte FIFOs for reception and transmission
- Dual target address capability
- Glitch suppression
- Independent controller and target interrupt generation
- Controller operation with arbitration, clock synchronization, multiple controller support
- Hardware support for SMBus and PMBus
 - Clock low timeout detection and interrupt
 - Quick command capability
- Hardware support for DMA with separate channels for transmit and receive

18.1.3 Functional Block Diagram

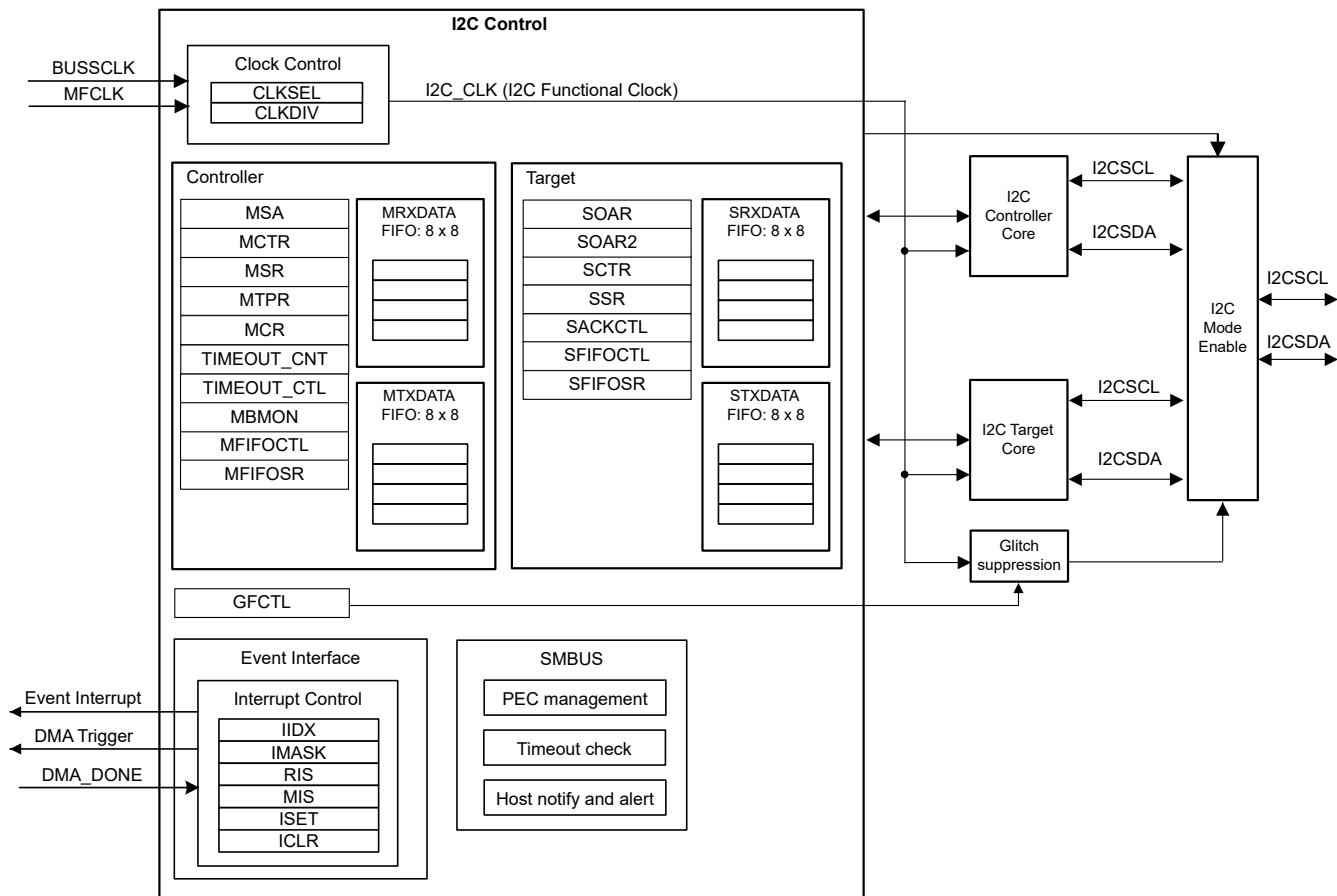


Figure 18-1. I2C Functional Block Diagram

18.1.4 Environment and External Connections

The I²C mode supports any target or controller I²C-compatible device. Figure 18-2 shows an example of an I²C bus. Each I²C peripheral instance is comprised of both controller and target functions where the target can be addressed with 2 independent user defined addresses. A device connected to the I²C bus can be considered as the controller or the target when performing data transfers. A controller initiates a data transfer and generates the clock signal SCL. Any device addressed by a controller is considered a target.

I²C data is communicated using the serial data (SDA) pin and the serial clock (SCL) pin. Both SDA and SCL are open-drain bidirectional and must be connected to a positive supply voltage using a pullup resistor.

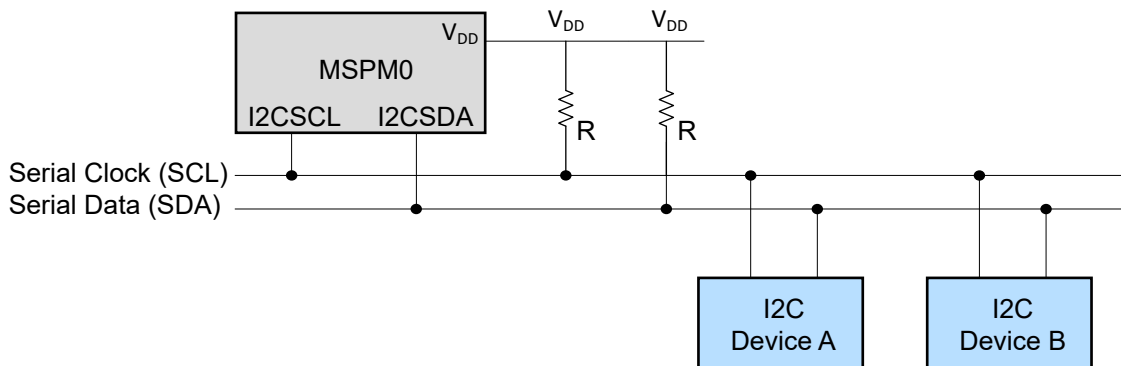


Figure 18-2. Bus Connection Diagram

18.2 I²C Operation

This section describes the operation of the I²C peripheral.

18.2.1 Clock Control

18.2.1.1 Clock Select and I²C Speed

Standard, Fast, and Fast Plus modes are selected using a value in the I²C Controller Timer Period (I2CMTPR) register that results in a maximum SCL frequency of:

- 100 kbps for Standard mode
- 400 kbps for Fast mode
- 1 Mbps for Fast mode Plus

The I²C frequency I2C_FREQ is determined by the I2C_CLK frequency and bitfields TPR, SCL_LP, and SCL_HP where:

- I2C_CLK is the functional clock frequency to the I²C module. Note that the I²C internal functional clock is first divided from the source clock:
 - Use I2Cx.CLKSEL register to select the source of the I²C functional clock
 - BUSSCLK: the current bus clock is selected as the source for I²C. The current bus clock depends on power domain. If the I²C instance is in power domain 1 (PD1) refer to MCLK, if the I²C instance is in power domain 0 (PD0) refer to ULPCLK.
 - MFCLK: MFCLK is selected as the source for I²C, refer to MFCLK.
 - Use I2Cx.CLKDIV register to select divide ratio of I²C function clock, options are from divide by 1 to 8.
- SCL_LP is the low phase of SCL (which must be fixed at 6)
- SCL_HP is the high phase of SCL (which must be fixed at 4)
- TPR is the programmed value of the TPR bits in the I2Cx.MTPR register. This value is determined by replacing the known variables in the equation below and solving for TPR

The I²C frequency is calculated as follows:

$$I2C_FREQ = I2C_CLK / ((1+TPR) * (SCL_LP + SCL_HP)) \quad (24)$$

For example, if the I²C functional clock frequency is 32 MHz and target SCL frequency is 400 kHz:

I2C_CLK = 32 MHz

I2C_FREQ = 400 kHz

SCL_LP = 6, SCL_HP = 4

$TPR = (I2C_CLK / (I2C_FREQ * (4 + 6))) - 1$

TPR = 7 (0x07)

Table 18-1. Examples of Controller Clock Setting for Typical Clock Configurations

Functional Clock	TPR Bits Standard Mode 100-kHz SCL	TPR Bits Fast Mode 400-kHz SCL	TPR Bits Fast Mode Plus 1000-kHz SCL
4 MHz	0x03	-	-
8 MHz	0x07	0x01	-
20 MHz	0x13	0x04	0x01
32 MHz	0x1F	0x07	0x02 ⁽¹⁾
40 MHz	0x27	0x09	0x03

(1) With 32-MHz functional clock, TPR = 0x01 generates 1.6-MHz SCL frequency, and TPR = 0x02 generates 1.067-MHz SCL frequency.

I²C functional clock must be greater than or equal to 20 times the SCL frequency, $I2C_CLK \geq 20 \times I2C_FREQ$.

The following minimum functional clock frequencies are required when running certain I²C clock speeds:

- I²C_CLK ≥ 2 MHz when working with I²C speed 0 to 100 kHz
- I²C_CLK ≥ 8 MHz when working with I²C speed 100 to 400 kHz
- I²C_CLK ≥ 20 MHz when working with I²C speed 400 kHz to 1 MHz

18.2.1.2 Clock Startup

The selected clock source is always available and the frequency depends on the power mode, for more information refer to the PMU/Clock section. After enabling the I²C module by setting the I2C.PWREN.ENABLE bit, the module is ready to start receiving and transmitting data.

18.2.2 Signal Descriptions

The I²C bus consists of a clock signal and a data signal. The clock signal can be generated either internally (during controller operation) or externally (during target operation).

Table 18-2. I²C signal descriptions

Device Pin	Function
I2Cx_SCL	I ² C clock signal
I2Cx_SDA	I ² C data signal

18.2.3 General Architecture

18.2.3.1 I²C Bus Functional Overview

The I²C bus uses only two signals: SDA and SCL. SDA is the bidirectional serial data line and SCL is the bidirectional serial clock line. The bus is considered idle when both lines are in high state and no transfer is ongoing.

Every transaction on the I²C bus is 9-bits long, consisting of 8 data bits and 1 acknowledge bit. A transfer is defined as the time between a valid START and STOP condition—as described in [Figure 18-3](#). The number of bytes per transfer is unrestricted; however, each data byte must be followed by an acknowledge bit and data must be transferred MSB first. When a receiver cannot receive another complete byte, it can hold the clock line SCL Low and force the transmitter into a wait state, this process is commonly known as clock stretching. The data transfer continues when the receiver releases the clock SCL.

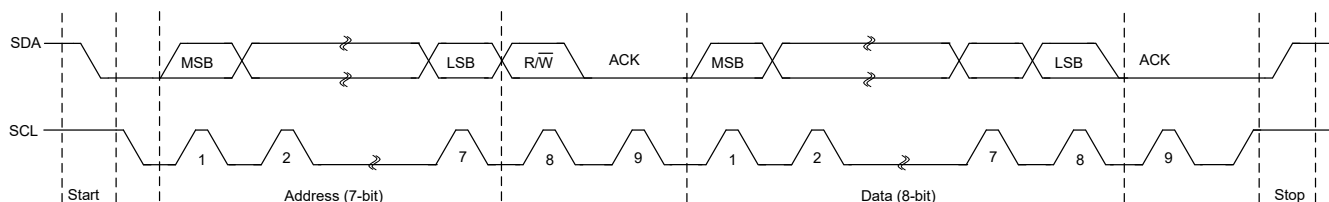


Figure 18-3. Module Data Transfer

The data on the SDA line must be stable during the high period of the clock, and the data line can only change when SCL is low (see [Figure 18-4](#)), otherwise START or STOP conditions are generated.

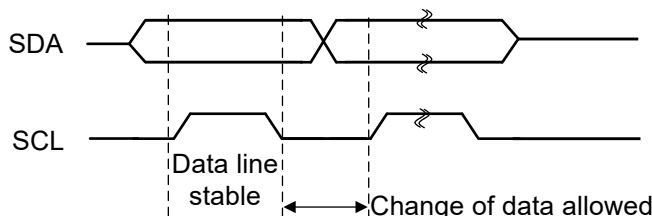


Figure 18-4. Data Change Period

18.2.3.2 START and STOP Conditions

The protocol of the I²C bus defines two states to begin and end a transaction: START and STOP. A high-to-low transition on the SDA line while the SCL is high is defined as a START condition, and a low-to-high transition on the SDA line while SCL is high is defined as a STOP condition. START and STOP conditions are always generated by the controller. The bus is considered busy after a START condition and free after a STOP condition.

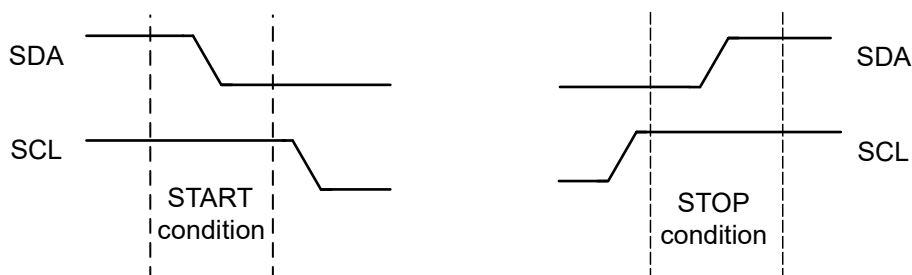


Figure 18-5. START and STOP Conditions

The STOP bit determines if the transaction stops at the end of the data cycle or continues on to a repeated START condition.

To generate a single transaction, the I²C controller target address I2Cx.MSA.SADDR register is written with the desired address, the I2Cx.MSA.DIR bit should be set to 1 to start a receive operation and 0 to start a transmit operation, and the control register (I2Cx.MCTR) is written with ACK = X (0 or 1), STOP = 1, START = 1, and RUN = 1 to perform the operation and generate a STOP at the end. When the operation is completed (or aborted due an error) the interrupt flags are set. When the I²C module operates in Controller receiver mode, the ACK bit is normally (in case of no error) set causing the I²C bus controller to transmit an acknowledge automatically after each byte. This bit must be cleared when the I²C bus controller requires no further data to be transmitted from the target transmitter. For more information on I²C controller mode configuration please refer to [Section 18.2.4.1](#)

When operating in target mode, the START and STOP bits in the CPU_INT.RIS register indicate detection of start and stop conditions on the bus and the CPU_INT.IMASK can be configured to allow START and STOP to be promoted to controller interrupts (when interrupts are enabled). For more information on I²C target mode configuration please refer to [Section 18.2.4.2](#)

Bus Status Flags:

- BUSBSY is set when a START is detected on the bus and sequentially cleared when a STOP condition is detected on the bus. It is also cleared after a clock timeout I2Cx.MSR.CLKTO is set once both SCL and SDA are pulled to high.
- BUSY is also set after a START/RESTART, and cleared after I2Cx.MCTR.MBLN bytes of data is transferred. It is also cleared after data is NACK 'd or a STOP.
- IDLE is set when the Controller I²C State machine is in the IDLE state indicating no ongoing transfer.

18.2.3.3 Data Format with 7-Bit Address

Data transfers follow the format shown in [Figure 18-6](#). After the START condition, a target address is transmitted. This address is 7-bits long followed by an eighth bit, which is a data direction bit (this bit is only as Controller mode, DIR bit in the I2Cx.MSA register). If the I2Cx.MSA.DIR bit is 0, it indicates a transmit operation (send), and if it is set to 1, it indicates a request to receive data (receive). A data transfer is always terminated by a STOP condition generated by the controller; however, a controller can initiate communications with another device on the bus by generating a repeated START condition and addressing another target without first generating a STOP condition, see section [Repeated Start](#). Various combinations of receive/transmit formats are then possible within a single transfer. The ninth bit is the Acknowledge bit, which is described in [Section 18.2.3.4](#).

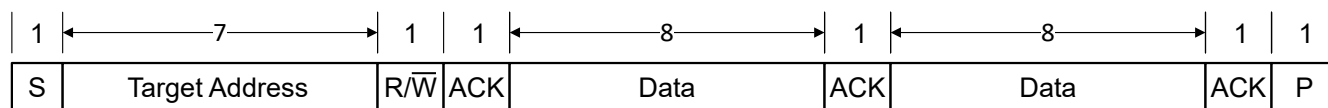


Figure 18-6. Data Format with 7-Bit Address

With the I2Cx.SCTR.GENCALL bit the I²C module can be enabled to respond on a General Call on the I²C bus. The General Call is identified by address of 0x00 and the R/W bit set to 0. The General Call interrupt can be enabled with the CPU_INT.IMASK.GENCALL bit.

18.2.3.4 Acknowledge

All bus transactions have a required acknowledge clock cycle that is generated by the controller. During the acknowledge cycle, the transmitter (which can be the controller or target) releases the SDA line. To acknowledge the transaction, the receiver must pull down SDA during the acknowledge clock cycle. The acknowledge cycle must comply with the data validity requirements.

When a target receiver does not acknowledge the target address, SDA must be left high by the target so that the controller can generate a STOP condition and abort the current transfer or generate a repeated START condition to start a new transfer. If the controller device is acting as a receiver during a transfer, it is responsible for acknowledging each transfer made by the target. Because the controller controls the number of bytes in the transfer, it signals the end of data to the target transmitter by not generating an acknowledge on the last data byte. The target transmitter must then release SDA to allow the controller to generate the STOP or a repeated START condition.

A target can generate ACK/NACK manually or automatically. When I2Cx.SACKCTL.ACKOEN=0, the target will send automatic ACK. Note that due to the FIFO, the device will receive and ACK all bytes automatically until the RX FIFO is full. Setting SACKCTL.ACKOEN =1 enables manual ACK. Manual ACK override can be used to evaluate each received byte or to slow down the communication when automatic FIFO reception is not desired. When manual ACK override operation is enabled, the I²C target module's clock is pulled low after the last data bit until this SACKCTL.ACKOVAL is written with the indicated response. The reception of new data is indicated by the SRXDONE interrupt flag.

If the controller receives a NACK while transmitting data the NACK and MTXDONE bit will be set in the RIS registers. If there is still data in the FIFO the TXEMPTY bit will not be set to inform software that a TX FIFO flush may be required.

18.2.3.5 Repeated Start

The direction of data flow on SDA can be changed by the controller, without first stopping a transfer, by issuing a repeated START condition. This is called a RESTART, see data format in [Figure 18-7](#). After a RESTART is issued, the target address is again sent out with the new data direction specified by the R/W bit.

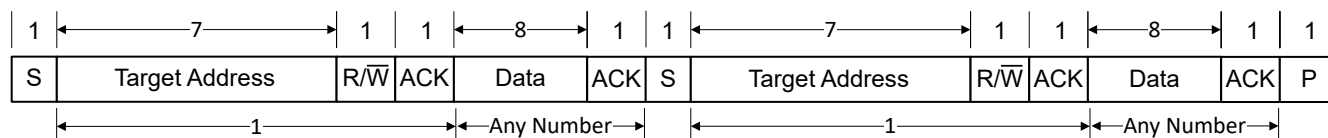


Figure 18-7. Data Format with Repeated Start

A repeated start sequence for a Controller transmit is as follows:

1. When the device is in the idle state, the Controller writes the target address to the I2Cx.MSA register and configures the DIR bit for the desired transfer type.
2. Data is written to the I2Cx.MTXDATA register.
3. When the BUSY bit in the MSR register is 0, the BURSTRUN and START bit in the I2Cx.MCTR register need to be set to initiate a transfer.
4. Wait till the BUSY bit in the I2Cx.MSR register gets 0.

5. The Controller does not generate a STOP condition but instead writes another target address to the I2Cx.MSA register and then sets the BURSTRUN and START bit again with a write operation to initiate the repeated START.

A repeated start sequence for a Controller receive is similar:

1. When the device is in idle, the Controller writes the target address to the I2Cx.MSA register and configures the DIR bit for the desired transfer type.
2. When the BUSY bit in the I2Cx.MSR register is 0, the BURSTRUN and START bit in the I2Cx.MCTR register need to be set to initiate a transfer.
3. The controller reads data from the I2Cx.MRXDATA register.
4. Wait till the BUSY bit in the I2Cx.MSR register gets 0.
5. The Controller does not generate a STOP condition but instead writes another target address to the I2Cx.MSA register and then sets the BURSTRUN and START bit again with a write operation to initiate the repeated START.

18.2.3.6 SCL Clock Low Timeout

The I²C target can extend the transaction by pulling the clock low to slow down communication. The I²C module has a 12-bit programmable counter that is used to track how long the clock has been held low. The upper 8 bits of the count value are software programmable through the I2CTIMEOUT_CTL register. The CNTL value programmed in the I2CTIMEOUT_CTL.TCNTLA register has to be greater than 0x01 to enable the timeout feature. Please note that the low timeout configuration needs to be set only during initialization and not during active state.

The application can program the eight most significant bits of the counter to reflect the acceptable cumulative low period in a transaction. Each count is equal to a timeout period of $(1 + \text{TPR}) \times 12$ of functional clocks FCLK where the TPR is the programmable timer period. The Timeout counter A counts for the entire time SCL is held Low continuously. When SCL is high, the Timeout counter A is reloaded with the value in the I2CTIMEOUT_CTL.TCNTLA register bit, and begins counting down from this value at the falling edge of SCL.

The BUSSCLK clock generated for the timeout counter keeps running irrespective of the programmed I²C speed even if SCL is held low on the bus.

The I²C clock low timeout period is calculated as follows:

- Cumulative clock low period = Timeout counter * One timeout period
- One timeout period = BUSSCLK period * $(1 + \text{TPR}) \times 12$
- Timeout counter = I2CTIMEOUT_CTL.TCNTLA register (this register contains the upper 8-bits of a 12-bit counter value, the lower 4-bits are set to 0)

As an example, if the BUSSCLK: clock is 20 MHz and the I²C module was operating at 100-kHz speed, the TPR would be equal to 19. See [Section 18.2.1.1](#) on how TPR is calculated. One timeout period is equal to $(1 / 20 \text{ MHz}) \times (1 + 19) \times 12$ or 12 μs . Programming the I2CTIMEOUT_CTL.TCNTLA register to 0xDA would translate to the value 0xDA0, because the lower 4-bits are set to 0x0. This would translate to a decimal value of 3488 clocks or a cumulative clock low period of $3488 \times 12 \mu\text{s}$, or 41.856 ms at 100 kHz.

The TIMEOUTA bit in the I²C Controller Raw Interrupt Status (RIS) register is set when the clock timeout period is reached, allowing the controller to start corrective action to resolve the remote target state. In addition, the BUSBSY bit in the I²C Controller Status MSR register is set. This bit is cleared after I²C goes to idle or during the I²C controller reset. The status of the raw SDA and SCL signals are readable by software through the SDA and SCL bits in the I²C Controller Bus Monitor MBMON register to help determine the state of the remote target.

In the event of a timeout condition, application software must choose how it intends to attempt bus recovery. If a timeout is detected before the end of a transfer (receive or transmit), software should flush the FIFO before initializing the next transfer. The clock low timeout is needed for SMBus and PMBus implementation.

Note

The Controller clock low timeout counter register I2C_TIMEOUT_CNT.TCNTA counts for the entire time SCL is held low continuously. When SCL is high, the counter is reloaded with the value in the I2C_TIMEOUT_CTL.TCNTLA register, and begins counting down from this value at the falling edge of SCL.

18.2.3.7 Clock Stretching

In controller mode, the clock stretching can be disabled if no targets on the bus support it, allowing the controller to reach the maximum speed on the bus. Otherwise the clock can be slowed by a target keeping the clock low or due to the clock status detection delay within the I²C module.

To ensure compliance to the I²C specification, clock stretching needs to be enabled. Clock stretching is activated when either the RX FIFO full or TX FIFO empty is set. Clock stretching support can be enabled or disabled by configuring the CLKSTRETCH bit in I2Cx.MCR register.

In the target mode, clock stretching is enabled by default and it is signaled by the TREQ and RREQ bits of the I²C target status register I2Cx.SSR.

- When TREQ bit is set, it indicates the I²C controller has been addressed as a target transmitter and is using clock stretching to delay the controller until data has been written to the STXDATA FIFO (Target TX FIFO is empty).
- When RREQ bit is set, it indicates the I²C controller has outstanding receive data from the I2C controller and is using clock stretching to delay the controller until the data has been read from the SRXDATA FIFO (Target RX FIFO is full).

Note

Clock stretching in target mode may be used together with an [asynchronous fast clock request](#) to support bringing the device into a suspended low power mode state upon detection of an I2C start bit, enabling the I2C module to support 100kHz (standard mode) or 400kHz (fast mode) operation out of low power modes where the bus clock speed is below the minimum oversampling speed required by the respective mode. Refer to the [clock selection and I²C speed section](#) for the minimum frequency requirements. When clock stretching is used together with an asynchronous fast clock request, it is possible for the device to wait in STOP or STANDBY mode when the I²C is idle, and when an I²C bus edge is seen, the fast clock request will requests SYSOSC at base frequency and the bus clock will switch to SYSOSC at base frequency, allowing the I²C module to process the bus transaction and wake the processor if an interrupt is generated.

18.2.3.8 Dual Address

The I²C target interface supports dual address capability for the target. An additional programmable I²C Target Own Address Register I2CX.SOAR is provided and can be matched if enabled. When dual address disabled (I2Cx.SOAR2.OAR2EN=0), the I²C target provides an ACK on the bus if the address matches the OAR field in the I2Cx.SOAR register. In dual address mode (I2Cx.SOAR2.OAR2EN=1), the I²C target provides an ACK on the bus if either the OAR field in the I2Cx.SOAR register or the OAR2 field in the I2Cx.SOAR2 register is matched.

The OAR2SEL bit in the I2Cx.SSR register indicates if the address that was ACKed is the alternate address or not. When this bit is clear, it indicates either the primary address match or no OAR2 address match.

18.2.3.9 Arbitration

A controller can start a transfer only if the bus is idle. It's possible for two or more controllers to generate a START condition within minimum hold time of the START condition. In these situations, an arbitration scheme takes place on the SDA line, while SCL is high (see [Figure 18-8](#)). The first controller transmitter that generates a logic high is overruled by the opposing controller generating a logic low and the losing controller releases the bus until the bus is idle again.

Arbitration can take place over several bits. Its first stage is a comparison of address bits, and if both controllers are trying to address the same device, arbitration continues on to the comparison of data bits.

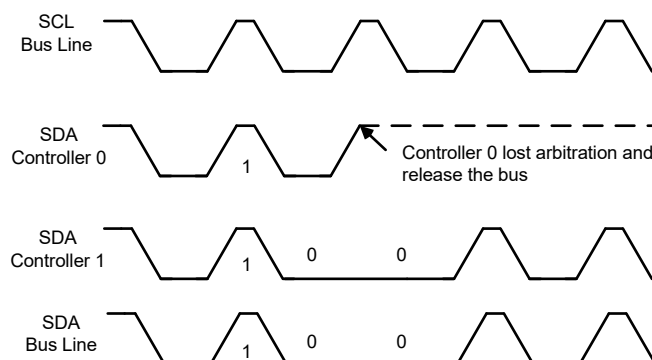


Figure 18-8. Arbitration Procedure Between Two Controller Transmitters

When an arbitration lost is detected the I2Cx.MSR.ARBLOST flag is set. It will be reset by the hardware with the next STOP condition detected on the bus. Additionally the ARBLOST flags in CPU_INT.RIS registers are set.

If arbitration is lost when the I²C controller has initiated a transfer, the application should execute the following steps to correctly handle the arbitration loss:

- Flush TX FIFO
- Clear and mask the TX Empty interrupt by the TXEMPTY bit through the IMASK and ICLR register.
- Once the bus is IDLE, the TXFIFO can be filled and enabled, the TXEMPTY bit can be unmasked and a new transaction can be initiated.

18.2.3.10 Multiple Controller Mode

When operating in a multiple controller system, the I2Cx.MCR.MMST bit must be set.

During the arbitration procedure, the clocks from the different controllers must be synchronized. A device that first generates a low period on SCL overrules the other devices, forcing them to start their own low periods. SCL is then held low by the device with the longest low period. The other devices must wait for SCL to be released before starting their high periods

In multiple-controller configuration, the clock synchronization (Figure 18-9) during the arbitration is enabled, the SCL high time counts once the SCL line has been detected high. If not enabled the high time counts as soon as the SCL line has been set high by the I²C controller which allows the I²C to reach the maximum speed by the I2Cx.MTPR register specified speed.

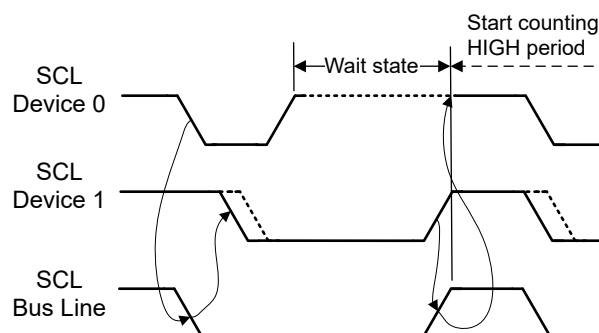


Figure 18-9. Clock Synchronization

18.2.3.11 Glitch Suppression

The I²C module supports glitch suppression on the SCL and SDA lines to meet the 50ns glitch suppression as specified in the I²C specification.

Analog glitch Filter

By default, an analog glitch filter is enabled and configured to suppress spikes with a pulse width up to 50 ns. I²C spec advises to suppress noise spikes of less than 50ns. The user can disable this filter by clear the I2Cx.GFCTL.AGFEN bit and the suppression pulse width can also be configured by using I2Cx.GFCTL.AGFEN. The analog glitch filter can only be used to wake up the I²C in low-power mode.

Digital glitch Filter

The DGFSEL bits in the I2Cx.GFCTL register can be programmed to provide glitch suppression on the SCL and SDA lines and assure proper signal values. The glitch suppression value is in terms of the I²C functional clocks. All signals are delayed internally when glitch suppression is nonzero. For example, if DGFSEL bit is set to 0x7, 31 clocks should be added onto the calculation for the expected transaction time, for more information please see register description. Digital glitch filter can't be used to wake up the I2C from low-power mode.

Table 18-3. Glitch Suppression Filter

	Analog Glitch Filter	Digital Glitch Filter
Default	Default enabled with 50 ns	Default bypassed
Pulse width of suppressed spikes	Configurable 5 ns, 10 ns, 25 ns, 50 ns	Programmable I ² C clock cycle 1, 2, 3, 4, 8, 16, 31
Benefits	Available without needing clock	• Programmable length to provide extra filtering • Stable filtering length
Limitation	Variation with temperature, voltage, process	Does not work in low-power mode wakeup when there is no sufficient clock . Enabled only after 3 clock cycles after start of I ² C packet

Note

The glitch filter configuration within the register I2Cx.GFCTL should only be modified while the I²C module for controller and target is not enabled.

18.2.3.12 FIFO operation

The receive data register I2Cx.MRXDATA (for controller mode) and I2Cx.SRXDATA (for target mode) are user accessible and contains the current character to be read from the RX FIFO stack. The last received character from the receive shift register will be push to the end of the FIFO Stack.

The transmit data register I2Cx.MTXDATA (for controller mode) and I2Cx.STXDATA (for target mode) are user accessible and holds the data last written data to the TX FIFO. The TX FIFO contains the data waiting to be moved into the transmit shift register and transmitted on SDA.

FIFOs are available for the controller and target receive and transmit. Each FIFO entry has a width of 8 bits and should be accessed in byte mode. Each FIFO has a programmable threshold point (configured by RXTRIG and TXTRIG bits in the I2Cx.MFIFOCTL register for controller mode and I2Cx.SFIFOCTL register for target mode) which indicates when the FIFO service interrupt should be generated. Additionally, a FIFO receive full and transmit empty interrupt can be enabled in the interrupt mask (IMASK) registers for the controller and target.

The content of the FIFO can be flushed with setting TXFLUSH or RXFLUSH bit to 1 in the I2Cx.FIFOCTL registers. When the I²C gets reset the content of the FIFO needs also to be cleared. FIFO clear should only be executed while the I²C is in IDLE mode. Before triggering the flush the FIFO interrupts should be disabled and after flush has completed the interrupt flags needs to be checked.

18.2.3.12.1 Flushing Stale Tx Data in Target Mode

In most use cases, the user does not want to transmit leftover data of the I2C peripheral Tx FIFO from previous frame in the next frame. The device provides a mechanism for the software to choose whether to flush stale data or not.

A status bit `SSR.STALE_TXFIFO` represents whether the data present inside I2C peripheral TX FIFO is stale or not. A control bit `SCTR.TXWAIT_STALE_TXFIFO` is used to enable modified empty indication to target logic - indicate empty to I2C peripheral FSM when Tx FIFO is empty OR stale data present in Tx FIFO. The `SCTR.TXEMPTY_ON_TREQ` control bit allows the `RIS.STXEMPTY` interrupt to be used to indicate the TREQ condition, the condition when SCL is being stretched waiting for transmit data from the I2C peripheral.

Recommended Sequence

1. When `SCTR.TXWAIT_STALE_TXFIFO` is set, on STOP, restart, or timeout, I2C peripheral FSM gets an empty indication even though stale data is present inside I2C peripheral Tx FIFO.
2. I2C module doesn't immediately generate an TX FIFO empty interrupt/DMA request. Instead, waits till the controller on the I2C bus attempts to read data from the I2C peripheral, at which point the I2C peripheral clock stretches.
3. The I2C peripheral issues the clock stretch (TREQ) interrupt to the CPU when `SCTR.TXEMPTY_ON_TREQ` is set.
4. Application software in the ISR checks for a `SSR.STALE_TXFIFO` flag and flushes the TX FIFO using `SFIFCTL.TXFLUSH`, which also clears the status of `SSR.STALE_TXFIFO`.

18.2.3.13 Loopback mode

The I²C modules can be placed into an internal loopback mode for diagnostic or debug work by setting the `LPBK` bit in the I²C controller configuration `I2Cx.MCR` register. In loopback mode, the SDA and SCL signals from the controller part of the I²C are tied to the SDA and SCL signals of the target part of the I²C module to allow internal testing of the device without having to connect the I/Os. The `SWUEN` bit should be set to 0 for internal loopback to work correctly.

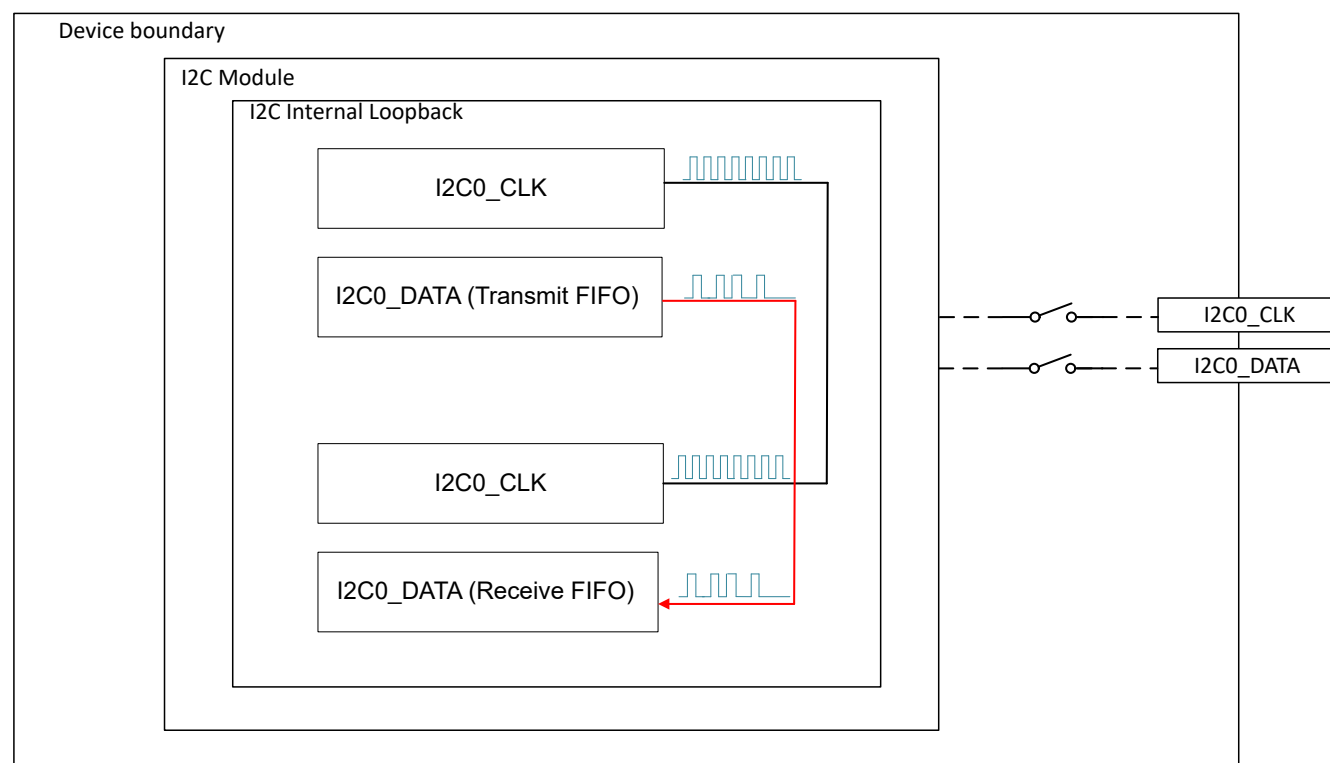


Figure 18-10. I2C Internal Loopback mode

18.2.3.14 Burst Mode

A burst mode is provided for the controller module which allows a sequence of data transfers using the DMA or software to handle the data in the FIFO. The burst mode is enabled by setting the `MBLEN` bits in the controller

control register I2Cx.MCTR to a value greater than '1'. This sets the number of bytes transferred by a burst. A copy of this value is automatically written to the MBCNT bits in the I²C controller status register I2Cx.MSR to be used as a down-counter during the burst transfer.

The bytes written to the I²C FIFO are transferred to the RX FIFO or TX FIFO depending on whether a transmit or receive is being executed. If data is not acknowledged (NACK) during a BURST and the STOP bit is set in the I2Cx.MCTR register, the transfer terminates. If the STOP bit is not set, software must issue a STOP or repeated START when a NACK interrupt is asserted. In the case of a NACK, the MBCNT bits in the I2Cx.MSR register can be used to determine the amount of data that was transferred prior to the burst termination. If the address is not acknowledged (NACK) during a transfer, then a STOP is issued.

18.2.3.15 DMA Operation

The I²C provides an interface to the DMA controller with two channels. The DMA operation of the I²C is enabled through the I²C event register and DMA peripheral registers. When the DMA functionality is enabled, the I²C asserts a DMA request on the selected channel when the associated FIFO can transfer or receive data.

For more information about I²C event and DMA, please refer to [Interrupt and Events Support](#) and [DMA Trigger Publisher](#) sections.

Note

Per DMA Channel only one event source should be enabled by the IMASK register at the same time. The DMA transaction descriptor needs to match for the selected trigger with a correct setup for either Controller or Target and RX or TX configuration. The DMA trigger should only be changed when no I²C transfer is ongoing and a previous triggered DMA transfer has been finished. If this cannot be ensured the I²C and DMA channel should be disabled first.

18.2.3.16 Low-Power Operation

I²C supports operation in low-power modes.

Supported power mode for controller mode:

- Controller mode with 100-kHz speed can operate in RUN, SLEEP, STOP power mode
- Controller mode with 400-kHz speed can operate in RUN, SLEEP power mode
- Controller mode with 1-MHz speed can operate in RUN, SLEEP power mode

In low-power mode, the I²C interface automatically request the clock selected in the CLKSEL control register after detecting a start condition. The clock request is released after detection of the STOP condition.

Supported power mode and wakeup mode for target mode:

- Target mode with 100-kHz speed can operate in RUN, SLEEP, STOP power mode
- Target mode with 100-kHz speed can't operate in STANDBY mode because the maximum bus clock in STANDBY mode is 32 kHz and support for 100-kHz speed requires a minimum 400-kHz clock. However, in STANDBY mode the I2C target can still wakeup from the start bit and perform the Asynchronous Fast Clock Requests to temporarily get a 32-MHz clock to receive the data until the FIFO or address match interrupt wakes up the CPU.
- Target mode with 400-kHz speed in RUN, SLEEP power mode
- Target mode with 1-MHz speed in RUN, SLEEP power mode

18.2.4 Protocol Descriptions

18.2.4.1 I²C Controller Mode

As showing in the [function block diagram](#) section, I²C peripheral has a set of specific controller registers to configure the operation when the module is configured as an I²C target mode.

18.2.4.1.1 Controller Configuration

The I²C Controller Control register I2Cx.MCTR and I²C Controller Target Address register I2Cx.MSA are used for controlling controller transmit and receive modes. The following settings can be modified to control the different transactions.

- Length indicates the number of bytes for the transaction and is configured by I2Cx.MCTR.MBLEN bit
- Direction (transmit or receive) is configured by I2Cx.MSA.DIR bit
- ACK generation is configured by I2Cx.MCTR.MBLEN bit
- STOP condition generation is configured by I2Cx.MCTR.STOP bit
- START or repeated START condition generation is configured by I2Cx.MCTR.START bit
- RUN enable the controller transaction and is configured by I2Cx.MCTR.BURSTRUN bit

Controller send data transactions

Table 18-4. Start Transmit From Idle Mode

Length	Direction	ACK	STOP	START	RUN	Format	Comments
n (n>0)	0	x	0 or 1	1	1	START+ADDR+R/W+ DATA*n+(STOP)	Sending of STOP depends on STOP bit

Table 18-5. Continue Transmit When Last Transmission Finished Without stop

Length	Direction	ACK	STOP	START	RUN	Format	Comments
n (n>0)	0	x	0 or 1	0	1	DATA*n+ (ACK/NACK)+ (STOP)	Sending of STOP depends on STOP bit

If there is a NACK response from the target, the controller will automatically send out a stop to finish the transmit. The Controller will be unable to send a RESTART after ADDR or DATA NACK.

Controller receive data transactions

Table 18-6. Start Receive From Idle Mode

Length	Direction	ACK	STOP	START	RUN	Format	Comments
n (n>0)	1	0 or 1	0 or 1	1	1	START+ADDR +R/ W+DATA *n +(ACK/NACK) +(STOP)	The last data ACK or NACK depend on ACK bit; additional sending of STOP depends on STOP bit

Table 18-7. Continue Receive When Last Receive Finished Without STOP or NACK

Length	Direction	ACK	STOP	START	RUN	Format	Comments
n (n>0)	1	0 or 1	0 or 1	0	1	DATA*n+ (ACK/NACK) + (STOP)	The last data followed by ACK or NACK depend on ACK bit; additional sending of STOP depends on STOP bit

This configuration is not allowed if last transaction ended with NACK, as NACK can only be followed by STOP or RESTART. The ACK and STOP bits should not be set to 1 at the same time, as the target needs to be informed to release bus lines before sending out STOP.

Controller repeated start transactions

If last transmit or receive finished without stop, a repeat start can be generated to initiate a new transaction

Table 18-8. Repeated Start Transmit

Length	Direction	ACK	STOP	START	RUN	Format	Comments
n (n>0)	0	0 or 1	0 or 1	1	1	Restart+ADDR +R/ W+DATA*n +(STOP)	Additional sending of STOP depends on STOP bit

If there is a NACK response from the target, the controller will automatically send out a stop to finish the transmit. The Controller will be unable to send a RESTART after ADDR or DATA NACK.

Table 18-9. Repeated Start Receive

Length	Direction	ACK	STOP	START	RUN	Format	Comments
n (n>0)	1	0 or 1	0 or 1	1	1	Restart+ADDR +R/ W+DATA*n +(ACK/NACK) +(STOP)	The last data followed by ACK or NACK depend on ACK bit; additional sending of STOP depends on STOP bit

The ACK and STOP bits should not be set to 1 at the same time, as the target needs to be informed to release bus lines before sending out STOP.

Controller send STOP only transaction

Table 18-10. Send STOP only

Length	Direction	ACK	STOP	START	RUN	Format	Comments
n (n>0)	X	X	1	0	1	STOP	STOP command

It is only allowed to send after previous transaction success finished, and STOP can't be sent without a NACK to the target if controller is currently in receive mode

Controller Quick Command transaction

The Quick command could only be sent at the transaction beginning, not following other transactions (without stop) or repeat start.

Table 18-11. Controller quick command

Length	Direction	ACK	STOP	START	RUN	Format	Comments
0	0/1	X	1	1	1	START+ADDR +R/ W+STOP	Quick command

18.2.4.1.2 Controller Mode Operation

I²C Controller Initialization

1. Configure SDA and SCL pin functions and select as input by using the IOMUX registers.
2. Reset the peripheral using I2Cx.RSTCTL register
3. Enable the power to peripheral using I2Cx.PWREN register
4. Select and configure the I²C clock using the CLKCTL and CLKDIV registers.
5. Set the desired SCL clock speed of by writing the TPR bit in I2Cx.MTPR register with the correct value. For more information about how to calculate TPR value refer to [Clock Control](#) section. For example, with 20MHz I²C clock to achieve 100 kbps SCL clock, TPR value will be 19 (0x13) so we can write the I2Cx.MTPR register with the value of 0x13.
6. Specify the target address and mode (transmit or receive) for the next operation by writing the I2Cx.MSA register. For example, if the target address is 0x3B (MSA.SADDR[7:1] = 0x3B) and we want to transmit (MSA.DIR[0] = 0x0) data then we can write the I2Cx.MSA register with a value of 0x76.
7. If controller is transmitting data, user can place data (byte) to be transmitted in the data register by writing the I2Cx.MTXDATA register with the desired data.

8. Configure the controller transmit or receive mode by writing the I2Cx.MCTR register. For more information on how to configure I2Cx.MCTR register for different mode please see [Controller Configuration](#) section above.
9. Enable desired interrupts and/or DMA event by using CPU_INT.IMASK register.

I²C Controller Status

User can read the I2Cx.MSR register to check the current state of the I²C controller.

Table 18-12. Controller Status Register

Bit Field	Description
BUSY	I2C controller busy. The BUSY bit is set during an ongoing transaction.
ERR	I2C error. The error can be from the target address not being acknowledged or the transmit data not being acknowledged.
ADRACK	Acknowledge address. This bit is set if the transmitted address was not acknowledged.
DATAACK	Acknowledge data. This bit is set if the transmitted data was not acknowledged.
ARBLST	Arbitration lost. This bit is set if controller lost arbitration.
IDLE	I2C bus idle.
BUSBSY	I2C bus busy. The bit changes based on the START and STOP conditions, set if bus is busy.
CLKTO	Clock timeout error. This bit is set if the clock timeout error has occurred.
MBCNT	I2C controller transaction count. This field contains the current countdown value of the transaction.

I²C Controller Receiver Mode

For controller to start receive data out of the idle mode, user needs to set the START bit in I2Cx.MCTR register to generate the start condition. Then the controller automatically sends the START condition followed by the target address as soon as it detects that the bus is free. All the process below should be followed.

I2Cx.MSA.DIR is set to 1 to enable receive mode, I2Cx.MCTR.START is set to generate start condition, I2Cx.MBLEN can be programmed to indicate the number of bytes (n) for the receive operation. I2Cx.MCTR.ACK and I2Cx.MCTR.STOP bit can be set or clear based on user configuration. I2Cx.MCTR.BURSTRUN is set to start the operation. The packet format is START+ADDR+R+DATA*n +(ACK/NACK) + (STOP). The last data ACK/NACK depend on ACK bit, additional sending of STOP depends on STOP bit.

After last byte is received, the MRXDONE (0x01) interrupt in CPU_INT.IIDX register is set to indicate that controller receive transaction is completed. User can use the MRXFIFOTRG (0x03) interrupt in CPU_INT.IIDX register to read the data from the receive FIFO. This interrupt will trigger when controller RX FIFO contains >= defined bytes, the trigger level can be defined by using RXTRIG bit in I2Cx.MFIFOCTL register. The flow chart of controller receiver mode is shown in [Figure 18-11](#).

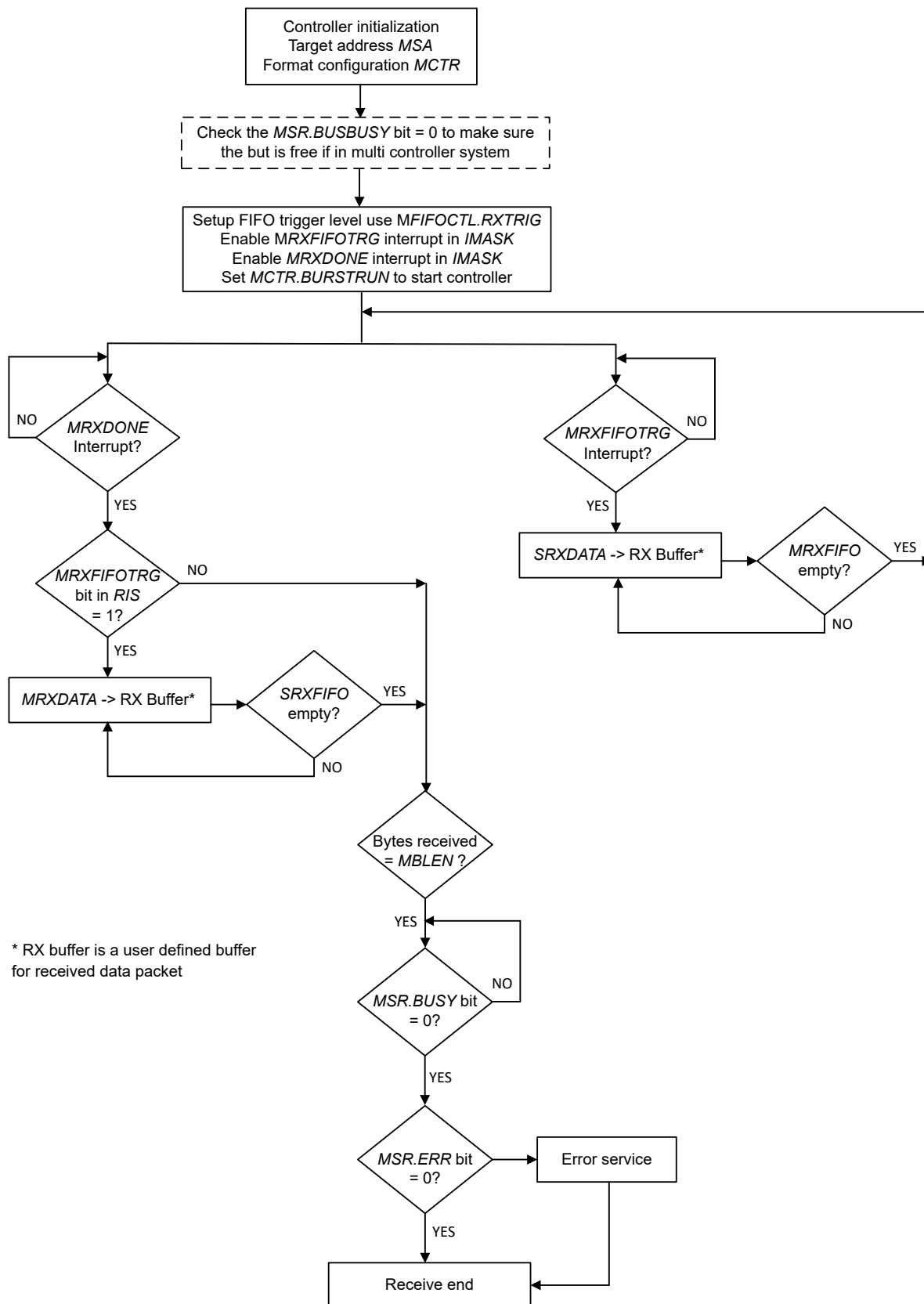


Figure 18-11. Controller Receiver Mode

I²C Controller transmitter Mode

For controller, to start transmit data out of the idle mode, user need to set the START bit in I2Cx.MCTR register to generate the start condition. Then the controller automatically sends the START condition followed by the target address as soon as it detects that the bus is free. The data written into the MTXFIFO is transmitted if arbitration is not lost during transmission of the target address. All the process below should be followed.

I2Cx.MSA.DIR is cleared to enable transmit mode, I2Cx.MCTR.START is set to generate start condition, I2Cx.MBLEN can be programmed to indicate the number of bytes (n) for the transmit operation. I2Cx.MCTR.STOP bit can be set or clear based on user configuration. I2Cx.MCTR.BURSTRUN is set to start the operation. The packet format is START+ADDR+W+DATA*n + (STOP), sending of STOP depends on STOP bit.

After last byte is transmitted, the MTXDONE (0x02) interrupt in CPU_INT.IIDX register is set to indicate that controller transmit transaction is completed. User can also set use the MTXEMPTY (0x06) interrupt in CPU_INT.IIDX register to see if the MTXFIFO is empty and ready to load more data. This interrupt will trigger if all data in the transmit FIFO have been shifted out. The flow chart of controller receiver mode is shown in [Figure 18-12](#).

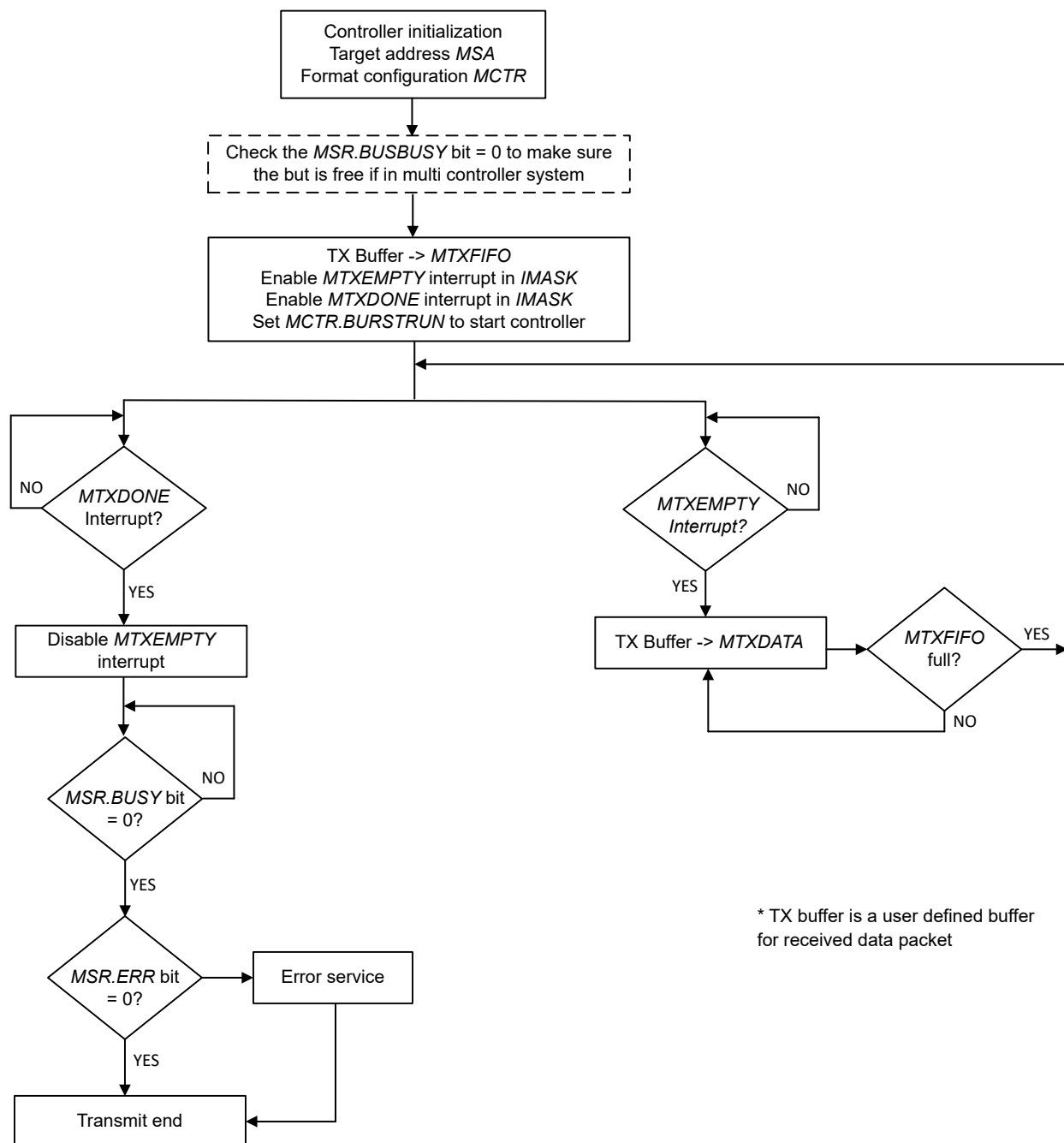


Figure 18-12. Controller Transmitter Mode

18.2.4.1.3 Read On TX Empty

Typically, in any I²C transaction one would need to first write the target in order to set the register number, then do a repeated start + read to get back the data value. However, the RD_ON_EMPTY flag allows one to do two I²C transactions with a single setup of the I²C controller. One can set up both the write and read together; with the limitation being that the "write" phase cannot send more than the data that fits inside the TX FIFO. This is an optimization to minimize interrupt processing requirements. The high level set up is below:

- Set MSA to target address with RECEIVE = 1 (RD_ON_TXEMPTY must have I2C_MSR_DIR_RECEIVE set)
- Set MCTR to initiate the transaction, in this use case write:

- MBLEN= size for READ phase
- RD_ON_TXEMPTY
- ACK_DISABLE(my use case will NACK+STOP after read)
- STOP_ENABLE
- START_ENABLE
- BURST_ENABLE

The expected behavior with this set up is that the controller will START, transmit all TXFIFO data and when TXFIFO is empty will automatically do a repeated START, read MBLEN bytes, then STOP .

18.2.4.2 I²C Target Mode

18.2.4.2.1 Target Mode Operation

I²C Target Initialization

1. Configure SDA and SCL pin functions and select as input by using the IOMUX registers.
2. Reset the peripheral using I2Cx.RSTCTL register
3. Enable the power to peripheral using I2Cx.PWREN register
4. Select and configure the I²C clock using the CLKCTL and CLKDIV registers.
5. Configure at least one target address by writing the 7-bit address to I2Cx.SOAR register. The additional target address can be enabled and configured by using I2Cx.SOAR2 register.
6. Enable desired interrupts and/or DMA event by using CPU_INT.IMASK register.
7. The general call response can be enabled by setting the GENCALL bit in I2Cx.SCTR register.
8. Enable the I²C target mode by setting the ACTIVE bit in I2Cx.SCTR register.

I²C Target Status

User can read the I2Cx.SSR register to check the current state of the I²C target.

Table 18-13. Target Status Register

Bit Field	Description
RREQ	This bit is set if the I2C controller has outstanding receive data from the I2C controller and is using clock stretching to delay the controller until the data has been read from the SRXDATA FIFO (Target RX FIFO is full)
TREQ	This bit is set if the I2C controller has been addressed as a target transmitter and is using clock stretching to delay the controller until data has been written to the STXDATA FIFO (Target TX FIFO is empty).
OAR2SEL	This bit is set if SOAR2.OAR2 address matched and ACKed by the target.
QCMDST	Quick command status value. This bit is 0 if the last transaction was a normal transaction or a transaction has not occurred. This bit is set if the last transaction was a quick command transaction
QCMDRW	Quick command read / write status. This bit only has meaning when the QCMDST bit is set. This bit is 0 if quick command was a write. This bit is set if quick command was a read.

I²C Target Receiver Mode

Target receiver mode is entered when the target address transmitted by the controller matches its own address and a cleared R/W bit is received. In target receiver mode, serial data bits received on SDA are shifted in with the clock pulses that are generated by the controller device. The target device does not generate the clock, but it can hold SCL low if intervention of the CPU is required after a byte has been received.

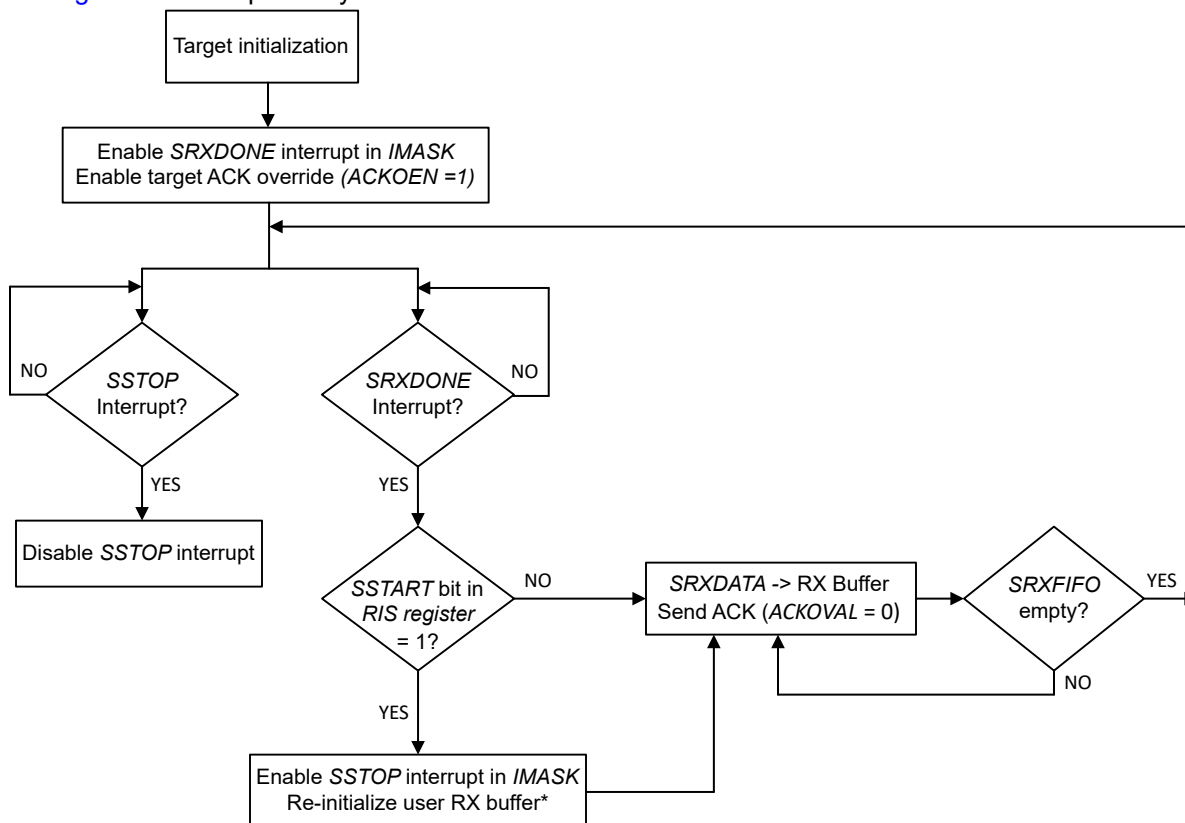
After the a data byte is received, the SRXDONE (0x11) interrupt in CPU_INT.IIDX register is set to indicate that a byte has been received. The I²C module automatically acknowledges the received data or user can choice to manually send acknowledge after each byte received by configuring the I2Cx.SACKCTL register.

When the controller generates a START condition, the SSTART (0x17) interrupt in CPU_INT.IIDX register is set. When the controller generates a STOP condition, the SSTOP (0x18) interrupt in CPU_INT.IIDX register is set.

User can also set use the SRXFIFOTRG (0x13) interrupt in CPU_INT.IIDX register to read the data from the receive FIFO. This interrupt will trigger when receive FIFO contains >= defined bytes, the trigger level can be defined by using RXTRIG bit in I2Cx.SFIFOCTL register.

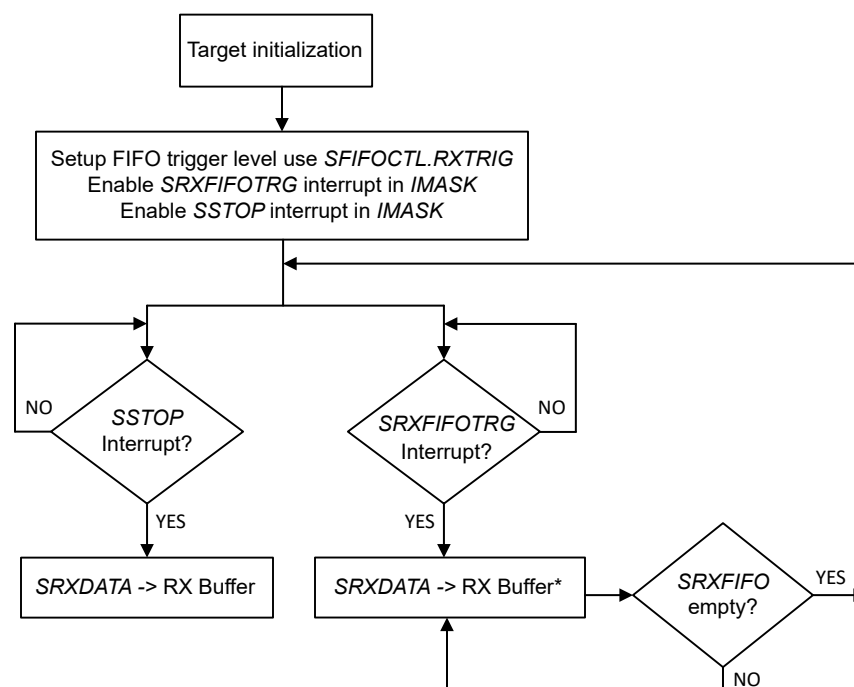
The SRXDONE approach could be used if target wants to slow down communication to evaluate reception of every byte, while the SRXFIFOTRG approach could be used to maximize throughput and avoid clock stretching.

The flow chart of using SRXDONE and SRXFIFOTRG interrupt to read the receive data are shown in [Figure 18-13](#) and [Figure 18-14](#) respectively.



* RX buffer is a user defined buffer
for received data packet

Figure 18-13. Target Receiver Mode using SRXDONE and ACK override



* RX buffer is a user defined buffer
for received data packet

Figure 18-14. Target Receiver Mode using SRXFIFOTRG and automatic ACK

I²C Target Transmitter Mode

Target transmitter mode is entered when the target address transmitted by the controller is identical to its own address with a set R/W bit. The target transmitter shifts the serial data out on SDA with the clock pulses that are generated by the controller device. The target device does not generate the clock, but it can hold SCL low if intervention of the CPU is required after a byte has been transmitted.

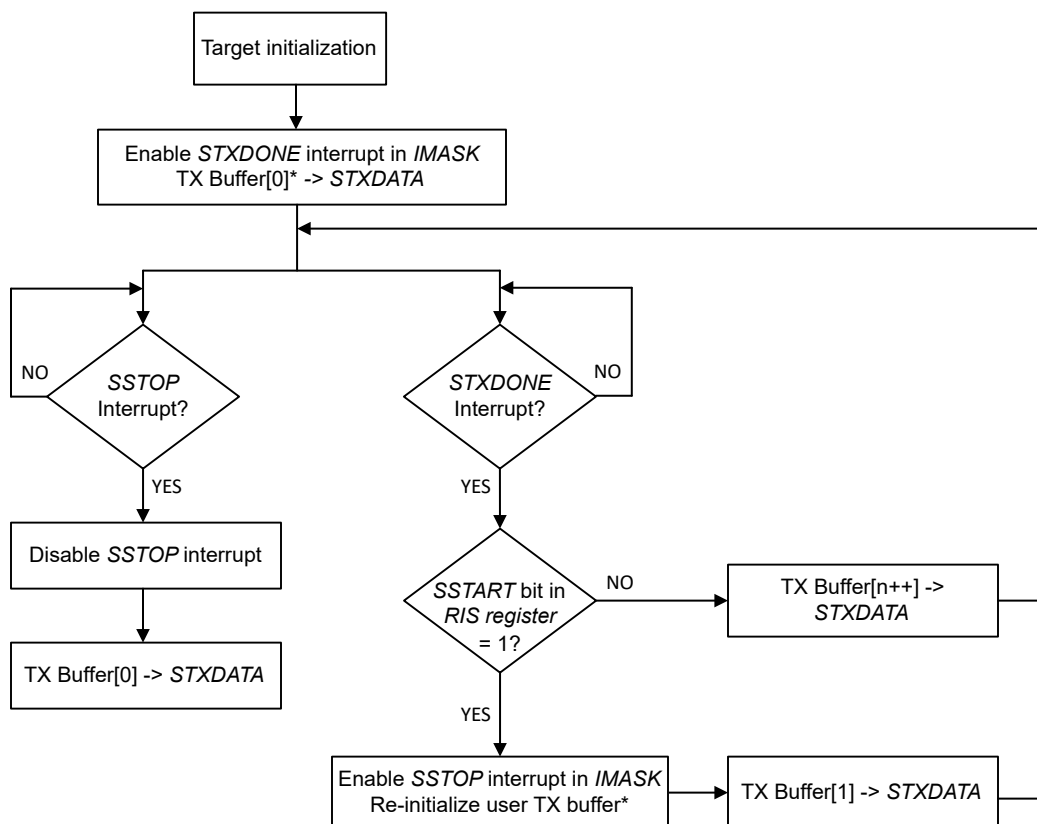
After a data byte is transmitted, the STXDONE (0x12) interrupt in CPU_INT.IIDX register is set to indicate that a byte has been transmitted.

When the controller generates a START condition, the SSTART (0x17) interrupt in CPU_INT.IIDX register is set. When the controller generates a STOP condition, the SSTOP (0x18) interrupt in CPU_INT.IIDX register is set.

User can also set use the STXFIFOTRG (0x14) interrupt in CPU_INT.IIDX register to load the data to the transmit FIFO. This interrupt will trigger when transmit FIFO contains <= defined bytes, the trigger level can be defined by using TXTRIG bit in I2Cx.SFIFOCTL register.

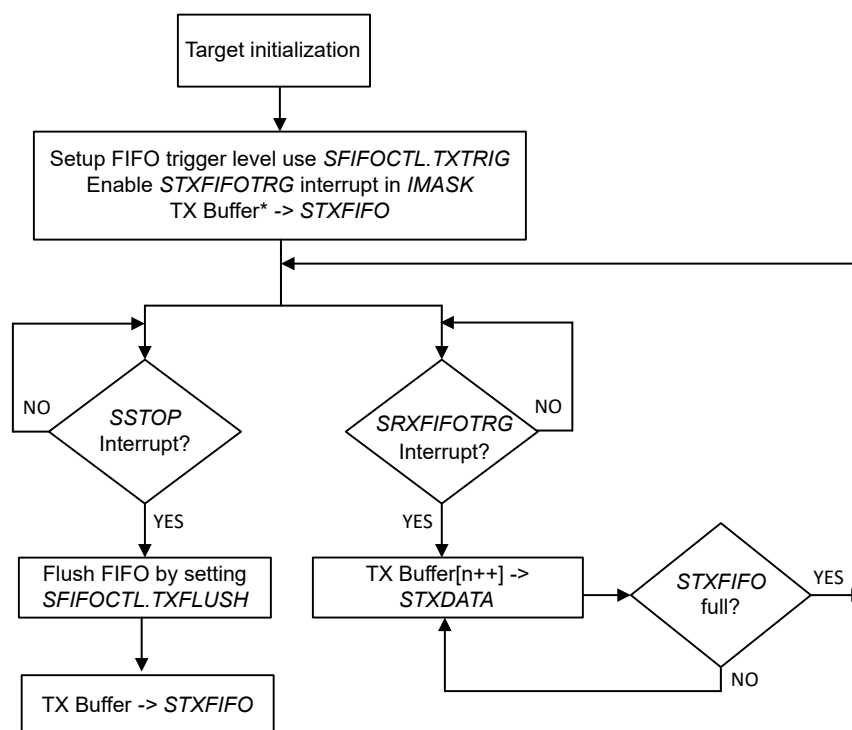
The STXDONE approach could be used if target wants to slow down communication to evaluate reception of every byte, while the STXFIFOTRG approach could be used to maximize throughput and avoid clock stretching.

The flow chat of using STXDONE and STXFIFOTRG interrupt to transmit data are shown in [Figure 18-15](#) and [Figure 18-16](#).



* TX buffer is a user defined buffer
for transmit data packet

Figure 18-15. Target Transmitter Mode using STXDONE



* TX buffer is a user defined buffer
for received data packet

Figure 18-16. Target Transmitter Mode using STXFIFOTRG

18.2.5 Reset Considerations

Software Reset Considerations

A Software reset can be executed by setting the RESETASSERT bit together with the KEY bit in the I2Cx.RSTCTL register. We recommend issues a reset only after terminating a transaction.

Hardware Reset Considerations

A hardware reset also initializes the IO configuration. This sets the IOs to a high-impedance state and with the external pullup resistors for I²C the lines pulled high.

Table 18-14 shows the behavior of status bits when Controller or Target gets disabled.

Table 18-14. Status Bits When Controller is Disabled

Register	Bit	Behavior When Controller Disabled	Behavior When Target Disabled	Behavior After Controller/Target Enabled
I2Cx.MSR	Busy	Reset State	Don't care	updates Start condition sending
	ERR	Reset State	Don't care	updates on next event detected
	ADRACK	Reset State	Don't care	updates on next event detected
	DATAACK	Reset State	Don't care	updates on next event detected
	ARBLST	Reset State	Don't care	updates on next event detected
	IDLE	Reset State	Don't care	updates on next event detected
	BUSBSY	Reset State	Don't care	updates on next Start detected on bus (or SDA or SCL is low)
	CLKTO	Reset State	Don't care	updates on next event detected
	MBCNT	Reset State	Don't care	updates on next event detected
I2Cx.MCLKCNT	CLKCNT	Reset State	Don't care	updates with Controller enable when SCL is high

Table 18-14. Status Bits When Controller is Disabled (continued)

Register	Bit	Behavior When Controller Disabled	Behavior When Target Disabled	Behavior After Controller/Target Enabled
I2Cx.MBMON	SCL	Reset State	Don't care	updates with Controller enable
	SDA	Reset State	Don't care	updates with Controller enable

Table 18-15. Status Bits When Target is Disabled

Register	Bit	Behavior When Controller Disabled	Behavior When Target Disabled	Behavior After Controller/Target Enabled
I2Cx.SSR	RREQ	Don't care	Reset State	updates on next event detected
	TREQ	Don't care	Reset State	updates on next event detected
	OAR2SEL	Don't care	Reset State	updates on next event detected
	QCMDST	Don't care	Reset State	updates on next event detected
	QCMDRW	Don't care	Reset State	updates on next event detected
I2Cx.SFIFOSR	RXFIFOCNT	Don't care	Unchanged	updates on FIFO access
	TXFIFOCNT	Don't care	Unchanged	updates on FIFO access

18.2.6 Initialization

Please see [Section 18.2.4.1.2](#) for controller initialization and [Section 18.2.4.2.1](#) for target initialization.

Note

The configuration registers of the I2C module (including the clock configuration, mode configuration, and timeout count) must not be modified by application software when an I2C data transfer is in progress.

18.2.7 Interrupt and Events Support

The I²C module contains three [event publishers](#) and no [event subscribers](#). One event publisher (CPU_INT) manages I²C interrupt requests (IRQs) to the CPU subsystem through a [static event route](#). The second and third event publishers (DMA_TRIG1, DMA_TRIG0) are used to setup the trigger signaling for the DMA through [DMA event route](#).

The I²C events are summarized in [Table 18-16](#).

Table 18-16. I2C Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU interrupt	Publisher	I ² C	CPU Subsystem	Static route	CPU_INT registers	Fixed interrupt route from I ² C to CPU
DMA trigger	Publisher	I ² C	DMA	DMA event route	DMA_TRIG1 registers	Fixed interrupt route from I ² C to DMA
DMA trigger	Publisher	I ² C	DMA	DMA event route	DMA_TRIG0 registers	Fixed interrupt route from I ² C to DMA

18.2.7.1 CPU Interrupt Event Publisher (CPU_INT)

The I²C module provides 24 interrupt sources which can be configured to source a [CPU interrupt event](#). In order of decreasing interrupt priority, the CPU interrupt events from the I²C are:

Table 18-17. I²C CPU Interrupt Event Conditions for Controller (CPU_INT)

IIDX STAT	Name	Description
0x01	MRXDONE	Controller receive transaction completed interrupt
0x02	MTXDONE	Controller transmit transaction completed interrupt
0x03	MRXFIFOTRG	Controller receive FIFO trigger. Trigger when RX FIFO contains >= defined bytes
0x04	MTXFIFOTRG	Controller transmit FIFO trigger. Trigger when Transmit FIFO contains <= defined bytes
0x05	MRXFIFOFULL	Controller RXFIFO full event. This interrupt is set if an RX FIFO is full.
0x06	MTXEMPTY	Controller transmit FIFO empty interrupt. This interrupt is set if all data in the Transmit FIFO have been shifted out and the transmit goes into idle mode.
0x07	MCLKTO	Controller clock timeout interrupt
0x08	MNACK	Address/Data NACK interrupt
0x09	MSTART	Controller START detection interrupt
0x0A	MSTOP	Controller STOP detection interrupt
0x0B	MARBLOST	Controller arbitration lost interrupt
0x0C	MDMA_DONE_TX	Controller DMA TX done signal (see next section for more detail)
0x0D	MDMA_DONE_RX	Controller DMA RX done signal (see next section for more detail)

Table 18-18. I²C CPU Interrupt Event Conditions for Target (CPU_INT)

IIDX STAT	Name	Description
0x11	SRXDONE	Target receive transaction completed interrupt
0x12	STXDONE	Target transmit transaction completed interrupt
0x13	SRXFIFOTRG	Target receive FIFO trigger. It will trigger when receive FIFO contains >= defined bytes
0x14	STXFIFOTRG	Target transmit FIFO trigger. It will trigger when transmit FIFO contains <= defined bytes
0x15	SXFIFOFULL	Target RXFIFO full event. This interrupt is set if an RX FIFO is full.
0x16	STXEMPTY	Target transmit FIFO empty interrupt. This interrupt is set if all data in the Target Transmit FIFO have been shifted out and the transmit goes into idle mode.
0x17	SSTART	Target START detection interrupt
0x18	SSTOP	Target STOP detection interrupt
0x19	SGENCALL	General call interrupt
0x1A	SDMA_DONE_TX	Target DMA TX done signal (see next section for more detail)
0x1B	SDMA_DONE_RX	Target DMA RX done signal (see next section for more detail)

The CPU interrupt event configuration is managed with the CPU_INT event management registers. See [Section 7.2.5](#) for guidance on configuring the Event registers for CPU interrupts.

18.2.7.2 DMA Trigger Publisher (DMA_TRIG1, DMA_TRIG0)

DMA_TRIG1 and DMA_TRIG0 registers are used to setup the trigger signaling for the DMA. This can be setup in a flexible way to trigger the DMA for Controller or Target and receive or transmit events with the following four trigger conditions:

Table 18-19. I²C DMA Trigger Condition (DMA_TRIG1 and DMA_TRIG0)

IIDX STAT	Name	Description
0x01	MRXFIFOTRG	Controller receive FIFO trigger. Trigger when RX FIFO contains >= defined bytes
0x02	MTXFIFOTRG	Controller transmit FIFO trigger. Trigger when Transmit FIFO contains <= defined bytes
0x03	SRXFIFOTRG	Target receive FIFO trigger. Trigger when RX FIFO contains >= defined bytes
0x04	STXFIFOTRG	Target transmit FIFO trigger. Trigger when Transmit FIFO contains <= defined bytes

The DMA trigger event configuration is managed with the DMA_TRIG1 and DMA_TRIG0 event management registers. See [Section 7.2.5](#) for guidance on configuring the Event registers and [Section 7.1.3.2](#) for on how DMA trigger event works. DMA_TRIG1 and DMA_TRIG0 are two event management registers that correspond to two DMA channels.

As shown in [Figure 18-17](#), each DMA channel can be triggered by any of the conditions listed in [Table 18-19](#) and it can generate either the controller DMA done signal or target DMA done signal.

For example, the user can configure the DMA_TRIG1 trigger using MTXFIFOTRG and the DMA_TRIG0 trigger using SRXFIFOTRG. When the Channel 1 DMA status changes to done, the MDMA_DONE_TX and MDMA_DONE_RX interrupts will set, and when the Channel 2 DMA status change to done, the SDMA_DONE_TX and SDMA_DONE_RX interrupts will set.

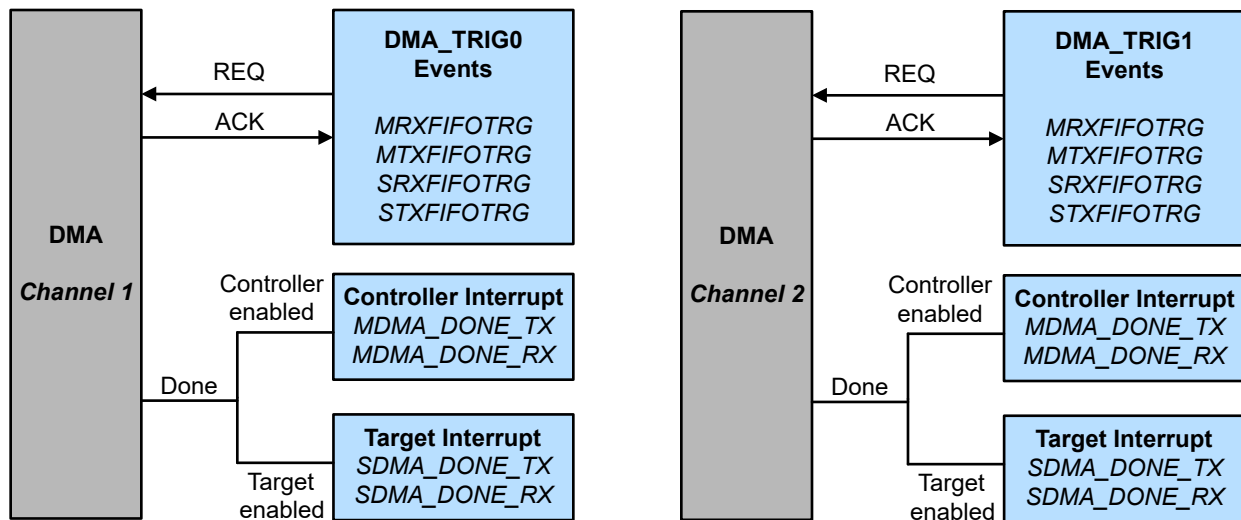


Figure 18-17. I²C DMA Trigger and Status

18.2.8 Emulation Modes

The module behavior while the device is in debug mode is controlled by the FREE and SOFT bits in PDBGCTL register.

When the device is in debug mode and set into halt mode below behavior can be configured.

Table 18-20. Debug Mode Peripheral Behavior

PDBGCTL.FREE	PDBGCTL.SOFT	Function
1	x	Modules continues operation
0	0	Module stops immediately
0	1	Module stops after the next transfer has been finished

This page intentionally left blank.

Chapter 19

I2C Registers



Table 19-1 lists the memory-mapped registers for the I2C registers. All register offset addresses not listed in Table 19-1 should be considered as reserved locations and the register contents should not be modified.

Table 19-1. I2C Registers

Offset	Acronym	Register Name	Section
800h	PWREN	Power enable	Section 19.1
804h	RSTCTL	Reset Control	Section 19.2
808h	CLKCFG	Peripheral Clock Configuration Register	Section 19.3
814h	STAT	Status Register	Section 19.4
1000h	CLKDIV	Clock Divider	Section 19.5
1004h	CLKSEL	Clock Select for Ultra Low Power peripherals	Section 19.6
1018h	PDBGCTL	Peripheral Debug Control	Section 19.7
1020h	IIDX	Interrupt index	Section 19.8
1028h	IMASK	Interrupt mask	Section 19.9
1030h	RIS	Raw interrupt status	Section 19.10
1038h	MIS	Masked interrupt status	Section 19.11
1040h	ISET	Interrupt set	Section 19.12
1048h	ICLR	Interrupt clear	Section 19.13
1050h	IIDX	Interrupt index	Section 19.14
1058h	IMASK	Interrupt mask	Section 19.15
1060h	RIS	Raw interrupt status	Section 19.16
1068h	MIS	Masked interrupt status	Section 19.17
1070h	ISET	Interrupt set	Section 19.18
1078h	ICLR	Interrupt clear	Section 19.19
1080h	IIDX	Interrupt index	Section 19.20
1088h	IMASK	Interrupt mask	Section 19.21
1090h	RIS	Raw interrupt status	Section 19.22
1098h	MIS	Masked interrupt status	Section 19.23
10A0h	ISET	Interrupt set	Section 19.24
10A8h	ICLR	Interrupt clear	Section 19.25
10E0h	EVT_MODE	Event Mode	Section 19.26
10E4h	INTCTL	Interrupt control register	Section 19.27
10FCh	DESC	Module Description	Section 19.28
1200h	GFCTL	I2C Glitch Filter Control	Section 19.29
1204h	TIMEOUT_CTL	I2C Timeout Count Control Register	Section 19.30
1208h	TIMEOUT_CNT	I2C Timeout Count Register	Section 19.31
1210h	MSA	I2C Controller Target Address Register	Section 19.32
1214h	MCTR	I2C Controller Control Register	Section 19.33
1218h	MSR	I2C Controller Status Register	Section 19.34

Table 19-1. I2C Registers (continued)

Offset	Acronym	Register Name	Section
121Ch	MRXDATA	I2C Controller RXData	Section 19.35
1220h	MTXDATA	I2C Controller TXData	Section 19.36
1224h	MTPR	I2C Controller Timer Period	Section 19.37
1228h	MCR	I2C Controller Configuration	Section 19.38
1234h	MBMON	I2C Controller Bus Monitor	Section 19.39
1238h	MFIFOCTL	I2C Controller FIFO Control	Section 19.40
123Ch	MFIFOSR	I2C Controller FIFO Status Register	Section 19.41
1250h	SOAR	I2C Target Own Address	Section 19.42
1254h	SOAR2	I2C Target Own Address 2	Section 19.43
1258h	SCTR	I2C Target Control Register	Section 19.44
125Ch	SSR	I2C Target Status Register	Section 19.45
1260h	SRXDATA	I2C Target RXData	Section 19.46
1264h	STXDATA	I2C Target TXData	Section 19.47
126Ch	SFIFOCTL	I2C Target FIFO Control	Section 19.48
1270h	SFIFOSR	I2C Target FIFO Status Register	Section 19.49

Complex bit access types are encoded to fit into small table cells. [Table 19-2](#) shows the codes that are used for access types in this section.

Table 19-2. I2C Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value

19.1 PWREN Register (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 19-1](#) and described in [Table 19-3](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 19-1. PWREN Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R-0h							R/WK-0h

Table 19-3. PWREN Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R	0h	
0	ENABLE	R/WK	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

19.2 RSTCTL Register (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 19-2](#) and described in [Table 19-4](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 19-2. RSTCTL Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCLR	RESETASSERT
R-0h						R	
						WK-0h	WK-0h

Table 19-4. RSTCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	R	0h	
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

19.3 CLKCFG Register (Offset = 808h) [Reset = 00000000h]

CLKCFG is shown in [Figure 19-3](#) and described in [Table 19-5](#).

Return to the [Summary Table](#).

Peripheral Clock Configuration Register

Figure 19-3. CLKCFG Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							BLOCKASYNC
R-0h							R/W-0h
7	6	5	4	3	2	1	0
RESERVED							
R-0h							

Table 19-5. CLKCFG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to Allow State Change -- 0xA9 A9h = key value to allow change field of GPRCM
23-9	RESERVED	R	0h	
8	BLOCKASYNC	R/W	0h	Async Clock Request is blocked from starting SYSOSC or forcing bus clock to 32MHz 0h = Not block async clock request 1h = Block async clock request
7-0	RESERVED	R	0h	

19.4 STAT Register (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 19-4](#) and described in [Table 19-6](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 19-4. STAT Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED						RESETSTKY	
R-0h						R-0h	
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							
R-0h							

Table 19-6. STAT Register Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0x0	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

19.5 CLKDIV Register (Offset = 1000h) [Reset = 00000000h]

CLKDIV is shown in [Figure 19-5](#) and described in [Table 19-7](#).

Return to the [Summary Table](#).

This register is used to specify module-specific divide ratio of the functional clock

Figure 19-5. CLKDIV Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												RATIO			
R-0h												R/W-0h			

Table 19-7. CLKDIV Register Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R	0h	
2-0	RATIO	R/W	0h	Selects divide ratio of module clock 0h = Do not divide clock source 1h = Divide clock source by 2 2h = Divide clock source by 3 3h = Divide clock source by 4 4h = Divide clock source by 5 5h = Divide clock source by 6 6h = Divide clock source by 7 7h = Divide clock source by 8

19.6 CLKSEL Register (Offset = 1004h) [Reset = 00000000h]

CLKSEL is shown in [Figure 19-6](#) and described in [Table 19-8](#).

Return to the [Summary Table](#).

Clock source selection.

Figure 19-6. CLKSEL Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				BUSCLK_SEL	MFCLK_SEL	RESERVED	
R-0h				R/W-0h	R/W-0h	R-0h	

Table 19-8. CLKSEL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	BUSCLK_SEL	R/W	0h	Selects BUSCLK as clock source if enabled 0h = Does not select this clock as a source 1h = Select this clock as a source
2	MFCLK_SEL	R/W	0h	Selects MFCLK as clock source if enabled 0h = Does not select this clock as a source 1h = Select this clock as a source
1-0	RESERVED	R	0h	

19.7 PDBGCTL Register (Offset = 1018h) [Reset = 00000000h]

PDBGCTL is shown in [Figure 19-7](#) and described in [Table 19-9](#).

Return to the [Summary Table](#).

This register can be used by the software developer to control the behavior of the peripheral relative to the 'Core Halted' input

Figure 19-7. PDBGCTL Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED						SOFT	FREE
R-0h						R/W-0h	R/W-0h

Table 19-9. PDBGCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1	SOFT	R/W	0h	Soft halt boundary control. This function is only available, if FREE is set to 'STOP' 0h = The peripheral will halt immediately, even if the resultant state will result in corruption if the system is restarted 1h = The peripheral blocks the debug freeze until it has reached a boundary where it can resume without corruption
0	FREE	R/W	0h	Free run control 0h = The peripheral freezes functionality while the Core Halted input is asserted and resumes when it is deasserted. 1h = The peripheral ignores the state of the Core Halted input

19.8 IIDX Register (Offset = 1020h) [Reset = 00000000h]

IIDX is shown in [Figure 19-8](#) and described in [Table 19-10](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index.

Figure 19-8. IIDX Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 19-10. IIDX Register Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	I2C Module Interrupt Vector Value. This register provides the highest priority interrupt index. A read clears the corresponding interrupt flag in RIS and MISC. 15h-1Fh = Reserved 00h = No interrupt pending 01h = Controller data received 02h = Controller data transmitted 03h = Controller receive FIFO Trigger Level 04h = Controller transmit FIFO Trigger level 5h = RX FIFO FULL Event/interrupt pending 6h = Transmit FIFO/Buffer Empty Event/interrupt pending 08h = Address/Data NACK 09h = Start Event 0Ah = Stop Event 0Bh = Arbitration Lost Ch = DMA DONE on Channel TX Dh = DMA DONE on Channel RX Eh = Controller PEC Receive Error Event Fh = Timeout A Event 10h = Timeout B Event 11h = Target Data Event 12h = Target Data Event 13h = Target receive FIFO Trigger Level 14h = Target transmit FIFO Trigger level 15h = RX FIFO FULL Event/interrupt pending 16h = Transmit FIFO/Buffer Empty Event/interrupt pending 17h = Start Event 18h = Stop Event 19h = General Call Event 1Ah = DMA DONE on Channel TX 1Bh = DMA DONE on Channel RX 1Ch = Target PEC receive error event 1Dh = Target TX FIFO underflow 1Eh = Target RX FIFO overflow event 1Fh = Target arbitration lost event 20h = Interrupt overflow event

19.9 IMASK Register (Offset = 1028h) [Reset = 00000000h]

IMASK is shown in [Figure 19-9](#) and described in [Table 19-11](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is un-masked. Un-masking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 19-9. IMASK Register

31	30	29	28	27	26	25	24
INTR_OVFL	SARBLOST	SRX_OVFL	STX_UNFL	SPEC_RX_ER R	SDMA_DONE_ RX	SDMA_DONE_ TX	SGENCALL
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
SSTOP	SSTART	STXEMPTY	SRXFIFOFULL	STXFIFOTRG	SRXFIFOTRG	STXDONE	SRXDONE
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
TIMEOUTB	TIMEOUTA	MPEC_RX_ER R	MDMA_DONE_ RX	MDMA_DONE_ TX	MARBLOST	MSTOP	MSTART
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
MNACK	RESERVED	MTXEMPTY	MRXFIFOFULL	MTXFIFOTRG	MRXFIFOTRG	MTXDONE	MRXDONE
R/W-0h	R-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 19-11. IMASK Register Field Descriptions

Bit	Field	Type	Reset	Description
31	INTR_OVFL	R/W	0h	Interrupt Overflow Interrupt Mask 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
30	SARBLOST	R/W	0h	Target Arbitration Lost 0h = Clear Set Interrupt Mask 1h = Set Interrupt Mask
29	SRX_OVFL	R/W	0h	Target RX FIFO overflow 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
28	STX_UNFL	R/W	0h	Target TX FIFO underflow 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
27	SPEC_RX_ERR	R/W	0h	Target RX Pec Error Interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
26	SDMA_DONE_RX	R/W	0h	Target DMA Done on Event Channel RX 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
25	SDMA_DONE_TX	R/W	0h	Target DMA Done on Event Channel TX 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
24	SGENCALL	R/W	0h	General Call Interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
23	SSTOP	R/W	0h	Stop Condition Interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

Table 19-11. IMASK Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
22	SSTART	R/W	0h	Start Condition Interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
21	STXEMPTY	R/W	0h	Target Transmit FIFO Empty interrupt mask. This interrupt is set if all data in the Transmit FIFO have been shifted out and the transmit goes into idle mode. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
20	SRXFIFOFULL	R/W	0h	RXFIFO full event. This interrupt is set if an Target RX FIFO is full. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
19	STXFIFOTRG	R/W	0h	Target Transmit FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
18	SRXFIFOTRG	R/W	0h	Target Receive FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
17	STXDONE	R/W	0h	Target Transmit Transaction completed Interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
16	SRXDONE	R/W	0h	Target Receive Data Interrupt Signals that a byte has been received 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
15	TIMEOUTB	R/W	0h	Timeout B Interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
14	TIMEOUTA	R/W	0h	Timeout A Interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
13	MPEC_RX_ERR	R/W	0h	Controller RX Pec Error Interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
12	MDMA_DONE_RX	R/W	0h	DMA Done on Event Channel RX 0h = Interrupt disabled 1h = Set Interrupt Mask
11	MDMA_DONE_TX	R/W	0h	DMA Done on Event Channel TX 0h = Interrupt disabled 1h = Set Interrupt Mask
10	MARBLOST	R/W	0h	Arbitration Lost Interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
9	MSTOP	R/W	0h	STOP Detection Interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
8	MSTART	R/W	0h	START Detection Interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
7	MNACK	R/W	0h	Address/Data NACK Interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
6	RESERVED	R	0h	
5	MTXEMPTY	R/W	0h	Transmit FIFO Empty interrupt mask. This interrupt is set if all data in the Transmit FIFO have been shifted out and the transmit goes into idle mode. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

Table 19-11. IMASK Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
4	MRXFIFOFULL	R/W	0h	RXFIFO full event. This interrupt is set if an RX FIFO is full. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
3	MTXFIFOTRG	R/W	0h	Controller Transmit FIFO Trigger Trigger when Transmit FIFO contains <= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	MRXFIFOTRG	R/W	0h	Controller Receive FIFO Trigger Trigger when RX FIFO contains >= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	MTXDONE	R/W	0h	Controller Transmit Transaction completed Interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	MRXDONE	R/W	0h	Controller Receive Transaction completed Interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

19.10 RIS Register (Offset = 1030h) [Reset = 00000000h]

RIS is shown in [Figure 19-10](#) and described in [Table 19-12](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 19-10. RIS Register

31	30	29	28	27	26	25	24
INTR_OVFL	SARBLOST	SRX_OVFL	STX_UNFL	SPEC_RX_ER R	SDMA_DONE_ RX	SDMA_DONE_ TX	SGENCALL
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
SSTOP	SSTART	STXEMPTY	SRXFIFOFULL	STXFIFOTRG	SRXFIFOTRG	STXDONE	SRXDONE
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
TIMEOUTB	TIMEOUTA	MPEC_RX_ER R	MDMA_DONE_ RX	MDMA_DONE_ TX	MARBLOST	MSTOP	MSTART
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
MNACK	RESERVED	MTXEMPTY	MRXFIFOFULL	MTXFIFOTRG	MRXFIFOTRG	MTXDONE	MRXDONE
R-0h	R-0h	R-0h	R-0h	R/W-0h	R/W-0h	R-0h	R-0h

Table 19-12. RIS Register Field Descriptions

Bit	Field	Type	Reset	Description
31	INTR_OVFL	R	0h	Interrupt overflow interrupt It is set when SSTART or SSTOP interrupts overflow i.e. occur twice without being serviced 0h = Interrupt did not occur 1h = Interrupt occurred
30	SARBLOST	R	0h	Target Arbitration Lost 0h = Interrupt did not occur 1h = Interrupt occurred
29	SRX_OVFL	R	0x0	Target RX FIFO overflow 0h = Interrupt did not occur 1h = Interrupt Occured
28	STX_UNFL	R	0x0	Target TX FIFO underflow 0h = Interrupt did not occur 1h = Interrupt occurred
27	SPEC_RX_ERR	R	0x0	Target RX Pec Error Interrupt 0h = Interrupt did not occur 1h = Interrupt occurred
26	SDMA_DONE_RX	R	0x0	DMA Done on Event Channel RX 0h = Clear interrupt 1h = Set interrupt
25	SDMA_DONE_TX	R	0x0	DMA Done on Event Channel TX 0h = Clear interrupt 1h = Set interrupt
24	SGENCALL	R	0x0	General Call Interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

Table 19-12. RIS Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
23	SSTOP	R	0x0	Stop Condition Interrupt 0h = Clear Interrupt 1h = Set interrupt
22	SSTART	R	0x0	Start Condition Interrupt 0h = Clear interrupt 1h = Set Interrupt
21	STXEMPTY	R	0x0	Transmit FIFO Empty interrupt mask. This interrupt is set if all data in the Transmit FIFO have been shifted out and the transmit goes into idle mode. 0h = Interrupt did not occur 1h = Set Interrupt Mask
20	SRXFIFOFULL	R	0x0	RXFIFO full event. This interrupt is set if an RX FIFO is full. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
19	STXFIFOTRG	R	0x0	Target Transmit FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
18	SRXFIFOTRG	R	0x0	Target Receive FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
17	STXDONE	R	0x0	Target Transmit Transaction completed Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask
16	SRXDONE	R	0x0	Target Receive Data Interrupt Signals that a byte has been received 0h = Interrupt did not occur 1h = Set Interrupt Mask
15	TIMEOUTB	R	0x0	Timeout B Interrupt 0h = Interrupt did not occur 1h = Interrupt occurred
14	TIMEOUTA	R	0x0	Timeout A Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask
13	MPEC_RX_ERR	R	0x0	Controller RX Pec Error Interrupt 0h = Interrupt did not occur 1h = Interrupt Occured
12	MDMA_DONE_RX	R	0x0	DMA Done on Event Channel RX 0h = Interrupt disabled 1h = Set Interrupt Mask
11	MDMA_DONE_TX	R	0x0	DMA Done on Event Channel TX 0h = Interrupt disabled 1h = Set Interrupt Mask
10	MARBLOST	R	0x0	Arbitration Lost Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask
9	MSTOP	R	0x0	STOP Detection Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask
8	MSTART	R	0x0	START Detection Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask
7	MNACK	R	0x0	Address/Data NACK Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask
6	RESERVED	R	0h	

Table 19-12. RIS Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
5	MTXEMPTY	R	0h	Transmit FIFO Empty interrupt mask. This interrupt is set if all data in the Transmit FIFO have been shifted out and the transmit goes into idle mode. 0h = Interrupt did not occur 1h = Set Interrupt Mask
4	MRXFIFOFULL	R	0x0	RXFIFO full event. This interrupt is set if an RX FIFO is full. 0h = Interrupt did not occur 1h = Set Interrupt Mask
3	MTXFIFOTRG	R/W	0h	Controller Transmit FIFO Trigger Trigger when Transmit FIFO contains <= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	MRXFIFOTRG	R/W	0x0	Controller Receive FIFO Trigger Trigger when RX FIFO contains >= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	MTXDONE	R	0x0	Controller Transmit Transaction completed Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask
0	MRXDONE	R	0x0	Controller Receive Transaction completed Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask

19.11 MIS Register (Offset = 1038h) [Reset = 00000000h]

MIS is shown in [Figure 19-11](#) and described in [Table 19-13](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 19-11. MIS Register

31	30	29	28	27	26	25	24
INTR_OVFL	SARBLOST	SRX_OVFL	STX_UNFL	SPEC_RX_ER R	SDMA_DONE_ RX	SDMA_DONE_ TX	SGENCALL
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
SSTOP	SSTART	STXEMPTY	SRXFIFOFULL	STXFIFOTRG	SRXFIFOTRG	STXDONE	SRXDONE
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
TIMEOUTB	TIMEOUTA	MPEC_RX_ER R	MDMA_DONE_ RX	MDMA_DONE_ TX	MARBLOST	MSTOP	MSTART
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
MNACK	RESERVED	MTXEMPTY	MRXFIFOFULL	MTXFIFOTRG	MRXFIFOTRG	MTXDONE	MRXDONE
R/W-0h	R-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 19-13. MIS Register Field Descriptions

Bit	Field	Type	Reset	Description
31	INTR_OVFL	R/W	0h	Interrupt overflow 0h = Interrupt did not occur 1h = Interrupt occurred
30	SARBLOST	R/W	0h	Target Arbitration Lost 0h = Clear interrupt mask 1h = Set interrupt mask
29	SRX_OVFL	R/W	0h	Target RX FIFO overflow 0h = Clear interrupt mask 1h = Set interrupt mask
28	STX_UNFL	R/W	0h	Target TX FIFO underflow 0h = Clear interrupt mask 1h = Set interrupt mask
27	SPEC_RX_ERR	R/W	0h	Target RX Pec Error Interrupt 0h = Clear interrupt mask 1h = Set interrupt mask
26	SDMA_DONE_RX	R/W	0h	DMA Done on Event Channel RX 0h = Clear MIS 1h = Set MIS
25	SDMA_DONE_TX	R/W	0h	DMA Done on Event Channel TX 0h = Clear MIS 1h = Set MIS
24	SGENCALL	R/W	0h	General Call Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask
23	SSTOP	R/W	0h	Target STOP Detection Interrupt 0h = Clear MIS 1h = Set MIS
22	SSTART	R/W	0h	Target START Detection Interrupt 0h = Clear MIS 1h = Set MIS

Table 19-13. MIS Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	STXEMPTY	R/W	0h	Transmit FIFO Empty interrupt mask. This interrupt is set if all data in the Transmit FIFO have been shifted out and the transmit goes into idle mode. 0h = Interrupt did not occur 1h = Set Interrupt Mask
20	SRXFIFOFULL	R/W	0h	RXFIFO full event. This interrupt is set if an RX FIFO is full. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
19	STXFIFOTRG	R/W	0h	Target Transmit FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
18	SRXFIFOTRG	R/W	0h	Target Receive FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
17	STXDONE	R/W	0h	Target Transmit Transaction completed Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask
16	SRXDONE	R/W	0h	Target Receive Data Interrupt Signals that a byte has been received 0h = Interrupt did not occur 1h = Set Interrupt Mask
15	TIMEOUTB	R/W	0h	Timeout B Interrupt 0h = Clear interrupt mask 1h = Set interrupt mask
14	TIMEOUTA	R/W	0h	Timeout A Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask
13	MPEC_RX_ERR	R/W	0h	Controller RX Pec Error Interrupt 0h = Clear interrupt mask 1h = Set interrupt mask
12	MDMA_DONE_RX	R/W	0h	DMA Done on Event Channel RX 0h = Interrupt disabled 1h = Set Interrupt Mask
11	MDMA_DONE_TX	R/W	0h	DMA Done on Event Channel TX 0h = Interrupt disabled 1h = Set Interrupt Mask
10	MARBLOST	R/W	0h	Arbitration Lost Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask
9	MSTOP	R/W	0h	STOP Detection Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask
8	MSTART	R/W	0h	START Detection Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask
7	MNACK	R/W	0h	Address/Data NACK Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask
6	RESERVED	R	0h	
5	MTXEMPTY	R/W	0h	Transmit FIFO Empty interrupt mask. This interrupt is set if all data in the Transmit FIFO have been shifted out and the transmit goes into idle mode. 0h = Interrupt did not occur 1h = Set Interrupt Mask
4	MRXFIFOFULL	R/W	0h	RXFIFO full event. This interrupt is set if the RX FIFO is full. 0h = Interrupt did not occur 1h = Set Interrupt Mask

Table 19-13. MIS Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
3	MTXFIFOTRG	R/W	0h	Controller Transmit FIFO Trigger Trigger when Transmit FIFO contains <= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	MRXFIFOTRG	R/W	0h	Controller Receive FIFO Trigger Trigger when RX FIFO contains >= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	MTXDONE	R/W	0h	Controller Transmit Transaction completed Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask
0	MRXDONE	R/W	0h	Controller Receive Data Interrupt 0h = Interrupt did not occur 1h = Set Interrupt Mask

19.12 ISET Register (Offset = 1040h) [Reset = 00000000h]

ISET is shown in [Figure 19-12](#) and described in [Table 19-14](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 19-12. ISET Register

31	30	29	28	27	26	25	24
INTR_OVFL	SARBLOST	SRX_OVFL	STX_UNFL	SPEC_RX_ER R	SDMA_DONE_ RX	SDMA_DONE_ TX	SGENCALL
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
SSTOP	SSTART	STXEMPTY	SRXFIFOFULL	STXFIFOTRG	SRXFIFOTRG	STXDONE	SRXDONE
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
TIMEOUTB	TIMEOUTA	MPEC_RX_ER R	MDMA_DONE_ RX	MDMA_DONE_ TX	MARBLOST	MSTOP	MSTART
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
MNACK	RESERVED	MTXEMPTY	MRXFIFOFULL	MTXFIFOTRG	MRXFIFOTRG	MTXDONE	MRXDONE
W-0h	R-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 19-14. ISET Register Field Descriptions

Bit	Field	Type	Reset	Description
31	INTR_OVFL	W	0h	Interrupt overflow 0h = No effect 1h = Set interrupt
30	SARBLOST	W	0h	Target Arbitration Lost 0h = Writing 0 has no effect 1h = Set interrupt
29	SRX_OVFL	W	0h	Target RX FIFO overflow 0h = Writing 0 has no effect 1h = Set interrupt
28	STX_UNFL	W	0h	Target TX FIFO underflow 0h = Writing 0 has no effect 1h = Set interrupt
27	SPEC_RX_ERR	W	0h	Target RX Pec Error Interrupt 0h = Writing 0 has no effect 1h = Set interrupt
26	SDMA_DONE_RX	W	0h	DMA Done on Event Channel RX 0h = Writing 0 has no effect 1h = Set interrupt
25	SDMA_DONE_TX	W	0h	DMA Done on Event Channel TX 0h = Writing 0 has no effect 1h = Set interrupt
24	SGENCALL	W	0h	General Call Interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
23	SSTOP	W	0h	Stop Condition Interrupt 0h = Writing 0 has no effect 1h = Set interrupt

Table 19-14. ISET Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
22	SSTART	W	0h	Start Condition Interrupt 0h = Writing 0 has no effect 1h = Set interrupt
21	STXEMPTY	W	0h	Transmit FIFO Empty interrupt mask. This interrupt is set if all data in the Transmit FIFO have been shifted out and the transmit goes into idle mode. 0h = Writing 0 has no effect 1h = Set Interrupt Mask
20	SRXFIFOFULL	W	0h	RXFIFO full event. This interrupt is set if an RX FIFO is full. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
19	STXFIFOTRG	W	0h	Target Transmit FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
18	SRXFIFOTRG	W	0h	Target Receive FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
17	STXDONE	W	0h	Target Transmit Transaction completed Interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
16	SRXDONE	W	0h	Target Receive Data Interrupt Signals that a byte has been received 0h = Writing 0 has no effect 1h = Set Interrupt Mask
15	TIMEOUTB	W	0h	Timeout B Interrupt 0h = Writing 0 has no effect 1h = Set interrupt
14	TIMEOUTA	W	0h	Timeout A interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
13	MPEC_RX_ERR	W	0h	Controller RX Pec Error Interrupt 0h = Writing 0 has no effect 1h = Set interrupt
12	MDMA_DONE_RX	W	0h	DMA Done on Event Channel RX 0h = Interrupt disabled 1h = Set Interrupt Mask
11	MDMA_DONE_TX	W	0h	DMA Done on Event Channel TX 0h = Interrupt disabled 1h = Set Interrupt Mask
10	MARBLOST	W	0h	Arbitration Lost Interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
9	MSTOP	W	0h	STOP Detection Interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
8	MSTART	W	0h	START Detection Interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
7	MNACK	W	0h	Address/Data NACK Interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
6	RESERVED	R	0h	
5	MTXEMPTY	W	0h	Transmit FIFO Empty interrupt mask. This interrupt is set if all data in the Transmit FIFO have been shifted out and the transmit goes into idle mode. 0h = Writing 0 has no effect 1h = Set Interrupt Mask

Table 19-14. ISET Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
4	MRXFIFOFULL	W	0h	RXFIFO full event. 0h = Writing 0 has no effect 1h = Set Interrupt Mask
3	MTXFIFOTRG	W	0h	Controller Transmit FIFO Trigger Trigger when Transmit FIFO contains <= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	MRXFIFOTRG	W	0h	Controller Receive FIFO Trigger Trigger when RX FIFO contains >= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	MTXDONE	W	0h	Controller Transmit Transaction completed Interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
0	MRXDONE	W	0h	Controller Receive Data Interrupt Signals that a byte has been received 0h = Writing 0 has no effect 1h = Set Interrupt Mask

19.13 ICLR Register (Offset = 1048h) [Reset = 00000000h]

ICLR is shown in [Figure 19-13](#) and described in [Table 19-15](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 19-13. ICLR Register

31	30	29	28	27	26	25	24
INTR_OVFL	SARBLOST	SRX_OVFL	STX_UNFL	SPEC_RX_ER R	SDMA_DONE_ RX	SDMA_DONE_ TX	SGENCALL
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
SSTOP	SSTART	STXEMPTY	SRXFIFOFULL	STXFIFOTRG	SRXFIFOTRG	STXDONE	SRXDONE
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
15	14	13	12	11	10	9	8
TIMEOUTB	TIMEOUTA	MPEC_RX_ER R	MDMA_DONE_ RX	MDMA_DONE_ TX	MARBLOST	MSTOP	MSTART
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
MNACK	RESERVED	MTXEMPTY	MRXFIFOFULL	MTXFIFOTRG	MRXFIFOTRG	MTXDONE	MRXDONE
W-0h	R-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 19-15. ICLR Register Field Descriptions

Bit	Field	Type	Reset	Description
31	INTR_OVFL	W	0h	Interrupt overflow 0h = No effect 1h = Clear interrupt
30	SARBLOST	W	0h	Target Arbitration Lost 0h = Writing 0 has no effect 1h = Clear Interrupt
29	SRX_OVFL	W	0h	Target RX FIFO overflow 0h = Writing 0 has no effect 1h = Clear Interrupt
28	STX_UNFL	W	0h	Target TX FIFO underflow 0h = Writing 0 has no effect 1h = Clear Interrupt
27	SPEC_RX_ERR	W	0h	Target RX Pec Error Interrupt 0h = Writing 0 has no effect 1h = Clear Interrupt
26	SDMA_DONE_RX	W	0h	DMA Done on Event Channel RX 0h = Writing 0 has no effect 1h = Clear interrupt
25	SDMA_DONE_TX	W	0h	DMA Done on Event Channel TX 0h = Writing 0 has no effect 1h = Clear interrupt
24	SGENCALL	W	0h	General Call Interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
23	SSTOP	W	0h	Target STOP Detection Interrupt 0h = Writing 0 has no effect 1h = Clear interrupt
22	SSTART	W	0h	Target START Detection Interrupt 0h = Writing 0 has no effect 1h = Clear interrupt

Table 19-15. ICLR Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	STXEMPTY	W	0h	Transmit FIFO Empty interrupt mask. This interrupt is set if all data in the Transmit FIFO have been shifted out and the transmit goes into idle mode. 0h = Writing 0 has no effect 1h = Set Interrupt Mask
20	SRXFIFOFULL	W	0h	RXFIFO full event. This interrupt is set if an RX FIFO is full. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
19	STXFIFOTRG	W	0h	Target Transmit FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
18	SRXFIFOTRG	W	0h	Target Receive FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
17	STXDONE	W	0h	Target Transmit Transaction completed Interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
16	SRXDONE	W	0h	Target Receive Data Interrupt Signals that a byte has been received 0h = Writing 0 has no effect 1h = Set Interrupt Mask
15	TIMEOUTB	W	0h	Timeout B Interrupt 0h = Writing 0 has no effect 1h = Clear Interrupt
14	TIMEOUTA	W	0h	Timeout A interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
13	MPEC_RX_ERR	W	0h	Controller RX Pec Error Interrupt 0h = Writing 0 has no effect 1h = Clear Interrupt
12	MDMA_DONE_RX	W	0h	DMA Done on Event Channel RX 0h = Interrupt disabled 1h = Set Interrupt Mask
11	MDMA_DONE_TX	W	0h	DMA Done on Event Channel TX 0h = Interrupt disabled 1h = Set Interrupt Mask
10	MARBLOST	W	0h	Arbitration Lost Interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
9	MSTOP	W	0h	STOP Detection Interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
8	MSTART	W	0h	START Detection Interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
7	MNACK	W	0h	Address/Data NACK Interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
6	RESERVED	R	0h	
5	MTXEMPTY	W	0h	Transmit FIFO Empty interrupt mask. This interrupt is set if all data in the Transmit FIFO have been shifted out and the transmit goes into idle mode. 0h = Writing 0 has no effect 1h = Set Interrupt Mask
4	MRXFIFOFULL	W	0h	RXFIFO full event. 0h = Writing 0 has no effect 1h = Set Interrupt Mask

Table 19-15. ICLR Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
3	MTXFIFOTRG	W	0h	Controller Transmit FIFO Trigger Trigger when Transmit FIFO contains <= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	MRXFIFOTRG	W	0h	Controller Receive FIFO Trigger Trigger when RX FIFO contains >= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	MTXDONE	W	0h	Controller Transmit Transaction completed Interrupt 0h = Writing 0 has no effect 1h = Set Interrupt Mask
0	MRXDONE	W	0h	Controller Receive Data Interrupt Signals that a byte has been received 0h = Writing 0 has no effect 1h = Set Interrupt Mask

19.14 IIDX Register (Offset = 1050h) [Reset = 00000000h]

IIDX is shown in [Figure 19-14](#) and described in [Table 19-16](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index.

Figure 19-14. IIDX Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 19-16. IIDX Register Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	I2C Module Interrupt Vector Value. This register provides the highest priority interrupt index. A read clears the corresponding interrupt flag in RIS and MISC. 15h-1Fh = Reserved 00h = No interrupt pending 01h = Controller receive FIFO Trigger Level 02h = Controller transmit FIFO Trigger level 03h = Target receive FIFO Trigger Level 04h = Target transmit FIFO Trigger level

19.15 IMASK Register (Offset = 1058h) [Reset = 00000000h]

IMASK is shown in [Figure 19-15](#) and described in [Table 19-17](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is un-masked. Un-masking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 19-15. IMASK Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				STXFIFOTRG	SRXFIFOTRG	MTXFIFOTRG	MRXFIFOTRG
R-0h				R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 19-17. IMASK Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	STXFIFOTRG	R/W	0h	Target Transmit FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	SRXFIFOTRG	R/W	0h	Target Receive FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	MTXFIFOTRG	R/W	0h	Controller Transmit FIFO Trigger Trigger when Transmit FIFO contains <= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	MRXFIFOTRG	R/W	0h	Controller Receive FIFO Trigger Trigger when RX FIFO contains >= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

19.16 RIS Register (Offset = 1060h) [Reset = 00000000h]

RIS is shown in [Figure 19-16](#) and described in [Table 19-18](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 19-16. RIS Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				STXFIFOTRG	SRXFIFOTRG	MTXFIFOTRG	MRXFIFOTRG
R-0h				R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 19-18. RIS Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	STXFIFOTRG	R/W	0h	Target Transmit FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	SRXFIFOTRG	R/W	0h	Target Receive FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	MTXFIFOTRG	R/W	0h	Controller Transmit FIFO Trigger Trigger when Transmit FIFO contains <= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	MRXFIFOTRG	R/W	0h	Controller Receive FIFO Trigger Trigger when RX FIFO contains >= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

19.17 MIS Register (Offset = 1068h) [Reset = 00000000h]

MIS is shown in [Figure 19-17](#) and described in [Table 19-19](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 19-17. MIS Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				STXFIFOTRG	SRXFIFOTRG	MTXFIFOTRG	MRXFIFOTRG
R-0h				R-0h	R-0h	R-0h	R-0h

Table 19-19. MIS Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	STXFIFOTRG	R	0h	Target Transmit FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	SRXFIFOTRG	R	0h	Target Receive FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	MTXFIFOTRG	R	0h	Controller Transmit FIFO Trigger Trigger when Transmit FIFO contains <= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	MRXFIFOTRG	R	0h	Controller Receive FIFO Trigger Trigger when RX FIFO contains >= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

19.18 ISET Register (Offset = 1070h) [Reset = 00000000h]

ISET is shown in [Figure 19-18](#) and described in [Table 19-20](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 19-18. ISET Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				STXFIFOTRG	SRXFIFOTRG	MTXFIFOTRG	MRXFIFOTRG
R-0h				W-0h	W-0h	W-0h	W-0h

Table 19-20. ISET Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	STXFIFOTRG	W	0h	Target Transmit FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	SRXFIFOTRG	W	0h	Target Receive FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	MTXFIFOTRG	W	0h	Controller Transmit FIFO Trigger Trigger when Transmit FIFO contains <= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	MRXFIFOTRG	W	0h	Controller Receive FIFO Trigger Trigger when RX FIFO contains >= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

19.19 ICLR Register (Offset = 1078h) [Reset = 00000000h]

ICLR is shown in [Figure 19-19](#) and described in [Table 19-21](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 19-19. ICLR Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				STXFIFOTRG	SRXFIFOTRG	MTXFIFOTRG	MRXFIFOTRG
R-0h				W-0h	W-0h	W-0h	W-0h

Table 19-21. ICLR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	STXFIFOTRG	W	0h	Target Transmit FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	SRXFIFOTRG	W	0h	Target Receive FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	MTXFIFOTRG	W	0h	Controller Transmit FIFO Trigger Trigger when Transmit FIFO contains <= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	MRXFIFOTRG	W	0h	Controller Receive FIFO Trigger Trigger when RX FIFO contains >= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

19.20 IIDX Register (Offset = 1080h) [Reset = 00000000h]

IIDX is shown in [Figure 19-20](#) and described in [Table 19-22](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index.

Figure 19-20. IIDX Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 19-22. IIDX Register Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	I2C Module Interrupt Vector Value. This register provides the highest priority interrupt index. A read clears the corresponding interrupt flag in RIS and MISC. 15h-1Fh = Reserved 00h = No interrupt pending 01h = Controller receive FIFO Trigger Level 02h = Controller transmit FIFO Trigger level 03h = Target receive FIFO Trigger Level 04h = Target transmit FIFO Trigger level

19.21 IMASK Register (Offset = 1088h) [Reset = 00000000h]

IMASK is shown in [Figure 19-21](#) and described in [Table 19-23](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is un-masked. Un-masking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 19-21. IMASK Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				STXFIFOTRG	SRXFIFOTRG	MTXFIFOTRG	MRXFIFOTRG
R-0h				R-0h	R-0h	R-0h	R-0h

Table 19-23. IMASK Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	STXFIFOTRG	R	0h	Target Transmit FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	SRXFIFOTRG	R	0h	Target Receive FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	MTXFIFOTRG	R	0h	Controller Transmit FIFO Trigger Trigger when Transmit FIFO contains <= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	MRXFIFOTRG	R	0h	Controller Receive FIFO Trigger Trigger when RX FIFO contains >= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

19.22 RIS Register (Offset = 1090h) [Reset = 00000000h]

RIS is shown in [Figure 19-22](#) and described in [Table 19-24](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 19-22. RIS Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				STXFIFOTRG	SRXFIFOTRG	MTXFIFOTRG	MRXFIFOTRG
R-0h				R-0h	R-0h	R-0h	R-0h

Table 19-24. RIS Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	STXFIFOTRG	R	0h	Target Transmit FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	SRXFIFOTRG	R	0h	Target Receive FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	MTXFIFOTRG	R	0h	Controller Transmit FIFO Trigger Trigger when Transmit FIFO contains <= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	MRXFIFOTRG	R	0h	Controller Receive FIFO Trigger Trigger when RX FIFO contains >= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

19.23 MIS Register (Offset = 1098h) [Reset = 00000000h]

MIS is shown in [Figure 19-23](#) and described in [Table 19-25](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 19-23. MIS Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				STXFIFOTRG	SRXFIFOTRG	MTXFIFOTRG	MRXFIFOTRG
R-0h				R-0h	R-0h	R-0h	R-0h

Table 19-25. MIS Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	STXFIFOTRG	R	0h	Target Transmit FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	SRXFIFOTRG	R	0h	Target Receive FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	MTXFIFOTRG	R	0h	Controller Transmit FIFO Trigger Trigger when Transmit FIFO contains <= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	MRXFIFOTRG	R	0h	Controller Receive FIFO Trigger Trigger when RX FIFO contains >= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

19.24 ISET Register (Offset = 10A0h) [Reset = 00000000h]

ISET is shown in [Figure 19-24](#) and described in [Table 19-26](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 19-24. ISET Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				STXFIFOTRG	SRXFIFOTRG	MTXFIFOTRG	MRXFIFOTRG
R-0h				R-0h	R-0h	R-0h	R-0h

Table 19-26. ISET Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	STXFIFOTRG	R	0h	Target Transmit FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	SRXFIFOTRG	R	0h	Target Receive FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	MTXFIFOTRG	R	0h	Controller Transmit FIFO Trigger Trigger when Transmit FIFO contains <= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	MRXFIFOTRG	R	0h	Controller Receive FIFO Trigger Trigger when RX FIFO contains >= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

19.25 ICLR Register (Offset = 10A8h) [Reset = 00000000h]

ICLR is shown in [Figure 19-25](#) and described in [Table 19-27](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 19-25. ICLR Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				STXFIFOTRG	SRXFIFOTRG	MTXFIFOTRG	MRXFIFOTRG
R-0h				R-0h	R-0h	R-0h	R-0h

Table 19-27. ICLR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	STXFIFOTRG	R	0h	Target Transmit FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	SRXFIFOTRG	R	0h	Target Receive FIFO Trigger 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	MTXFIFOTRG	R	0h	Controller Transmit FIFO Trigger Trigger when Transmit FIFO contains <= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	MRXFIFOTRG	R	0h	Controller Receive FIFO Trigger Trigger when RX FIFO contains >= defined bytes 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

19.26 EVT_MODE Register (Offset = 10E0h) [Reset = 00000000h]

EVT_MODE is shown in [Figure 19-26](#) and described in [Table 19-28](#).

Return to the [Summary Table](#).

Event mode register. It is used to select whether each line is disabled, in software mode (software clears the RIS) or in hardware mode (hardware clears the RIS)

Figure 19-26. EVT_MODE Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED		EVT2_CFG		INT1_CFG		INT0_CFG	
R-0h		R-0h		R-0h		R-0h	

Table 19-28. EVT_MODE Register Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R	0h	
5-4	EVT2_CFG	R	0h	Event line mode select for event corresponding to none.DMA_TRIG0 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
3-2	INT1_CFG	R	0h	Event line mode select for event corresponding to none.DMA_TRIG1 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
1-0	INT0_CFG	R	0h	Event line mode select for event corresponding to none.CPU_INT 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.

19.27 INTCTL Register (Offset = 10E4h) [Reset = 00000000h]

INTCTL is shown in [Figure 19-27](#) and described in [Table 19-29](#).

Return to the [Summary Table](#).

Interrupt control register

Figure 19-27. INTCTL Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							INTEVAL
R-0h							W-0h

Table 19-29. INTCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	INTEVAL	W	0h	Writing a 1 to this field re-evaluates the interrupt sources. 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS.

19.28 DESC Register (Offset = 10FCh) [Reset = 00000000h]

DESC is shown in [Figure 19-28](#) and described in [Table 19-30](#).

Return to the [Summary Table](#).

This register identifies the peripheral and its exact version.

Figure 19-28. DESC Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODULEID															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FEATUREVER				INSTNUM				MAJREV				MINREV			
R-0h				R/W-0h				R-0h				R-0h			

Table 19-30. DESC Register Field Descriptions

Bit	Field	Type	Reset	Description
31-16	MODULEID	R/W	0h	Module identification contains a unique peripheral identification number. The assignments are maintained in a central database for all of the platform modules to ensure uniqueness. 0h = Smallest value FFFFh = Highest possible value
15-12	FEATUREVER	R	0x0	Feature Set for the module *instance* 0h = Smallest value Fh = Highest possible value
11-8	INSTNUM	R/W	0x0	Instance Number within the device. This will be a parameter to the RTL for modules that can have multiple instances 0h = Smallest value Fh = Highest possible value
7-4	MAJREV	R	0h	Major rev of the IP 0h = Smallest value Fh = Highest possible value
3-0	MINREV	R	0x0	Minor rev of the IP 0h = Smallest value Fh = Highest possible value

19.29 GFCTL Register (Offset = 1200h) [Reset = 00000000h]

GFCTL is shown in [Figure 19-29](#) and described in [Table 19-31](#).

Return to the [Summary Table](#).

This register controls the glitch filter on the SCL and SDA lines

Figure 19-29. GFCTL Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED				CHAIN	AGFSEL		AGFEN
R-0h				R/W-0h	R/W-0h		R/W-0h
7	6	5	4	3	2	1	0
RESERVED					DGFSEL		
R-0h					R/W-0h		

Table 19-31. GFCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-12	RESERVED	R	0h	
11	CHAIN	R/W	0h	Analog and digital noise filters chaining enable. 0h = When 0, chaining is disabled and only digital filter output is available to IP logic for oversampling 1h = When 1, analog and digital glitch filters are chained and the output of the combination is made available to IP logic for oversampling
10-9	AGFSEL	R/W	0h	Analog Glitch Suppression Pulse Width This field controls the pulse width select for the analog glitch suppression on SCL and SDA lines. See device datasheet for exact values. (ULP I2C only) 0h = Pulses shorter than 5ns length are filtered. 1h = Pulses shorter than 10ns length are filtered. 2h = Pulses shorter than 25ns length are filtered. 3h = Pulses shorter than 50ns length are filtered.
8	AGFEN	R/W	0h	Analog Glitch Suppression Enable 0h = Analog Glitch Filter disable 1h = Analog Glitch Filter enable
7-3	RESERVED	R	0h	
2-0	DGFSEL	R/W	0x0	Glitch Suppression Pulse Width This field controls the pulse width select for glitch suppression on the SCL and SDA lines. The following values are the glitch suppression values in terms of functional clocks. (Core Domain only) 0h = Bypass 1h = 1 clock 2h = 2 clocks 3h = 3 clocks 4h = 4 clocks 5h = 8 clocks 6h = 16 clocks 7h = 31 clocks

19.30 TIMEOUT_CTL Register (Offset = 1204h) [Reset = 00020002h]

TIMEOUT_CTL is shown in [Figure 19-30](#) and described in [Table 19-32](#).

Return to the [Summary Table](#).

This register contains controls for Timeout Counters A and B

Figure 19-30. TIMEOUT_CTL Register

31	30	29	28	27	26	25	24
TCNTBEN	RESERVED						
R/W-0h	R-0h						
23	22	21	20	19	18	17	16
TCNTLB							
R/W-2h							
15	14	13	12	11	10	9	8
TCNTAEN	RESERVED						
R/W-0h	R-0h						
7	6	5	4	3	2	1	0
TCNTLA							
R/W-2h							

Table 19-32. TIMEOUT_CTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31	TCNTBEN	R/W	0h	Timeout Counter B Enable 0h = Disable Timeout Counter B 1h = Enable Timeout Counter B
30-24	RESERVED	R	0h	
23-16	TCNTLB	R/W	2h	Timeout Count B Load: Counter B is used for SCL High Detection. This field contains the upper 8 bits of a 12-bit pre-load value for the Timeout B count. NOTE: The value of CNTLB must be greater than 1h. Each count is equal to 1* clock period. For example, with 10MHz functional clock one timeout period will be equal to 1*100ns. 0h = Smallest possible value FFh = Highest possible value
15	TCNTAEN	R/W	0h	Timeout Counter A Enable 0h = Disable Timeout Counter B 1h = Enable Timeout Counter B
14-8	RESERVED	R	0h	
7-0	TCNTLA	R/W	2h	Timeout counter A load value Counter A is used for SCL low detection. This field contains the upper 8 bits of a 12-bit pre-load value for the Timeout A count. NOTE: The value of CNTLA must be greater than 1h. Each count is equal to 520 times the timeout period of functional clock. For example, with 8MHz functional clock and a 100KHz operating I2C clock, one timeout period will be equal to (1 / 8MHz) * 520 or 65 us. 0h = Smallest Value FFh = Highest possible value

19.31 TIMEOUT_CNT Register (Offset = 1208h) [Reset = 00020002h]

TIMEOUT_CNT is shown in [Figure 19-31](#) and described in [Table 19-33](#).

Return to the [Summary Table](#).

This register contains the upper 8 bits of a 12-bit current counter values for counter A and B. The lower four bits of the counter are not user visible and are always 0h.

Figure 19-31. TIMEOUT_CNT Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED								TCNTB								RESERVED								TCNTA							
R-0h								R-2h								R-0h								R-2h							

Table 19-33. TIMEOUT_CNT Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	RESERVED	R	0h	
23-16	TCNTB	R	2h	Timeout Count B Current Count: This field contains the upper 8 bits of a 12-bit current counter for timeout counter B 0h = Smallest Value FFh = Highest possible value
15-8	RESERVED	R	0h	
7-0	TCNTA	R	2h	Timeout Count A Current Count: This field contains the upper 8 bits of a 12-bit current counter for timeout counter A 0h = Smallest Value FFh = Highest possible value

19.32 MSA Register (Offset = 1210h) [Reset = 00000000h]

MSA is shown in [Figure 19-32](#) and described in [Table 19-34](#).

Return to the [Summary Table](#).

I2C Controller Target Address Register

Figure 19-32. MSA Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
MMODE	RESERVED				SADDR		
R/W-0h	R-0h				R/W-0h		
7	6	5	4	3	2	1	0
SADDR						DIR	
R/W-0h						R-0h	

Table 19-34. MSA Register Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15	MMODE	R/W	0h	This bit selects the addressing mode to be used in Controller mode When 0, 7-bit addressing is used. When 1, 10-bit addressing is used. 0h = 7-bit addressing mode 1h = 10-bit addressing mode
14-11	RESERVED	R	0h	
10-1	SADDR	R/W	0h	I2C Target Address This field specifies bits A9 through A0 of the Target address. In 7-bit addressing mode as selected by MSA.MODE bit, the top 3 bits are don't care 0h = Smallest value 3FFh = Highest possible value
0	DIR	R	0h	Receive/Send The DIR bit specifies if the next Controller operation is a Receive (High) or Transmit (Low). 0h = Transmit 1h = Receive 0h = The Controller is in transmit mode. 1h = The Controller is in receive mode.

19.33 MCTR Register (Offset = 1214h) [Reset = 00000000h]

MCTR is shown in [Figure 19-33](#) and described in [Table 19-35](#).

Return to the [Summary Table](#).

This control register configures the I2C controller operation. The START bit generates the START or REPEATED START condition. The STOP bit determines if the cycle stops at the end of the data cycle or continues to the next transfer cycle, which could be a repeated START. To generate a single transmit cycle, the I2C Controller Target Address (MSA) register is written with the desired address, the RS bit is cleared, and this register is written with ACK = X (0 or 1), STOP = 1, START = 1, and RUN = 1 to perform the operation and stop. When the operation is completed (or aborted due an error), an byte transaction completed interrupt becomes active and the data may be read from the MRXDATA register. When the I2C module operates in Controller receiver mode, a set ACK bit causes the I2C bus controller to transmit an acknowledge automatically after each byte. This bit must be cleared when the I2C bus controller requires no further data to be transmitted from the Target transmitter.

Figure 19-33. MCTR Register

31	30	29	28	27	26	25	24
RESERVED				MBLEN			
R-0h				R/W-0h			
23	22	21	20	19	18	17	16
MBLEN							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED		RD_ON_TXEM PTY	MACKOEN	ACK	STOP	START	BURSTRUN
R-0h		R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 19-35. MCTR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-28	RESERVED	R	0h	
27-16	MBLEN	R/W	0h	I2C transaction length This field contains the programmed length of bytes of the Transaction. 0h = Smallest value FFFh = Highest possible value
15-6	RESERVED	R	0h	
5	RD_ON_TXEMPTY	R/W	0h	Read on TX Empty 0h = No special behavior 1h = When 1 the Controller will transmit all bytes from the TX FIFO before continuing with the programmed Burst Run Read. If the DIR is not set to Read in the MSA then this bit is ignored. The Start must be set in the MCTR for proper I2C protocol. The Controller will first send the Start Condition, I2C Address with R/W bit set to write, before sending the bytes in the TX FIFO. When the TX FIFO is empty, the I2C transaction will continue as programmed in MTCR and MSA without sending a Stop Condition. This is intended to be used to perform simple I2C command based reads transition that will complete after initiating them without having to get an interrupt to turn the bus around.

Table 19-35. MCTR Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
4	MACKOEN	R/W	0h	Controller ACK override Enable 0h = No special behavior 1h = When 1 and the Controller is receiving data and the number of bytes indicated in MBLEN have been received, the state machine will generate an rxdone interrupt and wait at the start of the ACK for FW to indicate if an ACK or NACK should be sent. The ACK or NACK is selected by writing the MCTR register and setting ACK accordingly. The other fields in this register can also be written at this time to continue on with the transaction. If a NACK is sent the state machine will automatically send a Stop.
3	ACK	R/W	0h	Data Acknowledge Enable. Software needs to configure this bit to send the ACK or NACK. 0h = The last received data byte of a transaction is not acknowledged automatically by the Controller. 1h = The last received data byte of a transaction is acknowledged automatically by the Controller.
2	STOP	R/W	0h	Generate STOP 0h = The controller does not generate the STOP condition. 1h = The controller generates the STOP condition.
1	START	R/W	0h	Generate START 0h = The controller does not generate the START condition. 1h = The controller generates the START or repeated START condition.
0	BURSTRUN	R/W	0h	I2C Controller Enable and start transaction 0h = In standard mode, this encoding means the Controller is unable to transmit or receive data. 1h = The Controller is able to transmit or receive data.

19.34 MSR Register (Offset = 1218h) [Reset = 00000000h]

MSR is shown in [Figure 19-34](#) and described in [Table 19-36](#).

Return to the [Summary Table](#).

The status register indicates the state of the I2C bus controller.

Figure 19-34. MSR Register

31	30	29	28	27	26	25	24
RESERVED				MBCNT			
R-0h				R-0h			
23	22	21	20	19	18	17	16
MBCNT							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED	BUSBSY	IDLE	ARBLST	DATAACK	ADRACK	ERR	BUSY
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 19-36. MSR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-28	RESERVED	R	0h	
27-16	MBCNT	R	0x0	I2C Controller Transaction Count This field contains the current count-down value of the transaction. 0h = Smallest value FFFh = Highest possible value
15-7	RESERVED	R	0h	
6	BUSBSY	R	0x0	I2C Bus is Busy Controller State Machine will wait until this bit is cleared before starting a transaction. When first enabling the Controller in multi Controller environments, FW should wait for one I2C clock period after setting ACTIVE high before writing to the MTCR register to start the transaction so that if SCL goes low it will trigger the BUSBSY. 0h = The I2C bus is idle. 1h = 'This Status bit is set on a START or when SCL goes low. It is cleared on a STOP, or when a SCL high bus busy timeout occurs and SCL and SDA are both high. This status is cleared when the ACTIVE bit is low. Note that the Controller State Machine will wait until this bit is cleared before starting an I2C transaction. When first enabling the Controller in multi Controller environments, FW should wait for one I2C clock period after setting ACTIVE high before writing to the MTCR register to start the transaction so that if SCL goes low it will trigger the BUSBSY.
5	IDLE	R	0h	I2C Idle 0h = The I2C controller is not idle. 1h = The I2C controller is idle.
4	ARBLST	R	0x0	Arbitration Lost 0h = The I2C controller won arbitration. 1h = The I2C controller lost arbitration.
3	DATAACK	R	0x0	Acknowledge Data 0h = The transmitted data was acknowledged 1h = The transmitted data was not acknowledged.

Table 19-36. MSR Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	ADRACK	R	0x0	Acknowledge Address 0h = The transmitted address was acknowledged 1h = The transmitted address was not acknowledged.
1	ERR	R	0x0	Error The error can be from the Target address not being acknowledged or the transmit data not being acknowledged. 0h = No error was detected on the last operation. 1h = An error occurred on the last operation.
0	BUSY	R	0x0	I2C Controller FSM Busy The BUSY bit is set during an ongoing transaction, so is set during the transmit/receive of the amount of data set in MBLN including START, RESTART, Address and STOP signal generation when required for the current transaction. 0h = The controller is idle. 1h = The controller is busy.

19.35 MRXDATA Register (Offset = 121Ch) [Reset = 00000000h]

MRXDATA is shown in [Figure 19-35](#) and described in [Table 19-37](#).

Return to the [Summary Table](#).

I2C Controller RX FIFO Read Data Byte

This field contains the current byte being read in the RX FIFO stack.

If the FIFO is disabled, the data byte and status are stored in the receiving holding register (the bottom word of the receive FIFO). The received data can be retrieved by reading this register.

Figure 19-35. MRXDATA Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								VALUE							
R-0h																								R-0h							

Table 19-37. MRXDATA Register Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	VALUE	R	0h	Received Data. This field contains the last received data. 0h = Smallest value FFh = Highest possible value

19.36 MTXDATA Register (Offset = 1220h) [Reset = 00000000h]

MTXDATA is shown in [Figure 19-36](#) and described in [Table 19-38](#).

Return to the [Summary Table](#).

I2C Controller Transmit Data Register.

This register is the transmit data register (the interface to the FIFOs). For transmitted data, if the FIFO is enabled, data written to this location is pushed onto the transmit FIFO. If the FIFO is disabled, data is stored in the transmitter holding register (the bottom word of the transmit FIFO).

Figure 19-36. MTXDATA Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								VALUE							
R-0h																								R/W-0h							

Table 19-38. MTXDATA Register Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	VALUE	R/W	0h	Transmit Data This byte contains the data to be transferred during the next transaction. 0h = Smallest value FFh = Highest possible value

19.37 MTPR Register (Offset = 1224h) [Reset = 00000001h]

MTPR is shown in [Figure 19-37](#) and described in [Table 19-39](#).

Return to the [Summary Table](#).

This register is programmed to set the timer period for the SCL clock and assign the SCL clock to standard mode.

Figure 19-37. MTPR Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								TPR							
R-0h																								R/W-1h							

Table 19-39. MTPR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-7	RESERVED	R	0h	
6-0	TPR	R/W	1h	Timer Period This field is used in the equation to configure SCL_PERIOD : $\text{SCL_PERIOD} = (1 + \text{TPR}) \times (\text{SCL_LP} + \text{SCL_HP}) \times \text{INT_CLK_PRD}$ where: SCL_PRD is the SCL line period (I2C clock). TPR is the Timer Period register value (range of 1 to 127). SCL_LP is the SCL Low period (fixed at 6). SCL_HP is the SCL High period (fixed at 4). CLK_PRD is the functional clock period in ns. 0h = Smallest value 7Fh = Highest possible value

19.38 MCR Register (Offset = 1228h) [Reset = 00000000h]

MCR is shown in [Figure 19-38](#) and described in [Table 19-40](#).

Return to the [Summary Table](#).

Controller configuration register

Figure 19-38. MCR Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							LPBK
R-0h							R/W-0h
7	6	5	4	3	2	1	0
RESERVED					CLKSTRETCH	MMST	ACTIVE
R-0h					R/W-0h	R/W-0h	R/W-0h

Table 19-40. MCR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-9	RESERVED	R	0h	
8	LPBK	R/W	0h	I2C Loopback 0h = Normal operation. 1h = The controller in a test mode loopback configuration.
7-3	RESERVED	R	0h	
2	CLKSTRETCH	R/W	0h	Clock Stretching. This bit controls the support for clock stretching of the I2C bus. 0h = Disables the clock stretching detection. This can be disabled if no Target on the bus does support clock stretching, so that the maximum speed on the bus can be reached. 1h = Enables the clock stretching detection. Enabling the clock stretching ensures compliance to the I2C standard but could limit the speed due the clock stretching.
1	MMST	R/W	0h	MultiController mode. In MultiController mode the SCL high time counts once the SCL line has been detected high. If this is not enabled the high time counts as soon as the SCL line has been set high by the I2C controller. 0h = Disable MultiController mode. 1h = Enable MultiController mode.
0	ACTIVE	R/W	0h	Device Active After this bit has been set, it should not be set again unless it has been cleared by writing a 0 or by a reset, otherwise transfer failures may occur. 0h = Disables the I2C Controller operation. 1h = Enables the I2C Controller operation.

19.39 MBMON Register (Offset = 1234h) [Reset = 00000003h]

MBMON is shown in [Figure 19-39](#) and described in [Table 19-41](#).

Return to the [Summary Table](#).

This register is used to determine the SCL and SDA signal status.

Figure 19-39. MBMON Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED														SDA	SCL
R-0h														R-1h	R-1h

Table 19-41. MBMON Register Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1	SDA	R	1h	I2C SDA Status 0h = The I2CSDA signal is low. 1h = The I2CSDA signal is high. Note: During and right after reset, the SDA pin is in GPIO input mode without the internal pull enabled. For proper I2C operation, the user should have the external pull-up resistor in place before starting any I2C operations.
0	SCL	R	1h	I2C SCL Status 0h = The I2CSCL signal is low. 1h = The I2CSCL signal is high. Note: During and right after reset, the SCL pin is in GPIO input mode without the internal pull enabled. For proper I2C operation, the user should have the external pull-up resistor in place before starting any I2C operations.

19.40 MFIFOCTL Register (Offset = 1238h) [Reset = 00000000h]

MFIFOCTL is shown in [Figure 19-40](#) and described in [Table 19-42](#).

Return to the [Summary Table](#).

I2C Controller FIFO Control

Figure 19-40. MFIFOCTL Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RXFLUSH	RESERVED				RXTRIG		
R/W-0h	R-0h				R/W-0h		
7	6	5	4	3	2	1	0
TXFLUSH	RESERVED				TXTRIG		
R/W-0h	R-0h				R/W-0h		

Table 19-42. MFIFOCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15	RXFLUSH	R/W	0h	RX FIFO Flush Setting this bit will Flush the RX FIFO. Before clearing this bit to stop Flush the RXFIFOCNT should be checked to be 0 and indicating that the Flush has completed. 0h = Do not Flush FIFO 1h = Flush FIFO
14-11	RESERVED	R	0h	
10-8	RXTRIG	R/W	0h	RX FIFO Trigger Indicates at what fill level in the RX FIFO a trigger will be generated. Note: Programming RXTRIG to 0x0 has no effect since no data is present to transfer out of RX FIFO. 0h = Trigger when RX FIFO contains >= 1 byte 1h = Trigger when RX FIFO contains >= 2 byte 2h = Trigger when RX FIFO contains >= 3 byte 3h = Trigger when RX FIFO contains >= 4 byte 4h = Trigger when RX FIFO contains >= 5 byte 5h = Trigger when RX FIFO contains >= 6 byte 6h = Trigger when RX FIFO contains >= 7 byte 7h = Trigger when RX FIFO contains >= 8 byte
7	TXFLUSH	R/W	0h	TX FIFO Flush Setting this bit will Flush the TX FIFO. Before clearing this bit to stop Flush the TXFIFOCNT should be checked to be 8 and indicating that the Flush has completed. 0h = Do not Flush FIFO 1h = Flush FIFO
6-3	RESERVED	R	0h	

Table 19-42. MFIFOCTL Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2-0	TXTRIG	R/W	0h	TX FIFO Trigger Indicates at what fill level in the TX FIFO a trigger will be generated. 0h = Trigger when the TX FIFO is empty. 1h = Trigger when TX FIFO contains ≤ 1 byte 2h = Trigger when TX FIFO contains ≤ 2 byte 3h = Trigger when TX FIFO contains ≤ 3 byte 4h = Trigger when TX FIFO contains ≤ 4 byte 5h = Trigger when TX FIFO contains ≤ 5 byte 6h = Trigger when TX FIFO contains ≤ 6 byte 7h = Trigger when TX FIFO contains ≤ 7 byte

19.41 MFIFOSR Register (Offset = 123Ch) [Reset = 00000800h]

MFIFOSR is shown in [Figure 19-41](#) and described in [Table 19-43](#).

Return to the [Summary Table](#).

I2C Controller FIFO Status Register

Note: this Register should only be read when BUSY is 0

Figure 19-41. MFIFOSR Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
TXFLUSH	RESERVED			TXFIFOCNT			
R/W-0h	R-0h			R-8h			
7	6	5	4	3	2	1	0
RXFLUSH	RESERVED			RXFIFOCNT			
R/W-0h	R-0h			R-0h			

Table 19-43. MFIFOSR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15	TXFLUSH	R/W	0h	TX FIFO Flush When this bit is set a Flush operation for the TX FIFO is active. Clear the TXFLUSH bit in the control register to stop. 0h = FIFO Flush not active 1h = FIFO Flush active
14-12	RESERVED	R	0h	
11-8	TXFIFOCNT	R	8h	Number of Bytes which could be put into the TX FIFO 0h = Smallest value 8h = Highest possible value
7	RXFLUSH	R/W	0h	RX FIFO Flush When this bit is set a Flush operation for the RX FIFO is active. Clear the RXFLUSH bit in the control register to stop. 0h = FIFO Flush not active 1h = FIFO Flush active
6-4	RESERVED	R	0h	
3-0	RXFIFOCNT	R	0h	Number of Bytes which could be read from the RX FIFO 0h = Smallest value 8h = Highest possible value

19.42 SOAR Register (Offset = 1250h) [Reset = 00004000h]

SOAR is shown in [Figure 19-42](#) and described in [Table 19-44](#).

Return to the [Summary Table](#).

This register consists of seven address bits that identify the I2C device on the I2C bus.

Figure 19-42. SOAR Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
SMODE	OAREN	RESERVED				OAR	
R/W-0h	R/W-1h	R-0h				R/W-0h	
7	6	5	4	3	2	1	0
OAR							
R/W-0h							

Table 19-44. SOAR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15	SMODE	R/W	0h	This bit selects the addressing mode to be used in Target mode. When 0, 7-bit addressing is used. When 1, 10-bit addressing is used. 0h = Enable 7-bit addressing 1h = Enable 10-bit addressing
14	OAREN	R/W	1h	I2C Target Own Address Enable 0h = Disable OAR address 1h = Enable OAR address
13-10	RESERVED	R	0h	
9-0	OAR	R/W	0h	I2C Target Own Address: This field specifies bits A9 through A0 of the Target address. In 7-bit addressing mode as selected by I2CSOAR.MODE bit, the top 3 bits are don't care 0h = Smallest value 3FFh = Highest possible value

19.43 SOAR2 Register (Offset = 1254h) [Reset = 00000000h]

SOAR2 is shown in [Figure 19-43](#) and described in [Table 19-45](#).

Return to the [Summary Table](#).

This register consists of seven address bits that identify the alternate address for the I2C device on the I2C bus.

Figure 19-43. SOAR2 Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED	OAR2_MASK						
R-0h				R/W-0h			
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
OAR2EN	OAR2						
R/W-0h				R/W-0h			

Table 19-45. SOAR2 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-23	RESERVED	R	0h	
22-16	OAR2_MASK	R/W	0h	I2C Target Own Address 2 Mask: This field specifies bits A6 through A0 of the Target address. The bits with value '1' in SOAR2.OAR2_MASK field will make the corresponding incoming address bits to match by default regardless of the value inside SOAR2.OAR2 i.e. corresponding SOAR2.OAR2 bit is a don't care. 0h = Minimum Value 7Fh = Maximum Value
15-8	RESERVED	R	0h	
7	OAR2EN	R/W	0h	I2C Target Own Address 2 Enable 0h = The alternate address is disabled. 1h = Enables the use of the alternate address in the OAR2 field.
6-0	OAR2	R/W	0h	I2C Target Own Address 2 This field specifies the alternate OAR2 address. 0h = Smallest value 7Fh = Highest possible value

19.44 SCTR Register (Offset = 1258h) [Reset = 00000404h]

SCTR is shown in [Figure 19-44](#) and described in [Table 19-46](#).

Return to the [Summary Table](#).

I2C Target Control Register

Figure 19-44. SCTR Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED					SWUEN	EN_DEFDEVADR	EN_ALRESPADR
R-0h					R/W-1h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
EN_DEFHOSTADR	RXFULL_ON_REQ	TXWAIT_STALE_TXFIFO	TXTRIG_TXMODE	TXEMPTY_ON_TREQ	SCLKSTRETCH	GENCALL	ACTIVE
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R-0h	R/W-1h	R/W-0h	R/W-0h

Table 19-46. SCTR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-11	RESERVED	R	0h	
10	SWUEN	R/W	1h	Target Wakeup Enable 0h = When 0, the Target is not allowed to clock stretch on START detection 1h = When 1, the Target is allowed to clock stretch on START detection and wait for faster clock to be available. This allows clean wake up support for I2C in low power mode use cases
9	EN_DEFDEVADR	R/W	0h	Enable Default device address 0h = When this bit is 0, the default device address is not matched. NOTE: it may still be matched if programmed inside SOAR/SOAR2. 1h = When this bit is 1, default device address of 7'h110_0001 is always matched by the Target address match logic.
8	EN_ALRESPADR	R/W	0h	Enable Alert Response Address 0h = When this bit is 0, the alert response address is not matched. NOTE: it may still be matched if programmed inside SOAR/SOAR2 1h = When this bit is 1, alert response address of 7'h000_1100 is always matched by the Target address match logic.
7	EN_DEFHOSTADR	R/W	0h	Enable Default Host Address 0h = When this bit is 0, the default host address is not matched NOTE: it may still be matched if programmed inside SOAR/SOAR2 1h = When this bit is 1, default host address of 7'h000_1000 is always matched by the Target address match logic.

Table 19-46. SCTR Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
6	RXFULL_ON_RREQ	R/W	0h	Rx full interrupt generated on RREQ condition as indicated in SSR 0h = When 0, RIS:SRXFULL will be set when only the Target RX FIFO is full. This allows the SRXFULL interrupt to be used to indicate that the I2C bus is being clock stretched and that the FW must either read the RX FIFO or ACK/NACK the current Rx byte. 1h = When 1, RIS:SRXFULL will be set when the Target State Machine is in the RX_WAIT or RX_ACK_WAIT states which occurs when the I2C transaction is clock stretched because the RX FIFO is full or the ACKOEN has been set and the state machine is waiting for FW to ACK/NACK the current byte.
5	TXWAIT_STALE_TXFIFO	R/W	0h	Tx transfer waits when stale data in Tx FIFO. This prevents stale bytes left in the TX FIFO from automatically being sent on the next I2C packet. Note: this should be used with TXEMPTY_ON_TREQ set to prevent the Target State Machine from waiting for TX FIFO data without an interrupt notification when the FIFO data is stale. 0h = When 0, the TX FIFO empty signal to the Target State Machine indicates that the TX FIFO is empty. 1h = When 1, the TX FIFO empty signal to the Target State Machine will indicate that the TX FIFO is empty or that the TX FIFO data is stale. The TX FIFO data is determined to be stale when there is data in the TX FIFO when the Target State Machine leaves the TXMODE as defined in the SSR register. This can occur is a Stop or timeout occur when there are bytes left in the TX FIFO.
4	TXTRIG_TXMODE	R/W	0h	Tx Trigger when Target FSM is in Tx Mode 0h = No special behavior 1h = When 1, RIS:TXFIFOTRG will be set when the Target TX FIFO has reached the trigger level AND the Target State Machine is in the TXMODE as defined in the SSR register. When cleared RIS:TXFIFOTRG will be set when the Target TX FIFO is at or above the trigger level. This setting can be used to hold off the TX DMA until a transaction starts. This allows the DMA to be configured when the I2C is idle but have it wait till the transaction starts to load the Target TX FIFO, so it can load from a memory buffer that might be changing over time.
3	TXEMPTY_ON_TREQ	R	0h	Tx Empty Interrupt on TREQ 0h = When 0, RIS:STXEMPTY will be set when only the Target TX FIFO is empty. This allows the STXEMPTY interrupt to be used to indicate that the I2C bus is being clock stretched and that Target TX data is required. 1h = When 1, RIS:STXEMPTY will be set when the Target State Machine is in the TX_WAIT state which occurs when the TX FIFO is empty AND the I2C transaction is clock stretched waiting for the FIFO to receive data.
2	SCLKSTRETCH	R/W	1h	Target Clock Stretch Enable 0h = Target clock stretching is disabled 1h = Target clock stretching is enabled
1	GENCALL	R/W	0h	General call response enable Modify only when UCSWRST = 1. 0b = Do not respond to a general call 1b = Respond to a general call 0h = Do not respond to a general call 1h = Respond to a general call
0	ACTIVE	R/W	0h	Device Active. Setting this bit enables the Target functionality. 0h = Disables the I2C Target operation. 1h = Enables the I2C Target operation.

19.45 SSR Register (Offset = 125Ch) [Reset = 00000000h]

SSR is shown in [Figure 19-45](#) and described in [Table 19-47](#).

Return to the [Summary Table](#).

This register functions as a control register when written, and a status register when read.

Figure 19-45. SSR Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED					ADDRMATCH		
R-0h				R-0h			
15	14	13	12	11	10	9	8
ADDRMATCH							STALE_TXFIFO
R-0h							R-0h
7	6	5	4	3	2	1	0
TXMODE	BUSBSY	QCMDRW	QCMDST	OAR2SEL	RXMODE	TREQ	RREQ
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 19-47. SSR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-19	RESERVED	R	0h	
18-9	ADDRMATCH	R	0h	Indicates the address for which Target address match happened 0h = Minimum Value 3FFh = Maximum Value
8	STALE_TXFIFO	R	0h	Stale Tx FIFO 0h = Tx FIFO is not stale 1h = The TX FIFO is stale. This occurs when the TX FIFO was not emptied during the previous I2C transaction.
7	TXMODE	R	0h	Target FSM is in TX MODE 0h = The Target State Machine is not in TX_DATA, TX_WAIT, TX_ACK or ADDR_ACK state with the bus direction set to read. 1h = The Target State Machine is in TX_DATA, TX_WAIT, TX_ACK or ADDR_ACK state with the bus direction set to read.
6	BUSBSY	R	0h	I2C bus is busy 0h = The I2C Bus is not busy 1h = The I2C Bus is busy. This is cleared on a timeout.
5	QCMDRW	R	0h	Quick Command Read / Write This bit only has meaning when the QCMDST bit is set. Value Description: 0: Quick command was a write 1: Quick command was a read 0h = Quick command was a write 1h = Quick command was a read
4	QCMDST	R	0h	Quick Command Status Value Description: 0: The last transaction was a normal transaction or a transaction has not occurred. 1: The last transaction was a Quick Command transaction 0h = The last transaction was a normal transaction or a transaction has not occurred. 1h = The last transaction was a Quick Command transaction.

Table 19-47. SSR Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
3	OAR2SEL	R	0h	OAR2 Address Matched This bit gets reevaluated after every address comparison. 0h = Either the OAR2 address is not matched or the match is in legacy mode. 1h = OAR2 address matched and ACKed by the Target.
2	RXMODE	R	0h	Target FSM is in Rx MODE 0h = The Target State Machine is not in the RX_DATA, RX_ACK, RX_WAIT, RX_ACK_WAIT or ADDR_ACK state with the bus direction set to write. 1h = The Target State Machine is in the RX_DATA, RX_ACK, RX_WAIT, RX_ACK_WAIT or ADDR_ACK state with the bus direction set to write.
1	TREQ	R	0h	Transmit Request 0h = No outstanding transmit request. 1h = The I2C controller has been addressed as a Target transmitter and is using clock stretching to delay the Controller until data has been written to the STXDATA FIFO (Target TX FIFO is empty).
0	RREQ	R	0h	Receive Request 0h = No outstanding receive data. 1h = The I2C controller has outstanding receive data from the I2C Controller and is using clock stretching to delay the Controller until the data has been read from the SRXDATA FIFO (Target RX FIFO is full).

19.46 SRXDATA Register (Offset = 1260h) [Reset = 00000000h]

SRXDATA is shown in [Figure 19-46](#) and described in [Table 19-48](#).

Return to the [Summary Table](#).

I2C Target RX FIFO Read Data Byte

This field contains the current byte being read in the RX FIFO stack.

If the FIFO is disabled, the data byte and status are stored in the receiving holding register (the bottom word of the receive FIFO). The received data can be retrieved by reading this register.

Figure 19-46. SRXDATA Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								VALUE							
R-0h																								R-0h							

Table 19-48. SRXDATA Register Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	VALUE	R	0h	Received Data. This field contains the last received data. 0h = Smallest value FFh = Highest possible value

19.47 STXDATA Register (Offset = 1264h) [Reset = 00000000h]

STXDATA is shown in [Figure 19-47](#) and described in [Table 19-49](#).

Return to the [Summary Table](#).

I2C Target Transmit Data Register.

This register is the transmit data register (the interface to the FIFOs). For transmitted data, if the FIFO is enabled, data written to this location is pushed onto the transmit FIFO. If the FIFO is disabled, data is stored in the transmitter holding register (the bottom word of the transmit FIFO).

Figure 19-47. STXDATA Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																VALUE															
R-0h																R/W-0h															

Table 19-49. STXDATA Register Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	VALUE	R/W	0h	Transmit Data This byte contains the data to be transferred during the next transaction. 0h = Smallest value FFh = Highest possible value

19.48 SFIFOCTL Register (Offset = 126Ch) [Reset = 00000000h]

SFIFOCTL is shown in [Figure 19-48](#) and described in [Table 19-50](#).

Return to the [Summary Table](#).

I2C Target FIFO Control

Figure 19-48. SFIFOCTL Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RXFLUSH	RESERVED				RXTRIG		
R/W-0h	R-0h				R/W-0h		
7	6	5	4	3	2	1	0
TXFLUSH	RESERVED				TXTRIG		
R/W-0h	R-0h				R/W-0h		

Table 19-50. SFIFOCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15	RXFLUSH	R/W	0h	RX FIFO Flush Setting this bit will Flush the RX FIFO. Before clearing this bit to stop Flush the RXFIFOCNT should be checked to be 0 and indicating that the Flush has completed. 0h = Do not Flush FIFO 1h = Flush FIFO
14-11	RESERVED	R	0h	
10-8	RXTRIG	R/W	0h	RX FIFO Trigger Indicates at what fill level in the RX FIFO a trigger will be generated. Note: Programming RXTRIG to 0x0 has no effect since no data is present to transfer out of RX FIFO. 4h = Trigger when RX FIFO contains >= 5 byte 5h = Trigger when RX FIFO contains >= 6 byte 6h = Trigger when RX FIFO contains >= 7 byte 7h = Trigger when RX FIFO contains >= 8 byte
7	TXFLUSH	R/W	0h	TX FIFO Flush Setting this bit will Flush the TX FIFO. Before clearing this bit to stop Flush the TXFIFOCNT should be checked to be 8 and indicating that the Flush has completed. 0h = Do not Flush FIFO 1h = Flush FIFO
6-3	RESERVED	R	0h	
2-0	TXTRIG	R/W	0h	TX FIFO Trigger Indicates at what fill level in the TX FIFO a trigger will be generated. 4h = Trigger when TX FIFO contains ≤ 4 byte 5h = Trigger when TX FIFO contains ≤ 5 byte 6h = Trigger when TX FIFO contains ≤ 6 byte 7h = Trigger when TX FIFO contains ≤ 7 byte

19.49 SFIFOSR Register (Offset = 1270h) [Reset = 00000800h]

SFIFOSR is shown in [Figure 19-49](#) and described in [Table 19-51](#).

Return to the [Summary Table](#).

I2C Target FIFO Status Register

Note: this Register should only be read when BUSY is 0

Figure 19-49. SFIFOSR Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
TXFLUSH	RESERVED			TXFIFOCNT			
R/W-0h	R-0h			R-8h			
7	6	5	4	3	2	1	0
RXFLUSH	RESERVED			RXFIFOCNT			
R/W-0h	R-0h			R-0h			

Table 19-51. SFIFOSR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15	TXFLUSH	R/W	0h	TX FIFO Flush When this bit is set a Flush operation for the TX FIFO is active. Clear the TXFLUSH bit in the control register to stop. 0h = FIFO Flush not active 1h = FIFO Flush active
14-12	RESERVED	R	0h	
11-8	TXFIFOCNT	R	8h	Number of Bytes which could be put into the TX FIFO 0h = Smallest value 8h = Highest possible value
7	RXFLUSH	R/W	0h	RX FIFO Flush When this bit is set a Flush operation for the RX FIFO is active. Clear the RXFLUSH bit in the control register to stop. 0h = FIFOFlush not active 1h = FIFO Flush active
6-4	RESERVED	R	0h	
3-0	RXFIFOCNT	R	0h	Number of Bytes which could be read from the RX FIFO 0h = Smallest value 8h = Highest possible value



The Modular Controller Area Network (MCAN) peripheral supports both communication through classic CAN and CAN-FD protocols.

20.1 MCAN Overview.....	1066
20.2 MCAN Environment.....	1067
20.3 CAN Network Basics.....	1068
20.4 MCAN Functional Description.....	1069
20.5 MCAN Integration.....	1118
20.6 Interrupt and Event Support.....	1119
20.7 MCAN Registers.....	1120

20.1 MCAN Overview

The Controller Area Network (CAN) is a serial communications protocol that efficiently supports distributed real-time control with a high level of reliability. CAN has high immunity to electrical interference and the ability to detect various type of errors. In CAN, many short messages are broadcast to the entire network, which provides data consistency in every node of the system.

The MCAN module supports both classic CAN and CAN FD (CAN with flexible data-rate) protocols. The CAN FD feature allows higher throughput and increased payload per data frame. Classic CAN and CAN FD devices can coexist on the same network without any conflict provided that partial network transceivers, which can detect and ignore CAN FD without generating bus errors, are used by the classic CAN devices. The MCAN module is compliant to ISO 11898-1:2015.

Note

The CAN FD feature is not available on all devices. Refer to the device-specific data sheet for more information.

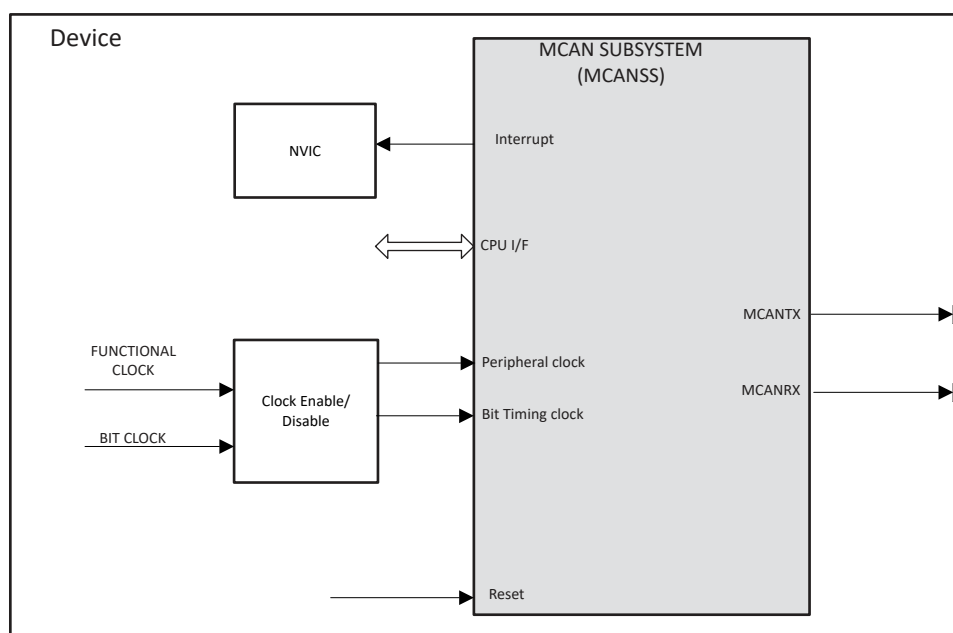


Figure 20-1. MCAN Block Diagram

20.1.1 MCAN Features

The MCAN module implements the following features:

- Conforms with CAN Protocol 2.0 A, B and ISO 11898-1:2015
- Full CAN FD support (up to 64 data bytes)
- AUTOSAR and SAE J1939 support
- Up to 32 dedicated transmit buffers
- Configurable transmit FIFO, up to 32 elements
- Configurable transmit queue, up to 32 elements
- Configurable transmit Event FIFO, up to 32 elements
- Up to 14 dedicated receive buffers
- Two configurable receive FIFOs, up to 14 elements each with 1kB message RAM.
- Up to 128 filter elements
- Loop-back mode for self-test
- Maskable interrupt (two configurable interrupt lines, correctable ECC, counter overflow, and clock stop or wakeup)
- Non-maskable interrupt (uncorrectable ECC)
- Two clock domains (CAN clock and host clock)
- ECC check for Message RAM
- Clock stop and wakeup support
- Timestamp counter
- 1-Mbps nominal bit rate, 8-Mbps data bit rate

Non-supported features:

- Host bus firewall
- Clock calibration
- Debug over CAN

20.2 MCAN Environment

The CAN network physical layer consists of a two-wire differential bus, usually twisted pair, and provides a high level of interference immunity. An external CAN transceiver IC is needed to access the bus.

Figure 20-2 shows typical MCAN wiring. Table 20-1 describes the external signals of the MCAN module.

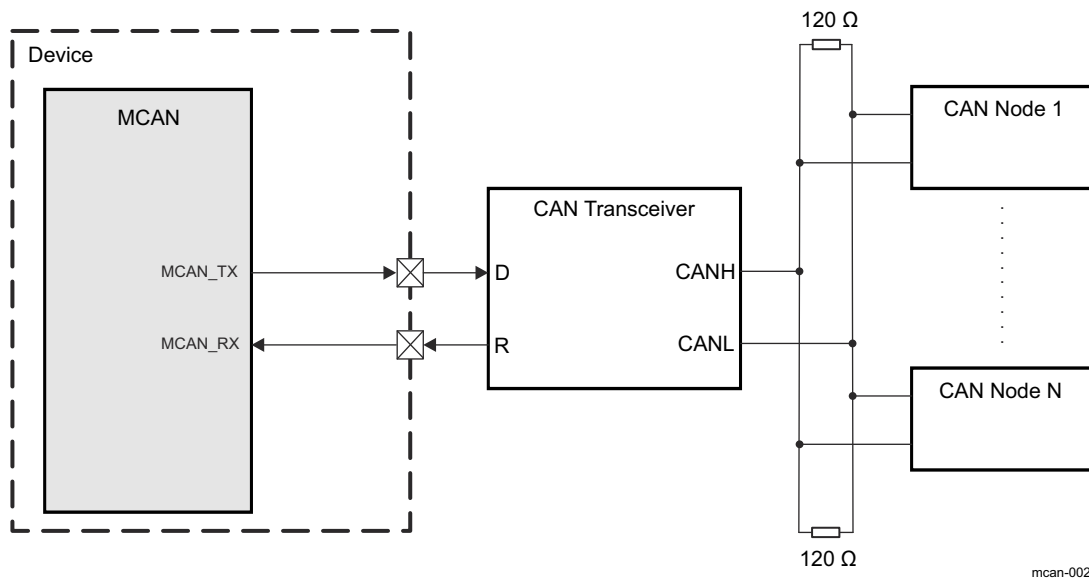


Figure 20-2. MCAN Typical Bus Wiring

Table 20-1. MCAN I/O Description

Module Signal	I/O	Description	Value at Reset
MCAN_RX	Input	Serial data input from external CAN transceiver.	HiZ
MCAN_TX	Output	Serial data output to external CAN transceiver.	HiZ

Note

See the *Terminal Configurations and Functions* section in the device data sheet and the *General-Purpose Input/Output (GPIO)* chapter to configure this peripheral to be connected to the device pins.

20.3 CAN Network Basics

The network basics are:

- The CAN bus is a 2-wire differential bus using non-return-to-zero (NRZ) encoding and has two states:
 - Recessive state (logical 1)
 - Dominant state (logical 0)
- When the bus is idle, any node can initiate a transmission to any other node. When two or more nodes (ECUs) attempt to transmit at the same time, a nondestructive arbitration technique sends messages in order of priority so that no messages are lost.
- The message transmission is multicast. Data messages transmitted are identifier-based, not address-based.
- The content of the message is labeled by the identifier that is unique throughout the network (for example: RPM, temperature, position, pressure, and so forth).
- All nodes on the network receive the message and each performs an acceptance test on the identifier. If the message is relevant, it is processed; otherwise, it is ignored.
- The unique identifier also determines the priority of the message (the lower the numerical value of the identifier, the higher the priority).
- Data is transmitted and received using message frames, consisting of the following basic fields:
 - Arbitration field
 - Control field
 - Data field (up to 8 bytes for classical CAN and up to 64 bytes for CAN FD)
 - CRC field
 - ACK field

For more information, see *ISO 11898-1:2015: CAN data link layer and physical signaling*.

MCAN power up and clock sequence :

1. Select CANCLK from the available clock sources: the HFCLK (HFXT or HFCLK_IN) or the SYSPLL (SYSPLLCLK1).
2. Enable the corresponding clock source.
3. Wait until the clock source is stable (SYSCTL_CLKSTATUS[*GOOD] bit = 1).
4. Enable MCAN power by setting PWREN for MCAN.
5. Wait until MCAN is ready (SYSCTL_SYSSTATUS[MCAN0READY] = 1).

20.4 MCAN Functional Description

The MCAN module performs CAN protocol communication according to ISO 11898-1:2015. The bit rate can be programmed to values up to 8Mbit/s. Additional transceiver hardware is required for the connection to the physical layer (CAN bus).

For communication on a CAN network, individual message frames can be configured. The message frames and identifier masks are stored in the Message RAM.

All functions concerning the handling of messages are implemented in the Message Handler.

The register set of the MCAN module can be accessed directly through the module interface. These registers are used to control and configure the CAN core and the Message Handler, and to access the Message RAM.

Figure 20-3 shows the MCAN module block diagram, followed by the description of the MCAN module blocks.

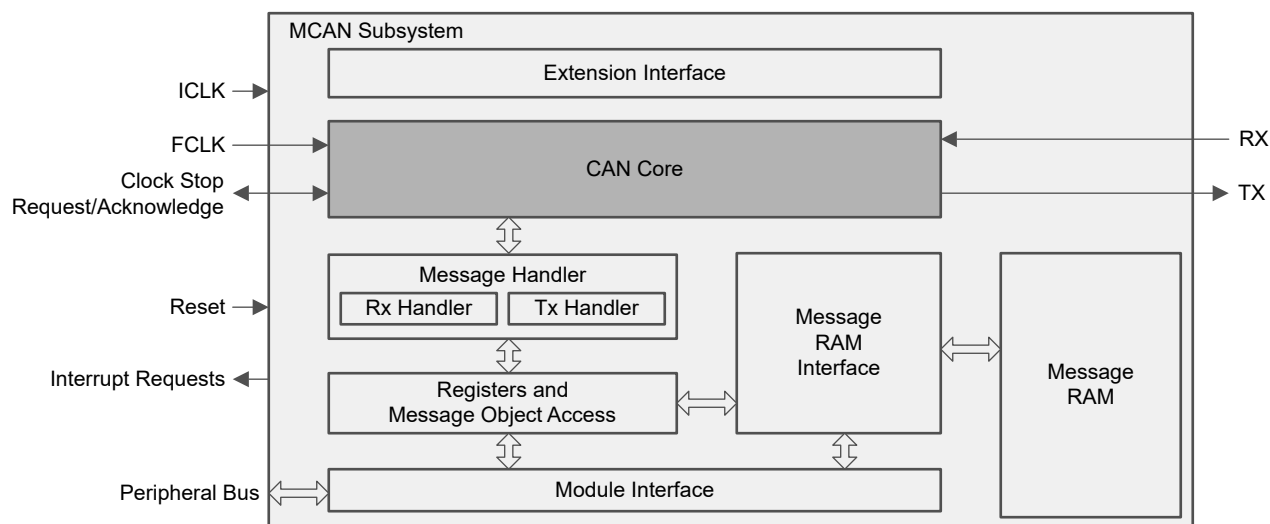


Figure 20-3. MCAN Block Diagram

- **CAN Core:** The CAN core consists of the CAN protocol controller and the Rx/Tx shift register. It handles all ISO 11898-1:2015 protocol functions and supports 11-bit and 29-bit identifiers.
- **Message Handler:** the Message Handler (Rx Handler and Tx Handler) is a state machine that controls the data transfer between the single-ported Message RAM and the CAN core's Rx/Tx shift register. It also handles the acceptance filtering and Interrupt generation as programmed in the control registers.
- **Message RAM:** the main purpose of the Message RAM is to store Rx/Tx messages, Tx Event elements, and Message ID Filter elements (for more information, see [Section 20.4.19](#)).
- **Message RAM Interface:** enables a connection between the Message RAM and the other blocks in the MCAN module.
- **Registers and Message Object Access:** Data consistency is ensured by indirect accesses to the message objects. During normal operation, all software and DMA accesses to the Message RAM are done through interface registers. The interface registers have the same word-length as the Message RAM.
- **Module Interface:** The MCAN module registers are accessed by the user's software through a 32-bit peripheral bus interface.
- **Clocking:** Two clocks are provided to the MCAN module: the peripheral synchronous clock (interface clock - MCAN_ICLK) and the peripheral asynchronous clock (functional clock - MCAN_FCLK).
- **Extension Interface:** All selected internal status and control signals are routed to this interface (except for the indication signals of configuration change enable bit (MCAN_CCCR.CCE) and Interrupt Register bits (MCAN_IR).

20.4.1 Clock Set up

Peripheral asynchronous clock (MCAN_FCLK) for MCAN can be clocked either through HFXT or SYSPLL. This clock configuration has to be done through SYSCTL REGISTERS. *Refer to CANCLK (CAN-FD Functional Clock) under the [Clocks](#) section.*

20.4.2 Module Clocking Requirements

Two clocks are provided to the MCAN module:

- Host Clock : peripheral synchronous clock (MCAN_ICLK) as the general module clock source, and
- CAN Clock: peripheral asynchronous clock (MCAN_FCLK) provided to the CAN core for generating the CAN bit timing.

Within the MCAN module, there is a synchronization mechanism implemented to make sure there is safe data transfer between the two clock domains. There is synchronization between the signals from the Host clock domain to the CAN clock domain and conversely, and between the reset signal (MCAN_RST) to the Host clock domain and to the CAN clock domain.

Note

MCAN_ICLK must always be higher or equal to MCAN_FCLK, to achieve a stable functionality of the MCAN module: $f_{\text{ICLK}} \geq f_{\text{FCLK}}$

The CAN-FD supports higher speeds of operation and as such has more stringent timing requirements than the classic CAN. For performance, TI recommends using the lowest N-divider value that maintains a working PLL REF_CLK for the system. Lower N-divider values increase the loop bandwidth of the PLL, which in turn improves timing margins for CAN-FD.

The CAN-FD supports higher speeds of operation and as such has more stringent timing requirements than the classic CAN. For performance, TI recommends using the lowest N-divider value that maintains a working PLL REF_CLK for the system. Lower N-divider values increase the loop bandwidth of the PLL, which in turn improves timing margins for CAN-FD. The peripheral asynchronous clock (MCAN_FCLK) can be clocked either through HFXT or SYSPLL. The configuration of this has to be done through the SYSCTL registers. Refer to [Clocks](#) module for more information

20.4.3 Interrupt Requests

The MCAN module generates interrupt requests. It is configured through the Host CPU. The Suspend mode prevents the interrupt requests from propagating to the Host CPU. The MCAN core has two interrupt lines and 30 internal interrupt sources. Each source can be configured to drive one of the two interrupt lines. The interrupts are 'level high' interrupts. The MCAN core provides two interrupt requests (MCANSS_INT0 and MCANSS_INT1).

For more information, see the following registers:

- Interrupt Register (MCAN_IR)
- Interrupt Enable (MCAN_IE)
- Interrupt Line Select (MCAN_ILS)
- Interrupt Line Enable (MCAN_ILE)

The MCAN module supports External Timestamp Counter. The External Timestamp Counter produces an interrupt when it rolls over (see [Section 20.4.12.1](#)).

For more information, see the following registers:

- Interrupt Clear Shadow Register (MCANSS_ICS)
- Interrupt Raw Status Register (MCANSS_IRS)
- Interrupt Enable Clear Shadow Register (MCANSS_IECS)
- Interrupt Enable Register (MCANSS_IE)
- Interrupt Enable Status Register (MCANSS_IES)
- End Of Interrupt Register (MCANSS_EOI)
- External Timestamp Prescaler Register (MCANSS_EXT_TS_PRESCALER)
- External Timestamp Unserviced Interrupts Counter Register (MCANSS_EXT_TS_UNSERVICED_INTR_CNTR)

To clear IRQ_INT0, IRQ_INT1 and TS_WAKE interrupts, write to the EOI bit field for the corresponding interrupt number that is described in the MCANSS_EOI register.

After servicing an interrupt (external timestamp, interrupt 0/1), write '1' in the corresponding bit in MCANSS_EOI register to clear the interrupt. In case of an ECC interrupt, after clearing the ECC interrupt source, application software must also write a '1' to the EOI registers (MCANERR_SEC_EOI.EOI_WR/ CANERR_DED_EOI.EOI_WR). For more information see [Section 20.4.14.2](#)

The IRQ sequence is:

1. Enable one of the interrupt sources by setting corresponding bit in IMASK.
2. Set MCANSS_IE to enable the IRQ, set MCANSS_ILS for line0 or line1.
3. Wait for the interrupt source to be triggered.
4. The interrupt is triggered and the application jumps into IRQ service routine (ISR).
5. In the ISR, check if IRQ is triggered by the expected source by reading IIDX. Reading IIDX clears the IRQ source, so there is no need to write ICLR to clear the IRQ.

Due to the design of MCAN, step (6) is required in the ISR to clear IRQ:

6. Write 1 to MCAN_IR to clear the interrupt. If the source is IRQ_INT0, IRQ_INT1, or TS_WAKE, also clear MCANSS_EOI, otherwise the next IRQ will not be recognized.

20.4.4 Operating Modes

The operating modes are discussed in the following sections.

20.4.4.1 Normal Operation

Once the MCAN module is initialized and the MCAN_CCCR.INIT bit is reset to zero, the MCAN module synchronizes itself to the CAN bus and is ready for communication. After passing the acceptance filtering, received messages including Message Identifier (ID) and Data Length Code (DLC) are stored into a dedicated Rx Buffer or into Rx FIFO 0/Rx FIFO 1.

For messages to be transmitted, dedicated Tx buffers, and a Tx FIFO or a Tx queue can be initialized or updated.

Note

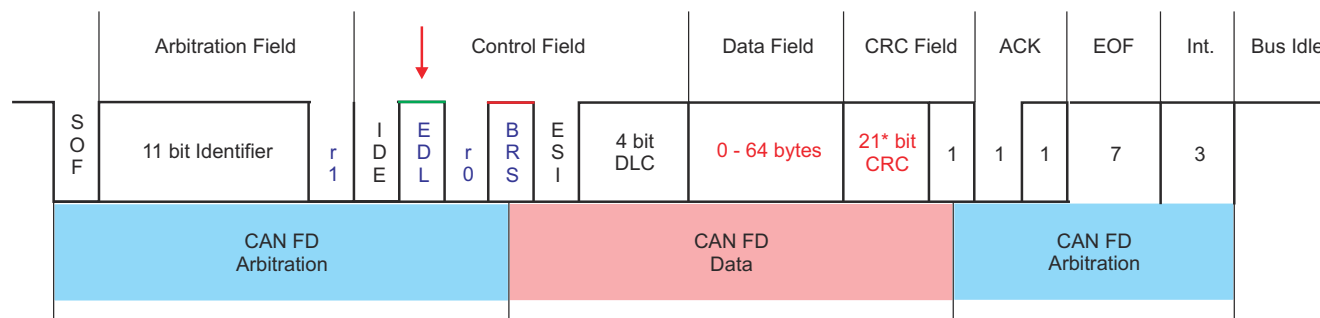
Automated transmission upon reception of remote frames is not supported.

20.4.4.2 CAN Classic

CAN supports transmission of data up to 8 bytes with standard (11 bit) or (29 bit) identifier.

20.4.4.3 CAN FD Operation

The CAN FD standard allows extended frames to be sent, up to 64 data bytes in a single frame at a higher bit rate for the data phase of a frame, up to 8 Mbps. The CAN FD standard introduces the ability to switch from one bit rate to another. Extended Data Length (EDL), as shown in Figure 20-4 and described in Table 20-2, sets a data length of up to 8 or 64 data bytes. Bit Rate Switching (BRS) indicates whether two bit rates (the data phase is transmitted at a different bit rate compared to the arbitration phase) are enabled.



* 17 bit CRC for data fields with up to 16 bytes

mcan-004a

Figure 20-4. CAN FD Frame

Table 20-2. CAN FD Frame Description

Bit	Description
SOF	Start of Frame
IDE	Identifier extension (for 29 bit extended ID)
FDF	Flexible Data Format
BRS	Bit Rate Switching
ESI	Error Status Indicator
DLC	Data Length Code
CRC	Cyclic Redundancy check

There are two variants of CAN FD frame transmission:

- CAN FD frame transmission without bit rate switching
- CAN FD frame transmission where control field, data field, and CRC field are transmitted with a higher bit rate than the beginning and the end of the frame

In the CAN frames, FDF = recessive (logical 1) signifies a CAN FD frame, FDF = dominant (logical 0) signifies a Classic CAN frame. In a CAN FD frame, the two bits following FDF - res and BRS, decide whether the bit rate inside of this CAN FD frame is switched. A CAN FD bit rate switch is signified by res = dominant and BRS = recessive. Note that the coding of res = recessive is reserved for future expansion of the protocol. If the MCAN module receives a frame with FDF = recessive and res = recessive, the MCAN signals a Protocol Exception Event by setting the MCAN_PSR.PXE bit. When Protocol Exception Handling is enabled (MCAN_CCCR.PXHD = 0), this causes the operation state to change from Receiver (MCAN_PSR.ACT = 10) to Integrating (MCAN_PSR.ACT = 00) at the next sample point. In case Protocol Exception Handling is disabled (MCAN_CCCR.PXHD = 1), the MCAN treats a recessive bit as an error and responds with an error frame.

CAN FD operation is enabled by programming the MCAN_CCCR.FDOE bit. If MCAN_CCCR.FDOE = 1, transmission and reception of CAN FD frames is enabled. Transmission and reception of Classic CAN frames is always possible. Whether a CAN FD frame or a Classic CAN frame is transmitted can be configured using the FDF bit in the respective Tx Buffer element.

With MCAN_CCCR.FDOE = 0, received frames are interpreted as Classic CAN frames, which leads to the transmission of an error frame when receiving a CAN FD frame. When CAN FD operation is disabled, no CAN FD frames are transmitted even if the FDF bit of a Tx Buffer element is set. The MCAN_CCCR.FDOE and MCAN_CCCR.BRSE bits can only be changed while the MCAN_CCCR.INIT and MCAN_CCCR.CCE bits are both set. With MCAN_CCCR.FDOE = 0, the setting of bits FDF and BRS is ignored and frames are transmitted in Classic CAN format.

With MCAN_CCCR.FDOE = 1 and MCAN_CCCR.BRSE = 0, only FDF bit of a Tx Buffer element is evaluated. With MCAN_CCCR.FDOE = 1 and MCAN_CCCR.BRSE = 1, transmission of CAN FD frames with bit rate switching is enabled. All Tx Buffer elements with bits FDF and BRS set are transmitted in CAN FD format with bit rate switching.

A mode change during CAN operation is only recommended under the following conditions:

- The failure rate in the CAN FD data phase is significantly higher than in the CAN FD arbitration phase. In this case, disable the CAN FD bit rate switching option for transmissions.
- During system startup, all nodes are transmitting Classic CAN messages until verified that the nodes are able to communicate in CAN FD format. If this is true, all nodes switch to CAN FD operation.
- Wakeup messages in CAN Partial Networking have to be transmitted in Classic CAN format.
- End-of-line programming in case not all nodes are CAN FD capable. Non CAN FD nodes are held in Silent mode until programming has completed. Then all nodes switch back to Classic CAN communication.

The coding of the DLC in the CAN FD format differs from the standard CAN format. The DLC codes 0 to 8 have the same coding as in standard CAN (0 to 8 data bytes), the codes 9 to 15, which in standard CAN all code a data field of 8 bytes, are coded according to [Table 20-3](#).

Table 20-3. DLC Coding in CAN FD

DLC	9	10	11	12	13	14	15
Number of Data Bytes	12	16	20	24	32	48	64

For CAN FD frames, the bit timing is switched inside the frame after the BRS (Bit Rate Switch) bit in case this bit is recessive. In the CAN FD arbitration phase, before the BRS bit, the nominal CAN bit timing (see [Figure 20-5](#)) is used as configured by the Nominal Bit Timing and Prescaler Register (MCAN_NBTP). In the following CAN FD data phase, the data phase bit timing is used as configured by the Data Bit Timing and Prescaler Register (MCAN_DBTP). The bit timing is switched back from the data phase timing at the CRC delimiter or when an error is detected, whichever occurs first.

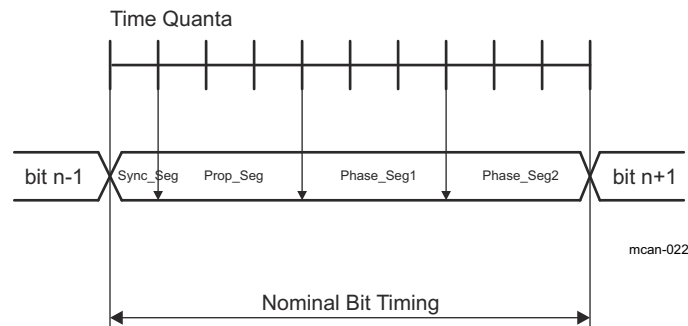


Figure 20-5. CAN Bit Timing

The maximum configurable data phase bit timing depends on the CAN clock frequency (MCAN_FCLK). Example: with MCAN_FCLK = 20 MHz and the shortest configurable bit time of 4 t_q (time quanta), the bit rate in the data phase is 5 Mbit/s.

In both data frame formats, CAN FD and CAN FD with bit rate switching, the value of the Error Status Indicator (ESI) bit depends on the transmitter error state (see MCAN_PSR.RESI bit) monitored at the start of the transmission. If the transmitter has an error passive flag, the ESI bit is transmitted recessive; else, the ESI bit is transmitted dominant.

20.4.5 Software Initialization

A software initialization begins when the MCAN_CCCR.INIT bit is set to 1. This is done either by software or by a hardware reset, when an uncorrected bit error is detected in the Message RAM, or by going to a Bus_Off state. While the MCAN_CCCR.INIT bit is set, the message transfer is stopped and the status of the output TX pin is recessive (high). The counters of the Error Management Logic (EML) are unchanged. Setting the MCAN_CCCR.INIT bit does not change any configuration register. Resetting the MCAN_CCCR.INIT bit finishes the software initialization. After waiting for the occurrence of a sequence of 11 consecutive recessive bits (indication for Bus_Idle state) the message transfer starts.

Access to the MCAN configuration registers is only enabled when both MCAN_CCCR.INIT and MCAN_CCCR.CCE bits are set (write protection).

The MCAN_CCCR.CCE bit can only be set/reset while the MCAN_CCCR.INIT = 1. The MCAN_CCCR.CCE bit is automatically reset when the MCAN_CCCR.INIT bit is reset.

The following registers are reset when the MCAN_CCCR.CCE bit is set:

- MCAN_HPMS - High Priority Message Status
- MCAN_RXF0S - Rx FIFO 0 Status
- MCAN_RFX1S - Rx FIFO 1 Status
- MCAN_TXFQS - Tx FIFO/Queue Status
- MCAN_TXBRP - Tx Buffer Request Pending
- MCAN_TXBTO - Tx Buffer Transmission Occurred
- MCAN_TXBCF - Tx Buffer Cancellation Finished
- MCAN_TXEFS - Tx Event FIFO Status

The Timeout Counter value MCAN_TOCV.TOC field is preset to the value configured by the MCAN_TOCC.TOP field when the MCAN_CCCR.CCE bit is set.

In addition, the Tx Handler and Rx Handler are held in idle state while MCAN_CCCR.CCE = 1.

The following registers are only writable while MCAN_CCCR.CCE = 0

- MCAN_TXBAR - Tx Buffer Add Request
- MCAN_TXBCR - Tx Buffer Cancellation Request

MCAN_CCCR.TEST and MCAN_CCCR.MON bits can only be set by the Host CPU while MCAN_CCCR.INIT = 1 and MCAN_CCCR.CCE = 1. Both bits are reset at any time. The MCAN_CCCR.DAR bit can only be set/reset while MCAN_CCCR.INIT = 1 and MCAN_CCCR.CCE = 1.

Table 20-4 shows the steps to configure the MCAN module.

Table 20-4. Steps to Configure MCAN Module

Step	Operation	Description	Pseudo Code
1	Initialize MCAN_CCCR	Set MCAN_CCCR.INIT bit and check that the bit has been set	INIT = 1; If INIT ≠ 1, wait until set
2	Unlock protected registers	Set MCAN_CCCR.CCE bit	CCE = 1;
3	Configure CAN mode	Set MCAN_CCCR.FDOE bit to CAN FD	FDOE = 1 for CAN FD FDOE = 0 for Classic CAN
4	Configure Bit Rate Switching	Set MCAN_CCCR.BRSE bit	BRSE = 1 for bit rate switching BRSE = 0 for no bit rate switching
5	Set nominal bit timing ⁽¹⁾	Set MCAN_NBTP register	
6	Lock protected registers	Clear MCAN_CCCR.CCE bit	CCE = 0;
7	Return MCAN module to normal operation	Clear MCAN_CCCR.INIT bit and check that the bit has been cleared	INIT = 0; If INIT ≠ 0, wait until cleared

(1) See the MCAN_NBTP register on how to program CAN bit timing in the *MCAN_REGS Registers* section.

20.4.6 Transmitter Delay Compensation

20.4.6.1 Description

When only one CAN FD node is transmitting and all other nodes are receivers, the length of the bus line has no impact. When transmitting using the TX pin, the MCAN module receives the transmitted data from its CAN transceiver using the RX pin. The received data is delayed. If the transmitter delay is greater than TSEG1 (time segment before sample point), a bit error is detected.

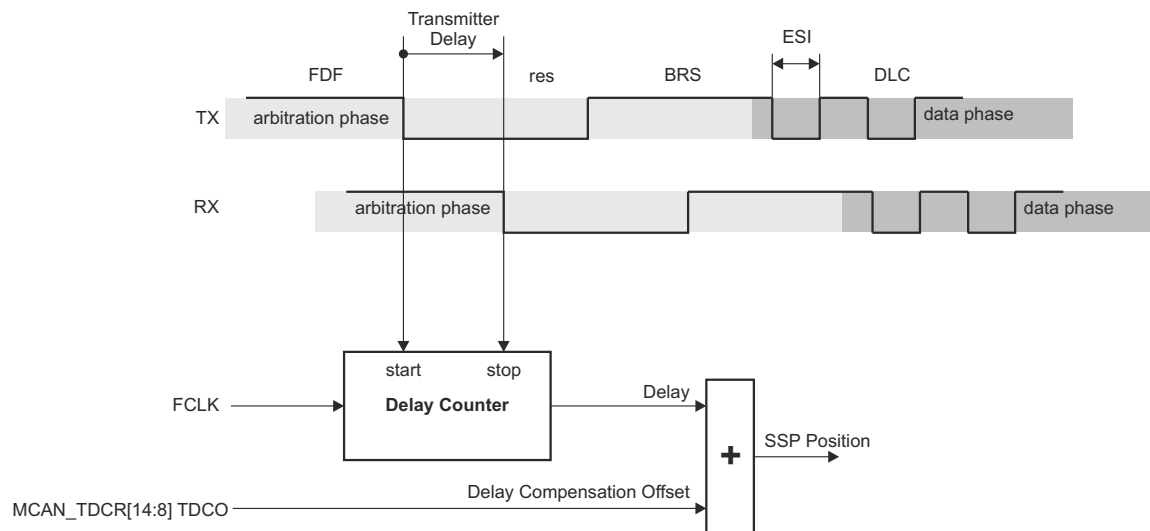
The MCAN module provides a delay compensation mechanism to compensate for the transmitter delay. The compensation mechanism enables transmission with higher bit rates during the CAN FD data phase independent of the delay of a specific CAN transceiver. Without transmitter delay compensation the bit rate in the data phase is limited by the transmitter delay.

The mechanism enables configurations where the data bit time is shorter than the transmitter delay (it is described in detail in ISO 11898-1:2015). The transmitter delay compensation is enabled by setting the MCAN_DBTP.TDC bit to 1.

The delayed transmit data is compared against the received data at the Secondary Sample Point (SSP) to check for bit errors during the data phase of transmitting nodes. If a bit error is detected, the transmitter reacts on this bit error at the next following regular sample point. During the arbitration phase, the delay compensation is always disabled.

The received bit is compared against the transmitted bit at the SSP. The SSP position is defined as the sum of the measured delay from the MCAN transmit output TX pin through the transceiver to the receive input RX pin plus the transmitter delay compensation offset configured by the MCAN_TDCR.TDCO field (see [Figure 20-6](#)). The transmitter delay compensation offset is used to adjust the position of the SSP inside the received bit (example: half of the bit time in the data phase). The position of the SSP is rounded down to the next integer number of mtq.

The actual transmitter delay compensation value can be checked by reading the MCAN_PSR.TDCV field. This field is cleared when the MCAN_CCCR.INIT bit is set and is updated at each transmission of CAN FD frame while the MCAN_DBTP.TDC bit is set.



mcan-005

Figure 20-6. Transmitter Delay Measurement

20.4.6.2 Transmitter Delay Compensation Measurement

When transmitter delay compensation is enabled (by programming MCAN_DBTP.TDC = 1), the measurement is started within each transmitted CAN FD frame at the falling edge of FDF bit to bit r0. The measurement is stopped when this edge is seen at the receive input RX pin of the transmitter. The resolution of this measurement is one mtq (see [Figure 20-6](#)). The mtq (minimum time quantum) dimension is equal to the CAN clock period (MCAN_FCLK).

The use of a transmitter delay compensation filter window can be enabled by programming the MCAN_TDCR.TDCF field. This filter feature defines a minimum value for the SSP position to avoid the case in which a dominant glitch inside the received FDF bit ends the delay compensation measurement before the falling edge of the received res bit, resulting in an early taken SSP position. Dominant edges on the RX pin that result in an earlier SSP position are ignored for transmitter delay measurement. The measurement is stopped when the SSP position is at least MCAN_TDCR.TDCF field and the RX pin is low.

The following boundary conditions have to be considered:

- The sum of the measured delay from the TX pin to the RX pin and the configured transmitter delay compensation offset (MCAN_TDCR.TDCO field) has to be less than six bit times in the data phase.
- The sum of the measured delay from the TX pin to the RX pin and the configured transmitter delay compensation offset (MCAN_TDCR.TDCO) field has to be less or equal 127 mtq. In case this sum exceeds 127 mtq, the maximum value of 127 mtq is used for transmitter delay compensation.
- The data phase ends at the sample point of the CRC delimiter, that stops checking of receive bits at the SSPs.

20.4.7 Restricted Operation Mode

In restricted operation mode, the CAN node is able to receive data and remote frames and to give acknowledgment to valid frames, but the node does not send data frames, remote frames, active error frames, or overload frames. In case of an error condition or overload condition, the node does not send dominant bits; instead the node waits for the occurrence of bus idle condition to resynchronize itself to the CAN communication. The receive and transmit error counters (MCAN_ECR.REC and MCAN_ECR.TEC) are frozen while CAN error logging (MCAN_ECR.CEL) is active. The Host CPU can set the MCAN module into Restricted Operation Mode by setting the MCAN_CCCR.ASM bit. The bit can only be set by the Host CPU at any time when both MCAN_CCCR.CCE and MCAN_CCCR.INIT bits are set to 1.

The restricted operation mode is automatically entered when the Tx Handler is not able to read data from the Message RAM in time. To leave restricted operation mode, the Host CPU has to reset the MCAN_CCCR.ASM bit. This mode can be used in applications that adapt themselves to different CAN bit rates. In this case, the application tests different bit rates and leaves the restricted operation mode after the node has received a valid frame.

Note

The Restricted Operation Mode must not be combined with the Loop Back Mode.

20.4.8 Bus Monitoring Mode

Entering bus monitoring mode is done by setting the MCAN_CCCR.MON bit to 1. In this mode (see ISO 11898-1:2015, *Bus Monitoring* section), the MCAN module is able to receive valid data and remote frames, but cannot start a transmission. The MCAN module sends only recessive bits on the CAN bus. If the MCAN module is required to send a dominant bit (ACK bit, overload flag, active error flag), the bit is rerouted internally so that the MCAN module monitors this dominant bit, although the CAN bus may remain in recessive state. In bus monitoring mode, the MCAN_TXBRP register is held in reset state. The bus monitoring mode can be used to analyze the traffic on a CAN bus without affecting the bus by the transmission of dominant bits. [Figure 20-7](#) shows the connection of the TX and RX signals to the MCAN module in bus monitoring mode.

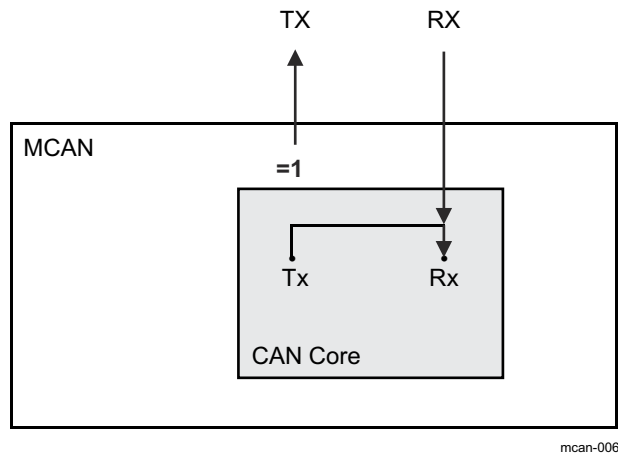


Figure 20-7. Connection of Signals in Bus Monitoring Mode

20.4.9 Disabled Automatic Retransmission (DAR) Mode

According to the CAN Specification (see ISO11898-1:2015, *Recovery Management* section), the MCAN module provides means for automatic retransmission of frames that have lost arbitration or that have been disturbed by errors during transmission. By default automatic retransmission is enabled (see the MCAN_CCCR.DAR bit).

20.4.9.1 Frame Transmission in DAR Mode

In DAR mode all transmissions are automatically canceled after they started on the CAN bus. A Tx buffer's Tx Request Pending (MCAN_TXBRP[xx]) TRPx bit is reset after successful transmission, when a transmission has not yet been started at the point of cancellation, has been aborted due to lost arbitration, or when an error occurred during frame transmission.

Successful transmission:

- Corresponding Tx Buffer Transmission Occurred (MCAN_TXBTO.TOx) TOx bit is set
- Corresponding Tx Buffer Cancellation Finished (MCAN_TXBCF.CFx) CFx bit is not set

Successful transmission in spite of cancellation:

- Corresponding Tx Buffer Transmission Occurred (MCAN_TXBTO.TOx) TOx bit is set
- Corresponding Tx Buffer Cancellation Finished (MCAN_TXBCF.CFx) CFx bit is set

Arbitration lost or frame transmission disturbed:

- Corresponding Tx Buffer Transmission Occurred (MCAN_TXBTO.TOx) TOx bit is not set
- Corresponding Tx Buffer Cancellation Finished (MCAN_TXBCF.CFx) CFx bit is set

In the case of a successful frame transmission, and if storage of Tx events is enabled, a Tx Event FIFO element is written with Event Type ET = 10 (transmission in spite of cancellation).

20.4.10 Clock Stop Mode

Entering the clock stop mode can be achieved by programming one of two bits:

- Clock stop request bit in MCAN_IP (MCAN_CCCR.CSR)
- Stop request bit in MCAN_WRAPPER (MCANSS_CLKCTL.STOPREQ)

The register bit within the MCAN_IP (MCAN_CCCR.CSR) reads as 1 as long as the MCAN_WRAPPER bit (MCANSS_CLKCTL.STOPREQ) is asserted.

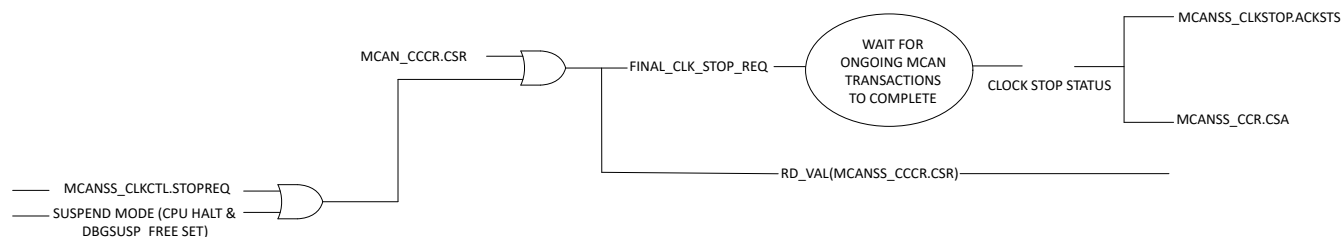


Figure 20-8. Clock Stop Request

After all pending transmission requests have completed, the MCAN module waits until the bus idle state is detected and then sets MCAN_CCCR.INIT to 1 to prevent further CAN transfers. The MCAN module then acknowledges the MCAN is ready for power down by setting the clock stop acknowledge bit MCAN_CCCR.CSA to 1 (MCANSS_CLKSTS.CLKSTOP_ACKSTS reflects the same value as well). In this state, before clocks are switched off, further register accesses can be made. However, a write access to the MCAN_CCCR.INIT bit has no effect. Module clock inputs MCAN_ICLK and MCAN_FCLK can now be switched off using MCANSS_CLKEN.CLK_REQEN.

To exit the power-down mode, the application has to turn on module clocks before clearing the clock stop request bit (make sure both MCAN_CCCR.CSR and MCANSS_CLKCTL.STOPREQ are cleared). MCAN acknowledges removal of the clock request by clearing the MCANSS_CLKSTS.CLKSTOP_ACKSTS

and MCAN_CCCR.CSA bits. The application can now restart CAN communication by clearing the MCAN_CCCR.INIT bit.

Automatic wakeup from the power-down mode (due to activity on MCAN Rx) is supported through the MCANSS_CTRL.AUTOWAKEUP and MCANSS_CTRL.WAKEUPREQEN bits (for more information, see [Section 20.4.10.2](#)).

Clock stop and automatic wakeup is illustrated in [Figure 20-9](#).

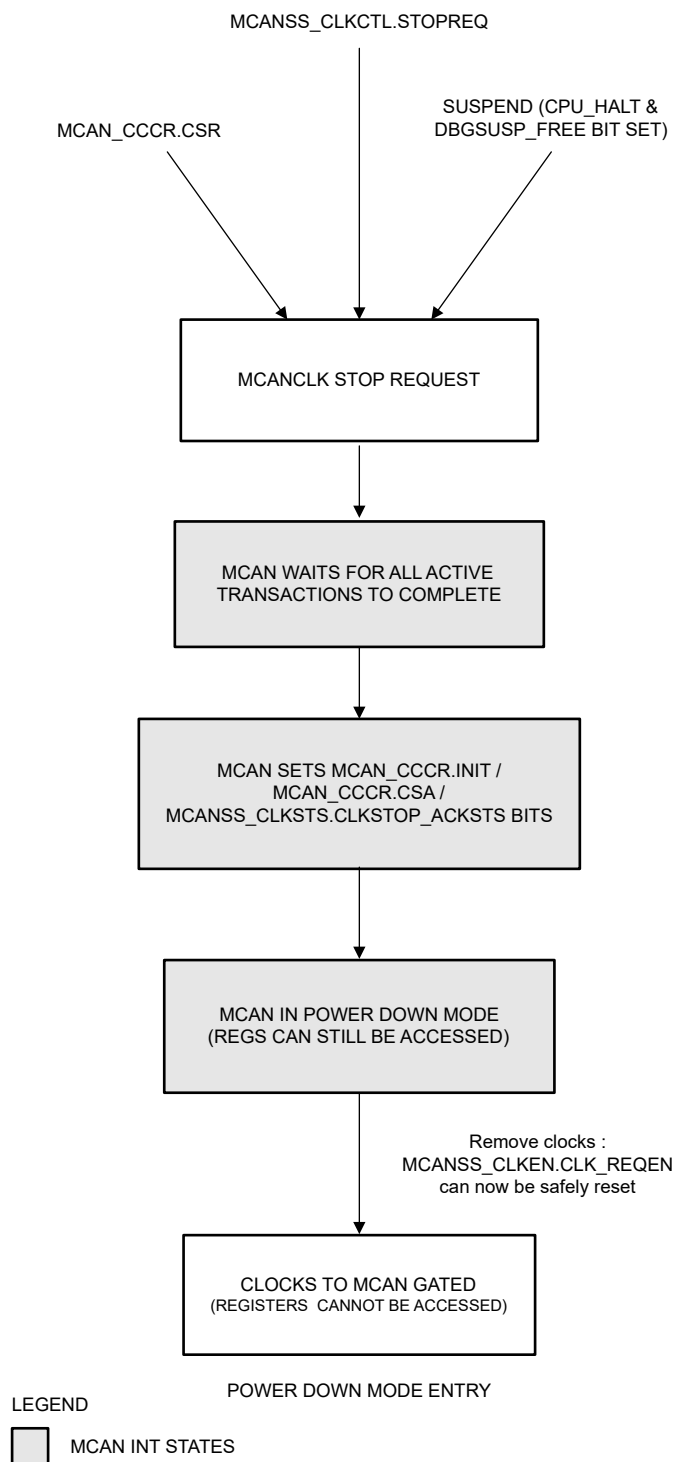


Figure 20-9. Power Down Entry

20.4.10.1 Suspend Mode

The MCAN module supports two suspend modes:

- Immediate
- Graceful

In a graceful suspend mode (see the MCANSS_CTRL.DBGSUSP_FREE bit) when the suspend request is asserted, a clock stop request to the MCAN core is performed. The MCAN core responds with a clock stop acknowledge when all pending Tx messages have been processed and an Idle line had been detected. At that point, the MCAN_CCCR.INIT bit is set and the MCAN core stays Idle. The suspend state can be verified by reading the MCAN_CCCR.INIT bit.

The automatic wakeup feature is enabled by setting the MCANSS_CTRL.AUTOWAKEUP and MCANSS_CTRL.WAKEUPREQEN bits to 1 (for more information, see [Section 20.4.10.2](#)). When suspend request is removed, if no external clock stop request is active, a read-modify-write to the MCAN_CCCR.INIT bit is performed to clear the bit.

During suspend mode the auto-clear feature is disabled. The following register fields have an auto-clear feature:

- MCAN_ECR.CEL
- MCAN_PSR.LEC
- MCAN_PSR.DLEC
- MCAN_PSR.RESI
- MCAN_PSR.RBRS
- MCAN_PSR.RFDF
- MCAN_PSR.PXE

20.4.10.2 Wakeup Request

Issuing a clock stop request puts the MCAN module into power-down mode (Sleep Mode). During transition from IDLE to ACTIVE, if the MCANSS_CTRL.AUTOWAKEUP and MCANSS_CTRL.WAKEUPREQEN bits are enabled, after the MCAN Core respond to the removal of the clock stop request with removing the clock stop acknowledge, a read-modify-write is issued to clear the MCAN_CCCR.INIT bit and the MCAN core resumes operation. Please note that after clock stop request has been removed by the hardware the first frame (wakeup frame) is not received. This is because after the clock stop is issued; there are no active clocks running into the IP. Therefore after removing the clock stop request; the wakeup frame that enables clock has to be re-transmitted.

If the MCANSS_CTRL.WAKEUPREQEN bit is set, the MCAN module provides a wakeup request on the following wakeup event:

- The receive RX pin is dominant (logical 0)

The wakeup request is deasserted when any of the following conditions occur:

- Clock stop request is removed and clock stop acknowledge is deasserted
- A reset is applied to the MCAN module

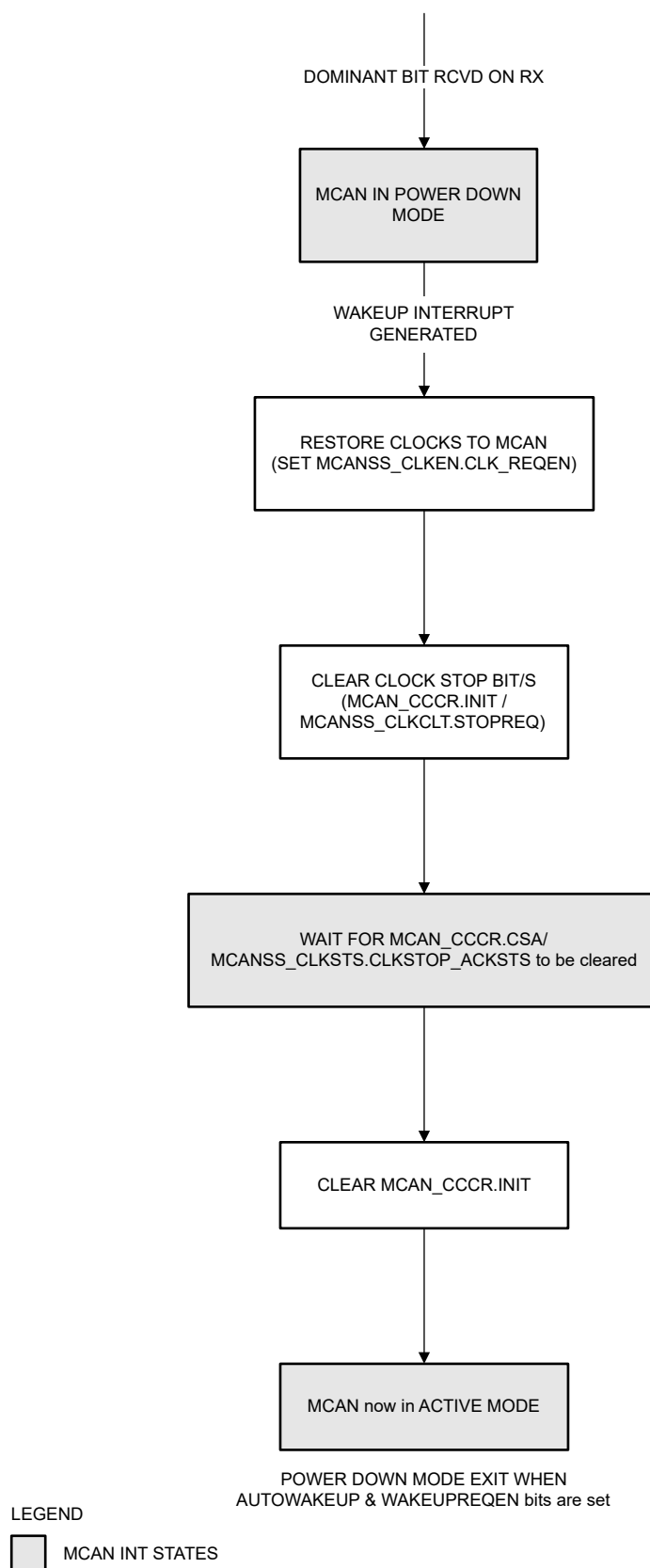


Figure 20-10. Auto Wakeup Enabled Exit from Power Down

Please note the suggested user notes while issuing a wakeup request .

- If there is a message that is being received on the CAN Bus when a Clock Stop Request (CSR) is issued; it will create a wakeup interrupt at the end of the reception resulting in the the MCAN module not entering the clock stop mode. To avoid unnecessary wake ups; the user should check the node activity bit (ACT) in the protocol status register(PSR) ensuring an idle state before setting the request.
- The user should also avoid creating a SW polling loop to only exit with a stop clock acknowledge (CSA) since the timing of the software check and wake up interrupt clearing the CSA could cause an infinite software loop
- If there is excessive noise on the Rx pin, it may cause frequent wake up. To avoid this it is recommended that the user waits for a specified time to see if a wake up was received and if not, assert the clock stop state.
- The wakeup logic can be unexpectedly disabled if MCAN is reset while clock stop request is asserted. Therefore, the user should avoid asserting a soft reset to MCAN while CSR is asserted

20.4.11 Test Modes

The MCAN_TEST register write access is enabled by setting the test mode enable MCAN_CCCR.TEST bit to 1. The MCAN_TEST register allows the configuration of the test modes and test functions.

The transmit (TX) pin has four different output functions which can be selected by programming the MCAN_TEST.TX field. The default function is the serial data output. The pin can also be driven with a constant dominant or recessive value. It is also possible to drive the sample-point signal to monitor the bit-timing.

The actual value of the receive (RX) pin can be monitored from MCAN_TEST.RX bit. Both functions can be used to check the physical layer. Due to the synchronization mechanism between the CAN clock (MCANx_FCLK) and Host clock (MCANx_ICLK) domain, there can be a delay of several Host clock periods between writing to the MCAN_TEST.TX field until the new configuration is visible at the output TX pin. This applies also when reading input RX pin by way of the MCAN_TEST.RX bit.

Note

Test modes can be used for self-test only. The software control for TX pin interferes with all CAN protocol functions. It is not recommended to use test modes for an application.

20.4.11.1 External Loop Back Mode

The MCAN module can be set into external loop back mode by programming MCAN_TEST.LBCK to 1. In loop backmode, the MCAN treats its own transmitted messages as received messages and stores them (if they pass acceptance filtering) into an Rx Buffer or an Rx FIFO as shown in below figure. Figure shows the connection of the TX and RX pins to the MCAN module in external loop back mode.

This mode is provided for hardware self-test. To be independent from external stimulation, the MCAN module ignores acknowledge errors (recessive bit sampled in the acknowledge slot of a data/remote frame) in loop back mode. In this mode, the MCAN module performs an internal feedback from the Tx output to the Rx input. The actual value of the RX input pin is disregarded by the MCAN module. The transmitted messages are monitored at the TX pin.

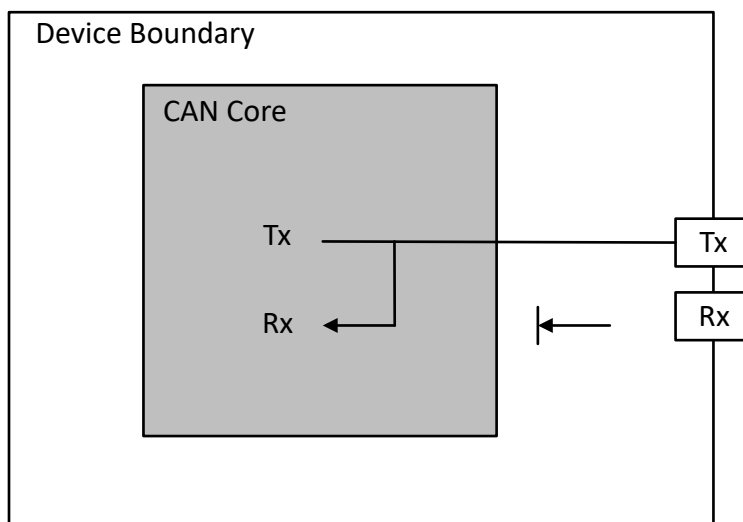


Figure 20-11. External Loop Back Mode

20.4.11.2 Internal Loop Back Mode

The MCAN module can be set into internal loop back mode by programming MCAN_TEST.LBCK and MCAN_CCCR.MON bits to 1. The internal loop back mode is used for a Hot Self-test. The Hot Self-test allows

the MCAN module to be tested without affecting a running CAN system connected to the TX and RX pins. In this mode, the RX pin is disconnected from the MCAN module and the TX pin is held recessive. [Figure 20-12](#) shows the connection of the TX and RX pins to the MCAN module in internal loop back mode.

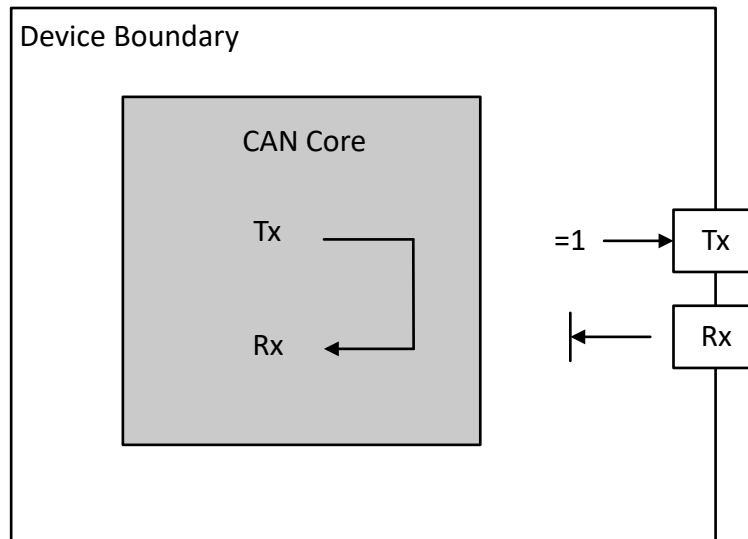


Figure 20-12. Internal Loop Back Mode

20.4.12 Timestamp Generation

The MCAN module has integrated a 16-bit wrap-around counter for timestamp generation. The timestamp counter prescaler MCAN_TSCC.TCP field can be configured to clock the counter in multiples of CAN bit times (1-16). The counter is readable by way of the MCAN_TSCV.TSC field. A write access to the MCAN_TSCV register resets the counter to zero. When the timestamp counter wraps around the interrupt MCAN_IR.TSW flag is set. On start of a frame reception/transmission the counter value is captured and stored into the timestamp section of an Rx Buffer/Rx FIFO (RXTS[15:0]) or Tx Event FIFO (TXTS[15:0]) element. For more information, see [Section 20.4.19](#).

20.4.12.1 External Timestamp Counter

For CAN FD operation mode, the MCAN core requires an external timestamp counter. An externally generated 16-bit vector can substitute the integrated 16-bit CAN bit time counter (internal timestamp counter) for receive and transmit timestamp generation. An external 16-bit timestamp counter can be used by programming the MCAN_TSCC.TSS field.

The external timestamp counter uses the interface clock (MCANx_ICLK) as a reference clock. The MCAN core accepts a 16-bit timestamp. A 24-bit prescaler provides a programmable resolution for the timestamp (see MCANSS_EXT_TS_PRESCALER.PRESCALER bit field). The external timestamp counter can be enabled or disabled through the MCANSS_CTRL.EXT_TS_CNTR_EN bit. When disabled, the counter is reset back to zero. While enabled, the counter keeps incrementing. When the timestamp rolls over, the MCAN_IRQ_TS interrupt is generated.

When the timestamp rolls over, the MCANSS_IRS register is set. The MCANSS_IE register can be affected by writing to the MCANSS_IESS register to set or to the MCANSS_IECS register to clear. The MCANSS_IESS register is a shadow register mapped to the same address as the MCANSS_IE register. The level interrupt is a reflection of both MCANSS_IRS and MCANSS_IE being set. The MCANSS_IES register reflects the level interrupt. When a rollover event occurs, the interrupt counter is incremented. Writing to the MCANSS_ICS register to clear the MCANSS_IRS register also decrements the interrupt counter. Writing to the MCANSS_EOI register issues another pulse, if the interrupt counter is not zero.

The rollover event can be artificially simulated by software through writing to the Interrupt Set Shadow register (MCANSS_ISS). The MCANSS_ISS register is a shadow register mapped to the same address as the MCANSS_IRS register.

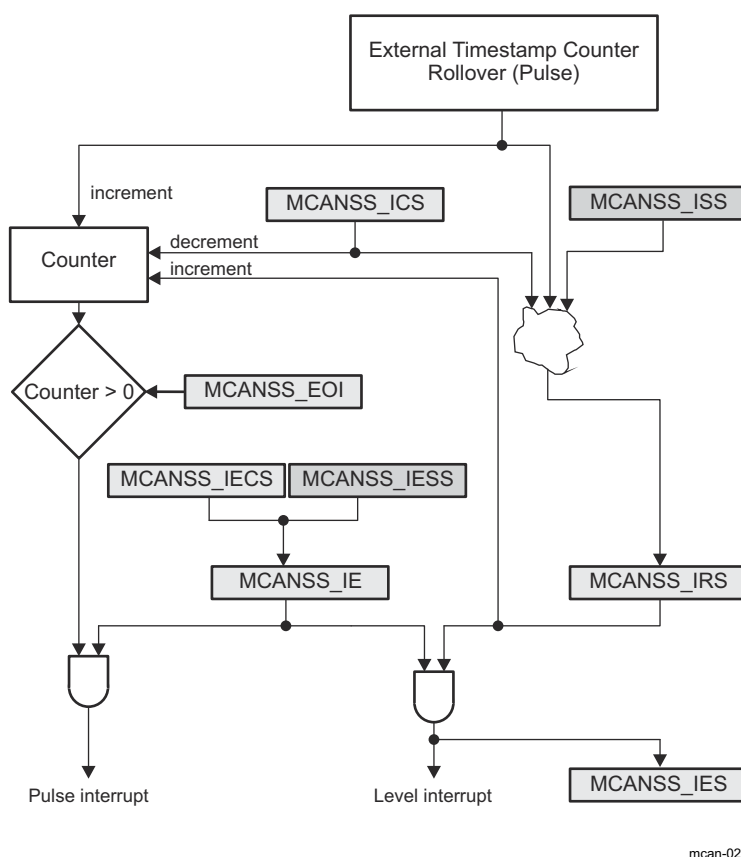


Figure 20-13. External Timestamp Counter Interrupt

20.4.13 Timeout Counter

The MCAN module has an integrated a 16-bit timeout counter. The timeout counter is used to signal timeout conditions for the Rx FIFO 0, Rx FIFO 1, and Tx Event FIFO Message RAM elements. The timeout counter is configured using the MCAN_TOCC register and is enabled using the MCAN_TOCC.ETOC bit. The timeout counter operates as down-counter and uses the same prescaler programmed by the MCAN_TSICC.TCP field as the timestamp counter. The actual counter value can be monitored from the MCAN_TOCV.TOC field. The timeout counter can be started only when MCAN_CCCR.INIT = 0 and stopped when MCAN_CCCR.INIT = 1 (example: when the MCAN enters Bus_Off state). The operation mode is selected by the MCAN_TOCC.TOS field. When continuous mode is selected, the counter starts when MCAN_CCCR.INIT = 0, a write to the MCAN_TOCV register presets the counter to the value configured by the MCAN_TOCC.TOP field and continues down-counting.

In case the timeout counter is controlled by one of the FIFOs, an empty FIFO presets the counter to the value configured by the MCAN_TOCC.TOP field. Down-counting is started when the first FIFO element is stored. Writing to the MCAN_TOCV register has no effect. When the counter reaches zero, the interrupt MCAN_IR.TOO flag is set.

In continuous mode, the counter is immediately restarted at the value configured by the MCAN_TOCC.TOP field.

Note

The clock signal for the timeout counter is derived from the CAN core sample point signal. Therefore, the point in time where the timeout counter is decremented can vary due to the synchronization/re-synchronization mechanism of the CAN core. If the baud rate switch feature in CAN FD is used, the timeout counter is clocked differently in arbitration and data field.

20.4.14 Safety

The Message Memory is wrapped in an ECC wrapper providing SECDED parity functionality. The ECC wrapper is controlled by an ECC aggregator.

20.4.14.1 ECC Wrapper

The ECC wrapper provides single error correction (SEC) and double error detection (DED) parity to the message memory content. The ECC wrapper has side band signals for error notification. The ECC Wrapper implements an error injection test mode.

The error correction is done using a lazy write back. When an error is detected, the error is noted in a FIFO queue that waits for an access gap to write the data back and refresh the memory. If a transaction writes new data to the compromised entry before the lazy write back completes, the write back is discarded.

20.4.14.2 ECC Aggregator

This section describes the functional details of the ECC aggregator module.

20.4.14.2.1 ECC Aggregator Overview

The ECC aggregator module supports the following general features:

- Provides a mechanism to control and monitor the ECC RAM in the MCAN module.
- Provides software access to all the ECC related registers.
- Supports software readable status of ECC single/double-bit errors and associated info such as RAM address and data bits that are in error.
- Aggregates level pending status from the ECC RAM into a single interrupt to the Host CPU.

The following feature is not supported:

- Statistics such as tracking the number of single and double-bit errors. If needed, these operations can be handled by software.

20.4.14.2.2 ECC Aggregator Registers

There are three groups of registers in the ECC aggregator module:

- **Global registers:** Aggregator Revision Register (MCANERR_REV), ECC Vector Register (MCANERR_VECTOR), Misc Status Register (MCANERR_STAT), ECC Control Register (MCANERR_CTRL), and ECC Wrapper Revision Register (MCANERR_WRAP_REV).
- **Control and status registers:** ECC Error Control Registers (MCANERR_ERR_CTRL1 and MCANERR_ERR_CTRL2) and ECC Error Status Registers (MCANERR_ERR_STAT1, MCANERR_ERR_STAT2, and MCANERR_ERR_STAT3).
- **Interrupt registers:** interrupt status, interrupt enable set, interrupt enable clear, and EOI (End Of Interrupt) registers that are part of a standard interrupt module. For more information, see the following registers:
 - MCANERR_SEC_EOI
 - MCANERR_SEC_STATUS
 - MCANERR_SEC_ENABLE_SET
 - MCANERR_SEC_ENABLE_CLR
 - MCANERR_DED_EOI
 - MCANERR_DED_STATUS
 - MCANERR_DED_ENABLE_SET
 - MCANERR_DED_ENABLE_CLR

20.4.14.3 Reads to ECC Control and Status Registers

The reads to the ECC control and status registers are triggered by writing a 'read message' to the ECC Vector Register as:

- Software writes value (the ECC RAM ID) to the MCANERR_VECTOR.ECC_VECTOR field to select the ECC RAM for control or status.
- Software writes 1 to the MCANERR_VECTOR.RD_SVBUS bit to trigger a read.
- Software writes read address to the MCANERR_VECTOR.RD_SVBUS_ADDRESS field.
- Software then polls the MCANERR_VECTOR.RD_SVBUS_DONE bit to check if the bit is 1. This bit indicates that the read operation has completed.
- Software reads the data from the ECC control or status register. The following clock cycle (MCAN_ICLK) returns the read data.

20.4.14.4 ECC Interrupts

The ECC aggregator module aggregates the level pending status from the ECC RAM into a single EOI-handshake based interrupt to the Host CPU. Software is expected to follow the sequence described:

- Software enables the interrupts for the ECC RAM by writing to the MCANERR_SEC_ENABLE_SET/MCANERR_DED_ENABLE_SET register.
- Software writes the ECC RAM ID in the MCANERR_VECTOR.ECC_VECTOR.
- Software writes the MCANERR_VECTOR.RD_SVBUS bit to trigger the read.
- Software writes the MCANERR_ERR_STAT1 register address to the MCANERR_VECTOR.RD_SVBUS_ADDRESS field. Software needs to load the 'read message' in the rMCANERR_VECTOR register again, if the software needs to read the MCANERR_ERR_STAT2 register.
- Software polls the MCANERR_VECTOR.RD_SVBUS_DONE bit. When this bit is set, a read of the MCANERR_ERR_STAT1/MCANERR_ERR_STAT2 register is performed.
- After the interrupt has been serviced, software clears the interrupt status by writing to the MCANERR_ERR_STAT1.CLR_ECC_SEC or MCANERR_ERR_STAT1.CLR_ECC_DED bit depending on the type of the ECC error.
- Software polls the MCANERR_ERR_STAT1 register to verify that the status bit has been cleared.
- Software writes to the MCANERR_SEC_EOI/MCANERR_DED_EOI register to clear the interrupt.
- After clearing the ECC interrupt source, the application software must also write 1 to the MCANERR_SEC_EOI.EOI_WR /MCANERR_DED_EOI.EOI_WR bits.

20.4.15 Tx Handling

The Tx handler is used to handle the Tx requests. It controls the transfer of transmit messages from the dedicated Tx buffers, the Tx FIFO, and the Tx Queue to the CAN Core, the Tx Event FIFO, and the Put and GetIndex operations. The MCAN module supports up to 32 Tx buffers. These Tx buffers can be configured as dedicated Tx buffers, Tx FIFO, or Tx Queue and as combination of dedicated Tx buffers/Tx FIFO or dedicated Tx buffers/Tx Queue. For each Tx Buffer element Classical CAN or CAN FD transmission mode can be configured. Table below shows the possible configurations for message transmission.

Table 20-5. Message Transmission configurations

MCAN_CCCR Register		Buffer Element		Frame Transmission
BRSE	FDOE	FDF	BRS	
ignored	0	ignored	ignored	Classic CAN
0	1	0	ignored	Classic CAN
0	1	1	ignored	CAN FD without bit rate switching
1	1	0	ignored	Classic CAN
1	1	1	0	CAN FD without bit rate switching
1	1	1	1	CAN FD with bit rate switching

When the Tx Buffer RequestPending (MCAN_TXBRP) register is updated, or when a transmission has been started the Tx Handler starts scanning to check for the highest priority pending Tx request. The Tx Buffer with the lowest Message ID has highest priority.

Note

AUTOSARrequires at least three Tx Queue buffers and support of transmit cancellation

20.4.15.1 Transmit Pause

The transmit pause feature is intended for use in CAN networks where the CAN Message IDs are specific and cannot easily be changed. These Message IDs may have a higher priority than other defined Message IDs, while in a specific application their relative priority should be inverse. This allows for a case where one ECU sends a burst of CAN messages that cause another ECU's CAN messages to be delayed (paused).

The transmit pause feature is enabled by the MCAN_CCCR.TXP bit. By default this bit is disabled (MCAN_CCCR.TXP= 0). Each time after successfully transmitted message, a pause for two CAN bit times occurs before the start of the next transmission. This allows the other CAN nodes in the network to transmit messages even if their Message IDs have lower priority.

20.4.15.2 Dedicated Tx Buffers

DedicatedTx buffers are intended for message transmission under complete control of the Host CPU. There are two options.

- Each dedicated Tx Buffer is configured with a specific Message ID
- Two or more dedicated Tx buffers are configured with the same Message ID. In this case the Tx Buffer with the lowest buffer number is transmitted first

After the data section has been updated, a transmission is requested by an Add Request. This is done using the MCAN_TXBAR[x]ARn bit (where x = 0 to 31). The requested messages arbitrate internally with messages from an optional Tx FIFO or Tx Queue and externally with messages on the CAN bus, and are sent out according to their Message ID.

The following table shows the Tx Buffer, Tx FIFO, and Tx Queue Element Size. A Dedicated Tx Buffer allocates element size 32-bit words in the Message RAM. The start address of a dedicated Tx Buffer in the Message RAM is calculated by adding transmit buffer index from 0 to 31 (MCAN_TXFQS.TFQP) × Element size to the Tx Buffer start address MCAN_TXBC.TBSA field

Table 20-6. Tx Buffer, Tx FIFO, Tx Queue Element Size

MCAN_TXESC.TBDS	DataField [Bytes]	Element Size [RAM Words]
000	8	4
001	12	5
010	16	6
011	20	7
100	24	8
101	32	10
110	48	14
111	64	18

20.4.15.3 Tx FIFO

Tx FIFO mode is configured by setting bit MCAN_TXBC.TFQM = 0. The stored in the Tx FIFO messages are transmitted starting with the message referenced by the Get Index MCAN_TXFQS.TFGI field. After each transmission the Get Index is incremented until the Tx FIFO is empty. The Tx FIFO Free Level MCAN_TXFQS.TFFL field indicates the number of the available free Tx FIFO elements. The Tx FIFO allows transmission of messages with the same Message ID from different Tx buffers in the order these messages have been written to the Tx FIFO.

New transmit messages have to be written to the Tx FIFO starting with the Tx Buffer referenced by the Put Index MCAN_TXFQS.TFQP field. After each Add Request (MCAN_TXBAR[x] ARn = 1) the Put Index is incremented to the next free Tx FIFO element. When the Put Index reaches the Get Index (MCAN_TXFQS.TFQP = MCAN_TXFQS.TFGI), Tx FIFO Full condition is signaled by bit MCAN_TXFQS.TFQF = 1. In this case no further messages should be written to the Tx FIFO until the next message has been transmitted and the Get Index has been incremented.

The number of requested Tx buffers should not exceed the number of free Tx buffers as indicated by the Tx FIFO Free Level MCAN_TXFQS.TFFL field.

In case a transmission request for the Tx Buffer referenced by the Get Index is canceled, the Get Index is incremented to the next Tx Buffer with pending transmission request and the Tx FIFO Free Level MCAN_TXFQS.TFFL field is recalculated. In case transmission cancellation is applied to any other Tx Buffer - the Get Index and the FIFO Free Level remain unchanged.

A Tx FIFO element allocates element size 32-bit words in the Message RAM. The start address of the next available (free) Tx FIFO Buffer is calculated by adding Tx FIFO/Queue Put Index MCAN_TXFQS.TFQP (from 0 to 31) × Element Size to the Tx Buffer Start Address MCAN_TXBC.TBSA field.

20.4.15.4 Tx Queue

Tx Queue mode is configured by setting bit MCAN_TXBC.TFQM = 1. The stored in the Tx Queue messages are transmitted starting with the highest priority message (lowest Message ID). In case two or more Queue buffers are configured with the same Message ID, the Queue Buffer with the lowest buffer number is transmitted first.

New transmit messages have to be written to the Tx FIFO starting with the Tx Buffer referenced by the Put Index MCAN_TXFQS.TFQP field. Each Add Request cyclically increments the Put Index to the next free Tx Buffer. In case of Tx Queue Full condition (MCAN_TXFQS.TFQF = 1), the Put Index is not valid and no further message should be written to the Tx Queue until at least one of the requested messages has been sent out or a pending transmission request has been canceled.

The application may use the MCAN_TXBRP register instead of the Put Index and may place messages to any Tx Buffer without pending transmission request.

A Tx Queue Buffer allocates element size 32-bit words in the Message RAM. The start address of the next available (free) Tx Queue Buffer is calculated by adding Tx FIFO/Queue Put Index MCAN_TXFQS.TFQP (from 0 to 31) × Element Size to the Tx Buffer Start Address MCAN_TXBC.TBSA field.

20.4.15.5 Mixed Dedicated Tx Buffers/Tx FIFO

For this combination the Tx buffers section in the Message RAM is separated in two parts:

- Dedicated Tx buffers: the number of dedicated Tx buffers is configured by MCAN_TXBC.NDTB field.
- Tx FIFO: the number of Tx buffers assigned to the Tx FIFO is configured by the MCAN_TXBC.TFQS field. If the MCAN_TXBC.TFQS field is empty(zero)- only dedicated Tx buffers are used.

Tx prioritization:

- Scan Dedicated Tx buffers and oldest pending Tx FIFO Buffer (referenced by the MCAN_TXFQS.TFGI field)
- Buffer with lowest Message ID gets highest priority and is transmitted next
- The following figure shows Mixed Dedicated Tx buffers/Tx FIFO example.

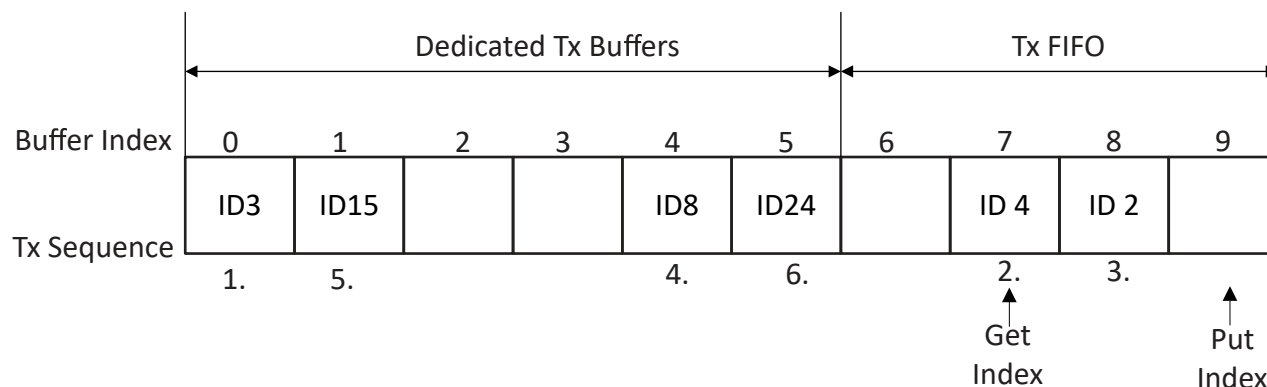


Figure 20-14. Mixed Dedicated Tx Buffers/ Tx FIFO (example)

20.4.15.6 Mixed Dedicated Tx Buffers/Tx Queue

For this combination the Tx buffers section in the Message RAM is separated in two parts:

- Dedicated Tx buffers: the number of Dedicated Tx buffers is configured by the MCAN_TXBC.NDTB field.
- Tx Queue: the number of Tx buffers assigned to the Tx Queue is configured by the MCAN_TXBC.TFQS field
If MCAN_TXBC.TFQS field is empty (zero) - only Dedicated Tx buffers are used.

Tx prioritization:

- Scan all Tx buffers with activated transmission request
- Tx Buffer with lowest Message ID gets highest priority and is transmitted next
- The following figure shows Mixed Dedicated Tx buffers/Tx Queue example.

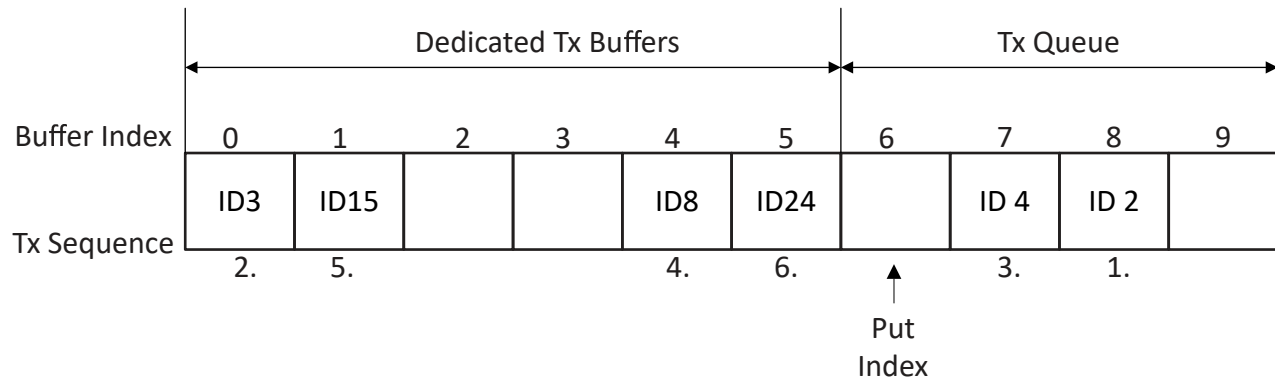


Figure 20-15. Mixed Dedicated Tx Buffers/ Tx FIFO (Example)

20.4.15.7 Transmit Cancellation

This feature is especially intended for gateway and AUTOSAR based applications. The Host CPU can cancel a requested transmission from a dedicated Tx Buffer or a Tx Queue Buffer by setting bit MCAN_TXBCR[n] CRn = 1 (where n = 0 - 31). The corresponding bit position n is equivalent to the number of the Tx buffer.

Transmit cancellation is not intended for Tx FIFO operation.

Successful cancellation is signaled by setting the corresponding bit of the MCAN_TXBCF register (MCAN_TXBCF[n] CFn = 1).

If transmission from a Tx Buffer is already ongoing and a transmit cancellation is requested, the corresponding MCAN_TXBRP[n] TRPn bit remains set as long as the transmission is in progress. If the transmission was successful, the corresponding MCAN_TXBTO[n] TOn and MCAN_TXBCF[n] CFn bits are set. If the transmission was not successful, only the corresponding bit MCAN_TXBCF[n] CFn = 1.

Note

If pending transmission is canceled immediately before this transmission could have been started, a short time window occurs where no transmission is started even if another message is also pending in this node. This may enable another node to transmit a message that may have a lower priority than the second message in this node.

20.4.15.8 Tx Event Handling

To support Tx Event Handling, the Message RAM has implemented a Tx Event FIFO section. Up to 32 Tx Event FIFO elements can be configured. Please see the Tx Event FIFO element chapter. After message transmission on the CAN bus, Message ID and Timestamp are stored in a Tx Event FIFO element. To link a Tx Event to a Tx Event FIFO element, the Message Marker from the transmitted Tx Buffer is copied into the Tx Event FIFO element.

A Tx Event FIFO full condition is signaled by the MCAN_IR.TEFF bit. In this case no further elements are written to the Tx Event FIFO until at least one element has been read out and the Tx Event FIFO Get Index has been incremented (MCAN_TXEFS.EFGI). In case a Tx Event occurs while the Tx Event FIFO is full, this event is rejected and interrupt flag MCAN_IR.TEFL bit is set.

The Tx Event FIFO watermark can be configured to avoid a Tx Event FIFO overflow. When the Tx Event FIFO fill level reaches the Tx Event FIFO watermark configured by the MCAN_TXEFC.EFWM field, interrupt flag MCAN_IR.TEFW is set. When reading from the Tx Event FIFO, two times the Tx Event FIFO Get Index MCAN_TXEFS.EFGI field has to be added to the Tx Event FIFO start address MCAN_TXEFC.EFSA field.

20.4.15.9 FIFO Acknowledge Handling

The Get Indices of the two Rx FIFOs (Rx FIFO 0 or Rx FIFO 1) and the Tx Event FIFO are controlled by writing to the corresponding FIFO Acknowledge Index (see MCAN_RXF0A, MCAN_RXF1A, and MCAN_TXEFA). Writing to the FIFO Acknowledge Index will set the FIFO Get Index to the FIFO Acknowledge Index plus one and thereby updates the FIFO Fill Level.

There are two use cases:

- A single element has been read from the FIFO: the Get Index value is written to the FIFO Acknowledge Index.
- A sequence of elements has been read from the FIFO: the Get Index value (Index of the last element read) is written to the FIFO Acknowledge Index at the end of that read sequence.

The Host CPU has free access to the Message RAM. The special care has to be taken when reading FIFO elements in an arbitrary order (Get Index not considered). This can be useful when reading a High Priority Message from one of the two Rx FIFOs. In this case the FIFO's Acknowledge Index should not be written because this would set the Get Index to a wrong position and also changes the FIFO's Fill Level. In this case some of the older FIFO elements would be lost.

Note

The application has to ensure that a valid value is written to the FIFO Acknowledge Index. The MCAN module does not check for erroneous values.

20.4.16 Rx Handling

The Rx Handler controls the following operations:

- Acceptance filtering
- The transfer of received messages to the Rx buffers or to one of the two Rx FIFOs (Rx FIFO 0 or Rx FIFO 1)
- Rx FIFO Put and Get Index operations

20.4.16.1 Acceptance Filtering

The MCAN module employs two sets of acceptance filters - one set for standard and one set for extended identifiers. These filters can be assigned to an Rx Buffer or to one of the two Rx FIFOs.

The main features of the filter elements are:

- Each filter element can be configured as:
 - Range filter (from - to)
 - Filter for specific IDs (for one or two dedicated IDs)
 - Classic bit mask filter
- Each filter element can be enabled/disabled individually
- Each filter element can be configured for acceptance or rejection filtering
- Filters are checked sequentially and execution (acceptance filtering procedure) stops at the first matching filter element or when the end of the filter list is reached

Related configuration registers are:

- Global Filter Configuration (MCAN_GFC) register
- Standard ID Filter Configuration (MCAN_SIDFC) register
- Extended ID Filter Configuration (MCAN_XIDFC) register
- Extended ID AND Mask (MCAN_XIDAM) register

Depending on the configuration of the filter element (see SFEC/EFEC in [Section 20.4.19](#)) if filter matches, one of the following actions is performed:

- Received frame is stored in FIFO 0 or FIFO 1
- Received frame is stored in Rx Buffer
- Received frame is stored in Rx Buffer and generation of pulse at filter event pin is performed. This is high level single MCAN_ICLK pulse.
- Received frame is rejected
- Set High Priority Message interrupt flag MCAN_IR.HPM
- Set High Priority Message interrupt flag MCAN_IR.HPM and store received frame in FIFO 0 or FIFO 1

Acceptance filtering starts when complete Message ID is received. Acceptance filtering stops at the first matching enabled filter element or when the end of the filter list is reached. If a filter element matches - the Rx Handler starts writing the received message data in portions of 32-bit to the matching Rx Buffer or Rx FIFO. If an error condition occurs (for example: CRC error), this message is rejected with the following impact on the affected Rx Buffer or Rx FIFO:

- Rx Buffer: New Data flag (MCAN_NDAT1/MCAN_NDAT2) of matching Rx Buffer is not set, but Rx Buffer (partly) overwritten with received data (for error type, see MCAN_PSR.LEC and MCAN_PSR.DLEC fields, respectively).
- Rx FIFO: Put index of matching Rx FIFO is not updated, but related Rx FIFO element (partly) overwritten with received data (for error type, see MCAN_PSR.LEC and MCAN_PSR.DLEC fields, respectively). If matching Rx FIFO is configured to operate in overwrite mode, the boundary conditions described in [Section 20.4.17.2](#) have to be considered.

Note

When an accepted message is written to one of the two Rx FIFOs, or into an Rx Buffer, the unmodified received identifier is stored independent of the filters used. The result of the acceptance filter process is strongly depending on the sequence of configured filter elements.

20.4.16.1.1 Range Filter

Each filter element can be configured to operate as Range Filter (Standard Filter Type SFT = 00/Extended Filter Type EFT = 00). The filter matches for all received message frames with IDs in the range from SFID1 to SFID2 (SFID2 ≥ SFID1) respectively in the range from EFID1 to EFID2 (EFID2 ≥ EFID1). For more information see [Section 20.4.19.5](#) and [Section 20.4.19.6](#).

There are two options for range filtering of extended frames:

- Extended Filter Type EFT = 00: The Extended ID AND Mask (MCAN_XIDAM) is used for Range Filtering. The Message ID of received frames is ANDed with the Extended ID AND Mask (MCAN_XIDAM) before the range filter is applied.
- Extended Filter Type EFT = 11: The Extended ID AND Mask (MCAN_XIDAM) is not used for Range Filtering.

20.4.16.1.2 Filter for Specific IDs

Each filter element can be configured to filter one or two dedicated Message IDs (Standard Filter Type SFT = 01/Extended Filter Type EFT = 01). To filter only one specific Message ID, the filter element has to be configured with SFID1 = SFID2 respectively EFID1 = EFID2. For more information, see [Section 20.4.19.5](#) and [Section 20.4.19.6](#).

20.4.16.1.3 Classic Bit Mask Filter

Classic bit mask filtering can filter groups of Message IDs (Standard Filter Type SFT = 10/Extended Filter Type EFT = 10). This is done by masking single bits of a received Message ID. In this case SFID1/EFID1 element is used as Message ID filter, while the SFID2/EFID2 element is used as filter mask.

A 0 bit at the filter mask (SFID2/EFID2) masks out the corresponding bit position of the configured Message ID filter (SFID1/EFID1) and the value of the received Message ID at that bit position is not relevant for acceptance filtering. Only those bits of the received Message ID where the corresponding mask bits are 1 are relevant for acceptance filtering.

There are two interesting cases:

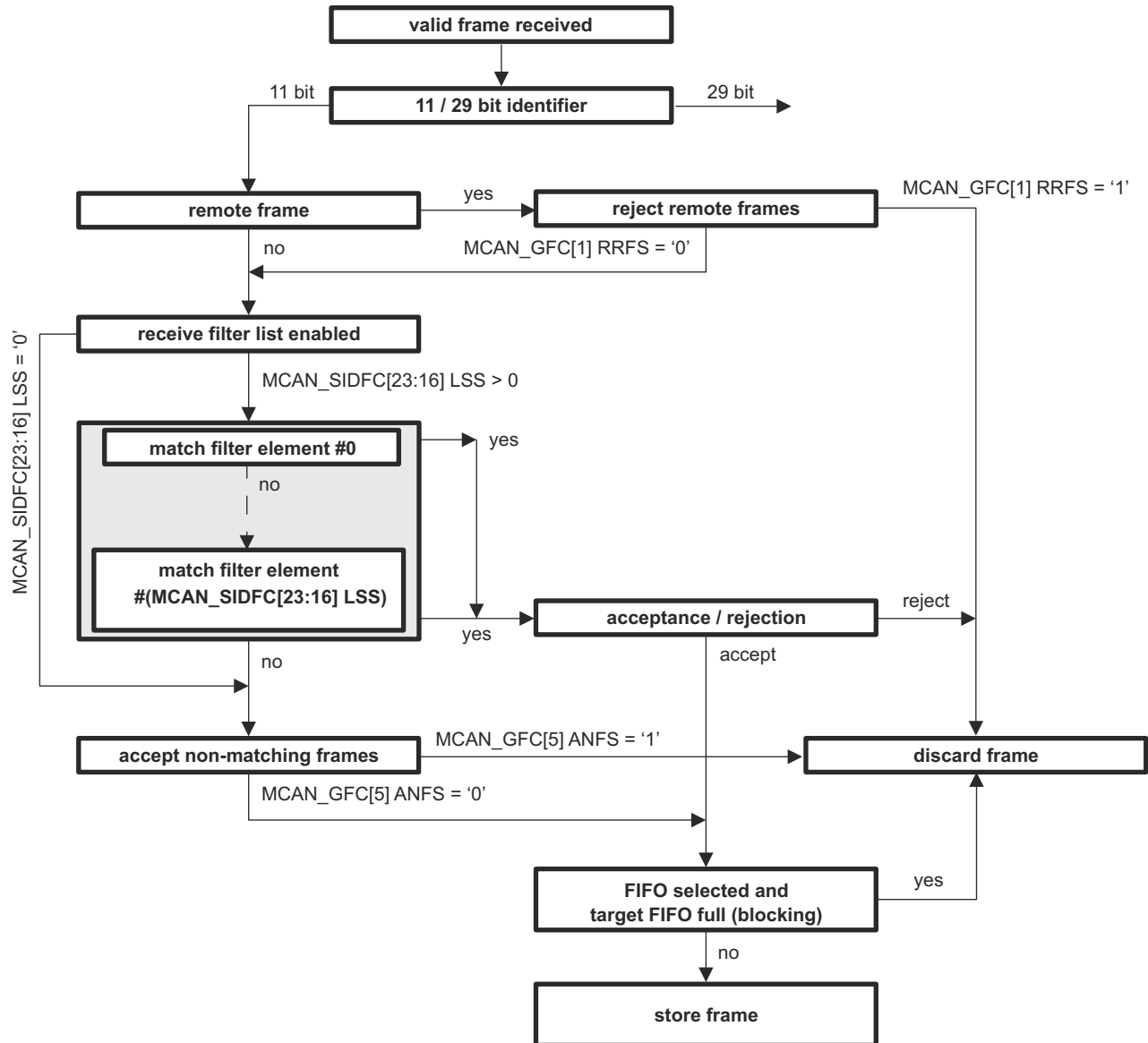
- All mask bits are 1: a match occurs only when the received Message ID and the configured Message ID filter are identical.
- All mask bits are 0: all Message IDs match.

20.4.16.1.4 Standard Message ID Filtering

Figure 20-16 shows the standard Message ID (11-bit ID) filtering flow. Section 20.4.19.5 describes the standard Message ID filter element.

The Remote Transmission Request (RTR) and Extended Identifier (XTD) bits of the received frames are compared against the list of configured filter elements. This is controlled by the following registers:

- Global Filter Configuration (MCAN_GFC) register
- Standard ID Filter Configuration (MCAN_SIDFC) register



mcan-009

Figure 20-16. Standard Message ID Filter Path

Note

In MCAN_GFC[1]RRFS and MCAN_GFC[5]ANFS; the [1] and [5] denotes the bit position for the RRFS and ANFS bits respectively in the MCAN_GFC register

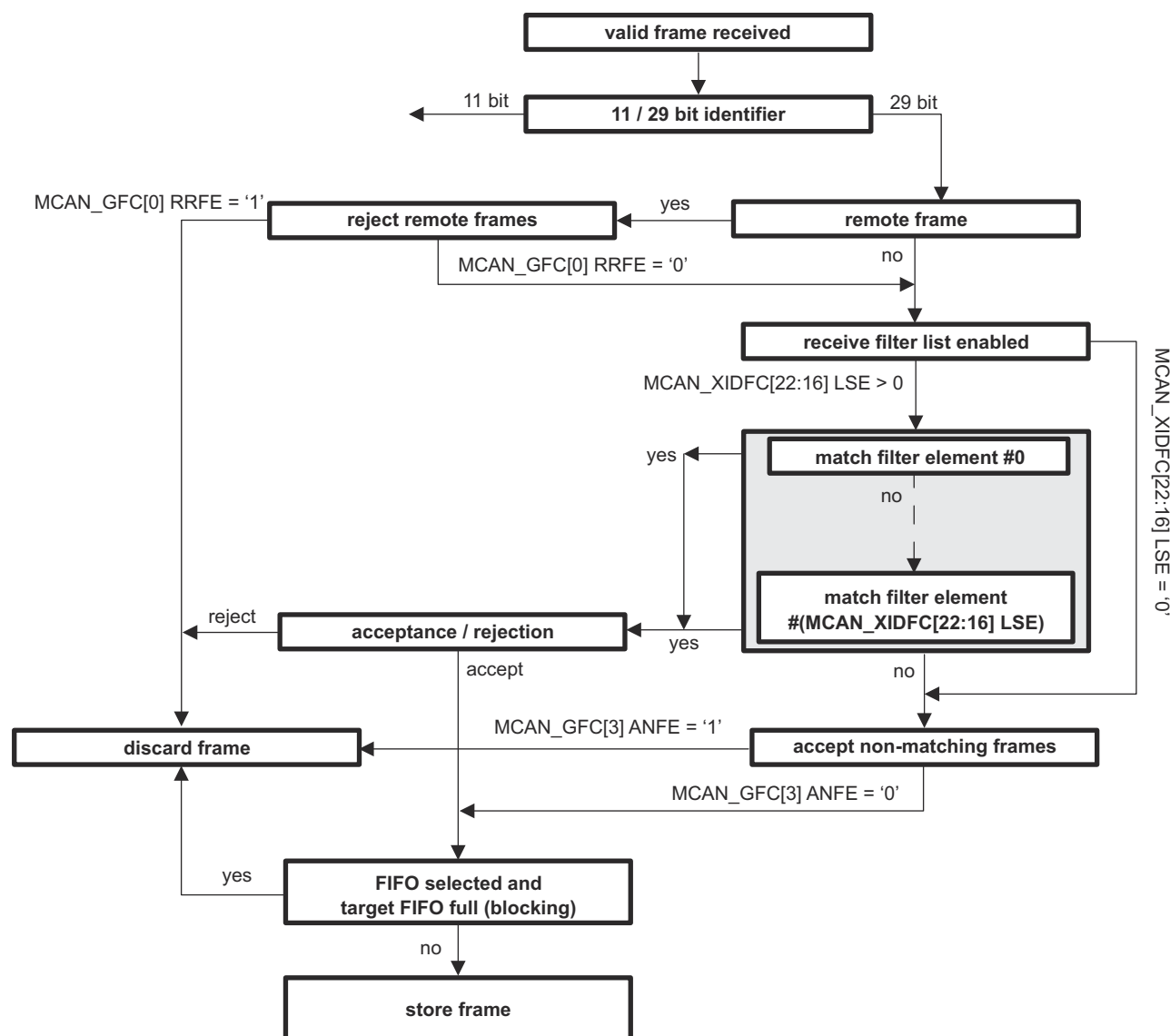
20.4.16.1.5 Extended Message ID Filtering

Figure 20-17 shows the extended Message ID (29-bit ID) filtering flow. Section 20.4.19.6 describes the extended Message ID filter element.

The Remote Transmission Request (RTR) and Extended Identifier (XTD) bits of the received frames are compared against the list of configured filter elements. This is controlled by the following registers:

- Global Filter Configuration (MCAN_GFC) register
- Extended ID Filter Configuration (MCAN_XIDFC) register

Note that before the filter list is executed, the received identifier is ANDed with the Extended ID AND Mask (MCAN_XIDAM).



mcan-010

Figure 20-17. Extended Message ID Filter Path

In MCAN_GFC[0]RRFS and MCAN_GFC[3]ANFS; the [0] and [3] denotes the bit position for the RRFS and ANFS bits respectively in the MCAN_GFC register

20.4.17 Rx FIFOs

The configuration of the Rx FIFOs (Rx FIFO 0 and Rx FIFO 1) can be done by way of the MCAN_RXF0C and MCAN_RXF1C registers. Each Rx FIFO can be configured to store up to 64 received messages.

After acceptance filtering the received messages that passed are transferred to the Rx FIFO. The filter mechanisms available for the Rx FIFO 0 and Rx FIFO 1 are described in [Section 20.4.16.1](#). The Rx FIFO element is described in [Section 20.4.19.2](#).

The Rx FIFO watermark can be used to prevent an Rx FIFO overflow. If the Rx FIFO fill level reaches the Rx FIFO watermark configured by the MCAN_RXFnC[30:24] FnWM filed (where: n = 0 or 1), an interrupt flag MCAN_IR.RF0W/MCAN_IR.RF1W is set.

When the Rx FIFO Put Index reaches the Rx FIFO Get Index (MCAN_RXFnS[21:16] FnPI = MCAN_RXFnS[13:8] FnGI), an Rx FIFO Full condition is signaled by the MCAN_RXFnS[24] FnF status bit and interrupt flag MCAN_IR.RF0F/MCAN_IR.RF1F is set. [Figure 20-18](#) shows Rx FIFO Status. The FIFOs fill level is presented in the MCAN_RXFnS[6:0] FnFL field (the number of elements stored in Rx FIFO).

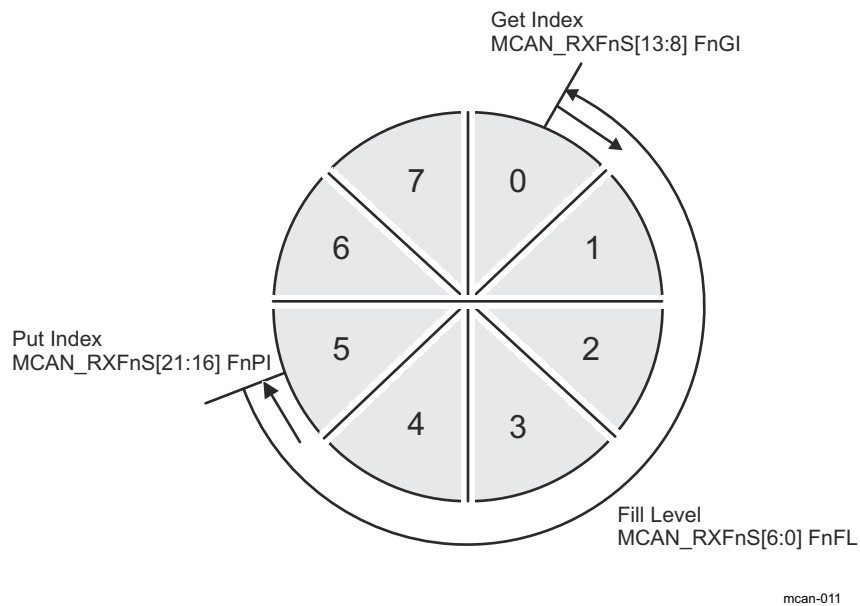


Figure 20-18. Rx FIFO Status

Rx FIFOs start address in the Message RAM (MCAN_RXFnC[15:2]FnSA field) has to be configured when reading from an Rx FIFO (Rx FIFO Get Index - MCAN_RXFnS[13:8] FnGI). [Table 20-7](#) presents Rx Buffer/Rx FIFO Element Size for different Rx Buffer / Rx FIFO Data Field Size which is configured by way of the MCAN_RXESC register.

Table 20-7. Rx Buffer/Rx FIFO Element Size

MCAN_RXESC Register RBDS/F0DS/F1DS Bits	Data Field [bytes]	FIFO Element Size [RAM words]
000	8	4
001	12	5
010	16	6
011	20	7
100	24	8
101	32	10
110	48	14
111	64	18

20.4.17.1 Rx FIFO Blocking Mode

The Rx FIFO blocking mode is the default operation mode for the Rx FIFOs and is configured by MCAN_RXFnC[31] FnOM = 0.

If an Rx FIFO full condition is reached (MCAN_RXFnS[21:16] FnPI = MCAN_RXFnS[13:8] FnGI), no further messages are written to the corresponding Rx FIFO until at least one message has been read out and the Rx FIFO Get Index has been incremented. An Rx FIFO full condition is signaled by the MCAN_RXFnS[24] FnF = 1 and interrupt flag MCAN_IR.RF0F/MCAN_IR.RF1F is set.

In case a message is received while the corresponding Rx FIFO is full, this message is rejected and the message lost condition is signaled by MCAN_RXFnS[25] RFnL = 1 and interrupt flag MCAN_IR.RF0L/MCAN_IR.RF1L is set.

20.4.17.2 Rx FIFO Overwrite Mode

The Rx FIFO overwrite mode is configured by MCAN_RXFnC[31] FnOM = 1. When an Rx FIFO full condition is reached (MCAN_RXFnS[21:16] FnPI = MCAN_RXFnS[13:8] FnGI) signaled by MCAN_RXFnS[24] FnF = 1, the next accepted message for the FIFO overwrites the oldest FIFO message. Put index/Get index are both incremented by one.

In overwrite mode if an Rx FIFO full condition is signaled, reading of the Rx FIFO elements starts at least at get index + 1. The reason for this is a received message is written to the Message RAM (Put index) while the Host CPU is reading from the Message RAM (Get index). In this case, inconsistent data can be read from the respective Rx FIFO element. The problem is solved by adding an offset to the Get index when reading from the Rx FIFO. The offset depends on how fast the Host CPU accesses the Rx FIFO. Figure 20-19 shows an offset of two with respect to the Get index when reading the Rx FIFO. In this case, the two messages stored in element 1 and 2 are lost.

After reading from the Rx FIFO, the number of the last element read has to be written to the Rx FIFO Acknowledge Index MCAN_RXFnA[5:0] FnAI. This increments the get index to that element number. In case the Put index has not been incremented to this Rx FIFO element, the Rx FIFO full condition is reset (MCAN_RXFnS[24] FnF = 0).

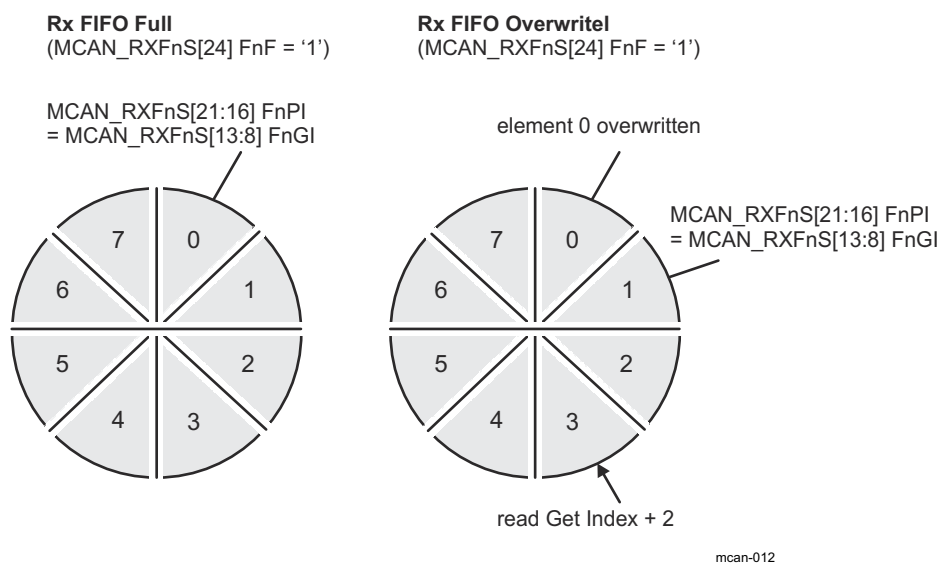


Figure 20-19. Rx FIFO Overflow Handling

20.4.18 Dedicated Rx Buffers

The MCAN supports up to 64 dedicated Rx buffers. The start address of the Rx buffers section in the Message RAM is configured by way of the MCAN_RXBC.RBSA field. To store in an Rx Buffer a Standard or Extended Message ID Filter Element with SFEC/EFEC = 111 and SFID2/EFID2[10:9] = 00 has to be configured (see [Section 20.4.19.5](#) and [Section 20.4.19.6](#)).

After a received message has been accepted by a filter element, the message is stored into the Rx Buffer in the Message RAM referenced by the filter element (the format is the same as for an Rx FIFO element). In addition, the flag MCAN_IR.DRX (message stored in Dedicated Rx Buffer) is set.

[Table 20-8](#) shows Example Filter Configuration for Rx buffers.

Table 20-8. Example Filter Configuration for Rx Buffers

Filter Element	SFID1[10:0] EFID1[28:0]	SFID2[10:9] EFID2[10:9]	SFID2[5:0] EFID2[5:0]
0	ID message 1	00	00 0000
1	ID message 2	00	00 0001
2	ID message 3	00	00 0010

After the last word of a matching received message has been written to the Message RAM, the respective New Data flag in register MCAN_NDAT1/MCAN_NDAT2 is set. As long as the New Data flag is set, the respective Rx Buffer is locked against updates from received matching frames. The New Data flags have to be reset by the Host CPU by writing a 1 to the respective bit position.

While an Rx buffer's New Data flag is set, a Message ID Filter Element referencing this specific Rx Buffer will not match, causing the acceptance filtering to continue. Following Message ID Filter Elements can cause the received message to be stored into another Rx Buffer, or into an Rx FIFO, or the message can be rejected, depending on filter configuration.

20.4.18.1 Rx Buffer Handling

Rx Buffer Handling include the following steps:

- Reset interrupt flag MCAN_IR.DRX
- Read New Data registers
- Read messages from Message RAM
- Reset New Data flags of processed messages

20.4.19 Message RAM

The MCAN module has a Message RAM. The main purpose of the Message RAM is to store:

- Received Messages
- Transmit Messages
- Tx Event Elements
- Message ID Filter Elements

20.4.19.1 Message RAM Configuration

The MCAN module can have different Message RAM sizes. An example of the MCAN module being configured for 1KB size with a width of 32 bits is described here.

The Message RAM can include each of the sections listed in [Message RAM Configuration](#). It is not necessary to configure each of the sections (a section in the Message RAM can be 0) and there is no restriction with respect to the sequence of the sections. For parity checking or ECC, a respective number of bits has to be added to each word. When the MCAN module addresses the Message RAM, it addresses 32-bit words. The start addresses are configurable and they are 32-bit word addresses.

The element size can be configured for:

- Rx FIFO 0, by way of the MCAN_RXESC.F0DS field
- Rx FIFO 1 by way of the MCAN_RXESC.F1DS field
- Rx buffers, by way of the MCAN_RXESC.RBDS field
- Tx buffers, by way of the MCAN_TXESC.TBDS field

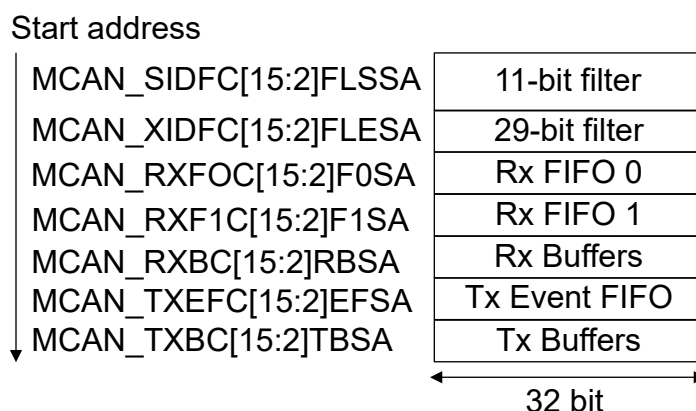


Figure 20-20. Message RAM Configuration

The host CPU configures the following information in the message RAM:

- Start addresses of the memory sections
- Number of elements in each section
- The size of the elements in some sections

Note

The MCAN module does not check for errors in the Message RAM configuration. The configuration of the start addresses of the different sections and the number of elements of each section has to be done carefully. This prevents falsification or loss of data.

An example of the max elements that can be accommodated for each section based on a RAM size of 1kB for the Rx FIFO is provided in [Message Transmission configurations](#) table

Table 20-9. Message Transmission configurations

Frame Size, DLC Code (Bytes)	Element Size (Words)	Maximum Elements
8	4	64
12	5	51
16	6	43
20	7	37
24	8	32
32	10	26
48	14	18

Table 20-9. Message Transmission configurations (continued)

Frame Size, DLC Code (Bytes)	Element Size (Words)	Maximum Elements
64	18	14

20.4.19.2 Rx Buffer and FIFO Element

Up to 64 Rx buffers and two Rx FIFOs can be configured in the Message RAM. Each Rx FIFO section can be configured to store up to 64 received messages. The element size can be configured for storage of CAN FD messages with up to 64 bytes data field by way of the MCAN_RXESC register.

Figure 20-21 shows the Rx Buffer/Rx FIFO element structure. Table 20-10 shows the Rx Buffer/Rx FIFO element field descriptions.

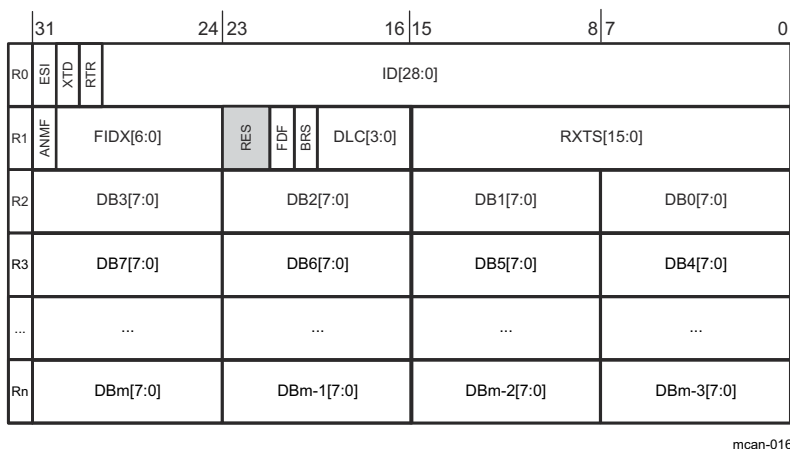


Figure 20-21. Rx Buffer/Rx FIFO Element Structure

Table 20-10. Rx Buffer/Rx FIFO Element Field Descriptions

Word	Bits	Field Name	Description
R0	31	ESI	Error State Indicator 0x0: Transmitting node is error active 0x1: Transmitting node is error passive
	30	XTD	Extended Identifier Signals to the Host CPU whether the received frame has a standard or extended identifier. 0x0: 11-bit standard identifier 0x1: 29-bit extended identifier
	29	RTR	Remote Transmission Request Signals to the Host CPU whether the received frame is a data frame or a remote frame. 0x0: Received frame is a data frame 0x1: Received frame is a remote frame Note: There are no remote frames in CAN FD format. In case a CAN FD frame was received (FDF = 1), RTR bit reflects the state of the reserved r1 bit (RES[23]). In CAN FD frames (FDF=1), the dominant RRS (Remote Request Substitution) bit replaces the RTR (Remote Transmission Request) bit.
	28:0	ID[28:0]	Identifier Standard or extended identifier depending on XTD bit. A standard identifier is stored into ID[28:18].

Table 20-10. Rx Buffer/Rx FIFO Element Field Descriptions (continued)

Word	Bits	Field Name	Description
R1	31	ANMF	Accepted Non-matching Frame Acceptance of non-matching frames can be enabled using the MCAN_GFC.ANFS and MCAN_GFC.ANFE fields. 0x0: Received frame matching filter index FIDX field 0x1: Received frame did not match any Rx filter element
	30:24	FIDX[6:0]	Filter Index 0x0-0x7F (0-127): Index of matching Rx acceptance filter element (invalid if ANMF = 1). Range is 0 to MCAN_SIDFC.LSS - 1 respectively MCAN_XIDFC.LSE - 1.
	23:22	RES	Reserved
	21	FDF	FD Format 0x0: Standard frame format 0x1: CAN FD frame format (new DLC-coding and CRC)
	20	BRS	Bit Rate Switch 0x0: Frame received without bit rate switching 0x1: Frame received with bit rate switching
	19:16	DLC[3:0]	Data Length Code 0x0-0x8 (0-8): CAN + CAN FD: received frame has 0-8 data bytes 0x9-0xF (9-15): CAN: received frame has 8 data bytes 0x9-0xF (9-15): CAN FD: received frame has 12/16/20/24/32/48/64 data bytes
R2	15:0	RXTS[15:0]	Rx Timestamp Timestamp Counter value captured on start of frame reception. Resolution depending on configuration of the Timestamp Counter Prescaler MCAN_TSCC.TCP.
	31:24	DB3[7:0]	Data Byte 3
	23:16	DB2[7:0]	Data Byte 2
	15:8	DB1[7:0]	Data Byte 1
R3	7:0	DB0[7:0]	Data Byte 0
	31:24	DB7[7:0]	Data Byte 7
	23:16	DB6[7:0]	Data Byte 6
	15:8	DB5[7:0]	Data Byte 5
Rn	7:0	DB4[7:0]	Data Byte 4
	...	...	...
	31:24	DBm[7:0]	Data Byte m
	23:16	DBm-1[7:0]	Data Byte m-1
Rn	15:8	DBm-2[7:0]	Data Byte m-2
	7:0	DBm-3[7:0]	Data Byte m-3

Note: Depending on the configuration of the element size (MCAN_RXESC), between two and sixteen 32-bit words (Rn = 3-17) are used for storage of a CAN message's data field.

20.4.19.3 Tx Buffer Element

The Tx buffers section can be configured to hold dedicated Tx buffers as well as a Tx FIFO/Tx Queue. In case that the Tx buffers section is shared by dedicated Tx buffers and a Tx FIFO/Tx Queue, the dedicated Tx buffers start at the beginning of the Tx buffers section followed by the buffers assigned to the Tx FIFO or Tx Queue. The Tx Handler makes difference between dedicated Tx buffers and Tx FIFO/Tx Queue by way of the MCAN_TXBC.TFQS and MCAN_TXBC.NDTB fields. The element size can be configured for storage of CAN FD messages with up to 64 bytes data field by way of the MCAN_TXESC register.

Figure 20-22 shows the Tx Buffer element structure. Table 20-11 shows the Tx Buffer element field descriptions.

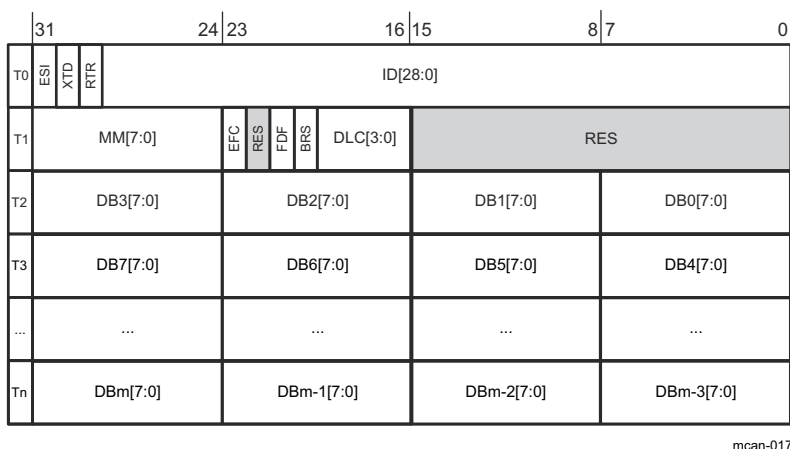


Figure 20-22. Tx Buffer Element Structure

Table 20-11. Tx Buffer Element Field Descriptions

Word	Bits	Field Name	Description
T0	31	ESI	Error State Indicator 0x0: ESI bit in CAN FD format depends only on error passive flag 0x1: ESI bit in CAN FD format transmitted recessive Note: The ESI bit of the transmit buffer is ORed with the error passive flag to decide the value of the ESI bit in the transmitted CAN FD frame. As required by the CAN FD protocol specification, an error active node can optionally transmit the ESI bit recessive, but an error passive node always transmits the ESI bit recessive.
	30	XTD	Extended Identifier 0x0: 11-bit standard identifier 0x1: 29-bit extended identifier
	29	RTR	Remote Transmission Request 0x0: Transmit data frame 0x1: Transmit remote frame Note: When RTR = 1, the MCAN module transmits a remote frame according to ISO11898-1:2015, even if the MCAN_CCCR.FDOE bit enables the transmission in CAN FD format.
	28:0	ID[28:0]	Identifier Standard or extended identifier depending on XTD bit. A standard identifier has to be written to ID[28:18].

Table 20-11. Tx Buffer Element Field Descriptions (continued)

Word	Bits	Field Name	Description
T1	31:24	MM[7:0]	Message Marker Written by Host CPU during Tx Buffer configuration. Copied into Tx Event FIFO element for identification of Tx message status (see also MM[7:0] field in Table 20-12).
	23	EFC	Event FIFO Control 0x0: Don't store Tx events 0x1: Store Tx events
	22	RES	Reserved
	21	FDF	FD Format 0x0: Frame transmitted in Classic CAN format 0x1: Frame transmitted in CAN FD format
	20	BRS	Bit Rate Switch 0x0: CAN FD frames transmitted without bit rate switching 0x1: CAN FD frames transmitted with bit rate switching Note: ESI, FDF, and BRS bits are only evaluated when CAN FD operation is enabled using the MCAN_CCCR.FDOE bit. BRS bit is only evaluated when MCAN_CCCR.BRSE = 1.
T2	19:16	DLC[3:0]	Data Length Code 0x0-0x8 (0-8): CAN + CAN FD: transmit frame has 0-8 data bytes 0x9-0xF (9-15): CAN: transmit frame has 8 data bytes 0x9-0xF (9-15): CAN FD: transmit frame has 12/16/20/24/32/48/64 data bytes
	15:0	RES	Reserved
	31:24	DB3[7:0]	Data Byte 3
	23:16	DB2[7:0]	Data Byte 2
	15:8	DB1[7:0]	Data Byte 1
T3	7:0	DB0[7:0]	Data Byte 0
	31:24	DB7[7:0]	Data Byte 7
	23:16	DB6[7:0]	Data Byte 6
	15:8	DB5[7:0]	Data Byte 5
Tn	7:0	DB4[7:0]	Data Byte 4
	...	...	...
	31:24	DBm[7:0]	Data Byte m
	23:16	DBm-1[7:0]	Data Byte m-1
Tn	15:8	DBm-2[7:0]	Data Byte m-2
	7:0	DBm-3[7:0]	Data Byte m-3

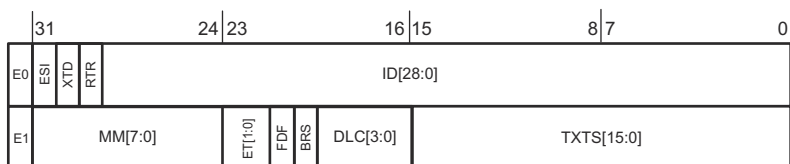
Note

Depending on the configuration of the element size (MCAN_TXESC), between two and sixteen 32-bit words (Tn = 3-17) are used for storage of a CAN message's data field.

20.4.19.4 Tx Event FIFO Element

Each element stores information about transmitted messages. By reading the Tx Event FIFO the Host CPU gets this information in the order the messages were transmitted. Status information about the Tx Event FIFO can be obtained from the MCAN_TXEFS register.

Figure 20-23 shows the Tx Event FIFO element structure. Table 20-12 shows the Tx Event FIFO element field descriptions.



mcan-018

Figure 20-23. Tx Event FIFO Element Structure

Table 20-12. Tx Event FIFO Element Field Descriptions

Word	Bits	Field Name	Description
E0	31	ESI	Error State Indicator 0x0: Transmitting node is error active 0x1: Transmitting node is error passive
	30	XTD	Extended Identifier 0x0: 11-bit standard identifier 0x1: 29-bit extended identifier
	29	RTR	Remote Transmission Request 0x0: Data frame transmitted 0x1: Remote frame transmitted
	28:0	ID[28:0]	Identifier Standard or extended identifier depending on XTD bit. A standard identifier has to be written to ID[28:18].

Table 20-12. Tx Event FIFO Element Field Descriptions (continued)

Word	Bits	Field Name	Description
E1	31:24	MM[7:0]	Message Marker Copied from Tx Buffer into Tx Event FIFO element for identification of Tx message status (see also MM[7:0] field in Table 20-11).
	23:22	ET[1:0]	Event Type 0x0: Reserved 0x1: Tx event 0x2: Transmission in spite of cancellation (always set for transmissions in DAR mode) 0x3: Reserved
	21	FDF	FD Format 0x0: Standard frame format 0x1: CAN FD frame format (new DLC-coding and CRC)
	20	BRS	Bit Rate Switch 0x0: Frame transmitted without bit rate switching 0x1: Frame transmitted with bit rate switching
	19:16	DLC[3:0]	Data Length Code 0x0-0x8 (0-8): CAN + CAN FD: frame with 0-8 data bytes transmitted 0x9-0xF (9-15): CAN: frame with 8 data bytes transmitted 0x9-0xF (9-15): CAN FD: frame with 12/16/20/24/32/48/64 data bytes transmitted
	15:0	TXTS[15:0]	Tx Timestamp Timestamp Counter value captured on start of frame transmission. Resolution depending on configuration of the Timestamp Counter Prescaler MCAN_TSCC.TCP filed.

20.4.19.5 Standard Message ID Filter Element

Up to 128 filter elements can be configured for 11-bit standard IDs. When accessing a Standard Message ID Filter element, the element address is the Filter List Standard Start Address MCAN_SIDFC.FLSSA field plus the index of the filter element (0-127).

[Figure 20-24](#) shows the Standard Message ID Filter element structure. [Table 20-13](#) shows the Standard Message ID Filter element field descriptions.

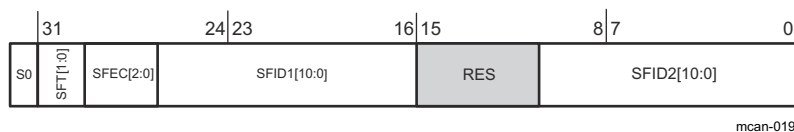

Figure 20-24. Standard Message ID Filter Element Structure

Table 20-13. Standard Message ID Filter Element Field Descriptions

Word	Bits	Field Name	Description
S0	31:30	SFT[1:0]	Standard Filter Type 0x0: Range filter from SFID1 to SFID2 (SFID2 ≥ SFID1) 0x1: Dual ID filter for SFID1 or SFID2 0x2: Classic filter: SFID1 = filter; SFID2 = mask 0x3: Filter element disabled Note: With SFT = 11 the filter element is disabled and the acceptance filtering continues (same behavior as with SFEC = 000)
	29:27	SFEC[2:0]	Standard Filter Element Configuration All enabled filter elements are used for acceptance filtering of standard frames. Acceptance filtering stops at the first matching enabled filter element or when the end of the filter list is reached. If SFEC = 100, 101, or 110 match sets interrupt flag MCAN_IR.HPM and, if enabled, an interrupt is generated. In this case, the MCAN_HPMS register is updated with the status of the priority match. 0x0: Disable filter element 0x1: Store in Rx FIFO 0 if filter matches 0x2: Store in Rx FIFO 1 if filter matches 0x3: Reject ID if filter matches 0x4: Set priority if filter matches 0x5: Set priority and store in FIFO 0 if filter matches 0x6: Set priority and store in FIFO 1 if filter matches 0x7: Store into Rx Buffer , configuration of SFT[1:0] ignored
	26:16	SFID1[10:0]	Standard Filter ID 1 When filtering for Rx buffers this field defines the ID of a standard message to be stored. The received identifiers must match exactly, no masking mechanism is used.
	15:11	RES	Reserved
	10:0	SFID2[10:0]	Standard Filter ID 2 This bit field has a different meaning depending on the configuration of SFEC: - SFEC = 001 - 110: Second ID of standard ID filter element - SFEC = 111: Filter for Rx buffers
		SFID2[10:9]	This field is decides whether the received message is stored into an Rx Buffer or treated as message A, B, or C of the debug message sequence. 0x0: Store message into an Rx Buffer 0x1: Debug Message A 0x2: Debug Message B 0x3: Debug Message C Note: Debug feature is not supported.
		SFID2[8:6]	This field is used to control the filter event pins at the Extension Interface. A one at the respective bit position enables generation of a pulse at the related filter event pin with the duration of one MCAN_ICKL period in case the filter matches. Note: Only two filter event pins are supported.
		SFID2[5:0]	This field defines the offset to the Rx Buffer Start Address MCAN_RXBC.RBSA field for storage of a matching message.

20.4.19.6 Extended Message ID Filter Element

Up to 64 filter elements can be configured for 29-bit extended IDs. When accessing an Extended Message ID Filter element, the element address is the Filter List Extended Start Address MCAN_XIDFC.FLESA field plus two times the index of the filter element (0-63).

Figure 20-25 shows the Extended Message ID Filter element structure. Table 20-14 shows the Extended Message ID Filter element field descriptions.

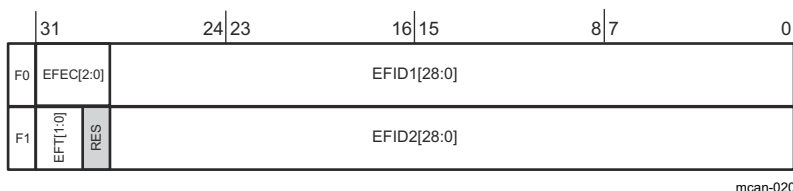


Figure 20-25. Extended Message ID Filter Element Structure

Table 20-14. Extended Message ID Filter Element Field Descriptions

Word	Bits	Field Name	Description
F0	31:29	EFEC[2:0]	Extended Filter Element Configuration All enabled filter elements are used for acceptance filtering of extended frames. Acceptance filtering stops at the first matching enabled filter element or when the end of the filter list is reached. If EFEC = 100, 101, or 110 match sets interrupt flag MCAN_IR.HPM and, if enabled, an interrupt is generated. In this case, the MCAN_HPMS register is updated with the status of the priority match. 0x0: Disable filter element 0x1: Store in Rx FIFO 0 if filter matches 0x2: Store in Rx FIFO 1 if filter matches 0x3: Reject ID if filter matches 0x4: Set priority if filter matches 0x5: Set priority and store in FIFO 0 if filter matches 0x6: Set priority and store in FIFO 1 if filter matches 0x7: Store into Rx Buffer or as debug message, configuration of EFT[1:0] ignored
	28:0	EFID1[28:0]	Extended Filter ID 1 First ID of extended ID filter element. When filtering for Rx buffers this field defines the ID of an extended message to be stored. The received identifiers must match exactly, only XIDAM masking mechanism (see Section 20.4.16.1.5) is used.

Table 20-14. Extended Message ID Filter Element Field Descriptions (continued)

Word	Bits	Field Name	Description
F1	31:30	EFT[1:0]	Extended Filter Type 0x0: Range filter from EFID1 to EFID2 (EFID2 ≥ EFID1) 0x1: Dual ID filter for EFID1 or EFID2 0x2: Classic filter: EFID1 = filter, EFID2 = mask 0x3: Range filter from EFID1 to EFID2 (EFID2 ≥ EFID1), XIDAM mask not applied
	29	RES	Reserved
	28:0	EFID2[28:0]	Extended Filter ID 2 This bit field has a different meaning depending on the configuration of EFEC: - EFEC = 001 - 110: Second ID of extended ID filter element - EFEC = 111: Filter for Rx buffers
		EFID2[10:9]	This field decides whether the received message is stored into an Rx Buffer or treated as message A, B, or C of the debug message sequence. 0x0: Store message into an Rx Buffer 0x1: Debug Message A 0x2: Debug Message B 0x3: Debug Message C Note: Debug feature is not supported.
		EFID2[8:6]	This field is used to control the filter event pins at the Extension Interface. A one at the respective bit position enables generation of a pulse at the related filter event pin with the duration of one MCAN_ICKL period in case the filter matches. Note: Only two filter event pins are supported.
		EFID2[5:0]	This field defines the offset to the Rx Buffer Start Address MCAN_RXBC.RBSA field for storage of a matching message.

20.5 MCAN Integration

Figure 20-26 shows the integration of the MCAN module in the device.

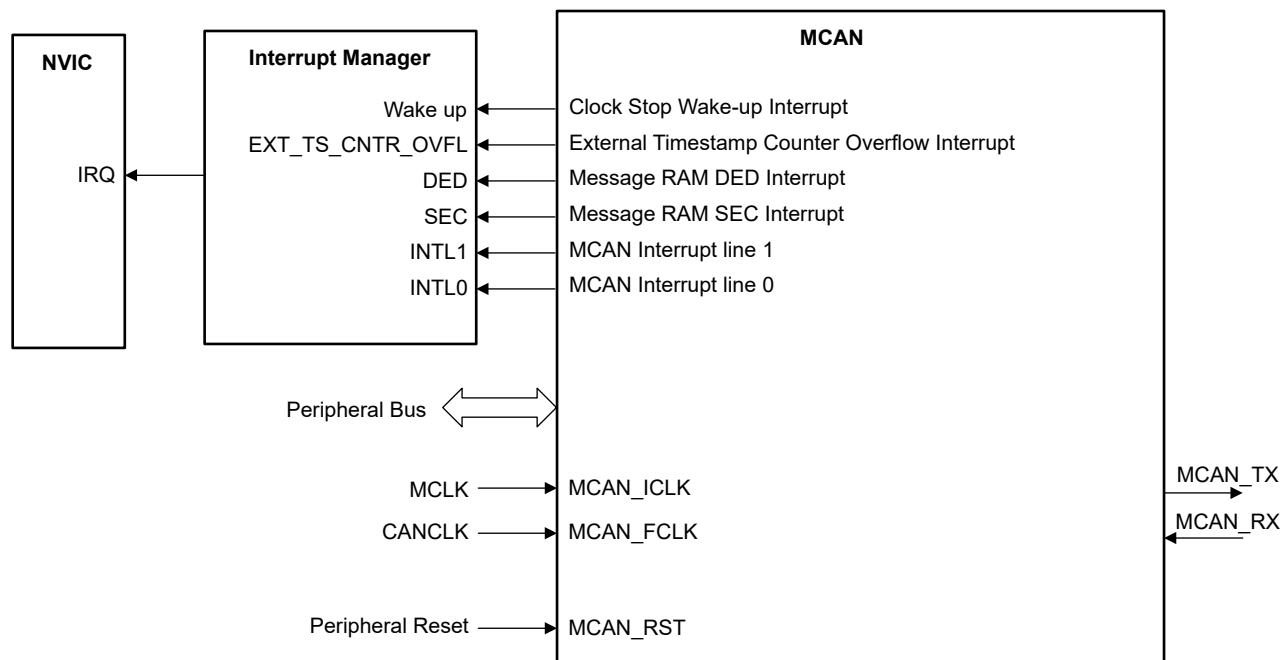


Figure 20-26. MCAN Integration

Table 20-15 summarizes the integration of the MCAN module in the device.

Table 20-15. MCAN Clocks and Resets

Destination Signal Name	Source Signal Name	Description
Clocks		
MCAN_ICLK	MCLK	Interface clock for the MCAN module
MCAN_FCLK	CANCLK	Bit timing clock for MCAN
Resets		
MCAN_RST	Peripheral Reset	Asynchronous reset signal to the MCAN module

20.6 Interrupt and Event Support

The MCAN module contains one [event publisher](#) (CPU_INT) that manages MCAN interrupt requests (IRQs) to the CPU subsystem via a [static event route](#).

The MCAN events are summarized in [Table 20-16](#).

Table 20-16. MCAN Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU interrupt	Publisher	MCAN	CPU Subsystem	Static route	CPU_INT registers	Fixed interrupt route from MCAN to CPU

20.6.1 CPU Interrupt Event Publisher (CPU_INT)

The MCAN module provides different interrupt sources which can be configured to source a [CPU interrupt event](#). In order of decreasing interrupt priority, the CPU interrupt events from the MCAN are shown in [Table 20-17](#).

Table 20-17. MCAN Interrupt Event Conditions (CPU_INT)

Index (IIDX)	Name	Description
0x1	MCAN_IRQ_INT0	MCAN interrupt 0
0x2	MCAN_IRQ_INT1	MCAN interrupt 1
0x3	MCAN_IRQ_ECC	MCAN ECC SEC (single error correct) interrupt

Table 20-17. MCAN Interrupt Event Conditions (CPU_INT) (continued)

Index (IIDX)	Name	Description
0x4	MCAN_IRQ_ECC_UNCORR	MCAN ECC uncorrectable / DED (double error detect) interrupt
0x5	MCANSS_IRQ_TS_CNTR_OVFL	MCAN timestamp counter overflow interrupt
0x6	MCANSS_IRQ_TS_WAKE	MCAN clock stop wakeup interrupt

The CPU interrupt event configuration is managed with the CPU_INT event management registers. Interrupt (RIS) flags are cleared upon software reading the IIDX register or writing to the respective ICLR register bits.

See [Section 7.2.5](#) for guidance on configuring the event registers for CPU interrupts.

20.7 MCAN Registers

This section describes the Modular Controller Area Network (MCAN) module registers.

Chapter 21

MCAN Registers



Table 21-1 lists the memory-mapped registers for the MCAN registers. All register offset addresses not listed in Table 21-1 should be considered as reserved locations and the register contents should not be modified.

Table 21-1. MCAN Registers

Offset	Acronym	Register Name	Group	Section
6004h	CANRX	CAN RX IO		Go
6008h	CANTX	CAN TX IO		Go
6204h	CANRX	FUPDATE version of CANRX		Go
6208h	CANTX	FUPDATE version of CANTX		Go
6480h	CPU_CONNECT_0	CPU Connect		Go
6800h	PWREN	Power enable		Go
6804h	RSTCTL	Reset Control		Go
6814h	STAT	Status Register		Go
7000h	MCAN_CREL	MCAN Core Release Register		Go
7004h	MCAN_ENDN	MCAN Endian Register		Go
700Ch	MCAN_DBTP	MCAN Data Bit Timing and Prescaler Register		Go
7010h	MCAN_TEST	MCAN Test Register		Go
7014h	MCAN_RWD	MCAN RAM Watchdog		Go
7018h	MCAN_CCCR	MCAN CC Control Register		Go
701Ch	MCAN_NBTP	MCAN Nominal Bit Timing and Prescaler Register		Go
7020h	MCAN_TSCC	MCAN Timestamp Counter Configuration		Go
7024h	MCAN_TSCV	MCAN Timestamp Counter Value		Go
7028h	MCAN_TOCC	MCAN Timeout Counter Configuration		Go
702Ch	MCAN_TOCV	MCAN Timeout Counter Value		Go
7040h	MCAN_ECR	MCAN Error Counter Register		Go
7044h	MCAN_PSR	MCAN Protocol Status Register		Go
7048h	MCAN_TDCR	MCAN Transmitter Delay Compensation Register		Go
7050h	MCAN_IR	MCAN Interrupt Register		Go
7054h	MCAN_IE	MCAN Interrupt Enable		Go
7058h	MCAN_ILS	MCAN Interrupt Line Select		Go
705Ch	MCAN_ILE	MCAN Interrupt Line Enable		Go
7080h	MCAN_GFC	MCAN Global Filter Configuration		Go
7084h	MCAN_SIDFC	MCAN Standard ID Filter Configuration		Go
7088h	MCAN_XIDFC	MCAN Extended ID Filter Configuration		Go
7090h	MCAN_XIDAM	MCAN Extended ID and Mask		Go
7094h	MCAN_HPMS	MCAN High Priority Message Status		Go

Table 21-1. MCAN Registers (continued)

Offset	Acronym	Register Name	Group	Section
7098h	MCAN_NDAT1	MCAN New Data 1	Go	
709Ch	MCAN_NDAT2	MCAN New Data 2	Go	
70A0h	MCAN_RXF0C	MCAN Rx FIFO 0 Configuration	Go	
70A4h	MCAN_RXF0S	MCAN Rx FIFO 0 Status	Go	
70A8h	MCAN_RXF0A	MCAN Rx FIFO 0 Acknowledge	Go	
70ACh	MCAN_RXBC	MCAN Rx Buffer Configuration	Go	
70B0h	MCAN_RXF1C	MCAN Rx FIFO 1 Configuration	Go	
70B4h	MCAN_RXF1S	MCAN Rx FIFO 1 Status	Go	
70B8h	MCAN_RXF1A	MCAN Rx FIFO 1 Acknowledge	Go	
70BCh	MCAN_RXESC	MCAN Rx Buffer / FIFO Element Size Configuration	Go	
70C0h	MCAN_TXBC	MCAN Tx Buffer Configuration	Go	
70C4h	MCAN_TXFQS	MCAN Tx FIFO / Queue Status	Go	
70C8h	MCAN_TXESC	MCAN Tx Buffer Element Size Configuration	Go	
70CCh	MCAN_TXBRP	MCAN Tx Buffer Request Pending	Go	
70D0h	MCAN_TXBAR	MCAN Tx Buffer Add Request	Go	
70D4h	MCAN_TXBCR	MCAN Tx Buffer Cancellation Request	Go	
70D8h	MCAN_TXBTO	MCAN Tx Buffer Transmission Occurred	Go	
70DCh	MCAN_TXBCF	MCAN Tx Buffer Cancellation Finished	Go	
70E0h	MCAN_TXBTIE	MCAN Tx Buffer Transmission Interrupt Enable	Go	
70E4h	MCAN_TXBCIE	MCAN Tx Buffer Cancellation Finished Interrupt Enable	Go	
70F0h	MCAN_TXEFC	MCAN Tx Event FIFO Configuration	Go	
70F4h	MCAN_TXEFS	MCAN Tx Event FIFO Status	Go	
70F8h	MCAN_TXEFA	MCAN Tx Event FIFO Acknowledge	Go	
7200h	MCANSS_PID	MCAN Subsystem Revision Register	Go	
7204h	MCANSS_CTRL	MCAN Subsystem Control Register	Go	
7208h	MCANSS_STAT	MCAN Subsystem Status Register	Go	
720Ch	MCANSS_ICS	MCAN Subsystem Interrupt Clear Shadow Register	Go	
7210h	MCANSS_IRS	MCAN Subsystem Interrupt Raw Status Register	Go	
7214h	MCANSS_IECS	MCAN Subsystem Interrupt Enable Clear Shadow Register	Go	
7218h	MCANSS_IE	MCAN Subsystem Interrupt Enable Register	Go	
721Ch	MCANSS_IES	MCAN Subsystem Interrupt Enable Status	Go	
7220h	MCANSS_EOI	MCAN Subsystem End of Interrupt	Go	
7224h	MCANSS_EXT_TS_PRESCALER	MCAN Subsystem External Timestamp Prescaler 0	Go	
7228h	MCANSS_EXT_TS_UNSERVICED_INTR_CNTR	MCAN Subsystem External Timestamp Unserved Interrupts Counter	Go	
7400h	MCANERR_REV	MCAN Error Aggregator Revision Register	Go	
7408h	MCANERR_VECTOR	MCAN ECC Vector Register	Go	
740Ch	MCANERR_STAT	MCAN Error Misc Status	Go	

Table 21-1. MCAN Registers (continued)

Offset	Acronym	Register Name	Group	Section
7410h	MCANERR_WRAP_REV	MCAN ECC Wrapper Revision Register		Go
7414h	MCANERR_CTRL	MCAN ECC Control		Go
7418h	MCANERR_ERR_CTRL1	MCAN ECC Error Control 1 Register		Go
741Ch	MCANERR_ERR_CTRL2	MCAN ECC Error Control 2 Register		Go
7420h	MCANERR_ERR_STAT1	MCAN ECC Error Status 1 Register		Go
7424h	MCANERR_ERR_STAT2	MCAN ECC Error Status 2 Register		Go
7428h	MCANERR_ERR_STAT3	MCAN ECC Error Status 3 Register		Go
743Ch	MCANERR_SEC_EOI	MCAN Single Error Corrected End of Interrupt Register		Go
7440h	MCANERR_SEC_STATUS	MCAN Single Error Corrected Interrupt Status Register		Go
7480h	MCANERR_SEC_ENABLE_SET	MCAN Single Error Corrected Interrupt Enable Set Register		Go
74C0h	MCANERR_SEC_ENABLE_CLR	MCAN Single Error Corrected Interrupt Enable Clear Register		Go
753Ch	MCANERR_DED_EOI	MCAN Double Error Detected End of Interrupt Register		Go
7540h	MCANERR_DED_STATUS	MCAN Double Error Detected Interrupt Status Register		Go
7580h	MCANERR_DED_ENABLE_SET	MCAN Double Error Detected Interrupt Enable Set Register		Go
75C0h	MCANERR_DED_ENABLE_CLR	MCAN Double Error Detected Interrupt Enable Clear Register		Go
7600h	MCANERR_AGGR_ENABLE_SET	MCAN Error Aggregator Enable Set Register		Go
7604h	MCANERR_AGGR_ENABLE_CLR	MCAN Error Aggregator Enable Clear Register		Go
7608h	MCANERR_AGGR_STATUS_SET	MCAN Error Aggregator Status Set Register		Go
760Ch	MCANERR_AGGR_STATUS_CLR	MCAN Error Aggregator Status Clear Register		Go
7820h	IIDX	Interrupt Index Register	CPU_INT	Go
7828h	IMASK	Interrupt mask	CPU_INT	Go
7830h	RIS	Raw interrupt status	CPU_INT	Go
7838h	MIS	Masked interrupt status	CPU_INT	Go
7840h	ISSET	Interrupt set	CPU_INT	Go
7848h	ICLR	Interrupt clear	CPU_INT	Go
78E0h	EVT_MODE	Event Mode		Go
78FCh	DESC	Module Description		Go
7900h	MCANSS_CLKEN	MCAN module clock enable		Go
7904h	MCANSS_CLKDIV	Clock divider		Go
7908h	MCANSS_CLKCTL	MCAN-SS clock stop control register		Go
790Ch	MCANSS_CLKSTS	MCANSS clock stop status register		Go

Complex bit access types are encoded to fit into small table cells. [Table 21-2](#) shows the codes that are used for access types in this section.

Table 21-2. MCAN Access Type Codes

Access Type	Code	Description
Read Type		

Table 21-2. MCAN Access Type Codes (continued)

Access Type	Code	Description
R	R	Read
R-0	R -0	Read Returns 0s
RC	R C	Read to Clear
RS	R S	Read to Set
Write Type		
W	W	Write
W1C	W 1C	Write 1 to clear
W1S	W 1S	Write 1 to set
W1SQ	W 1S Q	Write 1 to set Qualified. A condition must be met for this operation to occur.
WD	W D	Write Decrement. Decrements the specified bit field by the amount written.
WI	W I	Write Increment. Increments the specified bit field by the amount written.
WK	W K	Write Write protected by a key
WQ	W Q	Write Qualified. A condition must be met for this operation to occur.
Reset or Default Value		
-n		Value after reset or the default value

21.1 CANRX (Offset = 6004h) [Reset = 00000000h]

CANRX is shown in [Figure 21-1](#) and described in [Table 21-3](#).

Return to the [Summary Table](#).

CAN RX IO

Figure 21-1. CANRX

31	30	29	28	27	26	25	24
RESERVED	GFLT	SLEW	WCOMP	WUEN	INV	HIGHZ1	HIGHZ0
R-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
RESERVED		DRV		HYSTEN	INENA	PIPU	PIPD
R-0h		R/W-0h		R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
GSTATE							
R/W-0h							
7	6	5	4	3	2	1	0
PSTATE							
R/W-0h							

Table 21-3. CANRX Field Descriptions

Bit	Field	Type	Reset	Description
31	RESERVED	R	0h	
30	GFLT	R/W	0h	Glitch Filter Enable 0h = No internal glitch filter 1h = Use internal glitch filter
29	SLEW	R/W	0h	Reserved Slew Rate Control 0h = No Slew Rate Control 1h = Use Slew Rate Control
28	WCOMP	R/W	0h	Wake up compare value 0h = Match 0 will wake 1h = Match 1 will wake
27	WUEN	R/W	0h	Wake up enable 0h = Wake up not enabled 1h = Wake up enabled
26	INV	R/W	0h	Invert digital input/output relative to peripheral/GPIO 0h = Input and output are non-inverted 1h = Input and output are inverted
25	HIGHZ1	R/W	0h	High-Z instead of high output 0h = Pin can be driven high 1h = Pin is tri-stated instead of driven high
24	HIGHZ0	R/W	0h	High-Z instead of low output 0h = Pin can be driven low 1h = Pin is tri-stated instead of driven low
23	RESERVED	R	0h	
22-20	DRV	R/W	0h	Drive strength options 0h = Lowest drive strength 1h = Drive strength 2/8 2h = Drive strength 3/8 3h = Drive strength 4/8 4h = Drive strength 5/8 5h = Drive strength 6/8 6h = Drive strength 7/8 7h = Highest drive strength

Table 21-3. CANRX Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	HYSTEN	R/W	0h	Hysteresis enable 0h = No hysteresis 1h = Hysteresis on
18	INENA	R/W	0h	Input enable 0h = Inputs 0 to connected core 1h = Inputs IO pad value to connected core
17	PIPU	R/W	0h	Pull up enable 0h = No pull up 1h = Pull up
16	PIPD	R/W	0h	Pull down enable 0h = No pull down 1h = Pull down
15-14	GSTATE	R/W	0h	GPIO Channel State 0h = G-Channel is in Unassigned State 1h = G-Channel is in Handover State 2h = G-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = G-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
13-8	RESERVED	R	0h	
7-6	PSTATE	R/W	0h	Peripheral-Analog Channel State 0h = P-Channel is in Unassigned State 1h = P-Channel is in Handover State 2h = P-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = P-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
5-0	RESERVED	R	0h	

21.2 CANTX (Offset = 6008h) [Reset = 00000000h]

CANTX is shown in [Figure 21-2](#) and described in [Table 21-4](#).

Return to the [Summary Table](#).

CAN TX IO

Figure 21-2. CANTX

31	30	29	28	27	26	25	24
RESERVED	GFLT	SLEW	WCOMP	WUEN	INV	HIGHZ1	HIGHZ0
R-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
RESERVED	DRV			HYSTEN	INENA	PIPU	PIPD
R-0h	R/W-0h			R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
GSTATE		RESERVED					
R/W-0h		R-0h					
7	6	5	4	3	2	1	0
PSTATE		RESERVED					
R/W-0h		R-0h					

Table 21-4. CANTX Field Descriptions

Bit	Field	Type	Reset	Description
31	RESERVED	R	0h	
30	GFLT	R/W	0h	Glitch Filter Enable 0h = No internal glitch filter 1h = Use internal glitch filter
29	SLEW	R/W	0h	Reserved Slew Rate Control 0h = No Slew Rate Control 1h = Use Slew Rate Control
28	WCOMP	R/W	0h	Wake up compare value 0h = Match 0 will wake 1h = Match 1 will wake
27	WUEN	R/W	0h	Wake up enable 0h = Wake up not enabled 1h = Wake up enabled
26	INV	R/W	0h	Invert digital input/output relative to peripheral/GPIO 0h = Input and output are non-inverted 1h = Input and output are inverted
25	HIGHZ1	R/W	0h	High-Z instead of high output 0h = Pin can be driven high 1h = Pin is tri-stated instead of driven high
24	HIGHZ0	R/W	0h	High-Z instead of low output 0h = Pin can be driven low 1h = Pin is tri-stated instead of driven low
23	RESERVED	R	0h	
22-20	DRV	R/W	0h	Drive strength options 0h = Lowest drive strength 1h = Drive strength 2/8 2h = Drive strength 3/8 3h = Drive strength 4/8 4h = Drive strength 5/8 5h = Drive strength 6/8 6h = Drive strength 7/8 7h = Highest drive strength

Table 21-4. CANTX Field Descriptions (continued)

Bit	Field	Type	Reset	Description
19	HYSTEN	R/W	0h	Hysteresis enable 0h = No hysteresis 1h = Hysteresis on
18	INENA	R/W	0h	Input enable 0h = Inputs 0 to connected core 1h = Inputs IO pad value to connected core
17	PIPU	R/W	0h	Pull up enable 0h = No pull up 1h = Pull up
16	PIPD	R/W	0h	Pull down enable 0h = No pull down 1h = Pull down
15-14	GSTATE	R/W	0h	GPIO Channel State 0h = G-Channel is in Unassigned State 1h = G-Channel is in Handover State 2h = G-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = G-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
13-8	RESERVED	R	0h	
7-6	PSTATE	R/W	0h	Peripheral-Analog Channel State 0h = P-Channel is in Unassigned State 1h = P-Channel is in Handover State 2h = P-Channel is in Connected State and not Locked (That is F field is allowed to change without going back through Unassigned state) 3h = P-Channel is in Connected State and Locked (That is F field is not allowed to change to a different non-Zero value until both G and P channels go to Unassigned)
5-0	RESERVED	R	0h	

21.3 CANRX (Offset = 6204h) [Reset = 00000000h]

CANRX is shown in [Figure 21-3](#) and described in [Table 21-5](#).

Return to the [Summary Table](#).

FUPDATE version of CANRX

Figure 21-3. CANRX

31	30	29	28	27	26	25	24
RESERVED				IOADDR			
R-0h				W-0h			
23	22	21	20	19	18	17	16
IOADDR							
W-0h							
15	14	13	12	11	10	9	8
IOADDR							
W-0h							
7	6	5	4	3	2	1	0
IOADDR						LOCK	GSEL
W-0h						W-0h	W-0h

Table 21-5. CANRX Field Descriptions

Bit	Field	Type	Reset	Description
31-28	RESERVED	R	0h	
27-2	IOADDR	W	0h	IO Address. This is the address that corresponds to the SOC address[27:2] of the module IP instance specific IO signal in the "Full Write" subregion of the pinmux subregion.
1	LOCK	W	0h	Sets lock bit 0h = Writing this value has no effect 1h = Set channel lock bit
0	GSEL	W	0h	GPIO channel Select 0: Select the P-Channel for the F update 1: Select the G-Channel for the F update 0h = Select the P-Channel for the F update 1h = Select the G-Channel for the F update

21.4 CANTX (Offset = 6208h) [Reset = 00000000h]

CANTX is shown in [Figure 21-4](#) and described in [Table 21-6](#).

Return to the [Summary Table](#).

FUPDATE version of CANTX

Figure 21-4. CANTX

31	30	29	28	27	26	25	24
RESERVED				IOADDR			
R-0h				W-0h			
23	22	21	20	19	18	17	16
IOADDR							
W-0h							
15	14	13	12	11	10	9	8
IOADDR							
W-0h							
7	6	5	4	3	2	1	0
IOADDR						LOCK	GSEL
W-0h						W-0h	W-0h

Table 21-6. CANTX Field Descriptions

Bit	Field	Type	Reset	Description
31-28	RESERVED	R	0h	
27-2	IOADDR	W	0h	IO Address. This is the address that corresponds to the SOC address[27:2] of the module IP instance specific IO signal in the "Full Write" subregion of the pinmux subregion.
1	LOCK	W	0h	Sets lock bit 0h = Writing this value has no effect 1h = Set channel lock bit
0	GSEL	W	0h	GPIO channel Select 0: Select the P-Channel for the F update 1: Select the G-Channel for the F update 0h = Select the P-Channel for the F update 1h = Select the G-Channel for the F update

21.5 CPU_CONNECT_0 (Offset = 6480h) [Reset = 00000000h]

CPU_CONNECT_0 is shown in [Figure 21-5](#) and described in [Table 21-7](#).

Return to the [Summary Table](#).

Directly connect peripheral publisher port to application processor

Figure 21-5. CPU_CONNECT_0

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED						CPUSS0_CON N	RESERVED
R-0h						R/W-0h	R-0h

Table 21-7. CPU_CONNECT_0 Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1	CPUSS0_CONN	R/W	0h	CPUSS0 connect bit. 0h = The CPU is not connected. 1h = The CPU is connected.
0	RESERVED	R	0h	

21.6 PWREN (Offset = 6800h) [Reset = 00000000h]

PWREN is shown in [Figure 21-6](#) and described in [Table 21-8](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 21-6. PWREN

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R-0h							R/WK-0h

Table 21-8. PWREN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R	0h	
0	ENABLE	R/WK	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

21.7 RSTCTL (Offset = 6804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 21-7](#) and described in [Table 21-9](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 21-7. RSTCTL

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCLR	RESETASSERT
R-0h						R	WK-0h
						WK-0h	WK-0h

Table 21-9. RSTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	R	0h	
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

21.8 STAT (Offset = 6814h) [Reset = 00000000h]

STAT is shown in [Figure 21-8](#) and described in [Table 21-10](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 21-8. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-0h							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							
R-0h							

Table 21-10. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0x0	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

21.9 MCAN_CREL (Offset = 7000h) [Reset = 32380608h]

MCAN_CREL is shown in [Figure 21-9](#) and described in [Table 21-11](#).

Return to the [Summary Table](#).

MCAN Core Release Register

Figure 21-9. MCAN_CREL

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
REL				STEP				SUBSTEP				YEAR			
R-3h				R-2h				R-3h				R-8h			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MON								DAY							
R-6h								R-8h							

Table 21-11. MCAN_CREL Field Descriptions

Bit	Field	Type	Reset	Description
31-28	REL	R	3h	Core Release. One digit, BCD-coded.
27-24	STEP	R	2h	Step of Core Release. One digit, BCD-coded.
23-20	SUBSTEP	R	3h	Sub-Step of Core Release. One digit, BCD-coded.
19-16	YEAR	R	8h	Time Stamp Year. One digit, BCD-coded.
15-8	MON	R	6h	Time Stamp Month. Two digits, BCD-coded.
7-0	DAY	R	8h	Time Stamp Day. Two digits, BCD-coded.

21.10 MCAN_ENDN (Offset = 7004h) [Reset = 87654321h]

MCAN_ENDN is shown in [Figure 21-10](#) and described in [Table 21-12](#).

Return to the [Summary Table](#).

MCAN Endian Register

Figure 21-10. MCAN_ENDN

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETV																															
R-87654321h																															

Table 21-12. MCAN_ENDN Field Descriptions

Bit	Field	Type	Reset	Description
31-0	ETV	R	87654321h	Endianness Test Value. Reading the constant value maintained in this register allows software to determine the endianness of the host CPU.

21.11 MCAN_DBTP (Offset = 700Ch) [Reset = 0000A33h]

MCAN_DBTP is shown in [Figure 21-11](#) and described in [Table 21-13](#).

Return to the [Summary Table](#).

This register is only writable if bits CCCR.CCE and CCCR.INIT are set. The CAN bit time may be programmed in the range of 4 to 49 time quanta. The CAN time quantum may be programmed in the range of 1 to 32 m_can_cclk periods. $tq = (DBRP + 1) mtq$.

DTSEG1 is the sum of Prop_Seg and Phase_Seg1. DTSEG2 is Phase_Seg2.

Therefore the length of the bit time is (programmed values) (DTSEG1 + DTSEG2 + 3) tq or (functional values) (Sync_Seg + Prop_Seg + Phase_Seg1 + Phase_Seg2) tq.

The Information Processing Time (IPT) is zero, meaning the data for the next bit is available at the first clock edge after the sample point.

Figure 21-11. MCAN_DBTP

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
TDC	RESERVED				DBRP		
R/WQ-0h	R-0h				R/WQ-0h		
15	14	13	12	11	10	9	8
RESERVED				DTSEG1			
R-0h				R/WQ-Ah			
7	6	5	4	3	2	1	0
DTSEG2				DSJW			
R/WQ-3h				R/WQ-3h			

Table 21-13. MCAN_DBTP Field Descriptions

Bit	Field	Type	Reset	Description
31-24	RESERVED	R	0h	
23	TDC	R/WQ	0h	Transmitter Delay Compensation 0 Transmitter Delay Compensation disabled 1 Transmitter Delay Compensation enabled
22-21	RESERVED	R	0h	
20-16	DBRP	R/WQ	0h	Data Bit Rate Prescaler. The value by which the oscillator frequency is divided for generating the bit time quanta. The bit time is built up from a multiple of this quanta. Valid values for the Bit Rate Prescaler are 0 to 31. The actual interpretation by the hardware of this value is such that one more than the value programmed here is used. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
15-13	RESERVED	R	0h	
12-8	DTSEG1	R/WQ	Ah	Data Time Segment Before Sample Point. Valid values are 0 to 31. The actual interpretation by the hardware of this value is such that one more than the programmed value is used. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
7-4	DTSEG2	R/WQ	3h	Data Time Segment After Sample Point. Valid values are 0 to 15. The actual interpretation by the hardware of this value is such that one more than the programmed value is used. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.

Table 21-13. MCAN_DBTP Field Descriptions (continued)

Bit	Field	Type	Reset	Description
3-0	DSJW	R/WQ	3h	Data Resynchronization Jump Width. Valid values are 0 to 15. The actual interpretation by the hardware of this value is such that one more than the value programmed here is used. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.

21.12 MCAN_TEST (Offset = 7010h) [Reset = 000000X0h]

MCAN_TEST is shown in [Figure 21-12](#) and described in [Table 21-14](#).

Return to the [Summary Table](#).

Write access to the Test Register has to be enabled by setting bit CCCR.TEST to '1'. All Test Register functions are set to their reset values when bit CCCR.TEST is reset.

Loop Back Mode and software control of the internal CAN TX pin are hardware test modes. Programming of TX ? '00' may disturb the message transfer on the CAN bus.

Figure 21-12. MCAN_TEST

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RX	TX		LBCK	RESERVED			
R-X	R/WQ-0h		R/WQ-0h	R-0h			

Table 21-14. MCAN_TEST Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7	RX	R	X	Receive Pin. Monitors the actual value of the CAN receive pin. 0h = DOMINANT : The CAN bus is dominant (CAN RX pin = '0') 1h = RECESSIVE : The CAN bus is recessive (CAN RX pin = '1')
6-5	TX	R/WQ	0h	Control of Transmit Pin 00 CAN TX pin controlled by the CAN Core, updated at the end of the CAN bit time 01 Sample Point can be monitored at CAN TX pin 10 Dominant ('0') level at CAN TX pin 11 Recessive ('1') at CAN TX pin Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
4	LBCK	R/WQ	0h	Loop Back Mode. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'. 0h = DISABLE : Reset value, Loop Back Mode is disabled 1h = ENABLE : Loop Back Mode is enabled
3-0	RESERVED	R	0h	

21.13 MCAN_RWD (Offset = 7014h) [Reset = 00000000h]

MCAN_RWD is shown in [Figure 21-13](#) and described in [Table 21-15](#).

Return to the [Summary Table](#).

MCAN RAM Watchdog

Figure 21-13. MCAN_RWD

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																WDV								WDC							
R-0h																R-0h								R/WQ-0h							

Table 21-15. MCAN_RWD Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15-8	WDV	R	0h	Watchdog Value. Actual Message RAM Watchdog Counter Value. The RAM Watchdog monitors the READY output of the Message RAM. A Message RAM access via the MCAN's Generic Master Interface starts the Message RAM Watchdog Counter with the value configured by the WDC field. The counter is reloaded with WDC when the Message RAM signals successful completion by activating its READY output. In case there is no response from the Message RAM until the counter has counted down to zero, the counter stops and interrupt flag MCAN_IR.WDI is set. The RAM Watchdog Counter is clocked by the host (system) clock.
7-0	WDC	R/WQ	0h	Watchdog Configuration. Start value of the Message RAM Watchdog Counter. With the reset value of '00' the counter is disabled. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.

21.14 MCAN_CCCR (Offset = 7018h) [Reset = 00000001h]

MCAN_CCCR is shown in [Figure 21-14](#) and described in [Table 21-16](#).

Return to the [Summary Table](#).

MCAN CC Control Register

Figure 21-14. MCAN_CCCR

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
NISO	TXP	EFBI	PXHD	RESERVED		BRSE	FDOE
R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R-0h		R/WQ-0h	R/WQ-0h
7	6	5	4	3	2	1	0
TEST	DAR	MON	CSR	CSA	ASM	CCE	INIT
R/W1SQ-0h	R/WQ-0h	R/W1SQ-0h	R/W-0h	R-0h	R/W1SQ-0h	R/WQ-0h	R/W-1h

Table 21-16. MCAN_CCCR Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15	NISO	R/WQ	0h	Non ISO Operation. If this bit is set, the MCAN uses the CAN FD frame format as specified by the Bosch CAN FD Specification V1.0. 0 CAN FD frame format according to ISO 11898-1:2015 1 CAN FD frame format according to Bosch CAN FD Specification V1.0
14	TXP	R/WQ	0h	Transmit Pause. If this bit is set, the MCAN pauses for two CAN bit times before starting the next transmission after itself has successfully transmitted a frame. 0 Transmit pause disabled 1 Transmit pause enabled Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
13	EFBI	R/WQ	0h	Edge Filtering during Bus Integration 0 Edge filtering disabled 1 Two consecutive dominant tq required to detect an edge for hard synchronization Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
12	PXHD	R/WQ	0h	Protocol Exception Handling Disable 0 Protocol exception handling enabled 1 Protocol exception handling disabled Note: When protocol exception handling is disabled, the MCAN will transmit an error frame when it detects a protocol exception condition. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
11-10	RESERVED	R	0h	

Table 21-16. MCAN_CCCR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
9	BRSE	R/WQ	0h	Bit Rate Switch Enable 0 Bit rate switching for transmissions disabled 1 Bit rate switching for transmissions enabled Note: When CAN FD operation is disabled FDOE = '0', BRSE is not evaluated. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
8	FDOE	R/WQ	0h	Flexible Datarate Operation Enable 0 FD operation disabled 1 FD operation enabled Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
7	TEST	R/W1SQ	0h	Test Mode Enable 0 Normal operation, register TEST holds reset values 1 Test Mode, write access to register TEST enabled Qualified Write 1 to Set is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
6	DAR	R/WQ	0h	Disable Automatic Retransmission 0 Automatic retransmission of messages not transmitted successfully enabled 1 Automatic retransmission disabled Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
5	MON	R/W1SQ	0h	Bus Monitoring Mode. Bit MON can only be set by SW when both CCE and INIT are set to '1'. The bit can be reset by SW at any time. 0 Bus Monitoring Mode is disabled 1 Bus Monitoring Mode is enabled Qualified Write 1 to Set is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
4	CSR	R/W	0h	Clock Stop Request 0 No clock stop is requested 1 Clock stop requested. When clock stop is requested, first INIT and then CSA will be set after all pending transfer requests have been completed and the CAN bus reached idle.
3	CSA	R	0h	Clock Stop Acknowledge 0 No clock stop acknowledged 1 MCAN may be set in power down by stopping the Host and CAN clocks
2	ASM	R/W1SQ	0h	Restricted Operation Mode. Bit ASM can only be set by SW when both CCE and INIT are set to '1'. The bit can be reset by SW at any time. 0 Normal CAN operation 1 Restricted Operation Mode active Qualified Write 1 to Set is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
1	CCE	R/WQ	0h	Configuration Change Enable 0 The CPU has no write access to the protected configuration registers 1 The CPU has write access to the protected configuration registers (while CCCR.INIT = '1') Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
0	INIT	R/W	1h	Initialization 0 Normal Operation 1 Initialization is started Note: Due to the synchronization mechanism between the two clock domains, there may be a delay until the value written to INIT can be read back. Therefore the programmer has to assure that the previous value written to INIT has been accepted by reading INIT before setting INIT to a new value.

21.15 MCAN_NBTP (Offset = 701Ch) [Reset = 06000A03h]

MCAN_NBTP is shown in [Figure 21-15](#) and described in [Table 21-17](#).

Return to the [Summary Table](#).

This register is only writable if bits CCCR.CCE and CCCR.INIT are set. The CAN bit time may be programmed in the range of 4 to 385 time quanta. The CAN time quantum may be programmed in the range of 1 to 512 m_can_cclk periods. $tq = (NBRP + 1) mtq$.

NTSEG1 is the sum of Prop_Seg and Phase_Seg1. NTSEG2 is Phase_Seg2.

Therefore the length of the bit time is (programmed values) (NTSEG1 + NTSEG2 + 3) tq or (functional values) (Sync_Seg + Prop_Seg + Phase_Seg1 + Phase_Seg2) tq.

The Information Processing Time (IPT) is zero, meaning the data for the next bit is available at the first clock edge after the sample point.

Note: With a CAN clock of 8 MHz, the reset value of 0x06000A03 configures the MCAN for a bit rate of 500 kBit/s.

Figure 21-15. MCAN_NBTP

31	30	29	28	27	26	25	24
NSJW							NBRP
R/WQ-3h							R/WQ-0h
23	22	21	20	19	18	17	16
NBRP							
R/WQ-0h							
15	14	13	12	11	10	9	8
NTSEG1							
R/WQ-Ah							
7	6	5	4	3	2	1	0
RESERVED	NTSEG2						
R-0h	R/WQ-3h						

Table 21-17. MCAN_NBTP Field Descriptions

Bit	Field	Type	Reset	Description
31-25	NSJW	R/WQ	3h	Nominal (Re)Synchronization Jump Width. Valid values are 0 to 127. The actual interpretation by the hardware of this value is such that one more than the value programmed here is used. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
24-16	NBRP	R/WQ	0h	Nominal Bit Rate Prescaler. The value by which the oscillator frequency is divided for generating the bit time quanta. The bit time is built up from a multiple of this quanta. Valid values for the Bit Rate Prescaler are 0 to 511. The actual interpretation by the hardware of this value is such that one more than the value programmed here is used. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
15-8	NTSEG1	R/WQ	Ah	Nominal Time Segment Before Sample Point. Valid values are 1 to 255. The actual interpretation by the hardware of this value is such that one more than the programmed value is used. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
7	RESERVED	R	0h	

Table 21-17. MCAN_NBTP Field Descriptions (continued)

Bit	Field	Type	Reset	Description
6-0	NTSEG2	R/WQ	3h	Nominal Time Segment After Sample Point. Valid values are 1 to 127. The actual interpretation by the hardware of this value is such that one more than the programmed value is used. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.

21.16 MCAN_TSCC (Offset = 7020h) [Reset = 00000000h]

MCAN_TSCC is shown in [Figure 21-16](#) and described in [Table 21-18](#).

Return to the [Summary Table](#).

MCAN Timestamp Counter Configuration

Figure 21-16. MCAN_TSCC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED												TCP			
R-0h												R/WQ-0h			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED													TSS		
R-0h													R/WQ-0h		

Table 21-18. MCAN_TSCC Field Descriptions

Bit	Field	Type	Reset	Description
31-20	RESERVED	R	0h	
19-16	TCP	R/WQ	0h	Timestamp Counter Prescaler. Configures the timestamp and timeout counters time unit in multiples of CAN bit times. Valid values are 0 to 15. The actual interpretation by the hardware of this value is such that one more than the value programmed here is used. Note: With CAN FD an external counter is required for timestamp generation (TSS = '10'). Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
15-2	RESERVED	R	0h	
1-0	TSS	R/WQ	0h	Timestamp Select 00 Timestamp counter value always 0x0000 01 Timestamp counter value incremented according to TCP 10 External timestamp counter value used 11 Same as '00' Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.

21.17 MCAN_TSCV (Offset = 7024h) [Reset = 00000000h]

MCAN_TSCV is shown in [Figure 21-17](#) and described in [Table 21-19](#).

Return to the [Summary Table](#).

MCAN Timestamp Counter Value

Figure 21-17. MCAN_TSCV

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																TSC															
R-0h																R/W-0h															

Table 21-19. MCAN_TSCV Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15-0	TSC	R/W	0h	Timestamp Counter. The internal/external Timestamp Counter value is captured on start of frame (both Rx and Tx). When TSCC.TSS = '01', the Timestamp Counter is incremented in multiples of CAN bit times, (1...16), depending on the configuration of TSCC.TCP. A wrap around sets interrupt flag IR.TSW. Write access resets the counter to zero. When TSCC.TSS = '10', TSC reflects the External Timestamp Counter value, and a write access has no impact. Note: A 'wrap around' is a change of the Timestamp Counter value from non-zero to zero not caused by write access to MCAN_TSCV.

21.18 MCAN_TOCC (Offset = 7028h) [Reset = FFFF0000h]

MCAN_TOCC is shown in [Figure 21-18](#) and described in [Table 21-20](#).

Return to the [Summary Table](#).

MCAN Timeout Counter Configuration

Figure 21-18. MCAN_TOCC

31	30	29	28	27	26	25	24
TOP							
R/WQ-FFFFh							
23	22	21	20	19	18	17	16
TOP							
R/WQ-FFFFh							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED					TOS		ETOC
R-0h					R/WQ-0h		R/WQ-0h

Table 21-20. MCAN_TOCC Field Descriptions

Bit	Field	Type	Reset	Description
31-16	TOP	R/WQ	FFFFh	Timeout Period. Start value of the Timeout Counter (down-counter). Configures the Timeout Period. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
15-3	RESERVED	R	0h	
2-1	TOS	R/WQ	0h	Timeout Select. When operating in Continuous mode, a write to TOCV presets the counter to the value configured by TOCC.TOP and continues down-counting. When the Timeout Counter is controlled by one of the FIFOs, an empty FIFO presets the counter to the value configured by TOCC.TOP. Down-counting is started when the first FIFO element is stored. 00 Continuous operation 01 Timeout controlled by Tx Event FIFO 10 Timeout controlled by Rx FIFO 0 11 Timeout controlled by Rx FIFO 1 Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
0	ETOC	R/WQ	0h	Enable Timeout Counter 0 Timeout Counter disabled 1 Timeout Counter enabled Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.

21.19 MCAN_TOCV (Offset = 702Ch) [Reset = 0000FFFFh]

MCAN_TOCV is shown in [Figure 21-19](#) and described in [Table 21-21](#).

Return to the [Summary Table](#).

MCAN Timeout Counter Value

Figure 21-19. MCAN_TOCV

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																TOC															
R-0h																R/W-FFFFh															

Table 21-21. MCAN_TOCV Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15-0	TOC	R/W	FFFFh	Timeout Counter. The Timeout Counter is decremented in multiples of CAN bit times, (1...16), depending on the configuration of TSCC.TCP. When decremented to zero, interrupt flag IR.TOO is set and the Timeout Counter is stopped. Start and reset/restart conditions are configured via TOCC.TOS.

21.20 MCAN_ECR (Offset = 7040h) [Reset = 00000000h]

MCAN_ECR is shown in [Figure 21-20](#) and described in [Table 21-22](#).

Return to the [Summary Table](#).

MCAN Error Counter Register

Figure 21-20. MCAN_ECR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED								CEL							
R-0h								RC-0h							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RP		REC						TEC							
R-0h				R-0h								R-0h			

Table 21-22. MCAN_ECR Field Descriptions

Bit	Field	Type	Reset	Description
31-24	RESERVED	R	0h	
23-16	CEL	RC	0h	CAN Error Logging. The counter is incremented each time when a CAN protocol error causes the Transmit Error Counter or the Receive Error Counter to be incremented. It is reset by read access to CEL. The counter stops at 0xFF; the next increment of TEC or REC sets interrupt flag IR.ELO. Note: When CCCR.ASM is set, the CAN protocol controller does not increment TEC and REC when a CAN protocol error is detected, but CEL is still incremented.
15	RP	R	0h	Receive Error Passive 0 The Receive Error Counter is below the error passive level of 128 1 The Receive Error Counter has reached the error passive level of 128
14-8	REC	R	0h	Receive Error Counter. Actual state of the Receive Error Counter, values between 0 and 127. Note: When CCCR.ASM is set, the CAN protocol controller does not increment TEC and REC when a CAN protocol error is detected, but CEL is still incremented.
7-0	TEC	R	0h	Transmit Error Counter. Actual state of the Transmit Error Counter, values between 0 and 255. Note: When CCCR.ASM is set, the CAN protocol controller does not increment TEC and REC when a CAN protocol error is detected, but CEL is still incremented.

21.21 MCAN_PSR (Offset = 7044h) [Reset = 00000707h]

MCAN_PSR is shown in [Figure 21-21](#) and described in [Table 21-23](#).

Return to the [Summary Table](#).

MCAN Protocol Status Register

Figure 21-21. MCAN_PSR

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED				TDCV			
R-0h				R-0h			
15	14	13	12	11	10	9	8
RESERVED		PXE	RFDF	RBRS	RESI	DLEC	
R-0h		RC-0h	RC-0h	RC-0h	RC-0h	RS-7h	
7	6	5	4	3	2	1	0
BO	EW	EP	ACT		LEC		
R-0h	R-0h	R-0h	R-0h		RS-7h		

Table 21-23. MCAN_PSR Field Descriptions

Bit	Field	Type	Reset	Description
31-23	RESERVED	R	0h	
22-16	TDCV	R	0h	Transmitter Delay Compensation Value. Position of the secondary sample point, defined by the sum of the measured delay from the internal CAN TX signal to the internal CAN RX signal and TDCR.TDCO. The SSP position is, in the data phase, the number of mtq between the start of the transmitted bit and the secondary sample point. Valid values are 0 to 127 mtq.
15	RESERVED	R	0h	
14	PXE	RC	0h	Protocol Exception Event 0 No protocol exception event occurred since last read access 1 Protocol exception event occurred
13	RFDF	RC	0h	Received a CAN FD Message. This bit is set independent of acceptance filtering. 0 Since this bit was reset by the CPU, no CAN FD message has been received 1 Message in CAN FD format with FDF flag set has been received
12	RBRS	RC	0h	BRS Flag of Last Received CAN FD Message. This bit is set together with RFDF, independent of acceptance filtering. 0 Last received CAN FD message did not have its BRS flag set 1 Last received CAN FD message had its BRS flag set
11	RESI	RC	0h	ESI Flag of Last Received CAN FD Message. This bit is set together with RFDF, independent of acceptance filtering. 0 Last received CAN FD message did not have its ESI flag set 1 Last received CAN FD message had its ESI flag set
10-8	DLEC	RS	7h	Data Phase Last Error Code. Type of last error that occurred in the data phase of a CAN FD format frame with its BRS flag set. Coding is the same as for LEC. This field will be cleared to zero when a CAN FD format frame with its BRS flag set has been transferred (reception or transmission) without error.
7	BO	R	0h	Bus_Off Status 0 The M_CAN is not Bus_Off 1 The M_CAN is in Bus_Off state

Table 21-23. MCAN_PSR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
6	EW	R	0h	Warning Status 0 Both error counters are below the Error_Warning limit of 96 1 At least one of error counter has reached the Error_Warning limit of 96
5	EP	R	0h	Error Passive 0 The M_CAN is in the Error_Active state. It normally takes part in bus communication and sends an active error flag when an error has been detected 1 The M_CAN is in the Error_Passive state
4-3	ACT	R	0h	Node Activity. Monitors the module's CAN communication state. 00 Synchronizing - node is synchronizing on CAN communication 01 Idle - node is neither receiver nor transmitter 10 Receiver - node is operating as receiver 11 Transmitter - node is operating as transmitter Note: ACT is set to '00' by a Protocol Exception Event.
2-0	LEC	RS	7h	Last Error Code. The LEC indicates the type of the last error to occur on the CAN bus. This field will be cleared to '0' when a message has been transferred (reception or transmission) without error. 0 No Error: No error occurred since LEC has been reset by successful reception or transmission. 1 Stuff Error: More than 5 equal bits in a sequence have occurred in a part of a received message where this is not allowed. 2 Form Error: A fixed format part of a received frame has the wrong format. 3 AckError: The message transmitted by the MCAN was not acknowledged by another node. 4 Bit1Error: During the transmission of a message (with the exception of the arbitration field), the device wanted to send a recessive level (bit of logical value '1'), but the monitored bus value was dominant. 5 Bit0Error: During the transmission of a message (or acknowledge bit, or active error flag, or overload flag), the device wanted to send a dominant level (data or identifier bit logical value '0'), but the monitored bus value was recessive. During Bus_Off recovery this status is set each time a sequence of 11 recessive bits has been monitored. This enables the CPU to monitor the proceeding of the Bus_Off recovery sequence (indicating the bus is not stuck at dominant or continuously disturbed). 6 CRCErrror: The CRC check sum of a received message was incorrect. The CRC of an incoming message does not match with the CRC calculated from the received data. 7 NoChange: Any read access to the Protocol Status Register re-initializes the LEC to '7'. When the LEC shows the value '7', no CAN bus event was detected since the last CPU read access to the Protocol Status Register. Note: When a frame in CAN FD format has reached the data phase with BRS flag set, the next CAN event (error or valid frame) will be shown in DLEC instead of LEC. An error in a fixed stuff bit of a CAN FD CRC sequence will be shown as a Form Error, not Stuff Error. Note: The Bus_Off recovery sequence (see ISO 11898-1:2015) cannot be shortened by setting or resetting CCCR.INIT. If the device goes Bus_Off, it will set CCCR.INIT of its own accord, stopping all bus activities. Once CCCR.INIT has been cleared by the CPU, the device will then wait for 129 occurrences of Bus Idle (129 * 11 consecutive recessive bits) before resuming normal operation. At the end of the Bus_Off recovery sequence, the Error Management Counters will be reset. During the waiting time after the resetting of CCCR.INIT, each time a sequence of 11 recessive bits has been monitored, a Bit0Error code is written to PSR.LEC, enabling the CPU to readily check up whether the CAN bus is stuck at dominant or continuously disturbed and to monitor the Bus_Off recovery sequence. ECR.REC is used to count these sequences.

21.22 MCAN_TDCR (Offset = 7048h) [Reset = 00000000h]

MCAN_TDCR is shown in [Figure 21-22](#) and described in [Table 21-24](#).

Return to the [Summary Table](#).

MCAN Transmitter Delay Compensation Register

Figure 21-22. MCAN_TDCR

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED				TDCO			
R-0h				R/WQ-0h			
7	6	5	4	3	2	1	0
RESERVED				TDCF			
R-0h				R/WQ-0h			

Table 21-24. MCAN_TDCR Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	R	0h	
14-8	TDCO	R/WQ	0h	Transmitter Delay Compensation Offset. Offset value defining the distance between the measured delay from the internal CAN TX signal to the internal CAN RX signal and the secondary sample point. Valid values are 0 to 127 mtq. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
7	RESERVED	R	0h	
6-0	TDCF	R/WQ	0h	Transmitter Delay Compensation Filter Window Length. Defines the minimum value for the SSP position, dominant edges on the internal CAN RX signal that would result in an earlier SSP position are ignored for transmitter delay measurement. The feature is enabled when TDCF is configured to a value greater than TDCO. Valid values are 0 to 127 mtq. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.

21.23 MCAN_IR (Offset = 7050h) [Reset = 80000000h]

MCAN_IR is shown in [Figure 21-23](#) and described in [Table 21-25](#).

Return to the [Summary Table](#).

The flags are set when one of the listed conditions is detected (edge-sensitive). The flags remain set until the Host clears them. A flag is cleared by writing a '1' to the corresponding bit position. Writing a '0' has no effect. A hard reset will clear the register. The configuration of IE controls whether an interrupt is generated. The configuration of ILS controls on which interrupt line an interrupt is signalled.

Figure 21-23. MCAN_IR

31	30	29	28	27	26	25	24
RESERVED		ARA	PED	PEA	WDI	BO	EW
R-0h		R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h
23	22	21	20	19	18	17	16
EP	ELO	BEU	RESERVED	DRX	TOO	MRAF	TSW
R/W1C-0h	R/W1C-0h	R/W1C-0h	R-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h
15	14	13	12	11	10	9	8
TEFL	TEFF	TEFW	TEFN	TFE	TCF	TC	HPM
R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h
7	6	5	4	3	2	1	0
RF1L	RF1F	RF1W	RF1N	RF0L	RF0F	RF0W	RF0N
R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h

Table 21-25. MCAN_IR Field Descriptions

Bit	Field	Type	Reset	Description
31-30	RESERVED	R	0h	
29	ARA	R/W1C	0h	Access to Reserved Address 0 No access to reserved address occurred 1 Access to reserved address occurred
28	PED	R/W1C	0h	Protocol Error in Data Phase (Data Bit Time is used) 0 No protocol error in data phase 1 Protocol error in data phase detected (PSR.DLEC ? 0,7)
27	PEA	R/W1C	0h	Protocol Error in Arbitration Phase (Nominal Bit Time is used) 0 No protocol error in arbitration phase 1 Protocol error in arbitration phase detected (PSR.LEC ? 0,7)
26	WDI	R/W1C	0h	Watchdog Interrupt 0 No Message RAM Watchdog event occurred 1 Message RAM Watchdog event due to missing READY
25	BO	R/W1C	0h	Bus_Off Status 0 Bus_Off status unchanged 1 Bus_Off status changed
24	EW	R/W1C	0h	Warning Status 0 Error_Warning status unchanged 1 Error_Warning status changed
23	EP	R/W1C	0h	Error Passive 0 Error_Passive status unchanged 1 Error_Passive status changed
22	ELO	R/W1C	0h	Error Logging Overflow 0 CAN Error Logging Counter did not overflow 1 Overflow of CAN Error Logging Counter occurred

Table 21-25. MCAN_IR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	BEU	R/W1C	0h	Bit Error Uncorrected. Message RAM bit error detected, uncorrected. This bit is set when a double bit error is detected by the ECC aggregator attached to the Message RAM. An uncorrected Message RAM bit error sets CCCR.INIT to '1'. This is done to avoid transmission of corrupted data. 0 No bit error detected when reading from Message RAM 1 Bit error detected, uncorrected (e.g. parity logic)
20	RESERVED	R	0h	
19	DRX	R/W1C	0h	Message Stored to Dedicated Rx Buffer. The flag is set whenever a received message has been stored into a dedicated Rx Buffer. 0 No Rx Buffer updated 1 At least one received message stored into an Rx Buffer
18	TOO	R/W1C	0h	Timeout Occurred 0 No timeout 1 Timeout reached
17	MRAF	R/W1C	0h	Message RAM Access Failure. The flag is set, when the Rx Handler: - has not completed acceptance filtering or storage of an accepted message until the arbitration field of the following message has been received. In this case acceptance filtering or message storage is aborted and the Rx Handler starts processing of the following message. - was not able to write a message to the Message RAM. In this case message storage is aborted. In both cases the FIFO put index is not updated resp. the New Data flag for a dedicated Rx Buffer is not set, a partly stored message is overwritten when the next message is stored to this location. The flag is also set when the Tx Handler was not able to read a message from the Message RAM in time. In this case message transmission is aborted. In case of a Tx Handler access failure the MCAN is switched into Restricted Operation Mode. To leave Restricted Operation Mode, the Host CPU has to reset CCCR.ASM. 0 No Message RAM access failure occurred 1 Message RAM access failure occurred
16	TSW	R/W1C	0h	Timestamp Wraparound 0 No timestamp counter wrap-around 1 Timestamp counter wrapped around
15	TEFL	R/W1C	0h	Tx Event FIFO Element Lost 0 No Tx Event FIFO element lost 1 Tx Event FIFO element lost, also set after write attempt to Tx Event FIFO of size zero
14	TEFF	R/W1C	0h	Tx Event FIFO Full 0 Tx Event FIFO not full 1 Tx Event FIFO full
13	TEFW	R/W1C	0h	Tx Event FIFO Watermark Reached 0 Tx Event FIFO fill level below watermark 1 Tx Event FIFO fill level reached watermark
12	TEFN	R/W1C	0h	Tx Event FIFO New Entry 0 Tx Event FIFO unchanged 1 Tx Handler wrote Tx Event FIFO element
11	TFE	R/W1C	0h	Tx FIFO Empty 0 Tx FIFO non-empty 1 Tx FIFO empty
10	TCF	R/W1C	0h	Transmission Cancellation Finished 0 No transmission cancellation finished 1 Transmission cancellation finished
9	TC	R/W1C	0h	Transmission Completed 0 No transmission completed 1 Transmission completed

Table 21-25. MCAN_IR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
8	HPM	R/W1C	0h	High Priority Message 0 No high priority message received 1 High priority message received
7	RF1L	R/W1C	0h	Rx FIFO 1 Message Lost 0 No Rx FIFO 1 message lost 1 Rx FIFO 1 message lost, also set after write attempt to Rx FIFO 1 of size zero
6	RF1F	R/W1C	0h	Rx FIFO 1 Full 0 Rx FIFO 1 not full 1 Rx FIFO 1 full
5	RF1W	R/W1C	0h	Rx FIFO 1 Watermark Reached 0 Rx FIFO 1 fill level below watermark 1 Rx FIFO 1 fill level reached watermark
4	RF1N	R/W1C	0h	Rx FIFO 1 New Message 0 No new message written to Rx FIFO 1 1 New message written to Rx FIFO 1
3	RF0L	R/W1C	0h	Rx FIFO 0 Message Lost 0 No Rx FIFO 0 message lost 1 Rx FIFO 0 message lost, also set after write attempt to Rx FIFO 0 of size zero
2	RF0F	R/W1C	0h	Rx FIFO 0 Full 0 Rx FIFO 0 not full 1 Rx FIFO 0 full
1	RF0W	R/W1C	0h	Rx FIFO 0 Watermark Reached 0 Rx FIFO 0 fill level below watermark 1 Rx FIFO 0 fill level reached watermark
0	RF0N	R/W1C	0h	Rx FIFO 0 New Message 0 No new message written to Rx FIFO 0 1 New message written to Rx FIFO 0

21.24 MCAN_IE (Offset = 7054h) [Reset = 00000000h]

MCAN_IE is shown in [Figure 21-24](#) and described in [Table 21-26](#).

Return to the [Summary Table](#).

MCAN Interrupt Enable

Figure 21-24. MCAN_IE

31	30	29	28	27	26	25	24
RESERVED		ARAE	PEDE	PEAE	WDIE	BOE	EWE
R-0h		R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
EPE	ELOE	BEUE	BECE	DRXE	TOOE	MRAFE	TSWE
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
TEFLE	TEFFE	TEFWE	TEFNE	TFEE	TCFE	TCE	HPME
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
RF1LE	RF1FE	RF1WE	RF1NE	RF0LE	RF0FE	RF0WE	RF0NE
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 21-26. MCAN_IE Field Descriptions

Bit	Field	Type	Reset	Description
31-30	RESERVED	R	0h	
29	ARAE	R/W	0h	Access to Reserved Address Enable
28	PEDE	R/W	0h	Protocol Error in Data Phase Enable
27	PEAE	R/W	0h	Protocol Error in Arbitration Phase Enable
26	WDIE	R/W	0h	Watchdog Interrupt Enable
25	BOE	R/W	0h	Bus_Off Status Enable
24	EWE	R/W	0h	Warning Status Enable
23	EPE	R/W	0h	Error Passive Enable
22	ELOE	R/W	0h	Error Logging Overflow Enable
21	BEUE	R/W	0h	Bit Error Uncorrected Enable
20	BECE	R/W	0h	Bit Error Corrected Enable A separate interrupt line reserved for corrected bit errors is provided via the MCAN_ERROR_REGS. It is advised for the user to use these registers and leave this bit cleared to '0'.
19	DRXE	R/W	0h	Message Stored to Dedicated Rx Buffer Enable
18	TOOE	R/W	0h	Timeout Occurred Enable
17	MRAFE	R/W	0h	Message RAM Access Failure Enable
16	TSWE	R/W	0h	Timestamp Wraparound Enable
15	TEFLE	R/W	0h	Tx Event FIFO Element Lost Enable
14	TEFFE	R/W	0h	Tx Event FIFO Full Enable
13	TEFWE	R/W	0h	Tx Event FIFO Watermark Reached Enable
12	TEFNE	R/W	0h	Tx Event FIFO New Entry Enable
11	TFEE	R/W	0h	Tx FIFO Empty Enable
10	TCFE	R/W	0h	Transmission Cancellation Finished Enable
9	TCE	R/W	0h	Transmission Completed Enable
8	HPME	R/W	0h	High Priority Message Enable

Table 21-26. MCAN_IE Field Descriptions (continued)

Bit	Field	Type	Reset	Description
7	RF1LE	R/W	0h	Rx FIFO 1 Message Lost Enable
6	RF1FE	R/W	0h	Rx FIFO 1 Full Enable
5	RF1WE	R/W	0h	Rx FIFO 1 Watermark Reached Enable
4	RF1NE	R/W	0h	Rx FIFO 1 New Message Enable
3	RF0LE	R/W	0h	Rx FIFO 0 Message Lost Enable
2	RF0FE	R/W	0h	Rx FIFO 0 Full Enable
1	RF0WE	R/W	0h	Rx FIFO 0 Watermark Reached Enable
0	RF0NE	R/W	0h	Rx FIFO 0 New Message Enable

21.25 MCAN_ILS (Offset = 7058h) [Reset = 00000000h]

MCAN_ILS is shown in [Figure 21-25](#) and described in [Table 21-27](#).

Return to the [Summary Table](#).

The Interrupt Line Select register assigns an interrupt generated by a specific interrupt flag from the Interrupt Register to one of the two module interrupt lines. For interrupt generation the respective interrupt line has to be enabled via ILE.EINT0 and ILE.EINT1.

Figure 21-25. MCAN_ILS

31	30	29	28	27	26	25	24
RESERVED		ARAL	PEDL	PEAL	WDIL	BOL	EWL
R-0h		R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
EPL	ELOL	BEUL	BECL	DRXL	TOOL	MRAFL	TSWL
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
TEFLL	TEFFL	TEFWL	TEFNL	TFEL	TCFL	TCL	HPML
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
RF1LL	RF1FL	RF1WL	RF1NL	RF0LL	RF0FL	RF0WL	RF0NL
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 21-27. MCAN_ILS Field Descriptions

Bit	Field	Type	Reset	Description
31-30	RESERVED	R	0h	
29	ARAL	R/W	0h	Access to Reserved Address Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
28	PEDL	R/W	0h	Protocol Error in Data Phase Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
27	PEAL	R/W	0h	Protocol Error in Arbitration Phase Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
26	WDIL	R/W	0h	Watchdog Interrupt Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
25	BOL	R/W	0h	Bus_Off Status Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
24	EWL	R/W	0h	Warning Status Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
23	EPL	R/W	0h	Error Passive Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
22	ELOL	R/W	0h	Error Logging Overflow Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
21	BEUL	R/W	0h	Bit Error Uncorrected Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1

Table 21-27. MCAN_ILS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
20	BECL	R/W	0h	Bit Error Corrected Line A separate interrupt line reserved for corrected bit errors is provided via the MCAN_ERROR_REGS. It is advised for the user to use these registers and leave the MCAN_IE.BECE bit cleared to '0' (disabled), thereby relegating this bit to not applicable.
19	DRXL	R/W	0h	Message Stored to Dedicated Rx Buffer Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
18	TOOL	R/W	0h	Timeout Occurred Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
17	MRAFL	R/W	0h	Message RAM Access Failure Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
16	TSWL	R/W	0h	Timestamp Wraparound Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
15	TEFLL	R/W	0h	Tx Event FIFO Element Lost Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
14	TEFFL	R/W	0h	Tx Event FIFO Full Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
13	TEFWL	R/W	0h	Tx Event FIFO Watermark Reached Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
12	TEFNL	R/W	0h	Tx Event FIFO New Entry Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
11	TFEL	R/W	0h	Tx FIFO Empty Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
10	TCFL	R/W	0h	Transmission Cancellation Finished Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
9	TCL	R/W	0h	Transmission Completed Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
8	HPML	R/W	0h	High Priority Message Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
7	RF1LL	R/W	0h	Rx FIFO 1 Message Lost Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
6	RF1FL	R/W	0h	Rx FIFO 1 Full Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
5	RF1WL	R/W	0h	Rx FIFO 1 Watermark Reached Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
4	RF1NL	R/W	0h	Rx FIFO 1 New Message Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
3	RF0LL	R/W	0h	Rx FIFO 0 Message Lost Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1

Table 21-27. MCAN_ILS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	RF0FL	R/W	0h	Rx FIFO 0 Full Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
1	RF0WL	R/W	0h	Rx FIFO 0 Watermark Reached Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1
0	RF0NL	R/W	0h	Rx FIFO 0 New Message Line 0 Interrupt source is assigned to Interrupt Line 0 1 Interrupt source is assigned to Interrupt Line 1

21.26 MCAN_ILE (Offset = 705Ch) [Reset = 00000000h]

MCAN_ILE is shown in [Figure 21-26](#) and described in [Table 21-28](#).

Return to the [Summary Table](#).

MCAN Interrupt Line Enable

Figure 21-26. MCAN_ILE

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED						EINT1	EINT0
R-0h						R/W-0h	R/W-0h

Table 21-28. MCAN_ILE Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1	EINT1	R/W	0h	Enable Interrupt Line 1 0 Interrupt Line 1 is disabled 1 Interrupt Line 1 is enabled
0	EINT0	R/W	0h	Enable Interrupt Line 0 0 Interrupt Line 0 is disabled 1 Interrupt Line 0 is enabled

21.27 MCAN_GFC (Offset = 7080h) [Reset = 00000000h]

MCAN_GFC is shown in [Figure 21-27](#) and described in [Table 21-29](#).

Return to the [Summary Table](#).

MCAN Global Filter Configuration

Figure 21-27. MCAN_GFC

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED		ANFS		ANFE		RRFS	RRFE
R-0h		R/WQ-0h		R/WQ-0h		R/WQ-0h	R/WQ-0h

Table 21-29. MCAN_GFC Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R	0h	
5-4	ANFS	R/WQ	0h	Accept Non-matching Frames Standard. Defines how received messages with 11-bit IDs that do not match any element of the filter list are treated. 00 Accept in Rx FIFO 0 01 Accept in Rx FIFO 1 10 Reject 11 Reject Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
3-2	ANFE	R/WQ	0h	Accept Non-matching Frames Extended. Defines how received messages with 29-bit IDs that do not match any element of the filter list are treated. 00 Accept in Rx FIFO 0 01 Accept in Rx FIFO 1 10 Reject 11 Reject Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
1	RRFS	R/WQ	0h	Reject Remote Frames Standard 0 Filter remote frames with 11-bit standard IDs 1 Reject all remote frames with 11-bit standard IDs Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
0	RRFE	R/WQ	0h	Reject Remote Frames Extended 0 Filter remote frames with 29-bit extended IDs 1 Reject all remote frames with 29-bit extended IDs Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.

21.28 MCAN_SIDFC (Offset = 7084h) [Reset = 00000000h]

MCAN_SIDFC is shown in [Figure 21-28](#) and described in [Table 21-30](#).

Return to the [Summary Table](#).

MCAN Standard ID Filter Configuration

Figure 21-28. MCAN_SIDFC

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
LSS							
R/WQ-0h							
15	14	13	12	11	10	9	8
FLSSA							
R/WQ-0h							
7	6	5	4	3	2	1	0
FLSSA						RESERVED	
R/WQ-0h						R-0h	

Table 21-30. MCAN_SIDFC Field Descriptions

Bit	Field	Type	Reset	Description
31-24	RESERVED	R	0h	
23-16	LSS	R/WQ	0h	List Size Standard 0 No standard Message ID filter 1-128 Number of standard Message ID filter elements >128 Values greater than 128 are interpreted as 128
15-2	FLSSA	R/WQ	0h	Filter List Standard Start Address. Start address of standard Message ID filter list (32-bit word address).
1-0	RESERVED	R	0h	

21.29 MCAN_XIDFC (Offset = 7088h) [Reset = 00000000h]

MCAN_XIDFC is shown in [Figure 21-29](#) and described in [Table 21-31](#).

Return to the [Summary Table](#).

MCAN Extended ID Filter Configuration

Figure 21-29. MCAN_XIDFC

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED		LSE					
R-0h		R/WQ-0h					
15	14	13	12	11	10	9	8
FLESA							
R/WQ-0h							
7	6	5	4	3	2	1	0
FLESA						RESERVED	
R/WQ-0h						R-0h	

Table 21-31. MCAN_XIDFC Field Descriptions

Bit	Field	Type	Reset	Description
31-23	RESERVED	R	0h	
22-16	LSE	R/WQ	0h	Filter List Extended Start Address. Start address of extended Message ID filter list (32-bit word address). Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
15-2	FLESA	R/WQ	0h	List Size Extended 0 No extended Message ID filter 1-64 Number of extended Message ID filter elements >64 Values greater than 64 are interpreted as 64 Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
1-0	RESERVED	R	0h	

21.30 MCAN_XIDAM (Offset = 7090h) [Reset = 1FFFFFFFh]

MCAN_XIDAM is shown in [Figure 21-30](#) and described in [Table 21-32](#).

Return to the [Summary Table](#).

MCAN Extended ID and Mask

Figure 21-30. MCAN_XIDAM

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVE D			EIDM																												
R-0h			R/WQ-1FFFFFFFh																												

Table 21-32. MCAN_XIDAM Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R	0h	
28-0	EIDM	R/WQ	1FFFFFFFh	Extended ID Mask. For acceptance filtering of extended frames the Extended ID AND Mask is ANDed with the Message ID of a received frame. Intended for masking of 29-bit IDs in SAE J1939. With the reset value of all bits set to one the mask is not active. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.

21.31 MCAN_HPMS (Offset = 7094h) [Reset = 00000000h]

MCAN_HPMS is shown in [Figure 21-31](#) and described in [Table 21-33](#).

Return to the [Summary Table](#).

This register is updated every time a Message ID filter element configured to generate a priority event matches. This can be used to monitor the status of incoming high priority messages and to enable fast access to these messages.

Figure 21-31. MCAN_HPMS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
FLST		FIDX					
R-0h		R-0h					
7	6	5	4	3	2	1	0
MSI		BIDX					
R-0h		R-0h					

Table 21-33. MCAN_HPMS Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15	FLST	R	0h	Filter List. Indicates the filter list of the matching filter element. 0 Standard Filter List 1 Extended Filter List
14-8	FIDX	R	0h	Filter Index. Index of matching filter element. Range is 0 to SIDFC.LSS - 1 resp. XIDFC.LSE - 1.
7-6	MSI	R	0h	Message Storage Indicator 00 No FIFO selected 01 FIFO message lost 10 Message stored in FIFO 0 11 Message stored in FIFO 1
5-0	BIDX	R	0h	Buffer Index. Index of Rx FIFO element to which the message was stored. Only valid when MSI(1) = '1'.

21.32 MCAN_NDAT1 (Offset = 7098h) [Reset = 00000000h]

MCAN_NDAT1 is shown in [Figure 21-32](#) and described in [Table 21-34](#).

Return to the [Summary Table](#).

MCAN New Data 1

Figure 21-32. MCAN_NDAT1

31	30	29	28	27	26	25	24
ND31	ND30	ND29	ND28	ND27	ND26	ND25	ND24
R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h
23	22	21	20	19	18	17	16
ND23	ND22	ND21	ND20	ND19	ND18	ND17	ND16
R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h
15	14	13	12	11	10	9	8
ND15	ND14	ND13	ND12	ND11	ND10	ND9	ND8
R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h
7	6	5	4	3	2	1	0
ND7	ND6	ND5	ND4	ND3	ND2	ND1	ND0
R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h

Table 21-34. MCAN_NDAT1 Field Descriptions

Bit	Field	Type	Reset	Description
31	ND31	R/W1C	0h	New Data RX Buffer 31 0 Rx Buffer not updated 1 Rx Buffer updated from new message
30	ND30	R/W1C	0h	New Data RX Buffer 30 0 Rx Buffer not updated 1 Rx Buffer updated from new message
29	ND29	R/W1C	0h	New Data RX Buffer 29 0 Rx Buffer not updated 1 Rx Buffer updated from new message
28	ND28	R/W1C	0h	New Data RX Buffer 28 0 Rx Buffer not updated 1 Rx Buffer updated from new message
27	ND27	R/W1C	0h	New Data RX Buffer 27 0 Rx Buffer not updated 1 Rx Buffer updated from new message
26	ND26	R/W1C	0h	New Data RX Buffer 26 0 Rx Buffer not updated 1 Rx Buffer updated from new message
25	ND25	R/W1C	0h	New Data RX Buffer 25 0 Rx Buffer not updated 1 Rx Buffer updated from new message
24	ND24	R/W1C	0h	New Data RX Buffer 24 0 Rx Buffer not updated 1 Rx Buffer updated from new message
23	ND23	R/W1C	0h	New Data RX Buffer 23 0 Rx Buffer not updated 1 Rx Buffer updated from new message
22	ND22	R/W1C	0h	New Data RX Buffer 22 0 Rx Buffer not updated 1 Rx Buffer updated from new message

Table 21-34. MCAN_NDAT1 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	ND21	R/W1C	0h	New Data RX Buffer 21 0 Rx Buffer not updated 1 Rx Buffer updated from new message
20	ND20	R/W1C	0h	New Data RX Buffer 20 0 Rx Buffer not updated 1 Rx Buffer updated from new message
19	ND19	R/W1C	0h	New Data RX Buffer 19 0 Rx Buffer not updated 1 Rx Buffer updated from new message
18	ND18	R/W1C	0h	New Data RX Buffer 18 0 Rx Buffer not updated 1 Rx Buffer updated from new message
17	ND17	R/W1C	0h	New Data RX Buffer 17 0 Rx Buffer not updated 1 Rx Buffer updated from new message
16	ND16	R/W1C	0h	New Data RX Buffer 16 0 Rx Buffer not updated 1 Rx Buffer updated from new message
15	ND15	R/W1C	0h	New Data RX Buffer 15 0 Rx Buffer not updated 1 Rx Buffer updated from new message
14	ND14	R/W1C	0h	New Data RX Buffer 14 0 Rx Buffer not updated 1 Rx Buffer updated from new message
13	ND13	R/W1C	0h	New Data RX Buffer 13 0 Rx Buffer not updated 1 Rx Buffer updated from new message
12	ND12	R/W1C	0h	New Data RX Buffer 12 0 Rx Buffer not updated 1 Rx Buffer updated from new message
11	ND11	R/W1C	0h	New Data RX Buffer 11 0 Rx Buffer not updated 1 Rx Buffer updated from new message
10	ND10	R/W1C	0h	New Data RX Buffer 10 0 Rx Buffer not updated 1 Rx Buffer updated from new message
9	ND9	R/W1C	0h	New Data RX Buffer 9 0 Rx Buffer not updated 1 Rx Buffer updated from new message
8	ND8	R/W1C	0h	New Data RX Buffer 8 0 Rx Buffer not updated 1 Rx Buffer updated from new message
7	ND7	R/W1C	0h	New Data RX Buffer 7 0 Rx Buffer not updated 1 Rx Buffer updated from new message
6	ND6	R/W1C	0h	New Data RX Buffer 6 0 Rx Buffer not updated 1 Rx Buffer updated from new message
5	ND5	R/W1C	0h	New Data RX Buffer 5 0 Rx Buffer not updated 1 Rx Buffer updated from new message
4	ND4	R/W1C	0h	New Data RX Buffer 4 0 Rx Buffer not updated 1 Rx Buffer updated from new message
3	ND3	R/W1C	0h	New Data RX Buffer 3 0 Rx Buffer not updated 1 Rx Buffer updated from new message

Table 21-34. MCAN_NDAT1 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	ND2	R/W1C	0h	New Data RX Buffer 2 0 Rx Buffer not updated 1 Rx Buffer updated from new message
1	ND1	R/W1C	0h	New Data RX Buffer 1 0 Rx Buffer not updated 1 Rx Buffer updated from new message
0	ND0	R/W1C	0h	New Data RX Buffer 0 0 Rx Buffer not updated 1 Rx Buffer updated from new message

21.33 MCAN_NDAT2 (Offset = 709Ch) [Reset = 00000000h]

MCAN_NDAT2 is shown in [Figure 21-33](#) and described in [Table 21-35](#).

Return to the [Summary Table](#).

MCAN New Data 2

Figure 21-33. MCAN_NDAT2

31	30	29	28	27	26	25	24
ND63	ND62	ND61	ND60	ND59	ND58	ND57	ND56
R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h
23	22	21	20	19	18	17	16
ND55	ND54	ND53	ND52	ND51	ND50	ND49	ND48
R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h
15	14	13	12	11	10	9	8
ND47	ND46	ND45	ND44	ND43	ND42	ND41	ND40
R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h
7	6	5	4	3	2	1	0
ND39	ND38	ND37	ND36	ND35	ND34	ND33	ND32
R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h	R/W1C-0h

Table 21-35. MCAN_NDAT2 Field Descriptions

Bit	Field	Type	Reset	Description
31	ND63	R/W1C	0h	New Data RX Buffer 63 0 Rx Buffer not updated 1 Rx Buffer updated from new message
30	ND62	R/W1C	0h	New Data RX Buffer 62 0 Rx Buffer not updated 1 Rx Buffer updated from new message
29	ND61	R/W1C	0h	New Data RX Buffer 61 0 Rx Buffer not updated 1 Rx Buffer updated from new message
28	ND60	R/W1C	0h	New Data RX Buffer 60 0 Rx Buffer not updated 1 Rx Buffer updated from new message
27	ND59	R/W1C	0h	New Data RX Buffer 59 0 Rx Buffer not updated 1 Rx Buffer updated from new message
26	ND58	R/W1C	0h	New Data RX Buffer 58 0 Rx Buffer not updated 1 Rx Buffer updated from new message
25	ND57	R/W1C	0h	New Data RX Buffer 57 0 Rx Buffer not updated 1 Rx Buffer updated from new message
24	ND56	R/W1C	0h	New Data RX Buffer 56 0 Rx Buffer not updated 1 Rx Buffer updated from new message
23	ND55	R/W1C	0h	New Data RX Buffer 55 0 Rx Buffer not updated 1 Rx Buffer updated from new message
22	ND54	R/W1C	0h	New Data RX Buffer 54 0 Rx Buffer not updated 1 Rx Buffer updated from new message

Table 21-35. MCAN_NDAT2 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
21	ND53	R/W1C	0h	New Data RX Buffer 53 0 Rx Buffer not updated 1 Rx Buffer updated from new message
20	ND52	R/W1C	0h	New Data RX Buffer 52 0 Rx Buffer not updated 1 Rx Buffer updated from new message
19	ND51	R/W1C	0h	New Data RX Buffer 51 0 Rx Buffer not updated 1 Rx Buffer updated from new message
18	ND50	R/W1C	0h	New Data RX Buffer 50 0 Rx Buffer not updated 1 Rx Buffer updated from new message
17	ND49	R/W1C	0h	New Data RX Buffer 49 0 Rx Buffer not updated 1 Rx Buffer updated from new message
16	ND48	R/W1C	0h	New Data RX Buffer 48 0 Rx Buffer not updated 1 Rx Buffer updated from new message
15	ND47	R/W1C	0h	New Data RX Buffer 47 0 Rx Buffer not updated 1 Rx Buffer updated from new message
14	ND46	R/W1C	0h	New Data RX Buffer 46 0 Rx Buffer not updated 1 Rx Buffer updated from new message
13	ND45	R/W1C	0h	New Data RX Buffer 45 0 Rx Buffer not updated 1 Rx Buffer updated from new message
12	ND44	R/W1C	0h	New Data RX Buffer 44 0 Rx Buffer not updated 1 Rx Buffer updated from new message
11	ND43	R/W1C	0h	New Data RX Buffer 43 0 Rx Buffer not updated 1 Rx Buffer updated from new message
10	ND42	R/W1C	0h	New Data RX Buffer 42 0 Rx Buffer not updated 1 Rx Buffer updated from new message
9	ND41	R/W1C	0h	New Data RX Buffer 41 0 Rx Buffer not updated 1 Rx Buffer updated from new message
8	ND40	R/W1C	0h	New Data RX Buffer 40 0 Rx Buffer not updated 1 Rx Buffer updated from new message
7	ND39	R/W1C	0h	New Data RX Buffer 39 0 Rx Buffer not updated 1 Rx Buffer updated from new message
6	ND38	R/W1C	0h	New Data RX Buffer 38 0 Rx Buffer not updated 1 Rx Buffer updated from new message
5	ND37	R/W1C	0h	New Data RX Buffer 37 0 Rx Buffer not updated 1 Rx Buffer updated from new message
4	ND36	R/W1C	0h	New Data RX Buffer 36 0 Rx Buffer not updated 1 Rx Buffer updated from new message
3	ND35	R/W1C	0h	New Data RX Buffer 35 0 Rx Buffer not updated 1 Rx Buffer updated from new message

Table 21-35. MCAN_NDAT2 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2	ND34	R/W1C	0h	New Data RX Buffer 34 0 Rx Buffer not updated 1 Rx Buffer updated from new message
1	ND33	R/W1C	0h	New Data RX Buffer 33 0 Rx Buffer not updated 1 Rx Buffer updated from new message
0	ND32	R/W1C	0h	New Data RX Buffer 32 0 Rx Buffer not updated 1 Rx Buffer updated from new message

21.34 MCAN_RXF0C (Offset = 70A0h) [Reset = 00000000h]

MCAN_RXF0C is shown in [Figure 21-34](#) and described in [Table 21-36](#).

Return to the [Summary Table](#).

MCAN Rx FIFO 0 Configuration

Figure 21-34. MCAN_RXF0C

31	30	29	28	27	26	25	24
F0OM	F0WM						
R/WQ-0h				R/WQ-0h			
23	22	21	20	19	18	17	16
RESERVED	F0S						
R-0h				R/WQ-0h			
15	14	13	12	11	10	9	8
F0SA							
R/WQ-0h							
7	6	5	4	3	2	1	0
F0SA						RESERVED	
R/WQ-0h						R-0h	

Table 21-36. MCAN_RXF0C Field Descriptions

Bit	Field	Type	Reset	Description
31	F0OM	R/WQ	0h	FIFO 0 Operation Mode. FIFO 0 can be operated in blocking or in overwrite mode. 0 FIFO 0 blocking mode 1 FIFO 0 overwrite mode Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
30-24	F0WM	R/WQ	0h	Rx FIFO 0 Watermark 0 Watermark interrupt disabled 1-64 Level for Rx FIFO 0 watermark interrupt (IR.RF0W) >64 Watermark interrupt disabled Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
23	RESERVED	R	0h	
22-16	F0S	R/WQ	0h	Rx FIFO 0 Size. The Rx FIFO 0 elements are indexed from 0 to F0S-1. 0 No Rx FIFO 0 1-64 Number of Rx FIFO 0 elements >64 Values greater than 64 are interpreted as 64 Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
15-2	F0SA	R/WQ	0h	Rx FIFO 0 Start Address. Start address of Rx FIFO 0 in Message RAM (32-bit word address). Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
1-0	RESERVED	R	0h	

21.35 MCAN_RXF0S (Offset = 70A4h) [Reset = 00000000h]

MCAN_RXF0S is shown in [Figure 21-35](#) and described in [Table 21-37](#).

Return to the [Summary Table](#).

MCAN Rx FIFO 0 Status

Figure 21-35. MCAN_RXF0S

31	30	29	28	27	26	25	24
RESERVED						RF0L	F0F
R-0h						R-0h	R-0h
23	22	21	20	19	18	17	16
RESERVED				F0PI			
R-0h				R-0h			
15	14	13	12	11	10	9	8
RESERVED				F0GI			
R-0h				R-0h			
7	6	5	4	3	2	1	0
RESERVED		F0FL					
R-0h		R-0h					

Table 21-37. MCAN_RXF0S Field Descriptions

Bit	Field	Type	Reset	Description
31-26	RESERVED	R	0h	
25	RF0L	R	0h	Rx FIFO 0 Message Lost. This bit is a copy of interrupt flag IR.RF0L. When IR.RF0L is reset, this bit is also reset. 0 No Rx FIFO 0 message lost 1 Rx FIFO 0 message lost, also set after write attempt to Rx FIFO 0 of size zero Note: Overwriting the oldest message when RXF0C.F0OM = '1' will not set this flag.
24	F0F	R	0h	Rx FIFO 0 Full 0 Rx FIFO 0 not full 1 Rx FIFO 0 full
23-22	RESERVED	R	0h	
21-16	F0PI	R	0h	Rx FIFO 0 Put Index. Rx FIFO 0 write index pointer, range 0 to 63.
15-14	RESERVED	R	0h	
13-8	F0GI	R	0h	Rx FIFO 0 Get Index. Rx FIFO 0 read index pointer, range 0 to 63.
7	RESERVED	R	0h	
6-0	F0FL	R	0h	Rx FIFO 0 Fill Level. Number of elements stored in Rx FIFO 0, range 0 to 64.

21.36 MCAN_RXF0A (Offset = 70A8h) [Reset = 00000000h]

MCAN_RXF0A is shown in [Figure 21-36](#) and described in [Table 21-38](#).

Return to the [Summary Table](#).

MCAN Rx FIFO 0 Acknowledge

Figure 21-36. MCAN_RXF0A

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																										F0AI					
R-0h																										R/W-0h					

Table 21-38. MCAN_RXF0A Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R	0h	
5-0	F0AI	R/W	0h	Rx FIFO 0 Acknowledge Index. After the Host has read a message or a sequence of messages from Rx FIFO 0 it has to write the buffer index of the last element read from Rx FIFO 0 to F0AI. This will set the Rx FIFO 0 Get Index RXF0S.F0GI to F0AI + 1 and update the FIFO 0 Fill Level RXF0S.F0FL.

21.37 MCAN_RXBC (Offset = 70ACh) [Reset = 00000000h]

MCAN_RXBC is shown in [Figure 21-37](#) and described in [Table 21-39](#).

Return to the [Summary Table](#).

MCAN Rx Buffer Configuration

Figure 21-37. MCAN_RXBC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																RBSA											RESE RVED				
R-0h																R/WQ-0h											R-0h				

Table 21-39. MCAN_RXBC Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15-2	RBSA	R/WQ	0h	Rx Buffer Start Address. Configures the start address of the Rx Buffers section in the Message RAM (32-bit word address). +1466
1-0	RESERVED	R	0h	

21.38 MCAN_RXF1C (Offset = 70B0h) [Reset = 00000000h]

MCAN_RXF1C is shown in [Figure 21-38](#) and described in [Table 21-40](#).

Return to the [Summary Table](#).

MCAN Rx FIFO 1 Configuration

Figure 21-38. MCAN_RXF1C

31	30	29	28	27	26	25	24
F1OM	F1WM						
R/WQ-0h				R/WQ-0h			
23	22	21	20	19	18	17	16
RESERVED	F1S						
R-0h				R/WQ-0h			
15	14	13	12	11	10	9	8
F1SA							
R/WQ-0h							
7	6	5	4	3	2	1	0
F1SA						RESERVED	
R/WQ-0h						R-0h	

Table 21-40. MCAN_RXF1C Field Descriptions

Bit	Field	Type	Reset	Description
31	F1OM	R/WQ	0h	FIFO 1 Operation Mode. FIFO 1 can be operated in blocking or in overwrite mode. 0 FIFO 1 blocking mode 1 FIFO 1 overwrite mode Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
30-24	F1WM	R/WQ	0h	Rx FIFO 1 Watermark 0 Watermark interrupt disabled 1-64 Level for Rx FIFO 1 watermark interrupt (IR.RF1W) >64 Watermark interrupt disabled Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
23	RESERVED	R	0h	
22-16	F1S	R/WQ	0h	Rx FIFO 1 Size. The Rx FIFO 1 elements are indexed from 0 to F1S - 1. 0 No Rx FIFO 1 1-64 Number of Rx FIFO 1 elements >64 Values greater than 64 are interpreted as 64 Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
15-2	F1SA	R/WQ	0h	Rx FIFO 1 Start Address Start address of Rx FIFO 1 in Message RAM (32-bit word address).
1-0	RESERVED	R	0h	

21.39 MCAN_RXF1S (Offset = 70B4h) [Reset = 00000000h]

MCAN_RXF1S is shown in [Figure 21-39](#) and described in [Table 21-41](#).

Return to the [Summary Table](#).

MCAN Rx FIFO 1 Status

Figure 21-39. MCAN_RXF1S

31	30	29	28	27	26	25	24
DMS	RESERVED					RF1L	F1F
R-0h	R-0h					R-0h	R-0h
23	22	21	20	19	18	17	16
RESERVED		F1PI					
R-0h		R-0h					
15	14	13	12	11	10	9	8
RESERVED		F1GI					
R-0h		R-0h					
7	6	5	4	3	2	1	0
RESERVED		F1FL					
R-0h		R-0h					

Table 21-41. MCAN_RXF1S Field Descriptions

Bit	Field	Type	Reset	Description
31-30	DMS	R	0h	Debug Message Status 00 Idle state, wait for reception of debug messages 01 Debug message A received 10 Debug messages A, B received 11 Debug messages A, B, C received
29-26	RESERVED	R	0h	
25	RF1L	R	0h	Rx FIFO 1 Message Lost. This bit is a copy of interrupt flag IR.RF1L. When IR.RF1L is reset, this bit is also reset. 0 No Rx FIFO 1 message lost 1 Rx FIFO 1 message lost, also set after write attempt to Rx FIFO 1 of size zero Note: Overwriting the oldest message when RXF1C.F1OM = '1' will not set this flag.
24	F1F	R	0h	Rx FIFO 1 Full 0 Rx FIFO 1 not full 1 Rx FIFO 1 full
23-22	RESERVED	R	0h	
21-16	F1PI	R	0h	Rx FIFO 1 Put Index. Rx FIFO 1 write index pointer, range 0 to 63.
15-14	RESERVED	R	0h	
13-8	F1GI	R	0h	Rx FIFO 1 Get Index. Rx FIFO 1 read index pointer, range 0 to 63.
7	RESERVED	R	0h	
6-0	F1FL	R	0h	Rx FIFO 1 Fill Level. Number of elements stored in Rx FIFO 1, range 0 to 64.

21.40 MCAN_RXF1A (Offset = 70B8h) [Reset = 00000000h]

MCAN_RXF1A is shown in [Figure 21-40](#) and described in [Table 21-42](#).

Return to the [Summary Table](#).

MCAN Rx FIFO 1 Acknowledge

Figure 21-40. MCAN_RXF1A

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																										F1AI					
R-0h																										R/W-0h					

Table 21-42. MCAN_RXF1A Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R	0h	
5-0	F1AI	R/W	0h	Rx FIFO 1 Acknowledge Index. After the Host has read a message or a sequence of messages from Rx FIFO 1 it has to write the buffer index of the last element read from Rx FIFO 1 to F1AI. This will set the Rx FIFO 1 Get Index RXF1S.F1GI to F1AI + 1 and update the FIFO 1 Fill Level RXF1S.F1FL.

21.41 MCAN_RXESC (Offset = 70BCh) [Reset = 00000000h]

MCAN_RXESC is shown in [Figure 21-41](#) and described in [Table 21-43](#).

Return to the [Summary Table](#).

Configures the number of data bytes belonging to an Rx Buffer / Rx FIFO element. Data field sizes >8 bytes are intended for CAN FD operation only.

Figure 21-41. MCAN_RXESC

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED					RBDS		
R-0h					R/WQ-0h		
7	6	5	4	3	2	1	0
RESERVED		F1DS		RESERVED		F0DS	
R-0h		R/WQ-0h		R-0h		R/WQ-0h	

Table 21-43. MCAN_RXESC Field Descriptions

Bit	Field	Type	Reset	Description
31-11	RESERVED	R	0h	
10-8	RBDS	R/WQ	0h	Rx Buffer Data Field Size 000 8 byte data field 001 12 byte data field 010 16 byte data field 011 20 byte data field 100 24 byte data field 101 32 byte data field 110 48 byte data field 111 64 byte data field Note: In case the data field size of an accepted CAN frame exceeds the data field size configured for the matching Rx Buffer or Rx FIFO, only the number of bytes as configured by RXESC are stored to the Rx Buffer resp. Rx FIFO element. The rest of the frame's data field is ignored. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
7	RESERVED	R	0h	
6-4	F1DS	R/WQ	0h	Rx FIFO 1 Data Field Size 000 8 byte data field 001 12 byte data field 010 16 byte data field 011 20 byte data field 100 24 byte data field 101 32 byte data field 110 48 byte data field 111 64 byte data field Note: In case the data field size of an accepted CAN frame exceeds the data field size configured for the matching Rx Buffer or Rx FIFO, only the number of bytes as configured by RXESC are stored to the Rx Buffer resp. Rx FIFO element. The rest of the frame's data field is ignored. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.

Table 21-43. MCAN_RXESC Field Descriptions (continued)

Bit	Field	Type	Reset	Description
3	RESERVED	R	0h	
2-0	F0DS	R/WQ	0h	Rx FIFO 0 Data Field Size 000 8 byte data field 001 12 byte data field 010 16 byte data field 011 20 byte data field 100 24 byte data field 101 32 byte data field 110 48 byte data field 111 64 byte data field Note: In case the data field size of an accepted CAN frame exceeds the data field size configured for the matching Rx Buffer or Rx FIFO, only the number of bytes as configured by RXESC are stored to the Rx Buffer resp. Rx FIFO element. The rest of the frame's data field is ignored. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.

21.42 MCAN_TXBC (Offset = 70C0h) [Reset = 00000000h]

MCAN_TXBC is shown in [Figure 21-42](#) and described in [Table 21-44](#).

Return to the [Summary Table](#).

MCAN Tx Buffer Configuration

Figure 21-42. MCAN_TXBC

31	30	29	28	27	26	25	24
RESERVED	TFQM	TFQS					
R-0h	R/WQ-0h	R/WQ-0h					
23	22	21	20	19	18	17	16
RESERVED		NDTB					
R-0h		R/WQ-0h					
15	14	13	12	11	10	9	8
TBSA							
R/WQ-0h							
7	6	5	4	3	2	1	0
TBSA						RESERVED	
R/WQ-0h						R-0h	

Table 21-44. MCAN_TXBC Field Descriptions

Bit	Field	Type	Reset	Description
31	RESERVED	R	0h	
30	TFQM	R/WQ	0h	Tx FIFO/Queue Mode 0 Tx FIFO operation 1 Tx Queue operation Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
29-24	TFQS	R/WQ	0h	Transmit FIFO/Queue Size 0 No Tx FIFO/Queue 1-32 Number of Tx Buffers used for Tx FIFO/Queue >32 Values greater than 32 are interpreted as 32 Note: Be aware that the sum of TFQS and NDTB may be not greater than 32. There is no check for erroneous configurations. The Tx Buffers section in the Message RAM starts with the dedicated Tx Buffers. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
23-22	RESERVED	R	0h	
21-16	NDTB	R/WQ	0h	Number of Dedicated Transmit Buffers 0 No Dedicated Tx Buffers 1-32 Number of Dedicated Tx Buffers >32 Values greater than 32 are interpreted as 32 Note: Be aware that the sum of TFQS and NDTB may be not greater than 32. There is no check for erroneous configurations. The Tx Buffers section in the Message RAM starts with the dedicated Tx Buffers. Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.
15-2	TBSA	R/WQ	0h	Tx Buffers Start Address. Start address of Tx Buffers section in Message RAM (32-bit word address). Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.

Table 21-44. MCAN_TXBC Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1-0	RESERVED	R	0h	

21.43 MCAN_TXFQS (Offset = 70C4h) [Reset = 00000000h]

MCAN_TXFQS is shown in [Figure 21-43](#) and described in [Table 21-45](#).

Return to the [Summary Table](#).

The Tx FIFO/Queue status is related to the pending Tx requests listed in register TXBRP. Therefore the effect of Add/Cancellation requests may be delayed due to a running Tx scan (TXBRP not yet updated).

Figure 21-43. MCAN_TXFQS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED		TFQF	TFQP				
R-0h		R-0h	R-0h				
15	14	13	12	11	10	9	8
RESERVED			TFGI				
R-0h			R-0h				
7	6	5	4	3	2	1	0
RESERVED		TFFL					
R-0h		R-0h					

Table 21-45. MCAN_TXFQS Field Descriptions

Bit	Field	Type	Reset	Description
31-22	RESERVED	R	0h	
21	TFQF	R	0h	Tx FIFO/Queue Full 0 Tx FIFO/Queue not full 1 Tx FIFO/Queue full
20-16	TFQP	R	0h	Tx FIFO/Queue Put Index. Tx FIFO/Queue write index pointer, range 0 to 31. Note: In case of mixed configurations where dedicated Tx Buffers are combined with a Tx FIFO or a Tx Queue, the Put and Get Indices indicate the number of the Tx Buffer starting with the first dedicated Tx Buffers. Example: For a configuration of 12 dedicated Tx Buffers and a Tx FIFO of 20 Buffers a Put Index of 15 points to the fourth buffer of the Tx FIFO.
15-13	RESERVED	R	0h	
12-8	TFGI	R	0h	Tx FIFO Get Index. Tx FIFO read index pointer, range 0 to 31. Read as zero when Tx Queue operation is configured (TXBC.TFQM = '1'). Note: In case of mixed configurations where dedicated Tx Buffers are combined with a Tx FIFO or a Tx Queue, the Put and Get Indices indicate the number of the Tx Buffer starting with the first dedicated Tx Buffers. Example: For a configuration of 12 dedicated Tx Buffers and a Tx FIFO of 20 Buffers a Put Index of 15 points to the fourth buffer of the Tx FIFO.
7-6	RESERVED	R	0h	
5-0	TFFL	R	0h	Tx FIFO Free Level. Number of consecutive free Tx FIFO elements starting from TFGI, range 0 to 32. Read as zero when Tx Queue operation is configured (TXBC.TFQM = '1').

21.44 MCAN_TXESC (Offset = 70C8h) [Reset = 00000000h]

MCAN_TXESC is shown in [Figure 21-44](#) and described in [Table 21-46](#).

Return to the [Summary Table](#).

Configures the number of data bytes belonging to a Tx Buffer element. Data field sizes > 8 bytes are intended for CAN FD operation only.

Figure 21-44. MCAN_TXESC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												TBDS			
R-0h												R/WQ-0h			

Table 21-46. MCAN_TXESC Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R	0h	
2-0	TBDS	R/WQ	0h	Tx Buffer Data Field Size 000 8 byte data field 001 12 byte data field 010 16 byte data field 011 20 byte data field 100 24 byte data field 101 32 byte data field 110 48 byte data field 111 64 byte data field Note: In case the data length code DLC of a Tx Buffer element is configured to a value higher than the Tx Buffer data field size TXESC.TBDS, the bytes not defined by the Tx Buffer are transmitted as '0xCC' (padding bytes). Qualified Write is possible only with CCCR.CCE='1' and CCCR.INIT='1'.

21.45 MCAN_TXBRP (Offset = 70CCh) [Reset = 00000000h]

MCAN_TXBRP is shown in [Figure 21-45](#) and described in [Table 21-47](#).

Return to the [Summary Table](#).

MCAN Tx Buffer Request Pending

Figure 21-45. MCAN_TXBRP

31	30	29	28	27	26	25	24
TRP31	TRP30	TRP29	TRP28	TRP27	TRP26	TRP25	TRP24
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
TRP23	TRP22	TRP21	TRP20	TRP19	TRP18	TRP17	TRP16
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
TRP15	TRP14	TRP13	TRP12	TRP11	TRP10	TRP9	TRP8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
TRP7	TRP6	TRP5	TRP4	TRP3	TRP2	TRP1	TRP0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 21-47. MCAN_TXBRP Field Descriptions

Bit	Field	Type	Reset	Description
31	TRP31	R	0h	Transmission Request Pending 31. See description for bit 0.
30	TRP30	R	0h	Transmission Request Pending 30. See description for bit 0.
29	TRP29	R	0h	Transmission Request Pending 29. See description for bit 0.
28	TRP28	R	0h	Transmission Request Pending 28. See description for bit 0.
27	TRP27	R	0h	Transmission Request Pending 27. See description for bit 0.
26	TRP26	R	0h	Transmission Request Pending 26. See description for bit 0.
25	TRP25	R	0h	Transmission Request Pending 25. See description for bit 0.
24	TRP24	R	0h	Transmission Request Pending 24. See description for bit 0.
23	TRP23	R	0h	Transmission Request Pending 23. See description for bit 0.
22	TRP22	R	0h	Transmission Request Pending 22. See description for bit 0.
21	TRP21	R	0h	Transmission Request Pending 21. See description for bit 0.
20	TRP20	R	0h	Transmission Request Pending 20. See description for bit 0.
19	TRP19	R	0h	Transmission Request Pending 19. See description for bit 0.
18	TRP18	R	0h	Transmission Request Pending 18. See description for bit 0.
17	TRP17	R	0h	Transmission Request Pending 17. See description for bit 0.
16	TRP16	R	0h	Transmission Request Pending 16. See description for bit 0.
15	TRP15	R	0h	Transmission Request Pending 15. See description for bit 0.
14	TRP14	R	0h	Transmission Request Pending 14. See description for bit 0.
13	TRP13	R	0h	Transmission Request Pending 13. See description for bit 0.
12	TRP12	R	0h	Transmission Request Pending 12. See description for bit 0.
11	TRP11	R	0h	Transmission Request Pending 11. See description for bit 0.
10	TRP10	R	0h	Transmission Request Pending 10. See description for bit 0.
9	TRP9	R	0h	Transmission Request Pending 9. See description for bit 0.
8	TRP8	R	0h	Transmission Request Pending 8. See description for bit 0.
7	TRP7	R	0h	Transmission Request Pending 7. See description for bit 0.

Table 21-47. MCAN_TXBRP Field Descriptions (continued)

Bit	Field	Type	Reset	Description
6	TRP6	R	0h	Transmission Request Pending 6. See description for bit 0.
5	TRP5	R	0h	Transmission Request Pending 5. See description for bit 0.
4	TRP4	R	0h	Transmission Request Pending 4. See description for bit 0.
3	TRP3	R	0h	Transmission Request Pending 3. See description for bit 0.
2	TRP2	R	0h	Transmission Request Pending 2. See description for bit 0.
1	TRP1	R	0h	Transmission Request Pending 1. See description for bit 0.
0	TRP0	R	0h	Transmission Request Pending 0. Each Tx Buffer has its own Transmission Request Pending bit. The bits are set via register TXBAR. The bits are reset after a requested transmission has completed or has been cancelled via register TXBCR. TXBRP bits are set only for those Tx Buffers configured via TXBC. After a TXBRP bit has been set, a Tx scan is started to check for the pending Tx request with the highest priority (Tx Buffer with lowest Message ID). A cancellation request resets the corresponding transmission request pending bit of register TXBRP. In case a transmission has already been started when a cancellation is requested, this is done at the end of the transmission, regardless whether the transmission was successful or not. The cancellation request bits are reset directly after the corresponding TXBRP bit has been reset. After a cancellation has been requested, a finished cancellation is signalled via TXBCF - after successful transmission together with the corresponding TXBTO bit - when the transmission has not yet been started at the point of cancellation - when the transmission has been aborted due to lost arbitration - when an error occurred during frame transmission In DAR mode all transmissions are automatically cancelled if they are not successful. The corresponding TXBCF bit is set for all unsuccessful transmissions. 0 No transmission request pending 1 Transmission request pending Note: TXBRP bits which are set while a Tx scan is in progress are not considered during this particular Tx scan. In case a cancellation is requested for such a Tx Buffer, this Add Request is cancelled immediately, the corresponding TXBRP bit is reset.

21.46 MCAN_TXBAR (Offset = 70D0h) [Reset = 00000000h]

MCAN_TXBAR is shown in [Figure 21-46](#) and described in [Table 21-48](#).

Return to the [Summary Table](#).

MCAN Tx Buffer Add Request

Figure 21-46. MCAN_TXBAR

31	30	29	28	27	26	25	24
AR31	AR30	AR29	AR28	AR27	AR26	AR25	AR24
R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h
23	22	21	20	19	18	17	16
AR23	AR22	AR21	AR20	AR19	AR18	AR17	AR16
R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h
15	14	13	12	11	10	9	8
AR15	AR14	AR13	AR12	AR11	AR10	AR9	AR8
R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h
7	6	5	4	3	2	1	0
AR7	AR6	AR5	AR4	AR3	AR2	AR1	AR0
R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h

Table 21-48. MCAN_TXBAR Field Descriptions

Bit	Field	Type	Reset	Description
31	AR31	R/WQ	0h	Add Request 31. See description for bit 0.
30	AR30	R/WQ	0h	Add Request 30. See description for bit 0.
29	AR29	R/WQ	0h	Add Request 29. See description for bit 0.
28	AR28	R/WQ	0h	Add Request 28. See description for bit 0.
27	AR27	R/WQ	0h	Add Request 27. See description for bit 0.
26	AR26	R/WQ	0h	Add Request 26. See description for bit 0.
25	AR25	R/WQ	0h	Add Request 25. See description for bit 0.
24	AR24	R/WQ	0h	Add Request 24. See description for bit 0.
23	AR23	R/WQ	0h	Add Request 23. See description for bit 0.
22	AR22	R/WQ	0h	Add Request 22. See description for bit 0.
21	AR21	R/WQ	0h	Add Request 21. See description for bit 0.
20	AR20	R/WQ	0h	Add Request 20. See description for bit 0.
19	AR19	R/WQ	0h	Add Request 19. See description for bit 0.
18	AR18	R/WQ	0h	Add Request 18. See description for bit 0.
17	AR17	R/WQ	0h	Add Request 17. See description for bit 0.
16	AR16	R/WQ	0h	Add Request 16. See description for bit 0.
15	AR15	R/WQ	0h	Add Request 15. See description for bit 0.
14	AR14	R/WQ	0h	Add Request 14. See description for bit 0.
13	AR13	R/WQ	0h	Add Request 13. See description for bit 0.
12	AR12	R/WQ	0h	Add Request 12. See description for bit 0.
11	AR11	R/WQ	0h	Add Request 11. See description for bit 0.
10	AR10	R/WQ	0h	Add Request 10. See description for bit 0.
9	AR9	R/WQ	0h	Add Request 9. See description for bit 0.
8	AR8	R/WQ	0h	Add Request 8. See description for bit 0.
7	AR7	R/WQ	0h	Add Request 7. See description for bit 0.

Table 21-48. MCAN_TXBAR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
6	AR6	R/WQ	0h	Add Request 6. See description for bit 0.
5	AR5	R/WQ	0h	Add Request 5. See description for bit 0.
4	AR4	R/WQ	0h	Add Request 4. See description for bit 0.
3	AR3	R/WQ	0h	Add Request 3. See description for bit 0.
2	AR2	R/WQ	0h	Add Request 2. See description for bit 0.
1	AR1	R/WQ	0h	Add Request 1. See description for bit 0.
0	AR0	R/WQ	0h	Add Request 0. Each Tx Buffer has its own Add Request bit. Writing a '1' will set the corresponding Add Request bit; writing a '0' has no impact. This enables the Host to set transmission requests for multiple Tx Buffers with one write to TXBAR. TXBAR bits are set only for those Tx Buffers configured via TXBC. When no Tx scan is running, the bits are reset immediately, else the bits remain set until the Tx scan process has completed. 0 No transmission request added 1 Transmission requested added Note: If an add request is applied for a Tx Buffer with pending transmission request (corresponding TXBRP bit already set), this add request is ignored. Qualified Write is possible only with CCCR.CCE='0'

21.47 MCAN_TXBCR (Offset = 70D4h) [Reset = 00000000h]

MCAN_TXBCR is shown in [Figure 21-47](#) and described in [Table 21-49](#).

Return to the [Summary Table](#).

MCAN Tx Buffer Cancellation Request

Figure 21-47. MCAN_TXBCR

31	30	29	28	27	26	25	24
CR31	CR30	CR29	CR28	CR27	CR26	CR25	CR24
R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h
23	22	21	20	19	18	17	16
CR23	CR22	CR21	CR20	CR19	CR18	CR17	CR16
R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h
15	14	13	12	11	10	9	8
CR15	CR14	CR13	CR12	CR11	CR10	CR9	CR8
R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h
7	6	5	4	3	2	1	0
CR7	CR6	CR5	CR4	CR3	CR2	CR1	CR0
R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h	R/WQ-0h

Table 21-49. MCAN_TXBCR Field Descriptions

Bit	Field	Type	Reset	Description
31	CR31	R/WQ	0h	Cancellation Request 31. See description for bit 0.
30	CR30	R/WQ	0h	Cancellation Request 30. See description for bit 0.
29	CR29	R/WQ	0h	Cancellation Request 29. See description for bit 0.
28	CR28	R/WQ	0h	Cancellation Request 28. See description for bit 0.
27	CR27	R/WQ	0h	Cancellation Request 27. See description for bit 0.
26	CR26	R/WQ	0h	Cancellation Request 26. See description for bit 0.
25	CR25	R/WQ	0h	Cancellation Request 25. See description for bit 0.
24	CR24	R/WQ	0h	Cancellation Request 24. See description for bit 0.
23	CR23	R/WQ	0h	Cancellation Request 23. See description for bit 0.
22	CR22	R/WQ	0h	Cancellation Request 22. See description for bit 0.
21	CR21	R/WQ	0h	Cancellation Request 21. See description for bit 0.
20	CR20	R/WQ	0h	Cancellation Request 20. See description for bit 0.
19	CR19	R/WQ	0h	Cancellation Request 19. See description for bit 0.
18	CR18	R/WQ	0h	Cancellation Request 18. See description for bit 0.
17	CR17	R/WQ	0h	Cancellation Request 17. See description for bit 0.
16	CR16	R/WQ	0h	Cancellation Request 16. See description for bit 0.
15	CR15	R/WQ	0h	Cancellation Request 15. See description for bit 0.
14	CR14	R/WQ	0h	Cancellation Request 14. See description for bit 0.
13	CR13	R/WQ	0h	Cancellation Request 13. See description for bit 0.
12	CR12	R/WQ	0h	Cancellation Request 12. See description for bit 0.
11	CR11	R/WQ	0h	Cancellation Request 11. See description for bit 0.
10	CR10	R/WQ	0h	Cancellation Request 10. See description for bit 0.
9	CR9	R/WQ	0h	Cancellation Request 9. See description for bit 0.
8	CR8	R/WQ	0h	Cancellation Request 8. See description for bit 0.
7	CR7	R/WQ	0h	Cancellation Request 7. See description for bit 0.

Table 21-49. MCAN_TXBCR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
6	CR6	R/WQ	0h	Cancellation Request 6. See description for bit 0.
5	CR5	R/WQ	0h	Cancellation Request 5. See description for bit 0.
4	CR4	R/WQ	0h	Cancellation Request 4. See description for bit 0.
3	CR3	R/WQ	0h	Cancellation Request 3. See description for bit 0.
2	CR2	R/WQ	0h	Cancellation Request 2. See description for bit 0.
1	CR1	R/WQ	0h	Cancellation Request 1. See description for bit 0.
0	CR0	R/WQ	0h	Cancellation Request 0. Each Tx Buffer has its own Cancellation Request bit. Writing a '1' will set the corresponding Cancellation Request bit; writing a '0' has no impact. This enables the Host to set cancellation requests for multiple Tx Buffers with one write to TXBCR. TXBCR bits are set only for those Tx Buffers configured via TXBC. The bits remain set until the corresponding bit of TXBRP is reset. 0 No cancellation pending 1 Cancellation pending Qualified Write is possible only with CCCR.CCE='0'

21.48 MCAN_TXBTO (Offset = 70D8h) [Reset = 00000000h]

MCAN_TXBTO is shown in [Figure 21-48](#) and described in [Table 21-50](#).

Return to the [Summary Table](#).

MCAN Tx Buffer Transmission Occurred

Figure 21-48. MCAN_TXBTO

31	30	29	28	27	26	25	24
TO31	TO30	TO29	TO28	TO27	TO26	TO25	TO24
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
TO23	TO22	TO21	TO20	TO19	TO18	TO17	TO16
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
TO15	TO14	TO13	TO12	TO11	TO10	TO9	TO8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
TO7	TO6	TO5	TO4	TO3	TO2	TO1	TO0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 21-50. MCAN_TXBTO Field Descriptions

Bit	Field	Type	Reset	Description
31	TO31	R	0h	Transmission Occurred 31. See description for bit 0.
30	TO30	R	0h	Transmission Occurred 30. See description for bit 0.
29	TO29	R	0h	Transmission Occurred 29. See description for bit 0.
28	TO28	R	0h	Transmission Occurred 28. See description for bit 0.
27	TO27	R	0h	Transmission Occurred 27. See description for bit 0.
26	TO26	R	0h	Transmission Occurred 26. See description for bit 0.
25	TO25	R	0h	Transmission Occurred 25. See description for bit 0.
24	TO24	R	0h	Transmission Occurred 24. See description for bit 0.
23	TO23	R	0h	Transmission Occurred 23. See description for bit 0.
22	TO22	R	0h	Transmission Occurred 22. See description for bit 0.
21	TO21	R	0h	Transmission Occurred 21. See description for bit 0.
20	TO20	R	0h	Transmission Occurred 20. See description for bit 0.
19	TO19	R	0h	Transmission Occurred 19. See description for bit 0.
18	TO18	R	0h	Transmission Occurred 18. See description for bit 0.
17	TO17	R	0h	Transmission Occurred 17. See description for bit 0.
16	TO16	R	0h	Transmission Occurred 16. See description for bit 0.
15	TO15	R	0h	Transmission Occurred 15. See description for bit 0.
14	TO14	R	0h	Transmission Occurred 14. See description for bit 0.
13	TO13	R	0h	Transmission Occurred 13. See description for bit 0.
12	TO12	R	0h	Transmission Occurred 12. See description for bit 0.
11	TO11	R	0h	Transmission Occurred 11. See description for bit 0.
10	TO10	R	0h	Transmission Occurred 10. See description for bit 0.
9	TO9	R	0h	Transmission Occurred 9. See description for bit 0.
8	TO8	R	0h	Transmission Occurred 8. See description for bit 0.
7	TO7	R	0h	Transmission Occurred 7. See description for bit 0.

Table 21-50. MCAN_TXBTO Field Descriptions (continued)

Bit	Field	Type	Reset	Description
6	TO6	R	0h	Transmission Occurred 6. See description for bit 0.
5	TO5	R	0h	Transmission Occurred 5. See description for bit 0.
4	TO4	R	0h	Transmission Occurred 4. See description for bit 0.
3	TO3	R	0h	Transmission Occurred 3. See description for bit 0.
2	TO2	R	0h	Transmission Occurred 2. See description for bit 0.
1	TO1	R	0h	Transmission Occurred 1. See description for bit 0.
0	TO0	R	0h	Transmission Occurred 0. Each Tx Buffer has its own Transmission Occurred bit. The bits are set when the corresponding TXBRP bit is cleared after a successful transmission. The bits are reset when a new transmission is requested by writing a '1' to the corresponding bit of register TXBAR. 0 No transmission occurred 1 Transmission occurred

21.49 MCAN_TXBCF (Offset = 70DCh) [Reset = 00000000h]

MCAN_TXBCF is shown in [Figure 21-49](#) and described in [Table 21-51](#).

Return to the [Summary Table](#).

MCAN Tx Buffer Cancellation Finished

Figure 21-49. MCAN_TXBCF

31	30	29	28	27	26	25	24
CF31	CF30	CF29	CF28	CF27	CF26	CF25	CF24
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
CF23	CF22	CF21	CF20	CF19	CF18	CF17	CF16
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
15	14	13	12	11	10	9	8
CF15	CF14	CF13	CF12	CF11	CF10	CF9	CF8
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
CF7	CF6	CF5	CF4	CF3	CF2	CF1	CF0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 21-51. MCAN_TXBCF Field Descriptions

Bit	Field	Type	Reset	Description
31	CF31	R	0h	Cancellation Finished 31. See description for bit 0.
30	CF30	R	0h	Cancellation Finished 30. See description for bit 0.
29	CF29	R	0h	Cancellation Finished 29. See description for bit 0.
28	CF28	R	0h	Cancellation Finished 28. See description for bit 0.
27	CF27	R	0h	Cancellation Finished 27. See description for bit 0.
26	CF26	R	0h	Cancellation Finished 26. See description for bit 0.
25	CF25	R	0h	Cancellation Finished 25. See description for bit 0.
24	CF24	R	0h	Cancellation Finished 24. See description for bit 0.
23	CF23	R	0h	Cancellation Finished 23. See description for bit 0.
22	CF22	R	0h	Cancellation Finished 22. See description for bit 0.
21	CF21	R	0h	Cancellation Finished 21. See description for bit 0.
20	CF20	R	0h	Cancellation Finished 20. See description for bit 0.
19	CF19	R	0h	Cancellation Finished 19. See description for bit 0.
18	CF18	R	0h	Cancellation Finished 18. See description for bit 0.
17	CF17	R	0h	Cancellation Finished 17. See description for bit 0.
16	CF16	R	0h	Cancellation Finished 16. See description for bit 0.
15	CF15	R	0h	Cancellation Finished 15. See description for bit 0.
14	CF14	R	0h	Cancellation Finished 14. See description for bit 0.
13	CF13	R	0h	Cancellation Finished 13. See description for bit 0.
12	CF12	R	0h	Cancellation Finished 12. See description for bit 0.
11	CF11	R	0h	Cancellation Finished 11. See description for bit 0.
10	CF10	R	0h	Cancellation Finished 10. See description for bit 0.
9	CF9	R	0h	Cancellation Finished 9. See description for bit 0.
8	CF8	R	0h	Cancellation Finished 8. See description for bit 0.
7	CF7	R	0h	Cancellation Finished 7. See description for bit 0.

Table 21-51. MCAN_TXBCF Field Descriptions (continued)

Bit	Field	Type	Reset	Description
6	CF6	R	0h	Cancellation Finished 6. See description for bit 0.
5	CF5	R	0h	Cancellation Finished 5. See description for bit 0.
4	CF4	R	0h	Cancellation Finished 4. See description for bit 0.
3	CF3	R	0h	Cancellation Finished 3. See description for bit 0.
2	CF2	R	0h	Cancellation Finished 2. See description for bit 0.
1	CF1	R	0h	Cancellation Finished 1. See description for bit 0.
0	CF0	R	0h	Cancellation Finished 0. Each Tx Buffer has its own Cancellation Finished bit. The bits are set when the corresponding TXBRP bit is cleared after a cancellation was requested via TXBCR. In case the corresponding TXBRP bit was not set at the point of cancellation, CF is set immediately. The bits are reset when a new transmission is requested by writing a '1' to the corresponding bit of register TXBAR. 0 No transmit buffer cancellation 1 Transmit buffer cancellation finished

21.50 MCAN_TXBTIE (Offset = 70E0h) [Reset = 00000000h]

MCAN_TXBTIE is shown in [Figure 21-50](#) and described in [Table 21-52](#).

Return to the [Summary Table](#).

MCAN Tx Buffer Transmission Interrupt Enable

Figure 21-50. MCAN_TXBTIE

31	30	29	28	27	26	25	24
TIE31	TIE30	TIE29	TIE28	TIE27	TIE26	TIE25	TIE24
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
TIE23	TIE22	TIE21	TIE20	TIE19	TIE18	TIE17	TIE16
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
TIE15	TIE14	TIE13	TIE12	TIE11	TIE10	TIE9	TIE8
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
TIE7	TIE6	TIE5	TIE4	TIE3	TIE2	TIE1	TIE0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 21-52. MCAN_TXBTIE Field Descriptions

Bit	Field	Type	Reset	Description
31	TIE31	R/W	0h	Transmission Interrupt Enable 31. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
30	TIE30	R/W	0h	Transmission Interrupt Enable 30. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
29	TIE29	R/W	0h	Transmission Interrupt Enable 29. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
28	TIE28	R/W	0h	Transmission Interrupt Enable 28. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
27	TIE27	R/W	0h	Transmission Interrupt Enable 27. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
26	TIE26	R/W	0h	Transmission Interrupt Enable 26. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
25	TIE25	R/W	0h	Transmission Interrupt Enable 25. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
24	TIE24	R/W	0h	Transmission Interrupt Enable 24. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable

Table 21-52. MCAN_TXBTIE Field Descriptions (continued)

Bit	Field	Type	Reset	Description
23	TIE23	R/W	0h	Transmission Interrupt Enable 23. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
22	TIE22	R/W	0h	Transmission Interrupt Enable 22. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
21	TIE21	R/W	0h	Transmission Interrupt Enable 21. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
20	TIE20	R/W	0h	Transmission Interrupt Enable 20. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
19	TIE19	R/W	0h	Transmission Interrupt Enable 19. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
18	TIE18	R/W	0h	Transmission Interrupt Enable 18. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
17	TIE17	R/W	0h	Transmission Interrupt Enable 17. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
16	TIE16	R/W	0h	Transmission Interrupt Enable 16. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
15	TIE15	R/W	0h	Transmission Interrupt Enable 15. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
14	TIE14	R/W	0h	Transmission Interrupt Enable 14. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
13	TIE13	R/W	0h	Transmission Interrupt Enable 13. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
12	TIE12	R/W	0h	Transmission Interrupt Enable 12. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
11	TIE11	R/W	0h	Transmission Interrupt Enable 11. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
10	TIE10	R/W	0h	Transmission Interrupt Enable 10. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable

Table 21-52. MCAN_TXBTIE Field Descriptions (continued)

Bit	Field	Type	Reset	Description
9	TIE9	R/W	0h	Transmission Interrupt Enable 9. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
8	TIE8	R/W	0h	Transmission Interrupt Enable 8. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
7	TIE7	R/W	0h	Transmission Interrupt Enable 7. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
6	TIE6	R/W	0h	Transmission Interrupt Enable 6. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
5	TIE5	R/W	0h	Transmission Interrupt Enable 5. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
4	TIE4	R/W	0h	Transmission Interrupt Enable 4. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
3	TIE3	R/W	0h	Transmission Interrupt Enable 3. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
2	TIE2	R/W	0h	Transmission Interrupt Enable 2. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
1	TIE1	R/W	0h	Transmission Interrupt Enable 1. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable
0	TIE0	R/W	0h	Transmission Interrupt Enable 0. Each Tx Buffer has its own Transmission Interrupt Enable bit. 0 Transmission interrupt disabled 1 Transmission interrupt enable

21.51 MCAN_TXBCIE (Offset = 70E4h) [Reset = 00000000h]

MCAN_TXBCIE is shown in [Figure 21-51](#) and described in [Table 21-53](#).

Return to the [Summary Table](#).

MCAN Tx Buffer Cancellation Finished Interrupt Enable

Figure 21-51. MCAN_TXBCIE

31	30	29	28	27	26	25	24
CFIE31	CFIE30	CFIE29	CFIE28	CFIE27	CFIE26	CFIE25	CFIE24
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
CFIE23	CFIE22	CFIE21	CFIE20	CFIE19	CFIE18	CFIE17	CFIE16
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
CFIE15	CFIE14	CFIE13	CFIE12	CFIE11	CFIE10	CFIE9	CFIE8
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
CFIE7	CFIE6	CFIE5	CFIE4	CFIE3	CFIE2	CFIE1	CFIE0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 21-53. MCAN_TXBCIE Field Descriptions

Bit	Field	Type	Reset	Description
31	CFIE31	R/W	0h	Cancellation Finished Interrupt Enable 31. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
30	CFIE30	R/W	0h	Cancellation Finished Interrupt Enable 30. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
29	CFIE29	R/W	0h	Cancellation Finished Interrupt Enable 29. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
28	CFIE28	R/W	0h	Cancellation Finished Interrupt Enable 28. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
27	CFIE27	R/W	0h	Cancellation Finished Interrupt Enable 27. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
26	CFIE26	R/W	0h	Cancellation Finished Interrupt Enable 26. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
25	CFIE25	R/W	0h	Cancellation Finished Interrupt Enable 25. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
24	CFIE24	R/W	0h	Cancellation Finished Interrupt Enable 24. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled

Table 21-53. MCAN_TXBCIE Field Descriptions (continued)

Bit	Field	Type	Reset	Description
23	CFIE23	R/W	0h	Cancellation Finished Interrupt Enable 23. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
22	CFIE22	R/W	0h	Cancellation Finished Interrupt Enable 22. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
21	CFIE21	R/W	0h	Cancellation Finished Interrupt Enable 21. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
20	CFIE20	R/W	0h	Cancellation Finished Interrupt Enable 20. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
19	CFIE19	R/W	0h	Cancellation Finished Interrupt Enable 19. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
18	CFIE18	R/W	0h	Cancellation Finished Interrupt Enable 18. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
17	CFIE17	R/W	0h	Cancellation Finished Interrupt Enable 17. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
16	CFIE16	R/W	0h	Cancellation Finished Interrupt Enable 16. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
15	CFIE15	R/W	0h	Cancellation Finished Interrupt Enable 15. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
14	CFIE14	R/W	0h	Cancellation Finished Interrupt Enable 14. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
13	CFIE13	R/W	0h	Cancellation Finished Interrupt Enable 13. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
12	CFIE12	R/W	0h	Cancellation Finished Interrupt Enable 12. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
11	CFIE11	R/W	0h	Cancellation Finished Interrupt Enable 11. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
10	CFIE10	R/W	0h	Cancellation Finished Interrupt Enable 10. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled

Table 21-53. MCAN_TXBCIE Field Descriptions (continued)

Bit	Field	Type	Reset	Description
9	CFIE9	R/W	0h	Cancellation Finished Interrupt Enable 9. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
8	CFIE8	R/W	0h	Cancellation Finished Interrupt Enable 8. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
7	CFIE7	R/W	0h	Cancellation Finished Interrupt Enable 7. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
6	CFIE6	R/W	0h	Cancellation Finished Interrupt Enable 6. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
5	CFIE5	R/W	0h	Cancellation Finished Interrupt Enable 5. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
4	CFIE4	R/W	0h	Cancellation Finished Interrupt Enable 4. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
3	CFIE3	R/W	0h	Cancellation Finished Interrupt Enable 3. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
2	CFIE2	R/W	0h	Cancellation Finished Interrupt Enable 2. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
1	CFIE1	R/W	0h	Cancellation Finished Interrupt Enable 1. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled
0	CFIE0	R/W	0h	Cancellation Finished Interrupt Enable 0. Each Tx Buffer has its own Cancellation Finished Interrupt Enable bit. 0 Cancellation finished interrupt disabled 1 Cancellation finished interrupt enabled

21.52 MCAN_TXEFC (Offset = 70F0h) [Reset = 00000000h]

MCAN_TXEFC is shown in [Figure 21-52](#) and described in [Table 21-54](#).

Return to the [Summary Table](#).

MCAN Tx Event FIFO Configuration

Figure 21-52. MCAN_TXEFC

31	30	29	28	27	26	25	24
RESERVED				EFWM			
R-0h				R/WQ-0h			
23	22	21	20	19	18	17	16
RESERVED				EFS			
R-0h				R/WQ-0h			
15	14	13	12	11	10	9	8
EFSA							
R/WQ-0h							
7	6	5	4	3	2	1	0
EFSA						RESERVED	
R/WQ-0h						R-0h	

Table 21-54. MCAN_TXEFC Field Descriptions

Bit	Field	Type	Reset	Description
31-30	RESERVED	R	0h	
29-24	EFWM	R/WQ	0h	Event FIFO Watermark 0 Watermark interrupt disabled 1-32 Level for Tx Event FIFO watermark interrupt (IR.TEFW) >32 Watermark interrupt disabled
23-22	RESERVED	R	0h	
21-16	EFS	R/WQ	0h	Event FIFO Size. The Tx Event FIFO elements are indexed from 0 to EFS - 1. 0 Tx Event FIFO disabled 1-32 Number of Tx Event FIFO elements >32 Values greater than 32 are interpreted as 32
15-2	EFSA	R/WQ	0h	Event FIFO Start Address. Start address of Tx Event FIFO in Message RAM (32-bit word address).
1-0	RESERVED	R	0h	

21.53 MCAN_TXEFS (Offset = 70F4h) [Reset = 00000000h]

MCAN_TXEFS is shown in [Figure 21-53](#) and described in [Table 21-55](#).

Return to the [Summary Table](#).

MCAN Tx Event FIFO Status

Figure 21-53. MCAN_TXEFS

31	30	29	28	27	26	25	24
RESERVED						TEFL	EFF
R-0h						R-0h	R-0h
23	22	21	20	19	18	17	16
RESERVED				EFPI			
R-0h				R-0h			
15	14	13	12	11	10	9	8
RESERVED				EFGI			
R-0h				R-0h			
7	6	5	4	3	2	1	0
RESERVED			EFFL				
R-0h			R-0h				

Table 21-55. MCAN_TXEFS Field Descriptions

Bit	Field	Type	Reset	Description
31-26	RESERVED	R	0h	
25	TEFL	R	0h	Tx Event FIFO Element Lost. This bit is a copy of interrupt flag IR.TEFL. When IR.TEFL is reset, this bit is also reset. 0 No Tx Event FIFO element lost 1 Tx Event FIFO element lost, also set after write attempt to Tx Event FIFO of size zero.
24	EFF	R	0h	Event FIFO Full 0 Tx Event FIFO not full 1 Tx Event FIFO full
23-21	RESERVED	R	0h	
20-16	EFPI	R	0h	Event FIFO Put Index. Tx Event FIFO write index pointer, range 0 to 31.
15-13	RESERVED	R	0h	
12-8	EFGI	R	0h	Event FIFO Get Index. Tx Event FIFO read index pointer, range 0 to 31.
7-6	RESERVED	R	0h	
5-0	EFFL	R	0h	Event FIFO Fill Level. Number of elements stored in Tx Event FIFO, range 0 to 32.

21.54 MCAN_TXEFA (Offset = 70F8h) [Reset = 00000000h]

MCAN_TXEFA is shown in [Figure 21-54](#) and described in [Table 21-56](#).

Return to the [Summary Table](#).

MCAN Tx Event FIFO Acknowledge

Figure 21-54. MCAN_TXEFA

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																												EFAI			
R-0h																												R/W-0h			

Table 21-56. MCAN_TXEFA Field Descriptions

Bit	Field	Type	Reset	Description
31-5	RESERVED	R	0h	
4-0	EFAI	R/W	0h	Event FIFO Acknowledge Index. After the Host has read an element or a sequence of elements from the Tx Event FIFO it has to write the index of the last element read from Tx Event FIFO to EFAI. This will set the Tx Event FIFO Get Index TXEFS.EFGI to EFAI + 1 and update the Event FIFO Fill Level TXEFS.EFFL.

21.55 MCANSS_PID (Offset = 7200h) [Reset = 68E04901h]

MCANSS_PID is shown in [Figure 21-55](#) and described in [Table 21-57](#).

Return to the [Summary Table](#).

MCAN Subsystem Revision Register

Figure 21-55. MCANSS_PID

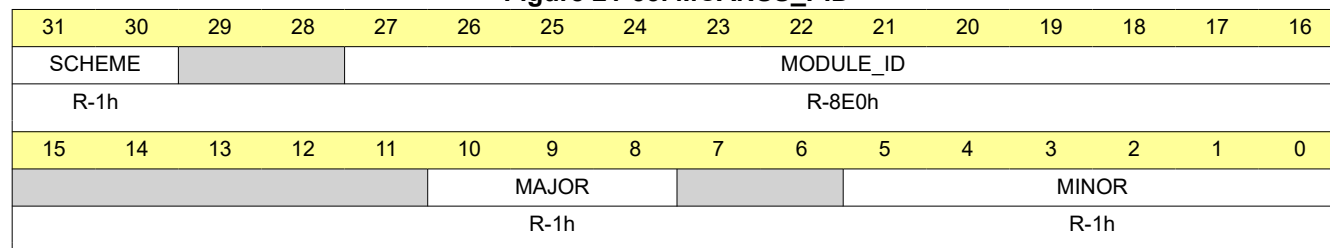


Table 21-57. MCANSS_PID Field Descriptions

Bit	Field	Type	Reset	Description
31-30	SCHEME	R	1h	PID Register Scheme
27-16	MODULE_ID	R	8E0h	Module Identification Number
10-8	MAJOR	R	1h	Major Revision of the MCAN Subsystem
5-0	MINOR	R	1h	Minor Revision of the MCAN Subsystem

21.56 MCANSS_CTRL (Offset = 7204h) [Reset = 00000008h]

MCANSS_CTRL is shown in [Figure 21-56](#) and described in [Table 21-58](#).

Return to the [Summary Table](#).

MCAN Subsystem Control Register

Figure 21-56. MCANSS_CTRL

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED	EXT_TS_CNTR_EN	AUTOWAKEUP	WAKEUPREQEN	DBGSUSP_FREE	RESERVED		
R-0h	R/W-0h	R/W-0h	R/W-0h	R/W-1h	R-0h		

Table 21-58. MCANSS_CTRL Field Descriptions

Bit	Field	Type	Reset	Description
31-7	RESERVED	R	0h	
6	EXT_TS_CNTR_EN	R/W	0h	External Timestamp Counter Enable. 0 External timestamp counter disabled 1 External timestamp counter enabled
5	AUTOWAKEUP	R/W	0h	Automatic Wakeup Enable. Enables the MCANSS to automatically clear the MCAN CCCR.INIT bit, fully waking the MCAN up, on an enabled wakeup request. 0 Disable the automatic write to CCCR.INIT 1 Enable the automatic write to CCCR.INIT
4	WAKEUPREQEN	R/W	0h	Wakeup Request Enable. Enables the MCANSS to wakeup on CAN RXD activity. 0 Disable wakeup request 1 Enables wakeup request
3	DBGSUSP_FREE	R/W	1h	Debug Suspend Free Bit. Enables debug suspend. 0 Honor debug suspend 1 Disregard debug suspend
2-0	RESERVED	R	0h	

21.57 MCANSS_STAT (Offset = 7208h) [Reset = 0000000Xh]

MCANSS_STAT is shown in [Figure 21-57](#) and described in [Table 21-59](#).

Return to the [Summary Table](#).

MCAN Subsystem Status Register

Figure 21-57. MCANSS_STAT

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED					ENABLE_FDO E	MEM_INIT_DO NE	RESET
R-0h					R-X	R-0h	R-0h

Table 21-59. MCANSS_STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R	0h	
2	ENABLE_FDOE	R	X	Enable FD (Flexible Data-Rate) Configuration Reflects the value of mcanss_enable_fdoe configuration port, -h = mcanss_enable_fdoe.
1	MEM_INIT_DONE	R	0h	Memory Initialization Done. 0 Message RAM initialization is in progress 1 Message RAM is initialized for use
0	RESET	R	0h	Soft Reset Status. 0 Not in reset 1 Reset is in progress

21.58 MCANSS_ICS (Offset = 720Ch) [Reset = 00000000h]

MCANSS_ICS is shown in [Figure 21-58](#) and described in [Table 21-60](#).

Return to the [Summary Table](#).

MCAN Subsystem Interrupt Clear Shadow Register

Figure 21-58. MCANSS_ICS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							EXT_TS_CNTR_OVFL
R-0h							R-0/W1C-0h

Table 21-60. MCANSS_ICS Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	EXT_TS_CNTR_OVFL	R-0/W1C	0h	External Timestamp Counter Overflow Interrupt Status Clear. Reads always return a 0. 0 Write of '0' has no effect 1 Write of '1' clears the MCANSS_IRS.EXT_TS_CNTR_OVFL bit

21.59 MCANSS_IRS (Offset = 7210h) [Reset = 00000000h]

MCANSS_IRS is shown in [Figure 21-59](#) and described in [Table 21-61](#).

Return to the [Summary Table](#).

MCAN Subsystem Interrupt Raw Status Register

Figure 21-59. MCANSS_IRS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							EXT_TS_CNTR_OVFL
R-0h							R/W1S-0h

Table 21-61. MCANSS_IRS Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	EXT_TS_CNTR_OVFL	R/W1S	0h	External Timestamp Counter Overflow Interrupt Status. This bit is set by HW or by a SW write of '1'. To clear, use the MCANSS_IC.S_EXT_TS_CNTR_OVFL bit. 0 External timestamp counter has not overflowed 1 External timestamp counter has overflowed When this bit is set to '1' by HW or SW, the MCANSS_EXT_TS_UNSERVICED_INTR_CNTR.EXT_TS_INTR_CNTR bit field will increment by 1.

21.60 MCANSS_IECS (Offset = 7214h) [Reset = 00000000h]

MCANSS_IECS is shown in [Figure 21-60](#) and described in [Table 21-62](#).

Return to the [Summary Table](#).

MCAN Subsystem Interrupt Enable Clear Shadow Register

Figure 21-60. MCANSS_IECS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							EXT_TS_CNTR_OVFL
R-0h							R-0/W1C-0h

Table 21-62. MCANSS_IECS Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	EXT_TS_CNTR_OVFL	R-0/W1C	0h	External Timestamp Counter Overflow Interrupt Enable Clear. Reads always return a 0. 0 Write of '0' has no effect 1 Write of '1' clears the MCANSS_IES.EXT_TS_CNTR_OVFL bit

21.61 MCANSS_IE (Offset = 7218h) [Reset = 00000000h]

MCANSS_IE is shown in [Figure 21-61](#) and described in [Table 21-63](#).

Return to the [Summary Table](#).

MCAN Subsystem Interrupt Enable Register

Figure 21-61. MCANSS_IE

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							EXT_TS_CNTR _OVFL
R-0h							R/W1S-0h

Table 21-63. MCANSS_IE Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	EXT_TS_CNTR_OVFL	R/W1S	0h	External Timestamp Counter Overflow Interrupt Enable. A write of '0' has no effect. A write of '1' sets the MCANSS_IES.EXT_TS_CNTR_OVFL bit.

21.62 MCANSS_IES (Offset = 721Ch) [Reset = 00000000h]

MCANSS_IES is shown in [Figure 21-62](#) and described in [Table 21-64](#).

Return to the [Summary Table](#).

MCAN Subsystem Interrupt Enable Status

Figure 21-62. MCANSS_IES

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							EXT_TS_CNTR_OVFL
R-0h							R-0h

Table 21-64. MCANSS_IES Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	EXT_TS_CNTR_OVFL	R	0h	External Timestamp Counter Overflow Interrupt Enable Status. To set, use the CANSS_IE.EXT_TS_CNTR_OVFL bit. To clear, use the MCANSS_IECS.EXT_TS_CNTR_OVFL bit. 0 External timestamp counter overflow interrupt is not enabled 1 External timestamp counter overflow interrupt is enabled

21.63 MCANSS_EOI (Offset = 7220h) [Reset = 00000000h]

MCANSS_EOI is shown in [Figure 21-63](#) and described in [Table 21-65](#).

Return to the [Summary Table](#).

MCAN Subsystem End of Interrupt

Figure 21-63. MCANSS_EOI

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								EOI							
R-0h																								R-0/W1S-0h							

Table 21-65. MCANSS_EOI Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	EOI	R-0/W1S	0h	End of Interrupt. A write to this register will clear the associated interrupt. If the unserviced interrupt counter is > 1, another interrupt is generated. 0x00 External TS Interrupt is cleared 0x01 MCAN(0) interrupt is cleared 0x02 MCAN(1) interrupt is cleared Other writes are ignored.

21.64 MCANSS_EXT_TS_PRESCALER (Offset = 7224h) [Reset = 00000000h]

MCANSS_EXT_TS_PRESCALER is shown in [Figure 21-64](#) and described in [Table 21-66](#).

Return to the [Summary Table](#).

MCAN Subsystem External Timestamp Prescaler 0

Figure 21-64. MCANSS_EXT_TS_PRESCALER

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED								PRESCALER																							
R-0h								R/W-0h																							

Table 21-66. MCANSS_EXT_TS_PRESCALER Field Descriptions

Bit	Field	Type	Reset	Description
31-24	RESERVED	R	0h	
23-0	PRESCALER	R/W	0h	External Timestamp Prescaler Reload Value. The external timestamp count rate is the host (system) clock rate divided by this value, except in the case of 0. A zero value in this bit field will act identically to a value of 0x000001.

21.65 MCANSS_EXT_TS_UNSERVICED_INTR_CNTR (Offset = 7228h) [Reset = 00000000h]

MCANSS_EXT_TS_UNSERVICED_INTR_CNTR is shown in [Figure 21-65](#) and described in [Table 21-67](#).

Return to the [Summary Table](#).

MCAN Subsystem External Timestamp Unserviced Interrupts Counter

Figure 21-65. MCANSS_EXT_TS_UNSERVICED_INTR_CNTR

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				EXT_TS_INTR_CNTR			
R-0h				R-0h			

Table 21-67. MCANSS_EXT_TS_UNSERVICED_INTR_CNTR Field Descriptions

Bit	Field	Type	Reset	Description
31-5	RESERVED	R	0h	
4-0	EXT_TS_INTR_CNTR	R	0h	External Timestamp Counter Unserviced Rollover Interrupts. If this value is > 1, an MCANSS_EOI write of '1' to bit 0 will issue another interrupt. The status of this bit field is affected by the MCANSS_IRS.EXT_TS_CNTR_OVFL bit field.

21.66 MCANERR_REV (Offset = 7400h) [Reset = 66A0EA00h]

MCANERR_REV is shown in [Figure 21-66](#) and described in [Table 21-68](#).

Return to the [Summary Table](#).

MCAN Error Aggregator Revision Register

Figure 21-66. MCANERR_REV

31	30	29	28	27	26	25	24
SCHEME				MODULE_ID			
R-1h				R-6A0h			
23	22	21	20	19	18	17	16
MODULE_ID							
R-6A0h							
15	14	13	12	11	10	9	8
					REVMAJ		
R-2h							
7	6	5	4	3	2	1	0
		REVMIN					
R-0h							

Table 21-68. MCANERR_REV Field Descriptions

Bit	Field	Type	Reset	Description
31-30	SCHEME	R	1h	PID Register Scheme
27-16	MODULE_ID	R	6A0h	Module Identification Number
10-8	REVMAJ	R	2h	Major Revision of the Error Aggregator
5-0	REVMIN	R	0h	Minor Revision of the Error Aggregator

21.67 MCANERR_VECTOR (Offset = 7408h) [Reset = 00000000h]

MCANERR_VECTOR is shown in [Figure 21-67](#) and described in [Table 21-69](#).

Return to the [Summary Table](#).

Each error detection and correction (EDC) controller has a bank of error registers (offsets 0x10 - 0x3B) associated with it. These registers are accessed via an internal serial bus (SVBUS). To access them through the ECC aggregator the controller ID desired must be written to the ECC_VECTOR field, together with the RD_SVBUS trigger and RD_SVBUS_ADDRESS bit field. This initiates the serial read which consummates by setting the RD_SVBUS_DONE bit. At this point the addressed register may be read by a normal CPU read of the appropriate offset address.

Figure 21-67. MCANERR_VECTOR

31	30	29	28	27	26	25	24
RESERVED							RD_SVBUS_D ONE
R-0h							R-0h
23	22	21	20	19	18	17	16
RD_SVBUS_ADDRESS							
R/W-0h							
15	14	13	12	11	10	9	8
RD_SVBUS	RESERVED				ECC_VECTOR		
R-0/W1S-0h	R-0h				R/W-0h		
7	6	5	4	3	2	1	0
ECC_VECTOR							
R/W-0h							

Table 21-69. MCANERR_VECTOR Field Descriptions

Bit	Field	Type	Reset	Description
31-25	RESERVED	R	0h	
24	RD_SVBUS_DONE	R	0h	Read Completion Flag
23-16	RD_SVBUS_ADDRESS	R/W	0h	Read Address Offset
15	RD_SVBUS	R-0/W1S	0h	Read Trigger
14-11	RESERVED	R	0h	
10-0	ECC_VECTOR	R/W	0h	ECC RAM ID. Each error detection and correction (EDC) controller has a bank of error registers (offsets 0x10 - 0x3B) associated with it. These registers are accessed via an internal serial bus (SVBUS). To access them through the ECC aggregator the controller ID desired must be written to the ECC_VECTOR field, together with the RD_SVBUS trigger and RD_SVBUS_ADDRESS bit field. This initiates the serial read which consummates by setting the RD_SVBUS_DONE bit. At this point the addressed register may be read by a normal CPU read of the appropriate offset address. 0x000 Message RAM ECC controller is selected Others Reserved (do not use) Subsequent writes through the SVBUS (offsets 0x10 - 0x3B) have a delayed completion. To avoid conflicts, perform a read back of a register within this range after writing.

21.68 MCANERR_STAT (Offset = 740Ch) [Reset = 00000002h]

MCANERR_STAT is shown in [Figure 21-68](#) and described in [Table 21-70](#).

Return to the [Summary Table](#).

MCAN Error Misc Status

Figure 21-68. MCANERR_STAT

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED											NUM_RAMs																				
R-0h											R-2h																				

Table 21-70. MCANERR_STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-11	RESERVED	R	0h	
10-0	NUM_RAMs	R	2h	Number of RAMs. Number of ECC RAMs serviced by the aggregator.

21.69 MCANERR_WRAP_REV (Offset = 7410h) [Reset = 66A46A02h]

MCANERR_WRAP_REV is shown in [Figure 21-69](#) and described in [Table 21-71](#).

Return to the [Summary Table](#).

This register is accessed through the ECC aggregator via an internal serial bus. To access, the appropriate procedure must be first followed in the MCAN ECC Vector Register.

Figure 21-69. MCANERR_WRAP_REV

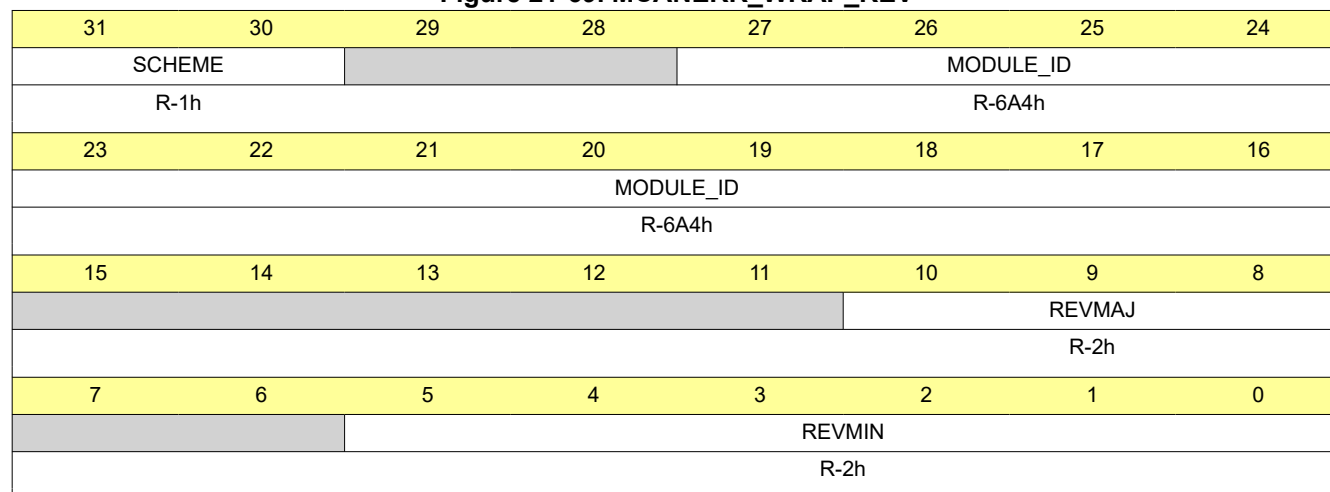


Table 21-71. MCANERR_WRAP_REV Field Descriptions

Bit	Field	Type	Reset	Description
31-30	SCHEME	R	1h	PID Register Scheme
27-16	MODULE_ID	R	6A4h	Module Identification Number
10-8	REVMAJ	R	2h	Major Revision of the Error Aggregator
5-0	REVMIN	R	2h	Minor Revision of the Error Aggregator

21.70 MCANERR_CTRL (Offset = 7414h) [Reset = 00000187h]

MCANERR_CTRL is shown in [Figure 21-70](#) and described in [Table 21-72](#).

Return to the [Summary Table](#).

This register is accessed through the ECC aggregator via an internal serial bus. To access, the appropriate procedure must be first followed in the MCAN ECC Vector Register.

Figure 21-70. MCANERR_CTRL

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							CHECK_SVBU S_TIMEOUT
R-0h							R/W-1h
7	6	5	4	3	2	1	0
RESERVED	ERROR_ONCE	FORCE_N_ROW	FORCE_DED	FORCE_SEC	ENABLE_RMW	ECC_CHECK	ECC_ENABLE
R-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-1h	R/W-1h	R/W-1h

Table 21-72. MCANERR_CTRL Field Descriptions

Bit	Field	Type	Reset	Description
31-9	RESERVED	R	0h	
8	CHECK_SVBUS_TIMEOUT	R/W	1h	Enables Serial VBUS timeout mechanism
7	RESERVED	R	0h	
6	ERROR_ONCE	R/W	0h	If this bit is set, the FORCE_SEC/FORCE_DED will inject an error to the specified row only once. The FORCE_SEC bit will be cleared once a writeback happens. If writeback is not enabled, this error will be cleared the cycle following the read when the data is corrected. For double-bit errors, the FORCE_DED bit will be cleared the cycle following the double-bit error. Any subsequent reads will not force an error.
5	FORCE_N_ROW	R/W	0h	Enable single/double-bit error on the next RAM read, regardless of the MCANERR_ERR_CTRL1.ECC_ROW setting. For write through mode, this applies to writes as well as reads.
4	FORCE_DED	R/W	0h	Force double-bit error. Cleared the cycle following the error if ERROR_ONCE is asserted. For write through mode, this applies to writes as well as reads. MCANERR_ERR_CTRL1 and MCANERR_ERR_CTRL2 should be configured prior to setting this bit.
3	FORCE_SEC	R/W	0h	Force single-bit error. Cleared on a writeback or the cycle following the error if ERROR_ONCE is asserted. For write through mode, this applies to writes as well as reads. MCANERR_ERR_CTRL1 and MCANERR_ERR_CTRL2 should be configured prior to setting this bit.
2	ENABLE_RMW	R/W	1h	Enable read-modify-write on partial word writes
1	ECC_CHECK	R/W	1h	Enable ECC Check. ECC is completely bypassed if both ECC_ENABLE and ECC_CHECK are '0'.
0	ECC_ENABLE	R/W	1h	Enable ECC Generation

21.71 MCANERR_ERR_CTRL1 (Offset = 7418h) [Reset = 00000000h]

MCANERR_ERR_CTRL1 is shown in [Figure 21-71](#) and described in [Table 21-73](#).

Return to the [Summary Table](#).

This register is accessed through the ECC aggregator via an internal serial bus. To access, the appropriate procedure must be first followed in the MCAN ECC Vector Register.

Figure 21-71. MCANERR_ERR_CTRL1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC_ROW																															
R/W-0h																															

Table 21-73. MCANERR_ERR_CTRL1 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	ECC_ROW	R/W	0h	Row address where FORCE_SEC or FORCE_DED needs to be applied. This is ignored if FORCE_N_ROW is set.

21.72 MCANERR_ERR_CTRL2 (Offset = 741Ch) [Reset = 00000000h]

MCANERR_ERR_CTRL2 is shown in [Figure 21-72](#) and described in [Table 21-74](#).

Return to the [Summary Table](#).

This register is accessed through the ECC aggregator via an internal serial bus. To access, the appropriate procedure must be first followed in the MCAN ECC Vector Register.

Figure 21-72. MCANERR_ERR_CTRL2

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC_BIT2																ECC_BIT1															
R/W-0h																R/W-0h															

Table 21-74. MCANERR_ERR_CTRL2 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	ECC_BIT2	R/W	0h	Second column/data bit that needs to be flipped when FORCE_DED is set
15-0	ECC_BIT1	R/W	0h	Column/Data bit that needs to be flipped when FORCE_SEC or FORCE_DED is set

21.73 MCANERR_ERR_STAT1 (Offset = 7420h) [Reset = 00000000h]

MCANERR_ERR_STAT1 is shown in [Figure 21-73](#) and described in [Table 21-75](#).

Return to the [Summary Table](#).

This register is accessed through the ECC aggregator via an internal serial bus. To access, the appropriate procedure must be first followed in the MCAN ECC Vector Register.

Figure 21-73. MCANERR_ERR_STAT1

31	30	29	28	27	26	25	24
ECC_BIT1							
R-0h							
23	22	21	20	19	18	17	16
ECC_BIT1							
R-0h							
15	14	13	12	11	10	9	8
CLR_CTRL_REG_ERROR	RESERVED		CLR_ECC_OTHER	CLR_ECC_DED		CLR_ECC_SEC	
R/W1S-0h	R-0h		R/W1C-0h	R/WD-0h		R/WD-0h	
7	6	5	4	3	2	1	0
CTRL_REG_ERROR	RESERVED		ECC_OTHER	ECC_DED		ECC_SEC	
R/W1S-0h	R-0h		R/W1S-0h	R/WI-0h		R/WI-0h	

Table 21-75. MCANERR_ERR_STAT1 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	ECC_BIT1	R	0h	ECC Error Bit Position. Indicates the bit position in the RAM data that is in error on an SEC error. Only valid on an SEC error. 0 Bit 0 is in error 1 Bit 1 is in error 2 Bit 2 is in error 3 Bit 3 is in error ... 31 Bit 31 is in error >32 Invalid
15	CLR_CTRL_REG_ERROR	R/W1S	0h	Writing a '1' clears the CTRL_REG_ERROR bit
14-13	RESERVED	R	0h	
12	CLR_ECC_OTHER	R/W1C	0h	Writing a '1' clears the ECC_OTHER bit.
11-10	CLR_ECC_DED	R/WD	0h	Clear ECC_DED. A write of a non-zero value to this bit field decrements the ECC_DED bit field by the value provided.
9-8	CLR_ECC_SEC	R/WD	0h	Clear ECC_SEC. A write of a non-zero value to this bit field decrements the ECC_SEC bit field by the value provided.
7	CTRL_REG_ERROR	R/W1S	0h	Control Register Error. A bit field in the control register is in an ambiguous state. This means that the redundancy registers have detected a state where not all values are the same and has defaulted to the reset state. S/W needs to re-write these registers to a known state. A write of 1 will set this interrupt flag.
6-5	RESERVED	R	0h	
4	ECC_OTHER	R/W1S	0h	SEC While Writeback Error Status 0 No SEC error while writeback pending 1 Indicates that successive single-bit errors have occurred while a writeback is still pending

Table 21-75. MCANERR_ERR_STAT1 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
3-2	ECC_DED	R/WI	0h	Double Bit Error Detected Status. A 2-bit saturating counter of the number of DED errors that have occurred since last cleared. 0 No double-bit error detected 1 One double-bit error was detected 2 Two double-bit errors were detected 3 Three double-bit errors were detected A write of a non-zero value to this bit field increments it by the value provided.
1-0	ECC_SEC	R/WI	0h	Single Bit Error Corrected Status. A 2-bit saturating counter of the number of SEC errors that have occurred since last cleared. 0 No single-bit error detected 1 One single-bit error was detected and corrected 2 Two single-bit errors were detected and corrected 3 Three single-bit errors were detected and corrected A write of a non-zero value to this bit field increments it by the value provided.

21.74 MCANERR_ERR_STAT2 (Offset = 7424h) [Reset = 00000000h]

MCANERR_ERR_STAT2 is shown in [Figure 21-74](#) and described in [Table 21-76](#).

Return to the [Summary Table](#).

This register is accessed through the ECC aggregator via an internal serial bus. To access, the appropriate procedure must be first followed in the MCAN ECC Vector Register.

Figure 21-74. MCANERR_ERR_STAT2

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC_ROW																															
R-0h																															

Table 21-76. MCANERR_ERR_STAT2 Field Descriptions

Bit	Field	Type	Reset	Description
31-0	ECC_ROW	R	0h	Indicates the row address where the single or double-bit error occurred. This value is address offset/4.

21.75 MCANERR_ERR_STAT3 (Offset = 7428h) [Reset = 00000000h]

MCANERR_ERR_STAT3 is shown in [Figure 21-75](#) and described in [Table 21-77](#).

Return to the [Summary Table](#).

This register is accessed through the ECC aggregator via an internal serial bus. To access, the appropriate procedure must be first followed in the MCAN ECC Vector Register.

Figure 21-75. MCANERR_ERR_STAT3

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED						CLR_SVBUS_T IMEOUT	RESERVED
R-0h						R-0/W1C-0h	R-0h
7	6	5	4	3	2	1	0
RESERVED						SVBUS_TIMEO UT	WB_PEND
R-0h						R-0/W1S-0h	R-0h

Table 21-77. MCANERR_ERR_STAT3 Field Descriptions

Bit	Field	Type	Reset	Description
31-10	RESERVED	R	0h	
9	CLR_SVBUS_TIMEOUT	R-0/W1C	0h	Write 1 to clear the Serial VBUS Timeout Flag
8-2	RESERVED	R	0h	
1	SVBUS_TIMEOUT	R-0/W1S	0h	Serial VBUS Timeout Flag. Write 1 to set.
0	WB_PEND	R	0h	Delayed Write Back Pending Status 0 No write back pending 1 An ECC data correction write back is pending

21.76 MCANERR_SEC_EOI (Offset = 743Ch) [Reset = 00000000h]

MCANERR_SEC_EOI is shown in [Figure 21-76](#) and described in [Table 21-78](#).

Return to the [Summary Table](#).

MCAN Single Error Corrected End of Interrupt Register

Figure 21-76. MCANERR_SEC_EOI

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							EOI_WR
R-0h							R-0/W1S-0h

Table 21-78. MCANERR_SEC_EOI Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	EOI_WR	R-0/W1S	0h	Write to this register indicates that software has acknowledged the pending interrupt and the next interrupt can be sent to the host. Note that a write to the MCANERR_ERR_STAT1.CLR_ECC_SEC goes through the SVBUS and has a delayed completion. To avoid an additional interrupt, read the MCANERR_ERR_STAT1 register back prior to writing to this bit field.

21.77 MCANERR_SEC_STATUS (Offset = 7440h) [Reset = 00000000h]

MCANERR_SEC_STATUS is shown in [Figure 21-77](#) and described in [Table 21-79](#).

Return to the [Summary Table](#).

MCAN Single Error Corrected Interrupt Status Register

Figure 21-77. MCANERR_SEC_STATUS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							MSGMEM_PEND
R-0h							R-0-0h

Table 21-79. MCANERR_SEC_STATUS Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	MSGMEM_PEND	R-0	0h	Message RAM SEC Interrupt Pending 0 No SEC interrupt is pending 1 SEC interrupt is pending

21.78 MCANERR_SEC_ENABLE_SET (Offset = 7480h) [Reset = 00000000h]

MCANERR_SEC_ENABLE_SET is shown in [Figure 21-78](#) and described in [Table 21-80](#).

Return to the [Summary Table](#).

MCAN Single Error Corrected Interrupt Enable Set Register

Figure 21-78. MCANERR_SEC_ENABLE_SET

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							MSGMEM_ENABLE_SET
R-0h							R/W1S-0h

Table 21-80. MCANERR_SEC_ENABLE_SET Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	MSGMEM_ENABLE_SET	R/W1S	0h	Message RAM SEC Interrupt Pending Enable Set. Writing a 1 to this bit enables the Message RAM SEC error interrupts. Writing a 0 has no effect. Reads return the corresponding enable bit's current value.

21.79 MCANERR_SEC_ENABLE_CLR (Offset = 74C0h) [Reset = 00000000h]

MCANERR_SEC_ENABLE_CLR is shown in [Figure 21-79](#) and described in [Table 21-81](#).

Return to the [Summary Table](#).

MCAN Single Error Corrected Interrupt Enable Clear Register

Figure 21-79. MCANERR_SEC_ENABLE_CLR

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							MSGMEM_ENA BLE_CLR
R-0h							R/W1C-0h

Table 21-81. MCANERR_SEC_ENABLE_CLR Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	MSGMEM_ENABLE_CLR	R/W1C	0h	Message RAM SEC Interrupt Pending Enable Clear. Writing a 1 to this bit disables the Message RAM SEC error interrupts. Writing a 0 has no effect. Reads return the corresponding enable bit's current value.

21.80 MCANERR_DED_EOI (Offset = 753Ch) [Reset = 00000000h]

MCANERR_DED_EOI is shown in [Figure 21-80](#) and described in [Table 21-82](#).

Return to the [Summary Table](#).

MCAN Double Error Detected End of Interrupt Register

Figure 21-80. MCANERR_DED_EOI

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							EOI_WR
R-0h							R-0/W1S-0h

Table 21-82. MCANERR_DED_EOI Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	EOI_WR	R-0/W1S	0h	Write to this register indicates that software has acknowledged the pending interrupt and the next interrupt can be sent to the host. Note that a write to the MCANERR_ERR_STAT1.CLR_ECC_DED goes through the SVBUS and has a delayed completion. To avoid an additional interrupt, read the MCANERR_ERR_STAT1 register back prior to writing to this bit field.

21.81 MCANERR_DED_STATUS (Offset = 7540h) [Reset = 00000000h]

MCANERR_DED_STATUS is shown in [Figure 21-81](#) and described in [Table 21-83](#).

Return to the [Summary Table](#).

MCAN Double Error Detected Interrupt Status Register

Figure 21-81. MCANERR_DED_STATUS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							MSGMEM_PEN D
R-0h							R-0-0h

Table 21-83. MCANERR_DED_STATUS Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	MSGMEM_PEND	R-0	0h	Message RAM DED Interrupt Pending 0 No DED interrupt is pending 1 DED interrupt is pending

21.82 MCANERR_DED_ENABLE_SET (Offset = 7580h) [Reset = 00000000h]

MCANERR_DED_ENABLE_SET is shown in [Figure 21-82](#) and described in [Table 21-84](#).

Return to the [Summary Table](#).

MCAN Double Error Detected Interrupt Enable Set Register

Figure 21-82. MCANERR_DED_ENABLE_SET

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							MSGMEM_ENABLE_SET
R-0h							R/W1S-0h

Table 21-84. MCANERR_DED_ENABLE_SET Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	MSGMEM_ENABLE_SET	R/W1S	0h	Message RAM DED Interrupt Pending Enable Set. Writing a 1 to this bit enables the Message RAM DED error interrupts. Writing a 0 has no effect. Reads return the corresponding enable bit's current value.

21.83 MCANERR_DED_ENABLE_CLR (Offset = 75C0h) [Reset = 00000000h]

MCANERR_DED_ENABLE_CLR is shown in [Figure 21-83](#) and described in [Table 21-85](#).

Return to the [Summary Table](#).

MCAN Double Error Detected Interrupt Enable Clear Register

Figure 21-83. MCANERR_DED_ENABLE_CLR

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							MSGMEM_ENA BLE_CLR
R-0h							R/W1C-0h

Table 21-85. MCANERR_DED_ENABLE_CLR Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	MSGMEM_ENABLE_CLR	R/W1C	0h	Message RAM DED Interrupt Pending Enable Clear. Writing a 1 to this bit disables the Message RAM DED error interrupts. Writing a 0 has no effect. Reads return the corresponding enable bit's current value.

21.84 MCANERR_AGGR_ENABLE_SET (Offset = 7600h) [Reset = 00000000h]

MCANERR_AGGR_ENABLE_SET is shown in [Figure 21-84](#) and described in [Table 21-86](#).

Return to the [Summary Table](#).

MCAN Error Aggregator Enable Set Register

Figure 21-84. MCANERR_AGGR_ENABLE_SET

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED						ENABLE_TIME OUT_SET	ENABLE_PARI TY_SET
R-0h						R/W1S-0h	R/W1S-0h

Table 21-86. MCANERR_AGGR_ENABLE_SET Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1	ENABLE_TIMEOUT_SET	R/W1S	0h	Write 1 to enable timeout errors. Reads return the corresponding enable bit's current value.
0	ENABLE_PARITY_SET	R/W1S	0h	Write 1 to enable parity errors. Reads return the corresponding enable bit's current value.

21.85 MCANERR_AGGR_ENABLE_CLR (Offset = 7604h) [Reset = 00000000h]

MCANERR_AGGR_ENABLE_CLR is shown in [Figure 21-85](#) and described in [Table 21-87](#).

Return to the [Summary Table](#).

MCAN Error Aggregator Enable Clear Register

Figure 21-85. MCANERR_AGGR_ENABLE_CLR

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED						ENABLE_TIME OUT_CLR	ENABLE_PARI TY_CLR
R-0h						R/W1C-0h	R/W1C-0h

Table 21-87. MCANERR_AGGR_ENABLE_CLR Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1	ENABLE_TIMEOUT_CLR	R/W1C	0h	Write 1 to disable timeout errors. Reads return the corresponding enable bit's current value.
0	ENABLE_PARITY_CLR	R/W1C	0h	Write 1 to disable parity errors. Reads return the corresponding enable bit's current value.

21.86 MCANERR_AGGR_STATUS_SET (Offset = 7608h) [Reset = 00000000h]

MCANERR_AGGR_STATUS_SET is shown in [Figure 21-86](#) and described in [Table 21-88](#).

Return to the [Summary Table](#).

MCAN Error Aggregator Status Set Register

Figure 21-86. MCANERR_AGGR_STATUS_SET

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				SVBUS_TIMEOUT		AGGR_PARITY_ERR	
R-0h				R/WI-0h		R/WI-0h	

Table 21-88. MCANERR_AGGR_STATUS_SET Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3-2	SVBUS_TIMEOUT	R/WI	0h	Aggregator Serial VBUS Timeout Error Status 2-bit saturating counter of the number of SVBUS timeout errors that have occurred since last cleared. 0 No timeout errors have occurred 1 One timeout error has occurred 2 Two timeout errors have occurred 3 Three timeout errors have occurred A write of a non-zero value to this bit field increments it by the value provided.
1-0	AGGR_PARITY_ERR	R/WI	0h	Aggregator Parity Error Status 2-bit saturating counter of the number of parity errors that have occurred since last cleared. 0 No parity errors have occurred 1 One parity error has occurred 2 Two parity errors have occurred 3 Three parity errors have occurred A write of a non-zero value to this bit field increments it by the value provided.

21.87 MCANERR_AGGR_STATUS_CLR (Offset = 760Ch) [Reset = 00000000h]

MCANERR_AGGR_STATUS_CLR is shown in [Figure 21-87](#) and described in [Table 21-89](#).

Return to the [Summary Table](#).

MCAN Error Aggregator Status Clear Register

Figure 21-87. MCANERR_AGGR_STATUS_CLR

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				SVBUS_TIMEOUT		AGGR_PARITY_ERR	
R-0h				R/WD-0h		R/WD-0h	

Table 21-89. MCANERR_AGGR_STATUS_CLR Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3-2	SVBUS_TIMEOUT	R/WD	0h	Aggregator Serial VBUS Timeout Error Status 2-bit saturating counter of the number of SVBUS timeout errors that have occurred since last cleared. 0 No timeout errors have occurred 1 One timeout error has occurred 2 Two timeout errors have occurred 3 Three timeout errors have occurred A write of a non-zero value to this bit field decrements it by the value provided.
1-0	AGGR_PARITY_ERR	R/WD	0h	Aggregator Parity Error Status 2-bit saturating counter of the number of parity errors that have occurred since last cleared. 0 No parity errors have occurred 1 One parity error has occurred 2 Two parity errors have occurred 3 Three parity errors have occurred A write of a non-zero value to this bit field decrements it by the value provided.

21.88 IIDX (Offset = 7820h) [Reset = 00000000h]

IIDX is shown in [Figure 21-88](#) and described in [Table 21-90](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS](#) and [MIS](#) are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 21-88. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 21-90. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 00h = No interrupt pending. 1h = MCAN Interrupt Line 0 interrupt pending. 2h = MCAN Interrupt Line 1 interrupt pending. 3h = Message RAM SEC (Single Error Correction) interrupt pending. 4h = Message RAM DED (Double Error Detection) interrupt pending. 5h = External Timestamp Counter Overflow interrupt pending. 6h = Clock Stop Wake Up interrupt pending.

21.89 IMASK (Offset = 7828h) [Reset = 00000000h]

IMASK is shown in [Figure 21-89](#) and described in [Table 21-91](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is un-masked. Un-masking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 21-89. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED		WAKEUP	EXT_TS_CNTR_OVFL	DED	SEC	INTL1	INTL0
R-0h		-0	-0	-0	-0	-0	-0

Table 21-91. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R	0h	
5	WAKEUP	R/W	0h	Clock Stop Wake Up interrupt mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
4	EXT_TS_CNTR_OVFL	R/W	0h	External Timestamp Counter Overflow interrupt mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
3	DED	R/W	0h	Message RAM DED interrupt mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	SEC	R/W	0h	Message RAM SEC interrupt mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	INTL1	R/W	0h	MCAN Interrupt Line 1 mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	INTL0	R/W	0h	MCAN Interrupt Line 0 mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

21.90 RIS (Offset = 7830h) [Reset = 00000000h]

RIS is shown in [Figure 21-90](#) and described in [Table 21-92](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 21-90. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED		WAKEUP	EXT_TS_CNTR_OVFL	DED	SEC	INTL1	INTL0
R-0h		-0	-0	-0	-0	-0	-0

Table 21-92. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R	0h	
5	WAKEUP	R	0h	Clock Stop Wake Up interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
4	EXT_TS_CNTR_OVFL	R	0h	External Timestamp Counter Overflow interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
3	DED	R	0h	Message RAM DED interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
2	SEC	R	0h	Message RAM SEC interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
1	INTL1	R	0h	MCAN Interrupt Line 1. 0h = Interrupt did not occur 1h = Interrupt occurred
0	INTL0	R	0h	MCAN Interrupt Line 0. 0h = Interrupt did not occur 1h = Interrupt occurred

21.91 MIS (Offset = 7838h) [Reset = 00000000h]

MIS is shown in [Figure 21-91](#) and described in [Table 21-93](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 21-91. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED		WAKEUP	EXT_TS_CNTR_OVFL	DED	SEC	INTL1	INTL0
R-0h		-0	-0	-0	-0	-0	-0

Table 21-93. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R	0h	
5	WAKEUP	R	0h	Masked Clock Stop Wake Up interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
4	EXT_TS_CNTR_OVFL	R	0h	Masked External Timestamp Counter Overflow interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
3	DED	R	0h	Masked Message RAM DED interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
2	SEC	R	0h	Masked Message RAM SEC interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred
1	INTL1	R	0h	Masked MCAN Interrupt Line 1. 0h = Interrupt did not occur 1h = Interrupt occurred
0	INTL0	R	0h	Masked MCAN Interrupt Line 0. 0h = Interrupt did not occur 1h = Interrupt occurred

21.92 ISET (Offset = 7840h) [Reset = 00000000h]

ISET is shown in [Figure 21-92](#) and described in [Table 21-94](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 21-92. ISET

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED		WAKEUP	EXT_TS_CNTR_OVFL	DED	SEC	INTL1	INTL0
R-0h		-0	-0	-0	-0	-0	-0

Table 21-94. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R	0h	
5	WAKEUP	W	0h	Set Clock Stop Wake Up interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
4	EXT_TS_CNTR_OVFL	W	0h	Set External Timestamp Counter Overflow interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
3	DED	W	0h	Set Message RAM DED interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
2	SEC	W	0h	Set Message RAM SEC interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt
1	INTL1	W	0h	Set MCAN Interrupt Line 1. 0h = Writing 0 has no effect 1h = Set Interrupt
0	INTL0	W	0h	Set MCAN Interrupt Line 0. 0h = Writing 0 has no effect 1h = Set Interrupt

21.93 ICLR (Offset = 7848h) [Reset = 00000000h]

ICLR is shown in [Figure 21-93](#) and described in [Table 21-95](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 21-93. ICLR

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED		WAKEUP	EXT_TS_CNTR_OVFL	DED	SEC	INTL1	INTL0
R-0h		-0	-0	-0	-0	-0	-0

Table 21-95. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R	0h	
5	WAKEUP	W	0h	Clear Clock Stop Wake Up interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
4	EXT_TS_CNTR_OVFL	W	0h	Clear External Timestamp Counter Overflow interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
3	DED	W	0h	Clear Message RAM DED interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
2	SEC	W	0h	Clear Message RAM SEC interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt
1	INTL1	W	0h	Clear MCAN Interrupt Line 1. 0h = Writing 0 has no effect 1h = Clear Interrupt
0	INTL0	W	0h	Clear MCAN Interrupt Line 0. 0h = Writing 0 has no effect 1h = Clear Interrupt

21.94 EVT_MODE (Offset = 78E0h) [Reset = 00000000h]

EVT_MODE is shown in [Figure 21-94](#) and described in [Table 21-96](#).

Return to the [Summary Table](#).

Event mode register. It is used to select whether each line is disabled, in software mode (software clears the RIS) or in hardware mode (hardware clears the RIS)

Figure 21-94. EVT_MODE

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED						INT0_CFG	
R-0h						R-0h	

Table 21-96. EVT_MODE Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1-0	INT0_CFG	R	0h	Event line mode select for event corresponding to [IPSTANDARD.CPU_INT] 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.

21.95 DESC (Offset = 78FCh) [Reset = 00000000h]

DESC is shown in [Figure 21-95](#) and described in [Table 21-97](#).

Return to the [Summary Table](#).

This register identifies the peripheral and its exact version.

Figure 21-95. DESC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODULEID															
-0															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FEATUREVER				RESERVED				MAJREV				MINREV			
-0				R-0h				-0				-0			

Table 21-97. DESC Field Descriptions

Bit	Field	Type	Reset	Description
31-16	MODULEID	R	0x0	Module identification contains a unique peripheral identification number. The assignments are maintained in a central database for all of the platform modules to ensure uniqueness. 0h = Smallest value FFFFh = Highest possible value
15-12	FEATUREVER	R	0x0	Feature Set for the module *instance* 0h = MCAN module with CAN-FD mode enabled <<Internal Note: This is an in-IP paper spin variant. How does this map to the SYS_MCAN_ENABLE_FD choice value?>> 1h = MCAN module with CAN-FD mode disabled <<Internal Note: This is an in-IP paper spin variant. How does this map to the SYS_MCAN_ENABLE_FD choice value?>>
11-8	RESERVED	R	0h	
7-4	MAJREV	R	0x0	Major rev of the IP 0h = Smallest value Fh = Highest possible value
3-0	MINREV	R	0x0	Minor rev of the IP 0h = Smallest value Fh = Highest possible value

21.96 MCANSS_CLKEN (Offset = 7900h) [Reset = 00000000h]

MCANSS_CLKEN is shown in [Figure 21-96](#) and described in [Table 21-98](#).

Return to the [Summary Table](#).

MCAN module clock (functional clock and Vbusp to access MCAN module MMRs) enable register
<Internal note> This IP-specific MMR itself is not clock gated, as long as IP is enabled by the paper-spin configuration

Figure 21-96. MCANSS_CLKEN

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							CLK_REQEN
R-0h							-0

Table 21-98. MCANSS_CLKEN Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	CLK_REQEN	RW	0x0	MCAN functional and MCAN/MCANSS MMR clock request enable bit 0h = MCAN module functional clock and Vbusp is not requested. These clocks are gated to the MCAN module. 1h = Setting this bit requests MCAN module functional clock and Vbusp. These clocks are not gated to MCAN module.

21.97 MCANSS_CLKDIV (Offset = 7904h) [Reset = 00000000h]

MCANSS_CLKDIV is shown in [Figure 21-97](#) and described in [Table 21-99](#).

Return to the [Summary Table](#).

Needs to go to the Management aperture once available

Figure 21-97. MCANSS_CLKDIV

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED													RATIO		
R-0h													R/W-0h		

Table 21-99. MCANSS_CLKDIV Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1-0	RATIO	R/W	0x0	Clock divide ratio specification. Enables configuring clock divide settings for the MCAN functional clock input to the MCAN-SS. 0h (R/W) = Divides input clock by 1 1h (R/W) = Divides input clock by 2 2h (R/W) = Divides input clock by 4 3h (R/W) = Divides input clock by 1

21.98 MCANSS_CLKCTL (Offset = 7908h) [Reset = 00000000h]

MCANSS_CLKCTL is shown in [Figure 21-98](#) and described in [Table 21-100](#).

Return to the [Summary Table](#).

MCANSS clock stop control MMR.

<Internal note> Bus clock for the wrapper MMRs (including this MMR) is not gated by this register.

Figure 21-98. MCANSS_CLKCTL

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							WKUP_GLTFILT_EN
R-0h							R/W-0h
7	6	5	4	3	2	1	0
RESERVED			WAKEUP_INT_EN	RESERVED			STOPREQ
R-0h			R/W-0h	R-0h			-0

Table 21-100. MCANSS_CLKCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-9	RESERVED	R	0h	
8	WKUP_GLTFILT_EN	R/W	0h	Setting this bit enables the glitch filter on MCAN RXD input, which wakes up the MCAN controller to exit clock gating. 0h = Disable glitch filter enable on RXD input when MCAN is in clock stop mode (waiting for event on RXD input for clock stop wakeup). 1h = Enable glitch filter enable on RXD input when MCAN is in clock stop mode (waiting for event on RXD input for clock stop wakeup).
7-5	RESERVED	R	0h	
4	WAKEUP_INT_EN	R/W	0h	This bit controls enabling or disabling the MCAN IP clock stop wakeup interrupt (when MCANSS_CTRL.WAKEUPREQEN wakeup request is enabled to wakeup MCAN IP upon CAN RXD activity) 0h = Disable MCAN IP clock stop wakeup interrupt 1h = Enable MCAN IP clock stop wakeup interrupt
3-1	RESERVED	R	0h	
0	STOPREQ	R/W	0h	This bit is used to enable/disable MCAN clock (both host clock and functional clock) gating request. Note: This bit can be reset by HW by Clock-Stop Wake-up via CAN RX Activity. See spec for more details. 0h = Disable MCAN-SS clock stop request 1h = Enable MCAN-SS clock stop request

21.99 MCANSS_CLKSTS (Offset = 790Ch) [Reset = 00000000h]

MCANSS_CLKSTS is shown in [Figure 21-99](#) and described in [Table 21-101](#).

Return to the [Summary Table](#).

MCANSS clock stop status register to indicate status of clock stop mechanism

Figure 21-99. MCANSS_CLKSTS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							CCLKDONE
R-0h							R-0h
7	6	5	4	3	2	1	0
RESERVED			STOPREQ_HW_OVR	RESERVED			CLKSTOP_ACKSTS
R-0h			R-0h	R-0h			R-0h

Table 21-101. MCANSS_CLKSTS Field Descriptions

Bit	Field	Type	Reset	Description
31-9	RESERVED	R	0h	
8	CCLKDONE	R	0h	This bit indicates the status of MCAN controller clock request from GPRCM. 0h = MCAN controller clock is not available to the MCAN IP. 1h = MCAN controller clock is enabled and available to the MCAN IP.
7-5	RESERVED	R	0h	
4	STOPREQ_HW_OVR	R	0h	MCANSS clock stop HW override status bit. This bit indicates when the MCANSS_CLKCTL.STOPREQ bit has been cleared by HW when a clock-stop wake-up event via CAN RX activity is triggered. 0h = MCANSS_CLKCTL.STOPREQ bit has not been cleared by HW. 1h = MCANSS_CLKCTL.STOPREQ bit has been cleared by HW.
3-1	RESERVED	R	0h	
0	CLKSTOP_ACKSTS	R	0h	Clock stop acknowledge status from MCAN IP 0h = No clock stop acknowledged. 1h = Clock stop has been acknowledged by MCAN IP; MCAN-SS may be clock gated by stopping both the CAN host and functional clocks.



The cyclic redundancy check (CRC) accelerator generates signatures for a given data sequence based on the CRC16-CCITT polynomial or the CRC32-ISO3309 polynomial.

22.1 CRC Overview.....	1252
22.2 CRC Operation.....	1252
22.3 CRC Registers.....	1255

22.1 CRC Overview

The CRC accelerator produces CRC signatures for given sequences of data. The CRC16-CCITT and CRC32-ISO3309 polynomial functions are supported. Identical input data sequences result in identical CRC signatures when the CRC is initialized with a fixed seed value. Different sequences of input data, in general, result in different signatures for a given CRC function.

Key features of the CRC accelerator include:

- Support for CRC16-CCITT and CRC32-ISO3309 polynomial
- Fast single-cycle computation of new CRC output for each data input (no wait states)
- Support for input/output bit reversal
- Support for little or big endian operation
- Byte, half-word, or word input to CRCIN
- 512-word CRCIN_IDX input field in which all addresses are mapped to CRCIN, supporting use of a standard C-style memcpy() routine to load data into the CRC module for data lengths up to 2KB

22.1.1 CRC16-CCITT

For CRC16-CCITT, the CRC signature is generated based on the polynomial given in the 16-bit CCITT standard, as shown in [the equation below](#).

$$f(x) = x^{16} + x^{12} + x^5 + 1 \quad (25)$$

The CRC16-CCITT digest size is 16 bits (half-word).

22.1.2 CRC32-ISO3309

For CRC32-ISO3309, the CRC signature is generated based on the polynomial given in the ISO3309 Ethernet standard, as shown in [the equation below](#).

$$f(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1 \quad (26)$$

The digest size for CRC32-ISO3309 is 32 bits.

22.2 CRC Operation

The CRC generator is initialized by [configuring the desired mode of operation in the CRCCTRL register](#), followed by writing the seed value to the CRCSEED register. After the seed is loaded to the CRCSEED register, the CRCOUT register will reflect the SEED value loaded to CRCSEED.

Note

If the endianness mode is configured to big endian before the seed value is written to CRCSEED, then the byte order of the seed value written to CRCSEED is swapped when the seed value is loaded into the CRC module.

Once initialized, data can be input into the CRC generator by writing to the CRCIN register using byte (8-bit), half-word (16-bit), or word (32-bit) writes. The CPU or the DMA can be used to move input data into the CRC accelerator.

Note

Byte writes need not be word aligned; a byte write to any byte location in CRCIN will be interpreted the same way (adding the 8 written bits to the computed CRC). Half-word writes also need not be word aligned, but they must be half-word aligned. For example, a half-word can be written to BIT0-BIT15 or BIT16-BIT31 of CRCIN, but not to BIT8-BIT23.

When using the CRC generator to verify a data set, all data to be included in the CRC calculation must be written to the CRCIN register in the same order as was used to calculate the original CRC signature. When using

the CRC generator to create a new signature to be used in the future for verification, be sure to load the data the same way and with the same settings when performing verification.

The current CRC output can be read at any time by reading the CRCOUT register.

22.2.1 CRC Generator Implementation

The CRC generator is implemented with a set of XOR trees. After a set of 8, 16, or 32 bits is provided to the CRC accelerator by writing to the CRCIN register, a calculation for the whole set of input bits is performed. When new data is written to CRCIN, the CRC generator updates the CRC output in a single cycle. Bus wait states are not required to load data back-to-back into the CRC generator.

22.2.2 Configuration

The CRC accelerator supports polynomial selection, bit reversal selection, and byte order (endianness) selection. This section describes these configuration aspects.

The CRC accelerator must be enabled before being used through the PWREN register (see [peripheral power enable](#)).

The CRC accelerator is in power domain 1 ([PD1](#)), and as such can only be active in RUN or SLEEP mode. If the CRC accelerator is configured to be enabled by application software, and the device enters STOP or STANDBY mode, SYSCTL will force the CRC into a disabled state until the device exists STOP or STANDBY mode. All CRC register contents are retained when the CRC is forced to a disabled state in STOP or STANDBY mode.

The CRC module only runs from the PD1 bus clock ([MCLK](#)).

22.2.2.1 Polynomial Selection

The desired polynomial (CRC16-CCITT or CRC32-ISO3309) is selected with the POLYSIZE bit in the CRCCTRL register. The polynomial to be used must be selected before loading the seed value and any data values.

When the CRC generator is configured for a 16-bit digest (CRC16-CCITT), the following conditions apply:

- The upper half-word (16 bits) of the CRCSEED register are ignored and only the lower half-word is used
- The upper half-word (16 bits) of the CRCOUT register read back as zero and only the lower half-word is valid

When the CRC generator is configured for CRC32-ISO3309, all 32 bits of CRCSEED and CRCOUT are valid.

22.2.2.2 Bit Order

The various CRC standards were defined in the era of main frame computers. At that time, BIT0 was treated as the MSB. In modern computing, BIT0 is typically the LSB.

The Arm Cortex-M0+ CPU treats BIT0 as the LSB, as is typical in modern CPUs and MCUs. This sometimes causes confusion, because BIT0 has been treated as the LSB in some cases and as the MSB in other cases. Therefore, the CRC accelerator provides a bit order reversal capability to support both conventions.

Bit order reversal can be enabled by setting the BITREVERSE bit in the CRCCTRL register, giving the following behavior for input and output data:

- **Input data:** The bit order of each input byte written to the CRCIN register is reversed before it is passed to the CRC generator to be used for CRC calculation
- **Output data:** The bit order of the 16-bit or 32-bit CRC result is reversed when read from the CRCOUT register

Note

If input data must be provided bit-reversed, but the output is to be read not reversed, the BITREVERSE bit can be set before loading data to CRCIN and then cleared after all data is written to CRCIN but before the resulting signature is read from CRCOUT. Likewise, it is possible to load input data to CRCIN with BITREVERSE cleared (not reversed), and flip the output before reading it (by setting BITREVERSE before reading CRCOUT).

22.2.2.3 Byte Swap

The bit `OUTPUT_BYTESWAP` in the register `CRCCTRL` can be used to enable or disable CRC output byte swap. This bit controls whether the output is byte-swapped upon a read of the `CRCOUT` register. If `CRCOUT` is accessed as a half-word, and the `OUTPUT_BYTESWAP` is set to 1, then the two bytes in the 16-bit access are swapped and returned. Using B0, B1, B2 and B3 to identify Byte 0, Byte 1, Byte 2, Byte 3. B1 is returned as B0 and B0 is returned as B1. If `CRCOUT` is accessed as a word, and the `OUTPUT_BYTESWAP` is set to 1, then the four bytes in the 32-bit read are swapped. B3 is returned as B0, B2 is returned as B1, B1 is returned as B2 and B0 is returned as B3.

Note that if the CRC `POLYSIZE` is 16-bit and a 32-bit read of `CRCOUT` is performed with `OUTPUT_BYTESWAP` enabled, then the output is: MSB LSB 0x0 0x0 B0 B1. If the CRC `POLYSIZE` is 16-bit and a 32-bit read of `CRCOUT` is performed with `OUTPUT_BYTESWAP` disabled, then the output is: MSB LSB 0x0 0x0 B1 B0.

22.2.2.4 Byte Order

When working with half-word or word input data, the input byte order can be configured as either little endian or big endian. The default configuration is little endian. To reverse the byte order when using half-word or word inputs, set the `INPUT_ENDIANNESS` bit in the `CRCCTRL` register.

Reversing the endianness will cause the following translation for half-word and word writes:

Table 22-1. CRCIN Byte Order Translation

Endianness	Data Written to CRCIN	Data Applied to CRC Logic
0 (little)	0x1234	0x1234
1 (big)	0x1234	0x3412
0 (little)	0x12345678	0x12345678
1 (big)	0x12345678	0x78563412

Note

If the `ENDIANNESS` bit is set before the seed value is written to `CRCSEED`, then the byte order of the seed value written to `CRCSEED` is also reversed when it is loaded into the CRC, and the seed value will read back reversed when reading the `CRCOUT` register after writing to the `CRCSEED` register.

22.2.2.5 CRC C Library Compatibility

To simplify loading of data by software into the CRC, the CRC accelerator provides a 512-word (2KB) `CRCIN_IDX` region. Within the `CRCIN_IDX` region, a write to any word is re-mapped as, and functionally equivalent to, a write to the `CRCIN` register. This mechanism enables the use of the standard C library `memcpy()` routine to copy data from SRAM or flash into the CRC, provided that the source data is less than 2KB (2,048 bytes).

22.3 CRC Registers

Table 22-2 lists the memory-mapped registers for the CRC registers. All register offset addresses not listed in Table 22-2 should be considered as reserved locations and the register contents should not be modified.

Table 22-2. CRC Registers

Offset	Acronym	Register Name	Group	Section
800h	PWREN	Power enable		Go
804h	RSTCTL	Reset Control		Go
814h	STAT	Status Register		Go
1004h	CLKSEL	Clock Select		Go
10FCh	DESC	Module Description		Go
1100h	CRCCTRL	CRC Control Register		Go
1104h	CRCSEED	CRC Seed Register		Go
1108h	CRCIN	CRC Input Data Register		Go
110Ch	CRCOUT	CRC Output Result Register		Go
1800h + formula	CRCIN_IDX[y]	CRC Input Data Array Register		Go

Complex bit access types are encoded to fit into small table cells. Table 22-3 shows the codes that are used for access types in this section.

Table 22-3. CRC Access Type Codes

Access Type	Code	Description
Read Type		
H	H	Set or cleared by hardware
R	R	Read
Write Type		
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value
Register Array Variables		
i,j,k,l,m,n		When these variables are used in a register name, an offset, or an address, they refer to the value of a register array where the register is part of a group of repeating registers. The register groups form a hierarchical structure and the array is represented with a formula.
y		When this variable is used in a register name, an offset, or an address it refers to the value of a register array.

22.3.1 PWREN (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 22-1](#) and described in [Table 22-4](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 22-1. PWREN

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-0h							R/WK-0h

Table 22-4. PWREN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	ENABLE	R/WK	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

22.3.2 RSTCTL (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 22-2](#) and described in [Table 22-5](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 22-2. RSTCTL

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCL R	RESETASSERT
R-0h						WK-0h	WK-0h

Table 22-5. RSTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	R	0h	Reserved
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

22.3.3 STAT (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 22-3](#) and described in [Table 22-6](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 22-3. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-0h							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							
R-0h							

Table 22-6. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	Reserved
16	RESETSTKY	R	0h	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	Reserved

22.3.4 CLKSEL (Offset = 1004h) [Reset = 00000001h]

CLKSEL is shown in [Figure 22-4](#) and described in [Table 22-7](#).

Return to the [Summary Table](#).

Clock source selection

Figure 22-4. CLKSEL

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							MCLK_SEL
R-0h							R-1h

Table 22-7. CLKSEL Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	MCLK_SEL	R	1h	Selects main clock (MCLK) if enabled 0h = Does not select this clock as a source 1h = Select this clock as a source

22.3.5 DESC (Offset = 10FCh) [Reset = 20117010h]

DESC is shown in [Figure 22-5](#) and described in [Table 22-8](#).

Return to the [Summary Table](#).

This register identifies the peripheral and its exact version.

Figure 22-5. DESC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODULEID															
R-2011h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FEATUREVER				INSTNUM				MAJREV				MINREV			
R-7h				R-0h				R-1h				R-0h			

Table 22-8. DESC Field Descriptions

Bit	Field	Type	Reset	Description
31-16	MODULEID	R	2011h	Module identification contains a unique peripheral identification number. The assignments are maintained in a central database for all of the platform modules to ensure uniqueness. 0h = Smallest value FFFFh = Highest possible value
15-12	FEATUREVER	R	7h	Feature Set for the module *instance* 0h = Smallest value Fh = Highest possible value
11-8	INSTNUM	R	0h	Instance Number within the device. This will be a parameter to the RTL for modules that can have multiple instances 0h = Smallest value Fh = Highest possible value
7-4	MAJREV	R	1h	Major rev of the IP 0h = Smallest value Fh = Highest possible value
3-0	MINREV	R	0h	Minor rev of the IP 0h = Smallest value Fh = Highest possible value

22.3.6 CRCCTRL (Offset = 1100h) [Reset = X]

CRCCTRL is shown in [Figure 22-6](#) and described in [Table 22-9](#).

Return to the [Summary Table](#).

CRC Control Register. Configuration control of the CRC.

Figure 22-6. CRCCTRL

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED			OUTPUT_BYT ESWAP	RESERVED	INPUT_ENDIA NNESS	BITREVERSE	POLYSIZE
R-0h			R/W-0h	R-0h	R/W-0h	R/W-0h	R/W-0h

Table 22-9. CRCCTRL Field Descriptions

Bit	Field	Type	Reset	Description
31-5	RESERVED	R	0h	Reserved
4	OUTPUT_BYTESWAP	R/W	0h	CRC Output Byteswap Enable. This bit controls whether the output is byte-swapped upon a read of the CRCOUT register. If CRCOUT is accessed as a half-word, and the OUTPUT_BYTESWAP is set to 1, then the two bytes in the 16-bit access are swapped and returned. B1 is returned as B0 B0 is returned as B1 If CRCOUT is accessed as a word, and the OUTPUT_BYTESWAP is set to 1, then the four bytes in the 32-bit read are swapped. B3 is returned as B0 B2 is returned as B1 B1 is returned as B2 B0 is returned as B3 Note that if the CRC POLYSIZE is 16-bit and a 32-bit read of CRCOUT is performed with OUTPUT_BYTESWAP enabled, then the output is: MSB LSB 0x0 0x0 B0 B1 If the CRC POLYSIZE is 16-bit and a 32-bit read of CRCOUT is performed with OUTPUT_BYTESWAP disabled, then the output is: MSB LSB 0x0 0x0 B1 B0 0h = Output byteswapping is disabled 1h = Output byteswapping is enabled.
3	RESERVED	R	0h	Reserved
2	INPUT_ENDIANNES	R/W	0h	CRC Endian. This bit indicates the byte order within a word or half word of input data. 0h = LSB is lowest memory address and first to be processed. 1h = LSB is highest memory address and last to be processed.

Table 22-9. CRCCTRL Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1	BITREVERSE	R/W	0h	CRC Bit Input and output Reverse. This bit indicates that the bit order of each input byte used for the CRC calculation is reversed before it is passed to the generator, and that the bit order of the calculated CRC is be reversed when read from CRC_RESULT. 0h = Bit order is not reversed. 1h = Bit order is reversed.
0	POLYSIZE	R/W	0h	This bit indicates which CRC calculation is performed by the generator. 0h = CRC-32 ISO-3309 calculation is performed 1h = CRC-16 CCITT is performed

22.3.7 CRCSEED (Offset = 1104h) [Reset = 00000000h]

CRCSEED is shown in [Figure 22-7](#) and described in [Table 22-10](#).

Return to the [Summary Table](#).

CRC Seed Register. The Data written to this register is used to initialize the CRC result with this SEED value. Note that in 16-bit mode only the lower 16-bits of this value are used. After writing this register the CRC Output Result Register will reflect this value.

Figure 22-7. CRCSEED

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SEED																															
W-0h																															

Table 22-10. CRCSEED Field Descriptions

Bit	Field	Type	Reset	Description
31-0	SEED	W	0h	Seed Data 00000000h = Minimum value FFFFFFFFh = Maximum value

22.3.8 CRCIN (Offset = 1108h) [Reset = 00000000h]

CRCIN is shown in [Figure 22-8](#) and described in [Table 22-11](#).

Return to the [Summary Table](#).

CRC Input Data Register. The Data written to this register is used along with the current CRC result to calculate the next CRC result. This is done in a single clock cycle and requires no wait states. This register can be written as a byte, half word or word transfer and the correct number of bits will be used for the next CRC result. This register is also mapped to a range of registers starting at 0xTDB_X000 and ending at 0xTDB_XFFF to allow memcpy to be used instead of DMA for CRC calculations that do not exceed the bounds of the memory range of this register.

Figure 22-8. CRCIN

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA																															
W-0h																															

Table 22-11. CRCIN Field Descriptions

Bit	Field	Type	Reset	Description
31-0	DATA	W	0h	Input Data 00000000h = Minimum value FFFFFFFFh = Maximum value

22.3.9 CRCOUT (Offset = 110Ch) [Reset = 00000000h]

CRCOUT is shown in [Figure 22-9](#) and described in [Table 22-12](#).

Return to the [Summary Table](#).

CRC Output Result Register. This register stores the result of the current CRC calculation. Note when configured for 16-bit mode the upper bits will read back 0. Note that if output inversion is set in the CRC Control register it will be applied.

Figure 22-9. CRCOUT

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESULT																															
R-0h																															

Table 22-12. CRCOUT Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESULT	R	0h	Result 00000000h = Minimum value FFFFFFFFh = Maximum value

22.3.10 CRCIN_IDX[y] (Offset = 1800h + formula) [Reset = 00000000h]

CRCIN_IDX[y] is shown in [Figure 22-10](#) and described in [Table 22-13](#).

Return to the [Summary Table](#).

This register is dual mapped to CRCIN and is intended to allow operation with C memcpy routine.

Offset = 1800h + (y * 4h); where y = 0h to 1FFh

Figure 22-10. CRCIN_IDX[y]

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA																															
HW-0h																															

Table 22-13. CRCIN_IDX[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-0	DATA	HW	0h	Input Data 00000000h = Minimum value FFFFFFFFh = Maximum value



The AES accelerator module accelerates encryption and decryption operations in hardware based on the FIPS PUB 197 advanced encryption standard (AES).

23.1 AES Overview.....	1268
23.2 AES Operation.....	1269
23.3 AES Registers.....	1290

23.1 AES Overview

The AES accelerator module performs encryption and decryption of 128-bit data blocks with a 128-bit or 256-bit key in hardware according to the advanced encryption standard (AES). AES is a symmetric-key block cipher algorithm specified in FIPS PUB 197.

The AES accelerator features include:

- AES 128-bit block encryption and decryption
- DMA trigger support for automating ECB, CBC, OFB, and CFB block cipher modes as defined in NIST SP 800-38
- Support for accelerating CTR cipher mode by encrypting precalculated (nonce || counter) blocks and accelerating XOR of plaintext with the generated key stream
- Support for accelerating CBC-MAC tag computation (CBC DMA mode with zero initialization vector)
- On-the-fly key expansion for encryption and decryption
- Offline key generation for decryption
- Shadow register storing the initial key for all key lengths
- 8-bit byte or 32-bit word access to provide key data, input data, and output data
- AES ready interrupt
- Supported in RUN and SLEEP (see the *Operating Modes* section of the device technical reference manual)

The AES accelerator hardware consists of the 128-bit state memory and associated input/output registers, the AES encryption/decryption core and control logic, and the 256-bit AES key memory and associated input register. The AES hardware is shown in Figure 23-1.

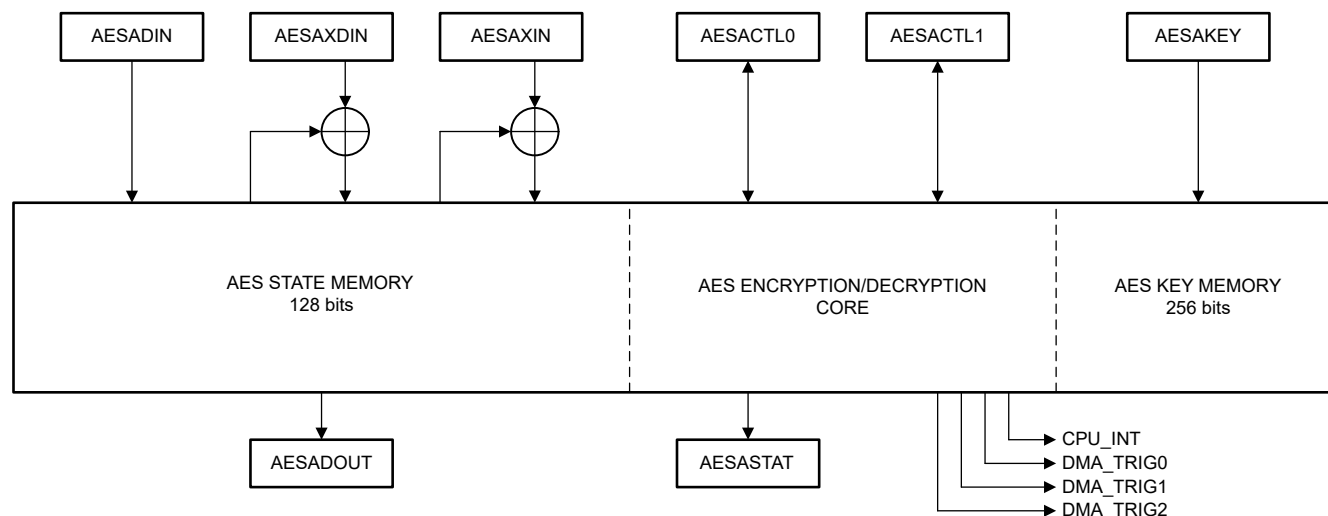


Figure 23-1. AES Accelerator Block Diagram

23.1.1 AES Performance

The AES accelerator provides fast encryption and decryption of 128-bit blocks. AES accelerator performance in both cycles and execution time for block encryption and block decryption (with pregenerated decryption key) is given in Table 23-1.

Table 23-1. AES Hardware Accelerator Key Performance Metrics

AES Key Length	Encryption (OPx==0x0)			Decryption (OPx==0x3)		
	Cycles	Time (32 MHz)	Time (80 MHz)	Cycles	Time (32 MHz)	Time (80 MHz)
128-bit	168	5.25 μ s	2.10 μ s	168	5.25 μ s	2.10 μ s
256-bit	234	7.31 μ s	2.93 μ s	234	7.31 μ s	2.93 μ s

23.2 AES Operation

The AES accelerator is configured with user software, and has two general modes of operation (configured with the AESACTL0 register):

- **Single block mode**
 - One 128-bit data block is encrypted or decrypted at a time under software control
 - No DMA triggers are generated directly by the AES hardware; application software writes and reads all data or configures DMA to load data with an alternate trigger
 - This mode can also be used to pregenerate the last round key needed for automated decryption in block cipher mode
- **Automated block cipher mode**
 - Multiple 128-bit data blocks can be encrypted or decrypted in an automated way using ECB, CBC, CFB, or OFB block cipher modes of operation
 - DMA triggers are generated to move data in and out of the accelerator automatically for a specified number of blocks with the assistance of 2 or 3 DMA channels (depending on the selected block cipher mode)

Regardless of single block mode or block cipher mode, the key length and operation type are specified by the KL and OP fields in the AESACTL0 register. The KLx and OPx fields are not reset by the AES software reset control (the SWRST bit in the AESACTL0 register). Whenever the KLx or OPx fields are changed, the key memory is reset.

- The KLx field selects the AES key size which is to be used (128 or 256 bits).
- The OPx field selects the AES operation to perform (encrypt, decrypt, generate last round decryption key, decrypt with pregenerated last round decryption key).

The KLx and OPx combination determines the cycle count per block operation, as shown in [Table 23-2](#).

Table 23-2. AES Operation Overview by OPx (operation) and KLx (key length) Configuration

AESOPx	AESKLx	Operation	Clock Cycles
00	00	AES128 encryption	168
	10	AES256 encryption	234
01	00	AES128 decryption	215
	10	AES256 decryption	292
10	00	AES128 encryption key schedule is performed (to generate last roundkey)	53
	10	AES256 encryption key schedule is performed (to generate last roundkey)	68
11	00	AES128 decryption (with last roundkey) is performed	168
	10	AES256 decryption (with last roundkey) is performed	234

Internally, the AES algorithm's operations are performed on a two-dimensional array of bytes called the STATE. The STATE consists of four rows of bytes, each containing four bytes, independent of whether AES128 or AES256 is performed. The input is assigned to the STATE array as shown in [Figure 23-2](#), with in[0] being the first data byte written into one of the AES accelerators input registers (AESADIN, AESAXDIN, and AESXIN). The encrypt or decrypt operations are then conducted on the STATE array, after which its final values can be read from the output with out[0] being the first data byte read from the AES accelerator data output register (AESADOUT).

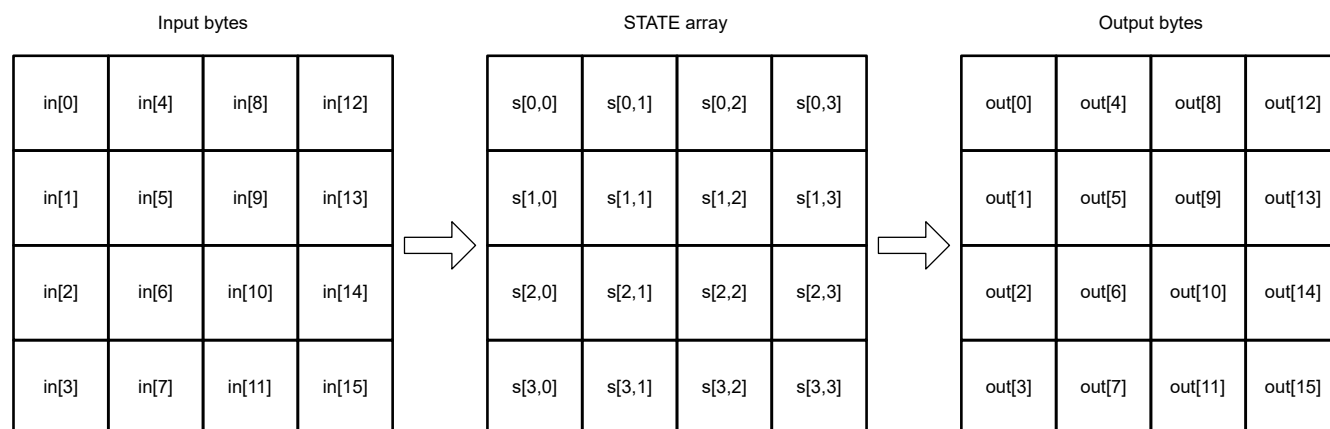


Figure 23-2. AES State Array Input and Output

If an encryption is to be performed, the initial state is called plaintext. If a decryption is to be performed, the initial state is called ciphertext.

23.2.1 AES Register Access Rules

The AES accelerator must be enabled before being configured for use through the PWREN register (see [peripheral power enable](#)). The AES accelerator is located in power domain 1 (PD1) and as such is only available in RUN and SLEEP modes. If the AES accelerator is enabled by application software, entry into STOP or STANDBY low-power modes will force the AES accelerator to be disabled while the device is in STOP or STANDBY mode, and the AES accelerator register contents will be lost (including any key or state contents).

Data Sizes for Reading and Writing Keys and Data

The module allows 32-bit word or 8-bit byte access to all key and data registers (AESAKEY, AESADIN, AESAXDIN, AESAXIN, and AESADOUT). Word and byte access must not be mixed while reading from or writing into one of the registers. However, it is possible to write one of the registers using byte access and another using word access.

General Access Restrictions

While the AES accelerator is busy (AESBUSY bit is set in the AESASTAT register):

- AESADOUT always reads as zero.
- The DOUTCNTx counter, the DOUTRD flag, and the DINWR flag are reset.
- Any attempt to change OPx, KLx, DINWR, or KEYWR is ignored.
- Writing to AESAKEY, AESADIN, AESAXDIN, or AESAXIN aborts the current operation, the complete module is reset (except for the IMASK registers, OPx, and KLx), and the AES error flag ERRFG is set.

AESADIN, AESAXDIN, AESAXIN, and AESAKEY are write-only registers and always read as zero.

Writing data into AESADIN, AESAXDIN, or AESAXIN influences the content of the corresponding output data; for example, writing in[0] alters out[0], writing in[1] alters out[1], and so on. However, interleaved operation is possible; for example, first reading out[0] and then writing in[0], and continuing with reading out[1] and then writing in[1], and so on. This interleaved operation must be either byte or word access on in[x] and out[x].

Access Restriction With Block Cipher Modes Enabled

When automated block cipher modes are enabled (CMEN is set in AESACTL0) and a cipher block operation is being processed (BLKCNTx > 0), writes to the following bits are ignored, independent of the BUSY bit status: CMEN, CMx, KLx, OPx, and BLKCNTx. Writing to AESAKEY aborts the cipher block mode operation if BUSY = 1, and the complete module is reset (except for the IMASK registers, OPx, and KLx).

23.2.2 Loading the Key

The key can be loaded by writing to the AESAKEY register or by setting the KEYWR bit. Depending on the selected key length (KLx in the AESCTL0 register), a different number of bits must be loaded:

- If KLx = 00, the 128-bit key must be loaded using either 16 byte-writes or 4 word-writes to AESAKEY
- If KLx = 10, the 256-bit key must be loaded using either 32 byte-writes or 8 word-writes to AESAKEY

The key memory is reset after changing the KLx bits.

If a key was loaded previously without changing the OPx bits, the KEYWR flag is cleared with the first write access to AESAKEY.

If an operation is triggered without writing a new key, the last key is used. The key must always be written before writing the data.

The AES algorithm operates not only on the STATE but also on the key. To avoid the need to reload the key for each operation, a key buffer is included such that the key expansion operations for round key generation do not remove the cipher key loaded in AESAKEY.

23.2.3 Loading Data

The state can be loaded by writing to AESADIN, AESAXDIN, or AESAXIN with 16 byte writes or 4 word writes. Do not mix byte and word mode when writing the state. Writing to a mixture of AESADIN, AESAXDIN, and AESAXIN using the same byte or word data format is allowed. When the 16th byte or 4th word of the state is written, DINWR is set in the AESASTAT register.

When writing to AESADIN, the corresponding byte or word of the state is overwritten. If AESADIN is used to write the last byte or word of the state, encryption or decryption starts automatically.

When writing to AESAXDIN, the corresponding byte or word is XORed with the current byte or word of the state. If AESAXDIN is used to write the last byte or word of the state, encryption or decryption starts automatically.

Writing to AESAXIN has the same behavior as writing to AESAXDIN: the corresponding byte or word is XORed with the current byte or word of the state; however, writing the last byte or word of the state using AESAXIN does not start encryption or decryption.

Note

The AESAXIN register can be used to accelerate the XOR operation of plaintext with an encrypted key stream (nonce || counter) when using the AES accelerator to assist with implementation of a CTR cipher mode. In a CTR cipher mode scenario, a plaintext block can be loaded to AESAXIN after encryption of the (nonce || counter) value has completed, as the encrypted (nonce || counter) value will be held in the STATE. After loading data into AESAXIN, the (plaintext ⊕ (nonce || counter)) output can be read from AESADOUT.

23.2.4 Reading Data

The STATE can be read when the BUSY bit in AESASTAT is cleared. The STATE is read using 16 byte reads or 4 word reads from AESADOUT. When all 16 bytes are read, the DOUTRD flag in AESASTAT indicates completion of the read.

23.2.5 Triggering an Encryption or Decryption

An encrypt or decrypt operation is triggered if the STATE was completely written in the AESADIN or AESAXDIN registers. Alternatively, the bit DINWR bit in the AESASTAT register can be set to trigger an operation if CMEN is cleared.

23.2.6 Single Block Operations

Single block operations include encryption, decryption, and decryption key generation.

23.2.6.1 Encryption

To perform encryption:

1. Set OPx = 0x0 to select encryption. Note that changing the OPx bits clears the KEYWR flag and a new key must be loaded in the next step.
2. Load the key as described in [Section 23.2.2](#).
3. Load the STATE (data) as described in [Section 23.2.3](#). After the data is loaded, the AES module starts the encryption process.
4. Wait for the operation to complete (wait for the BUSY flag to clear).
5. After the encryption is complete, the result can be read from AESADOUT as described in [Section 23.2.4](#).
6. To encrypt additional data with the same key loaded in step 2, write the new data into AESADIN after the results of the operation on the previous data were read from AESADOUT. When an additional 16 data bytes are written, the module automatically starts the encryption using the key loaded in step 2.

Note

Setting the DINWR bit in the AESASTAT register triggers another encryption, and the AES accelerator starts the encryption using the output data from the previous encryption as the input data. This is useful in certain block cipher modes with feedback.

The encryption process is given in [Figure 23-3](#) (128-bit key with 10 rounds shown).

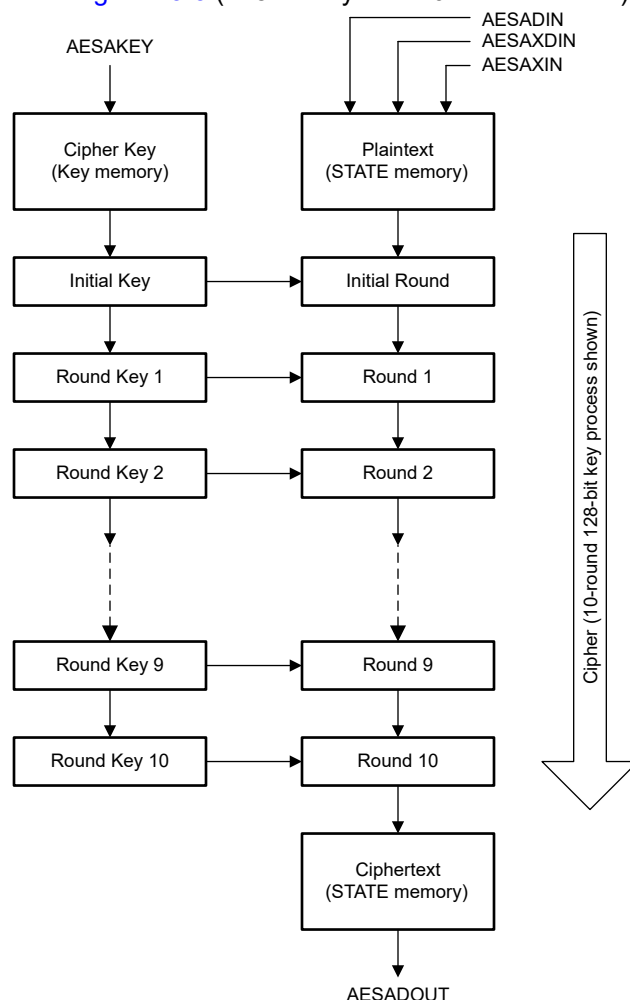


Figure 23-3. AES Encryption (OPx=0x0, 128-bit Key)

23.2.6.2 Decryption

To perform decryption:

1. Set OPx = 0x1 to select decryption using the same key used for encryption, or set OPx = 0x3 if the first round key required for decryption (the last round key used for encryption) is **already pregenerated** and will be loaded in step 2. Changing the OPx bits clears the KEYWR flag, and a new key must be loaded in step 2.
2. Load the key according to [Section 23.2.2](#).
3. Load the STATE (data) according to [Section 23.2.3](#). After the data is loaded, the AES module starts the decryption.
4. Wait for the operation to complete (wait for the BUSY flag to clear).
5. After the decryption is ready, the result can be read from AESADOUT as described in [Section 23.2.4](#).
6. If additional data should be decrypted with the same key loaded in step 2, new data can be written into AESADIN after the results of the operation on the previous data were read from AESADOUT. When additional 16 data bytes are written, the module automatically starts the decryption using the key loaded in step 2.

The decryption process with decryption round key generation is given in [Figure 23-4](#) (128-bit key with 10 rounds shown).

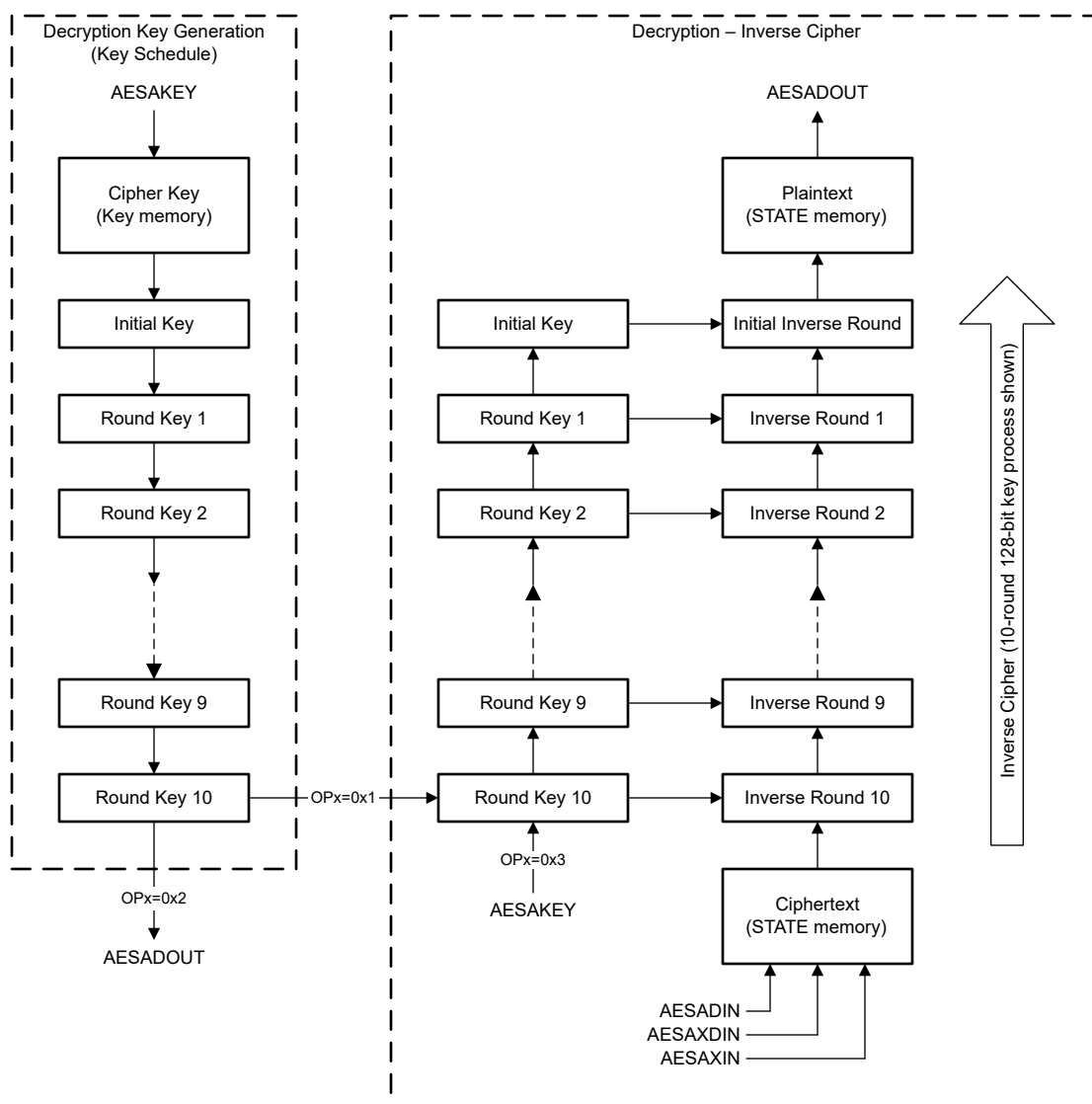


Figure 23-4. AES Decryption (OPx = 0x1, 128-bit Key)

23.2.6.2.1 Pregenerating a Decryption Key

When decryption is performed using $OPx=0x1$, the original cipher key used for encryption is loaded into AESAKEY. Decryption requires execution of the key schedule to obtain the round key used for the last round of encryption. That final encryption round key is then used by the decryption logic as the first round key for decryption. In the case of $OPx=0x1$, before the decryption begins the key schedule will be executed on the key material loaded into AESAKEY to obtain the last round key. Once the last round key used for encryption is generated, the decryption process begins automatically.

While $OPx=0x1$ is convenient for single block cases (as the original encryption key can be loaded without special handling), it is not efficient for handling multiple blocks, because the key schedule will be executed for each block to obtain the last round key which is needed to start the decryption process. To reduce the cycle count required for decryption, it is possible to pregenerate the last round encryption key using $OPx=0x2$. Then that pregenerated round key can be used to immediately start any decryption operations by loading the pregenerated key material and running the $OPx=0x3$ decryption operation, which assumes that the key material is the round key from the last round of encryption.

To execute the key schedule and obtain the last round key used for encryption (first round key used for decryption) without actually running a decryption operation, follow the steps below:

1. Set OPx to $0x2$ to select decryption key generation.
2. Load the cipher key used for encryption into AESAKEY as described in [Section 23.2.2](#). Generation will start as soon as the key material is loaded.
3. While the AES module is busy executing the key schedule, the AESBUSY bit is set. 53 clock cycles are required to complete the key generation for a 128-bit cipher key. After completion, the AESRDY interrupt is set and the 128-bit result can be read from the AESDOUT. AESRDY is cleared when reading AESADOUT or writing to AESAKEY or AESADIN.

Once a decryption first round key is generated, it can be loaded into AESAKEY and used for decryption with $OPx=3$.

23.2.7 Block Cipher Mode Operations

Block cipher modes are used to implement the electronic code book (ECB), cipher block chaining (CBC), output feedback (OFB) and cipher feedback (CFB) block cipher modes using AES as the underlying block cipher. These modes work together with the DMA through 2 or 3 DMA triggers to support easy and fast encryption or decryption of more than 128 bits.

23.2.7.1 Electronic Codebook (ECB) Mode

The electronic codebook (ECB) cipher is the simplest block cipher mode. The plaintext data is divided into 128-bit blocks, and each block is encrypted and decrypted independently from any other block. The ECB cipher is shown in [Figure 23-5](#). Note that each 128-bit data block can be encrypted or decrypted individually without any knowledge of the other blocks of plaintext or ciphertext.

While ECB is simple to understand and implement, it has a key disadvantage: the same 128-bit plaintext block is always encrypted into the same ciphertext block, allowing patterns in the ciphertext to be detected.

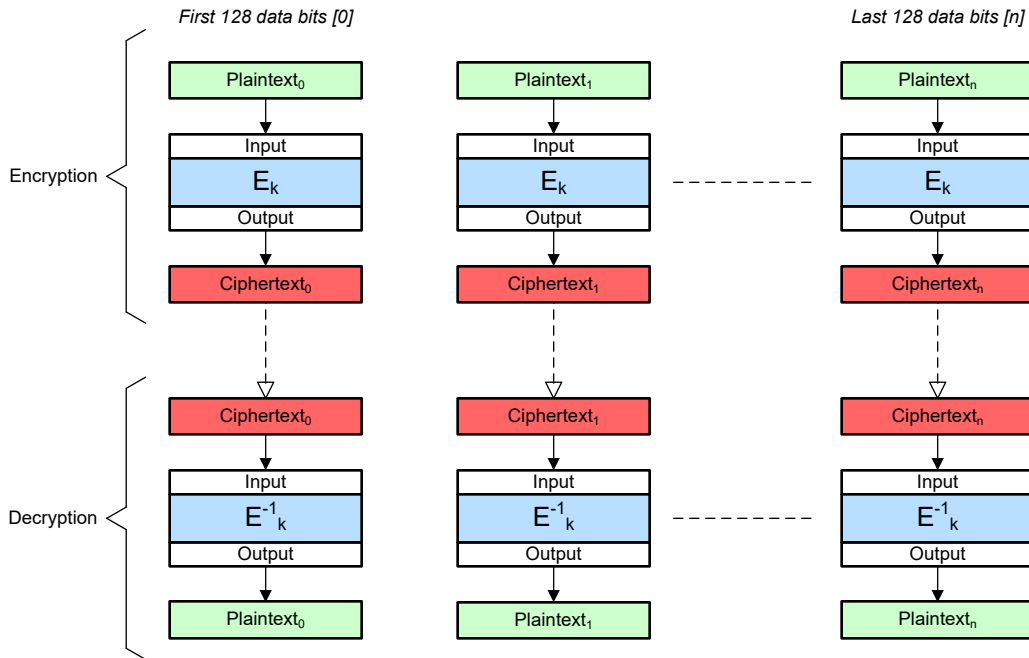


Figure 23-5. ECB Cipher

The AES accelerator supports automated encryption and decryption of more than 128 bits of data in ECB block cipher mode through the use of DMA together with the AES accelerator. ECB mode utilizes two DMA channels (referred to as DMA_A, DMA_B in this section) for output and input to the AES accelerator, respectively. The DMA channel usage for ECB mode encryption and decryption operations is given in Table 23-3.

Table 23-3. ECB Block Cipher DMA Behavior

AES Operation			AES DMA Triggers		
CMEN	CMx	OPx	DMA_A (DMA_TRIG0)	DMA_B (DMA_TRIG1)	DMA_C (DMA_TRIG2)
0x1	0x0 (ECB)	0x0 (encryption)	Used to read ciphertext from AESADOUT	Used to write plaintext to AESADIN, which also triggers start of block encryption when 128 bits are written to AESADIN	Not used
		0x1/0x2 (decryption)	Used to read plaintext from AESADOUT	Used to write ciphertext to AESADIN, which also triggers start of block decryption when 128 bits are written to AESADIN	Not used

23.2.7.1.1 ECB Encryption

ECB mode encryption of N blocks of plaintext into N blocks of ciphertext without CPU interaction is achieved through the use of 2 DMA channels (referred to as DMA_A and DMA_B). To implement ECB encryption follow these steps:

- Configure the AESACTL0 register for block cipher encryption mode using ECB:
 - Set CMEN to enable block cipher mode
 - Set CMx to ECB mode
 - Set OPx to 0x0 (encryption mode)
- Load key as described in Section 23.2.2
- Configure DMA_A channel for saving ciphertext:
 - Set DMA channel trigger selection to AES0 trigger
 - Set DMA channel source address to AESADOUT
 - Set DMA channel destination address to location where ciphertext is to be stored (for example, SRAM)
 - Set DMA channel transfer size to $N \times 4$

- e. Set DMA channel mode to single transfer mode
- f. In the AES event registers, unmask DMA0 in the IMASK register of DMA_TRIG0
4. Configure DMA_B channel for loading plaintext:
 - a. Set DMA channel trigger selection to AES1 trigger
 - b. Set DMA channel source address to location where plaintext is stored (for example, SRAM)
 - c. Set DMA channel destination address to AESADIN
 - d. Set DMA channel transfer size to $N*4$
 - e. Set DMA channel mode to single transfer mode
 - f. In the AES event registers, unmask DMA1 in the IMASK register of DMA_TRIG1
5. Configure and enable the DMA interrupt for the DMA_A channel in the DMA controller
6. Start encryption by writing the block count N to BLKCNTx in the AESACTL1 register
7. Wait for the DMA_A channel interrupt which indicates completion of the operation. The ciphertext output will be stored in the location configured in step 3c.

23.2.7.1.2 ECB Decryption

ECB mode decryption of N blocks of ciphertext into N blocks of plaintext without CPU interaction is achieved through the use of 2 DMA channels (referred to as DMA_A and DMA_B). To implement ECB decryption follow these steps:

1. Configure the AESACTL0 register to pregenerate the decryption key (last round key):
 - a. Clear CMEN to disable block cipher mode
 - b. Set OPx to 0x2 (decryption key generation)
 - c. Write key into AESAKEY register as described in [Section 23.2.2](#)
 - d. Wait for the BUSY status in the AESSTAT register to clear, indicating key generation has completed
2. Configure the AESACTL0 register for block cipher decryption mode using ECB:
 - a. Set CMEN to enable block cipher mode
 - b. Set CMx to ECB mode
 - c. Set OPx to 0x3 (decryption mode)
3. Set AESKEYWR bit in AESSTAT (to use pregenerated key from step 1)
4. Configure DMA_A channel for saving plaintext:
 - a. Set DMA channel trigger selection to AES0 trigger
 - b. Set DMA channel source address to AESADOUT
 - c. Set DMA channel destination address to location where plaintext is to be stored (for example, SRAM)
 - d. Set DMA channel transfer size to $N*4$
 - e. Set DMA channel mode to single transfer mode
 - f. In the AES event registers, unmask DMA0 in the IMASK register of DMA_TRIG0
5. Configure DMA_B channel for loading ciphertext:
 - a. Set DMA channel trigger selection to AES1 trigger
 - b. Set DMA channel source address to location where ciphertext is stored (for example, SRAM)
 - c. Set DMA channel destination address to AESADIN
 - d. Set DMA channel transfer size to $N*4$
 - e. Set DMA channel mode to single transfer mode
 - f. In the AES event registers, unmask DMA1 in the IMASK register of DMA_TRIG1
6. Configure and enable the DMA interrupt for the DMA_A channel in the DMA controller
7. Start decryption by writing the block count N to BLKCNTx in the AESACTL1 register
8. Wait for the DMA_A channel interrupt which indicates completion of the operation. The plaintext output will be stored in the location configured in step 3c.

23.2.7.2 Cipher Block Chaining (CBC) Mode

The cipher block chaining (CBC) cipher mode builds upon the ECB cipher mode to make the ciphertext output for each block dependent not only on the plaintext and the cipher key k , but also upon the ciphertext of the previous block. The CBC cipher is shown in [Figure 23-6](#). Like ECB mode, the plaintext data is divided into 128-bit blocks. Unlike ECB mode, in CBC mode each new plaintext block is bit-wise XORed with the previous ciphertext block to create the input to the AES block cipher.

In CBC mode, an unpredictable initialization vector (IV) must be provided. The initialization vector is XORed with the first plaintext block, as there is no "previous" ciphertext block to XOR the first plaintext block with.

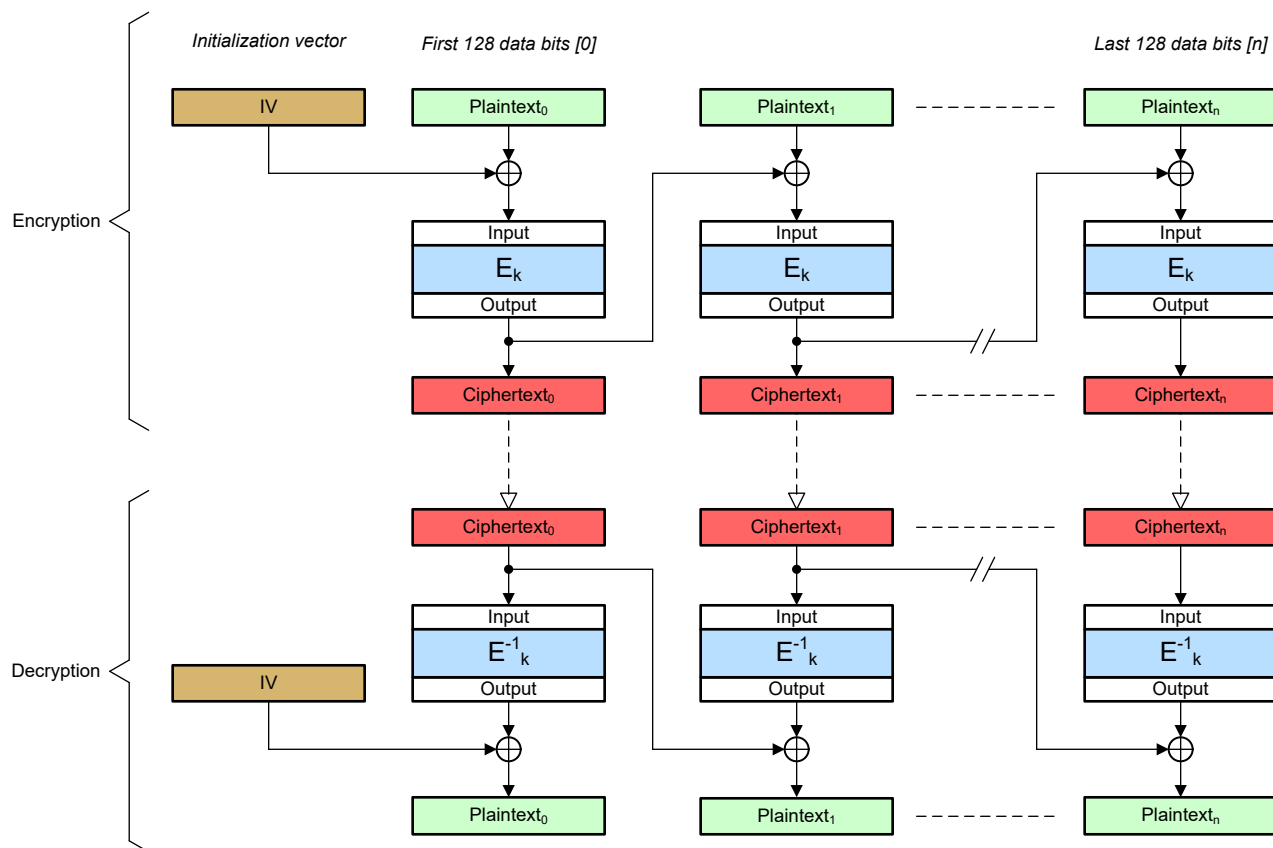


Figure 23-6. CBC Cipher

The AES accelerator supports automated encryption and decryption of more than 128 bits of data in CBC block cipher mode through the use of DMA together with the AES accelerator. CBC mode utilizes two DMA channels (referred to as DMA_A, DMA_B in this section) for output and input to the AES accelerator, respectively, when encrypting. CBC mode utilizes three DMA channels (DMA_A, DMA_B, and DMA_C) for input, output, and input to the AES accelerator when decrypting. The DMA channel usage for CBC mode encryption and decryption operations is given in [Table 23-4](#).

Table 23-4. CBC Block Cipher DMA Behavior

AES Operation			AES DMA Triggers		
CMEN	CMx	OPx	DMA_A (DMA_TRIG0)	DMA_B (DMA_TRIG1)	DMA_C (DMA_TRIG2)
0x1	0x1 (CBC)	0x0 (encryption)	Used to read ciphertext from AESADOUT	Used to write plaintext to AESAXIN, which also triggers start of block encryption when 128 bits are written to AESADIN	Not used
		0x1/0x2 (decryption)	Used to write the initialization vector and previous ciphertext block to AESAXIN	Used to read plaintext from AESADOUT	Used to write next ciphertext to AESADIN, which also triggers start of block decryption when 128 bits are written to AESADIN

23.2.7.2.1 CBC Encryption

CBC-mode encryption of N blocks of plaintext into N blocks of ciphertext without CPU interaction is achieved through the use of 2 DMA channels (referred to as DMA_A and DMA_B). To implement CBC encryption follow these steps:

- Configure the AESACTL0 register for block cipher encryption mode using CBC:
 - Reset AES module to clear internal state memory (set SWRST in the AESACTL0 register)
 - Set CMEN to enable block cipher mode
 - Set CMx to CBC mode
 - Set OPx to 0x0 (encryption mode)
- Load key as described in [Section 23.2.2](#)
- Load initialization vector (IV) into AESAXIN
 - Loading to AESAXIN does not start any encryption
 - The STATE must be reset (step 1a above) so that the IV is XORed with zeros when loaded
- Configure DMA_A channel for saving ciphertext:
 - Set DMA channel trigger selection to AES0 trigger
 - Set DMA channel source address to AESADOUT
 - Set DMA channel destination address to location where ciphertext is to be stored (for example, SRAM)
 - Set DMA channel transfer size to $N*4$ words
 - Set DMA channel mode to single transfer mode
 - In the AES event registers, unmask DMA0 in the IMASK register of DMA_TRIG0
- Configure DMA_B channel for loading plaintext:
 - Set DMA channel trigger selection to AES1 trigger
 - Set DMA channel source address to location where plaintext is stored (for example, SRAM)
 - Set DMA channel destination address to AESAXDIN
 - Set DMA channel transfer size to $N*4$ words
 - Set DMA channel mode to single transfer mode
 - In the AES event registers, unmask DMA1 in the IMASK register of DMA_TRIG1
- Configure and enable the DMA interrupt for the DMA_A channel in the DMA controller
- Start encryption by writing the block count N to BLKCNTx in the AESACTL1 register
- Wait for the DMA_A channel interrupt which indicates completion of the operation. The ciphertext output will be stored in the location configured in step 4c.

23.2.7.2.2 CBC Decryption

CBC-mode decryption of N blocks of ciphertext into N blocks of plaintext without CPU interaction is achieved through the use of 3 DMA channels (referred to as DMA_A, DMA_B, and DMA_C). To implement CBC decryption follow these steps:

- Configure the AESACTL0 register to pre-generate the decryption key (last round key):
 - Clear CMEN to disable block cipher mode
 - Set OPx to 0x2 (decryption key generation)

- c. Write key into AESAKEY register as described in [Section 23.2.2](#)
 - d. Wait for the BUSY status in the AESSTAT register to clear, indicating key generation has completed
2. Configure the AESACTL0 register for block cipher decryption mode using CBC:
 - a. Set CMEN to enable block cipher mode (0x1)
 - b. Set CMx to 0x1 (CBC mode)
 - c. Set OPx to 0x3 (decryption mode)
3. Set the AESKEYWR bit in AESSTAT (to use pre-generate key from step 1)
4. Configure DMA_A channel for loading the initialization vector and ciphertext:
 - a. Set up the ciphertext buffer in memory and pre-pend the initialization vector (IV) to the ciphertext
 - b. Set DMA channel trigger selection to AES0 trigger
 - c. Set DMA channel source address to location where initialization vector is stored in front of the following ciphertext (for example, SRAM)
 - d. Set DMA channel destination address to AESAXIN
 - e. Set DMA channel transfer size to $N \times 4$ words
 - f. Set DMA channel mode to single transfer mode
 - g. In the AES event registers, unmask DMA0 in the IMASK register of DMA_TRIG
5. Configure DMA_B channel for saving plaintext:
 - a. Set DMA channel trigger selection to AES1 trigger
 - b. Set DMA channel source address to AESADOUT
 - c. Set DMA channel destination address to location where plaintext is to be stored (for example, SRAM)
 - d. Set DMA channel transfer size to $N \times 4$ words
 - e. Set DMA channel mode to single transfer mode
 - f. In the AES event registers, unmask DMA1 in the IMASK register of DMA_TRIG
6. Configure DMA_C channel for loading ciphertext:
 - a. Set DMA channel trigger selection to AES2 trigger
 - b. Set DMA channel source address to location where ciphertext is stored after the IV (for example, SRAM)
 - c. Set DMA channel destination address to AESADIN
 - d. Set DMA channel transfer size to $N \times 4$ words
 - e. Set DMA channel mode to single transfer mode
 - f. In the AES event registers, unmask DMA2 in the IMASK register of DMA_TRIG
7. Configure and enable the DMA interrupt for the DMA_A and DMA_B channels in the DMA controller
8. Start decryption by writing the block count N to BLKCNTx in the AESACTL1 register
9. Wait for the DMA_B channel interrupt which indicates completion of the operation. The plaintext output will be stored in the location configured in step 5c.

23.2.7.3 Output Feedback (OFB) Mode

The output feedback mode leverages an initialization vector (IV) to generate a keystream by repeatedly encrypting the IV with the cipher key. The output ciphertext is obtained by XORing plaintext with the encrypted and re-encrypted versions of the initialization vector. The OFB cipher is shown in [Figure 23-7](#).

In OFB mode, the initialization vector must be a nonce (number used once). To prevent loss of confidentiality, each IV must only be used one time with a given key, and any value passed into the cipher E_k for a given key k must not be used as an initialization vector with the same key k .

Note

As OFB is a stream cipher, the AES block function is used in the forward (encryption) mode when performing OFB encryption or OFB decryption. When decrypting, the keystream is simply regenerated again to be XORed with the ciphertext, giving back the plaintext.

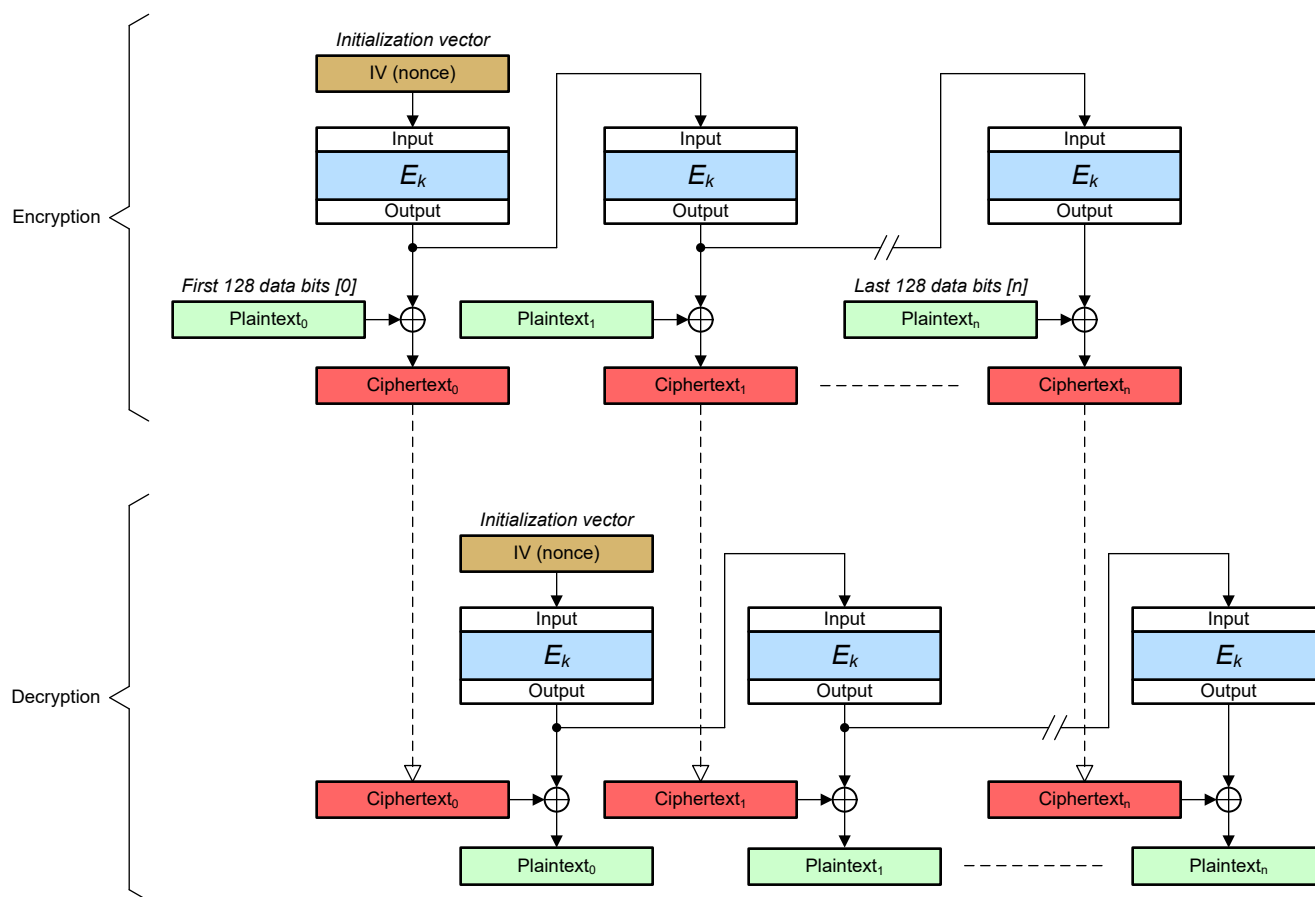


Figure 23-7. OFB Cipher

The AES accelerator supports automated encryption and decryption of more than 128 bits of data in OFB block cipher mode through the use of DMA together with the AES accelerator. OFB mode utilizes three DMA channels (referred to as DMA_A, DMA_B, and DMA_C in this section) for input, output, and input to the AES accelerator, respectively. The DMA channel usage for OFB mode encryption and decryption operations is given in [Table 23-5](#).

Table 23-5. OFB Cipher DMA Behavior

AES Operation			AES DMA Triggers		
CMEN	CMx	OPx	DMA_A (DMA_TRIG0)	DMA_B (DMA_TRIG1)	DMA_C (DMA_TRIG2DMA_TRIG)
0x1	0x2 (OFB)	0x0 (encryption)	Used to write plaintext of the current block to AESAXIN	Used to read ciphertext from AESADOUT	Used to write the plaintext of the current block to AESAXDIN, which also triggers the next encryption
		0x1 (decryption)	Used to write the ciphertext of the current block to AESAXIN	Used to read the plaintext from AESADOUT	Used to write the ciphertext of the current block to AESAXDIN, which also triggers the next decryption

23.2.7.3.1 OFB Encryption

OFB mode encryption of N blocks of plaintext into N blocks of ciphertext without CPU interaction is achieved through the use of 3 DMA channels (referred to as DMA_A, DMA_B, and DMA_C). To implement OFB encryption follow these steps:

1. Configure the AESACTL0 register for block cipher encryption mode using OFB:

- a. Reset AES module to clear the internal state memory (set SWRST in the AESACTL0 register)
 - b. Set CMEN to enable block cipher mode
 - c. Set CMx to OFB mode (0x2)
 - d. Set OPx to encryption mode (0x0)
2. Load key as described in [Section 23.2.2](#)
3. Load initialization vector (IV) into AESAXIN
 - a. Loading to AESAXIN does not start any encryption
 - b. The STATE must be reset (step 1a above) so that the IV is XORed with zeros when loaded
4. Configure the DMA_A channel for loading plaintext:
 - a. Set the DMA channel trigger selection to AES0 trigger
 - b. Set the DMA channel source address to the location where plaintext is stored (for example, SRAM)
 - c. Set the DMA channel destination address to AESAXIN
 - d. Set the DMA channel transfer size to $N \times 4$ words
 - e. Set the DMA channel mode to single transfer mode
 - f. In the AES event registers, unmask DMA0 in the IMASK register of DMA_TRIG
- 0
5. Configure the DMA_B channel for reading ciphertext:
 - a. Set the DMA channel trigger selection for AES1 trigger
 - b. Set the DMA channel source address to AESADOUT
 - c. Set the DMA channel destination address to the location where ciphertext is to be stored (for example, SRAM)
 - d. Set the DMA channel transfer size to $N \times 4$ words
 - e. Set the DMA channel mode to single transfer mode
 - f. In the AES event registers, unmask DMA1 in the IMASK register of DMA_TRIG
- 1
6. Configure the DMA_C channel for loading plaintext:
 - a. Set the DMA channel trigger selection for AES2 trigger
 - b. Set the DMA channel source address to the location where plaintext is stored (for example, SRAM)
 - c. Set the DMA channel destination address to AESAXDIN
 - d. Set the DMA channel transfer size to $N \times 4$ words
 - e. Set the DMA channel mode to single transfer mode
7. Configure and enable the DMA interrupt for the DMA_B channel in the DMA controller
8. Start encryption by writing the block count N to BLKCNTx in the AESACTL1 register and setting the DINWR bit in the AESASTAT register
9. Wait for the DMA_B channel interrupt which indicates completion of the operation. The ciphertext output will be stored in the location configured in step 5c.

23.2.7.3.2 OFB Decryption

OFB mode decryption of N blocks of ciphertext into N blocks of plaintext without CPU interaction is achieved through the use of 3 DMA channels (referred to as DMA_A, DMA_B, and DMA_C). To implement OFB decryption follow these steps:

1. Configure the AESACTL0 register for block cipher decryption mode using OFB:
 - a. Reset the AES module to clear the internal state memory (set SWRST in the AESCTL0 register)
 - b. Set the CMEN to enable block cipher mode
 - c. Set CMx to OFB mode (0x2)
 - d. Set OPx to decryption mode (0x1)
2. Load key as described in [Section 23.2.2](#)
3. Load initialization vector IV into AESAXIN
 - a. Loading to AESAXIN does not start any decryption
 - b. The STATE must be reset (step 1a above) so that the IV is XORed with zeros when loaded
4. Configure the DMA_A channel for loading ciphertext:
 - a. Set the DMA channel trigger selection to AES0 trigger

- b. Set the DMA channel source address to the location where ciphertext is stored (for example, SRAM)
 - c. Set the DMA channel destination address to AESAXIN
 - d. Set the DMA channel transfer size to $N \times 4$ words
 - e. Set the DMA channel mode to single transfer mode
5. Configure the DMA_B channel for reading plaintext:
 - a. Set the DMA channel trigger selection for AES1 trigger
 - b. Set the DMA channel source address to AESADOUT
 - c. Set the DMA channel destination address to the location where plaintext is to be stored (for example, SRAM)
 - d. Set the DMA channel transfer size to $N \times 4$ words
 - e. Set the DMA channel mode to single transfer mode
 - f. In the AES event registers, unmask DMA1 in the IMASK register of DMA_TRIG
- 1
6. Configure the DMA_C channel for loading ciphertext:
 - a. Set the DMA channel trigger selection for AES2 trigger
 - b. Set the DMA channel source address to the location where ciphertext is stored (for example, SRAM)
 - c. Set the DMA channel destination address to AESAXDIN
 - d. Set the DMA channel transfer size to $N \times 4$ words
 - e. Set the DMA channel mode to single transfer mode
7. Configure and enable the DMA interrupt for the DMA_B channel in the DMA controller
8. Start decryption by writing the block count N to BLKCNTR in the AESACTL1 register and setting the DINWR bit in the AESASTAT register
9. Wait for the DMA_B channel interrupt which indicates completion of the operation. The plaintext output will be stored in the location configured in step 5c.

23.2.7.4 Cipher Feedback (CFB) Mode

The cipher feedback (CFB) mode is similar to the [output feedback \(OFB\) mode](#), with the key difference being that the input block to the block cipher E used to generate the key stream is taken from the previous ciphertext block (after being XORed with the plaintext), vs. taken before being XORed with the plaintext (as in the case of OFB). As a result, the keystream is dependent upon the plaintext, which is not true of OFB. The CFB cipher is shown in [Figure 23-8](#).

Like OFB, CFB requires an initialization vector (IV). In CFB mode, the initialization vector (IV) must be unpredictable.

Note

As CFB is a stream cipher, the AES block function is used in the forward (encryption) mode when performing CFB encryption or CFB decryption.

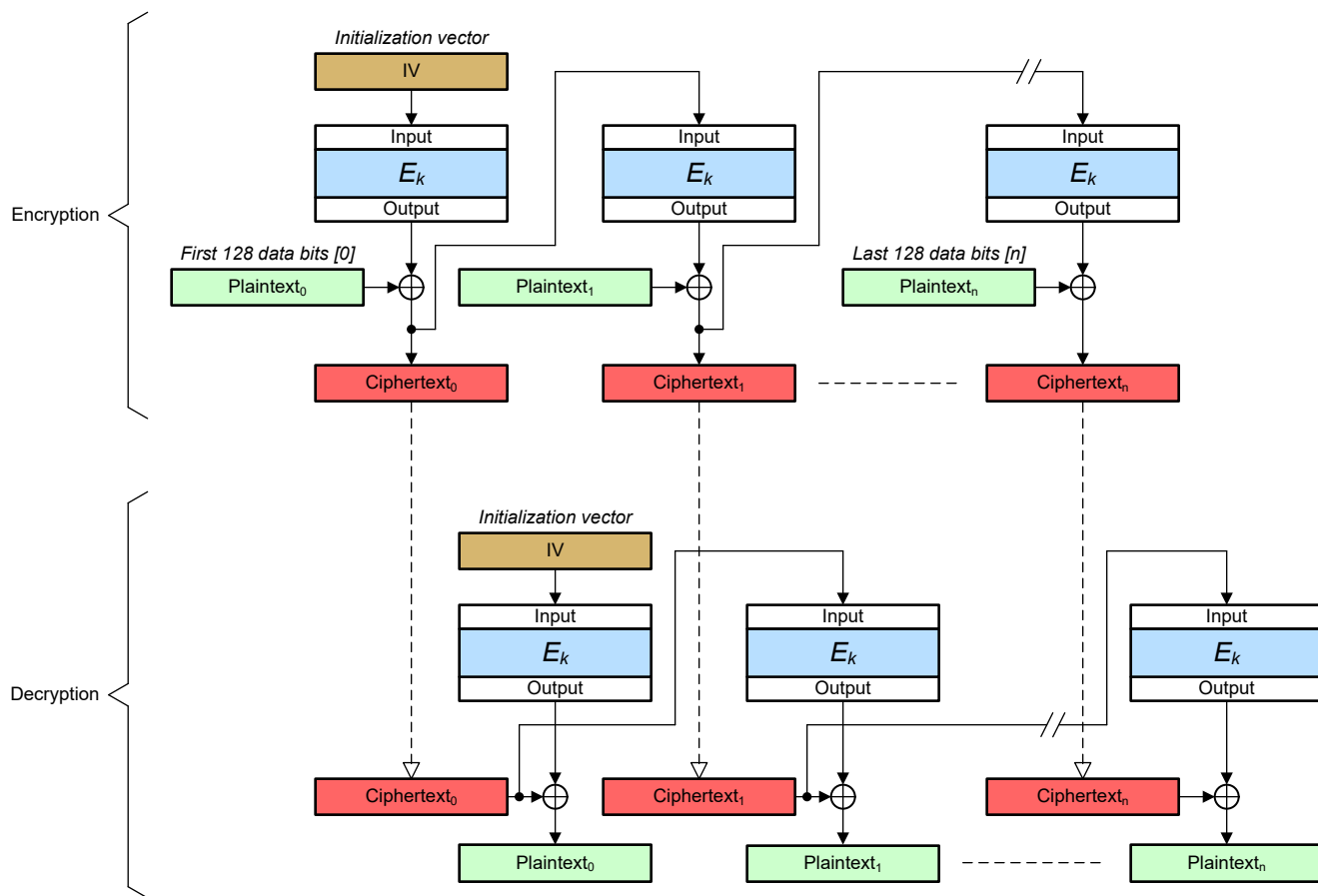


Figure 23-8. CFB Cipher

The AES accelerator supports automated encryption and decryption of more than 128 bits of data in CFB block cipher mode through the use of DMA together with the AES accelerator. CFB mode utilizes three DMA channels (referred to as DMA_A, DMA_B, and DMA_C in this section) for input, output, and input to the AES accelerator, respectively. The DMA channel usage for CFB mode encryption and decryption operations is given in [Table 23-6](#).

Table 23-6. CFB Cipher DMA Behavior

AES Operation			AES DMA Triggers		
CMEN	CMx	OPx	DMA_A (DMA_TRIG0)	DMA_B (DMA_TRIG1)	DMA_C (DMA_TRIG2)
0x1	0x3 (CFB)	0x0 (encryption)	Used to write plaintext of the current block to AESAXIN	Used to read ciphertext from AESADOUT, which also triggers the next encryption	Not used
		0x1 (decryption)	Used to write the ciphertext of the current block to AESAXIN	Used to read the plaintext from AESADOUT	Used to write the ciphertext of the current block to AESADIN, which also triggers the next decryption

23.2.7.4.1 CFB Encryption

CFB mode encryption of N blocks of plaintext into N blocks of ciphertext without CPU interaction is achieved through the use of two DMA channels (referred to as DMA_A, DMA_B). To implement CFB encryption follow these steps:

1. Configure the AESACTL0 register for block cipher encryption mode using OFB:
 - a. Reset AES module to clear the internal state memory (set SWRST in the AESACTL0 register)
 - b. Set CMEN to enable block cipher mode

- c. Set CMx to CFB mode (0x3)
 - d. Set OPx to encryption mode (0x0)
2. Load key as described in [Section 23.2.2](#)
3. Load initialization vector (IV) into AESAXIN
 - a. Loading to AESAXIN does not start any encryption
 - b. The STATE must be reset (step 1a above) so that the IV is XORed with zeros when loaded
4. Configure the DMA_A channel for loading plaintext:
 - a. Set the DMA channel trigger selection to AES0 trigger
 - b. Set the DMA channel source address to the location where plaintext is stored (for example, SRAM)
 - c. Set the DMA channel destination address to AESAXIN
 - d. Set the DMA channel transfer size to $N*4$ words
 - e. Set the DMA channel mode to single transfer mode
5. Configure the DMA_B channel for reading ciphertext:
 - a. Set the DMA channel trigger selection for AES1 trigger
 - b. Set the DMA channel source address to AESADOUT
 - c. Set the DMA channel destination address to the location where ciphertext is to be stored (for example, SRAM)
 - d. Set the DMA channel transfer size to $N*4$ words
 - e. Set the DMA channel mode to single transfer mode
 - f. In the AES event registers, unmask DMA1 in the IMASK register of DMA_TRIG
- 1
6. Configure and enable the DMA interrupt for the DMA_B channel in the DMA controller
7. Start encryption by writing the block count N to BLKCNTx in the AESACTL1 register and setting the DINWR bit in the AESASTAT register
8. Wait for the DMA_B channel interrupt which indicates completion of the operation. The ciphertext output will be stored in the location configured in step 5c.

23.2.7.4.2 CFB Decryption

CFB mode decryption of N blocks of ciphertext into N blocks of plaintext without CPU interaction is achieved through the use of 3 DMA channels (referred to as DMA_A, DMA_B, and DMA_C). To implement CFB decryption follow these steps:

1. Configure the AESACTL0 register for block cipher decryption mode using OFB:
 - a. Reset the AES module to clear the internal state memory (set SWRST in the AESCTL0 register)
 - b. Set the CMEN to enable block cipher mode
 - c. Set CMx to CFB mode (0x3)
 - d. Set OPx to decryption mode (0x1)
2. Load key as described in [Section 23.2.2](#)
3. Load initialization vector IV into AESAXIN
 - a. Loading to AESAXIN does not start any decryption
 - b. The STATE must be reset (step 1a above) so that the IV is XORed with zeros when loaded
4. Configure the DMA_A channel for loading ciphertext:
 - a. Set the DMA channel trigger selection to AES0 trigger
 - b. Set the DMA channel source address to the location where ciphertext is stored (for example, SRAM)
 - c. Set the DMA channel destination address to AESAXIN
 - d. Set the DMA channel transfer size to $N*4$ words
 - e. Set the DMA channel mode to single transfer mode
5. Configure the DMA_B channel for reading plaintext:
 - a. Set the DMA channel trigger selection for AES1 trigger
 - b. Set the DMA channel source address to AESADOUT
 - c. Set the DMA channel destination address to the location where plaintext is to be stored (for example, SRAM)
 - d. Set the DMA channel transfer size to $N*4$ words
 - e. Set the DMA channel mode to single transfer mode

- f. In the AES event registers, unmask DMA1 in the IMASK register of DMA_TRIG1
6. Configure the DMA_C channel for loading ciphertext:
 - a. Set the DMA channel trigger selection for AES2 trigger
 - b. Set the DMA channel source address to the location where ciphertext is stored (for example, SRAM)
 - c. Set the DMA channel destination address to AESADIN
 - d. Set the DMA channel transfer size to $N \times 4$ words
 - e. Set the DMA channel mode to single transfer mode
7. Configure and enable the DMA interrupt for the DMA_B channel in the DMA controller
8. Start decryption by writing the block count N to BLKCNTx in the AESACTL1 register and setting the DINWR bit in the AESASTAT register
9. Wait for the DMA_B channel interrupt which indicates completion of the operation. The plaintext output will be stored in the location configured in step 5c.

23.2.7.5 Counter (CTR) Mode

The counter mode leverages a nonce (number used once) and a counting integer to generate a keystream by appending the counter to the nonce and encrypting the combined nonce || counter value with the cipher key.

The nonce must only be used once with a given key k . The counter value can start from any value and is incremented for each 128-bit block of data.

The keystream is derived by encrypting the nonce || counter value for each 128-bit data block with the cipher key k . The output ciphertext is then obtained by XORing the plaintext with the encrypted nonce || counter value for each data block. The CTR cipher is shown in [Figure 23-9](#).

Note

As CTR mode is a stream cipher, the AES block function is used in the forward (encryption) mode when performing CTR encryption or CTR decryption. When decrypting, the keystream is simply regenerated again to be XORed with the ciphertext, giving back the plaintext.

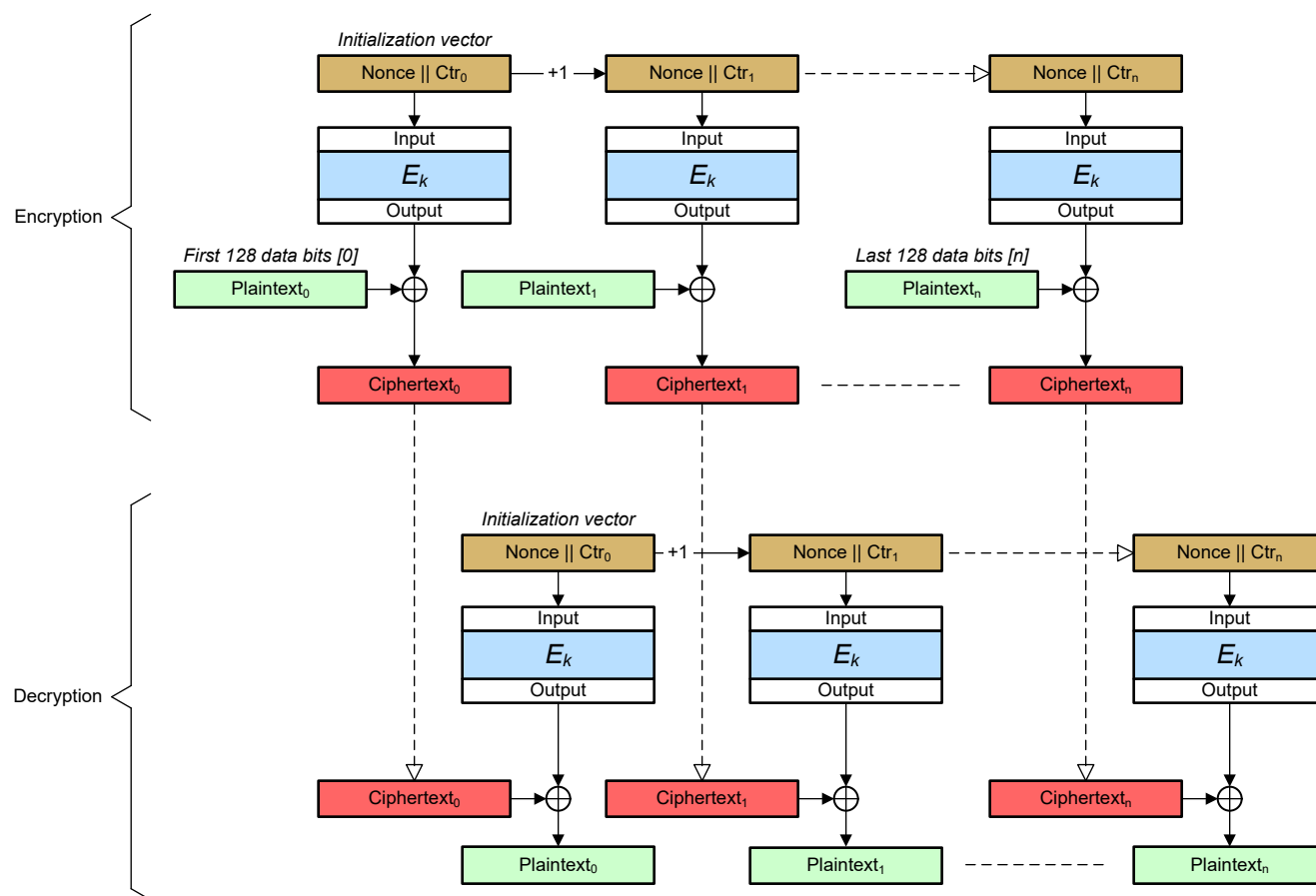


Figure 23-9. CTR Cipher

The AES accelerator can be used to accelerate CTR ciphers by accelerating the encryption of each nonce || counter value with the cipher key k , and accelerating the XOR operation of the plaintext (in the case of encryption) or ciphertext (in the case of decryption) with the keystream. DMA trigger generation is not supported when implementing a CTR cipher.

23.2.7.5.1 CTR Encryption

CTR-mode encryption of N blocks of plaintext into N blocks of ciphertext is achieved by following the steps below:

1. Configure the AESACTL0 register for single block encryption mode:
 - a. Clear CMEN to disable block cipher mode (if previously enabled)
 - b. Set OPx to encryption mode (0x0)
2. Load key as described in [Section 23.2.2](#)
3. Load first (nonce || counter) value into AESADIN as described in [Section 23.2.3](#)
4. Wait for the encryption process to complete (wait for the BUSY flag to clear)
5. The encrypted (nonce || counter) value is now in the STATE. The first ciphertext block can be obtained by XORing the first plaintext block with the keystream in the STATE.
 - a. Load the first plaintext block into AESAXIN (XORing the plaintext with the keystream, but not starting any encryption operation)
6. Read the first block of ciphertext from AESADOUT as described in [Section 23.2.4](#)
7. Repeat steps 3-6 for additional data blocks, incrementing the counter value each time.

23.2.7.5.2 CTR Decryption

CTR-mode decryption of N blocks of ciphertext into N blocks of plaintext is achieved by following the same steps given for encryption ([Section 23.2.7.5.1](#)), but substituting the ciphertext for the plaintext and the reverse. Note that only the forward cipher is used in CTR mode; the encryption and decryption operations are identical and the same software implementation can be used for both, with substitution of ciphertext for plaintext in the case of decryption.

23.2.8 AES Events

The AES module contains four [event publishers](#) and no [event subscribers](#). One event publisher (CPU_INT) manages AES interrupt requests (IRQs) to the CPU subsystem through a [static event route](#). The second, third, and fourth event publishers (DMA_TRIG0, DMA_TRIG1, and DMA_TRIG2) can be used to publish AES events to the DMA through [DMA event routes](#).

The AES events are summarized in [Table 23-7](#).

Table 23-7. AES Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU Interrupt Event	Publisher	AES	CPU Subsystem	Static route	CPU_INT registers	Fixed interrupt route from RTC to CPU
DMA Trigger Event 0	Publisher	AES	DMA	DMA route	DMA_TRIG 0 registers	DMA trigger 0 for block cipher modes
DMA Trigger Event 1	Publisher	AES	DMA	DMA route	DMA_TRIG 1 registers	DMA trigger 1 for block cipher modes
DMA Trigger Event 2	Publisher	AES	DMA	DMA route	DMA_TRIG 2 registers	DMA trigger 2 for block cipher modes

In general, the CPU interrupt event is used to communicate completion of an AES operation to the CPU, and the DMA triggers are used together to implement the block cipher modes (ECB, CBC, OFB, and CFB) using the DMA together with the AES accelerator.

When the CMEN bit is set in AESACTL0, the AES module will trigger DMA trigger event 0, DMA trigger event 1, and DMA trigger event 2 to execute different block cipher modes of operation together with the DMA.

For example, when using ECB encryption with AESCMEN=0x1, DMA trigger event 0 is triggered four times for DMA word access to read out AESADOUT, and AES trigger 1 is triggered 4 times to fill the next data into AESADIN. The AES module generates a trigger for each word, and so the DMA must be configured in repeated single channel mode. See the block cipher mode section for details on how the DMA must be configured to support the ECB, CBC, OFB, and CFB block cipher modes.

23.2.8.1 CPU Interrupt Event Publisher (CPU_EVENT)

The AES module provides four interrupt sources which can be configured to source a [CPU interrupt event](#). In order of decreasing interrupt priority, the CPU interrupt events from the AES are given in [Table 23-8](#).

Table 23-8. AES CPU Interrupt Event Conditions (CPU_EVENT)

Index (IIDX)	Name	Description
0	AESRDY	Indicates that the AES module has completed the selected operation and the results can be read from AESADOUT (if applicable).
1	DMA0	Not to be used for normal operation.
2	DMA1	Not to be used for normal operation.
3	DMA2	Not to be used for normal operation.

The CPU interrupt event configuration is managed with the event management registers. See [Section 7.2.5](#) for guidance on configuring these registers for CPU interrupts.

23.2.8.2 DMA Trigger Event Publisher (DMA_TRIG0)

The AES module provides four trigger sources which can be configured to source DMA trigger 0. In order of decreasing interrupt priority, the DMA0 trigger events from the AES are given in [Table 23-9](#). When the DMA0 channel is needed by the AES for block cipher operations, the DMA0 trigger should be unmasked in the IMASK register of DMA_TRIG0 and the DMA should be configured as needed to support the AES operation.

Table 23-9. AES DMA Trigger 0 Event Conditions (DMA_TRIG0)

Index (IIDX)	Name	Description
0	AESRDY	Not to be used for normal operation.
1	DMA0	DMA0 trigger indication
2	DMA1	Not to be used for normal operation.
3	DMA2	Not to be used for normal operation.

The DMA trigger 0 event configuration is managed with the DMA_TRIG0 event management registers. See [Section 7.2.5](#) for guidance on configuring the event registers for DMA triggers.

23.2.8.3 DMA Trigger Event Publisher (DMA_TRIG1)

The AES module provides four trigger sources which can be configured to source DMA trigger 1. In order of decreasing interrupt priority, the DMA1 trigger events from the AES are given in [Table 23-10](#). When the DMA1 channel is needed by the AES for block cipher operations, the DMA1 trigger should be unmasked in the IMASK register of DMA_TRIG1 and the DMA should be configured as needed to support the AES operation.

Table 23-10. AES DMA Trigger 1 Event Conditions (DMA_TRIG1)

Index (IIDX)	Name	Description
0	AESRDY	Not to be used for normal operation.
1	DMA0	Not to be used for normal operation.
2	DMA1	DMA1 trigger indication
3	DMA2	Not to be used for normal operation.

The DMA trigger 1 event configuration is managed with the DMA_TRIG1 event management registers. See [Section 7.2.5](#) for guidance on configuring the event registers for DMA triggers.

23.2.8.4 DMA Trigger Event Publisher (DMA_TRIG2)

The AES module provides four trigger sources which can be configured to source DMA trigger 2. In order of decreasing interrupt priority, the DMA2 trigger events from the AES are given in [Table 23-11](#). When the DMA2 channel is needed by the AES for block cipher operations, the DMA2 trigger should be unmasked in the IMASK register of DMA_TRIG2 and the DMA should be configured as needed to support the AES operation.

Table 23-11. AES DMA Trigger 2 Event Conditions (DMA_TRIG2)

Index (IIDX)	Name	Description
0	AESRDY	Not to be used for normal operation.
1	DMA0	Not to be used for normal operation.
2	DMA1	Not to be used for normal operation.
3	DMA2	DMA2 trigger indication

The DMA trigger 2 event configuration is managed with the DMA_TRIG2 event management registers. See [Section 7.2.5](#) for guidance on configuring the event registers for DMA triggers.

23.3 AES Registers

Table 23-12 lists the memory-mapped registers for the AES registers. All register offset addresses not listed in Table 23-12 should be considered as reserved locations and the register contents should not be modified.

Table 23-12. AES Registers

Offset	Acronym	Register Name	Group	Section
800h	PWREN	Power enable		Go
804h	RSTCTL	Reset Control		Go
814h	STAT	Status Register		Go
1018h	PDBGCTL	Peripheral Debug Control		Go
1020h	IIDX	Interrupt Index Register	CPU_INT	Go
1028h	IMASK	Interrupt mask	CPU_INT	Go
1030h	RIS	Raw interrupt status	CPU_INT	Go
1038h	MIS	Masked interrupt status	CPU_INT	Go
1040h	ISSET	Interrupt set	CPU_INT	Go
1048h	ICLR	Interrupt clear	CPU_INT	Go
1050h	IIDX	Interrupt Index Register	DMA_TRIG0	Go
1058h	IMASK	Interrupt mask	DMA_TRIG0	Go
1060h	RIS	Raw interrupt status	DMA_TRIG0	Go
1068h	MIS	Masked interrupt status	DMA_TRIG0	Go
1070h	ISSET	Interrupt set	DMA_TRIG0	Go
1078h	ICLR	Interrupt clear	DMA_TRIG0	Go
1080h	IIDX	Interrupt Index Register	DMA_TRIG1	Go
1088h	IMASK	Interrupt mask	DMA_TRIG1	Go
1090h	RIS	Raw interrupt status	DMA_TRIG1	Go
1098h	MIS	Masked interrupt status	DMA_TRIG1	Go
10A0h	ISSET	Interrupt set	DMA_TRIG1	Go
10A8h	ICLR	Interrupt clear	DMA_TRIG1	Go
10B0h	IIDX	Interrupt Index Register	DMA_TRIG2	Go
10B8h	IMASK	Interrupt mask	DMA_TRIG2	Go
10C0h	RIS	Raw interrupt status	DMA_TRIG2	Go
10C8h	MIS	Masked interrupt status	DMA_TRIG2	Go
10D0h	ISSET	Interrupt set	DMA_TRIG2	Go
10D8h	ICLR	Interrupt clear	DMA_TRIG2	Go
10E0h	EVT_MODE	Event Mode		Go
1100h	AESACTL0	AES accelerator control register 0		Go
1104h	AESACTL1	AES accelerator control register 1		Go
1108h	AESASTAT	aes accelerator status register		Go
110Ch	AESAKEY	aes accelerator key register		Go
1110h	AESADIN	aes accelerator data in register		Go
1114h	AESADOUT	aes accelerator data out register		Go
1118h	AESAXDIN	aes accelerator xored data in register		Go
111Ch	AESAXIN	aes accelerator xored data in register (no trigger)		Go

Complex bit access types are encoded to fit into small table cells. Table 23-13 shows the codes that are used for access types in this section.

Table 23-13. AES Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value

23.3.1 PWREN (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 23-10](#) and described in [Table 23-14](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 23-10. PWREN

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-0h							R/WK-0h

Table 23-14. PWREN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	ENABLE	R/WK	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

23.3.2 RSTCTL (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 23-11](#) and described in [Table 23-15](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 23-11. RSTCTL

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCL R	RESETASSERT
W-0h						WK-0h	WK-0h

Table 23-15. RSTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	W	0h	
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

23.3.3 STAT (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 23-12](#) and described in [Table 23-16](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 23-12. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED							
R-							

Table 23-16. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0h	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

23.3.4 PDBGCTL (Offset = 1018h) [Reset = 00000003h]

PDBGCTL is shown in [Figure 23-13](#) and described in [Table 23-17](#).

Return to the [Summary Table](#).

This register can be used by the software developer to control the behavior of the peripheral relative to the 'Core Halted' input

Figure 23-13. PDBGCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED						SOFT	FREE
R/W-						R/W-1h	R/W-1h

Table 23-17. PDBGCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R/W	0h	
1	SOFT	R/W	1h	Soft halt boundary control. This function is only available, if FREE is set to 'STOP' 0h = The peripheral will halt immediately, even if the resultant state will result in corruption if the system is restarted 1h = The peripheral blocks the debug freeze until it has reached a boundary where it can resume without corruption
0	FREE	R/W	1h	Free run control 0h = The peripheral freezes functionality while the Core Halted input is asserted and resumes when it is deasserted. 1h = The peripheral ignores the state of the Core Halted input

23.3.5 IIDX (Offset = 1020h) [Reset = 00000000h]

IIDX is shown in [Figure 23-14](#) and described in [Table 23-18](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 23-14. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 23-18. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 00h = No interrupt pending 1h = AES ready interrupt, set when the selected AES operation was completed and the result can be read from AESADOUT

23.3.6 IMASK (Offset = 1028h) [Reset = 00000000h]

IMASK is shown in [Figure 23-15](#) and described in [Table 23-19](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 23-15. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							AESRDY
R/W-0h							R/W-0h

Table 23-19. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R/W	0h	
0	AESRDY	R/W	0h	AES ready interrupt, set when the selected AES operation was completed and the result can be read from AESADOUT. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

23.3.7 RIS (Offset = 1030h) [Reset = 00000000h]

RIS is shown in [Figure 23-16](#) and described in [Table 23-20](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 23-16. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							AESRDY
R-0h							R-0h

Table 23-20. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	AESRDY	R	0h	AES ready interrupt, set when the selected AES operation was completed and the result can be read from AESADOUT. 0h = Interrupt did not occur 1h = Interrupt occurred

23.3.8 MIS (Offset = 1038h) [Reset = 00000000h]

MIS is shown in [Figure 23-17](#) and described in [Table 23-21](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 23-17. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							AESRDY
R-0h							R-0h

Table 23-21. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	AESRDY	R	0h	AES ready interrupt, set when the selected AES operation was completed and the result can be read from AESADOUT. 0h = Interrupt did not occur 1h = Interrupt occurred

23.3.9 ISET (Offset = 1040h) [Reset = 00000000h]

ISET is shown in [Figure 23-18](#) and described in [Table 23-22](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 23-18. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED							AESRDY
W-0h							W-0h

Table 23-22. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	W	0h	
0	AESRDY	W	0h	AES ready interrupt, set when the selected AES operation was completed and the result can be read from AESADOUT. 0h = Writing 0 has no effect 1h = Set Interrupt

23.3.10 ICLR (Offset = 1048h) [Reset = 00000000h]

ICLR is shown in [Figure 23-19](#) and described in [Table 23-23](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 23-19. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED							AESRDY
W-0h							W-0h

Table 23-23. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	W	0h	
0	AESRDY	W	0h	AES ready interrupt, set when the selected AES operation was completed and the result can be read from AESADOUT. 0h = Writing 0 has no effect 1h = Clear Interrupt

23.3.11 IIDX (Offset = 1050h) [Reset = 00000000h]

IIDX is shown in [Figure 23-20](#) and described in [Table 23-24](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 23-20. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 23-24. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 00h = No interrupt pending 2h = AES trigger 0 DMA

23.3.12 IMASK (Offset = 1058h) [Reset = 00000000h]

IMASK is shown in [Figure 23-21](#) and described in [Table 23-25](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 23-21. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED						DMA0	RESERVED
R/W-0h						R/W-0h	R/W-0h

Table 23-25. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R/W	0h	
1	DMA0	R/W	0h	DMA0 event mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	RESERVED	R/W	0h	

23.3.13 RIS (Offset = 1060h) [Reset = 00000000h]

RIS is shown in [Figure 23-22](#) and described in [Table 23-26](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 23-22. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED						DMA0	RESERVED
R-0h						R-0h	R-0h

Table 23-26. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1	DMA0	R	0h	DMA0 event 0h = Interrupt did not occur 1h = Interrupt occurred
0	RESERVED	R	0h	

23.3.14 MIS (Offset = 1068h) [Reset = 00000000h]

MIS is shown in [Figure 23-23](#) and described in [Table 23-27](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 23-23. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED						DMA0	RESERVED
R-0h						R-0h	R-0h

Table 23-27. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1	DMA0	R	0h	DMA0 event 0h = Interrupt did not occur 1h = Interrupt occurred
0	RESERVED	R	0h	

23.3.15 ISET (Offset = 1070h) [Reset = 00000000h]

ISET is shown in [Figure 23-24](#) and described in [Table 23-28](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 23-24. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED						DMA0	RESERVED
W-0h						W-0h	W-0h

Table 23-28. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	W	0h	
1	DMA0	W	0h	DMA0 0h = Writing 0 has no effect 1h = Set Interrupt
0	RESERVED	W	0h	

23.3.16 ICLR (Offset = 1078h) [Reset = 00000000h]

ICLR is shown in [Figure 23-25](#) and described in [Table 23-29](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 23-25. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED						DMA0	RESERVED
W-0h						W-0h	W-0h

Table 23-29. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	W	0h	
1	DMA0	W	0h	DMA0 event 0h = Writing 0 has no effect 1h = Clear Interrupt
0	RESERVED	W	0h	

23.3.17 IIDX (Offset = 1080h) [Reset = 00000000h]

IIDX is shown in [Figure 23-26](#) and described in [Table 23-30](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 23-26. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 23-30. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 00h = No interrupt pending 3h = AES trigger 1 DMA

23.3.18 IMASK (Offset = 1088h) [Reset = 00000000h]

IMASK is shown in [Figure 23-27](#) and described in [Table 23-31](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 23-27. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED					DMA1	RESERVED	
R/W-0h					R/W-0h	R/W-0h	

Table 23-31. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R/W	0h	
2	DMA1	R/W	0h	DMA1 event mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1-0	RESERVED	R/W	0h	

23.3.19 RIS (Offset = 1090h) [Reset = 00000000h]

RIS is shown in [Figure 23-28](#) and described in [Table 23-32](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 23-28. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED					DMA1	RESERVED	
R-0h					R-0h	R-0h	

Table 23-32. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R	0h	
2	DMA1	R	0h	DMA1 event 0h = Interrupt did not occur 1h = Interrupt occurred
1-0	RESERVED	R	0h	

23.3.20 MIS (Offset = 1098h) [Reset = 00000000h]

MIS is shown in [Figure 23-29](#) and described in [Table 23-33](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 23-29. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED					DMA1	RESERVED	
R-0h					R-0h	R-0h	

Table 23-33. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R	0h	
2	DMA1	R	0h	DMA1 event 0h = Interrupt did not occur 1h = Interrupt occurred
1-0	RESERVED	R	0h	

23.3.21 ISET (Offset = 10A0h) [Reset = 00000000h]

ISET is shown in [Figure 23-30](#) and described in [Table 23-34](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 23-30. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED					DMA1	RESERVED	
W-0h					W-0h	W-0h	

Table 23-34. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	W	0h	
2	DMA1	W	0h	DMA1 event 0h = Writing 0 has no effect 1h = Set Interrupt
1-0	RESERVED	W	0h	

23.3.22 ICLR (Offset = 10A8h) [Reset = 00000000h]

ICLR is shown in [Figure 23-31](#) and described in [Table 23-35](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 23-31. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED					DMA1	RESERVED	
W-0h					W-0h	W-0h	

Table 23-35. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	W	0h	
2	DMA1	W	0h	DMA1 event 0h = Writing 0 has no effect 1h = Clear Interrupt
1-0	RESERVED	W	0h	

23.3.23 IIDX (Offset = 10B0h) [Reset = 00000000h]

IIDX is shown in [Figure 23-32](#) and described in [Table 23-36](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 23-32. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 23-36. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 00h = No interrupt pending 4h = AES trigger 2 DMA

23.3.24 IMASK (Offset = 10B8h) [Reset = 00000000h]

IMASK is shown in [Figure 23-33](#) and described in [Table 23-37](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 23-33. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED				DMA2	RESERVED		
R/W-0h				R/W-0h	R/W-0h		

Table 23-37. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3	DMA2	R/W	0h	DMA2 event mask. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2-0	RESERVED	R/W	0h	

23.3.25 RIS (Offset = 10C0h) [Reset = 00000000h]

RIS is shown in [Figure 23-34](#) and described in [Table 23-38](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 23-34. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				DMA2	RESERVED		
R-0h				R-0h	R-0h		

Table 23-38. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	DMA2	R	0h	DMA2 event 0h = Interrupt did not occur 1h = Interrupt occurred
2-0	RESERVED	R	0h	

23.3.26 MIS (Offset = 10C8h) [Reset = 00000000h]

MIS is shown in [Figure 23-35](#) and described in [Table 23-39](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 23-35. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				DMA2	RESERVED		
R-0h				R-0h	R-0h		

Table 23-39. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	DMA2	R	0h	DMA2 event 0h = Interrupt did not occur 1h = Interrupt occurred
2-0	RESERVED	R	0h	

23.3.27 ISET (Offset = 10D0h) [Reset = 00000000h]

ISET is shown in [Figure 23-36](#) and described in [Table 23-40](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 23-36. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED				DMA2	RESERVED		
W-0h				W-0h	W-0h		

Table 23-40. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	W	0h	
3	DMA2	W	0h	DMA2 event 0h = Writing 0 has no effect 1h = Set Interrupt
2-0	RESERVED	W	0h	

23.3.28 ICLR (Offset = 10D8h) [Reset = 00000000h]

ICLR is shown in [Figure 23-37](#) and described in [Table 23-41](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 23-37. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED				DMA2	RESERVED		
W-0h				W-0h	W-0h		

Table 23-41. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	W	0h	
3	DMA2	W	0h	DMA2 event 0h = Writing 0 has no effect 1h = Clear Interrupt
2-0	RESERVED	W	0h	

23.3.29 EVT_MODE (Offset = 10E0h) [Reset = 000000A9h]

EVT_MODE is shown in [Figure 23-38](#) and described in [Table 23-42](#).

Return to the [Summary Table](#).

Event mode register. It is used to select whether each line is disabled, in software mode (software clears the RIS) or in hardware mode (hardware clears the RIS)

Figure 23-38. EVT_MODE

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
EVT3_CFG		EVT2_CFG		EVT1_CFG		INT0_CFG	
R-2h		R-2h		R-2h		R-1h	

Table 23-42. EVT_MODE Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R/W	0h	
7-6	EVT3_CFG	R	2h	Event line mode select for event corresponding to none.DMA_TRIG2 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
5-4	EVT2_CFG	R	2h	Event line mode select for event corresponding to none.DMA_TRIG1 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
3-2	EVT1_CFG	R	2h	Event line mode select for event corresponding to none.DMA_TRIG0 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
1-0	INT0_CFG	R	1h	Event line mode select for event corresponding to none.CPU_INT 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.

23.3.30 AESACTL0 (Offset = 1100h) [Reset = 00000000h]

AESACTL0 is shown in [Figure 23-39](#) and described in [Table 23-43](#).

Return to the [Summary Table](#).

AES accelerator control register 0

Figure 23-39. AESACTL0

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
CMEN	RESERVED			ERRFG	RESERVED		
R/W-0h	R/W-0h			R/W-0h	R/W-0h		
7	6	5	4	3	2	1	0
SWRST	CMx		RESERVED	KLx		OPx	
R/W-0h	R/W-0h		R/W-0h	R/W-0h		R/W-0h	

Table 23-43. AESACTL0 Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15	CMEN	R/W	0h	AESCMEN enables the support of the cipher modes ECB, CBC, OFB and CFB together with the DMA. Writes are ignored when AESCMEN = 1 and AESBLKCNTx > 0. 0 = No DMA triggers are generated. 1 = DMA cipher mode support operation is enabled and the corresponding DMA triggers are generated. 0h = No DMA triggers are generated. 1h = DMA cipher mode support operation is enabled and the corresponding DMA triggers are generated.
14-12	RESERVED	R/W	0h	
11	ERRFG	R/W	0h	AES error flag. AESAKEY or AESADIN were written while an AES operation was in progress. The bit must be cleared by software. 0b = No error 1b = Error occurred 0h (R/W) = No error 1h (R/W) = Error occurred
10-8	RESERVED	R/W	0h	
7	SWRST	R/W	0h	AES software reset. Immediately resets the complete AES accelerator module even when busy except for the AESRDYIE, the AESKLx and the AESOPx bits. It also clears the (internal) state memory. The AESSWRST bit is automatically reset and is always read as zero. 0b = No reset 1b = Reset AES accelerator module 0h = No reset. 1h = Reset AES accelerator module.
6-5	CMx	R/W	0h	AES cipher mode select. These bits are ignored for AESCMEN = 0. Writes are ignored when AESCMEN = 1 and AESBLKCNTx > 0. 00b = ECB 01b = CBC 10b = OFB 11b = CFB 0h = ECB 1h = CBC 2h = OFB 3h = CFB
4	RESERVED	R/W	0h	

Table 23-43. AESACTL0 Field Descriptions (continued)

Bit	Field	Type	Reset	Description
3-2	KLx	R/W	0h	AES key length. These bits define which of the 1 AES standards is performed. The AESKLx bits are not reset by AESSWRST = 1. Writes are ignored when AESCMEN = 1 and AESBLKCNTx > 0. 0h = The key size is 128 bit. 2h = The key size is 256 bit.
1-0	OPx	R/W	0h	AES operation. The AESOPx bits are not reset by AESSWRST = 1. Writes are ignored when AESCMEN = 1 and AESBLKCNTx > 0. 00b = Encryption. 01b = Decryption. The provided key is the same key used for encryption. 10b = Generate first round key required for decryption. 11b = Decryption. The provided key is the first round key required for decryption. 0h (R/W) = Encryption 1h (R/W) = Decryption. The provided key is the same key used for encryption. 2h (R/W) = Generate first round key required for decryption. 3h (R/W) = Decryption. The provided key is the first round key required for decryption.

23.3.31 AESACTL1 (Offset = 1104h) [Reset = 00000000h]

AESACTL1 is shown in [Figure 23-40](#) and described in [Table 23-44](#).

Return to the [Summary Table](#).

AES accelerator control register 1

Figure 23-40. AESACTL1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								BLKCNTx							
R/W-0h																								R/W-0h							

Table 23-44. AESACTL1 Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R/W	0h	
7-0	BLKCNTx	R/W	0h	Cipher Block Counter. Number of blocks to be encrypted or decrypted with block cipher modes enabled (AESCME = 1). Ignored if AESCMEN = 0. The block counter decrements with each performed encryption or decryption. Writes are ignored when AESCMEN = 1 and AESBLKCNTx > 0. 00h = Smallest value FFh = Highest possible value

23.3.32 AESASTAT (Offset = 1108h) [Reset = 00000000h]

AESASTAT is shown in [Figure 23-41](#) and described in [Table 23-45](#).

Return to the [Summary Table](#).

AES accelerator Status Register.

Figure 23-41. AESASTAT

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
DOUTCNTx				DINCNTx			
R-0h				R-0h			
7	6	5	4	3	2	1	0
KEYCNTx				DOUTRD	DINWR	KEYWR	BUSY
R-0h				R-0h	R/W-0h	R/W-0h	R-0h

Table 23-45. AESASTAT Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R	0h	
15-12	DOUTCNTx	R	0h	Bytes read from AESADOUT. Reset when AESDOUTRD is reset. If AESDOUTCNTx = 0 and AESDOUTRD = 0, no bytes were read. If AESDOUTCNTx = 0 and AESDOUTRD = 1, all bytes were read. 0h = Smallest possible value Fh = Highest possible value
11-8	DINCNTx	R	0h	Bytes written to AESADIN, AESAXDIN or AESAXIN. Reset when AESDINWR is reset. If AESDINCNTx = 0 and AESDINWR = 0, no bytes were written. If AESDINCNTx = 0 and AESDINWR = 1, all bytes were written. 0h = Smallest possible value Fh = Highest possible value
7-4	KEYCNTx	R	0h	Bytes written to AESAKEY when AESKLx = 00, half-words written to AESAKEY if AESKLx = b10. Reset when AESKEYWR is reset. If AESKEYCNTx = 0 and AESKEYWR = 0, no bytes were written. If AESKEYCNTx = 0 and AESKEYWR = 1, all bytes were written. 0h = Smallest possible value Fh = Highest possible value
3	DOUTRD	R	0h	All 16 bytes read from AESADOUT. AESDOUTRD is reset by PUC, AESSWRST, an error condition, changing AESOPx, changing AESKLx, when the AES accelerator is busy, and when the output data is read again. 0 = Not all bytes read 1 = All bytes read 0h = Not all bytes read 1h = All bytes read
2	DINWR	R/W	0h	All 16 bytes written to AESADIN, AESAXDIN or AESAXIN. Changing its state by software also resets the AESDINCNTx bits. AESDINWR is reset by PUC, AESSWRST, an error condition, changing AESOPx, changing AESKLx, the start to (over)write the data, and when the AES accelerator is busy. Because it is reset when AESOPx or AESKLx is changed it can be set by software again to indicate that the current data is still valid. 0 = Not all bytes written 1 = All bytes written 0h = Not all bytes written 1h = All bytes written

Table 23-45. AESASTAT Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1	KEYWR	R/W	0h	All bytes written to AESAKEY. This bit can be modified by software but it must not be reset by software (1→0) if AESCMEN=1. Changing its state by software also resets the AESKEYCNTx bits. AESKEYWR is reset by PUC, AESSWRST, an error condition, changing AESOPx, changing AESKLx, and the start to (over)write a new key. Because it is reset when AESOPx is changed it can be set by software again to indicate that the loaded key is still valid. 0h = Not all bytes written 1h = All bytes written
0	BUSY	R	0h	AES accelerator module busy; encryption, decryption, or key generation in progress. 0 = Not busy 1 = Busy 0h = Not busy 1h = Busy

23.3.33 AESAKEY (Offset = 110Ch) [Reset = 00000000h]

AESAKEY is shown in [Figure 23-42](#) and described in [Table 23-46](#).

Return to the [Summary Table](#).

aes accelerator key register

Figure 23-42. AESAKEY

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEY3x								KEY2x								KEY1x								KEY0x							
W-0h								W-0h								W-0h								W-0h							

Table 23-46. AESAKEY Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY3x	W	0h	AES key byte n+3 when AESAKEY is written as word. Do not use these bits for byte access. Do not mix word and byte access. Always reads as zero. The key is reset by PUC or by AESSWRST = 1. 00h = Smallest possible value FFh = Highest possible value
23-16	KEY2x	W	0h	AES key byte n+2 when AESAKEY is written as word. Do not use these bits for byte access. Do not mix word and byte access. Always reads as zero. The key is reset by PUC or by AESSWRST = 1. 00h = Smallest possible value FFh = Highest possible value
15-8	KEY1x	W	0h	AES key byte n+1 when AESAKEY is written as word. Do not use these bits for byte access. Do not mix word and byte access. Always reads as zero. The key is reset by PUC or by AESSWRST = 1. 00h = Smallest possible value FFh = Highest possible value
7-0	KEY0x	W	0h	AES key byte n when AESAKEY is written as word. AES next key byte when AESAKEY is written as byte. Do not mix word and byte access. Always reads as zero. The key is reset by PUC or by AESSWRST = 1. 00h = Smallest possible value FFh = Highest possible value

23.3.34 AESADIN (Offset = 1110h) [Reset = 00000000h]

AESADIN is shown in [Figure 23-43](#) and described in [Table 23-47](#).

Return to the [Summary Table](#).

aes accelerator data in register

Figure 23-43. AESADIN

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIN3x								DIN2x								DIN1x								DIN0x							
W-0h								W-0h								W-0h								W-0h							

Table 23-47. AESADIN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	DIN3x	W	0h	AES data in byte n+3 when AESADIN is written as word. Do not use these bits for byte access. Do not mix word and byte access. Always reads as zero. 00h = Smallest possible value FFh = Highest possible value
23-16	DIN2x	W	0h	AES data in byte n+2 when AESADIN is written as word. Do not use these bits for byte access. Do not mix word and byte access. Always reads as zero. 00h = Smallest possible value FFh = Highest possible value
15-8	DIN1x	W	0h	AES data in byte n+1 when AESADIN is written as word. Do not use these bits for byte access. Do not mix word and byte access. Always reads as zero. 00h = Smallest possible value FFh = Highest possible value
7-0	DIN0x	W	0h	AES data in byte n when AESADIN is written as word. AES next data in byte when AESADIN is written as byte. Do not mix word and byte access. Always reads as zero. 00h = Smallest possible value FFh = Highest possible value

23.3.35 AESADOUT (Offset = 1114h) [Reset = 00000000h]

AESADOUT is shown in [Figure 23-44](#) and described in [Table 23-48](#).

Return to the [Summary Table](#).

aes accelerator data out register

Figure 23-44. AESADOUT

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DOUT3x								DOUT2x								DOUT1x								DOUT0x							
R-0h								R-0h								R-0h								R-0h							

Table 23-48. AESADOUT Field Descriptions

Bit	Field	Type	Reset	Description
31-24	DOUT3x	R	0h	AES data out byte n+3 when AESADOUT is read as word. Do not use these bits for byte access. Do not mix word and byte access. 00h = Smallest possible value FFh = Highest possible value
23-16	DOUT2x	R	0h	AES data out byte n+2 when AESADOUT is read as word. Do not use these bits for byte access. Do not mix word and byte access. 00h = Smallest possible value FFh = Highest possible value
15-8	DOUT1x	R	0h	AES data out byte n+1 when AESADOUT is read as word. Do not use these bits for byte access. Do not mix word and byte access. 00h = Smallest possible value FFh = Highest possible value
7-0	DOUT0x	R	0h	AES data out byte n when AESADOUT is read as word. AES next data out byte when AESADOUT is read as byte. Do not mix word and byte access. 00h = Smallest possible value FFh = Highest possible value

23.3.36 AESAXDIN (Offset = 1118h) [Reset = 00000000h]

AESAXDIN is shown in [Figure 23-45](#) and described in [Table 23-49](#).

Return to the [Summary Table](#).

aes accelerator xored data in register

Figure 23-45. AESAXDIN

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
XDIN3x								XDIN2x								XDIN1x								XDIN0x							
W-0h								W-0h								W-0h								W-0h							

Table 23-49. AESAXDIN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	XDIN3x	W	0h	AES data in byte n+3 when AESAXDIN is written as word. Do not use these bits for byte access. Do not mix word and byte access. Always reads as zero. 00h = Smallest possible value FFh = Highest possible value
23-16	XDIN2x	W	0h	AES data in byte n+2 when AESAXDIN is written as word. Do not use these bits for byte access. Do not mix word and byte access. Always reads as zero. 00h = Smallest possible value FFh = Highest possible value
15-8	XDIN1x	W	0h	AES data in byte n+1 when AESAXDIN is written as word. Do not use these bits for byte access. Do not mix word and byte access. Always reads as zero. 00h = Smallest possible value FFh = Highest possible value
7-0	XDIN0x	W	0h	AES data in byte n when AESAXDIN is written as word. AES next data in byte when AESAXDIN is written as byte. Do not mix word and byte access. Always reads as zero. 00h = Smallest possible value FFh = Highest possible value

23.3.37 AESAXIN (Offset = 111Ch) [Reset = 00000000h]

AESAXIN is shown in [Figure 23-46](#) and described in [Table 23-50](#).

Return to the [Summary Table](#).

aes accelerator xored data in register (no trigger)

Figure 23-46. AESAXIN

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
XIN3x								XIN2x								XIN1x								XIN0x							
W-0h								W-0h								W-0h								W-0h							

Table 23-50. AESAXIN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	XIN3x	W	0h	AES data in byte n+3 when AESAXIN is written as word. Do not use these bits for byte access. Do not mix word and byte access. Always reads as zero. 00h = Smallest possible value FFh = Highest possible value
23-16	XIN2x	W	0h	AES data in byte n+2 when AESAXIN is written as word. Do not use these bits for byte access. Do not mix word and byte access. Always reads as zero. 00h = Smallest possible value FFh = Highest possible value
15-8	XIN1x	W	0h	AES data in byte n+1 when AESAXIN is written as word. Do not use these bits for byte access. Do not mix word and byte access. Always reads as zero. 00h = Smallest possible value FFh = Highest possible value
7-0	XIN0x	W	0h	AES data in byte n when AESAXIN is written as word. AES next data in byte when AESAXIN is written as byte. Do not mix word and byte access. Always reads as zero. 00h = Smallest possible value FFh = Highest possible value



The true random number generator (TRNG) block is an entropy source which can be used to generate random bit sequences.

24.1 TRNG Overview.....	1332
24.2 TRNG Operation.....	1332
24.3 TRNG Registers.....	1339

24.1 TRNG Overview

The true random number generator (TRNG) module securely generates random 32-bit numbers through an internal analog entropy source and digital conditioning/decimation blocks. The TRNG can be used as an entropy source to derive true random seed values when implementing a deterministic random bit generator (DRBG). The TRNG is designed for use in creating a deterministic random bit generator (DRBG) system which can pass the NIST SP800-22 statistical test suite for cryptographic random number generators.

Key features of the TRNG module include:

- 32-bit true random number output
- Functional across the entire device supply range (voltage and temperature)
- $\Delta\Sigma$ -modulator based analog entropy source
- Conditioning block
- Configurable decimation block
- Integrated startup and continuous health tests, compliant to NIST SP800-22
- Dedicated internal LDO regulator to defend against power manipulation attacks

24.2 TRNG Operation

The TRNG contains two main components: the analog block (containing the LDO regulator and entropy source), and the digital block (containing the state machine, startup logic, conditioning, decimation, health tests, and register interface). Figure 24-1 shows the TRNG blocks.

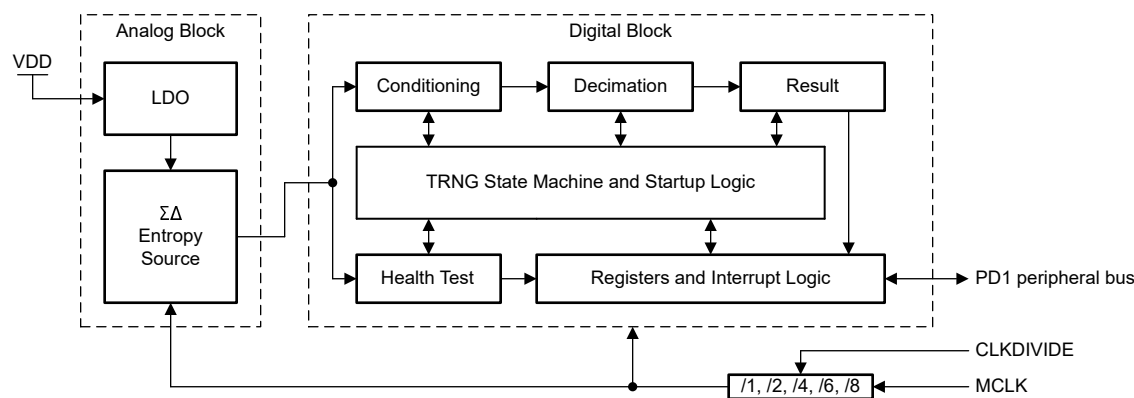


Figure 24-1. TRNG Block Diagram

24.2.1 TRNG Generation Data Path

The random number generation data path begins in the analog block. The analog block contains a dedicated low drop-out regulator which supplies power to the entropy source, improving resistance to power manipulation attacks on the MCU supply. The sigma-delta entropy source itself derives entropy through delta-sigma modulation of Johnson-Nyquist noise.

During operation, the entropy source output is digitized and the digital output is sent to the digital conditioning block to generate a random bit stream. The conditioning block implements a stream cipher scheme. Following the conditioning block, a decimation block is provided to boost entropy by accumulating the entropy of a configurable number of samples. The output of the decimation block is finally captured in a result word holding register to be read by application software.

Note

The purpose of the decimation block is to increase the overall entropy of the TRNG by merging entropy from multiple samples. The decimation block will accumulate bits from the conditioning block, and decimate down to 1 sample every n samples, where n is the decimation value. Decimation is implemented as a bit-wise exclusive-or function of n conditioned bits. The decimation integer n can be set between 1 and 8.

24.2.2 Clock Configuration and Output Rate

The TRNG functional clock is derived from [MCLK](#). The TRNG requires a functional clock which is within a specified frequency range for proper operation. Review the device-specific data sheet for the allowed TRNG frequency range on a given device. A clock divider is included in the TRNG to derive the frequency for the TRNG to use. The clock divider is specified through the **RATIO** field in the **CLKDIVIDE** register in the TRNG. This field must be set after the TRNG is enabled (through the **PWREN** register) but before the [TRNG state](#) is moved from the **OFF** state. Running the TRNG at a frequency outside the range specified in the device data sheet can lead to unexpected behavior.

The output rate of the TRNG module is dependent upon the TRNG functional clock frequency and the selected decimation rate. A minimum of 32 cycles of the TRNG functional clock are required to capture 32 random bits when the decimation rate is set to **DECIM_RATE=0x0** (decimate-by-1, or no decimation). When the decimation rate is set to **DECIM_RATE=0x3** (decimate-by-4), 128 cycles are required to capture 32 random bits. At a 10MHz clock rate with the decimation rate set to decimate-by-4, the time required to capture 32 random data bits is 12.8µs. [Equation 27](#) gives the formula for calculating the time required to generate 32 random bits based on the TRNG functional clock frequency and the selected decimation rate.

$$t_{\text{GENERATE}} = (32 * (\text{DECIM_RATE} + 1)) / f_{\text{TRNG}} \quad (27)$$

Note

A decimation rate of decimate-by-4 (**DECIM_RATE=0x3**) or greater is required to obtain the minimum entropy which is required to pass the NIST SP800-22 statistical test suite. TI recommends using decimate-by-4 or greater when using the TRNG in cryptographic applications.

Note

When the **DECIM_RATE** field is changed, a **NORM_FUNC** command must be re-sent to the TRNG for the new rate to take effect (by writing 0x3 to the **CMD** field in the **CTL** register).

When 32 bits of random data are captured and available in the **DATA_CAPTURE** register, the TRNG asserts the **IRQ_CAPTURED_RDY** interrupt to indicate to the processor that data is ready. Once the data is read by the processor, capture of the next 32 bits of random data begins.

24.2.3 Behavior in Low Power Modes

The TRNG is available for use in [RUN](#) and [SLEEP](#) modes. It is not available in any other operating mode, and all TRNG configuration settings and data are lost when the device transitions to any operating mode below **SLEEP**. When using low power operating modes below **SLEEP**, the TRNG must be re-configured for use as needed when waking the device from **STOP**, **STANDBY**, or **SHUTDOWN** mode.

24.2.4 Health Tests

Three health test mechanisms are provided: a digital block power-on self-test, analog block power-on self-test, and a runtime self-test. The digital and analog power-on self-tests are executed by sending the corresponding **CMD** to the TRNG state machine when the TRNG is in the **NORM_FUNC** state. The runtime self-test is always running when the TRNG is in the **NORM_FUNC** state.

24.2.4.1 Digital Block Startup Self-Test

The digital startup health test is run by application software when powering up the TRNG module. This built-in self-test verifies that the digital block is functioning properly by running predefined sequences of digital samples through the complete digital block while checking for expected outputs. The test sequence includes eight tests. Each test requires 1024 TRNG clock cycles to complete, as 1024 samples are input to the digital block for each test.

The results of the digital startup self-test are reported in the **DIG_TEST** field of the **TEST_RESULTS** register. Each test reports its status in an individual result bit. As each test passes, the corresponding bit in **DIG_TEST** is set by hardware. Upon completion of the digital start up self test, a pass condition is indicated if all eight bits

in the DIG_TEST field of the TEST_RESULTS register are set. If any bit in the DIG_TEST field is not set, this indicates a failing test. The TRNG is not usable if any of the digital startup health tests fail.

Note

The digital self-test changes the decimation rate when executing. This is a part of the self-test feature.

Note

The digital self-test injects deterministic values into the data path as a part of the test sequence. Following completion of the digital block startup self-test, the first data read from the DATA_CAPTURE register in normal (NORM_FUNC) operating mode is a deterministic value from the digital self-test, and not a true random number. **Always discard the first DATA_CAPTURE value after running the digital startup self-test.**

24.2.4.2 Analog Block Startup Self-Test

The analog startup health test is run by application software when powering up the TRNG module. This test verifies that the analog block is functioning properly by capturing 4,096 consecutive analog samples and verifying that the samples pass a health test.

If the health test fails, the health fail (IRQ_HEALTH_FAIL) and command done (IRQ_CMD_DONE) interrupts are both asserted, and the TRNG state machine enters the ERROR state. The result of the analog start up self-test is also reported in the ANA_TEST bit in the TEST_RESULTS register. If the test passed, the ANA_TEST bit is set. If the test failed, the ANA_TEST bit is left cleared.

If the test does fail, this indicates that a severe loss of entropy in the analog block was detected during the test. This means that the entropy source can not be used in its current state. The probability of this test returning a false positive (probability that a failure was asserted but the analog block is operating correctly) is statistically very low, but not zero. This means that there is a small probability that a failure could be indicated in certain cases even if the TRNG analog block is operating correctly. To ensure that a test failure is legitimate and that there is not a loss of entropy, the analog startup health test can be repeated to validate the failure. In the event of a failure, the following procedure is recommended:

1. Clear the IRQ_HEALTH_FAIL interrupt
2. Power off the TRNG
3. Run the analog startup health test again
4. Follow the appropriate action below based on the test result:
 - a. If the test passed, the TRNG can be used
 - b. If the test failed a second time, go to step 1 and attempt to run the test a third time
 - c. If the test failed a third time, there is catastrophic entropy loss and the TRNG should not be used

24.2.4.3 Runtime Health Test

During normal operation of the TRNG, raw data from the entropy source is sent through the health test logic to ensure that there is sufficient entropy present on an ongoing basis. The tests are designed to statistically check for a minimum of 0.3 bits of entropy per sample. The TRNG implements both repetition count and adaptive proportion continuous self-tests.

24.2.4.3.1 Repetition Count Test

The repetition count test quickly detects failures that cause the entropy source to remain stuck on a single output value for an extended period of time. The repetition count test fails if the entropy source outputs the same bit value for 135 consecutive samples.

24.2.4.3.2 Adaptive Proportion Test

The adaptive proportion test detects failures that cause a disproportionate number of samples to be the same bit value and/or bit pattern over a window of 1024 samples. The adaptive proportion test fails any of the conditions in [Table 24-1](#) are violated. For example, the test will fail if more than 912 samples within a 1024 sample window are the same bit value.

Table 24-1. Adaptive Proportion Test Bounds

Bit Pattern	Allowable Range of Bit Pattern Occurrences
1	$112 < n < 912$
10	$211 < n < 341$
001	$97 < n < 192$
1011	$11 < n < 104$

The bit patterns in [Table 24-1](#) used for adaptive proportion testing are designed to effectively detect catastrophic loss of entropy in the TRNG while also minimizing the occurrence of false positive failure detections.

24.2.4.3.3 Handling Runtime Health Test Failures

In the event that the run-time health test identifies that insufficient entropy can be present, the health fail (IRQ_HEALTH_FAIL) interrupt is asserted and the TRNG state machine enters the ERROR state. Due to the random nature of the TRNG module, it is possible for a health failure to be indicated in some cases even when the TRNG is working properly. To determine if there is a true loss of entropy, the following procedure is recommended:

1. Clear the IRQ_HEALTH_FAIL interrupt
2. Power off the TRNG
3. Power on the TRNG to normal mode again
 - a. If the health failure is not asserted again, the TRNG can be used
 - b. If the health test fails a second time, go to step 1 and attempt to run the test a third time
 - c. If the health test fails a third time, there is catastrophic entropy loss and the TRNG should not be used

The TRNG does provide diagnostic information to indicate which test(s) failed. To determine which runtime test(s) are failing, check the REP_FAIL and ADAP_FAIL bits in the STAT register for the repetition test and adaptive proportion test, respectively.

24.2.5 Configuration

The TRNG module is configured and used by application software and is accessed through the PD1 CPU-only peripheral bus. The DMA does not have access to the TRNG.

The TRNG contains a state machine which manages the current operating mode of the TRNG. Application software can configure the TRNG through commands to change the current state. The TRNG also sources an interrupt to the CPU to communicate status.

Note

Writing to a TRNG configuration register before a CMD is written to the TRNG can cause an unexpected command fail interrupt request (IRQ_CMD_FAIL) to be asserted. Mask the TRNG command fail interrupt when writing to TRNG configuration registers until a CMD is written to the TRNG. Clear the IRQ_CMD_FAIL interrupt status after writing the CMD and before enabling the IRQ_CMD_FAIL interrupt.

24.2.5.1 TRNG State Machine

The TRNG includes a state machine which manages the current operating state of the TRNG. The TRNG is configured for use by sending commands to the TRNG to change its state. The TRNG has seven states, given in [Table 24-2](#).

Table 24-2. TRNG States

Code	State Name	Description
0x0	OFF	Analog block is powered off, digital block is disabled
0x1	PWRUP_ES	TRNG is in the process of powering up the entropy source and will transition to NORM_FUNC.
0x2	PWRDOWN_ES	TRNG is in the process of powering down the entropy source and will transition to OFF.

Table 24-2. TRNG States (continued)

Code	State Name	Description
0x3	NORM_FUNC	TRNG is running and generating random bits normally.
0x7	TEST_DIG	TRNG is running the digital power-on self-test .
0xB	TEST_ANA	TRNG is running the analog power-on self-test .
0xA	ERROR	Operation halted due to an error condition. The analog remains powered but the health and conditioning logic is stopped.

The current TRNG state can be determined by application software. To check the current state, read the FSM_STATE field in the STAT register of the TRNG.

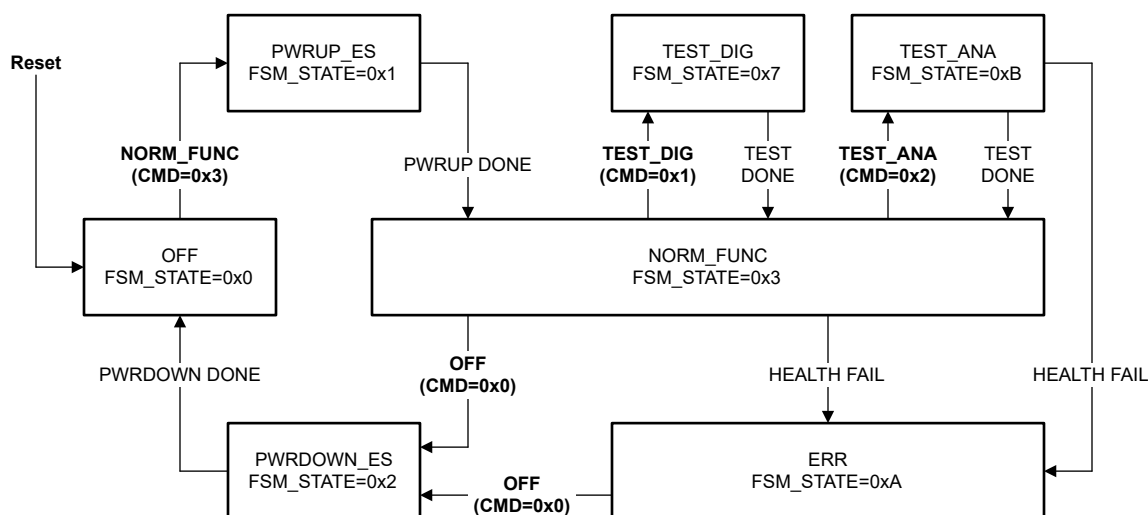
24.2.5.1.1 Changing TRNG States

Application software can set the operating state of the TRNG by writing a new command value to the CMD field in the CTL register. While the TRNG state machine has 7 states, only 4 of the 7 states are entered by user control with CMDs (OFF, TEST_DIG, TEST_ANA, and NORM_FUNC). The other 3 states (PWRUP_ES, PWRDOWN_ES, and ERR) are entered by a hardware only when powering up or down the analog block or when an error condition occurs. The operating state change commands for the TRNG are given in [Table 24-3](#).

Table 24-3. TRNG Operating Mode Commands

CMD	Mode	Description
0x0	PWROFF	The TRNG analog blocks are powered off, and the TRNG digital blocks are clock gated.
0x1	TEST_DIG	The TRNG digital power-on self-tests are executed and the result is reported in the TEST_RESULTS register.
0x2	TEST_ANA	The TRNG analog power-on self-tests are executed and the result is reported in the TEST_RESULTS register.
0x3	NORM_FUNC	The TRNG is put into normal operating mode, in which samples from the entropy source are collected, conditioned, decimated, and placed in the DATA_CAPTURE register. The continuous statistical health tests run in this mode.

From a given state, only certain state changes are possible. For example, from OFF only a CMD to switch to NORM_FUNC is supported. From NORM_FUNC, CMDs to switch to TEST_DIG, TEST_ANA, and OFF are supported. From the ERR state, only a switch to OFF is supported. The state flow for the TRNG is given in [Figure 24-2](#).


Figure 24-2. TRNG State Flow

A new command can only be written to the CMD field after any previously issued commands have run to completion. If a command is written at an invalid time, it will be rejected and a command fail interrupt will be asserted.

24.2.5.2 Using the TRNG

Use the following procedure to start the TRNG:

1. Enable the TRNG by setting the ENABLE bit together with the KEY in the TRNG PWREN register.
2. Configure the TRNG clock divider to ensure that the TRNG functional clock is within the allowable range (10MHz typical, see the device data sheet for additional detail). The clock divider is configured by programming the required value to the RATIO field of the CLKDIVIDE register. As an example, if MCLK is 80MHz, the RATIO field shall be set to 0x7 (divide-by-8) to provide a 10MHz functional clock to the TRNG module.
3. Verify that the TRNG interrupts are disabled (interrupt mask bits are cleared to mask interrupts).
4. Move the TRNG from the default OFF state to the NORM_FUNC state by writing the NORM_FUNC command (0x3) to the CMD field in the CTL register of the TRNG. Wait for the IRQ_CMD_DONE interrupt flag to be set, indicating that the CMD completed.
5. Run the [digital block start-up self-test routine](#) to ensure the TRNG digital is functioning properly:
 - a. Move the TRNG from the NORM_FUNC state to the TEST_DIG state by writing the TEST_DIG command (0x1) to the CMD field in the CTL register. Wait for the IRQ_CMD_DONE interrupt flag to be set, indicating that the digital self-test has completed.
 - b. Check that all 8 digital tests passed by ensuring the DIG_TEST field in the TEST_RESULTS register are set (DIG_TEST=0xFF).
 - c. After the digital test, the TRNG will return to the NORM_FUNC state automatically.
6. Run the [analog block start-up self-test routine](#) to ensure that the TRNG analog is functioning properly:
 - a. Move the TRNG from the NORM_FUNC state to the TEST_ANA state by writing the TEST_ANA command (0x2) to the CMD field in the CTL register. Wait for the IRQ_CMD_DONE interrupt flag to be set, indicating that the analog self-test has completed.
 - b. Check that the analog test passed by verifying that the ANA_TEST bit in the TEST_RESULTS register was set.
 - c. After the analog test, if the test passed the TRNG will return to the NORM_FUNC state automatically. If the test failed, the TRNG enters the ERR state and must be brought to the OFF state before attempting to use it again.
7. Configure the TRNG for normal operation after running start-up self-tests:
 - a. Clear the IRQ_CAPTURED_RDY_IRQ status by setting the corresponding ICLR bit, as this may have been set during the self-tests performed earlier.
 - b. Set the decimation rate to the desired value by programming the new decimation rate into the DECIM_RATE field of the CTL register, followed by sending the NORM_FUNC command again (by writing 0x3 to the CMD field in the CTL register). A decimation rate of 4 (DECIM_RATE=0x3) or greater is recommended.
 - c. Enable the health fail interrupt by setting the IRQ_HEALTH_FAIL bit in the IMASK register.
 - d. Enable the data captured interrupt by setting the IRQ_CAPTURED_RDY bit in the IMASK register.
8. Wait for the first IRQ_CAPTURED_RDY_IRQ, and read the DATA_CAPTURE register. This value (the first value read from DATA_CAPTURE after running a startup self-test) is not a true random value and must be read and discarded before collecting true random data from the DATA_CAPTURE register.
9. When the IRQ_CAPTURED_RDY_IRQ is again asserted, random bits are available for read-out in the DATA_CAPTURE register.
10. If the IRQ_HEALTH_FAIL_IRQ is asserted, a low entropy condition was found and the TRNG will have automatically switched to the ERR state to stop operation. To exit the ERR state, clear the IRQ_HEALTH_FAIL interrupt. Then, transition the TRNG to the OFF state by sending an OFF command (0x0) to the CMD field in the CTL register. Wait for the IRQ_CMD_DONE interrupt flag to be set, then return to step #2 to power up the TRNG again to test if sufficient entropy is again available.

24.2.5.3 TRNG Events

The TRNG module contains one [event publisher](#) and no [event subscribers](#). One event publisher (CPU_INT) manages TRNG interrupt requests (IRQs) to the CPU subsystem through a [static event route](#).

[Table 24-4](#) lists the TRNG events.

Table 24-4. TRNG Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU Interrupt Event	Publisher	TRNG	CPU Subsystem	Static route	CPU_INT registers	Fixed interrupt route from TRNG to CPU

24.2.5.3.1 CPU Interrupt Event Publisher (CPU_INT)

The TRNG module provides four interrupt sources which can be configured to source a [CPU interrupt event](#).

[Table 24-5](#) lists the TRNG interrupt conditions.

Table 24-5. TRNG CPU Interrupt Conditions (CPU_INT)

Index (IIDX)	Name	Description
1	IRQ_HEALTH_FAIL	Indicates that a health test has failed.
2	IRQ_CMD_FAIL	Indicates that a CMD which was issued has failed.
3	IRQ_CMD_DONE	Indicates that a CMD which was issued has completed.
4	IRQ_CAPTURED_RDY	Indicates that a new 32-bit data word containing random bits is available to be read by the processor.

The CPU interrupt event configuration is managed with the CPU_INT event management registers. See [Section 7.2.5](#) for guidance on configuring these registers for CPU interrupts.

24.3 TRNG Registers

Table 24-6 lists the memory-mapped registers for the TRNG registers. All register offset addresses not listed in Table 24-6 should be considered as reserved locations and the register contents should not be modified.

Table 24-6. TRNG Registers

Offset	Acronym	Register Name	Group	Section
800h	PWREN	Power enable		Go
804h	RSTCTL	Reset Control		Go
814h	STAT	Status Register		Go
1020h	IIDX	Interrupt index	CPU_INT	Go
1028h	IMASK	Interrupt mask	CPU_INT	Go
1030h	RIS	Raw interrupt status	CPU_INT	Go
1038h	MIS	Masked interrupt status	CPU_INT	Go
1040h	ISSET	Interrupt set	CPU_INT	Go
1048h	ICLR	Interrupt clear	CPU_INT	Go
10FCh	DESC	Module descriptions		Go
1100h	CTL	Controls the command and decimation rate		Go
1104h	STAT	Status register that informs health test results and last issued command		Go
1108h	DATA_CAPTURE	Captured word buffer of RNG data		Go
110Ch	TEST_RESULTS	Test results from TEST_ANA and TEST_DIG		Go
1110h	CLKDIVIDE	Clock Divider		Go

Complex bit access types are encoded to fit into small table cells. Table 24-7 shows the codes that are used for access types in this section.

Table 24-7. TRNG Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value

24.3.1 PWREN (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 24-3](#) and described in [Table 24-8](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 24-3. PWREN

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-0h							R/WK-0h

Table 24-8. PWREN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	ENABLE	R/WK	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

24.3.2 RSTCTL (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 24-4](#) and described in [Table 24-9](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 24-4. RSTCTL

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCL R	RESETASSERT
W-0h						WK-0h	WK-0h

Table 24-9. RSTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	W	0h	
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

24.3.3 STAT (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 24-5](#) and described in [Table 24-10](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 24-5. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED							
R-							

Table 24-10. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0h	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

24.3.4 IIDX (Offset = 1020h) [Reset = X]

IIDX is shown in [Figure 24-6](#) and described in [Table 24-11](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. 0h means no event pending. Interrupt 1 is the highest priority, 2 next highest, 4, 8, ... 2^{31} is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in the RIS and MIS are cleared as well. After a read from the CPU (not from the debug interface), the register must be updated with the next highest priority interrupt, if none are pending, then it should display 0h.

Figure 24-6. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-X																								R-0h							

Table 24-11. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	X	
7-0	STAT	R	0h	Interrupt index status 0h = No bit is set means there is no pending interrupt request 1h = Indicates that a health test has failed. The TRNG is in an error state until the interrupt is cleared. 2h = Indicates that the just issued command was rejected and is not being performed. 3h = Indicates that the current command/mode is done. This may have different meanings based on the mode: OFF --> Power has been turned off PWRUP_DIG --> Digital powerup tests are done PWRUP_ANA --> Analog powerup tests are done NORM_FUNC --> No IRQ, since mode runs indefinitely until a new command is issued 4h = Indicates that the captured word buffer is ready to be copied to memory

24.3.5 IMASK (Offset = 1028h) [Reset = X]

IMASK is shown in [Figure 24-7](#) and described in [Table 24-12](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt mask is set.

Figure 24-7. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED							
R/W-X							
15	14	13	12	11	10	9	8
RESERVED							
R/W-X							
7	6	5	4	3	2	1	0
RESERVED				IRQ_CAPTURE D_RDY	IRQ_CMD_DO NE	IRQ_CMD_FAIL	IRQ_HEALTH_ FAIL
R/W-X				R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 24-12. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	X	
3	IRQ_CAPTURED_RDY	R/W	0h	Mask for IRQ_CAPTURED_RDY. Indicates to the CPU that the Captured Word is ready to be read. 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
2	IRQ_CMD_DONE	R/W	0h	Mask for IRQ_CMD_DONE. Indicates that a command has finished 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
1	IRQ_CMD_FAIL	R/W	0h	Masked interrupt source for IRQ_CMD_FAIL. Indicates that the just issued command/mode has been rejected. 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
0	IRQ_HEALTH_FAIL	R/W	0h	Mask for IRQ_HEALTH_FAIL. Indicates that a health test has failed. 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set

24.3.6 RIS (Offset = 1030h) [Reset = X]

RIS is shown in [Figure 24-8](#) and described in [Table 24-13](#).

Return to the [Summary Table](#).

Raw interrupt status reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 24-8. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-X							
23	22	21	20	19	18	17	16
RESERVED							
R-X							
15	14	13	12	11	10	9	8
RESERVED							
R-X							
7	6	5	4	3	2	1	0
RESERVED				IRQ_CAPTURE D_RDY	IRQ_CMD_DO NE	IRQ_CMD_FAIL	IRQ_HEALTH_ FAIL
R-X				R-0h	R-0h	R-0h	R-0h

Table 24-13. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	X	
3	IRQ_CAPTURED_RDY	R	0h	Indicates to the CPU that the Captured Word is ready to be read. Reading the IIDX will clear this interrupt. 0h = IRQ_CAPTURED_READY did not occur 1h = IRQ_CAPTURED_READY occurred
2	IRQ_CMD_DONE	R	0h	Raw interrupt source for IRQ_CMD_DONE. Indicates that the issued command/mode has completed. 0h = IRQ_CMD_DONE did not occur 1h = IRQ_CMD_DONE occurred
1	IRQ_CMD_FAIL	R	0h	Masked interrupt source for IRQ_CMD_FAIL. Indicates that the just issued command/mode has been rejected. 0h = IRQ_CMD_FAIL did not occur 1h = IRQ_CMD_FAIL occurred
0	IRQ_HEALTH_FAIL	R	0h	Indicates to the CPU that any of the health tests have failed. Reading the IIDX will clear this interrupt. 0h = IRQ_CAPTURED_READY did not occur 1h = IRQ_CAPTURED_READY occurred

24.3.7 MIS (Offset = 1038h) [Reset = X]

MIS is shown in [Figure 24-9](#) and described in [Table 24-14](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 24-9. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-X							
23	22	21	20	19	18	17	16
RESERVED							
R-X							
15	14	13	12	11	10	9	8
RESERVED							
R-X							
7	6	5	4	3	2	1	0
RESERVED				IRQ_CAPTURE D_RDY	IRQ_CMD_DO NE	IRQ_CMD_FAIL	IRQ_HEALTH_ FAIL
R-X				R-0h	R-0h	R-0h	R-0h

Table 24-14. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	X	
3	IRQ_CAPTURED_RDY	R	0h	Masked interrupt result for CAPTURED_READY. Indicates to the CPU that the Captured Word is ready to be read. Reading the IIDX will clear this interrupt. 0h = IRQ_CAPTURED_READY did not request an interrupt service routine 1h = IRQ_CAPTURED_READY requests an interrupt service routine
2	IRQ_CMD_DONE	R	0h	Masked interrupt source for IRQ_CMD_DONE. Indicates that the issued command/mode has completed. 0h = IRQ_CAPTURED_READY did not request an interrupt service routine 1h = IRQ_CMD_DONE requests an interrupt service routine
1	IRQ_CMD_FAIL	R	0h	Masked interrupt source for IRQ_CMD_FAIL. Indicates that the just issued command/mode has been rejected. 0h = IRQ_CMD_FAIL did not request an interrupt service routine 1h = IRQ_CMD_FAIL requests an interrupt service routine
0	IRQ_HEALTH_FAIL	R	0h	Masked interrupt result for HEALTH_FAIL. Indicates to the CPU that any of the health tests have failed for the latest 1024-bit window. 0h = IRQ_CAPTURED_READY did not request an interrupt service routine 1h = IRQ_CAPTURED_READY requests an interrupt service routine

24.3.8 ISET (Offset = 1040h) [Reset = 00000000h]

ISET is shown in [Figure 24-10](#) and described in [Table 24-15](#).

Return to the [Summary Table](#).

ISET allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 24-10. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-							
23	22	21	20	19	18	17	16
RESERVED							
W-							
15	14	13	12	11	10	9	8
RESERVED							
W-							
7	6	5	4	3	2	1	0
RESERVED				IRQ_CAPTURE D_RDY	IRQ_CMD_DO NE	IRQ_CMD_FAIL	IRQ_HEALTH_ FAIL
W-				W-	W-	W-	W-

Table 24-15. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	W	0h	
3	IRQ_CAPTURED_RDY	W	0h	Indicates to the CPU that the Captured Word is ready to be read. Reading the IIDX or DATA_CAPTURE registers will clear this interrupt. 0h = Writing a 0 has no effect 1h = RIS bit corresponding to CAPTURED_READY is set
2	IRQ_CMD_DONE	W	0h	Write to turn on CMD_DONE IRQ. Indicates that the last issued TRNG command has finished. 0h = Writing a 0 has no effect. 1h = RIS bit corresponding to CMD_DONE is set
1	IRQ_CMD_FAIL	W	0h	Masked interrupt source for IRQ_CMD_FAIL. Indicates that the just issued command/mode has been rejected. 0h = Writing a 0 has no effect. 1h = RIS bit corresponding to CMD_FAIL is set
0	IRQ_HEALTH_FAIL	W	0h	Indicates to the CPU that any of the health tests have failed. Reading the IIDX or DATA_CAPTURE registers will clear this interrupt. 0h = Writing a 0 has no effect 1h = RIS bit corresponding to HEALTH_FAIL is set

24.3.9 ICLR (Offset = 1048h) [Reset = 00000000h]

ICLR is shown in [Figure 24-11](#) and described in [Table 24-16](#).

Return to the [Summary Table](#).

Write a 1 to clear corresponding Interrupt.

Figure 24-11. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-							
23	22	21	20	19	18	17	16
RESERVED							
W-							
15	14	13	12	11	10	9	8
RESERVED							
W-							
7	6	5	4	3	2	1	0
RESERVED				IRQ_CAPTURE D_RDY	IRQ_CMD_DO NE	IRQ_CMD_FAIL	IRQ_HEALTH_ FAIL
W-				W-	W-	W-	W-

Table 24-16. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	W	0h	
3	IRQ_CAPTURED_RDY	W	0h	Indicates to the CPU that the Captured Word is ready to be read. Reading the IIDX or DATA_CAPTURE registers will clear this interrupt. 0h = Writing a 0 has no effect 1h = RIS bit corresponding to CAPTURED_READY is cleared
2	IRQ_CMD_DONE	W	0h	Write to turn off CMD_DONE IRQ. Indicates that the last issued TRNG command has finished. 0h = Writing a 0 has no effect. 1h = RIS bit corresponding to CMD_DONE is cleared
1	IRQ_CMD_FAIL	W	0h	Masked interrupt source for IRQ_CMD_FAIL. Indicates that the just issued command/mode has been rejected. 0h = Writing a 0 has no effect. 1h = RIS bit corresponding to CMD_FAIL is cleared
0	IRQ_HEALTH_FAIL	W	0h	Indicates to the CPU that any of the health tests have failed. Reading the IIDX or DATA_CAPTURE registers will clear this interrupt. 0h = Writing a 0 has no effect 1h = RIS bit corresponding to CAPTURED_READY is cleared

24.3.10 DESC (Offset = 10FCh) [Reset = 05110000h]

DESC is shown in [Figure 24-12](#) and described in [Table 24-17](#).

Return to the [Summary Table](#).

This register is used to specify clock source selection for any special modules that need to choose between MCLK and another special clock source. The register is not expected to be present on most standard SVT peripherals but may be present on modules such as the ADC. The register is not present on VDDCOREULP domain peripherals and will be reserved, reading 0.

Figure 24-12. DESC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODULEID															
R-511h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FEATUREVER				INSTNUM				MAJREV				MINREV			
R-0h				R-0h				R-0h				R-0h			

Table 24-17. DESC Field Descriptions

Bit	Field	Type	Reset	Description
31-16	MODULEID	R	511h	Module Identifier - An internal TI page has been created to request unique module IDs
15-12	FEATUREVER	R	0h	Feature Set for the module *instance*
11-8	INSTNUM	R	0h	Instance Number within the device. This will be a parameter to the RTL for modules that can have multiple instances
7-4	MAJREV	R	0h	Major rev of the IP
3-0	MINREV	R	0h	Minor rev of the IP

24.3.11 CTL (Offset = 1100h) [Reset = X]

CTL is shown in [Figure 24-13](#) and described in [Table 24-18](#).

Return to the [Summary Table](#).

Affects various parameters of the TRNG system

Figure 24-13. CTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-X							
23	22	21	20	19	18	17	16
RESERVED			PWRUP_PSTART_CFG		PWRUP_PCHRG_CFG		PWRUP_CLKDIV
R/W-X			R/W-2h		R/W-2h		R/W-0h
15	14	13	12	11	10	9	8
RESERVED					DECIM_RATE		
R/W-X					R/W-0h		
7	6	5	4	3	2	1	0
RESERVED						CMD	
R/W-X						R/W-0h	

Table 24-18. CTL Field Descriptions

Bit	Field	Type	Reset	Description
31-21	RESERVED	R/W	X	
20-19	PWRUP_PSTART_CFG	R/W	2h	Configure pulse startup sequence length b00 = Disabled b01 = rise at 10us, fall at 50us b10 = rise at 10us, fall at 70us (default) b11 = rise at 10us, fall at 90us
18-17	PWRUP_PCHRG_CFG	R/W	2h	Configure PCHARGE sequence length b00 = Disabled b01 = 20 us PCHARGE b10 = 30 us PCHARGE (default) b11 = 40 us PCHARGE
16	PWRUP_CLKDIV	R/W	0h	When '1', the powerup sequence will take twice as long (i.e., clock frequency halved)
15-11	RESERVED	R/W	X	
10-8	DECIM_RATE	R/W	0h	Set decimation rate. Decimate by n 0x0 = Decimation by 1 (no decimation) 0x1 = Decimation by 2 (Skip every other sample) ... 0x7 = Decimation by 8 (Take every 8th sample)
7-2	RESERVED	R/W	X	

Table 24-18. CTL Field Descriptions (continued)

Bit	Field	Type	Reset	Description
1-0	CMD	R/W	0h	Sets the TRNG mode through a command. The mode will not be updated until the previous command is done, as indicated by IRQ_CMD_DONE. 00 --> OFF 01 --> PWRUP_DIG 10 --> PWRUP_ANA 11 --> NORM_FUNC 0h = Turns the power off of the analog source and clocks the digital interface 1h = Initiates the powerup test sequence for the digital components. This verifies that the digital components are properly working. IRQ_CMD_DONE indicates that the test is done. The results of this test are in bits 0:6 in TEST_RESULTS register 2h = Initiates the powerup test sequence for the analog TRNG. This verifies that the analog component is generating sequences with enough entropy. IRQ_CMD_DONE indicates that the test is done. The results of this test are in bit 7 in TEST_RESULTS register 3h = Normal operating mode for TRNG. All components are turned on.

24.3.12 STAT (Offset = 1104h) [Reset = X]

STAT is shown in [Figure 24-14](#) and described in [Table 24-19](#).

Return to the [Summary Table](#).

Status register that informs health test result, last issued command, and current FSM state

Figure 24-14. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-X							
23	22	21	20	19	18	17	16
RESERVED				FSM_STATE			
R-X				R-0h			
15	14	13	12	11	10	9	8
RESERVED						ISSUED_CMD	
R-X						R-0h	
7	6	5	4	3	2	1	0
RESERVED						REP_FAIL	ADAP_FAIL
R-X						R-0h	R-0h

Table 24-19. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-20	RESERVED	R	X	
19-16	FSM_STATE	R	0h	Current state of the front end FSM (behind a clock domain crossing). 2 reads are REQUIRED as there is a chance of metastability when reading this States: 0000: OFF 0001: PWRUP_ES 0011: NORM_FUNC 0111: TEST_DIG 1011: TEST_ANA 1010: ERROR 0010: PWRDOWN_ES
15-10	RESERVED	R	X	
9-8	ISSUED_CMD	R	0h	Indicates the last accepted command that is issued to the TRNG interface. Upon writing a valid command, this register will update and the command will be in progress until CMD_DONE_IRQ is set. CMD_DONE_IRQ indicates that the state is in PWROFF, NORM_FUNC, or ERROR. These states will accept new commands. 00 --> OFF 01 --> PWRUP_DIG 10 --> PWRUP_ANA 11 --> NORM_FUNC
7-2	RESERVED	R	X	
1	REP_FAIL	R	0h	Indicates that the repetition counter test caused the most recent failure. Thus, the health count numbers are most likely not for a complete 1024-bit window.
0	ADAP_FAIL	R	0h	Indicates that the Adaptive Proportion Test (1,2,3, or 4-bit counters) failed by having too many or too few counted samples in the last 1024 bit window.

24.3.13 DATA_CAPTURE (Offset = 1108h) [Reset = 00000000h]

DATA_CAPTURE is shown in [Figure 24-15](#) and described in [Table 24-20](#).

Return to the [Summary Table](#).

Captured data from decimation block

Figure 24-15. DATA_CAPTURE

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BUFFER																															
R-0h																															

Table 24-20. DATA_CAPTURE Field Descriptions

Bit	Field	Type	Reset	Description
31-0	BUFFER	R	0h	Captured Data from the Decimation Block

24.3.14 TEST_RESULTS (Offset = 110Ch) [Reset = X]

TEST_RESULTS is shown in [Figure 24-16](#) and described in [Table 24-21](#).

Return to the [Summary Table](#).

Includes a bit describing which startup test failed. The first 8 bits check the condition of the digital components in the digital startup validation check and the 9th bit checks that the entropy source is producing samples that have enough entropy. This register will read all '0s when a command is not finished. Each of the tests are active low, indicating a failed test with '0' and a passed test with '1'.

Figure 24-16. TEST_RESULTS

31	30	29	28	27	26	25	24
RESERVED							
R-X							
23	22	21	20	19	18	17	16
RESERVED							
R-X							
15	14	13	12	11	10	9	8
RESERVED							ANA_TEST
R-X							R-0h
7	6	5	4	3	2	1	0
DIG_TEST							
R-0h							

Table 24-21. TEST_RESULTS Field Descriptions

Bit	Field	Type	Reset	Description
31-9	RESERVED	R	X	
8	ANA_TEST	R	0h	Runs through 4096 samples from an enabled entropy source and verifies that none of the health tests failed, indicating sufficient entropy was produced by the analog components
7-0	DIG_TEST	R	0h	Bit 0 indicates if the first decimation rate test and health test(verifies conditioning, decimation, and captured buffer) fails and Bit 1 indicates if the second decimation test and health test fails Bit 0 - decim_test0 (decim = 0x0) Bit 1 - decim_test1 (decim = 0x1) ...

24.3.15 CLKDIVIDE (Offset = 1110h) [Reset = 00000000h]

CLKDIVIDE is shown in [Figure 24-17](#) and described in [Table 24-22](#).

Return to the [Summary Table](#).

This register is used to specify module-specific divide ratio of the functional clock , and it only supports even division for TRNG.

Figure 24-17. CLKDIVIDE

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												RATIO			
R/W-0h												R/W-0h			

Table 24-22. CLKDIVIDE Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R/W	0h	
2-0	RATIO	R/W	0h	Selects divide ratio of module clock 0h = Do not divide clock source 1h = Divide clock source by 2 3h = Divide clock source by 4 5h = Divide clock source by 6 7h = Divide clock source by 8

This page intentionally left blank.



The timer module (TIMx) is a timer counting module with multiple compare/capture blocks. Based on the device, two types of timers are available: general-purpose timers (TIMG) and advanced control timers (TIMA). Both timers include many common features that can be used for a variety of functions such as measuring the input signal edge and period (capture mode) or generating output waveforms (compare mode output) like PWMs. See the device-specific data sheet to determine which timers and timer instances are available.

25.1 TIMx Overview.....	1358
25.2 TIMx Operation.....	1361
25.3 Timers (TIMx) Registers.....	1409

25.1 TIMx Overview

The timer module (TIMx) is a timer counting module with multiple compare/capture blocks. Based on the device, two types of timers are available: general-purpose timers (TIMG) and advanced control timers (TIMA). Both timers use a common timer architecture, which allows for easy migration between timer instances with common functions. This minimizes the need to write extra software for timer-based applications and allows for easy porting and maintenance between TIMx instances.

Note

See [Section 25.1.3](#) to determine the common features available between TIMG and TIMA instances.

In this section:

- "TIMx" indicates a common feature available on TIMG and TIMA.
- "TIMA" indicates a feature available only on TIMA.
- "TIMG" indicates a feature available only on TIMG.

25.1.1 TIMG Overview

The TIMG module consists of 16-bit and 32-bit auto reload counters driven by a programmable prescaler with two capture/compare (CC) blocks for multiple capture/compares, PWM outputs, and interval timing. TIMG also has extensive event generation capabilities, including counter overflow, reload, and capture/compare actions for a variety of use cases.

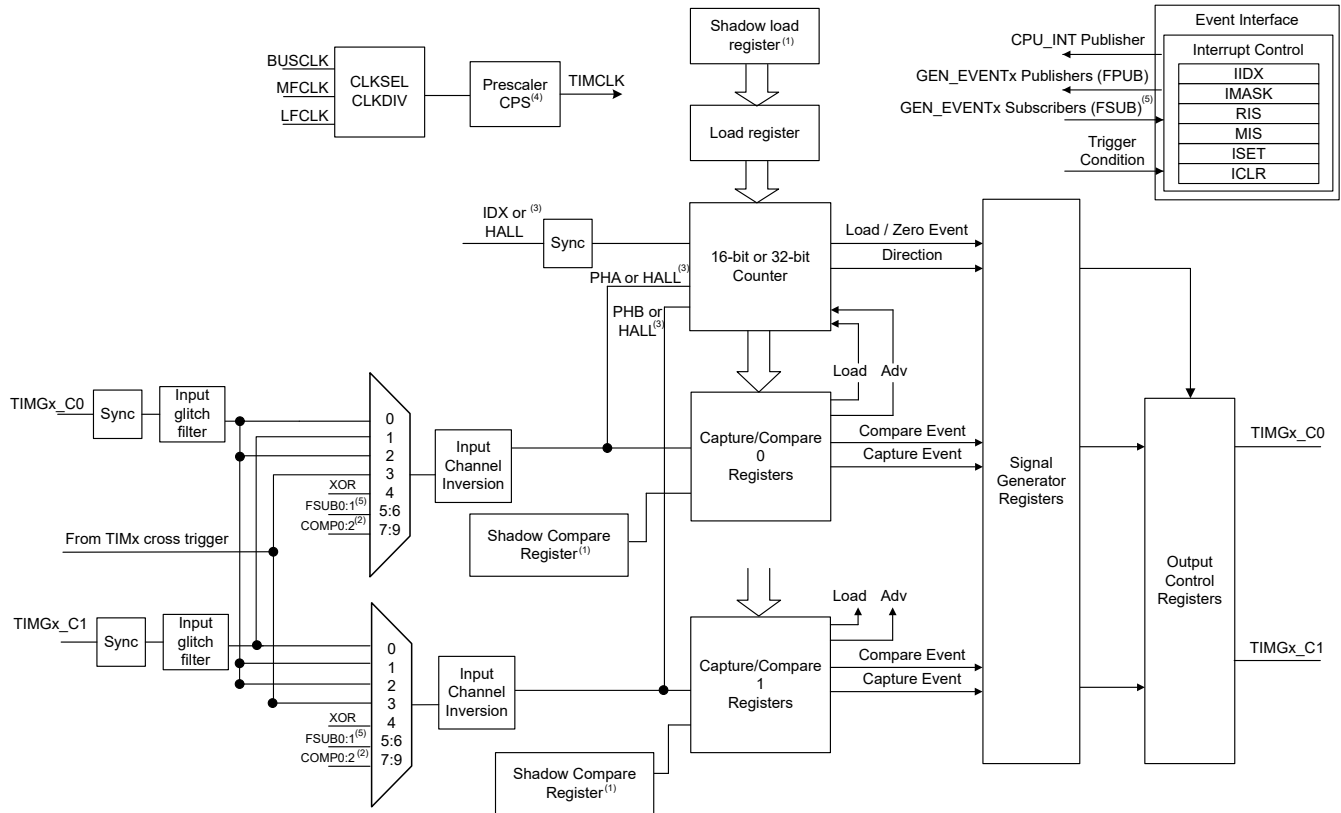
25.1.1.1 TIMG Features

Specific features for TIMG include:

- 16-bit or 32-bit up, down, or up-down counter, with repeat-reload mode
- 8-bit programmable prescaler to divide the counter clock frequency
- Up to two independent channels for
 - Output compare
 - Input capture
 - PWM output (Edge-Aligned and Center-Aligned)
 - One-shot mode
- Shadow register mode for load and compare values (see [Section 25.2.4](#))
- Support for quadrature encoder interface (QEI) (see [Section 25.2.3.1.3](#))
- 3-input Hall sensor mode for position sensing and speed computation (see [Section 25.2.3.1.4](#))
- Support synchronization and cross trigger among different TIMx instances in the same power domain (see [Section 25.2.7](#))
- Support CPU interrupt generation and cross peripherals (such as ADC, DAC, etc.) using the Event (see [Section 25.2.9](#))

25.1.1.2 Functional Block Diagram

[Figure 25-1](#) shows the TIMG block diagram.



- (1) TIMG0-3 and TIMG8-12 do not have shadow load and compare registers
(2) Devices with comparator only
(3) TIMG8-TIMG11 support QE1 and Hall input mode
(4) Not supported on TIMG12 and TIMG13
(5) Generic event (GEN_EVENTx) subscribers can be used to trigger any TIMx instance from any generic event publisher (GPIO, COMP, ADC, etc.)

Figure 25-1. TIMx Functional Block Diagram

25.1.2 TIMA Overview

The TIMA module consists of a 16-bit auto reload counter driven by a programmable prescaler with up to four capture/compare (CC) blocks for multiple capture/compares, PWM outputs with deadband insertion, and interval timing. TIMA has extensive event generation capabilities from different counter events such as overflow, reload, and from each of the capture/compare registers. It also has the hardware design to handle the fault signal generated by internal or external circuitry to indicate a fault in the system.

25.1.2.1 TIMA Features

- 16-bit up, down, or up-down counter, with repeat-reload mode
- Selectable and configurable clock source
- 8-bit programmable prescaler to divide the counter clock frequency
- Repeat counter to generate an interrupt or event only after a given number of cycles of the counter (see [Section 25.2.1.2](#))
- Up to four independent channels for:
 - Output compare
 - Input capture
 - PWM output (Edge-Aligned and Center-Aligned)
 - One-shot mode
- Two additional capture/compare channels for internal events (CC4/CC5)
- Shadow register for load and compare values (see [Section 25.2.4](#))
- Complementary PWM output with programmable deadband insertion (see [Section 25.2.5.2.4](#))
- Asymmetric PWM output (see [Section 25.2.5.5](#))

- Fault handling mechanisms to ensure the output signals are in a safe user-defined state when a fault condition is encountered (see [Section 25.2.6](#))
- Support synchronization and cross trigger among different TIMx instances in the same power domain (see [Section 25.2.7](#))
- Support CPU interrupt generation and cross peripherals (such as ADC, DAC, etc.) using the Event (see [Section 25.2.9](#))

25.1.2.2 Functional Block Diagram

Figure 25-2 shows the TIMA block diagram.

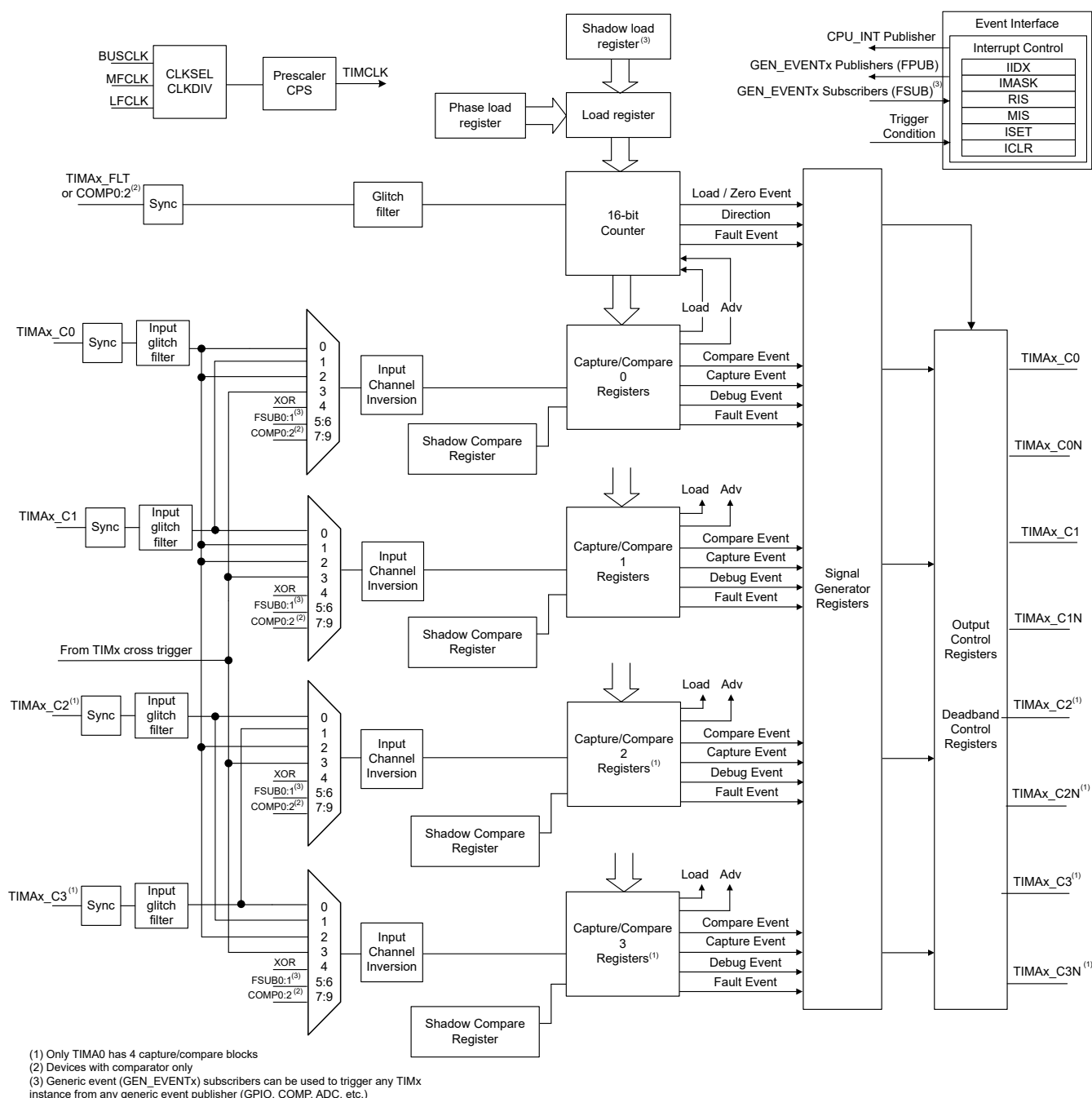


Figure 25-2. TIMA Block Diagram

25.1.3 TIMx Instance Configuration

Table 25-1 shows the TIMx instance configuration for TIMA and TIMG instances.

Table 25-1. TIMx Instance Configuration

Instance	Power Domain	Counter Resolution	Prescaler	Repeat Counter	CCP Channels (External/Internal)	External PWM Channels	Phase Load	Shadow Load	Shadow CCs	Deadband	Fault Handler	QEI / Hall Input Mode
TIMG0	PD0	16-bit	8-bit	-	2	2	-	-	-	-	-	-
TIMG1	PD0	16-bit	8-bit	-	2	2	-	-	-	-	-	-
TIMG2	PD0	16-bit	8-bit	-	2	2	-	-	-	-	-	-
TIMG3	PD0	16-bit	8-bit	-	2	2	-	-	-	-	-	-
TIMG4	PD0	16-bit	8-bit	-	2	2	-	Yes	Yes	-	-	-
TIMG5	PD0	16-bit	8-bit	-	2	2	-	Yes	Yes	-	-	-
TIMG6	PD1	16-bit	8-bit	-	2	2	-	Yes	Yes	-	-	-
TIMG7	PD1	16-bit	8-bit	-	2	2	-	Yes	Yes	-	-	-
TIMG8	PD0	16-bit	8-bit	-	2	2	-	-	-	-	-	Yes
TIMG9	PD0	16-bit	8-bit	-	2	2	-	-	-	-	-	Yes
TIMG10	PD1	16-bit	8-bit	-	2	2	-	-	-	-	-	Yes
TIMG11	PD1	16-bit	8-bit	-	2	2	-	-	-	-	-	Yes
TIMG12	PD1	32-bit	-	-	2	2	-	-	Yes	-	-	-
TIMG13	PD0	32-bit	-	-	2	2	-	-	Yes	-	-	-
TIMG14	PD1	16-bit	8-bit	-	4	4	-	-	-	-	-	-
TIMA0	PD1	16-bit	8-bit	Yes	4/2	8	Yes	Yes	Yes	Yes	Yes	-
TIMA1	PD1	16-bit	8-bit	Yes	2/2	4	Yes	Yes	Yes	Yes	Yes	-

Note

On TIMA instances, external PWM channels are pairs of complementary PWM output signals with deadband generation with respect to the CC block instance, such as TIMA0_C0 and TIMA0_C0N. For independent PWM output generation, separate non-inverting channels must be used, such as TIMA0_C0 and TIMA0_C1.

Note

Internal CC channels 4 and 5 (CC_45) can be used for internal compare events and are available in TIMA only.

Note

Look at the device-specific data sheet to check which TIMx instances are available on the device.

25.2 TIMx Operation

The TIMx module is configured with user software. The setup and operation of TIMx is discussed in the following sections.

Note

There are register arrays in the timer module to group registers with same bit fields for different capture/compare ports. For example, TIMx.CC_01[0/1] is a register array that contains the capture/compare registers for both CCP0 and CCP1. Access TIMx.CC_01[0] to read the CC0 capture/compare value, and access TIMx.CC_01[1] to read the CC1 capture/compare value.

Note

TIMx supports event subscribers and event publishers through event register arrays. For more information, please see the Events chapter.

25.2.1 Timer Counter

All TIMx instances have 16-bit counter blocks except for TIMG12 and TIMG13, which have 32-bit counter blocks. The timer counter register (TIMx.CTR) can count down, up-and-down, or up depending on the operation mode. It can also be read or written with software. Each count occurs with each rising edge of the TIMCLK signal or with both edges of external signals.

Enabling the TIMx Counter

The counter is clocked by the prescaler output TIMCLK. The counter enable bit TIMx.CTRCTL.EN can be enabled in two ways:

- Set in software manually
- After a load condition (LCOND) or zero condition (ZCOND) is met, and the counter value after enable (CVAE) is changed to the load value or zero value, respectively.

Note

The ability to write the counter register while TIMx is running is possible but should be avoided because the user write can collide with a load, zero, or advance event. Depending on the prescaler ratio, the application cannot predict the timing of the write, which can affect the correct timer period.

25.2.1.1 Clock Source Select and Prescaler

The TIMx clock (TIMCLK) can be sourced from an internal clock or an external signal trigger to advance the clock.

25.2.1.1.1 Internal Clock and Prescaler

The TIMx clock (TIMCLK) can be sourced from BUSCLK, MFCLK and LFCLK by setting the TIMx.CLKSEL register. It can also be divided by a ratio by setting the TIMx.CLKDIV register and a prescaler (if present). The selected source clock is always available and the frequency depends on the power mode. For more information, see the [Clock Module \(CKM\)](#) section.

The TIMCLK can come from the following sources:

- BUSCLK: the current bus clock is selected as the source for TIMx. The current bus clock depends on power domain.
 - If the TIMx instance is in Power Domain 1 (PD1), refer to [MCLK](#).
 - If the TIMx instance is in Power Domain 0 (PD0), refer to [ULPCLK](#).
- MFCLK: MFCLK is selected as the source for TIMx, refer to [MFCLK](#).
- LFCLK: LFCLK is selected as the source for TIMx, refer to [LFCLK](#).

The selected clock source can be passed directly to TIMx or divided by the 8-bit prescaler. The prescaler setting can be configured with register TIMx.CPS.PCNT bit. The selected TIMCLK source is divided by a value of (PCNT+1). A PCNT value of 0 divides TIMCLK by 1, effectively bypassing the divider. A PCNT value of greater than 0 divides the TIMCLK source to generate a slower clock.

TIMx also has a software mechanism for disabling the timer clock. Set TIMx.CCLKCTL.CLKEN to 0 to put the timer in an IDLE state.

TIMCLK Configuration

To configure the clock source, divider, and prescaler:

1. Select the TIMCLK clock source (BUSCLK, MFCLK, or LFCLK) using the CLKSEL register.
2. Optionally divide the TIMCLK using CLKDIV.RATIO.

3. In TIMx instances with prescalers, optionally set a prescaler using CPS.PCNT.
4. Enable the TIMCLK by setting CCLKCTL.CLKEN = 1.

The frequency of TIMCLK is determined using [Equation 28](#).

$$f_{TIMCLK} = \frac{f_{CLK_SOURCE}}{((CLKDIV.RATIO + 1) * (CPS.PCNT + 1))} \quad (28)$$

25.2.1.1.2 External Signal Trigger

The counter can also advance (increment or decrement) by using an external signal on the timer input pin or by triggering from an event from other peripherals in the system. This can be configured by using the advance condition setting (TIMx.CCCTL_xy[0/1].ACOND) to specify what creates the advance event. To specify what event advances the counter, use the TIMx.CTRCTL.CAC setting.

The counter can advance using the internal clock TIMCLK, a different edge of a timer external input, or an internal trigger event from other peripherals.

25.2.1.2 Repeat Counter (TIMA only)

In TIMA only, the repeat counter (RC) is an 8-bit counter that provides the mechanism to suppress unnecessary events and generate real events for optimal interrupt generation. Specifically, the repeat counter can suppress Load, Compare, and Zero events in the case where the timer is generating events that repeat for a known number of cycles, such as a periodic PWM output waveform. This prevents generating excessive and unnecessary interrupts every timer period.

When the timer counter (TIMA.CTR) is advancing, the repeat counter (TIMA.RC) advances once the counter reloads (TIMA.CTR = 0). The user can set the how many timer counter reloads occur until generating the interrupts and events by setting the TIMA.RCLD register. Once TIMA.RC = TIMA.RCLD, the repeat counter is reset back to zero and a Repeat Counter Zero event occurs (REPC) in the Interrupt and Event Status registers.

Note

If the counter is configured to stop counting in a debug or fault condition, the repeat counter should also be stopped. See [Section 25.2.6](#) and [Section 25.2.10](#) for more details.

Additionally, the repeat counter provides the ability to suppress generation of Zero, Load, and Compare events when TIMA.RC does not equal zero.

- Zero and Load events are suppressed by setting TIMA.CTRCTL.SLZERCNEZ register bit
- Compare events (see [Table 25-15](#)) are suppressed by setting the TIMA.CCCTL_xy[0/1].SCERCNEZ bit

[Table 25-2](#) shows the repeat counter behavior with respect to the timer counter and repeat counter load value.

Table 25-2. Repeat Counter Behavior

TIMA.CTR is advancing (+1)	Counter value	TIMA.RC = TIMA.RCLD	Repeat Counter Behavior	Suppress Load and Zero Events (SLZERCNEZ = 1)	Suppress Compare Events (SCERCNEZ = 1)
No	-	-	Does not advance	Yes	Yes
Yes	TIMA.CTR ≠ 0	-	Does not advance	Yes	Yes
Yes	TIMA.CTR = 0	No	Advance (+1)	Yes	Yes
Yes	TIMA.CTR = 0	Yes	TIMA.RC = 0	No	No

Repeat counter example

As shown in [Figure 25-3](#), the TIMA.CTR is configured for down-counting mode and zero events are generated once TIMA.CTR = 0. To suppress interrupt generation until 4 timer reloads occur, set TIMA.RCLD = 4 and

TIMA.CTRCTL.SLZERCNEZ = 1 to suppress zero and load events until RC = 0 (which occurs once TIMA.RC = TIMA.RCLD).

Note

The use of the repeat counter does not affect the output signal generation. All events are generated regardless of the repeat counter value sent to the signal generator unit.

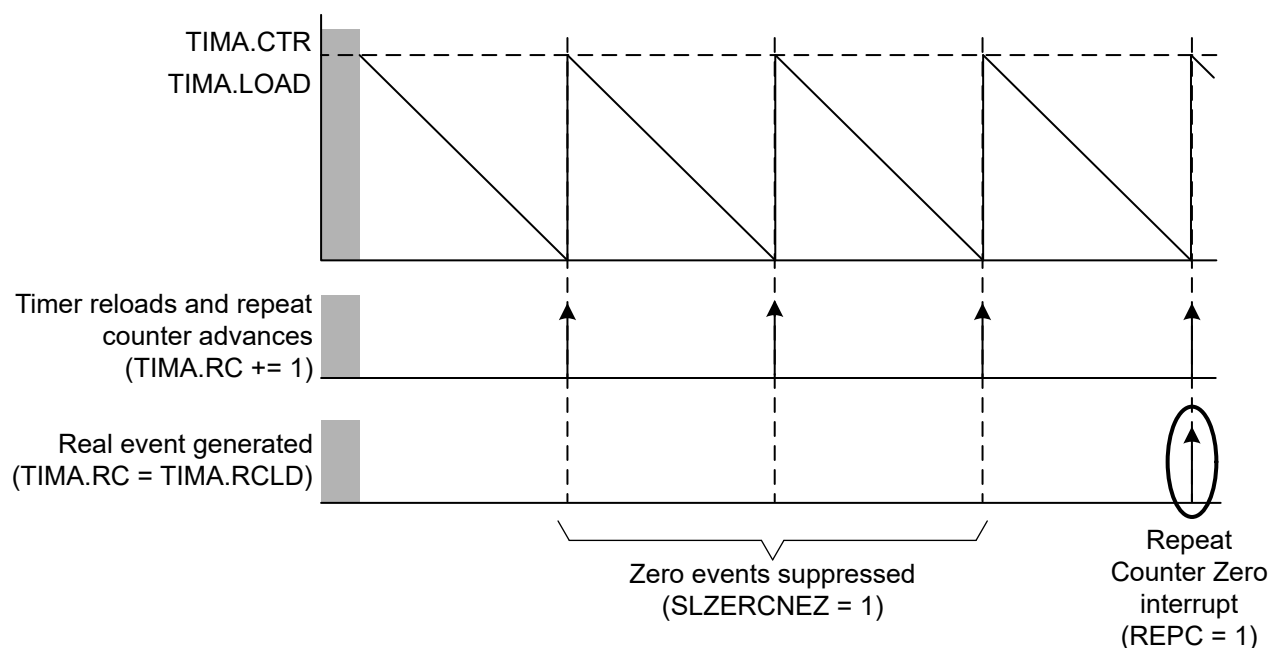


Figure 25-3. Event Suppressed by Repeat Counter with TIMx.RCLD = 4 in Down Counting Mode

25.2.2 Counting Mode Control

When the device is out of reset, TIMx is disabled. Writing 1 to the TIMx.CTRCTL.EN bit enables the counter. This bit is automatically cleared if TIMx.CTRCTL.REPEAT=0 (do not automatically reload), and the counter value equals zero.

TIMx has three counting modes when enabled: down, up/down, and up. The operating mode is selected by TIMx.CTRCTL.CM bit (shown in [Table 25-3](#)). After the counter is enabled, the timer will start counting from the count value after enable (TIMx.CTRCTL.CVAE) setting.

Table 25-3. TIMx Counting Modes (CM)

TIMx.CTRCTL.CM	Counting Mode
0	Down
1	Up/Down
2	Up

Table 25-4. TIMx Counter Value after Enable (CVAE)

Count Value After Enable (CVAE)	Description
0	LOAD value
1	Unchanged from current value
2	Zero value

Note

There is no dependency of CVAE on counting modes.

25.2.2.1 One-shot and Periodic Modes

[Figure 25-4](#) shows TIMx working in both one-shot mode and periodic mode.

One-shot Mode

When TIMx.CTRCTL.REPEAT bit is set to 0, TIMx does not advance when:

- TIMx.CTR value reaches 0 in either down- or up/down-counting mode
- TIMx.CTR value reaches TIMx.LOAD in up-counting mode

Periodic Mode (Counter Reload)

When TIMx.CTRCTL.REPEAT is set to 1h, TIMx automatically repeats once a zero event occurs. This happens when:

- TIMx.CTR reaches 0 in either down- or up/down-counting mode
 - In down-counting mode, a zero event is followed by a load event at the next advance condition
 - In up/down-counting mode, a zero event is followed by an advance event (+1)
- TIMx.CTR reaches TIMx.LOAD in up-counting mode
 - A load event is followed by a zero at the next advance condition

TIMx.CTRCTL.REPEAT can also be set to 3h for TIMx to repeat only when not in a debug condition. If there is a debug condition, TIMx will count to the zero event and repeat only once the debug condition is removed.

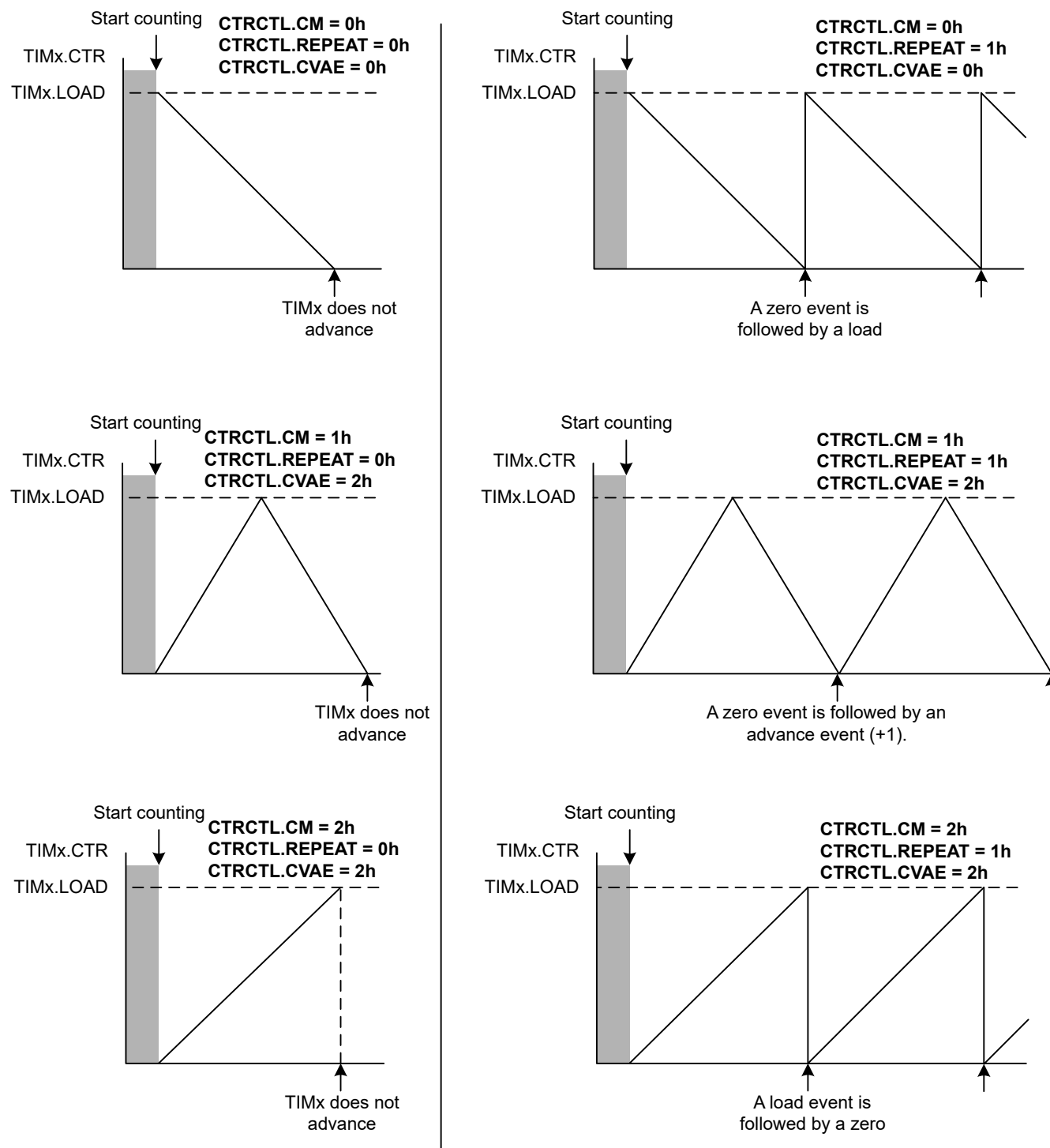


Figure 25-4. One-shot and Periodic Mode Behavior

25.2.2.2 Down Counting Mode

In down counting mode (CM = 0), TIMx counts from the value defined in TIMx.LOAD (CVAE = 0) down to zero. When the TIMx.CTR value equals zero and TIMx.CTRCTL.REPEAT is set to 1, the TIMx.LOAD value is loaded into TIMx.CTR and the timer repeats the down counting pattern as shown in [Figure 25-5](#).

A Zero event is generated when TIMx counts to zero. A Load event is generated when TIMx counts from zero to the TIMx.LOAD value. Figure 25-6 shows the event generating cycle.

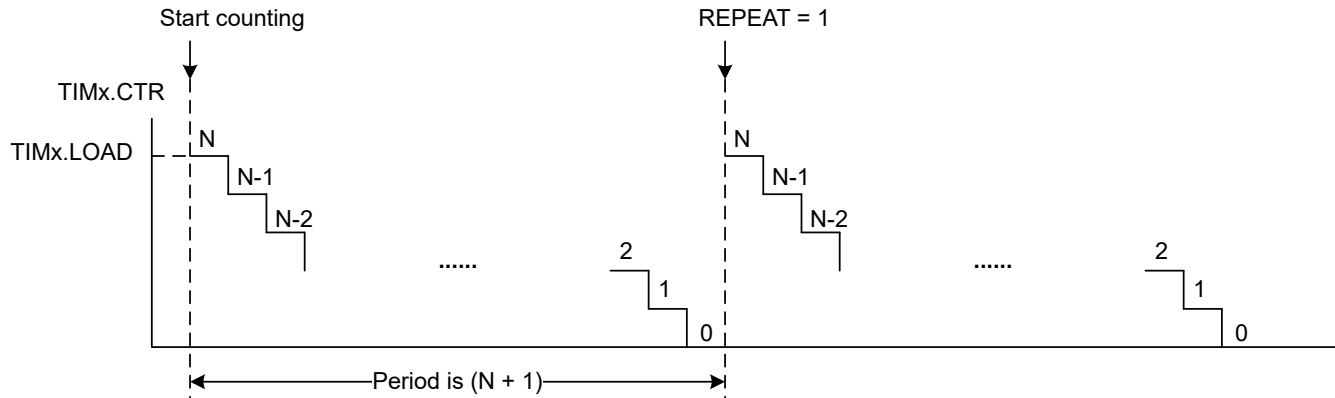


Figure 25-5. Down Counting Mode, CVAE = 0

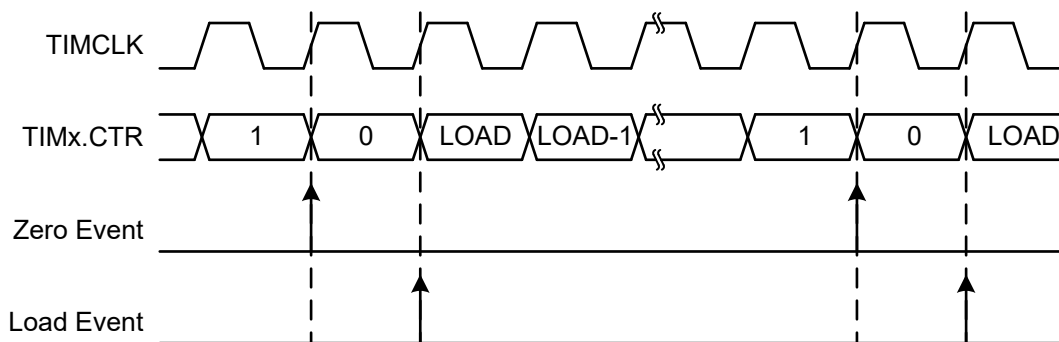


Figure 25-6. Down Counting Mode Event Generation

25.2.2.3 Up/Down Counting Mode

The Up/Down counting mode can count in an down-up direction or an up-down direction depending on TIMx.CTRCTL.CVAE value. The TIMx.CTRCTL.CVAE bits specify the initialization condition of the counter.

Table 25-5. Counter Value after Enable Initialization Conditions

TIMx.CTRCTL.CVAE Value	Counter Value After Enable
0x0	Load Value
0x1	No Change
0x2	Zero

Counting in down-up direction

When TIMx.CTRCTL.CVAE = 0, TIMx.CTR is set to TIMx.LOAD register value and TIMx counts in the down direction. When it reaches zero, a Zero event is generated and TIMx counts back up to TIMx.LOAD value. A Load event is generated when it reaches TIMx.LOAD value.

Figure 25-7 shows TIMx counting in the down-up direction when TIMx.CTRCTL.CVAE = 0.

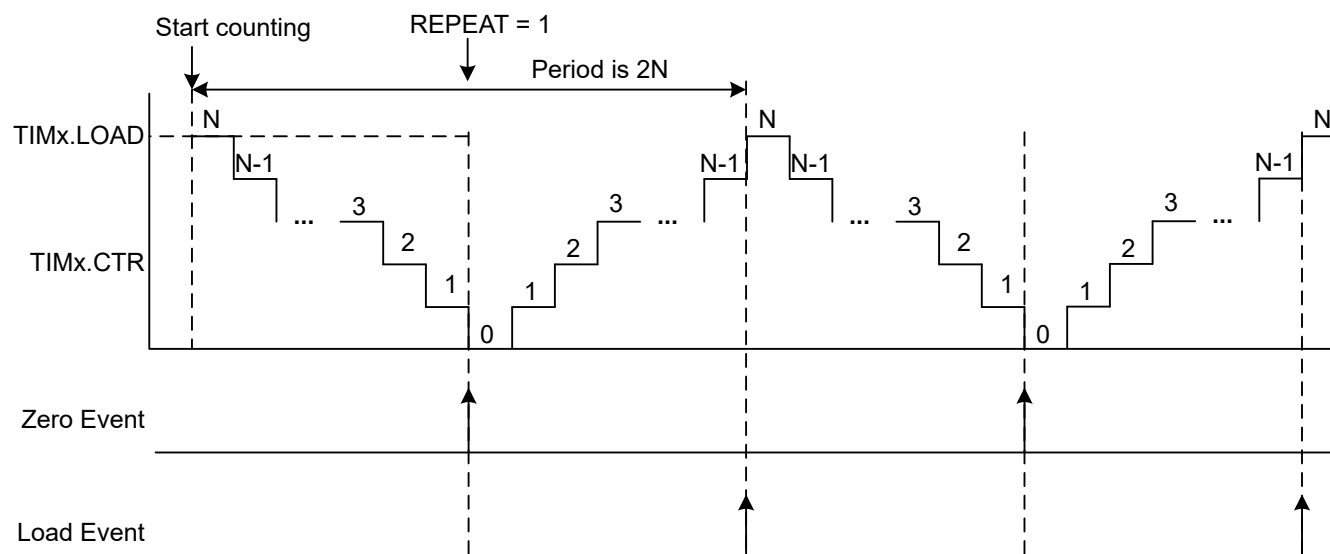


Figure 25-7. Down-up Counting Mode and Event Generation, CVAE = 0

Counting in up-down direction

When $TIMx.CTRCTL.CVAE = 2$, $TIMx.CTR$ is set to zero and $TIMx$ counts in the up direction. When it reaches $TIMx.LOAD$, a Load event is generated and $TIMx$ counts back down to zero. A Zero event is generated when it reaches zero.

Figure 25-8 shows $TIMx$ counting in up-down direction when $TIMx.CTRCTL.CVAE = 2$.

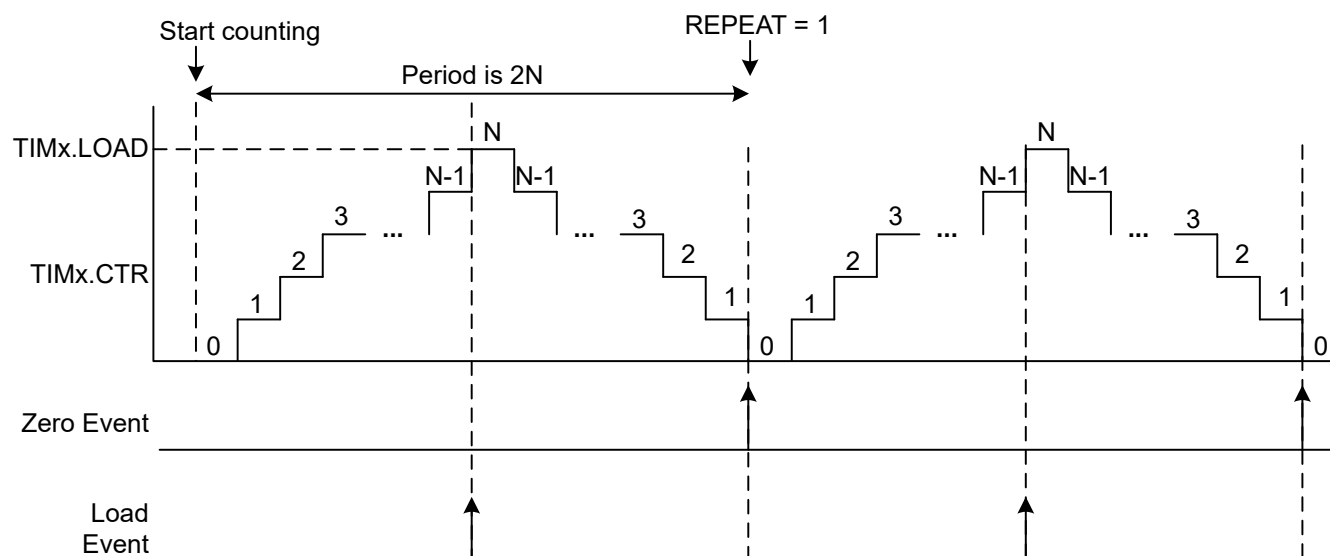


Figure 25-8. Up-down Counting Mode and Event Generation, CVAE = 2

25.2.2.4 Up Counting Mode

In up counting mode, $TIMx$ counts from zero up to the value defined in $TIMx.LOAD$. When the $TIMx.CTR$ value equals $TIMx.LOAD$ and $TIMx.CTRCTL.REPEAT$ is not equal to 0, the zero is loaded into $TIMx.CTR$ and the timer repeats the up counting pattern as shown in Figure 25-5.

A Load event is generated when $TIMx$ counts to $TIMx.LOAD$. A Zero event is generated when $TIMx$ counts from $TIMx.LOAD$ to the zero value. Figure 25-6 shows the event generating cycle.

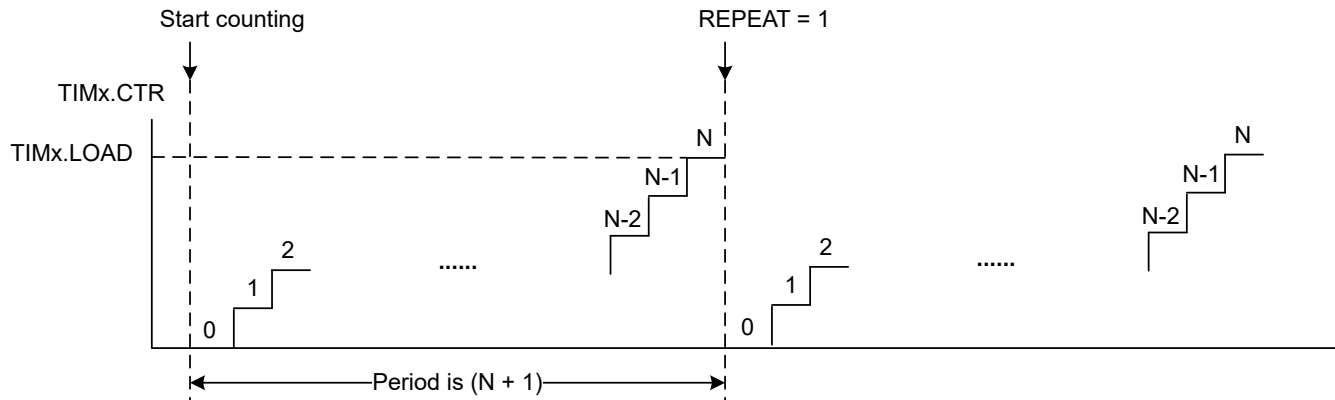


Figure 25-9. Up Counting Mode, CVAE = 2

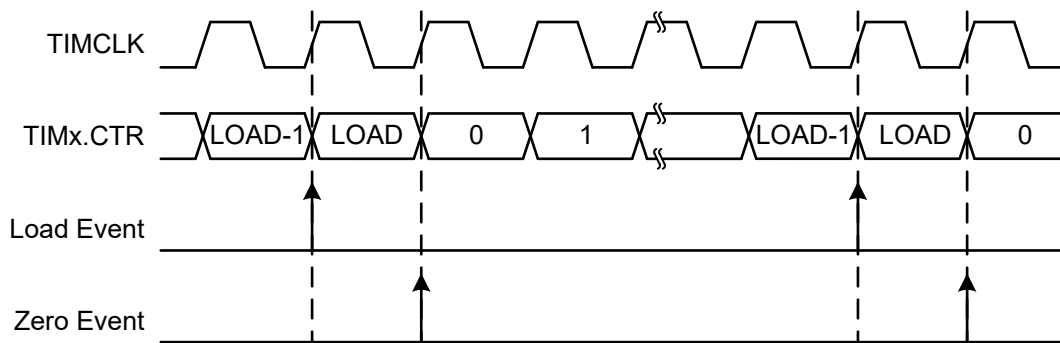


Figure 25-10. Up Counting Mode Event Generation

25.2.2.5 Phase Load (TIMA only)

In TIMA only, the phase load register TIMA.PL provides the capability for TIMA.CTR to count from a value other than zero or TIMA.LOAD in Up/Down counting mode. Phase load is used to generate asymmetric center-aligned PWM output signals with a controlled phase shift between different timer instances.

When TIMA.PL is nonzero, phase load is enabled by setting TIMA.CTRCTL.PLEN = 1 and triggered when TIMA.CTTRIG.TRIG = 1. When phase load is triggered while TIMA.CTRCTL.CVAE = 0, the timer counts from the TIMA.PL value in the down direction. When phase load is triggered while TIMA.CTRCTL.CVAE = 1, the TIMx counts from the TIMA.PL value in the up direction.

TIMA.PL is latched when the timer starts, and TIMA.PL is synchronized every time when the counter reaches the previously latched TIMA.PL value. [Figure 25-11](#) shows how the phase load register works when the timer is counting in the up-down direction and the phase load value changes to a new value.

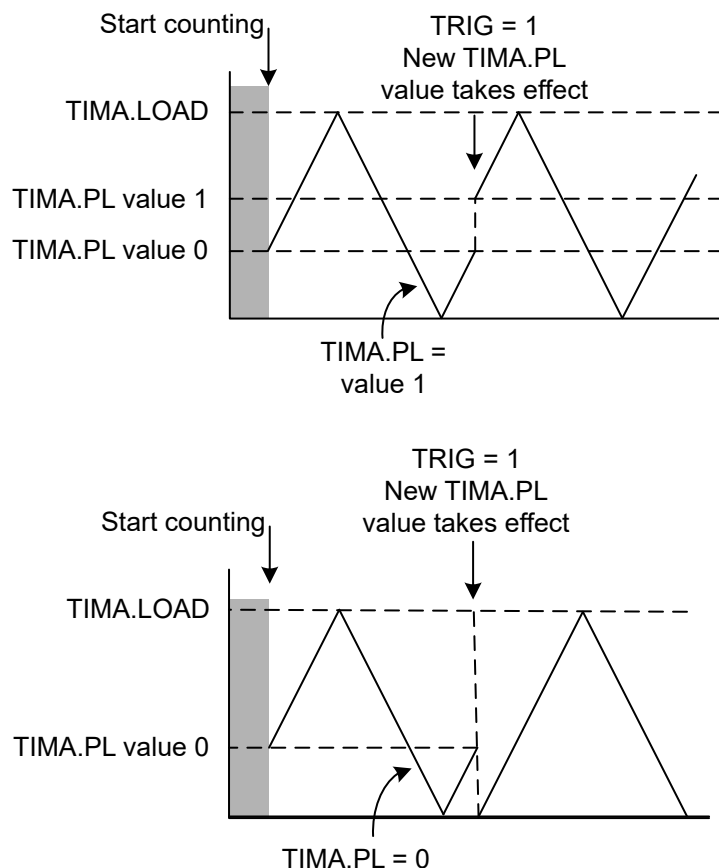


Figure 25-11. Phase Load Register Synchronization in Up-Down Mode

25.2.3 Capture/Compare Module

The capture/compare (CC) block is used for capture events or compare events. TIMG has up to 2 identical capture/compare blocks and TIMA has up to 4 identical capture/compare blocks present to support external or internal signals. Additionally, TIMA provides 2 additional CC blocks (CC4 and CC5) for compare events only from internal signals. Any of the TIMx capture/compare blocks may be used to capture timings of an input signal or to generate time intervals.

25.2.3.1 Capture Mode

Capture mode is selected when the TIMx.CCCTL_xy[0/1].COC bit is set to 1. Capture mode is used to generate capture events and record time intervals, which is useful for speed computation or time measurements.

Key registers for configuring capture mode:

- **TIMx.LOAD**: the contents of this register are copied to counter (TIMx.CTR) on any operation designated to do a "load". This value is also used to compare with the counter value for generating a "Load Event" which can be used for interrupt, trigger, or signal generation actions.
- **TIMx.CC_xy[0/1]**: this is a register that can be used as either a capture register to acquire or record the next counter value on an event, or a compare register to the current counter to create an event.
- **TIMx.CCCTL_xy[0/1]**: this register controls the operations of the respective CC (capture/compare) blocks. In capture mode, it can configure whether a rising edge or falling edge generates a load, zero, advance, or capture condition. In compare mode, it controls which sources generate different types of compare events.
- **TIMx.CTRCTL**: this register provides control over the counter operation in different conditions.
- **TIMx.IFCTL_xy[0/1]**: this register controls the input filtering (FE, FP, CPV), selection (ISEL), and inversion (INV) for the associated CC block.

25.2.3.1.1 Input Selection, Counter Conditions, and Inversion

The TIMx.IFCTL register is used for selecting input source, filtering, and final inversion options for the capture/compare block.

Figure 25-12 shows the block diagram for the TIMx capture block with two CC channels.

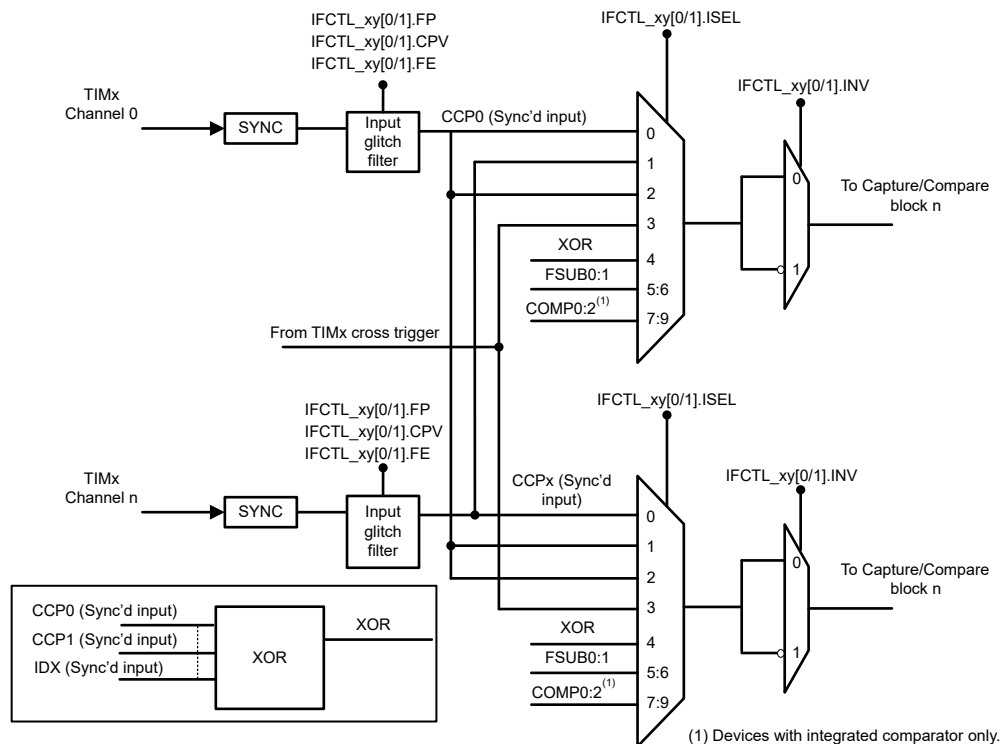


Figure 25-12. TIMx Capture Block Diagram

25.2.3.1.1.1 CCP Input Edge Synchronization

When using a capture/compare pin (CCP) as an input, configure the pin control management register (PINCMx) for the TIMx CCP input. Refer to the device data sheet for TIMx CCP pinmux input options, such as TIMG0_C0.

The CCP input signal is always passed through a synchronizer, and the input state (high or low) must be greater than one TIMCLK clock period for the synchronizer to detect the edge. CCP input edge detection requires at least one TIMCLK cycle to synchronize the edge input. Timing in the first TIMCLK cycle is uncertain because the edge detection cannot be predicted in the first TIMCLK period.

When the capture condition occurs, an additional TIMCLK cycle is required to generate the capture event.

25.2.3.1.1.2 CCP Input Pulse Conditions

The TIMx.CCCTL_xy[0/1] register can control whether each timer instance generates a zero, load, capture, or advance pulse based on the edge or polarity of the CCP input signal or trigger edge. The conditions that can be generated are:

- Advance condition (ACOND)
- Load condition (LCOND)
- Zero condition (ZCOND)
- Capture condition (CCOND)

Advance conditions

By default, the timer advances based on each TIMCLK. (ACOND = 0h). However, the timer can also advance based off the specified TIMx.CCCTL_xy[0/1].ACOND settings below.

Table 25-6. Advance pulse condition settings (ACOND)

ACOND	Condition
0h	Each TIMCLK
1h	Rising edge of CCP or trigger assertion edge
2h	Falling edge of CCP or trigger de-assertion edge
3h	Either edge of CCP or trigger
5h	CCP high or trigger assertion

Load, zero, and capture conditions

Load, zero, and capture pulses can be generated the LCOND, ZCOND, and CCOND condition settings below in the TIMx.CCCTL_xy[0/1] register.

Table 25-7. Load, zero, and capture condition settings (LCOND, ZCOND, CCOND)

LCOND	ZCOND	CCOND	Condition
N/A	N/A	0h	None
1h	1h	1h	Rising edge of CCP or trigger assertion edge
2h	2h	2h	Falling edge of CCP or trigger de-assertion edge
3h	3h	3h	Either edge of CCP or trigger

25.2.3.1.1.3 Counter Control Operation

To specify what CC instance controls the load, zero, or advance event, the CZC, CAC, and CLC fields are used for configuration in the TIMx.CTRCTL register.

See [Table 25-8](#) for counter zero control settings. For example, if a timer triggers a ZCOND event in Channel 1, then CZC should be set to 1h to register that a ZCOND event in channel 1 triggers the zero event.

Table 25-8. Counter Zero Control (CZC) settings

TIMx.CTRCTL.CZC	Setting
0h	Channel 0 ZCOND event zeroes the TIMx instance
1h	Channel 1 ZCOND event zeroes the TIMx instance
2h	Channel 2 ZCOND event zeroes the TIMx instance (4 CC timer only)
3h	Channel 3 ZCOND event zeroes the TIMx instance (4 CC timer only)
4h	2-input QEI mode. See Section 25.2.3.1.3
5h	3-input QEI mode. See Section 25.2.3.1.3

See [Table 25-9](#) for counter load control settings. For example, if a timer triggers a LCOND event in Channel 2, then CLC should be set to 2h to register that a LCOND event in channel 2 triggers the load event.

Table 25-9. Counter Load Control (CLC) settings

TIMx.CTRCTL.CLC	Setting
0h	Channel 0 LCOND event loads the TIMx instance
1h	Channel 1 LCOND event loads the TIMx instance
2h	Channel 2 LCOND event loads the TIMx instance (4 CC timer only)
3h	Channel 3 LCOND event loads the TIMx instance (4 CC timer only)
4h	2-input QEI mode. See Section 25.2.3.1.3
5h	3-input QEI mode. See Section 25.2.3.1.3

See [Table 25-10](#) for counter advance control settings. For example, if a timer triggers a ACOND event in Channel 3, then CAC should be set to 3h to register that a ACOND event in channel 3 triggers the advance event.

Table 25-10. Counter Advance Control (CAC) settings

TIMx.CTRCTL.CAC	Setting
0h	Channel 0 ACOND event advances the TIMx instance
1h	Channel 1 ACOND event advances the TIMx instance
2h	Channel 2 ACOND event advances the TIMx instance (4 CC timer only)
3h	Channel 3 ACOND event advances the TIMx instance (4 CC timer only)
4h	2-input QEI mode. See Section 25.2.3.1.3
5h	3-input QEI mode. See Section 25.2.3.1.3

25.2.3.1.1.4 CCP Input Filtering

The input glitch filter can be enabled by setting the TIMx. IFCTL_01[0/1].FE bit. The filter period is configured by setting the TIMx. IFCTL_01[0/1].FP bit.

A consecutive period or majority voting format selected by the TIMx. IFCTL_xy[0/1].CPV bit is used to select the criteria for a CCP input signal.

- **Consecutive period** - The CCP input signal must be at the specified level for the defined number of FP timer clocks for the CCP input to be processed.
- **Majority voting** - The filter ignores one clock of opposite logic over the filter period. For example, over the number of FP samples of the input, up to 1 sample can be of an opposite logic value (glitch) without affecting the output.

The example shown in [Figure 25-13](#) shows the difference between consecutive period and majority voting formats with a digital filter implemented to capture a CCP input signal of 3 TIMCLK periods.

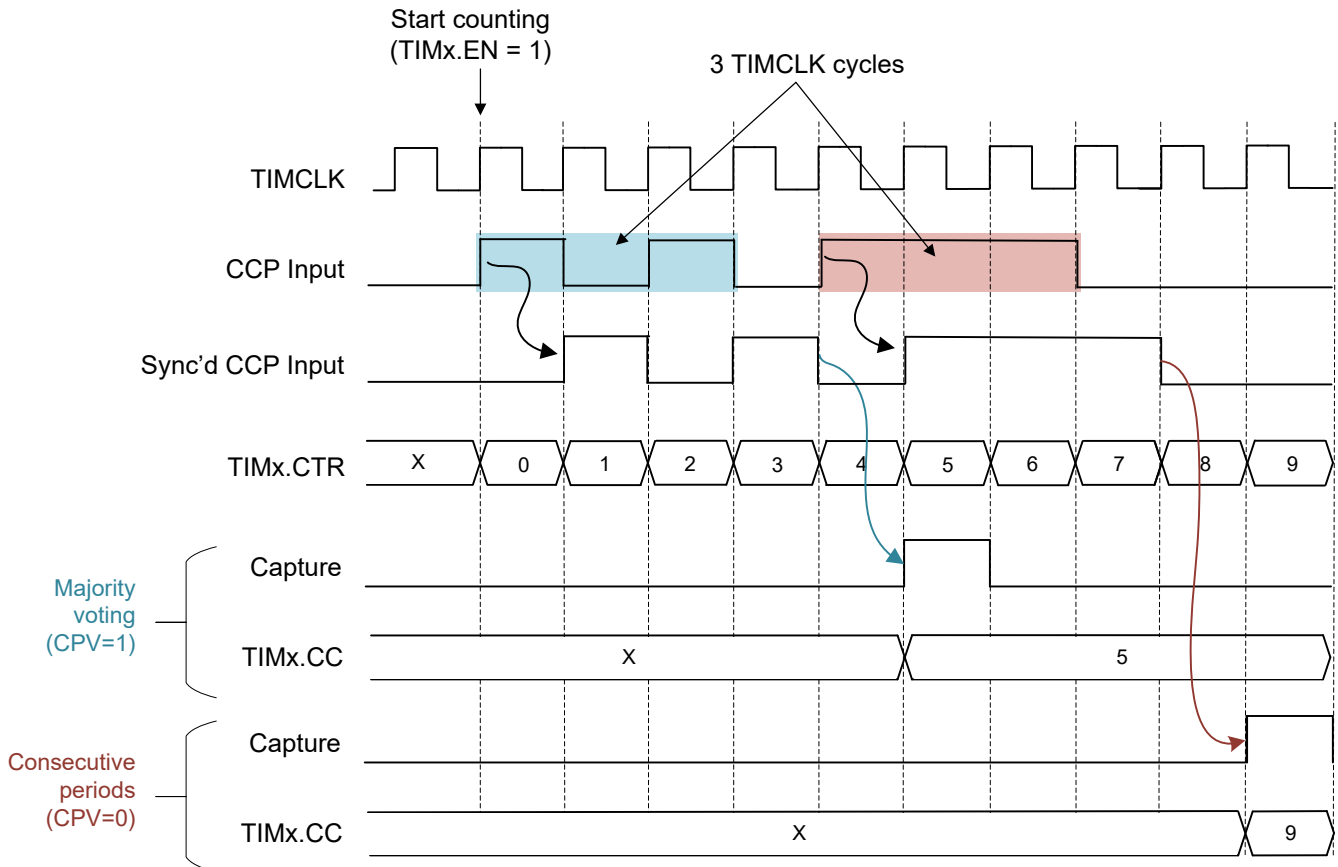


Figure 25-13. Consecutive Period and Majority Voting for Input Glitch Filtering using FP = 0 (3 TIMCLK cycles)

25.2.3.1.1.5 Input Selection

The input select bits TIMx.IFCTL_xy[0/1].ISEL select the input source to the filter input as the corresponding CCP input, CCP0 for cross-triggering across CC blocks, an external trigger, XOR (used in Hall input mode), event subscribers, or comparator inputs.

Table 25-11. Input Selection Options for TIMx CC Instances

Input selection (TIMx.IFCTL_xy[0/1].ISEL)	Source
0h	TIMx CCP of the corresponding capture compare unit
1h	Input pair CCPx of the capture compare unit. [CCP0/CCP1, CCP2/CCP3]
2h	TIMx CCP0
3h	Cross trigger signal
4h	XOR of CCP inputs. Used in Hall input mode. See Section 25.2.3.1.4
5h	Subscriber event 0 (FSUB0). See Section 25.2.9.2
6h	Subscriber event 1 (FSUB1). See Section 25.2.9.2
7h	COMP0_OUT (devices with comparator only)
8h	COMP1_OUT (devices with comparator only)
9h	COMP2_OUT (devices with comparator only)

25.2.3.1.2 Use Cases

Several different use cases can be achieved in capture mode and are discussed in the following sections.

25.2.3.1.2.1 Edge Time Capture

Edge time capture measures the time (in TIMCLK cycles) from the start of the capture operation to the signal edge. The counter is loaded when TIMx is enabled and counts with each TIMCLK period until the CCP edge is detected, which triggers the capture of the timer value and generates a capture event. The capture edge time is equivalent to the difference between the starting value of the counter and the capture value in TIMx.CC_xy[0/1] register.

Edge Time Capture Configuration

1. Set the TIMx.LOAD value.
2. In the CTRCTL register, set the desired counter control settings for:
 - a. Counting mode (CM) and counter value after enable (CVAE) (see as described in [Section 25.2.2](#))
 - b. Zero (CZC), advance (CAC), and load control (CLC) to specify what condition controls zeroing, advancing, or loading the counter
 - c. Repeat or one-shot mode (REPEAT)
3. Set TIMx.CCCTL_xy[0/1].COC = 1 for capture mode.
4. Configure CCP as an input for the CC block by setting respective bit in the CCPD registers. For instance, if TIMx Channel 0 is an input, set CCPD.C0CCP0 = 0.
5. For the corresponding CC block control register (CCCTL_01[0/1]), set CCOND to the corresponding setting to capture events based off the input signal condition (rising and/or falling edge). Additionally, set ZCOND or LCOND depending on the counting mode used.
6. Configure input capture settings in the TIMx.IFCTL_xy[0/1] register as described in [Section 25.2.3.1.1](#).
7. Enable the counter by setting EN = 1 or waiting for a capture event to occur from the input edge.

Example using up-counting mode for rising edge capture

In up-counting mode starting from zero (CM = 2, CVAE = 2), TIMx can be configured to generate a zero pulse and start the counter from the configured capture event (CCOND) by setting ZCOND to 1.

The expected internal timing for a rising or positive edge time capture in up-counting mode is shown in [Figure 25-14](#).

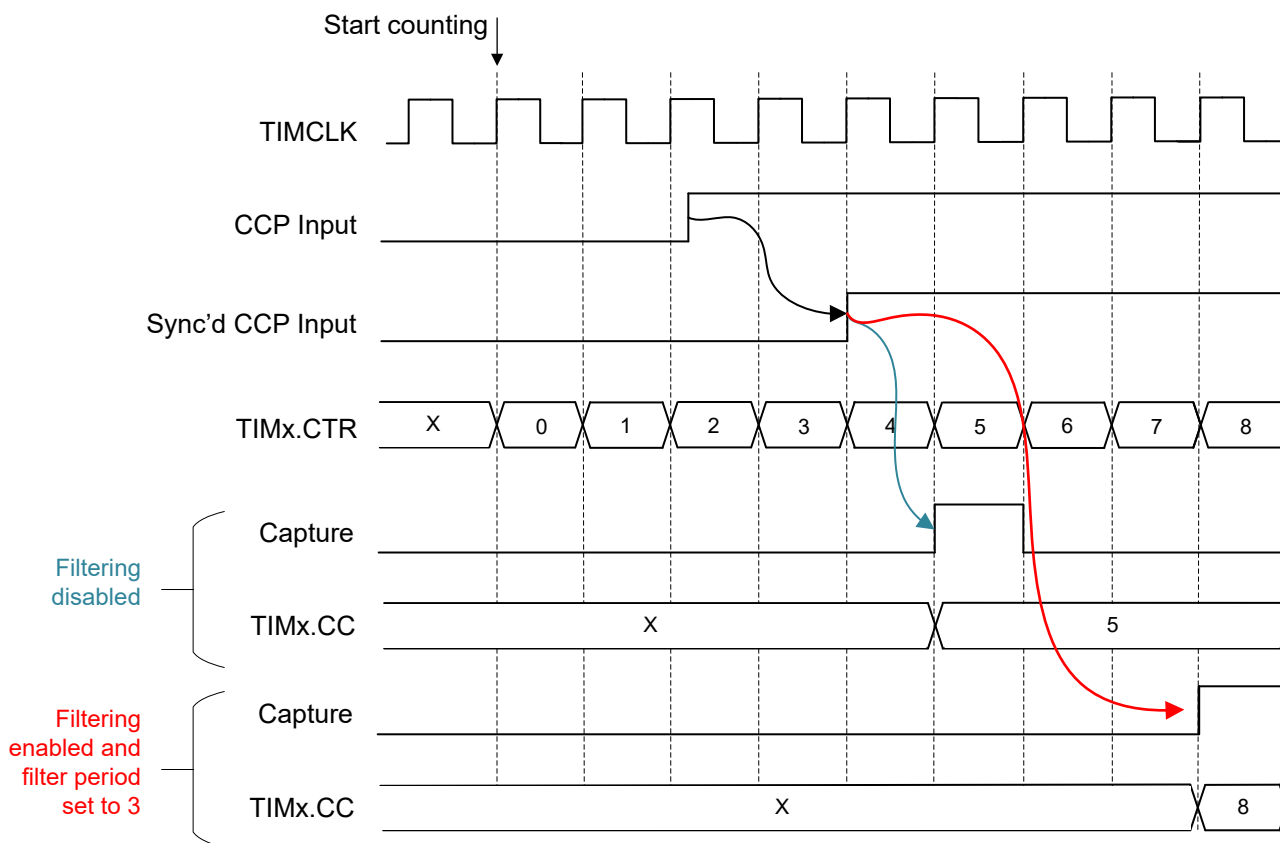


Figure 25-14. Edge Time Capture Mode in Up-Counting Mode, CVAE = 2

25.2.3.1.2.2 Period Capture

Period capture measures the period of a signal on an input CCP in TIMCLK cycles. On each positive (or negative) edge of the CCP input, the TIMx.CTR value is both captured into the TIMx.CC register to generate a capture event. The period capture time is equivalent to the difference between the starting value of the counter generated to the capture event, or time between reoccurring capture events for a periodic input signal.

Period Capture Configuration

1. Set the TIMx.LOAD value.
2. In the CTRCTL register, set the desired counter control settings for:
 - a. Counting mode (CM) and counter value after enable (CVAE) (see as described in [Section 25.2.2](#))
 - b. Advance (CAC) to specify what condition controls advancing the counter
 - c. Repeat or one-shot mode (REPEAT)
3. Set TIMx.CCCTL_xy[0/1].COC = 1 for capture mode.
4. Configure CCP as an input for the CC block by setting respective bit in the CCPD registers. For instance, if TIMx Channel 0 is an input, set CCPD.C0CCP0 = 0.
5. For the corresponding CC block control register (CCCTL_01[0/1]),
 - a. Set CCOND to the corresponding setting to capture events based off the input signal condition (rising and/or falling edge)
 - b. Set ZCOND or LCOND depending on the counting mode used
6. Configure input capture settings in the TIMx.IFCTL_xy[0/1] register as described in [Section 25.2.3.1.1](#).
7. Enable the counter by setting EN = 1 or waiting for a capture event to occur from the input edge.

Example using up-counting mode for rising-edge period capture

In up-counting mode starting from zero (CM = 2, CVAE = 2), TIMx channel 0 can be configured to generate a capture event from a rising edge input by setting CCOND = 1h. After enabling the counter, when a rising

edge input is detected, the counter will capture the counter value in TIMx.CC. After the capture event, set the TIMx.LOAD value back to 0 to reset the timer counter for the periodic CCP input signal.

The expected internal timing for a period capture in up-counting mode using two rising edges is shown in [Figure 25-14](#).

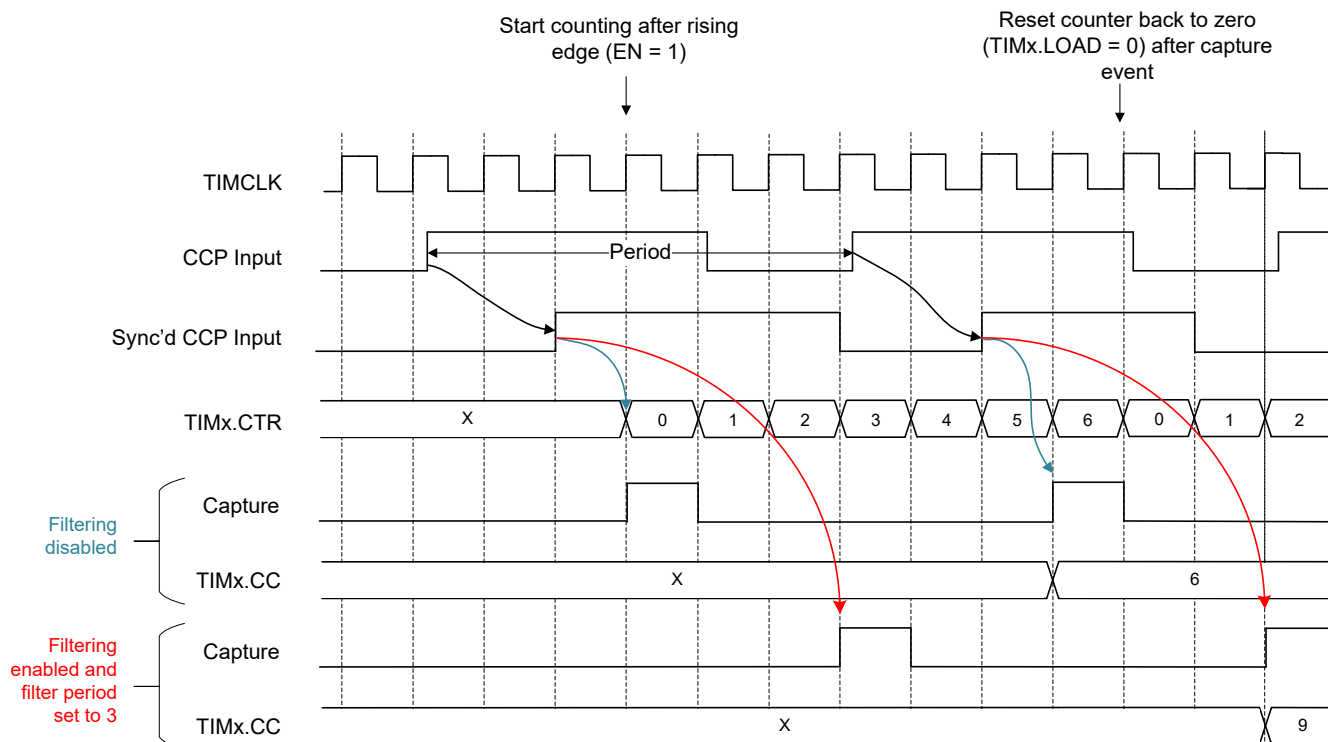


Figure 25-15. Period Capture Mode in Up-Counting Mode, CVAE = 2

25.2.3.1.2.3 Pulse Width Capture

Pulse width capture measures the high-time of a signal on CCP. The high time is the number of TIMCLK periods from rising edge to falling edge of the CCP input, and is useful for applications such as measuring the duty cycle of an PWM input signal. The counter is loaded at the positive edge and captured at the negative edge (capture event is generated).

Pulse-Width Capture Configuration

1. Set the TIMx.LOAD value.
2. In the CTRCTL register, set the desired counter control settings for:
 - a. Counting mode (CM) and counter value after enable (CVAE) (see as described in [Section 25.2.2](#))
 - b. Zero (CZC), advance (CAC), and load control (CLC) to specify what condition controls zeroing, advancing, or loading the counter
 - c. Repeat or one-shot mode (REPEAT)
3. Set TIMx.CCCTL_xy[0/1].COC = 1 for capture mode.
4. Configure CCP as an input for the CC block by setting respective bit in the CCPD registers. For instance, if TIMx Channel 0 is an input, set CCPD.C0CCP0 = 0.
5. For the corresponding CC block control register (CCCTL_01[0/1]), set CCOND to the corresponding setting to capture events based off the input signal condition (rising and/or falling edge). Additionally, set ZCOND or LCOND depending on the counting mode used.
6. Configure input capture settings in the TIMx.IFCTL_xy[0/1] register as described in [Section 25.2.3.1.1](#).
7. Enable the counter by setting EN = 1 or waiting for a capture event to occur from the input edge.

Example using up-counting mode for pulse width capture

In up-counting mode starting from zero (CM = 2, CVAE = 2), TIMx channel 0 can be configured to generate a zero pulse and start the counter from the configured capture event (CCOND) by setting ZCOND to 1. To start the counter, a load condition can be triggered from the CCP rising edge input by setting LCOND = 1.

The expected internal timing for a pulse width capture in up-counting mode using a rising and falling edge is shown in [Figure 25-14](#).

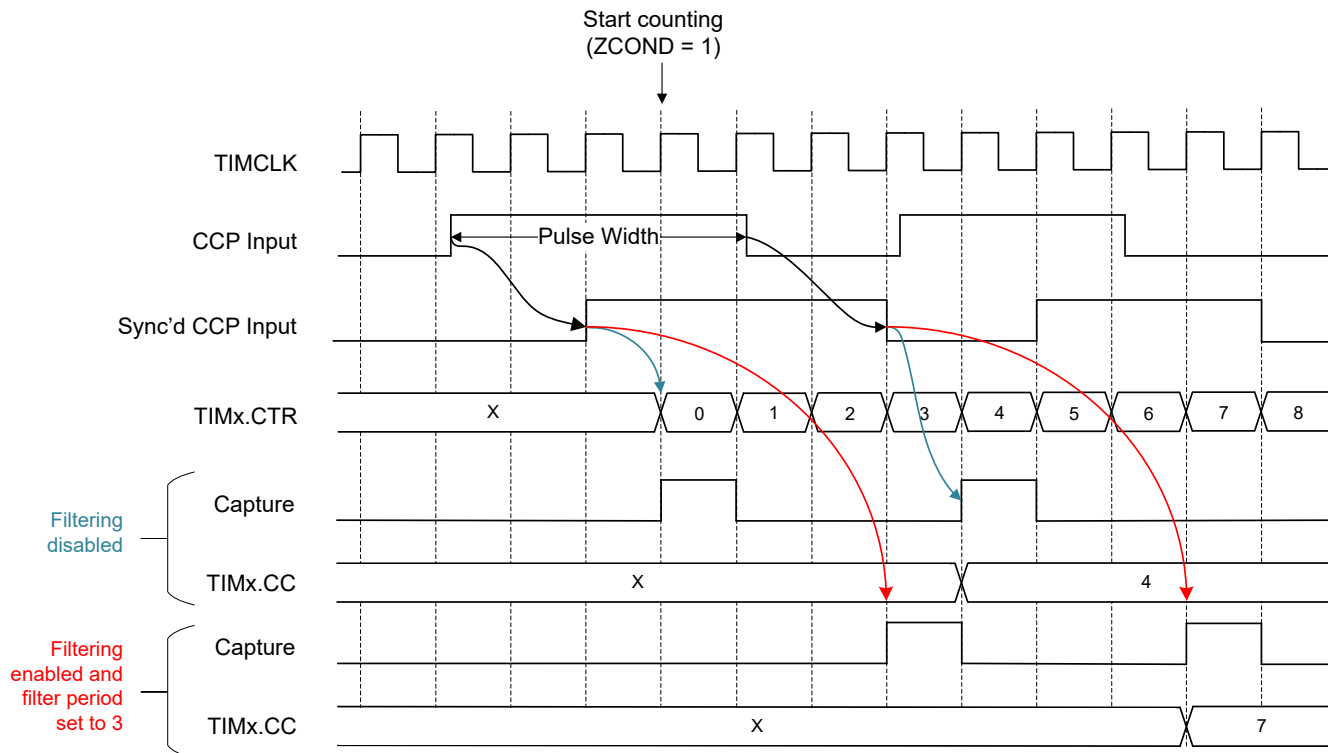


Figure 25-16. Pulse-Width Capture Mode

25.2.3.1.2.4 Combined Pulse Width and Period Time

Using two capture registers can combine pulse-width and period capture of a single input waveform. The input signal can be externally connected to CCP channel 0, and the IFCTL_01[1] register can be configured to have the input connected to CCP channel 1 internally so capture register 0 (TIMx.CC0) captures pulse width and capture register 1 (TIMx.CC1) captures period. The expected internal timing for combined pulse-width and period capture is shown in [Figure 25-17](#).

Pulse-Width and Period Capture Configuration

- Set the TIMx.LOAD value.
- In the CTRCTL register, set the desired counter control settings for:
 - Counting mode (CM) and counter value after enable (CVAE) (see as described in [Section 25.2.2](#))
 - Zero (CZC), advance (CAC), and load control (CLC) to specify what condition controls zeroing, advancing, or loading the counter
 - Repeat or one-shot mode (REPEAT)
- Set TIMx.CCCTL_xy[0/1].COC = 1 for capture mode for each CC channel.
- Configure CCP as an input for each CC block by setting respective bits in the CCPD register. For instance, if TIMx Channel 0 is an input, set CCPD.C0CCP0 = 0.
- For the corresponding CC block control register (CCCTL_01[0/1]),
 - Set CCOND to the corresponding setting to capture events based off the input signal condition (rising and/or falling edge)
 - Set ZCOND or LCOND depending on the counting mode used.

6. Configure input capture settings in the TIMx.IFCTL_xy[0/1] register as described in [Section 25.2.3.1.1](#).
7. Enable the counter by setting EN = 1 or waiting for a capture event to occur from the input edge.

Example using pulse-width and period time capture

In up counting mode, TIMx can be configured to generate a zero pulse and start the counter from the configured capture event (CCOND) by setting ZCOND to 1.

The expected internal timing for a pulse-width and period capture in up-counting mode using two CC blocks is shown in [Figure 25-14](#).

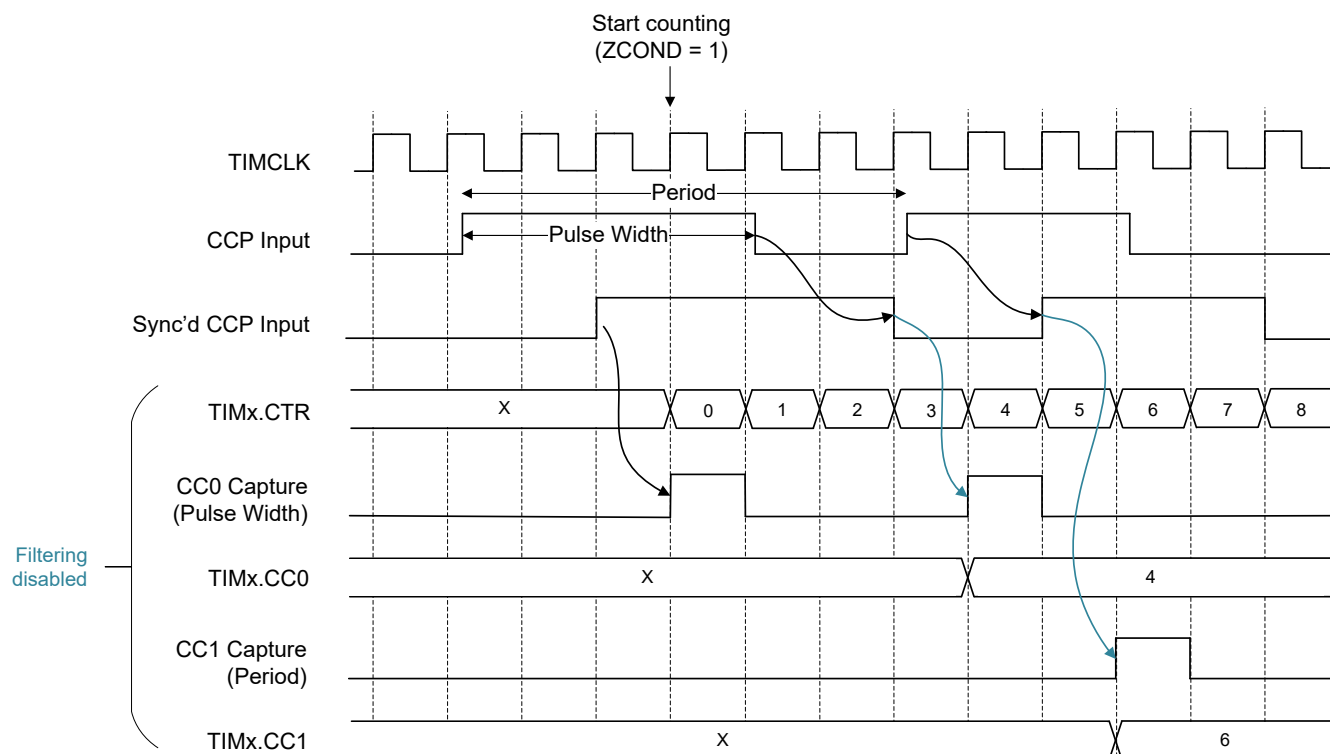


Figure 25-17. Combined Pulse-Width and Period Capture

25.2.3.1.3 QEI Mode (TIMG with QEI support only)

In TIMGx instances with QEI support, Quadrature Encoder Interface (QEI) mode provides an interface to the output of a quadrature encoder. It decodes the quadrature encoded data to provide the information on the relative positioning and movement of a linear or rotary motion.

The QEI consists of two Gray coded quadrature input signals PHA and PHB, and an index input signal IDX. All input signals go to CCP inputs of a single counter, such that PHA and PHB are mapped to CCP0 and CCP1, and IDX is brought in as a separate input. An error detection mechanism can report erroneous transitions to avoid improper signal decodings.

Note

See [Section 25.1.3](#) and the device-specific data sheet for TIMG instances that support QEI / Hall Input Mode.

25.2.3.1.3.1 QEI With 2-Signal

QEI is used to decode signals from optical position encoders. Optical position encoders typically output 2 signals (PHA/PHB) which are often used to measure rotary or linear movement of physical items with rotating shafts,

such as 3-phase motors. The measurement provided by the interface is incremental, which means as movement occurs, the interface provides the ability to capture the relative change from the previous position.

When operating in QEI mode, a counter accumulates the incremental updates, deriving current position from initial position and the accumulated change. The initial position can be determined by the signal of the IDX input (3-signal QEI mode) or by other software means (software directly setting the initial position). The capture and compare register can be used to store the position value by defining a capture condition.

When a direction change (DC) occurs, a DC interrupt is generated in the RIS register.

Figure 25-18 shows the state machine in the QEI interface to detect the directional rotation from the two CCP input signals, PHA and PHB.

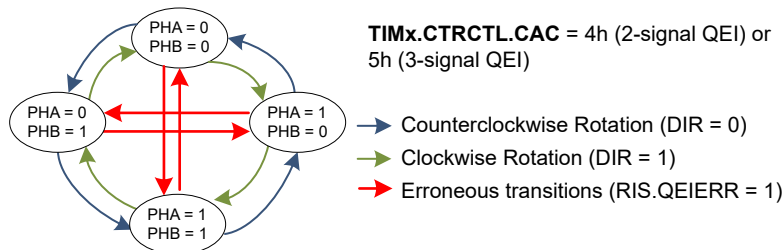


Figure 25-18. State Machine for 2-signal and 3-signal QEI mode

QEI 2-Signal Mode Configuration

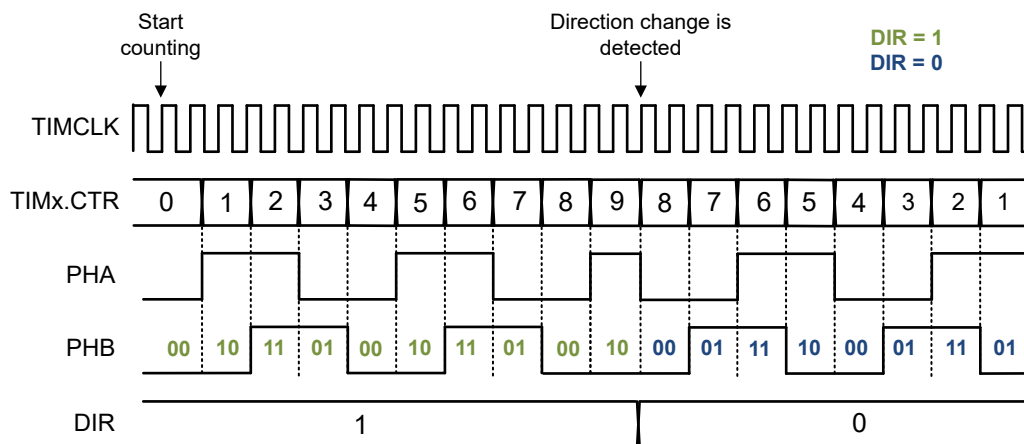
1. Configure PINCMx for TIMGx_C0 (PHA) and TIMGx_C1 (PHB).
2. Set TIMG.CCCTL_01[0].COC = 1 and TIMG.CCCTL_01[1].COC = 1 for capture mode for both CCP channels 0 and 1 (PHA and PHB).
3. Configure CCP as an input for each CC block by setting respective bits in the CCPD register. For instance, if TIMx Channel 0 is an input, set CCPD.C0CCP0 = 0.
4. Set the TIMx.LOAD to the encoder resolution, such as 4000.
5. In the CTRCTL register, set the CAC, CZC, and CLC bits to 4h.
6. Configure input capture settings as described in [Section 25.2.3.1.1](#), if desired.
7. Enable the counter to start counting by setting EN = 1.

Example using QEI mode with 2 input signals

The behavior of PHA/PHB follows [Table 25-12](#) and [Figure 25-19](#).

Table 25-12. PHA/PHB State Table and Counter Actions

Current State (PHA, PHB)	Next State (PHA, PHB)	Direction (DIR)	Counter Action
00	10	1 (Up)	+1 (If TIMx.CTR = LOAD, then CTR = 0)
10	11		
11	01		
01	00		
00	01	0 (Down)	-1 (If TIMx.CTR = 0, then CTR = TIMx.LOAD)
01	11		
11	10		
10	00		


Figure 25-19. 2-Signal QEI Operation

25.2.3.1.3.2 QEI With Index Input

3-signal QEI mode is similar to the 2-signal mode with additional index input signal IDX. Generally, IDX input pulses once per rotation which can be used to reset the counter. It is used in one of two applications:

- One IDX pulse is generated each movement cycle. In this case, the accumulated position represents the fraction of the movement cycle.
- An IDX pulse is generated at a specific position in a non-cyclic movement.

QEI 3-Signal Mode Configuration

1. Configure PINCMx for TIMGx_C0 (PHA), TIMGx_C1 (PHB), and TIMGx_IDX (IDX).
2. Set TIMG.CCCTL_01[0].COC = 1 and TIMG.CCCTL_01[1].COC = 1 for capture mode for both CCP channels 0 and 1 (PHA and PHB).
3. Configure CCP as an input for each CC block by setting respective bits in the CCPD register. For instance, if TIMx Channel 0 is an input, set CCPD.C0CCP0 = 0.
4. Set the TIMx.LOAD value to the encoder resolution, such as 4000.
5. In the CTRCTL register, set the CAC, CZC, and CLC bits to 5h.
6. Configure input capture settings as described in [Section 25.2.3.1.1](#), if desired.
7. Enable the counter to start counting by setting EN = 1.

The IDX input is sampled when a rising edge is detected. The IDX signal affects the counter value depending on the direction as shown in [Table 25-13](#) and

Table 25-13. Relation of IDX Input and Counter Value

Direction (DIR)	IDX	Counter Action
1	Rising	Zero (TIMx.CTR is set to zero)
0	Rising	Load (TIMx.CTR is set to TIMx.LOAD)

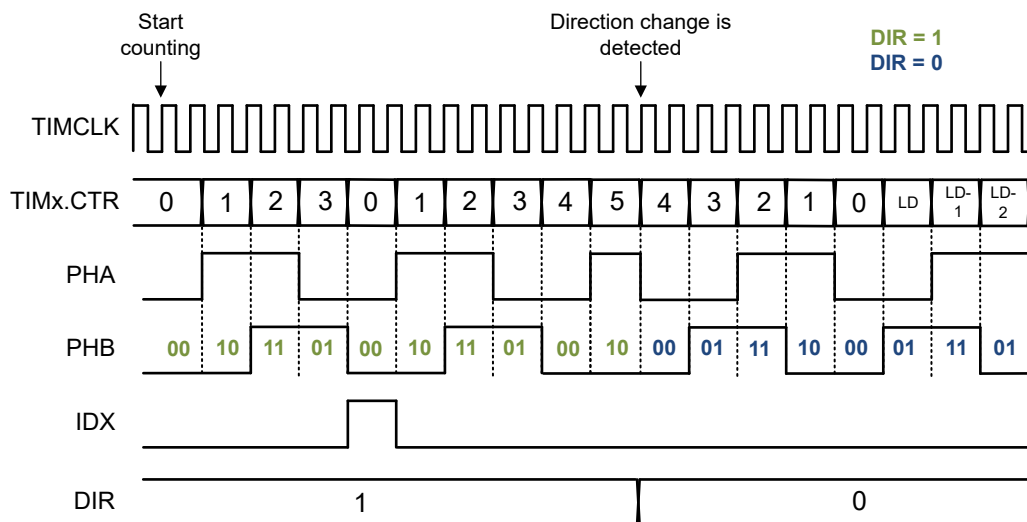


Figure 25-20. 2-Signal QEI with Index Input Operation

25.2.3.1.3.3 QEI Error Detection

The QEI module can detect erroneous transactions or state errors as shown in [Figure 25-18](#). A QEIERR interrupt is generated and the counter or direction signal does not change in the error state.

Note

QEIERR can occur if the TIMCLK period is slower than the period of the PHA or PHB signals.

25.2.3.1.4 Hall Input Mode (TIMG with QEI support only)

In TIMGx instances with QEI support, three digital Hall signals can be input into CCP channel 0 (CCP0), CCP channel input 1 (CCP1), and IDX for position control of 3-phase Hall-sensored motor applications. Hall signals are used to detect real-time motor position in motor control applications and can be used for speed computation measurements, position control, or motor stall status.

Note

See [Section 25.1.3](#) and the device-specific data sheet for TIMG instances that support QEI / Hall Input Mode.

[Table 25-14](#) shows the signal mapping for Hall signals A (U), B (V), and C (W) to TIMG capture/compare input signals.

Table 25-14. Hall Input and TIMx Input Signal Mapping

Hall input signal	TIMx input
HALL A / HALL U	CCP0
HALL B / HALL V	CCP1
HALL C / HALL W	IDX

Note

Hall input signals should be digital inputs from Hall sensor ICs with a pullup to VCC.

As shown in [Figure 25-21](#), the input capture module provides a 3-input XOR of synced CCP0, CCP1, and IDX signals to create a frequency generator (FG) signal. The XOR output signal is selected when IFCTL_xy[0/1].ISEL is set to 4h. See [Figure 25-21](#) for XOR option in the input capture block diagram.

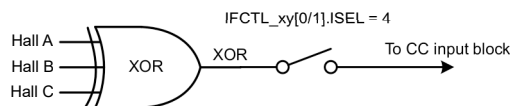


Figure 25-21. Hall 3-input XOR for Frequency Generator (FG) Signal to CC input block

The XOR'ed output signal is propagated to the CC block and a period or pulse-width capture can be used to compute the linear motor speed in relation to the calculated period or pulse width in the TIMx.CC register. See [Section 25.2.3.1.2.2](#) and [Section 25.2.3.1.2.3](#) on how to calculate period and pulse-width captures based on the XOR'ed input signal.

Figure 25-22 shows the input signal to the CC block which can be used for speed calculations.

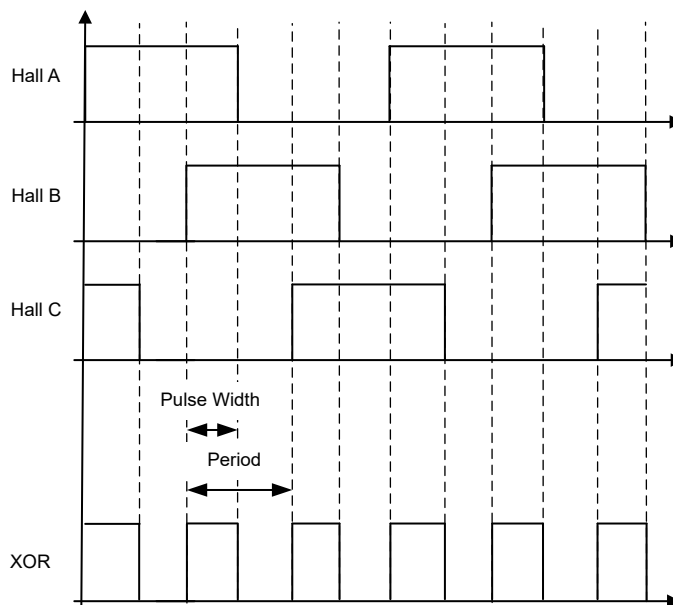


Figure 25-22. Hall 3-Input XOR Signal used with Pulse Width or Period Capture for Speed Computation

Hall Input Mode Configuration

1. Configure PINCMx for TIMGx_C0 (HALLA), TIMGx_C1 (HALLB), and TIMGx_IDX (HALLC).
2. Set TIMG.CCCTL_01[0].COC = 1 and TIMG.CCCTL_01[1].COC = 1 for capture mode for both CCP channels 0 and 1 (HALLA and HALLB).
3. Configure CCP as an input for each CC block by setting respective bits in the CCPD register. For instance, if TIMx Channel 0 is an input, set CCPD.C0CCP0 = 0.
4. Set the TIMx.LOAD value.
5. Set TIMG.IFCTL_xy[0/1].ISEL = 4h to select the XOR option for Hall signals.
6. Enable the counter to start counting by setting EN = 1.

25.2.3.2 Compare Mode

Compare mode is selected when TIMx.CCCTL_xy[0/1].COC = 0. Compare mode is used to generate event or PWM output signals at specific time intervals.

Many types of compare mode events can be generated based on the configuration of the CC action control register CCACT_xy[0/1]. A compare event occurs for a CC channel when TIMx.CTR counts up (CCU) or down (CCD) to the value in TIMx.CC_xy[0/1]. Additionally, secondary compare up (CC2U) or down (CC2D) events for a CC channel can be generated from another CC block's CCU or CCD event with respect to its TIMx.CC_xy[0/1] value in the same timer instance. Compare events can be used to generate a timing base internally or generate a PWM output with specific profiles using active, inactive, or toggle action behaviors.

Note

Secondary compare events in up/down counting mode can be used to generate PWM signals with twice the PWM frequency (but half the PWM resolution).

In TIMA only, an additional internal 5th and 6th CC block (TIMA.CC_45[0/1]) can be used for secondary compare events while continuing to use CC channels with dedicated output pins for external PWM signal generation.

Table 25-15 shows the types of compare mode events that can be generated and conditions to generate the events.

Table 25-15. Compare Mode Events

Event	Name	Event condition
CCDn (n = CC channel)	Capture/compare down event	When timer is counting down, TIMx.CTR = TIMx.CC_xy[0/1]
CCUn (n = CC channel)	Capture/compare up event	When timer is counting up, TIMx.CTR = TIMx.CC_xy[0/1]
CC2Dn (n = CC channel)	Secondary capture/compare down event	Occurs when CC2SELD is configured for the source of the CCDn event
CC2Un (n = CC channel)	Secondary capture/compare up event	Occurs when CC2SELU is configured for the source of the CCUn event

Note

Look at the device specific data sheet to check how many CC channels are available in each TIMx instance on the device.

25.2.3.2.1 Edge Count

In addition to event or PWM output generation, compare mode can also be used for input signal edge counting to determine when a number of edges has been detected. In edge count operation, a CCP input edge can advance the counter based on the ACOND condition. The counter register is initialized with the starting value, and the number of detected CCP input edges at any time can increment or decrement depending on the counting mode configuration. The user can count rising edges, falling edges, or both edges by configuring the CCOND value.

Edge Count Configuration

1. Set TIMx.CCCTL_xy[0/1].COC = 0 for compare mode.
2. Optionally set the corresponding TIMx.CC_xy[0/1] to a compare value to generate a compare interrupt when the counter reaches this value.
3. In the CTRCTL register, set the desired counter control settings for:
 - a. Counting mode (CM) and counter value after enable (CVAE) (see as described in Section 25.2.2)
 - b. Zero (CZC), advance (CAC), and load control (CLC) to specify what condition controls zeroing, advancing, or loading the counter
 - c. Repeat or one-shot mode (REPEAT)
4. Set ACOND to a setting to advance the counter based on the input edge polarity.
5. Configure input capture settings as described in Section 25.2.3.1.1, if desired.
6. Enable the counter by setting EN = 1.

Example using edge count operation using up-counting mode

In up-counting mode starting from zero (CM = 2, CVAE = 2), the expected internal timing for rising edge count operation to increment the counter is shown in Figure 25-23.

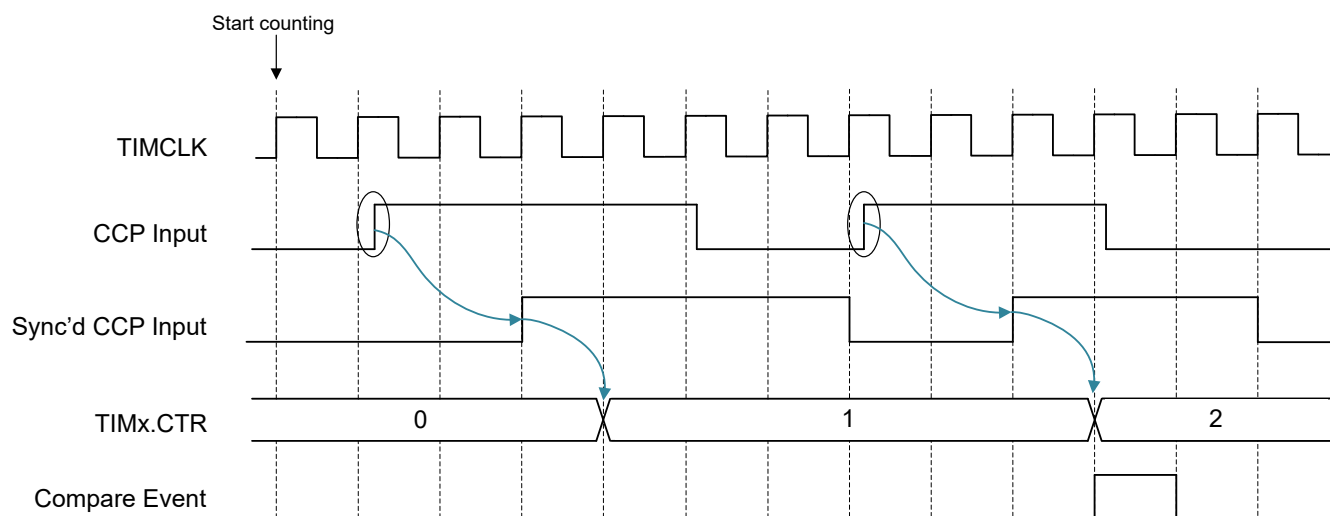


Figure 25-23. Edge Count Operation to Generate Compare Event (TIMx.CC = 2)

25.2.4 Shadow Load and Shadow Compare

Some timer modules have a shadow load and shadow compare register feature which gives the user the flexibility of holding the update of load and CC values until a certain event occurs. This is useful in timing-critical applications where PWM control signals need to be updated with correct timings, such as duty cycle updates. Refer to the timer features for specific configurations of the TIMx modules.

Note

See [Section 25.1.3](#) and the device-specific data sheet for TIMx instances that support Shadow Load and Shadow Compare.

25.2.4.1 Shadow Load

The shadow load feature allows holding the update of load values until a zero event occurs. To enable shadow loading, set the TIMx.GCTL.SHDWLDEN bit.

If the TIMx module has a shadow load feature, there is an internal shadow register for the load value (TIMx.LOAD). The shadow register will update the load value at a zero event as shown in [Figure 25-24](#).

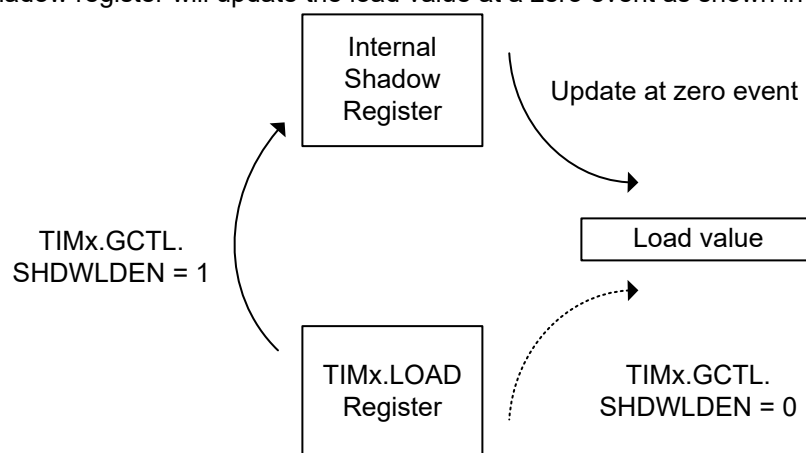


Figure 25-24. Shadow Load Update Load Value

When TIMx.GCTL.SHDWLDEN = 1, load values update at zero events for all counting modes. Consult the counting mode operations below to determine if a shadow load is needed:

- In down-counting modes, since TIMx.LOAD value is updated when a zero event occurs, a shadow load is not needed in these modes.
- In up/down counting mode, TIMx.LOAD is compared with the counter value to determine if the peak is reached and when to start to counting down. A shadow load is necessary to ensure that TIMx counts up to the load value before the zero event, or else the load value can update immediately and cause incorrect timings.
- In up-counting mode, the timer counts to TIMx.LOAD. A shadow load is necessary to ensure that TIMx counts up to the load value before the zero event, or else the load value can update immediately and cause incorrect timings.

Figure 25-25 shows an example of how shadow load and shadow compare takes effect at the zero event for both the TIMx.LOAD and TIMx.CC value in up/down counting mode.

25.2.4.2 Shadow Compare

When shadow compare is enabled for updating the capture/compare register (TIMx.CC), the value written to the respective compare register is first stored into a shadow compare register and then transferred to the compare register at different events configured by setting the TIMx.CCCTL_xy[0/1].CCUPD bits.

Additionally, the capture/compare action register (TIMx.CCACT) has the ability to update the action at different events configured by setting the TIMx.CCCTL_xy[0/1].CCACTUPD bits.

Table 25-16 shows the settings for configuring when shadow compare and actions occur at different events.

Table 25-16. Shadow Compare and Action Update Behavior

Bit Field	Value	Description/Comment
CCUPD / CCACTUPD	0	The value written to TIMx.CC register take effect immediately.
	1	The value written to the TIMx.CC register is stored in a shadow compare register and gets transferred to the TIMx.CC register in the TIMCLK cycle following a zero event (TIMx.CTR value equals 0).
	2	The value written to the TIMx.CC register is stored in a shadow compare register and gets transferred to the TIMx.CC register in the TIMCLK cycle following a compare (down) event (TIMx.CTR value equals TIMx.CC)
	3	The value written to the TIMx.CC register is stored in a shadow compare register and gets transferred to the TIMx.CC register in the TIMCLK cycle following a compare (up) event (TIMx.CTR value equals TIMx.CC)
	4	The value written to the TIMx.CC register is stored in a shadow compare register and gets transferred to the TIMx.CC register in the TIMCLK cycle following a zero or load event (TIMx.CTR value equals 0 or TIMx.CTR equals TIMx.LOAD). Note: this update mechanism is defined for use only in up/down counting mode.
	5	The value written to the TIMx.CC register is stored in a shadow compare register and gets transferred to the TIMx.CC register in the TIMCLK cycle following a zero event and the repeat count equaling zero (TIMx.CTR value equals 0 and TIMx.RC equals 0)
	6	The value written to the TIMx.CC register is stored in a shadow compare register, and gets transferred to the TIMx.CC register in the TIMCLK cycle following a trigger pulse. See Section 25.2.7 .

Figure 25-25 shows an example of how shadow load and shadow compare takes effect at the zero event for both the TIMx.LOAD and TIMx.CC value in up/down counting mode.

TIMx.GCTL.SHDWLDEN = 1

TIMx.CCCTL_xy[0/1].CCUPD = 1h (update CC register after zero event)

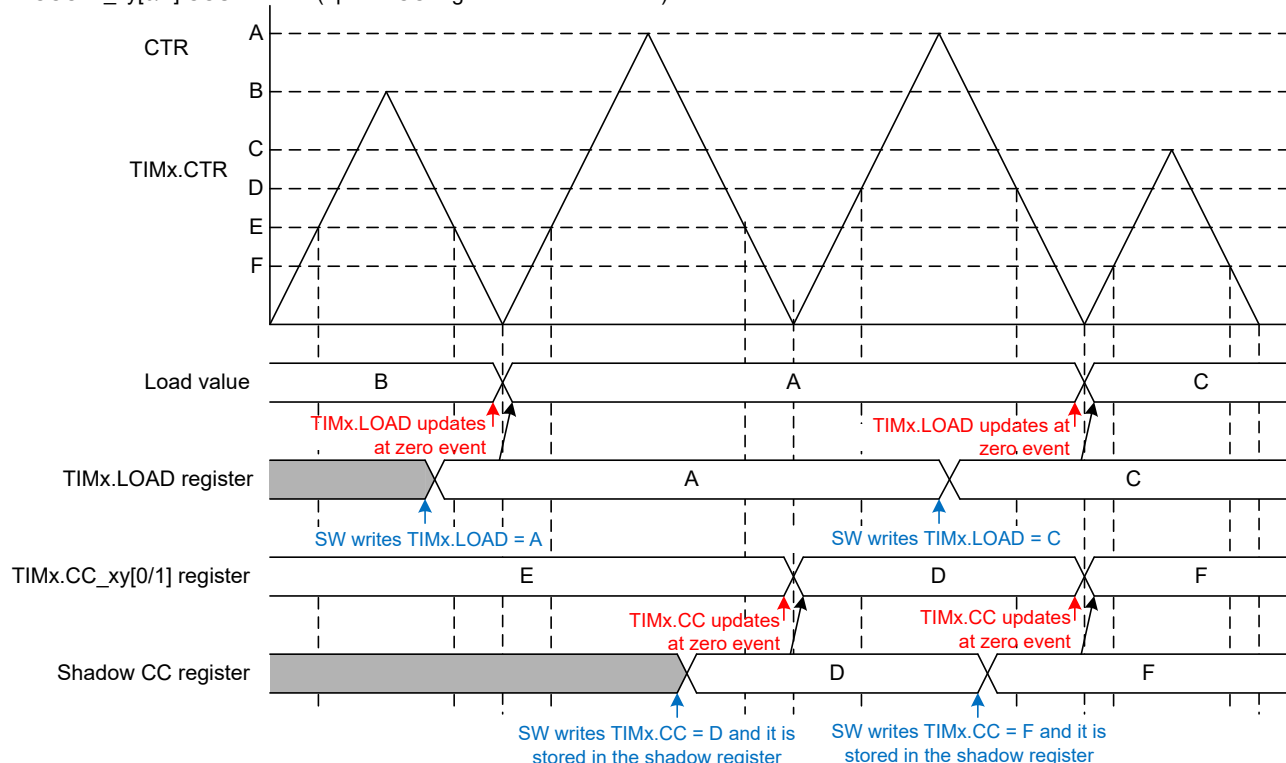


Figure 25-25. Shadow Load and Shadow Compare Taking Effect at Zero Event in Up/Down Mode

25.2.5 Output Generator

The output signal generation unit can be used with the counter and capture/compare modules to generate desired pulse-width modulation (PWM) output waveforms, event signals, synchronized capture inputs, or the counter direction. Many output waveforms are generated from counter events (load, zero, counter direction) and the capture/compare block (compare match).

TIMA and TIMG have many common features in the output generation signal unit. Additionally, TIMA has advanced output generation features such as complimentary output signals, deadband insertion, and fault generation.

Figure 25-27 shows the TIMG output block diagram.

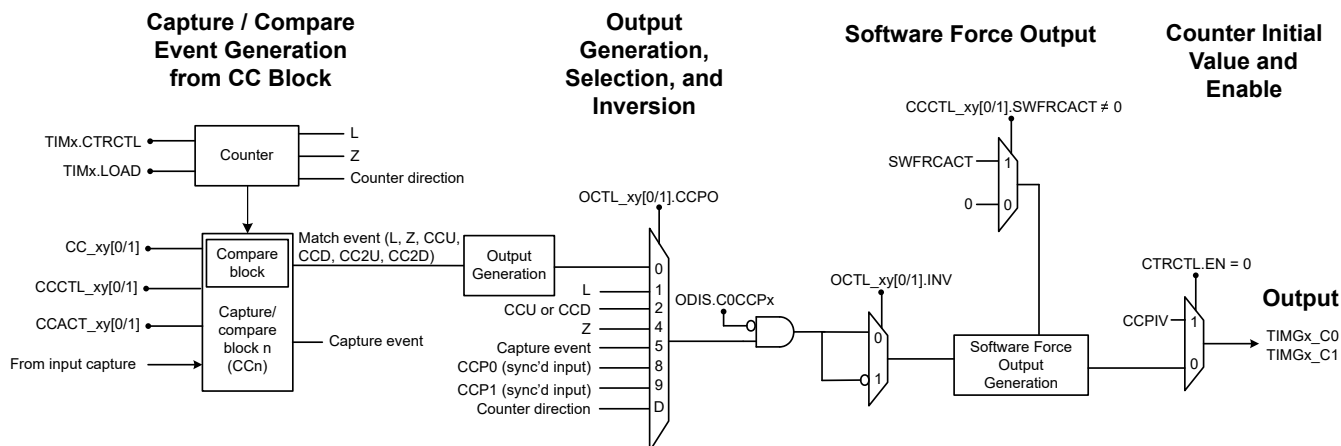


Figure 25-26. Output Connection for TIMG

Figure 25-27 shows the TIMA output block diagram.

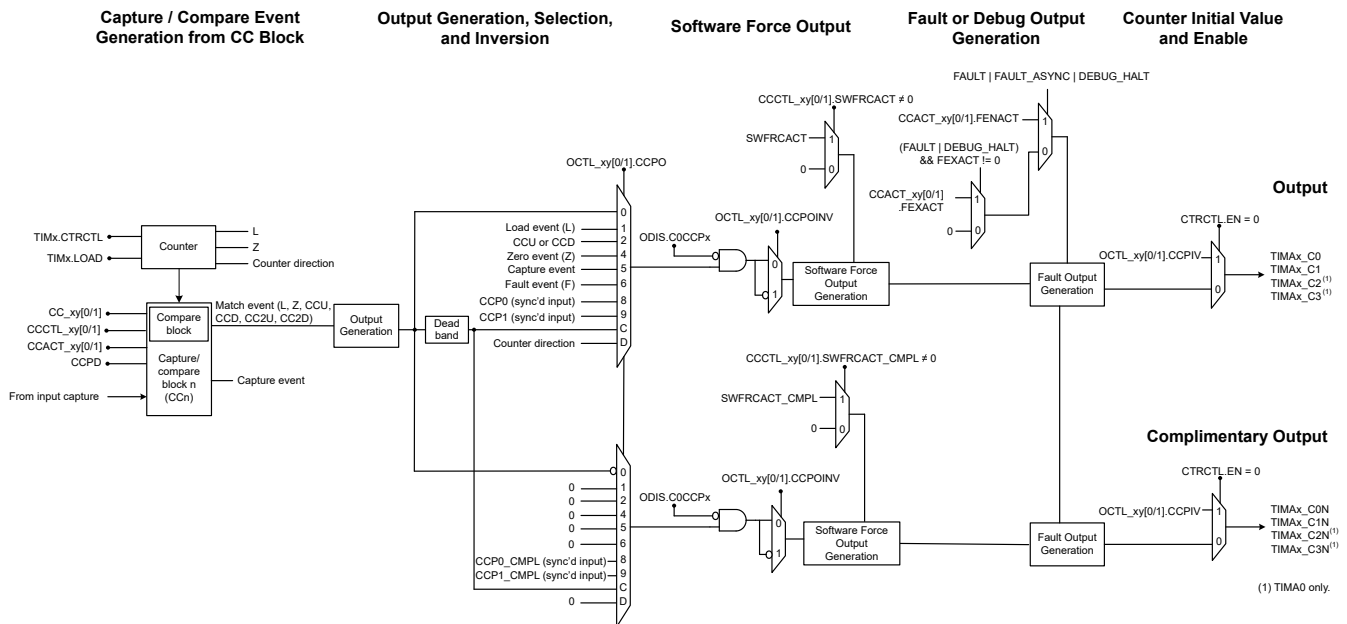


Figure 25-27. Output Connection for TIMA

Signal Generator Actions

Table 25-17 shows the types of signal generator actions capable by the output generator. Signal generator actions are configured in the CCACT_xy[0/1] register for zero, load, and compare events. For types of compare events, see Table 25-15.

Table 25-17. Signal Generator Actions from Compare Event

Value	Action
0h	Event is disabled and a lower priority event is selected if asserting
1h	CCP output value is set high
2h	CCP output value is set low
3h	CCP output value is toggled

The key registers for generation of output signals are:

- LOAD:** the contents of this register are copied to the counter (TIMx.CTR) on any operation designated to do a "load". This value is also used to compare with the counter value for generating a "Load Event" that can be used for interrupt, trigger, or signal generator actions.
- CCPD:** this register configures the direction of the CCP pins as inputs or outputs.
- CC_xy[0/1]:** this is a register used as a compare value to the current counter to create an match event.
- CTRCTL:** this register provides control over the counter operation in different conditions.
- CCCTL_xy[0/1]:** this register controls the operations of the respective CC registers and the counter
- OCTL_xy[0/1]:** this register controls the output of the capture-compare portion of the counter. This includes the ability to select the source of what is driven out along with initial condition values and final inversion options.
- CACT_xy[0/1]:** this register controls the actions of the signal generator of the capture-compare portion based on the events created in the counter block, the capture and compare block, and debug events.
- ODIS:** this register disables the output signal selected by OCTL.CCPO (before conditional inversion) to allow software the ability to hold the CCP output low during configuration or shutdown.

These are key registers for configuring the compare mode to generate PWM signals:

25.2.5.1 Configuration

There are five stages to configuring output signal generation in TIMx devices:

- Counter and CC Block Event Generation
- Output Generation, Selection and Inversion
- Software Force Output
- Fault Output Generation (TIMA only)
- Counter Initial Value and Enable

Counter and CC Block Event Generation

The counter block contains the counter and produces a load event (L), zero event (Z), and direction of counting based on the counting mode used.

The CC blocks contain the CC register and can generate two types of output signals: compare match events and capture events. Please see [Table 25-15](#) for the compare events that can be generated.

Output Generation, Selection and Inversion

The TIMx.CCACT register specifies the waveform generation of a CCP output depending on the counting mode and counter compare actions.

TIMx.OCTL_xy[0/1].CCPO controls the CCP output selection from the output generation unit, output generation unit with deadband (TIMA only), counter events, compare events, capture events, fault events, or signal inputs. The output disable register (ODIS) can optionally disable the CCP output to optionally hold the CCP output low during configuration or shutdown. TIMx.OCTL_xy[0/1].INV controls final inversion options.

Note

Mux selections for synchronized inputs are tied to 0 for TIMAx CC2 and CC3 instances. Do not use TIMAx.OCTL_23[0/1].CCPO = 8 or 9.

On TIMA devices only, CCP complimentary output channels can be generated from the output generation unit (denoted by "N" in the signal name). For instance, TIMA0 channel 2 (TIMA0_C2) can also produce a complimentary output (TIMA0_C2N). The CCPO and INV bits also controls the selection and inversion options for the complementary output.

Complimentary outputs with deadband insertion are a common use case for inverter-based applications with half-bridge topologies. For more information, please see [Section 25.2.5.2.4](#).

Software Force Output

The output of the signal generator can be overwritten in software by setting CCCTL_xy[0/1].SWFRACT to a nonzero setting. For TIMA devices only, the complementary output of the signal generator can be overwritten by setting CCCTL_xy[0/1].SWFRACT_CMPL to a nonzero setting.

For more information, see [Section 25.2.5.3](#).

Fault / Debug Output Generation (TIMA only)

On TIMA devices, the CCP output can be overwritten after the software force output block if there is a system fault (FAULT), fault condition upon exit (FEXACT), fault condition upon entry (FENACT), an asynchronous fault (FAULT_ASYNC), or the debugger is halted (DEBUG_HALT).

For more information, see [Section 25.2.6](#) and [Section 25.2.10](#).

Counter Compare Initial Value and Enable

To specify an initial value for the CCP output while the counter is disabled, set OCTL_xy[0/1].CCPIV to 0 for a low value or 1 for a high value. This is useful for applications where CCP outputs need to be in a default state before enabling the counter.

To enable the counter, set TIMx.CTRCTL.EN to 1.

25.2.5.2 Use Cases

Several different use cases can be achieved with the output generator and are discussed in the following sections.

25.2.5.2.1 Edge-Aligned PWM

To generate edge-aligned PWMs, TIMx can be configured for up- or down-counting mode and the PWM period in TIMCLK cycles is $TIMx.LOAD + 1$. The waveform uses load, zero, and compare events to drive the CCPx output high or low depending on the configuration settings of the compare/capture block and counter.

Edge-Aligned PWM Configuration

To generate edge-aligned PWMs using compare match events from the counter:

1. In the TIMx.CTRCTL register, set the desired counter control settings for:
 - a. Up-counting ($CM = 2$) or down-counting mode ($CM = 0$) and counter value after enable (CVAE) (see as described in [Section 25.2.2](#))
 - b. Zero (CZC), advance (CAC), and load control (CLC) to specify what condition controls zeroing, advancing, or loading the counter
 - c. Repeat or one-shot mode (REPEAT)
2. Set the TIMx.LOAD value to configure the PWM period.
3. Set the TIMx.CC_xy[0/1] value to configure the duty cycle.
4. Set TIMx.CCCTL_xy[0/1].COC = 1 for compare mode.
5. Configure CCP as an output for the CC block by setting respective bit in the CCPD registers. For instance, if TIMx Channel 0 is an output, set CCPD.C0CCP0 = 1.
6. In TIMx.CCACT_xy[0/1], set the CCP output action settings for compare events, zero events, load events, software force action, or fault events (TIMA only).
7. In TIMx.OCTL_xy[0/1], set CCPO = 0 to select the signal generator output.
8. Enable the corresponding CCP output by setting ODIS.C0CCPn to 1 for the corresponding counter n.
9. Configure polarity of the signal using the CCPOINV bit, and configure CCPIV to specify the CCP output state while disabled.
10. Enable the counter by setting TIMx.CTRCTL.EN = 1.

Example using edge-aligned PWM in down-counting mode

A typical 2-channel edge-aligned PWM generation for down-counting mode is shown in [Figure 25-28](#) with the following edge-aligned PWM output waveforms:

- CCP0 output generates:
 - High pulse-width from TIMx.LOAD to TIMx.CC0 value (LACT = 1h)
 - Low pulse-width from TIMx.CC0 value to zero (CDACT = 2h)
- CCP1 output generates:
 - High pulse-width from TIMx.LOAD to TIMx.CC1 value (LACT = 1h)
 - Low pulse-width from TIMx.CC1 value to zero (CDACT = 2h)

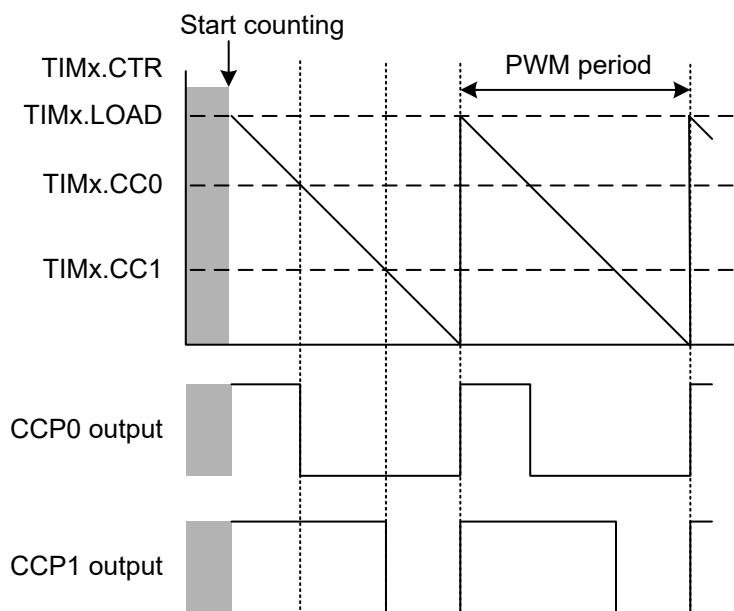


Figure 25-28. Edge-Aligned PWM Signals in Down-Counting Mode

Example using edge-aligned PWM in up-counting mode

A typical 2-channel edge-aligned PWM generation for up-counting mode is shown in [Figure 25-28](#) with the following edge-aligned PWM output waveforms:

- CCP0 output generates:
 - High pulse-width from zero to TIMx.CC0 value (ZACT = 1h)
 - Low pulse-width from TIMx.CC0 value to TIMx.LOAD (CUACT = 2h)
- CCP1 output generates:
 - High pulse-width from zero to TIMx.CC1 value (ZACT = 1h)
 - Low pulse-width from TIMx.CC1 value to TIMx.LOAD (CUACT = 2h)

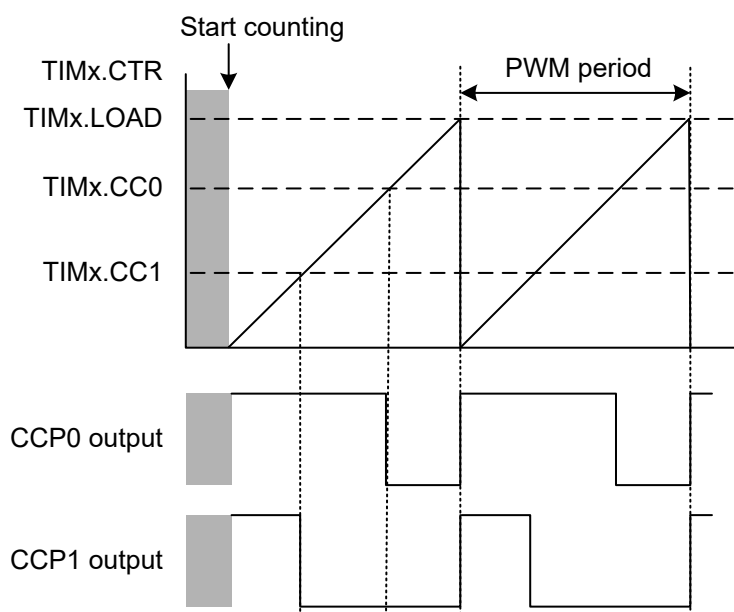


Figure 25-29. Edge-Aligned PWM Signals in Up-Counting Mode

25.2.5.2.2 Center-Aligned PWM

To generate center-aligned PWMs, TIMx is configured for up/down counting mode and the TIMx.LOAD value contains the half-period. The waveform uses up compare events and down compare events to drive the CCPx output high or low depending on the configuration settings of the compare/capture block and counter.

In TIMCLK cycles, the PWM period is $(2 * \text{TIMx.LOAD})$ and the duty cycle is $1 - (\text{TIMx.CC_xy}[0/1] / \text{TIMx.LOAD})$.

Center-Aligned PWM Configuration

To generate center-aligned PWMs using compare match events from the counter:

1. In the TIMx.ctrctl register, set the desired counter control settings for:
 - a. Up/down counting mode (CM = 1) and counter value after enable (CVAE) (see as described in [Section 25.2.2](#))
 - b. Zero (CZC), advance (CAC), and load control (CLC) to specify what condition controls zeroing, advancing, or loading the counter
 - c. Repeat or one-shot mode (REPEAT)
2. Set the TIMx.LOAD value to configure the PWM period.
3. Set the TIMx.CC_xy[0/1] value to configure the duty cycle.
4. Set TIMx.CCCTL_xy[0/1].COC = 1 for compare mode.
5. Configure CCP as an output for the CC block by setting respective bit in the CCPD registers. For instance, if TIMx Channel 0 is an output, set CCPD.C0CCP0 = 1.
6. In TIMx.CCACT_xy[0/1], set the CCP output action settings for compare events, zero events, load events, software force action, or fault events (TIMA only).
7. In TIMx.OCTL_xy[0/1], set CCPO = 0 to select the signal generator output.
8. Enable the corresponding CCP output by setting ODIS.C0CCPn to 1 for the corresponding counter n.
9. Configure polarity of the signal using the CCPOINV bit, and configure CCPIV to specify the CCP output state while disabled.
10. Enable the counter by setting TIMx.ctrctl.EN = 1.

Example using center-aligned PWM in up/down counting mode

A typical 2-channel center-aligned PWM generation using up/down counting mode is shown in [Figure 25-30](#) with the following center-aligned PWM output waveforms:

- CCP0 output generates:
 - High pulse-width from TIMx.CC0 compare up event to TIMx.CC0 compare down event (CUACT = 1h)
 - Low pulse-width from TIMx.CC0 compare down event to TIMx.CC0 compare up event (CDACT = 2h)
- CCP1 output generates:
 - High pulse-width from TIMx.CC0 compare up event to TIMx.CC0 compare down event (CUACT = 1h)
 - Low pulse-width from TIMx.CC0 compare down event to TIMx.CC0 compare up event (CDACT = 2h)

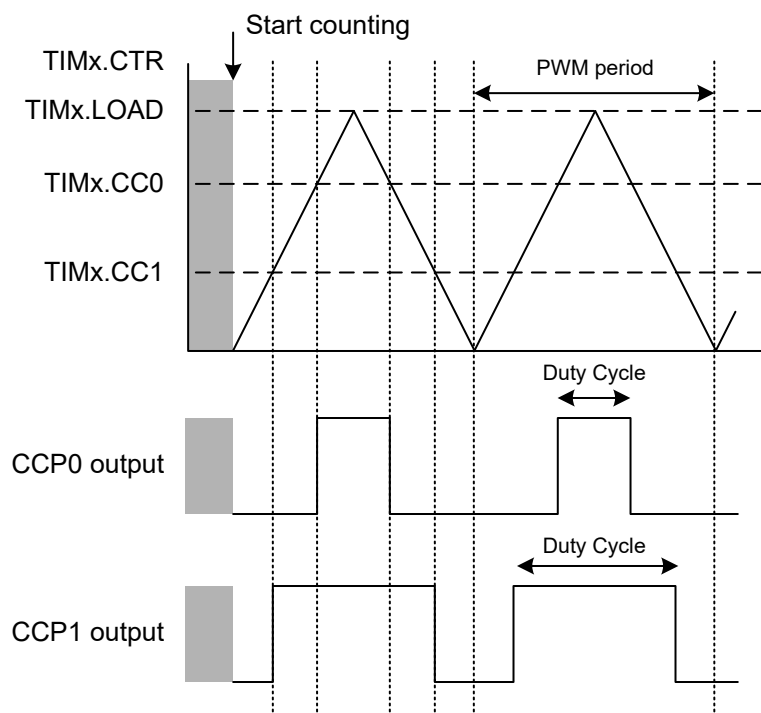


Figure 25-30. Center-Aligned PWM

25.2.5.2.3 Asymmetric PWM (TIMA only)

In TIMA only, asymmetric PWMs can be generated by generating two synchronized center-aligned PWM signals with a controlled phase shift. To generate the asymmetric PWM signals, the phase load feature is used as described in [Section 25.2.2.5](#).

Asymmetric PWM Configuration

To generate asymmetric PWMs using compare match events from the counter:

1. Synchronize TIMA0 and TIMA1 using a cross trigger as described in [Section 25.2.7](#).
2. Configure two center-aligned PWMs as described in [Section 25.2.5.2.2](#) using TIMA0 and TIMA1. TIMA0 and TIMA1 should have the same load value (TIMA.LOAD) and compare value (TIMA.CC_xy[0/1]) to generate the same PWM frequency and duty cycle.
3. Add a phase shift value for TIMA0 or TIMA1 by configuring the phase load value TIMA.PL as described in [Section 25.2.2.5](#).
4. Enable the counter by setting TIMA.CTRCTL.EN = 1.

[Figure 25-31](#) shows an example of asymmetric PWM configuration using CCP channel 0 of TIMA0 and TIMA1.

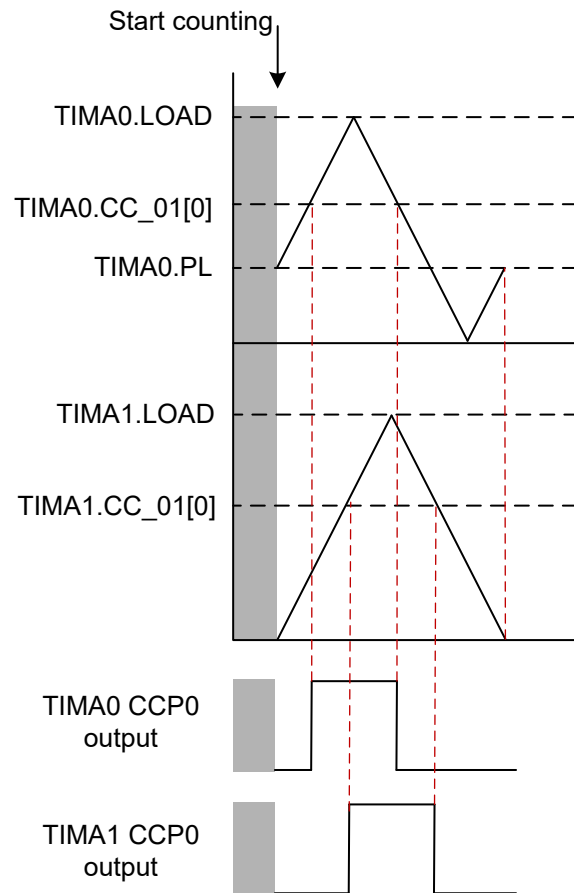


Figure 25-31. Asymmetric PWM Configuration with Phase Load for CCP channel 0 of TIMA0 and TIMA1

25.2.5.2.4 Complementary PWM with Deadband Insertion (TIMA only)

TIMA provides the option of generating complimentary PWM outputs with deadband insertion (non-overlapping transitions in complimentary PWM signals) from a signal PWM reference signal. Deadband is useful for applications with half-bridge control to avoid shoot-through conditions, such as motor driver or inverter-based applications.

TIMA provides this optional feature on complimentary CCP output channels, such as TIMA0_C2 and TIMA0_C2N for a reference PWM signal on TIMA0 CCP output channel 2.

The deadband control register (TIMA.DBCTL) is programmed with the deadband mode and timing information. The deadband mode is Mode 0 or Mode 1, which can be selected using the M1_ENABLE bit, and the timing information to control the deadband width in TIMCLK cycles is selected by the RISEDELAY and FALLDELAY bit fields. See [Table 25-18](#) for the configuration and relationship between deadband mode and deadband width settings.

Table 25-18. Deadband Modes and Delay Timing Configuration in DBCTL register

Deadband Mode	Bitfield	Description	Counting Mode
Mode 0	M1_ENABLE = 0	RISEDELAY and FALLDELAY is applied with respect to the output generator signal's rising and falling edges to generate CCP and CCP complimentary signal	Any

Table 25-18. Deadband Modes and Delay Timing Configuration in DBCTL register (continued)

Deadband Mode	Bitfield	Description	Counting Mode
Mode 1	M1_ENABLE = 1	- CCP signal is the same as output generator signal - RISEDELAY and FALLDELAY is applied to CCP complimentary signal	Up/down counting mode only

Deadband timing equation and example

The equations for configuring RISEDELAY and FALLDELAY from TIMCLK frequency and deadband timing is shown in [Equation 29](#) and [Equation 30](#).

$$RISEDELAY = f_{TIMCLK} \times t_{dead_rise} \quad (29)$$

$$FALLDELAY = f_{TIMCLK} \times t_{dead_fall} \quad (30)$$

For example, if 400 ns of deadband is required when using a TIMCLK frequency of 80 MHz, and Mode 1 is used with center-aligned PWMs to generate equal deadband every PWM period, then RISEDELAY = FALLDELAY = (80 MHz) * (400 ns) = 32.

Complimentary PWM with Deadband Configuration

1. Configure a PWM output for an edge-aligned PWM ([Section 25.2.5.2.1](#)) or center-aligned PWM ([Section 25.2.5.2.2](#)) for any CCP output channel in TIMA.
2. Configure TIMA.DBCTL with the specified deadband mode (M1_ENABLE) and deadband width RISEDELAY and FALLDELAY, depending on the deadband mode.
3. In TIMx.OCTL_xy[0/1], set CCPO = 0xC to select the signal generator with deadband output.
4. Enable the counter by setting TIMx.CTRCTL.EN = 1.

Example 1 - Complimentary PWM outputs with deadband using edge-aligned PWM in down-counting mode

For edge-aligned PWM, Mode 0 can only be used for deadband insertion mode. See [Figure 25-32](#) for inserting configurable deadband using down counting mode, TIMA output channel 0, and edge-aligned PWM. For center-aligned PWM, there are two modes, as shown in and [Figure 25-33](#).

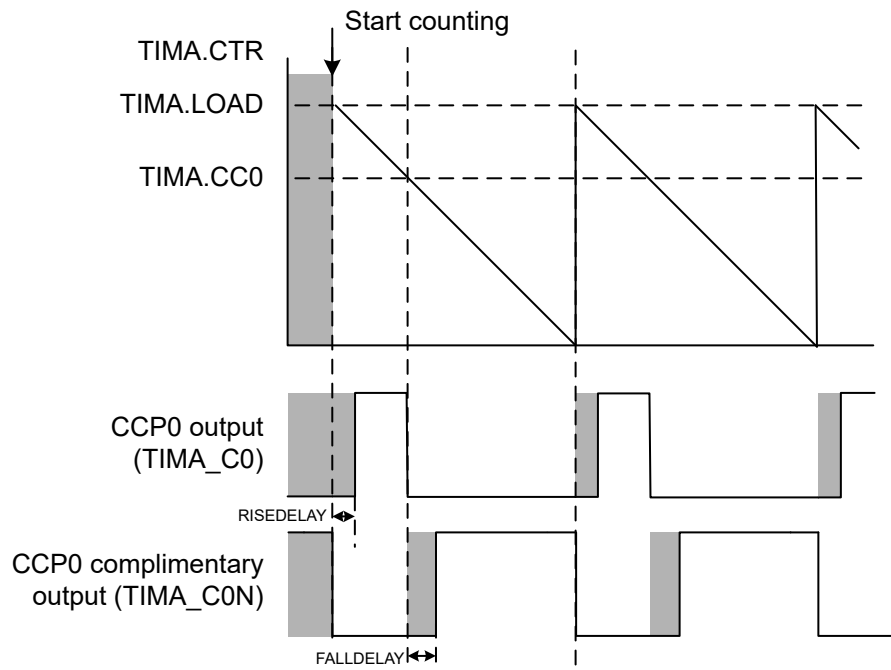


Figure 25-32. Deadband Insertion for Edge-Aligned PWM in Down-Counting Mode using TIMA CC output channel 0 (M1_ENABLE = 0)

Example 2- Complimentary PWM outputs with deadband using center-aligned PWM

For center-aligned PWM, Mode 0 or Mode 1 can be used for deadband insertion mode. See [Figure 25-33](#) For inserting configurable deadband using up/down counting mode, TIMA output channel 0, and center-aligned PWMs for both Mode 0 and Mode 1.

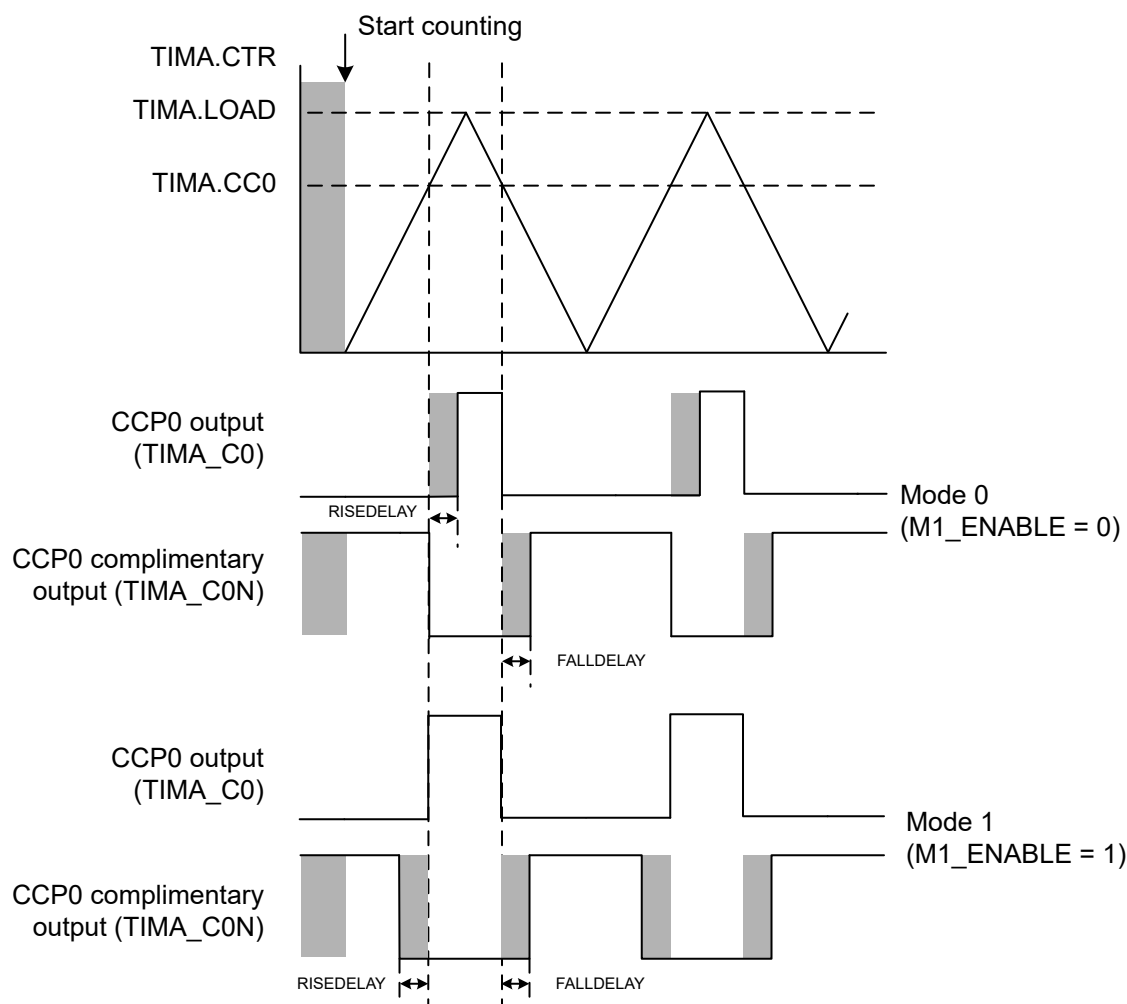


Figure 25-33. Dead Band Insertion for Center-Aligned PWM (Mode 0 and Mode 1)

25.2.5.3 Forced Output

Each output channel signal can be forced to a high or low level directly by software, independently of any comparison between the compare register and the counter.

The output of the CCP channel can be forced to high or low by setting the SWFRCACT bit in the TIMx.CCACT_xy[0/1] register.

Additionally, in TIMA only, the complimentary output channel can also be forced to high or low by setting the SWFRCACT_CMPL bit in the TIMx.CCACT_xy[0/1] register.

Table 25-19 shows the software force output action configuration options.

Table 25-19. Force Output Action Configuration

Bit Field	Value	Description/Comment
SWFRCACT / SWFRCACT_CMPL	0	No forced output. Output is directly from the signal generation block.
	1	Force output high
	2	Force output low

25.2.6 Fault Handler (TIMA only)

In TIMA only, there are internal and external fault inputs which can be used to control the generation of PWM signals. The intended use of these inputs is as a mechanism for internal or external circuitry to indicate a fault in

the system. This allows the hardware to react quickly to the external fault while optionally signaling an interrupt for software correction and leaving the output signals in a safe state.

It is important to consider the following basic properties of faults in a system, such as:

- Fault input selection (fault signal from external IC, internal signal, etc.)
- How long a fault condition lasts, or the fault condition duration
- How the counter reacts to the entry and exit of a fault condition
- How the output signal reacts to the entry and exit of a fault condition

Fault conditions are synchronously detected using TIMCLK or asynchronously detected. Synchronous faults have a configurable glitch filter and can generate a latched fault event. Asynchronous faults cannot be latched and do not generate a fault event. The CCP output can be configured for either type of fault upon entry and exit conditions.

The fault handler logic diagrams are split into three parts: asynchronous faults, synchronous faults, and fault output generation.

Figure 25-34 shows the asynchronous fault handler logic connections.

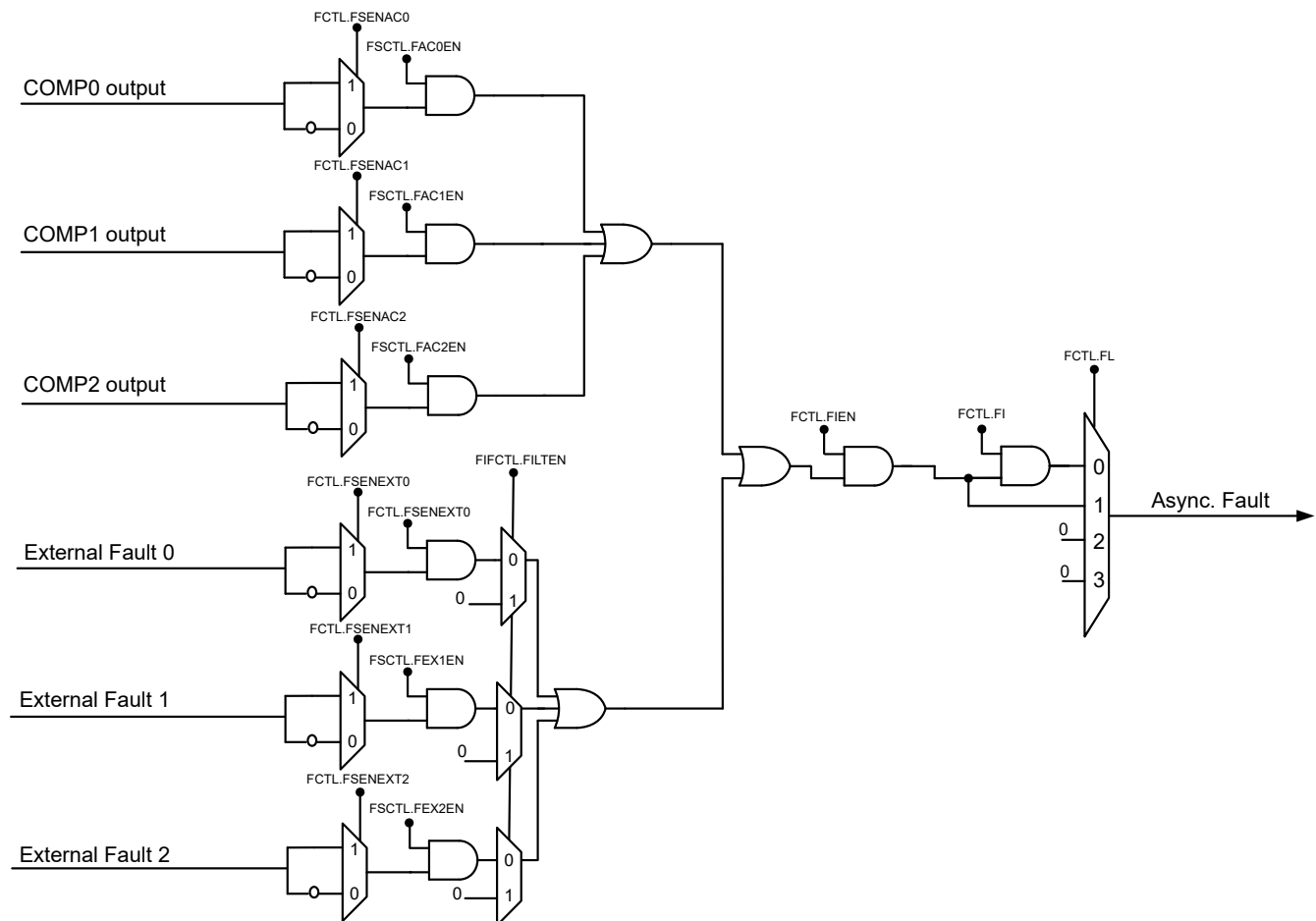


Figure 25-34. Asynchronous Fault Handler Connections

Figure 25-35 shows the synchronous fault handler logic connections.

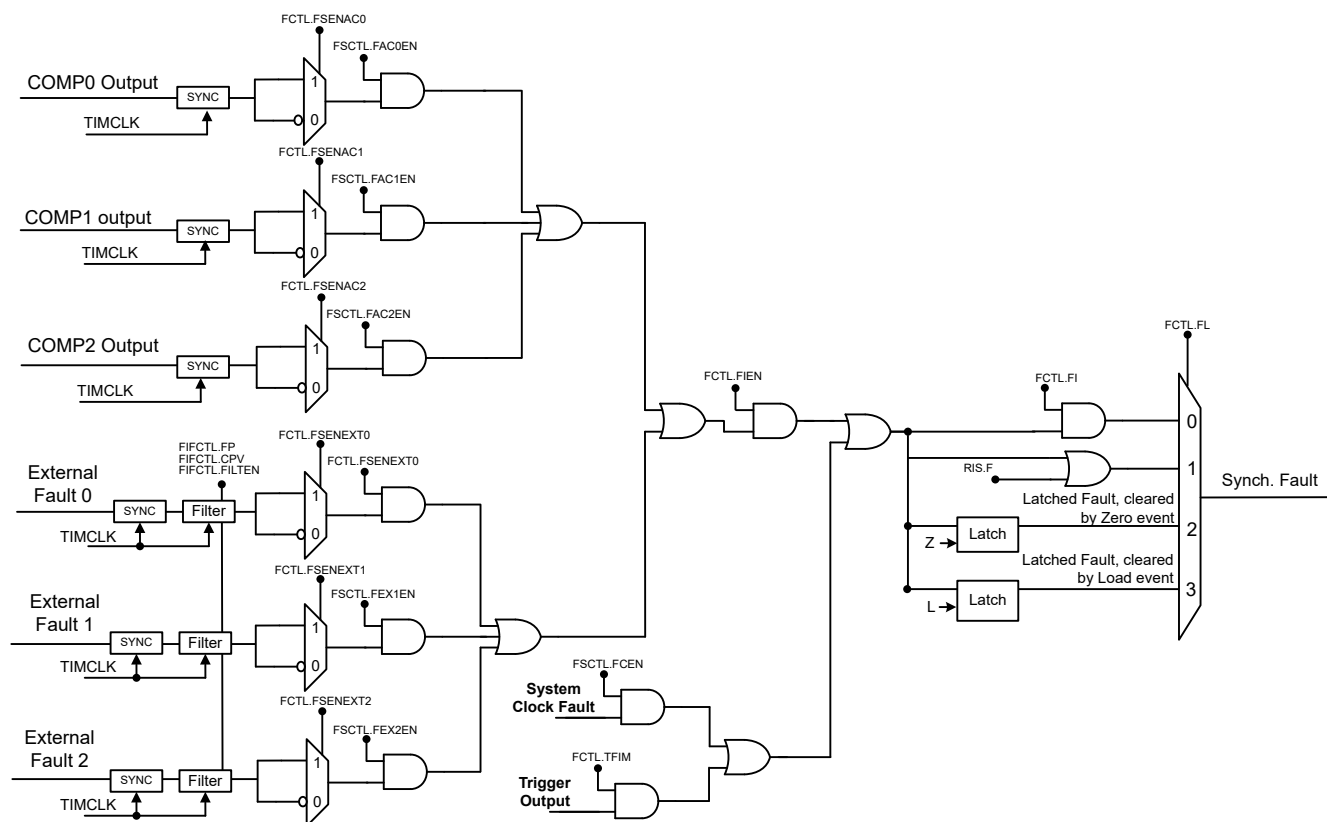


Figure 25-35. Synchronous Fault Handler Connections

Figure 25-36 shows the fault output generation logic connections.

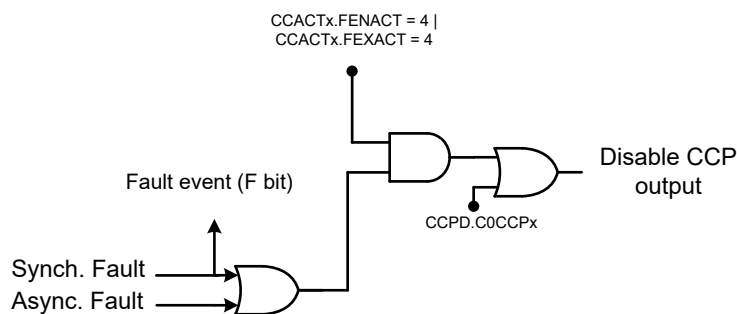


Figure 25-36. Fault Output Generation Connections

Key registers for configuring the fault handler are:

- **TIMA.FCTL**: this register controls the fault inputs, fault detection, and error handling behavior.
- **TIMA.FSCTL**: this register controls the fault source selection and enable.
- **TIMA.FIFCTL**: this register controls the input filtering (FILTEN, FP, CPV) for the fault input.
- **TIMA.CCACT_xy[0/1]**: this register controls the actions of the signal generator of the capture-compare portion based on fault events.

25.2.6.1 Fault Input Conditioning

The comparator and external fault pin fault source input passes through a two TIMCLK synchronization stage and the fault input can be filtered through the glitch filter using the fault input filter (TIMA.FIFCTL) register.

The fault input glitch filter can be enabled by setting the TIMA.FIFCTL.FILTEN bit. The filter period is configured by setting the TIMA.FIFCTL.FP bit.

A consecutive period or majority voting format selected by the TIMA.FIFCTL.CPV bit is used to select the criteria for a CCP input signal.

- **Consecutive period** - The fault input signal must be at the specified level for the defined number of FP timer clocks for the fault input to be processed.
- **Majority voting** - The filter ignores one clock of opposite logic over the filter period. For example, over the number of FP samples of the fault input, up to 1 sample may be of an opposite logic value (glitch) without affecting the output.

The example shown in [Figure 25-13](#) shows the difference between consecutive period and majority voting formats with a digital filter implemented to capture a fault input of 3 TIMCLK periods.

25.2.6.2 Fault Input Sources

The fault control (FCTL) and fault source control (FSCTL) registers are used to select the polarity and enable various fault input sources as shown in [Table 25-20](#).

To enable the final input for fault detection, set TIMA.FTCL.FIEN = 1.

There are four types of fault input sources that are available for synchronous or asynchronous fault detection:

Comparator (COMP) output

The comparator output is useful for fault detection when COMPs are used for detecting overcurrent or overvoltage events. To enable the comparator output for fault detection, set the TIMA.FSCTL.FACxEN bit, and configure the polarity to detect the fault using TIMA.FCTL.FSENACx bit (x = 0, 1, or 2 for COMP instance).

External Fault Pin

Many IC devices include a fault detection pin (i.e. nFAULT) that an MCU can detect when there is a fault condition in the system. There are 3 fault external signal pins (TIMA_FLTx) connected to every TIMA module, where x = 0, 1, or 2. Each signal pin can be enabled by setting the TIMA.FSCTL.FEXxEN bit, and the polarity of this signal to trigger a fault condition can be configured by using TIMA.FCTL.FSENEXTx bit (where x = 0, 1, or 2 for each TIMA_FLTx pin).

System Clock Fault

Any system clock fault can be used to trigger the PWM output(s) to a Hi-Z state. This can be enabled by setting the TIMA.FSCTL.FCEN bit.

Note

When a SYSCLK fault occurs, a device reset is generated. Various TIMA fault entry and exit options are invalid while the device is in reset.

Trigger

A trigger can be configured to generate a fault condition is detected. This is useful for performing diagnostics or creating fault dependencies from other peripherals in the event fabric. For trigger configuration, please see [Section 25.2.7](#). The fault input mask can be enabled by setting the TIMA.FSCTL.TFIM bit.

Table 25-20. Fault Input Sources and Configuration

Signal name	Input source	Fault type	Polarity Bit	Enable Bit
COMP0_OUT	COMP0 output	Synchronous or Asynchronous	FSENAC0	FAC0EN
COMP1_OUT	COMP1 output		FSENAC1	FAC1EN
COMP2_OUT	COMP2 output		FSENAC2	FAC2EN
TIMA_FLT0	External Fault 0		FSENEXT0	FSENEXT0
TIMA_FLT1	External Fault 1		FSENEXT1	FSENEXT1
TIMA_FLT2	External Fault 2		FSENEXT2	FSENEXT2
SYSCLK	System Clock	Synchronous	-	FCEN
TRIG	Trigger Output		-	TFIM

25.2.6.3 Counter Behavior With Fault Conditions

There are two settings for specifying the counter behavior in fault conditions: TIMA.CTRCTL.FB (during fault behavior) and TIMA.CTRCTL.FRB (fault resume behavior). The counter should continue to be enabled (TIMA.CTRCTL.EN = 1) during the fault handler behavior.

The counter behavior of the fault condition is described in [Table 25-21](#) and [Figure 25-37](#).

Table 25-21. Counter Behavior in Fault Condition With TIMA.CTRCTL Register

Bit Fields				Counter Behavior
FB	FRB	CVAE	REPEAT	
0	X	X	0	Ignores fault mode. Counter continues to count during fault and stops when reaches zero.
			1/3	Ignores fault mode. Counter continues to count during fault and repeat.
1	0	X	0/1/3	Reacts immediately to fault mode. The counter stops counting immediately and throughout the fault mode. Upon exit of fault mode, the counter continues counting from where it left off.
	1	0	X	Reacts immediately to fault mode. The counter stops counting immediately and throughout the debug mode. Upon exit of fault mode, the counter restarts from LOAD value (restarts a down count).
		1		Reacts immediately to fault mode. The counter stops counting immediately and throughout the debug mode. Upon exit of debug mode, the counter restarts from where it paused at fault entry.
		2		Reacts immediately to fault mode. The counter stops counting immediately and throughout the fault mode. Upon exit of fault mode, it restarts from 0 value (restarts an up/down count).

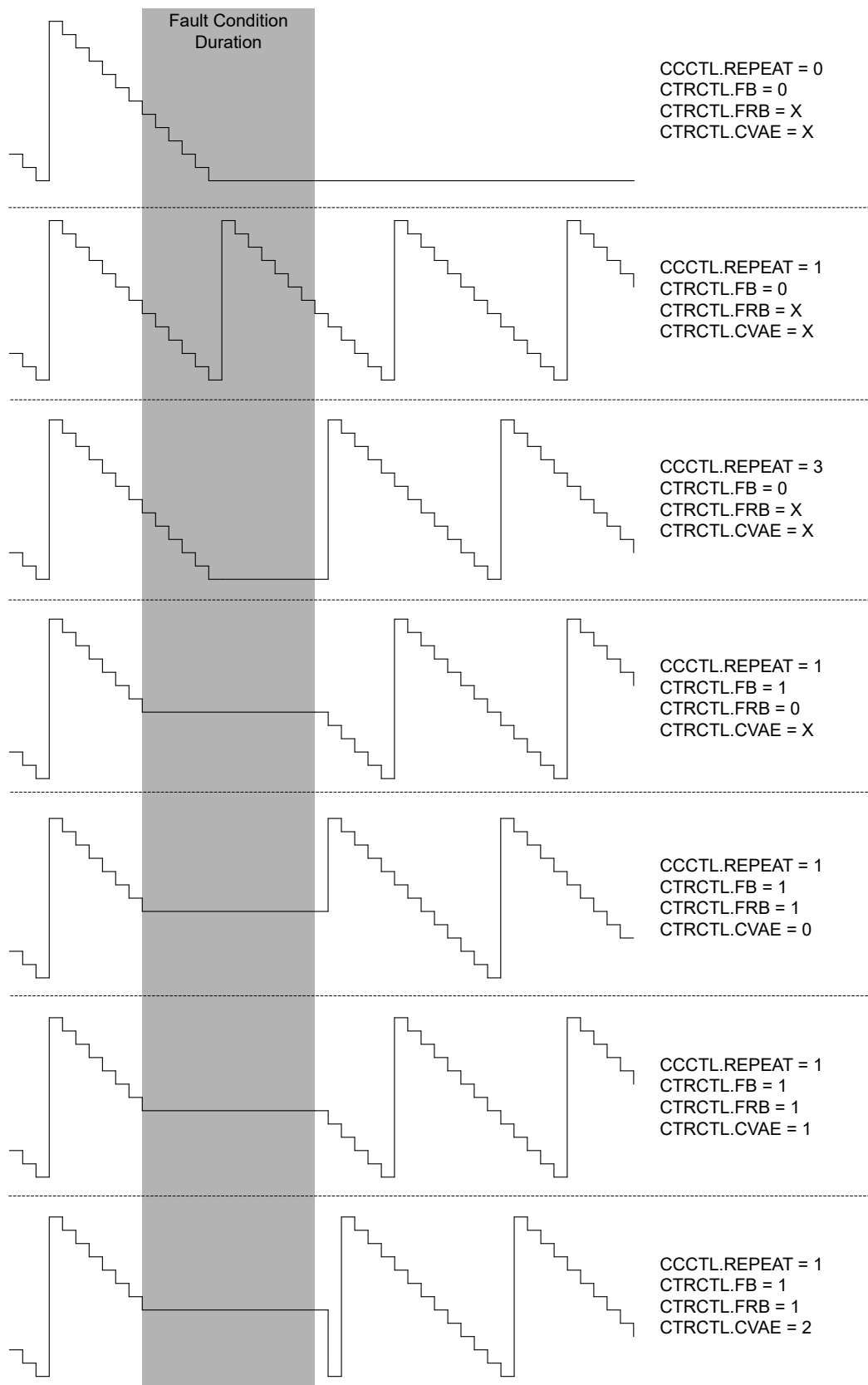


Figure 25-37. Counter Behavior in Fault Condition with TIMx.CTRCTL Register

25.2.6.4 Output Behavior With Fault Conditions

There are two settings for the CCP output channel behavior in a fault condition, TIMA.CCACT_01[0/1].FEXACT (fault exit behavior) and TIMA.CCACT_01[0/1].FENACT (fault entry behavior). The output behavior of fault condition is described in [Table 25-22](#).

Table 25-22. CCP Output Behavior in Fault Condition With TIMA.CCACT Register

Bit Fields		Output Behavior
FEXACT	FENACT	
0		The CCP output value is unaffected by the fault event
1		CCP output value is set high
2		CCP output value is set low
3		CCP output value toggles
4		CCP output is tristated (Hi-Z)

Note

Fault entry and exit behavior is not dependent on the counter being enabled. They will always update the output when in fault mode. Enabling the counter through a software write when in fault mode will generate a load event that will be captured in RIS but the counter will not proceed. Load events will not affect the output when in fault mode. Fault events takes the absolute priority and no events can update the output until you are out of fault mode.

25.2.7 Synchronization With Cross Trigger

When using a main-secondary timer configuration by connecting multiple timers together, the cross-trigger feature can instruct multiple timer modules in the same power domain or across different power domains using the event fabric to start counting simultaneously.

Cross-triggers can be enabled using software, compare events from other timer instances, zero or load events, or generic subscriber events. Some applications may require more than one counter block that can be simultaneously started across the same power domain (such as TIMA0 and TIMA1) or different power domains (such as TIMA0 and TIMG0).

This configuration uses cross triggers from a main timer module as the input trigger condition for the secondary timers. The timer cross trigger is essentially the combined logic of the hardware and software conditions that control the EN bit in the TIMx.CTRCTL register.

The cross triggers that are outputted from the main timer are connected to the external trigger input of other secondary timer modules. As shown in [Figure 25-38](#), TIMGx is the main timer and TIMAx is the secondary timer that will be cross triggered in the configuration example.

Note

For power domains and cross trigger selection sources enabled for timer instances, refer to the device-specific data sheet.

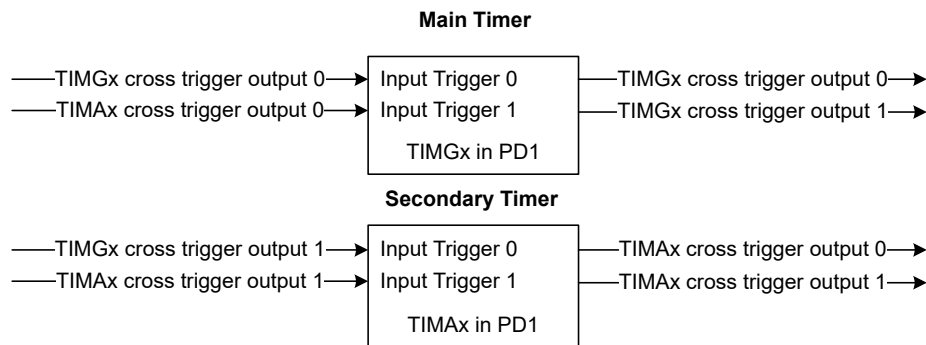


Figure 25-38. Cross Trigger Connections for Main Timer (TIMGx) and Secondary Timer (TIMAx) in Power Domain 1

25.2.7.1 Main Timer Cross Trigger Configuration

The following steps are used to configure the main timer cross trigger (which is TIMGx in this example):

1. Configure the main timer (which triggers other secondary timers) for the desired function, such as PWM output generation or using compare mode, to trigger other peripherals. See [Section 25.2.5](#) for how to configure for PWM generation.
2. Select which cross trigger output needs to be generated. For example, in [Figure 25-38](#), TIMGx cross trigger 1 can be used to trigger TIMAx and TIMGx cross trigger 0 can be used to trigger itself.
3. Enable the cross trigger output function by setting TIMx.CTTRIGCTL.CTEN bit to 1.
4. Choose how to trigger the start of these connected timers, which can be a software trigger or hardware trigger from a subscriber port, zero, load, or compare event.
 - a. For a software event trigger, set the TIMx.CTTRIG.TRIG bit.
 - b. For a hardware trigger event, select the source for the trigger using TIMx.CTTRIGCTL.EVTCTTRIGSEL and enable the hardware trigger by setting TIMx.CTTRIGCTL.EVTCTEN.

[Figure 25-39](#) shows the connection logic and registers.

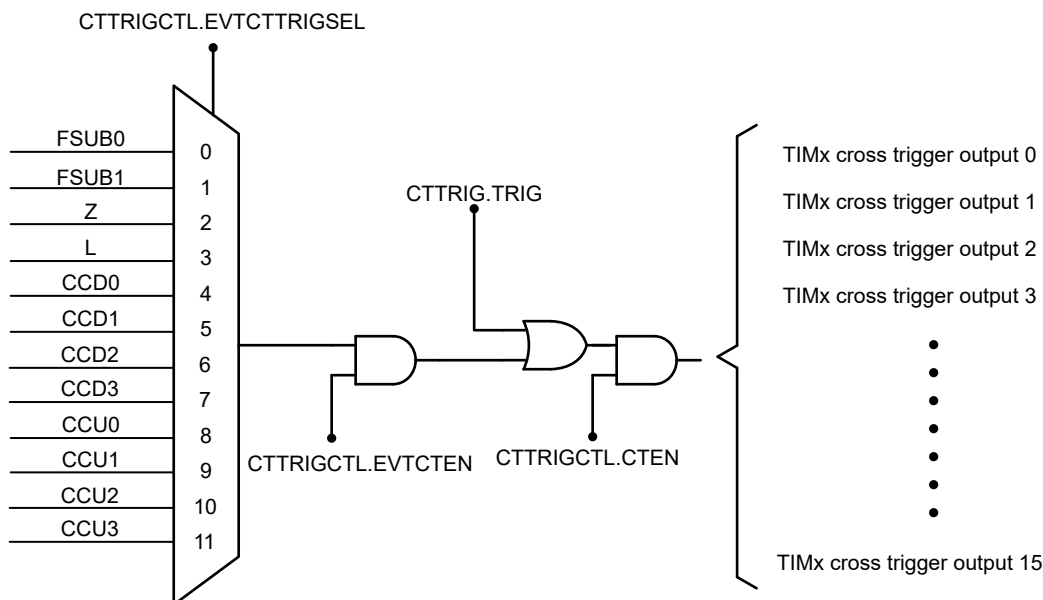


Figure 25-39. Main Timer Cross Trigger Output Configuration

25.2.7.2 Secondary Timer Cross Trigger Configuration

The following steps are used to configure the secondary timer cross trigger (which is TIMAx in this example):

1. Configure the secondary timer (triggered by the main timer) for the desired function for this timer, such as PWM output generation or using compare mode, to trigger other peripherals. See [Section 25.2.5](#) for how to configure for PWM generation.
2. Select which input trigger to use according to the device-specific data sheet. Using the example connection in [Figure 25-38](#), TIMAx must be triggered by TIMGx and the cross trigger output 1 of TIMGx is connected to input trigger 0 of TIMAx. Therefore, select input trigger 0 of TIMAx by setting TIMA.TSEL.ETSEL bit to 0.
3. Enable the input trigger function by setting the TIMA.TSEL.TE bit to 1.
4. Set TIMAx.IFCTL_01[0].ISEL = 3 and TIMAx.IFCTL_01[1].ISEL = 3 to select the trigger as the input source.
 - a. For a center-aligned PWM, set the TIMA.CCCTL_01[0].ZCOND and TIMA.CCCTL_01[1].ZCOND bits to 1 to use a trigger assertion edge for a zero event.
 - b. For an edge-aligned PWM, set the TIMA.CCCTL_01[0].LCOND and TIMA.CCCTL_01[1].LCOND bits to 1 to use the trigger assertion edge for a load event.
5. The TIMx.CTRCTL.EN bit is set as the result of an LCOND or ZCOND condition being met, and the counter value changes to the load value or zero value, respectively.

As the main timer TIMGx must also trigger itself, complete the previous configuration steps for TIMAx to trigger TIMGx itself.

[Figure 25-40](#) shows the logic connection.

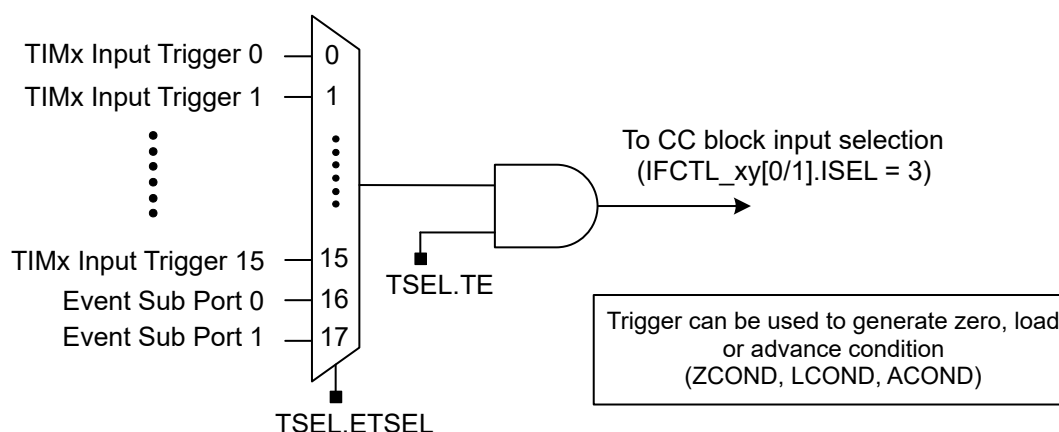


Figure 25-40. Secondary Timer Cross Trigger Input Configuration

Note

Refer to "TIMx Cross Trigger Map" in the device-specific data sheet for enabled cross trigger mapping using the ETSEL bit. For instance, if the timer instances of a device all use trigger input 0 (TRIG0) to cross trigger other timers, then only TRIG0 can be used to cross trigger other instances.

25.2.8 Low Power Operation

For detailed information on the low power modes in terms of available clock source and behaviors, refer to [Section 2.1.1](#).

Timer modules in power domain PD0 can be active and configured to continue counting in all power modes except SHUTDOWN mode. See [Section 2.1.1](#) for the available clock sources in each low-power mode. The user needs to configure the proper clock to source the timer in low-power mode.

Timer modules in power domain PD1 can only be active in RUN and SLEEP modes. When the system goes to STOP or STANDBY mode, the timer modules will be forced to a disabled state and resume when the systems moves back to RUN or SLEEP modes.

25.2.9 Interrupt and Event Support

TIMx interrupts and events can be configured to any peripheral of the device using the Event Manager. The timer can generate interrupts or events as an **event publisher**, and be triggered from other peripheral events (such as

GPIO, comparator, ADC, etc.) as an **event subscriber**. See [Section 7.2.5](#) for guidance on configuring the event registers for CPU interrupts or generic events.

The TIMx module contains three [event publishers](#) and two [event subscribers](#).

- One event publisher (CPU_INT) manages TIMx interrupt requests (IRQs) to the CPU subsystem through a [static event route](#).
- The second and third event (GEN_EVENT0 and GEN_EVENT1) are used to setup the generic event publishers and subscribers through [Generic route](#).

TIMx events are summarized in [Table 25-23](#).

Table 25-23. TIMx Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU interrupt	Publisher	TIMx	CPU Subsystem	Static route	CPU_INT registers	Fixed interrupt route from TIMx to CPU
Generic publisher event	Publisher	TIMx	Other peripherals	Generic route	GEN_EVENT0 and FPUB_0 registers	Configurable interrupt route from TIMx to other peripherals
Generic publisher event	Publisher	TIMx	Other peripherals	Generic route	GEN_EVENT 1 and FPUB_1 registers	Configurable interrupt route from TIMx to other peripherals
Generic subscriber event	Subscriber	Other peripherals	TIMx	Generic route	FSUB_0	Configurable interrupt route from other peripherals to TIMx
Generic subscriber event	Subscriber	Other peripherals	TIMx	Generic route	FSUB_1	Configurable interrupt route from other peripherals to TIMx

25.2.9.1 CPU Interrupt Event Publisher (CPU_INT)

The TIMx module provides 18 interrupt sources (depending on the specific TIMx module features) which can be configured to source a [CPU interrupt event](#). The CPU interrupt event configuration is managed with the CPU_INT event management registers. [Table 25-24](#) lists the CPU interrupt events from the TIMx in order of decreasing interrupt priority.

Table 25-24. TIMx CPU Interrupt Event Conditions (CPU_INT)

IIDX STAT	Name	Description	Timer Module
0x01	Z	Zero event interrupt. This interrupt is set when there is a zero event.	TIMx
0x02	L	Load event interrupt. This interrupt is set when there is a load event.	TIMx
0x05	CCD0	Capture or compare 0 down event. This interrupt is set when there is a down compare match event at CC0.	TIMx
0x06	CCD1	Capture or compare 1 down event. This interrupt is set when there is a down compare match event at CC1.	TIMx
0x07	CCD2	Capture or compare 2 down event. This interrupt is set when there is a down compare match event at CC2. This interrupt is only available for TIMA0.	TIMx
0x08	CCD3	Capture or compare 3 down event. This interrupt is set when there is a down compare match event at CC3. This interrupt is only available for TIMA0.	TIMx
0x09	CCU0	Capture or compare 0 up event. This interrupt is set when there is a up compare match event at CC0.	TIMx
0x0A	CCU1	Capture or compare 1 up event. This interrupt is set when there is a up compare match event at CC1.	TIMx
0x0B	CCU2	Capture or compare 2 up event. This interrupt is set when there is a up compare match event at CC2.	TIMx
0x0C	CCU3	Capture or compare 3 up event. This interrupt is set when there is a up compare match event at CC3.	TIMx
0x0D	CCD4	Capture or compare 4 down event. This interrupt is set when there is a down compare match event at CC4. This interrupt is only available for TIMA modules.	TIMA

Table 25-24. TIMx CPU Interrupt Event Conditions (CPU_INT) (continued)

IIDX STAT	Name	Description	Timer Module
0x0E	CCD5	Capture or compare 5 down event. This interrupt is set when there is a down compare match event at CC5. This interrupt is only available for TIMA modules.	TIMA
0x0F	CCU4	Capture or compare 4 up event. This interrupt is set when there is a up compare match event at CC4. This interrupt is only available for TIMA modules.	TIMA
0x10	CCU5	Capture or compare 5 up event. This interrupt is set when there is a up compare match event at CC5. This interrupt is only available for TIMA modules.	TIMA
0x19	F	Fault event interrupt. This interrupt is set when there is a fault condition event. See Section 25.2.6 . This interrupt is only available for TIMA modules with fault handler features.	TIMA
0x1A	TOV	Trigger overflow interrupt. This interrupt is set if a trigger event is generated while the associated trigger channel is active.	TIMx
0x1B	REPC	Repeat counter zero interrupt. This bit controls the generation of an interrupt if the repeat counter value transitions from a non-zero value to zero. This interrupt is only available for TIMA modules with a repeat counter feature.	TIMA
0x1C	DC	Direction change interrupt, used in QEI mode. This interrupt is only available for TIMG modules with QEI features.	TIMG
0x1D	QEIERR	Direction change interrupt, used in QEI mode. This interrupt is only available for TIMG modules with QEI features.	TIMG

See [Section 7.2.5](#) for guidance on configuring the event registers for CPU interrupts.

25.2.9.2 Generic Event Publisher and Subscriber (GEN_EVENT0 and GEN_EVENT1)

A generic route is a route in which the comparator peripheral publishing the event is configured to use one of several available generic route channels to publish its event to another entity (or entities, in the case of a splitter route), where an entity can be another peripheral, a generic DMA trigger event, or a generic CPU event.

The GEN_EVENT0 and GEN_EVENT1 registers are used to select a peripheral condition ([Table 25-25](#)) to use for publishing or subscribing an event. FPUB_0 and FPUB_1 are the publisher port registers and are used to configure which generic route channel to use to broadcast the event. FSUB_0 and FSUB_1 are the subscriber port registers and are used to configure which generic route channel to use to subscribe the event. Other peripherals, the DMA, or the CPU can subscribe to this event by configuring its subscriber port to listen on the same generic route channel which the publishing peripheral is connected to.

For example, through the use of a generic event channel, it is possible to directly start an ADC conversion from a TIMx event by connecting a TIMx FPUB_x and ADC FSUB_0 to the same generic event channel. Refer to [Section 7.1.3.3](#) and [Section 7.2.3](#) for how generic event route works.

Table 25-25. TIMx Generic Event Conditions (GEN_EVENT0 and GEN_EVENT1)

IIDX STAT	Name	Description	Timer Module
0x01	Z	Zero event interrupt. This interrupt is set when there is a zero event.	TIMx
0x02	L	Load event interrupt. This interrupt is set when there is a load event.	TIMx
0x05	CCD0	Capture or compare 0 down event. This interrupt is set when there is a down compare match event at CC0.	TIMx
0x06	CCD1	Capture or compare 1 down event. This interrupt is set when there is a down compare match event at CC1.	TIMx
0x07	CCD2	Capture or compare 2 down event. This interrupt is set when there is a down compare match event at CC2. This interrupt is only available for TIMA0.	TIMx
0x08	CCD3	Capture or compare 3 down event. This interrupt is set when there is a down compare match event at CC3. This interrupt is only available for TIMA0.	TIMx
0x09	CCU0	Capture or compare 0 up event. This interrupt is set when there is a up compare match event at CC0.	TIMx
0x0A	CCU1	Capture or compare 1 up event. This interrupt is set when there is a up compare match event at CC1.	TIMx
0x0B	CCU2	Capture or compare 2 up event. This interrupt is set when there is a up compare match event at CC2.	TIMx

Table 25-25. TIMx Generic Event Conditions (GEN_EVENT0 and GEN_EVENT1) (continued)

IIDX STAT	Name	Description	Timer Module
0x0C	CCU3	Capture or compare 3 up event. This interrupt is set when there is a up compare match event at CC3.	TIMx
0x0D	CCD4	Capture or compare 4 down event. This interrupt is set when there is a down compare match event at CC4. This interrupt is only available for TIMA modules.	TIMA
0x0E	CCD5	Capture or compare 5 down event. This interrupt is set when there is a down compare match event at CC5. This interrupt is only available for TIMA modules.	TIMA
0x0F	CCU4	Capture or compare 4 up event. This interrupt is set when there is a up compare match event at CC4. This interrupt is only available for TIMA modules.	TIMA
0x10	CCU5	Capture or compare 5 up event. This interrupt is set when there is a up compare match event at CC5. This interrupt is only available for TIMA modules.	TIMA
0x19	F	Fault event interrupt. This interrupt is set when there is a fault condition event. See Section 25.2.6 . This interrupt is only available for TIMA modules with fault handler features.	TIMA
0x1A	TOV	Trigger overflow interrupt. This interrupt is set if a trigger event is generated while the associated trigger channel is active.	TIMx
0x1B	REPC	Repeat counter zero interrupt. This bit controls the generation of an interrupt if the repeat counter value transitions from a non-zero value to zero. This interrupt is only available for TIMA modules with a repeat counter feature.	TIMA
0x1C	DC	Direction change interrupt, used in QEI mode. This interrupt is only available for TIMG modules with QEI features.	TIMG
0x1D	QEIERR	Direction change interrupt, used in QEI mode. This interrupt is only available for TIMG modules with QEI features.	TIMG

See [Section 7.2.5](#) for guidance on configuring the event registers.

25.2.9.3 Generic Subscriber Event Example (COMP to TIMx)

A common use case is to directly trigger a timer action (reset, PWM output, etc.) from the comparator output. The Event Manager can be used to set up the comparator as a generic event publisher (FPUB_1) and the timer as a generic event subscriber (FSUB_0 or FSUB_1) listening for an event from the publisher.

Comparator configuration (Publisher):

1. Set COMPIFG bit in GEN_EVENT0 IMASK register to mask the comparator output interrupt. For more info, see the COMP chapter.
2. Configure FPUB_1 register in comparator module to connect to event channel y.
3. Configure and enable the comparator.

TIMx configuration (Subscriber):

1. Configure FSUB_0 or FSUB_1 register in the TIMx module to connect to event channel y. Channel y must not be in use by another peripheral.
2. Set TIMx.IFCTL_xy[0/1].ISEL = 5 or 6 to select the CC input source as FSUB0 or FSUB1 for the TIMx module. See [Figure 25-12](#) and [Section 25.2.3.1.1.5](#).
3. Configure and enable the timer.

Note

The comparator output (COMP0:2) has two more options for cross-peripherals:

- Input to the TIMx CC block directly using TIMx.IFCTL_xy[0/1].ISEL = 7h, 8h, or 9h.
 - This is a lower latency path and is useful for applications such as cycle-by-cycle overcurrent limiting. See [Figure 25-12](#).
 - Using the timer's cross trigger path to configure the event subscriber port as a trigger source by configuring the TIMx.TSEL.ETSEL = 0x10 for FSUB0 or to 0x11 for FSUB1.
 - Enable the input trigger function by setting the TIMx.TSEL.TE bit to 1.
 - Set TIMx.IFCTL_xy[0/1].ISEL bit to 3 to select the cross trigger as input source for the TIMx module.
 - This is useful for using an event subscriber to trigger multiple timer instances in the same power domain, such as COMP to multiple TIMG instances.
-

25.2.10 Debug Handler (TIMA only)

In TIMA only, the timer output and counter behavior in CPU halt debug mode can also be configured by software using the PDBGCTL and CCACT_xy[0/1] registers.

25.3 Timers (TIMx) Registers

Table 25-26 lists the memory-mapped registers for the Timers (TIMx) registers. All register offset addresses not listed in Table 25-26 should be considered as reserved locations and the register contents should not be modified.

Table 25-26. TIMERS (TIMX) Registers

Offset	Acronym	Register Name	Group	Section
400h	FSUB_0	Subscriber Port 0		Go
404h	FSUB_1	Subscriber Port 1		Go
444h	FPUB_0	Publisher Port 0		Go
448h	FPUB_1	Publisher Port 1		Go
800h	PWREN	Power enable		Go
804h	RSTCTL	Reset Control		Go
814h	STAT	Status Register		Go
1000h	CLKDIV	Clock Divider		Go
1008h	CLKSEL	Clock Select for Ultra Low Power peripherals		Go
1018h	PDBGCTL	Peripheral Debug Control		Go
1020h	IIDX	Interrupt index	CPU_INT	Go
1028h	IMASK	Interrupt mask	CPU_INT	Go
1030h	RIS	Raw interrupt status	CPU_INT	Go
1038h	MIS	Masked interrupt status	CPU_INT	Go
1040h	ISSET	Interrupt set	CPU_INT	Go
1048h	ICLR	Interrupt clear	CPU_INT	Go
1050h	IIDX	Interrupt index	GEN_EVENT 0	Go
1058h	IMASK	Interrupt mask	GEN_EVENT 0	Go
1060h	RIS	Raw interrupt status	GEN_EVENT 0	Go
1068h	MIS	Masked interrupt status	GEN_EVENT 0	Go
1070h	ISSET	Interrupt set	GEN_EVENT 0	Go
1078h	ICLR	Interrupt clear	GEN_EVENT 0	Go
1080h	IIDX	Interrupt index	GEN_EVENT 1	Go
1088h	IMASK	Interrupt mask	GEN_EVENT 1	Go
1090h	RIS	Raw interrupt status	GEN_EVENT 1	Go
1098h	MIS	Masked interrupt status	GEN_EVENT 1	Go
10A0h	ISSET	Interrupt set	GEN_EVENT 1	Go
10A8h	ICLR	Interrupt clear	GEN_EVENT 1	Go
10E0h	EVT_MODE	Event Mode		Go
10FCh	DESC	Module Description		Go
1100h	CCPD	CCP Direction		Go

Table 25-26. TIMERS (TIMX) Registers (continued)

Offset	Acronym	Register Name	Group	Section
1104h	ODIS	Output Disable		Go
1108h	CCLKCTL	Counter Clock Control Register		Go
110Ch	CPS	Clock Prescale Register		Go
1110h	CPSV	Clock prescale count status register		Go
1114h	CTTRIGCTL	Timer Cross Trigger Control Register		Go
111Ch	CTTRIG	Timer Cross Trigger Register		Go
1120h	FSCTL	Fault Source Control		Go
1124h	GCTL	Global control register		Go
1800h	CTR	Counter Register		Go
1804h	CTRCTL	Counter Control Register		Go
1808h	LOAD	Load Register		Go
1810h + formula	CC_01[y]	Capture or Compare Register 0/1		Go
1818h + formula	CC_23[y]	Capture or Compare Register 0/1		Go
1820h + formula	CC_45[y]	The CC_45 register are a registers which can be used as compare to the current CTR to create an events CC4U, CC4D, CC5U and CC5D.		Go
1830h + formula	CCCTL_01[y]	Capture or Compare Control Registers		Go
1838h + formula	CCCTL_23[y]	Capture or Compare Control Registers 0/1		Go
1840h + formula	CCCTL_45[y]	Capture or Compare Control Registers 2/3		Go
1850h + formula	OCTL_01[y]	CCP Output Control Registers 4/5		Go
1858h + formula	OCTL_23[y]	CCP Output Control Registers 0/1		Go
1870h + formula	CCACT_01[y]	Capture or Compare Action Registers 2/3		Go
1878h + formula	CCACT_23[y]	Capture or Compare Action Registers 0/1		Go
1880h + formula	IFCTL_01[y]	Input Filter Control Register 0/1		Go
1888h + formula	IFCTL_23[y]	Input Filter Control Register 2/3		Go
18A0h	PL	Phase Load Register		Go
18A4h	DBCTL	Dead Band insertion control register		Go
18B0h	TSEL	Trigger Select Register		Go
18B4h	RC	Repeat counter Register		Go
18B8h	RCLD	Repeat counter load Register		Go
18BCh	QDIR	QEI Count Direction Register		Go
18D0h	FCTL	Fault Control Register		Go
18D4h	FIFCTL	Fault input Filter control register		Go

Complex bit access types are encoded to fit into small table cells. [Table 25-27](#) shows the codes that are used for access types in this section.

Table 25-27. Timers (TIMx) Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
K	K	Write protected by a key
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value
Register Array Variables		
i,j,k,l,m,n		When these variables are used in a register name, an offset, or an address, they refer to the value of a register array where the register is part of a group of repeating registers. The register groups form a hierarchical structure and the array is represented with a formula.
y		When this variable is used in a register name, an offset, or an address it refers to the value of a register array.

25.3.1 FSUB_0 (Offset = 400h) [Reset = 00000000h]

FSUB_0 is shown in [Figure 25-41](#) and described in [Table 25-28](#).

Return to the [Summary Table](#).

Subscriber port

Figure 25-41. FSUB_0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-0h												R/W-0h			

Table 25-28. FSUB_0 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 15.

25.3.2 FSUB_1 (Offset = 404h) [Reset = 00000000h]

FSUB_1 is shown in [Figure 25-42](#) and described in [Table 25-29](#).

Return to the [Summary Table](#).

Subscriber port

Figure 25-42. FSUB_1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-0h												R/W-0h			

Table 25-29. FSUB_1 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 15.

25.3.3 FPUB_0 (Offset = 444h) [Reset = 00000000h]

FPUB_0 is shown in [Figure 25-43](#) and described in [Table 25-30](#).

Return to the [Summary Table](#).

Publisher port

Figure 25-43. FPUB_0

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-0h												R/W-0h			

Table 25-30. FPUB_0 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 15.

25.3.4 FPUB_1 (Offset = 448h) [Reset = 00000000h]

FPUB_1 is shown in [Figure 25-44](#) and described in [Table 25-31](#).

Return to the [Summary Table](#).

Publisher port

Figure 25-44. FPUB_1

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-0h												R/W-0h			

Table 25-31. FPUB_1 Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 15.

25.3.5 PWREN (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 25-45](#) and described in [Table 25-32](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 25-45. PWREN

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-0h							K-0h

Table 25-32. PWREN Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	ENABLE	K	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

25.3.6 RSTCTL (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 25-46](#) and described in [Table 25-33](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 25-46. RSTCTL

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCL R	RESETASSERT
W-0h						WK-0h	WK-0h

Table 25-33. RSTCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	W	0h	
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

25.3.7 STAT (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 25-47](#) and described in [Table 25-34](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 25-47. STAT

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED							
R-							

Table 25-34. STAT Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0h	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

25.3.8 CLKDIV (Offset = 1000h) [Reset = 00000000h]

CLKDIV is shown in [Figure 25-48](#) and described in [Table 25-35](#).

Return to the [Summary Table](#).

This register is used to specify module-specific divide ratio of the functional clock

Figure 25-48. CLKDIV

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												RATIO			
R/W-0h												R/W-0h			

Table 25-35. CLKDIV Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R/W	0h	
2-0	RATIO	R/W	0h	Selects divide ratio of module clock 0h = Do not divide clock source 1h = Divide clock source by 2 2h = Divide clock source by 3 3h = Divide clock source by 4 4h = Divide clock source by 5 5h = Divide clock source by 6 6h = Divide clock source by 7 7h = Divide clock source by 8

25.3.9 CLKSEL (Offset = 1008h) [Reset = 00000000h]

CLKSEL is shown in [Figure 25-49](#) and described in [Table 25-36](#).

Return to the [Summary Table](#).

Clock Source Select Register

Figure 25-49. CLKSEL

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED				BUSCLK_SEL	MFCLK_SEL	LFCLK_SEL	RESERVED
R/W-0h				R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 25-36. CLKSEL Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3	BUSCLK_SEL	R/W	0h	Selects BUSCLK as clock source if enabled 0h = Does not select this clock as a source 1h = Select this clock as a source
2	MFCLK_SEL	R/W	0h	Selects MFCLK as clock source if enabled 0h = Does not select this clock as a source 1h = Select this clock as a source
1	LFCLK_SEL	R/W	0h	Selects LFCLK as clock source if enabled 0h = Does not select this clock as a source 1h = Select this clock as a source
0	RESERVED	R/W	0h	

25.3.10 PDBGCTL (Offset = 1018h) [Reset = 00000003h]

PDBGCTL is shown in [Figure 25-50](#) and described in [Table 25-37](#).

Return to the [Summary Table](#).

This register can be used by the software developer to control the behavior of the peripheral relative to the 'Core Halted' input

Figure 25-50. PDBGCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED						SOFT	FREE
R/W-						R/W-1h	R/W-1h

Table 25-37. PDBGCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R/W	0h	
1	SOFT	R/W	1h	Soft halt boundary control. This function is only available, if FREE is set to 'STOP' 0h = The peripheral will halt immediately, even if the resultant state will result in corruption if the system is restarted 1h = The peripheral blocks the debug freeze until it has reached a boundary where it can resume without corruption
0	FREE	R/W	1h	Free run control 0h = The peripheral freezes functionality while the Core Halted input is asserted and resumes when it is deasserted. 1h = The peripheral ignores the state of the Core Halted input

25.3.11 IIDX (Offset = 1020h) [Reset = 00000000h]

IIDX is shown in [Figure 25-51](#) and described in [Table 25-38](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 25-51. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 25-38. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 00h = No interrupt pending 01h = Interrupt Source: Zero event (Z) 02h = Interrupt Source: Load event (L) 05h = Interrupt Source: Capture or compare down event (CCD0) 06h = Interrupt Source: Capture or compare down event (CCD1) 07h = Interrupt Source: Capture or compare down event (CCD2) 08h = Interrupt Source: Capture or compare down event (CCD3) 09h = Interrupt Source: Capture or compare up event (CCU0) 0Ah = Interrupt Source: Capture or compare up event (CCU1) 0Bh = Interrupt Source: Capture or compare up event (CCU2) 0Ch = Interrupt Source: Capture or compare up event (CCU3) 0Dh = Interrupt Source: Compare down event (CCD4) 0Eh = Interrupt Source: Compare down event (CCD5) 0Fh = Interrupt Source: Compare down event (CCU4) 10h = Interrupt Source: Compare down event (CCU5) 19h = Interrupt Source: Fault Event generated an interrupt. (F) 1Ah = Interrupt Source: Trigger overflow (TOV) 1Bh = Interrupt Source: Repeat Counter Zero (REPC) 1Ch = Interrupt Source: Direction Change (DC) 1Dh = Interrupt Source: QEI Incorrect state transition error (QEIERR)

25.3.12 IMASK (Offset = 1028h) [Reset = 00000000h]

IMASK is shown in [Figure 25-52](#) and described in [Table 25-39](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.”

Figure 25-52. IMASK

31	30	29	28	27	26	25	24
RESERVED			QEIERR	DC	REPC	TOV	F
R/W-			R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
CCU5	CCU4	CCD5	CCD4	CCU3	CCU2	CCU1	CCU0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
CCD3	CCD2	CCD1	CCD0	RESERVED		L	Z
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-		R/W-0h	R/W-0h

Table 25-39. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R/W	0h	
28	QEIERR	R/W	0h	QEIERR Event mask 0h = Disable Event 1h = Enable Event
27	DC	R/W	0h	Direction Change Event mask 0h = Disable Event 1h = Enable Event
26	REPC	R/W	0h	Repeat Counter Zero Event mask 0h = Disable Event 1h = Enable Event
25	TOV	R/W	0h	Trigger Overflow Event mask 0h = Disable Event 1h = Enable Event
24	F	R/W	0h	Fault Event mask 0h = Disable Event 1h = Enable Event
23-16	RESERVED	R/W	0h	
15	CCU5	R/W	0h	Compare UP event mask CCP5 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
14	CCU4	R/W	0h	Compare UP event mask CCP4 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
13	CCD5	R/W	0h	Compare DN event mask CCP5 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
12	CCD4	R/W	0h	Compare DN event mask CCP4 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

Table 25-39. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	CCU3	R/W	0h	Capture or Compare UP event mask CCP3 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
10	CCU2	R/W	0h	Capture or Compare UP event mask CCP2 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
9	CCU1	R/W	0h	Capture or Compare UP event mask CCP1 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
8	CCU0	R/W	0h	Capture or Compare UP event mask CCP0 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
7	CCD3	R/W	0h	Capture or Compare DN event mask CCP3 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
6	CCD2	R/W	0h	Capture or Compare DN event mask CCP2 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
5	CCD1	R/W	0h	Capture or Compare DN event mask CCP1 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
4	CCD0	R/W	0h	Capture or Compare DN event mask CCP0 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
3-2	RESERVED	R/W	0h	
1	L	R/W	0h	Load Event mask 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	Z	R/W	0h	Zero Event mask 0h = Disable Event 1h = Enable Event

25.3.13 RIS (Offset = 1030h) [Reset = 00000000h]

RIS is shown in [Figure 25-53](#) and described in [Table 25-40](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 25-53. RIS

31	30	29	28	27	26	25	24
RESERVED			QEIERR	DC	REPC	TOV	F
R-0h			R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
CCU5	CCU4	CCD5	CCD4	CCU3	CCU2	CCU1	CCU0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
CCD3	CCD2	CCD1	CCD0	RESERVED		L	Z
R-0h	R-0h	R-0h	R-0h	R-0h		R-0h	R-0h

Table 25-40. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R	0h	
28	QEIERR	R	0h	QEIERR, set on an incorrect state transition on the encoder interface. 0h = Event Cleared 1h = Event Set
27	DC	R	0h	Direction Change 0h = Event Cleared 1h = Event Set
26	REPC	R	0h	Repeat Counter Zero 0h = Event Cleared 1h = Event Set
25	TOV	R	0h	Trigger overflow 0h = Event Cleared 1h = Event Set
24	F	R	0h	Fault 0h = Event Cleared 1h = Event Set
23-16	RESERVED	R	0h	
15	CCU5	R	0h	Compare up event generated an interrupt CCP5 0h = Event Cleared 1h = Event Set
14	CCU4	R	0h	Compare up event generated an interrupt CCU4 0h = Event Cleared 1h = Event Set
13	CCD5	R	0h	Compare down event generated an interrupt CCD5 0h = Event Cleared 1h = Event Set

Table 25-40. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
12	CCD4	R	0h	Compare down event generated an interrupt CCD4 0h = Event Cleared 1h = Event Set
11	CCU3	R	0h	Capture or compare up event generated an interrupt CCP3 0h = Event Cleared 1h = Event Set
10	CCU2	R	0h	Capture or compare up event generated an interrupt CCP2 0h = Event Cleared 1h = Event Set
9	CCU1	R	0h	Capture or compare up event generated an interrupt CCP1 0h = Event Cleared 1h = Event Set
8	CCU0	R	0h	Capture or compare up event generated an interrupt CCP0 0h = Event Cleared 1h = Event Set
7	CCD3	R	0h	Capture or compare down event generated an interrupt CCP3 0h = Event Cleared 1h = Event Set
6	CCD2	R	0h	Capture or compare down event generated an interrupt CCP2 0h = Event Cleared 1h = Event Set
5	CCD1	R	0h	Capture or compare down event generated an interrupt CCP1 0h = Event Cleared 1h = Event Set
4	CCD0	R	0h	Capture or compare down event generated an interrupt CCP0 0h = Event Cleared 1h = Event Set
3-2	RESERVED	R	0h	
1	L	R	0h	Load event generated an interrupt. 0h = Event Cleared 1h = Event Set
0	Z	R	0h	Zero event generated an interrupt. 0h = Event Cleared 1h = Event Set

25.3.14 MIS (Offset = 1038h) [Reset = 00000000h]

MIS is shown in [Figure 25-54](#) and described in [Table 25-41](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 25-54. MIS

31	30	29	28	27	26	25	24
RESERVED			QEIERR	DC	REPC	TOV	F
R-0h			R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
CCU5	CCU4	CCD5	CCD4	CCU3	CCU2	CCU1	CCU0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
CCD3	CCD2	CCD1	CCD0	RESERVED		L	Z
R-0h	R-0h	R-0h	R-0h	R-0h		R-0h	R-0h

Table 25-41. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R	0h	
28	QEIERR	R	0h	QEIERR 0h = Event Cleared 1h = Event Set
27	DC	R	0h	Direction Change 0h = Event Cleared 1h = Event Set
26	REPC	R	0h	Repeat Counter Zero 0h = Event Cleared 1h = Event Set
25	TOV	R	0h	Trigger overflow 0h = Event Cleared 1h = Event Set
24	F	R	0h	Fault 0h = Event Cleared 1h = Event Set
23-16	RESERVED	R	0h	
15	CCU5	R	0h	Compare up event generated an interrupt CCP5 0h = Event Cleared 1h = Event Set
14	CCU4	R	0h	Compare up event generated an interrupt CCP4 0h = Event Cleared 1h = Event Set
13	CCD5	R	0h	Compare down event generated an interrupt CCP5 0h = Event Cleared 1h = Event Set
12	CCD4	R	0h	Compare down event generated an interrupt CCP4 0h = Event Cleared 1h = Event Set

Table 25-41. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	CCU3	R	0h	Capture or compare up event generated an interrupt CCP3 0h = Event Cleared 1h = Event Set
10	CCU2	R	0h	Capture or compare up event generated an interrupt CCP2 0h = Event Cleared 1h = Event Set
9	CCU1	R	0h	Capture or compare up event generated an interrupt CCP1 0h = Event Cleared 1h = Event Set
8	CCU0	R	0h	Capture or compare up event generated an interrupt CCP0 0h = Event Cleared 1h = Event Set
7	CCD3	R	0h	Capture or compare down event generated an interrupt CCP3 0h = Event Cleared 1h = Event Set
6	CCD2	R	0h	Capture or compare down event generated an interrupt CCP2 0h = Event Cleared 1h = Event Set
5	CCD1	R	0h	Capture or compare down event generated an interrupt CCP1 0h = Event Cleared 1h = Event Set
4	CCD0	R	0h	Capture or compare down event generated an interrupt CCP0 0h = Event Cleared 1h = Event Set
3-2	RESERVED	R	0h	
1	L	R	0h	Load event generated an interrupt. 0h = Event Cleared 1h = Event Set
0	Z	R	0h	Zero event generated an interrupt. 0h = Event Cleared 1h = Event Set

25.3.15 ISET (Offset = 1040h) [Reset = 00000000h]

ISET is shown in [Figure 25-55](#) and described in [Table 25-42](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 25-55. ISET

31	30	29	28	27	26	25	24
RESERVED			QEIRR	DC	REPC	TOV	F
W-0h			W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
CCU5	CCU4	CCD5	CCD4	CCU3	CCU2	CCU1	CCU0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
CCD3	CCD2	CCD1	CCD0	RESERVED		L	Z
W-0h	W-0h	W-0h	W-0h	W-0h		W-0h	W-0h

Table 25-42. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	W	0h	
28	QEIRR	W	0h	QEIRR event SET 0h = Writing 0 has no effect. 1h = Event Set
27	DC	W	0h	Direction Change event SET 0h = Writing 0 has no effect. 1h = Event Set
26	REPC	W	0h	Repeat Counter Zero event SET 0h = Writing 0 has no effect. 1h = Event Set
25	TOV	W	0h	Trigger Overflow event SET 0h = Writing 0 has no effect. 1h = Event Set
24	F	W	0h	Fault event SET 0h = Writing 0 has no effect. 1h = Event Set
23-16	RESERVED	W	0h	
15	CCU5	W	0h	Compare up event 5 SET 0h = Writing 0 has no effect. 1h = Event Set
14	CCU4	W	0h	Compare up event 4 SET 0h = Writing 0 has no effect. 1h = Event Set
13	CCD5	W	0h	Compare down event 5 SET 0h = Writing 0 has no effect. 1h = Event Set
12	CCD4	W	0h	Compare down event 4 SET 0h = Writing 0 has no effect. 1h = Event Set

Table 25-42. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	CCU3	W	0h	Capture or compare up event SET 0h = Writing 0 has no effect. 1h = Event Set
10	CCU2	W	0h	Capture or compare up event SET 0h = Writing 0 has no effect. 1h = Event Set
9	CCU1	W	0h	Capture or compare up event SET 0h = Writing 0 has no effect. 1h = Event Set
8	CCU0	W	0h	Capture or compare up event SET 0h = Writing 0 has no effect. 1h = Event Set
7	CCD3	W	0h	Capture or compare down event SET 0h = Writing 0 has no effect. 1h = Event Set
6	CCD2	W	0h	Capture or compare down event SET 0h = Writing 0 has no effect. 1h = Event Set
5	CCD1	W	0h	Capture or compare down event SET 0h = Writing 0 has no effect. 1h = Event Set
4	CCD0	W	0h	Capture or compare down event SET 0h = Writing 0 has no effect. 1h = Event Set
3-2	RESERVED	W	0h	
1	L	W	0h	Load event SET 0h = Writing 0 has no effect. 1h = Event Set
0	Z	W	0h	Zero event SET 0h = Writing 0 has no effect. 1h = Event Set

25.3.16 ICLR (Offset = 1048h) [Reset = 00000000h]

ICLR is shown in [Figure 25-56](#) and described in [Table 25-43](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 25-56. ICLR

31	30	29	28	27	26	25	24
RESERVED			QEIRR	DC	REPC	TOV	F
W-0h			W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
CCU5	CCU4	CCD5	CCD4	CCU3	CCU2	CCU1	CCU0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
CCD3	CCD2	CCD1	CCD0	RESERVED		L	Z
W-0h	W-0h	W-0h	W-0h	W-0h		W-0h	W-0h

Table 25-43. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	W	0h	
28	QEIRR	W	0h	QEIRR event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
27	DC	W	0h	Direction Change event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
26	REPC	W	0h	Repeat Counter Zero event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
25	TOV	W	0h	Trigger Overflow event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
24	F	W	0h	Fault event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
23-16	RESERVED	W	0h	
15	CCU5	W	0h	Compare up event 5 CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
14	CCU4	W	0h	Compare up event 4 CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
13	CCD5	W	0h	Compare down event 5 CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
12	CCD4	W	0h	Compare down event 4 CLEAR 0h = Writing 0 has no effect. 1h = Event Clear

Table 25-43. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	CCU3	W	0h	Capture or compare up event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
10	CCU2	W	0h	Capture or compare up event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
9	CCU1	W	0h	Capture or compare up event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
8	CCU0	W	0h	Capture or compare up event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
7	CCD3	W	0h	Capture or compare down event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
6	CCD2	W	0h	Capture or compare down event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
5	CCD1	W	0h	Capture or compare down event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
4	CCD0	W	0h	Capture or compare down event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
3-2	RESERVED	W	0h	
1	L	W	0h	Load event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
0	Z	W	0h	Zero event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear

25.3.17 IIDX (Offset = 1050h) [Reset = 00000000h]

IIDX is shown in [Figure 25-57](#) and described in [Table 25-44](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 25-57. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 25-44. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 00h = No interrupt pending 01h = Interrupt Source: Zero event (Z) 02h = Interrupt Source: Load event (L) 05h = Interrupt Source: Capture or compare down event (CCD0) 06h = Interrupt Source: Capture or compare down event (CCD1) 07h = Interrupt Source: Capture or compare down event (CCD2) 08h = Interrupt Source: Capture or compare down event (CCD3) 09h = Interrupt Source: Capture or compare up event (CCU0) 0Ah = Interrupt Source: Capture or compare up event (CCU1) 0Bh = Interrupt Source: Capture or compare up event (CCU2) 0Ch = Interrupt Source: Capture or compare up event (CCU3) 0Dh = Interrupt Source: Compare down event (CCD4) 0Eh = Interrupt Source: Compare down event (CCD5) 0Fh = Interrupt Source: Compare down event (CCU4) 10h = Interrupt Source: Compare down event (CCU5) 19h = Interrupt Source: Fault Event generated an interrupt. (F) 1Ah = Interrupt Source: Trigger overflow (TOV) 1Bh = Interrupt Source: Repeat Counter Zero (REPC) 1Ch = Interrupt Source: Direction Change (DC) 1Dh = Interrupt Source: QEI Incorrect state transition error (QEIERR)

25.3.18 IMASK (Offset = 1058h) [Reset = 00000000h]

IMASK is shown in [Figure 25-58](#) and described in [Table 25-45](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.”

Figure 25-58. IMASK

31	30	29	28	27	26	25	24
RESERVED			QEIERR	DC	REPC	TOV	F
R/W-			R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
CCU5	CCU4	CCD5	CCD4	CCU3	CCU2	CCU1	CCU0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
CCD3	CCD2	CCD1	CCD0	RESERVED		L	Z
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-		R/W-0h	R/W-0h

Table 25-45. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R/W	0h	
28	QEIERR	R/W	0h	QEIERR Event mask 0h = Disable Event 1h = Enable Event
27	DC	R/W	0h	Direction Change Event mask 0h = Disable Event 1h = Enable Event
26	REPC	R/W	0h	Repeat Counter Zero Event mask 0h = Disable Event 1h = Enable Event
25	TOV	R/W	0h	Trigger Overflow Event mask 0h = Disable Event 1h = Enable Event
24	F	R/W	0h	Fault Event mask 0h = Disable Event 1h = Enable Event
23-16	RESERVED	R/W	0h	
15	CCU5	R/W	0h	Compare UP event mask CCP5 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
14	CCU4	R/W	0h	Compare UP event mask CCP4 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
13	CCD5	R/W	0h	Compare DN event mask CCP5 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
12	CCD4	R/W	0h	Compare DN event mask CCP4 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

Table 25-45. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	CCU3	R/W	0h	Capture or Compare UP event mask CCP3 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
10	CCU2	R/W	0h	Capture or Compare UP event mask CCP2 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
9	CCU1	R/W	0h	Capture or Compare UP event mask CCP1 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
8	CCU0	R/W	0h	Capture or Compare UP event mask CCP0 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
7	CCD3	R/W	0h	Capture or Compare DN event mask CCP3 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
6	CCD2	R/W	0h	Capture or Compare DN event mask CCP2 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
5	CCD1	R/W	0h	Capture or Compare DN event mask CCP1 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
4	CCD0	R/W	0h	Capture or Compare DN event mask CCP0 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
3-2	RESERVED	R/W	0h	
1	L	R/W	0h	Load Event mask 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	Z	R/W	0h	Zero Event mask 0h = Disable Event 1h = Enable Event

25.3.19 RIS (Offset = 1060h) [Reset = 00000000h]

RIS is shown in [Figure 25-59](#) and described in [Table 25-46](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 25-59. RIS

31	30	29	28	27	26	25	24
RESERVED			QEIERR	DC	REPC	TOV	F
R-0h			R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
CCU5	CCU4	CCD5	CCD4	CCU3	CCU2	CCU1	CCU0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
CCD3	CCD2	CCD1	CCD0	RESERVED		L	Z
R-0h	R-0h	R-0h	R-0h	R-0h		R-0h	R-0h

Table 25-46. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R	0h	
28	QEIERR	R	0h	QEIERR, set on an incorrect state transition on the encoder interface. 0h = Event Cleared 1h = Event Set
27	DC	R	0h	Direction Change 0h = Event Cleared 1h = Event Set
26	REPC	R	0h	Repeat Counter Zero 0h = Event Cleared 1h = Event Set
25	TOV	R	0h	Trigger overflow 0h = Event Cleared 1h = Event Set
24	F	R	0h	Fault 0h = Event Cleared 1h = Event Set
23-16	RESERVED	R	0h	
15	CCU5	R	0h	Compare up event generated an interrupt CCP5 0h = Event Cleared 1h = Event Set
14	CCU4	R	0h	Compare up event generated an interrupt CCU4 0h = Event Cleared 1h = Event Set
13	CCD5	R	0h	Compare down event generated an interrupt CCD5 0h = Event Cleared 1h = Event Set

Table 25-46. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
12	CCD4	R	0h	Compare down event generated an interrupt CCD4 0h = Event Cleared 1h = Event Set
11	CCU3	R	0h	Capture or compare up event generated an interrupt CCP3 0h = Event Cleared 1h = Event Set
10	CCU2	R	0h	Capture or compare up event generated an interrupt CCP2 0h = Event Cleared 1h = Event Set
9	CCU1	R	0h	Capture or compare up event generated an interrupt CCP1 0h = Event Cleared 1h = Event Set
8	CCU0	R	0h	Capture or compare up event generated an interrupt CCP0 0h = Event Cleared 1h = Event Set
7	CCD3	R	0h	Capture or compare down event generated an interrupt CCP3 0h = Event Cleared 1h = Event Set
6	CCD2	R	0h	Capture or compare down event generated an interrupt CCP2 0h = Event Cleared 1h = Event Set
5	CCD1	R	0h	Capture or compare down event generated an interrupt CCP1 0h = Event Cleared 1h = Event Set
4	CCD0	R	0h	Capture or compare down event generated an interrupt CCP0 0h = Event Cleared 1h = Event Set
3-2	RESERVED	R	0h	
1	L	R	0h	Load event generated an interrupt. 0h = Event Cleared 1h = Event Set
0	Z	R	0h	Zero event generated an interrupt. 0h = Event Cleared 1h = Event Set

25.3.20 MIS (Offset = 1068h) [Reset = 00000000h]

MIS is shown in [Figure 25-60](#) and described in [Table 25-47](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 25-60. MIS

31	30	29	28	27	26	25	24
RESERVED			QEIERR	DC	REPC	TOV	F
R-0h			R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
CCU5	CCU4	CCD5	CCD4	CCU3	CCU2	CCU1	CCU0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
CCD3	CCD2	CCD1	CCD0	RESERVED		L	Z
R-0h	R-0h	R-0h	R-0h	R-0h		R-0h	R-0h

Table 25-47. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R	0h	
28	QEIERR	R	0h	QEIERR 0h = Event Cleared 1h = Event Set
27	DC	R	0h	Direction Change 0h = Event Cleared 1h = Event Set
26	REPC	R	0h	Repeat Counter Zero 0h = Event Cleared 1h = Event Set
25	TOV	R	0h	Trigger overflow 0h = Event Cleared 1h = Event Set
24	F	R	0h	Fault 0h = Event Cleared 1h = Event Set
23-16	RESERVED	R	0h	
15	CCU5	R	0h	Compare up event generated an interrupt CCP5 0h = Event Cleared 1h = Event Set
14	CCU4	R	0h	Compare up event generated an interrupt CCP4 0h = Event Cleared 1h = Event Set
13	CCD5	R	0h	Compare down event generated an interrupt CCP5 0h = Event Cleared 1h = Event Set
12	CCD4	R	0h	Compare down event generated an interrupt CCP4 0h = Event Cleared 1h = Event Set

Table 25-47. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	CCU3	R	0h	Capture or compare up event generated an interrupt CCP3 0h = Event Cleared 1h = Event Set
10	CCU2	R	0h	Capture or compare up event generated an interrupt CCP2 0h = Event Cleared 1h = Event Set
9	CCU1	R	0h	Capture or compare up event generated an interrupt CCP1 0h = Event Cleared 1h = Event Set
8	CCU0	R	0h	Capture or compare up event generated an interrupt CCP0 0h = Event Cleared 1h = Event Set
7	CCD3	R	0h	Capture or compare down event generated an interrupt CCP3 0h = Event Cleared 1h = Event Set
6	CCD2	R	0h	Capture or compare down event generated an interrupt CCP2 0h = Event Cleared 1h = Event Set
5	CCD1	R	0h	Capture or compare down event generated an interrupt CCP1 0h = Event Cleared 1h = Event Set
4	CCD0	R	0h	Capture or compare down event generated an interrupt CCP0 0h = Event Cleared 1h = Event Set
3-2	RESERVED	R	0h	
1	L	R	0h	Load event generated an interrupt. 0h = Event Cleared 1h = Event Set
0	Z	R	0h	Zero event generated an interrupt. 0h = Event Cleared 1h = Event Set

25.3.21 ISET (Offset = 1070h) [Reset = 00000000h]

ISET is shown in [Figure 25-61](#) and described in [Table 25-48](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 25-61. ISET

31	30	29	28	27	26	25	24
RESERVED			QEIRR	DC	REPC	TOV	F
W-0h			W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
CCU5	CCU4	CCD5	CCD4	CCU3	CCU2	CCU1	CCU0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
CCD3	CCD2	CCD1	CCD0	RESERVED		L	Z
W-0h	W-0h	W-0h	W-0h	W-0h		W-0h	W-0h

Table 25-48. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	W	0h	
28	QEIRR	W	0h	QEIRR event SET 0h = Writing 0 has no effect. 1h = Event Set
27	DC	W	0h	Direction Change event SET 0h = Writing 0 has no effect. 1h = Event Set
26	REPC	W	0h	Repeat Counter Zero event SET 0h = Writing 0 has no effect. 1h = Event Set
25	TOV	W	0h	Trigger Overflow event SET 0h = Writing 0 has no effect. 1h = Event Set
24	F	W	0h	Fault event SET 0h = Writing 0 has no effect. 1h = Event Set
23-16	RESERVED	W	0h	
15	CCU5	W	0h	Compare up event 5 SET 0h = Writing 0 has no effect. 1h = Event Set
14	CCU4	W	0h	Compare up event 4 SET 0h = Writing 0 has no effect. 1h = Event Set
13	CCD5	W	0h	Compare down event 5 SET 0h = Writing 0 has no effect. 1h = Event Set
12	CCD4	W	0h	Compare down event 4 SET 0h = Writing 0 has no effect. 1h = Event Set

Table 25-48. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	CCU3	W	0h	Capture or compare up event SET 0h = Writing 0 has no effect. 1h = Event Set
10	CCU2	W	0h	Capture or compare up event SET 0h = Writing 0 has no effect. 1h = Event Set
9	CCU1	W	0h	Capture or compare up event SET 0h = Writing 0 has no effect. 1h = Event Set
8	CCU0	W	0h	Capture or compare up event SET 0h = Writing 0 has no effect. 1h = Event Set
7	CCD3	W	0h	Capture or compare down event SET 0h = Writing 0 has no effect. 1h = Event Set
6	CCD2	W	0h	Capture or compare down event SET 0h = Writing 0 has no effect. 1h = Event Set
5	CCD1	W	0h	Capture or compare down event SET 0h = Writing 0 has no effect. 1h = Event Set
4	CCD0	W	0h	Capture or compare down event SET 0h = Writing 0 has no effect. 1h = Event Set
3-2	RESERVED	W	0h	
1	L	W	0h	Load event SET 0h = Writing 0 has no effect. 1h = Event Set
0	Z	W	0h	Zero event SET 0h = Writing 0 has no effect. 1h = Event Set

25.3.22 ICLR (Offset = 1078h) [Reset = 00000000h]

ICLR is shown in [Figure 25-62](#) and described in [Table 25-49](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 25-62. ICLR

31	30	29	28	27	26	25	24
RESERVED			QEIRR	DC	REPC	TOV	F
W-0h			W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
CCU5	CCU4	CCD5	CCD4	CCU3	CCU2	CCU1	CCU0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
CCD3	CCD2	CCD1	CCD0	RESERVED		L	Z
W-0h	W-0h	W-0h	W-0h	W-0h		W-0h	W-0h

Table 25-49. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	W	0h	
28	QEIRR	W	0h	QEIRR event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
27	DC	W	0h	Direction Change event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
26	REPC	W	0h	Repeat Counter Zero event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
25	TOV	W	0h	Trigger Overflow event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
24	F	W	0h	Fault event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
23-16	RESERVED	W	0h	
15	CCU5	W	0h	Compare up event 5 CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
14	CCU4	W	0h	Compare up event 4 CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
13	CCD5	W	0h	Compare down event 5 CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
12	CCD4	W	0h	Compare down event 4 CLEAR 0h = Writing 0 has no effect. 1h = Event Clear

Table 25-49. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	CCU3	W	0h	Capture or compare up event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
10	CCU2	W	0h	Capture or compare up event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
9	CCU1	W	0h	Capture or compare up event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
8	CCU0	W	0h	Capture or compare up event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
7	CCD3	W	0h	Capture or compare down event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
6	CCD2	W	0h	Capture or compare down event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
5	CCD1	W	0h	Capture or compare down event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
4	CCD0	W	0h	Capture or compare down event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
3-2	RESERVED	W	0h	
1	L	W	0h	Load event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
0	Z	W	0h	Zero event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear

25.3.23 IIDX (Offset = 1080h) [Reset = 00000000h]

IIDX is shown in [Figure 25-63](#) and described in [Table 25-50](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 25-63. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 25-50. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 00h = No interrupt pending 01h = Interrupt Source: Zero event (Z) 02h = Interrupt Source: Load event (L) 05h = Interrupt Source: Capture or compare down event (CCD0) 06h = Interrupt Source: Capture or compare down event (CCD1) 07h = Interrupt Source: Capture or compare down event (CCD2) 08h = Interrupt Source: Capture or compare down event (CCD3) 09h = Interrupt Source: Capture or compare up event (CCU0) 0Ah = Interrupt Source: Capture or compare up event (CCU1) 0Bh = Interrupt Source: Capture or compare up event (CCU2) 0Ch = Interrupt Source: Capture or compare up event (CCU3) 0Dh = Interrupt Source: Compare down event (CCD4) 0Eh = Interrupt Source: Compare down event (CCD5) 0Fh = Interrupt Source: Compare down event (CCU4) 10h = Interrupt Source: Compare down event (CCU5) 19h = Interrupt Source: Fault Event generated an interrupt. (F) 1Ah = Interrupt Source: Trigger overflow (TOV) 1Bh = Interrupt Source: Repeat Counter Zero (REPC) 1Ch = Interrupt Source: Direction Change (DC) 1Dh = Interrupt Source: QEI Incorrect state transition error (QEIERR)

25.3.24 IMASK (Offset = 1088h) [Reset = 00000000h]

IMASK is shown in [Figure 25-64](#) and described in [Table 25-51](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.”

Figure 25-64. IMASK

31	30	29	28	27	26	25	24
RESERVED			QEIERR	DC	REPC	TOV	F
R/W-			R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
CCU5	CCU4	CCD5	CCD4	CCU3	CCU2	CCU1	CCU0
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
CCD3	CCD2	CCD1	CCD0	RESERVED		L	Z
R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-		R/W-0h	R/W-0h

Table 25-51. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R/W	0h	
28	QEIERR	R/W	0h	QEIERR Event mask 0h = Disable Event 1h = Enable Event
27	DC	R/W	0h	Direction Change Event mask 0h = Disable Event 1h = Enable Event
26	REPC	R/W	0h	Repeat Counter Zero Event mask 0h = Disable Event 1h = Enable Event
25	TOV	R/W	0h	Trigger Overflow Event mask 0h = Disable Event 1h = Enable Event
24	F	R/W	0h	Fault Event mask 0h = Disable Event 1h = Enable Event
23-16	RESERVED	R/W	0h	
15	CCU5	R/W	0h	Compare UP event mask CCP5 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
14	CCU4	R/W	0h	Compare UP event mask CCP4 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
13	CCD5	R/W	0h	Compare DN event mask CCP5 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
12	CCD4	R/W	0h	Compare DN event mask CCP4 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

Table 25-51. IMASK Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	CCU3	R/W	0h	Capture or Compare UP event mask CCP3 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
10	CCU2	R/W	0h	Capture or Compare UP event mask CCP2 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
9	CCU1	R/W	0h	Capture or Compare UP event mask CCP1 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
8	CCU0	R/W	0h	Capture or Compare UP event mask CCP0 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
7	CCD3	R/W	0h	Capture or Compare DN event mask CCP3 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
6	CCD2	R/W	0h	Capture or Compare DN event mask CCP2 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
5	CCD1	R/W	0h	Capture or Compare DN event mask CCP1 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
4	CCD0	R/W	0h	Capture or Compare DN event mask CCP0 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
3-2	RESERVED	R/W	0h	
1	L	R/W	0h	Load Event mask 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	Z	R/W	0h	Zero Event mask 0h = Disable Event 1h = Enable Event

25.3.25 RIS (Offset = 1090h) [Reset = 00000000h]

RIS is shown in [Figure 25-65](#) and described in [Table 25-52](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 25-65. RIS

31	30	29	28	27	26	25	24
RESERVED			QEIERR	DC	REPC	TOV	F
R-0h			R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
CCU5	CCU4	CCD5	CCD4	CCU3	CCU2	CCU1	CCU0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
CCD3	CCD2	CCD1	CCD0	RESERVED		L	Z
R-0h	R-0h	R-0h	R-0h	R-0h		R-0h	R-0h

Table 25-52. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R	0h	
28	QEIERR	R	0h	QEIERR, set on an incorrect state transition on the encoder interface. 0h = Event Cleared 1h = Event Set
27	DC	R	0h	Direction Change 0h = Event Cleared 1h = Event Set
26	REPC	R	0h	Repeat Counter Zero 0h = Event Cleared 1h = Event Set
25	TOV	R	0h	Trigger overflow 0h = Event Cleared 1h = Event Set
24	F	R	0h	Fault 0h = Event Cleared 1h = Event Set
23-16	RESERVED	R	0h	
15	CCU5	R	0h	Compare up event generated an interrupt CCP5 0h = Event Cleared 1h = Event Set
14	CCU4	R	0h	Compare up event generated an interrupt CCU4 0h = Event Cleared 1h = Event Set
13	CCD5	R	0h	Compare down event generated an interrupt CCD5 0h = Event Cleared 1h = Event Set

Table 25-52. RIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
12	CCD4	R	0h	Compare down event generated an interrupt CCD4 0h = Event Cleared 1h = Event Set
11	CCU3	R	0h	Capture or compare up event generated an interrupt CCP3 0h = Event Cleared 1h = Event Set
10	CCU2	R	0h	Capture or compare up event generated an interrupt CCP2 0h = Event Cleared 1h = Event Set
9	CCU1	R	0h	Capture or compare up event generated an interrupt CCP1 0h = Event Cleared 1h = Event Set
8	CCU0	R	0h	Capture or compare up event generated an interrupt CCP0 0h = Event Cleared 1h = Event Set
7	CCD3	R	0h	Capture or compare down event generated an interrupt CCP3 0h = Event Cleared 1h = Event Set
6	CCD2	R	0h	Capture or compare down event generated an interrupt CCP2 0h = Event Cleared 1h = Event Set
5	CCD1	R	0h	Capture or compare down event generated an interrupt CCP1 0h = Event Cleared 1h = Event Set
4	CCD0	R	0h	Capture or compare down event generated an interrupt CCP0 0h = Event Cleared 1h = Event Set
3-2	RESERVED	R	0h	
1	L	R	0h	Load event generated an interrupt. 0h = Event Cleared 1h = Event Set
0	Z	R	0h	Zero event generated an interrupt. 0h = Event Cleared 1h = Event Set

25.3.26 MIS (Offset = 1098h) [Reset = 00000000h]

MIS is shown in [Figure 25-66](#) and described in [Table 25-53](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 25-66. MIS

31	30	29	28	27	26	25	24
RESERVED			QEIRR	DC	REPC	TOV	F
R-0h			R-0h	R-0h	R-0h	R-0h	R-0h
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
CCU5	CCU4	CCD5	CCD4	CCU3	CCU2	CCU1	CCU0
R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h	R-0h
7	6	5	4	3	2	1	0
CCD3	CCD2	CCD1	CCD0	RESERVED		L	Z
R-0h	R-0h	R-0h	R-0h	R-0h		R-0h	R-0h

Table 25-53. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	R	0h	
28	QEIRR	R	0h	QEIRR 0h = Event Cleared 1h = Event Set
27	DC	R	0h	Direction Change 0h = Event Cleared 1h = Event Set
26	REPC	R	0h	Repeat Counter Zero 0h = Event Cleared 1h = Event Set
25	TOV	R	0h	Trigger overflow 0h = Event Cleared 1h = Event Set
24	F	R	0h	Fault 0h = Event Cleared 1h = Event Set
23-16	RESERVED	R	0h	
15	CCU5	R	0h	Compare up event generated an interrupt CCP5 0h = Event Cleared 1h = Event Set
14	CCU4	R	0h	Compare up event generated an interrupt CCP4 0h = Event Cleared 1h = Event Set
13	CCD5	R	0h	Compare down event generated an interrupt CCP5 0h = Event Cleared 1h = Event Set
12	CCD4	R	0h	Compare down event generated an interrupt CCP4 0h = Event Cleared 1h = Event Set

Table 25-53. MIS Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	CCU3	R	0h	Capture or compare up event generated an interrupt CCP3 0h = Event Cleared 1h = Event Set
10	CCU2	R	0h	Capture or compare up event generated an interrupt CCP2 0h = Event Cleared 1h = Event Set
9	CCU1	R	0h	Capture or compare up event generated an interrupt CCP1 0h = Event Cleared 1h = Event Set
8	CCU0	R	0h	Capture or compare up event generated an interrupt CCP0 0h = Event Cleared 1h = Event Set
7	CCD3	R	0h	Capture or compare down event generated an interrupt CCP3 0h = Event Cleared 1h = Event Set
6	CCD2	R	0h	Capture or compare down event generated an interrupt CCP2 0h = Event Cleared 1h = Event Set
5	CCD1	R	0h	Capture or compare down event generated an interrupt CCP1 0h = Event Cleared 1h = Event Set
4	CCD0	R	0h	Capture or compare down event generated an interrupt CCP0 0h = Event Cleared 1h = Event Set
3-2	RESERVED	R	0h	
1	L	R	0h	Load event generated an interrupt. 0h = Event Cleared 1h = Event Set
0	Z	R	0h	Zero event generated an interrupt. 0h = Event Cleared 1h = Event Set

25.3.27 ISET (Offset = 10A0h) [Reset = 00000000h]

ISET is shown in [Figure 25-67](#) and described in [Table 25-54](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 25-67. ISET

31	30	29	28	27	26	25	24
RESERVED			QEIRR	DC	REPC	TOV	F
W-0h			W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
CCU5	CCU4	CCD5	CCD4	CCU3	CCU2	CCU1	CCU0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
CCD3	CCD2	CCD1	CCD0	RESERVED		L	Z
W-0h	W-0h	W-0h	W-0h	W-0h		W-0h	W-0h

Table 25-54. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	W	0h	
28	QEIRR	W	0h	QEIRR event SET 0h = Writing 0 has no effect. 1h = Event Set
27	DC	W	0h	Direction Change event SET 0h = Writing 0 has no effect. 1h = Event Set
26	REPC	W	0h	Repeat Counter Zero event SET 0h = Writing 0 has no effect. 1h = Event Set
25	TOV	W	0h	Trigger Overflow event SET 0h = Writing 0 has no effect. 1h = Event Set
24	F	W	0h	Fault event SET 0h = Writing 0 has no effect. 1h = Event Set
23-16	RESERVED	W	0h	
15	CCU5	W	0h	Compare up event 5 SET 0h = Writing 0 has no effect. 1h = Event Set
14	CCU4	W	0h	Compare up event 4 SET 0h = Writing 0 has no effect. 1h = Event Set
13	CCD5	W	0h	Compare down event 5 SET 0h = Writing 0 has no effect. 1h = Event Set
12	CCD4	W	0h	Compare down event 4 SET 0h = Writing 0 has no effect. 1h = Event Set

Table 25-54. ISET Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	CCU3	W	0h	Capture or compare up event SET 0h = Writing 0 has no effect. 1h = Event Set
10	CCU2	W	0h	Capture or compare up event SET 0h = Writing 0 has no effect. 1h = Event Set
9	CCU1	W	0h	Capture or compare up event SET 0h = Writing 0 has no effect. 1h = Event Set
8	CCU0	W	0h	Capture or compare up event SET 0h = Writing 0 has no effect. 1h = Event Set
7	CCD3	W	0h	Capture or compare down event SET 0h = Writing 0 has no effect. 1h = Event Set
6	CCD2	W	0h	Capture or compare down event SET 0h = Writing 0 has no effect. 1h = Event Set
5	CCD1	W	0h	Capture or compare down event SET 0h = Writing 0 has no effect. 1h = Event Set
4	CCD0	W	0h	Capture or compare down event SET 0h = Writing 0 has no effect. 1h = Event Set
3-2	RESERVED	W	0h	
1	L	W	0h	Load event SET 0h = Writing 0 has no effect. 1h = Event Set
0	Z	W	0h	Zero event SET 0h = Writing 0 has no effect. 1h = Event Set

25.3.28 ICLR (Offset = 10A8h) [Reset = 00000000h]

ICLR is shown in [Figure 25-68](#) and described in [Table 25-55](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 25-68. ICLR

31	30	29	28	27	26	25	24
RESERVED			QEIRR	DC	REPC	TOV	F
W-0h			W-0h	W-0h	W-0h	W-0h	W-0h
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
CCU5	CCU4	CCD5	CCD4	CCU3	CCU2	CCU1	CCU0
W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h	W-0h
7	6	5	4	3	2	1	0
CCD3	CCD2	CCD1	CCD0	RESERVED		L	Z
W-0h	W-0h	W-0h	W-0h	W-0h		W-0h	W-0h

Table 25-55. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-29	RESERVED	W	0h	
28	QEIRR	W	0h	QEIRR event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
27	DC	W	0h	Direction Change event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
26	REPC	W	0h	Repeat Counter Zero event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
25	TOV	W	0h	Trigger Overflow event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
24	F	W	0h	Fault event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
23-16	RESERVED	W	0h	
15	CCU5	W	0h	Compare up event 5 CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
14	CCU4	W	0h	Compare up event 4 CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
13	CCD5	W	0h	Compare down event 5 CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
12	CCD4	W	0h	Compare down event 4 CLEAR 0h = Writing 0 has no effect. 1h = Event Clear

Table 25-55. ICLR Field Descriptions (continued)

Bit	Field	Type	Reset	Description
11	CCU3	W	0h	Capture or compare up event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
10	CCU2	W	0h	Capture or compare up event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
9	CCU1	W	0h	Capture or compare up event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
8	CCU0	W	0h	Capture or compare up event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
7	CCD3	W	0h	Capture or compare down event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
6	CCD2	W	0h	Capture or compare down event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
5	CCD1	W	0h	Capture or compare down event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
4	CCD0	W	0h	Capture or compare down event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
3-2	RESERVED	W	0h	
1	L	W	0h	Load event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear
0	Z	W	0h	Zero event CLEAR 0h = Writing 0 has no effect. 1h = Event Clear

25.3.29 EVT_MODE (Offset = 10E0h) [Reset = 00000029h]

EVT_MODE is shown in [Figure 25-69](#) and described in [Table 25-56](#).

Return to the [Summary Table](#).

Event mode register. It is used to select whether each line is disabled, in software mode (software clears the RIS) or in hardware mode (hardware clears the RIS)

Figure 25-69. EVT_MODE

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED		EVT2_CFG		EVT1_CFG		EVT0_CFG	
R/W-0h		R-2h		R-2h		R-1h	

Table 25-56. EVT_MODE Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R/W	0h	
5-4	EVT2_CFG	R	2h	Event line mode select for event corresponding to GEN_EVENT1 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
3-2	EVT1_CFG	R	2h	Event line mode select for event corresponding to GEN_EVENT0 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
1-0	EVT0_CFG	R	1h	Event line mode select for event corresponding to CPU_INT 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.

25.3.30 DESC (Offset = 10FCh) [Reset = 11110000h]

DESC is shown in [Figure 25-70](#) and described in [Table 25-57](#).

Return to the [Summary Table](#).

This register identifies the peripheral and its exact version.

Figure 25-70. DESC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODULEID															
R-1111h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FEATUREVER				INSTNUM				MAJREV				MINREV			
R-				R-				R-				R-			

Table 25-57. DESC Field Descriptions

Bit	Field	Type	Reset	Description
31-16	MODULEID	R	1111h	Module identification contains a unique peripheral identification number. The assignments are maintained in a central database for all of the platform modules to ensure uniqueness. 0h = Smallest value FFFFh = Highest possible value
15-12	FEATUREVER	R	0h	Feature Set for the module *instance* 0h = Smallest value Fh = Highest possible value
11-8	INSTNUM	R	0h	Instance Number within the device. This will be a parameter to the RTL for modules that can have multiple instances 0h = Smallest value Fh = Highest possible value
7-4	MAJREV	R	0h	Major rev of the IP 0h = Smallest value Fh = Highest possible value
3-0	MINREV	R	0h	Minor rev of the IP 0h = Smallest value Fh = Highest possible value

25.3.31 CCPD (Offset = 1100h) [Reset = 00000000h]

CCPD is shown in [Figure 25-71](#) and described in [Table 25-58](#).

Return to the [Summary Table](#).

CCP Direction. Controls whether CCP is used as an input or an output.

Figure 25-71. CCPD

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED				C0CCP3	C0CCP2	C0CCP1	C0CCP0
R/W-				R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 25-58. CCPD Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3	C0CCP3	R/W	0h	CCP3 direction 0h = Input 1h = Output
2	C0CCP2	R/W	0h	CCP2 direction 0h = input 1h = Output
1	C0CCP1	R/W	0h	CCP1 direction 0h = Input 1h = Output
0	C0CCP0	R/W	0h	CCP0 direction 0h = Input 1h = Output

25.3.32 ODIS (Offset = 1104h) [Reset = 00000000h]

ODIS is shown in [Figure 25-72](#) and described in [Table 25-59](#).

Return to the [Summary Table](#).

The ODIS register output is inverted and then ANDed with the output signal selected by the OCTL register CCPO field (before conditional inversion) to allow software the ability to hold the CCP output low during configuration or shutdown.

Figure 25-72. ODIS

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED				C0CCP3	C0CCP2	C0CCP1	C0CCP0
R/W-				R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 25-59. ODIS Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3	C0CCP3	R/W	0h	Counter CCP3 Disable Mask Defines whether CCP3 of Counter n is forced low or not 0h = Output function as selected by the OCTL register CCPO field are provided to output inversion block. 1h = CCP output is forced low.
2	C0CCP2	R/W	0h	Counter CCP2 Disable Mask Defines whether CCP2 of Counter n is forced low or not 0h = Output function as selected by the OCTL register CCPO field are provided to output inversion block. 1h = CCP output is forced low.
1	C0CCP1	R/W	0h	Counter CCP1 Disable Mask Defines whether CCP0 of Counter n is forced low or not 0h = Output function as selected by the OCTL register CCPO field are provided to output inversion block. 1h = CCP output is forced low.
0	C0CCP0	R/W	0h	Counter CCP0 Disable Mask Defines whether CCP0 of Counter n is forced low or not 0h = Output function as selected by the OCTL register CCPO field are provided to output inversion block. 1h = CCP output is forced low.

25.3.33 CCLKCTL (Offset = 1108h) [Reset = 00000000h]

CCLKCTL is shown in [Figure 25-73](#) and described in [Table 25-60](#).

Return to the [Summary Table](#).

The CCLKCTL register provides a SW mechanism for gating the TIMER clock if the module is expected not to be used but the power domain is alive.

This effectively puts the IP in an IDLE state

Figure 25-73. CCLKCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED							CLKEN
R/W-							R/W-0h

Table 25-60. CCLKCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R/W	0h	
0	CLKEN	R/W	0h	Clock Enable Disables the clock gating to the module. SW has to explicitly program the value to 0 to gate the clock. 0h = Clock is disabled. 1h = Clock is enabled

25.3.34 CPS (Offset = 110Ch) [Reset = 00000000h]

CPS is shown in [Figure 25-74](#) and described in [Table 25-61](#).

Return to the [Summary Table](#).

The CPS register provides the value for the clock pre-scaler.

Figure 25-74. CPS

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								PCNT							
R/W-																								R/W-0h							

Table 25-61. CPS Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R/W	0h	
7-0	PCNT	R/W	0h	Pre-Scale Count This field specifies the pre-scale count value. The selected TIMCLK source is divided by a value of (PCNT+1). A PCNT value of 0 divides TIMCLK by 1, effectively bypassing the divider. A PCNT value of greater than 0 divides the TIMCLK source generating a slower clock 0h = Minimum value FFh = Maximum Value

25.3.35 CPSV (Offset = 1110h) [Reset = 00000000h]

CPSV is shown in [Figure 25-75](#) and described in [Table 25-62](#).

Return to the [Summary Table](#).

The CPSV register provides the ability to read the current clock prescale count value.

Figure 25-75. CPSV

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								CPSVAL							
R-																								R-0h							

Table 25-62. CPSV Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	CPSVAL	R	0h	Current Prescale Count Value 0h = Minimum value FFh = Maximum Value

25.3.36 CTTRIGCTL (Offset = 1114h) [Reset = 00000000h]

CTTRIGCTL is shown in [Figure 25-76](#) and described in [Table 25-63](#).

Return to the [Summary Table](#).

Cross Timer Trigger Control Register

This register is used to control the cross trigger connections for enables and faults of different timer instances in the same power domain. Please refer to sections Timer Module Cross Trigger (In/Out) and Fault Cross Triggering for details.

Figure 25-76. CTTRIGCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED				EVTCTTRIGSEL			
R/W-				R/W-0h			
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED						EVTCTEN	CTEN
R/W-						R/W-0h	R/W-0h

Table 25-63. CTTRIGCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-20	RESERVED	R/W	0h	
19-16	EVTCTTRIGSEL	R/W	0h	Used to Select the subscriber port that should be used for input cross trigger. 0h = Use FSUB0 as cross trigger source. 1h = Use FSUB1 as cross trigger source. 2h = Use Zero event as cross trigger source. 3h = Use Load event as cross trigger source. 4h = Use CCD0 event as cross trigger source. 5h = Use CCD1 event as cross trigger source. 6h = Use CCD2 event as cross trigger source. 7h = Use CCD3 event as cross trigger source. 8h = Use CCU0 event as cross trigger source. 9h = Use CCU1 event as cross trigger source. Ah = Use CCU2 event as cross trigger source. Bh = Use CCU3 event as cross trigger source.
15-2	RESERVED	R/W	0h	
1	EVTCTEN	R/W	0h	Enable the Input Trigger Conditions to the Timer module as a condition for Cross Triggers. 0h = Cross trigger generation disabled. 1h = Cross trigger generation enabled
0	CTEN	R/W	0h	Timer Cross trigger enable. This field is used to enable whether the SW or HW logic can generate a timer cross trigger event in the system. These cross triggers are connected to the respective timer trigger in of the other timer IPs in the SOC power domain. The timer cross trigger is essentially the combined logic of the HW and SW conditions controlling EN bit in the CTRCTL register. 0h = Cross trigger generation disabled. 1h = Cross trigger generation enabled

25.3.37 CTTRIG (Offset = 111Ch) [Reset = 00000000h]

CTTRIG is shown in [Figure 25-77](#) and described in [Table 25-64](#).

Return to the [Summary Table](#).

Cross Timer Trigger Register

This register is used to trigger the timer instances connected and enabled using CTTRIGCTL and CTTRIGMSK registers.

Figure 25-77. CTTRIG

31	30	29	28	27	26	25	24
RESERVED							
W-							
23	22	21	20	19	18	17	16
RESERVED							
W-							
15	14	13	12	11	10	9	8
RESERVED							
W-							
7	6	5	4	3	2	1	0
RESERVED							TRIG
W-							W-0h

Table 25-64. CTTRIG Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	W	0h	
0	TRIG	W	0h	Generate Cross Trigger This bit when programmed will generate a synchronized trigger condition all the cross trigger enabled Timer instances including current timer instance. 0h = Cross trigger generation disabled 1h = Generate Cross trigger pulse

25.3.38 FSCTL (Offset = 1120h) [Reset = 00000000h]

FSCTL is shown in [Figure 25-78](#) and described in [Table 25-65](#).

Return to the [Summary Table](#).

The FSCTL register controls the fault source selection and enable. There are 5 input fault sources either through synchronous path processing or asynchronous path.

Figure 25-78. FSCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED	FEX2EN	FEX1EN	FEX0EN	FAC2EN	FAC1EN	FAC0EN	FCEN
R/W-	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 25-65. FSCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-7	RESERVED	R/W	0h	
6	FEX2EN	R/W	0h	This field controls whether the fault is caused by external fault pin 2. 0h = Disable 1h = Enable
5	FEX1EN	R/W	0h	This field controls whether the fault is caused by external fault pin 1. 0h = Disable 1h = Enable
4	FEX0EN	R/W	0h	This field controls whether the fault is caused by external fault pin 0. 0h = Disable 1h = Enable
3	FAC2EN	R/W	0h	This field controls whether the fault is caused by COMP2 output. 0h = Disable 1h = Enable
2	FAC1EN	R/W	0h	This field controls whether the fault is caused by COMP1 output. 0h = Disable 1h = Enable
1	FAC0EN	R/W	0h	This field controls whether the fault signal is caused by COMP0 output. 0h = Disable 1h = Enable
0	FCEN	R/W	0h	This field controls whether the fault is caused by the system clock fault. 0h = Disable 1h = Enable

25.3.39 GCTL (Offset = 1124h) [Reset = 00000001h]

GCTL is shown in [Figure 25-79](#) and described in [Table 25-66](#).

Return to the [Summary Table](#).

Global control register

Figure 25-79. GCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED							SHDWLDEN
R/W-							R/W-1h

Table 25-66. GCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R/W	0h	
0	SHDWLDEN	R/W	1h	Enables shadow to active load of buffered registers and register fields. 0h = Disable 1h = Enable

25.3.40 CTR (Offset = 1800h) [Reset = 00000000h]

CTR is shown in [Figure 25-80](#) and described in [Table 25-67](#).

Return to the [Summary Table](#).

This is the TIMER counter register.

This can be set by SW. However, the writes will be unpredictable if the software tries to set a value while the counter is running.

Figure 25-80. CTR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																CCTR															
R/W-0h																R/W-0h															

Table 25-67. CTR Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15-0	CCTR	R/W	0h	Current Counter value 0h = Minimum value 00FFFFFFh = Maximum Value

25.3.41 CTRCTL (Offset = 1804h) [Reset = 0000FF80h]

CTRCTL is shown in [Figure 25-81](#) and described in [Table 25-68](#).

Return to the [Summary Table](#).

This register provides control over the counter operation.
The configuration can change as well as setting the EN bit in a single write.
There is no requirement to change the configuration first and then do an additional write to set the EN bit.

Figure 25-81. CTRCTL

31	30	29	28	27	26	25	24
RESERVED		CVAE		RESERVED		PLEN	
R/W-0h		R/W-0h		R/W-0h		R/W-0h	
23	22	21	20	19	18	17	16
SLZERCNEZ	RESERVED			FRB	FB	DRB	RESERVED
R/W-0h		R/W-0h		R/W-0h	R/W-0h	R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
CZC			CAC			CLC	
R/W-7h			R/W-7h			R/W-7h	
7	6	5	4	3	2	1	0
CLC	RESERVED	CM		REPEAT		EN	
R/W-7h	R/W-0h	R/W-0h		R/W-0h		R/W-0h	

Table 25-68. CTRCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-30	RESERVED	R/W	0h	
29-28	CVAE	R/W	0h	Counter Value After Enable. This field specifies the initialization condition of the counter when the EN bit is changed from 0 to 1 by a write to the CTRCTL register. Note that an external event can also cause the EN bit to go active. 0h = The counter is set to the LOAD register value 1h = The counter value is unchanged from its current value which could have been initialized by software 2h = The counter is set to zero
27-25	RESERVED	R/W	0h	
24	PLEN	R/W	0h	Phase Load Enable. This bit allows the timer to have phase load feature. 0h = Disabled 1h = Enabled
23	SLZERCNEZ	R/W	0h	Suppress Load and Zero Events if Repeat Counter is Not Equal to Zero. This bit suppresses the generation of the Z (zero) and L (load) events from the counter when the repeat counter (RC) value is not 0. 0h = Disabled. Z and L events are always generated from the counter when their conditions are generated. 1h = Enabled. Z and L events are generated from the counter when their conditions are generated and the RC register value is 0.
22-20	RESERVED	R/W	0h	
19	FRB	R/W	0h	Fault Resume Behavior This bit specifies what the device does following the release/exit of fault condition. 0h = Resume counting 1h = Perform the action as specified by the CVAE field.

Table 25-68. CTRCTL Field Descriptions (continued)

Bit	Field	Type	Reset	Description
18	FB	R/W	0h	Fault Behavior This bit specifies whether the counter continues running or suspends during a fault mode. There is a separate control under REPEAT to indicate whether counting is to suspend at next Counter==0 0h = Continues counting 1h = Suspends counting
17	DRB	R/W	0h	Debug Resume Behavior This bit specifies what the device does following the release/exit of debug mode. 0h = Resume counting 1h = Perform the action as specified by the CVAE field.
16	RESERVED	R/W	0h	
15-13	CZC	R/W	7h	Counter Zero Control This field specifies what controls the counter operation with respect to zeroing the counter value. Encodings 1-3 are present based on the CCPC parameter value. Bits 4-5 are present based on the HQEI parameter value. Any encodings not provided are documented as reserved. 0h = CCCTL_0 ZCOND 1h = CCCTL_1 ZCOND 2h = CCCTL_2 ZCOND This value exists when there are 4 channels. 3h = CCCTL_3 ZCOND This value exists when there are 4 channels. 4h = Controlled by 2-input QEI mode This value exists when TIMER support QEI feature. 5h = Controlled by 3-input QEI mode This value exists when TIMER support QEI feature.
12-10	CAC	R/W	7h	Counter Advance Control. This field specifies what controls the counter operation with respect to advancing (incrementing or decrementing) the counter value. Encodings 1-3 are present based on the CCPC parameter value. Bits 4-5 are present based on the HQEI parameter value. Any encodings not provided are documented as reserved. 0h = CCCTL_0 ACOND 1h = CCCTL_1 ACOND 2h = CCCTL_2 ACOND This value exists when there are 4 channels. 3h = CCCTL_3 ACOND This value exists when there are 4 channels. 4h = Controlled by 2-input QEI mode This value exists when TIMER support QEI feature. 5h = Controlled by 3-input QEI mode This value exists when TIMER support QEI feature.
9-7	CLC	R/W	7h	Counter Load Control. This field specifies what controls the counter operation with respect to setting the counter to the LD register value. Encodings 1-3 are present based on the CCPC parameter value. Bits 4-5 are present based on the HQEI parameter value. Any encodings not provided are documented as reserved. 0h = CCCTL_0 LCOND 1h = CCCTL_1 LCOND 2h = CCCTL_2 LCOND This value exists when there are 4 channels. 3h = CCCTL_3 LCOND This value exists when there are 4 channels. 4h = Controlled by 2 input QEI mode. This value exists when TIMER support QEI feature. 5h = Controlled by 3 input QEI mode. This value exists when TIMER support QEI feature.
6	RESERVED	R/W	0h	

Table 25-68. CTRCTL Field Descriptions (continued)

Bit	Field	Type	Reset	Description
5-4	CM	R/W	0h	Count Mode 0h = Down 1h = Up/Down 2h = Counter counts up.
3-1	REPEAT	R/W	0h	Repeat. The repeat bit controls whether the counter continues to advance following a zero event, or the exiting of a debug or fault condition. If counting down, a zero event is followed by a load at the next advance condition. If counting up-down, a zero event is followed by an advance event (+1). The intent of encoding 3 is that if the debug condition is in effect, the generation of the load pulse is deferred until the debug condition is over. This allows the counter to reach zero before counting is suspended. 0h = Does not automatically advance following a zero event. 1h = Continues to advance following a zero event. 2h = Reserved 3h = Continues to advance following a zero event if the debug mode is not in effect, or following the release of the debug mode. 4h = Reserved
0	EN	R/W	0h	Counter Enable. This bit allows the timer to advance This bit is automatically cleared if REPEAT=0 (do not automatically reload) and the counter value equals zero. CPU Write: A register write that sets the EN bit, the counter value is set per the CVAE value. Hardware: This bit may also be set as the result of an LCOND or ZCOND condition being met and the counter value changed to the load value or zero value, respectively. 0h = Disabled 1h = Enabled

25.3.42 LOAD (Offset = 1808h) [Reset = 00000000h]

LOAD is shown in [Figure 25-82](#) and described in [Table 25-69](#).

Return to the [Summary Table](#).

The contents of LOAD register are copied to CTR on any operation designated to do a "LOAD". The LOAD is used to compare with the CTR for generating a "Load Event" that can be used for interrupt, trigger, or signal generator actions.

Figure 25-82. LOAD

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																LD															
R/W-0h																R/W-0h															

Table 25-69. LOAD Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15-0	LD	R/W	0h	Load Value 0h = Minimum value 00FFFFFFh = Maximum Value

25.3.43 CC_01[y] (Offset = 1810h + formula) [Reset = 00000000h]

CC_01[y] is shown in [Figure 25-83](#) and described in [Table 25-70](#).

Return to the [Summary Table](#).

The CC_01 register is a register that can be used as either a capture register, to capture the next CTR value on an event, or a compare to the current CTR to create an event. It cannot operate concurrently as both. There are two Capture-Compare slices of hardware for each counter, hence there are two CC_01 registers per timer. On a capture event, the next value of the CTR is loaded so that CTR and CC_01 (which captured) will be equal on the cycle that an interrupt or trigger is created from the capture action.

Offset = 1810h + (y * 4h); where y = 0h to 1h

Figure 25-83. CC_01[y]

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																CCVAL															
R/W-0h																R/W-0h															

Table 25-70. CC_01[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15-0	CCVAL	R/W	0h	Capture or compare value 0h = Minimum value FFFFh = Maximum Value

25.3.44 CC_23[y] (Offset = 1818h + formula) [Reset = 00000000h]

CC_23[y] is shown in [Figure 25-84](#) and described in [Table 25-71](#).

Return to the [Summary Table](#).

The CC_23 register is a register that can be used as either a capture register, to capture the next CTR value on an event, or a compare to the current CTR to create an event. It cannot operate concurrently as both. There are two Capture-Compare slices of hardware for each counter, hence there are two CC_01 registers per timer. On a capture event, the next value of the CTR is loaded so that CTR and CC_01 (which captured) will be equal on the cycle that an interrupt or trigger is created from the capture action.

Offset = 1818h + (y * 4h); where y = 0h to 1h

Figure 25-84. CC_23[y]

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																CCVAL															
R/W-0h																R/W-0h															

Table 25-71. CC_23[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15-0	CCVAL	R/W	0h	Capture or compare value 0h = Minimum value FFFFh = Maximum Value

25.3.45 CC_45[y] (Offset = 1820h + formula) [Reset = 00000000h]

CC_45[y] is shown in [Figure 25-85](#) and described in [Table 25-72](#).

Return to the [Summary Table](#).

The CC_45 register are a registers which can be used as compare to the current CTR to create an events CC4U, CC4D, CC5U and CC5D.

Offset = 1820h + (y * 4h); where y = 0h to 1h

Figure 25-85. CC_45[y]

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																CCVAL															
R/W-0h																R/W-0h															

Table 25-72. CC_45[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15-0	CCVAL	R/W	0h	Capture or compare value 0h = Minimum value FFFFh = Maximum Value

25.3.46 CCCTL_01[y] (Offset = 1830h + formula) [Reset = 00000000h]

CCCTL_01[y] is shown in [Figure 25-86](#) and described in [Table 25-73](#).

Return to the [Summary Table](#).

The CCCTL_01 registers control the operations of the respective CC registers and the counter.

Offset = 1830h + (y * 4h); where y = 0h to 1h

Figure 25-86. CCCTL_01[y]

31	30	29	28	27	26	25	24
CC2SELD			CCTUPD			SCERCNEZ	CC2SELU
R/W-0h			R/W-0h			R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
CC2SELU		RESERVED	CCUPD			COC	RESERVED
R/W-0h		R/W-0h	R/W-0h			R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
RESERVED	ZCOND			RESERVED	LCOND		
R/W-0h		R/W-0h		R/W-0h		R/W-0h	
7	6	5	4	3	2	1	0
RESERVED	ACOND			RESERVED	CCOND		
R/W-0h		R/W-0h		R/W-0h		R/W-0h	

Table 25-73. CCCTL_01[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-29	CC2SELD	R/W	0h	Selects the source second CCD event. 0h = Selects CCD from CC0. 1h = Selects CCD from CC1. 2h = Selects CCD from CC2. 3h = Selects CCD from CC3. 4h = Selects CCD from CC4. 5h = Selects CCD from CC5.

Table 25-73. CCCTL_01[y] Field Descriptions (continued)

Bit	Field	Type	Reset	Description
28-26	CCACTUPD	R/W	0h	CCACT shadow register Update Method This field controls how updates to the CCACT shadow register are performed 0h = Value written to the CCACT register has immediate effect. 1h = Following a zero event (CTR=0) Writes to the CCACTx_y register are stored in shadow register and transferred to CCACTx_y in the TIMCLK cycle following CTR equals 0. 2h = Following a CCD event (CTR=CC_xy) Writes to the CCACTx_y register are stored in shadow register and transferred to CCACTx_y in the TIMCLK cycle following CTR equals the CCx_y register value. 3h = Following a CCU event (CTR=CC_xy) Writes to the CCACTx_y register are stored in shadow register and transferred to CCACTx_y in the TIMCLK cycle following CTR equals the CCx_y register value. 4h = Following a zero event (CTR=0) or load event (CTR = LOAD) Writes to the CCACTx_y register are stored in shadow register and transferred to CCACTx_y in the TIMCLK cycle following CTR equals 0 or CTR. Equals LDn. Note this update mechanism is defined for use only in configurations using up/down counting. This mode is not intended for use in down count configurations. 5h = Following a zero event (CTR=0) with repeat count also zero (RC=0). Writes to the CCACTx_y register are stored in shadow register and transferred to CCACTx_y in the TIMCLK cycle following CTR equals 0 and if RC equal 0. 6h = On a TRIG pulse, the value stored in CCACT_xy shadow register is loaded into CCACT_xy register.
25	SCERCNEZ	R/W	0h	Suppress Compare Event if Repeat Counter is Not Equal to Zero This bit suppresses the generation of the compare (CCD, CCU and RC) events from the counter when the repeat counter (RC) value is not 0. 0h = CCD, CCU and RC events are always generated from the counter when their conditions are generated. 1h = CCD, CCU and RC events are generated from the counter when their conditions are generated and the RC register value is 0.

Table 25-73. CCCTL_01[y] Field Descriptions (continued)

Bit	Field	Type	Reset	Description
24-22	CC2SELU	R/W	0h	Selects the source second CCU event. 0h = Selects CCU from CC0. 1h = Selects CCU from CC1. 2h = Selects CCU from CC2. 3h = Selects CCU from CC3. 4h = Selects CCU from CC4. 5h = Selects CCU from CC5.
21	RESERVED	R/W	0h	
20-18	CCUPD	R/W	0h	Capture and Compare Update Method This field controls how updates to the shadow capture and compare register are performed (when operating in compare mode, COC=0). 0h = Writes to the CCx_y register is written to the register directly and has immediate effect. 1h = Following a zero event (CTR=0) Writes to the CCx_y register are stored in shadow register and transferred to CCx_y in the TIMCLK cycle following CTR equals 0. 2h = Following a CCD event (CTR=CC_xy) Writes to the CCx_y register are stored in shadow register and transferred to CCx_y in the TIMCLK cycle following CTR equals the CCx_y register value. 3h = Following a CCU event (CTR=CC_xy) Writes to the CCx_y register are stored in shadow register and transferred to CCx_y in the TIMCLK cycle following CTR equals the CCx_y register value. 4h = Following a zero event(CTR=0) or load event (CTR=LOAD) Writes to the CCx_y register are stored in shadow register and transferred to ECCx_y in the TIMCLK cycle following CTR equals 0 or CTR. Equals LD. Note this update mechanism is defined for use only in configurations using up/down counting. This mode is not intended for use in down count configurations. 5h = Following a zero event (CTR=0) with repeat count also zero (RC=0). Writes to the CCx_y register are stored in shadow register and transferred to CCx_y in the TIMCLK cycle following CTR equals 0 and if RC equal 0. 6h = Following a TRIG pulse. Writes to the CCx_y register are stored in shadow register and transferred to CCx_y

Table 25-73. CCCTL_01[y] Field Descriptions (continued)

Bit	Field	Type	Reset	Description
17	COC	R/W	0h	Capture or Compare. Specifies whether the corresponding CC register is used as a capture register or a compare register (never both). 0h = Compare 1h = Capture
16-15	RESERVED	R/W	0h	
14-12	ZCOND	R/W	0h	Zero Condition. This field specifies the condition that generates a zero pulse. 1h = Rising edge of CCP or trigger assertion edge 2h = Falling edge of CCP or trigger de-assertion edge 3h = Either edge of CCP or trigger change (assertion/de-assertion edge)
11	RESERVED	R/W	0h	
10-8	LCOND	R/W	0h	Load Condition. Specifies the condition that generates a load pulse. 1h = Rising edge of CCP or trigger assertion edge 2h = Falling edge of CCP or trigger de-assertion edge 3h = Either edge of CCP or trigger change (assertion/de-assertion edge)
7	RESERVED	R/W	0h	
6-4	ACOND	R/W	0h	Advance Condition. Specifies the condition that generates an advance pulse. 0h = Each TIMCLK 1h = Rising edge of CCP or trigger assertion edge 2h = Falling edge of CCP or trigger de-assertion edge 3h = Either edge of CCP or trigger change (assertion/de-assertion edge) 5h = CCP High or Trigger assertion (level)
3	RESERVED	R/W	0h	
2-0	CCOND	R/W	0h	Capture Condition. Specifies the condition that generates a capture pulse. 0h = None (never captures) 1h = Rising edge of CCP or trigger assertion edge 2h = Falling edge of CCP or trigger de-assertion edge 3h = Either edge of CCP or trigger change (assertion/de-assertion edge)

25.3.47 CCCTL_23[y] (Offset = 1838h + formula) [Reset = 00000000h]

CCCTL_23[y] is shown in [Figure 25-87](#) and described in [Table 25-74](#).

Return to the [Summary Table](#).

The CCCTL registers control the operations of the respective CC registers and the counter.

Offset = 1838h + (y * 4h); where y = 0h to 1h

Figure 25-87. CCCTL_23[y]

31	30	29	28	27	26	25	24
CC2SELD			CCACTUPD			SCERCNEZ	CC2SELU
R/W-0h			R/W-0h			R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
CC2SELU		RESERVED	CCUPD			COC	RESERVED
R/W-0h		R/W-0h	R/W-0h			R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
RESERVED	ZCOND			RESERVED	LCOND		
R/W-0h		R/W-0h		R/W-0h		R/W-0h	
7	6	5	4	3	2	1	0
RESERVED	ACOND			RESERVED	CCOND		
R/W-0h		R/W-0h		R/W-0h		R/W-0h	

Table 25-74. CCCTL_23[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-29	CC2SELD	R/W	0h	Selects the source second CCD event. 0h = Selects CCD from CC0. 1h = Selects CCD from CC1. 2h = Selects CCD from CC2. 3h = Selects CCD from CC3. 4h = Selects CCD from CC4. 5h = Selects CCD from CC5.

Table 25-74. CCCTL_23[y] Field Descriptions (continued)

Bit	Field	Type	Reset	Description
28-26	CCACTUPD	R/W	0h	CCACT shadow register Update Method This field controls how updates to the CCACT shadow register are performed 0h = Value written to the CCACTx_y register has immediate effect. 1h = Following a zero event (CTR=0) Writes to the CCACTx_y register are stored in shadow register and transferred to CCACTx_y in the TIMCLK cycle following CTR equals 0. 2h = Following a CCD event (CTR=CC_xy) Writes to the CCACTx_y register are stored in shadow register and transferred to CCACTx_y in the TIMCLK cycle following CTR equals the CCx_y register value. 3h = Following a CCU event (CTR=cc_xy) Writes to the CCACTx_y register are stored in shadow register and transferred to CCACTx_y in the TIMCLK cycle following CTR equals the CCx_y register value. 4h = Following a zero event (CTR=0) or load event (CTR=LOAD) Writes to the CCACTx_y register are stored in shadow register and transferred to CCACTx_y in the TIMCLK cycle following CTR equals 0 or CTR. Equals LDn. Note this update mechanism is defined for use only in configurations using up/down counting. This mode is not intended for use in down count configurations. 5h = Following a zero event (CTR=0) with repeat count also zero (RC=0). Writes to the CCACTx_y register are stored in shadow register and transferred to CCACTx_y in the TIMCLK cycle following CTR equals 0 and if RC equal 0. 6h = On a TRIG pulse, the value stored in CCACTx_y shadow register is loaded into CCACTx_y active register.
25	SCERCNEZ	R/W	0h	Suppress Compare Event if Repeat Counter is Not Equal to Zero This bit suppresses the generation of the compare (CCD, CCU and RC) events from the counter when the repeat counter (RCn) value is not 0. 0h = CCD, CCU and RC events are always generated from the counter when their conditions are generated. 1h = CCD, CCU and RC events are generated from the counter when their conditions are generated and the RC register value is 0.

Table 25-74. CCCTL_23[y] Field Descriptions (continued)

Bit	Field	Type	Reset	Description
24-22	CC2SELU	R/W	0h	Selects the source second CCU event. 0h = Selects CCU from CC0. 1h = Selects CCU from CC1. 2h = Selects CCU from CC2. 3h = Selects CCU from CC3. 4h = Selects CCU from CC4. 5h = Selects CCU from CC5.
21	RESERVED	R/W	0h	
20-18	CCUPD	R/W	0h	Capture and Compare Update Method This field controls how updates to the shadow capture and compare register are performed (when operating in compare mode, COC=0). 0h = Writes to the CCx_y register is written to the register directly and has immediate effect. 1h = Following a zero event (CTR=0) Writes to the CCx_y register are stored in shadow register and transferred to CCx_y in the TIMCLK cycle following CTR equals 0. 2h = Following a CCD event (CTR=CC_xy) Writes to the CCx_y register are stored in shadow register and transferred to CCx_y in the TIMCLK cycle following CTR equals the CCx_y register value. 3h = Following a CCU event (CTR=CC_xy) Writes to the CCx_y register are stored in shadow register and transferred to CCx_y in the TIMCLK cycle following CTR equals the CCx_y register value. 4h = Following a zero or load event Writes to the CCx_y register are stored in shadow register and transferred to CCx_y in the TIMCLK cycle following CTR equals 0 or CTR. Equals LDn. Note this update mechanism is defined for use only in configurations using up/down counting. This mode is not intended for use in down count configurations. 5h = Following a zero event (CTR=0) with repeat count also zero (RC=0). Writes to the CCx_y register are stored in shadow register and transferred to CCx_y in the TIMCLK cycle following CTR equals 0 and if RC equal 0. 6h = Following a TRIG pulse. Writes to the CCx_y register are stored in shadow register and transferred to CCx_y #xD; 0.

Table 25-74. CCCTL_23[y] Field Descriptions (continued)

Bit	Field	Type	Reset	Description
17	COC	R/W	0h	Capture or Compare. Specifies whether the corresponding CC register is used as a capture register or a compare register (never both). 0h = Compare 1h = Capture
16-15	RESERVED	R/W	0h	
14-12	ZCOND	R/W	0h	Zero Condition. This field specifies the condition that generates a zero pulse. 4h-Fh = Reserved 1h = Rising edge of CCP or trigger assertion edge 2h = Falling edge of CCP or trigger de-assertion edge 3h = Either edge of CCP or trigger change (assertion/de-assertion edge)
11	RESERVED	R/W	0h	
10-8	LCOND	R/W	0h	Load Condition. Specifies the condition that generates a load pulse. 4h-Fh = Reserved 1h = Rising edge of CCP or trigger assertion edge 2h = Falling edge of CCP or trigger de-assertion edge 3h = Either edge of CCP or trigger change (assertion/de-assertion edge)
7	RESERVED	R/W	0h	
6-4	ACOND	R/W	0h	Advance Condition. Specifies the condition that generates an advance pulse. 6h-Fh = Reserved 0h = Each TIMCLK 1h = Rising edge of CCP or trigger assertion edge 2h = Falling edge of CCP or trigger de-assertion edge 3h = Either edge of CCP or trigger change (assertion/de-assertion edge) 5h = CCP High or Trigger assertion (level)
3	RESERVED	R/W	0h	
2-0	CCOND	R/W	0h	Capture Condition. Specifies the condition that generates a capture pulse. 4h-Fh = Reserved 0h = None (never captures) 1h = Rising edge of CCP or trigger assertion edge 2h = Falling edge of CCP or trigger de-assertion edge 3h = Either edge of CCP or trigger change (assertion/de-assertion edge)

25.3.48 CCCTL_45[y] (Offset = 1840h + formula) [Reset = 00000000h]

CCCTL_45[y] is shown in [Figure 25-88](#) and described in [Table 25-75](#).

Return to the [Summary Table](#).

The CCCTL registers control the operations of the respective CC registers and the counter.

Offset = 1840h + (y * 4h); where y = 0h to 1h

Figure 25-88. CCCTL_45[y]

31	30	29	28	27	26	25	24
RESERVED						SCERCNEZ	RESERVED
R/W-0h						R/W-0h	R/W-0h
23	22	21	20	19	18	17	16
RESERVED			CCUPD			RESERVED	
R/W-0h			R/W-0h			R/W-0h	
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							
R/W-0h							

Table 25-75. CCCTL_45[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-26	RESERVED	R/W	0h	
25	SCERCNEZ	R/W	0h	Suppress Compare Event if Repeat Counter is Not Equal to Zero This bit suppresses the generation of the compare (CCD, CCU and RC) events from the counter when the repeat counter (RC) value is not 0. 0h = CCD, CCU and RC events are always generated from the counter when their conditions are generated. 1h = CCD, CCU and RC events are generated from the counter when their conditions are generated and the RC register value is 0.
24-21	RESERVED	R/W	0h	

Table 25-75. CCCTL_45[y] Field Descriptions (continued)

Bit	Field	Type	Reset	Description
20-18	CCUPD	R/W	0h	Capture and Compare Update Method This field controls how updates to the shadow capture and compare register are performed (when operating in compare mode, COC=0). 0h = Writes to the CCx_y register is written to the register directly and has immediate effect. 1h = Following a zero event (CTR=0) Writes to the CCx_y register are stored in shadow register and transferred to ECCx_y in the TIMCLK cycle following CTR equals 0. 2h = Following a CCD event (CTR=CC_xy) Writes to the CCx_y register are stored in shadow register and transferred to CCx_y in the TIMCLK cycle following CTR equals the CCx_y register value. 3h = Following a CCU event (CTR=CC_xy) Writes to the CCx_y register are stored in shadow register and transferred to CCx_y in the TIMCLK cycle following CTR equals the CCx_y register value. 4h = Following a zero event (CTR=0) or load event (CTR=LOAD) Writes to the CCx_y register are stored in shadow register and transferred to CCx_y in the TIMCLK cycle following CTR equals 0 or CTR. Equals LD. Note this update mechanism is defined for use only in configurations using up/down counting. This mode is not intended for use in down count configurations. 5h = Following a zero event (CTR=0) with repeat count also zero (RC=0). Writes to the CCx_y register are stored in shadow register and transferred to CCx_y in the TIMCLK cycle following CTR equals 0 and if RC equal 0. 6h = Following a TRIG pulse. Writes to the CCx_y register are stored in shadow register and transferred to CCx_y #xD; 0.
17-0	RESERVED	R/W	0h	

25.3.49 OCTL_01[y] (Offset = 1850h + formula) [Reset = 00000000h]

OCTL_01[y] is shown in [Figure 25-89](#) and described in [Table 25-76](#).

Return to the [Summary Table](#).

The OCTL_01 register controls the CCP output of the Capture-Compare slice of the counter. This includes the ability to select the source of what is driven out along with initial condition values and final inversion options.

Offset = 1850h + (y * 4h); where y = 0h to 1h

Figure 25-89. OCTL_01[y]

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED		CCPIV	CCPOINV	CCPO			
R/W-0h		R/W-0h	R/W-0h	R/W-0h			

Table 25-76. OCTL_01[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R/W	0h	
5	CCPIV	R/W	0h	CCP Initial Value This bit specifies the logical value put on the signal generator state while the counter is disabled (CTRCTL.EN == 0). 0h = Low 1h = High
4	CCPOINV	R/W	0h	CCP Output Invert The output as selected by CCPO is conditionally inverted. 0h = No inversion 1h = Invert
3-0	CCPO	R/W	0h	CCP Output Source 0h = Signal generator value (for example, PWM, triggered PWM) PWM 1h = Load event 2h = CCU event or CCD event 4h = Zero event 5h = Capture event 6h = Fault condition 8h = Mirror CCP of first capture and compare register to other capture compare blocks 9h = Mirror CCP of second capture and compare register in other capture compare blocks Ch = Signal generator output after deadband insertion Dh = Counter direction

25.3.50 OCTL_23[y] (Offset = 1858h + formula) [Reset = 00000000h]

OCTL_23[y] is shown in [Figure 25-90](#) and described in [Table 25-77](#).

Return to the [Summary Table](#).

The OCTL register controls the CCP output of the Capture-Compare slice of the counter. This includes the ability to select the source of what is driven out along with initial condition values and final inversion options.

Offset = 1858h + (y * 4h); where y = 0h to 1h

Figure 25-90. OCTL_23[y]

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED		CCPIV	CCPOINV	CCPO			
R/W-0h		R/W-0h	R/W-0h	R/W-0h			

Table 25-77. OCTL_23[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R/W	0h	
5	CCPIV	R/W	0h	CCP Initial Value This bit specifies the logical value put on the signal generator state while the counter is disabled (CTRCTL.EN == 0). 0h = Low 1h = High
4	CCPOINV	R/W	0h	CCP Output Invert The output as selected by CCPO is conditionally inverted. 0h = No inversion 1h = Invert
3-0	CCPO	R/W	0h	CCP Output Source 0h = Signal generator value (for example, PWM, triggered PWM) 1h = Load condition 2h = CCU event or CCD event 4h = Zero event 5h = Capture event 6h = Fault Condition 8h = Mirror CCP of first capture and compare register in other capture compare blocks 9h = Mirror CCP of second capture and compare register in other capture compare blocks Ch = Deadband Inserted Output Dh = Counter direction

25.3.51 CCACT_01[y] (Offset = 1870h + formula) [Reset = 00000000h]

CCACT_01[y] is shown in [Figure 25-91](#) and described in [Table 25-78](#).

Return to the [Summary Table](#).

The CCACT_01 register controls the actions of the signal generator of the capture-compare slice based on the events created in the counter block, the capture and compare block and debug events.

Offset = 1870h + (y * 4h); where y = 0h to 1h

Figure 25-91. CCACT_01[y]

31	30	29	28	27	26	25	24
SWFRCACT_CMPL		SWFRCACT		FEXACT		FENACT	
R/W-0h		R/W-0h		R/W-0h		R/W-0h	
23	22	21	20	19	18	17	16
FENACT		RESERVED					CC2UACT
R/W-0h		R/W-0h					R/W-0h
15	14	13	12	11	10	9	8
CC2UACT	RESERVED	CC2DACT		RESERVED	CUACT		RESERVED
R/W-0h	R/W-0h	R/W-0h		R/W-0h	R/W-0h		R/W-0h
7	6	5	4	3	2	1	0
CDACT		RESERVED	LACT		RESERVED	ZACT	
R/W-0h		R/W-0h	R/W-0h		R/W-0h	R/W-0h	

Table 25-78. CCACT_01[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-30	SWFRCACT_CMPL	R/W	0h	CCP Complimentary output Action on Software Force Output This field describes the resulting action of software force. This action has a shadow register, which will be updated under specific condition. So that this register cannot take into effect immediately. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP Complimentary output value is set high 2h = CCP Complimentary output value is set low
29-28	SWFRCACT	R/W	0h	CCP Output Action on Software Force Output This field describes the resulting action of software force. This action has a shadow register, which will be updated under specific condition. So that this register cannot take into effect immediately. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low
27-25	FEXACT	R/W	0h	CCP Output Action on Fault Exit This field describes the resulting action of the signal generator upon exiting the fault condition. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled 4h = CCP output value is tristated

Table 25-78. CCACT_01[y] Field Descriptions (continued)

Bit	Field	Type	Reset	Description
24-22	FENACT	R/W	0h	CCP Output Action on Fault Entry This field describes the resulting action of the signal generator upon detecting a fault. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled 4h = CCP output value is tristated
21-17	RESERVED	R/W	0h	
16-15	CC2UACT	R/W	0h	CCP Output Action on CC2U event. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled
14	RESERVED	R/W	0h	
13-12	CC2DACT	R/W	0h	CCP Output Action on CC2D event. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled
11	RESERVED	R/W	0h	
10-9	CUACT	R/W	0h	CCP Output Action on Compare (Up) This field describes the resulting action of the signal generator upon detecting a compare event while counting up. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled
8	RESERVED	R/W	0h	
7-6	CDACT	R/W	0h	CCP Output Action on Compare (Down) This field describes the resulting action of the signal generator upon detecting a compare event while counting down. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled
5	RESERVED	R/W	0h	
4-3	LACT	R/W	0h	CCP Output Action on Load Specifies what changes occur to CCP output as the result of a load event. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled
2	RESERVED	R/W	0h	
1-0	ZACT	R/W	0h	CCP Output Action on Zero Specifies what changes occur to CCP output as the result of a zero event. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled

25.3.52 CCACT_23[y] (Offset = 1878h + formula) [Reset = 00000000h]

CCACT_23[y] is shown in [Figure 25-92](#) and described in [Table 25-79](#).

Return to the [Summary Table](#).

The CCACT register controls the actions of the signal generator of the capture-compare slice based on the events created in the counter block, the capture and compare block and debug events.

Offset = 1878h + (y * 4h); where y = 0h to 1h

Figure 25-92. CCACT_23[y]

31	30	29	28	27	26	25	24
SWFRCACT_CMPL		SWFRCACT		FEXACT		FENACT	
R/W-0h		R/W-0h		R/W-0h		R/W-0h	
23	22	21	20	19	18	17	16
FENACT		RESERVED					CC2UACT
R/W-0h		R/W-0h					R/W-0h
15	14	13	12	11	10	9	8
CC2UACT	RESERVED	CC2DACT		RESERVED	CUACT		RESERVED
R/W-0h	R/W-0h	R/W-0h		R/W-0h	R/W-0h		R/W-0h
7	6	5	4	3	2	1	0
CDACT		RESERVED	LACT		RESERVED	ZACT	
R/W-0h		R/W-0h	R/W-0h		R/W-0h	R/W-0h	

Table 25-79. CCACT_23[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-30	SWFRCACT_CMPL	R/W	0h	CCP Complimentary Output Action on Software Force Output This field describes the resulting action of software force. This action has a shadow register, which will be updated under specific condition. So that this register cannot take into effect immediately. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP Complimentary output value is set high 2h = CCP Complimentary output value is set low
29-28	SWFRCACT	R/W	0h	CCP Output Action on Software Force Output This field describes the resulting action of software force. This action has a shadow register, which will be updated under specific condition. So that this register cannot take into effect immediately. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low
27-25	FEXACT	R/W	0h	CCP Output Action on Fault Exit This field describes the resulting action of the signal generator upon exiting the fault condition. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled 4h = CCP output value is tristated

Table 25-79. CCACT_23[y] Field Descriptions (continued)

Bit	Field	Type	Reset	Description
24-22	FENACT	R/W	0h	CCP Output Action on Fault Entry This field describes the resulting action of the signal generator upon detecting a fault. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled 4h = CCP output value is tristated
21-17	RESERVED	R/W	0h	
16-15	CC2UACT	R/W	0h	CCP Output Action on CC2U event. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled
14	RESERVED	R/W	0h	
13-12	CC2DACT	R/W	0h	CCP Output Action on CC2D event. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled
11	RESERVED	R/W	0h	
10-9	CUACT	R/W	0h	CCP Output Action on Compare (Up) This field describes the resulting action of the signal generator upon detecting a compare event while counting up. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled
8	RESERVED	R/W	0h	
7-6	CDACT	R/W	0h	CCP Output Action on Compare (Down) This field describes the resulting action of the signal generator upon detecting a compare event while counting down. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled
5	RESERVED	R/W	0h	
4-3	LACT	R/W	0h	CCP Output Action on Load Specifies what changes occur to CCP output as the result of a load event. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled
2	RESERVED	R/W	0h	
1-0	ZACT	R/W	0h	CCP Output Action on Zero Specifies what changes occur to CCP output as the result of a zero event. 0h = This event is disabled and a lower priority event is selected if asserting. The CCP output value is unaffected by the event. 1h = CCP output value is set high 2h = CCP output value is set low 3h = CCP output value is toggled

25.3.53 IFCTL_01[y] (Offset = 1880h + formula) [Reset = 00000000h]

IFCTL_01[y] is shown in [Figure 25-93](#) and described in [Table 25-80](#).

Return to the [Summary Table](#).

The IFCTL_01 register controls the input selection and inversion for the associated Capture-Compare slice.

Offset = 1880h + (y * 4h); where y = 0h to 1h

Figure 25-93. IFCTL_01[y]

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED			FE	CPV	RESERVED	FP	
R/W-0h			R/W-0h	R/W-0h	R/W-0h	R/W-0h	
7	6	5	4	3	2	1	0
INV	RESERVED			ISEL			
R/W-0h	R/W-0h			R/W-0h			

Table 25-80. IFCTL_01[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-13	RESERVED	R/W	0h	
12	FE	R/W	0h	Filter Enable This bit controls whether the input is filtered by the input filter or bypasses to the edge detect. 0h = Bypass. 1h = Filtered.
11	CPV	R/W	0h	Consecutive Period/Voting Select This bit controls whether the input filter uses a stricter consecutive period count or majority voting. 0h = Consecutive Periods The input must be at a specific logic level for the period defined by FP before it is passed to the filter output. 1h = Voting The filter ignores one clock of opposite logic over the filter period. I.e. Over FP samples of the input, up to 1 sample may be of an opposite logic value (glitch) without affecting the output.
10	RESERVED	R/W	0h	
9-8	FP	R/W	0h	Filter Period. This field specifies the sample period for the input filter. I.e. The input is sampled for FP timer clocks during filtering. 0h = The division factor is 3 1h = The division factor is 5 2h = The division factor is 8

Table 25-80. IFCTL_01[y] Field Descriptions (continued)

Bit	Field	Type	Reset	Description
7	INV	R/W	0h	Input Inversion This bit controls whether the selected input is inverted. 0h = Noninverted 1h = Inverted
6-4	RESERVED	R/W	0h	
3-0	ISEL	R/W	0h	Input Select (CCP0) This field selects the input source to the filter input. 4h-7h = Reserved 0h = CCP of the corresponding capture compare unit PWM 0h 1h = Input pair CCPX of the capture compare unit. For CCP0 input pair is CCP1 and for CCP1 input pair is CCP0. 2h = CCP0 of the counter 捕获 2h 3h = Trigger 或内部时钟的compare用 待验证 4h = XOR of CCP inputs as input source (Used in Hall input mode). 5h = subscriber 0 event as input source. 6h = subscriber 1 event as input source. 7h = Comparator 0 output. 8h = Comparator 1 output. 9h = Comparator 2 output.

25.3.54 IFCTL_23[y] (Offset = 1888h + formula) [Reset = 00000000h]

IFCTL_23[y] is shown in [Figure 25-94](#) and described in [Table 25-81](#).

Return to the [Summary Table](#).

The IFCTL register controls the input selection and inversion for the associated Capture-Compare slice.

Offset = 1888h + (y * 4h); where y = 0h to 1h

Figure 25-94. IFCTL_23[y]

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED			FE	CPV	RESERVED	FP	
R/W-0h			R/W-0h	R/W-0h	R/W-0h	R/W-0h	
7	6	5	4	3	2	1	0
INV	RESERVED			ISEL			
R/W-0h	R/W-0h			R/W-0h			

Table 25-81. IFCTL_23[y] Field Descriptions

Bit	Field	Type	Reset	Description
31-13	RESERVED	R/W	0h	
12	FE	R/W	0h	Filter Enable This bit controls whether the input is filtered by the input filter or bypasses to the edge detect. 0h = Bypass. 1h = Filtered.
11	CPV	R/W	0h	Consecutive Period/Voting Select This bit controls whether the input filter uses a stricter consecutive period count or majority voting. 0h = Consecutive Periods The input must be at a specific logic level for the period defined by FP before it is passed to the filter output. 1h = Voting The filter ignores one clock of opposite logic over the filter period. I.e. Over FP samples of the input, up to 1 sample may be of an opposite logic value (glitch) without affecting the output.
10	RESERVED	R/W	0h	
9-8	FP	R/W	0h	Filter Period. This field specifies the sample period for the input filter. I.e. The input is sampled for FP timer clocks during filtering. 0h = The division factor is 3 1h = The division factor is 5 2h = The division factor is 8

Table 25-81. IFCTL_23[y] Field Descriptions (continued)

Bit	Field	Type	Reset	Description
7	INV	R/W	0h	Input Inversion This bit controls whether the selected input is inverted. 0h = Noninverted 1h = Inverted
6-4	RESERVED	R/W	0h	
3-0	ISEL	R/W	0h	Input Select (CCP0) This field selects the input source to the filter input. 4h-7h = Reserved 0h = CCP of the corresponding capture compare unit 1h = Input pair CCPX of the capture compare unit. For CCP0 input pair is CCP1 and for CCP1 input pair is CCP0. 2h = CCP0 of the counter 3h = Trigger 4h = XOR of CCP inputs as input source (Used in Hall input mode). 5h = subscriber 0 event as input source. 6h = subscriber 1 event as input source. 7h = Comparator 0 output. 8h = Comparator 1 output. 9h = Comparator 2 output.

25.3.55 PL (Offset = 18A0h) [Reset = 00000000h]

PL is shown in [Figure 25-95](#) and described in [Table 25-82](#).

Return to the [Summary Table](#).

This is the phase load register.

Figure 25-95. PL

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																PHASE															
R/W-0h																R/W-0h															

Table 25-82. PL Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15-0	PHASE	R/W	0h	Phase Load value 0h = Minimum value 00FFFFFFh = Maximum Value

25.3.56 DBCTL (Offset = 18A4h) [Reset = 00000000h]

DBCTL is shown in [Figure 25-96](#) and described in [Table 25-83](#).

Return to the [Summary Table](#).

The DBCTL register controls the dead band insertion of the pulse width modulated output.

Figure 25-96. DBCTL

31	30	29	28	27	26	25	24
RESERVED				FALLDELAY			
R/W-				R/W-0h			
23	22	21	20	19	18	17	16
FALLDELAY							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED			M1_ENABLE	RISEDELAY			
R/W-			R/W-	R/W-0h			
7	6	5	4	3	2	1	0
RISEDELAY							
R/W-0h							

Table 25-83. DBCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-28	RESERVED	R/W	0h	
27-16	FALLDELAY	R/W	0h	Fall Delay The number of TIMCLK periods inserted between the fall edge of CCP signal and the rise edge of CCP complimentary signal. 0h = Minimum value FFFh = Maximum Value
15-13	RESERVED	R/W	0h	
12	M1_ENABLE	R/W	0h	Dead Band Mode 1 Enable. 0h = Disabled 1h = Enabled
11-0	RISEDELAY	R/W	0h	Rise Delay The number of TIMCLK periods inserted between the falling edge of CCP signal and the rising edge of CCP complimentary signal. 0h = Minimum value FFFh = Maximum Value

25.3.57 TSEL (Offset = 18B0h) [Reset = 00000000h]

TSEL is shown in [Figure 25-97](#) and described in [Table 25-84](#).

Return to the [Summary Table](#).

The TSEL register controls the input trigger enable and selection of the trigger source. Trigger sources are generated by other peripherals through their respective publisher ports (subscribed in by the timer's subscriber port).

Figure 25-97. TSEL

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED						TE	RESERVED				ETSEL				
R/W-0h						R/W-0h		R/W-0h			R/W-0h				

Table 25-84. TSEL Field Descriptions

Bit	Field	Type	Reset	Description
31-10	RESERVED	R/W	0h	
9	TE	R/W	0h	Trigger Enable. This selects whether a trigger is enabled or not for this counter 0x0 = Triggers are not used 0x1 = Triggers are used as selected by the ETSEL field 0h = Triggers are not used. 1h = Triggers are used as selected by the IE, ITSEL and ETSEL fields.
8-5	RESERVED	R/W	0h	
4-0	ETSEL	R/W	0h	External Trigger Select. This selects which System Event is used if the input filter selects trigger. Triggers 0-15 are used to connect triggers generated by other timer modules. Refer to the SoC data sheet for details related to timer trigger sources. Triggers 16 and 17 are connected to event manager subscriber ports. Event lines 18-31 are reserved for future use. 0h = TRIGx = External trigger input from TIM x. 1h = TRIGx = External trigger input from TIM x. 2h = TRIGx = External trigger input from TIM x. 3h = TRIGx = External trigger input from TIM x. 4h = TRIGx = External trigger input from TIM x. 5h = TRIGx = External trigger input from TIM x. 6h = TRIGx = External trigger input from TIM x. 7h = TRIGx = External trigger input from TIM x. 8h = TRIGx = External trigger input from TIM x. 9h = TRIGx = External trigger input from TIM x. Ah = TRIGx = External trigger input from TIM x. Bh = TRIGx = External trigger input from TIM x. Ch = TRIGx = External trigger input from TIM x. Dh = TRIGx = External trigger input from TIM x. Eh = TRIGx = External trigger input from TIM x. Fh = TRIGx = External trigger input from TIM x. 10h = TRIG_SUBx = External trigger input from subscriber port x. 11h = TRIG_SUBx = External trigger input from subscriber port x.

25.3.58 RC (Offset = 18B4h) [Reset = 00000000h]

RC is shown in [Figure 25-98](#) and described in [Table 25-85](#).

Return to the [Summary Table](#).

Repeat counter is to reduce interrupt overhead. The repeat counter provides the mechanism to suppress un-necessary interrupts;
reducing the number of interrupts generated by each event type to 1 for the program number of periods. Specifically, the repeat timer may suppress Load, Compare (up/down, normal/shadow), and Zero events.

Figure 25-98. RC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								RC							
R-0h																								R-0h							

Table 25-85. RC Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	RC	R	0h	Repeat Counter Value 0h = Minimum value FFh = Maximum Value

25.3.59 RCLD (Offset = 18B8h) [Reset = 00000000h]

RCLD is shown in [Figure 25-99](#) and described in [Table 25-86](#).

Return to the [Summary Table](#).

The load register value is transferred to the counter when the counter load input is asserted.

Figure 25-99. RCLD

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								RCLD							
R/W-0h																								R/W-0h							

Table 25-86. RCLD Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R/W	0h	
7-0	RCLD	R/W	0h	Repeat Counter Load Value This field provides the value loaded into the repeat counter at a load event following the repeat counter value equaling 0. 0h = Minimum value FFh = Maximum Value

25.3.60 QDIR (Offset = 18BCh) [Reset = 00000000h]

QDIR is shown in [Figure 25-100](#) and described in [Table 25-87](#).

Return to the [Summary Table](#).

The QDIR register provides the direction of count which is intended for use when operating the counter in QE1.

Figure 25-100. QDIR

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R-															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED															DIR
R-															R-0h

Table 25-87. QDIR Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	DIR	R	0h	Direction of count 0h = Down (Phase B leads Phase A) 1h = Up (Phase A leads Phase B)

25.3.61 FCTL (Offset = 18D0h) [Reset = 00000000h]

FCTL is shown in [Figure 25-101](#) and described in [Table 25-88](#).

Return to the [Summary Table](#).

The FCTL register controls the fault inputs, fault detection and error handling behavior.

Figure 25-101. FCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED		FSENEXT2	FSENEXT1	FSENEXT0	FSENAC2	FSENAC1	FSENAC0
R/W-		R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h
7	6	5	4	3	2	1	0
TFIM	RESERVED		FL		FI	RESERVED	FIEN
R/W-0h	R/W-		R/W-0h		R/W-0h	R/W-	R/W-0h

Table 25-88. FCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-14	RESERVED	R/W	0h	
13	FSENEXT2	R/W	0h	Specifies whether the external fault pin2 high/low is treated as fault condition. 0h = Fault Input is active low. 1h = Fault Input is active high.
12	FSENEXT1	R/W	0h	Specifies whether the external fault pin1 high/low is treated as fault condition. 0h = Fault Input is active low. 1h = Fault Input is active high.
11	FSENEXT0	R/W	0h	Specifies whether the external fault pin0 high/low is treated as fault condition. 0h = Fault Input is active low. 1h = Fault Input is active high.
10	FSENAC2	R/W	0h	Specifies whether the COMP2 output high/low is treated as fault condition. 0h = Fault Input is active low. 1h = Fault Input is active high.
9	FSENAC1	R/W	0h	Specifies whether the COMP1 output high/low is treated as fault condition. 0h = Fault Input is active low. 1h = Fault Input is active high.
8	FSENAC0	R/W	0h	Specifies whether the COMP0 output high/low is treated as fault condition. 0h = Fault Input is active low. 1h = Fault Input is active high.
7	TFIM	R/W	0h	Trigger Fault Input Mask Specifies whether the selected trigger participates as a fault input. If enabled and the trigger asserts, the trigger is treated as a fault. 0h = Selected trigger does not participate in fault condition generation 1h = Selected trigger participates in fault condition generation
6-5	RESERVED	R/W	0h	

Table 25-88. FCTL Field Descriptions (continued)

Bit	Field	Type	Reset	Description
4-3	FL	R/W	0h	Fault Latch mode Specifies whether the fault condition is latched and configures the latch clear conditions. 0h = Overall fault condition is not dependent on the F bit in RIS 1h = Overall fault condition is dependent on the F bit in RIS 2h = Fault condition is latched. Fault condition is cleared on a zero event if the fault input is 0. 3h = Fault condition is latched. Fault condition is cleared on a load event if the fault input is 0.
2	FI	R/W	0h	Fault Input Specifies whether the overall fault condition is dependent on the sensed fault pin. 0h = Overall Fault condition is not dependent on sensed input. 1h = Overall Fault condition is dependent on sensed input.
1	RESERVED	R/W	0h	
0	FIEN	R/W	0h	Fault Input Enable This bit enables the input for fault detection. 0h = Fault Input Disabled 1h = Fault Input Enabled

25.3.62 FIFCTL (Offset = 18D4h) [Reset = 00000000h]

FIFCTL is shown in [Figure 25-102](#) and described in [Table 25-89](#).

Return to the [Summary Table](#).

The FIFCTL register controls the filtering for the fault input.

Figure 25-102. FIFCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED			FILTEN	CPV	RESERVED	FP	
R/W-			R/W-0h	R/W-0h	R/W-	R/W-0h	

Table 25-89. FIFCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-5	RESERVED	R/W	0h	
4	FILTEN	R/W	0h	Filter Enable This bit controls whether the input is filtered by the input filter or bypasses to go directly to the optional pre-scale filter and then to the edge detect. 0h = Bypass 1h = Filtered.
3	CPV	R/W	0h	Consecutive Period/Voting Select This bit controls whether the input filter uses a stricter consecutive period count or majority voting. 0h = Consecutive Periods. The input must be at a specific logic level for the period defined by FP before it is passed to the filter output. 1h = Voting. The filter ignores one clock of opposite logic over the filter period. I.e. Over FP samples of the input, up to 1 sample may be of an opposite logic value (glitch) without affecting the output
2	RESERVED	R/W	0h	
1-0	FP	R/W	0h	Filter Period This field specifies the sample period for the input filter. I.e. The input is sampled for FP timer clocks during filtering. 0h = Filter Period 3 1h = Filter Period 5 2h = Filter Period 8



The real-time clock (RTC) module provides clock counters with calendar mode, a flexible programmable alarm, offset calibration, and a provision for temperature compensation.

26.1 Overview	1504
26.2 Basic Operation	1504
26.3 Configuration	1506
26.4 RTC Registers	1514

26.1 Overview

The real-time clock (RTC) module provides time-keeping for the application, with counters for seconds, minutes, hours, day of the week, day of the month, and year, in selectable binary or binary-coded decimal (BCD) format. A variety of programmable interrupt alarms are provided.

Key features of the RTC include:

- Real-time clock and calendar mode providing seconds, minutes, hours, day of week, day of month, and year
- Selectable binary or binary-coded decimal (BCD) format
- Leap-year correction (valid for year 1901 through 2099)
- Two customizable calendar alarm interrupts based on minute, hour, day of the week, and day of the month
- Interval alarm interrupt to wake every minute, every hour, at midnight, or at noon
- Periodic interrupt to wake at 4096, 2048, 1024, 512, 256, or 128 Hz
- Periodic interrupt to wake at 64, 32, 16, 8, 4, 2, 1, and 0.5 Hz
- Interrupt capability down to STANDBY mode with STOPCLKSTBY
- Calibration for crystal offset error and crystal temperature drift (up to ± 240 ppm total)
- RTC clock output to pin for calibration

26.2 Basic Operation

The RTC utilizes two prescaler blocks (RT0PS, RT1PS) to generate a 128Hz and 1Hz clock, respectively, from the source 32.768kHz RTCCLK. The prescaler blocks each support generating a periodic interrupt alarm at a variety of rates from 4096Hz down to 0.5Hz. The 1Hz clock output from RT1PS ticks the counter block, which tracks seconds, minutes, hours, and day of the week. The counter block also sources an interval alarm interrupt once per minute, per hour, at midnight, or at noon. The counter midnight output sources the calendar block, which provides day of the month, month, and year, with leap year correction. Two independent calendar alarm interrupts are provided to wake the application at specific times.

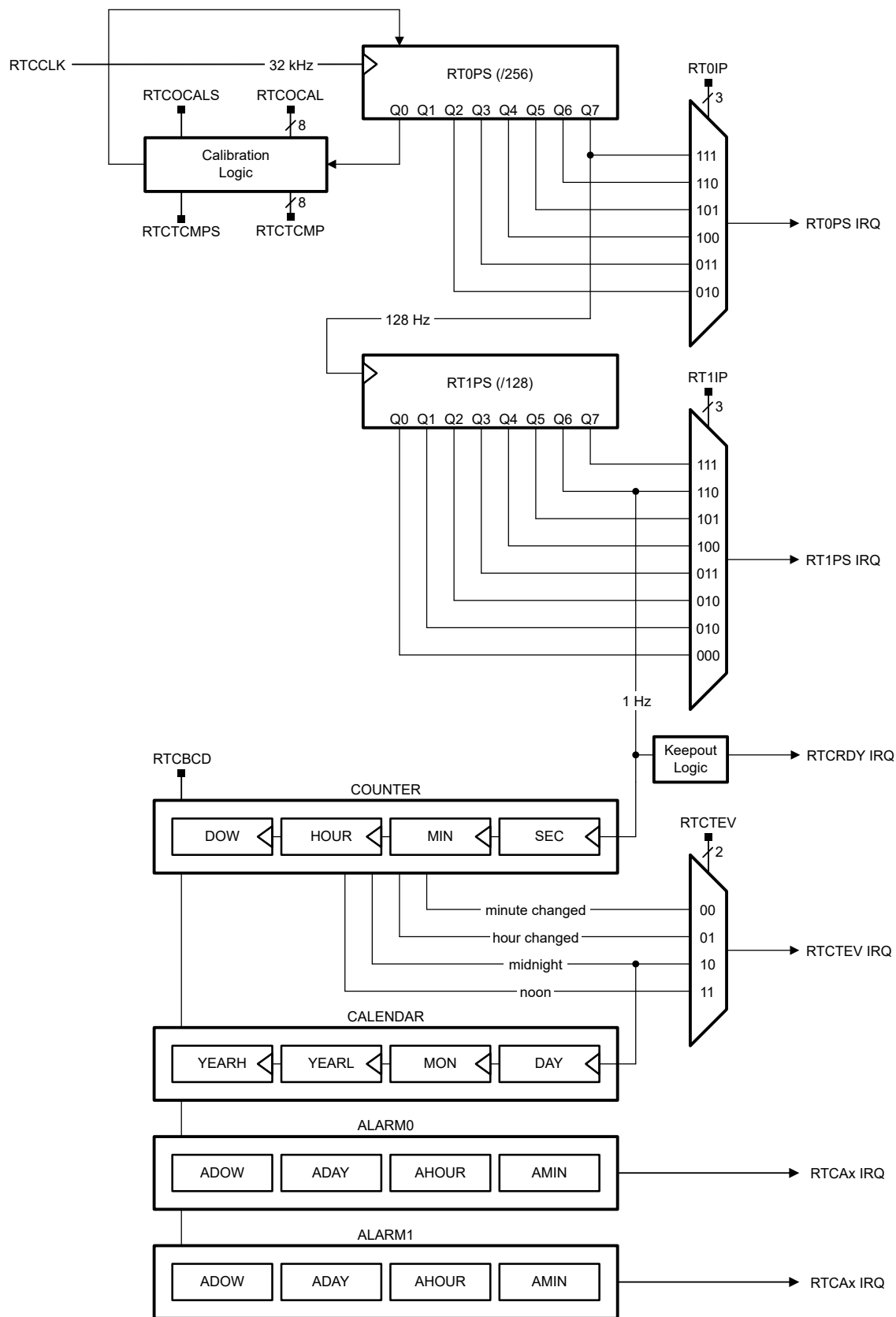


Figure 26-1. RTC Block Diagram

26.3 Configuration

The RTC is configured with the RTC peripheral registers. The RTC must be enabled before being configured for use by setting the ENABLE bit in the PWREN register (see [peripheral power enable](#)) together with the matching PWREN KEY value.

Application software may reset the RTC state at any time by setting the RESETASSERT bit in the RSTCTL register of the RTC together with the matching RSTCTL KEY value. The RESETSTKY bit in the STAT register of the RTC will be set following a reset of the RTC. This flag may be cleared by setting the RESETSTKYCLR bit in the RSTCTL register of the RTC. As such, software may determine if the RTC has been reset since the sticky reset bit (RESETSTKY) was last cleared. Many RTC registers have no initial conditions. These registers must be configured by application software before use.

To start the RTC counters, the MODCLKEN bit in the CLKCTL register must be set by application software. The MODCLKEN bit is initially cleared following a reset of the RTC.

When enabled and configured, the RTC runs in all power modes except SHUTDOWN. The RTC is not reset by a CPURST, SYSRST, or NRST pin triggered BOOTRST (see the [reset control](#) section for a detailed overview of device reset levels). As such, the RTC continues to operate even through software invocation of the bootstrap loader (BSL) or a hold of <1s on the external NRST pin.

26.3.1 Clocking

The RTC is clocked directly from [RTCCLK](#), which is sourced either from the internal [LFOSC](#) or the external low frequency crystal oscillator ([LFXT](#)). The LFOSC and LFXT both require startup time. Ensure that the oscillator sourcing LFCLK in the application is configured and that LFCLK is operational before enabling power to the RTC module.

Two prescale dividers, RT0PS and RT1PS, are automatically configured to provide a 1-second clock interval for the RTC. The LFCLK must be running at 32 kHz clock frequency for proper RTC operation. The RT0PS divider is sourced directly from LFCLK at 32 kHz. The output of the RT0PS divider block is 32 kHz/256, or 128Hz, and is used to source the input of the RT1PS divider block. The RT1PS output is 128Hz/128, or 1Hz, and is used to source the real-time clock counter registers with the required 1-second time interval.

Clearing the MODCLKEN bit in the CLKCTL register of the RTC halts the propagation of LFCLK to the real-time counters and the prescale counters (RT0PS and RT1PS) in the RTC. It does not have any effect on the LFCLK itself.

26.3.2 Reading and Writing to RTC Peripheral Registers

The peripheral bus clock (ULPCLK) is asynchronous to the RTC clock source (RTCCLK). As a result, special care must be taken when reading and writing certain RTC peripheral registers.

Counter/Calendar Registers

The RTC counter/calendar registers are updated once per second. To prevent reading any counter/calendar register at the time of an update (which could result in an invalid time being read), a keep-out window is provided. The keep-out window is approximately 128/32768 seconds before the counters update. The read-only RTCRDY bit in the STA register is reset during the keep-out window, and set outside the keep-out window period. Any read of the counter/calendar registers while RTCRDY bit in the STA register is reset can potentially be invalid, and the time read should be ignored.

The RTCRDY interrupt mechanism can also be used to safely read the RTC counter/calendar registers. When the RTCRDY interrupt is enabled, an interrupt is generated based on the rising edge of the RTCRDY bit, causing the RTCRDY interrupt flag to be set. At this point, the application has nearly a complete second to safely read any or all of the RTC registers. To ensure a safe read, the RTCRDY bit in the STA register can be read again in the interrupt service routine and other interrupts can be disabled. This synchronization process prevents reading the time value during a counter/calendar transition. The RTCRDY interrupt flag is reset automatically when the interrupt is serviced, or can be reset by software.

The RTC counter/calendar registers can be written to at any time. Writes to the calendar registers take 2 to 3 RTCCLK cycles to take effect. If a back-to-back write is done on counter/calendar registers, then it is possible that for 2-3 RTCCLK cycles, the register is set to an undefined value. Therefore, back-to-back writes to calendar registers need to be avoided. Note that due to the synchronization, if a read immediately follows a write to the calendar registers, the read back value is always the actual counter value, which can be different than the written value due to the 2-3 RTCCLK cycles required to synchronize the newly written value.

The following registers are subject to the restrictions above: SEC, MIN, HOUR, DAY, MON, YEAR.

Control Register

The RTC control register (CTL) can be read at any time. Writes to the CTL register should only be done when the TEV (interval timer) interrupt is disabled and after the RTCRDY interrupt is set.

The following register is subject to the restrictions above: CTL.

Alarm Registers

The RTC alarm configuration registers can be read at any time. Writes to the alarm registers must be done when the corresponding alarm interrupt is disabled, and after the RTCRDY interrupt is set.

The following registers are subject to the restrictions above: A1MIN, A1HOUR, A1DAY, A2MIN, A2HOUR, A2DAY.

Offset Correction and Temperature Compensation Registers

The offset correction register (CAL) and temperature compensation register (TCMP) must only be written when the RTCTCRDY bit in the STA register is set. RTCTCRDY is a read-only bit which is set when the hardware is ready to take in new correction or compensation values. Writes applied when RTCTCRDY is cleared have no effect. The RTCTCOK status bit is provided in the STA register. If a write to CAL or TCMP was successful, RTCTCOK will be set, else it will be reset. RTCTCOK will hold its status until the next write attempt. If a write is unsuccessful, software needs to re-attempt the write when RTCTCRDY is set.

The following registers are subject to the restrictions above: TCMP, CAL.

Prescaler Control Registers

The prescaler interval control registers can be read at any time. Writes to the prescaler control registers should only be done when the PS0 and PS1 interrupts are disabled.

The following registers are subject to the restrictions above: PSCTL.

26.3.3 Binary vs. BCD

The RTC provides seconds, minutes, hours, day of week, day of month, month, and year in selectable binary or binary-coded decimal (BCD) format. When the RTCBCD bit in the CTL register of the RTC is set, BCD format is selected. When BCD format is selected, only the BCD bits are valid in the calendar registers. The format must be selected before the time is set.

26.3.4 Leap Year Handling

The calendar includes a leap-year algorithm which considers all years evenly divisible by four as leap years. This algorithm is accurate from the year 1901 through 2099.

26.3.5 Calendar Alarm Configuration

The RTC provides a flexible alarm system. There are two user-programmable alarms (A0 and A1) which can be programmed based on the settings contained in the alarm registers for minutes, hours, day of week, and day of month.

Each alarm register contains an alarm enable (AE) bit that can be used to enable the respective alarm register. By setting the AE bits of the various alarm registers, a variety of alarm events can be generated.

- **Example 1:** A user wants to set an alarm every hour at 15 minutes past the hour (for example, at 00:15:00, 01:15:00, 02:15:00, and so on). This is possible by setting AxMIN to 15. By setting the AE bit of the AxMIN and clearing all other AE bits of the alarm registers, the alarm is enabled. When enabled, the RTCax interrupt flag is set when the count transitions from 00:14:59 to 00:15:00, 01:14:59 to 01:15:00, 02:14:59 to 02:15:00, and so on.
- **Example 2:** A user wants to set an alarm every day at 04:00:00. This is possible by setting AxHOUR to 4. By setting the AE bit of the AxHOUR and clearing all other AE bits of the alarm registers, the alarm is enabled. When enabled, the RTCax interrupt flag is set when the count transitions from 03:59:59 to 04:00:00.
- **Example 3:** A user wants to set an alarm for 06:30:00. AxHOUR would be set to 6 and AxMIN would be set to 30. By setting the AE bits of AxHOUR and AxMIN, the alarm is enabled. When enabled, the RTCax interrupt flag is set when the time count transitions from 06:29:59 to 06:30:00. In this case, the alarm event occurs every day at 06:30:00.
- **Example 4:** A user wants to set an alarm every Tuesday at 06:30:00. AxDOW in the AxDAY register would be set to 2, AxHOUR would be set to 6 and AxMIN would be set to 30. By setting the AE bits of AxDOW, AxHOUR and AxMIN, the alarm is enabled. When enabled, the RTCax interrupt flag is set when the time count transitions from 06:29:59 to 06:30:00 and the DOW transitions from 1 to 2.
- **Example 5:** A user wants to set an alarm the fifth day of each month at 06:30:00. AxDAY would be set to 5, AxHOUR would be set to 6 and AxMIN would be set to 30. By setting the AE bits of AxDAY, AxHOUR and AxMIN, the alarm is enabled. When enabled, the RTCax interrupt flag is set when the time count transitions from 06:29:59 to 06:30:00 and the RTCDAY equals 5.

Note

Invalid alarm settings

Invalid alarm settings are not checked through hardware. It is the user's responsibility that valid alarm settings are entered.

Note

Invalid time and date values

Writing of invalid date and/or time information or data values outside the legal ranges specified in the calendar and alarm registers can result in unpredictable behavior.

Note

Setting the alarm

Before setting an initial alarm, all alarm registers including the AE bits should be cleared.

To prevent potential erroneous alarm conditions from occurring, the alarms should be disabled by clearing the RTCax interrupt enable, RTCax interrupt flag, and AE bits before writing initial or new time values to the RTC time registers.

26.3.6 Interval Alarm Configuration

The interval alarm is provided in addition to the two calendar alarms, and can be configured to generate an interval alarm event when the following events occur:

- MIN changes
- HOUR changes
- Every day at midnight
- Every day at noon

The interval is selected by programming the RTCTEVTX field in the CTL register of the RTC. The interval alarm sources the RTCTEV interrupt.

26.3.7 Periodic Alarm Configuration

Two periodic alarms are provided, and can be configured to generate two periodic time bases for the application.

The RT0PS periodic alarm can be configured to generate a periodic interrupt at one of the following rates: 4096Hz, 2048Hz, 1024Hz, 512Hz, 256Hz, or 128Hz. The RT0PS period is set by the RT0IP bits in the PSCTL register. The RT0PS event sources the RT0PS interrupt.

The RT1PS periodic alarm can be configured to generate a periodic interrupt at one of the following rates: 64Hz, 32Hz, 16Hz, 8Hz, 4Hz, 2Hz, 1Hz, or 0.5Hz. The RT1PS period is set by the RT1IP bits in the PSCTL register. The RT1PS event sources the RT1PS interrupt.

Note

Changing the settings of the periodic alarm configuration (RT0IP, RT1IP) while the corresponding prescaler is running can result in setting the corresponding interrupt flags. To avoid this, the RT0PS and RT1PS interrupts should be masked when changing RT0IP or RT1IP while the prescaler is running.

26.3.8 Calibration

The RTC module provides calibration mechanisms for reducing timekeeping error due to crystal offset error and crystal temperature drift. [Crystal offset error](#) can be corrected by measuring the crystal frequency through the RTC_OUT output and adjusting calibration registers in the RTC. [Crystal temperature drift](#) can be corrected by measuring the system temperature with software, computing a correction factor in ppm based on the measured temperature, and applying the correction factor to the RTC.

Up to ± 240 ppm of offset correction or up to ± 240 ppm of temperature compensation correction can be applied individually, but the total correction allowed when both offset correction and temperature compensation are used together is ± 240 ppm. This means that the signed addition of offset error and temperature compensation should not exceed the maximum of ± 240 ppm, else the values in excess of ± 240 ppm will be ignored by hardware.

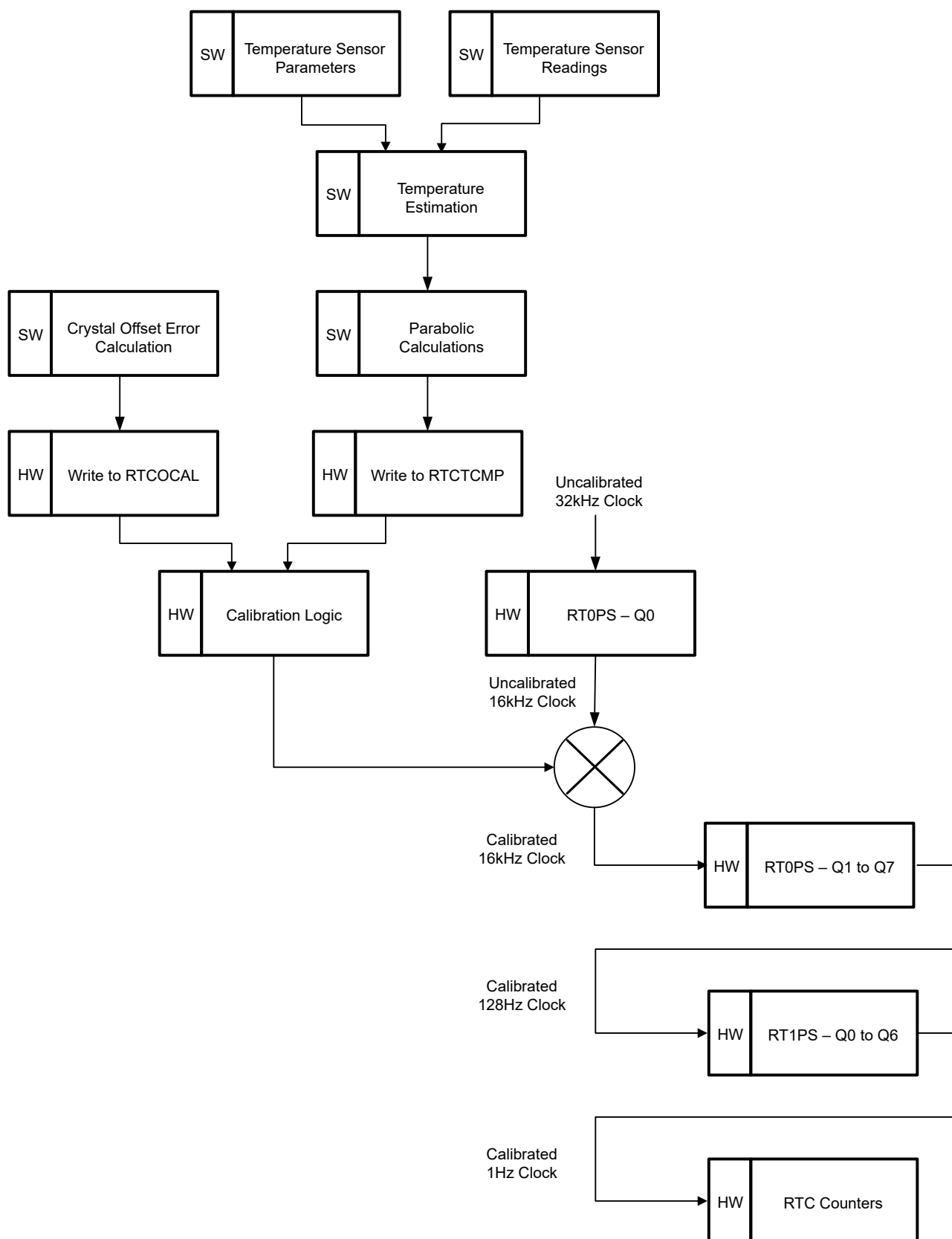


Figure 26-2. RTC Calibration

26.3.8.1 Crystal Offset Error

The RTC module can be calibrated for crystal manufacturing tolerance or offset error for higher timekeeping accuracy. A crystal frequency error of up to ± 240 ppm can be calibrated smoothly over a period of 60 seconds.

The CAL register is used to adjust the frequency. The calibration value is written into the RTCOCALX field in the CAL register, and each LSB in this field represents approximately ± 1 ppm correction (the sign is based on the RTCOCALS bit in the CAL register). When RTCOCALS is set, up calibration is selected and each LSB in RTCOCALX represents +1 ppm adjustment. When RTCOCALS is cleared, down calibration is selected and each LSB in RTCOCALX represents -1 ppm adjustment. The RTCOCALX field is 8 bits. Software can write a value of up to 256ppm into this register, but the maximum frequency error that can be corrected is only 240ppm. Software is responsible for writing legal values into RTCOCALX. Offset error calibration is inactive when the RTC is not enabled (MODCLKEN is cleared) or when RTCOCALX is zero. Writing to the RTCOCAL register resets the temperature compensation to zero.

Note

The CAL register must only be accessed via half-word (16-bit) or word (32-bit) wide operations to ensure that the sign bit is set together with the calibration value.

To calibrate the RTC frequency, the RTC_OUT output signal is available on a pin. The RTCCALFX field in the CAL register can be used to select the frequency rate of the output signal to the pin. When RTCCALFX = 0x0, no signal is output on the RTC_OUT pin. The other settings of the RTCCALFX select one of the three frequency options to output: 512Hz, 256Hz, or 1Hz. RTC_OUT output can be measured, and the result of this measurement can be applied to the RTCOCALS and RTCOCALX bits to effectively reduce the initial offset of the RTCCLK.

26.3.8.1.1 Offset Error Correction Mechanism

In the RTC, offset error calibration takes place over a period of 60 seconds. To achieve approximately ± 1 ppm correction, the 16kHz clock (Q0 output of RT0PS) is adjusted to add or subtract 1 clock pulse. This correction happens once every quarter second until the programmed ppm error is compensated.

All three possible output frequencies 512Hz, 256Hz, and 1Hz at the RTC_OUT pin are affected by calibration settings. RT0PS interrupt triggered by RT0PS – Q2 to Q7 is based on calibrated clock. RT1PS interrupt (RT1PS) and RTC clock time event interrupt (RTCDEV) are also based on the calibrated clock.

The following can be used as a guide for setting RTCOCALS and RTCOCALX, given $f_{\text{RTCCLK}} = f_{\text{RTCCLK, meas}} \times$ divider factor (set by RTCCALFX):

Slow Crystal ($f_{\text{RTCCLK}} < 32768\text{Hz}$)

- Set RTCOCALS = 1
- Set RTCOCALX to Round ($60 \times 16384 \times (1 - f_{\text{RTCCLK}}/32768)$)

As an example for up calibration, when the measured frequency is 511.9658Hz against the reference frequency of 512Hz, the frequency error is approximately 67ppm low. In order to increase the frequency by 67ppm, RTCOCALS should be set, and RTCOCALx should be set to Round ($60 \times 16384 \times (1 - 511.9658 \times 64 / 32768)$) = 66.

Fast Crystal ($f_{\text{ACLK}} \geq 32768\text{Hz}$)

- Set RTCOCALS = 0
- Set RTCOCALX to Round ($60 \times 16384 \times (1 - f_{\text{RTCCLK}}/32768)$)

As an example for down calibration, when the measured frequency is 512.0241Hz against the reference frequency of 512Hz, the frequency error is approximately 47ppm high. In order to decrease the frequency by 47ppm, RTCOCALS should be cleared, and RTCOCALx should be set to Round ($60 \times 16384 \times (1 - 512.0241 \times 64 / 32768)$) = 46.

26.3.8.2 Crystal Temperature Error

The output frequency of a crystal can vary significantly due to drift in temperature. A hybrid software and hardware approach can be applied to achieve temperature compensation for the RTC. The RTC supports temperature compensation up to ± 240 ppm.

Application software can make use of the on-chip temperature sensor to measure the device temperature at desired intervals (for example, every few seconds or minutes) to approximate the ambient temperature of the circuit. Then, software can be used to do parabolic calculations to determine the corresponding frequency error in ppm. This frequency error can then be written into the TCMP register for temperature compensation.

The RTCTCMPX field contains 8 bits which provide frequency correction up to ± 240 ppm. The LSB represents ± 1 ppm based on the RTCTCMPS bit in the TCMP register. When RTCTCMPS is set, each step in RTCTCMPX represents a +1ppm adjustment (up calibration). When the RTCTCMPS bit is cleared, each step in RTCTCMPX represents a -1ppm adjustment (down calibration).

26.3.8.2.1 Temperature Drift Correction Mechanism

When the temperature compensation value is written into RTCTCMPX, it is added together with the offset error calibration value and the resulting value is taken into account from the next calibration cycle onwards. The ongoing calibration cycle will not be affected by writes into the TCMP register.

Reading the TCMPX at any time returns the cumulative correction value (offset + temperature) which is the signed addition of RTCOCALX and RTCTCMPX, and the updated sign bit (RTCTCMPS) of the addition result. Note that writing to the RTCOCAL register will reset the temperature compensation value to zero.

For example, when RTCOCALX is +150 ppm, and RTCTCMP is written with +200 ppm, the effective value taken in for the next calibration cycle is +240 ppm (saturated). If the RTCOCALX is +150 ppm, and RTCTCMP is written with +50 ppm, the effective value taken for the next calibration cycle is +200 ppm (40 ppm of compensation headroom remains).

To achieve effective temperature compensation, software is responsible to:

1. Measure the temperature as often as required by the operating conditions, crystal performance, and targeted accuracy
2. Calculate the frequency error due to temperature
3. Write the temperature compensation value into RTCTCMPX while not exceeding the maximum combined correction limit of ± 240 ppm

Writing to the TCMP register for temperature compensation requires 60 seconds (one minute) to take effect as a part of the next calibration cycle. Therefore, if temperature must be measured more frequently than once per minute (for example, once every 5 seconds) then it needs to average the error over one minute and update the TCMP register once per minute.

Note

The TCMP register must only be accessed via half-word (16-bit) or word (32-bit) wide operations to ensure that the sign bit is set together with the calibration value.

26.3.9 RTC Events

The RTC module contains two [event publishers](#) and no [event subscribers](#). One event publisher (CPU_INT) manages RTC interrupt requests (IRQs) to the CPU subsystem through a [static event route](#). The second event publisher (GEN_EVENT) can be used to publish RTC events to a subscriber through a [generic event route channel](#).

The RTC events are summarized in [Table 26-1](#).

Table 26-1. RTC Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU interrupt event	Publisher	RTC	CPU Subsystem	Static route	CPU_INT registers	Fixed interrupt route from RTC to CPU
Generic event	Publisher	RTC	Generic event channel	Generic route (FPUB_0)	GEN_EVENT registers, FPUB_0 register	Trigger generic event channel from RTC

26.3.9.1 CPU Interrupt Event Publisher (CPU_INT)

The RTC module provides six interrupt sources which can be configured to source a [CPU interrupt event](#). In order of decreasing interrupt priority, the CPU interrupt events from the RTC are given in [Table 26-2](#).

Table 26-2. RTC CPU Interrupt Event Conditions (CPU_INT)

Index (IIDX)	Name	Description
0	RTCRDY	Indicates that the RTC counter/calendar registers are safe to be read for approximately one second
1	RTCTEV	Interval interrupt, configurable as once per minute, per hour, at midnight, or at noon
2	RTCA1	Calendar alarm 1 interrupt
3	RTCA2	Calendar alarm 2 interrupt
4	RTC0PS	Prescaler 0 periodic alarm interrupt
5	RTC1PS	Prescaler 1 periodic alarm interrupt

The CPU interrupt event configuration is managed with the CPU_INT event management registers. See [Section 7.2.5](#) for guidance on configuring the event registers for CPU interrupts.

26.3.9.2 Generic Event Publisher (GEN_EVENT)

The RTC module provides six interrupt sources, of which one can be configured to publish an event to a generic event route channel are given in [Table 26-3](#).

Table 26-3. RTC Generic Event Publisher Conditions (GEN_EVENT)

Index	Name	Description
0	RTCRDY	Indicates that the RTC counter/calendar registers are safe to be read for approximately one second
1	RTCTEV	Interval interrupt, configurable as once per minute, per hour, at midnight, or at noon
2	RTCA1	Calendar alarm 1 interrupt
3	RTCA2	Calendar alarm 2 interrupt
4	RTC0PS	Prescaler 0 periodic alarm interrupt
5	RTC1PS	Prescaler 1 periodic alarm interrupt

The generic event publisher configuration is managed with the GEN_EVENT event management registers. See [Section 7.2.5](#) for guidance on configuring the event registers for generic event publishers.

The generic event channel which generic event is to publish to must be selected by writing the target generic channel ID to the **FPUB_0** register in the RTC. See [Section 7.1.3.3](#) for guidance on configuring generic event routes.

If this publisher is not used in an application, the FPUB_0 register can be left in a disconnected state (set equal to zero) and no events should be unmasked through the MIS register in the RTC GEN_EVENT register set.

26.4 RTC Registers

Table 26-4 lists the memory-mapped registers for the RTC registers. All register offset addresses not listed in Table 26-4 should be considered as reserved locations and the register contents should not be modified.

Table 26-4. RTC Registers

Offset	Acronym	Register Name	Section
444h	FPUB_0	Publisher Port 0	FPUB_0 Register (Offset = 444h) [Reset = 00000000h]
800h	PWREN	Power enable	PWREN Register (Offset = 800h) [Reset = 00000000h]
804h	RSTCTL	Reset Control	RSTCTL Register (Offset = 804h) [Reset = 00000000h]
808h	CLKCFG	Peripheral Clock Configuration Register	CLKCFG Register (Offset = 808h) [Reset = 00000000h]
814h	STAT	Status Register	STAT Register (Offset = 814h) [Reset = 00000000h]
1004h	CLKSEL	Clock Select for Ultra Low Power peripherals	CLKSEL Register (Offset = 1004h) [Reset = 00000002h]
1020h	IIDX	Interrupt Index Register	IIDX Register (Offset = 1020h) [Reset = 00000000h]
1028h	IMASK	Interrupt mask	IMASK Register (Offset = 1028h) [Reset = 00000000h]
1030h	RIS	Raw interrupt status	RIS Register (Offset = 1030h) [Reset = 00000000h]
1038h	MIS	Masked interrupt status	MIS Register (Offset = 1038h) [Reset = 00000000h]
1040h	ISSET	Interrupt set	ISSET Register (Offset = 1040h) [Reset = 00000000h]
1048h	ICLR	Interrupt clear	ICLR Register (Offset = 1048h) [Reset = 00000000h]
1050h	IIDX	Interrupt Index Register	IIDX Register (Offset = 1050h) [Reset = 00000000h]
1058h	IMASK	Interrupt mask	IMASK Register (Offset = 1058h) [Reset = 00000000h]
1060h	RIS	Raw interrupt status	RIS Register (Offset = 1060h) [Reset = 00000000h]
1068h	MIS	Masked interrupt status	MIS Register (Offset = 1068h) [Reset = 00000000h]
1070h	ISSET	Interrupt set	ISSET Register (Offset = 1070h) [Reset = 00000000h]
1078h	ICLR	Interrupt clear	ICLR Register (Offset = 1078h) [Reset = 00000000h]
10E0h	EVT_MODE	Event Mode	EVT_MODE Register (Offset = 10E0h) [Reset = 00000009h]
10FCh	DESC	RTC Descriptor Register	DESC Register (Offset = 10FCh) [Reset = 09118010h]
1100h	CLKCTL	RTC Clock Control Register	CLKCTL Register (Offset = 1100h) [Reset = 00000000h]
1104h	DBGCTL	RTC Module Debug Control Register	DBGCTL Register (Offset = 1104h) [Reset = 00000000h]
1108h	CTL	RTC Control Register	CTL Register (Offset = 1108h) [Reset = 00000000h]
110Ch	STA	RTC Status Register	STA Register (Offset = 110Ch) [Reset = 00000000h]

Table 26-4. RTC Registers (continued)

Offset	Acronym	Register Name	Section
1110h	CAL	RTC Clock Offset Calibration Register	CAL Register (Offset = 1110h) [Reset = 00000000h]
1114h	TCMP	RTC Temperature Compensation Register	TCMP Register (Offset = 1114h) [Reset = 00000000h]
1118h	SEC	RTC Seconds Register - Calendar Mode With Binary / BCD Format	SEC Register (Offset = 1118h) [Reset = X]
111Ch	MIN	RTC Minutes Register - Calendar Mode With Binary / BCD Format	MIN Register (Offset = 111Ch) [Reset = X]
1120h	HOURL	RTC Hours Register - Calendar Mode With Binary / BCD Format	HOURL Register (Offset = 1120h) [Reset = X]
1124h	DAY	RTC Day Of Week / Month Register - Calendar Mode With Binary / BCD Format	DAY Register (Offset = 1124h) [Reset = X]
1128h	MON	RTC Month Register - Calendar Mode With Binary / BCD Format	MON Register (Offset = 1128h) [Reset = X]
112Ch	YEAR	RTC Year Register - Calendar Mode With Binary / BCD Format	YEAR Register (Offset = 112Ch) [Reset = X]
1130h	A1MIN	RTC Minute Alarm Register - Calendar Mode With Binary / BCD Format	A1MIN Register (Offset = 1130h) [Reset = 00000000h]
1134h	A1HOUR	RTC Hours Alarm Register - Calendar Mode With Binary / BCD Format	A1HOUR Register (Offset = 1134h) [Reset = 00000000h]
1138h	A1DAY	RTC Alarm Day Of Week / Month Register - Calendar Mode With Binary / BCD Format	A1DAY Register (Offset = 1138h) [Reset = 00000000h]
113Ch	A2MIN	RTC Minute Alarm Register - Calendar Mode With Binary / BCD Format	A2MIN Register (Offset = 113Ch) [Reset = 00000000h]
1140h	A2HOUR	RTC Hours Alarm Register - Calendar Mode With Binary / BCD Format	A2HOUR Register (Offset = 1140h) [Reset = 00000000h]
1144h	A2DAY	RTC Alarm Day Of Week / Month Register - Calendar Mode With Binary / BCD Format	A2DAY Register (Offset = 1144h) [Reset = 00000000h]
1148h	PSCTL	RTC Prescale Timer 0/1 Control Register	PSCTL Register (Offset = 1148h) [Reset = 00000008h]

Complex bit access types are encoded to fit into small table cells. [Table 26-5](#) shows the codes that are used for access types in this section.

Table 26-5. RTC Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
R-0	R -0	Read Returns 0s
Write Type		
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value

26.4.1 FPUB_0 Register (Offset = 444h) [Reset = 00000000h]

FPUB_0 is shown in [Figure 26-3](#) and described in [Table 26-6](#).

Return to the [Summary Table](#).

Publisher port

Figure 26-3. FPUB_0 Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R/W-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED												CHANID			
R/W-0h												R/W-0h			

Table 26-6. FPUB_0 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-0	CHANID	R/W	0h	0 = disconnected. 1-15 = connected to channelID = CHANID. 0h = A value of 0 specifies that the event is not connected Fh = Consult your device data sheet as the actual allowed maximum may be less than 15.

26.4.2 PWREN Register (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 26-4](#) and described in [Table 26-7](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 26-4. PWREN Register

31	30	29	28	27	26	25	24
KEY							
R-0/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-0h							R/WK-0h

Table 26-7. PWREN Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	R-0/W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	ENABLE	R/WK	0h	Enable the power KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

26.4.3 RSTCTL Register (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 26-5](#) and described in [Table 26-8](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 26-5. RSTCTL Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCL R	RESETASSERT
W-0h						WK-0h	WK-0h

Table 26-8. RSTCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	W	0h	
1	RESETSTKYCLR	WK	0h	Clear the RESETSTKY bit in the STAT register KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

26.4.4 CLKCFG Register (Offset = 808h) [Reset = 00000000h]

CLKCFG is shown in [Figure 26-6](#) and described in [Table 26-9](#).

Return to the [Summary Table](#).

Peripheral Clock Configuration Register

Figure 26-6. CLKCFG Register

31	30	29	28	27	26	25	24
KEY							
R-0/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							BLOCKASYNC
R/W-0h							R/WK-0h
7	6	5	4	3	2	1	0
RESERVED							
R/W-0h							

Table 26-9. CLKCFG Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	R-0/W	0h	KEY to Allow State Change -- 0xA9 A9h (W) = key value to allow change field of GPRCM
23-9	RESERVED	R/W	0h	
8	BLOCKASYNC	R/WK	0h	Async Clock Request is blocked from starting SYSOSC or forcing bus clock to 32MHz KEY must be set to A9h to write to this bit. 0h = Not block async clock request 1h = Block async clock request
7-0	RESERVED	R/W	0h	

26.4.5 STAT Register (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 26-7](#) and described in [Table 26-10](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 26-7. STAT Register

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED							
R-							

Table 26-10. STAT Register Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0h	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

26.4.6 CLKSEL Register (Offset = 1004h) [Reset = 00000002h]

CLKSEL is shown in [Figure 26-8](#) and described in [Table 26-11](#).

Return to the [Summary Table](#).

Clock source selection for ULP peripherals

Figure 26-8. CLKSEL Register

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							
R-							
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED						LFCLK_SEL	RESERVED
R-						R-1h	R-

Table 26-11. CLKSEL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1	LFCLK_SEL	R	1h	Selects LFCLK as clock source if enabled 0h = LFCLK is disabled as clock source 1h = LFCLK is enabled as clock source
0	RESERVED	R	0h	

26.4.7 IIDX Register (Offset = 1020h) [Reset = 00000000h]

IIDX is shown in [Figure 26-9](#) and described in [Table 26-12](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 26-9. IIDX Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 26-12. IIDX Register Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 00h = No interrupt pending 1h = RTC-Ready interrupt; Interrupt flag: RTCRDY 2h = Time-Event interrupt; Interrupt flag: RTCTEV 3h = Alarm-1 interrupt; Interrupt flag: RTCA1 4h = Alarm-2 interrupt; Interrupt flag: RTCA2 5h = Prescaler-0 interrupt; Interrupt flag: RT0PS 6h = Prescaler-1 interrupt; Interrupt flag: RT1PS

26.4.8 IMASK Register (Offset = 1028h) [Reset = 00000000h]

IMASK is shown in [Figure 26-10](#) and described in [Table 26-13](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 26-10. IMASK Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED		RT1PS	RT0PS	RTCA2	RTCA1	RTCTEV	RTCRDY
R/W-0h		R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 26-13. IMASK Register Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R/W	0h	
5	RT1PS	R/W	0h	Enable Prescaler-1 interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
4	RT0PS	R/W	0h	Enable Prescaler-0 interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
3	RTCA2	R/W	0h	Enable Alarm-2 interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	RTCA1	R/W	0h	Enable Alarm-1 interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	RTCTEV	R/W	0h	Enable Time-Event interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	RTCRDY	R/W	0h	Enable RTC-Ready interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

26.4.9 RIS Register (Offset = 1030h) [Reset = 00000000h]

RIS is shown in [Figure 26-11](#) and described in [Table 26-14](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 26-11. RIS Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED		RT1PS	RT0PS	RTCA2	RTCA1	RTCTEV	RTCRDY
R-0h		R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 26-14. RIS Register Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R	0h	
5	RT1PS	R	0h	Raw Prescaler-1 interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
4	RT0PS	R	0h	Raw Prescaler-0 interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
3	RTCA2	R	0h	Raw Alarm-2 interrupts status 0h = Interrupt did not occur 1h = Interrupt occurred
2	RTCA1	R	0h	Raw Alarm-1 interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
1	RTCTEV	R	0h	Raw Time-Event interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
0	RTCRDY	R	0h	Raw RTC-Ready interrupts status 0h = Interrupt did not occur 1h = Interrupt occurred

26.4.10 MIS Register (Offset = 1038h) [Reset = 00000000h]

MIS is shown in [Figure 26-12](#) and described in [Table 26-15](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 26-12. MIS Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED		RT1PS	RT0PS	RTCA2	RTCA1	RTCTEV	RTCRDY
R-0h		R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 26-15. MIS Register Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R	0h	
5	RT1PS	R	0h	Masked Prescaler-1 interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
4	RT0PS	R	0h	Masked Prescaler-0 interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
3	RTCA2	R	0h	Masked Alarm-2 interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
2	RTCA1	R	0h	Masked Alarm-1 interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
1	RTCTEV	R	0h	Masked Time-Event interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
0	RTCRDY	R	0h	Masked RTC-Ready interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred

26.4.11 ISET Register (Offset = 1040h) [Reset = 00000000h]

ISET is shown in [Figure 26-13](#) and described in [Table 26-16](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 26-13. ISET Register

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED		RT1PS	RT0PS	RTCA2	RTCA1	RTCDEV	RTCRDY
W-0h		W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 26-16. ISET Register Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	W	0h	
5	RT1PS	W	0h	Set Prescaler-1 interrupt 0h = Writing 0 has no effect 1h = Set Interrupt
4	RT0PS	W	0h	Set Prescaler-0 interrupt 0h = Writing 0 has no effect 1h = Set Interrupt
3	RTCA2	W	0h	Set Alarm-2 interrupt 0h = Writing 0 has no effect 1h = Set Interrupt
2	RTCA1	W	0h	Set Alarm-1 interrupt 0h = Writing 0 has no effect 1h = Set Interrupt
1	RTCDEV	W	0h	Set Time-Event interrupt 0h = Writing 0 has no effect 1h = Set Interrupt
0	RTCRDY	W	0h	Set RTC-Ready interrupt 0h = Writing 0 has no effect 1h = Set Interrupt

26.4.12 ICLR Register (Offset = 1048h) [Reset = 00000000h]

ICLR is shown in [Figure 26-14](#) and described in [Table 26-17](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 26-14. ICLR Register

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED		RT1PS	RT0PS	RTCA2	RTCA1	RTCDEV	RTCRDY
W-0h		W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 26-17. ICLR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	W	0h	
5	RT1PS	W	0h	Clear Prescaler-1 interrupt 0h = Writing 0 has no effect 1h = Clear Interrupt
4	RT0PS	W	0h	Clear Prescaler-0 interrupt 0h = Writing 0 has no effect 1h = Clear Interrupt
3	RTCA2	W	0h	Clear Alarm-2 interrupt 0h = Writing 0 has no effect 1h = Clear Interrupt
2	RTCA1	W	0h	Clear Alarm-1 interrupt 0h = Writing 0 has no effect 1h = Clear Interrupt
1	RTCDEV	W	0h	Clear Time-Event interrupt 0h = Writing 0 has no effect 1h = Clear Interrupt
0	RTCRDY	W	0h	Clear RTC-Ready interrupt 0h = Writing 0 has no effect 1h = Clear Interrupt

26.4.13 IIDX Register (Offset = 1050h) [Reset = 00000000h]

IIDX is shown in [Figure 26-15](#) and described in [Table 26-18](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. Value 0x00 means no event pending. Interrupt 1 is the highest priority, IIDX next highest, 4, 8, ... IIDX^31 is the least priority. That is, the least bit position that is set to 1 denotes the highest priority pending interrupt. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred. On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in [RIS] and [MIS] are cleared as well. After a read from the CPU (not from the debug interface), the register is updated with the next highest priority interrupt, if none are pending, then it should display 0x0.

Figure 26-15. IIDX Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 26-18. IIDX Register Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 00h = No interrupt pending 1h = RTC-Ready interrupt; Interrupt flag: RTCRDY 2h = Time-Event interrupt; Interrupt flag: RTCTEV 3h = Alarm-1 interrupt; Interrupt flag: RTCA1 4h = Alarm-2 interrupt; Interrupt flag: RTCA2 5h = Prescaler-0 interrupt; Interrupt flag: RT0PS 6h = Prescaler-1 interrupt; Interrupt flag: RT1PS

26.4.14 IMASK Register (Offset = 1058h) [Reset = 00000000h]

IMASK is shown in [Figure 26-16](#) and described in [Table 26-19](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 26-16. IMASK Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED		RT1PS	RT0PS	RTCA2	RTCA1	RTCTEV	RTCRDY
R/W-0h		R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 26-19. IMASK Register Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R/W	0h	
5	RT1PS	R/W	0h	Enable Prescaler-1 interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
4	RT0PS	R/W	0h	Enable Prescaler-0 interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
3	RTCA2	R/W	0h	Enable Alarm-2 interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	RTCA1	R/W	0h	Enable Alarm-1 interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	RTCTEV	R/W	0h	Enable Time-Event interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	RTCRDY	R/W	0h	Enable RTC-Ready interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

26.4.15 RIS Register (Offset = 1060h) [Reset = 00000000h]

RIS is shown in [Figure 26-17](#) and described in [Table 26-20](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 26-17. RIS Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED		RT1PS	RT0PS	RTCA2	RTCA1	RTCTEV	RTCRDY
R-0h		R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 26-20. RIS Register Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R	0h	
5	RT1PS	R	0h	Raw Prescaler-1 interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
4	RT0PS	R	0h	Raw Prescaler-0 interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
3	RTCA2	R	0h	Raw Alarm-2 interrupts status 0h = Interrupt did not occur 1h = Interrupt occurred
2	RTCA1	R	0h	Raw Alarm-1 interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
1	RTCTEV	R	0h	Raw Time-Event interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
0	RTCRDY	R	0h	Raw RTC-Ready interrupts status 0h = Interrupt did not occur 1h = Interrupt occurred

26.4.16 MIS Register (Offset = 1068h) [Reset = 00000000h]

MIS is shown in [Figure 26-18](#) and described in [Table 26-21](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 26-18. MIS Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED		RT1PS	RT0PS	RTCA2	RTCA1	RTCDEV	RTCRDY
R-0h		R-0h	R-0h	R-0h	R-0h	R-0h	R-0h

Table 26-21. MIS Register Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	R	0h	
5	RT1PS	R	0h	Masked Prescaler-1 interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
4	RT0PS	R	0h	Masked Prescaler-0 interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
3	RTCA2	R	0h	Masked Alarm-2 interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
2	RTCA1	R	0h	Masked Alarm-1 interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
1	RTCDEV	R	0h	Masked Time-Event interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred
0	RTCRDY	R	0h	Masked RTC-Ready interrupt status 0h = Interrupt did not occur 1h = Interrupt occurred

26.4.17 ISET Register (Offset = 1070h) [Reset = 00000000h]

ISET is shown in [Figure 26-19](#) and described in [Table 26-22](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 26-19. ISET Register

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED		RT1PS	RT0PS	RTCA2	RTCA1	RTCTEV	RTCRDY
W-0h		W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 26-22. ISET Register Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	W	0h	
5	RT1PS	W	0h	Set Prescaler-1 interrupt 0h = Writing 0 has no effect 1h = Set Interrupt
4	RT0PS	W	0h	Set Prescaler-0 interrupt 0h = Writing 0 has no effect 1h = Set Interrupt
3	RTCA2	W	0h	Set Alarm-2 interrupt 0h = Writing 0 has no effect 1h = Set Interrupt
2	RTCA1	W	0h	Set Alarm-1 interrupt 0h = Writing 0 has no effect 1h = Set Interrupt
1	RTCTEV	W	0h	Set Time-Event interrupt 0h = Writing 0 has no effect 1h = Set Interrupt
0	RTCRDY	W	0h	Set RTC-Ready interrupt 0h = Writing 0 has no effect 1h = Set Interrupt

26.4.18 ICLR Register (Offset = 1078h) [Reset = 00000000h]

ICLR is shown in [Figure 26-20](#) and described in [Table 26-23](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 26-20. ICLR Register

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED		RT1PS	RT0PS	RTCA2	RTCA1	RTCTEV	RTCRDY
W-0h		W-0h	W-0h	W-0h	W-0h	W-0h	W-0h

Table 26-23. ICLR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-6	RESERVED	W	0h	
5	RT1PS	W	0h	Clear Prescaler-1 interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
4	RT0PS	W	0h	Clear Prescaler-0 interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
3	RTCA2	W	0h	Clear Alarm-2 interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
2	RTCA1	W	0h	Clear Alarm-1 interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
1	RTCTEV	W	0h	Clear Time-Event interrupt 0h = Clear Interrupt Mask 1h = Set Interrupt Mask
0	RTCRDY	W	0h	Clear RTC-Ready interrupt 0h = Clear Interrupt Mask 1h = Clear Interrupt

26.4.19 EVT_MODE Register (Offset = 10E0h) [Reset = 00000009h]

EVT_MODE is shown in [Figure 26-21](#) and described in [Table 26-24](#).

Return to the [Summary Table](#).

Event mode register. It is used to select whether each line is disabled, in software mode (software clears the RIS) or in hardware mode (hardware clears the RIS)

Figure 26-21. EVT_MODE Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED				EVT1_CFG		EVT0_CFG	
R/W-				R-2h		R-1h	

Table 26-24. EVT_MODE Register Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3-2	EVT1_CFG	R	2h	Event line mode 1 select 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. The software ISR clears the associated RIS flag. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.
1-0	EVT0_CFG	R	1h	Event line mode 0 select 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. The software ISR clears the associated RIS flag. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.

26.4.20 DESC Register (Offset = 10FCh) [Reset = 09118010h]

DESC is shown in [Figure 26-22](#) and described in [Table 26-25](#).

Return to the [Summary Table](#).

RTC Descriptor Register

Figure 26-22. DESC Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODULEID															
R-911h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FEATUREVER				INSTNUM				MAJREV				MINREV			
R-8h				R-0h				R-1h				R-0h			

Table 26-25. DESC Register Field Descriptions

Bit	Field	Type	Reset	Description
31-16	MODULEID	R	911h	Module identifier. This ID is unique for each module. 0x0911 = Module ID of the RTC Module 0000h = Minimum value FFFFh = Maximum value
15-12	FEATUREVER	R	8h	Feature set of this module. Differentiates the complexity of the actually instantiated module if there are differences. 0h = Minimum value Fh = Maximum value
11-8	INSTNUM	R	0h	Instantiated version. Describes which instance of the module accessed. 0h = This is the default, if there is only one instance - like for SSIM
7-4	MAJREV	R	1h	Major revision. This number holds the module revision and is incremented by the module developers. n = Major version (see device-specific data sheet) 0h = Minimum value Fh = Maximum value
3-0	MINREV	R	0h	Minor revision. This number holds the module revision and is incremented by the module developers. n = Minor module revision (see device-specific data sheet) 0h = Minimum value Fh = Maximum value

26.4.21 CLKCTL Register (Offset = 1100h) [Reset = 00000000h]

CLKCTL is shown in [Figure 26-23](#) and described in [Table 26-26](#).

Return to the [Summary Table](#).

RTC Clock Control Register

Figure 26-23. CLKCTL Register

31	30	29	28	27	26	25	24
MODCLKEN	RESERVED						
R/W-0h				R/W-0h			
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							
R/W-0h							

Table 26-26. CLKCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31	MODCLKEN	R/W	0h	This bit enables the supply of the 32kHz clock to the RTC. It will not power-up the 32kHz crystal oscillator this needs to be done in the Clock System Module. 0h = 32kHz clock is not supplied to the RTC. 1h = 32kHz clock is supplied to the RTC.
30-0	RESERVED	R/W	0h	

26.4.22 DBGCTL Register (Offset = 1104h) [Reset = 00000000h]

DBGCTL is shown in [Figure 26-24](#) and described in [Table 26-27](#).

Return to the [Summary Table](#).

RTC Module Debug Control Register

Figure 26-24. DBGCTL Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED						DBGINT	DBGRUN
R/W-0h						R/W-0h	R/W-0h

Table 26-27. DBGCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R/W	0h	
1	DBGINT	R/W	0h	Debug Interrupt Enable. 0h = Interrupts of the module will not be captured anymore if CPU is in debug state. Which means no update to the RTCRIS, RTCMISC and RTCIIDX register. 1h = Interrupts are enabled in debug mode. Interrupt requests are signaled to the interrupt controller. If the flags are required by software (polling mode) the DGBINT bit need to be set to 1.
0	DBGRUN	R/W	0h	Debug Run. 0h = Counter is halted if CPU is in debug state. 1h = Continue to operate normally ignoring the debug state of the CPU.

26.4.23 CTL Register (Offset = 1108h) [Reset = 00000000h]

CTL is shown in [Figure 26-25](#) and described in [Table 26-28](#).

Return to the [Summary Table](#).

RTC Control Register

Figure 26-25. CTL Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RTCB CD	RESERVED					RTCTEV TX	
R/W-0h	R/W-0h					R/W-0h	

Table 26-28. CTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R/W	0h	
7	RTCB CD	R/W	0h	Real-time clock BCD select. Selects BCD counting for real-time clock. 0h = Binary code selected 1h = Binary coded decimal (BCD) code selected
6-2	RESERVED	R/W	0h	
1-0	RTCTEV TX	R/W	0h	Real-time clock time event. 0h = Minute changed. 1h = Hour changed. 2h = Every day at midnight (00:00). 3h = Every day at noon (12:00).

26.4.24 STA Register (Offset = 110Ch) [Reset = 00000000h]

STA is shown in [Figure 26-26](#) and described in [Table 26-29](#).

Return to the [Summary Table](#).

RTC Status Register

Figure 26-26. STA Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED					RTCTCOK	RTCTCRDY	RTCRDY
R-0h					R-0h	R-0h	R-0h

Table 26-29. STA Register Field Descriptions

Bit	Field	Type	Reset	Description
31-3	RESERVED	R	0h	
2	RTCTCOK	R	0h	Real-time clock temperature compensation write OK. This is a read-only bit that indicates if the write to RTCTCMP is successful or not. 0h = Write to RTCTCMP is unsuccessful 1h = Write to RTCTCMP is successful
1	RTCTCRDY	R	0h	Real-time clock temperature compensation ready. This is a read only bit that indicates when the RTCTCMPx can be written. Write to RTCTCMPx should be avoided when RTCTCRDY is reset. 0h = Real-time clock temperature compensation not ready 1h = Real-time clock temperature compensation ready
0	RTCRDY	R	0h	Real-time clock ready. This bit indicates when the real-time clock time values are safe for reading. 0h = RTC time values in transition 1h = RTC time values safe for reading

26.4.25 CAL Register (Offset = 1110h) [Reset = 00000000h]

CAL is shown in [Figure 26-27](#) and described in [Table 26-30](#).

Return to the [Summary Table](#).

RTC Clock Offset Calibration Register

Figure 26-27. CAL Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED						RTCCALFX	
R/W-0h				R/W-0h			
15	14	13	12	11	10	9	8
RTCOALS	RESERVED						
R/W-0h				R/W-0h			
7	6	5	4	3	2	1	0
RTCOCALX							
R/W-0h							

Table 26-30. CAL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-18	RESERVED	R/W	0h	
17-16	RTCCALFX	R/W	0h	Real-time clock calibration frequency. Selects frequency output to RTC_OUT pin for calibration measurement. The corresponding port must be configured for the peripheral module function. 0h = No frequency output to RTC_OUT pin 1h = 512 Hz 2h = 256 Hz 3h = 1 Hz
15	RTCOALS	R/W	0h	Real-time clock offset error calibration sign. This bit decides the sign of offset error calibration. 0h = Down calibration. Frequency adjusted down. 1h = Up calibration. Frequency adjusted up.
14-8	RESERVED	R/W	0h	
7-0	RTCOALX	R/W	0h	Real-time clock offset error calibration. Each LSB represents approximately +1ppm (RTCOALXS = 1) or -1ppm (RTCOALXS = 0) adjustment in frequency. Maximum effective calibration value is +/-240ppm. Excess values written above +/-240ppm will be ignored by hardware. 0h = Minimum effective calibration value. FFh = Maximum effective calibration value is +/-240ppm. Excess values written above +/-240ppm will be ignored by hardware.

26.4.26 TCMP Register (Offset = 1114h) [Reset = 00000000h]

TCMP is shown in [Figure 26-28](#) and described in [Table 26-31](#).

Return to the [Summary Table](#).

RTC Temperature Compensation Register

Figure 26-28. TCMP Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RTCTCMPS	RESERVED						
R/W-0h	R/W-0h						
7	6	5	4	3	2	1	0
RTCTCMPX							
R/W-0h							

Table 26-31. TCMP Register Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15	RTCTCMPS	R/W	0h	Real-time clock temperature compensation sign. This bit decides the sign of temperature compensation. 0h = Down calibration. Frequency adjusted down. 1h = Up calibration. Frequency adjusted up.
14-8	RESERVED	R/W	0h	
7-0	RTCTCMPX	R/W	0h	Real-time clock temperature compensation. Value written into this register is used for temperature compensation of RTC. Each LSB represents approximately +1ppm (RTCTCMPS = 1) or -1ppm (RTCTCMPS = 0) adjustment in frequency. Maximum effective calibration value is +/-240ppm. Excess values written above +/-240ppm are ignored by hardware. Reading from RTCTCMP register at any time returns the cumulative value which is the signed addition of RTCOALx and RTCTCMPX values, and the updated sign bit (RTCTCMPS) of the addition result. 00h = Minimum value FFh = Maximum value

26.4.27 SEC Register (Offset = 1118h) [Reset = X]

SEC is shown in [Figure 26-29](#) and described in [Table 26-32](#).

Return to the [Summary Table](#).

RTC Seconds Register - Calendar Mode With Binary / BCD Format

Figure 26-29. SEC Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED		SECHIGHBCD			SECLOWBCD		
R/W-0h		R/W-X			R/W-X		
7	6	5	4	3	2	1	0
RESERVED		SECBIN					
R/W-0h		R/W-X					

Table 26-32. SEC Register Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	R/W	0h	
14-12	SECHIGHBCD	R/W	X	Seconds BCD – high digit (0 to 5). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 5h = Maximum value
11-8	SECLOWBCD	R/W	X	Seconds BCD – low digit (0 to 9). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 9h = Maximum value
7-6	RESERVED	R/W	0h	
5-0	SECBIN	R/W	X	Seconds Binary (0 to 59). If RTCBCD=1 write to these bits will be ignored and read give the value 0. 00h = Minimum value 3Bh = Maximum value

26.4.28 MIN Register (Offset = 111Ch) [Reset = X]

MIN is shown in [Figure 26-30](#) and described in [Table 26-33](#).

Return to the [Summary Table](#).

RTC Minutes Register - Calendar Mode With Binary / BCD Format

Figure 26-30. MIN Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED	MINHIGHBCD			MINLOWBCD			
R/W-0h	R/W-X			R/W-X			
7	6	5	4	3	2	1	0
RESERVED		MINBIN					
R/W-0h		R/W-X					

Table 26-33. MIN Register Field Descriptions

Bit	Field	Type	Reset	Description
31-15	RESERVED	R/W	0h	
14-12	MINHIGHBCD	R/W	X	Minutes BCD – high digit (0 to 5). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 5h = Maximum value
11-8	MINLOWBCD	R/W	X	Minutes BCD – low digit (0 to 9). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 9h = Maximum value
7-6	RESERVED	R/W	0h	
5-0	MINBIN	R/W	X	Minutes Binary (0 to 59). If RTCBCD=1 write to these bits will be ignored and read give the value 0. 00h = Minimum value 3Bh = Maximum value

26.4.29 HOUR Register (Offset = 1120h) [Reset = X]

HOUR is shown in [Figure 26-31](#) and described in [Table 26-34](#).

Return to the [Summary Table](#).

RTC Hours Register - Calendar Mode With Binary / BCD Format

Figure 26-31. HOUR Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED		HOURHIGHBCD		HOURLOWBCD			
R/W-0h		R/W-X		R/W-X			
7	6	5	4	3	2	1	0
RESERVED			HOURBIN				
R/W-0h			R/W-X				

Table 26-34. HOUR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-14	RESERVED	R/W	0h	
13-12	HOURHIGHBCD	R/W	X	Hours BCD – high digit (0 to 2). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value. 2h = Maximum value.
11-8	HOURLOWBCD	R/W	X	Hours BCD – low digit (0 to 9). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value. 9h = Maximum value.
7-5	RESERVED	R/W	0h	
4-0	HOURBIN	R/W	X	Hours Binary (0 to 23). If RTCBCD=1 write to these bits will be ignored and read give the value 0. 00h = Minimum value. 17h = Maximum value.

26.4.30 DAY Register (Offset = 1124h) [Reset = X]

DAY is shown in [Figure 26-32](#) and described in [Table 26-35](#).

Return to the [Summary Table](#).

RTC Day of Week/Month Register - Calendar Mode With Binary / BCD Format

Figure 26-32. DAY Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED		DOMHIGHBCD		DOMLOWBCD			
R/W-0h		R/W-X		R/W-X			
15	14	13	12	11	10	9	8
RESERVED			DOMBIN				
R/W-0h			R/W-X				
7	6	5	4	3	2	1	0
RESERVED					DOW		
R/W-0h					R/W-X		

Table 26-35. DAY Register Field Descriptions

Bit	Field	Type	Reset	Description
31-22	RESERVED	R/W	0h	
21-20	DOMHIGHBCD	R/W	X	Day of month BCD – high digit (0 to 3). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 3h = Maximum value
19-16	DOMLOWBCD	R/W	X	Day of month BCD – low digit (0 to 9). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 9h = Maximum value
15-13	RESERVED	R/W	0h	
12-8	DOMBIN	R/W	X	Day of month Binary (1 to 28, 29, 30, 31). If RTCBCD=1 write to these bits will be ignored and read give the value 0. 00h = Minimum value 1Fh = Maximum value
7-3	RESERVED	R/W	0h	
2-0	DOW	R/W	X	Day of week (0 to 6). These bits are valid if RTCBCD=1 or RTCBCD=0. 0h = Minimum value 6h = Maximum value

26.4.31 MON Register (Offset = 1128h) [Reset = X]

MON is shown in [Figure 26-33](#) and described in [Table 26-36](#).

Return to the [Summary Table](#).

RTC Month Register - Calendar Mode With Binary / BCD Format

Figure 26-33. MON Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED			MONHIGHBCD	MONLOWBCD			
R/W-0h			R/W-X	R/W-X			
7	6	5	4	3	2	1	0
RESERVED				MONBIN			
R/W-0h				R/W-X			

Table 26-36. MON Register Field Descriptions

Bit	Field	Type	Reset	Description
31-13	RESERVED	R/W	0h	
12	MONHIGHBCD	R/W	X	Month BCD – high digit (0 or 1). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 1h = Maximum value
11-8	MONLOWBCD	R/W	X	Month BCD – low digit (0 to 9). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 9h = Maximum value
7-4	RESERVED	R/W	0h	
3-0	MONBIN	R/W	X	Month Binary (1 to 12). If RTCBCD=1 write to these bits will be ignored and read give the value 0. 0h = Minimum value Ch = Maximum value

26.4.32 YEAR Register (Offset = 112Ch) [Reset = X]

YEAR is shown in [Figure 26-34](#) and described in [Table 26-37](#).

Return to the [Summary Table](#).

RTC Year Register - Calendar Mode With Binary / BCD Format

Figure 26-34. YEAR Register

31	30	29	28	27	26	25	24
RESERVED	CENTHIGHBCD				CENTLOWBCD		
R/W-0h	R/W-X				R/W-X		
23	22	21	20	19	18	17	16
DECADEBCD				YEARLOWESTBCD			
R/W-X				R/W-X			
15	14	13	12	11	10	9	8
RESERVED				YEARHIGHBIN			
R/W-0h				R/W-X			
7	6	5	4	3	2	1	0
YEARLOWBIN							
R/W-X							

Table 26-37. YEAR Register Field Descriptions

Bit	Field	Type	Reset	Description
31	RESERVED	R/W	0h	
30-28	CENTHIGHBCD	R/W	X	Century BCD – high digit (0 to 4). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 4h = Maximum value
27-24	CENTLOWBCD	R/W	X	Century BCD – low digit (0 to 9). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 9h = Maximum value
23-20	DECADEBCD	R/W	X	Decade BCD (0 to 9). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 9h = Maximum value
19-16	YEARLOWESTBCD	R/W	X	Year BCD – lowest digit (0 to 9). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 9h = Maximum value
15-12	RESERVED	R/W	0h	
11-8	YEARHIGHBIN	R/W	X	Year Binary – high byte. Valid values for Year are 0 to 4095. If RTCBCD=1 write to these bits will be ignored and read give the value 0. 0h = Minimum value Fh = Maximum value
7-0	YEARLOWBIN	R/W	X	Year Binary – low byte. Valid values for Year are 0 to 4095. If RTCBCD=1 write to these bits will be ignored and read give the value 0. 00h = Minimum value FFh = Maximum value

26.4.33 A1MIN Register (Offset = 1130h) [Reset = 00000000h]

A1MIN is shown in [Figure 26-35](#) and described in [Table 26-38](#).

Return to the [Summary Table](#).

RTC Minutes Alarm Register - Calendar Mode With Binary / BCD Format

Figure 26-35. A1MIN Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
AMINAEB CD	AMINHIGHBCD			AMINLOWBCD			
R/W-0h	R/W-0h			R/W-0h			
7	6	5	4	3	2	1	0
AMINAEBIN	RESERVED	AMINBIN					
R/W-0h	R/W-0h	R/W-0h					

Table 26-38. A1MIN Register Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15	AMINAEB CD	R/W	0h	Alarm Minutes BCD enable. If RTCBCD=0 this bit is always 0. Write to this bit will be ignored. 0h = Alarm disabled 1h = Alarm enabled
14-12	AMINHIGHBCD	R/W	0h	Alarm Minutes BCD – high digit (0 to 5). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 5h = Maximum value
11-8	AMINLOWBCD	R/W	0h	Alarm Minutes BCD – low digit (0 to 9). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 9h = Maximum value
7	AMINAEBIN	R/W	0h	Alarm Minutes Binary enable. If RTCBCD=1 this bit is always 0. Write to this bit will be ignored. 0h = Alarm disabled 1h = Alarm enabled
6	RESERVED	R/W	0h	
5-0	AMINBIN	R/W	0h	Alarm Minutes Binary (0 to 59). If RTCBCD=1 write to these bits will be ignored and read give the value 0. 00h = Minimum value 3Bh = Maximum value

26.4.34 A1HOUR Register (Offset = 1134h) [Reset = 00000000h]

A1HOUR is shown in [Figure 26-36](#) and described in [Table 26-39](#).

Return to the [Summary Table](#).

RTC Hours Alarm Register - Calendar Mode With Binary / BCD Format

Figure 26-36. A1HOUR Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
AHOURAEB CD	RESERVED	AHOURHIGHBCD		AHOURLOWBCD			
R/W-0h	R/W-0h	R/W-0h		R/W-0h			
7	6	5	4	3	2	1	0
AHOURAEBIN	RESERVED		AHOURBIN				
R/W-0h	R/W-0h		R/W-0h				

Table 26-39. A1HOUR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15	AHOURAEB CD	R/W	0h	Alarm Hours BCD enable. If RTCBCD=0 this bit is always 0. Write to this bit will be ignored. 0h = Alarm disabled 1h = Alarm enabled
14	RESERVED	R/W	0h	
13-12	AHOURHIGHBCD	R/W	0h	Alarm Hours BCD – high digit (0 to 2). If RTCBCD=0 write to these bits will be ignored and read give the value 0.. 0h = Minimum value 2h = Maximum value
11-8	AHOURLOWBCD	R/W	0h	Alarm Hours BCD – low digit (0 to 9). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 9h = Maximum value
7	AHOURAEBIN	R/W	0h	Alarm Hours Binary enable. If RTCBCD=1 this bit is always 0. Write to this bit will be ignored. 0h = Alarm disabled 1h = Alarm enabled
6-5	RESERVED	R/W	0h	
4-0	AHOURBIN	R/W	0h	Alarm Hours Binary (0 to 23). If RTCBCD=1 write to these bits will be ignored and read give the value 0. 00h = Minimum value 17h = Maximum value

26.4.35 A1DAY Register (Offset = 1138h) [Reset = 00000000h]

A1DAY is shown in [Figure 26-37](#) and described in [Table 26-40](#).

Return to the [Summary Table](#).

RTC Alarm Day Of Week / Month Register - Calendar Mode With Binary / BCD Format

Figure 26-37. A1DAY Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
ADOMAEB CD	RESERVED	ADOMHIGHBCD		ADOMLOWBCD			
R/W-0h	R/W-0h	R/W-0h		R/W-0h			
15	14	13	12	11	10	9	8
ADOMAEBIN	RESERVED		ADOMBIN				
R/W-0h	R/W-0h		R/W-0h				
7	6	5	4	3	2	1	0
ADOWAE	RESERVED				ADOW		
R/W-0h	R/W-0h				R/W-0h		

Table 26-40. A1DAY Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	RESERVED	R/W	0h	
23	ADOMAEB CD	R/W	0h	Alarm Day of month BCD enable. If RTCBCD=0 this bit is always 0. Write to this bit will be ignored. 0h = Alarm disabled 1h = Alarm enabled
22	RESERVED	R/W	0h	
21-20	ADOMHIGHBCD	R/W	0h	Alarm Day of month BCD – high digit (0 to 3). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 3h = Maximum value
19-16	ADOMLOWBCD	R/W	0h	Alarm Day of month BCD – low digit (0 to 9). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 9h = Maximum value
15	ADOMAEBIN	R/W	0h	Alarm Day of month Binary enable. If RTCBCD=1 this bit is always 0. Write to this bit will be ignored. 0h = Alarm disabled 1h = Alarm enabled
14-13	RESERVED	R/W	0h	
12-8	ADOMBIN	R/W	0h	Alarm Day of month Binary (1 to 28, 29, 30, 31) If RTCBCD=1 write to these bits will be ignored and read give the value 0. 00h = Minimum value 1Fh = Maximum value
7	ADOWAE	R/W	0h	Alarm Day of week enable. This bit are valid if RTCBCD=1 or RTCBCD=0. 0h = Alarm disabled 1h = Alarm enabled
6-3	RESERVED	R/W	0h	

Table 26-40. A1DAY Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2-0	ADOW	R/W	0h	Alarm Day of week (0 to 6). These bits are valid if RTCBCD=1 or RTCBCD=0. 0h = Minimum value 6h = Maximum value

26.4.36 A2MIN Register (Offset = 113Ch) [Reset = 00000000h]

A2MIN is shown in [Figure 26-38](#) and described in [Table 26-41](#).

Return to the [Summary Table](#).

RTC Minutes Alarm Register - Calendar Mode With Binary / BCD Format

Figure 26-38. A2MIN Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
AMINAEB CD		AMINHIGHBCD			AMINLOWBCD		
R/W-0h		R/W-0h			R/W-0h		
7	6	5	4	3	2	1	0
AMINAEBIN		RESERVED		AMINBIN			
R/W-0h		R/W-0h		R/W-0h			

Table 26-41. A2MIN Register Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15	AMINAEB CD	R/W	0h	Alarm Minutes BCD enable. If RTCBCD=0 this bit is always 0. Write to this bit will be ignored. 0h = Alarm disabled 1h = Alarm enabled
14-12	AMINHIGHBCD	R/W	0h	Alarm Minutes BCD – high digit (0 to 5). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 5h = Maximum value
11-8	AMINLOWBCD	R/W	0h	Alarm Minutes BCD – low digit (0 to 9). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 9h = Maximum value
7	AMINAEBIN	R/W	0h	Alarm Minutes Binary enable. If RTCBCD=1 this bit is always 0. Write to this bit will be ignored. 0h = Alarm disabled 1h = Alarm enabled
6	RESERVED	R/W	0h	
5-0	AMINBIN	R/W	0h	Alarm Minutes Binary (0 to 59). If RTCBCD=1 write to these bits will be ignored and read give the value 0. 00h = Minimum value 3Bh = Maximum value

26.4.37 A2HOUR Register (Offset = 1140h) [Reset = 00000000h]

A2HOUR is shown in [Figure 26-39](#) and described in [Table 26-42](#).

Return to the [Summary Table](#).

RTC Hours Alarm Register - Calendar Mode With Binary / BCD Format

Figure 26-39. A2HOUR Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
AHOURAEB CD	RESERVED	AHOURHIGHBCD		AHOURLOWBCD			
R/W-0h	R/W-0h	R/W-0h		R/W-0h			
7	6	5	4	3	2	1	0
AHOURAEBIN	RESERVED		AHOURBIN				
R/W-0h	R/W-0h		R/W-0h				

Table 26-42. A2HOUR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-16	RESERVED	R/W	0h	
15	AHOURAEB CD	R/W	0h	Alarm Hours BCD enable. If RTCBCD=0 this bit is always 0. Write to this bit will be ignored. 0h = Alarm disabled 1h = Alarm enabled
14	RESERVED	R/W	0h	
13-12	AHOURHIGHBCD	R/W	0h	Alarm Hours BCD – high digit (0 to 2). If RTCBCD=0 write to these bits will be ignored and read give the value 0.. 0h = Minimum value 2h = Maximum value
11-8	AHOURLOWBCD	R/W	0h	Alarm Hours BCD – low digit (0 to 9). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 9h = Maximum value
7	AHOURAEBIN	R/W	0h	Alarm Hours Binary enable. If RTCBCD=1 this bit is always 0. Write to this bit will be ignored. 0h = Alarm disabled 1h = Alarm enabled
6-5	RESERVED	R/W	0h	
4-0	AHOURBIN	R/W	0h	Alarm Hours Binary (0 to 23). If RTCBCD=1 write to these bits will be ignored and read give the value 0. 00h = Minimum value 17h = Maximum value

26.4.38 A2DAY Register (Offset = 1144h) [Reset = 00000000h]

A2DAY is shown in [Figure 26-40](#) and described in [Table 26-43](#).

Return to the [Summary Table](#).

RTC Alarm Day Of Week / Month Register - Calendar Mode With Binary / BCD Format

Figure 26-40. A2DAY Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
ADOMAEB CD	RESERVED	ADOMHIGHBCD		ADOMLOWBCD			
R/W-0h	R/W-0h	R/W-0h		R/W-0h			
15	14	13	12	11	10	9	8
ADOMAEBIN	RESERVED		ADOMBIN				
R/W-0h	R/W-0h		R/W-0h				
7	6	5	4	3	2	1	0
ADOWAE	RESERVED				ADOW		
R/W-0h	R/W-0h				R/W-0h		

Table 26-43. A2DAY Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	RESERVED	R/W	0h	
23	ADOMAEB CD	R/W	0h	Alarm Day of month BCD enable. If RTCBCD=0 this bit is always 0. Write to this bit will be ignored. 0h = Alarm disabled 1h = Alarm enabled
22	RESERVED	R/W	0h	
21-20	ADOMHIGHBCD	R/W	0h	Alarm Day of month BCD – high digit (0 to 3). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 3h = Maximum value
19-16	ADOMLOWBCD	R/W	0h	Alarm Day of month BCD – low digit (0 to 9). If RTCBCD=0 write to these bits will be ignored and read give the value 0. 0h = Minimum value 9h = Maximum value
15	ADOMAEBIN	R/W	0h	Alarm Day of month Binary enable. If RTCBCD=1 this bit is always 0. Write to this bit will be ignored. 0h = Alarm disabled 1h = Alarm enabled
14-13	RESERVED	R/W	0h	
12-8	ADOMBIN	R/W	0h	Alarm Day of month Binary (1 to 28, 29, 30, 31) If RTCBCD=1 write to these bits will be ignored and read give the value 0. 00h = Minimum value 1Fh = Maximum value
7	ADOWAE	R/W	0h	Alarm Day of week enable. This bit are valid if RTCBCD=1 or RTCBCD=0. 0h = Alarm disabled 1h = Alarm enabled
6-3	RESERVED	R/W	0h	

Table 26-43. A2DAY Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
2-0	ADOW	R/W	0h	Alarm Day of week (0 to 6). These bits are valid if RTCBCD=1 or RTCBCD=0. 0h = Minimum value 6h = Maximum value

26.4.39 PSCTL Register (Offset = 1148h) [Reset = 00000008h]

PSCTL is shown in [Figure 26-41](#) and described in [Table 26-44](#).

Return to the [Summary Table](#).

RTC Prescale Timer 0/1 Control Register

Figure 26-41. PSCTL Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED				RT1IP		RESERVED	
R/W-0h				R/W-0h		R/W-0h	
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED				RT0IP		RESERVED	
R/W-0h				R/W-2h		R/W-0h	

Table 26-44. PSCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-21	RESERVED	R/W	0h	
20-18	RT1IP	R/W	0h	Prescale timer 1 interrupt interval 0h = Divide by 2 - 15.6 millisecond interval 1h = Divide by 4 - 31.2 millisecond interval 2h = Divide by 8 - 62.5 millisecond interval 3h = Divide by 16 - 125 millisecond interval 4h = Divide by 32 - 250 millisecond interval 5h = Divide by 64 - 500 millisecond interval 6h = Divide by 128 - 1 second interval 7h = Divide by 256 - 2 second interval
17-5	RESERVED	R/W	0h	
4-2	RT0IP	R/W	2h	Prescale timer 0 interrupt interval 2h = Divide by 8 - 244 microsecond interval 3h = Divide by 16 - 488 microsecond interval 4h = Divide by 32 - 976 microsecond interval 5h = Divide by 64 - 1.95 millisecond interval 6h = Divide by 128 - 3.90 millisecond interval 7h = Divide by 256 - 7.81 millisecond interval
1-0	RESERVED	R/W	0h	



The window watchdog timer (WWDT) supervises code execution. If the application software does not successfully reset the window watchdog within the programmed open time window, the window watchdog generates a reset.

27.1 WWDT Overview	1558
27.2 WWDT Operation	1559
27.3 WWDT Registers	1563

27.1 WWDT Overview

The primary function of the window watchdog timer (WWDT) is to initiate a reset when correct operation of the device has failed due to an unexpected software or system delay. The WWDT can be programmed with a predefined time window within which the application software must restart the timer, indicating that application execution is proceeding normally. If application software fails to restart the timer within the specified window, the WWDT will issue a WWDT violation signal to SYSCCTL to generate a reset.

If watchdog functionality is not required in an application, the WWDT can also be configured as a basic system interval timer which is capable of generating periodic maskable interrupts to the CPU.

Key features of the WWDT include:

- A 25-bit counter with closed and open window
- Counter driven from LFCLK (fixed 32 kHz clock path) with a programmable clock divider
- Eight selectable watchdog timer periods
- Optional automatic suspension of counter when operating in low power modes
- Support for standard window watchdog mode or interval timer (non-watchdog) mode

Devices may have 1 or 2 WWDT instances. A WWDT0 violation generates a BOOTRST, which resets the peripheral and CPU state and also causes the boot configuration routine (BCR) to run. A WWDT1 violation generates a SYSRST, which resets the peripheral and CPU state but does not trigger execution of the BCR. As such, WWDT1 is well suited for recovering from execution stalls that result from software execution, while WWDT0 is well suited for catching larger issues such as a corrupted trim value, at the expense of a longer reset time.

27.1.1 Watchdog Mode

In watchdog mode, the WWDT is configured to count up to the specified WWDT period. The WWDT counter must be restarted with the configured open window of the WWDT period, or the WWDT will assert a WWDT violation to SYSCCTL and a reset will be generated.

The window watchdog timer supports detecting both a "too late" response as well as a "too early" response through the use of an optional closed window, as shown in [Figure 27-1](#). The WWDT period consists of a closed window period and an open window period. The closed window period begins first, followed by the open window period. The WWDT can only be restarted during the open window period. An attempt to restart the WWDT during the closed window period results in a violation. Following the closed window, if the WWDT is not restarted before the end of the open window, the WWDT period expires and a violation is also generated.

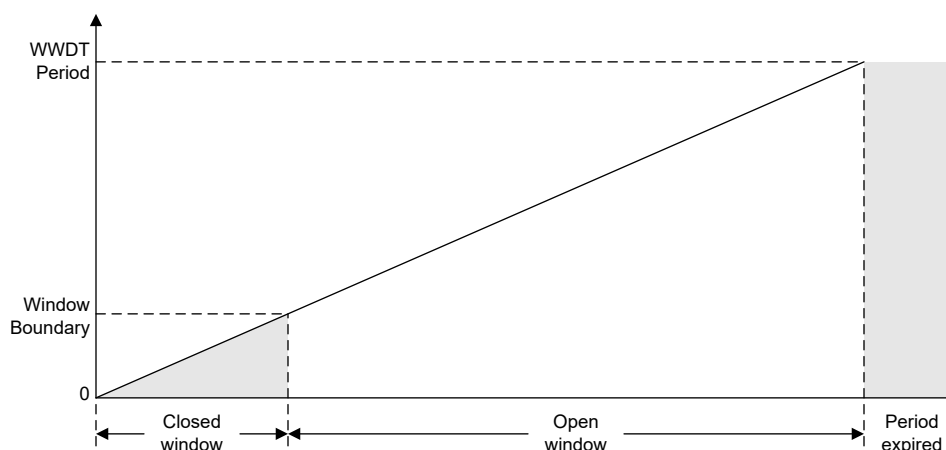


Figure 27-1. WWDT Functionality

If the closed window functionality is not desired, it can be disabled (set to 0%), giving traditional watchdog timer functionality where the WWDT can be reset any time before the WWDT period expires.

27.1.2 Interval Timer Mode

The WWDT can be used in interval timer mode to generate periodic interrupts to the CPU when not using the watchdog functionality. When used in interval timer mode, a WWDT interrupt is generated when the WWDT period expires, or when an incorrect password is applied to the WWDT control registers.

27.2 WWDT Operation

The WWDT must be enabled before being configured for use through the PWREN register (see [peripheral power enable](#)).

The WWDT is configured through the WWDTCTL0 and WWDTCTL1 registers. The registers are password protected. Any register access (read or write) must be a 32-bit access. Write access must also include the corresponding password in the most significant byte (0xC9 for WWDTCTL0, and 0xBE for WWDTCTL1). Attempting a register write without the correct password, or attempting a write with an access other than a 32-bit access generates a WWDT violation to SYSCTL. The password byte always reads as 0x00.

The WWDT is disabled and cleared after a SYSRST. The WWDTCTL0 register sets the static configuration of the WWDT, including: the clock divider, the timer period, the two closed window percentages, the timer mode (WWDT or interval), and the stop-in-sleep status. The first write (with a key match) to the WWDTCTL0 register enables the WWDT. Once the WWDT is enabled, the WWDTCTL0 register becomes write protected. Any attempt to write to the WWDTCTL0 register after the WWDT is enabled generates a WWDT violation to SYSCTL. The RUN bit in the WWDTSTAT register indicates that the WWDT is running.

Figure 27-2 shows the WWDT functional block diagram.

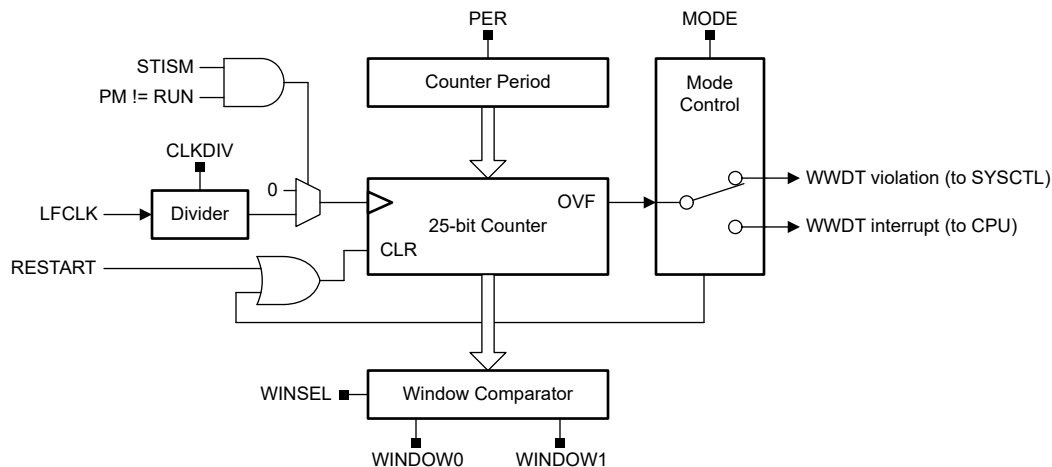


Figure 27-2. WWDT Diagram

27.2.1 Mode Selection

The WWDT operating mode (watchdog mode or interval timer mode) is selected by the MODE bit in the WWDTCTL0 register. Watchdog mode is the default mode (MODE cleared). Setting the MODE bit configures the WWDT for interval mode.

When the WWDT is in watchdog mode, the WWDT counter must be restarted within the open window period by writing the RESTART value (0x000000A7) to the WWDTCNTRST register. After a reset or restart, the WWDT counter will restart from zero. A failure to restart the WWDT within the open window or an attempt to restart the WWDT counter during the closed window will generate a WWDT violation to SYSCTL. Writing any value other than the RESTART value to the WWDTCNTRST register also generates a WWDT violation.

When the WWDT is in interval mode, the timer acts as an interval timer, generating WWDT interrupts to the CPU as specified by the WWDT period. As soon as the WWDT is enabled in interval mode, the WWDT interval timer interrupt will be asserted after the expiration of the timer. It is not necessary to restart the WWDT in interval timer mode.

27.2.2 Clock Configuration

The WWDT runs from the 32-kHz low-frequency clock (LFCLK). A clock divider supports dividing the input clock from /1 (no divide) to /8 (divide-by-8) using the CLKDIV field in the WWDTCTL0 register. The default CLKDIV setting is 0x03 (32 kHz divided by 4, or 8 kHz).

By running from the LFCLK, the WWDT time base is independent of the main clock (MCLK) and CPU clock (CPUCLK) time base, provided that these clocks are not also configured to be running from the LFCLK. While the time base may be considered as independent and derived from a separate oscillator source, LFCLK edges are synchronized to the MCLK before sourcing the WWDT to enable simple access to the memory-mapped registers from application software. A simplified view of the clock scheme is given in Figure 27-3. In Figure 27-3, the internal LFOSC is configured to set the LFCLK rate, and the internal SYSOSC sets the MCLK/CPUCLK rate. Clock selection muxes and dividers are excluded from the figure to simplify the view; the complete clock tree is provided in Section 2.3.3.

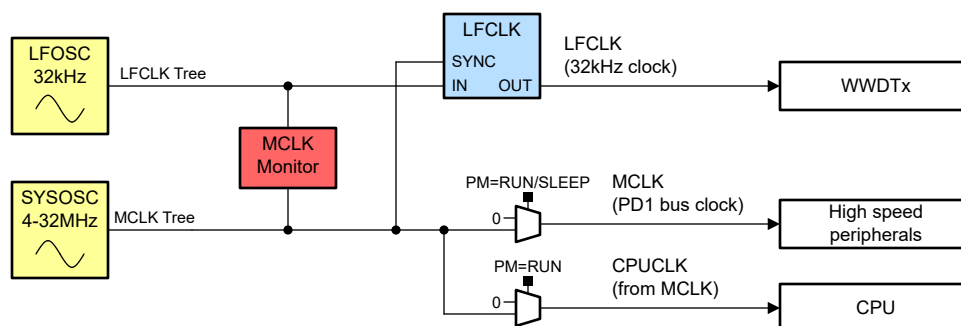


Figure 27-3. WWDT Simplified Clock Source Tree

In the event that the MCLK fails and synchronization of the LFCLK to the MCLK is lost, this failure may be detected by enabling the continuous MCLK monitor. When the MCLK monitor is enabled, a loss of MCLK is always a catastrophic failure which generates a BOOTRST within 12 LFCLK cycles. As a result, a loss of the MCLK tree, and corresponding loss of synchronization, does not prevent a BOOTRST from being generated.

Period Selection

The WWDT has a 25-bit counter which is initially stopped after a SYSRST. The WWDT period (total time interval) is set by the PER field in the WWDTCTL0 register. The total WWDT period is calculated as follows:

$$T_{WWDT} = (CLKDIV + 1) * PER_{COUNT} / 32768 \text{ Hz} \quad (31)$$

The total timer count PER_{COUNT} is selected to be one of 8 possible period count values, with the encoding given in Table 27-1.

Table 27-1. WWDT Period Total Timer Count

PER	0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7
PER_{COUNT}	2^{25}	2^{21}	2^{18}	2^{15}	2^{12}	2^{10}	2^8	2^6

The combination of the period selection (PER) and clock divider (CLKDIV) enable a wide range of WWDT periods, from as short as 1.95 ms to as long as 136.53 minutes. Table 27-2 gives all possible WWDT periods for a given combination of PER and CLKDIV.

Table 27-2. WWDT Period Timing Options

CLKDIV	PER							
	0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7
	min	min	s	s	s	ms	ms	ms
0x0 (/1)	17.07	1.07	8.00	1.00	0.13	31.25	7.81	1.95

Table 27-2. WWDT Period Timing Options (continued)

CLKDIV	PER							
	0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7
	<i>min</i>	<i>min</i>	<i>s</i>	<i>s</i>	<i>s</i>	<i>ms</i>	<i>ms</i>	<i>ms</i>
0x1 (/2)	34.13	2.13	16.00	2.00	0.25	62.50	15.63	3.91
0x2 (/3)	51.20	3.20	24.00	3.00	0.38	93.75	23.44	5.86
0x3 (/4)	68.27	4.27	32.00	4.00	0.50	125.00	32.25	7.81
0x4 (/5)	85.33	5.33	40.00	5.00	0.63	156.25	39.06	9.77
0x5 (/6)	102.40	6.40	48.00	6.00	0.75	187.50	46.88	11.72
0x6 (/7)	119.47	7.47	56.00	7.00	0.88	218.75	54.69	13.67
0x7 (/8)	136.53	8.53	64.00	8.00	1.00	250.00	62.50	15.63

Synchronization Delay

When starting or re-starting the WWDT counter, a maximum synchronization delay of one 32 kHz clock cycle (30.5μs) can occur before the WWDT counter begins counting from zero. The periods given in [Table 27-2](#) do not include this synchronization delay.

Closed Window Selection

Configuration of two closed window periods is supported by setting the WINDOW0 and WINDOW1 fields in the WWDTCTL0 register. The WINSEL bit in the WWDTCTL1 register determines the active window (either WINDOW0 or WINDOW1). Either window can be set to one of 8 possible window settings.

Table 27-3. WWDT Window Options

WINDOW	0x0	0x1	0x2	0x3	0x4	0x5	0x6	0x7
Closed window	0%	12.5%	18.75%	25%	50%	75%	81.25%	87.5%

Setting a WINDOWx value to 0x0 (0% closed, 100% open) is equivalent to disabling the window function of the WWDT. In this configuration, the WWDT can be restarted at any point during the WWDT period.

The active window selection can be changed after the WWDT has been enabled. When the WWDT is restarted by writing to the WWDTCNTRST register, the closed window selection (WINSEL) must not be changed for at least four 32-kHz clock cycles (≈122 μs).

27.2.3 Low-Power Mode Behavior

The WWDT counter can be configured to continue counting when the device is in a low-power mode (CPU is disabled) or to continue to run when the device is in a low-power mode.

The STISM bit in the WWDTCTL0 register controls if the WWDT counter stops counting in sleep mode. By default, the STISM bit is cleared, indicating that the WWDT continues to count in low-power modes. To stop the WWDT from counting in low-power modes, set the STISM bit when loading the WWDTCTL0 configuration to start the WWDT. In this case, when the low-power mode is exited and the CPU returns to operation, the WWDT counter resumes counting from the same value it held before entering the low-power mode.

27.2.4 Debug Behavior

The WWDT can be configured to stop counting or continue counting when the CPU is halted for debug by the debug subsystem. By default, the WWDT stops counting when the CPU is halted for debug and the device is in a debug state. To allow the WWDT to continue to free run when the CPU is stopped for debug, set the FREE bit in the PDBGCTL register.

27.2.5 WWDT Events

The WWDT module contains one [event publisher](#) and no [event subscribers](#). One event publisher (CPU_INT) manages WWDT interrupt requests (IRQs) to the CPU subsystem through a [static event route](#).

[Table 27-4](#) lists the WWDT events.

Table 27-4. WWDT Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU Interrupt Event	Publisher	WWDT	CPU Subsystem	Static route	CPU_INT registers	Fixed interrupt route from WWDT to CPU

27.2.5.1 CPU Interrupt Event Publisher (CPU_INT)

The WWDT module provides one interrupt source which can be configured to source a [CPU interrupt event](#). The WWDT interrupt conditions are given in [Table 27-5](#).

Table 27-5. WWDT CPU Interrupt Conditions (CPU_INT)

Index (IIDX)	Name	Description
0	INTTIM	Indicates that the WWDT interval timer period has expired

The CPU interrupt event configuration is managed with the CPU_INT event management registers. See [Section 7.2.5](#) for guidance on configuring the Event registers for CPU interrupts.

27.3 WWDT Registers

Table 27-6 lists the memory-mapped registers for the WWDT registers. All register offset addresses not listed in Table 27-6 should be considered as reserved locations and the register contents should not be modified.

Table 27-6. WWDT Registers

Offset	Acronym	Register Name	Section
800h	PWREN	Power enable	Section 27.3.1
804h	RSTCTL	Reset Control	Section 27.3.2
814h	STAT	Status Register	Section 27.3.3
1018h	PDBGCTL	Peripheral Debug Control	Section 27.3.4
1020h	IIDX	Interrupt index	Section 27.3.5
1028h	IMASK	Interrupt mask	Section 27.3.6
1030h	RIS	Raw interrupt status	Section 27.3.7
1038h	MIS	Masked interrupt status	Section 27.3.8
1040h	ISSET	Interrupt set	Section 27.3.9
1048h	ICLR	Interrupt clear	Section 27.3.10
10E0h	EVT_MODE	Event Mode	Section 27.3.11
10FCh	DESC	Module Description	Section 27.3.12
1100h	WWDTCTL0	Window Watchdog Timer Control Register 0	Section 27.3.13
1104h	WWDTCTL1	Window Watchdog Timer Control Register 0	Section 27.3.14
1108h	WWDTCNTRST	Window Watchdog Timer Counter Reset Register	Section 27.3.15
110Ch	WWDTSTAT	Window Watchdog Timer Status Register	Section 27.3.16

Complex bit access types are encoded to fit into small table cells. Table 27-7 shows the codes that are used for access types in this section.

Table 27-7. WWDT Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
Write Type		
K	K	Write protected by a key
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value

27.3.1 PWREN Register (Offset = 800h) [Reset = 00000000h]

PWREN is shown in [Figure 27-4](#) and described in [Table 27-8](#).

Return to the [Summary Table](#).

Register to control the power state

Figure 27-4. PWREN Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED							ENABLE
R/W-							K-0h

Table 27-8. PWREN Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow Power State Change 26h = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	ENABLE	K	0h	Enable the power Note: For safety devices the power cannot be disabled once enabled. KEY must be set to 26h to write to this bit. 0h = Disable Power 1h = Enable Power

27.3.2 RSTCTL Register (Offset = 804h) [Reset = 00000000h]

RSTCTL is shown in [Figure 27-5](#) and described in [Table 27-9](#).

Return to the [Summary Table](#).

Register to control reset assertion and de-assertion

Figure 27-5. RSTCTL Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED						RESETSTKYCL R	RESETASSERT
W-0h						WK-0h	WK-0h

Table 27-9. RSTCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	Unlock key B1h = KEY to allow write access to this register
23-2	RESERVED	W	0h	
1	RESETSTKYCLR	WK	0h	Clear RESETSTKY KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Clear reset sticky bit
0	RESETASSERT	WK	0h	Assert reset to the peripheral Note: For safety devices a watchdog reset by software is not possible. KEY must be set to B1h to write to this bit. 0h = Writing 0 has no effect 1h = Assert reset

27.3.3 STAT Register (Offset = 814h) [Reset = 00000000h]

STAT is shown in [Figure 27-6](#) and described in [Table 27-10](#).

Return to the [Summary Table](#).

peripheral enable and reset status

Figure 27-6. STAT Register

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							RESETSTKY
R-							R-0h
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED							
R-							

Table 27-10. STAT Register Field Descriptions

Bit	Field	Type	Reset	Description
31-17	RESERVED	R	0h	
16	RESETSTKY	R	0h	This bit indicates, if the peripheral was reset, since this bit was cleared by RESETSTKYCLR in the RSTCTL register 0h = The peripheral has not been reset since this bit was last cleared by RESETSTKYCLR in the RSTCTL register 1h = The peripheral was reset since the last bit clear
15-0	RESERVED	R	0h	

27.3.4 PDBGCTL Register (Offset = 1018h) [Reset = 00000000h]

PDBGCTL is shown in [Figure 27-7](#) and described in [Table 27-11](#).

Return to the [Summary Table](#).

This register can be used by the software developer to control the behavior of the peripheral relative to the 'Core Halted' input

Figure 27-7. PDBGCTL Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							FREE
R/W-0h							R/W-0h

Table 27-11. PDBGCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R/W	0h	
0	FREE	R/W	0h	Free run control 0h = The peripheral freezes functionality while the Core Halted input is asserted and resumes when it is deasserted. 1h = The peripheral ignores the state of the Core Halted input

27.3.5 IIDX Register (Offset = 1020h) [Reset = 00000000h]

IIDX is shown in [Figure 27-8](#) and described in [Table 27-12](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index.

Figure 27-8. IIDX Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																												STAT			
R-0h																												R-0h			

Table 27-12. IIDX Register Field Descriptions

Bit	Field	Type	Reset	Description
31-5	RESERVED	R	0h	
4-0	STAT	R	0h	Module Interrupt Vector Value. This register provides the highest priority interrupt index. A read clears the corresponding interrupt flag in RIS and MISC. 0h = No interrupt pending 1h = Interval Timer Interrupt; Interrupt Flag: INTTIM; Interrupt Priority: Highest

27.3.6 IMASK Register (Offset = 1028h) [Reset = 00000000h]

IMASK is shown in [Figure 27-9](#) and described in [Table 27-13](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.”

Figure 27-9. IMASK Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							INTTIM
R/W-0h							R/W-0h

Table 27-13. IMASK Register Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R/W	0h	
0	INTTIM	R/W	0h	Interval Timer Interrupt. 0h = Clear Interrupt Mask 1h = Set Interrupt Mask

27.3.7 RIS Register (Offset = 1030h) [Reset = 00000000h]

RIS is shown in [Figure 27-10](#) and described in [Table 27-14](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 27-10. RIS Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							INTTIM
R-0h							R-0h

Table 27-14. RIS Register Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	INTTIM	R	0h	Interval Timer Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred

27.3.8 MIS Register (Offset = 1038h) [Reset = 00000000h]

MIS is shown in [Figure 27-11](#) and described in [Table 27-15](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 27-11. MIS Register

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED							INTTIM
R-0h							R-0h

Table 27-15. MIS Register Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	INTTIM	R	0h	Interval Timer Interrupt. 0h = Interrupt did not occur 1h = Interrupt occurred

27.3.9 ISET Register (Offset = 1040h) [Reset = 00000000h]

ISET is shown in [Figure 27-12](#) and described in [Table 27-16](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 27-12. ISET Register

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED							INTTIM
W-0h							W-0h

Table 27-16. ISET Register Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	W	0h	
0	INTTIM	W	0h	Interval Timer Interrupt. 0h = Writing 0 has no effect 1h = Set Interrupt

27.3.10 ICLR Register (Offset = 1048h) [Reset = 00000000h]

ICLR is shown in [Figure 27-13](#) and described in [Table 27-17](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 27-13. ICLR Register

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED							INTTIM
W-0h							W-0h

Table 27-17. ICLR Register Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	W	0h	
0	INTTIM	W	0h	Interval Timer Interrupt. 0h = Writing 0 has no effect 1h = Clear Interrupt

27.3.11 EVT_MODE Register (Offset = 10E0h) [Reset = 00000001h]

EVT_MODE is shown in [Figure 27-14](#) and described in [Table 27-18](#).

Return to the [Summary Table](#).

Event mode register. It is used to select whether each line is disabled, in software mode (software clears the RIS) or in hardware mode (hardware clears the RIS)

Figure 27-14. EVT_MODE Register

31	30	29	28	27	26	25	24
RESERVED							
R/W-							
23	22	21	20	19	18	17	16
RESERVED							
R/W-							
15	14	13	12	11	10	9	8
RESERVED							
R/W-							
7	6	5	4	3	2	1	0
RESERVED						INT0_CFG	
R/W-						R-1h	

Table 27-18. EVT_MODE Register Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R/W	0h	
1-0	INT0_CFG	R	1h	Event line mode select for event corresponding to none.INT_EVENT[0] 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.

27.3.12 DESC Register (Offset = 10FCh) [Reset = 1F117010h]

DESC is shown in [Figure 27-15](#) and described in [Table 27-19](#).

Return to the [Summary Table](#).

This register identifies the peripheral and its exact version.

Figure 27-15. DESC Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODULEID															
R-1F11h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FEATUREVER				INSTNUM				MAJREV				MINREV			
R-7h				R-0h				R-1h				R-0h			

Table 27-19. DESC Register Field Descriptions

Bit	Field	Type	Reset	Description
31-16	MODULEID	R	1F11h	Module identification contains a unique peripheral identification number. The assignments are maintained in a central database for all of the platform modules to ensure uniqueness. 0h = Smallest value FFFFh = Highest possible value
15-12	FEATUREVER	R	7h	Feature Set for the module *instance* 0h = Smallest value Fh = Highest possible value
11-8	INSTNUM	R	0h	Instance Number within the device. This will be a parameter to the RTL for modules that can have multiple instances 0h = Smallest value Fh = Highest possible value
7-4	MAJREV	R	1h	Major rev of the IP 0h = Smallest value Fh = Highest possible value
3-0	MINREV	R	0h	Minor rev of the IP 0h = Smallest value Fh = Highest possible value

27.3.13 WWDCTL0 Register (Offset = 1100h) [Reset = 00000043h]

WWDCTL0 is shown in [Figure 27-16](#) and described in [Table 27-20](#).

Return to the [Summary Table](#).

Window Watchdog Timer Control 0 Register

NOTE: Write to this register is enabled after System Reset. The first successful write (key match) enables the Watchdog. When the watchdog is enabled all subsequent writes to this register activate the WWDT error signal to the ESM.

Figure 27-16. WWDCTL0 Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED						STISM	MODE
R/W-0h						R/W-0h	R/W-0h
15	14	13	12	11	10	9	8
RESERVED	WINDOW1			RESERVED	WINDOW0		
R/W-0h	R/W-0h			R/W-0h	R/W-0h		
7	6	5	4	3	2	1	0
RESERVED	PER			RESERVED	CLKDIV		
R/W-0h	R/W-4h			R/W-0h	R/W-3h		

Table 27-20. WWDCTL0 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow write access to this register. Writing to this register with an incorrect key activates the WWDT error signal to the ESM. Read as 0. C9h (W) = KEY to allow write access to this register
23-18	RESERVED	R/W	0h	
17	STISM	R/W	0h	Stop In Sleep Mode. The functionality of this bit requires that POLICY.HWCEN = 0. If POLICY.HWCEN = 1 the WWDT resets during sleep and needs re-configuration. Note: This bit has no effect for the global Window Watchdog as Sleep Mode is not supported. 0h = The WWDT continues to function in Sleep mode. 1h = The WWDT stops in Sleep mode and resumes where it was stopped after wakeup.
16	MODE	R/W	0h	Window Watchdog Timer Mode 0h = Window Watchdog Timer Mode. The WWDT will generate a error signal to the ESM when following conditions occur: - Timer Expiration (Timeout) - Reset WWDT during the active window closed period - Keyword violation 1h = Interval Timer Mode. The WWDT acts as an interval timer. It generates an interrupt on timeout.
15	RESERVED	R/W	0h	

Table 27-20. WWDTCTL0 Register Field Descriptions (continued)

Bit	Field	Type	Reset	Description
14-12	WINDOW1	R/W	0h	Closed window period in percentage of the timer interval. WWDTCTL1.WINSEL determines the active window setting (WWDTCTL0.WINDOW0 or WWDTCTL0.WINDOW1). 0h = 0% (No closed Window) 1h = 12.50% of the total timer period is closed window 2h = 18.75% of the total timer period is closed window 3h = 25% of the total timer period is closed window 4h = 50% of the total timer period is closed window 5h = 75% of the total timer period is closed window 6h = 81.25% of the total timer period is closed window 7h = 87.50% of the total timer period is closed window
11	RESERVED	R/W	0h	
10-8	WINDOW0	R/W	0h	Closed window period in percentage of the timer interval. WWDTCTL1.WINSEL determines the active window setting (WWDTCTL0.WINDOW0 or WWDTCTL0.WINDOW1). 0h = 0% (No closed Window) 1h = 12.50% of the total timer period is closed window 2h = 18.75% of the total timer period is closed window 3h = 25% of the total timer period is closed window 4h = 50% of the total timer period is closed window 5h = 75% of the total timer period is closed window 6h = 81.25% of the total timer period is closed window 7h = 87.50% of the total timer period is closed window
7	RESERVED	R/W	0h	
6-4	PER	R/W	4h	Timer Period of the WWDT. These bits select the total watchdog timer count. 0h = Total timer count is 2^{25} 1h = Total timer count is 2^{21} 2h = Total timer count is 2^{18} 3h = Total timer count is 2^{15} 4h = Total timer count is 2^{12} (default) 5h = Total timer count is 2^{10} 6h = Total timer count is 2^8 7h = Total timer count is 2^6
3	RESERVED	R/W	0h	
2-0	CLKDIV	R/W	3h	Module Clock Divider, Divide the clock source by CLKDIV+1. Divider values from /1 to /8 are possible. The clock divider is currently 4 bits. Bit 4 has no effect and should always be written with 0. 0h = Minimum value 7h = Maximum value

27.3.14 WWDTCCTL1 Register (Offset = 1104h) [Reset = 00000000h]

WWDTCCTL1 is shown in [Figure 27-17](#) and described in [Table 27-21](#).

Return to the [Summary Table](#).

Window Watchdog Timer Control 1 Register

Figure 27-17. WWDTCCTL1 Register

31	30	29	28	27	26	25	24
KEY							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED							WINSEL
R/W-0h							R/W-0h

Table 27-21. WWDTCCTL1 Register Field Descriptions

Bit	Field	Type	Reset	Description
31-24	KEY	W	0h	KEY to allow write access to this register. Writing to this register with an incorrect key activates the WWDTC error signal to the ESM. Read as 0. BEh (W) = KEY to allow write access to this register
23-1	RESERVED	R/W	0h	
0	WINSEL	R/W	0h	Close Window Select 0h = In window mode field WINDOW0 of WDDTCCTL0 defines the closed window size. 1h = In window mode field WINDOW1 of WDDTCCTL0 defines the closed window size.

27.3.15 WWDTCNTRST Register (Offset = 1108h) [Reset = 00000000h]

WWDTCNTRST is shown in [Figure 27-18](#) and described in [Table 27-22](#).

Return to the [Summary Table](#).

Window Watchdog Timer Counter Restart Register

Figure 27-18. WWDTCNTRST Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESTART																															
R/W-0h																															

Table 27-22. WWDTCNTRST Register Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RESTART	R/W	0h	Window Watchdog Timer Counter Restart Writing 00A7h to this register restarts the WWDT Counter. Writing any other value causes an error generation to the ESM. Read as 0. 0h = Minimum value FFFFFFFFh = Maximum value

27.3.16 WWDTSTAT Register (Offset = 110Ch) [Reset = 00000000h]

WWDTSTAT is shown in [Figure 27-19](#) and described in [Table 27-23](#).

Return to the [Summary Table](#).

Window Watchdog Timer Status Register

A write to this register has no effect.

Figure 27-19. WWDTSTAT Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RESERVED															
R-0h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED															RUN
R-0h															R-0h

Table 27-23. WWDTSTAT Register Field Descriptions

Bit	Field	Type	Reset	Description
31-1	RESERVED	R	0h	
0	RUN	R	0h	Watchdog running status flag. 0h = Watchdog counter stopped. 1h = Watchdog running.



The debug subsystem (DEBUGSS) is implemented in all MSPM0 devices. The DEBUGSS enables comprehensive debug of application software running on the processor during development by interfacing an external debug probe to the device systems through a serial wire debug (SWD) interface.

28.1 Overview	1582
28.2 Debug Features	1584
28.3 Behavior in Low Power Modes	1586
28.4 Restricting Debug Access	1587
28.5 Mailbox (DSSM)	1587

28.1 Overview

The debug subsystem (DEBUGSS) interfaces the serial wire debug (SWD) two-wire [physical interface](#) to multiple debug functions within the device. MSPM0 devices support debugging of processor execution, the device state, and the power state (through EnergyTrace technology). The DEBUGSS also provides a [mailbox system](#) for communicating with software through SWD.

Key features provided by the debug subsystem include:

- Two-wire (SWDIO, SWCLK) debug interface, compatible with both TI and 3rd party debug probes
 - On-chip pullup and pulldown resistors for SWDIO and SWCLK, respectively, enabled by default
 - Support for [disabling SWD functions](#) to use SWD pins as general-purpose input/output pins
 - Can wake the device from SHUTDOWN mode on valid SWD activity
- Debug of the processor
 - Run, halt, and step debug support
 - 4 hardware breakpoints (BPU)
 - 2 hardware watchpoints (DWT)
 - Instruction trace of up to 4 branches through the Arm micro trace buffer (MTB)
 - Unlimited software breakpoints
- Software-configurable peripheral behavior during processor debug
 - Ability to free run select peripherals through debug halt
 - Ability to halt select peripherals on a debug halt
 - Ability to request reset and mode changes to the PMCU
- Monitoring of CPU state through EnergyTrace technology
- Mailbox (DSSM) for passing data and control signals between the SWD interface and boot ROM (as well as application software)
- Support for various security features, including SWD lockout and password authenticated debugging

28.1.1 Debug Interconnect

The DEBUGSS architecture is given in [Figure 28-1](#).

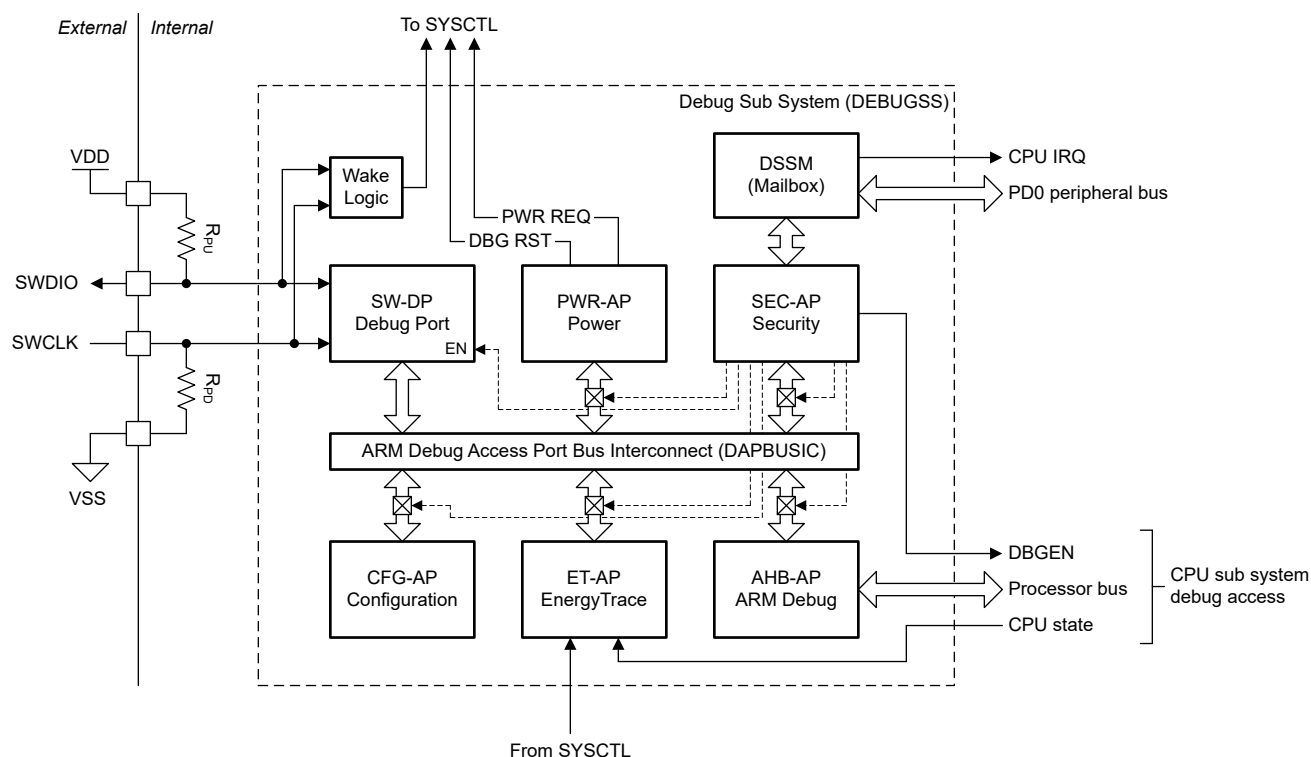


Figure 28-1. Debug Sub System Block Diagram

The SWD physical interface interacts with the Arm serial wire debug port (SW-DP) to gain access to the debug access port bus interconnect (DAPBUSIC) when the SW-DP is enabled. From TI, devices ship with the SW-DP enabled to allow SWD access to the device for development and production programming, but the SW-DP can be configured to be permanently disabled through the boot configuration policy (see [Section 28.4](#)).

The DAPBUSIC enables a debug probe to access one or more debug access ports. For a debug probe to be able to communicate with an access port, the SW-DP debug port must not be disabled by the device boot configuration policy, and the target access port must also not be disabled by the boot configuration policy. The available access ports are given in [Section 28.1.3](#).

The SWD and SW-DP also contain signaling to the PMCU module to support debug-generated resets and operating mode changes (see [Section 28.3](#)).

28.1.2 Physical Interface

Debug connections to the device are supported through an Arm serial wire debug (SWD) compliant interface. The SWD interface requires two connections:

- A bidirectional data line (SWDIO) used to send data to, and receive data from, the device
- A unidirectional clock line (SWCLK) driven by the debug probe connecting to the device

The SWD interface uses the standard logic levels of the device for SWD communication. See the device-specific data sheet for input and output logic levels for a given supply voltage (VDD). A SWCLK frequency of up to 10MHz is supported by the DEBUGSS.

During SWD operation, the SWDIO line can be driven high or driven low by either the target device or the debug probe. As either device can drive the line, when ownership of the shared SWDIO line is switched between the device and the debug probe, undriven time slots are inserted as a part of the SWD protocol. The primary purpose of the pullup resistor on the SWDIO line, and the pulldown resistor on the SWCLK line, is to place the SWD pins into a known state when no debug probe is attached. A minimum resistance of 100kΩ is recommended by Arm. The internal pullup/pulldown resistors fulfill this requirement and external resistors are not required for correct operation of the SWD interface.

After a power-on reset (POR), MSPM0 devices configure the SWD pins in SWD mode with an internal pullup resistor enabled on the SWDIO line and an internal pulldown resistor enabled on the SWCLK line. If the device configuration has not permanently disabled all SWD access, then the SWD interface is enabled during the boot process and a debug probe can be connected to the DEBUGSS.

In the event that a device was configured by software to enter SHUTDOWN mode, and a debug probe is then connected to the SWD pins with SWCLK active, wakeup logic will trigger an exit from SHUTDOWN mode and cause a BOR. A debug connection can then be established to the DEBUGSS after the BOR completes.

Upon physical connection of a debug probe, a configuration sequence must be sent from the debug probe to the target device to initiate a valid SWD connection with the SW-DP. An invalid sequence will not wake the device from SHUTDOWN mode. Once the sequence is applied and the SWD connection is established, communication with enabled debug access points is possible and the application code is alerted through assertion of the DEBUGSS PWRUPIFG interrupt. When the debug probe is disconnected and the SWD connection is lost, the PWRDWNIFG interrupt is asserted.

It is possible for application software to disable the SWD interface in SYSCTL, freeing the IO to be used for general purpose IO functionality. Review [Section 2.4.1.4](#) in SYSCTL for using the SWD pins for functionality other than SWD. Once software disables SWD functionality, it is not possible to re-enable it other than by triggering a POR. A POR will automatically re-enable the SWD functionality and put the SWD pins into SWD mode with pullup/pulldown resistors enabled. To re-gain debug access to a device which contains software that disables the SWD pins at startup, it is necessary to hold the device in a reset state with the NRST pin during a POR. This will prevent the application software from starting and will allow the debug probe to gain access to the device, at which point a mass erase DSSM command can be sent from the integrated development environment to the device via the debug probe to remove the application software which is disabling the SWD pins.

Note

BOR, BOOTRST, and SYSRST levels do reset the IOMUX logic, which will re-enable the pullup/pulldown resistors on the SWDIO/SWCLK pin. However, the SWD functionality remains disabled until the next POR. Because the device always powers up with the SWDIO pullup and SWCLK pulldown resistors enabled, the hardware design must accommodate this when using the SWD pins for functions other than SWD after startup. After reset, application software may disable the pullup/pulldown resistors in the IOMUX to free the SWD pins for other purposes.

28.1.3 Debug Access Ports

The debug access ports in the DEBUGSS are given in [Table 28-1](#).

Table 28-1. DEBUGSS Access Port Listing

APSEL	AP	Port Description	Purpose
0x0	AHB-AP	MCPUSS debug access port	Debug of the processor and peripherals
0x1	CFG-AP	Configuration access port	Access device type information
0x2	SEC-AP	Security access port	Access the debug mailbox (DSSM)
0x3	ET-AP	EnergyTrace™ technology access port	Read the power state data from EnergyTrace technology for power aware debug
0x4	PWR-AP	Power access port	Configure the device power states (interfaces with PMCU/SYSCTL)

The AHB-AP, PWR-AP, and ET-AP provide the complete device debug functionality (processor debug, peripheral and memory bus access, power state control, and processor state). See [Section 28.2](#) for more information.

The CFG-AP provides device information to the debug probe so that the debug probe can identify device characteristics, including the device part number and the device revision.

The SEC-AP provides access to the mailbox for communicating with software running on the device through SWD. See [Section 28.5](#) for more information.

28.2 Debug Features

The DEBUGSS supports [processor debug](#), processor trace, [peripheral debug](#), and [energy state debug](#).

28.2.1 Processor Debug

The Arm Cortex-M0+ processor supports a wide range of features to simplify debugging of application software during development. Key features supported by MSPM0 MCUs include:

- Ability to halt the processor through a assertion of a halt signal, a configured debug event (such as a hard fault entry or reset), or a BKPT instruction (for software breakpoints)
- Ability to step through instructions (with or without peripheral interrupts enabled)
- Ability to run through instructions (with or without peripheral interrupts enabled)
- Ability to read and write [CPU registers](#) when halted
- Ability to read exception information through the Cortex-M0+ system control space (SCS)
- Support for 4 hardware breakpoints
- Support for 2 hardware watchpoints
- Support for accessing the device memory map

28.2.1.1 Breakpoint Unit (BPU)

The breakpoint unit (BPU) provides 4 comparators which can be used to generate a debug event when the address of an instruction fetch matches the address programmed into the respective BPU comparator.

The BPU does not generate a debug event upon an address match for a data read or data write access.

Address matching is possible for half-word (16-bit) instructions and word (32-bit) instructions fetched from the CODE region (0x0000.0000 to 0x1FFF.FFFF).

If a debug scenario requires more than four breakpoints, software breakpoints can be used together with hardware breakpoints using the BKPT instruction. If debugging of code in the SRAM region is desired, hardware breakpoints are not available and software breakpoints must be inserted by the debug probe instead.

```
// Example of a breakpoint function in C (TI Arm CLANG compiler)
__BKPT(0);
```

28.2.1.2 Data Watchpoint and Trace Unit (DWT)

The data watchpoint and trace unit (DWT) provides 2 comparators which both support generating an event upon a data address match (watchpoint event) or an instruction address match (PC watchpoint event).

The DWT comparators support masking of the address, enabling an event to be generated when the processor attempts to access an address within a specified address range.

28.2.1.3 Processor Trace (MTB)

MSPM0G devices support basic instruction execution trace to obtain context of the sequence of execution which led to a certain state of the processor. The processor trace engine is based on the Arm CoreSight MTB-M0+ micro trace buffer.

The MTB captures the processor's program counter (PC) state when the PC changes in a non-sequential way due to a branch instruction or an exception. Load and store activity is not captured by the MTB. When non-sequential execution is detected by the MTB, the change is captured and stored into a small buffer memory (described in [Table 28-2](#)) which can be read out later by application software or the debug probe.

Table 28-2. MTB Buffer Memory

Start Address	End Address	Length
0x4040.3000	0x4040.3020	32B (4 trace packets)

Trace Packet Data

For each trace capture, the MTB stores the source address (the address which was branched from) and the destination address (the address which was branched to) into the buffer memory. Thus, two 32-bit words are used per trace capture packet. Because instructions are half-word aligned, the LSBs of the addresses are not required and are thus used to store additional context about the state into the trace packet.

- The LSB of the logged source address is modified by the MTB and is referred to as the "A" bit. The "A" bit is set to 0 or 1 based on the atomic state of the processor at the time of the branch.
 - When the "A" bit is captured as "0", the branch originated from an instruction in the code flow.
 - When the "A" bit is captured as "1", the branch originated from an exception or a PC update under debug.
- The LSB of the logged destination address is modified and is referred to as the "S" bit. The "S" bit is set to 0 or 1 based on whether the capture is the first to occur after tracing started.
 - When the "S" bit is captured as "1", the packet is the first packet after tracing started.
 - When the "S" bit is captured as "0", the packet is not the first packet after tracing started.

In the case of an exception return, two trace packets are stored into the buffer memory:

- The first packet has the "A" bit set to "0" and stores the following values:
 - The source address is the address of the instruction causing the exception return (the BX or POP).
 - The destination address is the EXC_RETURN value.
- The second packet has the "A" bit set to "1" and stores the following values:
 - The source address is the EXC_RETURN value.
 - The destination address is the address of the instruction where execution begins again after the exception.

28.2.2 Peripheral Debug

In addition to processor debug, the DEBUGSS can be used to access the device memory map from the perspective of the processor. Thus, a connected debug probe can be used to read and write memory-mapped peripheral registers, the system SRAM, and the flash memory.

Certain peripherals support advanced debug configuration options. These options are configured by application software (or optionally, the debug probe) by setting/clearing various debug control bits in the memory map of a given peripheral. In general, the debug behavior of a particular peripheral is specified in the PDBGCTL register of each peripheral. Many peripherals offer the option of halting the functional clock to the peripheral when the processor is halted for debug, thus pausing the peripheral together with the processor (default configuration), or letting the peripheral run even when the processor is halted for debug.

For example, the WWDT peripheral supports the FREE bit in the PDBGCTL register. Setting the FREE bit in PDBGCTL for a WWDT causes the WWDT counter to run even if the processor is halted for debug.

28.2.3 EnergyTrace Technology

The DEBUGSS in MSPM0 devices supports [EnergyTrace](#) technology. EnergyTrace technology enables power profiling of MCU devices running application code. This is very useful when developing an application which must be optimized for low-power operation.

Development tools from Texas Instruments, including the MSPM0 LaunchPad development tools, support hardware energy measurement of the target MSPM0 over time through EnergyTrace charge counting. This mechanism enables a developer to obtain an energy usage profile for an application, based on real current measurements with a wide dynamic range.

To give context to the energy measurements made by the hardware development tools supporting EnergyTrace technology, MSPM0 MCUs also enable EnergyTrace+. EnergyTrace+ is a component of the DEBUGSS that lets the debug probe log the state of the processor (RUN, SLEEP) and the current program counter value while the device is running. This state information can be then overlaid with energy measurements to determine if the cause of high current is the processor running or some other activity on the device.

TI's [Code Composer Studio](#) integrated development environment provides out-of-the-box support for EnergyTrace energy measurement and EnergyTrace+ processor state logging with MSPM0 devices.

28.3 Behavior in Low Power Modes

The DEBUGSS supports maintaining a debug connection through SWD in all operating modes except SHUTDOWN.

Access to device memory and peripherals is possible in RUN mode and SLEEP mode, in which a debug probe can be actively connected to the AHB-AP access port to interface with the processor. In STOP and STANDBY modes, a debug connection can be established and/or maintained with the DEBUGSS, but not with the CPU debug access port.

In SHUTDOWN mode, any active debug connection is terminated as the debug logic is powered down with the device VCORE. While a debug connection to the DEBUGSS is not possible while the device is in SHUTDOWN mode, a debug probe can cause the device to exit SHUTDOWN mode by attempting to communicate with the SWD pins. The device will detect attempted SWD communication even when the device is in SHUTDOWN. If activity is detected, a SHUTDOWN exit is initiated and the device will transition through a BOR state, after which a debug connection can be made to the DEBUGSS through SWD.

The DEBUGSS functionality by operating mode is given in [Table 28-3](#).

Table 28-3. DEBUGSS Functionality by Operating Mode

Capability	RUN	SLEEP	STOP	STANDBY	SHUTDOWN	NRST HOLD
Processor debug	Y	Y	N	N	N	N
Memory map access	Y	Y	N	N	N	N
Debug status through SW-DP	Y	Y	Y	Y	N	Y

Table 28-3. DEBUGSS Functionality by Operating Mode (continued)

Capability	RUN	SLEEP	STOP	STANDBY	SHUTDOWN	NRST HOLD
Debug state maintained	Y	Y	Y	Y	N	N
Wake from SWD	-	-	-	-	Y	-

28.4 Restricting Debug Access

The debug subsystem supports several methods for restricting access to the device through the SWD interface. The debug access policy is determined by the user configuration specified in the NONMAIN flash region.

There are 3 levels of access control, given in [Table 28-4](#). By default, products shipped from TI arrive in a "debug enabled" state where the device is fully open. This state is not recommended for production. For production, TI recommends changing the debug configuration to password protected or disabled.

Table 28-4. Debug Access Control

DEBUGSS Function	Debug Configuration		
	Debug Enabled (default)	Debug Enabled with Password	Debug Disabled
SW-DP (debug port)	EN	EN	DIS
CFG-AP	EN	EN	DIS
SEC-AP	EN	EN	DIS
ET-AP	EN	EN w/ PW	DIS
AHB-AP (CPU Debug)	EN	EN w/ PW	DIS

When debug is enabled with password, the debug access command together with the user-specified debug access password must be provided to the DEBUGSS [mailbox](#) by the debug probe, and a BOOTRST must be issued.

When debug is disabled, the SW-DP will be disabled during the boot process and any commands previously sent to the mailbox are ignored during boot. Following boot, any attempt to connect to the SW-DP is ignored.

It is possible to permanently lock debug access to the device by configuring the NONMAIN flash region to disable debug access while also configuring the NONMAIN flash region as statically write protected (locked). Locking the NONMAIN configuration has the added security of preventing the bootstrap loader (BSL) and application code from changing the debug security policy.

28.5 Mailbox (DSSM)

The debug subsystem mailbox (DSSM) enables a debug probe to pass messages to the target device through the SWD interface, as well as making it possible for the target device to return data to the debug probe.

The DSSM supports the following functions:

- Transmission of commands to the device during boot, including authenticating the debug probe for password-protected debug, mass erase, and factory reset operations
- Communicating with application software running on the target device when no other communication interface is present

Two 32-bit word data buffers are provided for TX data (debug probe to target device) and RX data (target device to debug probe). These data buffers are implemented as 32-bit memory-mapped registers in the DEBUGSS. In addition, TXCTL and RXCTL registers are provided for enabling flow control and indicating status of the mailbox.

Table 28-5. DSSM Register Functions

DSSM Register	Description	Debug Probe	Target Device	Actions
TX_DATA	Data buffer	RW	R	TXCTL.TRANSMIT is set on write by the debug probe, and cleared on a read by the target device; TXIFG is also set on a write by the debug probe
TXCTL	Flow control and status	RW	R	None
RX_DATA	Data buffer	R	RW	RXCTL.RECEIVE is set on write by the target device, and cleared on a read by the debug probe; RXIFG is also set on a write by the target device
RXCTL	Flow control and status	R	RW	None

The TXCTL and RXCTL registers provide TRANSMIT and RECEIVE flags, respectively, in the BIT0 position. The TRANSMIT bit is set in the TXCTL register when a debug probe writes data to the TX_DATA buffer register. The TRANSMIT flag will then remain set until the target device reads TX_DATA or a POR occurs. The RECEIVE flag is set in the RXCTL register when the target device writes data to the RX_DATA buffer register. The RECEIVE flag will then remain set until the debug probe reads the data from RX_DATA.

It is not possible for software running on the target device to write to TX_DATA, and it is also not possible for target software to clear the TRANSMIT flag other than by reading TX_DATA. The upper 31 bits of the TXCTL register contain generic flag bits which can be set or cleared by the debug probe to implement a protocol if desired. Only the debug probe can write to the TRANSMIT_FLAGS field in TXCTL.

In a similar way, only the target device software can write to RX_DATA and RXCTL. The debug probe cannot write to RX_DATA and it can only clear the RECEIVE flag in RXCTL by reading RX_DATA. BIT1 through BIT7 (0xFE) of RXCTL contains the RECEIVE_FLAGS field. Software on the target device can set or clear bits in the RECEIVE_FLAGS field to implement a protocol if desired. These flags can be read by the debug probe but can not be modified by the debug probe.

For a complete listing of DSSM commands which are supported by the boot configuration routine during device startup configuration, see [Section 1.4](#).

28.5.1 DSSM Events

The DSSM contains one [event publisher](#) and no [event subscribers](#). One event publisher (CPU_INT) manages DSSM interrupt requests (IRQs) to the CPU subsystem through a [static event route](#).

The DSSM events are summarized in [Table 28-6](#).

Table 28-6. DSSM Events

Event	Type	Source	Destination	Route	Configuration	Functionality
CPU Interrupt Event	Publisher	DEBUGSS	CPU Subsystem	Static route	CPU_INT registers	Fixed interrupt route from DEBUGSS to CPU

28.5.1.1 CPU Interrupt Event (CPU_INT)

The DSSM provides 4 interrupt sources which can be configured to source a [CPU interrupt event](#). In order of decreasing interrupt priority, the CPU interrupt events from the DSSM are given in [Table 28-7](#).

Table 28-7. DSSM CPU Interrupt Event Conditions (CPU_INT)

Index (IIDX)	Name	Description
0	TXIFG	Indicates that the TX_DATA buffer in the DSSM has received data.
1	RXIFG	Indicates that the data in RX_DATA buffer in the DSSM was read.

Table 28-7. DSSM CPU Interrupt Event Conditions (CPU_INT) (continued)

Index (IIDX)	Name	Description
2	PWRUPIFG	Indicates that the DEBUGSS was started due to a debug probe attaching to the device.
3	PWRDWNIFG	Indicates that the DEBUGSS was stopped due to a debug probe disconnecting from the device.

The CPU interrupt event configuration is managed with the CPU_INT event management registers. See [Section 7.2.5](#) for guidance on configuring the Event registers for CPU interrupts.

28.5.2 DEBUGSS Registers

Table 28-8 lists the memory-mapped registers for the DEBUGSS registers. All register offset addresses not listed in Table 28-8 should be considered as reserved locations and the register contents should not be modified.

Table 28-8. DEBUGSS Registers

Offset	Acronym	Register Name	Group	Section
1020h	IIDX	Interrupt index	CPU_INT	Go
1028h	IMASK	Interrupt mask	CPU_INT	Go
1030h	RIS	Raw interrupt status	CPU_INT	Go
1038h	MIS	Masked interrupt status	CPU_INT	Go
1040h	ISSET	Interrupt set	CPU_INT	Go
1048h	ICLR	Interrupt clear	CPU_INT	Go
10E0h	EVT_MODE	Event Mode		Go
10FCh	DESC	Module Description		Go
1100h	TXD	Transmit data register		Go
1104h	TXCTL	Transmit control register		Go
1108h	RXD	Receive data register		Go
110Ch	RXCTL	Receive control register		Go
1200h	SPECIAL_AUTH	Special enable authorization register		Go
1210h	APP_AUTH	Application CPU0 authorization register		Go

Complex bit access types are encoded to fit into small table cells. Table 28-9 shows the codes that are used for access types in this section.

Table 28-9. DEBUGSS Access Type Codes

Access Type	Code	Description
Read Type		
R	R	Read
R-0	R -0	Read Returns 0s
Write Type		
W	W	Write
WK	W K	Write Write protected by a key
Reset or Default Value		
-n		Value after reset or the default value

28.5.2.1 IIDX (Offset = 1020h) [Reset = 00000000h]

IIDX is shown in [Figure 28-2](#) and described in [Table 28-10](#).

Return to the [Summary Table](#).

This register provides the highest priority enabled interrupt index. 0xFF means no event pending. Interrupt 0x0 is the highest priority, 0x1 next highest, and 0xFE is the least priority. The priority order is fixed. However, users can implement their own prioritization schemes using other registers that expose the full set of interrupts that have occurred.

On each read, only one interrupt is indicated. On a read, the current interrupt (highest priority) is automatically cleared by the hardware and the corresponding interrupt flag in the RIS and MIS are cleared as well. After a read from the CPU (not from the debug interface), the register must be updated with the next highest priority interrupt, if none are pending, then it displays 0xFF.

Figure 28-2. IIDX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RESERVED																								STAT							
R-0h																								R-0h							

Table 28-10. IIDX Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R	0h	
7-0	STAT	R	0h	Interrupt index status 0h = No pending interrupt request 1h = TX interrupt 2h = RX interrupt 3h = Power-up interrupt. A debug session has started. 4h = Power-up interrupt. A debug session has started.

28.5.2.2 IMASK (Offset = 1028h) [Reset = 00000000h]

IMASK is shown in [Figure 28-3](#) and described in [Table 28-11](#).

Return to the [Summary Table](#).

Interrupt Mask. If a bit is set, then corresponding interrupt is unmasked. Unmasking the interrupt causes the raw interrupt to be visible in IIDX, as well as MIS.

Figure 28-3. IMASK

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RESERVED				PWRDWNIFG	PWRUPIFG	RXIFG	TXIFG
R/W-0h				R/W-0h	R/W-0h	R/W-0h	R/W-0h

Table 28-11. IMASK Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R/W	0h	
3	PWRDWNIFG	R/W	0h	Masks PWRDWNIFG in MIS register 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
2	PWRUPIFG	R/W	0h	Masks PWRUPIFG in MIS register 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
1	RXIFG	R/W	0h	Masks RXIFG in MIS register 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set
0	TXIFG	R/W	0h	Masks TXIFG in MIS register 0h = Interrupt is masked out 1h = Interrupt will request an interrupt service routine and corresponding bit in MIS will be set

28.5.2.3 RIS (Offset = 1030h) [Reset = 00000000h]

RIS is shown in [Figure 28-4](#) and described in [Table 28-12](#).

Return to the [Summary Table](#).

Raw interrupt status. Reflects all pending interrupts, regardless of masking. The RIS register allows the user to implement a poll scheme. A flag set in this register can be cleared by writing 1 to the ICLR register bit even if the corresponding IMASK bit is not enabled.

Figure 28-4. RIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				PWRDWNIFG	PWRUPIFG	RXIFG	TXIFG
R-0h				R-0h	R-0h	R-0h	R-0h

Table 28-12. RIS Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	PWRDWNIFG	R	0h	Raw interrupt status for PWRDWNIFG 0h = PWRUPIFG did not occur 1h = PWRUPIFG occurred
2	PWRUPIFG	R	0h	Raw interrupt status for PWRUPIFG 0h = PWRUPIFG did not occur 1h = PWRUPIFG occurred
1	RXIFG	R	0h	Raw interrupt status for RXIFG 0h = RXIFG did not occur 1h = RXIFG occurred
0	TXIFG	R	0h	Raw interrupt status for TXIFG 0h = TXIFG did not occur 1h = TXIFG occurred

28.5.2.4 MIS (Offset = 1038h) [Reset = 00000000h]

MIS is shown in [Figure 28-5](#) and described in [Table 28-13](#).

Return to the [Summary Table](#).

Masked interrupt status. This is an AND of the IMASK and RIS registers.

Figure 28-5. MIS

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				PWRDWNIFG	PWRUPIFG	RXIFG	TXIFG
R-0h				R-0h	R-0h	R-0h	R-0h

Table 28-13. MIS Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	PWRDWNIFG	R	0h	Masked interrupt status for PWRDWNIFG 0h = PWRUPIFG did not request an interrupt service routine 1h = PWRUPIFG requests an interrupt service routine
2	PWRUPIFG	R	0h	Masked interrupt status for PWRUPIFG 0h = PWRUPIFG did not request an interrupt service routine 1h = PWRUPIFG requests an interrupt service routine
1	RXIFG	R	0h	Masked interrupt status for RXIFG 0h = RXIFG did not request an interrupt service routine 1h = RXIFG requests an interrupt service routine
0	TXIFG	R	0h	Masked interrupt status for TXIFG 0h = TXIFG did not request an interrupt service routine 1h = TXIFG requests an interrupt service routine

28.5.2.5 ISET (Offset = 1040h) [Reset = 00000000h]

ISET is shown in [Figure 28-6](#) and described in [Table 28-14](#).

Return to the [Summary Table](#).

Interrupt set. Allows interrupts to be set by software (useful in diagnostics and safety checks). Writing a 1 to a bit in ISET will set the event and therefore the related RIS bit also gets set. If the interrupt is enabled through the mask, then the corresponding MIS bit is also set.

Figure 28-6. ISET

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED				PWRDWNIFG	PWRUPIFG	RXIFG	TXIFG
W-0h				W-0h	W-0h	W-0h	W-0h

Table 28-14. ISET Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	W	0h	
3	PWRDWNIFG	W	0h	Sets PWRDWNIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to PWRUPIFG is set
2	PWRUPIFG	W	0h	Sets PWRUPIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to PWRUPIFG is set
1	RXIFG	W	0h	Sets RXIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to RXIFG is set
0	TXIFG	W	0h	Sets TXIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to TXIFG is set

28.5.2.6 ICLR (Offset = 1048h) [Reset = 00000000h]

ICLR is shown in [Figure 28-7](#) and described in [Table 28-15](#).

Return to the [Summary Table](#).

Interrupt clear. Write a 1 to clear corresponding Interrupt.

Figure 28-7. ICLR

31	30	29	28	27	26	25	24
RESERVED							
W-0h							
23	22	21	20	19	18	17	16
RESERVED							
W-0h							
15	14	13	12	11	10	9	8
RESERVED							
W-0h							
7	6	5	4	3	2	1	0
RESERVED				PWRDWNIFG	PWRUPIFG	RXIFG	TXIFG
W-0h				W-0h	W-0h	W-0h	W-0h

Table 28-15. ICLR Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	W	0h	
3	PWRDWNIFG	W	0h	Clears PWRDWNIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to PWRUPIFG is cleared
2	PWRUPIFG	W	0h	Clears PWRUPIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to PWRUPIFG is cleared
1	RXIFG	W	0h	Clears RXIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to RXIFG is cleared
0	TXIFG	W	0h	Clears TXIFG in RIS register 0h = Writing a 0 has no effect 1h = RIS bit corresponding to TXIFG is cleared

28.5.2.7 EVT_MODE (Offset = 10E0h) [Reset = 00000001h]

EVT_MODE is shown in [Figure 28-8](#) and described in [Table 28-16](#).

Return to the [Summary Table](#).

Event mode register. It is used to select whether each line is disabled, in software mode (software clears the RIS) or in hardware mode (hardware clears the RIS)

Figure 28-8. EVT_MODE

31	30	29	28	27	26	25	24
RESERVED							
R-							
23	22	21	20	19	18	17	16
RESERVED							
R-							
15	14	13	12	11	10	9	8
RESERVED							
R-							
7	6	5	4	3	2	1	0
RESERVED						INT0_CFG	
R-						R-1h	

Table 28-16. EVT_MODE Field Descriptions

Bit	Field	Type	Reset	Description
31-2	RESERVED	R	0h	
1-0	INT0_CFG	R	1h	Event line mode select for peripheral events 0h = The interrupt or event line is disabled. 1h = The interrupt or event line is in software mode. Software must clear the RIS. 2h = The interrupt or event line is in hardware mode. The hardware (another module) clears automatically the associated RIS flag.

28.5.2.8 DESC (Offset = 10FCh) [Reset = 03400000h]

DESC is shown in [Figure 28-9](#) and described in [Table 28-17](#).

Return to the [Summary Table](#).

This register identifies the peripheral and its exact version.

Figure 28-9. DESC

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODULEID															
R-340h															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FEATUREVER				INSTNUM				MAJREV				MINREV			
R-0h				R-0h				R-0h				R-0h			

Table 28-17. DESC Field Descriptions

Bit	Field	Type	Reset	Description
31-16	MODULEID	R	340h	Module identification contains a unique peripheral identification number. The assignments are maintained in a central database for all of the platform modules to ensure uniqueness.
15-12	FEATUREVER	R	0h	Feature Set for the module *instance*
11-8	INSTNUM	R	0h	Instance Number within the device. This will be a parameter to the RTL for modules that can have multiple instances
7-4	MAJREV	R	0h	Major rev of the IP
3-0	MINREV	R	0h	Minor rev of the IP

28.5.2.9 TXD (Offset = 1100h) [Reset = 00000000h]

TXD is shown in [Figure 28-10](#) and described in [Table 28-18](#).

Return to the [Summary Table](#).

This register is used for data transfers from external debug tools to the DSSM module. The register is written by the debug tool and read by the CPU.

Figure 28-10. TXD

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TX_DATA																															
R-0h																															

Table 28-18. TXD Field Descriptions

Bit	Field	Type	Reset	Description
31-0	TX_DATA	R	0h	Contains data written by an external debug tool to the SEC-AP TXDATA register

28.5.2.10 TXCTL (Offset = 1104h) [Reset = 00000000h]

TXCTL is shown in [Figure 28-11](#) and described in [Table 28-19](#).

Return to the [Summary Table](#).

Transmit control register

Figure 28-11. TXCTL

31	30	29	28	27	26	25	24
TRANSMIT_FLAGS							
R-0h							
23	22	21	20	19	18	17	16
TRANSMIT_FLAGS							
R-0h							
15	14	13	12	11	10	9	8
TRANSMIT_FLAGS							
R-0h							
7	6	5	4	3	2	1	0
TRANSMIT_FLAGS							TRANSMIT
R-0h							R-0h

Table 28-19. TXCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-1	TRANSMIT_FLAGS	R	0h	Generic TX flags that can be set by external debug tool. Functionality is defined by SW.
0	TRANSMIT	R	0h	Indicates data request in DSSM.TXD, set on write via Debug AP to DSSM.TXD. A read of the DSSM.TXD register by SW will clear the TX field. The tool can check that TXD is empty by reading this field. 0h = TXD is empty 1h = TXD is full

28.5.2.11 RXD (Offset = 1108h) [Reset = 00000000h]

RXD is shown in [Figure 28-12](#) and described in [Table 28-20](#).

Return to the [Summary Table](#).

Receive data register. This register contains the data written by the CPU.

This data is read by external debug tool.

Figure 28-12. RXD

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RX_DATA																															
R/W-0h																															

Table 28-20. RXD Field Descriptions

Bit	Field	Type	Reset	Description
31-0	RX_DATA	R/W	0h	Contains data written by SM/OW.

28.5.2.12 RXCTL (Offset = 110Ch) [Reset = 00000000h]

RXCTL is shown in [Figure 28-13](#) and described in [Table 28-21](#).

Return to the [Summary Table](#).

Receive control register

Figure 28-13. RXCTL

31	30	29	28	27	26	25	24
RESERVED							
R/W-0h							
23	22	21	20	19	18	17	16
RESERVED							
R/W-0h							
15	14	13	12	11	10	9	8
RESERVED							
R/W-0h							
7	6	5	4	3	2	1	0
RECEIVE_FLAGS							RECEIVE
R/W-0h							R-0h

Table 28-21. RXCTL Field Descriptions

Bit	Field	Type	Reset	Description
31-8	RESERVED	R/W	0h	
7-1	RECEIVE_FLAGS	R/W	0h	Generic RX flags that can be set by SW and read by external debug tool. Functionality is defined by SW.
0	RECEIVE	R	0h	Indicates SW write to the DSSM.RXD register. A read of the DSSM.RXD register by SWD Access Port will clear the RX field. 0h = RXD empty 1h = RXD full

28.5.2.13 SPECIAL_AUTH (Offset = 1200h) [Reset = 00000013h]

SPECIAL_AUTH is shown in [Figure 28-14](#) and described in [Table 28-22](#).

Return to the [Summary Table](#).

This register is used to control ET-AP, DFT-TAP, SWD, CFG-AP and SEC-AP.

Figure 28-14. SPECIAL_AUTH

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED	PWRAPEN	AHBAPEN	CFGAPEN	ETAPEN	DFTAPEN	SWDPORTEN	SECAPEN
R-0h	R-0h	R-0h	R-1h	R-0h	R-0h	R-1h	R-1h

Table 28-22. SPECIAL_AUTH Field Descriptions

Bit	Field	Type	Reset	Description
31-7	RESERVED	R	0h	
6	PWRAPEN	R	0h	An active high input. When asserted (and SWD access is also permitted), the debug tools can then access the PWR-AP to power and reset state of the CPU. When deasserted, a DAPBUS firewall will isolate the AP and prevent access. 0h = Disable PWR-AP 1h = Enable PWR-AP
5	AHBAPEN	R	0h	Disabling / enabling debug access to the M0+ Core via the AHB-AP DAP bus isolation. 0h = Disable AHB-AP 1h = Enable AHB-AP
4	CFGAPEN	R	1h	An active high input. When asserted (and SWD access is also permitted), the debug tools can use the Config-AP to read device configuration information. When deasserted, a DAPBUS firewall will isolate the AP and prevent access to the Config-AP. 0h = Disable CFG-AP 1h = Enable CFG-AP
3	ETAPEN	R	0h	An active high input. When asserted (and SWD access is also permitted), the debug tools can then access an ET-AP external to the DebugSS lite. When deasserted, a DAPBUS firewall will isolate the AP and prevent access. 0h = Disable ET+ -AP 1h = Enable ET+ -AP
2	DFTAPEN	R	0h	An active high input. When asserted (and SWD access is also permitted), the debug tools can then access the DFT-AP external to the DebugSS lite. When deasserted, a DAPBUS firewall will isolate the AP and prevent access. 0h = Disable DFT-TAP 1h = Enable DFT-TAP
1	SWDPORTEN	R	1h	When asserted, the SW-DP functions normally. When deasserted, the SW-DP effectively disables all external debug access. 0h = Disable SWD port 1h = Enable SWD port

Table 28-22. SPECIAL_AUTH Field Descriptions (continued)

Bit	Field	Type	Reset	Description
0	SECAPEN	R	1h	An active high input. When asserted (and SWD access is also permitted), the debug tools can use the Security-AP to communicate with security control logic. When deasserted, a DAPBUS firewall will isolate the AP and prevent access to the Security-AP. 0h = Disable SEC-AP 1h = Enable SEC-AP

28.5.2.14 APP_AUTH (Offset = 1210h) [Reset = 00000000h]

APP_AUTH is shown in [Figure 28-15](#) and described in [Table 28-23](#).

Return to the [Summary Table](#).

This register is used to control DBGEN, NIDEN, SPIDEN, and SPNIDEN of Application CPU0. DBGEN, NIDEN are further processed by DSW based on Active and Debug IPF ID.

Figure 28-15. APP_AUTH

31	30	29	28	27	26	25	24
RESERVED							
R-0h							
23	22	21	20	19	18	17	16
RESERVED							
R-0h							
15	14	13	12	11	10	9	8
RESERVED							
R-0h							
7	6	5	4	3	2	1	0
RESERVED				SPNIDEN	SPIDEN	NIDEN	DBGEN
R-0h				R-0h	R-0h	R-0h	R-0h

Table 28-23. APP_AUTH Field Descriptions

Bit	Field	Type	Reset	Description
31-4	RESERVED	R	0h	
3	SPNIDEN	R	0h	Secure non-invasive debug enable. 0h = Invasive debug disabled 1h = Invasive debug enabled
2	SPIDEN	R	0h	Secure invasive debug enable. 0h = Invasive debug disabled 1h = Invasive debug enabled
1	NIDEN	R	0h	Controls non-invasive debug enable. 0h = Non-invasive debug disabled 1h = Non-invasive debug enabled
0	DBGEN	R	0h	Controls invasive debug enable. 0h = Invasive debug disabled 1h = Invasive debug enabled

Revision History



NOTE: Page numbers for previous revisions may differ from page numbers in the current version.

DATE	REVISION	NOTES
October 2023	A	Changes throughout for production release

IMPORTANT NOTICE AND DISCLAIMER

TI PROVIDES TECHNICAL AND RELIABILITY DATA (INCLUDING DATA SHEETS), DESIGN RESOURCES (INCLUDING REFERENCE DESIGNS), APPLICATION OR OTHER DESIGN ADVICE, WEB TOOLS, SAFETY INFORMATION, AND OTHER RESOURCES "AS IS" AND WITH ALL FAULTS, AND DISCLAIMS ALL WARRANTIES, EXPRESS AND IMPLIED, INCLUDING WITHOUT LIMITATION ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT OF THIRD PARTY INTELLECTUAL PROPERTY RIGHTS.

These resources are intended for skilled developers designing with TI products. You are solely responsible for (1) selecting the appropriate TI products for your application, (2) designing, validating and testing your application, and (3) ensuring your application meets applicable standards, and any other safety, security, regulatory or other requirements.

These resources are subject to change without notice. TI grants you permission to use these resources only for development of an application that uses the TI products described in the resource. Other reproduction and display of these resources is prohibited. No license is granted to any other TI intellectual property right or to any third party intellectual property right. TI disclaims responsibility for, and you will fully indemnify TI and its representatives against, any claims, damages, costs, losses, and liabilities arising out of your use of these resources.

TI's products are provided subject to [TI's Terms of Sale](#) or other applicable terms available either on [ti.com](https://www.ti.com) or provided in conjunction with such TI products. TI's provision of these resources does not expand or otherwise alter TI's applicable warranties or warranty disclaimers for TI products.

TI objects to and rejects any additional or different terms you may have proposed.

Mailing Address: Texas Instruments, Post Office Box 655303, Dallas, Texas 75265
Copyright © 2024, Texas Instruments Incorporated